

Ezairo® 8300 Hardware Reference

M-20852-019
December 2023

Table of Contents

Ezairo® 8300 Hardware Reference.....	1
Table of Contents.....	2
1. Introduction.....	39
1.1 Purpose.....	39
1.2 Intended Audience.....	39
1.3 Conventions.....	39
1.4 Further Reading.....	40
2. System Overview.....	42
2.1 System Architecture Introduction.....	42
2.1.1 System Control and Configuration.....	43
2.1.2 Application Overview.....	44
2.2 CFX Digital Signal Processor.....	47
2.2.1 CFX Memory.....	47
2.3 Arm Cortex-M3 Processor.....	48
2.3.1 Arm Cortex-M3 Processor Memory.....	48
2.4 LPDSP32 DSP.....	48
2.4.1 LPDSP32 Memories.....	49
2.5 HEAR Configurable Accelerator.....	49
2.5.1 HEAR Memory.....	49
2.6 Filter Engine.....	49
2.6.1 Filter Engine Memory.....	50
2.7 Neural Network Accelerator.....	50
2.8 FIFO Controller.....	50
2.9 Peripherals.....	51
2.9.1 Peripherals Accessible to the CFX and the Arm Cortex-M3 Core.....	51

onsemi Confidential
Ezairo 8300 Hardware Reference

2.9.2 Arm Cortex-M3 Processor Private Peripherals.....	51
2.10 Interfaces.....	51
2.10.1 Input/Output Configuration.....	51
2.10.2 Debug Interfaces.....	51
2.10.2.1 Activity Counter Control and Configuration Registers.....	52
2.10.2.1.1 SYSCTRL_CNT_CTRL.....	53
2.10.2.1.2 SYSCTRL_SYSCLK_CNT.....	53
2.10.2.1.3 SYSCTRL_CM3_CNT.....	53
2.10.2.1.4 SYSCTRL_DSP_CNT.....	53
2.10.2.1.5 SYSCTRL_CFX_CNT.....	54
2.10.2.1.6 SYSCTRL_HEAR_CNT.....	54
2.10.2.1.7 SYSCTRL_FENG_CNT.....	54
2.10.3 Interfaces for Communications and Signaling.....	54
2.11 Inputs and Outputs.....	54
2.11.1 Input/Output Controllers (IOCs) and Audio Multiplexers.....	54
2.11.2 Input Stage.....	55
2.11.3 Output Stage.....	55
2.12 Analog Support Blocks.....	55
2.12.1 Oscillators and Clocking.....	55
2.12.2 Power Supply, Power Management and Reset Handling.....	55
2.13 Default Register Values Overwritten by ROM.....	56
2.14 Device Identification.....	56
2.14.1 Device Identification Control and Configuration Registers.....	57
2.14.1.1 AHBREGS_CHIP_ID_NUM.....	57
2.14.1.2 D_CHIP_ID_FAMILY, CHIP_ID_FAMILY.....	58
2.14.1.3 D_CHIP_ID_VERSION, CHIP_ID_VERSION.....	58

2.14.1.4	D_CHIP_ID_REVISION, CHIP_ID_REVISION.....	58
2.14.1.5	D_CHIP_ID_CFX_DEBUG_VERSION, CHIP_ID_CFX_DEBUG_VERSION.....	58
2.14.1.6	D_CHIP_ID_CM3_DEBUG_VERSION, CHIP_ID_CM3_DEBUG_VERSION.....	58
3.	CFX Digital Signal Processor.....	59
3.1	Processor Overview.....	59
3.1.1	Inter-processor Communication.....	59
3.1.2	Boot-Up Control and Configuration Registers.....	59
3.1.2.1	BOOT_STATUS.....	60
3.1.3	Command Control and Configuration Registers.....	60
3.1.3.1	D_CM3_CMDS, CM3_CMDS.....	60
3.2	Interrupt Controller.....	61
3.2.1	Control and Configuration Registers.....	67
3.2.1.1	D_INT_MASTER_ENABLE, INT_MASTER_ENABLE.....	71
3.2.1.2	D_INT_ENABLE_0, INT_ENABLE_0.....	71
3.2.1.3	D_INT_ENABLE_1, INT_ENABLE_1.....	73
3.2.1.4	D_INT_ENABLE_2, INT_ENABLE_2.....	76
3.2.1.5	D_INT_STATUS_0, INT_STATUS_0.....	78
3.2.1.6	D_INT_STATUS_1, INT_STATUS_1.....	80
3.2.1.7	D_INT_STATUS_2, INT_STATUS_2.....	82
3.2.1.8	D_INT_PRIORITY_0, INT_PRIORITY_0.....	84
3.2.1.9	D_INT_PRIORITY_1, INT_PRIORITY_1.....	88
3.2.1.10	D_INT_PRIORITY_2, INT_PRIORITY_2.....	91
3.2.1.11	D_INT_EBL_0, INT_EBL_0.....	94
3.2.1.12	D_INT_EBL_1, INT_EBL_1.....	95
3.2.1.13	D_INT_EBL_2, INT_EBL_2.....	96
3.2.1.14	D_INT_EBL_3, INT_EBL_3.....	96

Ezairo 8300 Hardware Reference

3.2.1.15	D_INT_EBL_4, INT_EBL_4.....	97
3.2.1.16	D_INT_EBL_5, INT_EBL_5.....	98
3.2.1.17	D_INT_EBL_7, INT_EBL_7.....	98
3.2.1.18	D_INT_EBL_8, INT_EBL_8.....	99
3.2.1.19	D_INT_EBL_9, INT_EBL_9.....	100
3.2.1.20	D_INT_EBL_10, INT_EBL_10.....	101
3.2.1.21	D_INT_EBL_11, INT_EBL_11.....	101
3.2.1.22	D_INT_DIS_0, INT_DIS_0.....	102
3.2.1.23	D_INT_DIS_1, INT_DIS_1.....	103
3.2.1.24	D_INT_DIS_2, INT_DIS_2.....	103
3.2.1.25	D_INT_DIS_3, INT_DIS_3.....	104
3.2.1.26	D_INT_DIS_4, INT_DIS_4.....	105
3.2.1.27	D_INT_DIS_5, INT_DIS_5.....	105
3.2.1.28	D_INT_DIS_6, INT_DIS_6.....	106
3.2.1.29	D_INT_DIS_7, INT_DIS_7.....	107
3.2.1.30	D_INT_DIS_8, INT_DIS_8.....	107
3.2.1.31	D_INT_DIS_9, INT_DIS_9.....	108
3.2.1.32	D_INT_DIS_10, INT_DIS_10.....	109
3.2.1.33	D_INT_DIS_11, INT_DIS_11.....	109
3.2.1.34	D_INT_ACK_0, INT_ACK_0.....	110
3.2.1.35	D_INT_ACK_1, INT_ACK_1.....	111
3.2.1.36	D_INT_ACK_2, INT_ACK_2.....	111
3.2.1.37	D_INT_ACK_3, INT_ACK_3.....	112
3.2.1.38	D_INT_ACK_4, INT_ACK_4.....	113
3.2.1.39	D_INT_ACK_5, INT_ACK_5.....	113
3.2.1.40	D_INT_ACK_6, INT_ACK_6.....	114

3.2.1.41	D_INT_ACK_7, INT_ACK_7.....	115
3.2.1.42	D_INT_ACK_8, INT_ACK_8.....	116
3.2.1.43	D_INT_ACK_9, INT_ACK_9.....	116
3.2.1.44	D_INT_ACK_10, INT_ACK_10.....	117
3.2.1.45	D_INT_ACK_11, INT_ACK_11.....	118
3.2.1.46	D_INT_PRIO_0, INT_PRIO_0.....	118
3.2.1.47	D_INT_PRIO_1, INT_PRIO_1.....	119
3.2.1.48	D_INT_PRIO_2, INT_PRIO_2.....	120
3.2.1.49	D_INT_PRIO_3, INT_PRIO_3.....	121
3.2.1.50	D_INT_PRIO_4, INT_PRIO_4.....	122
3.2.1.51	D_INT_PRIO_5, INT_PRIO_5.....	123
3.2.1.52	D_INT_PRIO_6, INT_PRIO_6.....	124
3.2.1.53	D_INT_PRIO_7, INT_PRIO_7.....	125
3.2.1.54	D_INT_PRIO_8, INT_PRIO_8.....	126
3.2.1.55	D_INT_PRIO_9, INT_PRIO_9.....	127
3.2.1.56	D_INT_PRIO_10, INT_PRIO_10.....	128
3.2.1.57	D_INT_PRIO_11, INT_PRIO_11.....	129
4.	Arm Cortex-M3 Processor.....	131
4.1	Processor Overview.....	131
4.2	Arm Cortex-M3 Processor Debugging.....	132
4.3	Inter-processor Communication.....	132
4.4	Arm Cortex-M3 Control and Configuration Register Conventions.....	132
4.5	Control and Configuration Registers.....	133
4.5.1	SYSCTRL_DSP_CMD.....	133
4.5.2	SYSCTRL_CFX_CMD.....	134
4.5.3	SYSCTRL_BOOT_STATUS.....	134

4.6 Arm Cortex-M3 Private Peripherals.....	135
4.6.1 Nested Vectored Interrupt Controller (NVIC).....	135
4.6.1.1 NVIC and SCB Control and Configuration Registers.....	138
4.6.1.1.1 NVIC_ISER0.....	142
4.6.1.1.2 NVIC_ISER1.....	145
4.6.1.1.3 NVIC_ISER2.....	147
4.6.1.1.4 NVIC_ICER0.....	149
4.6.1.1.5 NVIC_ICER1.....	152
4.6.1.1.6 NVIC_ICER2.....	154
4.6.1.1.7 NVIC_ISPR0.....	156
4.6.1.1.8 NVIC_ISPR1.....	158
4.6.1.1.9 NVIC_ISPR2.....	161
4.6.1.1.10 NVIC_ICPR0.....	163
4.6.1.1.11 NVIC_ICPR1.....	166
4.6.1.1.12 NVIC_ICPR2.....	169
4.6.1.1.13 NVIC_IABR0.....	172
4.6.1.1.14 NVIC_IABR1.....	175
4.6.1.1.15 NVIC_IABR2.....	178
4.6.1.1.16 NVIC_IP0.....	180
4.6.1.1.17 NVIC_IP1.....	180
4.6.1.1.18 NVIC_IP2.....	181
4.6.1.1.19 NVIC_IP3.....	181
4.6.1.1.20 NVIC_IP4.....	181
4.6.1.1.21 NVIC_IP5.....	181
4.6.1.1.22 NVIC_IP6.....	181
4.6.1.1.23 NVIC_IP7.....	182

onsemi Confidential
Ezairo 8300 Hardware Reference

4.6.1.1.24	NVIC_IP8.....	182
4.6.1.1.25	NVIC_IP9.....	182
4.6.1.1.26	NVIC_IP10.....	182
4.6.1.1.27	NVIC_IP11.....	182
4.6.1.1.28	NVIC_IP12.....	183
4.6.1.1.29	NVIC_IP13.....	183
4.6.1.1.30	NVIC_IP14.....	183
4.6.1.1.31	NVIC_IP15.....	183
4.6.1.1.32	NVIC_IP16.....	183
4.6.1.1.33	NVIC_IP17.....	184
4.6.1.1.34	NVIC_IP18.....	184
4.6.1.1.35	NVIC_IP19.....	184
4.6.1.1.36	NVIC_IP20.....	184
4.6.1.1.37	NVIC_IP21.....	184
4.6.1.1.38	NVIC_IP22.....	185
4.6.1.1.39	NVIC_IP23.....	185
4.6.1.1.40	NVIC_STIR.....	185
4.6.1.1.41	SCnSCB_ICTR.....	185
4.6.1.1.42	SCB_CPUID.....	185
4.6.1.1.43	SCB_ICSR.....	186
4.6.1.1.44	SCB_VTOR.....	186
4.6.1.1.45	SCB_AIRCR.....	186
4.6.1.1.46	SCB_SCR.....	187
4.6.1.1.47	SCB_CCR.....	188
4.6.1.1.48	SCB_SHP0.....	189
4.6.1.1.49	SCB_SHP1.....	189

onsemi Confidential
Ezairo 8300 Hardware Reference

4.6.1.1.50	SCB_SHP2.....	189
4.6.1.1.51	SCB_SHCSR.....	189
4.6.1.1.52	SCB_CFSR.....	191
4.6.1.1.53	SCB_HFSR.....	191
4.6.1.1.54	SCB_DFSR.....	192
4.6.1.1.55	SCB_MMFAR.....	192
4.6.1.1.56	SCB_BFAR.....	192
4.6.1.2	Interrupt Controller Type Register.....	192
4.6.1.3	Interrupt Set Enable and Clear Enable Registers.....	192
4.6.1.4	Interrupt Set-Pending Registers and Interrupt Clear-Pending Registers.....	193
4.6.1.5	Active Bit Register.....	194
4.6.1.6	Interrupt Priority Registers.....	195
4.6.1.7	Registers Described by Arm Documentation.....	196
4.6.2	SysTick.....	196
4.6.2.1	SysTick Control and Configuration Registers.....	197
4.6.2.1.1	SysTick_CTRL.....	197
4.6.2.1.2	SysTick_LOAD.....	198
4.6.2.1.3	SysTick_VAL.....	198
4.6.2.1.4	SysTick_CALIB.....	198
4.6.3	Debug Controller.....	199
4.6.3.1	Halting Debug Configuration and Status.....	199
4.6.3.2	Debug Monitor Configuration.....	200
4.6.3.3	Arm Cortex-M3 Processor Core Register Access.....	200
4.6.3.4	Debug Fault Status Register.....	200
4.6.3.5	Arm Cortex-M3 Processor Debug Port Specific Control and Configuration Registers.....	201
4.6.3.5.1	DEBUG_DHCSR.....	201

4.6.3.5.2	DEBUG_DCRSR.....	202
4.6.3.5.3	DEBUG_DCRDR.....	203
4.6.3.5.4	DEBUG_DEMCR.....	203
5.	HEAR Configurable Accelerator.....	205
5.1	Accelerator Overview.....	205
5.1.1	Function Chains.....	205
5.1.2	HEAR Control.....	205
5.1.3	Control and Configuration Registers.....	206
5.1.3.1	D_SYSTEM_CTRL, SYSTEM_CTRL.....	206
5.2	Function Chain Controller.....	207
5.2.1	Control and Configuration Registers.....	207
5.2.1.1	D_HEAR_FC_ENABLE, HEAR_FC_ENABLE.....	208
5.2.1.2	D_HEAR_FC_PRIORITY, HEAR_FC_PRIORITY.....	209
5.2.1.3	D_HEAR_FC_STATUS, HEAR_FC_STATUS.....	212
5.2.1.4	D_HEAR_CMDS_0_TO_7, HEAR_CMDS_0_TO_7.....	214
5.2.1.5	D_HEAR_CMDS_8_TO_15, HEAR_CMDS_8_TO_15.....	215
5.3	Block Floating-Point.....	215
5.3.1	Control and Configuration Registers.....	216
5.3.1.1	D_HEAR_BFP_CFG, HEAR_BFP_CFG.....	216
5.3.1.2	D_HEAR_BFP_BLOCK_EXP, HEAR_BFP_BLOCK_EXP.....	216
6.	Filter Engine.....	217
6.1	Processor Overview.....	217
6.2	Functional Description.....	217
6.3	Filter Engine in the Audio Path.....	219
6.4	Filter Engine Control and Configuration.....	219
6.4.1	Filter Engine Control.....	219

Ezairo 8300 Hardware Reference

6.4.1.1	Master Control.....	220
6.4.1.2	Pause.....	220
6.4.1.3	Clean and Resume.....	220
6.4.1.4	Synchronized Start.....	220
6.4.2	Filter Engine Reset.....	221
6.4.3	Filter Engine Manual Execution.....	221
6.4.4	Filter Engine Configuration.....	221
6.4.4.1	Core Configuration Register (D_FENG_CFG).....	221
6.4.4.2	Input and Output Configuration Registers (D_FENG_CFG_IO).....	222
6.4.5	Filter Engine Filter Coefficients.....	222
6.4.6	Multi-Rate Counter.....	223
6.4.7	Phase Memory.....	223
6.4.8	Dynamic TC and SH Mapping.....	224
6.5	Filter Engine and Processor Interface.....	224
6.5.1	Filter Engine Interrupt.....	224
6.5.2	Filter Engine Write Protection.....	226
6.5.3	Memory and Register Access Conflicts.....	227
6.6	Data, Control and Configuration Registers.....	228
6.6.1	Data Register Groups.....	228
6.6.1.1	Extended-Precision Data Registers.....	228
6.6.1.2	Single-Precision Data Registers.....	228
6.6.1.3	Input Data Registers.....	228
6.6.1.4	Output Data Registers.....	228
6.6.2	Filter Engine Registers.....	228
6.6.2.1	FENG_INT_STATUS, D_FENG_INT_STATUS.....	231
6.6.2.2	FENG_INT_STATUS_IO, D_FENG_INT_STATUS_IO.....	233

Ezairo 8300 Hardware Reference

6.6.2.3	FENG_INT_ACK, D_FENG_INT_ACK.....	236
6.6.2.4	FENG_INT_ACK_IO, D_FENG_INT_ACK_IO.....	237
6.6.2.5	FENG_INT_ENABLE, D_FENG_INT_ENABLE.....	239
6.6.2.6	FENG_INT_ENABLE_IO, D_FENG_INT_ENABLE_IO.....	240
6.6.2.7	FENG_CFG, D_FENG_CFG.....	243
6.6.2.8	FENG_CFG_IO, D_FENG_CFG_IO.....	244
6.6.2.9	FENG_MRCNT, D_FENG_MRCNT.....	247
6.6.2.10	FENG_PGM_SECTOR_EN, D_FENG_PGM_SECTOR_EN.....	248
6.6.2.11	FENG_PHM_PH0_PH1, D_FENG_PHM_PH0_PH1.....	248
6.6.2.12	FENG_PHM_PH4_PH5, D_FENG_PHM_PH4_PH5.....	248
6.6.2.13	FENG_PHM_PH2_PH3, D_FENG_PHM_PH2_PH3.....	249
6.6.2.14	FENG_PHM_PH4_PH5, D_FENG_PHM_PH4_PH5.....	249
6.6.2.15	FENG_PHM_PH6_PH7, D_FENG_PHM_PH6_PH7.....	249
6.6.2.16	FENG_PHM_PH8_PH9, D_FENG_PHM_PH8_PH9.....	249
6.6.2.17	FENG_PHM_PH10_PH11, D_FENG_PHM_PH10_PH11.....	250
6.6.2.18	FENG_PHM_PH12_PH13, D_FENG_PHM_PH12_PH13.....	250
6.6.2.19	FENG_PHM_PH14_PH15, D_FENG_PHM_PH14_PH15.....	250
6.6.2.20	FENG_PHM_PH15_END, D_FENG_PHM_PH15_END.....	250
6.6.2.21	FENG_COEFF_OFFSET_EN_SHADOW0, D_FENG_COEFF_OFFSET_EN_SHADOW0....	251
6.6.2.22	FENG_COEFF_OFFSET_EN_SHADOW1, D_FENG_COEFF_OFFSET_EN_SHADOW1....	251
6.6.2.23	FENG_SH_SHADOW, D_FENG_SH_SHADOW.....	251
6.6.2.24	FENG_TC_SHADOW0, D_FENG_TC_SHADOW0.....	251
6.6.2.25	FENG_TC_SHADOW1, D_FENG_TC_SHADOW1.....	252
6.6.2.26	FENG_STATE_W_EN, D_FENG_STATE_W_EN.....	252
6.6.2.27	FENG_STATE_SHADOW, D_FENG_STATE_SHADOW.....	252
6.6.2.28	FENG_STATE_ACTIVE, D_FENG_STATE_ACTIVE.....	252

Ezairo 8300 Hardware Reference

6.6.2.29	FENG_COEFF_OFFSET_EN_ACTIVE0, D_FENG_COEFF_OFFSET_EN_ACTIVE0.....	252
6.6.2.30	FENG_COEFF_OFFSET_EN_ACTIVE1, D_FENG_COEFF_OFFSET_EN_ACTIVE1.....	252
6.6.2.31	FENG_SH_ACTIVE, D_FENG_SH_ACTIVE.....	253
6.6.2.32	FENG_TC_ACTIVE0, D_FENG_TC_ACTIVE0.....	253
6.6.2.33	FENG_TC_ACTIVE1, D_FENG_TC_ACTIVE1.....	253
6.6.2.34	FENG_CMDS, D_FENG_CMDS.....	254
6.6.2.35	FENG_CTRL, D_FENG_CTRL.....	254
6.6.2.36	FENG_WR_KEY, D_FENG_WR_KEY.....	255
6.6.2.37	FENG_WR_PROTECT, D_FENG_WR_PROTECT.....	255
6.6.2.38	FENG_PGM_ADDR, D_FENG_PGM_ADDR.....	256
6.6.2.39	FENG_RF_EXTENDED*, D_FENG_RF_EXTENDED_*	256
6.6.2.40	FENG_RF_SINGLE*, D_FENG_RF_SINGLE_*	256
6.6.2.41	FENG_RF_IN*, D_FENG_RF_IN_*	256
6.6.2.42	FENG_RF_OUT*, D_FENG_RF_OUT_*	256
7.	Neural Network Accelerator.....	257
7.1	Block Overview.....	257
7.2	Attributes and Hardware Features.....	258
7.2.0.0.1	Inputs.....	259
7.2.0.0.2	Weights.....	260
7.2.0.0.3	Biases.....	261
7.2.0.0.4	Outputs.....	262
7.3	Weight Interleaving and Compression.....	263
7.3.0.0.1	Interleaving with 4-Bit or 8-Bit Weights.....	263
7.3.0.0.2	Interleaving with 16-Bit Weights.....	264
7.3.0.0.3	Weight Compression.....	265
7.3.0.0.3.1	Implementing a Weight Lookup Table.....	267

Ezairo 8300 Hardware Reference

7.4 Suggested Development Flow.....	269
7.5 NNA Beyond Fully-Connected Layers.....	270
7.6 NNA Control.....	270
7.7 Interfacing to the Rest of the System.....	271
7.8 Neural-Network Accelerator Control and Configuration Registers.....	271
7.8.0.1 NNA_CFG.....	273
7.8.0.2 NNA_CTRL.....	276
7.8.0.3 NNA_INPUT_CNT.....	277
7.8.0.4 NNA_OUTPUT_CNT.....	278
7.8.0.5 NNA_CIRC_BASE.....	278
7.8.0.6 NNA_CIRC_SIZE.....	278
7.8.0.7 NNA_INPUT_PTR.....	278
7.8.0.8 NNA_WEIGHT_PTR.....	278
7.8.0.9 NNA_BIAS_PTR.....	278
7.8.0.10 NNA_OUTPUT_PTR.....	278
7.8.0.11 NNA_WEIGHT_LUT_0_3.....	278
7.8.0.12 NNA_WEIGHT_LUT_4_7.....	279
7.8.0.13 NNA_WEIGHT_LUT_8_11.....	279
7.8.0.14 NNA_WEIGHT_LUT_12_15.....	279
8. LPDSP32 Processor.....	280
8.1 System Integration.....	280
8.2 Memory Support.....	280
8.3 Interrupt Handling.....	281
8.4 Outgoing Interrupts.....	281
8.5 Control and Configuration Registers.....	282
8.5.1 SYSCTRL_DSP_CTRL.....	282

onsemi Confidential
Ezairo 8300 Hardware Reference

8.5.2	IMSK.....	283
8.5.3	IRQ_STAT.....	284
8.5.4	IRQ_CLR.....	285
8.5.5	CM3_CFX_CMD.....	287
9.	Power Components.....	288
9.1	Power Supply.....	288
9.1.1	Power Supply Inputs.....	290
9.1.1.1	Battery Supply Voltage (VBAT).....	290
9.1.1.2	Output Receiver Supply Voltage (VBATOD).....	290
9.1.1.3	Digital Output Supply Voltages (VDDO1, VDDO2, VDDO3, and VDDO4).....	290
9.1.2	Internal Power Supply Voltages.....	291
9.1.2.1	Internal Band Gap Reference Voltage (VREF).....	291
9.1.2.2	Internal Digital Supply Voltages (VDDC and VDDM).....	291
9.1.2.2.1	Adaptive Voltage Scaling for VDDC.....	294
9.1.2.3	Regulated Supply Voltage (VREG).....	295
9.1.2.4	Analog Supply Voltage (VDDA).....	295
9.1.3	Power Supply Outputs.....	296
9.1.3.1	Regulated Interface Supply Voltage (VDDIF).....	296
9.1.3.2	Microphone Supply Voltage (VMIC) and Microphone Ground Reference (GND_MIC).....	297
9.2	Power Management Unit.....	297
9.2.1	Power Supervisory Unit.....	298
9.2.2	Power-On-Reset (POR) Sequence.....	299
9.3	Resets.....	299
9.4	Standby Mode.....	300
9.5	Control and Configuration Registers.....	302
9.5.1	ANALOG_BG_CTRL.....	303

Ezairo 8300 Hardware Reference

9.5.2	ANALOG_PWR_CTRL.....	303
9.5.3	ANALOG_VMIC_CTRL.....	304
9.5.4	ANALOG_VDDOD_CTRL.....	304
9.5.5	ANALOG_CP_VDDA_CTRL.....	305
9.5.6	ANALOG_CP_VDDIF_CTRL.....	305
9.5.7	ANALOG_CP_VDDC_TRIM.....	306
9.5.8	ANALOG_CP_VDDC_CTRL.....	307
9.5.9	ANALOG_CP_VDDC_AUTO_CTRL.....	307
9.5.10	ANALOG_CP_VDDC_AUTO_LIMITS.....	308
9.5.11	ANALOG_CP_VDDM_TRIM.....	308
9.5.12	ANALOG_CP_VDDM_CTRL.....	309
9.5.13	ANALOG_CP_VDDCM_REF.....	309
9.5.14	ANALOG_CP_VDDCM_CFG.....	309
10.	Clock Components.....	311
10.1	Clock Generation.....	311
10.1.1	Current-Controlled Oscillator.....	311
10.1.2	Standby Oscillator.....	311
10.1.3	External Clock (EXTCLK).....	312
10.1.3.1	Clock Detector.....	312
10.2	Clock Distribution.....	312
10.2.1	System Clock (SYSCLK).....	313
10.2.2	Sigma-Delta Modulator Clock (SDMCLK).....	314
10.2.3	ADC Clock (ADCCLK).....	315
10.2.4	Slow Clock (SLOWCLK).....	315
10.2.5	User Clock (UCLK).....	316
10.2.6	UART Clock (UARTCLK).....	317

10.3	Clock Throttling.....	317
10.3.1	Multiplier-Based Clock Throttling.....	317
10.3.2	Frequency-Based Clock Throttling.....	318
10.3.2.1	Advanced Frequency-Based Clock Throttling Options.....	319
10.4	Control and Configuration Registers.....	321
10.4.1	D_CLK_CFG_0, CLK_CFG_0.....	322
10.4.2	D_CLK_CFG_1, CLK_CFG_1.....	323
10.4.3	D_CLK_CFG_2, CLK_CFG_2.....	327
10.4.4	D_CLK_CFG_CPA, CLK_CFG_CPA.....	328
10.4.5	D_CLK_CFG_CPIF, CLK_CFG_CPIF.....	328
10.4.6	D_CLK_CFG_CPCM, CLK_CFG_CPCM.....	329
10.4.7	D_CLK_ADCCLK_THROTTLING, CLK_ADCCLK_THROTTLING.....	330
10.4.8	D_CLK_SDMCLK_THROTTLING, CLK_SDMCLK_THROTTLING.....	331
10.4.9	D_CLK_SLOWCLK_THROTTLING, CLK_SLOWCLK_THROTTLING.....	331
10.4.10	D_CLK_THROTTLING_CTRL, CLK_THROTTLING_CTRL.....	331
10.4.11	D_CLK_MULT_THROTTLING_CFG, CLK_MULT_THROTTLING_CFG.....	332
10.4.12	D_CLK_THROTTLING_CFG, CLK_THROTTLING_CFG.....	332
10.4.13	D_CLK_MULTI_SYNC_CFG, CLK_MULTI_SYNC_CFG.....	333
10.4.14	D_CLK_EXTCLK_DET_CFG, CLK_EXTCLK_DET_CFG.....	333
10.4.15	D_CLK_EXTCLK_DET_STATUS, CLK_EXTCLK_DET_STATUS.....	333
10.4.16	ANALOG_OSC_CTRL_0.....	334
10.4.17	ANALOG_OSC_CTRL_1.....	334
10.4.18	ANALOG_OSC_CTRL_THROTTLING*.....	335
11.	Memory.....	336
11.1	Introduction.....	336
11.2	Memory Architecture.....	336

onsemi Confidential
Ezairo 8300 Hardware Reference

11.2.1	Memory Instances	339
11.2.1.1	Filter Engine Memory.....	341
11.2.1.2	eMMC Memory Instance.....	341
11.2.1.3	Memory Instance Power Control.....	341
11.2.2	Memory Bus Structures.....	343
11.2.3	Memory Controllers and Arbitration.....	344
11.2.3.1	CFX Memory Arbitration.....	344
11.2.3.2	CFX Memory Interface Write Buffers.....	345
11.2.3.2.1	Buffering and Flushing of Write Operations.....	345
11.2.3.2.2	Visible Effects of Write Buffering.....	346
11.2.3.3	Arm Cortex-M3 Memory Arbitration.....	347
11.2.3.4	Shared Memory Bus Controller.....	347
11.2.3.5	FIFO Controller.....	348
11.2.3.6	Filter Engine Memory Accesses.....	348
11.3	CFX Processor Memory Mapping.....	348
11.3.1	Program and P Memory.....	348
11.3.1.1	Program Memory Space.....	350
11.3.1.2	P Memory Space.....	350
11.3.1.3	Program Memory Cache.....	352
11.3.1.4	Interrupt Vector Mapping.....	353
11.3.1.5	Configuration Register Mapping.....	353
11.3.2	Data Memory.....	354
11.3.2.1	CFX Data Memories (RAM and ROM).....	356
11.4	Arm Cortex-M3 Processor Memory Mapping.....	358
11.4.1	Arm Cortex-M3 Memory Elements.....	360
11.4.2	Program Memory Cache.....	362

11.4.3	Configuration Register Mapping.....	363
11.5	HEAR Configurable Accelerator Memory Mapping.....	365
11.5.1	Microcode Memory.....	365
11.5.2	Shared Memory.....	365
11.6	LPDSP32 DSP Memory Mapping.....	367
11.6.1	Program Memory.....	367
11.6.1.1	Program Memory Cache.....	367
11.6.2	Data Memory.....	368
11.7	Filter Engine Memory.....	370
11.8	Control and Configuration Registers.....	371
11.8.1	SYSCTRL_MEM_POWER_CFG0.....	372
11.8.2	SYSCTRL_MEM_POWER_CFG1.....	375
11.8.3	SYSCTRL_MEM_ACCESS_CFG0.....	377
11.8.4	SYSCTRL_MEM_ACCESS_CFG1.....	380
11.8.5	SYSCTRL_MEM_ARBITER_CFG.....	383
11.8.6	SYSCTRL_MEM_ERROR.....	384
11.8.7	SYSCTRL_CM3_LOOP_CACHE_CFG.....	384
11.8.8	SYSCTRL_DSP_LOOP_CACHE_CFG.....	385
11.8.9	SYSCTRL_PRAM_CFG.....	385
12.	FIFO Controller.....	387
12.1	Introduction to the FIFO Controller.....	387
12.1.1	FIFO Control and Configuration Register Conventions.....	387
12.2	Defining Memory for Use as a FIFO.....	388
12.2.1	Control and Configuration Registers.....	390
12.2.1.1	D_FIFO_x_START, FIFO_x_START.....	390
12.2.1.2	D_FIFO_x_END, FIFO_x_END.....	390

Ezairo 8300 Hardware Reference

12.2.1.3	D_FIFO_x_SIZE, FIFO_x_SIZE	390
12.3	FIFO Buffer Data and Configuration	391
12.3.1	FIFO Input and Output Block	391
12.3.2	FIFO Processing Block	393
12.3.3	Automatic FIFO Mute	394
12.3.4	Typical FIFO Configurations	394
12.3.4.1	Input FIFO Configuration	394
12.3.4.2	Output FIFO Configuration	395
12.3.5	Control and Configuration Registers	396
12.3.5.1	D_FIFO_x_BASE_PTR, FIFO_x_BASE_PTR	397
12.3.5.2	D_FIFO_x_IOBLOCK_PTR, FIFO_x_IOBLOCK_PTR	397
12.3.5.3	D_FIFO_x_BLOCK_SIZE, FIFO_x_BLOCK_SIZE	397
12.3.5.4	D_FIFO_x_BLOCK_CNT, FIFO_x_BLOCK_CNT	397
12.3.5.5	D_FIFO_x_ACCESS, FIFO_x_ACCESS	397
12.3.5.6	D_FIFO_x_AUTO_MUTE, FIFO_x_AUTO_MUTE	397
12.4	FIFO Interrupt Configuration	398
12.4.1	Control and Configuration Registers	398
12.4.1.1	D_FIFO_INT_CFX_CFG*, FIFO_INT_CFX_CFG_*	398
12.4.1.2	D_FIFO_INT_HEAR_CFG*, FIFO_INT_HEAR_CFG_*	399
12.4.1.3	D_FIFO_INT_CM3_CFG*, FIFO_INT_CM3_CFG_*	400
12.5	Access Priority	400
13.	Input/Output Audio Control	401
13.1	Audio Multiplexing	401
13.1.1	Audio Multiplexing with Filter Engine Outputs	404
13.1.2	Audio Multiplexing of Audio Input Sources	405
13.1.3	Control and Configuration Registers	405

13.1.3.1	AUDIO_MUX_CFG.....	406
13.1.3.2	AUDIO_BYPASS_DATA*.....	407
13.1.3.3	AUDIO_IOC_FENG_CFG.....	407
13.2	Introduction to the Input/Output Controllers.....	411
13.3	IOC Configuration.....	413
13.4	IOC Input.....	413
13.4.1	Control and Configuration Registers.....	414
13.4.1.1	D_IOC*_INPUT_CFG, IOC*_INPUT_CFG.....	415
13.4.1.2	D_IOC*_INPUT_PCM_CFG, IOC*_INPUT_PCM_CFG.....	420
13.4.1.3	D_IOC*_FIFO_ACCESS_CFG, IOC*_FIFO_ACCESS_CFG.....	423
13.4.1.4	D_IOC*_IN0_DATA, IOC*_IN0_DATA.....	426
13.4.1.5	D_IOC*_IN1_DATA, IOC*_IN1_DATA.....	427
13.4.1.6	D_IOC*_IN2_DATA, IOC*_IN2_DATA.....	427
13.4.1.7	D_IOC*_IN3_DATA, IOC*_IN3_DATA.....	427
13.5	IOC Output.....	427
13.5.1	Control and Configuration Registers.....	428
13.5.1.1	D_IOC*_OUTPUT_CFG, IOC*_OUTPUT_CFG.....	428
13.5.1.2	D_IOC*_OUT0_DATA, IOC*_OUT0_DATA.....	435
13.5.1.3	D_IOC*_OUT1_DATA, IOC*_OUT1_DATA.....	435
13.6	IOC Data Transfer Timing.....	436
14.	Input Stage.....	438
14.1	Overview.....	438
14.2	Input Selection.....	440
14.3	Analog-to-Digital Converters (ADCs).....	440
14.4	Decimation Filtering.....	444
14.4.1	DC Removal Filters.....	446

14.5 Configuration Registers	446
14.5.1 AUDIO_ADC_CFG*.....	447
14.5.2 AUDIO_ADC_CTRL*.....	448
14.5.3 AUDIO_ADC_FDEC_CTRL*.....	450
14.5.4 AUDIO_ADC_DEC_CTRL*.....	450
14.5.5 AUDIO_ADC_SAMPLE_FREQ_CFG.....	452
14.5.6 AUDIO_DMIC_OUTPUT_CFG.....	453
14.5.7 AUDIO_OUTPUT_DRIVER_CFG.....	454
14.5.8 AUDIO_OUTPUT_GAIN.....	454
14.5.9 AUDIO_OUTPUT_LIMITER.....	454
14.5.10 AUDIO_ADC_OFFSET_TRIM0*.....	455
14.5.11 AUDIO_ADC_OFFSET_TRIM1*.....	455
14.5.12 AUDIO_ADC_OFFSET_TRIM2*.....	455
15. Output Stage.....	456
15.1 Overview.....	456
15.2 Interpolation Filtering.....	456
15.3 Sigma-Delta Modulator.....	457
15.3.1 Output Limiter.....	459
15.4 Output Driver.....	459
15.5 Configuration Registers.....	460
15.5.1 AUDIO_SDM_CTRL*.....	460
15.5.2 AUDIO_SDM_OFFSET_CTRL*.....	461
15.5.3 AUDIO_OUTPUT_TEST*.....	461
15.5.4 AUDIO_SDM_DCRM_CTRL*.....	462
16. Input/Output Pad Configuration.....	463
16.1 Overview.....	463

16.2	Functional Configuration.....	464
16.2.1	Special Functional Configurations.....	468
16.3	Physical Configuration.....	469
16.4	DIO Registers.....	469
16.4.1	DIO_CFG*.....	471
16.4.2	DIO_CM3_JTAG_PAD_CFG.....	473
16.4.3	DIO_CLKDIV_CFG.....	474
16.4.4	DIO_SRC_SYNC_SLAVE.....	476
16.4.5	DIO_SRC_PCM*.....	477
16.4.6	DIO_SRC_SPI*.....	481
16.4.6.1	DIO_SRC_SPI_IO*.....	484
16.4.7	DIO_SRC_I3C*.....	489
16.4.8	DIO_SRC_I2C*.....	491
16.4.9	DIO_SRC_CLKDIV.....	494
16.4.9.1	DIO_SRC_NMI.....	497
16.4.10	DIO_SRC_EMMC_0.....	498
16.4.11	DIO_SRC_EMMC_1.....	504
16.4.12	DIO_SRC_EMMC_2.....	509
16.4.13	DIO_SRC_DMIC_CLK.....	510
16.4.14	DIO_SRC_DMIC_DATA.....	512
16.4.15	DIO_SRC_DSP_JTAG.....	517
16.4.16	DIO_SRC_UART.....	521
17.	Debug Ports and Security.....	523
17.1	Debug.....	523
17.1.1	Primary Debug Ports for the CFX DSP and the Arm Cortex-M3 Processor.....	523
17.1.1.1	I2C Debug Ports.....	523

17.1.1.1.1	I2C Debug Port Control and Configuration Registers.....	525
17.1.1.1.1.1	D_DEBUG_CFG, DEBUG_CFG.....	526
17.1.1.1.1.2	D_DEBUG_DATA, DEBUG_DATA.....	526
17.1.1.1.1.3	D_I2C_CFX_DEBUG_ADDR1.....	526
17.1.1.1.1.4	D_I2C_CFX_DEBUG_ADDR2.....	526
17.1.1.1.1.5	D_I2C_CM3_DEBUG_ADDR.....	527
17.1.1.1.1.6	D_PRAM_SECURITY_CFG, PRAM_SECURITY_CFG.....	527
17.1.1.1.1.7	SYSCTRL_DBG_LOCK.....	528
17.1.1.2	SWJ-DP Debug Port.....	528
17.1.1.2.1	SWJ-DP as CFX Debug Port Bridge.....	529
17.1.2	CFX Debug Port Controller.....	529
17.1.2.1	Breakpoints.....	530
17.1.2.2	External Breakpoint Configuration.....	530
17.1.2.2.1	Double-Address Mode.....	531
17.1.2.2.2	Memory Conflict Modes.....	531
17.1.2.3	NOAHlink Digital Audio Streaming Configuration and Usage.....	531
17.1.2.4	CFX Debug Port Specific Control and Configuration Registers.....	533
17.1.2.4.1	D_EXT_BREAK_CFG, EXT_BREAK_CFG.....	533
17.1.2.4.2	D_EXT_BREAK_ADDR_LOW, EXT_BREAK_ADDR_LOW.....	534
17.1.2.4.3	D_EXT_BREAK_ADDR_HIGH, EXT_BREAK_ADDR_HIGH.....	534
17.1.3	Arm Cortex-M3 Debug.....	535
17.1.4	LPDSP32 DSP Debug Port.....	535
17.1.5	Control and Configuration Registers.....	535
17.1.5.1	SYSCTRL_DSP_DEBUG_CFG.....	536
17.1.6	Filter Engine Debug Support Registers.....	536
17.2	System Security.....	537

18. Interfaces.....	538
18.1 General-Purpose I/O (GPIO) Interface.....	538
18.1.1 GPIO Interrupts.....	539
18.1.2 GPIO Registers.....	540
18.1.2.1 DIO_INPUT_DATA_0.....	541
18.1.2.2 DIO_INPUT_DATA_1.....	543
18.1.2.3 DIO_GPIO_OUTPUT_DATA_0.....	544
18.1.2.4 DIO_GPIO_OUTPUT_DATA_1.....	546
18.1.2.5 DIO_GPIO_OUTPUT_DATA_0_SET.....	547
18.1.2.6 DIO_GPIO_OUTPUT_DATA_1_SET.....	548
18.1.2.7 DIO_GPIO_OUTPUT_DATA_0_CLR.....	548
18.1.2.8 DIO_GPIO_OUTPUT_DATA_1_CLR.....	549
18.1.2.9 DIO_DIR_0.....	550
18.1.2.10 DIO_DIR_1.....	553
18.1.2.11 DIO_MODE_0.....	555
18.1.2.12 DIO_MODE_1.....	558
18.1.2.13 DIO_INT_CFG*.....	559
18.1.2.14 DIO_INT_STATUS.....	561
18.1.2.15 DIO_INT_DEBOUNCE.....	562
18.2 Digital Microphone (DMIC) Inputs and Outputs.....	563
18.2.1 Digital Microphone Inputs.....	563
18.2.2 Digital Microphone Outputs.....	564
18.3 eMMC Interface.....	565
18.3.1 eMMC Registers.....	568
18.3.1.1 INTERFACE_EMMC_CTRL.....	571
18.3.1.2 EMMC_HRS00.....	571

Ezairo 8300 Hardware Reference

18.3.1.3	EMMC_HRS01.....	571
18.3.1.4	EMMC_HRS04.....	572
18.3.1.5	EMMC_HRS06.....	572
18.3.1.6	EMMC_HRS07.....	573
18.3.1.7	EMMC_HRS30.....	573
18.3.1.8	EMMC_HRS31.....	573
18.3.1.9	EMMC_HRS32.....	574
18.3.1.10	EMMC_HRS33.....	574
18.3.1.11	EMMC_HRS34.....	574
18.3.1.12	EMMC_HRS35.....	574
18.3.1.13	EMMC_HRS36.....	575
18.3.1.14	EMMC_HRS37.....	575
18.3.1.15	EMMC_HRS38.....	576
18.3.1.16	EMMC_CRS63.....	576
18.3.1.17	EMMC_SRS00.....	577
18.3.1.18	EMMC_SRS01.....	577
18.3.1.19	EMMC_SRS02.....	577
18.3.1.20	EMMC_SRS03.....	577
18.3.1.21	EMMC_SRS04.....	579
18.3.1.22	EMMC_SRS05.....	579
18.3.1.23	EMMC_SRS06.....	579
18.3.1.24	EMMC_SRS07.....	579
18.3.1.25	EMMC_SRS08.....	579
18.3.1.26	EMMC_SRS09.....	579
18.3.1.27	EMMC_SRS10.....	581
18.3.1.28	EMMC_SRS11.....	584

onsemi Confidential
Ezairo 8300 Hardware Reference

18.3.1.29	EMMC_SRS12.....	586
18.3.1.30	EMMC_SRS13.....	588
18.3.1.31	EMMC_SRS14.....	590
18.3.1.32	EMMC_SRS15.....	592
18.3.1.33	EMMC_SRS16.....	594
18.3.1.34	EMMC_SRS17.....	596
18.3.1.35	EMMC_SRS18.....	598
18.3.1.36	EMMC_SRS19.....	598
18.3.1.37	EMMC_SRS20.....	598
18.3.1.38	EMMC_SRS24.....	600
18.3.1.39	EMMC_SRS25.....	600
18.3.1.40	EMMC_SRS26.....	600
18.3.1.41	EMMC_SRS27.....	601
18.3.1.42	EMMC_SRS29.....	601
18.4	I2C Interfaces.....	601
18.4.1	Slave Mode Specific Configuration.....	606
18.4.2	Master Mode Specific Configuration.....	607
18.4.3	Setting the System Clock.....	608
18.4.4	I2C Interrupts.....	608
18.4.4.1	Operation Using Manual Accomplishment.....	609
18.4.4.2	Operation Using Auto Accomplishment.....	609
18.4.4.3	I2C Main Modes of Operation.....	610
18.4.5	I2C Registers.....	611
18.4.5.1	D_I2C_CFG.....	613
18.4.5.2	D_I2C_CTRL.....	617
18.4.5.3	D_I2C_ADDR_START.....	618

Ezairo 8300 Hardware Reference

18.4.5.4 D_I2C_STATUS.....	618
18.4.5.5 D_I2C_TX_DATA.....	621
18.4.5.6 D_I2C_RX_DATA.....	621
18.4.5.7 D_I2C_RX_DATA_MIRROR.....	621
18.4.5.8 D_I2C_WATCHDOG_CFG.....	621
18.4.5.9 D_I2C_ERROR_CNT.....	622
18.4.5.10 I2C*_CFG.....	622
18.4.5.11 I2C*_CTRL.....	626
18.4.5.12 I2C*_ADDR_START.....	627
18.4.5.13 I2C*_STATUS.....	627
18.4.5.14 I2C*_TX_DATA.....	630
18.4.5.15 I2C*_RX_DATA.....	630
18.4.5.16 I2C*_RX_DATA_MIRROR.....	630
18.5 Low-Speed Analog-to-Digital (LSAD).....	630
18.5.1 LSAD Input Configuration.....	630
18.5.2 LSAD Sampling Configuration.....	631
18.5.3 LSAD Output Data.....	632
18.5.4 Power Supply Monitoring.....	633
18.5.5 LSAD and Power Supply Monitoring Interrupt.....	634
18.5.6 LSAD Gain Compensation Calibration.....	634
18.5.7 LSAD Registers.....	635
18.5.7.1 LSAD_DATA_AUDIO_CH*.....	636
18.5.7.2 LSAD_DATA_AUDIO_SAT_CH*.....	636
18.5.7.3 LSAD_DATA_TRIM_CH*.....	636
18.5.7.4 LSAD_DATA_TRIM_SAT_CH*.....	636
18.5.7.5 LSAD_INPUT_SEL_CH*.....	636

18.5.7.6	LSAD_DATA_VBAT.....	637
18.5.7.7	LSAD_DATA_VDDC.....	637
18.5.7.8	LSAD_DATA_VDDM.....	637
18.5.7.9	LSAD_CFG.....	637
18.5.7.10	LSAD_OFFSET_COMP.....	639
18.5.7.11	LSAD_GAIN_COMP.....	639
18.5.7.12	LSAD_BATMON_CFG.....	639
18.5.7.13	LSAD_BATMON_COUNT_VAL.....	640
18.5.7.14	LSAD_BATMON_STATUS.....	640
18.6	Pulse Code Modulation (PCM) Interfaces.....	640
18.6.1	PCM Signal Configuration.....	642
18.6.1.1	Frame Signal Configuration and Timing.....	643
18.6.1.2	Data Serial Input and Output Configuration.....	646
18.6.2	PCM/I2S Data Interface.....	646
18.6.3	PCM Interrupt Configuration.....	647
18.6.4	I2S Configuration and Usage.....	648
18.6.5	PCM Registers.....	650
18.6.5.1	PCM*_CFG.....	651
18.6.5.2	PCM*_CTRL.....	655
18.6.5.3	PCM*_STATUS.....	655
18.6.5.4	PCM*_TX_DATA0.....	656
18.6.5.5	PCM*_TX_DATA1.....	656
18.6.5.6	PCM*_RX_DATA0.....	656
18.6.5.7	PCM*_RX_DATA1.....	656
18.7	Serial Peripheral Interfaces (SPI).....	656
18.7.1	SPI Data Transfers.....	657

18.7.2	SPI Interrupts.....	661
18.7.3	SPI DMA Control.....	662
18.7.4	SPI Registers.....	663
18.7.4.1	SPI*_CFG.....	664
18.7.4.2	SPI*_CTRL.....	666
18.7.4.3	SPI*_STATUS.....	667
18.7.4.4	SPI*_TX_DATA.....	668
18.7.4.5	SPI*_RX_DATA.....	668
18.7.4.6	SPI*_RX_DATA_NO_START.....	668
18.7.4.7	SPI*_RX_DATA_MIRROR.....	669
18.8	Universal Asynchronous Receiver-Transmitter (UART) Interfaces.....	669
18.8.1	UART Interrupts.....	670
18.8.2	UART Interface Registers.....	671
18.8.2.1	UART*_CFG.....	671
18.8.2.2	UART*_CTRL.....	673
18.8.2.3	UART*_STATUS.....	673
18.8.2.4	UART*_TX_DATA.....	674
18.8.2.5	UART*_RX_DATA.....	674
19.	Peripherals.....	675
19.1	Asynchronous Sample Rate Converter (ASRC).....	675
19.1.1	ASRC Control and Configuration Registers.....	679
19.1.1.1	ASRC_CFG.....	680
19.1.1.2	ASRC_CTRL.....	681
19.1.1.3	ASRC_STATUS.....	681
19.1.1.4	ASRC_CNT.....	682
19.1.1.5	ASRC_IN.....	682

Ezairo 8300 Hardware Reference

19.1.1.6 ASRC_OUT.....	682
19.1.1.7 ASRC_OUT_MIRROR.....	682
19.1.1.8 ASRC_PHASE_INC.....	683
19.1.1.9 ASRC_PHASE_CNT.....	683
19.1.1.10 ASRC_STATE_MEM*.....	683
19.2 Audio Sink Clock Counters.....	683
19.2.1 Audio Sink Control and Configuration Registers.....	685
19.2.1.1 AUDIOSINK*_CFG.....	686
19.2.1.2 AUDIOSINK*_CTRL.....	686
19.2.1.3 AUDIOSINK*_STATUS.....	686
19.2.1.4 AUDIOSINK*_CNT.....	687
19.2.1.5 AUDIOSINK*_PHASE_CNT.....	687
19.3 Cyclic Redundancy Check (CRC) Generators.....	687
19.3.1 CRC-CCITT Generators.....	687
19.3.2 CRC-32 Generator.....	688
19.3.3 Non-standard CRC Variants.....	688
19.3.4 CRC Registers.....	689
19.3.4.1 Arm Cortex-M3 Processor CRC Registers.....	689
19.3.4.1.1 CRC_CFG.....	690
19.3.4.1.2 CRC_VALUE.....	690
19.3.4.1.3 CRC_ADD_1.....	691
19.3.4.1.4 CRC_ADD_8.....	691
19.3.4.1.5 CRC_ADD_16.....	691
19.3.4.1.6 CRC_ADD_24.....	691
19.3.4.1.7 CRC_ADD_32.....	691
19.3.4.1.8 CRC_FINAL.....	691

19.3.4.2	CFX CRC Registers.....	692
19.3.4.2.1	D_CRC_VALUE.....	692
19.3.4.2.2	D_CRC_ADD_1.....	692
19.3.4.2.3	D_CRC_ADD_8.....	692
19.3.4.2.4	D_CRC_ADD_16.....	692
19.3.4.2.5	D_CRC_ADD_24.....	693
19.3.4.2.6	D_CRC_ADD_32.....	693
19.4	Direct Memory Access (DMA) Controller.....	693
19.4.1	Block Overview.....	693
19.4.2	Functional Description.....	694
19.4.2.1	DMA to Peripherals Interface.....	694
19.4.2.2	Configuration and Status.....	696
19.4.2.3	DMA Interrupt Configuration.....	699
19.4.2.4	Circular Mode.....	699
19.4.2.5	Word Size.....	700
19.4.2.6	Packing and Unpacking.....	700
19.4.2.7	Triggering DMA Channels.....	705
19.4.2.8	DMA Operation.....	705
19.4.2.9	DMA Channel Arbiter.....	707
19.4.3	DMA Control and Configuration Registers.....	707
19.4.3.1	DMA*_CFG0.....	712
19.4.3.2	DMA*_CFG1.....	718
19.4.3.3	DMA*_CTRL.....	718
19.4.3.4	DMA*_STATUS.....	721
19.4.3.5	DMA*_SRC_ADDR.....	721
19.4.3.6	DMA*_DEST_ADDR.....	721

Ezairo 8300 Hardware Reference

19.4.3.7	DMA*_CNTS.....	722
19.4.3.8	DMA*_SRC_BUFFER.....	722
19.5	G.722 Hardware Accelerators.....	722
19.5.1	G.722 Decoder.....	722
19.5.1.1	G.722 SB-ADPCM Decoders.....	723
19.5.1.2	G.722 Decoder QMF Filterbank.....	725
19.5.1.3	G.722 Decoder Concealment Support.....	725
19.5.1.4	G.722 Decoder Input and Output Interfaces.....	726
19.5.2	G.722 Encoder.....	727
19.5.2.1	G.722 Encoder QMF Filterbank.....	727
19.5.2.2	G.722 SB-ADPCM Encoders.....	728
19.5.2.3	G.722 Encoder Input and Output Interfaces.....	729
19.5.3	G.722 Decoder and Encoder Configuration Registers.....	731
19.5.3.1	G722_STATUS.....	732
19.5.3.2	G722_INT_ENABLE.....	732
19.5.3.3	G722_CFG.....	733
19.5.3.4	G722_CTRL.....	734
19.5.3.5	G722_ENC_IN.....	735
19.5.3.6	G722_ENC_OUT.....	735
19.5.3.7	G722_DEC_IN.....	735
19.5.3.8	G722_DEC_OUT.....	735
19.5.3.9	G722_DEC_FBIN.....	735
19.5.3.10	G722_DEC_PLT.....	735
19.5.3.11	G722_DEC_RLT.....	735
19.5.3.12	G722_DEC_SX.....	736
19.5.3.13	G722_DEC_LSB_QUANT.....	736

Ezairo 8300 Hardware Reference

19.5.3.14	G722_DEC_HSB_QUANT.....	736
19.6	Math Accelerators.....	736
19.6.1	Log 2.....	736
19.6.2	Exp 2.....	737
19.6.3	Roots.....	738
19.6.4	Division.....	739
19.6.5	Control and Configuration Registers.....	740
19.6.5.1	D_X_MATH_DIVISOR_TRG, X_MATH_DIVISOR_TRG.....	745
19.6.5.2	D_X_MATH_LOG_EXP_ROOT_LSB, X_MATH_LOG_EXP_ROOT_LSB.....	745
19.6.5.3	D_X_MATH_LOG2_OUT_LSB, X_MATH_LOG2_OUT_LSB.....	745
19.6.5.4	D_X_MATH_EXP2_OUT_LSB, X_MATH_EXP2_OUT_LSB.....	745
19.6.5.5	D_X_MATH_ROOT_OUT_LSB, X_MATH_ROOT_OUT_LSB.....	745
19.6.5.6	D_X_MATH_LOG2_IN_LSB, X_MATH_LOG2_IN_LSB.....	746
19.6.5.7	D_X_MATH_ABS_LOG2_IN_LSB, X_MATH_ABS_LOG2_IN_LSB.....	746
19.6.5.8	D_X_MATH_EXP2_IN_LSB, X_MATH_EXP2_IN_LSB.....	746
19.6.5.9	D_X_MATH_ABSNEG_EXP2_IN_LSB, X_MATH_ABSNEG_EXP2_IN_LSB.....	746
19.6.5.10	D_X_MATH_ROOT2_IN_LSB, X_MATH_ROOT2_IN_LSB.....	746
19.6.5.11	D_X_MATH_ABS_ROOT2_IN_LSB, X_MATH_ABS_ROOT2_IN_LSB.....	746
19.6.5.12	D_X_MATH_ROOT3_IN_LSB, X_MATH_ROOT3_IN_LSB.....	746
19.6.5.13	D_X_MATH_ROOT4_IN_LSB, X_MATH_ROOT4_IN_LSB.....	746
19.6.5.14	D_X_MATH_ABS_ROOT4_IN_LSB, X_MATH_ABS_ROOT4_IN_LSB.....	746
19.6.5.15	D_X_MATH_DIVISOR, X_MATH_DIVISOR.....	747
19.6.5.16	D_X_MATH_DIV_OUT_LSB, X_MATH_DIV_OUT_LSB.....	747
19.6.5.17	D_Y_MATH_DIV_STATUS, Y_MATH_DIV_STATUS.....	747
19.6.5.18	D_Y_MATH_DIVIDEND, Y_MATH_DIVIDEND.....	747
19.6.5.19	D_Y_MATH_LOG_EXP_STATUS, Y_MATH_LOG_EXP_STATUS.....	747

Ezairo 8300 Hardware Reference

19.6.5.20	D_Y_MATH_LOG2_OUT_MSB, Y_MATH_LOG2_OUT_MSB.....	748
19.6.5.21	D_Y_MATH_EXP2_OUT_MSB, Y_MATH_EXP2_OUT_MSB.....	748
19.6.5.22	D_Y_MATH_ROOT_OUT_MSB, Y_MATH_ROOT_OUT_MSB.....	748
19.6.5.23	D_Y_MATH_LOG2_IN_MSB, Y_MATH_LOG2_IN_MSB.....	748
19.6.5.24	D_Y_MATH_ABS_LOG2_IN_MSB, Y_MATH_ABS_LOG2_IN_MSB.....	748
19.6.5.25	D_Y_MATH_EXP2_IN_MSB, Y_MATH_EXP2_IN_MSB.....	748
19.6.5.26	D_Y_MATH_ABSNEG_EXP2_IN_MSB, Y_MATH_ABSNEG_EXP2_IN_MSB.....	749
19.6.5.27	D_Y_MATH_ROOT2_IN_MSB, Y_MATH_ROOT2_IN_MSB.....	749
19.6.5.28	D_Y_MATH_ABS_ROOT2_IN_MSB, Y_MATH_ABS_ROOT2_IN_MSB.....	749
19.6.5.29	D_Y_MATH_ROOT3_IN_MSB, Y_MATH_ROOT3_IN_MSB.....	749
19.6.5.30	D_Y_MATH_ROOT4_IN_MSB, Y_MATH_ROOT4_IN_MSB.....	749
19.6.5.31	D_Y_MATH_ABS_ROOT4_IN_MSB, Y_MATH_ABS_ROOT4_IN_MSB.....	749
19.6.5.32	D_Y_MATH_DIVIDEND_TRG, Y_MATH_DIVIDEND_TRG.....	749
19.6.5.33	D_Y_MATH_DIV_OUT_MSB, Y_MATH_DIV_OUT_MSB.....	749
19.7	Reed-Solomon Hardware Accelerators.....	750
19.7.1	Reed-Solomon Encoder.....	750
19.7.1.1	Using the Reed-Solomon Encoder with Interrupts.....	752
19.7.1.2	Using the Reed-Solomon Encoder with the DMA Controller.....	752
19.7.2	Reed-Solomon Decoder.....	753
19.7.2.1	Using the Reed-Solomon Decoder with Interrupts.....	755
19.7.2.2	Using the Reed-Solomon Decoder with the DMA Controller.....	756
19.7.3	Reed-Solomon Configuration and Control Registers.....	757
19.7.3.1	RS_STATUS.....	757
19.7.3.2	RS_INT_ENABLE.....	759
19.7.3.3	RS_CFG.....	760
19.7.3.4	RS_BLK_CNT.....	761

19.7.3.5 RS_CTRL.....	762
19.7.3.6 RS_ENC_IN.....	762
19.7.3.7 RS_ENC_OUT.....	762
19.7.3.8 RS_DEC_IN.....	762
19.7.3.9 RS_DEC_OUT.....	762
19.7.3.10 RS_PTR.....	762
19.8 Timers.....	762
19.8.1 Starting or Stopping Timers.....	763
19.8.2 Mode Selection.....	763
19.8.3 GPIO Interrupt Capture Mode.....	764
19.8.4 Timer Registers.....	764
19.8.4.1 TIMER*_CFG0.....	766
19.8.4.2 TIMER*_CFG1.....	766
19.8.4.3 TIMER*_CTRL.....	768
19.8.4.4 TIMER*_VAL.....	768
19.8.4.5 TIMER*_VAL_CAPTURE.....	768
19.9 Watchdog Timer.....	768
19.9.1 Watchdog Interrupt.....	769
19.9.2 Control and Configuration Registers.....	770
19.9.2.1 D_WATCHDOG_CFG, WATCHDOG_CFG.....	770
19.9.2.2 WATCHDOG_RESET, D_WATCHDOG_RESET.....	770
A. Control and Configuration Registers.....	771
A.1 System Control.....	773
A.2 Watchdog Timer Configuration.....	774
A.3 Boot Status.....	775
A.4 HEAR Function Chain Commands.....	776

onsemi Confidential
Ezairo 8300 Hardware Reference

A.5 PRAM Security.....	777
A.6 CFX/Arm Cortex-M3 Control and Configuration.....	778
A.7 External Breakpoint Configuration.....	779
A.8 Hardware CRC.....	780
A.9 Chip Version Information.....	781
A.10 Debug Port Configuration.....	782
A.11 Interrupt Configuration.....	783
A.12 Clock Configuration.....	816
A.13 IOC Configuration.....	819
A.14 Analog.....	827
A.15 Audio.....	831
A.16 I2C Interface.....	838
A.17 FENG Control and Configuration.....	842
A.18 FENG Register File.....	862
A.19 Chip Identification.....	863
A.20 System Control.....	864
A.21 General-Purpose Timers.....	875
A.22 DMA Controller Configuration and Control.....	879
A.23 Audio Sink Clock Counters.....	901
A.24 ASRC Configuration and Control.....	903
A.25 CRC Generator Control.....	905
A.26 G.722 Configuration and Control.....	907
A.27 Reed Solomon Configuration and Control.....	910
A.28 DIO Interface and Digital Pad control.....	913
A.29 PCM Interface Configuration and Control.....	920
A.30 SPI Interface Configuration and Control.....	925

Ezairo 8300 Hardware Reference

A.31	UART Interface Configuration and Control.....	931
A.32	I2C Interface Configuration and Control.....	933
A.33	Analog-to-Digital Converter and Battery Monitoring.....	940
A.34	eMMC and I3C Interface Control.....	943
A.35	I3C Interface Configuration and Control.....	945
A.36	eMMC Interface Configuration Control.....	985
A.37	Neural Network Accelerator.....	1007
A.38	SYSTICK Timer.....	1011
A.39	System Control and ID register not in the SCB.....	1012
A.40	Nested Vector Interrupt Controller.....	1013
A.41	System Control Block.....	1040
A.42	Debug Controller.....	1045
A.43	CFX Math Accelerators I/O and Control.....	1047
A.44	FIFO A0 Configuration and Control.....	1049
A.45	FIFO A1 Configuration and Control.....	1057
A.46	FIFO B0 Configuration and Control.....	1065
A.47	FIFO B1 Configuration and Control.....	1073
A.48	FIFO Interrupt Selection.....	1081
A.49	HEAR Interrupt Configuration and Control.....	1082
A.50	Block Floating Point Configuration and Control.....	1088
A.51	CFX Math Accelerators I/O.....	1089
B.	Sampling Frequency Reference.....	1091
C.	Glossary.....	1093

Page

CHAPTER 1

Introduction

1.1 PURPOSE

IMPORTANT: onsemi acknowledges that this document might contain the inappropriate terms “white list”, “master” and “slave”. We have a plan to work with other companies to identify an industry wide solution that can eradicate non-inclusive terminology but maintains the technical relationship of the original wording. Once new terminologies are agreed upon, future products will contain new terminology.

This manual provides a reference to the system hardware for application developers working with Ezairo® 8300. This manual describes all of the components belonging to the Ezairo 8300 chip, including:

- Data processing and control components
- Memory components
- Peripherals
- External interfaces

The data processing and control component information provided by this manual complements:

- The CFX DSP description in the *CFX DSP Architecture Manual for Ezairo 8300*
- The Arm® Cortex®-M3 core description in the *The Definitive Guide to the Arm Cortex-M3* and other third-party documentation for the Arm Cortex-M3 processor
- The HEAR coprocessor description in the *HEAR Configurable Accelerator Reference Manual for Ezairo 8300*
- The Arm Cortex-M3 core description in the *The Definitive Guide to the Arm Cortex-M3* and other third-party documentation for the Arm Cortex-M3 processor
- The Filter Engine coprocessor description in the *Filter Engine Reference Manual for Ezairo 8300*

It further describes how each component of the Ezairo 8300 chip can be used in the implementation of an application, and provides information about the configuration of the various system components.

The *Ezairo 8300 Hardware Reference* is a part of the Ezairo 8300 Software Development Kit (Ezairo 8300 SDK).

1.2 INTENDED AUDIENCE

This manual is primarily intended for engineers and other individuals who are responsible for developing and/or maintaining signal processing applications that make use of an Ezairo 8300 chip. People who are developing communications between an external device and Ezairo 8300, as well as those interested in the details of the audio components, will find this manual particularly helpful, as it focuses on the configuration and use of system components such as the HEAR Accelerator core in the implementation of a signal processing application by the CFX DSP core.

This manual assumes that readers are familiar with C and/or assembly-level programming, digital signal processing (DSP) application concepts, the design of basic audio systems, and DSP tools.

1.3 CONVENTIONS

The following typographical conventions are used in this manual:

- Control and configuration registers are shown in a special font using their full descriptors, depending on which core the register is accessing. The full description takes the form `<PREFIX><GROUP>_<REGISTER>`.

Ezairo 8300 Hardware Reference

- All control and configuration registers are accessible from both the CFX and Arm Cortex-M3 processors.
- A register prefix of `D_` is used:
 - For CFX and Arm Cortex-M3 processor accesses to digital components in the CFX area where similar components exist in the Arm Cortex-M3 area of the design (ex., `D_I2C_CFG`).
 - For CFX processor accesses to other digital components in the CFX area. This forces register names for digital entries in the CFX system to be backward compatible with Ezairo 7100.
- A register address prefixed with `P`: refers to the P memory address space, accessible through the P (IO) memory bus and used by the CFX DSP.
- A register address prefixed with `X`: or `Y`: refers to the CFX DSP's X or Y memory address spaces, which are used to store selected 24-bit registers and are accessed through the X and Y memory buses respectively.
- A register address with no prefix refers to the memory address space used by the Arm Cortex-M3 processor.
- A register address prefixed with `DMIO`: is the "Data Memory: IO" or "Data Memory: Input/Output" address space of the LPDSP32.

NOTE: In cases where there are multiple instances of a block of registers, the summary of the registers at the beginning of the Control and Configuration Register section will have slightly different names from the detailed register sections below that table. For example, the `DMA*_CFG0` registers are referred to as `DMA_CFG0` when we are defining bit-fields and settings.

- The firmware provides access to these registers in two ways:
 - In the flat header files (e.g.: `sk5_hw_flat_cid*.h`, `sk5_cfx_hw_flat_cid*.h`), each register is individually accessible by directly using the naming provided in this manual. This is helpful for assembly and low-level C programming.
 - In the normal header files (e.g.: `sk5_hw_cid*.h`, `sk5_cfx_hw_cid*.h`), each register group forms a structure, with the registers being defined as members within that structure. The structures defined by these header files provide access to registers under the naming conventions `PREFIX_GROUP->REGISTER` (for the structure) and `GROUP->REGISTER` (for the register).
 - For more information, see the Hardware Definitions chapter of the *Ezairo 8300 Firmware Reference*.
- Angle brackets "<" and ">" identify an optional parameter or indicate a placeholder for specific information. To use an optional parameter or replace a placeholder, specify the information within the brackets; do not include the brackets themselves.
- Default settings for registers and bit fields are marked with an asterisk (*).
- All sample rates specified are the final decimated sample rates, unless stated otherwise.
- In general, numbers are presented in decimal notation. In cases where hexadecimal or binary notation is more convenient, these numbers are identified by the prefixes "0x" and "0b" respectively. For example, the decimal number 123456 can also be represented as 0x1E240 or 0b11110001001000000.

1.4 FURTHER READING

For more information, refer to the following documents that are included in the Ezairo 8300 EDK:

- *Integrated Development Environment User's Guide for Ezairo 8300*
- *Ezairo 8300 Firmware Reference*
- *Audio Processor for Digital Hearing Aids - Ezairo 8300*
- *CFX DSP Architecture Manual for Ezairo 8300*
- *HEAR Configurable Accelerator Reference Manual for Ezairo 8300*
- *Filter Engine Reference Manual for Ezairo 8300*

For more information about the Arm Cortex-M3 processor, refer to the following documents:

Ezairo 8300 Hardware Reference

- *Cortex-M3 Technical Reference Manual*, revision r2p1
- *ARM and Thumb-2 Instruction Set Quick Reference Card*
- *Arm Cortex Microcontroller Software Interface Standard (CMSIS)*
- Yiu, Joseph *The Definitive Guide to the Arm Cortex-M3* (Newnes, December 2009) or a similar book describing the Arm Cortex-M3 core architecture

For more information about the LPDSP32 DSP, refer to these documents:

- *LPDSP32-V3 Hardware Reference Manual*
- *LPDSP32-V3 Block Diagram*
- *LPDSP32-V3 Interrupt Support Manual*
- *User Guide IP Programmer for LPDSP32-V3*

You can find Arm publications on their website at <http://infocenter.arm.com/help/index.jsp>.

Other publicly available standards that are relevant to this manual are:

- ITU-T G.722 (09/2012)
- *Specification of the Bluetooth® System*: <https://www.bluetooth.org/en-us/specification/adopted-specifications>

CHAPTER 2

System Overview

The Ezairo 8300 system is a mixed-signal system-on-chip (SoC) designed specifically for audio processing. It is based on an asymmetric architecture that includes six different processing cores: the CFX Digital Signal Processor (DSP), referred to in this group of topics as the CFX; the Arm Cortex-M3 processor; the HEAR Configurable Accelerator, referred to as the HEAR; the Filter Engine core, referred to as the Filter Engine; the LPDSP32 DSP, referred to as the LPDSP32; and the Neural Network Accelerator, referred to as the NNA.

- The CFX core acts as either the system master or a coprocessor to the Arm Cortex-M3 processor. This core:
 - Can be used to control the other cores, peripherals, interfaces and analog components.
 - Is typically used to coordinate the flow of signal data progressing through the system.
 - Can also be used for custom signal processing applications that are not handled by the HEAR, Filter Engine, or LPDSP32.
- The Arm Cortex-M3 processor functions as either a coprocessor to the CFX or as the system master. This core:
 - Can be used to control the other cores, peripherals, interfaces and analog components.
 - Is typically used to provide additional computation and support for external components, with a focus on supporting wireless capability and external devices that are part of the Ezairo 8300 system.
- The HEAR is designed to perform with both common signal processing operations and complex standard filterbanks, such as a WOLA filterbank, reducing the load on the CFX core.
- The Filter Engine is a coprocessor that provides basic filtering and arithmetic operations on audio signals, and is tightly integrated with the audio path to provide low delay signal processing.
- The LPDSP32 is a high-speed, high-capacity coprocessor used mainly for audio codecs, artificial intelligence applications using the NNA, and general purpose data processing.
- The NNA is a configurable hardware accelerator dedicated to supporting neural networks with high energy efficiency. It allows Ezairo 8300 to perform neural network computations with greatly enhanced efficiency and flexibility.

2.1 SYSTEM ARCHITECTURE INTRODUCTION

A top-level block overview of the components of the Ezairo 8300 system is provided in the "[Top-Level System Block Diagram](#)" figure (Figure 1). More information on the various component blocks is provided in the subsections that follow.

Ezairo 8300 Hardware Reference

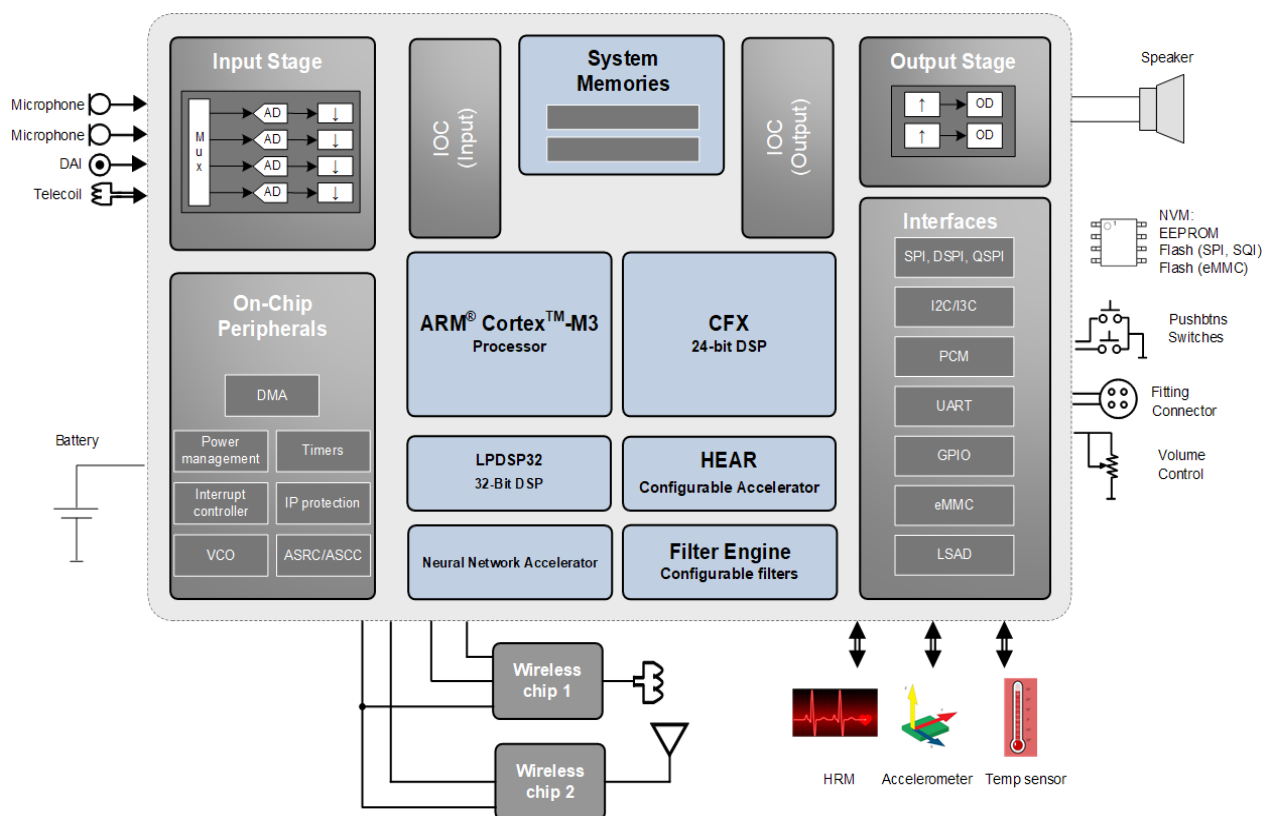


Figure 1. Top-Level System Block Diagram

2.1.1 System Control and Configuration

The CFX and the Arm Cortex-M3 processor both have master control of the Ezairo 8300 system. This means that each processor can:

- Control and configure all memory-mapped registers

NOTE: An exception to this is the Arm Cortex-M3 processor's private peripherals which are only accessible by the Arm Cortex-M3 processor.

- Access all system memories
- Configure and receive interrupts from all of the system's interrupt lines

Following a power-on reset (POR), the Ezairo 8300 system executes the Boot ROM portion of the Program ROM using the CFX. This ROM routine initializes all aspects of the system, including each of the other cores. Once complete, the Boot ROM loads a bootloader that executes on either the CFX core or the Arm Cortex-M3 core.

- When the Boot ROM loads a bootloader to the CFX, the CFX sets the `BOOT_STATUS_STATUS` bit from the `BOOT_STATUS` register to indicate that the CFX has been booted previously. The boot result is stored in `SYSVAR_SYS_ERRNO` (0x4008).

Ezairo 8300 Hardware Reference

- When the Boot ROM loads a bootloader to the Arm Cortex-M3 processor, the CFX sets the data at `D_CM3_START_ADDR` in program memory and triggers a `CM3_CMD_0` interrupt that is used to signal to a ROM routine running on the Arm Cortex-M3 processor to execute the bootloader application whose vector table is at the location that was placed into `D_CM3_START_ADDR`. When the Arm Cortex-M3 processor's simple ROM boots an application, the processor sets the `SYSCTRL_BOOT_STATUS_STATUS` bit from the `SYSCTRL_BOOT_STATUS` register to indicate that the Arm Cortex-M3 processor has been booted previously. The boot result is stored in `D_CM3_START_RESULT` (PMEM 0x805F).

For more information about the Program ROM, see the *Ezairo 8300 Firmware Reference*.

The inter-processor communication between the five system cores consists of:

- Interrupt/function chain triggering lines which provide inter-processor signals that can be used to signal events to each of the cores. These interrupts and function chain triggers are described in their respective processor configuration chapters.
- HEAR and LPDSP32 shared memories that are accessible by the CFX, the Arm Cortex-M3 processor, and the DMA. Part of the LPDSP32 shared memory is also accessible by the NNA.
- A set of first-in, first-out (FIFO) buffers that are used to:
 - Provide signal processing data from the input stage and Filter Engine for processing by the CFX, the HEAR, the Arm Cortex-M3 processor, and the DMA.
 - Transfer intermediate signal processing data between the CFX, the HEAR, the Arm Cortex-M3 processor, and the DMA.
 - Handle distribution of signal processing data from the CFX, the HEAR, the Arm Cortex-M3 processor, and the DMA to the output stage and Filter Engine.

2.1.2 Application Overview

A typical application executing on the Ezairo 8300 consists of:

- Signal processing code that is distributed across the five system cores. Typical use of the cores includes:
 - CFX: general-purpose DSP processing
 - HEAR: additional signal processing, with microcode modules supporting filterbank and vector processing of data blocks
 - LPDSP32, paired with an NNA: processing data using the NNA, or encoding/decoding data using an audio codec
 - Filter Engine: both low-latency processing, and to processes both decimated and undecimated input and output data, on a per-sample basis
 - Arm Cortex-M3 processor: general purpose micro controller functions for any additional signal processing, or to transfer signal data between Ezairo 8300 and other devices
- System configuration code that executes on the Arm Cortex-M3 processor or CFX. Typical configuration tasks for the Ezairo 8300 system include configuration of:
 - Input stage, PCM/I2S and DMIC interfaces, and the IOCs to obtain input signal information
 - IOC, output stage, and the PCM/I2S and DMIC interfaces to distribute output signal information
 - FIFO buffers for use in handling input, output and intermediate data
 - The other potential master processor (CFX or Arm Cortex-M3 processor), by setting configuration registers and loading a program to perform additional signal processing or external communications
 - The HEAR, by setting configuration registers and loading microcode to perform regular signal processing algorithms on the available data
 - The Filter Engine, by setting configuration registers and loading a phased program to perform any required input or output filtering in addition to low-delay path processing

Ezairo 8300 Hardware Reference

- The LPDSP32, by setting configuration registers and loading a program to perform regular signal processing algorithms
- Input and output pad configuration for the digital input/output (DIO) pads and external interfaces
- Clock and power components as needed for the user application
- Application code that handles communication with, and coordination of, the cores in the Ezairo 8300 system, that executes on the Arm Cortex-M3 processor, the CFX, and the LPDSP32.
- Application code that handles communication with, and/or signaling of, external components through the debug port and other external interfaces, which executes on the Arm Cortex-M3 processor or CFX.

Processing on the five system cores is supported by the input/output controllers (IOCs) and the FIFO controller, as follows:

- The IOCs can take inputs from an analog input stage, digital microphones, or pulse code modulation (PCM/I²S) interface input, and can deliver outputs to a direct digital output stage, digital microphone outputs, or a PCM/I²S interface output.
- The FIFO controller simplifies access to data by the CFX, the HEAR, Arm Cortex-M3 core, the DMA, and the IOCs, by providing configurable hardware-based FIFOs.

NOTE: The LPDSP32 is not connected to any FIFO buffers, but communicates with the Arm Cortex-M3 core and the CFX through shared memory. The DMA can be used to transfer data between the FIFO buffers and the LPDSP32 shared memory.

"Signal Data Flow in a Typical Application" figure (Figure 2), below, provides an overview of possible paths that signal data will typically flow through in the Ezairo 8300 system.

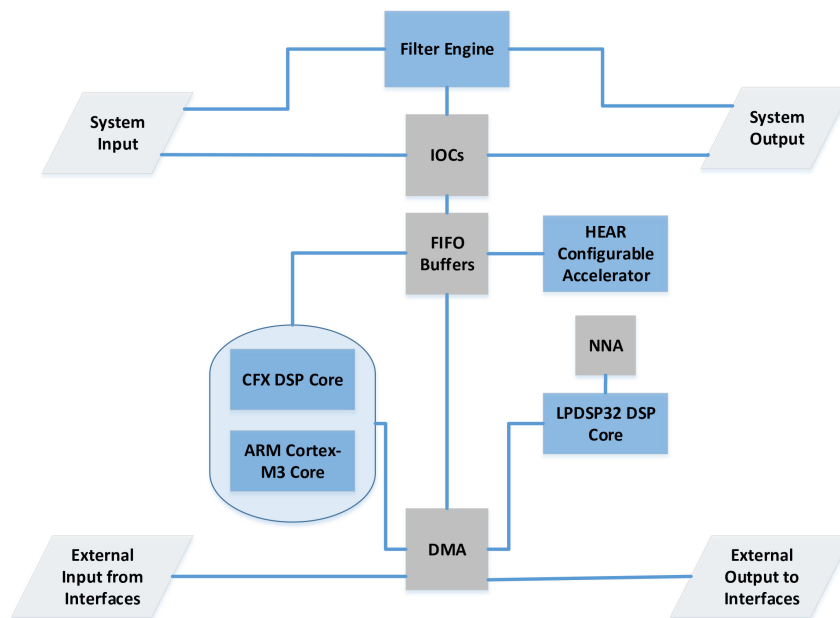


Figure 2. Signal Data Flow in a Typical Application

Signal data flow can be divided into three steps: data input, signal processing, and data output.

Data Input

For input, the signal data to be processed enters the system either through the input stage (either from the analog inputs or digital microphone inputs), or through one of the PCM/I²S interfaces. This data is processed through the input side of the IOCs directly to FIFOs in shared memory. Alternatively, the data from the input stage can be processed by the Filter Engine before, or instead of, being passed to the IOCs. Further information on configuration and data flow options can be found in [Chapter 13.1 "Audio Multiplexing" on page 401](#).

Signal Processing

The signal data stored in the FIFOs can then be processed through any number of data processing steps by functions implemented for the CFX, function chains available through the microcode configuration supplied for the HEAR, and/or functions implemented for the Arm Cortex-M3 processor or the LPDSP32. After each processing step, the data is typically written back to either

Ezairo 8300 Hardware Reference

an intermediate or output FIFO buffer. The LPDSP32 is the only core that does not have access to the FIFO buffers, but the DMA can be used to copy data between the FIFO buffers and the LPDSP32 shared memory.

The Filter Engine supports pre-filtering of input data, post-filtering of output data, and high-frequency/low group-delay processing. Processed data can be analyzed using the NNA, and the LPDSP32 can be used for decoding and encoding audio that is transferred over an external wireless radio chip.

Data Output

Once all of the desired data processing has been performed, the data is distributed from the system either directly through one or both of the PCM/I²S interfaces, from the outputs of the Filter Engine, or through the output side of the IOCs. As shown in [Chapter 13.1 "Audio Multiplexing" on page 401](#), the IOCs can then direct this output information to the PCM/I²S interfaces or the output stage (including outputs to both the direct-digital outputs and the digital microphone outputs).

Algorithms running on Ezairo 8300 devices can adapt their behavior based on external conditions or stimuli. For example, the system might receive an input signal attached to a DIO pin that is configured as an input to the low-speed analog-to-digital (LSAD) converter to provide signal data that is used to control the gain, and hence the output volume, of the output signal provided by the system. As these components of an application might be used for a variety of things, including collecting information from an external sensor, supplying information to another device in the larger system, or other configurations. This portion of an application is usually defined based on the overall product needs, and is specific to a given product; therefore, this document does not attempt to provide a generic view of such application components.

2.2 CFX DIGITAL SIGNAL PROCESSOR

The CFX is a user-programmable, general-purpose DSP with 32-bit instructions and a 24-bit data path. The CFX can serve many roles in the system: it can be used as the system master; it can be used to control and configure all peripherals and external interfaces; and it can be used to coordinate and assist in all data processing tasks performed by the six cores.

The CFX is closely tied to its local memories, and has access to all other memory-mapped components of the system except for the items mapped onto the Arm Cortex-M3 core's private peripheral bus.

More detailed information on the CFX is available in [Chapter 3.1 "Processor Overview" on page 59](#), and in the *CFX DSP Architecture Manual for Ezairo 8300*.

2.2.1 CFX Memory

The CFX uses a dual-Harvard memory bus architecture to provide separate buses for its program memory and two separate data memories.

The program memory space available to the CFX includes a program ROM as well as program RAM space to store a user application and interrupt vectors. This memory is accessible as instructions to the CFX through the instruction memory bus (referred to by the CFX toolchain as PMEM), and as data for the CFX through the P memory bus (IOMEM). Additionally, the P memory space also provides the CFX with access to:

- The HEAR core's microcode memory
- The Arm Cortex-M3 processor's:
 - Program memory
 - Data memory

Ezairo 8300 Hardware Reference

- Control and configuration registers
- The Filter Engine's memory and registers
- The LPDSP32's memory
- System control and configuration registers

The data memory areas available to the CFX are divided into two memory spaces that can be accessed either separately through the X memory and Y memory buses as 24-bit data, or together through these buses as 48-bit data in XY memory. These memory buses also provide access for the CFX to the HEAR shared memory, and the HEAR's configuration registers.

More detailed information about the memories and associated memory buses is available in [Chapter 11.1 "Introduction " on page 336](#).

2.3 ARM CORTEX-M3 PROCESSOR

The Arm Cortex-M3 processor is a general-purpose 32-bit microcontroller with an embedded multiplier and arithmetic logic unit (ALU) to handle typical control functions as well as DSP functions used for implementing a variety of applications. The Arm Cortex-M3 processor can be used as the system master or as a coprocessor that provides additional processing for the Ezairo 8300 system. This processing can be used to control communications using external components, such as wireless radio chips, and to process data from those communications. It can also be used to manage hardware acceleration peripherals, or to perform additional signal processing.

The Arm Cortex-M3 processor is closely tied to its local memories, and has access to all other memory-mapped components of the system.

More detailed information about the Arm Cortex-M3 processor is available from:

- [Chapter 4.1 "Processor Overview" on page 131](#)
- *The Cortex-M3 Technical Reference Manual*
- *The Definitive Guide to the Arm Cortex-M3* and other third-party documentation for the Arm Cortex-M3 processor

2.3.1 Arm Cortex-M3 Processor Memory

The Arm Cortex-M3 processor memory subsystem consists of a program memory area that is connected to the core's instruction and data buses, a data memory area connected to the core's system bus, and a private peripherals area. A 32-word read-only instruction cache, designed to increase power efficiency, has also been included on the I-Code memory bus between the Arm Cortex-M3 Processor and its addressable program memory. Additionally, the Arm Cortex-M3 processor can access the HEAR FIFO memory in parallel with memory-mapped components through its peripheral bus, and can access all CFX memories through a bridge, which can result in wait cycles on the Arm Cortex-M3 processor and, in certain situations, on the CFX. The Arm Cortex-M3 core is supplemented by an 8-channel DMA.

More detailed information about the Arm Cortex-M3 processor's memories and memory buses is available in [Chapter 11.1 "Introduction " on page 336](#).

2.4 LPDSP32 DSP

The LPDSP32 DSP is a low power, programmable, pipelined DSP that uses a dual-Harvard, dual-MAC architecture to efficiently process 32-bit signal data. This processor is included in the design to efficiently support advanced developments that can take advantage of the additional 32-bit data processing power that this core provides.

Ezairo 8300 Hardware Reference

This processor is closely tied to the Neural Network Accelerator (NNA) core and can be used to support and extend the Deep Learning and Artificial Intelligence (AI) functionality.

2.4.1 LPDSP32 Memories

The LPDSP32 uses a dual-Harvard memory bus architecture to provide separate buses for its program memory and two separate data memories. The LPDSP32 program memory is interpreted as 40-bit program memory read as two 20-bit reads from 32-bit memory. The LPDSP32 data memories are 32-bit.

2.5 HEAR CONFIGURABLE ACCELERATOR

The HEAR provides additional digital signal processing functionality within the Ezairo 8300 system. The signal processing functionality of the HEAR is configured through a set of pre-defined microcode modules. The microcode modules provide functions that can be linked together to form function chains. These function chains are triggered through the integrated function chain controller of the HEAR via commands issued by the CFX or Arm Cortex-M3 processor, or events from the FIFO controller.

The HEAR also has a number of other configurable components, and communicates with the CFX and Arm Cortex-M3 processor through its shared memories and CFX interrupts. The HEAR includes integrated block floating-point support, for maintaining signal fidelity when the processing steps demand a large dynamic range.

The HEAR is controlled by the system master, which controls the clock and execution state of the core, as well as the core's memory configuration. Either the CFX or the Arm Cortex-M3 processor can be used to configure and send commands to the HEAR's function chain controller.

More detailed information about the HEAR is available in [Chapter 5.1 "Accelerator Overview" on page 205](#), and in the *HEAR Configurable Accelerator Reference Manual for Ezairo 8300*.

2.5.1 HEAR Memory

The HEAR core uses a Harvard memory bus architecture that provides separate memories for the microcode modules and data.

Microcode memory contains the user-configured microcode modules for the HEAR. This memory is accessible from the CFX core and the Arm Cortex-M3 core through the P memory bus.

The HEAR shared memory is a set of distinct physical memories that are used to store temporary processing data, algorithm coefficients, and signal data, at all points during processing (from input, to partially processed signal data, to output). The CFX and Arm Cortex-M3 core can access this memory through the X and Y memory buses. Access to shared memory is managed by the shared memory bus controller. Some portions of this shared memory are further controlled by the FIFO controller, which provides FIFO access to other system components including the IOCs, the CFX, and the Arm Cortex-M3 processor.

More detailed information about the microcode and shared memories is available in [Chapter 5.1 "Accelerator Overview" on page 205](#), and in [Chapter 11.1 "Introduction " on page 336](#). Additional information about the FIFO controller and how it affects memory accesses is available in [Chapter 12.1 "Introduction to the FIFO Controller" on page 387](#).

2.6 FILTER ENGINE

The Filter Engine is a specialized processing core that is optimized to support pre- and post-processing of audio data. It also supports low-group delay processing of data at faster data rates than the system's sampling rate, by bypassing the input stage's decimation filters and output stage's interpolation filters. With flexible multiplexing

Ezairo 8300 Hardware Reference

between the Filter Engine signals and the default audio paths, the input and output stages can use the Filter Engine outputs to bypass processing with the default audio paths.

This processor is primarily designed to perform time-domain filter operations such as infinite-impulse response (IIR) filtering and finite impulse response (FIR) filtering. It supports these processing requirements using 64 second-order section (biquad) filters and 64 5-tap FIR filters that can be cascaded to form larger filtering structures.

The Filter Engine is controlled by the system master, which defines the Filter Engine program flow by controlling the execution and debug states of the core.

More detailed information about the Filter Engine is available in [Chapter 6.1 "Processor Overview" on page 217](#), and in the *Filter Engine Reference Manual for Ezairo 8300*.

2.6.1 Filter Engine Memory

The Filter Engine architecture includes the following memory elements:

- 256 word (36-bit per word) program memory
- Configuration registers
- 70 single-precision data registers (24 bits each) and 32 extended-precision data registers (54 bits each)
- 14 input registers and 12 output registers (24 bits each)
- 64 filter coefficient sets (each containing 5 coefficients, 28 bits per word) organized as a coefficient memory
- 64 state variable sets (each containing 5 variables, with 24 bits per word) organized as a state variable memory

More detailed information about the Filter Engine's memory structures is available in [Chapter 11.1 "Introduction" on page 336](#).

2.7 NEURAL NETWORK ACCELERATOR

The Neural Network Accelerator (NNA) is a hardware block designed to accelerate dense neural network heavy applications, or more generally, applications that must perform many multiply-accumulate (MAC) sequences operating over an input vector, with weights and bias, and applying an activation function.

The NNA's components include multipliers, accumulators, input registers, weight registers, a bias register, an input fetcher, a weight fetcher, weight decompression logic, a bias fetcher, shifters for bias and output, five types of activation functions (Linear, ReLu, Leaky ReLu, Sigmoid, and Tanh), and an output writer. Multiple weight compression methods are supported, including weight pruning.

For more information about the NNA, see [Chapter 7 "Neural Network Accelerator" on page 257](#).

2.8 FIFO CONTROLLER

The FIFO controller controls access to the subset of shared memory that can be configured for use as FIFO buffers. This controller has three purposes in managing access to shared memory:

- It remaps portions of the address space so that a hardware circular buffer is implemented, and the most recent samples in the buffer can always be accessed at the same logical address.
- It resolves access conflicts between the input/output controllers (IOC) and the three system cores that access these memory areas through the shared memory bus.
- It tracks incoming data to a buffer or outgoing data from a buffer. Changes to these buffers can trigger interrupts on the CFX, function chains on the HEAR, and interrupts on the Arm Cortex-M3 processor.

Ezairo 8300 Hardware Reference

More detailed information about configuring FIFOs and using the FIFO controller is available in [Chapter 12.1 "Introduction to the FIFO Controller" on page 387](#).

2.9 PERIPHERALS

2.9.1 Peripherals Accessible to the CFX and the Arm Cortex-M3 Core

The Ezairo 8300 system has a number of peripherals to support various tasks that the system needs to perform. These system peripherals include:

- An interrupt controller for the CFX, as described in [Section 3.2 "Interrupt Controller" on page 61](#).
- System security blocks that are used to restrict external access to application code and data, as described in [Chapter 17.1 "Debug" on page 523](#).
- Asynchronous sample rate converter and audio clock synchronization blocks
- Two CRC calculation units: one supporting a standard CRC-CCITT calculation, and one supporting both standard and non-standard forms of CRC32 and CRC-CCITT
- A dynamic-memory access (DMA) unit
- Hardware accelerators supporting Reed-Solomon error-correction coding, and the G.722 audio codec
- Math accelerators
- Four general-purpose timers
- A watchdog timer

More detailed information about configuring and using the peripherals is available in [Chapter 19.1 "Asynchronous Sample Rate Converter \(ASRC\)" on page 675](#).

2.9.2 Arm Cortex-M3 Processor Private Peripherals

Several system peripherals are accessible only to the Arm Cortex-M3 processor. They are:

- A standard nested-vectorized interrupt controller (NVIC)
- A system timer (SYSTICK)
- The debug controller for the Arm Cortex-M3 processor

For more information about the Arm Cortex-M3 processor's private peripherals, see [Section 4.6 "Arm Cortex-M3 Private Peripherals" on page 135](#).

2.10 INTERFACES

2.10.1 Input/Output Configuration

The majority of the interfaces in the Ezairo 8300 system are assigned to 36 data input/output (DIO) pads that are configured through the DIO block. Each of these pads can be configured as the source for a number of interfaces or as the outputs for a number of interface signals, system clocks and other system signals.

For more information about this DIO configuration, including physical configuration of the DIO pads, see [Chapter 16.1 "Overview" on page 463](#).

2.10.2 Debug Interfaces

The Ezairo 8300 system contains a number of debug interfaces to assist with debugging applications running on the system's programmable cores.

The CFX and Arm Cortex-M3 processor are supported by:

- Dedicated I/O pads for a shared I²C bus supporting:
 - An I²C debug interface for the CFX

Ezairo 8300 Hardware Reference

- An I²C debug interface for the Arm Cortex-M3 processor
- An SWJ-DP debug interface for the Arm Cortex-M3 processor, with memory-mapped registers that can be used to provide data to the I²C CFX debug interface

The CFX I²C debug interface, Arm Cortex-M3 processor's I²C debug interface, and the SWJ-DP debug interface are described in [Chapter 17.1 "Debug" on page 523](#). Further information about these debug ports can be found in the *Communication Protocols Manual for Ezairo 8300*.

The HEAR does not have a debug interface. Integration of microcode can be debugged using either the CFX or Arm Cortex-M3 processor debug interfaces.

The Filter Engine does not have a debug interface, but does provide a set of master control registers which can be used to debug programs written for this core. For more information on Filter Engine control support which can be used for debugging, see [Section 6.4.1 "Filter Engine Control" on page 219](#).

The LPDSP32 is supported by a JTAG debug interface that can be assigned to a set of DIO pads. For further information on the LPDSP32 debug port, see [Section 17.1.4 "LPDSP32 DSP Debug Port" on page 535](#).

The Ezairo 8300 system also contains activity counters for all five cores to assist with profiling an application that is running on the system. The counters are used to count the number of active execution cycles for each core since the activity counters were last cleared. The activity counters are collectively controlled by the `SYSCTRL_CNT_CTRL` register:

- To clear the activity counters, set the `SYSCTRL_CNT_CTRL_CNT_CLEAR` bit.
- To start the activity counters, set the `SYSCTRL_CNT_CTRL_CNT_START` bit.
- To stop the activity counters, set the `SYSCTRL_CNT_CTRL_CNT_STOP` bit.

IMPORTANT: The activity counters do not contain overflow protection, and simply wrap when they reach their maximum values while active. This limits accurate profiling to a maximum of 2^{32} active cycles on any core.

2.10.2.1 Activity Counter Control and Configuration Registers

Register Name	Register Description	Address
<code>SYSCTRL_CNT_CTRL</code>	Activity Counters Control	P:0x1010 0x40000040
<code>SYSCTRL_SYSCLK_CNT</code>	System Clock Counter Value	P:0x1011 0x40000044
<code>SYSCTRL_CM3_CNT</code>	CM3 Activity Counter Value	P:0x1012 0x40000048
<code>SYSCTRL_DSP_CNT</code>	DSP Activity Counter Value	P:0x1013 0x4000004C
<code>SYSCTRL_CFX_CNT</code>	CFX Activity Counter Value	P:0x1014

onsemi Confidential
Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
		0x40000050
SYSCTRL_HEAR_CNT	HEAR Activity Counter Value	P:0x1015 0x40000054
SYSCTRL_FENG_CNT	FENG Activity Counter Value	P:0x1016 0x40000058

2.10.2.1.1 SYSCTRL_CNT_CTRL

Bit Field	Read/Write	Field Name	Description
8	R	CNT_STATUS	Activity counters status bit
2	W	CNT_CLEAR	Clear activity counters
1	W	CNT_STOP	Stop activity counters
0	W	CNT_START	Start activity counters

Field Name	Value Symbol	Value Description	Hex Value
CNT_STATUS	CNT_STOPPED	Activity counters stopped	0x0*
	CNT_RUNNING	Activity counters running	0x1
CNT_CLEAR	CNT_CLEAR	Clear activity counters	0x1
CNT_STOP	CNT_STOP	Stop activity counters	0x1
CNT_START	CNT_START	Start activity counters	0x1

2.10.2.1.2 SYSCTRL_SYSClk_CNT

Bit Field	Read/Write	Field Name	Description
31:0	RW	SYSClk_CNT	System clock counter value

2.10.2.1.3 SYSCTRL_CM3_CNT

Bit Field	Read/Write	Field Name	Description
31:0	RW	CM3_CNT	Arm Cortex-M3 activity counter value

2.10.2.1.4 SYSCTRL_DSP_CNT

Bit Field	Read/Write	Field Name	Description
31:0	RW	DSP_CNT	DSP activity counter value

Ezairo 8300 Hardware Reference

2.10.2.1.5 SYSCTRL_CFX_CNT

Bit Field	Read/Write	Field Name	Description
31:0	RW	CFX_CNT	CFX activity counter value

2.10.2.1.6 SYSCTRL_HEAR_CNT

Bit Field	Read/Write	Field Name	Description
31:0	RW	HEAR_CNT	HEAR activity counter value

2.10.2.1.7 SYSCTRL_FENG_CNT

Bit Field	Read/Write	Field Name	Description
31:0	RW	FENG_CNT	FENG activity counter value

2.10.3 Interfaces for Communications and Signaling

The Ezairo 8300 system has a number of external interfaces that can be controlled by either the CFX or the Arm Cortex-M3 core. These interfaces provide support for a variety of types of external communications, as well as signaling through general-purpose input/output (GPIO) pads and low-speed analog-to-digital (LSAD) converters. The interfaces are generally accessible through the CFX, the Arm Cortex-M3 processor, and the DMA peripheral.

The external interfaces provided are:

- GPIOs
- An eMMC interface, with custom interface buffering
- Three general-purpose I²C interfaces, including one interface that uses the I²C bus from the shared I²C debug interface
- Two I²C interfaces
- 8 LSAD inputs (four internal signals, four external signals)
- Two PCM/I²S interfaces
- Two SPI interfaces, with support for full-duplex data or dual- and quad-SPI operating modes
- A UART interface

The shared I²S and PCM interfaces are designed to allow streaming of digitized signal data directly into or out of the system through either an IOC, the DMA, the CFX, or the Arm Cortex-M3 processor. This interface can be configured to communicate with any I²S device, many PCM devices, and the NOAHlink® (audio streaming) device. For more detailed configuration and use information, see [Section 18.6.2 “PCM/I2S Data Interface” on page 646](#).

For more detailed information about configuring and using the other external interfaces, see [Chapter 18.1 “General-Purpose I/O \(GPIO\) Interface” on page 538](#).

2.11 INPUTS AND OUTPUTS

2.11.1 Input/Output Controllers (IOCs) and Audio Multiplexers

The input/output controllers (IOCs) and audio multiplexers control the audio signal data input and output from the system. The IOCs can be directed to acquire input data from a combination of:

- The four input A/D channels belonging to the input stage

Ezairo 8300 Hardware Reference

- The four digital microphone inputs
- The eight IOC dedicated Filter Engine inputs
- Four shared PCM/I²S input channels from the two PCM/I²S interfaces

The IOCs are also used to direct output data, sending the data to a combination of:

- The two output data channels belonging to the output stage
- The two digital microphone outputs
- The four shared PCM/I²S output channels from the two PCM/I²S interfaces
- The four IOC dedicated Filter Engine inputs

The IOCs route this data to and from the HEAR shared memory via the FIFO controller. Input data from and output data to the IOCs are grouped into some combination of single-channel and multi-channel FIFO buffers.

For more detailed information about audio multiplexing and IOC configuration, see [Chapter 13.1 "Audio Multiplexing" on page 401](#).

2.11.2 Input Stage

The audio input stage on Ezairo 8300 is based on an ADC architecture that provides four input channels multiplexed from six analog inputs, with sufficient input dynamic range to not require an input preamplifier. Each of these input channels uses a number of components, including several filters and a 24-bit oversampling sigma-delta A/D converter, to provide audio input data to the IOCs for the rest of the Ezairo 8300 system to use.

For more detailed information about the configuration of the input stage and using the input stage in the acquisition of data, see [Chapter 14.1 "Overview" on page 438](#).

2.11.3 Output Stage

The audio output stage on Ezairo 8300 provides two output channels. Each output channel uses a fourth-order, 24-bit oversampling sigma-delta D/A converter as the source for a direct digital output, which can be used to drive zero-bias hearing aid receivers.

For more detailed information about the configuration of the output stage and the distribution of data through the output stage, see [Chapter 15.1 "Overview" on page 456](#).

2.12 ANALOG SUPPORT BLOCKS

2.12.1 Oscillators and Clocking

The system makes use of a current-controlled oscillator (CCO). Additionally, a standby clock oscillator can be used to clock the system for low-power profiles. The system clock can be selected from the CCO, the standby clock oscillator, or a divided external clock input.

For detailed information about the operation and configuration of the oscillator and clocking components, see [Chapter 10.1 "Clock Generation" on page 311](#).

2.12.2 Power Supply, Power Management and Reset Handling

The Ezairo 8300 system includes integrated power supply and power management components. These components supply power to the system and cover all configuration aspects of managing power consumption

Ezairo 8300 Hardware Reference

through regular and stand-by power modes. These components also provide a regulated power supply to system components that require voltage regulation, monitor the power supplied to the system as a whole, and provide soft power-down tools to limit power consumption by the chip.

The power supply components include reset handling blocks that provide clean handling of all possible reset sources, including power-on-reset and other power management related resets, input from the NRESET pad, and the expiry of the watchdog timer.

For details about the power supply and power management features, see [Chapter 9.1 "Power Supply" on page 288](#).

2.13 DEFAULT REGISTER VALUES OVERWRITTEN BY ROM

Some of the power-on register values are overwritten with ROM values. This manual specifies the ROM values in the register tables in each topic; the original hardware values before they are overwritten by the ROM are also shown in the ["Power-On Registers that Have Different ROM Default Values" table \(Table 1\)](#).

Table 1. Power-On Registers that Have Different ROM Default Values

Field Name	ROM Default Value	Startup Default Value
VDDOD_REG_TRIM	VDDOD_REG_TRIM_0P80V	VDDOD_REG_TRIM_1P15V
VDDOD_REG_EN	VDDOD_REG_ENABLE	VDDOD_REG_DISABLE
VDDCM_REF_SEL	VDDCM_REF_AUTO	VDDCM_REF_PMU

2.14 DEVICE IDENTIFICATION

The Ezairo 8300 chip provides a number of registers which contain static values that identify a chip and provide version information about the chip. These registers can be used to verify compatibility for both user applications and external applications communicating through the debug port. A list of these registers along with a brief description of each are in the ["Device Identification Registers" table \(Table 2\)](#).

Table 2. Device Identification Registers

Register	Description
CHIP_ID_FAMILY	The chip technology family to which the chip belongs. Ezairo 8300 belongs to chip family 10 (0xA).
CHIP_ID_VERSION	Version number for the chip. Used to indicate major updates which might not be backwards compatible. Ezairo 8300 uses a chip version of 0x01.
CHIP_ID_REVISION	Revision number for the chip. - An update of the minor revision number (bits [7:0]) indicates minor backwards-compatible or non-functional updates to the system that require no update to source code or binary objects. - An update of the major revision number (bits [15:8]) indicates updates that could affect source code or binary objects, and thus could require firmware library or software updates. When the major revision number is updated, the minor revision number is reset.

Ezairo 8300 Hardware Reference

Device Identification Registers (Continued)

Register	Description
CHIP_ID_CFX_DEBUG_VERSION	Version number of the CFX debug port communications protocol. Used to indicate which communication options are available for the chip.
CHIP_ID_CM3_DEBUG_VERSION	Version number of the Arm Cortex-M3 processor debug port communications protocol. Used to indicate which communication options are available for the chip.

The chip version is commonly written as X.YY.ZZ (for example, 1.02.03), where X is the chip version number, YY is the major revision number, and ZZ is the minor revision number. Chips are also identified using just the chip version number and major revision number as a chip identifier (CID) value. For a chip version of X.YY.ZZ, the CID would be XYY (for example, 102).

The chip family and version information is also duplicated in the AHBREGS_CHIP_ID_NUM register for compatibility with onsemi debug tools for devices that use the Arm Cortex-M3 processor.

2.14.1 Device Identification Control and Configuration Registers

Register Name	Register Description	Address
AHBREGS_CHIP_ID_NUM	Chip ID number	P:0x-7FFF001 0x1FFFFFFC
D_CHIP_ID_FAMILY CHIP_ID_FAMILY	Chip family number	P:0x0830 0x400020C0
D_CHIP_ID_VERSION CHIP_ID_VERSION	Chip version number	P:0x0831 0x400020C4
D_CHIP_ID_REVISION CHIP_ID_REVISION	Chip revision number	P:0x0832 0x400020C8
D_CHIP_ID_CFX_DEBUG_VERSION CHIP_ID_CFX_DEBUG_VERSION	CFX debug port version number	P:0x0833 0x400020CC
D_CHIP_ID_CM3_DEBUG_VERSION CHIP_ID_CM3_DEBUG_VERSION	Arm Cortex-M3 Debug version number	P:0x0834 0x400020D0

2.14.1.1 AHBREGS_CHIP_ID_NUM

Bit Field	Read/Write	Field Name	Description
31:24	R	CHIP_FAMILY	Chip Family number
23:16	R	CHIP_VERSION	Chip Version number
15:8	R	CHIP_MAJOR_REVISION	Chip Major Revision number
7:0	R	CHIP_MINOR_REVISION	Chip Minor Revision number

Ezairo 8300 Hardware Reference

2.14.1.2 D_CHIP_ID_FAMILY, CHIP_ID_FAMILY

Bit Field	Read/Write	Field Name	Description
7:0	R	CHIP_FAMILY	Chip family number

Field Name	Value Symbol	Value Description	Hex Value
CHIP_FAMILY	CHIP_FAMILY_NB	Ezairo 8300 family	0xA*

2.14.1.3 D_CHIP_ID_VERSION, CHIP_ID_VERSION

Bit Field	Read/Write	Field Name	Description
7:0	R	CHIP_VERSION	Chip version number

Field Name	Value Symbol	Value Description	Hex Value
CHIP_VERSION	CHIP_VERSION_NB	Value for SV3 (SHL)	0x1*

2.14.1.4 D_CHIP_ID_REVISION, CHIP_ID_REVISION

Bit Field	Read/Write	Field Name	Description
15:0	R	CHIP_REVISION	Chip revision number

Field Name	Value Symbol	Value Description	Hex Value
CHIP_REVISION	CHIP_REVISION_NB	Value for SV3 (SHL)	0x101*

2.14.1.5 D_CHIP_ID_CFX_DEBUG_VERSION, CHIP_ID_CFX_DEBUG_VERSION

Bit Field	Read/Write	Field Name	Description
15:0	R	CFX_DEBUG_VERSION	CFX debug port version number

Field Name	Value Symbol	Value Description	Hex Value
CFX_DEBUG_VERSION	CFX_DEBUG_VERSION_NB	Value for SV3 (SHL)	0x300*

2.14.1.6 D_CHIP_ID_CM3_DEBUG_VERSION, CHIP_ID_CM3_DEBUG_VERSION

Bit Field	Read/Write	Field Name	Description
15:0	R	CM3_DEBUG_VERSION	Arm Cortex-M3 debug version number

Field Name	Value Symbol	Value Description	Hex Value
CM3_DEBUG_VERSION	CM3_DEBUG_VERSION_NB	Value for SV3 (SHL)	0x300*

CHAPTER 3

CFX Digital Signal Processor

The CFX Digital Signal Processor (DSP) core is a programmable, pipe-lined DSP that uses a dual-Harvard, dual-MAC architecture to efficiently process 24-bit signal data. For information about the hardware features of this processor and details of the CFX instruction set, refer to the *CFX DSP Architecture Manual for Ezairo 8300*.

NOTE: The Ezairo 8300 system contains version 1.1 of the CFX DSP, which is substantially similar in design to the CFX DSP 1.0. As the instruction set for these tools is incompatible between cores, ensure that you are using the CFX DSP 1.1 tools when working with the Ezairo 8300 system.

3.1 PROCESSOR OVERVIEW

The CFX is one of the primary controllers for the Ezairo 8300 system. Since only it and the Arm Cortex-M3 core have access to other system components, it is one of the processors that provides an interface for configuration and control of all of the other system components.

The CFX is closely coupled to many of the memory components. As a result, it has access to all of the signal data that the system is processing.

All applications executing on the CFX DSP core can be debugged through the CFX debug controller. This controller provides access to all the CFX registers, and to the memory and memory-mapped elements available through the memory buses attached to the CFX DSP. This transparent view of the system facilitates efficient debugging by enabling you to:

- Review the current state of system components through your associated memory mapped control registers
- Use this information to troubleshoot most issues in your application

For further details about the debug controller and its interaction with the CFX DSP, see [Section 17.1.2 “CFX Debug Port Controller”](#) on page 529.

IMPORTANT: To minimize power consumption when the CFX DSP is waiting for an interrupt, the core should be held in a wait state using the sleep instruction. In C code, an application can use the `SYS_WAIT_FOR_INTERRUPT` macro (which wraps the `sleep()` intrinsic function) to place the core into this wait state.

3.1.1 Inter-processor Communication

As part of the Ezairo 8300 system, the CFX DSP is provided with a set of eight interrupt lines for signaling to the Arm Cortex-M3 processor. To trigger an interrupt on the Arm Cortex-M3 processor, the CFX DSP writes a 1 to the appropriate `D_CM3_CMDS_CM3_CMD_*` field in the `D_CM3_CMDS` register. Similarly, the CFX DSP can receive interrupts from the Arm Cortex-M3 processor, as described in [Section 4.3 “Inter-processor Communication”](#) on page 132.

3.1.2 Boot-Up Control and Configuration Registers

Register Name	Register Description	Address
<code>D_BOOT_STATUS</code>	CFX boot status	P:0x0803
<code>BOOT_STATUS</code>		0x4000200C

Ezairo 8300 Hardware Reference

3.1.2.1 BOOT_STATUS

Bit Field	Read/Write	Field Name	Description
0	R	STATUS	CFX boot status
0	W	SET	Set CFX boot status

Field Name	Value Symbol	Value Description	Hex Value
STATUS	CFX_NOT_BOOTED	CFX has not finished booting	0x0*
	CFX_BOOTED	CFX has finished booting	0x1
SET	CFX_BOOT_SET	Set CFX boot status	0x1

3.1.3 Command Control and Configuration Registers

Register Name	Register Description	Address
D_CM3_CMDS	CFX issued Arm Cortex-M3 Commands 0-7	P:0x080B
CM3_CMDS		0x4000202C

3.1.3.1 D_CM3_CMDS, CM3_CMDS

Bit Field	Read/Write	Field Name	Description
7	W	CM3_CMD_7	Write a 1 to issue Arm Cortex-M3 processor start command 7
6	W	CM3_CMD_6	Write a 1 to issue Arm Cortex-M3 processor start command 6
5	W	CM3_CMD_5	Write a 1 to issue Arm Cortex-M3 processor start command 5
4	W	CM3_CMD_4	Write a 1 to issue Arm Cortex-M3 processor start command 4
3	W	CM3_CMD_3	Write a 1 to issue Arm Cortex-M3 processor start command 3
2	W	CM3_CMD_2	Write a 1 to issue Arm Cortex-M3 processor start command 2
1	W	CM3_CMD_1	Write a 1 to issue Arm Cortex-M3 processor start command 1
0	W	CM3_CMD_0	Write a 1 to issue Arm Cortex-M3 processor start command 0

Field Name	Value Symbol	Value Description	Hex Value
CM3_CMD_7	CM3_CMD_7	Issue Arm Cortex-M3 processor command 7	0x1
CM3_CMD_6	CM3_CMD_6	Issue Arm Cortex-M3 processor command 6	0x1
CM3_CMD_5	CM3_CMD_5	Issue Arm Cortex-M3 processor command 5	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
CM3_CMD_4	CM3_CMD_4	Issue Arm Cortex-M3 processor command 4	0x1
CM3_CMD_3	CM3_CMD_3	Issue Arm Cortex-M3 processor command 3	0x1
CM3_CMD_2	CM3_CMD_2	Issue Arm Cortex-M3 processor command 2	0x1
CM3_CMD_1	CM3_CMD_1	Issue Arm Cortex-M3 processor command 1	0x1
CM3_CMD_0	CM3_CMD_0	Issue Arm Cortex-M3 processor command 0	0x1

3.2 INTERRUPT CONTROLLER

The Ezairo 8300 system contains an interrupt controller linked to the CFX DSP. This controller services all of the Ezairo 8300 interrupt sources. The controller is directly connected to the interfaces, peripherals and other components that can trigger an interrupt.

Each interrupt source is associated with an interrupt vector, which is mapped into the base of program RAM memory. These vectors must be filled with an unconditional branch to the desired interrupt handler for the interrupt source associated with the vector. The possible interrupt sources, along with their interrupt vectors and the ordering of interrupts, are listed in [Table 3](#), below.

NOTE: Details regarding what events or conditions trigger each interrupt condition for each interrupt source can be found elsewhere in this group of topics.

Table 3. Interrupt Sources and Vectors

Interrupt Source	Vector	Index
Watchdog Timer interrupt	D_VECT_WATCHDOG	0
ADC monitor interrupt	D_VECT_LSAD_BATMON	1
Audio sink interrupt 0	D_VECT_AUDIOSINK_0	2
Audio sink interrupt 1	D_VECT_AUDIOSINK_1	3
Timer 0 interrupt	D_VECT_TIMER_0	4
Timer 1 interrupt	D_VECT_TIMER_1	5
Timer 2 interrupt	D_VECT_TIMER_2	6
Timer 3 interrupt	D_VECT_TIMER_3	7
DMA channel 0 interrupt	D_VECT_DMA_0	8
DMA channel 1 interrupt	D_VECT_DMA_1	9
DMA channel 2 interrupt	D_VECT_DMA_2	10
DMA channel 3 interrupt	D_VECT_DMA_3	11

onsemi Confidential
Ezairo 8300 Hardware Reference

Interrupt Sources and Vectors (Continued)

Interrupt Source	Vector	Index
DMA channel 4 interrupt	D_VECT_DMA_4	12
DMA channel 5 interrupt	D_VECT_DMA_5	13
DMA channel 6 interrupt	D_VECT_DMA_6	14
DMA channel 7 interrupt	D_VECT_DMA_7	15
DIO0 interrupt	D_VECT_DIO_0	16
DIO1 interrupt	D_VECT_DIO_1	17
DIO2 interrupt	D_VECT_DIO_2	18
DIO3 interrupt	D_VECT_DIO_3	19
DIO4 interrupt	D_VECT_DIO_4	20
DIO5 interrupt	D_VECT_DIO_5	21
DIO6 interrupt	D_VECT_DIO_6	22
DIO7 interrupt	D_VECT_DIO_7	23
I ³ C (0) interrupt	D_VECT_I3C_0	24
I ³ C (1) interrupt	D_VECT_I3C_1	25
I ² C (0) interrupt	D_VECT_I2C_0	26
I ² C (1) interrupt	D_VECT_I2C_1	27
D_I2C interrupt	D_VECT_D_I2C	28
SPI (0) communication interrupt	D_VECT_SPI_0_COM	29
SPI (0) receive interrupt	D_VECT_SPI_0_RX	30
SPI (0) transmit interrupt	D_VECT_SPI_0_TX	31
SPI (1) communication interrupt	D_VECT_SPI_1_COM	32
SPI (1) receive interrupt	D_VECT_SPI_1_RX	33
SPI (1) transmit interrupt	D_VECT_SPI_1_TX	34
eMMC interrupt	D_VECT_EMMC	35
UART error interrupt	D_VECT_UART_ERROR	36
UART receive interrupt	D_VECT_UART_RX	37
UART transmit interrupt	D_VECT_UART_TX	38
Filter Engine interrupt	D_VECT_FENG	39
PCM (0) error interrupt	D_VECT_PCM_0_ERROR	40
PCM (0) receive and transmit interrupt	D_VECT_PCM_0_RX_TX	41
PCM (1) error interrupt	D_VECT_PCM_1_ERROR	42
PCM (1) receive and transmit interrupt	D_VECT_PCM_1_RX_TX	43
ASRC error interrupt	D_VECT_ASRC_ERROR	44

onsemi Confidential
Ezairo 8300 Hardware Reference

Interrupt Sources and Vectors (Continued)

Interrupt Source	Vector	Index
ASRC data input interrupt	D_VECT_ASRC_IN	45
ASRC data output interrupt	D_VECT_ASRC_OUT	46
ASRC idle interrupt	D_VECT_ASRC_IDLE	47
G.722 encoder input ready interrupt	D_VECT_G722_ENC_IN_RDY	48
G.722 encoder output ready interrupt	D_VECT_G722_ENC_OUT_RDY	49
G.722 decoder input ready interrupt	D_VECT_G722_DEC_IN_RDY	50
G.722 decoder output ready interrupt	D_VECT_G722_DEC_OUT_RDY	51
Reed-Solomon encoder input ready interrupt	D_VECT_RS_ENC_IN_RDY	52
Reed-Solomon encoder output ready interrupt	D_VECT_RS_ENC_OUT_RDY	53
Reed-Solomon decoder input ready interrupt	D_VECT_RS_DEC_IN_RDY	54
Reed-Solomon decoder output ready interrupt	D_VECT_RS_DEC_OUT_RDY	55
FIFO interrupt 0	D_VECT_FIFO_0	56
FIFO interrupt 1	D_VECT_FIFO_1	57
FIFO interrupt 2	D_VECT_FIFO_2	58
FIFO interrupt 3	D_VECT_FIFO_3	59
FIFO interrupt 4	D_VECT_FIFO_4	60
FIFO interrupt 5	D_VECT_FIFO_5	61
FIFO interrupt 6	D_VECT_FIFO_6	62
FIFO interrupt 7	D_VECT_FIFO_7	63
HEAR event interrupt 0	D_VECT_HEAR_0	64
HEAR event interrupt 1	D_VECT_HEAR_1	65
HEAR event interrupt 2	D_VECT_HEAR_2	66
HEAR event interrupt 3	D_VECT_HEAR_3	67
HEAR event interrupt 4	D_VECT_HEAR_4	68
HEAR event interrupt 5	D_VECT_HEAR_5	69
HEAR event interrupt 6	D_VECT_HEAR_6	70
HEAR event interrupt 7	D_VECT_HEAR_7	71
DSP event interrupt 0	D_VECT_DSP0	72
DSP event interrupt 1	D_VECT_DSP1	73
DSP event interrupt 2	D_VECT_DSP2	74

Ezairo 8300 Hardware Reference

Interrupt Sources and Vectors (Continued)

Interrupt Source	Vector	Index
DSP event interrupt 3	D_VECT_DSP3	75
DSP event interrupt 4	D_VECT_DSP4	76
DSP event interrupt 5	D_VECT_DSP5	77
DSP event interrupt 6	D_VECT_DSP6	78
DSP event interrupt 7	D_VECT_DSP7	79
Arm Cortex-M3 interrupt 0	D_VECT_CM3_0	80
Arm Cortex-M3 interrupt 1	D_VECT_CM3_1	81
Arm Cortex-M3 interrupt 2	D_VECT_CM3_2	82
Arm Cortex-M3 interrupt 3	D_VECT_CM3_3	83
Arm Cortex-M3 interrupt 4	D_VECT_CM3_4	84
Arm Cortex-M3 interrupt 5	D_VECT_CM3_5	85
Arm Cortex-M3 interrupt 6	D_VECT_CM3_6	86
Arm Cortex-M3 interrupt 7	D_VECT_CM3_7	87
Neural network accelerator interrupt	D_VECT_NNA	88
Clock detector interrupt	D_VECT_CLK_DET	89
Low-speed ADC interrupt	D_VECT_LSAD	90
Memory error interrupt	D_VECT_MEM_ERROR	91
G.722 communication interrupt	D_VECT_G722_COM	92
Reed-Solomon communication interrupt	D_VECT_RS_COM	93

Even when more than one interrupt is enabled, only one interrupt can be issued for handling at a time. As a result, an application might find it valuable to prioritize certain interrupts over others. Assign either normal or high priority to an interrupt by clearing or setting the appropriate bits in the `D_INT_PRIORITY_*` registers. This setting is also mapped into the `D_INT_PRIO_*` registers in 8-bit blocks, for efficient configuration of a subset of the interrupt sources. If multiple interrupts are pending, all high priority interrupts are handled before any normal priority interrupts are handled. As well, if multiple pending interrupts share a priority level, the lowest numbered interrupt (as numbered in [Table 3 on page 61](#)) is handled first.

The interrupt enable registers, `D_INT_ENABLE_*`, provide interrupt enable flags for each of the interrupt vectors. These bits are also mapped into the `D_INT_EBL_*` and `D_INT_DIS_*` registers. The `D_INT_EBL_*` registers provide an interface allowing you to only enable interrupts, which makes it unnecessary to read the interrupt enable register before enabling a set of interrupts. Similarly, the `D_INT_DIS_*` registers provide an interface allowing you to only disable interrupts. The `D_INT_MASTER_ENABLE_MASTER` bit in the `D_INT_MASTER_ENABLE` register acts as a master enable/disable for all interrupts regardless of their individual settings. This register provides a faster way to disable and enable interrupts globally on the CFX processor compared to clearing and setting the `IE` bit in the status register (as clearing the `IE` bit takes three cycles).

When an interrupt source generates an interrupt event, the interrupt status bit that corresponds to the interrupt source is set in the `D_INT_STATUS_*` register. The bits in this register can be manually cleared to acknowledge their linked interrupt, by writing a 1 to either this bit in the appropriate `D_INT_STATUS_*` register, or to the

Ezairo 8300 Hardware Reference

corresponding bit in one of the `D_INT_ACK_*` registers. The bit corresponding to an interrupt is also cleared when the CFX DSP handles the associated interrupt. When the CFX DSP is acknowledging an interrupt, no new interrupt signals for that interrupt are accepted until the following clock cycle.

NOTE: An interrupt that is not enabled is still marked in the status register, because the interrupt enable register only masks interrupt notification and not generation. If you want to ensure that the interrupt triggers only for new interrupt conditions, acknowledge interrupts prior to enabling the interrupt. This behavior can also be used to support event polling in the Ezairo 8300 system.

IMPORTANT: Unlike the case where the CFX DSP is automatically acknowledging an interrupt, when an interrupt is acknowledged by a user application, a new interrupt signal for the acknowledged interrupt is accepted immediately.

In summary, the interrupt controller selects which interrupt to issue, based on these criteria:

1. The interrupt controller starts with interrupts marked as pending in the interrupt status registers (`D_INT_STATUS_*`).
2. The interrupt enable settings in the `D_INT_ENABLE_*` registers are used to filter the pending interrupts.
3. All pending interrupts are removed if the `D_INT_MASTER_ENABLE_MASTER` bit is not set.
4. The remaining interrupts are split into high priority and normal priority.
5. The lowest-numbered interrupt with a high priority is then issued. If no high priority interrupts are pending, the lowest-numbered normal priority interrupt is issued.

These selection criteria are illustrated in [Figure 3 on page 66](#).

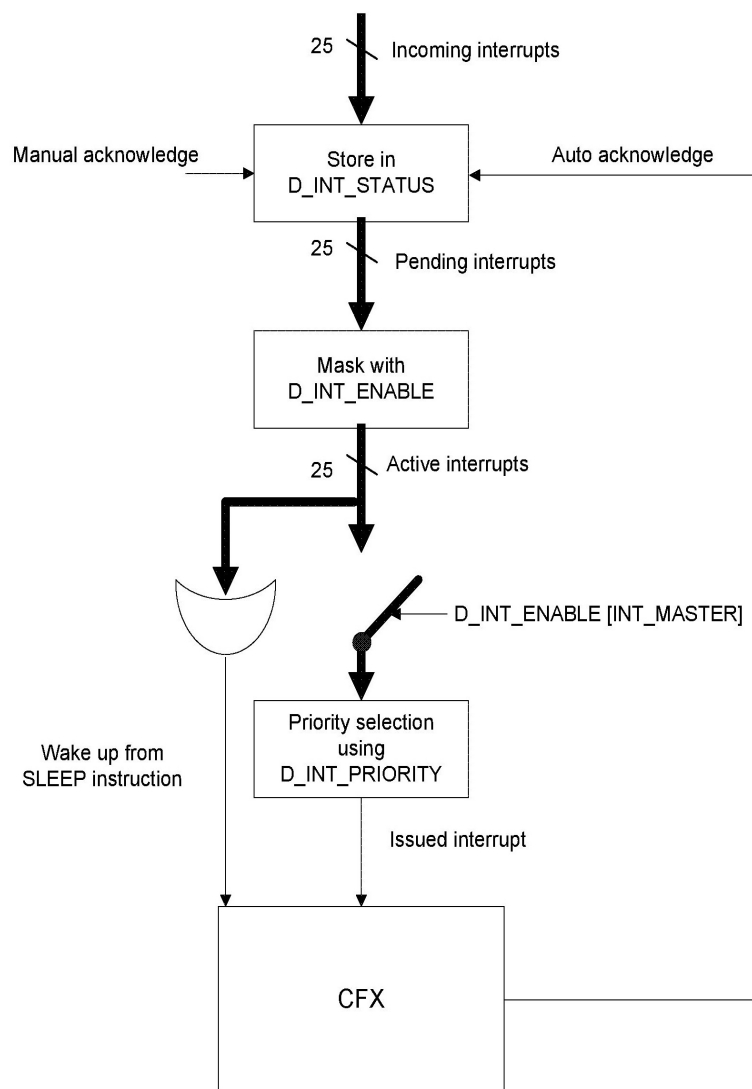


Figure 3. Criteria the Interrupt Controller Uses to Select an Interrupt to Issue

onsemi Confidential
Ezairo 8300 Hardware Reference

3.2.1 Control and Configuration Registers

Register Name	Register Description	Address
D_INT_MASTER_ENABLE INT_MASTER_ENABLE	Master Interrupt Enable register	P:0x0840 0x40002100
D_INT_ENABLE_0 INT_ENABLE_0	Interrupt Enable/Mask register 0	P:0x0841 0x40002104
D_INT_ENABLE_1 INT_ENABLE_1	Interrupt Enable/Mask register 1	P:0x0842 0x40002108
D_INT_ENABLE_2 INT_ENABLE_2	Interrupt Enable/Mask register 2	P:0x0843 0x4000210C
D_INT_STATUS_0 INT_STATUS_0	Interrupt ACK/Status register 0	P:0x0844 0x40002110
D_INT_STATUS_1 INT_STATUS_1	Interrupt ACK/Status register 1	P:0x0845 0x40002114
D_INT_STATUS_2 INT_STATUS_2	Interrupt ACK/Status register 2	P:0x0846 0x40002118
D_INT_PRIORITY_0 INT_PRIORITY_0	Interrupt priority setting 0	P:0x0847 0x4000211C
D_INT_PRIORITY_1 INT_PRIORITY_1	Interrupt priority setting 1	P:0x0848 0x40002120
D_INT_PRIORITY_2 INT_PRIORITY_2	Interrupt priority setting 2	P:0x0849 0x40002124
D_INT_EBL_0 INT_EBL_0	Enable (no disable) of interrupts 0 to 7	P:0x0850 0x40002140
D_INT_EBL_1 INT_EBL_1	Enable (no disable) of interrupts 8 to 15	P:0x0851 0x40002144
D_INT_EBL_2 INT_EBL_2	Enable (no disable) of interrupts 16 to 23	P:0x0852 0x40002148
D_INT_EBL_3	Enable (no disable) of interrupts 24 to 31	P:0x0853

onsemi Confidential
Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
INT_EBL_3		0x4000214C
D_INT_EBL_4	Enable (no disable) of interrupts 32 to 39	P:0x0854
INT_EBL_4		0x40002150
D_INT_EBL_5	Enable (no disable) of interrupts 40 to 47	P:0x0855
INT_EBL_5		0x40002154
D_INT_EBL_6	Enable (no disable) of interrupts 48 to 55	P:0x0856
INT_EBL_6		0x40002158
D_INT_EBL_7	Enable (no disable) of interrupts 56 to 63	P:0x0857
INT_EBL_7		0x4000215C
D_INT_EBL_8	Enable (no disable) of interrupts 64 to 71	P:0x0858
INT_EBL_8		0x40002160
D_INT_EBL_9	Enable (no disable) of interrupts 72 to 79	P:0x0859
INT_EBL_9		0x40002164
D_INT_EBL_10	Enable (no disable) of interrupts 80 to 87	P:0x085A
INT_EBL_10		0x40002168
D_INT_EBL_11	Enable (no disable) of interrupts 88 to 93	P:0x085B
INT_EBL_11		0x4000216C
D_INT_DIS_0	Disable (no enable) of interrupts 0 to 7	P:0x085C
INT_DIS_0		0x40002170
D_INT_DIS_1	Disable (no enable) of interrupts 8 to 15	P:0x085D
INT_DIS_1		0x40002174
D_INT_DIS_2	Disable (no enable) of interrupts 16 to 23	P:0x085E
INT_DIS_2		0x40002178
D_INT_DIS_3	Disable (no enable) of interrupts 24 to 31	P:0x085F
INT_DIS_3		0x4000217C
D_INT_DIS_4	Disable (no enable) of interrupts 32 to 39	P:0x0860
INT_DIS_4		0x40002180
D_INT_DIS_5	Disable (no enable) of interrupts 40 to 47	P:0x0861

onsemi Confidential
Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
INT_DIS_5		0x40002184
D_INT_DIS_6	Disable (no enable) of interrupts 48 to 55	P:0x0862
INT_DIS_6		0x40002188
D_INT_DIS_7	Disable (no enable) of interrupts 56 to 63	P:0x0863
INT_DIS_7		0x4000218C
D_INT_DIS_8	Disable (no enable) of interrupts 64 to 71	P:0x0864
INT_DIS_8		0x40002190
D_INT_DIS_9	Disable (no enable) of interrupts 72 to 79	P:0x0865
INT_DIS_9		0x40002194
D_INT_DIS_10	Disable (no enable) of interrupts 80 to 87	P:0x0866
INT_DIS_10		0x40002198
D_INT_DIS_11	Disable (no enable) of interrupts 88 to 93	P:0x0867
INT_DIS_11		0x4000219C
D_INT_ACK_0	Acknowledge/status of interrupts 0 to 7	P:0x0868
INT_ACK_0		0x400021A0
D_INT_ACK_1	Acknowledge/status of interrupts 8 to 15	P:0x0869
INT_ACK_1		0x400021A4
D_INT_ACK_2	Acknowledge/status of interrupts 16 to 23	P:0x086A
INT_ACK_2		0x400021A8
D_INT_ACK_3	Acknowledge/status of interrupts 24 to 31	P:0x086B
INT_ACK_3		0x400021AC
D_INT_ACK_4	Acknowledge/status of interrupts 32 to 39	P:0x086C
INT_ACK_4		0x400021B0
D_INT_ACK_5	Acknowledge/status of interrupts 40 to 47	P:0x086D
INT_ACK_5		0x400021B4
D_INT_ACK_6	Acknowledge/status of interrupts 48 to 55	P:0x086E
INT_ACK_6		0x400021B8
D_INT_ACK_7	Acknowledge/status of interrupts 56 to 63	P:0x086F

onsemi Confidential
Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
INT_ACK_7		0x400021BC
D_INT_ACK_8	Acknowledge/status of interrupts 64 to 71	P:0x0870
INT_ACK_8		0x400021C0
D_INT_ACK_9	Acknowledge/status of interrupts 72 to 79	P:0x0871
INT_ACK_9		0x400021C4
D_INT_ACK_10	Acknowledge/status of interrupts 80 to 87	P:0x0872
INT_ACK_10		0x400021C8
D_INT_ACK_11	Acknowledge/status of interrupts 88 to 93	P:0x0873
INT_ACK_11		0x400021CC
D_INT_PRIO_0	Priority level of interrupts 0 to 7	P:0x0874
INT_PRIO_0		0x400021D0
D_INT_PRIO_1	Priority level of interrupts 8 to 15	P:0x0875
INT_PRIO_1		0x400021D4
D_INT_PRIO_2	Priority level of interrupts 16 to 23	P:0x0876
INT_PRIO_2		0x400021D8
D_INT_PRIO_3	Priority level of interrupts 24 to 31	P:0x0877
INT_PRIO_3		0x400021DC
D_INT_PRIO_4	Priority level of interrupts 32 to 39	P:0x0878
INT_PRIO_4		0x400021E0
D_INT_PRIO_5	Priority level of interrupts 40 to 47	P:0x0879
INT_PRIO_5		0x400021E4
D_INT_PRIO_6	Priority level of interrupts 48 to 55	P:0x087A
INT_PRIO_6		0x400021E8
D_INT_PRIO_7	Priority level of interrupts 56 to 63	P:0x087B
INT_PRIO_7		0x400021EC
D_INT_PRIO_8	Priority level of interrupts 64 to 71	P:0x087C
INT_PRIO_8		0x400021F0
D_INT_PRIO_9	Priority level of interrupts 72 to 79	P:0x087D

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
INT_PRIO_9		0x400021F4
D_INT_PRIO_10	Priority level of interrupts 80 to 87	P:0x087E
INT_PRIO_10		0x400021F8
D_INT_PRIO_11	Priority level of interrupts 88 to 93	P:0x087F
INT_PRIO_11		0x400021FC

3.2.1.1 D_INT_MASTER_ENABLE, INT_MASTER_ENABLE

Bit Field	Read/Write	Field Name	Description
0	RW	MASTER	MASTER enable

Field Name	Value Symbol	Value Description	Hex Value
MASTER	INT_DIS_MASTER	Disable MASTER interrupt	0x0
	INT_EBL_MASTER	Enable MASTER interrupt	0x1*

3.2.1.2 D_INT_ENABLE_0, INT_ENABLE_0

Bit Field	Read/Write	Field Name	Description
31	RW	SPI_0_TX	SPI_0_TX enable
30	RW	SPI_0_RX	SPI_0_RX enable
29	RW	SPI_0_COM	SPI_0_COM enable
28	RW	D_I2C	D_I2C enable
27	RW	I2C_1	I2C_1 enable
26	RW	I2C_0	I2C_0 enable
25	RW	I3C_1	I3C_1 enable
24	RW	I3C_0	I3C_0 enable
23	RW	DIO_7	DIO_7 enable
22	RW	DIO_6	DIO_6 enable
21	RW	DIO_5	DIO_5 enable
20	RW	DIO_4	DIO_4 enable
19	RW	DIO_3	DIO_3 enable
18	RW	DIO_2	DIO_2 enable
17	RW	DIO_1	DIO_1 enable

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
16	RW	DIO_0	DIO_0 enable
15	RW	DMA_7	DMA_7 enable
14	RW	DMA_6	DMA_6 enable
13	RW	DMA_5	DMA_5 enable
12	RW	DMA_4	DMA_4 enable
11	RW	DMA_3	DMA_3 enable
10	RW	DMA_2	DMA_2 enable
9	RW	DMA_1	DMA_1 enable
8	RW	DMA_0	DMA_0 enable
7	RW	TIMER_3	TIMER_3 enable
6	RW	TIMER_2	TIMER_2 enable
5	RW	TIMER_1	TIMER_1 enable
4	RW	TIMER_0	TIMER_0 enable
3	RW	AUDIOSINK_1	AUDIOSINK_1 enable
2	RW	AUDIOSINK_0	AUDIOSINK_0 enable
1	RW	LSAD_BATMON	LSAD_BATMON enable
0	RW	WATCHDOG	WATCHDOG enable

Field Name	Value Symbol	Value Description	Hex Value
SPI_0_TX	INT_EBL_SPI_0_TX	Enable SPI_0_TX interrupt	0x1
SPI_0_RX	INT_EBL_SPI_0_RX	Enable SPI_0_RX interrupt	0x1
SPI_0_COM	INT_EBL_SPI_0_COM	Enable SPI_0_COM interrupt	0x1
D_I2C	INT_EBL_D_I2C	Enable D_I2C interrupt	0x1
I2C_1	INT_EBL_I2C_1	Enable I2C_1 interrupt	0x1
I2C_0	INT_EBL_I2C_0	Enable I2C_0 interrupt	0x1
I3C_1	INT_EBL_I3C_1	Enable I3C_1 interrupt	0x1
I3C_0	INT_EBL_I3C_0	Enable I3C_0 interrupt	0x1
DIO_7	INT_EBL_DIO_7	Enable DIO_7 interrupt	0x1
DIO_6	INT_EBL_DIO_6	Enable DIO_6 interrupt	0x1
DIO_5	INT_EBL_DIO_5	Enable DIO_5 interrupt	0x1
DIO_4	INT_EBL_DIO_4	Enable DIO_4 interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DIO_3	INT_EBL_DIO_3	Enable DIO_3 interrupt	0x1
DIO_2	INT_EBL_DIO_2	Enable DIO_2 interrupt	0x1
DIO_1	INT_EBL_DIO_1	Enable DIO_1 interrupt	0x1
DIO_0	INT_EBL_DIO_0	Enable DIO_0 interrupt	0x1
DMA_7	INT_EBL_DMA_7	Enable DMA_7 interrupt	0x1
DMA_6	INT_EBL_DMA_6	Enable DMA_6 interrupt	0x1
DMA_5	INT_EBL_DMA_5	Enable DMA_5 interrupt	0x1
DMA_4	INT_EBL_DMA_4	Enable DMA_4 interrupt	0x1
DMA_3	INT_EBL_DMA_3	Enable DMA_3 interrupt	0x1
DMA_2	INT_EBL_DMA_2	Enable DMA_2 interrupt	0x1
DMA_1	INT_EBL_DMA_1	Enable DMA_1 interrupt	0x1
DMA_0	INT_EBL_DMA_0	Enable DMA_0 interrupt	0x1
TIMER_3	INT_EBL_TIMER_3	Enable TIMER_3 interrupt	0x1
TIMER_2	INT_EBL_TIMER_2	Enable TIMER_2 interrupt	0x1
TIMER_1	INT_EBL_TIMER_1	Enable TIMER_1 interrupt	0x1
TIMER_0	INT_EBL_TIMER_0	Enable TIMER_0 interrupt	0x1
AUDIOSINK_1	INT_EBL_AUDIOSINK_1	Enable AUDIOSINK_1 interrupt	0x1
AUDIOSINK_0	INT_EBL_AUDIOSINK_0	Enable AUDIOSINK_0 interrupt	0x1
LSAD_BATMON	INT_EBL_LSAD_BATMON	Enable LSAD_BATMON interrupt	0x1
WATCHDOG	INT_EBL_WATCHDOG	Enable WATCHDOG interrupt	0x1

3.2.1.3 D_INT_ENABLE_1, INT_ENABLE_1

Bit Field	Read/Write	Field Name	Description
31	RW	FIFO_7	FIFO_7 enable
30	RW	FIFO_6	FIFO_6 enable
29	RW	FIFO_5	FIFO_5 enable
28	RW	FIFO_4	FIFO_4 enable
27	RW	FIFO_3	FIFO_3 enable
26	RW	FIFO_2	FIFO_2 enable
25	RW	FIFO_1	FIFO_1 enable
24	RW	FIFO_0	FIFO_0 enable
23	RW	RS_DEC_OUT_RDY	RS_DEC_OUT_RDY enable

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
22	RW	RS_DEC_IN_RDY	RS_DEC_IN_RDY enable
21	RW	RS_ENC_OUT_RDY	RS_ENC_OUT_RDY enable
20	RW	RS_ENC_IN_RDY	RS_ENC_IN_RDY enable
19	RW	G722_DEC_OUT_RDY	G722_DEC_OUT_RDY enable
18	RW	G722_DEC_IN_RDY	G722_DEC_IN_RDY enable
17	RW	G722_ENC_OUT_RDY	G722_ENC_OUT_RDY enable
16	RW	G722_ENC_IN_RDY	G722_ENC_IN_RDY enable
15	RW	ASRC_IDLE	ASRC_IDLE enable
14	RW	ASRC_OUT	ASRC_OUT enable
13	RW	ASRC_IN	ASRC_IN enable
12	RW	ASRC_ERROR	ASRC_ERROR enable
11	RW	PCM_1_RX_TX	PCM_1_RX_TX enable
10	RW	PCM_1_ERROR	PCM_1_ERROR enable
9	RW	PCM_0_RX_TX	PCM_0_RX_TX enable
8	RW	PCM_0_ERROR	PCM_0_ERROR enable
7	RW	FENG	FENG enable
6	RW	UART_0_TX	UART_0_TX enable
5	RW	UART_0_RX	UART_0_RX enable
4	RW	UART_0_ERROR	UART_0_ERROR enable
3	RW	EMMC	EMMC enable
2	RW	SPI_1_TX	SPI_1_TX enable
1	RW	SPI_1_RX	SPI_1_RX enable
0	RW	SPI_1_COM	SPI_1_COM enable

Field Name	Value Symbol	Value Description	Hex Value
FIFO_7	INT_EBL_FIFO_7	Enable FIFO_7 interrupt	0x1
FIFO_6	INT_EBL_FIFO_6	Enable FIFO_6 interrupt	0x1
FIFO_5	INT_EBL_FIFO_5	Enable FIFO_5 interrupt	0x1
FIFO_4	INT_EBL_FIFO_4	Enable FIFO_4 interrupt	0x1
FIFO_3	INT_EBL_FIFO_3	Enable FIFO_3 interrupt	0x1
FIFO_2	INT_EBL_FIFO_2	Enable FIFO_2 interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
FIFO_1	INT_EBL_FIFO_1	Enable FIFO_1 interrupt	0x1
FIFO_0	INT_EBL_FIFO_0	Enable FIFO_0 interrupt	0x1
RS_DEC_OUT_RDY	INT_EBL_RS_DEC_OUT_RDY	Enable RS_DEC_OUT_RDY interrupt	0x1
RS_DEC_IN_RDY	INT_EBL_RS_DEC_IN_RDY	Enable RS_DEC_IN_RDY interrupt	0x1
RS_ENC_OUT_RDY	INT_EBL_RS_ENC_OUT_RDY	Enable RS_ENC_OUT_RDY interrupt	0x1
RS_ENC_IN_RDY	INT_EBL_RS_ENC_IN_RDY	Enable RS_ENC_IN_RDY interrupt	0x1
G722_DEC_OUT_RDY	INT_EBL_G722_DEC_OUT_RDY	Enable G722_DEC_OUT_RDY interrupt	0x1
G722_DEC_IN_RDY	INT_EBL_G722_DEC_IN_RDY	Enable G722_DEC_IN_RDY interrupt	0x1
G722_ENC_OUT_RDY	INT_EBL_G722_ENC_OUT_RDY	Enable G722_ENC_OUT_RDY interrupt	0x1
G722_ENC_IN_RDY	INT_EBL_G722_ENC_IN_RDY	Enable G722_ENC_IN_RDY interrupt	0x1
ASRC_IDLE	INT_EBL_ASRC_IDLE	Enable ASRC_IDLE interrupt	0x1
ASRC_OUT	INT_EBL_ASRC_OUT	Enable ASRC_OUT interrupt	0x1
ASRC_IN	INT_EBL_ASRC_IN	Enable ASRC_IN interrupt	0x1
ASRC_ERROR	INT_EBL_ASRC_ERROR	Enable ASRC_ERROR interrupt	0x1
PCM_1_RX_TX	INT_EBL_PCM_1_RX_TX	Enable PCM_1_RX_TX interrupt	0x1
PCM_1_ERROR	INT_EBL_PCM_1_ERROR	Enable PCM_1_ERROR interrupt	0x1
PCM_0_RX_TX	INT_EBL_PCM_0_RX_TX	Enable PCM_0_RX_TX interrupt	0x1
PCM_0_ERROR	INT_EBL_PCM_0_ERROR	Enable PCM_0_ERROR interrupt	0x1
FENG	INT_EBL_FENG	Enable FENG interrupt	0x1
UART_0_TX	INT_EBL_UART_0_TX	Enable UART_0_TX interrupt	0x1
UART_0_RX	INT_EBL_UART_0_RX	Enable UART_0_RX interrupt	0x1
UART_0_ERROR	INT_EBL_UART_0_ERROR	Enable UART_0_ERROR interrupt	0x1
EMMC	INT_EBL_EMMC	Enable EMMC interrupt	0x1
SPI_1_TX	INT_EBL_SPI_1_TX	Enable SPI_1_TX interrupt	0x1
SPI_1_RX	INT_EBL_SPI_1_RX	Enable SPI_1_RX interrupt	0x1
SPI_1_COM	INT_EBL_SPI_1_COM	Enable SPI_1_COM interrupt	0x1

onsemi Confidential
Ezairo 8300 Hardware Reference

3.2.1.4 D_INT_ENABLE_2, INT_ENABLE_2

Bit Field	Read/Write	Field Name	Description
29	RW	RS_COM	RS_COM enable
28	RW	G722_COM	G722_COM enable
27	RW	MEM_ERROR	MEM_ERROR enable
26	RW	LSAD	LSAD enable
25	RW	CLK_DET	CLK_DET enable
24	RW	NNA	NNA enable
23	RW	CM3_7	CM3_7 enable
22	RW	CM3_6	CM3_6 enable
21	RW	CM3_5	CM3_5 enable
20	RW	CM3_4	CM3_4 enable
19	RW	CM3_3	CM3_3 enable
18	RW	CM3_2	CM3_2 enable
17	RW	CM3_1	CM3_1 enable
16	RW	CM3_0	CM3_0 enable
15	RW	DSP_7	DSP_7 enable
14	RW	DSP_6	DSP_6 enable
13	RW	DSP_5	DSP_5 enable
12	RW	DSP_4	DSP_4 enable
11	RW	DSP_3	DSP_3 enable
10	RW	DSP_2	DSP_2 enable
9	RW	DSP_1	DSP_1 enable
8	RW	DSP_0	DSP_0 enable
7	RW	HEAR_7	HEAR_7 enable
6	RW	HEAR_6	HEAR_6 enable
5	RW	HEAR_5	HEAR_5 enable
4	RW	HEAR_4	HEAR_4 enable
3	RW	HEAR_3	HEAR_3 enable
2	RW	HEAR_2	HEAR_2 enable
1	RW	HEAR_1	HEAR_1 enable
0	RW	HEAR_0	HEAR_0 enable

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
RS_COM	INT_EBL_RS_COM	Enable RS_COM interrupt	0x1
G722_COM	INT_EBL_G722_COM	Enable G722_COM interrupt	0x1
MEM_ERROR	INT_EBL_MEM_ERROR	Enable MEM_ERROR interrupt	0x1
LSAD	INT_EBL_LSAD	Enable LSAD interrupt	0x1
CLK_DET	INT_EBL_CLK_DET	Enable CLK_DET interrupt	0x1
NNA	INT_EBL_NNA	Enable NNA interrupt	0x1
CM3_7	INT_EBL_CM3_7	Enable CM3_7 interrupt	0x1
CM3_6	INT_EBL_CM3_6	Enable CM3_6 interrupt	0x1
CM3_5	INT_EBL_CM3_5	Enable CM3_5 interrupt	0x1
CM3_4	INT_EBL_CM3_4	Enable CM3_4 interrupt	0x1
CM3_3	INT_EBL_CM3_3	Enable CM3_3 interrupt	0x1
CM3_2	INT_EBL_CM3_2	Enable CM3_2 interrupt	0x1
CM3_1	INT_EBL_CM3_1	Enable CM3_1 interrupt	0x1
CM3_0	INT_EBL_CM3_0	Enable CM3_0 interrupt	0x1
DSP_7	INT_EBL_DSP_7	Enable DSP_7 interrupt	0x1
DSP_6	INT_EBL_DSP_6	Enable DSP_6 interrupt	0x1
DSP_5	INT_EBL_DSP_5	Enable DSP_5 interrupt	0x1
DSP_4	INT_EBL_DSP_4	Enable DSP_4 interrupt	0x1
DSP_3	INT_EBL_DSP_3	Enable DSP_3 interrupt	0x1
DSP_2	INT_EBL_DSP_2	Enable DSP_2 interrupt	0x1
DSP_1	INT_EBL_DSP_1	Enable DSP_1 interrupt	0x1
DSP_0	INT_EBL_DSP_0	Enable DSP_0 interrupt	0x1
HEAR_7	INT_EBL_HEAR_7	Enable HEAR_7 interrupt	0x1
HEAR_6	INT_EBL_HEAR_6	Enable HEAR_6 interrupt	0x1
HEAR_5	INT_EBL_HEAR_5	Enable HEAR_5 interrupt	0x1
HEAR_4	INT_EBL_HEAR_4	Enable HEAR_4 interrupt	0x1
HEAR_3	INT_EBL_HEAR_3	Enable HEAR_3 interrupt	0x1
HEAR_2	INT_EBL_HEAR_2	Enable HEAR_2 interrupt	0x1
HEAR_1	INT_EBL_HEAR_1	Enable HEAR_1 interrupt	0x1
HEAR_0	INT_EBL_HEAR_0	Enable HEAR_0 interrupt	0x1

Ezairo 8300 Hardware Reference

3.2.1.5 D_INT_STATUS_0, INT_STATUS_0

Bit Field	Read/Write	Field Name	Description
31	RW	SPI_0_TX	SPI_0_TX status/acknowledge
30	RW	SPI_0_RX	SPI_0_RX status/acknowledge
29	RW	SPI_0_COM	SPI_0_COM status/acknowledge
28	RW	D_I2C	D_I2C status/acknowledge
27	RW	I2C_1	I2C_1 status/acknowledge
26	RW	I2C_0	I2C_0 status/acknowledge
25	RW	I3C_1	I3C_1 status/acknowledge
24	RW	I3C_0	I3C_0 status/acknowledge
23	RW	DIO_7	DIO_7 status/acknowledge
22	RW	DIO_6	DIO_6 status/acknowledge
21	RW	DIO_5	DIO_5 status/acknowledge
20	RW	DIO_4	DIO_4 status/acknowledge
19	RW	DIO_3	DIO_3 status/acknowledge
18	RW	DIO_2	DIO_2 status/acknowledge
17	RW	DIO_1	DIO_1 status/acknowledge
16	RW	DIO_0	DIO_0 status/acknowledge
15	RW	DMA_7	DMA_7 status/acknowledge
14	RW	DMA_6	DMA_6 status/acknowledge
13	RW	DMA_5	DMA_5 status/acknowledge
12	RW	DMA_4	DMA_4 status/acknowledge
11	RW	DMA_3	DMA_3 status/acknowledge
10	RW	DMA_2	DMA_2 status/acknowledge
9	RW	DMA_1	DMA_1 status/acknowledge
8	RW	DMA_0	DMA_0 status/acknowledge
7	RW	TIMER_3	TIMER_3 status/acknowledge
6	RW	TIMER_2	TIMER_2 status/acknowledge
5	RW	TIMER_1	TIMER_1 status/acknowledge
4	RW	TIMER_0	TIMER_0 status/acknowledge
3	RW	AUDIOSINK_1	AUDIOSINK_1 status/acknowledge

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
2	RW	AUDIOSINK_0	AUDIOSINK_0 status/acknowledge
1	RW	LSAD_BATMON	LSAD_BATMON status/acknowledge
0	RW	WATCHDOG	WATCHDOG status/acknowledge

Field Name	Value Symbol	Value Description	Hex Value
SPI_0_TX	INT_ACK_SPI_0_TX	Acknowledge SPI_0_TX interrupt	0x1
SPI_0_RX	INT_ACK_SPI_0_RX	Acknowledge SPI_0_RX interrupt	0x1
SPI_0_COM	INT_ACK_SPI_0_COM	Acknowledge SPI_0_COM interrupt	0x1
D_I2C	INT_ACK_D_I2C	Acknowledge D_I2C interrupt	0x1
I2C_1	INT_ACK_I2C_1	Acknowledge I2C_1 interrupt	0x1
I2C_0	INT_ACK_I2C_0	Acknowledge I2C_0 interrupt	0x1
I3C_1	INT_ACK_I3C_1	Acknowledge I3C_1 interrupt	0x1
I3C_0	INT_ACK_I3C_0	Acknowledge I3C_0 interrupt	0x1
DIO_7	INT_ACK_DIO_7	Acknowledge DIO_7 interrupt	0x1
DIO_6	INT_ACK_DIO_6	Acknowledge DIO_6 interrupt	0x1
DIO_5	INT_ACK_DIO_5	Acknowledge DIO_5 interrupt	0x1
DIO_4	INT_ACK_DIO_4	Acknowledge DIO_4 interrupt	0x1
DIO_3	INT_ACK_DIO_3	Acknowledge DIO_3 interrupt	0x1
DIO_2	INT_ACK_DIO_2	Acknowledge DIO_2 interrupt	0x1
DIO_1	INT_ACK_DIO_1	Acknowledge DIO_1 interrupt	0x1
DIO_0	INT_ACK_DIO_0	Acknowledge DIO_0 interrupt	0x1
DMA_7	INT_ACK_DMA_7	Acknowledge DMA_7 interrupt	0x1
DMA_6	INT_ACK_DMA_6	Acknowledge DMA_6 interrupt	0x1
DMA_5	INT_ACK_DMA_5	Acknowledge DMA_5 interrupt	0x1
DMA_4	INT_ACK_DMA_4	Acknowledge DMA_4 interrupt	0x1
DMA_3	INT_ACK_DMA_3	Acknowledge DMA_3 interrupt	0x1
DMA_2	INT_ACK_DMA_2	Acknowledge DMA_2 interrupt	0x1
DMA_1	INT_ACK_DMA_1	Acknowledge DMA_1 interrupt	0x1
DMA_0	INT_ACK_DMA_0	Acknowledge DMA_0 interrupt	0x1
TIMER_3	INT_ACK_TIMER_3	Acknowledge TIMER_3 interrupt	0x1
TIMER_2	INT_ACK_TIMER_2	Acknowledge TIMER_2 interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
TIMER_1	INT_ACK_TIMER_1	Acknowledge TIMER_1 interrupt	0x1
TIMER_0	INT_ACK_TIMER_0	Acknowledge TIMER_0 interrupt	0x1
AUDIOSINK_1	INT_ACK_AUDIOSINK_1	Acknowledge AUDIOSINK_1 interrupt	0x1
AUDIOSINK_0	INT_ACK_AUDIOSINK_0	Acknowledge AUDIOSINK_0 interrupt	0x1
LSAD_BATMON	INT_ACK_LSAD_BATMON	Acknowledge LSAD_BATMON interrupt	0x1
WATCHDOG	INT_ACK_WATCHDOG	Acknowledge WATCHDOG interrupt	0x1

3.2.1.6 D_INT_STATUS_1, INT_STATUS_1

Bit Field	Read/Write	Field Name	Description
31	RW	FIFO_7	FIFO_7 status/acknowledge
30	RW	FIFO_6	FIFO_6 status/acknowledge
29	RW	FIFO_5	FIFO_5 status/acknowledge
28	RW	FIFO_4	FIFO_4 status/acknowledge
27	RW	FIFO_3	FIFO_3 status/acknowledge
26	RW	FIFO_2	FIFO_2 status/acknowledge
25	RW	FIFO_1	FIFO_1 status/acknowledge
24	RW	FIFO_0	FIFO_0 status/acknowledge
23	RW	RS_DEC_OUT_RDY	RS_DEC_OUT_RDY status/acknowledge
22	RW	RS_DEC_IN_RDY	RS_DEC_IN_RDY status/acknowledge
21	RW	RS_ENC_OUT_RDY	RS_ENC_OUT_RDY status/acknowledge
20	RW	RS_ENC_IN_RDY	RS_ENC_IN_RDY status/acknowledge
19	RW	G722_DEC_OUT_RDY	G722_DEC_OUT_RDY status/acknowledge
18	RW	G722_DEC_IN_RDY	G722_DEC_IN_RDY status/acknowledge
17	RW	G722_ENC_OUT_RDY	G722_ENC_OUT_RDY status/acknowledge
16	RW	G722_ENC_IN_RDY	G722_ENC_IN_RDY status/acknowledge
15	RW	ASRC_IDLE	ASRC_IDLE status/acknowledge
14	RW	ASRC_OUT	ASRC_OUT status/acknowledge
13	RW	ASRC_IN	ASRC_IN status/acknowledge
12	RW	ASRC_ERROR	ASRC_ERROR status/acknowledge
11	RW	PCM_1_RX_TX	PCM_1_RX_TX status/acknowledge

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
10	RW	PCM_1_ERROR	PCM_1_ERROR status/acknowledge
9	RW	PCM_0_RX_TX	PCM_0_RX_TX status/acknowledge
8	RW	PCM_0_ERROR	PCM_0_ERROR status/acknowledge
7	RW	FENG	FENG status/acknowledge
6	RW	UART_0_TX	UART_0_TX status/acknowledge
5	RW	UART_0_RX	UART_0_RX status/acknowledge
4	RW	UART_0_ERROR	UART_0_ERROR status/acknowledge
3	RW	EMMC	EMMC status/acknowledge
2	RW	SPI_1_TX	SPI_1_TX status/acknowledge
1	RW	SPI_1_RX	SPI_1_RX status/acknowledge
0	RW	SPI_1_COM	SPI_1_COM status/acknowledge

Field Name	Value Symbol	Value Description	Hex Value
FIFO_7	INT_ACK_FIFO_7	Acknowledge FIFO_7 interrupt	0x1
FIFO_6	INT_ACK_FIFO_6	Acknowledge FIFO_6 interrupt	0x1
FIFO_5	INT_ACK_FIFO_5	Acknowledge FIFO_5 interrupt	0x1
FIFO_4	INT_ACK_FIFO_4	Acknowledge FIFO_4 interrupt	0x1
FIFO_3	INT_ACK_FIFO_3	Acknowledge FIFO_3 interrupt	0x1
FIFO_2	INT_ACK_FIFO_2	Acknowledge FIFO_2 interrupt	0x1
FIFO_1	INT_ACK_FIFO_1	Acknowledge FIFO_1 interrupt	0x1
FIFO_0	INT_ACK_FIFO_0	Acknowledge FIFO_0 interrupt	0x1
RS_DEC_OUT_RDY	INT_ACK_RS_DEC_OUT_RDY	Acknowledge RS_DEC_OUT_RDY interrupt	0x1
RS_DEC_IN_RDY	INT_ACK_RS_DEC_IN_RDY	Acknowledge RS_DEC_IN_RDY interrupt	0x1
RS_ENC_OUT_RDY	INT_ACK_RS_ENC_OUT_RDY	Acknowledge RS_ENC_OUT_RDY interrupt	0x1
RS_ENC_IN_RDY	INT_ACK_RS_ENC_IN_RDY	Acknowledge RS_ENC_IN_RDY interrupt	0x1
G722_DEC_OUT_RDY	INT_ACK_G722_DEC_OUT_RDY	Acknowledge G722_DEC_OUT_RDY interrupt	0x1
G722_DEC_IN_RDY	INT_ACK_G722_DEC_IN_RDY	Acknowledge G722_DEC_IN_RDY interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
G722_ENC_OUT_RDY	INT_ACK_G722_ENC_OUT_RDY	Acknowledge G722_ENC_OUT_RDY interrupt	0x1
G722_ENC_IN_RDY	INT_ACK_G722_ENC_IN_RDY	Acknowledge G722_ENC_IN_RDY interrupt	0x1
ASRC_IDLE	INT_ACK_ASRC_IDLE	Acknowledge ASRC_IDLE interrupt	0x1
ASRC_OUT	INT_ACK_ASRC_OUT	Acknowledge ASRC_OUT interrupt	0x1
ASRC_IN	INT_ACK_ASRC_IN	Acknowledge ASRC_IN interrupt	0x1
ASRC_ERROR	INT_ACK_ASRC_ERROR	Acknowledge ASRC_ERROR interrupt	0x1
PCM_1_RX_TX	INT_ACK_PCM_1_RX_TX	Acknowledge PCM_1_RX_TX interrupt	0x1
PCM_1_ERROR	INT_ACK_PCM_1_ERROR	Acknowledge PCM_1_ERROR interrupt	0x1
PCM_0_RX_TX	INT_ACK_PCM_0_RX_TX	Acknowledge PCM_0_RX_TX interrupt	0x1
PCM_0_ERROR	INT_ACK_PCM_0_ERROR	Acknowledge PCM_0_ERROR interrupt	0x1
FENG	INT_ACK_FENG	Acknowledge FENG interrupt	0x1
UART_0_TX	INT_ACK_UART_0_TX	Acknowledge UART_0_TX interrupt	0x1
UART_0_RX	INT_ACK_UART_0_RX	Acknowledge UART_0_RX interrupt	0x1
UART_0_ERROR	INT_ACK_UART_0_ERROR	Acknowledge UART_0_ERROR interrupt	0x1
EMMC	INT_ACK_EMMC	Acknowledge EMMC interrupt	0x1
SPI_1_TX	INT_ACK_SPI_1_TX	Acknowledge SPI_1_TX interrupt	0x1
SPI_1_RX	INT_ACK_SPI_1_RX	Acknowledge SPI_1_RX interrupt	0x1
SPI_1_COM	INT_ACK_SPI_1_COM	Acknowledge SPI_1_COM interrupt	0x1

3.2.1.7 D_INT_STATUS_2, INT_STATUS_2

Bit Field	Read/Write	Field Name	Description
29	RW	RS_COM	RS_COM status/acknowledge
28	RW	G722_COM	G722_COM status/acknowledge
27	RW	MEM_ERROR	MEM_ERROR status/acknowledge
26	RW	LSAD	LSAD status/acknowledge
25	RW	CLK_DET	CLK_DET status/acknowledge

onsemi Confidential

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
24	RW	NNA	NNA status/acknowledge
23	RW	CM3_7	CM3_7 status/acknowledge
22	RW	CM3_6	CM3_6 status/acknowledge
21	RW	CM3_5	CM3_5 status/acknowledge
20	RW	CM3_4	CM3_4 status/acknowledge
19	RW	CM3_3	CM3_3 status/acknowledge
18	RW	CM3_2	CM3_2 status/acknowledge
17	RW	CM3_1	CM3_1 status/acknowledge
16	RW	CM3_0	CM3_0 status/acknowledge
15	RW	DSP_7	DSP_7 status/acknowledge
14	RW	DSP_6	DSP_6 status/acknowledge
13	RW	DSP_5	DSP_5 status/acknowledge
12	RW	DSP_4	DSP_4 status/acknowledge
11	RW	DSP_3	DSP_3 status/acknowledge
10	RW	DSP_2	DSP_2 status/acknowledge
9	RW	DSP_1	DSP_1 status/acknowledge
8	RW	DSP_0	DSP_0 status/acknowledge
7	RW	HEAR_7	HEAR_7 status/acknowledge
6	RW	HEAR_6	HEAR_6 status/acknowledge
5	RW	HEAR_5	HEAR_5 status/acknowledge
4	RW	HEAR_4	HEAR_4 status/acknowledge
3	RW	HEAR_3	HEAR_3 status/acknowledge
2	RW	HEAR_2	HEAR_2 status/acknowledge
1	RW	HEAR_1	HEAR_1 status/acknowledge
0	RW	HEAR_0	HEAR_0 status/acknowledge

Field Name	Value Symbol	Value Description	Hex Value
RS_COM	INT_ACK_RS_COM	Acknowledge RS_COM interrupt	0x1
G722_COM	INT_ACK_G722_COM	Acknowledge G722_COM interrupt	0x1
MEM_ERROR	INT_ACK_MEM_ERROR	Acknowledge MEM_ERROR interrupt	0x1
LSAD	INT_ACK_LSAD	Acknowledge LSAD interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
CLK_DET	INT_ACK_CLK_DET	Acknowledge CLK_DET interrupt	0x1
NNA	INT_ACK_NNA	Acknowledge NNA interrupt	0x1
CM3_7	INT_ACK_CM3_7	Acknowledge CM3_7 interrupt	0x1
CM3_6	INT_ACK_CM3_6	Acknowledge CM3_6 interrupt	0x1
CM3_5	INT_ACK_CM3_5	Acknowledge CM3_5 interrupt	0x1
CM3_4	INT_ACK_CM3_4	Acknowledge CM3_4 interrupt	0x1
CM3_3	INT_ACK_CM3_3	Acknowledge CM3_3 interrupt	0x1
CM3_2	INT_ACK_CM3_2	Acknowledge CM3_2 interrupt	0x1
CM3_1	INT_ACK_CM3_1	Acknowledge CM3_1 interrupt	0x1
CM3_0	INT_ACK_CM3_0	Acknowledge CM3_0 interrupt	0x1
DSP_7	INT_ACK_DSP_7	Acknowledge DSP_7 interrupt	0x1
DSP_6	INT_ACK_DSP_6	Acknowledge DSP_6 interrupt	0x1
DSP_5	INT_ACK_DSP_5	Acknowledge DSP_5 interrupt	0x1
DSP_4	INT_ACK_DSP_4	Acknowledge DSP_4 interrupt	0x1
DSP_3	INT_ACK_DSP_3	Acknowledge DSP_3 interrupt	0x1
DSP_2	INT_ACK_DSP_2	Acknowledge DSP_2 interrupt	0x1
DSP_1	INT_ACK_DSP_1	Acknowledge DSP_1 interrupt	0x1
DSP_0	INT_ACK_DSP_0	Acknowledge DSP_0 interrupt	0x1
HEAR_7	INT_ACK_HEAR_7	Acknowledge HEAR_7 interrupt	0x1
HEAR_6	INT_ACK_HEAR_6	Acknowledge HEAR_6 interrupt	0x1
HEAR_5	INT_ACK_HEAR_5	Acknowledge HEAR_5 interrupt	0x1
HEAR_4	INT_ACK_HEAR_4	Acknowledge HEAR_4 interrupt	0x1
HEAR_3	INT_ACK_HEAR_3	Acknowledge HEAR_3 interrupt	0x1
HEAR_2	INT_ACK_HEAR_2	Acknowledge HEAR_2 interrupt	0x1
HEAR_1	INT_ACK_HEAR_1	Acknowledge HEAR_1 interrupt	0x1
HEAR_0	INT_ACK_HEAR_0	Acknowledge HEAR_0 interrupt	0x1

3.2.1.8 D_INT_PRIORITY_0, INT_PRIORITY_0

Bit Field	Read/Write	Field Name	Description
31	RW	SPI_0_TX	SPI_0_TX interrupt priority
30	RW	SPI_0_RX	SPI_0_RX interrupt priority
29	RW	SPI_0_COM	SPI_0_COM interrupt priority

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
28	RW	D_I2C	D_I2C interrupt priority
27	RW	I2C_1	I2C_1 interrupt priority
26	RW	I2C_0	I2C_0 interrupt priority
25	RW	I3C_1	I3C_1 interrupt priority
24	RW	I3C_0	I3C_0 interrupt priority
23	RW	DIO_7	DIO_7 interrupt priority
22	RW	DIO_6	DIO_6 interrupt priority
21	RW	DIO_5	DIO_5 interrupt priority
20	RW	DIO_4	DIO_4 interrupt priority
19	RW	DIO_3	DIO_3 interrupt priority
18	RW	DIO_2	DIO_2 interrupt priority
17	RW	DIO_1	DIO_1 interrupt priority
16	RW	DIO_0	DIO_0 interrupt priority
15	RW	DMA_7	DMA_7 interrupt priority
14	RW	DMA_6	DMA_6 interrupt priority
13	RW	DMA_5	DMA_5 interrupt priority
12	RW	DMA_4	DMA_4 interrupt priority
11	RW	DMA_3	DMA_3 interrupt priority
10	RW	DMA_2	DMA_2 interrupt priority
9	RW	DMA_1	DMA_1 interrupt priority
8	RW	DMA_0	DMA_0 interrupt priority
7	RW	TIMER_3	TIMER_3 interrupt priority
6	RW	TIMER_2	TIMER_2 interrupt priority
5	RW	TIMER_1	TIMER_1 interrupt priority
4	RW	TIMER_0	TIMER_0 interrupt priority
3	RW	AUDIOSINK_1	AUDIOSINK_1 interrupt priority
2	RW	AUDIOSINK_0	AUDIOSINK_0 interrupt priority
1	RW	LSAD_BATMON	LSAD_BATMON interrupt priority
0	RW	WATCHDOG	WATCHDOG interrupt priority

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
SPI_0_TX	INT_NORMALPRIO_SPI_0_TX	SPI_0_TX has normal priority	0x0*
	INT_HIGHPRIO_SPI_0_TX	SPI_0_TX has high priority	0x1
SPI_0_RX	INT_NORMALPRIO_SPI_0_RX	SPI_0_RX has normal priority	0x0*
	INT_HIGHPRIO_SPI_0_RX	SPI_0_RX has high priority	0x1
SPI_0_COM	INT_NORMALPRIO_SPI_0_COM	SPI_0_COM has normal priority	0x0*
	INT_HIGHPRIO_SPI_0_COM	SPI_0_COM has high priority	0x1
D_I2C	INT_NORMALPRIO_D_I2C	D_I2C has normal priority	0x0*
	INT_HIGHPRIO_D_I2C	D_I2C has high priority	0x1
I2C_1	INT_NORMALPRIO_I2C_1	I2C_1 has normal priority	0x0*
	INT_HIGHPRIO_I2C_1	I2C_1 has high priority	0x1
I2C_0	INT_NORMALPRIO_I2C_0	I2C_0 has normal priority	0x0*
	INT_HIGHPRIO_I2C_0	I2C_0 has high priority	0x1
I3C_1	INT_NORMALPRIO_I3C_1	I3C_1 has normal priority	0x0*
	INT_HIGHPRIO_I3C_1	I3C_1 has high priority	0x1
I3C_0	INT_NORMALPRIO_I3C_0	I3C_0 has normal priority	0x0*
	INT_HIGHPRIO_I3C_0	I3C_0 has high priority	0x1
DIO_7	INT_NORMALPRIO_DIO_7	DIO_7 has normal priority	0x0*
	INT_HIGHPRIO_DIO_7	DIO_7 has high priority	0x1
DIO_6	INT_NORMALPRIO_DIO_6	DIO_6 has normal priority	0x0*
	INT_HIGHPRIO_DIO_6	DIO_6 has high priority	0x1
DIO_5	INT_NORMALPRIO_DIO_5	DIO_5 has normal priority	0x0*
	INT_HIGHPRIO_DIO_5	DIO_5 has high priority	0x1
DIO_4	INT_NORMALPRIO_DIO_4	DIO_4 has normal priority	0x0*
	INT_HIGHPRIO_DIO_4	DIO_4 has high priority	0x1
DIO_3	INT_NORMALPRIO_DIO_3	DIO_3 has normal priority	0x0*
	INT_HIGHPRIO_DIO_3	DIO_3 has high priority	0x1
DIO_2	INT_NORMALPRIO_DIO_2	DIO_2 has normal priority	0x0*
	INT_HIGHPRIO_DIO_2	DIO_2 has high priority	0x1
DIO_1	INT_NORMALPRIO_DIO_1	DIO_1 has normal priority	0x0*
	INT_HIGHPRIO_DIO_1	DIO_1 has high priority	0x1
DIO_0	INT_NORMALPRIO_DIO_0	DIO_0 has normal priority	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	INT_HIGHPRIO_DIO_0	DIO_0 has high priority	0x1
DMA_7	INT_NORMALPRIO_DMA_7	DMA_7 has normal priority	0x0*
	INT_HIGHPRIO_DMA_7	DMA_7 has high priority	0x1
DMA_6	INT_NORMALPRIO_DMA_6	DMA_6 has normal priority	0x0*
	INT_HIGHPRIO_DMA_6	DMA_6 has high priority	0x1
DMA_5	INT_NORMALPRIO_DMA_5	DMA_5 has normal priority	0x0*
	INT_HIGHPRIO_DMA_5	DMA_5 has high priority	0x1
DMA_4	INT_NORMALPRIO_DMA_4	DMA_4 has normal priority	0x0*
	INT_HIGHPRIO_DMA_4	DMA_4 has high priority	0x1
DMA_3	INT_NORMALPRIO_DMA_3	DMA_3 has normal priority	0x0*
	INT_HIGHPRIO_DMA_3	DMA_3 has high priority	0x1
DMA_2	INT_NORMALPRIO_DMA_2	DMA_2 has normal priority	0x0*
	INT_HIGHPRIO_DMA_2	DMA_2 has high priority	0x1
DMA_1	INT_NORMALPRIO_DMA_1	DMA_1 has normal priority	0x0*
	INT_HIGHPRIO_DMA_1	DMA_1 has high priority	0x1
DMA_0	INT_NORMALPRIO_DMA_0	DMA_0 has normal priority	0x0*
	INT_HIGHPRIO_DMA_0	DMA_0 has high priority	0x1
TIMER_3	INT_NORMALPRIO_TIMER_3	TIMER_3 has normal priority	0x0*
	INT_HIGHPRIO_TIMER_3	TIMER_3 has high priority	0x1
TIMER_2	INT_NORMALPRIO_TIMER_2	TIMER_2 has normal priority	0x0*
	INT_HIGHPRIO_TIMER_2	TIMER_2 has high priority	0x1
TIMER_1	INT_NORMALPRIO_TIMER_1	TIMER_1 has normal priority	0x0*
	INT_HIGHPRIO_TIMER_1	TIMER_1 has high priority	0x1
TIMER_0	INT_NORMALPRIO_TIMER_0	TIMER_0 has normal priority	0x0*
	INT_HIGHPRIO_TIMER_0	TIMER_0 has high priority	0x1
AUDIOSINK_1	INT_NORMALPRIO_AUDIOSINK_1	AUDIOSINK_1 has normal priority	0x0*
	INT_HIGHPRIO_AUDIOSINK_1	AUDIOSINK_1 has high priority	0x1
AUDIOSINK_0	INT_NORMALPRIO_AUDIOSINK_0	AUDIOSINK_0 has normal priority	0x0*
	INT_HIGHPRIO_AUDIOSINK_0	AUDIOSINK_0 has high priority	0x1
LSAD_BATMON	INT_NORMALPRIO_LSAD_BATMON	LSAD_BATMON has normal priority	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	INT_HIGHPRIO_LSAD_BATMON	LSAD_BATMON has high priority	0x1
WATCHDOG	INT_NORMALPRIO_WATCHDOG	WATCHDOG has normal priority	0x0*
	INT_HIGHPRIO_WATCHDOG	WATCHDOG has high priority	0x1

3.2.1.9 D_INT_PRIORITY_1, INT_PRIORITY_1

Bit Field	Read/Write	Field Name	Description
31	RW	FIFO_7	FIFO_7 interrupt priority
30	RW	FIFO_6	FIFO_6 interrupt priority
29	RW	FIFO_5	FIFO_5 interrupt priority
28	RW	FIFO_4	FIFO_4 interrupt priority
27	RW	FIFO_3	FIFO_3 interrupt priority
26	RW	FIFO_2	FIFO_2 interrupt priority
25	RW	FIFO_1	FIFO_1 interrupt priority
24	RW	FIFO_0	FIFO_0 interrupt priority
23	RW	RS_DEC_OUT_RDY	RS_DEC_OUT_RDY interrupt priority
22	RW	RS_DEC_IN_RDY	RS_DEC_IN_RDY interrupt priority
21	RW	RS_ENC_OUT_RDY	RS_ENC_OUT_RDY interrupt priority
20	RW	RS_ENC_IN_RDY	RS_ENC_IN_RDY interrupt priority
19	RW	G722_DEC_OUT_RDY	G722_DEC_OUT_RDY interrupt priority
18	RW	G722_DEC_IN_RDY	G722_DEC_IN_RDY interrupt priority
17	RW	G722_ENC_OUT_RDY	G722_ENC_OUT_RDY interrupt priority
16	RW	G722_ENC_IN_RDY	G722_ENC_IN_RDY interrupt priority
15	RW	ASRC_IDLE	ASRC_IDLE interrupt priority
14	RW	ASRC_OUT	ASRC_OUT interrupt priority
13	RW	ASRC_IN	ASRC_IN interrupt priority
12	RW	ASRC_ERROR	ASRC_ERROR interrupt priority
11	RW	PCM_1_RX_TX	PCM_1_RX_TX interrupt priority
10	RW	PCM_1_ERROR	PCM_1_ERROR interrupt priority
9	RW	PCM_0_RX_TX	PCM_0_RX_TX interrupt priority
8	RW	PCM_0_ERROR	PCM_0_ERROR interrupt priority
7	RW	FENG	FENG interrupt priority
6	RW	UART_0_TX	UART_0_TX interrupt priority

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
5	RW	UART_0_RX	UART_0_RX interrupt priority
4	RW	UART_0_ERROR	UART_0_ERROR interrupt priority
3	RW	EMMC	EMMC interrupt priority
2	RW	SPI_1_TX	SPI_1_TX interrupt priority
1	RW	SPI_1_RX	SPI_1_RX interrupt priority
0	RW	SPI_1_COM	SPI_1_COM interrupt priority

Field Name	Value Symbol	Value Description	Hex Value
FIFO_7	INT_NORMALPRIO_FIFO_7	FIFO_7 has normal priority	0x0*
	INT_HIGHPRIO_FIFO_7	FIFO_7 has high priority	0x1
FIFO_6	INT_NORMALPRIO_FIFO_6	FIFO_6 has normal priority	0x0*
	INT_HIGHPRIO_FIFO_6	FIFO_6 has high priority	0x1
FIFO_5	INT_NORMALPRIO_FIFO_5	FIFO_5 has normal priority	0x0*
	INT_HIGHPRIO_FIFO_5	FIFO_5 has high priority	0x1
FIFO_4	INT_NORMALPRIO_FIFO_4	FIFO_4 has normal priority	0x0*
	INT_HIGHPRIO_FIFO_4	FIFO_4 has high priority	0x1
FIFO_3	INT_NORMALPRIO_FIFO_3	FIFO_3 has normal priority	0x0*
	INT_HIGHPRIO_FIFO_3	FIFO_3 has high priority	0x1
FIFO_2	INT_NORMALPRIO_FIFO_2	FIFO_2 has normal priority	0x0*
	INT_HIGHPRIO_FIFO_2	FIFO_2 has high priority	0x1
FIFO_1	INT_NORMALPRIO_FIFO_1	FIFO_1 has normal priority	0x0*
	INT_HIGHPRIO_FIFO_1	FIFO_1 has high priority	0x1
FIFO_0	INT_NORMALPRIO_FIFO_0	FIFO_0 has normal priority	0x0*
	INT_HIGHPRIO_FIFO_0	FIFO_0 has high priority	0x1
RS_DEC_OUT_RDY	INT_NORMALPRIO_RS_DEC_OUT_RDY	RS_DEC_OUT_RDY has normal priority	0x0*
	INT_HIGHPRIO_RS_DEC_OUT_RDY	RS_DEC_OUT_RDY has high priority	0x1
RS_DEC_IN_RDY	INT_NORMALPRIO_RS_DEC_IN_RDY	RS_DEC_IN_RDY has normal priority	0x0*
	INT_HIGHPRIO_RS_DEC_IN_RDY	RS_DEC_IN_RDY has high priority	0x1
RS_ENC_OUT_RDY	INT_NORMALPRIO_RS_ENC_OUT_RDY	RS_ENC_OUT_RDY has normal priority	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	INT_HIGHPRIO_RS_ENC_OUT_RDY	RS_ENC_OUT_RDY has high priority	0x1
RS_ENC_IN_RDY	INT_NORMALPRIO_RS_ENC_IN_RDY	RS_ENC_IN_RDY has normal priority	0x0*
	INT_HIGHPRIO_RS_ENC_IN_RDY	RS_ENC_IN_RDY has high priority	0x1
G722_DEC_OUT_RDY	INT_NORMALPRIO_G722_DEC_OUT_RDY	G722_DEC_OUT_RDY has normal priority	0x0*
	INT_HIGHPRIO_G722_DEC_OUT_RDY	G722_DEC_OUT_RDY has high priority	0x1
G722_DEC_IN_RDY	INT_NORMALPRIO_G722_DEC_IN_RDY	G722_DEC_IN_RDY has normal priority	0x0*
	INT_HIGHPRIO_G722_DEC_IN_RDY	G722_DEC_IN_RDY has high priority	0x1
G722_ENC_OUT_RDY	INT_NORMALPRIO_G722_ENC_OUT_RDY	G722_ENC_OUT_RDY has normal priority	0x0*
	INT_HIGHPRIO_G722_ENC_OUT_RDY	G722_ENC_OUT_RDY has high priority	0x1
G722_ENC_IN_RDY	INT_NORMALPRIO_G722_ENC_IN_RDY	G722_ENC_IN_RDY has normal priority	0x0*
	INT_HIGHPRIO_G722_ENC_IN_RDY	G722_ENC_IN_RDY has high priority	0x1
ASRC_IDLE	INT_NORMALPRIO_ASRC_IDLE	ASRC_IDLE has normal priority	0x0*
	INT_HIGHPRIO_ASRC_IDLE	ASRC_IDLE has high priority	0x1
ASRC_OUT	INT_NORMALPRIO_ASRC_OUT	ASRC_OUT has normal priority	0x0*
	INT_HIGHPRIO_ASRC_OUT	ASRC_OUT has high priority	0x1
ASRC_IN	INT_NORMALPRIO_ASRC_IN	ASRC_IN has normal priority	0x0*
	INT_HIGHPRIO_ASRC_IN	ASRC_IN has high priority	0x1
ASRC_ERROR	INT_NORMALPRIO_ASRC_ERROR	ASRC_ERROR has normal priority	0x0*
	INT_HIGHPRIO_ASRC_ERROR	ASRC_ERROR has high priority	0x1
PCM_1_RX_TX	INT_NORMALPRIO_PCM_1_RX_TX	PCM_1_RX_TX has normal priority	0x0*
	INT_HIGHPRIO_PCM_1_RX_TX	PCM_1_RX_TX has high priority	0x1
PCM_1_ERROR	INT_NORMALPRIO_PCM_1_ERROR	PCM_1_ERROR has normal priority	0x0*
	INT_HIGHPRIO_PCM_1_ERROR	PCM_1_ERROR has high priority	0x1
PCM_0_RX_TX	INT_NORMALPRIO_PCM_0_RX_TX	PCM_0_RX_TX has normal priority	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	INT_HIGHPRIO_PCM_0_RX_TX	PCM_0_RX_TX has high priority	0x1
PCM_0_ERROR	INT_NORMALPRIO_PCM_0_ERROR	PCM_0_ERROR has normal priority	0x0*
	INT_HIGHPRIO_PCM_0_ERROR	PCM_0_ERROR has high priority	0x1
FENG	INT_NORMALPRIO_FENG	FENG has normal priority	0x0*
	INT_HIGHPRIO_FENG	FENG has high priority	0x1
UART_0_TX	INT_NORMALPRIO_UART_0_TX	UART_0_TX has normal priority	0x0*
	INT_HIGHPRIO_UART_0_TX	UART_0_TX has high priority	0x1
UART_0_RX	INT_NORMALPRIO_UART_0_RX	UART_0_RX has normal priority	0x0*
	INT_HIGHPRIO_UART_0_RX	UART_0_RX has high priority	0x1
UART_0_ERROR	INT_NORMALPRIO_UART_0_ERROR	UART_0_ERROR has normal priority	0x0*
	INT_HIGHPRIO_UART_0_ERROR	UART_0_ERROR has high priority	0x1
EMMC	INT_NORMALPRIO_EMMC	EMMC has normal priority	0x0*
	INT_HIGHPRIO_EMMC	EMMC has high priority	0x1
SPI_1_TX	INT_NORMALPRIO_SPI_1_TX	SPI_1_TX has normal priority	0x0*
	INT_HIGHPRIO_SPI_1_TX	SPI_1_TX has high priority	0x1
SPI_1_RX	INT_NORMALPRIO_SPI_1_RX	SPI_1_RX has normal priority	0x0*
	INT_HIGHPRIO_SPI_1_RX	SPI_1_RX has high priority	0x1
SPI_1_COM	INT_NORMALPRIO_SPI_1_COM	SPI_1_COM has normal priority	0x0*
	INT_HIGHPRIO_SPI_1_COM	SPI_1_COM has high priority	0x1

3.2.1.10 D_INT_PRIORITY_2, INT_PRIORITY_2

Bit Field	Read/Write	Field Name	Description
29	RW	RS_COM	RS_COM interrupt priority
28	RW	G722_COM	G722_COM interrupt priority
27	RW	MEM_ERROR	MEM_ERROR interrupt priority
26	RW	LSAD	LSAD interrupt priority
25	RW	CLK_DET	CLK_DET interrupt priority
24	RW	NNA	NNA interrupt priority
23	RW	CM3_7	CM3_7 interrupt priority
22	RW	CM3_6	CM3_6 interrupt priority

onsemi Confidential

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
21	RW	CM3_5	CM3_5 interrupt priority
20	RW	CM3_4	CM3_4 interrupt priority
19	RW	CM3_3	CM3_3 interrupt priority
18	RW	CM3_2	CM3_2 interrupt priority
17	RW	CM3_1	CM3_1 interrupt priority
16	RW	CM3_0	CM3_0 interrupt priority
15	RW	DSP_7	DSP_7 interrupt priority
14	RW	DSP_6	DSP_6 interrupt priority
13	RW	DSP_5	DSP_5 interrupt priority
12	RW	DSP_4	DSP_4 interrupt priority
11	RW	DSP_3	DSP_3 interrupt priority
10	RW	DSP_2	DSP_2 interrupt priority
9	RW	DSP_1	DSP_1 interrupt priority
8	RW	DSP_0	DSP_0 interrupt priority
7	RW	HEAR_7	HEAR_7 interrupt priority
6	RW	HEAR_6	HEAR_6 interrupt priority
5	RW	HEAR_5	HEAR_5 interrupt priority
4	RW	HEAR_4	HEAR_4 interrupt priority
3	RW	HEAR_3	HEAR_3 interrupt priority
2	RW	HEAR_2	HEAR_2 interrupt priority
1	RW	HEAR_1	HEAR_1 interrupt priority
0	RW	HEAR_0	HEAR_0 interrupt priority

Field Name	Value Symbol	Value Description	Hex Value
RS_COM	INT_NORMALPRIO_RS_COM	RS_COM has normal priority	0x0*
	INT_HIGHPRIO_RS_COM	RS_COM has high priority	0x1
G722_COM	INT_NORMALPRIO_G722_COM	G722_COM has normal priority	0x0*
	INT_HIGHPRIO_G722_COM	G722_COM has high priority	0x1
MEM_ERROR	INT_NORMALPRIO_MEM_ERROR	MEM_ERROR has normal priority	0x0*
	INT_HIGHPRIO_MEM_ERROR	MEM_ERROR has high priority	0x1
LSAD	INT_NORMALPRIO_LSAD	LSAD has normal priority	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	INT_HIGHPRIO_LSAD	LSAD has high priority	0x1
CLK_DET	INT_NORMALPRIO_CLK_DET	CLK_DET has normal priority	0x0*
	INT_HIGHPRIO_CLK_DET	CLK_DET has high priority	0x1
NNA	INT_NORMALPRIO_NNA	NNA has normal priority	0x0*
	INT_HIGHPRIO_NNA	NNA has high priority	0x1
CM3_7	INT_NORMALPRIO_CM3_7	CM3_7 has normal priority	0x0*
	INT_HIGHPRIO_CM3_7	CM3_7 has high priority	0x1
CM3_6	INT_NORMALPRIO_CM3_6	CM3_6 has normal priority	0x0*
	INT_HIGHPRIO_CM3_6	CM3_6 has high priority	0x1
CM3_5	INT_NORMALPRIO_CM3_5	CM3_5 has normal priority	0x0*
	INT_HIGHPRIO_CM3_5	CM3_5 has high priority	0x1
CM3_4	INT_NORMALPRIO_CM3_4	CM3_4 has normal priority	0x0*
	INT_HIGHPRIO_CM3_4	CM3_4 has high priority	0x1
CM3_3	INT_NORMALPRIO_CM3_3	CM3_3 has normal priority	0x0*
	INT_HIGHPRIO_CM3_3	CM3_3 has high priority	0x1
CM3_2	INT_NORMALPRIO_CM3_2	CM3_2 has normal priority	0x0*
	INT_HIGHPRIO_CM3_2	CM3_2 has high priority	0x1
CM3_1	INT_NORMALPRIO_CM3_1	CM3_1 has normal priority	0x0*
	INT_HIGHPRIO_CM3_1	CM3_1 has high priority	0x1
CM3_0	INT_NORMALPRIO_CM3_0	CM3_0 has normal priority	0x0*
	INT_HIGHPRIO_CM3_0	CM3_0 has high priority	0x1
DSP_7	INT_NORMALPRIO_DSP_7	DSP_7 has normal priority	0x0*
	INT_HIGHPRIO_DSP_7	DSP_7 has high priority	0x1
DSP_6	INT_NORMALPRIO_DSP_6	DSP_6 has normal priority	0x0*
	INT_HIGHPRIO_DSP_6	DSP_6 has high priority	0x1
DSP_5	INT_NORMALPRIO_DSP_5	DSP_5 has normal priority	0x0*
	INT_HIGHPRIO_DSP_5	DSP_5 has high priority	0x1
DSP_4	INT_NORMALPRIO_DSP_4	DSP_4 has normal priority	0x0*
	INT_HIGHPRIO_DSP_4	DSP_4 has high priority	0x1
DSP_3	INT_NORMALPRIO_DSP_3	DSP_3 has normal priority	0x0*
	INT_HIGHPRIO_DSP_3	DSP_3 has high priority	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DSP_2	INT_NORMALPRIO_DSP_2	DSP_2 has normal priority	0x0*
	INT_HIGHPRIO_DSP_2	DSP_2 has high priority	0x1
DSP_1	INT_NORMALPRIO_DSP_1	DSP_1 has normal priority	0x0*
	INT_HIGHPRIO_DSP_1	DSP_1 has high priority	0x1
DSP_0	INT_NORMALPRIO_DSP_0	DSP_0 has normal priority	0x0*
	INT_HIGHPRIO_DSP_0	DSP_0 has high priority	0x1
HEAR_7	INT_NORMALPRIO_HEAR_7	HEAR_7 has normal priority	0x0*
	INT_HIGHPRIO_HEAR_7	HEAR_7 has high priority	0x1
HEAR_6	INT_NORMALPRIO_HEAR_6	HEAR_6 has normal priority	0x0*
	INT_HIGHPRIO_HEAR_6	HEAR_6 has high priority	0x1
HEAR_5	INT_NORMALPRIO_HEAR_5	HEAR_5 has normal priority	0x0*
	INT_HIGHPRIO_HEAR_5	HEAR_5 has high priority	0x1
HEAR_4	INT_NORMALPRIO_HEAR_4	HEAR_4 has normal priority	0x0*
	INT_HIGHPRIO_HEAR_4	HEAR_4 has high priority	0x1
HEAR_3	INT_NORMALPRIO_HEAR_3	HEAR_3 has normal priority	0x0*
	INT_HIGHPRIO_HEAR_3	HEAR_3 has high priority	0x1
HEAR_2	INT_NORMALPRIO_HEAR_2	HEAR_2 has normal priority	0x0*
	INT_HIGHPRIO_HEAR_2	HEAR_2 has high priority	0x1
HEAR_1	INT_NORMALPRIO_HEAR_1	HEAR_1 has normal priority	0x0*
	INT_HIGHPRIO_HEAR_1	HEAR_1 has high priority	0x1
HEAR_0	INT_NORMALPRIO_HEAR_0	HEAR_0 has normal priority	0x0*
	INT_HIGHPRIO_HEAR_0	HEAR_0 has high priority	0x1

3.2.1.11 D_INT_EBL_0, INT_EBL_0

Bit Field	Read/Write	Field Name	Description
7	W	TIMER_3	TIMER_3 interrupt enable
6	W	TIMER_2	TIMER_2 interrupt enable
5	W	TIMER_1	TIMER_1 interrupt enable
4	W	TIMER_0	TIMER_0 interrupt enable
3	W	AUDIOSINK_1	AUDIOSINK_1 interrupt enable

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
2	W	AUDIOSINK_0	AUDIOSINK_0 interrupt enable
1	W	LSAD_BATMON	LSAD_BATMON interrupt enable
0	W	WATCHDOG	WATCHDOG interrupt enable

Field Name	Value Symbol	Value Description	Hex Value
TIMER_3	INT0_EBL_TIMER_3	Enable TIMER_3 interrupt	0x1
TIMER_2	INT0_EBL_TIMER_2	Enable TIMER_2 interrupt	0x1
TIMER_1	INT0_EBL_TIMER_1	Enable TIMER_1 interrupt	0x1
TIMER_0	INT0_EBL_TIMER_0	Enable TIMER_0 interrupt	0x1
AUDIOSINK_1	INT0_EBL_AUDIOSINK_1	Enable AUDIOSINK_1 interrupt	0x1
AUDIOSINK_0	INT0_EBL_AUDIOSINK_0	Enable AUDIOSINK_0 interrupt	0x1
LSAD_BATMON	INT0_EBL_LSAD_BATMON	Enable LSAD_BATMON interrupt	0x1
WATCHDOG	INT0_EBL_WATCHDOG	Enable WATCHDOG interrupt	0x1

3.2.1.12 D_INT_EBL_1, INT_EBL_1

Bit Field	Read/Write	Field Name	Description
7	W	DMA_7	DMA_7 interrupt enable
6	W	DMA_6	DMA_6 interrupt enable
5	W	DMA_5	DMA_5 interrupt enable
4	W	DMA_4	DMA_4 interrupt enable
3	W	DMA_3	DMA_3 interrupt enable
2	W	DMA_2	DMA_2 interrupt enable
1	W	DMA_1	DMA_1 interrupt enable
0	W	DMA_0	DMA_0 interrupt enable

Field Name	Value Symbol	Value Description	Hex Value
DMA_7	INT1_EBL_DMA_7	Enable DMA_7 interrupt	0x1
DMA_6	INT1_EBL_DMA_6	Enable DMA_6 interrupt	0x1
DMA_5	INT1_EBL_DMA_5	Enable DMA_5 interrupt	0x1
DMA_4	INT1_EBL_DMA_4	Enable DMA_4 interrupt	0x1
DMA_3	INT1_EBL_DMA_3	Enable DMA_3 interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DMA_2	INT1_EBL_DMA_2	Enable DMA_2 interrupt	0x1
DMA_1	INT1_EBL_DMA_1	Enable DMA_1 interrupt	0x1
DMA_0	INT1_EBL_DMA_0	Enable DMA_0 interrupt	0x1

3.2.1.13 D_INT_EBL_2, INT_EBL_2

Bit Field	Read/Write	Field Name	Description
7	W	DIO_7	DIO_7 interrupt enable
6	W	DIO_6	DIO_6 interrupt enable
5	W	DIO_5	DIO_5 interrupt enable
4	W	DIO_4	DIO_4 interrupt enable
3	W	DIO_3	DIO_3 interrupt enable
2	W	DIO_2	DIO_2 interrupt enable
1	W	DIO_1	DIO_1 interrupt enable
0	W	DIO_0	DIO_0 interrupt enable

Field Name	Value Symbol	Value Description	Hex Value
DIO_7	INT2_EBL_DIO_7	Enable DIO_7 interrupt	0x1
DIO_6	INT2_EBL_DIO_6	Enable DIO_6 interrupt	0x1
DIO_5	INT2_EBL_DIO_5	Enable DIO_5 interrupt	0x1
DIO_4	INT2_EBL_DIO_4	Enable DIO_4 interrupt	0x1
DIO_3	INT2_EBL_DIO_3	Enable DIO_3 interrupt	0x1
DIO_2	INT2_EBL_DIO_2	Enable DIO_2 interrupt	0x1
DIO_1	INT2_EBL_DIO_1	Enable DIO_1 interrupt	0x1
DIO_0	INT2_EBL_DIO_0	Enable DIO_0 interrupt	0x1

3.2.1.14 D_INT_EBL_3, INT_EBL_3

Bit Field	Read/Write	Field Name	Description
7	W	SPI_0_TX	SPI_0_TX interrupt enable
6	W	SPI_0_RX	SPI_0_RX interrupt enable
5	W	SPI_0_COM	SPI_0_COM interrupt enable
4	W	D_I2C	D_I2C interrupt enable

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
3	W	I2C_1	I2C_1 interrupt enable
2	W	I2C_0	I2C_0 interrupt enable
1	W	I3C_1	I3C_1 interrupt enable
0	W	I3C_0	I3C_0 interrupt enable

Field Name	Value Symbol	Value Description	Hex Value
SPI_0_TX	INT3_EBL_SPI_0_TX	Enable SPI_0_TX interrupt	0x1
SPI_0_RX	INT3_EBL_SPI_0_RX	Enable SPI_0_RX interrupt	0x1
SPI_0_COM	INT3_EBL_SPI_0_COM	Enable SPI_0_COM interrupt	0x1
D_I2C	INT3_EBL_D_I2C	Enable D_I2C interrupt	0x1
I2C_1	INT3_EBL_I2C_1	Enable I2C_1 interrupt	0x1
I2C_0	INT3_EBL_I2C_0	Enable I2C_0 interrupt	0x1
I3C_1	INT3_EBL_I3C_1	Enable I3C_1 interrupt	0x1
I3C_0	INT3_EBL_I3C_0	Enable I3C_0 interrupt	0x1

3.2.1.15 D_INT_EBL_4, INT_EBL_4

Bit Field	Read/Write	Field Name	Description
7	W	FENG	FENG interrupt enable
6	W	UART_0_TX	UART_0_TX interrupt enable
5	W	UART_0_RX	UART_0_RX interrupt enable
4	W	UART_0_ERROR	UART_0_ERROR interrupt enable
3	W	EMMC	EMMC interrupt enable
2	W	SPI_1_TX	SPI_1_TX interrupt enable
1	W	SPI_1_RX	SPI_1_RX interrupt enable
0	W	SPI_1_COM	SPI_1_COM interrupt enable

Field Name	Value Symbol	Value Description	Hex Value
FENG	INT4_EBL_FENG	Enable FENG interrupt	0x1
UART_0_TX	INT4_EBL_UART_0_TX	Enable UART_0_TX interrupt	0x1
UART_0_RX	INT4_EBL_UART_0_RX	Enable UART_0_RX interrupt	0x1
UART_0_ERROR	INT4_EBL_UART_0_ERROR	Enable UART_0_ERROR interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
EMMC	INT4_EBL_EMMC	Enable EMMC interrupt	0x1
SPI_1_TX	INT4_EBL_SPI_1_TX	Enable SPI_1_TX interrupt	0x1
SPI_1_RX	INT4_EBL_SPI_1_RX	Enable SPI_1_RX interrupt	0x1
SPI_1_COM	INT4_EBL_SPI_1_COM	Enable SPI_1_COM interrupt	0x1

3.2.1.16 D_INT_EBL_5, INT_EBL_5

Field Name	Value Symbol	Value Description	Hex Value
ASRC_IDLE	INT5_EBL_ASRC_IDLE	Enable ASRC_IDLE interrupt	0x1
ASRC_OUT	INT5_EBL_ASRC_OUT	Enable ASRC_OUT interrupt	0x1
ASRC_IN	INT5_EBL_ASRC_IN	Enable ASRC_IN interrupt	0x1
ASRC_ERROR	INT5_EBL_ASRC_ERROR	Enable ASRC_ERROR interrupt	0x1
PCM_1_RX_TX	INT5_EBL_PCM_1_RX_TX	Enable PCM_1_RX_TX interrupt	0x1
PCM_1_ERROR	INT5_EBL_PCM_1_ERROR	Enable PCM_1_ERROR interrupt	0x1
PCM_0_RX_TX	INT5_EBL_PCM_0_RX_TX	Enable PCM_0_RX_TX interrupt	0x1
PCM_0_ERROR	INT5_EBL_PCM_0_ERROR	Enable PCM_0_ERROR interrupt	0x1

Bit Field	Read/Write	Field Name	Description
7	W	ASRC_IDLE	ASRC_IDLE interrupt enable
6	W	ASRC_OUT	ASRC_OUT interrupt enable
5	W	ASRC_IN	ASRC_IN interrupt enable
4	W	ASRC_ERROR	ASRC_ERROR interrupt enable
3	W	PCM_1_RX_TX	PCM_1_RX_TX interrupt enable
2	W	PCM_1_ERROR	PCM_1_ERROR interrupt enable
1	W	PCM_0_RX_TX	PCM_0_RX_TX interrupt enable
0	W	PCM_0_ERROR	PCM_0_ERROR interrupt enable

3.2.1.17 D_INT_EBL_7, INT_EBL_7

Bit Field	Read/Write	Field Name	Description
7	W	FIFO_7	FIFO_7 interrupt enable
6	W	FIFO_6	FIFO_6 interrupt enable
5	W	FIFO_5	FIFO_5 interrupt enable

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
4	W	FIFO_4	FIFO_4 interrupt enable
3	W	FIFO_3	FIFO_3 interrupt enable
2	W	FIFO_2	FIFO_2 interrupt enable
1	W	FIFO_1	FIFO_1 interrupt enable
0	W	FIFO_0	FIFO_0 interrupt enable

Field Name	Value Symbol	Value Description	Hex Value
FIFO_7	INT7_EBL_FIFO_7	Enable FIFO_7 interrupt	0x1
FIFO_6	INT7_EBL_FIFO_6	Enable FIFO_6 interrupt	0x1
FIFO_5	INT7_EBL_FIFO_5	Enable FIFO_5 interrupt	0x1
FIFO_4	INT7_EBL_FIFO_4	Enable FIFO_4 interrupt	0x1
FIFO_3	INT7_EBL_FIFO_3	Enable FIFO_3 interrupt	0x1
FIFO_2	INT7_EBL_FIFO_2	Enable FIFO_2 interrupt	0x1
FIFO_1	INT7_EBL_FIFO_1	Enable FIFO_1 interrupt	0x1
FIFO_0	INT7_EBL_FIFO_0	Enable FIFO_0 interrupt	0x1

3.2.1.18 D_INT_EBL_8, INT_EBL_8

Bit Field	Read/Write	Field Name	Description
7	W	HEAR_7	HEAR_7 interrupt enable
6	W	HEAR_6	HEAR_6 interrupt enable
5	W	HEAR_5	HEAR_5 interrupt enable
4	W	HEAR_4	HEAR_4 interrupt enable
3	W	HEAR_3	HEAR_3 interrupt enable
2	W	HEAR_2	HEAR_2 interrupt enable
1	W	HEAR_1	HEAR_1 interrupt enable
0	W	HEAR_0	HEAR_0 interrupt enable

Field Name	Value Symbol	Value Description	Hex Value
HEAR_7	INT8_EBL_HEAR_7	Enable HEAR_7 interrupt	0x1
HEAR_6	INT8_EBL_HEAR_6	Enable HEAR_6 interrupt	0x1
HEAR_5	INT8_EBL_HEAR_5	Enable HEAR_5 interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
HEAR_4	INT8_EBL_HEAR_4	Enable HEAR_4 interrupt	0x1
HEAR_3	INT8_EBL_HEAR_3	Enable HEAR_3 interrupt	0x1
HEAR_2	INT8_EBL_HEAR_2	Enable HEAR_2 interrupt	0x1
HEAR_1	INT8_EBL_HEAR_1	Enable HEAR_1 interrupt	0x1
HEAR_0	INT8_EBL_HEAR_0	Enable HEAR_0 interrupt	0x1

3.2.1.19 D_INT_EBL_9, INT_EBL_9

Bit Field	Read/Write	Field Name	Description
7	W	DSP_7	DSP_7 interrupt enable
6	W	DSP_6	DSP_6 interrupt enable
5	W	DSP_5	DSP_5 interrupt enable
4	W	DSP_4	DSP_4 interrupt enable
3	W	DSP_3	DSP_3 interrupt enable
2	W	DSP_2	DSP_2 interrupt enable
1	W	DSP_1	DSP_1 interrupt enable
0	W	DSP_0	DSP_0 interrupt enable

Field Name	Value Symbol	Value Description	Hex Value
DSP_7	INT9_EBL_DSP_7	Enable DSP_7 interrupt	0x1
DSP_6	INT9_EBL_DSP_6	Enable DSP_6 interrupt	0x1
DSP_5	INT9_EBL_DSP_5	Enable DSP_5 interrupt	0x1
DSP_4	INT9_EBL_DSP_4	Enable DSP_4 interrupt	0x1
DSP_3	INT9_EBL_DSP_3	Enable DSP_3 interrupt	0x1
DSP_2	INT9_EBL_DSP_2	Enable DSP_2 interrupt	0x1
DSP_1	INT9_EBL_DSP_1	Enable DSP_1 interrupt	0x1
DSP_0	INT9_EBL_DSP_0	Enable DSP_0 interrupt	0x1

Ezairo 8300 Hardware Reference

3.2.1.20 D_INT_EBL_10, INT_EBL_10

Bit Field	Read/Write	Field Name	Description
7	W	CM3_7	CM3_7 interrupt enable
6	W	CM3_6	CM3_6 interrupt enable
5	W	CM3_5	CM3_5 interrupt enable
4	W	CM3_4	CM3_4 interrupt enable
3	W	CM3_3	CM3_3 interrupt enable
2	W	CM3_2	CM3_2 interrupt enable
1	W	CM3_1	CM3_1 interrupt enable
0	W	CM3_0	CM3_0 interrupt enable

Field Name	Value Symbol	Value Description	Hex Value
CM3_7	INT10_EBL_CM3_7	Enable CM3_7 interrupt	0x1
CM3_6	INT10_EBL_CM3_6	Enable CM3_6 interrupt	0x1
CM3_5	INT10_EBL_CM3_5	Enable CM3_5 interrupt	0x1
CM3_4	INT10_EBL_CM3_4	Enable CM3_4 interrupt	0x1
CM3_3	INT10_EBL_CM3_3	Enable CM3_3 interrupt	0x1
CM3_2	INT10_EBL_CM3_2	Enable CM3_2 interrupt	0x1
CM3_1	INT10_EBL_CM3_1	Enable CM3_1 interrupt	0x1
CM3_0	INT10_EBL_CM3_0	Enable CM3_0 interrupt	0x1

3.2.1.21 D_INT_EBL_11, INT_EBL_11

Bit Field	Read/Write	Field Name	Description
5	W	RS_COM	RS_COM interrupt enable
4	W	G722_COM	G722_COM interrupt enable
3	W	MEM_ERROR	MEM_ERROR interrupt enable
2	W	LSAD	LSAD interrupt enable
1	W	CLK_DET	CLK_DET interrupt enable
0	W	NNA	NNA interrupt enable

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
RS_COM	INT11_EBL_RS_COM	Enable RS_COM interrupt	0x1
G722_COM	INT11_EBL_G722_COM	Enable G722_COM interrupt	0x1
MEM_ERROR	INT11_EBL_MEM_ERROR	Enable MEM_ERROR interrupt	0x1
LSAD	INT11_EBL_LSAD	Enable LSAD interrupt	0x1
CLK_DET	INT11_EBL_CLK_DET	Enable CLK_DET interrupt	0x1
NNA	INT11_EBL_NNA	Enable NNA interrupt	0x1

3.2.1.22 D_INT_DIS_0, INT_DIS_0

Bit Field	Read/Write	Field Name	Description
7	W	TIMER_3	TIMER_3 interrupt disable
6	W	TIMER_2	TIMER_2 interrupt disable
5	W	TIMER_1	TIMER_1 interrupt disable
4	W	TIMER_0	TIMER_0 interrupt disable
3	W	AUDIOSINK_1	AUDIOSINK_1 interrupt disable
2	W	AUDIOSINK_0	AUDIOSINK_0 interrupt disable
1	W	LSAD_BATMON	LSAD_BATMON interrupt disable
0	W	WATCHDOG	WATCHDOG interrupt disable

Field Name	Value Symbol	Value Description	Hex Value
TIMER_3	INT0_DIS_TIMER_3	Disable TIMER_3 interrupt	0x1
TIMER_2	INT0_DIS_TIMER_2	Disable TIMER_2 interrupt	0x1
TIMER_1	INT0_DIS_TIMER_1	Disable TIMER_1 interrupt	0x1
TIMER_0	INT0_DIS_TIMER_0	Disable TIMER_0 interrupt	0x1
AUDIOSINK_1	INT0_DIS_AUDIOSINK_1	Disable AUDIOSINK_1 interrupt	0x1
AUDIOSINK_0	INT0_DIS_AUDIOSINK_0	Disable AUDIOSINK_0 interrupt	0x1
LSAD_BATMON	INT0_DIS_LSAD_BATMON	Disable LSAD_BATMON interrupt	0x1
WATCHDOG	INT0_DIS_WATCHDOG	Disable WATCHDOG interrupt	0x1

Ezairo 8300 Hardware Reference

3.2.1.23 D_INT_DIS_1, INT_DIS_1

Bit Field	Read/ Write	Field Name	Description
7	W	DMA_7	DMA_7 interrupt disable
6	W	DMA_6	DMA_6 interrupt disable
5	W	DMA_5	DMA_5 interrupt disable
4	W	DMA_4	DMA_4 interrupt disable
3	W	DMA_3	DMA_3 interrupt disable
2	W	DMA_2	DMA_2 interrupt disable
1	W	DMA_1	DMA_1 interrupt disable
0	W	DMA_0	DMA_0 interrupt disable

Field Name	Value Symbol	Value Description	Hex Value
DMA_7	INT1_DIS_DMA_7	Disable DMA_7 interrupt	0x1
DMA_6	INT1_DIS_DMA_6	Disable DMA_6 interrupt	0x1
DMA_5	INT1_DIS_DMA_5	Disable DMA_5 interrupt	0x1
DMA_4	INT1_DIS_DMA_4	Disable DMA_4 interrupt	0x1
DMA_3	INT1_DIS_DMA_3	Disable DMA_3 interrupt	0x1
DMA_2	INT1_DIS_DMA_2	Disable DMA_2 interrupt	0x1
DMA_1	INT1_DIS_DMA_1	Disable DMA_1 interrupt	0x1
DMA_0	INT1_DIS_DMA_0	Disable DMA_0 interrupt	0x1

3.2.1.24 D_INT_DIS_2, INT_DIS_2

Bit Field	Read/Write	Field Name	Description
7	W	DIO_7	DIO_7 interrupt disable
6	W	DIO_6	DIO_6 interrupt disable
5	W	DIO_5	DIO_5 interrupt disable
4	W	DIO_4	DIO_4 interrupt disable
3	W	DIO_3	DIO_3 interrupt disable
2	W	DIO_2	DIO_2 interrupt disable
1	W	DIO_1	DIO_1 interrupt disable
0	W	DIO_0	DIO_0 interrupt disable

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DIO_7	INT2_DIS_DIO_7	Disable DIO_7 interrupt	0x1
DIO_6	INT2_DIS_DIO_6	Disable DIO_6 interrupt	0x1
DIO_5	INT2_DIS_DIO_5	Disable DIO_5 interrupt	0x1
DIO_4	INT2_DIS_DIO_4	Disable DIO_4 interrupt	0x1
DIO_3	INT2_DIS_DIO_3	Disable DIO_3 interrupt	0x1
DIO_2	INT2_DIS_DIO_2	Disable DIO_2 interrupt	0x1
DIO_1	INT2_DIS_DIO_1	Disable DIO_1 interrupt	0x1
DIO_0	INT2_DIS_DIO_0	Disable DIO_0 interrupt	0x1

3.2.1.25 D_INT_DIS_3, INT_DIS_3

Bit Field	Read/Write	Field Name	Description
7	W	SPI_0_TX	SPI_0_TX interrupt disable
6	W	SPI_0_RX	SPI_0_RX interrupt disable
5	W	SPI_0_COM	SPI_0_COM interrupt disable
4	W	D_I2C	D_I2C interrupt disable
3	W	I2C_1	I2C_1 interrupt disable
2	W	I2C_0	I2C_0 interrupt disable
1	W	I3C_1	I3C_1 interrupt disable
0	W	I3C_0	I3C_0 interrupt disable

Field Name	Value Symbol	Value Description	Hex Value
SPI_0_TX	INT3_DIS_SPI_0_TX	Disable SPI_0_TX interrupt	0x1
SPI_0_RX	INT3_DIS_SPI_0_RX	Disable SPI_0_RX interrupt	0x1
SPI_0_COM	INT3_DIS_SPI_0_COM	Disable SPI_0_COM interrupt	0x1
D_I2C	INT3_DIS_D_I2C	Disable D_I2C interrupt	0x1
I2C_1	INT3_DIS_I2C_1	Disable I2C_1 interrupt	0x1
I2C_0	INT3_DIS_I2C_0	Disable I2C_0 interrupt	0x1
I3C_1	INT3_DIS_I3C_1	Disable I3C_1 interrupt	0x1
I3C_0	INT3_DIS_I3C_0	Disable I3C_0 interrupt	0x1

Ezairo 8300 Hardware Reference

3.2.1.26 D_INT_DIS_4, INT_DIS_4

Bit Field	Read/Write	Field Name	Description
7	W	FENG	FENG interrupt disable
6	W	UART_0_TX	UART_0_TX interrupt disable
5	W	UART_0_RX	UART_0_RX interrupt disable
4	W	UART_0_ERROR	UART_0_ERROR interrupt disable
3	W	EMMC	EMMC interrupt disable
2	W	SPI_1_TX	SPI_1_TX interrupt disable
1	W	SPI_1_RX	SPI_1_RX interrupt disable
0	W	SPI_1_COM	SPI_1_COM interrupt disable

Field Name	Value Symbol	Value Description	Hex Value
FENG	INT4_DIS_FENG	Disable FENG interrupt	0x1
UART_0_TX	INT4_DIS_UART_0_TX	Disable UART_0_TX interrupt	0x1
UART_0_RX	INT4_DIS_UART_0_RX	Disable UART_0_RX interrupt	0x1
UART_0_ERROR	INT4_DIS_UART_0_ERROR	Disable UART_0_ERROR interrupt	0x1
EMMC	INT4_DIS_EMMC	Disable EMMC interrupt	0x1
SPI_1_TX	INT4_DIS_SPI_1_TX	Disable SPI_1_TX interrupt	0x1
SPI_1_RX	INT4_DIS_SPI_1_RX	Disable SPI_1_RX interrupt	0x1
SPI_1_COM	INT4_DIS_SPI_1_COM	Disable SPI_1_COM interrupt	0x1

3.2.1.27 D_INT_DIS_5, INT_DIS_5

Bit Field	Read/Write	Field Name	Description
7	W	ASRC_IDLE	ASRC_IDLE interrupt disable
6	W	ASRC_OUT	ASRC_OUT interrupt disable
5	W	ASRC_IN	ASRC_IN interrupt disable
4	W	ASRC_ERROR	ASRC_ERROR interrupt disable
3	W	PCM_1_RX_TX	PCM_1_RX_TX interrupt disable
2	W	PCM_1_ERROR	PCM_1_ERROR interrupt disable
1	W	PCM_0_RX_TX	PCM_0_RX_TX interrupt disable
0	W	PCM_0_ERROR	PCM_0_ERROR interrupt disable

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
ASRC_IDLE	INT5_DIS_ASRC_IDLE	Disable ASRC_IDLE interrupt	0x1
ASRC_OUT	INT5_DIS_ASRC_OUT	Disable ASRC_OUT interrupt	0x1
ASRC_IN	INT5_DIS_ASRC_IN	Disable ASRC_IN interrupt	0x1
ASRC_ERROR	INT5_DIS_ASRC_ERROR	Disable ASRC_ERROR interrupt	0x1
PCM_1_RX_TX	INT5_DIS_PCM_1_RX_TX	Disable PCM_1_RX_TX interrupt	0x1
PCM_1_ERROR	INT5_DIS_PCM_1_ERROR	Disable PCM_1_ERROR interrupt	0x1
PCM_0_RX_TX	INT5_DIS_PCM_0_RX_TX	Disable PCM_0_RX_TX interrupt	0x1
PCM_0_ERROR	INT5_DIS_PCM_0_ERROR	Disable PCM_0_ERROR interrupt	0x1

3.2.1.28 D_INT_DIS_6, INT_DIS_6

Bit Field	Read/Write	Field Name	Description
7	W	RS_DEC_OUT_RDY	RS_DEC_OUT_RDY interrupt disable
6	W	RS_DEC_IN_RDY	RS_DEC_IN_RDY interrupt disable
5	W	RS_ENC_OUT_RDY	RS_ENC_OUT_RDY interrupt disable
4	W	RS_ENC_IN_RDY	RS_ENC_IN_RDY interrupt disable
3	W	G722_DEC_OUT_RDY	G722_DEC_OUT_RDY interrupt disable
2	W	G722_DEC_IN_RDY	G722_DEC_IN_RDY interrupt disable
1	W	G722_ENC_OUT_RDY	G722_ENC_OUT_RDY interrupt disable
0	W	G722_ENC_IN_RDY	G722_ENC_IN_RDY interrupt disable

Field Name	Value Symbol	Value Description	Hex Value
RS_DEC_OUT_RDY	INT6_DIS_RS_DEC_OUT_RDY	Disable RS_DEC_OUT_RDY interrupt	0x1
RS_DEC_IN_RDY	INT6_DIS_RS_DEC_IN_RDY	Disable RS_DEC_IN_RDY interrupt	0x1
RS_ENC_OUT_RDY	INT6_DIS_RS_ENC_OUT_RDY	Disable RS_ENC_OUT_RDY interrupt	0x1
RS_ENC_IN_RDY	INT6_DIS_RS_ENC_IN_RDY	Disable RS_ENC_IN_RDY interrupt	0x1
G722_DEC_OUT_RDY	INT6_DIS_G722_DEC_OUT_RDY	Disable G722_DEC_OUT_RDY interrupt	0x1
G722_DEC_IN_RDY	INT6_DIS_G722_DEC_IN_RDY	Disable G722_DEC_IN_RDY interrupt	0x1
G722_ENC_OUT_RDY	INT6_DIS_G722_ENC_OUT_RDY	Disable G722_ENC_OUT_RDY interrupt	0x1
G722_ENC_IN_RDY	INT6_DIS_G722_ENC_IN_RDY	Disable G722_ENC_IN_RDY interrupt	0x1

Ezairo 8300 Hardware Reference

3.2.1.29 D_INT_DIS_7, INT_DIS_7

Bit Field	Read/Write	Field Name	Description
7	W	FIFO_7	FIFO_7 interrupt disable
6	W	FIFO_6	FIFO_6 interrupt disable
5	W	FIFO_5	FIFO_5 interrupt disable
4	W	FIFO_4	FIFO_4 interrupt disable
3	W	FIFO_3	FIFO_3 interrupt disable
2	W	FIFO_2	FIFO_2 interrupt disable
1	W	FIFO_1	FIFO_1 interrupt disable
0	W	FIFO_0	FIFO_0 interrupt disable

Field Name	Value Symbol	Value Description	Hex Value
FIFO_7	INT7_DIS_FIFO_7	Disable FIFO_7 interrupt	0x1
FIFO_6	INT7_DIS_FIFO_6	Disable FIFO_6 interrupt	0x1
FIFO_5	INT7_DIS_FIFO_5	Disable FIFO_5 interrupt	0x1
FIFO_4	INT7_DIS_FIFO_4	Disable FIFO_4 interrupt	0x1
FIFO_3	INT7_DIS_FIFO_3	Disable FIFO_3 interrupt	0x1
FIFO_2	INT7_DIS_FIFO_2	Disable FIFO_2 interrupt	0x1
FIFO_1	INT7_DIS_FIFO_1	Disable FIFO_1 interrupt	0x1
FIFO_0	INT7_DIS_FIFO_0	Disable FIFO_0 interrupt	0x1

3.2.1.30 D_INT_DIS_8, INT_DIS_8

Bit Field	Read/Write	Field Name	Description
7	W	HEAR_7	HEAR_7 interrupt disable
6	W	HEAR_6	HEAR_6 interrupt disable
5	W	HEAR_5	HEAR_5 interrupt disable
4	W	HEAR_4	HEAR_4 interrupt disable
3	W	HEAR_3	HEAR_3 interrupt disable
2	W	HEAR_2	HEAR_2 interrupt disable
1	W	HEAR_1	HEAR_1 interrupt disable
0	W	HEAR_0	HEAR_0 interrupt disable

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
HEAR_7	INT8_DIS_HEAR_7	Disable HEAR_7 interrupt	0x1
HEAR_6	INT8_DIS_HEAR_6	Disable HEAR_6 interrupt	0x1
HEAR_5	INT8_DIS_HEAR_5	Disable HEAR_5 interrupt	0x1
HEAR_4	INT8_DIS_HEAR_4	Disable HEAR_4 interrupt	0x1
HEAR_3	INT8_DIS_HEAR_3	Disable HEAR_3 interrupt	0x1
HEAR_2	INT8_DIS_HEAR_2	Disable HEAR_2 interrupt	0x1
HEAR_1	INT8_DIS_HEAR_1	Disable HEAR_1 interrupt	0x1
HEAR_0	INT8_DIS_HEAR_0	Disable HEAR_0 interrupt	0x1

3.2.1.31 D_INT_DIS_9, INT_DIS_9

Bit Field	Read/Write	Field Name	Description
7	W	DSP_7	DSP_7 interrupt disable
6	W	DSP_6	DSP_6 interrupt disable
5	W	DSP_5	DSP_5 interrupt disable
4	W	DSP_4	DSP_4 interrupt disable
3	W	DSP_3	DSP_3 interrupt disable
2	W	DSP_2	DSP_2 interrupt disable
1	W	DSP_1	DSP_1 interrupt disable
0	W	DSP_0	DSP_0 interrupt disable

Field Name	Value Symbol	Value Description	Hex Value
DSP_7	INT9_DIS_DSP_7	Disable DSP_7 interrupt	0x1
DSP_6	INT9_DIS_DSP_6	Disable DSP_6 interrupt	0x1
DSP_5	INT9_DIS_DSP_5	Disable DSP_5 interrupt	0x1
DSP_4	INT9_DIS_DSP_4	Disable DSP_4 interrupt	0x1
DSP_3	INT9_DIS_DSP_3	Disable DSP_3 interrupt	0x1
DSP_2	INT9_DIS_DSP_2	Disable DSP_2 interrupt	0x1
DSP_1	INT9_DIS_DSP_1	Disable DSP_1 interrupt	0x1
DSP_0	INT9_DIS_DSP_0	Disable DSP_0 interrupt	0x1

Ezairo 8300 Hardware Reference

3.2.1.32 D_INT_DIS_10, INT_DIS_10

Bit Field	Read/Write	Field Name	Description
7	W	CM3_7	CM3_7 interrupt disable
6	W	CM3_6	CM3_6 interrupt disable
5	W	CM3_5	CM3_5 interrupt disable
4	W	CM3_4	CM3_4 interrupt disable
3	W	CM3_3	CM3_3 interrupt disable
2	W	CM3_2	CM3_2 interrupt disable
1	W	CM3_1	CM3_1 interrupt disable
0	W	CM3_0	CM3_0 interrupt disable

Field Name	Value Symbol	Value Description	Hex Value
CM3_7	INT10_DIS_CM3_7	Disable CM3_7 interrupt	0x1
CM3_6	INT10_DIS_CM3_6	Disable CM3_6 interrupt	0x1
CM3_5	INT10_DIS_CM3_5	Disable CM3_5 interrupt	0x1
CM3_4	INT10_DIS_CM3_4	Disable CM3_4 interrupt	0x1
CM3_3	INT10_DIS_CM3_3	Disable CM3_3 interrupt	0x1
CM3_2	INT10_DIS_CM3_2	Disable CM3_2 interrupt	0x1
CM3_1	INT10_DIS_CM3_1	Disable CM3_1 interrupt	0x1
CM3_0	INT10_DIS_CM3_0	Disable CM3_0 interrupt	0x1

3.2.1.33 D_INT_DIS_11, INT_DIS_11

Bit Field	Read/Write	Field Name	Description
5	W	RS_COM	RS_COM interrupt disable
4	W	G722_COM	G722_COM interrupt disable
3	W	MEM_ERROR	MEM_ERROR interrupt disable
2	W	LSAD	LSAD interrupt disable
1	W	CLK_DET	CLK_DET interrupt disable
0	W	NNA	NNA interrupt disable

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
RS_COM	INT11_DIS_RS_COM	Disable RS_COM interrupt	0x1
G722_COM	INT11_DIS_G722_COM	Disable G722_COM interrupt	0x1
MEM_ERROR	INT11_DIS_MEM_ERROR	Disable MEM_ERROR interrupt	0x1
LSAD	INT11_DIS_LSAD	Disable LSAD interrupt	0x1
CLK_DET	INT11_DIS_CLK_DET	Disable CLK_DET interrupt	0x1
NNA	INT11_DIS_NNA	Disable NNA interrupt	0x1

3.2.1.34 D_INT_ACK_0, INT_ACK_0

Bit Field	Read/Write	Field Name	Description
7	W	TIMER_3	TIMER_3 interrupt acknowledge
6	W	TIMER_2	TIMER_2 interrupt acknowledge
5	W	TIMER_1	TIMER_1 interrupt acknowledge
4	W	TIMER_0	TIMER_0 interrupt acknowledge
3	W	AUDIOSINK_1	AUDIOSINK_1 interrupt acknowledge
2	W	AUDIOSINK_0	AUDIOSINK_0 interrupt acknowledge
1	W	LSAD_BATMON	LSAD_BATMON interrupt acknowledge
0	W	WATCHDOG	WATCHDOG interrupt acknowledge

Field Name	Value Symbol	Value Description	Hex Value
TIMER_3	INT0_ACK_TIMER_3	Acknowledge TIMER_3 interrupt	0x1
TIMER_2	INT0_ACK_TIMER_2	Acknowledge TIMER_2 interrupt	0x1
TIMER_1	INT0_ACK_TIMER_1	Acknowledge TIMER_1 interrupt	0x1
TIMER_0	INT0_ACK_TIMER_0	Acknowledge TIMER_0 interrupt	0x1
AUDIOSINK_1	INT0_ACK_AUDIOSINK_1	Acknowledge AUDIOSINK_1 interrupt	0x1
AUDIOSINK_0	INT0_ACK_AUDIOSINK_0	Acknowledge AUDIOSINK_0 interrupt	0x1
LSAD_BATMON	INT0_ACK_LSAD_BATMON	Acknowledge LSAD_BATMON interrupt	0x1
WATCHDOG	INT0_ACK_WATCHDOG	Acknowledge WATCHDOG interrupt	0x1

Ezairo 8300 Hardware Reference

3.2.1.35 D_INT_ACK_1, INT_ACK_1

Bit Field	Read/Write	Field Name	Description
7	W	DMA_7	DMA_7 interrupt acknowledge
6	W	DMA_6	DMA_6 interrupt acknowledge
5	W	DMA_5	DMA_5 interrupt acknowledge
4	W	DMA_4	DMA_4 interrupt acknowledge
3	W	DMA_3	DMA_3 interrupt acknowledge
2	W	DMA_2	DMA_2 interrupt acknowledge
1	W	DMA_1	DMA_1 interrupt acknowledge
0	W	DMA_0	DMA_0 interrupt acknowledge

Field Name	Value Symbol	Value Description	Hex Value
DMA_7	INT1_ACK_DMA_7	Acknowledge DMA_7 interrupt	0x1
DMA_6	INT1_ACK_DMA_6	Acknowledge DMA_6 interrupt	0x1
DMA_5	INT1_ACK_DMA_5	Acknowledge DMA_5 interrupt	0x1
DMA_4	INT1_ACK_DMA_4	Acknowledge DMA_4 interrupt	0x1
DMA_3	INT1_ACK_DMA_3	Acknowledge DMA_3 interrupt	0x1
DMA_2	INT1_ACK_DMA_2	Acknowledge DMA_2 interrupt	0x1
DMA_1	INT1_ACK_DMA_1	Acknowledge DMA_1 interrupt	0x1
DMA_0	INT1_ACK_DMA_0	Acknowledge DMA_0 interrupt	0x1

3.2.1.36 D_INT_ACK_2, INT_ACK_2

Bit Field	Read/Write	Field Name	Description
7	W	DIO_7	DIO_7 interrupt acknowledge
6	W	DIO_6	DIO_6 interrupt acknowledge
5	W	DIO_5	DIO_5 interrupt acknowledge
4	W	DIO_4	DIO_4 interrupt acknowledge
3	W	DIO_3	DIO_3 interrupt acknowledge
2	W	DIO_2	DIO_2 interrupt acknowledge
1	W	DIO_1	DIO_1 interrupt acknowledge
0	W	DIO_0	DIO_0 interrupt acknowledge

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DIO_7	INT2_ACK_DIO_7	Acknowledge DIO_7 interrupt	0x1
DIO_6	INT2_ACK_DIO_6	Acknowledge DIO_6 interrupt	0x1
DIO_5	INT2_ACK_DIO_5	Acknowledge DIO_5 interrupt	0x1
DIO_4	INT2_ACK_DIO_4	Acknowledge DIO_4 interrupt	0x1
DIO_3	INT2_ACK_DIO_3	Acknowledge DIO_3 interrupt	0x1
DIO_2	INT2_ACK_DIO_2	Acknowledge DIO_2 interrupt	0x1
DIO_1	INT2_ACK_DIO_1	Acknowledge DIO_1 interrupt	0x1
DIO_0	INT2_ACK_DIO_0	Acknowledge DIO_0 interrupt	0x1

3.2.1.37 D_INT_ACK_3, INT_ACK_3

Bit Field	Read/Write	Field Name	Description
7	W	SPI_0_TX	SPI_0_TX interrupt acknowledge
6	W	SPI_0_RX	SPI_0_RX interrupt acknowledge
5	W	SPI_0_COM	SPI_0_COM interrupt acknowledge
4	W	D_I2C	D_I2C interrupt acknowledge
3	W	I2C_1	I2C_1 interrupt acknowledge
2	W	I2C_0	I2C_0 interrupt acknowledge
1	W	I3C_1	I3C_1 interrupt acknowledge
0	W	I3C_0	I3C_0 interrupt acknowledge

Field Name	Value Symbol	Value Description	Hex Value
SPI_0_TX	INT3_ACK_SPI_0_TX	Acknowledge SPI_0_TX interrupt	0x1
SPI_0_RX	INT3_ACK_SPI_0_RX	Acknowledge SPI_0_RX interrupt	0x1
SPI_0_COM	INT3_ACK_SPI_0_COM	Acknowledge SPI_0_COM interrupt	0x1
D_I2C	INT3_ACK_D_I2C	Acknowledge D_I2C interrupt	0x1
I2C_1	INT3_ACK_I2C_1	Acknowledge I2C_1 interrupt	0x1
I2C_0	INT3_ACK_I2C_0	Acknowledge I2C_0 interrupt	0x1
I3C_1	INT3_ACK_I3C_1	Acknowledge I3C_1 interrupt	0x1
I3C_0	INT3_ACK_I3C_0	Acknowledge I3C_0 interrupt	0x1

Ezairo 8300 Hardware Reference

3.2.1.38 D_INT_ACK_4, INT_ACK_4

Bit Field	Read/Write	Field Name	Description
7	W	FENG	FENG interrupt acknowledge
6	W	UART_0_TX	UART_0_TX interrupt acknowledge
5	W	UART_0_RX	UART_0_RX interrupt acknowledge
4	W	UART_0_ERROR	UART_0_ERROR interrupt acknowledge
3	W	EMMC	EMMC interrupt acknowledge
2	W	SPI_1_TX	SPI_1_TX interrupt acknowledge
1	W	SPI_1_RX	SPI_1_RX interrupt acknowledge
0	W	SPI_1_COM	SPI_1_COM interrupt acknowledge

Field Name	Value Symbol	Value Description	Hex Value
FENG	INT4_ACK_FENG	Acknowledge FENG interrupt	0x1
UART_0_TX	INT4_ACK_UART_0_TX	Acknowledge UART_0_TX interrupt	0x1
UART_0_RX	INT4_ACK_UART_0_RX	Acknowledge UART_0_RX interrupt	0x1
UART_0_ERROR	INT4_ACK_UART_0_ERROR	Acknowledge UART_0_ERROR interrupt	0x1
EMMC	INT4_ACK_EMMC	Acknowledge EMMC interrupt	0x1
SPI_1_TX	INT4_ACK_SPI_1_TX	Acknowledge SPI_1_TX interrupt	0x1
SPI_1_RX	INT4_ACK_SPI_1_RX	Acknowledge SPI_1_RX interrupt	0x1
SPI_1_COM	INT4_ACK_SPI_1_COM	Acknowledge SPI_1_COM interrupt	0x1

3.2.1.39 D_INT_ACK_5, INT_ACK_5

Bit Field	Read/Write	Field Name	Description
7	W	ASRC_IDLE	ASRC_IDLE interrupt acknowledge
6	W	ASRC_OUT	ASRC_OUT interrupt acknowledge
5	W	ASRC_IN	ASRC_IN interrupt acknowledge
4	W	ASRC_ERROR	ASRC_ERROR interrupt acknowledge
3	W	PCM_1_RX_TX	PCM_1_RX_TX interrupt acknowledge
2	W	PCM_1_ERROR	PCM_1_ERROR interrupt acknowledge
1	W	PCM_0_RX_TX	PCM_0_RX_TX interrupt acknowledge
0	W	PCM_0_ERROR	PCM_0_ERROR interrupt acknowledge

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
ASRC_IDLE	INT5_ACK_ASRC_IDLE	Acknowledge ASRC_IDLE interrupt	0x1
ASRC_OUT	INT5_ACK_ASRC_OUT	Acknowledge ASRC_OUT interrupt	0x1
ASRC_IN	INT5_ACK_ASRC_IN	Acknowledge ASRC_IN interrupt	0x1
ASRC_ERROR	INT5_ACK_ASRC_ERROR	Acknowledge ASRC_ERROR interrupt	0x1
PCM_1_RX_TX	INT5_ACK_PCM_1_RX_TX	Acknowledge PCM_1_RX_TX interrupt	0x1
PCM_1_ERROR	INT5_ACK_PCM_1_ERROR	Acknowledge PCM_1_ERROR interrupt	0x1
PCM_0_RX_TX	INT5_ACK_PCM_0_RX_TX	Acknowledge PCM_0_RX_TX interrupt	0x1
PCM_0_ERROR	INT5_ACK_PCM_0_ERROR	Acknowledge PCM_0_ERROR interrupt	0x1

3.2.1.40 D_INT_ACK_6, INT_ACK_6

Bit Field	Read/Write	Field Name	Description
7	W	RS_DEC_OUT_RDY	RS_DEC_OUT_RDY interrupt acknowledge
6	W	RS_DEC_IN_RDY	RS_DEC_IN_RDY interrupt acknowledge
5	W	RS_ENC_OUT_RDY	RS_ENC_OUT_RDY interrupt acknowledge
4	W	RS_ENC_IN_RDY	RS_ENC_IN_RDY interrupt acknowledge
3	W	G722_DEC_OUT_RDY	G722_DEC_OUT_RDY interrupt acknowledge
2	W	G722_DEC_IN_RDY	G722_DEC_IN_RDY interrupt acknowledge
1	W	G722_ENC_OUT_RDY	G722_ENC_OUT_RDY interrupt acknowledge
0	W	G722_ENC_IN_RDY	G722_ENC_IN_RDY interrupt acknowledge

Field Name	Value Symbol	Value Description	Hex Value
RS_DEC_OUT_RDY	INT6_ACK_RS_DEC_OUT_RDY	Acknowledge RS_DEC_OUT_RDY interrupt	0x1
RS_DEC_IN_RDY	INT6_ACK_RS_DEC_IN_RDY	Acknowledge RS_DEC_IN_RDY interrupt	0x1
RS_ENC_OUT_RDY	INT6_ACK_RS_ENC_OUT_RDY	Acknowledge RS_ENC_OUT_RDY interrupt	0x1
RS_ENC_IN_RDY	INT6_ACK_RS_ENC_IN_RDY	Acknowledge RS_ENC_IN_RDY interrupt	0x1
G722_DEC_OUT_RDY	INT6_ACK_G722_DEC_OUT_RDY	Acknowledge G722_DEC_OUT_RDY interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
G722_DEC_IN_RDY	INT6_ACK_G722_DEC_IN_RDY	Acknowledge G722_DEC_IN_RDY interrupt	0x1
G722_ENC_OUT_RDY	INT6_ACK_G722_ENC_OUT_RDY	Acknowledge G722_ENC_OUT_RDY interrupt	0x1
G722_ENC_IN_RDY	INT6_ACK_G722_ENC_IN_RDY	Acknowledge G722_ENC_IN_RDY interrupt	0x1

3.2.1.41 D_INT_ACK_7, INT_ACK_7

Bit Field	Read/Write	Field Name	Description
7	W	FIFO_7	FIFO_7 interrupt acknowledge
6	W	FIFO_6	FIFO_6 interrupt acknowledge
5	W	FIFO_5	FIFO_5 interrupt acknowledge
4	W	FIFO_4	FIFO_4 interrupt acknowledge
3	W	FIFO_3	FIFO_3 interrupt acknowledge
2	W	FIFO_2	FIFO_2 interrupt acknowledge
1	W	FIFO_1	FIFO_1 interrupt acknowledge
0	W	FIFO_0	FIFO_0 interrupt acknowledge

Field Name	Value Symbol	Value Description	Hex Value
FIFO_7	INT7_ACK_FIFO_7	Acknowledge FIFO_7 interrupt	0x1
FIFO_6	INT7_ACK_FIFO_6	Acknowledge FIFO_6 interrupt	0x1
FIFO_5	INT7_ACK_FIFO_5	Acknowledge FIFO_5 interrupt	0x1
FIFO_4	INT7_ACK_FIFO_4	Acknowledge FIFO_4 interrupt	0x1
FIFO_3	INT7_ACK_FIFO_3	Acknowledge FIFO_3 interrupt	0x1
FIFO_2	INT7_ACK_FIFO_2	Acknowledge FIFO_2 interrupt	0x1
FIFO_1	INT7_ACK_FIFO_1	Acknowledge FIFO_1 interrupt	0x1
FIFO_0	INT7_ACK_FIFO_0	Acknowledge FIFO_0 interrupt	0x1

Ezairo 8300 Hardware Reference

3.2.1.42 D_INT_ACK_8, INT_ACK_8

Bit Field	Read/Write	Field Name	Description
7	W	HEAR_7	HEAR_7 interrupt acknowledge
6	W	HEAR_6	HEAR_6 interrupt acknowledge
5	W	HEAR_5	HEAR_5 interrupt acknowledge
4	W	HEAR_4	HEAR_4 interrupt acknowledge
3	W	HEAR_3	HEAR_3 interrupt acknowledge
2	W	HEAR_2	HEAR_2 interrupt acknowledge
1	W	HEAR_1	HEAR_1 interrupt acknowledge
0	W	HEAR_0	HEAR_0 interrupt acknowledge

Field Name	Value Symbol	Value Description	Hex Value
HEAR_7	INT8_ACK_HEAR_7	Acknowledge HEAR_7 interrupt	0x1
HEAR_6	INT8_ACK_HEAR_6	Acknowledge HEAR_6 interrupt	0x1
HEAR_5	INT8_ACK_HEAR_5	Acknowledge HEAR_5 interrupt	0x1
HEAR_4	INT8_ACK_HEAR_4	Acknowledge HEAR_4 interrupt	0x1
HEAR_3	INT8_ACK_HEAR_3	Acknowledge HEAR_3 interrupt	0x1
HEAR_2	INT8_ACK_HEAR_2	Acknowledge HEAR_2 interrupt	0x1
HEAR_1	INT8_ACK_HEAR_1	Acknowledge HEAR_1 interrupt	0x1
HEAR_0	INT8_ACK_HEAR_0	Acknowledge HEAR_0 interrupt	0x1

3.2.1.43 D_INT_ACK_9, INT_ACK_9

Bit Field	Read/Write	Field Name	Description
7	W	DSP_7	DSP_7 interrupt acknowledge
6	W	DSP_6	DSP_6 interrupt acknowledge
5	W	DSP_5	DSP_5 interrupt acknowledge
4	W	DSP_4	DSP_4 interrupt acknowledge
3	W	DSP_3	DSP_3 interrupt acknowledge
2	W	DSP_2	DSP_2 interrupt acknowledge
1	W	DSP_1	DSP_1 interrupt acknowledge
0	W	DSP_0	DSP_0 interrupt acknowledge

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DSP_7	INT9_ACK_DSP_7	Acknowledge DSP_7 interrupt	0x1
DSP_6	INT9_ACK_DSP_6	Acknowledge DSP_6 interrupt	0x1
DSP_5	INT9_ACK_DSP_5	Acknowledge DSP_5 interrupt	0x1
DSP_4	INT9_ACK_DSP_4	Acknowledge DSP_4 interrupt	0x1
DSP_3	INT9_ACK_DSP_3	Acknowledge DSP_3 interrupt	0x1
DSP_2	INT9_ACK_DSP_2	Acknowledge DSP_2 interrupt	0x1
DSP_1	INT9_ACK_DSP_1	Acknowledge DSP_1 interrupt	0x1
DSP_0	INT9_ACK_DSP_0	Acknowledge DSP_0 interrupt	0x1

3.2.1.44 D_INT_ACK_10, INT_ACK_10

Bit Field	Read/Write	Field Name	Description
7	W	CM3_7	CM3_7 interrupt acknowledge
6	W	CM3_6	CM3_6 interrupt acknowledge
5	W	CM3_5	CM3_5 interrupt acknowledge
4	W	CM3_4	CM3_4 interrupt acknowledge
3	W	CM3_3	CM3_3 interrupt acknowledge
2	W	CM3_2	CM3_2 interrupt acknowledge
1	W	CM3_1	CM3_1 interrupt acknowledge
0	W	CM3_0	CM3_0 interrupt acknowledge

Field Name	Value Symbol	Value Description	Hex Value
CM3_7	INT10_ACK_CM3_7	Acknowledge CM3_7 interrupt	0x1
CM3_6	INT10_ACK_CM3_6	Acknowledge CM3_6 interrupt	0x1
CM3_5	INT10_ACK_CM3_5	Acknowledge CM3_5 interrupt	0x1
CM3_4	INT10_ACK_CM3_4	Acknowledge CM3_4 interrupt	0x1
CM3_3	INT10_ACK_CM3_3	Acknowledge CM3_3 interrupt	0x1
CM3_2	INT10_ACK_CM3_2	Acknowledge CM3_2 interrupt	0x1
CM3_1	INT10_ACK_CM3_1	Acknowledge CM3_1 interrupt	0x1
CM3_0	INT10_ACK_CM3_0	Acknowledge CM3_0 interrupt	0x1

Ezairo 8300 Hardware Reference

3.2.1.45 D_INT_ACK_11, INT_ACK_11

Bit Field	Read/Write	Field Name	Description
5	W	RS_COM	RS_COM interrupt acknowledge
4	W	G722_COM	G722_COM interrupt acknowledge
3	W	MEM_ERROR	MEM_ERROR interrupt acknowledge
2	W	LSAD	LSAD interrupt acknowledge
1	W	CLK_DET	CLK_DET interrupt acknowledge
0	W	NNA	NNA interrupt acknowledge

Field Name	Value Symbol	Value Description	Hex Value
RS_COM	INT11_ACK_RS_COM	Acknowledge RS_COM interrupt	0x1
G722_COM	INT11_ACK_G722_COM	Acknowledge G722_COM interrupt	0x1
MEM_ERROR	INT11_ACK_MEM_ERROR	Acknowledge MEM_ERROR interrupt	0x1
LSAD	INT11_ACK_LSAD	Acknowledge LSAD interrupt	0x1
CLK_DET	INT11_ACK_CLK_DET	Acknowledge CLK_DET interrupt	0x1
NNA	INT11_ACK_NNA	Acknowledge NNA interrupt	0x1

3.2.1.46 D_INT_PRIO_0, INT_PRIO_0

Bit Field	Read/Write	Field Name	Description
7	RW	TIMER_3	TIMER_3 interrupt priority
6	RW	TIMER_2	TIMER_2 interrupt priority
5	RW	TIMER_1	TIMER_1 interrupt priority
4	RW	TIMER_0	TIMER_0 interrupt priority
3	RW	AUDIOSINK_1	AUDIOSINK_1 interrupt priority
2	RW	AUDIOSINK_0	AUDIOSINK_0 interrupt priority
1	RW	LSAD_BATMON	LSAD_BATMON interrupt priority
0	RW	WATCHDOG	WATCHDOG interrupt priority

Field Name	Value Symbol	Value Description	Hex Value
TIMER_3	INT0_NORMALPRIO_TIMER_3	TIMER_3 has normal priority	0x0*
	INT0_HIGHPRIO_TIMER_3	TIMER_3 has high priority	0x1
TIMER_2	INT0_NORMALPRIO_TIMER_2	TIMER_2 has normal priority	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	INT0_HIGHPRIO_TIMER_2	TIMER_2 has high priority	0x1
TIMER_1	INT0_NORMALPRIO_TIMER_1	TIMER_1 has normal priority	0x0*
	INT0_HIGHPRIO_TIMER_1	TIMER_1 has high priority	0x1
TIMER_0	INT0_NORMALPRIO_TIMER_0	TIMER_0 has normal priority	0x0*
	INT0_HIGHPRIO_TIMER_0	TIMER_0 has high priority	0x1
AUDIOSINK_1	INT0_NORMALPRIO_AUDIOSINK_1	AUDIOSINK_1 has normal priority	0x0*
	INT0_HIGHPRIO_AUDIOSINK_1	AUDIOSINK_1 has high priority	0x1
AUDIOSINK_0	INT0_NORMALPRIO_AUDIOSINK_0	AUDIOSINK_0 has normal priority	0x0*
	INT0_HIGHPRIO_AUDIOSINK_0	AUDIOSINK_0 has high priority	0x1
LSAD_BATMON	INT0_NORMALPRIO_LSAD_BATMON	LSAD_BATMON has normal priority	0x0*
	INT0_HIGHPRIO_LSAD_BATMON	LSAD_BATMON has high priority	0x1
WATCHDOG	INT0_NORMALPRIO_WATCHDOG	WATCHDOG has normal priority	0x0*
	INT0_HIGHPRIO_WATCHDOG	WATCHDOG has high priority	0x1

3.2.1.47 D_INT_PRIO_1, INT_PRIO_1

Bit Field	Read/Write	Field Name	Description
7	RW	DMA_7	DMA_7 interrupt priority
6	RW	DMA_6	DMA_6 interrupt priority
5	RW	DMA_5	DMA_5 interrupt priority
4	RW	DMA_4	DMA_4 interrupt priority
3	RW	DMA_3	DMA_3 interrupt priority
2	RW	DMA_2	DMA_2 interrupt priority
1	RW	DMA_1	DMA_1 interrupt priority
0	RW	DMA_0	DMA_0 interrupt priority

Field Name	Value Symbol	Value Description	Hex Value
DMA_7	INT1_NORMALPRIO_DMA_7	DMA_7 has normal priority	0x0*
	INT1_HIGHPRIO_DMA_7	DMA_7 has high priority	0x1
DMA_6	INT1_NORMALPRIO_DMA_6	DMA_6 has normal priority	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	INT1_HIGHPRIO_DMA_6	DMA_6 has high priority	0x1
DMA_5	INT1_NORMALPRIO_DMA_5	DMA_5 has normal priority	0x0*
	INT1_HIGHPRIO_DMA_5	DMA_5 has high priority	0x1
DMA_4	INT1_NORMALPRIO_DMA_4	DMA_4 has normal priority	0x0*
	INT1_HIGHPRIO_DMA_4	DMA_4 has high priority	0x1
DMA_3	INT1_NORMALPRIO_DMA_3	DMA_3 has normal priority	0x0*
	INT1_HIGHPRIO_DMA_3	DMA_3 has high priority	0x1
DMA_2	INT1_NORMALPRIO_DMA_2	DMA_2 has normal priority	0x0*
	INT1_HIGHPRIO_DMA_2	DMA_2 has high priority	0x1
DMA_1	INT1_NORMALPRIO_DMA_1	DMA_1 has normal priority	0x0*
	INT1_HIGHPRIO_DMA_1	DMA_1 has high priority	0x1
DMA_0	INT1_NORMALPRIO_DMA_0	DMA_0 has normal priority	0x0*
	INT1_HIGHPRIO_DMA_0	DMA_0 has high priority	0x1

3.2.1.48 D_INT_PRIO_2, INT_PRIO_2

Bit Field	Read/Write	Field Name	Description
7	RW	DIO_7	DIO_7 interrupt priority
6	RW	DIO_6	DIO_6 interrupt priority
5	RW	DIO_5	DIO_5 interrupt priority
4	RW	DIO_4	DIO_4 interrupt priority
3	RW	DIO_3	DIO_3 interrupt priority
2	RW	DIO_2	DIO_2 interrupt priority
1	RW	DIO_1	DIO_1 interrupt priority
0	RW	DIO_0	DIO_0 interrupt priority

Field Name	Value Symbol	Value Description	Hex Value
DIO_7	INT2_NORMALPRIO_DIO_7	DIO_7 has normal priority	0x0*
	INT2_HIGHPRIO_DIO_7	DIO_7 has high priority	0x1
DIO_6	INT2_NORMALPRIO_DIO_6	DIO_6 has normal priority	0x0*
	INT2_HIGHPRIO_DIO_6	DIO_6 has high priority	0x1
DIO_5	INT2_NORMALPRIO_DIO_5	DIO_5 has normal priority	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	INT2_HIGHPRIO_DIO_5	DIO_5 has high priority	0x1
DIO_4	INT2_NORMALPRIO_DIO_4	DIO_4 has normal priority	0x0*
	INT2_HIGHPRIO_DIO_4	DIO_4 has high priority	0x1
DIO_3	INT2_NORMALPRIO_DIO_3	DIO_3 has normal priority	0x0*
	INT2_HIGHPRIO_DIO_3	DIO_3 has high priority	0x1
DIO_2	INT2_NORMALPRIO_DIO_2	DIO_2 has normal priority	0x0*
	INT2_HIGHPRIO_DIO_2	DIO_2 has high priority	0x1
DIO_1	INT2_NORMALPRIO_DIO_1	DIO_1 has normal priority	0x0*
	INT2_HIGHPRIO_DIO_1	DIO_1 has high priority	0x1
DIO_0	INT2_NORMALPRIO_DIO_0	DIO_0 has normal priority	0x0*
	INT2_HIGHPRIO_DIO_0	DIO_0 has high priority	0x1

3.2.1.49 D_INT_PRIO_3, INT_PRIO_3

Bit Field	Read/Write	Field Name	Description
7	RW	SPI_0_TX	SPI_0_TX interrupt priority
6	RW	SPI_0_RX	SPI_0_RX interrupt priority
5	RW	SPI_0_COM	SPI_0_COM interrupt priority
4	RW	D_I2C	D_I2C interrupt priority
3	RW	I2C_1	I2C_1 interrupt priority
2	RW	I2C_0	I2C_0 interrupt priority
1	RW	I3C_1	I3C_1 interrupt priority
0	RW	I3C_0	I3C_0 interrupt priority

Field Name	Value Symbol	Value Description	Hex Value
SPI_0_TX	INT3_NORMALPRIO_SPI_0_TX	SPI_0_TX has normal priority	0x0*
	INT3_HIGHPRIO_SPI_0_TX	SPI_0_TX has high priority	0x1
SPI_0_RX	INT3_NORMALPRIO_SPI_0_RX	SPI_0_RX has normal priority	0x0*
	INT3_HIGHPRIO_SPI_0_RX	SPI_0_RX has high priority	0x1
SPI_0_COM	INT3_NORMALPRIO_SPI_0_COM	SPI_0_COM has normal priority	0x0*
	INT3_HIGHPRIO_SPI_0_COM	SPI_0_COM has high priority	0x1
D_I2C	INT3_NORMALPRIO_D_I2C	D_I2C has normal priority	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	INT3_HIGHPRIO_D_I2C	D_I2C has high priority	0x1
I2C_1	INT3_NORMALPRIO_I2C_1	I2C_1 has normal priority	0x0*
	INT3_HIGHPRIO_I2C_1	I2C_1 has high priority	0x1
I2C_0	INT3_NORMALPRIO_I2C_0	I2C_0 has normal priority	0x0*
	INT3_HIGHPRIO_I2C_0	I2C_0 has high priority	0x1
I3C_1	INT3_NORMALPRIO_I3C_1	I3C_1 has normal priority	0x0*
	INT3_HIGHPRIO_I3C_1	I3C_1 has high priority	0x1
I3C_0	INT3_NORMALPRIO_I3C_0	I3C_0 has normal priority	0x0*
	INT3_HIGHPRIO_I3C_0	I3C_0 has high priority	0x1

3.2.1.50 D_INT_PRIO_4, INT_PRIO_4

Bit Field	Read/Write	Field Name	Description
7	RW	FENG	FENG interrupt priority
6	RW	UART_0_TX	UART_0_TX interrupt priority
5	RW	UART_0_RX	UART_0_RX interrupt priority
4	RW	UART_0_ERROR	UART_0_ERROR interrupt priority
3	RW	EMMC	EMMC interrupt priority
2	RW	SPI_1_TX	SPI_1_TX interrupt priority
1	RW	SPI_1_RX	SPI_1_RX interrupt priority
0	RW	SPI_1_COM	SPI_1_COM interrupt priority

Field Name	Value Symbol	Value Description	Hex Value
FENG	INT4_NORMALPRIO_FENG	FENG has normal priority	0x0*
	INT4_HIGHPRIO_FENG	FENG has high priority	0x1
UART_0_TX	INT4_NORMALPRIO_UART_0_TX	UART_0_TX has normal priority	0x0*
	INT4_HIGHPRIO_UART_0_TX	UART_0_TX has high priority	0x1
UART_0_RX	INT4_NORMALPRIO_UART_0_RX	UART_0_RX has normal priority	0x0*
	INT4_HIGHPRIO_UART_0_RX	UART_0_RX has high priority	0x1
UART_0_ERROR	INT4_NORMALPRIO_UART_0_ERROR	UART_0_ERROR has normal priority	0x0*
	INT4_HIGHPRIO_UART_0_ERROR	UART_0_ERROR has high priority	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
EMMC	INT4_NORMALPRIO_EMMC	EMMC has normal priority	0x0*
	INT4_HIGHPRIO_EMMC	EMMC has high priority	0x1
SPI_1_TX	INT4_NORMALPRIO_SPI_1_TX	SPI_1_TX has normal priority	0x0*
	INT4_HIGHPRIO_SPI_1_TX	SPI_1_TX has high priority	0x1
SPI_1_RX	INT4_NORMALPRIO_SPI_1_RX	SPI_1_RX has normal priority	0x0*
	INT4_HIGHPRIO_SPI_1_RX	SPI_1_RX has high priority	0x1
SPI_1_COM	INT4_NORMALPRIO_SPI_1_COM	SPI_1_COM has normal priority	0x0*
	INT4_HIGHPRIO_SPI_1_COM	SPI_1_COM has high priority	0x1

3.2.1.51 D_INT_PRIO_5, INT_PRIO_5

Bit Field	Read/Write	Field Name	Description
7	RW	ASRC_IDLE	ASRC_IDLE interrupt priority
6	RW	ASRC_OUT	ASRC_OUT interrupt priority
5	RW	ASRC_IN	ASRC_IN interrupt priority
4	RW	ASRC_ERROR	ASRC_ERROR interrupt priority
3	RW	PCM_1_RX_TX	PCM_1_RX_TX interrupt priority
2	RW	PCM_1_ERROR	PCM_1_ERROR interrupt priority
1	RW	PCM_0_RX_TX	PCM_0_RX_TX interrupt priority
0	RW	PCM_0_ERROR	PCM_0_ERROR interrupt priority

Field Name	Value Symbol	Value Description	Hex Value
ASRC_IDLE	INT5_NORMALPRIO_ASRC_IDLE	ASRC_IDLE has normal priority	0x0*
	INT5_HIGHPRIO_ASRC_IDLE	ASRC_IDLE has high priority	0x1
ASRC_OUT	INT5_NORMALPRIO_ASRC_OUT	ASRC_OUT has normal priority	0x0*
	INT5_HIGHPRIO_ASRC_OUT	ASRC_OUT has high priority	0x1
ASRC_IN	INT5_NORMALPRIO_ASRC_IN	ASRC_IN has normal priority	0x0*
	INT5_HIGHPRIO_ASRC_IN	ASRC_IN has high priority	0x1
ASRC_ERROR	INT5_NORMALPRIO_ASRC_ERROR	ASRC_ERROR has normal priority	0x0*
	INT5_HIGHPRIO_ASRC_ERROR	ASRC_ERROR has high priority	0x1
PCM_1_RX_TX	INT5_NORMALPRIO_PCM_1_RX_TX	PCM_1_RX_TX has normal priority	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	INT5_HIGHPRIO_PCM_1_RX_TX	PCM_1_RX_TX has high priority	0x1
PCM_1_ERROR	INT5_NORMALPRIO_PCM_1_ERROR	PCM_1_ERROR has normal priority	0x0*
	INT5_HIGHPRIO_PCM_1_ERROR	PCM_1_ERROR has high priority	0x1
PCM_0_RX_TX	INT5_NORMALPRIO_PCM_0_RX_TX	PCM_0_RX_TX has normal priority	0x0*
	INT5_HIGHPRIO_PCM_0_RX_TX	PCM_0_RX_TX has high priority	0x1
PCM_0_ERROR	INT5_NORMALPRIO_PCM_0_ERROR	PCM_0_ERROR has normal priority	0x0*
	INT5_HIGHPRIO_PCM_0_ERROR	PCM_0_ERROR has high priority	0x1

3.2.1.52 D_INT_PRIO_6, INT_PRIO_6

Bit Field	Read/Write	Field Name	Description
7	RW	RS_DEC_OUT_RDY	RS_DEC_OUT_RDY interrupt priority
6	RW	RS_DEC_IN_RDY	RS_DEC_IN_RDY interrupt priority
5	RW	RS_ENC_OUT_RDY	RS_ENC_OUT_RDY interrupt priority
4	RW	RS_ENC_IN_RDY	RS_ENC_IN_RDY interrupt priority
3	RW	G722_DEC_OUT_RDY	G722_DEC_OUT_RDY interrupt priority
2	RW	G722_DEC_IN_RDY	G722_DEC_IN_RDY interrupt priority
1	RW	G722_ENC_OUT_RDY	G722_ENC_OUT_RDY interrupt priority
0	RW	G722_ENC_IN_RDY	G722_ENC_IN_RDY interrupt priority

Field Name	Value Symbol	Value Description	Hex Value
RS_DEC_OUT_RDY	INT6_NORMALPRIO_RS_DEC_OUT_RDY	RS_DEC_OUT_RDY has normal priority	0x0*
	INT6_HIGHPRIO_RS_DEC_OUT_RDY	RS_DEC_OUT_RDY has high priority	0x1
RS_DEC_IN_RDY	INT6_NORMALPRIO_RS_DEC_IN_RDY	RS_DEC_IN_RDY has normal priority	0x0*
	INT6_HIGHPRIO_RS_DEC_IN_RDY	RS_DEC_IN_RDY has high priority	0x1
RS_ENC_OUT_RDY	INT6_NORMALPRIO_RS_ENC_OUT_RDY	RS_ENC_OUT_RDY has normal priority	0x0*
	INT6_HIGHPRIO_RS_ENC_OUT_RDY	RS_ENC_OUT_RDY has high priority	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
RS_ENC_IN_RDY	INT6_NORMALPRIO_RS_ENC_IN_RDY	RS_ENC_IN_RDY has normal priority	0x0*
	INT6_HIGHPRIO_RS_ENC_IN_RDY	RS_ENC_IN_RDY has high priority	0x1
G722_DEC_OUT_RDY	INT6_NORMALPRIO_G722_DEC_OUT_RDY	G722_DEC_OUT_RDY has normal priority	0x0*
	INT6_HIGHPRIO_G722_DEC_OUT_RDY	G722_DEC_OUT_RDY has high priority	0x1
G722_DEC_IN_RDY	INT6_NORMALPRIO_G722_DEC_IN_RDY	G722_DEC_IN_RDY has normal priority	0x0*
	INT6_HIGHPRIO_G722_DEC_IN_RDY	G722_DEC_IN_RDY has high priority	0x1
G722_ENC_OUT_RDY	INT6_NORMALPRIO_G722_ENC_OUT_RDY	G722_ENC_OUT_RDY has normal priority	0x0*
	INT6_HIGHPRIO_G722_ENC_OUT_RDY	G722_ENC_OUT_RDY has high priority	0x1
G722_ENC_IN_RDY	INT6_NORMALPRIO_G722_ENC_IN_RDY	G722_ENC_IN_RDY has normal priority	0x0*
	INT6_HIGHPRIO_G722_ENC_IN_RDY	G722_ENC_IN_RDY has high priority	0x1

3.2.1.53 D_INT_PRIO_7, INT_PRIO_7

Bit Field	Read/Write	Field Name	Description
7	RW	FIFO_7	FIFO_7 interrupt priority
6	RW	FIFO_6	FIFO_6 interrupt priority
5	RW	FIFO_5	FIFO_5 interrupt priority
4	RW	FIFO_4	FIFO_4 interrupt priority
3	RW	FIFO_3	FIFO_3 interrupt priority
2	RW	FIFO_2	FIFO_2 interrupt priority
1	RW	FIFO_1	FIFO_1 interrupt priority
0	RW	FIFO_0	FIFO_0 interrupt priority

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
FIFO_7	INT7_NORMALPRIO_FIFO_7	FIFO_7 has normal priority	0x0*
	INT7_HIGHPRIO_FIFO_7	FIFO_7 has high priority	0x1
FIFO_6	INT7_NORMALPRIO_FIFO_6	FIFO_6 has normal priority	0x0*
	INT7_HIGHPRIO_FIFO_6	FIFO_6 has high priority	0x1
FIFO_5	INT7_NORMALPRIO_FIFO_5	FIFO_5 has normal priority	0x0*
	INT7_HIGHPRIO_FIFO_5	FIFO_5 has high priority	0x1
FIFO_4	INT7_NORMALPRIO_FIFO_4	FIFO_4 has normal priority	0x0*
	INT7_HIGHPRIO_FIFO_4	FIFO_4 has high priority	0x1
FIFO_3	INT7_NORMALPRIO_FIFO_3	FIFO_3 has normal priority	0x0*
	INT7_HIGHPRIO_FIFO_3	FIFO_3 has high priority	0x1
FIFO_2	INT7_NORMALPRIO_FIFO_2	FIFO_2 has normal priority	0x0*
	INT7_HIGHPRIO_FIFO_2	FIFO_2 has high priority	0x1
FIFO_1	INT7_NORMALPRIO_FIFO_1	FIFO_1 has normal priority	0x0*
	INT7_HIGHPRIO_FIFO_1	FIFO_1 has high priority	0x1
FIFO_0	INT7_NORMALPRIO_FIFO_0	FIFO_0 has normal priority	0x0*
	INT7_HIGHPRIO_FIFO_0	FIFO_0 has high priority	0x1

3.2.1.54 D_INT_PRIO_8, INT_PRIO_8

Bit Field	Read/Write	Field Name	Description
7	RW	HEAR_7	HEAR_7 interrupt priority
6	RW	HEAR_6	HEAR_6 interrupt priority
5	RW	HEAR_5	HEAR_5 interrupt priority
4	RW	HEAR_4	HEAR_4 interrupt priority
3	RW	HEAR_3	HEAR_3 interrupt priority
2	RW	HEAR_2	HEAR_2 interrupt priority
1	RW	HEAR_1	HEAR_1 interrupt priority
0	RW	HEAR_0	HEAR_0 interrupt priority

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
HEAR_7	INT8_NORMALPRIO_HEAR_7	HEAR_7 has normal priority	0x0*
	INT8_HIGHPRIO_HEAR_7	HEAR_7 has high priority	0x1
HEAR_6	INT8_NORMALPRIO_HEAR_6	HEAR_6 has normal priority	0x0*
	INT8_HIGHPRIO_HEAR_6	HEAR_6 has high priority	0x1
HEAR_5	INT8_NORMALPRIO_HEAR_5	HEAR_5 has normal priority	0x0*
	INT8_HIGHPRIO_HEAR_5	HEAR_5 has high priority	0x1
HEAR_4	INT8_NORMALPRIO_HEAR_4	HEAR_4 has normal priority	0x0*
	INT8_HIGHPRIO_HEAR_4	HEAR_4 has high priority	0x1
HEAR_3	INT8_NORMALPRIO_HEAR_3	HEAR_3 has normal priority	0x0*
	INT8_HIGHPRIO_HEAR_3	HEAR_3 has high priority	0x1
HEAR_2	INT8_NORMALPRIO_HEAR_2	HEAR_2 has normal priority	0x0*
	INT8_HIGHPRIO_HEAR_2	HEAR_2 has high priority	0x1
HEAR_1	INT8_NORMALPRIO_HEAR_1	HEAR_1 has normal priority	0x0*
	INT8_HIGHPRIO_HEAR_1	HEAR_1 has high priority	0x1
HEAR_0	INT8_NORMALPRIO_HEAR_0	HEAR_0 has normal priority	0x0*
	INT8_HIGHPRIO_HEAR_0	HEAR_0 has high priority	0x1

3.2.1.55 D_INT_PRIO_9, INT_PRIO_9

Bit Field	Read/Write	Field Name	Description
7	RW	DSP_7	DSP_7 interrupt priority
6	RW	DSP_6	DSP_6 interrupt priority
5	RW	DSP_5	DSP_5 interrupt priority
4	RW	DSP_4	DSP_4 interrupt priority
3	RW	DSP_3	DSP_3 interrupt priority
2	RW	DSP_2	DSP_2 interrupt priority
1	RW	DSP_1	DSP_1 interrupt priority
0	RW	DSP_0	DSP_0 interrupt priority

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DSP_7	INT9_NORMALPRIO_DSP_7	DSP_7 has normal priority	0x0*
	INT9_HIGHPRIO_DSP_7	DSP_7 has high priority	0x1
DSP_6	INT9_NORMALPRIO_DSP_6	DSP_6 has normal priority	0x0*
	INT9_HIGHPRIO_DSP_6	DSP_6 has high priority	0x1
DSP_5	INT9_NORMALPRIO_DSP_5	DSP_5 has normal priority	0x0*
	INT9_HIGHPRIO_DSP_5	DSP_5 has high priority	0x1
DSP_4	INT9_NORMALPRIO_DSP_4	DSP_4 has normal priority	0x0*
	INT9_HIGHPRIO_DSP_4	DSP_4 has high priority	0x1
DSP_3	INT9_NORMALPRIO_DSP_3	DSP_3 has normal priority	0x0*
	INT9_HIGHPRIO_DSP_3	DSP_3 has high priority	0x1
DSP_2	INT9_NORMALPRIO_DSP_2	DSP_2 has normal priority	0x0*
	INT9_HIGHPRIO_DSP_2	DSP_2 has high priority	0x1
DSP_1	INT9_NORMALPRIO_DSP_1	DSP_1 has normal priority	0x0*
	INT9_HIGHPRIO_DSP_1	DSP_1 has high priority	0x1
DSP_0	INT9_NORMALPRIO_DSP_0	DSP_0 has normal priority	0x0*
	INT9_HIGHPRIO_DSP_0	DSP_0 has high priority	0x1

3.2.1.56 D_INT_PRIO_10, INT_PRIO_10

Bit Field	Read/Write	Field Name	Description
7	RW	CM3_7	CM3_7 interrupt priority
6	RW	CM3_6	CM3_6 interrupt priority
5	RW	CM3_5	CM3_5 interrupt priority
4	RW	CM3_4	CM3_4 interrupt priority
3	RW	CM3_3	CM3_3 interrupt priority
2	RW	CM3_2	CM3_2 interrupt priority
1	RW	CM3_1	CM3_1 interrupt priority
0	RW	CM3_0	CM3_0 interrupt priority

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
CM3_7	INT10_NORMALPRIO_CM3_7	CM3_7 has normal priority	0x0*
	INT10_HIGHPRIO_CM3_7	CM3_7 has high priority	0x1
CM3_6	INT10_NORMALPRIO_CM3_6	CM3_6 has normal priority	0x0*
	INT10_HIGHPRIO_CM3_6	CM3_6 has high priority	0x1
CM3_5	INT10_NORMALPRIO_CM3_5	CM3_5 has normal priority	0x0*
	INT10_HIGHPRIO_CM3_5	CM3_5 has high priority	0x1
CM3_4	INT10_NORMALPRIO_CM3_4	CM3_4 has normal priority	0x0*
	INT10_HIGHPRIO_CM3_4	CM3_4 has high priority	0x1
CM3_3	INT10_NORMALPRIO_CM3_3	CM3_3 has normal priority	0x0*
	INT10_HIGHPRIO_CM3_3	CM3_3 has high priority	0x1
CM3_2	INT10_NORMALPRIO_CM3_2	CM3_2 has normal priority	0x0*
	INT10_HIGHPRIO_CM3_2	CM3_2 has high priority	0x1
CM3_1	INT10_NORMALPRIO_CM3_1	CM3_1 has normal priority	0x0*
	INT10_HIGHPRIO_CM3_1	CM3_1 has high priority	0x1
CM3_0	INT10_NORMALPRIO_CM3_0	CM3_0 has normal priority	0x0*
	INT10_HIGHPRIO_CM3_0	CM3_0 has high priority	0x1

3.2.1.57 D_INT_PRIO_11, INT_PRIO_11

Bit Field	Read/Write	Field Name	Description
5	RW	RS_COM	RS_COM interrupt priority
4	RW	G722_COM	G722_COM interrupt priority
3	RW	MEM_ERROR	MEM_ERROR interrupt priority
2	RW	LSAD	LSAD interrupt priority
1	RW	CLK_DET	CLK_DET interrupt priority
0	RW	NNA	NNA interrupt priority

Field Name	Value Symbol	Value Description	Hex Value
RS_COM	INT11_NORMALPRIO_RS_COM	RS_COM has normal priority	0x0*
	INT11_HIGHPRIO_RS_COM	RS_COM has high priority	0x1
G722_COM	INT11_NORMALPRIO_G722_COM	G722_COM has normal priority	0x0*
	INT11_HIGHPRIO_G722_COM	G722_COM has high priority	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
MEM_ERROR	INT11_NORMALPRIO_MEM_ERROR	MEM_ERROR has normal priority	0x0*
	INT11_HIGHPRIO_MEM_ERROR	MEM_ERROR has high priority	0x1
LSAD	INT11_NORMALPRIO_LSAD	LSAD has normal priority	0x0*
	INT11_HIGHPRIO_LSAD	LSAD has high priority	0x1
CLK_DET	INT11_NORMALPRIO_CLK_DET	CLK_DET has normal priority	0x0*
	INT11_HIGHPRIO_CLK_DET	CLK_DET has high priority	0x1
NNA	INT11_NORMALPRIO_NNA	NNA has normal priority	0x0*
	INT11_HIGHPRIO_NNA	NNA has high priority	0x1

CHAPTER 4

Arm Cortex-M3 Processor

The Arm Cortex-M3 processor is a low-power microcontroller that implements the Armv7-M architecture, which features low interrupt latency and low-cost debugging.

NOTE: The Ezairo 8300 system contains version r2p1 of the Arm Cortex-M3 processor.

4.1 PROCESSOR OVERVIEW

In the Ezairo 8300 system, the Arm Cortex-M3 processor typically performs one or more of the following roles:

- The system master, configuring and updating the system execution
- A coprocessor to the CFX DSP that provides additional microcontroller computation for interface drivers and protocols executing on those interfaces
- A controller for managing hardware acceleration peripherals such as the Reed-Solomon, G.722 blocks, the asynchronous sample rate converter (ASRC), the audio sink clock counters (ASCC), the neural network accelerator (NNA), or the LPDSP32 DSP (which is expected to be used for codecs, a neural network, and similar use cases).

Some key features of the Arm Cortex-M3 processor include:

- The Thumb-2 (ISA) subset, consisting of all base Thumb-2 instructions, both 16-bit and 32-bit
- Harvard processor architecture enabling simultaneous instruction fetches with data load or store
- A three-stage pipeline
- Single cycle 32-bit multiply
- Hardware divide
- Thumb and debug states
- Handler and thread modes
- Low latency ISR entry and exit
- Processor state saving and restoration, with no instruction fetch overhead; the exception vector is fetched from memory in parallel with the state saving, enabling faster ISR entry
- Support for late-arriving interrupts
- A tightly coupled interface to an interrupt controller, enabling efficient processing of late-arriving interrupts
- Tail-chaining of interrupts, enabling back-to-back interrupt processing without the overhead of state saving and restoration between interrupts
- Interruptible-continued LDM/STM and PUSH/POP
- Registers:
 - 13 general-purpose 32-bit registers
 - Link Register (LR)
 - Program Counter (PC)
 - Program Status Register (xPSR)
 - Two banked SP registers
- Native support for 8-bit, 16-bit, and 32-bit data; some memory areas also support 1-bit data accesses

For power management, the processor can be placed, via firmware control, into Standby Mode, in which the processor clock is disabled. The Nested Vectored Interrupt Controller (NVIC) continues to run to enable exiting from a sleep state on an interrupt. For more information about the NVIC, see [Section 4.6.1 “Nested Vectored Interrupt Controller \(NVIC\)” on page 135](#).

Ezairo 8300 Hardware Reference

For information about the Arm Cortex-M3 processor's hardware features, see the *Cortex-M3 Technical Reference Manual*, or *The Definitive Guide to the Arm Cortex-M3* and other third-party documentation for the Arm Cortex-M3 processor. Further hardware details regarding the private peripherals accessible to the Arm Cortex-M3 processor in the Ezairo 8300 system are available in [Section 4.6 “Arm Cortex-M3 Private Peripherals”](#).

IMPORTANT: To minimize power consumption when the Arm Cortex-M3 core is waiting for an interrupt or potentially waiting for an interrupt and/or fault (generalized as an event), the core should be held in a wait state using the WFI or WFE instructions. In C code, an application can use the `SYS_WAIT_FOR_INTERRUPT` and `SYS_WAIT_FOR_EVENT` macros to place the core into the appropriate wait state.

4.2 ARM CORTEX-M3 PROCESSOR DEBUGGING

To support the Arm Cortex-M3 processor, there are two debug ports that each can access the standard Arm Debug Access Port (DAP). The DAP provides access to all of the Arm Cortex-M3 processor registers, and to the memory available through the memory buses attached to the Arm Cortex-M3 processor. This transparent view of the system enables a user to review the current state of the system components through their associated memory-mapped control registers, and to use this information to troubleshoot most issues in an application. For more information about debugging the Arm Cortex-M3 processor using the DAP, see the debug components of the NVIC as described in [Section 4.6.3 “Debug Controller”](#).

Interfacing with the DAP are the following debug ports:

1. The shared I²C debug port. This debug port shares two dedicated pads with the CFX debug port, but has its own address and command set. For more information about this debug port, see [Chapter 17.1 “Debug” on page 523](#), and the *Communication Protocols Manual for Ezairo 8300*.
2. The Serial-Wire Debug or JTAG Debug Port (SWJ-DP) debug port, connected to DIOs 30 to 33. This debug port interface can be configured in two modes, serial wire mode (SW-DP) using only DIOs 30 to 31, or JTAG mode (JTAG-DP) using all four connections. For information about configuring the DIOs for these modes, see [Section 16.2.1 “Special Functional Configurations” on page 468](#).

When using the SWJ-DP in SW-DP mode, data is transferred using a single clock and single data signal. This interface mode is useful when the number of debug port pins needs to be limited, but it is only half-duplex and has a relatively high overhead to transfer data. When using the SWJ-DP in JTAG-DP mode, data is transferred using a data control line, a pair of data lines, a clock, and an optional reset signal. This interface mode is useful when the number of debug port connections is not limited and a higher throughput is required, as JTAG mode uses less data overhead than SW mode and supports full-duplex transfers.

4.3 INTER-PROCESSOR COMMUNICATION

As part of the Ezairo 8300 system, the Arm Cortex-M3 processor is provided with a set of eight interrupt lines that it can use to signal to the CFX DSP. To trigger an interrupt on the CFX, the Arm Cortex-M3 processor writes a 1 to the appropriate `SYSCTRL_CFX_CMD_*` field from the `SYSCTRL_CFX_CMD` register. Similarly, it can receive interrupts from the CFX DSP as described in [Section 3.1.1 “Inter-processor Communication” on page 59](#).

4.4 ARM CORTEX-M3 CONTROL AND CONFIGURATION REGISTER CONVENTIONS

Control and configuration registers for the Arm Cortex-M3 core that are mapped onto the peripheral bus are accessible in three forms:

Ezairo 8300 Hardware Reference

- When accessed from the Arm Cortex-M3 core, these registers are accessed through CMSIS-compatible structures, with the first part of each register broken off as the name for the structure. To access the registers in this section, use the ">" operator to access the register (e.g., ECB_INT_STATUS is accessed as ECB->INT_STATUS). Structures for CFX registers controlling digital components that are accessed from the Arm Cortex-M3 processor in this way are prefixed with "D_".
- When accessed from the Arm Cortex-M3 core in the bit-band aliased areas, each register is defined as a CMSIS compatible structure that provides aliased access to each bit-band accessible field though a structure based on the register and bit names (e.g., the RS_ENC_IN_ERR bit from the ECB_INT_STATUS register is accessed as ECB_INT_STATUS->RS_ENC_IN_ERR_ALIAS).

NOTE: The CFX DSP does not have bit-band access support, so the bit-band aliased areas are not mapped into the CFX DSP's memory spaces.

4.5 CONTROL AND CONFIGURATION REGISTERS

Register Name	Register Description	Address
SYSCTRL_DSP_CMD	DSP Commands	P:0x1003 0x4000000C
SYSCTRL_CFX_CMD	CFX Commands	P:0x1004 0x40000010
SYSCTRL_BOOT_STATUS	CM3 Boot Status	P:0x1019 0x40000064

4.5.1 SYSCTRL_DSP_CMD

Bit Field	Read/Write	Field Name	Description
5	W	DSP_CMD_5	Write a 1 to issue DSP command 5
4	W	DSP_CMD_4	Write a 1 to issue DSP command 4
3	W	DSP_CMD_3	Write a 1 to issue DSP command 3
2	W	DSP_CMD_2	Write a 1 to issue DSP command 2
1	W	DSP_CMD_1	Write a 1 to issue DSP command 1
0	W	DSP_CMD_0	Write a 1 to issue DSP command 0

Field Name	Value Symbol	Value Description	Hex Value
DSP_CMD_5	DSP_CMD_5	Issue DSP command 5	0x1
DSP_CMD_4	DSP_CMD_4	Issue DSP command 4	0x1
DSP_CMD_3	DSP_CMD_3	Issue DSP command 3	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DSP_CMD_2	DSP_CMD_2	Issue DSP command 2	0x1
DSP_CMD_1	DSP_CMD_1	Issue DSP command 1	0x1
DSP_CMD_0	DSP_CMD_0	Issue DSP command 0	0x1

4.5.2 SYSCTRL_CFX_CMD

Bit Field	Read/Write	Field Name	Description
7	W	CFX_CMD_7	Write a 1 to issue CFX command 7
6	W	CFX_CMD_6	Write a 1 to issue CFX command 6
5	W	CFX_CMD_5	Write a 1 to issue CFX command 5
4	W	CFX_CMD_4	Write a 1 to issue CFX command 4
3	W	CFX_CMD_3	Write a 1 to issue CFX command 3
2	W	CFX_CMD_2	Write a 1 to issue CFX command 2
1	W	CFX_CMD_1	Write a 1 to issue CFX command 1
0	W	CFX_CMD_0	Write a 1 to issue CFX command 0

Field Name	Value Symbol	Value Description	Hex Value
CFX_CMD_7	CFX_CMD_7	Issue CFX command 7	0x1
CFX_CMD_6	CFX_CMD_6	Issue CFX command 6	0x1
CFX_CMD_5	CFX_CMD_5	Issue CFX command 5	0x1
CFX_CMD_4	CFX_CMD_4	Issue CFX command 4	0x1
CFX_CMD_3	CFX_CMD_3	Issue CFX command 3	0x1
CFX_CMD_2	CFX_CMD_2	Issue CFX command 2	0x1
CFX_CMD_1	CFX_CMD_1	Issue CFX command 1	0x1
CFX_CMD_0	CFX_CMD_0	Issue CFX command 0	0x1

4.5.3 SYSCTRL_BOOT_STATUS

Bit Field	Read/Write	Field Name	Description
0	R	STATUS	Arm Cortex-M3 boot status

Field Name	Value Symbol	Value Description	Hex Value
STATUS	CM3_NOT_BOOTED	Arm Cortex-M3 has not finished booting	0x0*
	CM3_BOOTED	Arm Cortex-M3 has finished booting	0x1

Ezairo 8300 Hardware Reference

4.6 ARM CORTEX-M3 PRIVATE PERIPHERALS

4.6.1 Nested Vectored Interrupt Controller (NVIC)

The Arm Cortex-M3 processor is closely tied to a nested vectored interrupt controller (NVIC), which is an integral part of the processor and provides the interrupt handling functionality. This block is implemented with the Arm Cortex-M3 processor and is described in the *Cortex-M3 Technical Reference Manual*.

The Arm Cortex-M3 processor as implemented for Ezairo 8300 uses pulse interrupts, which are sampled on the rising edge of SYSCLK. A pulse interrupt can be reasserted during the ISR, allowing the interrupt to be in the pending and active states at the same time. If another pulse arrives while the interrupt is still pending, the interrupt remains pending and the ISR runs only once.

The NVIC handles a non-maskable interrupt (NMI), several faults, predefined interrupts, and a set of general-purpose interrupts that are external to the Arm Cortex-M3 processor and linked to its interfaces and peripherals. A list of the interrupts supported by the NVIC for the Arm Cortex-M3 processor is provided in the "Interrupts in the Arm Cortex-M3 Processor" table (Table 4).

Table 4. Interrupts in the Arm Cortex-M3 Processor

Interrupt Enumeration Define	Enumeration Value	Vector Number	Description
Reset_IRQn	-15	1	Reset vector
NonMaskableInt_IRQn	-14	2	Non-maskable interrupt (NMI)
HardFault_IRQn	-13	3	Hard fault interrupt
MemoryManagement_IRQn	-12	4	Memory management interrupt
BusFault_IRQn	-11	5	Bus fault interrupt
UsageFault_IRQn	-10	6	Usage fault interrupt
SVCALL_IRQn	-5	11	SVCALL interrupt
DebugMonitor_IRQn	-4	12	Debug monitor interrupt
PendSV_IRQn	-2	14	PendSV interrupt
SysTick_IRQn	-1	15	System Tick interrupt
WATCHDOG_IRQn	0	16	Watchdog Timer interrupt
LSAD_BATMON_IRQn	1	17	LSAD battery monitor interrupt
AUDIOSINK_0_IRQn	2	18	Audio sink interrupt 0
AUDIOSINK_1_IRQn	3	19	Audio sink interrupt 1
TIMER_0_IRQn	4	20	Timer 0 interrupt
TIMER_1_IRQn	5	21	Timer 1 interrupt
TIMER_2_IRQn	6	22	Timer 2 interrupt
TIMER_3_IRQn	7	23	Timer 3 interrupt
DMA_0_IRQn	8	24	DMA channel 0 interrupt
DMA_1_IRQn	9	25	DMA channel 1 interrupt
DMA_2_IRQn	10	26	DMA channel 2 interrupt

onsemi Confidential
Ezairo 8300 Hardware Reference

Interrupts in the Arm Cortex-M3 Processor (Continued)

Interrupt Enumeration Define	Enumeration Value	Vector Number	Description
DMA_3_IRQn	11	27	DMA channel 3 interrupt
DMA_4_IRQn	12	28	DMA channel 4 interrupt
DMA_5_IRQn	13	29	DMA channel 5 interrupt
DMA_6_IRQn	14	30	DMA channel 6 interrupt
DMA_7_IRQn	15	31	DMA channel 7 interrupt
DIO_0_IRQn	16	32	DIO0 interrupt
DIO_1_IRQn	17	33	DIO1 interrupt
DIO_2_IRQn	18	34	DIO2 interrupt
DIO_3_IRQn	19	35	DIO3 interrupt
DIO_4_IRQn	20	36	DIO4 interrupt
DIO_5_IRQn	21	37	DIO5 interrupt
DIO_6_IRQn	22	38	DIO6 interrupt
DIO_7_IRQn	23	39	DIO7 interrupt
I3C_0_IRQn	24	40	I ³ C (0) interrupt
I3C_1_IRQn	25	41	I ³ C (1) interrupt
I2C_0_IRQn	26	42	I ² C (0) interrupt
I2C_1_IRQn	27	43	I ² C (1) interrupt
D_I2C_IRQn	28	44	D_I2C interrupt
SPI_0_COM_IRQn	29	45	SPI (0) communication interrupt
SPI_0_RX_IRQn	30	46	SPI (0) receive interrupt
SPI_0_TX_IRQn	31	47	SPI (0) transmit interrupt
SPI_1_COM_IRQn	32	48	SPI (1) communication interrupt
SPI_1_RX_IRQn	33	49	SPI (1) receive interrupt
SPI_1_TX_IRQn	34	50	SPI (1) transmit interrupt
EMMC_IRQn	35	51	eMMC interrupt
UART_ERROR_IRQn	36	52	UART error interrupt
UART_RX_IRQn	37	53	UART receive interrupt
UART_TX_IRQn	38	54	UART transmit interrupt
FENG_IRQn	39	55	Filter Engine interrupt
PCM_0_ERROR_IRQn	40	56	PCM (0) error interrupt
PCM_0_RX_TX_IRQn	41	57	PCM (0) receive and transmit interrupt
PCM_1_ERROR_IRQn	42	58	PCM (1) error interrupt
PCM_1_RX_TX_IRQn	43	59	PCM (1) receive and transmit interrupt

onsemi Confidential
Ezairo 8300 Hardware Reference

Interrupts in the Arm Cortex-M3 Processor (Continued)

Interrupt Enumeration Define	Enumeration Value	Vector Number	Description
ASRC_ERROR_IRQn	44	60	ASRC error interrupt
ASRC_IN_IRQn	45	61	ASRC data input interrupt
ASRC_OUT_IRQn	46	62	ASRC data output interrupt
ASRC_IDLE_IRQn	47	63	ASRC idle interrupt
G722_ENC_IN_RDY_IRQn	48	64	G.722 encoder input ready interrupt
G722_ENC_OUT_RDY_IRQn	49	65	G.722 encoder output ready interrupt
G722_DEC_IN_RDY_IRQn	50	66	G.722 decoder input ready interrupt
G722_DEC_OUT_RDY_IRQn	51	67	G.722 decoder output ready interrupt
RS_ENC_IN_RDY_IRQn	52	68	Reed-Solomon encoder input ready interrupt
RS_ENC_OUT_RDY_IRQn	53	69	Reed-Solomon encoder output ready interrupt
RS_DEC_IN_RDY_IRQn	54	70	Reed-Solomon decoder input ready interrupt
RS_DEC_OUT_RDY_IRQn	55	71	Reed-Solomon decoder output ready interrupt
FIFO_0_IRQn	56	72	FIFO interrupt 0
FIFO_1_IRQn	57	73	FIFO interrupt 1
FIFO_2_IRQn	58	74	FIFO interrupt 2
FIFO_3_IRQn	59	75	FIFO interrupt 3
FIFO_4_IRQn	60	76	FIFO interrupt 4
FIFO_5_IRQn	61	77	FIFO interrupt 5
FIFO_6_IRQn	62	78	FIFO interrupt 6
FIFO_7_IRQn	63	79	FIFO interrupt 7
HEAR_0_IRQn	64	80	HEAR event interrupt 0
HEAR_1_IRQn	65	81	HEAR event interrupt 1
HEAR_2_IRQn	66	82	HEAR event interrupt 2
HEAR_3_IRQn	67	83	HEAR event interrupt 3
HEAR_4_IRQn	68	84	HEAR event interrupt 4
HEAR_5_IRQn	69	85	HEAR event interrupt 5
HEAR_6_IRQn	70	86	HEAR event interrupt 6
HEAR_7_IRQn	71	87	HEAR event interrupt 7
DSP_0_IRQn	72	88	LPDSP32 event interrupt 0
DSP_1_IRQn	73	89	LPDSP32 event interrupt 1
DSP_2_IRQn	74	90	LPDSP32 event interrupt 2
DSP_3_IRQn	75	91	LPDSP32 event interrupt 3
DSP_4_IRQn	76	92	LPDSP32 event interrupt 4

Ezairo 8300 Hardware Reference

Interrupts in the Arm Cortex-M3 Processor (Continued)

Interrupt Enumeration Define	Enumeration Value	Vector Number	Description
DSP_5_IRQn	77	93	LPDSP32 event interrupt 5
DSP_6_IRQn	78	94	LPDSP32 event interrupt 6
DSP_7_IRQn	79	95	LPDSP32 event interrupt 7
CFX_0_IRQn	80	96	CFX interrupt 0
CFX_1_IRQn	81	97	CFX interrupt 1
CFX_2_IRQn	82	98	CFX interrupt 2
CFX_3_IRQn	83	99	CFX interrupt 3
CFX_4_IRQn	84	100	CFX interrupt 4
CFX_5_IRQn	85	101	CFX interrupt 5
CFX_6_IRQn	86	102	CFX interrupt 6
CFX_7_IRQn	87	103	CFX interrupt 7
NNA_IRQn	88	104	Neural network accelerator interrupt
CLK_DET_IRQn	89	105	Clock detector interrupt
LSAD_IRQn	90	106	Low-speed ADC interrupt
MEM_ERROR_IRQn	91	107	Memory error interrupt
G722_COM_IRQn	92	108	G.722 communication interrupt
RS_COM_IRQn	93	109	Reed-Solomon communication interrupt

4.6.1.1 NVIC and SCB Control and Configuration Registers

The following subsections describe their bit fields and use by the Ezairo 8300 microcontroller.

Register Name	Register Description	Address
NVIC_ISER0	NVIC External Interrupt Set Enable Register 0	P:0x28004840 0xE000E100
NVIC_ISER1	NVIC External Interrupt Set Enable Register 1	P:0x28004841 0xE000E104
NVIC_ISER2	NVIC External Interrupt Set Enable Register 2	P:0x28004842 0xE000E108
NVIC_ICER0	NVIC External Interrupt Clear Enable Register 0	P:0x28004860 0xE000E180
NVIC_ICER1	NVIC External Interrupt Clear Enable Register 1	P:0x28004861 0xE000E184

onsemi Confidential

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
NVIC_ICER2	NVIC External Interrupt Clear Enable Register 2	P:0x28004862 0xE000E188
NVIC_ISPR0	NVIC External Interrupt Set Pending Register 0	P:0x28004880 0xE000E200
NVIC_ISPR1	NVIC External Interrupt Set Pending Register 1	P:0x28004881 0xE000E204
NVIC_ISPR2	NVIC External Interrupt Set Pending Register 2	P:0x28004882 0xE000E208
NVIC_ICPR0	NVIC External Interrupt Clear Pending Register 0	P:0x280048A0 0xE000E280
NVIC_ICPR1	NVIC External Interrupt Clear Pending Register 1	P:0x280048A1 0xE000E284
NVIC_ICPR2	NVIC External Interrupt Clear Pending Register 2	P:0x280048A2 0xE000E288
NVIC_IABR0	NVIC External Interrupt Active Register 0	P:0x280048C0 0xE000E300
NVIC_IABR1	NVIC External Interrupt Active Register 1	P:0x280048C1 0xE000E304
NVIC_IABR2	NVIC External Interrupt Active Register 2	P:0x280048C2 0xE000E308
NVIC_IP0	NVIC External Interrupt Priority Register 0	P:0x28004900 0xE000E400
NVIC_IP1	NVIC External Interrupt Priority Register 1	P:0x28004901 0xE000E404
NVIC_IP2	NVIC External Interrupt Priority Register 2	P:0x28004902 0xE000E408
NVIC_IP3	NVIC External Interrupt Priority Register 3	P:0x28004903 0xE000E40C

onsemi Confidential

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
NVIC_IP4	NVIC External Interrupt Priority Register 4	P:0x28004904 0xE000E410
NVIC_IP5	NVIC External Interrupt Priority Register 5	P:0x28004905 0xE000E414
NVIC_IP6	NVIC External Interrupt Priority Register 6	P:0x28004906 0xE000E418
NVIC_IP7	NVIC External Interrupt Priority Register 7	P:0x28004907 0xE000E41C
NVIC_IP8	NVIC External Interrupt Priority Register 8	P:0x28004908 0xE000E420
NVIC_IP9	NVIC External Interrupt Priority Register 9	P:0x28004909 0xE000E424
NVIC_IP10	NVIC External Interrupt Priority Register 10	P:0x2800490A 0xE000E428
NVIC_IP11	NVIC External Interrupt Priority Register 11	P:0x2800490B 0xE000E42C
NVIC_IP12	NVIC External Interrupt Priority Register 12	P:0x2800490C 0xE000E430
NVIC_IP13	NVIC External Interrupt Priority Register 13	P:0x2800490D 0xE000E434
NVIC_IP14	NVIC External Interrupt Priority Register 14	P:0x2800490E 0xE000E438
NVIC_IP15	NVIC External Interrupt Priority Register 15	P:0x2800490F 0xE000E43C
NVIC_IP16	NVIC External Interrupt Priority Register 16	P:0x28004910 0xE000E440
NVIC_IP17	NVIC External Interrupt Priority Register 17	P:0x28004911 0xE000E444

onsemi Confidential

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
NVIC_IP18	NVIC External Interrupt Priority Register 18	P:0x28004911 0xE000E444
NVIC_IP19	NVIC External Interrupt Priority Register 19	P:0x28004911 0xE000E444
NVIC_IP20	NVIC External Interrupt Priority Register 20	P:0x28004911 0xE000E444
NVIC_IP21	NVIC External Interrupt Priority Register 21	P:0x28004911 0xE000E444
NVIC_IP22	NVIC External Interrupt Priority Register 22	P:0x28004911 0xE000E444
NVIC_IP23	NVIC External Interrupt Priority Register 23	P:0x28004911 0xE000E444
NVIC_STIR	NVIC Software Trigger Interrupt Register	P:0x28004BC0 0xE000EF00
SCnSCB_ICTR	Interrupt Controller Type Register	P:0x28004801 0xE000E004
SCB_CPUID	CPU ID Base Register	P:0x28004B40 0xE000ED00
SCB_ICSR	Interrupt Control and State Register	P:0x28004B41 0xE000ED04
SCB_VTOR	Vector Table Offset Register	P:0x28004B42 0xE000ED08
SCB_AIRCR	Application Interrupt and Reset Control Register	P:0x28004B43 0xE000ED0C
SCB_SCR	System Control Register	P:0x28004B44 0xE000ED10
SCB_CCR	Configuration Control Register	P:0x28004B45 0xE000ED14

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
SCB_SHP0	System Interrupt Priority Register 0	P:0x28004B46 0xE000ED18
SCB_SHP1	System Interrupt Priority Register 1	P:0x28004B47 0xE000ED1C
SCB_SHP2	System Interrupt Priority Register 2	P:0x28004B48 0xE000ED20
SCB_SHCSR	System Handler Control and State Register	P:0x28004B49 0xE000ED24
SCB_CFSR	Fault Status Register	P:0x28004B4A 0xE000ED28
SCB_HFSR	Hard Fault Status Register	P:0x28004B4B 0xE000ED2C
SCB_DFSR	Debug Fault Status Register	P:0x28004B4C 0xE000ED30
SCB_MMFSR	Memory Management Fault Address Register	P:0x28004B4D 0xE000ED34
SCB_BFAR	Bus Fault Address Register	P:0x28004B4E 0xE000ED38

IMPORTANT: The NVIC is a standard component provided with the Arm Cortex-M3 processor. The registers for this peripheral are defined in *core_cm3.h* and augmented by defines for the bit fields, bit settings, and sub-registers in *sk5_hw.h*.

4.6.1.1.1 NVIC_ISER0

Bit Field	Read/Write	Field Name	Description
31	RW	SPI_0_TX	SPI_0_TX interrupt set enable
30	RW	SPI_0_RX	SPI_0_RX interrupt set enable
29	RW	SPI_0_COM	SPI_0_COM interrupt set enable
28	RW	D_I2C	D_I2C interrupt set enable
27	RW	I2C_1	I2C_1 interrupt set enable

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
26	RW	I2C_0	I2C_0 interrupt set enable
25	RW	I3C_1	I3C_1 interrupt set enable
24	RW	I3C_0	I3C_0 interrupt set enable
23	RW	DIO_7	DIO_7 interrupt set enable
22	RW	DIO_6	DIO_6 interrupt set enable
21	RW	DIO_5	DIO_5 interrupt set enable
20	RW	DIO_4	DIO_4 interrupt set enable
19	RW	DIO_3	DIO_3 interrupt set enable
18	RW	DIO_2	DIO_2 interrupt set enable
17	RW	DIO_1	DIO_1 interrupt set enable
16	RW	DIO_0	DIO_0 interrupt set enable
15	RW	DMA_7	DMA_7 interrupt set enable
14	RW	DMA_6	DMA_6 interrupt set enable
13	RW	DMA_5	DMA_5 interrupt set enable
12	RW	DMA_4	DMA_4 interrupt set enable
11	RW	DMA_3	DMA_3 interrupt set enable
10	RW	DMA_2	DMA_2 interrupt set enable
9	RW	DMA_1	DMA_1 interrupt set enable
8	RW	DMA_0	DMA_0 interrupt set enable
7	RW	TIMER_3	TIMER_3 interrupt set enable
6	RW	TIMER_2	TIMER_2 interrupt set enable
5	RW	TIMER_1	TIMER_1 interrupt set enable
4	RW	TIMER_0	TIMER_0 interrupt set enable
3	RW	AUDIOSINK_1	AUDIOSINK_1 interrupt set enable
2	RW	AUDIOSINK_0	AUDIOSINK_0 interrupt set enable
1	RW	LSAD_BATMON	LSAD_BATMON interrupt set enable
0	RW	WATCHDOG	WATCHDOG interrupt set enable

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
SPI_0_TX	NVIC_SPI_0_TX_INT_ENABLE	Enable the SPI_0_TX interrupt	0x1
SPI_0_RX	NVIC_SPI_0_RX_INT_ENABLE	Enable the SPI_0_RX interrupt	0x1
SPI_0_COM	NVIC_SPI_0_COM_INT_ENABLE	Enable the SPI_0_COM interrupt	0x1
D_I2C	NVIC_D_I2C_INT_ENABLE	Enable the D_I2C interrupt	0x1
I2C_1	NVIC_I2C_1_INT_ENABLE	Enable the I2C_1 interrupt	0x1
I2C_0	NVIC_I2C_0_INT_ENABLE	Enable the I2C_0 interrupt	0x1
I3C_1	NVIC_I3C_1_INT_ENABLE	Enable the I3C_1 interrupt	0x1
I3C_0	NVIC_I3C_0_INT_ENABLE	Enable the I3C_0 interrupt	0x1
DIO_7	NVIC_DIO_7_INT_ENABLE	Enable the DIO_7 interrupt	0x1
DIO_6	NVIC_DIO_6_INT_ENABLE	Enable the DIO_6 interrupt	0x1
DIO_5	NVIC_DIO_5_INT_ENABLE	Enable the DIO_5 interrupt	0x1
DIO_4	NVIC_DIO_4_INT_ENABLE	Enable the DIO_4 interrupt	0x1
DIO_3	NVIC_DIO_3_INT_ENABLE	Enable the DIO_3 interrupt	0x1
DIO_2	NVIC_DIO_2_INT_ENABLE	Enable the DIO_2 interrupt	0x1
DIO_1	NVIC_DIO_1_INT_ENABLE	Enable the DIO_1 interrupt	0x1
DIO_0	NVIC_DIO_0_INT_ENABLE	Enable the DIO_0 interrupt	0x1
DMA_7	NVIC_DMA_7_INT_ENABLE	Enable the DMA_7 interrupt	0x1
DMA_6	NVIC_DMA_6_INT_ENABLE	Enable the DMA_6 interrupt	0x1
DMA_5	NVIC_DMA_5_INT_ENABLE	Enable the DMA_5 interrupt	0x1
DMA_4	NVIC_DMA_4_INT_ENABLE	Enable the DMA_4 interrupt	0x1
DMA_3	NVIC_DMA_3_INT_ENABLE	Enable the DMA_3 interrupt	0x1
DMA_2	NVIC_DMA_2_INT_ENABLE	Enable the DMA_2 interrupt	0x1
DMA_1	NVIC_DMA_1_INT_ENABLE	Enable the DMA_1 interrupt	0x1
DMA_0	NVIC_DMA_0_INT_ENABLE	Enable the DMA_0 interrupt	0x1
TIMER_3	NVIC_TIMER_3_INT_ENABLE	Enable the TIMER_3 interrupt	0x1
TIMER_2	NVIC_TIMER_2_INT_ENABLE	Enable the TIMER_2 interrupt	0x1
TIMER_1	NVIC_TIMER_1_INT_ENABLE	Enable the TIMER_1 interrupt	0x1
TIMER_0	NVIC_TIMER_0_INT_ENABLE	Enable the TIMER_0 interrupt	0x1
AUDIOSINK_1	NVIC_AUDIOSINK_1_INT_ENABLE	Enable the AUDIOSINK_1 interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
AUDIOSINK_0	NVIC_AUDIOSINK_0_INT_ENABLE	Enable the AUDIOSINK_0 interrupt	0x1
LSAD_BATMON	NVIC_LSAD_BATMON_INT_ENABLE	Enable the LSAD_BATMON interrupt	0x1
WATCHDOG	NVIC_WATCHDOG_INT_ENABLE	Enable the WATCHDOG interrupt	0x1

4.6.1.1.2 NVIC_ISER1

Bit Field	Read/Write	Field Name	Description
31	RW	FIFO_7	FIFO_7 interrupt set enable
30	RW	FIFO_6	FIFO_6 interrupt set enable
29	RW	FIFO_5	FIFO_5 interrupt set enable
28	RW	FIFO_4	FIFO_4 interrupt set enable
27	RW	FIFO_3	FIFO_3 interrupt set enable
26	RW	FIFO_2	FIFO_2 interrupt set enable
25	RW	FIFO_1	FIFO_1 interrupt set enable
24	RW	FIFO_0	FIFO_0 interrupt set enable
23	RW	RS_DEC_OUT_RDY	RS_DEC_OUT_RDY interrupt set enable
22	RW	RS_DEC_IN_RDY	RS_DEC_IN_RDY interrupt set enable
21	RW	RS_ENC_OUT_RDY	RS_ENC_OUT_RDY interrupt set enable
20	RW	RS_ENC_IN_RDY	RS_ENC_IN_RDY interrupt set enable
19	RW	G722_DEC_OUT_RDY	G722_DEC_OUT_RDY interrupt set enable
18	RW	G722_DEC_IN_RDY	G722_DEC_IN_RDY interrupt set enable
17	RW	G722_ENC_OUT_RDY	G722_ENC_OUT_RDY interrupt set enable
16	RW	G722_ENC_IN_RDY	G722_ENC_IN_RDY interrupt set enable
15	RW	ASRC_IDLE	ASRC_IDLE interrupt set enable
14	RW	ASRC_OUT	ASRC_OUT interrupt set enable
13	RW	ASRC_IN	ASRC_IN interrupt set enable
12	RW	ASRC_ERROR	ASRC_ERROR interrupt set enable
11	RW	PCM_1_RX_TX	PCM_1_RX_TX interrupt set enable
10	RW	PCM_1_ERROR	PCM_1_ERROR interrupt set enable
9	RW	PCM_0_RX_TX	PCM_0_RX_TX interrupt set enable
8	RW	PCM_0_ERROR	PCM_0_ERROR interrupt set enable

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
7	RW	FENG	FENG interrupt set enable
6	RW	UART_0_TX	UART_0_TX interrupt set enable
5	RW	UART_0_RX	UART_0_RX interrupt set enable
4	RW	UART_0_ERROR	UART_0_ERROR interrupt set enable
3	RW	EMMC	EMMC interrupt set enable
2	RW	SPI_1_TX	SPI_1_TX interrupt set enable
1	RW	SPI_1_RX	SPI_1_RX interrupt set enable
0	RW	SPI_1_COM	SPI_1_COM interrupt set enable

Field Name	Value Symbol	Value Description	Hex Value
FIFO_7	NVIC_FIFO_7_INT_ENABLE	Enable the TIMER_3 interrupt	0x1
FIFO_6	NVIC_FIFO_6_INT_ENABLE	Enable the TIMER_2 interrupt	0x1
FIFO_5	NVIC_FIFO_5_INT_ENABLE	Enable the TIMER_1 interrupt	0x1
FIFO_4	NVIC_FIFO_4_INT_ENABLE	Enable the TIMER_0 interrupt	0x1
FIFO_3	NVIC_FIFO_3_INT_ENABLE	Enable the AUDIOSINK_1 interrupt	0x1
FIFO_2	NVIC_FIFO_2_INT_ENABLE	Enable the AUDIOSINK_0 interrupt	0x1
FIFO_1	NVIC_FIFO_1_INT_ENABLE	Enable the LSAD_BATMON interrupt	0x1
FIFO_0	NVIC_FIFO_0_INT_ENABLE	Enable the WATCHDOG interrupt	0x1
RS_DEC_OUT_RDY	NVIC_RS_DEC_OUT_RDY_INT_ENABLE	Enable the interrupt	0x1
RS_DEC_IN_RDY	NVIC_RS_DEC_IN_RDY_INT_ENABLE	Enable the FIFO_7 interrupt	0x1
RS_ENC_OUT_RDY	NVIC_RS_ENC_OUT_RDY_INT_ENABLE	Enable the RS_ENC_OUT_RDY interrupt	0x1
RS_ENC_IN_RDY	NVIC_RS_ENC_IN_RDY_INT_ENABLE	Enable the RS_ENC_IN_RDY interrupt	0x1
G722_DEC_OUT_RDY	NVIC_G722_DEC_OUT_RDY_INT_ENABLE	Enable the G722_DEC_OUT_RDY interrupt	0x1
G722_DEC_IN_RDY	NVIC_G722_DEC_IN_RDY_INT_ENABLE	Enable the G722_DEC_IN_RDY interrupt	0x1
G722_ENC_OUT_RDY	NVIC_G722_ENC_OUT_RDY_INT_ENABLE	Enable the G722_ENC_OUT_RDY interrupt	0x1
G722_ENC_IN_RDY	NVIC_G722_ENC_IN_RDY_INT_ENABLE	Enable the G722_ENC_IN_RDY interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
ASRC_IDLE	NVIC_ASRC_IDLE_INT_ENABLE	Enable the ASRC_IDLE interrupt	0x1
ASRC_OUT	NVIC_ASRC_OUT_INT_ENABLE	Enable the ASRC_OUT interrupt	0x1
ASRC_IN	NVIC_ASRC_IN_INT_ENABLE	Enable the ASRC_IN interrupt	0x1
ASRC_ERROR	NVIC_ASRC_ERROR_INT_ENABLE	Enable the ASRC_ERROR interrupt	0x1
PCM_1_RX_TX	NVIC_PCM_1_RX_TX_INT_ENABLE	Enable the PCM_1_RX_TX interrupt	0x1
PCM_1_ERROR	NVIC_PCM_1_ERROR_INT_ENABLE	Enable the PCM_1_ERROR interrupt	0x1
PCM_0_RX_TX	NVIC_PCM_0_RX_TX_INT_ENABLE	Enable the PCM_0_RX_TX interrupt	0x1
PCM_0_ERROR	NVIC_PCM_0_ERROR_INT_ENABLE	Enable the PCM_0_ERROR interrupt	0x1
FENG	NVIC_FENG_INT_ENABLE	Enable the FENG interrupt	0x1
UART_0_TX	NVIC_UART_0_TX_INT_ENABLE	Enable the UART_0_TX interrupt	0x1
UART_0_RX	NVIC_UART_0_RX_INT_ENABLE	Enable the UART_0_RX interrupt	0x1
UART_0_ERROR	NVIC_UART_0_ERROR_INT_ENABLE	Enable the UART_0_ERROR interrupt	0x1
EMMC	NVIC_EMMC_INT_ENABLE	Enable the EMMC interrupt	0x1
SPI_1_TX	NVIC_SPI_1_TX_INT_ENABLE	Enable the SPI_1_TX interrupt	0x1
SPI_1_RX	NVIC_SPI_1_RX_INT_ENABLE	Enable the SPI_1_RX interrupt	0x1
SPI_1_COM	NVIC_SPI_1_COM_INT_ENABLE	Enable the SPI_1_COM interrupt	0x1

4.6.1.1.3 NVIC_ISER2

Bit Field	Read/Write	Field Name	Description
29	RW	RS_COM	RS_COM interrupt set enable
28	RW	G722_COM	G722_COM interrupt set enable
27	RW	MEM_ERROR	MEM_ERROR interrupt set enable
26	RW	LSAD	LSAD interrupt set enable
25	RW	CLK_DET	CLK_DET interrupt set enable
24	RW	NNA	NNA interrupt set enable
23	RW	CFX_7	CFX_7 interrupt set enable
22	RW	CFX_6	CFX_6 interrupt set enable
21	RW	CFX_5	CFX_5 interrupt set enable

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
20	RW	CFX_4	CFX_4 interrupt set enable
19	RW	CFX_3	CFX_3 interrupt set enable
18	RW	CFX_2	CFX_2 interrupt set enable
17	RW	CFX_1	CFX_1 interrupt set enable
16	RW	CFX_0	CFX_0 interrupt set enable
15	RW	DSP_7	DSP_7 interrupt set enable
14	RW	DSP_6	DSP_6 interrupt set enable
13	RW	DSP_5	DSP_5 interrupt set enable
12	RW	DSP_4	DSP_4 interrupt set enable
11	RW	DSP_3	DSP_3 interrupt set enable
10	RW	DSP_2	DSP_2 interrupt set enable
9	RW	DSP_1	DSP_1 interrupt set enable
8	RW	DSP_0	DSP_0 interrupt set enable
7	RW	HEAR_7	HEAR_7 interrupt set enable
6	RW	HEAR_6	HEAR_6 interrupt set enable
5	RW	HEAR_5	HEAR_5 interrupt set enable
4	RW	HEAR_4	HEAR_4 interrupt set enable
3	RW	HEAR_3	HEAR_3 interrupt set enable
2	RW	HEAR_2	HEAR_2 interrupt set enable
1	RW	HEAR_1	HEAR_1 interrupt set enable
0	RW	HEAR_0	HEAR_0 interrupt set enable

Field Name	Value Symbol	Value Description	Hex Value
RS_COM	NVIC_RS_COM_INT_ENABLE	Enable the RS_COM interrupt	0x1
G722_COM	NVIC_G722_COM_INT_ENABLE	Enable the G722_COM interrupt	0x1
MEM_ERROR	NVIC_MEM_ERROR_INT_ENABLE	Enable the MEM_ERROR interrupt	0x1
LSAD	NVIC_LSAD_INT_ENABLE	Enable the LSAD interrupt	0x1
CLK_DET	NVIC_CLK_DET_INT_ENABLE	Enable the CLK_DET interrupt	0x1
NNA	NVIC_NNA_INT_ENABLE	Enable the NNA interrupt	0x1
CFX_7	NVIC_CFX_7_INT_ENABLE	Enable the CFX_7 interrupt	0x1
CFX_6	NVIC_CFX_6_INT_ENABLE	Enable the CFX_6 interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
CFX_5	NVIC_CFX_5_INT_ENABLE	Enable the CFX_5 interrupt	0x1
CFX_4	NVIC_CFX_4_INT_ENABLE	Enable the CFX_4 interrupt	0x1
CFX_3	NVIC_CFX_3_INT_ENABLE	Enable the CFX_3 interrupt	0x1
CFX_2	NVIC_CFX_2_INT_ENABLE	Enable the CFX_2 interrupt	0x1
CFX_1	NVIC_CFX_1_INT_ENABLE	Enable the CFX_1 interrupt	0x1
CFX_0	NVIC_CFX_0_INT_ENABLE	Enable the CFX_0 interrupt	0x1
DSP_7	NVIC_DSP_7_INT_ENABLE	Enable the DSP_7 interrupt	0x1
DSP_6	NVIC_DSP_6_INT_ENABLE	Enable the DSP_6 interrupt	0x1
DSP_5	NVIC_DSP_5_INT_ENABLE	Enable the DSP_5 interrupt	0x1
DSP_4	NVIC_DSP_4_INT_ENABLE	Enable the DSP_4 interrupt	0x1
DSP_3	NVIC_DSP_3_INT_ENABLE	Enable the DSP_3 interrupt	0x1
DSP_2	NVIC_DSP_2_INT_ENABLE	Enable the DSP_2 interrupt	0x1
DSP_1	NVIC_DSP_1_INT_ENABLE	Enable the DSP_1 interrupt	0x1
DSP_0	NVIC_DSP_0_INT_ENABLE	Enable the DSP_0 interrupt	0x1
HEAR_7	NVIC_HEAR_7_INT_ENABLE	Enable the HEAR_7 interrupt	0x1
HEAR_6	NVIC_HEAR_6_INT_ENABLE	Enable the HEAR_6 interrupt	0x1
HEAR_5	NVIC_HEAR_5_INT_ENABLE	Enable the HEAR_5 interrupt	0x1
HEAR_4	NVIC_HEAR_4_INT_ENABLE	Enable the HEAR_4 interrupt	0x1
HEAR_3	NVIC_HEAR_3_INT_ENABLE	Enable the HEAR_3 interrupt	0x1
HEAR_2	NVIC_HEAR_2_INT_ENABLE	Enable the HEAR_2 interrupt	0x1
HEAR_1	NVIC_HEAR_1_INT_ENABLE	Enable the HEAR_1 interrupt	0x1
HEAR_0	NVIC_HEAR_0_INT_ENABLE	Enable the HEAR_0 interrupt	0x1

4.6.1.1.4 NVIC_ICER0

Bit Field	Read/Write	Field Name	Description
31	RW	SPI_0_TX	SPI_0_TX interrupt clear enable
30	RW	SPI_0_RX	SPI_0_RX interrupt clear enable
29	RW	SPI_0_COM	SPI_0_COM interrupt clear enable
28	RW	D_I2C	D_I2C interrupt clear enable
27	RW	I2C_1	I2C_1 interrupt clear enable
26	RW	I2C_0	I2C_0 interrupt clear enable
25	RW	I3C_1	I3C_1 interrupt clear enable

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
24	RW	I3C_0	I3C_0 interrupt clear enable
23	RW	DIO_7	DIO_7 interrupt clear enable
22	RW	DIO_6	DIO_6 interrupt clear enable
21	RW	DIO_5	DIO_5 interrupt clear enable
20	RW	DIO_4	DIO_4 interrupt clear enable
19	RW	DIO_3	DIO_3 interrupt clear enable
18	RW	DIO_2	DIO_2 interrupt clear enable
17	RW	DIO_1	DIO_1 interrupt clear enable
16	RW	DIO_0	DIO_0 interrupt clear enable
15	RW	DMA_7	DMA_7 interrupt clear enable
14	RW	DMA_6	DMA_6 interrupt clear enable
13	RW	DMA_5	DMA_5 interrupt clear enable
12	RW	DMA_4	DMA_4 interrupt clear enable
11	RW	DMA_3	DMA_3 interrupt clear enable
10	RW	DMA_2	DMA_2 interrupt clear enable
9	RW	DMA_1	DMA_1 interrupt clear enable
8	RW	DMA_0	DMA_0 interrupt clear enable
7	RW	TIMER_3	TIMER_3 interrupt clear enable
6	RW	TIMER_2	TIMER_2 interrupt clear enable
5	RW	TIMER_1	TIMER_1 interrupt clear enable
4	RW	TIMER_0	TIMER_0 interrupt clear enable
3	RW	AUDIOSINK_1	AUDIOSINK_1 interrupt clear enable
2	RW	AUDIOSINK_0	AUDIOSINK_0 interrupt clear enable
1	RW	LSAD_BATMON	LSAD_BATMON interrupt clear enable
0	RW	WATCHDOG	WATCHDOG interrupt clear enable

Field Name	Value Symbol	Value Description	Hex Value
SPI_0_TX	NVIC_SPI_0_TX_INT_DISABLE	Disable the SPI_0_TX interrupt	0x1
SPI_0_RX	NVIC_SPI_0_RX_INT_DISABLE	Disable the SPI_0_RX interrupt	0x1
SPI_0_COM	NVIC_SPI_0_COM_INT_DISABLE	Disable the SPI_0_COM interrupt	0x1
D_I2C	NVIC_D_I2C_INT_DISABLE	Disable the D_I2C interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
I2C_1	NVIC_I2C_1_INT_DISABLE	Disable the I2C_1 interrupt	0x1
I2C_0	NVIC_I2C_0_INT_DISABLE	Disable the I2C_0 interrupt	0x1
I3C_1	NVIC_I3C_1_INT_DISABLE	Disable the I3C_1 interrupt	0x1
I3C_0	NVIC_I3C_0_INT_DISABLE	Disable the I3C_0 interrupt	0x1
DIO_7	NVIC_DIO_7_INT_DISABLE	Disable the DIO_7 interrupt	0x1
DIO_6	NVIC_DIO_6_INT_DISABLE	Disable the DIO_6 interrupt	0x1
DIO_5	NVIC_DIO_5_INT_DISABLE	Disable the DIO_5 interrupt	0x1
DIO_4	NVIC_DIO_4_INT_DISABLE	Disable the DIO_4 interrupt	0x1
DIO_3	NVIC_DIO_3_INT_DISABLE	Disable the DIO_3 interrupt	0x1
DIO_2	NVIC_DIO_2_INT_DISABLE	Disable the DIO_2 interrupt	0x1
DIO_1	NVIC_DIO_1_INT_DISABLE	Disable the DIO_1 interrupt	0x1
DIO_0	NVIC_DIO_0_INT_DISABLE	Disable the DIO_0 interrupt	0x1
DMA_7	NVIC_DMA_7_INT_DISABLE	Disable the DMA_7 interrupt	0x1
DMA_6	NVIC_DMA_6_INT_DISABLE	Disable the DMA_6 interrupt	0x1
DMA_5	NVIC_DMA_5_INT_DISABLE	Disable the DMA_5 interrupt	0x1
DMA_4	NVIC_DMA_4_INT_DISABLE	Disable the DMA_4 interrupt	0x1
DMA_3	NVIC_DMA_3_INT_DISABLE	Disable the DMA_3 interrupt	0x1
DMA_2	NVIC_DMA_2_INT_DISABLE	Disable the DMA_2 interrupt	0x1
DMA_1	NVIC_DMA_1_INT_DISABLE	Disable the DMA_1 interrupt	0x1
DMA_0	NVIC_DMA_0_INT_DISABLE	Disable the DMA_0 interrupt	0x1
TIMER_3	NVIC_TIMER_3_INT_DISABLE	Disable the TIMER_3 interrupt	0x1
TIMER_2	NVIC_TIMER_2_INT_DISABLE	Disable the TIMER_2 interrupt	0x1
TIMER_1	NVIC_TIMER_1_INT_DISABLE	Disable the TIMER_1 interrupt	0x1
TIMER_0	NVIC_TIMER_0_INT_DISABLE	Disable the TIMER_0 interrupt	0x1
AUDIOSINK_1	NVIC_AUDIOSINK_1_INT_DISABLE	Disable the AUDIOSINK_1 interrupt	0x1
AUDIOSINK_0	NVIC_AUDIOSINK_0_INT_DISABLE	Disable the AUDIOSINK_0 interrupt	0x1
LSAD_BATMON	NVIC_LSAD_BATMON_INT_DISABLE	Disable the LSAD_BATMON interrupt	0x1
WATCHDOG	NVIC_WATCHDOG_INT_DISABLE	Disable the WATCHDOG interrupt	0x1

Ezairo 8300 Hardware Reference

4.6.1.1.5 NVIC_ICER1

Bit Field	Read/Write	Field Name	Description
31	RW	FIFO_7	FIFO_7 interrupt clear enable
30	RW	FIFO_6	FIFO_6 interrupt clear enable
29	RW	FIFO_5	FIFO_5 interrupt clear enable
28	RW	FIFO_4	FIFO_4 interrupt clear enable
27	RW	FIFO_3	FIFO_3 interrupt clear enable
26	RW	FIFO_2	FIFO_2 interrupt clear enable
25	RW	FIFO_1	FIFO_1 interrupt clear enable
24	RW	FIFO_0	FIFO_0 interrupt clear enable
23	RW	RS_DEC_OUT_RDY	RS_DEC_OUT_RDY interrupt clear enable
22	RW	RS_DEC_IN_RDY	RS_DEC_IN_RDY interrupt clear enable
21	RW	RS_ENC_OUT_RDY	RS_ENC_OUT_RDY interrupt clear enable
20	RW	RS_ENC_IN_RDY	RS_ENC_IN_RDY interrupt clear enable
19	RW	G722_DEC_OUT_RDY	G722_DEC_OUT_RDY interrupt clear enable
18	RW	G722_DEC_IN_RDY	G722_DEC_IN_RDY interrupt clear enable
17	RW	G722_ENC_OUT_RDY	G722_ENC_OUT_RDY interrupt clear enable
16	RW	G722_ENC_IN_RDY	G722_ENC_IN_RDY interrupt clear enable
15	RW	ASRC_IDLE	ASRC_IDLE interrupt clear enable
14	RW	ASRC_OUT	ASRC_OUT interrupt clear enable
13	RW	ASRC_IN	ASRC_IN interrupt clear enable
12	RW	ASRC_ERROR	ASRC_ERROR interrupt clear enable
11	RW	PCM_1_RX_TX	PCM_1_RX_TX interrupt clear enable
10	RW	PCM_1_ERROR	PCM_1_ERROR interrupt clear enable
9	RW	PCM_0_RX_TX	PCM_0_RX_TX interrupt clear enable
8	RW	PCM_0_ERROR	PCM_0_ERROR interrupt clear enable
7	RW	FENG	FENG interrupt clear enable
6	RW	UART_0_TX	UART_0_TX interrupt clear enable
5	RW	UART_0_RX	UART_0_RX interrupt clear enable
4	RW	UART_0_ERROR	UART_0_ERROR interrupt clear enable
3	RW	EMMC	EMMC interrupt clear enable

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
2	RW	SPI_1_TX	SPI_1_TX interrupt clear enable
1	RW	SPI_1_RX	SPI_1_RX interrupt clear enable
0	RW	SPI_1_COM	SPI_1_COM interrupt clear enable

Field Name	Value Symbol	Value Description	Hex Value
FIFO_7	NVIC_FIFO_7_INT_DISABLE	Disable the FIFO_7 interrupt	0x1
FIFO_6	NVIC_FIFO_6_INT_DISABLE	Disable the FIFO_6 interrupt	0x1
FIFO_5	NVIC_FIFO_5_INT_DISABLE	Disable the FIFO_5 interrupt	0x1
FIFO_4	NVIC_FIFO_4_INT_DISABLE	Disable the FIFO_4 interrupt	0x1
FIFO_3	NVIC_FIFO_3_INT_DISABLE	Disable the FIFO_3 interrupt	0x1
FIFO_2	NVIC_FIFO_2_INT_DISABLE	Disable the FIFO_2 interrupt	0x1
FIFO_1	NVIC_FIFO_1_INT_DISABLE	Disable the FIFO_1 interrupt	0x1
FIFO_0	NVIC_FIFO_0_INT_DISABLE	Disable the FIFO_0 interrupt	0x1
RS_DEC_OUT_RDY	NVIC_RS_DEC_OUT_RDY_INT_DISABLE	Disable the RS_DEC_OUT_RDY interrupt	0x1
RS_DEC_IN_RDY	NVIC_RS_DEC_IN_RDY_INT_DISABLE	Disable the RS_DEC_IN_RDY interrupt	0x1
RS_ENC_OUT_RDY	NVIC_RS_ENC_OUT_RDY_INT_DISABLE	Disable the RS_ENC_OUT_RDY interrupt	0x1
RS_ENC_IN_RDY	NVIC_RS_ENC_IN_RDY_INT_DISABLE	Disable the RS_ENC_IN_RDY interrupt	0x1
G722_DEC_OUT_RDY	NVIC_G722_DEC_OUT_RDY_INT_DISABLE	Disable the G722_DEC_OUT_RDY interrupt	0x1
G722_DEC_IN_RDY	NVIC_G722_DEC_IN_RDY_INT_DISABLE	Disable the G722_DEC_IN_RDY interrupt	0x1
G722_ENC_OUT_RDY	NVIC_G722_ENC_OUT_RDY_INT_DISABLE	Disable the G722_ENC_OUT_RDY interrupt	0x1
G722_ENC_IN_RDY	NVIC_G722_ENC_IN_RDY_INT_DISABLE	Disable the G722_ENC_IN_RDY interrupt	0x1
ASRC_IDLE	NVIC_ASRC_IDLE_INT_DISABLE	Disable the ASRC_IDLE interrupt	0x1
ASRC_OUT	NVIC_ASRC_OUT_INT_DISABLE	Disable the ASRC_OUT interrupt	0x1
ASRC_IN	NVIC_ASRC_IN_INT_DISABLE	Disable the ASRC_IN interrupt	0x1
ASRC_ERROR	NVIC_ASRC_ERROR_INT_DISABLE	Disable the ASRC_ERROR interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
PCM_1_RX_TX	NVIC_PCM_1_RX_TX_INT_DISABLE	Disable the PCM_1_RX_TX interrupt	0x1
PCM_1_ERROR	NVIC_PCM_1_ERROR_INT_DISABLE	Disable the PCM_1_ERROR interrupt	0x1
PCM_0_RX_TX	NVIC_PCM_0_RX_TX_INT_DISABLE	Disable the PCM_0_RX_TX interrupt	0x1
PCM_0_ERROR	NVIC_PCM_0_ERROR_INT_DISABLE	Disable the PCM_0_ERROR interrupt	0x1
FENG	NVIC_FENG_INT_DISABLE	Disable the FENG interrupt	0x1
UART_0_TX	NVIC_UART_0_TX_INT_DISABLE	Disable the UART_0_TX interrupt	0x1
UART_0_RX	NVIC_UART_0_RX_INT_DISABLE	Disable the UART_0_RX interrupt	0x1
UART_0_ERROR	NVIC_UART_0_ERROR_INT_DISABLE	Disable the UART_0_ERROR interrupt	0x1
EMMC	NVIC_EMMC_INT_DISABLE	Disable the EMMC interrupt	0x1
SPI_1_TX	NVIC_SPI_1_TX_INT_DISABLE	Disable the SPI_1_TX interrupt	0x1
SPI_1_RX	NVIC_SPI_1_RX_INT_DISABLE	Disable the SPI_1_RX interrupt	0x1
SPI_1_COM	NVIC_SPI_1_COM_INT_DISABLE	Disable the SPI_1_COM interrupt	0x1

4.6.1.1.6 NVIC_ICER2

Bit Field	Read/Write	Field Name	Description
29	RW	RS_COM	RS_COM interrupt clear enable
28	RW	G722_COM	G722_COM interrupt clear enable
27	RW	MEM_ERROR	MEM_ERROR interrupt clear enable
26	RW	LSAD	LSAD interrupt clear enable
25	RW	CLK_DET	CLK_DET interrupt clear enable
24	RW	NNA	NNA interrupt clear enable
23	RW	CFX_7	CFX_7 interrupt clear enable
22	RW	CFX_6	CFX_6 interrupt clear enable
21	RW	CFX_5	CFX_5 interrupt clear enable
20	RW	CFX_4	CFX_4 interrupt clear enable
19	RW	CFX_3	CFX_3 interrupt clear enable
18	RW	CFX_2	CFX_2 interrupt clear enable
17	RW	CFX_1	CFX_1 interrupt clear enable

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
16	RW	CFX_0	CFX_0 interrupt clear enable
15	RW	DSP_7	DSP_7 interrupt clear enable
14	RW	DSP_6	DSP_6 interrupt clear enable
13	RW	DSP_5	DSP_5 interrupt clear enable
12	RW	DSP_4	DSP_4 interrupt clear enable
11	RW	DSP_3	DSP_3 interrupt clear enable
10	RW	DSP_2	DSP_2 interrupt clear enable
9	RW	DSP_1	DSP_1 interrupt clear enable
8	RW	DSP_0	DSP_0 interrupt clear enable
7	RW	HEAR_7	HEAR_7 interrupt clear enable
6	RW	HEAR_6	HEAR_6 interrupt clear enable
5	RW	HEAR_5	HEAR_5 interrupt clear enable
4	RW	HEAR_4	HEAR_4 interrupt clear enable
3	RW	HEAR_3	HEAR_3 interrupt clear enable
2	RW	HEAR_2	HEAR_2 interrupt clear enable
1	RW	HEAR_1	HEAR_1 interrupt clear enable
0	RW	HEAR_0	HEAR_0 interrupt clear enable

Field Name	Value Symbol	Value Description	Hex Value
RS_COM	NVIC_RS_COM_INT_DISABLE	Disable the RS_COM interrupt	0x1
G722_COM	NVIC_G722_COM_INT_DISABLE	Disable the G722_COM interrupt	0x1
MEM_ERROR	NVIC_MEM_ERROR_INT_DISABLE	Disable the MEM_ERROR interrupt	0x1
LSAD	NVIC_LSAD_INT_DISABLE	Disable the LSAD interrupt	0x1
CLK_DET	NVIC_CLK_DET_INT_DISABLE	Disable the CLK_DET interrupt	0x1
NNA	NVIC_NNA_INT_DISABLE	Disable the NNA interrupt	0x1
CFX_7	NVIC_CFX_7_INT_DISABLE	Disable the CFX_7 interrupt	0x1
CFX_6	NVIC_CFX_6_INT_DISABLE	Disable the CFX_6 interrupt	0x1
CFX_5	NVIC_CFX_5_INT_DISABLE	Disable the CFX_5 interrupt	0x1
CFX_4	NVIC_CFX_4_INT_DISABLE	Disable the CFX_4 interrupt	0x1
CFX_3	NVIC_CFX_3_INT_DISABLE	Disable the CFX_3 interrupt	0x1
CFX_2	NVIC_CFX_2_INT_DISABLE	Disable the CFX_2 interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
CFX_1	NVIC_CFX_1_INT_DISABLE	Disable the CFX_1 interrupt	0x1
CFX_0	NVIC_CFX_0_INT_DISABLE	Disable the CFX_0 interrupt	0x1
DSP_7	NVIC_DSP_7_INT_DISABLE	Disable the DSP_7 interrupt	0x1
DSP_6	NVIC_DSP_6_INT_DISABLE	Disable the DSP_6 interrupt	0x1
DSP_5	NVIC_DSP_5_INT_DISABLE	Disable the DSP_5 interrupt	0x1
DSP_4	NVIC_DSP_4_INT_DISABLE	Disable the DSP_4 interrupt	0x1
DSP_3	NVIC_DSP_3_INT_DISABLE	Disable the DSP_3 interrupt	0x1
DSP_2	NVIC_DSP_2_INT_DISABLE	Disable the DSP_2 interrupt	0x1
DSP_1	NVIC_DSP_1_INT_DISABLE	Disable the DSP_1 interrupt	0x1
DSP_0	NVIC_DSP_0_INT_DISABLE	Disable the DSP_0 interrupt	0x1
HEAR_7	NVIC_HEAR_7_INT_DISABLE	Disable the HEAR_7 interrupt	0x1
HEAR_6	NVIC_HEAR_6_INT_DISABLE	Disable the HEAR_6 interrupt	0x1
HEAR_5	NVIC_HEAR_5_INT_DISABLE	Disable the HEAR_5 interrupt	0x1
HEAR_4	NVIC_HEAR_4_INT_DISABLE	Disable the HEAR_4 interrupt	0x1
HEAR_3	NVIC_HEAR_3_INT_DISABLE	Disable the HEAR_3 interrupt	0x1
HEAR_2	NVIC_HEAR_2_INT_DISABLE	Disable the HEAR_2 interrupt	0x1
HEAR_1	NVIC_HEAR_1_INT_DISABLE	Disable the HEAR_1 interrupt	0x1
HEAR_0	NVIC_HEAR_0_INT_DISABLE	Disable the HEAR_0 interrupt	0x1

4.6.1.1.7 NVIC_ISPR0

Bit Field	Read/Write	Field Name	Description
31	RW	SPI_0_TX	SPI_0_TX interrupt set pending
30	RW	SPI_0_RX	SPI_0_RX interrupt set pending
29	RW	SPI_0_COM	SPI_0_COM interrupt set pending
28	RW	D_I2C	D_I2C interrupt set pending
27	RW	I2C_1	I2C_1 interrupt set pending
26	RW	I2C_0	I2C_0 interrupt set pending
25	RW	I3C_1	I3C_1 interrupt set pending
24	RW	I3C_0	I3C_0 interrupt set pending
23	RW	DIO_7	DIO_7 interrupt set pending
22	RW	DIO_6	DIO_6 interrupt set pending
21	RW	DIO_5	DIO_5 interrupt set pending

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
20	RW	DIO_4	DIO_4 interrupt set pending
19	RW	DIO_3	DIO_3 interrupt set pending
18	RW	DIO_2	DIO_2 interrupt set pending
17	RW	DIO_1	DIO_1 interrupt set pending
16	RW	DIO_0	DIO_0 interrupt set pending
15	RW	DMA_7	DMA_7 interrupt set pending
14	RW	DMA_6	DMA_6 interrupt set pending
13	RW	DMA_5	DMA_5 interrupt set pending
12	RW	DMA_4	DMA_4 interrupt set pending
11	RW	DMA_3	DMA_3 interrupt set pending
10	RW	DMA_2	DMA_2 interrupt set pending
9	RW	DMA_1	DMA_1 interrupt set pending
8	RW	DMA_0	DMA_0 interrupt set pending
7	RW	TIMER_3	TIMER_3 interrupt set pending
6	RW	TIMER_2	TIMER_2 interrupt set pending
5	RW	TIMER_1	TIMER_1 interrupt set pending
4	RW	TIMER_0	TIMER_0 interrupt set pending
3	RW	AUDIOSINK_1	AUDIOSINK_1 interrupt set pending
2	RW	AUDIOSINK_0	AUDIOSINK_0 interrupt set pending
1	RW	LSAD_BATMON	LSAD_BATMON interrupt set pending
0	RW	WATCHDOG	WATCHDOG interrupt set pending

Field Name	Value Symbol	Value Description	Hex Value
SPI_0_TX	NVIC_SPI_0_TX_INT_SETPEND	Set the SPI_0_TX interrupt pending	0x1
SPI_0_RX	NVIC_SPI_0_RX_INT_SETPEND	Set the SPI_0_RX interrupt pending	0x1
SPI_0_COM	NVIC_SPI_0_COM_INT_SETPEND	Set the SPI_0_COM interrupt pending	0x1
D_I2C	NVIC_D_I2C_INT_SETPEND	Set the D_I2C interrupt pending	0x1
I2C_1	NVIC_I2C_1_INT_SETPEND	Set the I2C_1 interrupt pending	0x1
I2C_0	NVIC_I2C_0_INT_SETPEND	Set the I2C_0 interrupt pending	0x1
I3C_1	NVIC_I3C_1_INT_SETPEND	Set the I3C_1 interrupt pending	0x1
I3C_0	NVIC_I3C_0_INT_SETPEND	Set the I3C_0 interrupt pending	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DIO_7	NVIC_DIO_7_INT_SETPEND	Set the DIO_7 interrupt pending	0x1
DIO_6	NVIC_DIO_6_INT_SETPEND	Set the DIO_6 interrupt pending	0x1
DIO_5	NVIC_DIO_5_INT_SETPEND	Set the DIO_5 interrupt pending	0x1
DIO_4	NVIC_DIO_4_INT_SETPEND	Set the DIO_4 interrupt pending	0x1
DIO_3	NVIC_DIO_3_INT_SETPEND	Set the DIO_3 interrupt pending	0x1
DIO_2	NVIC_DIO_2_INT_SETPEND	Set the DIO_2 interrupt pending	0x1
DIO_1	NVIC_DIO_1_INT_SETPEND	Set the DIO_1 interrupt pending	0x1
DIO_0	NVIC_DIO_0_INT_SETPEND	Set the DIO_0 interrupt pending	0x1
DMA_7	NVIC_DMA_7_INT_SETPEND	Set the DMA_7 interrupt pending	0x1
DMA_6	NVIC_DMA_6_INT_SETPEND	Set the DMA_6 interrupt pending	0x1
DMA_5	NVIC_DMA_5_INT_SETPEND	Set the DMA_5 interrupt pending	0x1
DMA_4	NVIC_DMA_4_INT_SETPEND	Set the DMA_4 interrupt pending	0x1
DMA_3	NVIC_DMA_3_INT_SETPEND	Set the DMA_3 interrupt pending	0x1
DMA_2	NVIC_DMA_2_INT_SETPEND	Set the DMA_2 interrupt pending	0x1
DMA_1	NVIC_DMA_1_INT_SETPEND	Set the DMA_1 interrupt pending	0x1
DMA_0	NVIC_DMA_0_INT_SETPEND	Set the DMA_0 interrupt pending	0x1
TIMER_3	NVIC_TIMER_3_INT_SETPEND	Set the TIMER_3 interrupt pending	0x1
TIMER_2	NVIC_TIMER_2_INT_SETPEND	Set the TIMER_2 interrupt pending	0x1
TIMER_1	NVIC_TIMER_1_INT_SETPEND	Set the TIMER_1 interrupt pending	0x1
TIMER_0	NVIC_TIMER_0_INT_SETPEND	Set the TIMER_0 interrupt pending	0x1
AUDIOSINK_1	NVIC_AUDIOSINK_1_INT_SETPEND	Set the AUDIOSINK_1 interrupt pending	0x1
AUDIOSINK_0	NVIC_AUDIOSINK_0_INT_SETPEND	Set the AUDIOSINK_0 interrupt pending	0x1
LSAD_BATMON	NVIC_LSAD_BATMON_INT_SETPEND	Set the LSAD_BATMON interrupt pending	0x1
WATCHDOG	NVIC_WATCHDOG_INT_SETPEND	Set the WATCHDOG interrupt pending	0x1

4.6.1.1.8 NVIC_ISPR1

Bit Field	Read/Write	Field Name	Description
31	RW	FIFO_7	FIFO_7 interrupt set pending
30	RW	FIFO_6	FIFO_6 interrupt set pending
29	RW	FIFO_5	FIFO_5 interrupt set pending

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
28	RW	FIFO_4	FIFO_4 interrupt set pending
27	RW	FIFO_3	FIFO_3 interrupt set pending
26	RW	FIFO_2	FIFO_2 interrupt set pending
25	RW	FIFO_1	FIFO_1 interrupt set pending
24	RW	FIFO_0	FIFO_0 interrupt set pending
23	RW	RS_DEC_OUT_RDY	RS_DEC_OUT_RDY interrupt set pending
22	RW	RS_DEC_IN_RDY	RS_DEC_IN_RDY interrupt set pending
21	RW	RS_ENC_OUT_RDY	RS_ENC_OUT_RDY interrupt set pending
20	RW	RS_ENC_IN_RDY	RS_ENC_IN_RDY interrupt set pending
19	RW	G722_DEC_OUT_RDY	G722_DEC_OUT_RDY interrupt set pending
18	RW	G722_DEC_IN_RDY	G722_DEC_IN_RDY interrupt set pending
17	RW	G722_ENC_OUT_RDY	G722_ENC_OUT_RDY interrupt set pending
16	RW	G722_ENC_IN_RDY	G722_ENC_IN_RDY interrupt set pending
15	RW	ASRC_IDLE	ASRC_IDLE interrupt set pending
14	RW	ASRC_OUT	ASRC_OUT interrupt set pending
13	RW	ASRC_IN	ASRC_IN interrupt set pending
12	RW	ASRC_ERROR	ASRC_ERROR interrupt set pending
11	RW	PCM_1_RX_TX	PCM_1_RX_TX interrupt set pending
10	RW	PCM_1_ERROR	PCM_1_ERROR interrupt set pending
9	RW	PCM_0_RX_TX	PCM_0_RX_TX interrupt set pending
8	RW	PCM_0_ERROR	PCM_0_ERROR interrupt set pending
7	RW	FENG	FENG interrupt set pending
6	RW	UART_0_TX	UART_0_TX interrupt set pending
5	RW	UART_0_RX	UART_0_RX interrupt set pending
4	RW	UART_0_ERROR	UART_0_ERROR interrupt set pending
3	RW	EMMC	EMMC interrupt set pending
2	RW	SPI_1_TX	SPI_1_TX interrupt set pending
1	RW	SPI_1_RX	SPI_1_RX interrupt set pending
0	RW	SPI_1_COM	SPI_1_COM interrupt set pending

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
FIFO_7	NVIC_FIFO_7_INT_SETPEND	Set the FIFO_7 interrupt pending	0x1
FIFO_6	NVIC_FIFO_6_INT_SETPEND	Set the FIFO_6 interrupt pending	0x1
FIFO_5	NVIC_FIFO_5_INT_SETPEND	Set the FIFO_5 interrupt pending	0x1
FIFO_4	NVIC_FIFO_4_INT_SETPEND	Set the FIFO_4 interrupt pending	0x1
FIFO_3	NVIC_FIFO_3_INT_SETPEND	Set the FIFO_3 interrupt pending	0x1
FIFO_2	NVIC_FIFO_2_INT_SETPEND	Set the FIFO_2 interrupt pending	0x1
FIFO_1	NVIC_FIFO_1_INT_SETPEND	Set the FIFO_1 interrupt pending	0x1
FIFO_0	NVIC_FIFO_0_INT_SETPEND	Set the FIFO_0 interrupt pending	0x1
RS_DEC_OUT_RDY	NVIC_RS_DEC_OUT_RDY_INT_SETPEND	Set the RS_DEC_OUT_RDY interrupt pending	0x1
RS_DEC_IN_RDY	NVIC_RS_DEC_IN_RDY_INT_SETPEND	Set the RS_DEC_IN_RDY interrupt pending	0x1
RS_ENC_OUT_RDY	NVIC_RS_ENC_OUT_RDY_INT_SETPEND	Set the RS_ENC_OUT_RDY interrupt pending	0x1
RS_ENC_IN_RDY	NVIC_RS_ENC_IN_RDY_INT_SETPEND	Set the RS_ENC_IN_RDY interrupt pending	0x1
G722_DEC_OUT_RDY	NVIC_G722_DEC_OUT_RDY_INT_SETPEND	Set the G722_DEC_OUT_RDY interrupt pending	0x1
G722_DEC_IN_RDY	NVIC_G722_DEC_IN_RDY_INT_SETPEND	Set the G722_DEC_IN_RDY interrupt pending	0x1
G722_ENC_OUT_RDY	NVIC_G722_ENC_OUT_RDY_INT_SETPEND	Set the G722_ENC_OUT_RDY interrupt pending	0x1
G722_ENC_IN_RDY	NVIC_G722_ENC_IN_RDY_INT_SETPEND	Set the G722_ENC_IN_RDY interrupt pending	0x1
ASRC_IDLE	NVIC_ASRC_IDLE_INT_SETPEND	Set the ASRC_IDLE interrupt pending	0x1
ASRC_OUT	NVIC_ASRC_OUT_INT_SETPEND	Set the ASRC_OUT interrupt pending	0x1
ASRC_IN	NVIC_ASRC_IN_INT_SETPEND	Set the ASRC_IN interrupt pending	0x1
ASRC_ERROR	NVIC_ASRC_ERROR_INT_SETPEND	Set the ASRC_ERROR interrupt pending	0x1
PCM_1_RX_TX	NVIC_PCM_1_RX_TX_INT_SETPEND	Set the PCM_1_RX_TX interrupt pending	0x1
PCM_1_ERROR	NVIC_PCM_1_ERROR_INT_SETPEND	Set the PCM_1_ERROR interrupt pending	0x1
PCM_0_RX_TX	NVIC_PCM_0_RX_TX_INT_SETPEND	Set the PCM_0_RX_TX interrupt pending	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
PCM_0_ERROR	NVIC_PCM_0_ERROR_INT_SETPEND	Set the PCM_0_ERROR interrupt pending	0x1
FENG	NVIC_FENG_INT_SETPEND	Set the FENG interrupt pending	0x1
UART_0_TX	NVIC_UART_0_TX_INT_SETPEND	Set the UART_0_TX interrupt pending	0x1
UART_0_RX	NVIC_UART_0_RX_INT_SETPEND	Set the UART_0_RX interrupt pending	0x1
UART_0_ERROR	NVIC_UART_0_ERROR_INT_SETPEND	Set the UART_0_ERROR interrupt pending	0x1
EMMC	NVIC_EMMC_INT_SETPEND	Set the EMMC interrupt pending	0x1
SPI_1_TX	NVIC_SPI_1_TX_INT_SETPEND	Set the SPI_1_TX interrupt pending	0x1
SPI_1_RX	NVIC_SPI_1_RX_INT_SETPEND	Set the SPI_1_RX interrupt pending	0x1
SPI_1_COM	NVIC_SPI_1_COM_INT_SETPEND	Set the SPI_1_COM interrupt pending	0x1

4.6.1.1.9 NVIC_ISPR2

Bit Field	Read/Write	Field Name	Description
29	RW	RS_COM	RS_COM interrupt set pending
28	RW	G722_COM	G722_COM interrupt set pending
27	RW	MEM_ERROR	MEM_ERROR interrupt set pending
26	RW	LSAD	LSAD interrupt set pending
25	RW	CLK_DET	CLK_DET interrupt set pending
24	RW	NNA	NNA interrupt set pending
23	RW	CFX_7	CFX_7 interrupt set pending
22	RW	CFX_6	CFX_6 interrupt set pending
21	RW	CFX_5	CFX_5 interrupt set pending
20	RW	CFX_4	CFX_4 interrupt set pending
19	RW	CFX_3	CFX_3 interrupt set pending
18	RW	CFX_2	CFX_2 interrupt set pending
17	RW	CFX_1	CFX_1 interrupt set pending
16	RW	CFX_0	CFX_0 interrupt set pending
15	RW	DSP_7	DSP_7 interrupt set pending
14	RW	DSP_6	DSP_6 interrupt set pending
13	RW	DSP_5	DSP_5 interrupt set pending
12	RW	DSP_4	DSP_4 interrupt set pending

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
11	RW	DSP_3	DSP_3 interrupt set pending
10	RW	DSP_2	DSP_2 interrupt set pending
9	RW	DSP_1	DSP_1 interrupt set pending
8	RW	DSP_0	DSP_0 interrupt set pending
7	RW	HEAR_7	HEAR_7 interrupt set pending
6	RW	HEAR_6	HEAR_6 interrupt set pending
5	RW	HEAR_5	HEAR_5 interrupt set pending
4	RW	HEAR_4	HEAR_4 interrupt set pending
3	RW	HEAR_3	HEAR_3 interrupt set pending
2	RW	HEAR_2	HEAR_2 interrupt set pending
1	RW	HEAR_1	HEAR_1 interrupt set pending
0	RW	HEAR_0	HEAR_0 interrupt set pending

Field Name	Value Symbol	Value Description	Hex Value
RS_COM	NVIC_RS_COM_INT_SETPEND	Set the RS_COM interrupt pending	0x1
G722_COM	NVIC_G722_COM_INT_SETPEND	Set the G722_COM interrupt pending	0x1
MEM_ERROR	NVIC_MEM_ERROR_INT_SETPEND	Set the MEM_ERROR interrupt pending	0x1
LSAD	NVIC_LSAD_INT_SETPEND	Set the LSAD interrupt pending	0x1
CLK_DET	NVIC_CLK_DET_INT_SETPEND	Set the CLK_DET interrupt pending	0x1
NNA	NVIC_NNA_INT_SETPEND	Set the NNA interrupt pending	0x1
CFX_7	NVIC_CFX_7_INT_SETPEND	Set the CFX_7 interrupt pending	0x1
CFX_6	NVIC_CFX_6_INT_SETPEND	Set the CFX_6 interrupt pending	0x1
CFX_5	NVIC_CFX_5_INT_SETPEND	Set the CFX_5 interrupt pending	0x1
CFX_4	NVIC_CFX_4_INT_SETPEND	Set the CFX_4 interrupt pending	0x1
CFX_3	NVIC_CFX_3_INT_SETPEND	Set the CFX_3 interrupt pending	0x1
CFX_2	NVIC_CFX_2_INT_SETPEND	Set the CFX_2 interrupt pending	0x1
CFX_1	NVIC_CFX_1_INT_SETPEND	Set the CFX_1 interrupt pending	0x1
CFX_0	NVIC_CFX_0_INT_SETPEND	Set the CFX_0 interrupt pending	0x1
DSP_7	NVIC_DSP_7_INT_SETPEND	Set the DSP_7 interrupt pending	0x1
DSP_6	NVIC_DSP_6_INT_SETPEND	Set the DSP_6 interrupt pending	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DSP_5	NVIC_DSP_5_INT_SETPEND	Set the DSP_5 interrupt pending	0x1
DSP_4	NVIC_DSP_4_INT_SETPEND	Set the DSP_4 interrupt pending	0x1
DSP_3	NVIC_DSP_3_INT_SETPEND	Set the DSP_3 interrupt pending	0x1
DSP_2	NVIC_DSP_2_INT_SETPEND	Set the DSP_2 interrupt pending	0x1
DSP_1	NVIC_DSP_1_INT_SETPEND	Set the DSP_1 interrupt pending	0x1
DSP_0	NVIC_DSP_0_INT_SETPEND	Set the DSP_0 interrupt pending	0x1
HEAR_7	NVIC_HEAR_7_INT_SETPEND	Set the HEAR_7 interrupt pending	0x1
HEAR_6	NVIC_HEAR_6_INT_SETPEND	Set the HEAR_6 interrupt pending	0x1
HEAR_5	NVIC_HEAR_5_INT_SETPEND	Set the HEAR_5 interrupt pending	0x1
HEAR_4	NVIC_HEAR_4_INT_SETPEND	Set the HEAR_4 interrupt pending	0x1
HEAR_3	NVIC_HEAR_3_INT_SETPEND	Set the HEAR_3 interrupt pending	0x1
HEAR_2	NVIC_HEAR_2_INT_SETPEND	Set the HEAR_2 interrupt pending	0x1
HEAR_1	NVIC_HEAR_1_INT_SETPEND	Set the HEAR_1 interrupt pending	0x1
HEAR_0	NVIC_HEAR_0_INT_SETPEND	Set the HEAR_0 interrupt pending	0x1

4.6.1.1.10 NVIC_ICPR0

Bit Field	Read/Write	Field Name	Description
31	RW	SPI_0_TX	SPI_0_TX interrupt clear pending
30	RW	SPI_0_RX	SPI_0_RX interrupt clear pending
29	RW	SPI_0_COM	SPI_0_COM interrupt clear pending
28	RW	D_I2C	D_I2C interrupt clear pending
27	RW	I2C_1	I2C_1 interrupt clear pending
26	RW	I2C_0	I2C_0 interrupt clear pending
25	RW	I3C_1	I3C_1 interrupt clear pending
24	RW	I3C_0	I3C_0 interrupt clear pending
23	RW	DIO_7	DIO_7 interrupt clear pending
22	RW	DIO_6	DIO_6 interrupt clear pending
21	RW	DIO_5	DIO_5 interrupt clear pending
20	RW	DIO_4	DIO_4 interrupt clear pending
19	RW	DIO_3	DIO_3 interrupt clear pending
18	RW	DIO_2	DIO_2 interrupt clear pending
17	RW	DIO_1	DIO_1 interrupt clear pending

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
16	RW	DIO_0	DIO_0 interrupt clear pending
15	RW	DMA_7	DMA_7 interrupt clear pending
14	RW	DMA_6	DMA_6 interrupt clear pending
13	RW	DMA_5	DMA_5 interrupt clear pending
12	RW	DMA_4	DMA_4 interrupt clear pending
11	RW	DMA_3	DMA_3 interrupt clear pending
10	RW	DMA_2	DMA_2 interrupt clear pending
9	RW	DMA_1	DMA_1 interrupt clear pending
8	RW	DMA_0	DMA_0 interrupt clear pending
7	RW	TIMER_3	TIMER_3 interrupt clear pending
6	RW	TIMER_2	TIMER_2 interrupt clear pending
5	RW	TIMER_1	TIMER_1 interrupt clear pending
4	RW	TIMER_0	TIMER_0 interrupt clear pending
3	RW	AUDIOSINK_1	AUDIOSINK_1 interrupt clear pending
2	RW	AUDIOSINK_0	AUDIOSINK_0 interrupt clear pending
1	RW	LSAD_BATMON	LSAD_BATMON interrupt clear pending
0	RW	WATCHDOG	WATCHDOG interrupt clear pending

Field Name	Value Symbol	Value Description	Hex Value
SPI_0_TX	NVIC_SPI_0_TX_INT_CLRPEND	Clear the NVIC_SPI_0_TX_INT_CLRPEND interrupt status bit	0x1
SPI_0_RX	NVIC_SPI_0_RX_INT_CLRPEND	Clear the NVIC_SPI_0_RX_INT_CLRPEND interrupt status bit	0x1
SPI_0_COM	NVIC_SPI_0_COM_INT_CLRPEND	Clear the NVIC_SPI_0_COM_INT_CLRPEND interrupt status bit	0x1
D_I2C	NVIC_D_I2C_INT_CLRPEND	Clear the NVIC_D_I2C_INT_CLRPEND interrupt status bit	0x1
I2C_1	NVIC_I2C_1_INT_CLRPEND	Clear the NVIC_I2C_1_INT_CLRPEND interrupt status bit	0x1
I2C_0	NVIC_I2C_0_INT_CLRPEND	Clear the NVIC_I2C_0_INT_CLRPEND interrupt status bit	0x1
I3C_1	NVIC_I3C_1_INT_CLRPEND	Clear the NVIC_I3C_1_INT_CLRPEND interrupt status bit	0x1
I3C_0	NVIC_I3C_0_INT_CLRPEND	Clear the NVIC_I3C_0_INT_CLRPEND interrupt status bit	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		CLRPEND interrupt status bit	
DIO_7	NVIC_DIO_7_INT_CLRPEND	Clear the NVIC_DIO_7_INT_CLRPEND interrupt status bit	0x1
DIO_6	NVIC_DIO_6_INT_CLRPEND	Clear the NVIC_DIO_6_INT_CLRPEND interrupt status bit	0x1
DIO_5	NVIC_DIO_5_INT_CLRPEND	Clear the NVIC_DIO_5_INT_CLRPEND interrupt status bit	0x1
DIO_4	NVIC_DIO_4_INT_CLRPEND	Clear the NVIC_DIO_4_INT_CLRPEND interrupt status bit	0x1
DIO_3	NVIC_DIO_3_INT_CLRPEND	Clear the NVIC_DIO_3_INT_CLRPEND interrupt status bit	0x1
DIO_2	NVIC_DIO_2_INT_CLRPEND	Clear the NVIC_DIO_2_INT_CLRPEND interrupt status bit	0x1
DIO_1	NVIC_DIO_1_INT_CLRPEND	Clear the NVIC_DIO_1_INT_CLRPEND interrupt status bit	0x1
DIO_0	NVIC_DIO_0_INT_CLRPEND	Clear the NVIC_DIO_0_INT_CLRPEND interrupt status bit	0x1
DMA_7	NVIC_DMA_7_INT_CLRPEND	Clear the NVIC_DMA_7_INT_CLRPEND interrupt status bit	0x1
DMA_6	NVIC_DMA_6_INT_CLRPEND	Clear the NVIC_DMA_6_INT_CLRPEND interrupt status bit	0x1
DMA_5	NVIC_DMA_5_INT_CLRPEND	Clear the NVIC_DMA_5_INT_CLRPEND interrupt status bit	0x1
DMA_4	NVIC_DMA_4_INT_CLRPEND	Clear the NVIC_DMA_4_INT_CLRPEND interrupt status bit	0x1
DMA_3	NVIC_DMA_3_INT_CLRPEND	Clear the NVIC_DMA_3_INT_CLRPEND interrupt status bit	0x1
DMA_2	NVIC_DMA_2_INT_CLRPEND	Clear the NVIC_DMA_2_INT_CLRPEND interrupt status bit	0x1
DMA_1	NVIC_DMA_1_INT_CLRPEND	Clear the NVIC_DMA_1_INT_CLRPEND interrupt status bit	0x1
DMA_0	NVIC_DMA_0_INT_CLRPEND	Clear the NVIC_DMA_0_INT_CLRPEND interrupt status bit	0x1
TIMER_3	NVIC_TIMER_3_INT_CLRPEND	Clear the NVIC_TIMER_3_INT_CLRPEND interrupt status bit	0x1
TIMER_2	NVIC_TIMER_2_INT_CLRPEND	Clear the NVIC_TIMER_2_INT_CLRPEND interrupt status bit	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
TIMER_1	NVIC_TIMER_1_INT_CLRPEND	Clear the NVIC_TIMER_1_INT_CLRPEND interrupt status bit	0x1
TIMER_0	NVIC_TIMER_0_INT_CLRPEND	Clear the NVIC_TIMER_0_INT_CLRPEND interrupt status bit	0x1
AUDIOSINK_1	NVIC_AUDIOSINK_1_INT_CLRPEND	Clear the NVIC_AUDIOSINK_1_INT_CLRPEND interrupt status bit	0x1
AUDIOSINK_0	NVIC_AUDIOSINK_0_INT_CLRPEND	Clear the NVIC_AUDIOSINK_0_INT_CLRPEND interrupt status bit	0x1
LSAD_BATMON	NVIC_LSAD_BATMON_INT_CLRPEND	Clear the NVIC_LSAD_BATMON_INT_CLRPEND interrupt status bit	0x1
WATCHDOG	NVIC_WATCHDOG_INT_CLRPEND	Clear the NVIC_WATCHDOG_INT_CLRPEND interrupt status bit	0x1

4.6.1.1.11 NVIC_ICPR1

Bit Field	Read/Write	Field Name	Description
31	RW	FIFO_7	FIFO_7 interrupt clear pending
30	RW	FIFO_6	FIFO_6 interrupt clear pending
29	RW	FIFO_5	FIFO_5 interrupt clear pending
28	RW	FIFO_4	FIFO_4 interrupt clear pending
27	RW	FIFO_3	FIFO_3 interrupt clear pending
26	RW	FIFO_2	FIFO_2 interrupt clear pending
25	RW	FIFO_1	FIFO_1 interrupt clear pending
24	RW	FIFO_0	FIFO_0 interrupt clear pending
23	RW	RS_DEC_OUT_RDY	RS_DEC_OUT_RDY interrupt clear pending
22	RW	RS_DEC_IN_RDY	RS_DEC_IN_RDY interrupt clear pending
21	RW	RS_ENC_OUT_RDY	RS_ENC_OUT_RDY interrupt clear pending
20	RW	RS_ENC_IN_RDY	RS_ENC_IN_RDY interrupt clear pending
19	RW	G722_DEC_OUT_RDY	G722_DEC_OUT_RDY interrupt clear pending
18	RW	G722_DEC_IN_RDY	G722_DEC_IN_RDY interrupt clear pending
17	RW	G722_ENC_OUT_RDY	G722_ENC_OUT_RDY interrupt clear pending
16	RW	G722_ENC_IN_RDY	G722_ENC_IN_RDY interrupt clear pending
15	RW	ASRC_IDLE	ASRC_IDLE interrupt clear pending
14	RW	ASRC_OUT	ASRC_OUT interrupt clear pending
13	RW	ASRC_IN	ASRC_IN interrupt clear pending

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
12	RW	ASRC_ERROR	ASRC_ERROR interrupt clear pending
11	RW	PCM_1_RX_TX	PCM_1_RX_TX interrupt clear pending
10	RW	PCM_1_ERROR	PCM_1_ERROR interrupt clear pending
9	RW	PCM_0_RX_TX	PCM_0_RX_TX interrupt clear pending
8	RW	PCM_0_ERROR	PCM_0_ERROR interrupt clear pending
7	RW	FENG	FENG interrupt clear pending
6	RW	UART_0_TX	UART_0_TX interrupt clear pending
5	RW	UART_0_RX	UART_0_RX interrupt clear pending
4	RW	UART_0_ERROR	UART_0_ERROR interrupt clear pending
3	RW	EMMC	EMMC interrupt clear pending
2	RW	SPI_1_TX	SPI_1_TX interrupt clear pending
1	RW	SPI_1_RX	SPI_1_RX interrupt clear pending
0	RW	SPI_1_COM	SPI_1_COM interrupt clear pending

Field Name	Value Symbol	Value Description	Hex Value
FIFO_7	NVIC_FIFO_7_INT_CLRPEND	Clear the NVIC_FIFO_7_INT_CLRPEND interrupt status bit	0x1
FIFO_6	NVIC_FIFO_6_INT_CLRPEND	Clear the NVIC_FIFO_6_INT_CLRPEND interrupt status bit	0x1
FIFO_5	NVIC_FIFO_5_INT_CLRPEND	Clear the NVIC_FIFO_5_INT_CLRPEND interrupt status bit	0x1
FIFO_4	NVIC_FIFO_4_INT_CLRPEND	Clear the NVIC_FIFO_4_INT_CLRPEND interrupt status bit	0x1
FIFO_3	NVIC_FIFO_3_INT_CLRPEND	Clear the NVIC_FIFO_3_INT_CLRPEND interrupt status bit	0x1
FIFO_2	NVIC_FIFO_2_INT_CLRPEND	Clear the NVIC_FIFO_2_INT_CLRPEND interrupt status bit	0x1
FIFO_1	NVIC_FIFO_1_INT_CLRPEND	Clear the NVIC_FIFO_1_INT_CLRPEND interrupt status bit	0x1
FIFO_0	NVIC_FIFO_0_INT_CLRPEND	Clear the NVIC_FIFO_0_INT_CLRPEND interrupt status bit	0x1
RS_DEC_OUT_RDY	NVIC_RS_DEC_OUT_RDY_INT_CLRPEND	Clear the NVIC_RS_DEC_OUT_RDY_INT_CLRPEND interrupt status bit	0x1
RS_DEC_IN_RDY	NVIC_RS_DEC_IN_RDY_INT_	Clear the NVIC_RS_DEC_IN_RDY_	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	CLRPEND	INT_CLRPEND interrupt status bit	
RS_ENC_OUT_RDY	NVIC_RS_ENC_OUT_RDY_INT_CLRPEND	Clear the NVIC_RS_ENC_OUT_RDY_INT_CLRPEND interrupt status bit	0x1
RS_ENC_IN_RDY	NVIC_RS_ENC_IN_RDY_INT_CLRPEND	Clear the NVIC_RS_ENC_IN_RDY_INT_CLRPEND interrupt status bit	0x1
G722_DEC_OUT_RDY	NVIC_G722_DEC_OUT_RDY_INT_CLRPEND	Clear the NVIC_G722_DEC_OUT_RDY_INT_CLRPEND interrupt status bit	0x1
G722_DEC_IN_RDY	NVIC_G722_DEC_IN_RDY_INT_CLRPEND	Clear the NVIC_G722_DEC_IN_RDY_INT_CLRPEND interrupt status bit	0x1
G722_ENC_OUT_RDY	NVIC_G722_ENC_OUT_RDY_INT_CLRPEND	Clear the NVIC_G722_ENC_OUT_RDY_INT_CLRPEND interrupt status bit	0x1
G722_ENC_IN_RDY	NVIC_G722_ENC_IN_RDY_INT_CLRPEND	Clear the NVIC_G722_ENC_IN_RDY_INT_CLRPEND interrupt status bit	0x1
ASRC_IDLE	NVIC_ASRC_IDLE_INT_CLRPEND	Clear the NVIC_ASRC_IDLE_INT_CLRPEND interrupt status bit	0x1
ASRC_OUT	NVIC_ASRC_OUT_INT_CLRPEND	Clear the NVIC_ASRC_OUT_INT_CLRPEND interrupt status bit	0x1
ASRC_IN	NVIC_ASRC_IN_INT_CLRPEND	Clear the NVIC_ASRC_IN_INT_CLRPEND interrupt status bit	0x1
ASRC_ERROR	NVIC_ASRC_ERROR_INT_CLRPEND	Clear the NVIC_ASRC_ERROR_INT_CLRPEND interrupt status bit	0x1
PCM_1_RX_TX	NVIC_PCM_1_RX_TX_INT_CLRPEND	Clear the NVIC_PCM_1_RX_TX_INT_CLRPEND interrupt status bit	0x1
PCM_1_ERROR	NVIC_PCM_1_ERROR_INT_CLRPEND	Clear the NVIC_PCM_1_ERROR_INT_CLRPEND interrupt status bit	0x1
PCM_0_RX_TX	NVIC_PCM_0_RX_TX_INT_CLRPEND	Clear the NVIC_PCM_0_RX_TX_INT_CLRPEND interrupt status bit	0x1
PCM_0_ERROR	NVIC_PCM_0_ERROR_INT_CLRPEND	Clear the NVIC_PCM_0_ERROR_INT_CLRPEND interrupt status bit	0x1
FENG	NVIC_FENG_INT_CLRPEND	Clear the NVIC_FENG_INT_CLRPEND interrupt status bit	0x1
UART_0_TX	NVIC_UART_0_TX_INT_CLRPEND	Clear the NVIC_UART_0_TX_INT_CLRPEND interrupt status bit	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
UART_0_RX	NVIC_UART_0_RX_INT_CLRPEND	Clear the NVIC_UART_0_RX_INT_CLRPEND interrupt status bit	0x1
UART_0_ERROR	NVIC_UART_0_ERROR_INT_CLRPEND	Clear the NVIC_UART_0_ERROR_INT_CLRPEND interrupt status bit	0x1
EMMC	NVIC_EMMC_INT_CLRPEND	Clear the NVIC_EMMC_INT_CLRPEND interrupt status bit	0x1
SPI_1_TX	NVIC_SPI_1_TX_INT_CLRPEND	Clear the NVIC_SPI_1_TX_INT_CLRPEND interrupt status bit	0x1
SPI_1_RX	NVIC_SPI_1_RX_INT_CLRPEND	Clear the NVIC_SPI_1_RX_INT_CLRPEND interrupt status bit	0x1
SPI_1_COM	NVIC_SPI_1_COM_INT_CLRPEND	Clear the NVIC_SPI_1_COM_INT_CLRPEND interrupt status bit	0x1

4.6.1.1.12 NVIC_ICPR2

Bit Field	Read/Write	Field Name	Description
29	RW	RS_COM	RS_COM interrupt clear pending
28	RW	G722_COM	G722_COM interrupt clear pending
27	RW	MEM_ERROR	MEM_ERROR interrupt clear pending
26	RW	LSAD	LSAD interrupt clear pending
25	RW	CLK_DET	CLK_DET interrupt clear pending
24	RW	NNA	NNA interrupt clear pending
23	RW	CFX_7	CFX_7 interrupt clear pending
22	RW	CFX_6	CFX_6 interrupt clear pending
21	RW	CFX_5	CFX_5 interrupt clear pending
20	RW	CFX_4	CFX_4 interrupt clear pending
19	RW	CFX_3	CFX_3 interrupt clear pending
18	RW	CFX_2	CFX_2 interrupt clear pending
17	RW	CFX_1	CFX_1 interrupt clear pending
16	RW	CFX_0	CFX_0 interrupt clear pending
15	RW	DSP_7	DSP_7 interrupt clear pending
14	RW	DSP_6	DSP_6 interrupt clear pending
13	RW	DSP_5	DSP_5 interrupt clear pending
12	RW	DSP_4	DSP_4 interrupt clear pending
11	RW	DSP_3	DSP_3 interrupt clear pending

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
10	RW	DSP_2	DSP_2 interrupt clear pending
9	RW	DSP_1	DSP_1 interrupt clear pending
8	RW	DSP_0	DSP_0 interrupt clear pending
7	RW	HEAR_7	HEAR_7 interrupt clear pending
6	RW	HEAR_6	HEAR_6 interrupt clear pending
5	RW	HEAR_5	HEAR_5 interrupt clear pending
4	RW	HEAR_4	HEAR_4 interrupt clear pending
3	RW	HEAR_3	HEAR_3 interrupt clear pending
2	RW	HEAR_2	HEAR_2 interrupt clear pending
1	RW	HEAR_1	HEAR_1 interrupt clear pending
0	RW	HEAR_0	HEAR_0 interrupt clear pending

Field Name	Value Symbol	Value Description	Hex Value
RS_COM	NVIC_RS_COM_INT_CLRPEND	Clear the NVIC_RS_COM_INT_CLRPEND interrupt status bit	0x1
G722_COM	NVIC_G722_COM_INT_CLRPEND	Clear the NVIC_G722_COM_INT_CLRPEND interrupt status bit	0x1
MEM_ERROR	NVIC_MEM_ERROR_INT_CLRPEND	Clear the NVIC_MEM_ERROR_INT_CLRPEND interrupt status bit	0x1
LSAD	NVIC_LSAD_INT_CLRPEND	Clear the NVIC_LSAD_INT_CLRPEND interrupt status bit	0x1
CLK_DET	NVIC_CLK_DET_INT_CLRPEND	Clear the NVIC_CLK_DET_INT_CLRPEND interrupt status bit	0x1
NNA	NVIC_NNA_INT_CLRPEND	Clear the NVIC_NNA_INT_CLRPEND interrupt status bit	0x1
CFX_7	NVIC_CFX_7_INT_CLRPEND	Clear the NVIC_CFX_7_INT_CLRPEND interrupt status bit	0x1
CFX_6	NVIC_CFX_6_INT_CLRPEND	Clear the NVIC_CFX_6_INT_CLRPEND interrupt status bit	0x1
CFX_5	NVIC_CFX_5_INT_CLRPEND	Clear the NVIC_CFX_5_INT_CLRPEND interrupt status bit	0x1
CFX_4	NVIC_CFX_4_INT_CLRPEND	Clear the NVIC_CFX_4_INT_CLRPEND interrupt status bit	0x1
CFX_3	NVIC_CFX_3_INT_CLRPEND	Clear the NVIC_CFX_3_INT_CLRPEND interrupt status bit	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
CFX_2	NVIC_CFX_2_INT_CLRPEND	Clear the NVIC_CFX_2_INT_CLRPEND interrupt status bit	0x1
CFX_1	NVIC_CFX_1_INT_CLRPEND	Clear the NVIC_CFX_1_INT_CLRPEND interrupt status bit	0x1
CFX_0	NVIC_CFX_0_INT_CLRPEND	Clear the NVIC_CFX_0_INT_CLRPEND interrupt status bit	0x1
DSP_7	NVIC_DSP_7_INT_CLRPEND	Clear the NVIC_DSP_7_INT_CLRPEND interrupt status bit	0x1
DSP_6	NVIC_DSP_6_INT_CLRPEND	Clear the NVIC_DSP_6_INT_CLRPEND interrupt status bit	0x1
DSP_5	NVIC_DSP_5_INT_CLRPEND	Clear the NVIC_DSP_5_INT_CLRPEND interrupt status bit	0x1
DSP_4	NVIC_DSP_4_INT_CLRPEND	Clear the NVIC_DSP_4_INT_CLRPEND interrupt status bit	0x1
DSP_3	NVIC_DSP_3_INT_CLRPEND	Clear the NVIC_DSP_3_INT_CLRPEND interrupt status bit	0x1
DSP_2	NVIC_DSP_2_INT_CLRPEND	Clear the NVIC_DSP_2_INT_CLRPEND interrupt status bit	0x1
DSP_1	NVIC_DSP_1_INT_CLRPEND	Clear the NVIC_DSP_1_INT_CLRPEND interrupt status bit	0x1
DSP_0	NVIC_DSP_0_INT_CLRPEND	Clear the NVIC_DSP_0_INT_CLRPEND interrupt status bit	0x1
HEAR_7	NVIC_HEAR_7_INT_CLRPEND	Clear the NVIC_HEAR_7_INT_CLRPEND interrupt status bit	0x1
HEAR_6	NVIC_HEAR_6_INT_CLRPEND	Clear the NVIC_HEAR_6_INT_CLRPEND interrupt status bit	0x1
HEAR_5	NVIC_HEAR_5_INT_CLRPEND	Clear the NVIC_HEAR_5_INT_CLRPEND interrupt status bit	0x1
HEAR_4	NVIC_HEAR_4_INT_CLRPEND	Clear the NVIC_HEAR_4_INT_CLRPEND interrupt status bit	0x1
HEAR_3	NVIC_HEAR_3_INT_CLRPEND	Clear the NVIC_HEAR_3_INT_CLRPEND interrupt status bit	0x1
HEAR_2	NVIC_HEAR_2_INT_CLRPEND	Clear the NVIC_HEAR_2_INT_CLRPEND interrupt status bit	0x1
HEAR_1	NVIC_HEAR_1_INT_CLRPEND	Clear the NVIC_HEAR_1_INT_CLRPEND interrupt status bit	0x1
HEAR_0	NVIC_HEAR_0_INT_CLRPEND	Clear the NVIC_HEAR_0_INT_CLRPEND interrupt status bit	0x1

Ezairo 8300 Hardware Reference

4.6.1.1.13 NVIC_IABR0

Bit Field	Read/Write	Field Name	Description
31	R	SPI_0_TX	Set the SPI_0_TX interrupt as active
30	R	SPI_0_RX	Set the SPI_0_RX interrupt as active
29	R	SPI_0_COM	Set the SPI_0_COM interrupt as active
28	R	D_I2C	Set the D_I2C interrupt as active
27	R	I2C_1	Set the I2C_1 interrupt as active
26	R	I2C_0	Set the I2C_0 interrupt as active
25	R	I3C_1	Set the I3C_1 interrupt as active
24	R	I3C_0	Set the I3C_0 interrupt as active
23	R	DIO_7	Set the DIO_7 interrupt as active
22	R	DIO_6	Set the DIO_6 interrupt as active
21	R	DIO_5	Set the DIO_5 interrupt as active
20	R	DIO_4	Set the DIO_4 interrupt as active
19	R	DIO_3	Set the DIO_3 interrupt as active
18	R	DIO_2	Set the DIO_2 interrupt as active
17	R	DIO_1	Set the DIO_1 interrupt as active
16	R	DIO_0	Set the DIO_0 interrupt as active
15	R	DMA_7	Set the DMA_7 interrupt as active
14	R	DMA_6	Set the DMA_6 interrupt as active
13	R	DMA_5	Set the DMA_5 interrupt as active
12	R	DMA_4	Set the DMA_4 interrupt as active
11	R	DMA_3	Set the DMA_3 interrupt as active
10	R	DMA_2	Set the DMA_2 interrupt as active
9	R	DMA_1	Set the DMA_1 interrupt as active
8	R	DMA_0	Set the DMA_0 interrupt as active
7	R	TIMER_3	Set the TIMER_3 interrupt as active
6	R	TIMER_2	Set the TIMER_2 interrupt as active
5	R	TIMER_1	Set the TIMER_1 interrupt as active
4	R	TIMER_0	Set the TIMER_0 interrupt as active
3	R	AUDIOSINK_1	Set the AUDIOSINK_1 interrupt as active

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
2	R	AUDIOSINK_0	Set the AUDIOSINK_0 interrupt as active
1	R	LSAD_BATMON	Set the LSAD_BATMON interrupt as active
0	R	WATCHDOG	Set the WATCHDOG interrupt as active

Field Name	Value Symbol	Value Description	Hex Value
SPI_0_TX	NVIC_SPI_0_TX_INT_ACTIVE	The NVIC_SPI_0_TX_INT_ACTIVE interrupt is active	0x1
SPI_0_RX	NVIC_SPI_0_RX_INT_ACTIVE	The NVIC_SPI_0_RX_INT_ACTIVE interrupt is active	0x1
SPI_0_COM	NVIC_SPI_0_COM_INT_ACTIVE	The NVIC_SPI_0_COM_INT_ACTIVE interrupt is active	0x1
D_I2C	NVIC_D_I2C_INT_ACTIVE	The NVIC_D_I2C_INT_ACTIVE interrupt is active	0x1
I2C_1	NVIC_I2C_1_INT_ACTIVE	The NVIC_I2C_1_INT_ACTIVE interrupt is active	0x1
I2C_0	NVIC_I2C_0_INT_ACTIVE	The NVIC_I2C_0_INT_ACTIVE interrupt is active	0x1
I3C_1	NVIC_I3C_1_INT_ACTIVE	The NVIC_I3C_1_INT_ACTIVE interrupt is active	0x1
I3C_0	NVIC_I3C_0_INT_ACTIVE	The NVIC_I3C_0_INT_ACTIVE interrupt is active	0x1
DIO_7	NVIC_DIO_7_INT_ACTIVE	The NVIC_DIO_7_INT_ACTIVE interrupt is active	0x1
DIO_6	NVIC_DIO_6_INT_ACTIVE	The NVIC_DIO_6_INT_ACTIVE interrupt is active	0x1
DIO_5	NVIC_DIO_5_INT_ACTIVE	The NVIC_DIO_5_INT_ACTIVE interrupt is active	0x1
DIO_4	NVIC_DIO_4_INT_ACTIVE	The NVIC_DIO_4_INT_ACTIVE interrupt is active	0x1
DIO_3	NVIC_DIO_3_INT_ACTIVE	The NVIC_DIO_3_INT_ACTIVE interrupt is active	0x1
DIO_2	NVIC_DIO_2_INT_ACTIVE	The NVIC_DIO_2_INT_ACTIVE interrupt is active	0x1
DIO_1	NVIC_DIO_1_INT_ACTIVE	The NVIC_DIO_1_INT_ACTIVE interrupt is active	0x1
DIO_0	NVIC_DIO_0_INT_ACTIVE	The NVIC_DIO_0_INT_ACTIVE interrupt is active	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DMA_7	NVIC_DMA_7_INT_ACTIVE	The NVIC_DMA_7_INT_ACTIVE interrupt is active	0x1
DMA_6	NVIC_DMA_6_INT_ACTIVE	The NVIC_DMA_6_INT_ACTIVE interrupt is active	0x1
DMA_5	NVIC_DMA_5_INT_ACTIVE	The NVIC_DMA_5_INT_ACTIVE interrupt is active	0x1
DMA_4	NVIC_DMA_4_INT_ACTIVE	The NVIC_DMA_4_INT_ACTIVE interrupt is active	0x1
DMA_3	NVIC_DMA_3_INT_ACTIVE	The NVIC_DMA_3_INT_ACTIVE interrupt is active	0x1
DMA_2	NVIC_DMA_2_INT_ACTIVE	The NVIC_DMA_2_INT_ACTIVE interrupt is active	0x1
DMA_1	NVIC_DMA_1_INT_ACTIVE	The NVIC_DMA_1_INT_ACTIVE interrupt is active	0x1
DMA_0	NVIC_DMA_0_INT_ACTIVE	The NVIC_DMA_0_INT_ACTIVE interrupt is active	0x1
TIMER_3	NVIC_TIMER_3_INT_ACTIVE	The NVIC_TIMER_3_INT_ACTIVE interrupt is active	0x1
TIMER_2	NVIC_TIMER_2_INT_ACTIVE	The NVIC_TIMER_2_INT_ACTIVE interrupt is active	0x1
TIMER_1	NVIC_TIMER_1_INT_ACTIVE	The NVIC_TIMER_1_INT_ACTIVE interrupt is active	0x1
TIMER_0	NVIC_TIMER_0_INT_ACTIVE	The NVIC_TIMER_0_INT_ACTIVE interrupt is active	0x1
AUDIOSINK_1	NVIC_AUDIOSINK_1_INT_ACTIVE	The NVIC_AUDIOSINK_1_INT_ACTIVE interrupt is active	0x1
AUDIOSINK_0	NVIC_AUDIOSINK_0_INT_ACTIVE	The NVIC_AUDIOSINK_0_INT_ACTIVE interrupt is active	0x1
LSAD_BATMON	NVIC_LSAD_BATMON_INT_ACTIVE	The NVIC_LSAD_BATMON_INT_ACTIVE interrupt is active	0x1
WATCHDOG	NVIC_WATCHDOG_INT_ACTIVE	The NVIC_WATCHDOG_INT_ACTIVE interrupt is active	0x1

Ezairo 8300 Hardware Reference

4.6.1.1.14 NVIC_IABR1

Bit Field	Read/Write	Field Name	Description
31	R	FIFO_7	Set the FIFO_7 interrupt as active
30	R	FIFO_6	Set the FIFO_6 interrupt as active
29	R	FIFO_5	Set the FIFO_5 interrupt as active
28	R	FIFO_4	Set the FIFO_4 interrupt as active
27	R	FIFO_3	Set the FIFO_3 interrupt as active
26	R	FIFO_2	Set the FIFO_2 interrupt as active
25	R	FIFO_1	Set the FIFO_1 interrupt as active
24	R	FIFO_0	Set the FIFO_0 interrupt as active
23	R	RS_DEC_OUT_RDY	Set the RS_DEC_OUT_RDY interrupt as active
22	R	RS_DEC_IN_RDY	Set the RS_DEC_IN_RDY interrupt as active
21	R	RS_ENC_OUT_RDY	Set the RS_ENC_OUT_RDY interrupt as active
20	R	RS_ENC_IN_RDY	Set the RS_ENC_IN_RDY interrupt as active
19	R	G722_DEC_OUT_RDY	Set the G722_DEC_OUT_RDY interrupt as active
18	R	G722_DEC_IN_RDY	Set the G722_DEC_IN_RDY interrupt as active
17	R	G722_ENC_OUT_RDY	Set the G722_ENC_OUT_RDY interrupt as active
16	R	G722_ENC_IN_RDY	Set the G722_ENC_IN_RDY interrupt as active
15	R	ASRC_IDLE	Set the ASRC_IDLE interrupt as active
14	R	ASRC_OUT	Set the ASRC_OUT interrupt as active
13	R	ASRC_IN	Set the ASRC_IN interrupt as active
12	R	ASRC_ERROR	Set the ASRC_ERROR interrupt as active
11	R	PCM_1_RX_TX	Set the PCM_1_RX_TX interrupt as active
10	R	PCM_1_ERROR	Set the PCM_1_ERROR interrupt as active
9	R	PCM_0_RX_TX	Set the PCM_0_RX_TX interrupt as active
8	R	PCM_0_ERROR	Set the PCM_0_ERROR interrupt as active
7	R	FENG	Set the FENG interrupt as active
6	R	UART_0_TX	Set the UART_0_TX interrupt as active
5	R	UART_0_RX	Set the UART_0_RX interrupt as active
4	R	UART_0_ERROR	Set the UART_0_ERROR interrupt as active
3	R	EMMC	Set the EMMC interrupt as active

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
2	R	SPI_1_TX	Set the SPI_1_TX interrupt as active
1	R	SPI_1_RX	Set the SPI_1_RX interrupt as active
0	R	SPI_1_COM	Set the SPI_1_COM interrupt as active

Field Name	Value Symbol	Value Description	Hex Value
FIFO_7	NVIC_FIFO_7_INT_ACTIVE	The NVIC_FIFO_7_INT_ACTIVE interrupt is active	0x1
FIFO_6	NVIC_FIFO_6_INT_ACTIVE	The NVIC_FIFO_6_INT_ACTIVE interrupt is active	0x1
FIFO_5	NVIC_FIFO_5_INT_ACTIVE	The NVIC_FIFO_5_INT_ACTIVE interrupt is active	0x1
FIFO_4	NVIC_FIFO_4_INT_ACTIVE	The NVIC_FIFO_4_INT_ACTIVE interrupt is active	0x1
FIFO_3	NVIC_FIFO_3_INT_ACTIVE	The NVIC_FIFO_3_INT_ACTIVE interrupt is active	0x1
FIFO_2	NVIC_FIFO_2_INT_ACTIVE	The NVIC_FIFO_2_INT_ACTIVE interrupt is active	0x1
FIFO_1	NVIC_FIFO_1_INT_ACTIVE	The NVIC_FIFO_1_INT_ACTIVE interrupt is active	0x1
FIFO_0	NVIC_FIFO_0_INT_ACTIVE	The NVIC_FIFO_0_INT_ACTIVE interrupt is active	0x1
RS_DEC_OUT_RDY	NVIC_RS_DEC_OUT_RDY_INT_ACTIVE	The NVIC_RS_DEC_OUT_RDY_INT_ACTIVE interrupt is active	0x1
RS_DEC_IN_RDY	NVIC_RS_DEC_IN_RDY_INT_ACTIVE	The NVIC_RS_DEC_IN_RDY_INT_ACTIVE interrupt is active	0x1
RS_ENC_OUT_RDY	NVIC_RS_ENC_OUT_RDY_INT_ACTIVE	The NVIC_RS_ENC_OUT_RDY_INT_ACTIVE interrupt is active	0x1
RS_ENC_IN_RDY	NVIC_RS_ENC_IN_RDY_INT_ACTIVE	The NVIC_RS_ENC_IN_RDY_INT_ACTIVE interrupt is active	0x1
G722_DEC_OUT_RDY	NVIC_G722_DEC_OUT_RDY_INT_ACTIVE	The NVIC_G722_DEC_OUT_RDY_INT_ACTIVE interrupt is active	0x1
G722_DEC_IN_RDY	NVIC_G722_DEC_IN_RDY_INT_ACTIVE	The NVIC_G722_DEC_IN_RDY_INT_ACTIVE interrupt is active	0x1
G722_ENC_OUT_RDY	NVIC_G722_ENC_OUT_RDY_INT_ACTIVE	The NVIC_G722_ENC_OUT_RDY_INT_ACTIVE interrupt is active	0x1
G722_ENC_IN_RDY	NVIC_G722_ENC_IN_RDY_INT_ACTIVE	The NVIC_G722_ENC_IN_RDY_INT_ACTIVE interrupt is active	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
ASRC_IDLE	NVIC_ASRC_IDLE_INT_ACTIVE	The NVIC_ASRC_IDLE_INT_ACTIVE interrupt is active	0x1
ASRC_OUT	NVIC_ASRC_OUT_INT_ACTIVE	The NVIC_ASRC_OUT_INT_ACTIVE interrupt is active	0x1
ASRC_IN	NVIC_ASRC_IN_INT_ACTIVE	The NVIC_ASRC_IN_INT_ACTIVE interrupt is active	0x1
ASRC_ERROR	NVIC_ASRC_ERROR_INT_ACTIVE	The NVIC_ASRC_ERROR_INT_ACTIVE interrupt is active	0x1
PCM_1_RX_TX	NVIC_PCM_1_RX_TX_INT_ACTIVE	The NVIC_PCM_1_RX_TX_INT_ACTIVE interrupt is active	0x1
PCM_1_ERROR	NVIC_PCM_1_ERROR_INT_ACTIVE	The NVIC_PCM_1_ERROR_INT_ACTIVE interrupt is active	0x1
PCM_0_RX_TX	NVIC_PCM_0_RX_TX_INT_ACTIVE	The NVIC_PCM_0_RX_TX_INT_ACTIVE interrupt is active	0x1
PCM_0_ERROR	NVIC_PCM_0_ERROR_INT_ACTIVE	The NVIC_PCM_0_ERROR_INT_ACTIVE interrupt is active	0x1
FENG	NVIC_FENG_INT_ACTIVE	The NVIC_FENG_INT_ACTIVE interrupt is active	0x1
UART_0_TX	NVIC_UART_0_TX_INT_ACTIVE	The NVIC_UART_0_TX_INT_ACTIVE interrupt is active	0x1
UART_0_RX	NVIC_UART_0_RX_INT_ACTIVE	The NVIC_UART_0_RX_INT_ACTIVE interrupt is active	0x1
UART_0_ERROR	NVIC_UART_0_ERROR_INT_ACTIVE	The NVIC_UART_0_ERROR_INT_ACTIVE interrupt is active	0x1
EMMC	NVIC_EMMC_INT_ACTIVE	The NVIC_EMMC_INT_ACTIVE interrupt is active	0x1
SPI_1_TX	NVIC_SPI_1_TX_INT_ACTIVE	The NVIC_SPI_1_TX_INT_ACTIVE interrupt is active	0x1
SPI_1_RX	NVIC_SPI_1_RX_INT_ACTIVE	The NVIC_SPI_1_RX_INT_ACTIVE interrupt is active	0x1
SPI_1_COM	NVIC_SPI_1_COM_INT_ACTIVE	The NVIC_SPI_1_COM_INT_ACTIVE interrupt is active	0x1

Ezairo 8300 Hardware Reference

4.6.1.1.15 NVIC_IABR2

Bit Field	Read/Write	Field Name	Description
29	R	RS_COM	Set the RS_COM interrupt as active
28	R	G722_COM	Set the G722_COM interrupt as active
27	R	MEM_ERROR	Set the MEM_ERROR interrupt as active
26	R	LSAD	Set the LSAD interrupt as active
25	R	CLK_DET	Set the CLK_DET interrupt as active
24	R	NNA	Set the NNA interrupt as active
23	R	CFX_7	Set the CFX_7 interrupt as active
22	R	CFX_6	Set the CFX_6 interrupt as active
21	R	CFX_5	Set the CFX_5 interrupt as active
20	R	CFX_4	Set the CFX_4 interrupt as active
19	R	CFX_3	Set the CFX_3 interrupt as active
18	R	CFX_2	Set the CFX_2 interrupt as active
17	R	CFX_1	Set the CFX_1 interrupt as active
16	R	CFX_0	Set the CFX_0 interrupt as active
15	R	DSP_7	Set the DSP_7 interrupt as active
14	R	DSP_6	Set the DSP_6 interrupt as active
13	R	DSP_5	Set the DSP_5 interrupt as active
12	R	DSP_4	Set the DSP_4 interrupt as active
11	R	DSP_3	Set the DSP_3 interrupt as active
10	R	DSP_2	Set the DSP_2 interrupt as active
9	R	DSP_1	Set the DSP_1 interrupt as active
8	R	DSP_0	Set the DSP_0 interrupt as active
7	R	HEAR_7	Set the HEAR_7 interrupt as active
6	R	HEAR_6	Set the HEAR_6 interrupt as active
5	R	HEAR_5	Set the HEAR_5 interrupt as active
4	R	HEAR_4	Set the HEAR_4 interrupt as active
3	R	HEAR_3	Set the HEAR_3 interrupt as active
2	R	HEAR_2	Set the HEAR_2 interrupt as active
1	R	HEAR_1	Set the HEAR_1 interrupt as active
0	R	HEAR_0	Set the HEAR_0 interrupt as active

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
RS_COM	NVIC_RS_COM_INT_ACTIVE	The NVIC_RS_COM_INT_ACTIVE interrupt is active	0x1
G722_COM	NVIC_G722_COM_INT_ACTIVE	The NVIC_G722_COM_INT_ACTIVE interrupt is active	0x1
MEM_ERROR	NVIC_MEM_ERROR_INT_ACTIVE	The NVIC_MEM_ERROR_INT_ACTIVE interrupt is active	0x1
LSAD	NVIC_LSAD_INT_ACTIVE	The NVIC_LSAD_INT_ACTIVE interrupt is active	0x1
CLK_DET	NVIC_CLK_DET_INT_ACTIVE	The NVIC_CLK_DET_INT_ACTIVE interrupt is active	0x1
NNA	NVIC_NNA_INT_ACTIVE	The NVIC_NNA_INT_ACTIVE interrupt is active	0x1
CFX_7	NVIC_CFX_7_INT_ACTIVE	The NVIC_CFX_7_INT_ACTIVE interrupt is active	0x1
CFX_6	NVIC_CFX_6_INT_ACTIVE	The NVIC_CFX_6_INT_ACTIVE interrupt is active	0x1
CFX_5	NVIC_CFX_5_INT_ACTIVE	The NVIC_CFX_5_INT_ACTIVE interrupt is active	0x1
CFX_4	NVIC_CFX_4_INT_ACTIVE	The NVIC_CFX_4_INT_ACTIVE interrupt is active	0x1
CFX_3	NVIC_CFX_3_INT_ACTIVE	The NVIC_CFX_3_INT_ACTIVE interrupt is active	0x1
CFX_2	NVIC_CFX_2_INT_ACTIVE	The NVIC_CFX_2_INT_ACTIVE interrupt is active	0x1
CFX_1	NVIC_CFX_1_INT_ACTIVE	The NVIC_CFX_1_INT_ACTIVE interrupt is active	0x1
CFX_0	NVIC_CFX_0_INT_ACTIVE	The NVIC_CFX_0_INT_ACTIVE interrupt is active	0x1
DSP_7	NVIC_DSP_7_INT_ACTIVE	The NVIC_DSP_7_INT_ACTIVE interrupt is active	0x1
DSP_6	NVIC_DSP_6_INT_ACTIVE	The NVIC_DSP_6_INT_ACTIVE interrupt is active	0x1
DSP_5	NVIC_DSP_5_INT_ACTIVE	The NVIC_DSP_5_INT_ACTIVE interrupt is active	0x1
DSP_4	NVIC_DSP_4_INT_ACTIVE	The NVIC_DSP_4_INT_ACTIVE interrupt is active	0x1
DSP_3	NVIC_DSP_3_INT_ACTIVE	The NVIC_DSP_3_INT_ACTIVE interrupt is active	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DSP_2	NVIC_DSP_2_INT_ACTIVE	The NVIC_DSP_2_INT_ACTIVE interrupt is active	0x1
DSP_1	NVIC_DSP_1_INT_ACTIVE	The NVIC_DSP_1_INT_ACTIVE interrupt is active	0x1
DSP_0	NVIC_DSP_0_INT_ACTIVE	The NVIC_DSP_0_INT_ACTIVE interrupt is active	0x1
HEAR_7	NVIC_HEAR_7_INT_ACTIVE	The NVIC_HEAR_7_INT_ACTIVE interrupt is active	0x1
HEAR_6	NVIC_HEAR_6_INT_ACTIVE	The NVIC_HEAR_6_INT_ACTIVE interrupt is active	0x1
HEAR_5	NVIC_HEAR_5_INT_ACTIVE	The NVIC_HEAR_5_INT_ACTIVE interrupt is active	0x1
HEAR_4	NVIC_HEAR_4_INT_ACTIVE	The NVIC_HEAR_4_INT_ACTIVE interrupt is active	0x1
HEAR_3	NVIC_HEAR_3_INT_ACTIVE	The NVIC_HEAR_3_INT_ACTIVE interrupt is active	0x1
HEAR_2	NVIC_HEAR_2_INT_ACTIVE	The NVIC_HEAR_2_INT_ACTIVE interrupt is active	0x1
HEAR_1	NVIC_HEAR_1_INT_ACTIVE	The NVIC_HEAR_1_INT_ACTIVE interrupt is active	0x1
HEAR_0	NVIC_HEAR_0_INT_ACTIVE	The NVIC_HEAR_0_INT_ACTIVE interrupt is active	0x1

4.6.1.1.16 NVIC_IP0

Bit Field	Read/Write	Field Name	Description
31:24	RW	AUDIOSINK_1	Configure the AUDIOSINK_1 interrupt priority
23:16	RW	AUDIOSINK_0	Configure the AUDIOSINK_0 interrupt priority
15:8	RW	LSAD_BATMON	Configure the LSAD_BATMON interrupt priority
7:0	RW	WATCHDOG	Configure the WATCHDOG interrupt priority

4.6.1.1.17 NVIC_IP1

Bit Field	Read/Write	Field Name	Description
31:24	RW	TIMER_3	Configure the TIMER_3 interrupt priority
23:16	RW	TIMER_2	Configure the TIMER_2 interrupt priority
15:8	RW	TIMER_1	Configure the TIMER_1 interrupt priority
7:0	RW	TIMER_0	Configure the TIMER_0 interrupt priority

4.6.1.1.18 NVIC_IP2

Bit Field	Read/Write	Field Name	Description
31:24	RW	DMA_3	Configure the DMA_3 interrupt priority
23:16	RW	DMA_2	Configure the DMA_2 interrupt priority
15:8	RW	DMA_1	Configure the DMA_1 interrupt priority
7:0	RW	DMA_0	Configure the DMA_0 interrupt priority

4.6.1.1.19 NVIC_IP3

Bit Field	Read/Write	Field Name	Description
31:24	RW	DMA_7	Configure the DMA_7 interrupt priority
23:16	RW	DMA_6	Configure the DMA_6 interrupt priority
15:8	RW	DMA_5	Configure the DMA_5 interrupt priority
7:0	RW	DMA_4	Configure the DMA_4 interrupt priority

4.6.1.1.20 NVIC_IP4

Bit Field	Read/Write	Field Name	Description
31:24	RW	DIO_3	Configure the DIO_3 interrupt priority
23:16	RW	DIO_2	Configure the DIO_2 interrupt priority
15:8	RW	DIO_1	Configure the DIO_1 interrupt priority
7:0	RW	DIO_0	Configure the DIO_0 interrupt priority

4.6.1.1.21 NVIC_IP5

Bit Field	Read/Write	Field Name	Description
31:24	RW	DIO_7	Configure the DIO_7 interrupt priority
23:16	RW	DIO_6	Configure the DIO_6 interrupt priority
15:8	RW	DIO_5	Configure the DIO_5 interrupt priority
7:0	RW	DIO_4	Configure the DIO_4 interrupt priority

4.6.1.1.22 NVIC_IP6

Bit Field	Read/Write	Field Name	Description
31:24	RW	I2C_1	Configure the I2C_1 interrupt priority
23:16	RW	I2C_0	Configure the I2C_0 interrupt priority
15:8	RW	I3C_1	Configure the I3C_1 interrupt priority
7:0	RW	I3C_0	Configure the I3C_0 interrupt priority

Ezairo 8300 Hardware Reference

4.6.1.1.23 NVIC_IP7

Bit Field	Read/Write	Field Name	Description
31:24	RW	SPI_0_TX	Configure the SPI_0_TX interrupt priority
23:16	RW	SPI_0_RX	Configure the SPI_0_RX interrupt priority
15:8	RW	SPI_0_COM	Configure the SPI_0_COM interrupt priority
7:0	RW	D_I2C	Configure the D_I2C interrupt priority

4.6.1.1.24 NVIC_IP8

Bit Field	Read/Write	Field Name	Description
31:24	RW	EMMC	Configure the EMMC interrupt priority
23:16	RW	SPI_1_TX	Configure the SPI_1_TX interrupt priority
15:8	RW	SPI_1_RX	Configure the SPI_1_RX interrupt priority
7:0	RW	SPI_1_COM	Configure the SPI_1_COM interrupt priority

4.6.1.1.25 NVIC_IP9

Bit Field	Read/Write	Field Name	Description
31:24	RW	FENG	Configure the FENG interrupt priority
23:16	RW	UART_0_TX	Configure the UART_0_TX interrupt priority
15:8	RW	UART_0_RX	Configure the UART_0_RX interrupt priority
7:0	RW	UART_0_ERROR	Configure the UART_0_ERROR interrupt priority

4.6.1.1.26 NVIC_IP10

Bit Field	Read/Write	Field Name	Description
31:24	RW	PCM_1_RX_TX	Configure the PCM_1_RX_TX interrupt priority
23:16	RW	PCM_1_ERROR	Configure the PCM_1_ERROR interrupt priority
15:8	RW	PCM_0_RX_TX	Configure the PCM_0_RX_TX interrupt priority
7:0	RW	PCM_0_ERROR	Configure the PCM_0_ERROR interrupt priority

4.6.1.1.27 NVIC_IP11

Bit Field	Read/Write	Field Name	Description
31:24	RW	ASRC_IDLE	Configure the ASRC_IDLE interrupt priority
23:16	RW	ASRC_OUT	Configure the ASRC_OUT interrupt priority
15:8	RW	ASRC_IN	Configure the ASRC_IN interrupt priority
7:0	RW	ASRC_ERROR	Configure the ASRC_ERROR interrupt priority

Ezairo 8300 Hardware Reference

4.6.1.1.28 NVIC_IP12

Bit Field	Read/Write	Field Name	Description
31:24	RW	G722_DEC_OUT_RDY	Configure the G722_DEC_OUT_RDY interrupt priority
23:16	RW	G722_DEC_IN_RDY	Configure the G722_DEC_IN_RDY interrupt priority
15:8	RW	G722_ENC_OUT_RDY	Configure the G722_ENC_OUT_RDY interrupt priority
7:0	RW	G722_ENC_IN_RDY	Configure the G722_ENC_IN_RDY interrupt priority

4.6.1.1.29 NVIC_IP13

Bit Field	Read/Write	Field Name	Description
31:24	RW	RS_DEC_OUT_RDY	Configure the RS_DEC_OUT_RDY interrupt priority
23:16	RW	RS_DEC_IN_RDY	Configure the RS_DEC_IN_RDY interrupt priority
15:8	RW	RS_ENC_OUT_RDY	Configure the RS_ENC_OUT_RDY interrupt priority
7:0	RW	RS_ENC_IN_RDY	Configure the RS_ENC_IN_RDY interrupt priority

4.6.1.1.30 NVIC_IP14

Bit Field	Read/Write	Field Name	Description
31:24	RW	FIFO_3	Configure the FIFO_3 interrupt priority
23:16	RW	FIFO_2	Configure the FIFO_2 interrupt priority
15:8	RW	FIFO_1	Configure the FIFO_1 interrupt priority
7:0	RW	FIFO_0	Configure the FIFO_0 interrupt priority

4.6.1.1.31 NVIC_IP15

Bit Field	Read/Write	Field Name	Description
31:24	RW	FIFO_7	Configure the FIFO_7 interrupt priority
23:16	RW	FIFO_6	Configure the FIFO_6 interrupt priority
15:8	RW	FIFO_5	Configure the FIFO_5 interrupt priority
7:0	RW	FIFO_4	Configure the FIFO_4 interrupt priority

4.6.1.1.32 NVIC_IP16

Bit Field	Read/Write	Field Name	Description
31:24	RW	HEAR_3	Configure the HEAR_3 interrupt priority
23:16	RW	HEAR_2	Configure the HEAR_2 interrupt priority
15:8	RW	HEAR_1	Configure the HEAR_1 interrupt priority
7:0	RW	HEAR_0	Configure the HEAR_0 interrupt priority

4.6.1.1.33 NVIC_IP17

Bit Field	Read/Write	Field Name	Description
31:24	RW	HEAR_7	Configure the HEAR_7 interrupt priority
23:16	RW	HEAR_6	Configure the HEAR_6 interrupt priority
15:8	RW	HEAR_5	Configure the HEAR_5 interrupt priority
7:0	RW	HEAR_4	Configure the HEAR_4 interrupt priority

4.6.1.1.34 NVIC_IP18

Bit Field	Read/Write	Field Name	Description
31:24	RW	DSP_3	Configure the DSP_3 interrupt priority
23:16	RW	DSP_2	Configure the DSP_2 interrupt priority
15:8	RW	DSP_1	Configure the DSP_1 interrupt priority
7:0	RW	DSP_0	Configure the DSP_0 interrupt priority

4.6.1.1.35 NVIC_IP19

Bit Field	Read/Write	Field Name	Description
31:24	RW	DSP_7	Configure the DSP_7 interrupt priority
23:16	RW	DSP_6	Configure the DSP_6 interrupt priority
15:8	RW	DSP_5	Configure the DSP_5 interrupt priority
7:0	RW	DSP_4	Configure the DSP_4 interrupt priority

4.6.1.1.36 NVIC_IP20

Bit Field	Read/Write	Field Name	Description
31:24	RW	CFX_3	Configure the CFX_3 interrupt priority
23:16	RW	CFX_2	Configure the CFX_2 interrupt priority
15:8	RW	CFX_1	Configure the CFX_1 interrupt priority
7:0	RW	CFX_0	Configure the CFX_0 interrupt priority

4.6.1.1.37 NVIC_IP21

Bit Field	Read/Write	Field Name	Description
31:24	RW	CFX_7	Configure the CFX_7 interrupt priority
23:16	RW	CFX_6	Configure the CFX_6 interrupt priority
15:8	RW	CFX_5	Configure the CFX_5 interrupt priority
7:0	RW	CFX_4	Configure the CFX_4 interrupt priority

4.6.1.1.38 NVIC_IP22

Bit Field	Read/Write	Field Name	Description
31:24	RW	MEM_ERROR	Configure the MEM_ERROR interrupt priority
23:16	RW	LSAD	Configure the LSAD interrupt priority
15:8	RW	CLK_DET	Configure the CLK_DET interrupt priority
7:0	RW	NNA	Configure the NNA interrupt priority

4.6.1.1.39 NVIC_IP23

Bit Field	Read/Write	Field Name	Description
15:8	RW	RS_COM	Configure the RS_COM interrupt priority
7:0	RW	G722_COM	Configure the G722_COM interrupt priority

4.6.1.1.40 NVIC_STIR

Bit Field	Read/Write	Field Name	Description
7:0	W	INTID	The interrupt number to trigger

4.6.1.1.41 SCnSCB_ICTR

Bit Field	Read/Write	Field Name	Description
4:0	R	INTLINESNUM	Number of interrupt inputs in steps of 32

Field Name	Value Symbol	Value Description	Hex Value
INTLINESNUM	NVIC_INTLINESNUM_1_32	NVIC_INTLINESNUM_1_32	0x0
	NVIC_INTLINESNUM_33_64	NVIC_INTLINESNUM_33_64	0x1
	NVIC_INTLINESNUM_65_96	NVIC_INTLINESNUM_65_96	0x2*

4.6.1.1.42 SCB_CPUID

Bit Field	Read/Write	Field Name	Description
31:24	R	IMPLEMENTER	Implementer code
23:20	R	VARIANT	Implementation variant
19:16	R	ARCHITECTURE	Architecture number
15:4	R	PARTNO	Part number
3:0	R	REVISION	Revision code

4.6.1.1.43 SCB_ICSR

Bit Field	Read/Write	Field Name	Description
31	RW	NMIPENDSET	Indicates the NMI exception is pending
28	RW	PENDSVSET	Write 1 to set pending status for PendSV exception. Read indicates PendSV pending status
27	W	PENDSVCLR	Write 1 to clear PendSV exception pending status
26	RW	PENDSTSET	Write 1 to set pending status for SYSTICK exception. Read value indicates SYSTICK pending status
25	W	PENDSTCLR	Write 1 to clear SYSTICK exception pending status
23	R	ISRPREEMPT	Indicates that a pending interrupt is going to be active in the next step
22	R	ISRPENDING	Indicates that an external interrupt is pending
21:12	R	VECTPENDING	Pending external interrupt number
11	R	RETTOBASE	Set to 1 when the an exception handler is being run
9:0	R	VECTACTIVE	Number of current running interrupt service routine

Field Name	Value Symbol	Value Description	Hex Value
NMIPENDSET	NVIC_NMI_SETPEND	Set NMI exception to pending	0x1
PENDSVSET	NVIC_PENDSV_SETPEND	Set PendSV exception to pending	0x1
PENDSVCLR	NVIC_PENDSV_CLRPEND	Clear PendSV exception pending status	0x1
PENDSTSET	NVIC_SYSTICK_SETPEND	Set SYSTICK exception to pending	0x1
PENDSTCLR	NVIC_SYSTICK_CLRPEND	Clear SYSTICK exception pending status	0x1

4.6.1.1.44 SCB_VTOR

Bit Field	Read/Write	Field Name	Description
31:7	RW	TBLOFF	Table offset value in code or RAM region. Must be multiple of 128

4.6.1.1.45 SCB_AIRCR

Bit Field	Read/Write	Field Name	Description
31:16	RW	VECTKEY	Access key for writing this register . Must be set to 0x05FA to write to the other register fields
15	R	ENDIANESS	Indicates endianness for data.
10:8	RW	PRIGROUP	Priority group setting. Controls how many bits in the priority register

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
			are used for pre-empt priority vs. sub-priority
2	W	SYSRESETREQ	Requests a chip-level system reset
1	W	VECTCLRACTIVE	Clears all active exception state information
0	W	VECTRESET	Reset the Arm Cortex-M3

Field Name	Value Symbol	Value Description	Hex Value
VECTKEY	NVIC_VECTKEY_UNLOCK	Unlocks the NVIC_APP_INT_CTRL register	0x5FA
	NVIC_VECTKEY_UNLOCK_READBACK	Read back value when reading the VECTKEY field after unlocking	0xFA05*
ENDIANESS	NVIC_LITTLE_ENDIAN	Indicates core is configured for little endian data	0x0*
PRIGROUP	NVIC_PRIGROUP_4_0	Indicates 4 bits for pre-emption priority and 0 bits for sub-priority	0x3
	NVIC_PRIGROUP_3_1	Indicates 3 bits for pre-emption priority and 1 bit for sub-priority	0x4
	NVIC_PRIGROUP_2_2	Indicates 2 bits for pre-emption priority and 2 bits for sub-priority	0x5
	NVIC_PRIGROUP_1_3	Indicates 1 bit for pre-emption priority and 3 bits for sub-priority	0x6
	NVIC_PRIGROUP_0_4	Indicates 0 bits for pre-emption priority and 4 bits for sub-priority	0x7*
SYSRESETREQ	NVIC_SYSRESET	Requests a chip-level system reset	0x1
VECTCLRACTIVE	NVIC_CLEAR_ACTIVE_VECTORS	Clears all active exception state information	0x1
VECTRESET	NVIC_VECTRESET	Reset the Arm Cortex-M3	0x1

4.6.1.1.46 SCB_SCR

Bit Field	Read/Write	Field Name	Description
4	RW	SEVONPEND	Set to 1 to cause the WFE to wake up if a new interrupt is pending
2	RW	SLEEPDEEP	Enable SLEEPDEEP output signal when entering sleep mode
1	RW	SLEEPONEXIT	Enable sleep on exit feature when returning from handler to thread mode

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
SLEEPDEEP	NVIC_SLEEPDEEP_DISABLE	Disable SLEEPDEEP output signal when entering sleep mode	0x0*
	NVIC_SLEEPDEEP_ENABLE	Enable SLEEPDEEP output signal when entering sleep mode	0x1
SLEEPONEXIT	NVIC_SLEEPONEXIT_DISABLE	Disable sleep on exit feature	0x0*
	NVIC_SLEEPONEXIT_ENABLE	Enable sleep on exit feature	0x1

4.6.1.1.47 SCB_CCR

Bit Field	Read/Write	Field Name	Description
9	RW	STKALIGN	Force stack to be double-word aligned
8	RW	BFHFNMIEN	Ignore data bus faults in hard fault and NMI exceptions
4	RW	DIV_0_TRP	Trap on divide by zero
3	RW	UNALIGN_TRP	Trap on unaligned data access
1	RW	USERSETMPEND	Allow user code to write to software trigger interrupt register
0	RW	NONBASETHRDENA	Non base thread mode enable

Field Name	Value Symbol	Value Description	Hex Value
STKALIGN	NVIC_STKALIGN_DISABLE	Disable double-word stack alignment	0x0*
	NVIC_STKALIGN_ENABLE	Enable double-word stack alignment	0x1
BFHFNMIEN	NVIC_BFHFNMIEN_DISABLE	Disable ignoring of data bus faults in hard fault and NMI exception handlers. Bus faults in these handlers cause a lock-up	0x0*
	NVIC_BFHFNMIEN_ENABLE	Enable ignoring of data bus faults in hard fault and NMI exception handlers	0x1
DIV_0_TRP	NVIC_DIV_0_TRP_DISABLE	Disable trapping on divide by zero	0x0*
	NVIC_DIV_0_TRP_ENABLE	Enable trapping on divide by zero	0x1
UNALIGN_TRP	NVIC_UNALIGN_TRP_DISABLE	Disable trapping on unaligned data access	0x0*
	NVIC_UNALIGN_TRP_ENABLE	Enable trapping on unaligned data access	0x1
USERSETMPEND	NVIC_USERSETMPEND_DISABLE	Disable user code access to software trigger interrupt register	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	NVIC_USERSETMPEND_ENABLE	Enable user code access to software trigger interrupt register	0x1
NONBASETHRDENA	NVIC_NONBASETHRDENA_DISABLE	Only allow thread mode on return from last exception	0x0*
	NVIC_NONBASETHRDENA_ENABLE	Allow thread mode from any handler level by controlled return value	0x1

4.6.1.1.48 SCB_SHP0

Bit Field	Read/Write	Field Name	Description
23:16	RW	NVIC_USAGE_FAULT_PRIORITY	Configure the usage fault priority
15:8	RW	NVIC_BUS_FAULT_PRIORITY	Configure the bus fault priority
7:0	RW	NVIC_MEM_FAULT_PRIORITY	Configure the memory fault priority

4.6.1.1.49 SCB_SHP1

Bit Field	Read/Write	Field Name	Description
31:24	RW	NVIC_SVC_PRIORITY	Configure the SVC priority

4.6.1.1.50 SCB_SHP2

Bit Field	Read/Write	Field Name	Description
31:24	RW	NVIC_SYSTICK_PRIORITY	Configure the SysTick interrupt priority
23:16	RW	NVIC_PENDSV_PRIORITY	Configure the PendSV priority
7:0	RW	NVIC_MONITOR_PRIORITY	Configure the monitor priority

4.6.1.1.51 SCB_SHCSR

Bit Field	Read/Write	Field Name	Description
18	RW	USGFAULTENA	Usage fault handler enable
17	RW	BUSFAULTENA	Bus fault handler enable
16	RW	MEMFAULTENA	Memory management fault handler enable
15	RW	SVCALLPENDE	SVCALL is pending or was started and replaced by a higher priority exception
14	RW	BUSFAULTPENDE	Bus fault is pending or was started and replaced by a higher priority exception
13	RW	MEMFAULTPENDE	Memory management fault is pending or was started and replaced

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
			by a higher priority exception
12	RW	USGFAULTPENDE	Usage fault is pending or was started and replaced by a higher priority exception
11	RW	SYSTICKACT	SYSTICK exception handler is active
10	RW	PENDSVACT	PendSV exception handler is active
8	RW	MONITORACT	Debug monitor exception handler is active
7	RW	SVCALLACT	SVCALL exception handler is active
3	RW	USGFAULTACT	Usage fault exception handler is active
1	RW	BUSFAULTACT	Bus fault exception handler is active
0	RW	MEMFAULTACT	Memory management fault exception handler is active

Field Name	Value Symbol	Value Description	Hex Value
USGFAULTENA	NVIC_USGFAULT_DISABLE	Disable usage fault handler	0x0*
	NVIC_USGFAULT_ENABLE	Enable usage fault handler	0x1
BUSFAULTENA	NVIC_BUSFAULT_DISABLE	Disable bus fault handler	0x0*
	NVIC_BUSFAULT_ENABLE	Enable bus fault handler	0x1
MEMFAULTENA	NVIC_MEMFAULT_DISABLE	Disable memory management fault handler	0x0*
	NVIC_MEMFAULT_ENABLE	Enable memory management fault handler	0x1
SVCALLPENDE	NVIC_SVCALL_CLRPEND	SVCALL handler not pending	0x0*
	NVIC_SVCALL_SETPEND	SVCALL handler is pending	0x1
BUSFAULTPENDE	NVIC_BUSFAULT_CLRPEND	Bus fault handler not pending	0x0*
	NVIC_BUSFAULT_SETPEND	Bus fault handler is pending	0x1
MEMFAULTPENDE	NVIC_MEMFAULT_CLRPEND	Memory management fault handler not pending	0x0*
	NVIC_MEMFAULT_SETPEND	Memory management fault handler is pending	0x1
USGFAULTPENDE	NVIC_USGFAULT_CLRPEND	Usage fault handler not pending	0x0*
	NVIC_USGFAULT_SETPEND	Usage fault handler is pending	0x1
SYSTICKACT	NVIC_SYSTICK_ACTIVE	SysTick exception handler is active	0x1
PENDSVACT	NVIC_PENDSV_ACTIVE	PendSV exception handler is active	0x1
MONITORACT	NVIC_MONITOR_ACTIVE	Debug monitor exception handler is	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		active	
SVCALLACT	NVIC_SVCALL_ACTIVE	SVCALL exception is active	0x1
USGFAULTACT	NVIC_USGFAULT_ACTIVE	Usage fault exception handler is active	0x1
BUSFAULTACT	NVIC_BUSFAULT_ACTIVE	Bus fault exception handler is active	0x1
MEMFAULTACT	NVIC_MEMFAULT_ACTIVE	Memory management fault exception handler is active	0x1

4.6.1.1.52 SCB_CFSR

Bit Field	Read/Write	Field Name	Description
25	RW	DIVBYZERO	Indicates divide by zero has occurred
24	RW	UNALIGNED	Indicates unaligned access has occurred
19	RW	NOCP	Attempt to execute a coprocessor instruction
18	RW	INVPC	Attempt to execute exception with bad value in EXC_RETURN number
17	RW	INVSTATE	Attempt to switch to invalid (e.g. ARM) state
16	RW	UNDEFINSTR	Attempt to execute an undefined instruction
15	R	BFARVALID	Indicates if bus fault address register is valid
12	RW	STKERR	Indicates stacking error
11	RW	UNSTKERR	Indicates unstacking error
10	RW	IMPRECISERR	Indicates imprecise data access violation
9	RW	PRECISERR	Indicates precise data access violation
8	RW	IBUSERR	Indicates instruction access violation
7	R	MMARVALID	Indicates if memory management fault address register is valid
4	RW	MSTKERR	Indicates stacking error
3	RW	MUNSTKERR	Indicates unstacking error
1	RW	DACCVIOL	Indicates data access violation
0	RW	IACCVIOL	Indicates instruction access violation

4.6.1.1.53 SCB_HFSR

Bit Field	Read/Write	Field Name	Description
31	RW	DEBUGEVT	Indicates hard fault is triggered by debug event
30	RW	FORCED	Indicates hard fault is taken because of a lower priority (e.g., bus, memory management or usage) fault
1	RW	VECTBL	Indicates hard fault is taken due to failed vector fetch

4.6.1.1.54 SCB_DFSR

Bit Field	Read/Write	Field Name	Description
4	RW	EXTERNAL	Indicates external debug request signal asserted
3	RW	VCATCH	Indicates vector fetch occurred
2	RW	DWTTRAP	Indicates DWT match occurred
1	RW	BKPT	Indicates BKPT instruction executed
0	RW	HALTED	Indicates halt requested by NVIC

4.6.1.1.55 SCB_MMAR

Bit Field	Read/Write	Field Name	Description
31:0	R	NVIC_MMAR	

4.6.1.1.56 SCB_BFAR

Bit Field	Read/Write	Field Name	Description
31:0	R	NVIC_BFAR	

4.6.1.2 Interrupt Controller Type Register

The Interrupt Controller Type register (SCnSCB ICTR) indicates the number of interrupts supported by the NVIC. This register is from the Arm Cortex-M3 processor's SCnSCB register block. The ["Interrupt Controller Type Register Bit Assignments" table \(Table 5\)](#) describes this register.

Table 5. Interrupt Controller Type Register Bit Assignments

Bit Field	Field Name	Description
4:0	INTLINESNUM	Total number of interrupt line groups: Ezairo 8300 uses 69 external interrupts (NVIC_INTLINESNUM_65_96, hex value 0x2)

4.6.1.3 Interrupt Set Enable and Clear Enable Registers

Use the Interrupt Set Enable registers (NVIC_ISER[0] to NVIC_ISER[2]) to:

- Enable interrupts
- Determine which interrupts are currently enabled

Use the Interrupt Clear-Enable registers (NVIC_ICER[0] to NVIC_ICER[2]) to:

- Disable interrupts
- Determine which interrupts are currently disabled

These registers contain a bit for each of the external interrupts (vectors 16 to 109) listed in the ["Interrupts in the Arm Cortex-M3 Processor" table \(Table 4\)](#). Setting a bit in an Interrupt Set-Enable register enables the corresponding interrupt. Setting a bit in an Interrupt Clear-Enable register disables the corresponding interrupt. This helps to efficiently clear interrupts without affecting the state of other interrupts.

Ezairo 8300 Hardware Reference

When the enable bit of a pending interrupt is set, the processor activates the interrupt based on its priority. When the enable bit is cleared, asserting its interrupt signal pends the interrupt, but it is not possible to activate the interrupt, regardless of its priority. Therefore, a disabled interrupt can serve as a latched general-purpose I/O bit that can be read and cleared without invoking an interrupt.

NOTE: Clearing an Interrupt Set-Enable Register bit does not affect currently active interrupts. It only prevents new activations.

These registers are part of the Arm Cortex-M3 processor's *NVIC* register block. The "Interrupt Set-Enable Register Bit Assignments" table (Table 6) describes the field of the Interrupt Set-Enable registers. The "Interrupt Clear-Enable Register Bit Assignments" table (Table 7) describes the field of the Interrupt Clear-Enable registers.

Table 6. Interrupt Set-Enable Register Bit Assignments

Bit Field	Field Name	Description
31:0	SETENA	Interrupt set enable bits. For write operation: 1 = Enable interrupt 0 = No effect. For read operation: 1 = Interrupt enabled 0 = Interrupt disabled Writing 0 to a SETENA bit has no effect. Reading the bit returns its current enable state. Reset clears the SETENA fields.

Table 7. Interrupt Clear-Enable Register Bit Assignments

Bit Field	Field Name	Description
31:0	CLRENA	Interrupt clear-enable bits. For write operation: 1 = Disable interrupt 0 = No effect. For read operation: 1 = Enable interrupt 0 = Disable interrupt. Writing 0 to a CLRENA bit has no effect. Reading the bit returns its current enable state.

4.6.1.4 Interrupt Set-Pending Registers and Interrupt Clear-Pending Registers

Use the Interrupt Set-Pending Registers (*NVIC_ISPR[0]* to *NVIC_ISPR[2]*) to:

- Force interrupts into the pending state
- Determine which interrupts are currently pending

Use the Interrupt Clear-Pending Registers (*NVIC_ICPR[0]* to *NVIC_ICPR[2]*) to:

- Clear pending interrupts
- Determine which interrupts are currently pending

Ezairo 8300 Hardware Reference

These registers contain a bit for each of the external interrupts (vectors 16 to 109) listed in the "Interrupts in the Arm Cortex-M3 Processor" table (Table 4). Setting a bit in an Interrupt Set-Pending register causes the corresponding interrupt to be pending. Setting a bit in an Interrupt Clear-Pending register puts the interrupt into the non-pending state.

NOTE: Writing to an Interrupt Set-Pending register has no effect on an interrupt that is already pending or is disabled. Similarly, writing to an Interrupt Clear-Pending register has no effect on an interrupt that is active, unless it is also pending.

These registers are part of the Arm Cortex-M3 processor's NVIC register block. The "Interrupt Set-Pending Register Bit Assignments" table (Table 8) describes the Interrupt Set-Pending registers. The "Interrupt Clear-Pending Registers Bit Assignments" table (Table 9) describes the Interrupt Clear-Pending registers.

Table 8. Interrupt Set-Pending Register Bit Assignments

Bit Field	Field Name	Description
31:0	SETPEND	Interrupt set-pending bits: 1 = Pend the corresponding interrupt 0 = Corresponding interrupt not pending Writing 0 to a SETPEND bit has no effect. Reading the bit returns its current state.

Table 9. Interrupt Clear-Pending Registers Bit Assignments

Bit Field	Field Name	Description
31:0	CLRPEND	Interrupt clear-pending bits: 1 = Clear pending interrupt 0 = Do not clear pending interrupt Writing 0 to a CLRPEND bit has no effect. Reading the bit returns its current state.

4.6.1.5 Active Bit Register

Read the Active Bit registers (NVIC_IABR[0] to NVIC_IABR[2]) to determine which interrupts are active. This register contains a flag for each of the external interrupts (vectors 16 to 109) listed in the "Interrupts in the Arm Cortex-M3 Processor" table (Table 4). These registers are part of the Arm Cortex-M3 processor's NVIC register block. The "Active Bit Register Bit Assignments" table (Table 10) describes the Active Bit register.

Table 10. Active Bit Register Bit Assignments

Bit Field	Field Name	Description
31:0	ACTIVE	Interrupt active flags: 1 = Interrupt active or pre-empted and stacked 0 = Interrupt not active nor stacked.

Ezairo 8300 Hardware Reference

4.6.1.6 Interrupt Priority Registers

Use the Interrupt Priority Registers (NVIC_IP[0] to NVIC_IP[23]) to assign a priority to each of the available interrupts. Each byte in an Interrupt Priority can be used to set the priority for one of the external interrupts (vectors 16 to 84) listed in the "Interrupts in the Arm Cortex-M3 Processor" table (Table 4).

NOTE: Configuration of the interrupt priorities for standard Arm Cortex-M3 processor exceptions (vectors 4 to 15) are set using the System Handler Priority registers. For more information, see the *ARMv7M Architecture Reference Manual*.

The NVIC for the Arm Cortex-M3 processor in the Ezairo 8300 system has been implemented with three interrupt priority bits per interrupt. These three priority bits are MSB aligned in an eight-bit priority bit field as required by Arm. Generally, the lower the priority value, the higher the priority that interrupt is given.

The SCB_AIRCR_PRIGROUP bit field from the Application Interrupt and Reset Control register (see the *ARMv7M Architecture Reference Manual*) is used to divide the interrupt priority settings into interrupt groups, and to prioritize interrupts within those groups. The possible configurations for the division of the priority bit field into pre-emption priority and subgroup priority are shown in the "Division of Priority into Pre-Empt Priority and Subgroup Priority" table (Table 11).

Table 11. Division of Priority into Pre-Empt Priority and Subgroup Priority

PRIGROUP Setting	Pre-Empt Priority Field	Subgroup Priority Field
0x0 to 0x4	7:5	-
0x5	7:6	5
0x6	7	6:5
0x7	-	7:5

When choosing which interrupt to activate, the priority settings are applied as follows:

- When multiple interrupts are pending, but no interrupts are active, the interrupt with the lowest priority setting is activated. If more than one pending interrupt shares the lowest priority setting, the interrupt with the lower vector number is activated.
- If an interrupt is currently active, it can be pre-empted by any interrupt that is pending in a lower numbered group. If multiple interrupts that could pre-empt the active interrupt are pending, the interrupt with the lowest priority setting is activated. As before, if more than one pending interrupt shares the lowest priority setting, the interrupt with the lower vector number is activated.

For example, setting the SCB_AIRCR_PRIGROUP bit field to 0x6 divides the three interrupt priority bits, so that bits [7:6] are used to assign the interrupt to a group, and bit [5] assigns the interrupt a priority relative to the other interrupts in that group. Suppose the following interrupts are pending:

- TIMER_0 with priority set to 0xC0
- DMA_0 with priority set to 0xB0
- DIO_0 with priority set to 0xB0

In this example, the `DIO_0` and `DMA_0` interrupts have the lowest priority, and the `DMA0` interrupt is activated because it has a lower vector number (24 versus 32). If the `TIMER_0` interrupt is then pended with a priority of 0x80, the `DMA_0` interrupt is not pre-empted because the `TIMER_0`, `DMA_0`, and `DIO_0` interrupts all share the same priority group. If the `WATCHDOG` interrupt is then pended with a priority level of 0x20, the `DMA_0` interrupt is pre-empted because the `WATCHDOG` interrupt belongs to a higher priority interrupt group.

IMPORTANT: The reset, NMI and fault vectors have priority levels of -3, -2, and -1 respectively. As such, these events can always pre-empt interrupts with lower priorities.

These registers are part of the Arm Cortex-M3 processor's NVIC register block. The "Interrupt Priority Register Bit Assignments" table (Table 12) describes the bit assignments for a single interrupt within the Interrupt Priority registers.

Table 12. Interrupt Priority Register Bit Assignments

Bit Field	Field Name	Description
7:5	PRI_n	Priority of interrupt <i>n</i>

4.6.1.7 Registers Described by Arm Documentation

The following registers are documented in the *ARMv7M Architecture Reference Manual*:

- Interrupt Control State register
- Vector Table Offset register
- Application Interrupt and Reset Control register
- System Control register
- Configuration Control register
- System Handler Priority registers
- System Handler Control and State register
- Configurable Fault Status register
- Software Trigger Interrupt register

4.6.2 SysTick

The Arm Cortex-M3 core peripherals include the system tick (SysTick) count-down timer from the Arm Cortex-M3 processor implementation. This block is implemented as part of the NVIC, and is described in the *Cortex-M3 Technical Reference Manual*.

The clock used by the SysTick timer is selected using the `SysTick_CTRL_CLKSOURCE` bit from the `SysTick_CTRL` register. This timer can be clocked from the system clock (SYSCLK) or from the SysTick-specific reference clock (STCLK) that is divided from SLOWCLK by 32 while the core is active, or divided from SYSCLK by 2 when STANDBYCLK is the source for SYSCLK. For more information about SYSCLK, see Section 10.2.1 "System Clock (SYSCLK)" on page 313. For more information about SLOWCLK, see Section 10.2.4 "Slow Clock (SLOWCLK)" on page 315. The `SysTick_CALIB` register is configured to define a 10 ms timer period based on STCLK.

The delay provided by the SysTick timer is defined using the selected clock and a reload value loaded to the `SysTick_LOAD` register, as follows:

$$\text{DELAY} = \frac{(\text{SysTick_LOAD} + 1)}{f_{\text{SYS CLK or STCLK}}}$$

Ezairo 8300 Hardware Reference

The current value of the SysTick counter can be read at any time from the `SysTick_VAL` register.

The SysTick timer is enabled by setting the `SysTick_CTRL_ENABLE` bit in the `SysTick_CTRL` register. SysTick interrupts are enabled by setting the `SysTick_CTRL_TICKINT` bit in the `SysTick_CTRL` register. The `SysTick_CTRL_COUNTFLAG` bit in the `SysTick_CTRL` register indicates if the SysTick timer has reached zero since the last read of this register, and is cleared automatically after being read. This bit can be used if an application uses polling instead of interrupting to monitor for SysTick timer events.

IMPORTANT: If the SysTick timer is sourced from `SYSClk`, and the clock to the Arm Cortex-M3 processor is gated due to the use of a wait-for-interrupt (`WFI`) or wait-for-event (`WFE`) instruction, the SysTick timer is clocked at a much slower rate while waiting for the interrupt or event to occur. If using these instructions, we recommend using `STCLK` as the source for the SysTick timer, or using a general-purpose timer running at `SLOWCLK` divided by 2 if the clock source needs to be faster.

4.6.2.1 SysTick Control and Configuration Registers

Register Name	Register Description	Address
<code>SysTick_CTRL</code>	SYSTICK Control and Status Register	P:0x28004804 0xE000E010
<code>SysTick_LOAD</code>	SYSTICK Reload Value Register	P:0x28004805 0xE000E014
<code>SysTick_VAL</code>	SYSTICK Current Value Register	P:0x28004806 0xE000E018
<code>SysTick_CALIB</code>	SYSTICK Calibration Register	P:0x28004807 0xE000E01C

IMPORTANT: The SysTick timer is a standard component provided with the Arm Cortex-M3 processor. The registers for this peripheral are defined in `core_cm3.h` and augmented by defines for the bit fields, bit settings, and sub-registers in `sk5_hw.h`.

4.6.2.1.1 SysTick_CTRL

Bit Field	Read/Write	Field Name	Description
16	R	COUNTFLAG	Reads as 1 if SYSTICK counter has reached 0 since the last time the timer has reached 0. Clears to 0 on read
2	RW	CLKSOURCE	SYSTICK timer clock source
1	RW	TICKINT	SYSTICK timer interrupt enable
0	RW	ENABLE	SYSTICK timer enable

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
COUNTFLAG	SYSTICK_COUNTFLAG_NOT_ZERO	SYSTICK counter has not reached zero since last read	0x0*
	SYSTICK_COUNTFLAG_ZERO	SYSTICK counter has reached zero since last read	0x1
CLKSOURCE	SYSTICK_CLKSOURCE_EXTREF_CLK	Use external reference clock (STCLK)	0x0*
	SYSTICK_CLKSOURCE_CORE_CLK	Use the core clock	0x1
TICKINT	SYSTICK_TICKINT_DISABLE	Disable interrupt generation when SYSTICK timer reaches 0	0x0*
	SYSTICK_TICKINT_ENABLE	Enable interrupt generation when SYSTICK timer reaches 0	0x1
ENABLE	SYSTICK_DISABLE	Disable SYSTICK timer	0x0*
	SYSTICK_ENABLE	Enable SYSTICK Timer	0x1

4.6.2.1.2 SysTick_LOAD

Bit Field	Read/Write	Field Name	Description
23:0	RW	RELOAD	Counter reload value for the SYSTICK timer when it reaches 0

4.6.2.1.3 SysTick_VAL

Bit Field	Read/Write	Field Name	Description
23:0	RW	CURRENT	Current value of the SYSTICK counter value. Write to clear counter

4.6.2.1.4 SysTick_CALIB

Bit Field	Read/Write	Field Name	Description
31	R	NOREF	Indicates if a reference clock is available
30	R	SKEW	Indicates if calibration value is exactly 10 ms or not
23:0	R	TENMS	SYSTICK counter calibration value for 10 ms. A value of 0 means the calibration value is not available

Field Name	Value Symbol	Value Description	Hex Value
NOREF	SYSTICK_NOREF	No external reference clock available	0x1
	SYSTICK_REF	External reference clock available	0x0*
SKEW	SYSTICK_SKEW	Calibration value is not exactly 10 ms	0x1
	SYSTICK_NOSKEW	Calibration value is exactly 10 ms	0x0*

Ezairo 8300 Hardware Reference

4.6.3 Debug Controller

4.6.3.1 Halting Debug Configuration and Status

The Debug Halting Control and Status Register (DHCSR) provides status information on the processor state, enables core debugging, and allows an external system to halt and single-step the core. To write to this register, `DEBUG_HALT_KEY` must be written to the `CoreDebug_DHCSR_DBGKEY` bit field.

The DHCSR is used to configure the Arm Cortex-M3 processor for halting debug. To enable halting debug, set the `CoreDebug_DHCSR_C_DEBUGEN` bit. If halting debug is enabled:

- To halt the core, set the `CoreDebug_DHCSR_C_HALT` bit.
- To single-step the core, set the `CoreDebug_DHCSR_C_STEP` bit.
- To mask interrupts while single-stepping, set the `CoreDebug_DHCSR_C_MASKINTS` bit (the core must be halted to write to this bit).
- To break a stalled memory access, where the memory access might be stalled due to a memory conflict with another component of the Ezairo 8300 system, set the `CoreDebug_DHCSR_C_SNAPALL` bit.

The Debug Exception and Monitor Control Register (DEMCR) contains a number of possible exception conditions that the debug tools might want to monitor during debug. These vector catch configuration bits, when enabled, monitor for the specified fault or reset event. When a fault or event that is being monitored is detected, a core halt request is used to halt the core as soon as the currently executing instruction completes. Supported vector catch events include debug traps that trigger on:

- A core reset
- A memory management fault
- Usage faults for:
 - No coprocessor errors
 - Unaligned accesses or division by 0
 - State errors
- A bus fault
- Errors when handling an interrupt or exception
- A hard fault

The DHCSR also provides a variety of debug related status information, including:

- If the core has been reset or is resetting (`CoreDebug_DHCSR_S_RESET_ST`); this bit is cleared when read
- If an instruction has completed execution since this register has been last read; this bit is cleared when read
- If the core is in a locked state
- If the core is in Sleep Mode
- If the core has been halted
- If the most recent register read/write has completed; for more information, see [Section 4.6.3.3 “Arm Cortex-M3 Processor Core Register Access” on page 200](#)

NOTE: The DHCSR and all DEMCR bits that are not related to the debug monitor are only reset if a POR or similar occurs. These registers are not reset for a core reset. For more information about resets, see [Section 9.3 “Resets” on page 299](#).

CAUTION: We strongly recommend that only the debugger use the DHCSR, because accesses to this register from application code can interfere with the debug behavior of the Arm Cortex-M3 processor debug port.

4.6.3.2 Debug Monitor Configuration

The NVIC from the Arm Cortex-M3 processor contains a debug monitor that can be used to control debug activities. The debug monitor is tied to the debug monitor system interrupt (vector number 12), and is configured using the DEMCR register. To enable the debug monitor and debug monitor exception, set the CoreDebug_DEMCR_MON_EN bit from the DEMCR register. To manually pend the debug monitor exception, set the CoreDebug_DEMCR_MON_PEND bit from the DEMCR register. To single-step the core using the debug monitor (if the debug monitor is enabled), set the CoreDebug_DEMCR_MON_STEP bit from the DEMCR register.

The CoreDebug_DEMCR_MON_REQ bit from the DEMCR register indicates whether a debug monitor event has been caused by a manual request, or by a debug event (including a debug trap).

4.6.3.3 Arm Cortex-M3 Processor Core Register Access

The Arm Cortex-M3 processor debug port includes a pair of registers that the debug port uses to provide read and write access to the Arm Cortex-M3 processor's core registers: the Debug Core Register Selector Register (DCRSR) and the Debug Core Register Data Register (DCRDR). The DCRSR contains the selection of the register to be read or written to, and the type of access used. To define the read/write direction, write REGWNR_READ or REGWNR_WRITE to the CoreDebug_DCSR_REGWNR bit field. To set the register to be read, use the REGSEL_* bit settings for the CoreDebug_DCSR_REGSEL bit field from the DCRSR.

Data written using the DCRSR is copied from the DCRDR to the specified core register. Similarly, data read using the DCRSR is written to the DCRDR, where it can be accessed using debug port memory reads. If the selector register selects the core special registers, the data read or written is interpreted using the bit fields described in the "Debug Core Register Data Register Special Register Mapping" table (Table 13).

Table 13. Debug Core Register Data Register Special Register Mapping

Bit Field	Arm Cortex-M3 Processor Register
31:24	CONTROL
23:16	FAULTMASK
15:8	BASEPRI
7:0	PRIMASK

4.6.3.4 Debug Fault Status Register

The Debug Fault Status register (DFSR) is used to monitor debug events including:

- External debug requests
- Vector catches
- Data watchpoint matches
- BKPT instruction execution
- Halt requests

Each flag in the Debug Fault Status register is set independently when its debug condition occurs. The bits in this register are not set unless the event is caught. One of four things occurs if an event is detected:

If halting debug is enabled

Debug events stop the processor by going into debug mode.

If halting debug is disabled and the debug monitor is enabled

Debug events trigger a debug monitor handler call, if priority permits.

If halting debug and the debug monitor are both disabled

Some debug events are interpreted as Hard Faults, setting the SCB_HFSR_DBGEVT bit in the Hard Fault status register. This always includes BKPT events, and includes VCATCH events (see TBD for descriptions of the events) if those events are enabled in the Debug Exception and Monitor Control Register (see TBD).

If not interpreted as a hard fault, the debug event is ignored.

This register is part of the Arm Cortex-M3 processor's SCB register block.

4.6.3.5 Arm Cortex-M3 Processor Debug Port Specific Control and Configuration Registers

Register Name	Register Description	Address
DEBUG_DHCSR	Debug Halting Control and Status Register	P:0x28004B7C 0xE000EDF0
DEBUG_DCRSR	Debug Core Register Selector Register	P:0x28004B7D 0xE000EDF4
DEBUG_DCRDR	Debug Core Register Data Register	P:0x28004B7E 0xE000EDF8
DEBUG_DEMCR	Debug Exception and Monitor Control Register	P:0x28004B7F 0xE000EDFC

IMPORTANT: The Arm Cortex-M3 core debug port components are part of a standard component provided with the Arm Cortex-M3 processor. The registers for this peripheral are defined in *core_cm3.h* and augmented by defines for the bit fields, bit settings, and sub-registers in *sk5_hw.h*. These registers are only accessible through the Arm Cortex-M3 processor's private peripheral bus (see [Section 11.2 “Memory Architecture” on page 336](#)).

4.6.3.5.1 DEBUG_DHCSR

Bit Field	Read/Write	Field Name	Description
31:16	W	DBGKEY	Debug key must be written to this field in order to write the rest of the register
25	R	S_RESET_ST	Core has been reset or is being reset. Bit is cleared on read
24	R	S_RETIRE_ST	Instruction is completed since last read. Bit is cleared on read
19	R	S_LOCKUP	Indicates if core is in lockup state
18	R	S_SLEEP	Indicates if core is in sleep mode

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
17	R	S_HALT	Indicates is core is halted
16	R	S_REGRDY	Indicates register read/write operation is completed
5	RW	C_SNAPSTALL	Set to break a stalled memory access
3	RW	C_MASKINTS	Mask interrupts while stepping
2	RW	C_STEP	Single step the processor
1	RW	C_HALT	Halt the processor
0	RW	C_DEBUGEN	Enable halt mode debugging

Field Name	Value Symbol	Value Description	Hex Value
DBGKEY	DEBUG_HALT_KEY	DEBUG_HALT_KEY	0xA05F

4.6.3.5.2 DEBUG_DCRSR

Bit Field	Read/Write	Field Name	Description
16	W	REGWnR	Indicates direction of register transfer
4:0	W	REGSEL	Indicates register to be accessed

Field Name	Value Symbol	Value Description	Hex Value
REGWnR	REGWNR_READ	Indicates register read	0x0*
	REGWNR_WRITE	Indicates register write	0x1
REGSEL	REGSEL_R0	Select R0	0x0*
	REGSEL_R1	Select R1	0x1
	REGSEL_R2	Select R2	0x2
	REGSEL_R3	Select R3	0x3
	REGSEL_R4	Select R4	0x4
	REGSEL_R5	Select R5	0x5
	REGSEL_R6	Select R6	0x6
	REGSEL_R7	Select R7	0x7
	REGSEL_R8	Select R8	0x8
	REGSEL_R9	Select R9	0x9
	REGSEL_R10	Select R10	0xA
	REGSEL_R11	Select R11	0xB

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	REGSEL_R12	Select R12	0xC
	REGSEL_R13	Select R13	0xD
	REGSEL_R14	Select R14	0xE
	REGSEL_R15	Select R15	0xF
	REGSEL_PSRFLGS	Select xPSR/flags	0x10
	REGSEL_MSP	Select main stack pointer	0x11
	REGSEL_PSP	Select process stack pointer	0x12
	REGSEL_SPECREG	Access other registers including control, FAULTMASK, BASEPRI and PRIMASK	0x14

4.6.3.5.3 DEBUG_DCRDR

Bit Field	Read/Write	Field Name	Description
31:0	RW	DEBUG_REGDATA	Register read/write data for debugging

4.6.3.5.4 DEBUG_DEMCR

Bit Field	Read/Write	Field Name	Description
24	RW	TRCENA	Trace system enable
19	RW	MON_REQ	Indicates that the debug monitor is caused by a manual pending request rather than a hardware event
18	RW	MON_STEP	Single step the processor
17	RW	MON_PEND	Pend the monitor exception request
16	RW	MON_EN	Enable the debug monitor exception
10	RW	VC_HARDERR	Debug trap on hard faults
9	RW	VC_INTERR	Debug trap on interrupt service errors
8	RW	VC_BUSERR	Debug trap on bus faults
7	RW	VC_STATERR	Debug trap on usage fault state errors
6	RW	VC_CHKERR	Debug trap on fault-enabled checking errors (e.g. unaligned access, divide by zero, etc.)
5	RW	VC_NOCPERR	Debug trap on usage fault no coprocessor errors
4	RW	VC_MMERR	Debug trap on memory management fault
0	RW	VC_CORERESSET	Debug trap on core reset

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
MON_PEND	DEBUG_MON_CLRPEND	Clear the pending status of the debug monitor exception	0x0*
	DEBUG_MON_SETPEND	Set pending status of the debug monitor exception	0x1
MON_EN	DEBUG_MON_DISABLE	Disable debug monitor exceptions	0x0*
	DEBUG_MON_ENABLE	Enable debug monitor exceptions	0x1

CHAPTER 5

HEAR Configurable Accelerator

The HEAR Configurable Accelerator is a core that provides additional signal processing resources to the Ezairo 8300 system. Unlike the CFX DSP, the Arm Cortex-M3 processor, and the LPDSP32 processor, the HEAR is not programmable, does not interact directly with any external interfaces, and does not include direct debugging access for applications that make use of the core. For debugging algorithms that use the HEAR, the debug controller can view some information on the HEAR through memories available on the memory buses attached to the CFX or the Arm Cortex-M3 processor.

5.1 ACCELERATOR OVERVIEW

For information about the HEAR's hardware features, available microcode modules, and how to configure the HEAR Configurable Accelerator to work with a given microcode configuration, see the *HEAR Configurable Accelerator Reference Manual for Ezairo 8300*. Further hardware details regarding the function chain controller can be found in [Section 5.2 “Function Chain Controller” on page 207](#), and more information about the block floating-point feature that can be used with some microcode configurations is in [Section 5.3 “Block Floating-Point” on page 215](#).

5.1.1 Function Chains

The HEAR is a configurable 24-bit processing unit that runs data-processing functions defined in HEAR microcode configurations. These functions, which are typically used to process signal data within the system, are linked together into sequences of one or more functions known as microcode function chains. These function chains are executed by the HEAR Configurable Accelerator through triggered events, such as a command sent by the CFX DSP or Arm Cortex-M3 processor, or an interrupt from one of the configured FIFOs. These function chains normally operate on data stored in shared memory: usually an H $\times$ memory instance (see [Section 11.2 “Memory Architecture” on page 336](#) and [Chapter 12.1 “Introduction to the FIFO Controller” on page 387](#) for more information), an input FIFO, or an intermediate FIFO. Output from these chains is typically stored to another location in shared memory: usually another H $\times$ memory instance, an output FIFO, or an intermediate FIFO.

When any function completes, or when the end of a completed function chain is reached, the HEAR processor can send the controlling processor an interrupt through one of eight HEAR interrupts. These interrupts are used to notify the controlling processor when data is ready for additional processing, or to signal that the processor can issue another HEAR command.

5.1.2 HEAR Control

To pause and resume the HEAR processing, set the `SYSTEM_CTRL_HEAR_PAUSE` and `SYSTEM_CTRL_HEAR_RESUME` bits, respectively, in the `D_SYSTEM_CTRL` register. When the HEAR is paused, the accelerator clock is stopped, which prevents all function execution and all responses to the function chain controller. All incomplete accesses to shared memory by the HEAR are put into a non-blocking wait state, and these are completed once processing resumes on the HEAR. While the HEAR is paused, the CFX or the Arm Cortex-M3 processor and the IOCs can access shared memory at any address, without the HEAR accelerator introducing additional delays.

As bits in the `D_SYSTEM_CTRL` register only change state when set, there is no need to monitor the state of the other bits of this register when controlling the HEAR. However, the behavior of the system is undefined if both of these bits are set by the same instruction. Reading the `SYSTEM_HEAR_STATE_RUN` bit (which shares a bit position with the `SYSTEM_HEAR_STATE_PAUSE` bit) in the `D_SYSTEM_CTRL` register provides the current status of the HEAR.

IMPORTANT: While the HEAR is paused, function chain triggers are still collected in the `D_HEAR_FC_STATUS` register. When processing on the HEAR is resumed, these triggers cause their associated function chains to be executed, as described in [Section 5.2 “Function Chain Controller” on page 207](#).

Ezairo 8300 Hardware Reference

The HEAR can also be reset by setting the `SYSTEM_HEAR_RESET` bit in the `D_SYSTEM_CTRL` register. This resets the HEAR core and its peripherals, including the block floating point unit and the function chain controller.

IMPORTANT: The HEAR reset also resets the FIFO controllers, clearing the current FIFO register settings.

5.1.3 Control and Configuration Registers

Register Name	Register Description	Address
<code>D_SYSTEM_CTRL</code>	CFX/HEAR Configuration and Control	P:0x0800
<code>SYSTEM_CTRL</code>		0x40002000

5.1.3.1 D_SYSTEM_CTRL, SYSTEM_CTRL

Bit Field	Read/Write	Field Name	Description
9	R	<code>PRIORITY_STATUS</code>	CFX/HEAR priority status
8	R	<code>HEAR_RUNNING</code>	HEAR running status
7	W	<code>PRIORITY_HEAR</code>	Write a 1 to give HEAR priority
6	W	<code>PRIORITY_CFX</code>	Write a 1 to give CFX priority
5	W	<code>HEAR_PAUSE</code>	Write a 1 to pause HEAR
4	W	<code>HEAR_RESUME</code>	Write a 1 to run HEAR
3	W	<code>HEAR_RESET</code>	Write a 1 to reset HEAR
0	W	<code>WATCHDOG_REFRESH</code>	Write a 1 to reset the watchdog

Field Name	Value Symbol	Value Description	Hex Value
<code>PRIORITY_STATUS</code>	<code>SYSTEM_PRIORITY_STATE_HEAR</code>	HEAR has priority over CFX	0x0*
	<code>SYSTEM_PRIORITY_STATE_CFX</code>	CFX has priority over HEAR	0x1
<code>HEAR_RUNNING</code>	<code>SYSTEM_HEAR_STATE_PAUSE</code>	HEAR paused	0x0*
	<code>SYSTEM_HEAR_STATE_RUN</code>	HEAR running	0x1
<code>PRIORITY_HEAR</code>	<code>SYSTEM_PRIORITY_HEAR</code>	Give HEAR priority over CFX	0x1
<code>PRIORITY_CFX</code>	<code>SYSTEM_PRIORITY_CFX</code>	Give CFX priority over HEAR	0x1
<code>HEAR_PAUSE</code>	<code>SYSTEM_HEAR_PAUSE</code>	Pause HEAR	0x1
<code>HEAR_RESUME</code>	<code>SYSTEM_HEAR_RESUME</code>	Resume HEAR	0x1
<code>HEAR_RESET</code>	<code>SYSTEM_HEAR_RESET</code>	HEAR reset	0x1
<code>WATCHDOG_REFRESH</code>	<code>SYSTEM_WATCHDOG_REFRESH</code>	Watchdog reset	0x1

Ezairo 8300 Hardware Reference

5.2 FUNCTION CHAIN CONTROLLER

The CFX DSP configures the function chain controller of the HEAR to control the execution of the HEAR's configurable microcode function chains. Alternately, the Arm Cortex-M3 processor can configure the HEAR in the same way. For the remainder of this section, the CFX is used as the example processor for control, but similar behavior is possible with the Arm Cortex-M3 processor.

Function chains are typically associated with triggers that can be used to execute them. The function chain controller is directed to trigger specific function chains within a microcode configuration by setting bits in the `D_HEAR_FC_ENABLE` register.

The CFX DSP or Arm Cortex-M3 processor sends commands to start the execution of one or more of the HEAR function chains by setting the appropriate bit or bits in the `D_HEAR_CMDS_0_TO_7` and `D_HEAR_CMDS_8_TO_15` registers. The processing of function chains can also be triggered by HEAR FIFO interrupts in the system. In both cases, the microcode functions to be executed in the function chain depend on the microcode configuration used by the user application. Each function chain has an entry in a vector table stored at `D_HEAR_FC_VECT_BASE`, located in shared memory. These function chain vector table entries can be modified during an application's execution to change which function chains are mapped to the CFX commands and FIFO interrupts. For more information about the HEAR microcode, function chains, and possible microcode configurations, see the *HEAR Configurable Accelerator Reference Manual for Ezairo 8300*.

Setting the corresponding bit in the `D_HEAR_FC_PRIORITY` register prioritizes enabled function chains as either normal or high priority. This allows an application to select the function chains to process first, if more than one function chain is pending when the function chain controller initiates the processing of a function chain. If multiple function chains are pending, all high priority function chains are handled first. If multiple pending function chains share the same priority level, then function chains are executed with the following order of precedence:

1. Function chains triggered by CFX commands, starting with the lowest numbered command
2. Function chains triggered by FIFO events, starting with the lowest numbered FIFO function chain

The processing status of function chains can be observed through the `D_HEAR_FC_STATUS` register. The bit corresponding to a particular enabled function chain is set whenever the execution of the function chain is pending.

IMPORTANT: Writing to the `D_HEAR_FC_STATUS` register might result in undefined application behavior.

5.2.1 Control and Configuration Registers

Register Name	Register Description	Address
<code>D_HEAR_FC_ENABLE</code>	HEAR Function Chain Enable register	X:0x1230
<code>HEAR_FC_ENABLE</code>		0x208048C0
<code>D_HEAR_FC_PRIORITY</code>	HEAR Function Chain Priority register	X:0x1231
<code>HEAR_FC_PRIORITY</code>		0x208048C4
<code>D_HEAR_FC_STATUS</code>	HEAR Function Chain Status register	X:0x1237
<code>HEAR_FC_STATUS</code>		0x208048DC
<code>D_HEAR_CMDS_0_TO_7</code>	CFX issued HEAR commands 0-7	P:0x0804

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
HEAR_CMDS_0_TO_7		0x40002010
D_HEAR_CMDS_8_TO_15	CFX issued HEAR commands 8-15	P:0x0805
HEAR_CMDS_8_TO_15		0x40002014

5.2.1.1 D_HEAR_FC_ENABLE, HEAR_FC_ENABLE

Bit Field	Read/Write	Field Name	Description
23	RW	FIFO_7	Enable/disable HEAR FIFO function chain 7
22	RW	FIFO_6	Enable/disable HEAR FIFO function chain 6
21	RW	FIFO_5	Enable/disable HEAR FIFO function chain 5
20	RW	FIFO_4	Enable/disable HEAR FIFO function chain 4
19	RW	FIFO_3	Enable/disable HEAR FIFO function chain 3
18	RW	FIFO_2	Enable/disable HEAR FIFO function chain 2
17	RW	FIFO_1	Enable/disable HEAR FIFO function chain 1
16	RW	FIFO_0	Enable/disable HEAR FIFO function chain 0
15	RW	CMD15	Enable/disable CFX function chain 15
14	RW	CMD14	Enable/disable CFX function chain 14
13	RW	CMD13	Enable/disable CFX function chain 13
12	RW	CMD12	Enable/disable CFX function chain 12
11	RW	CMD11	Enable/disable CFX function chain 11
10	RW	CMD10	Enable/disable CFX function chain 10
9	RW	CMD9	Enable/disable CFX function chain 9
8	RW	CMD8	Enable/disable CFX function chain 8
7	RW	CMD7	Enable/disable CFX function chain 7
6	RW	CMD6	Enable/disable CFX function chain 6
5	RW	CMD5	Enable/disable CFX function chain 5
4	RW	CMD4	Enable/disable CFX function chain 4
3	RW	CMD3	Enable/disable CFX function chain 3
2	RW	CMD2	Enable/disable CFX function chain 2
1	RW	CMD1	Enable/disable CFX function chain 1
0	RW	CMD0	Enable/disable CFX function chain 0

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
FIFO_7	HEAR_FC_EBL_FIFO_7	Enable function chain FIFO_7	0x1
FIFO_6	HEAR_FC_EBL_FIFO_6	Enable function chain FIFO_6	0x1
FIFO_5	HEAR_FC_EBL_FIFO_5	Enable function chain FIFO_5	0x1
FIFO_4	HEAR_FC_EBL_FIFO_4	Enable function chain FIFO_4	0x1
FIFO_3	HEAR_FC_EBL_FIFO_3	Enable function chain FIFO_3	0x1
FIFO_2	HEAR_FC_EBL_FIFO_2	Enable function chain FIFO_2	0x1
FIFO_1	HEAR_FC_EBL_FIFO_1	Enable function chain FIFO_1	0x1
FIFO_0	HEAR_FC_EBL_FIFO_0	Enable function chain FIFO_0	0x1
CMD15	HEAR_FC_EBL_CMD15	Enable function chain CMD15	0x1
CMD14	HEAR_FC_EBL_CMD14	Enable function chain CMD14	0x1
CMD13	HEAR_FC_EBL_CMD13	Enable function chain CMD13	0x1
CMD12	HEAR_FC_EBL_CMD12	Enable function chain CMD12	0x1
CMD11	HEAR_FC_EBL_CMD11	Enable function chain CMD11	0x1
CMD10	HEAR_FC_EBL_CMD10	Enable function chain CMD10	0x1
CMD9	HEAR_FC_EBL_CMD9	Enable function chain CMD9	0x1
CMD8	HEAR_FC_EBL_CMD8	Enable function chain CMD8	0x1
CMD7	HEAR_FC_EBL_CMD7	Enable function chain CMD7	0x1
CMD6	HEAR_FC_EBL_CMD6	Enable function chain CMD6	0x1
CMD5	HEAR_FC_EBL_CMD5	Enable function chain CMD5	0x1
CMD4	HEAR_FC_EBL_CMD4	Enable function chain CMD4	0x1
CMD3	HEAR_FC_EBL_CMD3	Enable function chain CMD3	0x1
CMD2	HEAR_FC_EBL_CMD2	Enable function chain CMD2	0x1
CMD1	HEAR_FC_EBL_CMD1	Enable function chain CMD1	0x1
CMD0	HEAR_FC_EBL_CMD0	Enable function chain CMD0	0x1

5.2.1.2 D_HEAR_FC_PRIORITY, HEAR_FC_PRIORITY

Bit Field	Read/Write	Field Name	Description
23	RW	FIFO_7	Prioritize the HEAR FIFO function chain 7
22	RW	FIFO_6	Prioritize the HEAR FIFO function chain 6
21	RW	FIFO_5	Prioritize the HEAR FIFO function chain 5
20	RW	FIFO_4	Prioritize the HEAR FIFO function chain 4
19	RW	FIFO_3	Prioritize the HEAR FIFO function chain 3

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
18	RW	FIFO_2	Prioritize the HEAR FIFO function chain 2
17	RW	FIFO_1	Prioritize the HEAR FIFO function chain 1
16	RW	FIFO_0	Prioritize the HEAR FIFO function chain 0
15	RW	CFX_15	Prioritize the CFX function chain 15
14	RW	CFX_14	Prioritize the CFX function chain 14
13	RW	CMD13	Prioritize the CFX function chain 13
12	RW	CMD12	Prioritize the CFX function chain 12
11	RW	CMD11	Prioritize the CFX function chain 11
10	RW	CMD10	Prioritize the CFX function chain 10
9	RW	CMD9	Prioritize the CFX function chain 9
8	RW	CMD8	Prioritize the CFX function chain 8
7	RW	CMD7	Prioritize the CFX function chain 7
6	RW	CMD6	Prioritize the CFX function chain 6
5	RW	CMD5	Prioritize the CFX function chain 5
4	RW	CMD4	Prioritize the CFX function chain 4
3	RW	CMD3	Prioritize the CFX function chain 3
2	RW	CMD2	Prioritize the CFX function chain 2
1	RW	CMD1	Prioritize the CFX function chain 1
0	RW	CMD0	Prioritize the CFX function chain 0

Field Name	Value Symbol	Value Description	Hex Value
FIFO_7	HEAR_FC_NORMALPRIO_FIFO_7	Function chain FIFO_7 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_FIFO_7	Function chain FIFO_7 has high priority	0x1
FIFO_6	HEAR_FC_NORMALPRIO_FIFO_6	Function chain FIFO_6 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_FIFO_6	Function chain FIFO_6 has high priority	0x1
FIFO_5	HEAR_FC_NORMALPRIO_FIFO_5	Function chain FIFO_5 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_FIFO_5	Function chain FIFO_5 has high priority	0x1
FIFO_4	HEAR_FC_NORMALPRIO_FIFO_4	Function chain FIFO_4 has normal priority	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	HEAR_FC_HIGHPRIO_FIFO_4	Function chain FIFO_4 has high priority	0x1
FIFO_3	HEAR_FC_NORMALPRIO_FIFO_3	Function chain FIFO_3 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_FIFO_3	Function chain FIFO_3 has high priority	0x1
FIFO_2	HEAR_FC_NORMALPRIO_FIFO_2	Function chain FIFO_2 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_FIFO_2	Function chain FIFO_2 has high priority	0x1
FIFO_1	HEAR_FC_NORMALPRIO_FIFO_1	Function chain FIFO_1 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_FIFO_1	Function chain FIFO_1 has high priority	0x1
FIFO_0	HEAR_FC_NORMALPRIO_FIFO_0	Function chain FIFO_0 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_FIFO_0	Function chain FIFO_0 has high priority	0x1
CFX_15	HEAR_FC_NORMALPRIO_CFX_15	Function chain CFX_15 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CFX_15	Function chain CFX_15 has high priority	0x1
CFX_14	HEAR_FC_NORMALPRIO_CFX_14	Function chain CFX_14 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CFX_14	Function chain CFX_14 has high priority	0x1
CMD13	HEAR_FC_NORMALPRIO_CMD13	Function chain CMD13 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD13	Function chain CMD13 has high priority	0x1
CMD12	HEAR_FC_NORMALPRIO_CMD12	Function chain CMD12 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD12	Function chain CMD12 has high priority	0x1
CMD11	HEAR_FC_NORMALPRIO_CMD11	Function chain CMD11 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD11	Function chain CMD11 has high priority	0x1
CMD10	HEAR_FC_NORMALPRIO_CMD10	Function chain CMD10 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD10	Function chain CMD10 has high priority	0x1
CMD9	HEAR_FC_NORMALPRIO_CMD9	Function chain CMD9 has normal priority	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	HEAR_FC_HIGHPRIO_CMD9	Function chain CMD9 has high priority	0x1
CMD8	HEAR_FC_NORMALPRIO_CMD8	Function chain CMD8 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD8	Function chain CMD8 has high priority	0x1
CMD7	HEAR_FC_NORMALPRIO_CMD7	Function chain CMD7 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD7	Function chain CMD7 has high priority	0x1
CMD6	HEAR_FC_NORMALPRIO_CMD6	Function chain CMD6 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD6	Function chain CMD6 has high priority	0x1
CMD5	HEAR_FC_NORMALPRIO_CMD5	Function chain CMD5 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD5	Function chain CMD5 has high priority	0x1
CMD4	HEAR_FC_NORMALPRIO_CMD4	Function chain CMD4 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD4	Function chain CMD4 has high priority	0x1
CMD3	HEAR_FC_NORMALPRIO_CMD3	Function chain CMD3 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD3	Function chain CMD3 has high priority	0x1
CMD2	HEAR_FC_NORMALPRIO_CMD2	Function chain CMD2 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD2	Function chain CMD2 has high priority	0x1
CMD1	HEAR_FC_NORMALPRIO_CMD1	Function chain CMD1 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD1	Function chain CMD1 has high priority	0x1
CMD0	HEAR_FC_NORMALPRIO_CMD0	Function chain CMD0 has normal priority	0x0*
	HEAR_FC_HIGHPRIO_CMD0	Function chain CMD0 has high priority	0x1

5.2.1.3 D_HEAR_FC_STATUS, HEAR_FC_STATUS

Bit Field	Read/Write	Field Name	Description
23	R	FIFO_HEAR_7_PENDING	Read the HEAR FIFO pending function chain 7 status
23	W	FIFO_HEAR_7_CLEAR	Clear the HEAR FIFO pending function chain 7 status
22	R	FIFO_HEAR_6_PENDING	Read the HEAR FIFO pending function chain 6 status
22	W	FIFO_HEAR_6_CLEAR	Clear the HEAR FIFO pending function chain 6 status

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
21	R	FIFO_HEAR_5_PENDING	Read the HEAR FIFO pending function chain 5 status
21	W	FIFO_HEAR_5_CLEAR	Clear the HEAR FIFO pending function chain 5 status
20	R	FIFO_HEAR_4_PENDING	Read the HEAR FIFO pending function chain 4 status
20	W	FIFO_HEAR_4_CLEAR	Clear the HEAR FIFO pending function chain 4 status
19	R	FIFO_HEAR_3_PENDING	Read the HEAR FIFO pending function chain 3 status
19	W	FIFO_HEAR_3_CLEAR	Clear the HEAR FIFO pending function chain 3 status
18	R	FIFO_HEAR_2_PENDING	Read the HEAR FIFO pending function chain 2 status
18	W	FIFO_HEAR_2_CLEAR	Clear the HEAR FIFO pending function chain 2 status
17	R	FIFO_HEAR_1_PENDING	Read the HEAR FIFO pending function chain 1 status
17	W	FIFO_HEAR_1_CLEAR	Clear the HEAR FIFO pending function chain 1 status
16	R	FIFO_HEAR_0_PENDING	Read the HEAR FIFO pending function chain 0 status
16	W	FIFO_HEAR_0_CLEAR	Clear the HEAR FIFO pending function chain 0 status
15	R	CFX_15_PENDING	Read the CFX pending function chain 15 status
15	W	CFX_15_CLEAR	Clear the CFX pending function chain 15 status
14	R	CFX_14_PENDING	Read the CFX pending function chain 14 status
14	W	CFX_14_CLEAR	Clear the CFX pending function chain 14 status
13	R	CFX_13_PENDING	Read the CFX pending function chain 13 status
13	W	CFX_13_CLEAR	Clear the CFX pending function chain 13 status
12	R	CFX_12_PENDING	Read the CFX pending function chain 12 status
12	W	CFX_12_CLEAR	Clear the CFX pending function chain 12 status
11	R	CFX_11_PENDING	Read the CFX pending function chain 11 status
11	W	CFX_11_CLEAR	Clear the CFX pending function chain 11 status
10	R	CFX_10_PENDING	Read the CFX pending function chain 10 status
10	W	CFX_10_CLEAR	Clear the CFX pending function chain 10 status
9	R	CFX_9_PENDING	Read the CFX pending function chain 9 status
9	W	CFX_9_CLEAR	Clear the CFX pending function chain 9 status
8	R	CFX_8_PENDING	Read the CFX pending function chain 8 status
8	W	CFX_8_CLEAR	Clear the CFX pending function chain 8 status
7	R	CFX_7_PENDING	Read the CFX pending function chain 7 status
7	W	CFX_7_CLEAR	Clear the CFX pending function chain 7 status
6	R	CFX_6_PENDING	Read the CFX pending function chain 6 status

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
6	W	CFX_6_CLEAR	Clear the CFX pending function chain 6 status
5	R	CFX_5_PENDING	Read the CFX pending function chain 5 status
5	W	CFX_5_CLEAR	Clear the CFX pending function chain 5 status
4	R	CFX_4_PENDING	Read the CFX pending function chain 4 status
4	W	CFX_4_CLEAR	Clear the CFX pending function chain 4 status
3	R	CFX_3_PENDING	Read the CFX pending function chain 3 status
3	W	CFX_3_CLEAR	Clear the CFX pending function chain 3 status
2	R	CFX_2_PENDING	Read the CFX pending function chain 2 status
2	W	CFX_2_CLEAR	Clear the CFX pending function chain 2 status
1	R	CFX_1_PENDING	Read the CFX pending function chain 1 status
1	W	CFX_1_CLEAR	Clear the CFX pending function chain 1 status
0	R	CFX_0_PENDING	Read the CFX pending function chain 0 status
0	W	CFX_0_CLEAR	Clear the CFX pending function chain 0 status

5.2.1.4 D_HEAR_CMDS_0_TO_7, HEAR_CMDS_0_TO_7

Bit Field	Read/Write	Field Name	Description
7	W	HEAR_CMD_7	Write a 1 to issue HEAR start command 7
6	W	HEAR_CMD_6	Write a 1 to issue HEAR start command 6
5	W	HEAR_CMD_5	Write a 1 to issue HEAR start command 5
4	W	HEAR_CMD_4	Write a 1 to issue HEAR start command 4
3	W	HEAR_CMD_3	Write a 1 to issue HEAR start command 3
2	W	HEAR_CMD_2	Write a 1 to issue HEAR start command 2
1	W	HEAR_CMD_1	Write a 1 to issue HEAR start command 1
0	W	HEAR_CMD_0	Write a 1 to issue HEAR start command 0

Field Name	Value Symbol	Value Description	Hex Value
HEAR_CMD_7	HEAR_CMD_7	Issue HEAR command 7	0x1
HEAR_CMD_6	HEAR_CMD_6	Issue HEAR command 6	0x1
HEAR_CMD_5	HEAR_CMD_5	Issue HEAR command 5	0x1
HEAR_CMD_4	HEAR_CMD_4	Issue HEAR command 4	0x1
HEAR_CMD_3	HEAR_CMD_3	Issue HEAR command 3	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
HEAR_CMD_2	HEAR_CMD_2	Issue HEAR command 2	0x1
HEAR_CMD_1	HEAR_CMD_1	Issue HEAR command 1	0x1
HEAR_CMD_0	HEAR_CMD_0	Issue HEAR command 0	0x1

5.2.1.5 D_HEAR_CMDS_8_TO_15, HEAR_CMDS_8_TO_15

Bit Field	Read/Write	Field Name	Description
7	W	HEAR_CMD_15	Write a 1 to issue HEAR start command 15
6	W	HEAR_CMD_14	Write a 1 to issue HEAR start command 14
5	W	HEAR_CMD_13	Write a 1 to issue HEAR start command 13
4	W	HEAR_CMD_12	Write a 1 to issue HEAR start command 12
3	W	HEAR_CMD_11	Write a 1 to issue HEAR start command 11
2	W	HEAR_CMD_10	Write a 1 to issue HEAR start command 10
1	W	HEAR_CMD_9	Write a 1 to issue HEAR start command 9
0	W	HEAR_CMD_8	Write a 1 to issue HEAR start command 8

Field Name	Value Symbol	Value Description	Hex Value
HEAR_CMD_15	HEAR_CMD_15	Issue HEAR command 15	0x1
HEAR_CMD_14	HEAR_CMD_14	Issue HEAR command 14	0x1
HEAR_CMD_13	HEAR_CMD_13	Issue HEAR command 13	0x1
HEAR_CMD_12	HEAR_CMD_12	Issue HEAR command 12	0x1
HEAR_CMD_11	HEAR_CMD_11	Issue HEAR command 11	0x1
HEAR_CMD_10	HEAR_CMD_10	Issue HEAR command 10	0x1
HEAR_CMD_9	HEAR_CMD_9	Issue HEAR command 9	0x1
HEAR_CMD_8	HEAR_CMD_8	Issue HEAR command 8	0x1

5.3 BLOCK FLOATING-POINT

The HEAR Configurable Accelerator supports the use of a block floating-point data type for data calculations. Using this feature allows:

1. The detection of the maximum magnitude of a block of data
2. Shifting of data by a previous magnitude factor
3. Storing of the total shift amount (referred to as the block exponent)

Ezairo 8300 Hardware Reference

The block floating-point configuration is normally controlled by the microcode configuration running on the HEAR. As use of block floating-point is tightly coupled with a microcode configuration, only specific configuration details of the available bit fields and defined symbols for these bit fields are provided here. For more information about using this feature of the HEAR, see the *HEAR Configurable Accelerator Reference Manual for Ezairo 8300*.

This feature of the HEAR can also be configured by the controlling core through the D_HEAR_BFP_CFG register as part of the overall configuration of the HEAR. The block exponent bit field (HEAR_BFP_CFG_BLOCK_EXP) from the D_HEAR_BFP_CFG register has been mirrored into the low bits of the D_HEAR_BFP_BLOCK_EXP register, to allow efficient access to this important field.

5.3.1 Control and Configuration Registers

Register Name	Register Description	Address
D_HEAR_BFP_CFG	Configure block floating-point functionality of the HEAR	X:0x1260
HEAR_BFP_CFG		0x20804980
D_HEAR_BFP_BLOCK_EXP	Mirror of the block floating-point total shift/block exponent field from D_HEAR_BFP	X:0x1261
HEAR_BFP_BLOCK_EXP		0x20804984

5.3.1.1 D_HEAR_BFP_CFG, HEAR_BFP_CFG

Bit Field	Read/Write	Field Name	Description
21:16	RW	BLOCK_EXP	Stored block exponent for the data
15	RW	FMD_MODE	Enable/disable FMD mode
12:8	RW	BLOCK_MAX_MAG	Maximum magnitude for a block of data
5:4	RW	SCALE_FACTOR	Select a scaling factor for BFP
1:0	RW	FORMAT	Select a format based on the number of bits

Field Name	Value Symbol	Value Description	Hex Value
FMD_MODE	HEAR_BFP_FMD_DISABLE	Disable FMD mode	0x0*
	HEAR_BFP_FMD_ENABLE	Enable FMD mode	0x1
FORMAT	HEAR_BFP_FORMAT_24BITS	Allow no shifts	0x0*
	HEAR_BFP_FORMAT_23BITS	Allow 0 or 1 shift	0x1
	HEAR_BFP_FORMAT_22BITS	Allow 0, 1 or 2 shifts	0x2
	HEAR_BFP_FORMAT_21BITS	Allow 0, 1, 3 or 3 shifts	0x3

5.3.1.2 D_HEAR_BFP_BLOCK_EXP, HEAR_BFP_BLOCK_EXP

Bit Field	Read/Write	Field Name	Description
5:0	RW	MIRROR	Mirror of the block exponent for the data

CHAPTER 6

Filter Engine

The Filter Engine is a specialized processing core that is optimized for basic audio filtering of both decimated and undecimated data in the Ezairo 8300 system. The Filter Engine can be used to implement a set of FIR and IIR data filters that contain up to 320 coefficients. FIR filters are implemented using a direct-form structure, with 5 taps per section. IIR filters are implemented as cascaded second-order sections (biquads), each implemented as a direct-form I filter.

Some Filter Engine use cases include:

- Pre-filtering of input data before it is handled by the input/output controller (IOC)
- Post-filtering of output data before it is output to the sigma-delta modulator
- Alternate decimation and interpolation of input and output data (replacing the input stage decimation filtering described in [Section 14.4 “Decimation Filtering” on page 444](#) and output stage interpolation filtering described in [Section 15.2 “Interpolation Filtering” on page 456](#))
- Low group-delay processing of undecimated data
- Combining decimated and undecimated data

6.1 PROCESSOR OVERVIEW

The Filter Engine is programmable, but does not include direct debugging access. The CFX can monitor and control the Filter Engine state through control and configuration registers on the IOMEM bus.

The architecture of the Filter Engine includes a 256-word (36 bit per word) program memory. The Filter Engine cannot write to the program memory, but frequently reads from it.

The Filter Engine operates on the concept of a program cycle, where a sample-ready signal triggers the program execution. When this occurs, the Filter Engine is brought out of sleep state, executes the instructions in the program memory starting at address 0, and continues until all active instructions are executed, at which time the Filter Engine returns to sleep, waiting for a new event. The CFX DSP or Arm Cortex-M3 processor can monitor the current address of the program memory decoder using the `FENG_PGM_ADDR` register, but cannot modify its contents.

Information about the Filter Engine's features and instruction set can be found in the *Filter Engine Reference Manual for Ezairo 8300*.

6.2 FUNCTIONAL DESCRIPTION

The functional block diagram of the Filter Engine is provided in the ["Filter Engine Functional Block Diagram" figure \(Figure 4\)](#). It holds the following components:

1. A core block, which is designed to efficiently implement filtering operations such as Biquads and FIRs
2. An ALU to perform a total of 15 different arithmetic instructions
3. A multi-port register file, which stores intermediate values between the core stages as well as the ALU data. It also holds I/O mapped registers used in Filter Engine processing.
4. A program memory and instruction set to control the operations in the Filter Engine
5. A phase memory to support multirate processing
6. A decoder for fetching and decoding the instructions from the program memory. The decoder also uses multirate counters to decode phase memory content, in order to control the program memory address pointer increments.
7. A communication interface with the DSP or Arm Cortex-M3 processor

Ezairo 8300 Hardware Reference

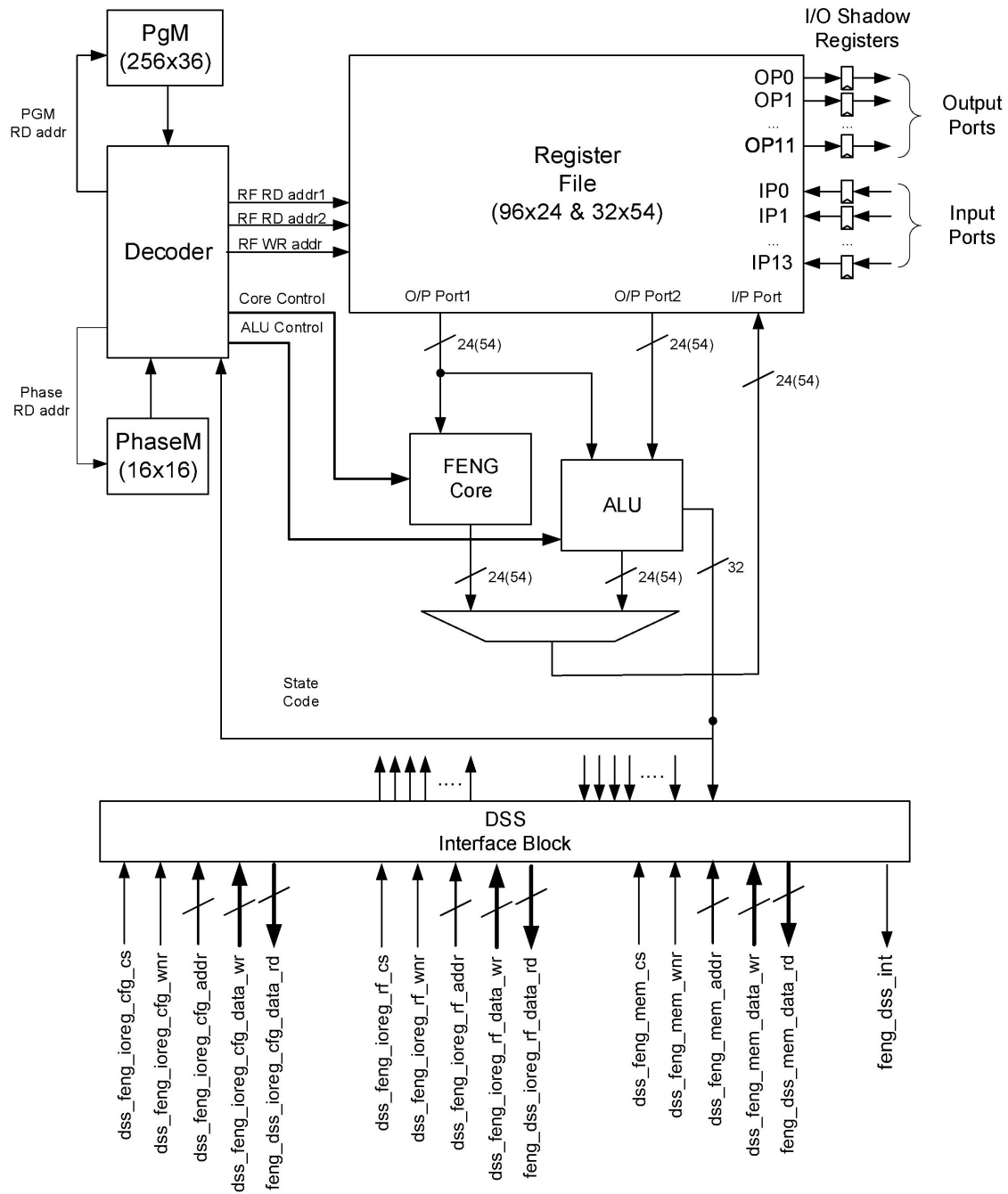


Figure 4. Filter Engine Functional Block Diagram

Ezairo 8300 Hardware Reference

6.3 FILTER ENGINE IN THE AUDIO PATH

The Filter Engine is linked to the audio input and output stages, offering the options of pre-processing the input signal and post-processing the output signal. The Filter Engine also plays an indispensable part in forming a low group-delay processing path, where the following conditions exist:

- The decimation filter at the input stage is bypassed.
- The interpolation filter at the output stage is bypassed.
- The audio data is processed at the undecimated sampling rate.

The "Audio Filtering and Multiplexing, Full Diagram" figure (Figure 26) shows how the Filter Engine input and output signals are connected to the input and output stages, and how the audio signals that bypass the Filter Engine are multiplexed with Filter Engine pre- and post-processed signals.

- Each audio input channel includes Filter Engine filters, offering the option of pre-processing the input signal from the decimation filters before the IOCs transfer this data to an input FIFO for use by the CFX, HEAR, LPDSP32, or Arm Cortex-M3 cores. The filters can be bypassed independently for each channel.
- Each audio output channel includes Filter Engine filters, offering the option of post-processing the output signal before it is sent to the interpolation filters. The filters can be bypassed independently for each channel.
- The Filter Engine can be configured to provide a low-delay processing path by using the undecimated input from a selected input channel's ADC, and providing this data to the selected output channel's sigma-delta modulator. If desired, this data can be combined with data from the interpolation filter before being routed to an output channel.
- The undecimated ADC data can also be routed to the input FIFO after the Filter Engine processing. The data at the undecimated rate can be brought back to the Filter Engine from the IOCs. This enables further processing at the undecimated rate in CFX DSP, Arm Cortex-M3 processor, HEAR, or LPDSP32 at the expense of further delay.

NOTE: The fractional delay observed by the input data can impact the synchronization (i.e., the sampling time) between the different ADC-driven inputs to the Filter Engine. By default, data items from the ADCs are sampled in a synchronized manner; however, changes to the fractional delay affect this synchronization. For more information, see [Section 14.3 "Analog-to-Digital Converters \(ADCs\)" on page 440](#).

For more information about multiplexing the Filter Engine with the audio input and output paths, see [Section 13.1.1 "Audio Multiplexing with Filter Engine Outputs" on page 404](#).

IMPORTANT: Because Filter Engine inputs 4 and 5 do not have a natural sampling interval, sampling of these inputs is synchronized to their associated interpolation filters, ensuring that they are sampled at the system sampling frequency. For this process, the interpolation filters for output channels 0 and 1 need to be enabled. This allows data to be captured at the interpolation input sampling rate.

The inputs 6 and 7 capture can be aligned with various events, such as interpolation filter input and output as well as all ADC undecimated sample captures. The mode of operation is defined by the `D_AUDIO_IOC_FENG_CFG` register.

6.4 FILTER ENGINE CONTROL AND CONFIGURATION

6.4.1 Filter Engine Control

For information on single-step and debugging, see [Chapter 17.1 "Debug" on page 523](#).

Ezairo 8300 Hardware Reference

6.4.1.1 Master Control

The CFX can start or stop the Filter Engine by setting or clearing the `FENG_CTRL_ENABLE` bit field of the `D_FENG_CTRL` register. When this field is cleared, the clock to the Filter Engine is gated immediately.

NOTE: Unless the CFX updates them, all Filter Engine registers hold their states while the Filter Engine is disabled.

6.4.1.2 Pause

When the Filter Engine is enabled, the CFX DSP can pause the Filter Engine the next time it begins execution, by setting the `FENG_CTRL_PAUSE` bit field of the `D_FENG_CTRL` register. When this field is set, the Filter Engine continues running until the next time it starts execution of a new program phase (i.e., the next time the Filter Engine starts execution of the root sector). At this time, the `FENG_CTRL_ENABLE` and `FENG_CTRL_PAUSE` bit fields are cleared.

When the Filter Engine is paused in its operation, all configuration registers and intermediate data register files (RF) can be accessed freely. But if the content of a memory is read, and the Filter Engine has been reading from the same memory with a different address during the same cycle, the Filter Engine operation is corrupted. The Filter Engine uses synchronous memories, where the result of the read is registered at the output of the memory. Therefore the Filter Engine requires this output register to retain its value, in order to complete operations normally.

To achieve this, before stepping the Filter Engine further, the last address it has accessed must be read again. During the program memory debugging, the programmer can check the `FENG_PGM_ADDR` register to retrieve the address to be read. For the coefficient memory and state variable memories, the programmer needs to check the corresponding instruction in the program address, and retrieve the coefficient and state variable addresses used by the Filter Engine in that execution cycle.

6.4.1.3 Clean and Resume

When restarting the Filter Engine in its normal execution after a PAUSE or DISABLE, and waiting for a new clean sample, the `FENG_CTRL_RESUME_CLEAN` bit under the `D_FENG_CTRL` register needs to be set. This bit automatically issues the `FENG_DECODER_RESET`, clears all the pending sample ready flags, and clears the multirate counters under the `D_FENG_MRCNT` register. During the execution cycle of this bit, the program memory decoder start is blocked. This ensures that the Filter Engine has a clean start without being affected by any of the ongoing activities while it is disabled. The actual audio registers, and other configuration registers, are not reset.

Note that this feature is supported by the firmware macro `SYS_FENG_DELAYEDPAUSE`. This macro sets the `FENG_PAUSE_ENABLE` and `FENG_CTRL_ENABLE` bits, causing the Filter Engine to pause the next time it starts execution rather than pausing immediately. The name `DELAYEDPAUSE` differentiates it from the firmware macro `SYS_FENG_PAUSE`, which pauses the Filter Engine immediately (see also [Section 6.4.1.1 “Master Control” on page 220](#)).

6.4.1.4 Synchronized Start

During multirate signal processing, the Filter Engine processing needs to be triggered by one of the faster input samples. If the high sample rate processing is performed along with the decimated rate processing, it is desirable to synchronize the base processing to a certain phase of the multirate counters.

To achieve this, the programmer can use the `SYNC_START` bit of the `D_FENG_CTRL` register. This master control bit ensures that the Filter Engine clocks are not enabled until the slow sample rate `sr_in` configured in the `MRCNT_SYNC_SOURCE` field of the `D_FENG_MRCNT` register arrives. Once this occurs, the consecutive high speed trigger sample starts the processing. The samples might be coming at the same time, in which case the processing starts

Ezairo 8300 Hardware Reference

immediately. This synchronizes the start of the processing to a slow rate sample. For the proper operation of the SYNC_START bit, the MRCNT_SYNC_EN bit in the D_FENG_MRCNT register does not need to be enabled. When the synchronized start occurs, the SYNC_START field is cleared.

6.4.2 Filter Engine Reset

The Filter Engine can be reset by using the two reset bits from the D_FENG_CMDS control register. When the Filter Engine is idle, setting the following bits produce the following results:

- FENG_CMDS_AUDIO_RESET bit set to FENG_AUDIO_RESET — this resets the audio path registers (excluding the memories).
- FENG_CMDS_DECODER_RESET bit set to FENG_DECODER_RESET — this resets the program decoder.

It is recommended to reset both the audio path and the program decoder during application initialization.

6.4.3 Filter Engine Manual Execution

Besides the input sample ready signal that triggers Filter Engine processing, the CFX can also manually start Filter Engine program execution. This is useful for debugging, or using the Filter Engine for filtering or performing arithmetic operations on data that is not directly linked to the input or output stage.

To enable manual starting of Filter Engine execution, set the FENG_CFG_SRI_SEL bit field of the D_FENG_CFG register to FENG_SRI_DISABLE. Assuming that the Filter Engine is enabled and idle, the CFX can start Filter Engine processing immediately by writing FENG_EXEC to the FENG_CMDS_EXEC bit field of the D_FENG_CMDS register. This updates the audio shadow registers, and executes the Filter Engine program code once. If the Filter Engine is already active, issuing the FENG_EXEC command results in the FENG_INT_STATUS_OVERLOAD bit field being set in the D_FENG_INT_STATUS register.

6.4.4 Filter Engine Configuration

The D_FENG_CFG and D_FENG_CFG_IO registers have fields that define some of the most basic aspects of Filter Engine behavior.

6.4.4.1 Core Configuration Register (D_FENG_CFG)

Coefficient offset

This specifies an offset to add to the coefficient set index if the corresponding bit is set in the D_FENG_COEFF_OFFSET_EN_ACTIVE0 or D_FENG_COEFF_OFFSET_EN_ACTIVE1 register. This is useful in dynamic coefficient switching. For more information, see [Section 6.4.5 “Filter Engine Filter Coefficients”](#) on page 222.

Sample-ready input that triggers Filter Engine processing

For sample-triggered Filter Engine processing, select one of the 14 input signals as the trigger for Filter Engine processing. If the application uses more than one sampling rate, set the FENG_CFG_SRI_SEL bit field to **x**, where SR_IN**x** has the highest sampling rate.

For manual execution of the Filter Engine program code, set the FENG_CFG_SRI_SEL bit field to FENG_SRI_DISABLE.

Ezairo 8300 Hardware Reference

NOTE: Input data received from the decimation filters for ADC0 and ADC2 arrives 1/8th of a sample period ahead of data from the decimation filters for ADC1 and ADC3 (inputs to the Filter Engine from the input stage decimation filters are inputs 0 to 3 in the "Input Stage Block Diagram" figure (Figure 28)).

6.4.4.2 Input and Output Configuration Registers (D_FENG_CFG_IO)

Input sample capture

Disable or enable each of the 14 Filter Engine input sources. If input x is enabled, then whenever a new sample arrives at input x , it is captured in the register D_FENG_RF_IN_ x and becomes available to both the Filter Engine and the CFX for processing.

Output shadow register capture modes

Shadow registers at the output ports can ensure that all the output samples, or a subset of them, are synchronized. When data is written to an output via an output data register (by the Filter Engine or the CFX), the data is transferred to the output shadow register, whose update timing is controlled by the FENG_CFG_OSR*_EBL bit field in the D_FENG_CFG_IO register. The twelve bits in this bit field control the shadow registers at the twelve output ports, in the following ways:

- If the corresponding bit is set, the output shadow register is updated at the start of program execution. Therefore, if some or all outputs have their FENG_CFG_OSR*_EBL bits set to 1, these output samples are synchronized.
- If the corresponding bit is cleared, the output shadow register is updated immediately. This option minimizes system delay and is especially useful for low group delay processing.

IMPORTANT: Be careful when using unsynchronized output sample updates. If the output sample is updated through the register file during Filter Engine program execution, the following restrictions apply:

- Do not use this output data register for intermediate data.
- Do not update this register file location through conditional processing by multiple instructions.
- In multi-rate processing, if the same register file location is updated in multiple phases, ensure that the delay, from the start of the program phase to the time of this register file location update, is the same in each program phase.

6.4.5 Filter Engine Filter Coefficients

If certain coefficient sets are used in the Filter Engine source code, the CFX must initialize them. Up to 64 coefficient sets, each supporting one biquad-IIR or 5-tap FIR filter step, can be stored at address locations 0x1A00 through 0x1B3F in IOMEM. Each set of coefficients (C0 to C63) occupies five words. For more information about filter coefficients, see the *Filter Engine Reference Manual for Ezairo 8300*.

To implement dynamic coefficient updates:

1. Specify a coefficient offset in the Filter Engine configuration register D_FENG_CFG (see Section 6.4.4 "Filter Engine Configuration" on page 221 for more information).
2. Set the coefficient offset enable bits that correspond to the coefficient set indexes, in the applicable Filter Engine configuration registers selected from D_FENG_COEFF_OFFSET_EN_SHADOW0, D_FENG_COEFF_OFFSET_EN_SHADOW1, D_FENG_COEFF_OFFSET_EN_ACTIVE0, and D_FENG_COEFF_OFFSET_EN_ACTIVE1. An update to the shadow register is transferred to the corresponding active register after the SHADOW_TRANSFER command is

Ezairo 8300 Hardware Reference

set in the `D_FENG_CMDS` register. The transfer occurs when the Filter Engine becomes idle, or at the start of the next program execution cycle if the Filter Engine is always busy. If the coefficients are static, we recommend that you do not toggle their corresponding `FENG_COEFF_OFFSET_EN_ACTIVE*` bits.

The CFX can also write to the active register to make the new value take effect immediately. However, use this feature with caution to avoid unintended memory access conflicts (see [Section 6.5.3 “Memory and Register Access Conflicts” on page 227](#)).

3. Update the inactive coefficients from CFX.

See the dynamic coefficient update section of the *Filter Engine Reference Manual for Ezairo 8300* for more information.

6.4.6 Multi-Rate Counter

The Filter Engine has two programmable 4-bit multi-rate counters, `MRCNT0` and `MRCNT1`. These can support up to three sampling rates in the system: `fs` (`FSin` (sampling rate of `sr_in` set by `SRI_SEL`)), `fs/N1` (`FSin/(MRCNT0_DR+1)`), and `fs/N2` (`FSin/(MRCNT1_DR+1)`).

If the Filter Engine only processes data at the same rate at which samples become available, do not use any multi-rate counters. If an application needs to process data at a decimated rate, `fs/N1`, in addition to the base sampling rate, set the `FENG_MRCNT_MRCNT0_DR` bit field of the `D_FENG_MRCNT` register to `(N1-1)`. Similarly, to configure a third sampling rate, `fs/N2`, use the second multi-rate counter by setting the `FENG_MRCNT_MRCNT1_DR` bit field of the `D_FENG_MRCNT` register to `(N2-1)`. Current counter values can be read from the bit fields `FENG_MRCNT_MRCNT0` and `FENG_MRCNT_MRCNT1` of the `D_FENG_MRCNT` register.

For information about multi-rate processing on the Filter Engine, refer to the *Filter Engine Reference Manual for Ezairo 8300*.

A phase difference between the counters can be introduced by initializing `MRCNT0` or `MRCNT1` with a non-zero value.

During multirate signal processing, the Filter Engine processing needs to be triggered by one of the faster input samples. If the high sample rate processing is performed along with the decimated rate processing, it is desirable to synchronize the base processing to a certain phase of the multirate counters. This synchronization can be ensured at each slow rate processing cycle by enabling the `MRCNT_SYNC_EN` field of the `D_FENG_MRCNT` register. In this mode, the `sr_in` configured in the `MRCNT_SYNC_SOURCE` field of the `D_FENG_MRCNT` register resets the `MRCNT0` and `MRCNT1` fields. That means the phase0 processing is always performed at this sample ready arrival. In this operation the `MRCNT0_DR` and `MRCNT1_DR` have to be equal, or an integer factor of each other, and are expected to enter to phase0 together at the same input sample configured in the `MRCNT_SYNC_SOURCE` field. The `sr_in` used for the synchronization needs to be at the slowest sample rate; otherwise, the synchronization enable can result in some of the multirate counters' phases being skipped.

6.4.7 Phase Memory

The Filter Engine uses the following bank of registers in the following ways:

- `D_FENG_PHM_PH0_PH1` to `D_FENG_PHM_PH14_PH15`, to store the starting addresses of program memory for each enabled program phase sector
- `D_FENG_PHM_PH15_END` register, for phase 15 program memory sector end
- `D_FENG_PGM_SECTOR_EN` register, to store the program phase sector enable

Ezairo 8300 Hardware Reference

Initialization data for the phase memory and program sector enable registers is generated automatically by the compilation tools. There is no need to configure these registers manually.

For more information about multi-rate processing and the program phase concept on the Filter Engine, refer to the *Filter Engine Reference Manual for Ezairo 8300*.

6.4.8 Dynamic TC and SH Mapping

The ATC (attack time constant) and DTC (decay time constant) fields of the LDET instruction can use a static value as part of the instruction entered as-is into the program memory; or the content of these field can be used as a dynamic address mapping to the `FENG_TC_ACTIVE` registers. Each LDET instruction in the Filter Engine program memory has the choice of defining the operation independently from the other instructions. If the dynamic mapping is enabled, the addresses 0 to 7 map the active time constants to the subfields in register `D_FENG_TC_ACTIVE0`. The remainder of TC addresses 8 to 15 retrieve the active content from `D_FENG_TC_ACTIVE1`. These registers' contents can be updated directly, or shadow registers can be used to align the update with all other configuration content to the same Filter Engine execution.

There are 11 instructions with an SH (shift value) field. Similar to the TC, the SH value of the instruction can be defined statically in the program memory, or the instruction can be entered in a such a way as to use this content as a dynamic address mapping to the `D_FENG_SH_ACTIVE` register. Each instruction in the Filter Engine program memory has the choice of working with static or dynamic mapping. The active register content can be updated directly, or its shadow register can be used to align the update with all other configuration content to the same Filter Engine execution.

6.5 FILTER ENGINE AND PROCESSOR INTERFACE

6.5.1 Filter Engine Interrupt

The Filter Engine can raise an interrupt to the CFX and Arm Cortex-M3 processors when a new sample has arrived, if a particular condition in its processing has occurred, or if a CMP instruction has met certain criteria. The Filter Engine interrupt can be set by any of the sources listed in the "[Filter Engine Interrupt Sources](#)" table (Table 14), below. These sources can be enabled using the `D_FENG_INT_ENABLE` and `D_FENG_INT_ENABLE_IO` configuration registers. After enabling an interrupt under the `D_FENG_INT_ENABLE_IO` register, the SRIN or OUT fields under the `D_FENG_INT_ENABLE` register need to be enabled to propagate these interrupt sources. To check the status of an interrupt source, read the corresponding bit value in the `D_FENG_INT_STATUS` register. The individual interrupt status values for the I/O registers are captured in the `D_FENG_INT_STATUS_IO` register.

When the interrupt is serviced, the corresponding `FENG_INT_STATUS` bit must be cleared in the `D_FENG_INT_ACK` register. In the case of an input sample interrupt, setting the `SRIN_ACK` field in the `D_FENG_INT_ACK` register clears all the `SRINx` interrupt statuses in the `D_FENG_INT_STATUS_IO` register as well. Similarly, setting the `OUT_ACK` field in the `D_FENG_INT_ACK` register clears all the `OUTx` interrupt statuses in the `D_FENG_INT_STATUS_IO` register. Individual `SRIN` and `OUT` interrupts can be cleared in the `D_FENG_INT_ACK_IO` register.

Ezairo 8300 Hardware Reference

Table 14. Filter Engine Interrupt Sources

Interrupt Source	Description
FENG_SRIN0	A new data sample has been received at IN0; a sample ready event is triggered if SRI0 has been selected as the SRI_SRC.
FENG_SRIN1	A new data sample has been received at IN1; a sample ready event is triggered if SRI1 has been selected as the SRI_SRC.
FENG_SRIN2	A new data sample has been received at IN2; a sample ready event is triggered if SRI2 has been selected as the SRI_SRC.
FENG_SRIN3	A new data sample has been received at IN3; a sample ready event is triggered if SRI3 has been selected as the SRI_SRC.
FENG_SRIN4	A new data sample has been received at IN4; a sample ready event is triggered if SRI4 has been selected as the SRI_SRC.
FENG_SRIN5	A new data sample has been received at IN5; a sample ready event is triggered if SRI5 has been selected as the SRI_SRC.
FENG_SRIN6	A new data sample has been received at IN6; a sample ready event is triggered if SRI6 has been selected as the SRI_SRC.
FENG_SRIN7	A new data sample has been received at IN7; a sample ready event is triggered if SRI7 has been selected as the SRI_SRC.
FENG_SRIN8	A new data sample has been received at IN8; a sample ready event is triggered if SRI8 as been selected as the SRI_SRC.
FENG_SRIN9	A new data sample has been received at IN9; a sample ready event is triggered if SRI9 has been selected as the SRI_SRC.
FENG_SRIN10	A new data sample has been received at IN10; a sample ready event is triggered if SRI10 has been selected as the SRI_SRC.
FENG_SRIN11	A new data sample has been received at IN11; a sample ready event is triggered if SRI11 has been selected as the SRI_SRC.
FENG_SRIN12	A new data sample has been received at IN12; a sample ready event is triggered if SRI12 has been selected as the SRI_SRC.
FENG_SRIN13	A new data sample has been received at IN13; a sample ready event is triggered if SRI13 has been selected as the SRI_SRC.
FENG_ISHC	The new input shadow register data values have been transferred into the register file (one system clock cycle after a sample ready event).
FENG_OUT0	New data is captured at D_FENG_RF_OUT_0
FENG_OUT1	New data is captured at D_FENG_RF_OUT_1
FENG_OUT2	New data is captured at D_FENG_RF_OUT_2
FENG_OUT3	New data is captured at D_FENG_RF_OUT_3
FENG_OUT4	New data is captured at D_FENG_RF_OUT_4
FENG_OUT5	New data is captured at D_FENG_RF_OUT_5
FENG_OUT6	New data is captured at D_FENG_RF_OUT_6
FENG_OUT7	New data is captured at D_FENG_RF_OUT_7

Ezairo 8300 Hardware Reference

Filter Engine Interrupt Sources (Continued)

Interrupt Source	Description
FENG_OUT8	New data is captured at D_FENG_RF_OUT_8
FENG_OUT9	New data is captured at D_FENG_RF_OUT_9
FENG_OUT10	New data is captured at D_FENG_RF_OUT_10
FENG_OUT11	New data is captured at D_FENG_RF_OUT_11
FENG_PGE	End of the program memory execution. The Filter Engine is idle until the next sample ready event.
FENG_CMP	A CMP instruction has met its interrupt condition.
FENG_REG_ERR	A write conflict has occurred on the configuration registers (including the Register File) between the CFX DSP or Arm Cortex-M3 processor and the Filter Engine. This interrupt is issued one cycle after the occurrence.
FENG_MEM_ERR	A memory access (either read or write) conflict has occurred when the processor acting as the system master attempts to access Filter Engine memories (state variables, coefficients, and program memory). This interrupt is issued one cycle after the occurrence.
FENG_CORE_SAT	Data saturation has been detected under the core block.
FENG_ALU_SAT	Data saturation has been detected under the ALU block.
FENG_OVERLOAD	A sample ready event or manual execution command has attempted to start Filter Engine execution while the Filter Engine is still active.

6.5.2 Filter Engine Write Protection

The Filter Engine is generally an autonomous block running the content from its program memory continuously, based on the arrival of sample ready signals. To prevent an unintentional access to its memory map, the D_FENG_WR_PROTECT register should be utilized. The D_FENG_WR_PROTECT register holds 5 separate control bits. Each bit is reset to a write protect disable mode of 0. They must be set intentionally to block write access to various regions of the Filter Engine.

The PGM_WR_PROTECT bit blocks write access to the program memory only. The SV_WR_PROTECT bit blocks write access to the State Variable registers. The COEF_WR_PROTECT bit blocks write access to the Coefficient registers. The RF_WR_PROTECT bit blocks write access to the full Register File. The last bit, CFG_WR_PROTECT, blocks write access to the Filter Engine registers in the IOREG region, excluding the registers D_FENG_WR_PROTECT, D_FENG_WR_KEY, D_FENG_INT_ACK_IO, and D_FENG_INT_ACK.

The write protect bits in the D_FENG_WR_PROTECT register can only be updated after writing the key value of 0xFE5870CF into the D_FENG_WR_KEY register. After writing this key value, only single write access to D_FENG_WR_PROTECT is allowed. This is independent of whether the access is to set the bit or to clear it. For each consecutive write to D_FENG_WR_PROTECT, the key must be re-entered.

D_FENG_WR_KEY is a write only address. The status of the key can be monitored with the WR_KEY_STATUS bit in D_FENG_WR_PROTECT. If the key has been entered properly, and the permitted write access to the D_FENG_WR_PROTECT register has not been executed, the WR_KEY_STATUS bit is set. After any write access to the D_FENG_WR_PROTECT register, WR_KEY_STATUS is cleared.

Ezairo 8300 Hardware Reference

6.5.3 Memory and Register Access Conflicts

The CFX can access Filter Engine memory over the P memory bus. If a memory access conflict (either read or write) occurs, the Filter Engine is granted memory access, and the CFX access is corrupted. When an access conflict occurs, any data read by the CFX is invalid, and the data to be written by the CFX is lost.

A memory access conflict occurs if the CFX attempts to access an address in the Filter Engine's memory areas (not including the register file or control and configuration areas). When a memory access conflict occurs, the `FENG_INT_STATUS_MEM_ERR` bit field is set in the `D_FENG_INT_STATUS` register. This occurs if the CFX attempts to:

- Access the Filter Engine coefficient memory while the Filter Engine is executing the first cycle of a `BIQ` or `FIR` instruction
- Access the state variable memory while the Filter Engine is executing a `BIQ` or `FIR` instruction
- Access the Filter Engine program memory while the Filter Engine is active

A register write conflict occurs if the CFX attempts to write to a configuration register (e.g., `D_FENG_MRCNT`) or a data register location (e.g., `D_FENG_RF_SINGLE_0`) when the Filter Engine is also writing to it. These registers are addressed in the Filter Engine's `IOREG` areas. When a register write conflict occurs, the `FENG_INT_STATUS_REG_ERR` bit field is set in the `D_FENG_INT_STATUS` register.

Unlike other Filter Engine registers, the `D_FENG_STATE_ACTIVE` register allows individual bit write-access from the CFX, and this is controlled by the `D_FENG_STATE_W_EN` register. This allows both the Filter Engine and the CFX to write to this register at the same time without a register write conflict, provided they are not writing to the same bits.

For more information about dividing the Filter Engine's memory-mapped components into `IOMEM` and `IOREG` memory areas, refer to [Section 11.2.1.1 “Filter Engine Memory” on page 341](#).

To avoid register write access conflicts on the state register, and align the processing with the new configuration, a set of shadow registers is provided. These registers can be written to safely by the CFX DSP or Arm Cortex-M3 processor at any time, and copied to their mapped active registers only after the `SHADOW_TRANSFER` command is issued. When the `SHADOW_TRANSFER` field in the `D_FENG_CMDS` register is set, the active registers are updated at the start of the next Filter Engine program execution. Only the shadow registers which have been written to since the last `SHADOW_TRANSFER` command are transferred to their active pairs. For the `FENG_COEF_OFFSET_EN` and `FENG_TC` configurations, it is acceptable to update only single addresses. The updated portion of the configuration is transferred into the active register, and the remaining active fields hold on to their state. These register pairs are listed below in the "Filter Engine Shadow/Active Register Pairs" table (Table 15).

Table 15. Filter Engine Shadow/Active Register Pairs

Shadow Register	Active Register
<code>D_FENG_COEFF_OFFSET_EN_SHADOW0</code>	<code>D_FENG_COEFF_OFFSET_EN_ACTIVE0</code>
<code>D_FENG_COEFF_OFFSET_EN_SHADOW1</code>	<code>D_FENG_COEFF_OFFSET_EN_ACTIVE1</code>
<code>D_FENG_STATE_SHADOW</code>	<code>D_FENG_STATE_ACTIVE</code>
<code>FENG_TC_SHADOW0</code>	<code>FENG_TC_ACTIVE0</code>
<code>FENG_TC_SHADOW1</code>	<code>FENG_TC_ACTIVE1</code>
<code>FENG_SH_SHADOW</code>	<code>FENG_SH_ACTIVE</code>

Ezairo 8300 Hardware Reference

6.6 DATA, CONTROL AND CONFIGURATION REGISTERS

6.6.1 Data Register Groups

There are 128 data registers with 24-bit or 54-bit word lengths to store input, output, and intermediate data. Reading and writing to the same register under the same instruction is permitted. In this case, the read occurs first; the read data is processed, and at the end of the instruction execution, the result is written to the same register.

All data registers can be accessed by the Filter Engine, the CFX, and the Arm Cortex-M3 processor. If a register access conflict occurs, it is resolved as described in [Section 6.5.3 “Memory and Register Access Conflicts” on page 227](#).

NOTE: For an explanation of the numeric formats used to describe each register group, see the section about numeric types in the *Ezairo 8300 Firmware Reference*.

6.6.1.1 Extended-Precision Data Registers

Registers `D_FENG_RF_EXTENDED_0` to `D_FENG_RF_EXTENDED_31` are used to store intermediate data with higher resolution, and are not linked to the I/O interface. These 54-bit extended-precision registers have an m4p50 format when used to store linear audio data, and an m1lp43 format when used to store log2-format data.

6.6.1.2 Single-Precision Data Registers

Registers `D_FENG_RF_SINGLE_0` to `D_FENG_RF_SINGLE_69` are used to store intermediate data with lower resolution, and are not linked to the I/O interface. These 24-bit single-precision registers have a frac24 format when used to store linear audio data, and an m8p16 format when used to store log2-format data.

6.6.1.3 Input Data Registers

Registers `D_FENG_RF_IN_0` to `D_FENG_RF_IN_13` hold input data and are linked to the input stage. Input data registers can also be used to store single-precision intermediate data if no more intermediate data registers are available. These 24-bit input data registers have a frac24 format when used to store linear audio data, and an m8p16 format when used to store log2-format data.

6.6.1.4 Output Data Registers

Registers `D_FENG_RF_OUT_0` to `D_FENG_RF_OUT_11` hold output data and are linked to the output stage. Output data registers can also be used to store single-precision intermediate data if no more intermediate data registers are available. These 24-bit output data registers have a frac24 format when used to store linear audio data, and an m8p16 format when used to store log2-format data.

6.6.2 Filter Engine Registers

Register Name	Register Description	Address
FENG_INT_STATUS	FENG Interrupt Status Register	0x40002C00
D_FENG_INT_STATUS		P:0x0B00
FENG_INT_STATUS_IO	FENG Input/Output Interrupt Status Register	0x40002C04
D_FENG_INT_STATUS_IO		P:0x0B01
FENG_INT_ACK	FENG Interrupt Acknowledge Register	0x40002C08
D_FENG_INT_ACK		P:0x0B02

onsemi Confidential

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
FENG_INT_ACK_IO	FENG Input/Output Interrupt Acknowledge Register	0x40002C0C
D_FENG_INT_ACK_IO		P:0x0B03
FENG_INT_ENABLE	FENG Interrupt Enable Register	0x40002C10
D_FENG_INT_ENABLE		P:0x0B04
FENG_INT_ENABLE_IO	FENG Input/Output Interrupt Enable Register	0x40002C14
D_FENG_INT_ENABLE_IO		P:0x0B05
FENG_CFG	FENG Configuration Register	0x40002C18
D_FENG_CFG		P:0x0B06
FENG_CFG_IO	FENG Input/Output Configuration Register	0x40002C1C
D_FENG_CFG_IO		P:0x0B07
FENG_MRCNT	FENG Multirate Counter Register Control	0x40002C20
D_FENG_MRCNT		P:0x0B08
FENG_PGM_SECTOR_EN	FENG Program Memory Sector Enables	0x40002C24
D_FENG_PGM_SECTOR_EN		P:0x0B09
FENG_PHM_PH0_PH1	FENG Phase Memory Phase 0 and Phase 1	0x40002C28
D_FENG_PHM_PH0_PH1		P:0x0B0A
FENG_PHM_PH2_PH3	FENG Phase Memory Phase 2 and Phase 3	0x40002C2C
D_FENG_PHM_PH2_PH3		P:0x0B0B
FENG_PHM_PH4_PH5	FENG Phase Memory Phase 4 and Phase 5	0x40002C30
D_FENG_PHM_PH4_PH5		P:0x0B0C
FENG_PHM_PH6_PH7	FENG Phase Memory Phase 6 and Phase 7	0x40002C34
D_FENG_PHM_PH6_PH7		P:0x0B0D
FENG_PHM_PH8_PH9	FENG Phase Memory Phase 8 and Phase 9	0x40002C38
D_FENG_PHM_PH8_PH9		P:0x0B0E
FENG_PHM_PH10_PH11	FENG Phase Memory Phase 10 and Phase 11	0x40002C3C
D_FENG_PHM_PH10_PH11		P:0x0B0F
FENG_PHM_PH12_PH13	FENG Phase Memory Phase 12 and Phase 13	0x40002C40
D_FENG_PHM_PH12_PH13		P:0x0B10
FENG_PHM_PH14_PH15	FENG Phase Memory Phase 14 and Phase 15	0x40002C44
D_FENG_PHM_PH14_PH15		P:0x0B11

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
FENG_PHM_PH15_END	FENG Phase Memory Phase 15 End	0x40002C48
D_FENG_PHM_PH15_END		P:0x0B12
FENG_COEFF_OFFSET_EN_SHADOW0	FENG Coefficients 0 to 31 Read Address Offset Enable Shadow Register	0x40002C4C
D_FENG_COEFF_OFFSET_EN_SHADOW0		P:0x0B13
FENG_COEFF_OFFSET_EN_SHADOW1	FENG Coefficients 32 to 63 Read Address Offset Enable Shadow Register	0x40002C50
D_FENG_COEFF_OFFSET_EN_SHADOW1		P:0x0B14
FENG_SH_SHADOW	FENG Instructions Dynamic Shift Values Shadow Register	0x40002C54
D_FENG_SH_SHADOW		P:0x0B15
FENG_TC_SHADOW0	FENG Instructions Dynamic Time Constant 0 to 7 Shadow Registers	0x40002C58
D_FENG_TC_SHADOW0		P:0x0B16
FENG_TC_SHADOW1	FENG Instructions Dynamic Time Constant 8 to 15 Shadow Registers	0x40002C5C
D_FENG_TC_SHADOW1		P:0x0B17
FENG_STATE_W_EN	FENG Conditional Processing State Register Write Enable	0x40002C60
D_FENG_STATE_W_EN		P:0x0B18
FENG_STATE_SHADOW	FENG Conditional Processing State Shadow Register	0x40002C64
D_FENG_STATE_SHADOW		P:0x0B19
FENG_STATE_ACTIVE	FENG Conditional Processing State Active Register	0x40002C68
D_FENG_STATE_ACTIVE		P:0x0B1A
FENG_COEFF_OFFSET_EN_ACTIVE0	FENG Coefficients 0 to 31 Read Address Offset Enable Active Register	0x40002C6C
D_FENG_COEFF_OFFSET_EN_ACTIVE0		P:0x0B1B
FENG_COEFF_OFFSET_EN_ACTIVE1	FENG Coefficients 32 to 63 Read Address Offset Enable Active Register	0x40002C70
D_FENG_COEFF_OFFSET_EN_ACTIVE1		P:0x0B1C
FENG_SH_ACTIVE	FENG Instructions Dynamic Shift Values Active Register	0x40002C74
D_FENG_SH_ACTIVE		P:0x0B1D
FENG_TC_ACTIVE0	FENG Instructions Dynamic Time Constant 0 to 7 Active Registers	0x40002C78
D_FENG_TC_ACTIVE0		P:0x0B1E
FENG_TC_ACTIVE1	FENG Instructions Dynamic Time Constant 8 to 15 Active Registers	0x40002C7C
D_FENG_TC_ACTIVE1		P:0x0B1F
FENG_CMDS	FENG Commands Register	0x40002C80
D_FENG_CMDS		P:0x0B20

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
FENG_CTRL	FENG Master Control	0x40002C84
D_FENG_CTRL		P:0x0B21
FENG_WR_KEY	FENG write protection register write access key	0x40002C88
D_FENG_WR_KEY		P:0x0B22
FENG_WR_PROTECT	FENG write protection register	0x40002C8C
D_FENG_WR_PROTECT		P:0x0B23
FENG_PGM_ADDR	FENG Program Memory Decoder Address	0x40002C90
D_FENG_PGM_ADDR		P:0x0B24
FENG_RF_EXTENDED_*	FENG 32 extended precision registers (RF_EXTENDED0 to RF_EXTENDED31)	0x40002D00-0x40002D7C
D_FENG_RF_EXTENDED_*		P:0x0B40-P:0x0B5F
FENG_RF_SINGLE_*	FENG 70 single precision registers (RF_SINGLE0 to RF_SINGLE69)	0x40002D80-0x40002E94
D_FENG_RF_SINGLE_*		P:0x0B60-P:0x0BA5
FENG_RF_IN_*	FENG 14 single precision input data registers (RF_IN0 to RF_IN13)	0x40002E98-0x40002ECC
D_FENG_RF_IN_*		P:0x0BA6-P:0x0BB3
FENG_RF_OUT_*	FENG 12 single precision output data registers (RF_OUT0 to RF_OUT11)	0x40002ED0-0x40002EFC
D_FENG_RF_OUT_*		P:0x0BB4-P:0x0BBF

6.6.2.1 FENG_INT_STATUS, D_FENG_INT_STATUS

Bit Field	Read/Write	Field Name	Description
9	R	SRIN	New FENG sample input and signal ready arrival interrupt status. Check the INT_STATUS_IO register for the individual input status
8	R	OUT	New data capture at FENG output interrupt status. Check the INT_STATUS_IO register for the individual output status
7	R	OVERLOAD	Processor overload interrupt status
6	R	ALU_SAT	ALU saturation interrupt status
5	R	CORE_SAT	Core saturation interrupt status
4	R	MEM_ERR	Illegal memory access interrupt status
3	R	REG_ERR	Register write access collision interrupt status

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
2	R	CMP	Compare instruction interrupt status
1	R	PGE	End of PGM execution interrupt status
0	R	ISHC	Input shadow registers capture in RF interrupt status

Field Name	Value Symbol	Value Description	Hex Value
SRIN	INT_PENDING_FENG_SRIN	A new input sample and sample ready signal have arrived	0x1
	INT_IDLE_FENG_SRIN	Interrupt is not pending	0x0*
OUT	INT_PENDING_FENG_OUT	New output data captured for transfer into output shadow register	0x1
	INT_IDLE_FENG_OUT	Interrupt is not pending	0x0*
OVERLOAD	INT_PENDING_FENG_OVERLOAD	Configured sample ready input is active before execution completion interrupt status	0x1
	INT_IDLE_FENG_OVERLOAD	Interrupt is not pending	0x0*
ALU_SAT	INT_PENDING_FENG_ALU_SAT	A data saturation is detected under the ALU block	0x1
	INT_IDLE_FENG_ALU_SAT	Interrupt is not pending	0x0*
CORE_SAT	INT_PENDING_FENG_CORE_SAT	A data saturation is detected under the FENG core block	0x1
	INT_IDLE_FENG_CORE_SAT	Interrupt is not pending	0x0*
MEM_ERR	INT_PENDING_FENG_MEM_ERR	CFX has executed an illegal memory access	0x1
	INT_IDLE_FENG_MEM_ERR	Interrupt is not pending	0x0*
REG_ERR	INT_PENDING_FENG_REG_ERR	CFX has tried to write into registers at the same time as FENG	0x1
	INT_IDLE_FENG_REG_ERR	Interrupt is not pending	0x0*
CMP	INT_PENDING_FENG_CMP	A CMP instruction with its interrupt field enabled has modified the FE_STATE register	0x1
	INT_IDLE_FENG_CMP	Interrupt is not pending	0x0*
PGE	INT_PENDING_FENG_PGE	FENG has finished executing the program	0x1
	INT_IDLE_FENG_PGE	Interrupt is not pending	0x0*
ISHC	INT_PENDING_FENG_ISHC	There are updated input shadow registers that have been transferred to the input registers	0x1
	INT_IDLE_FENG_ISHC	Interrupt is not pending	0x0*

Ezairo 8300 Hardware Reference

6.6.2.2 FENG_INT_STATUS_IO, D_FENG_INT_STATUS_IO

Bit Field	Read/Write	Field Name	Description
27	R	OUT11	New data capture at FENG output 11 interrupt status
26	R	OUT10	New data capture at FENG output 10 interrupt status
25	R	OUT9	New data capture at FENG output 9 interrupt status
24	R	OUT8	New data capture at FENG output 8 interrupt status
23	R	OUT7	New data capture at FENG output 7 interrupt status
22	R	OUT6	New data capture at FENG output 6 interrupt status
21	R	OUT5	New data capture at FENG output 5 interrupt status
20	R	OUT4	New data capture at FENG output 4 interrupt status
19	R	OUT3	New data capture at FENG output 3 interrupt status
18	R	OUT2	New data capture at FENG output 2 interrupt status
17	R	OUT1	New data capture at FENG output 1 interrupt status
16	R	OUT0	New data capture at FENG output 0 interrupt status
13	R	SRIN13	New FENG input 13 and signal ready arrival interrupt status
12	R	SRIN12	New FENG input 12 and signal ready arrival interrupt status
11	R	SRIN11	New FENG input 11 and signal ready arrival interrupt status
10	R	SRIN10	New FENG input 10 and signal ready arrival interrupt status
9	R	SRIN9	New FENG input 9 and signal ready arrival interrupt status
8	R	SRIN8	New FENG input 8 and signal ready arrival interrupt status
7	R	SRIN7	New FENG input 7 and signal ready arrival interrupt status
6	R	SRIN6	New FENG input 6 and signal ready arrival interrupt status
5	R	SRIN5	New FENG input 5 and signal ready arrival interrupt status
4	R	SRIN4	New FENG input 4 and signal ready arrival interrupt status
3	R	SRIN3	New FENG input 3 and signal ready arrival interrupt status
2	R	SRIN2	New FENG input 2 and signal ready arrival interrupt status
1	R	SRIN1	New FENG input 1 and signal ready arrival interrupt status
0	R	SRIN0	New FENG input 0 and signal ready arrival interrupt status

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
OUT11	INT_PENDING_FENG_OUT11	New output data captured for transfer into output shadow register	0x1
	INT_IDLE_FENG_OUT11	Interrupt is not pending	0x0*
OUT10	INT_PENDING_FENG_OUT10	New output data captured for transfer into output shadow register	0x1
	INT_IDLE_FENG_OUT10	Interrupt is not pending	0x0*
OUT9	INT_PENDING_FENG_OUT9	New output data captured for transfer into output shadow register	0x1
	INT_IDLE_FENG_OUT9	Interrupt is not pending	0x0*
OUT8	INT_PENDING_FENG_OUT8	New output data captured for transfer into output shadow register	0x1
	INT_IDLE_FENG_OUT8	Interrupt is not pending	0x0*
OUT7	INT_PENDING_FENG_OUT7	New output data captured for transfer into output shadow register	0x1
	INT_IDLE_FENG_OUT7	Interrupt is not pending	0x0*
OUT6	INT_PENDING_FENG_OUT6	New output data captured for transfer into output shadow register	0x1
	INT_IDLE_FENG_OUT6	Interrupt is not pending	0x0*
OUT5	INT_PENDING_FENG_OUT5	New output data captured for transfer into output shadow register	0x1
	INT_IDLE_FENG_OUT5	Interrupt is not pending	0x0*
OUT4	INT_PENDING_FENG_OUT4	New output data captured for transfer into output shadow register	0x1
	INT_IDLE_FENG_OUT4	Interrupt is not pending	0x0*
OUT3	INT_PENDING_FENG_OUT3	New output data captured for transfer into output shadow register	0x1
	INT_IDLE_FENG_OUT3	Interrupt is not pending	0x0*
OUT2	INT_PENDING_FENG_OUT2	New output data captured for transfer into output shadow register	0x1
	INT_IDLE_FENG_OUT2	Interrupt is not pending	0x0*
OUT1	INT_PENDING_FENG_OUT1	New output data captured for transfer into output shadow register	0x1
	INT_IDLE_FENG_OUT1	Interrupt is not pending	0x0*
OUT0	INT_PENDING_FENG_OUT0	New output data captured for transfer into output shadow register	0x1
	INT_IDLE_FENG_OUT0	Interrupt is not pending	0x0*
SRIN13	INT_PENDING_FENG_SRIN13	A new input sample and sample ready signal have arrived	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	INT_IDLE_FENG_SRIN13	Interrupt is not pending	0x0*
SRIN12	INT_PENDING_FENG_SRIN12	A new input sample and sample ready signal have arrived	0x1
	INT_IDLE_FENG_SRIN12	Interrupt is not pending	0x0*
SRIN11	INT_PENDING_FENG_SRIN11	A new input sample and sample ready signal have arrived	0x1
	INT_IDLE_FENG_SRIN11	Interrupt is not pending	0x0*
SRIN10	INT_PENDING_FENG_SRIN10	A new input sample and sample ready signal have arrived	0x1
	INT_IDLE_FENG_SRIN10	Interrupt is not pending	0x0*
SRIN9	INT_PENDING_FENG_SRIN9	A new input sample and sample ready signal have arrived	0x1
	INT_IDLE_FENG_SRIN9	Interrupt is not pending	0x0*
SRIN8	INT_PENDING_FENG_SRIN8	A new input sample and sample ready signal have arrived	0x1
	INT_IDLE_FENG_SRIN8	Interrupt is not pending	0x0*
SRIN7	INT_PENDING_FENG_SRIN7	A new input sample and sample ready signal have arrived	0x1
	INT_IDLE_FENG_SRIN7	Interrupt is not pending	0x0*
SRIN6	INT_PENDING_FENG_SRIN6	A new input sample and sample ready signal have arrived	0x1
	INT_IDLE_FENG_SRIN6	Interrupt is not pending	0x0*
SRIN5	INT_PENDING_FENG_SRIN5	A new input sample and sample ready signal have arrived	0x1
	INT_IDLE_FENG_SRIN5	Interrupt is not pending	0x0*
SRIN4	INT_PENDING_FENG_SRIN4	A new input sample and sample ready signal have arrived	0x1
	INT_IDLE_FENG_SRIN4	Interrupt is not pending	0x0*
SRIN3	INT_PENDING_FENG_SRIN3	A new input sample and sample ready signal have arrived	0x1
	INT_IDLE_FENG_SRIN3	Interrupt is not pending	0x0*
SRIN2	INT_PENDING_FENG_SRIN2	A new input sample and sample ready signal have arrived	0x1
	INT_IDLE_FENG_SRIN2	Interrupt is not pending	0x0*
SRIN1	INT_PENDING_FENG_SRIN1	A new input sample and sample ready signal have arrived	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	INT_IDLE_FENG_SRIN1	Interrupt is not pending	0x0*
SRIN0	INT_PENDING_FENG_SRIN0	A new input sample and sample ready signal have arrived	0x1
	INT_IDLE_FENG_SRIN0	Interrupt is not pending	0x0*

6.6.2.3 FENG_INT_ACK, D_FENG_INT_ACK

Bit Field	Read/Write	Field Name	Description
9	W	SRIN_ACK	FENG sample input and signal ready arrival interrupt acknowledge. Acknowledging with this bit clears the SRIN bit under the FENG_INT_STATUS as well as all the SRIN fields under the FENG_INT_STATUS_IO
8	W	OUT_ACK	New data capture at FENG output interrupt acknowledge. Acknowledging with this bit clears the OUT bit under the FENG_INT_STATUS as well as all the OUT fields under the FENG_INT_STATUS_IO
7	W	OVERLOAD_ACK	Processor overload interrupt acknowledge
6	W	ALU_SAT_ACK	ALU saturation interrupt acknowledge
5	W	CORE_SAT_ACK	Core saturation interrupt acknowledge
4	W	MEM_ERR_ACK	Illegal memory access interrupt acknowledge
3	W	REG_ERR_ACK	Register write access collision interrupt acknowledge
2	W	CMP_ACK	Compare instruction interrupt acknowledge
1	W	PGE_ACK	End of PGM execution interrupt acknowledge
0	W	ISHC_ACK	Input shadow registers capture in RF interrupt acknowledge

Field Name	Value Symbol	Value Description	Hex Value
SRIN_ACK	INT_ACK_FENG_SRIN	Acknowledge the interrupt. This clears all input status under FENG_INT_STATUS_IO as well	0x1
OUT_ACK	INT_ACK_FENG_OUT	Acknowledge the interrupt. This clears all output status under FENG_INT_STATUS_IO as well	0x1
OVERLOAD_ACK	INT_ACK_FENG_OVERLOAD	Acknowledge the interrupt	0x1
ALU_SAT_ACK	INT_ACK_FENG_ALU_SAT	Acknowledge the interrupt	0x1
CORE_SAT_ACK	INT_ACK_FENG_CORE_SAT	Acknowledge the interrupt	0x1
MEM_ERR_ACK	INT_ACK_FENG_MEM_ERR	Acknowledge the interrupt	0x1
REG_ERR_ACK	INT_ACK_FENG_REG_ERR	Acknowledge the interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
CMP_ACK	INT_ACK_FENG_CMP	Acknowledge the interrupt	0x1
PGE_ACK	INT_ACK_FENG_PGE	Acknowledge the interrupt	0x1
ISHC_ACK	INT_ACK_FENG_ISHC	Acknowledge the interrupt	0x1

6.6.2.4 FENG_INT_ACK_IO, D_FENG_INT_ACK_IO

Bit Field	Read/Write	Field Name	Description
27	W	OUT11_ACK	Data capture at FENG output11 interrupt acknowledge
26	W	OUT10_ACK	Data capture at FENG output10 interrupt acknowledge
25	W	OUT9_ACK	Data capture at FENG output9 interrupt acknowledge
24	W	OUT8_ACK	Data capture at FENG output8 interrupt acknowledge
23	W	OUT7_ACK	Data capture at FENG output7 interrupt acknowledge
22	W	OUT6_ACK	Data capture at FENG output6 interrupt acknowledge
21	W	OUT5_ACK	Data capture at FENG output5 interrupt acknowledge
20	W	OUT4_ACK	Data capture at FENG output4 interrupt acknowledge
19	W	OUT3_ACK	Data capture at FENG output3 interrupt acknowledge
18	W	OUT2_ACK	Data capture at FENG output2 interrupt acknowledge
17	W	OUT1_ACK	Data capture at FENG output1 interrupt acknowledge
16	W	OUT0_ACK	Data capture at FENG output0 interrupt acknowledge
13	W	SRIN13_ACK	FENG input13 and signal ready arrival interrupt acknowledge
12	W	SRIN12_ACK	FENG input12 and signal ready arrival interrupt acknowledge
11	W	SRIN11_ACK	FENG input11 and signal ready arrival interrupt acknowledge
10	W	SRIN10_ACK	FENG input10 and signal ready arrival interrupt acknowledge
9	W	SRIN9_ACK	FENG input9 and signal ready arrival interrupt acknowledge
8	W	SRIN8_ACK	FENG input8 and signal ready arrival interrupt acknowledge
7	W	SRIN7_ACK	FENG input7 and signal ready arrival interrupt acknowledge
6	W	SRIN6_ACK	FENG input6 and signal ready arrival interrupt acknowledge
5	W	SRIN5_ACK	FENG input5 and signal ready arrival interrupt acknowledge
4	W	SRIN4_ACK	FENG input4 and signal ready arrival interrupt acknowledge
3	W	SRIN3_ACK	FENG input3 and signal ready arrival interrupt acknowledge
2	W	SRIN2_ACK	FENG input2 and signal ready arrival interrupt acknowledge
1	W	SRIN1_ACK	FENG input1 and signal ready arrival interrupt acknowledge
0	W	SRIN0_ACK	FENG input0 and signal ready arrival interrupt acknowledge

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
OUT11_ACK	INT_ACK_FENG_OUT11	Acknowledge the interrupt	0x1
OUT10_ACK	INT_ACK_FENG_OUT10	Acknowledge the interrupt	0x1
OUT9_ACK	INT_ACK_FENG_OUT9	Acknowledge the interrupt	0x1
OUT8_ACK	INT_ACK_FENG_OUT8	Acknowledge the interrupt	0x1
OUT7_ACK	INT_ACK_FENG_OUT7	Acknowledge the interrupt	0x1
OUT6_ACK	INT_ACK_FENG_OUT6	Acknowledge the interrupt	0x1
OUT5_ACK	INT_ACK_FENG_OUT5	Acknowledge the interrupt	0x1
OUT4_ACK	INT_ACK_FENG_OUT4	Acknowledge the interrupt	0x1
OUT3_ACK	INT_ACK_FENG_OUT3	Acknowledge the interrupt	0x1
OUT2_ACK	INT_ACK_FENG_OUT2	Acknowledge the interrupt	0x1
OUT1_ACK	INT_ACK_FENG_OUT1	Acknowledge the interrupt	0x1
OUT0_ACK	INT_ACK_FENG_OUT0	Acknowledge the interrupt	0x1
SRIN13_ACK	INT_ACK_FENG_SRIN13	Acknowledge the interrupt	0x1
SRIN12_ACK	INT_ACK_FENG_SRIN12	Acknowledge the interrupt	0x1
SRIN11_ACK	INT_ACK_FENG_SRIN11	Acknowledge the interrupt	0x1
SRIN10_ACK	INT_ACK_FENG_SRIN10	Acknowledge the interrupt	0x1
SRIN9_ACK	INT_ACK_FENG_SRIN9	Acknowledge the interrupt	0x1
SRIN8_ACK	INT_ACK_FENG_SRIN8	Acknowledge the interrupt	0x1
SRIN7_ACK	INT_ACK_FENG_SRIN7	Acknowledge the interrupt	0x1
SRIN6_ACK	INT_ACK_FENG_SRIN6	Acknowledge the interrupt	0x1
SRIN5_ACK	INT_ACK_FENG_SRIN5	Acknowledge the interrupt	0x1
SRIN4_ACK	INT_ACK_FENG_SRIN4	Acknowledge the interrupt	0x1
SRIN3_ACK	INT_ACK_FENG_SRIN3	Acknowledge the interrupt	0x1
SRIN2_ACK	INT_ACK_FENG_SRIN2	Acknowledge the interrupt	0x1
SRIN1_ACK	INT_ACK_FENG_SRIN1	Acknowledge the interrupt	0x1
SRIN0_ACK	INT_ACK_FENG_SRIN0	Acknowledge the interrupt	0x1

Ezairo 8300 Hardware Reference

6.6.2.5 FENG_INT_ENABLE, D_FENG_INT_ENABLE

Bit Field	Read/Write	Field Name	Description
9	RW	SRIN	New FENG sample input and signal ready arrival interrupt enable. This enable propagates the SRIN interrupts enabled in the INT_ENABLE_IO register
8	RW	OUT	New data capture at FENG output interrupt enable. This enable propagates the OUT interrupts enabled in the INT_ENABLE_IO register
7	RW	OVERLOAD	Processor overload interrupt enable
6	RW	ALU_SAT	ALU saturation interrupt enable
5	RW	CORE_SAT	Core saturation interrupt enable
4	RW	MEM_ERR	Illegal memory access interrupt enable
3	RW	REG_ERR	Register write access collision interrupt enable
2	RW	CMP	Compare instruction interrupt enable
1	RW	PGE	End of PGM execution interrupt enable
0	RW	ISHC	Input shadow registers capture in RF interrupt enable

Field Name	Value Symbol	Value Description	Hex Value
SRIN	INT_DIS_FENG_SRIN	The SRINs cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN	This SRINs can set the FENG interrupt	0x1
OUT	INT_DIS_FENG_OUT	The output captures cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT	The output captures can set the FENG interrupt	0x1
OVERLOAD	INT_DIS_FENG_OVERLOAD	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OVERLOAD	This source can set the FENG interrupt	0x1
ALU_SAT	INT_DIS_FENG_ALU_SAT	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_ALU_SAT	This source can set the FENG interrupt	0x1
CORE_SAT	INT_DIS_FENG_CORE_SAT	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_CORE_SAT	This source can set the FENG interrupt	0x1
MEM_ERR	INT_DIS_FENG_MEM_ERR	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_MEM_ERR	This source can set the FENG interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
REG_ERR	INT_DIS_FENG_REG_ERR	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_REG_ERR	This source can set the FENG interrupt	0x1
CMP	INT_DIS_FENG_CMP	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_CMP	This source can set the FENG interrupt	0x1
PGE	INT_DIS_FENG_PGE	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_PGE	This source can set the FENG interrupt	0x1
ISHC	INT_DIS_FENG_ISHC	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_ISHC	This source can set the FENG interrupt	0x1

6.6.2.6 FENG_INT_ENABLE_IO, D_FENG_INT_ENABLE_IO

Bit Field	Read/Write	Field Name	Description
27	RW	OUT11	New data capture at FENG output 11 interrupt enable
26	RW	OUT10	New data capture at FENG output 10 interrupt enable
25	RW	OUT9	New data capture at FENG output 9 interrupt enable
24	RW	OUT8	New data capture at FENG output 8 interrupt enable
23	RW	OUT7	New data capture at FENG output 7 interrupt enable
22	RW	OUT6	New data capture at FENG output 6 interrupt enable
21	RW	OUT5	New data capture at FENG output 5 interrupt enable
20	RW	OUT4	New data capture at FENG output 4 interrupt enable
19	RW	OUT3	New data capture at FENG output 3 interrupt enable
18	RW	OUT2	New data capture at FENG output 2 interrupt enable
17	RW	OUT1	New data capture at FENG output 1 interrupt enable
16	RW	OUT0	New data capture at FENG output 0 interrupt enable
13	RW	SRIN13	New FENG input 13 and signal ready arrival interrupt enable
12	RW	SRIN12	New FENG input 12 and signal ready arrival interrupt enable
11	RW	SRIN11	New FENG input 11 and signal ready arrival interrupt enable
10	RW	SRIN10	New FENG input 10 and signal ready arrival interrupt enable
9	RW	SRIN9	New FENG input 9 and signal ready arrival interrupt enable
8	RW	SRIN8	New FENG input 8 and signal ready arrival interrupt enable

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
7	RW	SRIN7	New FENG input 7 and signal ready arrival interrupt enable
6	RW	SRIN6	New FENG input 6 and signal ready arrival interrupt enable
5	RW	SRIN5	New FENG input 5 and signal ready arrival interrupt enable
4	RW	SRIN4	New FENG input 4 and signal ready arrival interrupt enable
3	RW	SRIN3	New FENG input 3 and signal ready arrival interrupt enable
2	RW	SRIN2	New FENG input 2 and signal ready arrival interrupt enable
1	RW	SRIN1	New FENG input 1 and signal ready arrival interrupt enable
0	RW	SRIN0	New FENG input 0 and signal ready arrival interrupt enable

Field Name	Value Symbol	Value Description	Hex Value
OUT11	INT_DIS_FENG_OUT11	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT11	This source can set the FENG interrupt	0x1
OUT10	INT_DIS_FENG_OUT10	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT10	This source can set the FENG interrupt	0x1
OUT9	INT_DIS_FENG_OUT9	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT9	This source can set the FENG interrupt	0x1
OUT8	INT_DIS_FENG_OUT8	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT8	This source can set the FENG interrupt	0x1
OUT7	INT_DIS_FENG_OUT7	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT7	This source can set the FENG interrupt	0x1
OUT6	INT_DIS_FENG_OUT6	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT6	This source can set the FENG interrupt	0x1
OUT5	INT_DIS_FENG_OUT5	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT5	This source can set the FENG interrupt	0x1
OUT4	INT_DIS_FENG_OUT4	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT4	This source can set the FENG interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
OUT3	INT_DIS_FENG_OUT3	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT3	This source can set the FENG interrupt	0x1
OUT2	INT_DIS_FENG_OUT2	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT2	This source can set the FENG interrupt	0x1
OUT1	INT_DIS_FENG_OUT1	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT1	This source can set the FENG interrupt	0x1
OUT0	INT_DIS_FENG_OUT0	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_OUT0	This source can set the FENG interrupt	0x1
SRIN13	INT_DIS_FENG_SRIN13	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN13	This source can set the FENG interrupt	0x1
SRIN12	INT_DIS_FENG_SRIN12	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN12	This source can set the FENG interrupt	0x1
SRIN11	INT_DIS_FENG_SRIN11	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN11	This source can set the FENG interrupt	0x1
SRIN10	INT_DIS_FENG_SRIN10	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN10	This source can set the FENG interrupt	0x1
SRIN9	INT_DIS_FENG_SRIN9	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN9	This source can set the FENG interrupt	0x1
SRIN8	INT_DIS_FENG_SRIN8	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN8	This source can set the FENG interrupt	0x1
SRIN7	INT_DIS_FENG_SRIN7	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN7	This source can set the FENG interrupt	0x1
SRIN6	INT_DIS_FENG_SRIN6	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN6	This source can set the FENG interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
SRIN5	INT_DIS_FENG_SRIN5	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN5	This source can set the FENG interrupt	0x1
SRIN4	INT_DIS_FENG_SRIN4	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN4	This source can set the FENG interrupt	0x1
SRIN3	INT_DIS_FENG_SRIN3	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN3	This source can set the FENG interrupt	0x1
SRIN2	INT_DIS_FENG_SRIN2	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN2	This source can set the FENG interrupt	0x1
SRIN1	INT_DIS_FENG_SRIN1	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN1	This source can set the FENG interrupt	0x1
SRIN0	INT_DIS_FENG_SRIN0	This source cannot set an interrupt	0x0*
	INT_EBL_FENG_SRIN0	This source can set the FENG interrupt	0x1

6.6.2.7 FENG_CFG, D_FENG_CFG

Bit Field	Read/Write	Field Name	Description
9:4	RW	COEFF_OFFSET	The coefficient decoder offset
3:0	RW	SRI_SEL	Sample ready input starting the FENG processing

Field Name	Value Symbol	Value Description	Hex Value
SRI_SEL	FENG_SRI_SELECT_IN0	FENG executes on input sample ready 0	0x0
	FENG_SRI_SELECT_IN1	FENG executes on input sample ready 1	0x1
	FENG_SRI_SELECT_IN2	FENG executes on input sample ready 2	0x2
	FENG_SRI_SELECT_IN3	FENG executes on input sample ready 3	0x3
	FENG_SRI_SELECT_IN4	FENG executes on input sample ready 4	0x4
	FENG_SRI_SELECT_IN5	FENG executes on input sample ready 5	0x5
	FENG_SRI_SELECT_IN6	FENG executes on input sample ready	0x6

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		6	
	FENG_SRI_SELECT_IN7	FENG executes on input sample ready 7	0x7
	FENG_SRI_SELECT_IN8	FENG executes on input sample ready 8	0x8
	FENG_SRI_SELECT_IN9	FENG executes on input sample ready 9	0x9
	FENG_SRI_SELECT_IN10	FENG executes on input sample ready 10	0xA
	FENG_SRI_SELECT_IN11	FENG executes on input sample ready 11	0xB
	FENG_SRI_SELECT_IN12	FENG executes on input sample ready 12	0xC
	FENG_SRI_SELECT_IN13	FENG executes on input sample ready 13	0xD
	FENG_SRI_DISABLE	FENG is disabled	0xF*

6.6.2.8 FENG_CFG_IO, D_FENG_CFG_IO

Bit Field	Read/Write	Field Name	Description
27	RW	OSR11_EBL	Shadow register capture mode for output 11
26	RW	OSR10_EBL	Shadow register capture mode for output 10
25	RW	OSR9_EBL	Shadow register capture mode for output 9
24	RW	OSR8_EBL	Shadow register capture mode for output 8
23	RW	OSR7_EBL	Shadow register capture mode for output 7
22	RW	OSR6_EBL	Shadow register capture mode for output 6
21	RW	OSR5_EBL	Shadow register capture mode for output 5
20	RW	OSR4_EBL	Shadow register capture mode for output 4
19	RW	OSR3_EBL	Shadow register capture mode for output 3
18	RW	OSR2_EBL	Shadow register capture mode for output 2
17	RW	OSR1_EBL	Shadow register capture mode for output 1
16	RW	OSR0_EBL	Shadow register capture mode for output 0
13	RW	FIN13_EBL	Enable/Disable sample capture on input 13
12	RW	FIN12_EBL	Enable/Disable sample capture on input 12
11	RW	FIN11_EBL	Enable/Disable sample capture on input 11
10	RW	FIN10_EBL	Enable/Disable sample capture on input 10

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
9	RW	FIN9_EBL	Enable/Disable sample capture on input 9
8	RW	FIN8_EBL	Enable/Disable sample capture on input 8
7	RW	FIN7_EBL	Enable/Disable sample capture on input 7
6	RW	FIN6_EBL	Enable/Disable sample capture on input 6
5	RW	FIN5_EBL	Enable/Disable sample capture on input 5
4	RW	FIN4_EBL	Enable/Disable sample capture on input 4
3	RW	FIN3_EBL	Enable/Disable sample capture on input 3
2	RW	FIN2_EBL	Enable/Disable sample capture on input 2
1	RW	FIN1_EBL	Enable/Disable sample capture on input 1
0	RW	FIN0_EBL	Enable/Disable sample capture on input 0

Field Name	Value Symbol	Value Description	Hex Value
OSR11_EBL	FENG_OSR11_DISABLE	Disable the shadow register for FENG output 11	0x0
	FENG_OSR11_ENABLE	Enable the shadow register for FENG output 11	0x1*
OSR10_EBL	FENG_OSR10_DISABLE	Disable the shadow register for FENG output 10	0x0
	FENG_OSR10_ENABLE	Enable the shadow register for FENG output 10	0x1*
OSR9_EBL	FENG_OSR9_DISABLE	Disable the shadow register for FENG output 9	0x0
	FENG_OSR9_ENABLE	Enable the shadow register for FENG output 9	0x1*
OSR8_EBL	FENG_OSR8_DISABLE	Disable the shadow register for FENG output 8	0x0
	FENG_OSR8_ENABLE	Enable the shadow register for FENG output 8	0x1*
OSR7_EBL	FENG_OSR7_DISABLE	Disable the shadow register for FENG output 7	0x0
	FENG_OSR7_ENABLE	Enable the shadow register for FENG output 7	0x1*
OSR6_EBL	FENG_OSR6_DISABLE	Disable the shadow register for FENG output 6	0x0
	FENG_OSR6_ENABLE	Enable the shadow register for FENG output 6	0x1*
OSR5_EBL	FENG_OSR5_DISABLE	Disable the shadow register for FENG output 5	0x0

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	FENG_OSR5_ENABLE	Enable the shadow register for FENG output 5	0x1*
OSR4_EBL	FENG_OSR4_DISABLE	Disable the shadow register for FENG output 4	0x0
	FENG_OSR4_ENABLE	Enable the shadow register for FENG output 4	0x1*
OSR3_EBL	FENG_OSR3_DISABLE	Disable the shadow register for FENG output 3	0x0
	FENG_OSR3_ENABLE	Enable the shadow register for FENG output 3	0x1*
OSR2_EBL	FENG_OSR2_DISABLE	Disable the shadow register for FENG output 2	0x0
	FENG_OSR2_ENABLE	Enable the shadow register for FENG output 2	0x1*
OSR1_EBL	FENG_OSR1_DISABLE	Disable the shadow register for FENG output 1	0x0
	FENG_OSR1_ENABLE	Enable the shadow register for FENG output 1	0x1*
OSR0_EBL	FENG_OSR0_DISABLE	Disable the shadow register for FENG output 0	0x0
	FENG_OSR0_ENABLE	Enable the shadow register for FENG output 0	0x1*
FIN13_EBL	FENG_FIN13_DISABLE	Disable FENG input 13	0x0*
	FENG_FIN13_ENABLE	Enable FENG input 13	0x1
FIN12_EBL	FENG_FIN12_DISABLE	Disable FENG input 12	0x0*
	FENG_FIN12_ENABLE	Enable FENG input 12	0x1
FIN11_EBL	FENG_FIN11_DISABLE	Disable FENG input 11	0x0*
	FENG_FIN11_ENABLE	Enable FENG input 11	0x1
FIN10_EBL	FENG_FIN10_DISABLE	Disable FENG input 10	0x0*
	FENG_FIN10_ENABLE	Enable FENG input 10	0x1
FIN9_EBL	FENG_FIN9_DISABLE	Disable FENG input 9	0x0*
	FENG_FIN9_ENABLE	Enable FENG input 9	0x1
FIN8_EBL	FENG_FIN8_DISABLE	Disable FENG input 8	0x0*
	FENG_FIN8_ENABLE	Enable FENG input 8	0x1
FIN7_EBL	FENG_FIN7_DISABLE	Disable FENG input 7	0x0*
	FENG_FIN7_ENABLE	Enable FENG input 7	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
FIN6_EBL	FENG_FIN6_DISABLE	Disable FENG input 6	0x0*
	FENG_FIN6_ENABLE	Enable FENG input 6	0x1
FIN5_EBL	FENG_FIN5_DISABLE	Disable FENG input 5	0x0*
	FENG_FIN5_ENABLE	Enable FENG input 5	0x1
FIN4_EBL	FENG_FIN4_DISABLE	Disable FENG input 4	0x0*
	FENG_FIN4_ENABLE	Enable FENG input 4	0x1
FIN3_EBL	FENG_FIN3_DISABLE	Disable FENG input 3	0x0*
	FENG_FIN3_ENABLE	Enable FENG input 3	0x1
FIN2_EBL	FENG_FIN2_DISABLE	Disable FENG input 2	0x0*
	FENG_FIN2_ENABLE	Enable FENG input 2	0x1
FIN1_EBL	FENG_FIN1_DISABLE	Disable FENG input 1	0x0*
	FENG_FIN1_ENABLE	Enable FENG input 1	0x1
FIN0_EBL	FENG_FIN0_DISABLE	Disable FENG input 0	0x0*
	FENG_FIN0_ENABLE	Enable FENG input 0	0x1

6.6.2.9 FENG_MRCNT, D_FENG_MRCNT

Bit Field	Read/Write	Field Name	Description
19	RW	MRCNT_SYNC_EN	The continuous synchronization enable of the MRCNT Phase0
18:16	RW	MRCNT_SYNC_SOURCE	Sample ready input to synchronize the Multirate Counters
15:12	RW	MRCNT1	Multirate Counter 1 Value
11:8	RW	MRCNT0	Multirate Counter 0 Value
7:4	RW	MRCNT1_DR	Multirate Counter 1 Divide Ratio (0 to 15)
3:0	RW	MRCNT0_DR	Multirate Counter 0 Divide Ratio (0 to 15)

Field Name	Value Symbol	Value Description	Hex Value
MRCNT_SYNC_EN	FENG_MRCNT_SYNC_DISABLE	The MRCNT synchronization is disabled	0x0*
	FENG_MRCNT_SYNC_ENABLE	The configured SRIN clears the MRCNTs to zero	0x1
MRCNT_SYNC_SOURCE	FENG_MRCNT_SYNC_IN0	FENG synchronizes the MRCNTs to the SR_IN0	0x0*
	FENG_MRCNT_SYNC_IN1	FENG synchronizes the MRCNTs to the SR_IN1	0x1
	FENG_MRCNT_SYNC_IN2	FENG synchronizes the MRCNTs to	0x2

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		the SR_IN2	
	FENG_MRCNT_SYNC_IN3	FENG synchronizes the MRCNTs to the SR_IN3	0x3
	FENG_MRCNT_SYNC_IN4	FENG synchronizes the MRCNTs to the SR_IN4	0x4
	FENG_MRCNT_SYNC_IN5	FENG synchronizes the MRCNTs to the SR_IN5	0x5
	FENG_MRCNT_SYNC_IN6	FENG synchronizes the MRCNTs to the SR_IN6	0x6
	FENG_MRCNT_SYNC_IN7	FENG synchronizes the MRCNTs to the SR_IN7	0x7

6.6.2.10 FENG_PGM_SECTOR_EN, D_FENG_PGM_SECTOR_EN

Bit Field	Read/Write	Field Name	Description
31:16	RW	CNT1	FENG_MRCNT_MRCNT1 counter phase controlled sectors enable
15:0	RW	CNT0	FENG_MRCNT_MRCNT0 counter phase controlled sectors enable

6.6.2.11 FENG_PHM_PH0_PH1, D_FENG_PHM_PH0_PH1

Bit Field	Read/Write	Field Name	Description
31:24	RW	PHM_CNT1PH1	FENG_MRCNT_MRCNT1 phase 1 controlled program memory sector start
23:16	RW	PHM_CNT0PH1	FENG_MRCNT_MRCNT0 phase 1 controlled program memory sector start
15:8	RW	PHM_CNT1PH0	FENG_MRCNT_MRCNT1 phase 0 controlled program memory sector start
7:0	RW	PHM_CNT0PH0	FENG_MRCNT_MRCNT0 phase 0 controlled program memory sector start

6.6.2.12 FENG_PHM_PH4_PH5, D_FENG_PHM_PH4_PH5

Bit Field	Read/Write	Field Name	Description
31:24	RW	PHM_CNT1PH5	FENG_MRCNT_MRCNT1 phase 5 controlled program memory sector start
23:16	RW	PHM_CNT0PH5	FENG_MRCNT_MRCNT0 phase 5 controlled program memory sector start
15:8	RW	PHM_CNT1PH4	FENG_MRCNT_MRCNT1 phase 4 controlled program memory sector start
7:0	RW	PHM_CNT0PH4	FENG_MRCNT_MRCNT0 phase 4 controlled program memory sector start

Ezairo 8300 Hardware Reference

6.6.2.13 FENG_PHM_PH2_PH3, D_FENG_PHM_PH2_PH3

Bit Field	Read/Write	Field Name	Description
31:24	RW	PHM_CNT1PH3	FENG_MRCNT_MRCNT1 phase 3 controlled program memory sector start
23:16	RW	PHM_CNT0PH3	FENG_MRCNT_MRCNT0 phase 3 controlled program memory sector start
15:8	RW	PHM_CNT1PH2	FENG_MRCNT_MRCNT1 phase 2 controlled program memory sector start
7:0	RW	PHM_CNT0PH2	FENG_MRCNT_MRCNT0 phase 2 controlled program memory sector start

6.6.2.14 FENG_PHM_PH4_PH5, D_FENG_PHM_PH4_PH5

Bit Field	Read/Write	Field Name	Description
31:24	RW	PHM_CNT1PH5	FENG_MRCNT_MRCNT1 phase 5 controlled program memory sector start
23:16	RW	PHM_CNT0PH5	FENG_MRCNT_MRCNT0 phase 5 controlled program memory sector start
15:8	RW	PHM_CNT1PH4	FENG_MRCNT_MRCNT1 phase 4 controlled program memory sector start
7:0	RW	PHM_CNT0PH4	FENG_MRCNT_MRCNT0 phase 4 controlled program memory sector start

6.6.2.15 FENG_PHM_PH6_PH7, D_FENG_PHM_PH6_PH7

Bit Field	Read/Write	Field Name	Description
31:24	RW	PHM_CNT1PH7	FENG_MRCNT_MRCNT1 phase 7 controlled program memory sector start
23:16	RW	PHM_CNT0PH7	FENG_MRCNT_MRCNT0 phase 7 controlled program memory sector start
15:8	RW	PHM_CNT1PH6	FENG_MRCNT_MRCNT1 phase 6 controlled program memory sector start
7:0	RW	PHM_CNT0PH6	FENG_MRCNT_MRCNT0 phase 6 controlled program memory sector start

6.6.2.16 FENG_PHM_PH8_PH9, D_FENG_PHM_PH8_PH9

Bit Field	Read/Write	Field Name	Description
31:24	RW	PHM_CNT1PH9	FENG_MRCNT_MRCNT1 phase 9 controlled program memory sector start
23:16	RW	PHM_CNT0PH9	FENG_MRCNT_MRCNT0 phase 9 controlled program memory sector start
15:8	RW	PHM_CNT1PH8	FENG_MRCNT_MRCNT1 phase 8 controlled program memory sector start

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
7:0	RW	PHM_CNT0PH8	FENG_MRCNT_MRCNT0 phase 8 controlled program memory sector start

6.6.2.17 FENG_PHM_PH10_PH11, D_FENG_PHM_PH10_PH11

Bit Field	Read/Write	Field Name	Description
31:24	RW	PHM_CNT1PH11	FENG_MRCNT_MRCNT1 phase 11 controlled program memory sector start
23:16	RW	PHM_CNT0PH11	FENG_MRCNT_MRCNT0 phase 11 controlled program memory sector start
15:8	RW	PHM_CNT1PH10	FENG_MRCNT_MRCNT1 phase 10 controlled program memory sector start
7:0	RW	PHM_CNT0PH10	FENG_MRCNT_MRCNT0 phase 10 controlled program memory sector start

6.6.2.18 FENG_PHM_PH12_PH13, D_FENG_PHM_PH12_PH13

Bit Field	Read/Write	Field Name	Description
31:24	RW	PHM_CNT1PH13	FENG_MRCNT_MRCNT1 phase 13 controlled program memory sector start
23:16	RW	PHM_CNT0PH13	FENG_MRCNT_MRCNT0 phase 13 controlled program memory sector start
15:8	RW	PHM_CNT1PH12	FENG_MRCNT_MRCNT1 phase 12 controlled program memory sector start
7:0	RW	PHM_CNT0PH12	FENG_MRCNT_MRCNT0 phase 12 controlled program memory sector start

6.6.2.19 FENG_PHM_PH14_PH15, D_FENG_PHM_PH14_PH15

Bit Field	Read/Write	Field Name	Description
31:24	RW	PHM_CNT1PH15	FENG_MRCNT_MRCNT1 phase 15 controlled program memory sector start
23:16	RW	PHM_CNT0PH15	FENG_MRCNT_MRCNT0 phase 15 controlled program memory sector start
15:8	RW	PHM_CNT1PH14	FENG_MRCNT_MRCNT1 phase 14 controlled program memory sector start
7:0	RW	PHM_CNT0PH14	FENG_MRCNT_MRCNT0 phase 14 controlled program memory sector start

6.6.2.20 FENG_PHM_PH15_END, D_FENG_PHM_PH15_END

Bit Field	Read/Write	Field Name	Description
7:0	RW	PH15_END	FENG_MRCNT_MRCNT1 phase 15 program memory sector end

Ezairo 8300 Hardware Reference

6.6.2.21 FENG_COEFF_OFFSET_EN_SHADOW0, D_FENG_COEFF_OFFSET_EN_SHADOW0

Bit Field	Read/Write	Field Name	Description
31:0	RW	COEFF_OFFSET_EN_SHADOW0	The coefficients 0 to 31 offset enable shadow register

6.6.2.22 FENG_COEFF_OFFSET_EN_SHADOW1, D_FENG_COEFF_OFFSET_EN_SHADOW1

Bit Field	Read/Write	Field Name	Description
31:0	RW	COEFF_OFFSET_EN_SHADOW1	The coefficients 32 to 63 offset enable shadow register

6.6.2.23 FENG_SH_SHADOW, D_FENG_SH_SHADOW

Bit Field	Read/Write	Field Name	Description
23:21	RW	SHREG7_SHADOW	SH shadow value for address 7 mapping
20:18	RW	SHREG6_SHADOW	SH shadow value for address 6 mapping
17:15	RW	SHREG5_SHADOW	SH shadow value for address 5 mapping
14:12	RW	SHREG4_SHADOW	SH shadow value for address 4 mapping
11:9	RW	SHREG3_SHADOW	SH shadow value for address 3 mapping
8:6	RW	SHREG2_SHADOW	SH shadow value for address 2 mapping
5:3	RW	SHREG1_SHADOW	SH shadow value for address 1 mapping
2:0	RW	SHREG0_SHADOW	SH shadow value for address 0 mapping

6.6.2.24 FENG_TC_SHADOW0, D_FENG_TC_SHADOW0

Bit Field	Read/Write	Field Name	Description
31:28	RW	TCREG7_SHADOW	ATC or DTC shadow value for address 7 mapping
27:24	RW	TCREG6_SHADOW	ATC or DTC shadow value for address 6 mapping
23:20	RW	TCREG5_SHADOW	ATC or DTC shadow value for address 5 mapping
19:16	RW	TCREG4_SHADOW	ATC or DTC shadow value for address 4 mapping
15:12	RW	TCREG3_SHADOW	ATC or DTC shadow value for address 3 mapping
11:8	RW	TCREG2_SHADOW	ATC or DTC shadow value for address 2 mapping
7:4	RW	TCREG1_SHADOW	ATC or DTC shadow value for address 1 mapping
3:0	RW	TCREG0_SHADOW	ATC or DTC shadow value for address 0 mapping

Ezairo 8300 Hardware Reference

6.6.2.25 FENG_TC_SHADOW1, D_FENG_TC_SHADOW1

Bit Field	Read/Write	Field Name	Description
31:28	RW	TCREG15_SHADOW	ATC or DTC shadow value for address 15 mapping
27:24	RW	TCREG14_SHADOW	ATC or DTC shadow value for address 14 mapping
23:20	RW	TCREG13_SHADOW	ATC or DTC shadow value for address 13 mapping
19:16	RW	TCREG12_SHADOW	ATC or DTC shadow value for address 12 mapping
15:12	RW	TCREG11_SHADOW	ATC or DTC shadow value for address 11 mapping
11:8	RW	TCREG10_SHADOW	ATC or DTC shadow value for address 10 mapping
7:4	RW	TCREG9_SHADOW	ATC or DTC shadow value for address 9 mapping
3:0	RW	TCREG8_SHADOW	ATC or DTC shadow value for address 8 mapping

6.6.2.26 FENG_STATE_W_EN, D_FENG_STATE_W_EN

Bit Field	Read/Write	Field Name	Description
31:0	RW	STATE_W_EN	The conditional processing state registers write enable

6.6.2.27 FENG_STATE_SHADOW, D_FENG_STATE_SHADOW

Bit Field	Read/Write	Field Name	Description
31:0	RW	STATE_SHADOW	The conditional processing state shadow register

6.6.2.28 FENG_STATE_ACTIVE, D_FENG_STATE_ACTIVE

Bit Field	Read/Write	Field Name	Description
31:0	RW	STATE_ACTIVE	The conditional processing state active register

6.6.2.29 FENG_COEFF_OFFSET_EN_ACTIVE0, D_FENG_COEFF_OFFSET_EN_ACTIVE0

Bit Field	Read/Write	Field Name	Description
31:0	RW	COEFF_OFFSET_EN_ACTIVE0	The coefficients 0 to 31 offset enable active register

6.6.2.30 FENG_COEFF_OFFSET_EN_ACTIVE1, D_FENG_COEFF_OFFSET_EN_ACTIVE1

Bit Field	Read/Write	Field Name	Description
31:0	RW	COEFF_OFFSET_EN_ACTIVE1	The coefficients 32 to 63 offset enable active register

Ezairo 8300 Hardware Reference

6.6.2.31 FENG_SH_ACTIVE, D_FENG_SH_ACTIVE

Bit Field	Read/Write	Field Name	Description
23:21	RW	SHREG7_ACTIVE	SH active value for address 7
20:18	RW	SHREG6_ACTIVE	SH active value for address 6
17:15	RW	SHREG5_ACTIVE	SH active value for address 5
14:12	RW	SHREG4_ACTIVE	SH active value for address 4
11:9	RW	SHREG3_ACTIVE	SH active value for address 3
8:6	RW	SHREG2_ACTIVE	SH active value for address 2
5:3	RW	SHREG1_ACTIVE	SH active value for address 1
2:0	RW	SHREG0_ACTIVE	SH active value for address 0

6.6.2.32 FENG_TC_ACTIVE0, D_FENG_TC_ACTIVE0

Bit Field	Read/Write	Field Name	Description
31:28	RW	TCREG7_ACTIVE	ATC or DTC active value for address 7 mapping
27:24	RW	TCREG6_ACTIVE	ATC or DTC active value for address 6 mapping
23:20	RW	TCREG5_ACTIVE	ATC or DTC active value for address 5 mapping
19:16	RW	TCREG4_ACTIVE	ATC or DTC active value for address 4 mapping
15:12	RW	TCREG3_ACTIVE	ATC or DTC active value for address 3 mapping
11:8	RW	TCREG2_ACTIVE	ATC or DTC active value for address 2 mapping
7:4	RW	TCREG1_ACTIVE	ATC or DTC active value for address 1 mapping
3:0	RW	TCREG0_ACTIVE	ATC or DTC active value for address 0 mapping

6.6.2.33 FENG_TC_ACTIVE1, D_FENG_TC_ACTIVE1

Bit Field	Read/Write	Field Name	Description
31:28	RW	TCREG15_ACTIVE	ATC or DTC active value for address 15 mapping
27:24	RW	TCREG14_ACTIVE	ATC or DTC active value for address 14 mapping
23:20	RW	TCREG13_ACTIVE	ATC or DTC active value for address 13 mapping
19:16	RW	TCREG12_ACTIVE	ATC or DTC active value for address 12 mapping
15:12	RW	TCREG11_ACTIVE	ATC or DTC active value for address 11 mapping
11:8	RW	TCREG10_ACTIVE	ATC or DTC active value for address 10 mapping
7:4	RW	TCREG9_ACTIVE	ATC or DTC active value for address 9 mapping
3:0	RW	TCREG8_ACTIVE	ATC or DTC active value for address 8 mapping

Ezairo 8300 Hardware Reference

6.6.2.34 FENG_CMD5, D_FENG_CMD5

Bit Field	Read/Write	Field Name	Description
3	W	SHADOW_TRANSFER	FENG Shadow to Active Transfer
2	W	EXEC	FENG Program Execution Start
1	W	DECODER_RESET	FENG Decoder Reset
0	W	AUDIO_RESET	FENG Audio Reset

Field Name	Value Symbol	Value Description	Hex Value
SHADOW_TRANSFER	FENG_SHADOW_TRANSFER	Transfer data from the updated shadow registers to active registers	0x1
EXEC	FENG_EXEC	Start a single FENG execution sequence	0x1
DECODER_RESET	FENG_DECODER_RESET	Reset the FENG decoder registers	0x1
AUDIO_RESET	FENG_AUDIO_RESET	Reset the FENG audio registers	0x1

6.6.2.35 FENG_CTRL, D_FENG_CTRL

Bit Field	Read/Write	Field Name	Description
4	W	SYNC_START	If set, the FENG_ENABLE bit is set only after the SRIN arrival defined by the MRCNT_SYNC_SOURCE.
3	W	RESUME_CLEAN	If set, the DECODER_RESET is issued, all the pending sample ready flags are cleared and multirate counters are reset. Thereafter the ENABLE bit is set
2	RW	PAUSE	If set, at the next execution start FENG is paused
1	W	SINGLE_STEP	If set, the CTRL_ENABLE is set for one clock cycle
0	RW	ENABLE	If cleared, all the clocks are shut down (CFX can still access FENG)

Field Name	Value Symbol	Value Description	Hex Value
SYNC_START	FENG_SYNC_START_DISABLE	FENG is not enabled.	0x0*
	FENG_SYNC_START_ENABLE	FENG is enabled after the arrival of the selected SRIN	0x1
RESUME_CLEAN	FENG_RESUME_CLEAN_DISABLE	FENG is not enabled.	0x0*
	FENG_RESUME_CLEAN_ENABLE	FENG is enabled after clearing out queued data	0x1
PAUSE	FENG_PAUSE_DISABLE	FENG can execute	0x0*
	FENG_PAUSE_ENABLE	FENG is paused at the start of the next program execution cycle	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
SINGLE_STEP	FENG_STEP	FENG executes 1 clock cycle of its program	0x1
ENABLE	FENG_CTRL_DISABLE	FENG is disabled; CFX accesses provide a clock edge to the region accessed	0x0*
	FENG_CTRL_ENABLE	FENG is enabled	0x1

6.6.2.36 FENG_WR_KEY, D_FENG_WR_KEY

Bit Field	Read/Write	Field Name	Description
31:0	W	FENG_WR_KEY	The FENG_WR_PROTECT register write access grant key

6.6.2.37 FENG_WR_PROTECT, D_FENG_WR_PROTECT

Bit Field	Read/Write	Field Name	Description
5	R	WR_KEY_STATUS	If set, the FENG_WR_KEY has been entered properly and there has not been an access to FENG_WR_PROTECT thereafter
4	RW	CFG_WR_PROTECT	If set, the configuration registers content cannot be updated
3	RW	RF_WR_PROTECT	If set, the register file content cannot be updated
2	RW	COEF_WR_PROTECT	If set, the coefficient memory cannot be updated
1	RW	SV_WR_PROTECT	If set, the state variable memory cannot be updated
0	RW	PGM_WR_PROTECT	If set, the program memory cannot be updated

Field Name	Value Symbol	Value Description	Hex Value
WR_KEY_STATUS	WR_BLOCKED	The FENG_WR_PROTECT address access blocked	0x0*
	WR_ALLOWED	The FENG_WR_PROTECT address access allowed	0x1
CFG_WR_PROTECT	CFG_WR_ENABLE	The configuration registers write access is allowed	0x0*
	CFG_WR_DISABLE	The configuration registers write access is blocked	0x1
RF_WR_PROTECT	RF_WR_ENABLE	The register file write access is allowed	0x0*
	RF_WR_DISABLE	The register file write access is blocked	0x1
COEF_WR_PROTECT	COEF_WR_ENABLE	The coefficient memory write access is allowed	0x0*
	COEF_WR_DISABLE	The coefficient memory write access is blocked	0x1
SV_WR_PROTECT	SV_WR_ENABLE	The state variable memory write	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		access is allowed	
	SV_WR_DISABLE	The state variable memory write access is blocked	0x1
PGM_WR_PROTECT	PGM_WR_ENABLE	The program memory write access is allowed	0x0*
	PGM_WR_DISABLE	The program memory write access is blocked	0x1

6.6.2.38 FENG_PGM_ADDR, D_FENG_PGM_ADDR

Bit Field	Read/Write	Field Name	Description
7:0	R	DECODER	Program memory decoder address

6.6.2.39 FENG_RF_EXTENDED*, D_FENG_RF_EXTENDED_*

Bit Field	Read/Write	Field Name	Description
26:0	RW	RF_EXTENDED	Data read in m5p23 format, written in m4p23 format (sign extended to 32bits)

6.6.2.40 FENG_RF_SINGLE*, D_FENG_RF_SINGLE_*

Bit Field	Read/Write	Field Name	Description
23:0	RW	RF_SINGLE	Data in m1p23 format (sign extended to 32bits)

6.6.2.41 FENG_RF_IN*, D_FENG_RF_IN_*

Bit Field	Read/Write	Field Name	Description
23:0	RW	RF_IN	Data in m1p23 format (sign extended to 32bits)

6.6.2.42 FENG_RF_OUT*, D_FENG_RF_OUT_*

Bit Field	Read/Write	Field Name	Description
23:0	RW	RF_OUT	Data in m1p23 format (sign extended to 32bits)

CHAPTER 7

Neural Network Accelerator

The Neural Network Accelerator (NNA) is a hardware block that allows running neural networks efficiently. The accelerator can execute a single layer of a fully populated or sparsely populated neural network in a single task without any processor intervention.

7.1 BLOCK OVERVIEW

After writing the input data to the NNA input buffer and starting the NNA (by setting the appropriate flag), the NNA computes the output results. When the accelerator is started, a full vector set (layer) is processed and an interrupt occurs when processing is completed. The accelerator can execute a single layer of a fully populated or sparsely populated neural network in a single task without any processor intervention. Layers with up to 1023 inputs and 1023 outputs are supported.

Replacing computationally heavy MAC loops with NNA calls is most easily understood with the fully-connected layer architecture, but it is also possible to use the NNA with other architectures (such as convolutional neural nets) by packing groups of filter coefficients and passing them to the NNA.

The block diagram in the "NNA Block Diagram" figure (Figure 5) shows the structure of the components of the NNA.

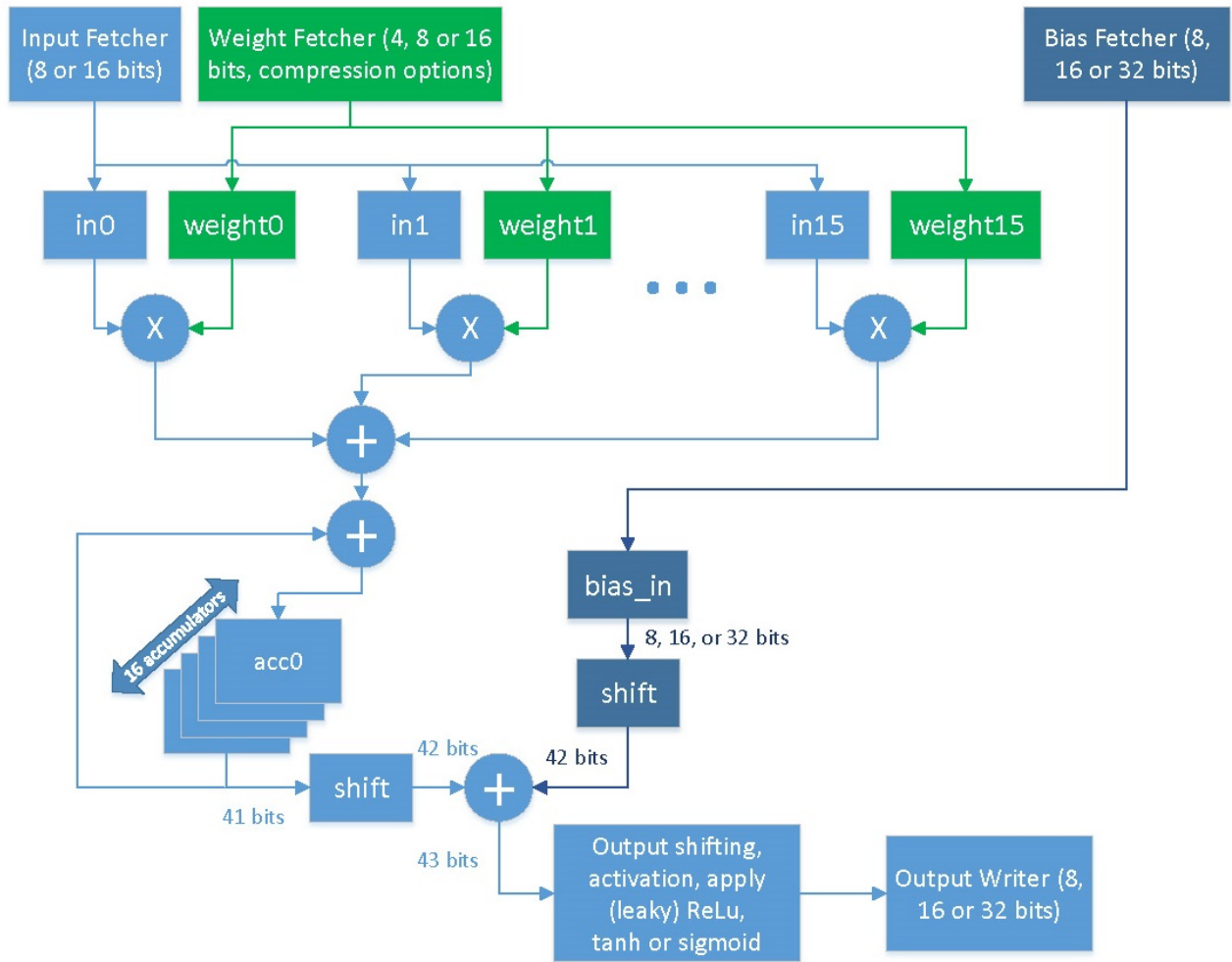


Figure 5. NNA Block Diagram

LPDSP32 and the Arm Cortex-M3 processor have direct Peripheral Bus access to the accelerator, and the CFX has access through a bus bridge to the Arm Cortex-M3 processor's system bus. When writing to the same register at the same time, the Arm Cortex-M3 core, DMA, or CFX write has priority and the LPDSP32 write is ignored. Input fetcher, weight fetcher, bias fetcher, and output writer share 2 x 32-bit data memory buses, and have access to all memories that are mapped to the LPDSP32 data memory A (DMA), providing up to 352 KB of total available RAM. If the LPDSP32 and the accelerator access a logical instance in the same cycle, then the accelerator is stalled. Data can be fetched starting at any byte address, and each fetch can read up to 64 bits of data in a cycle.

7.2 ATTRIBUTES AND HARDWARE FEATURES

The NNA processes input and weight data read from memory, accumulating the intermediate data. An optional bias is applied to this accumulated data before applying an activation function to form output data that is written to memory. The calculation by the NNA for each layer can be described as:

$$output_i = F_{activation}(\sum_{j=0}^{INPUT_CNT-1} weight_{i,j} * input_j + bias_i)$$

NOTE: The 16 multipliers and accumulators are 41-bit to avoid overflow in the output calculation.

NOTE: Multiplier and accumulator parallelism is reduced to 8 when using 16-bit weights.

Input, weight, bias, and output data used by the NNA is treated as:

- Signed
- Stored in little endian format when accessed from or stored to memory
- Configurable to be interpreted as fractional or integer data in multiplication operations using the `NNA_CFG_MULT_MODE` from the `NNA_CFG` register

For more information on configuration of each of the NNA data sets see the following sub-sections.

7.2.0.0.1 Inputs

The input registers are loaded with 8- or 16-bit data from the input data; use the `NNA_CFG_INPUT_BITS` bit from the `NNA_CFG` register to select the data width. Input data that is smaller than the specified input data size (for example, 8 bits stored in the 16 bit input register), must be left aligned and the unused data bits are ignored (set to zero in calculations).

Input data is read fetched from memory starting from the address pointed to by the `NNA_INPUT_PTR` register and loaded to the input registers. The input pointer must point to data that is word aligned to the word size specified by `NNA_CFG_INPUT_BITS`. The number of inputs read for each layer (or number of synapses per output) is set using the `NNA_INPUT_CNT` register.

NOTE: The `NNA_INPUT_PTR` register is not updated while the accelerator runs. It needs to be updated when a new layer is executed. Similarly, the `NNA_INPUT_CNT` register is not updated when a new layer is executed. This counter only needs to be reconfigured if the next layer uses a different number of inputs.

Input data can be read using a circular buffer, which means that the NNA can cycle though inputs up to 1023 in length, processing the configured number of inputs. The circular buffer is disabled when `NNA_CIRC_BASE` and `NNA_CIRC_SIZE` are both set to zero.

If using a circular buffer `NNA_CIRC_BASE` is set to the start address of the circular buffer, `NNA_CIRC_SIZE` is set to the size of the circular buffer, and `NNA_INPUT_PTR` is set to the start of the input data in the circular buffer. The size is specified in words with the bit widths specified by `NNA_CFG_INPUT_BITS` bit from the `NNA_CFG` register. When the input pointer crosses the bounding of `NNA_CIRC_BASE + NNA_CIRC_SIZE * (INPUT_BITS + 1)`, then it wraps back to `NNA_CIRC_BASE`.

The input read order when `NNA_INPUT_CNT` is equal to `NNA_CIRC_SIZE` is shown in the "Input Read Order when `NNA_INPUT_CNT = NNA_CIRC_SIZE`" figure (Figure 6):

Ezairo 8300 Hardware Reference

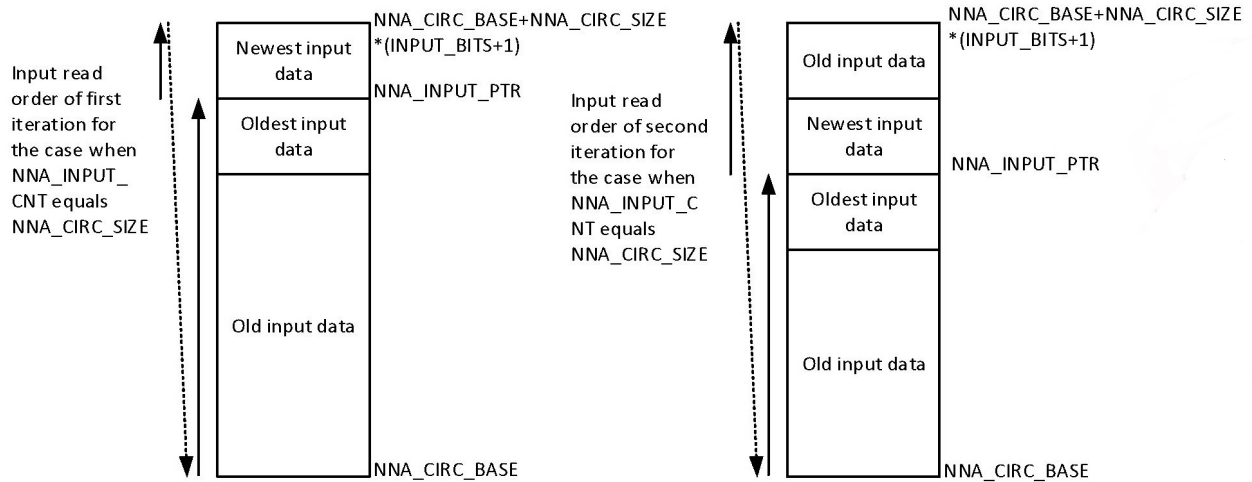


Figure 6. Input Read Order when $NNA_INPUT_CNT = NNA_CIRC_SIZE$

The input read order when $NNA_INPUT_CNT = NNA_CIRC_SIZE / 2$ is shown in the "Input Read Order when $NNA_INPUT_CNT = NNA_CIRC_SIZE$ Divided by 2" figure (Figure 7):

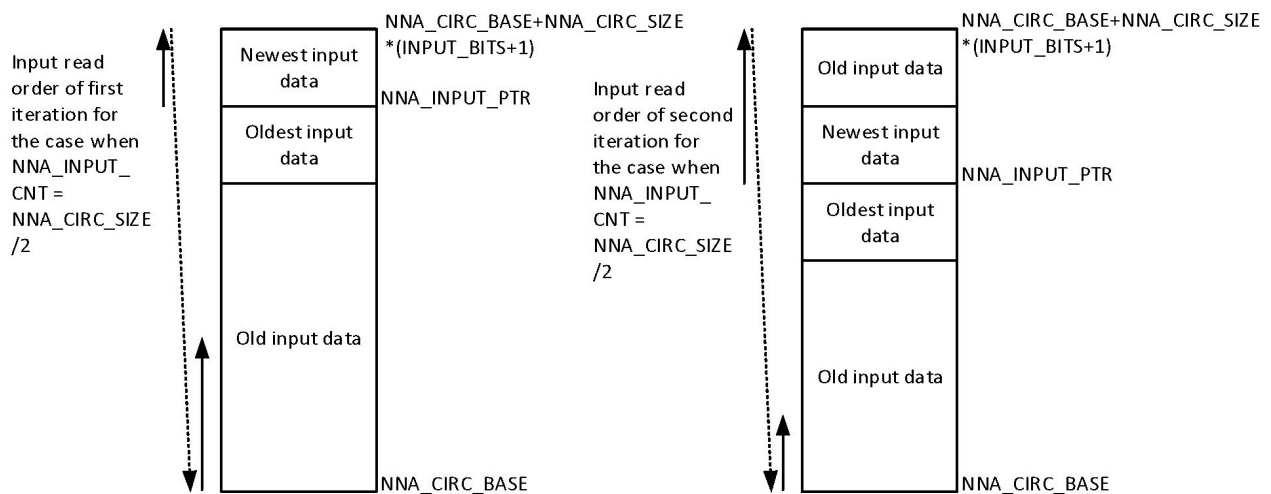


Figure 7. Input Read Order when $NNA_INPUT_CNT = NNA_CIRC_SIZE$ Divided by 2

7.2.0.0.2 Weights

The number of weights required for each layer is equal to the number of inputs times the number of outputs. This weight data is loaded from memory into the 16 weight registers to be multiplied by the input data. Data is loaded starting from memory indicated by the weight pointer, as stored in the NNA_WEIGHT_PTR register. The 16

Ezairo 8300 Hardware Reference

weight registers are loaded with weights from the specified weight data. The weight registers are 8 bits wide, and number of bits in each weight is configured using the `NNA_CFG_WEIGHT_BITS` bit-field from the `NNA_CFG` register. When configured for:

- 4-bit weights are expanded to 8 bits using the `NNA_WEIGHT_LUT_*_*` registers as a programmable look-up table.
- 8-bit weights are loaded directly to the weight registers
- 16-bit weights use pairs of weight registers, reducing the effective number of registers to eight.

NOTE: The weight pointer must be word aligned to the word size specified by `NNA_CFG_WEIGHT_BITS` for 8-, 16-bit weights. For 4-bit weights, the weight pointer must be byte aligned and an even number of inputs must be used for each layer.

The weight pointer is updated while the accelerator runs. The pointer does not need to be reconfigured if the weights for the next layer to be processed are located in memory directly after the weights of the current layer being processed.

NOTE: There are restrictions to the number of weight inputs when weight compression is enabled, depending on the type of weights being used. For more information about weight compression/pruning restrictions, see [Section 7.3.0.0.3 “Weight Compression” on page 265](#).

Weight data must be stored in a special interleaved format for all but the smallest neural network architectures, as described in [Section 7.2 “Attributes and Hardware Features” on page 258](#). The weight pointer, stored in the `NNA_WEIGHT_PTR` register, is updated while the accelerator runs. The pointer does not need to be reconfigured if the weights for the next layer to be processed are located in memory directly after the weights of the current layer being processed.

$$\frac{1}{1 + e^{-x}}$$

$$\frac{2}{1 + e^{-2x}} - 1$$

7.2.0.0.3 Biases

An optional bias can be applied to data before processing using the activation function to form outputs. This bias and the number of bits of precision for the bias (selected from 8-, 16-, or 32-bits of precision) are configurable using the `NNA_CFG_BIAS_MODE` bit-field from the `NNA_CFG` register.

The bias is right aligned and sign extended, and optionally shifted left, before it is added to the accumulation of the inputs multiplied with the weights for an output point. This is performed before the output shift value is applied, i.e.: if an output right shift is used, then an equivalent left shift of the bias will align it on the right in the output. Select the appropriate bias shift using the `NNA_CFG_BIAS_SHIFT` bit-field in the `NNA_CFG` register.

Data used for the bias is loaded from memory starting at the bias pointer stored in the `NNA_BIAS_POINTER` register. The bias pointer must point to data that is word aligned to the word size specified by `NNA_CFG_BIAS_MODE`. The bias pointer is updated while the accelerator runs. It does not need to be reconfigured if the biases for the next layer directly follow the biases for the current layer.

7.2.0.0.4 Outputs

Output precision can be set to 8, 16, or 32 bits, configurable using the `NNA_CFG_OUTPUT_BITS` field from the `NNA_CFG` register. Outputs may be referred to as nodes in other documentation. Output data is written to memory starting from the address pointed to by the `NNA_OUTPUT_PTR` register. The output pointer must point to data that is word aligned to the word size specified by `NNA_CFG_OUTPUT_BITS`. The number of outputs written for each layer is set using the `NNA_OUTPUT_CNT` register, which is decremented to zero while the accelerator runs.

NOTE: The `NNA_OUTPUT_CNT` register always ends with an output count of zero, and needs to be reconfigured for the next layer. The output pointer in `NNA_OUTPUT_PTR` is updated while the accelerator runs. The pointer does not need to be reconfigured if the outputs for the next layer directly follow the outputs for the current layer being processed.

There are five activation functions that can be applied to the biased accumulator data, selected using the `NNA_CFG_ACTIVATION_FUNCTION` bit field from the `NNA_CFG` register. They are:

- Linear (no activation)
- ReLU
 - To use this activation, use the `NNA_RELU` bit-setting and set the `NNA_CFG_RELU_CFG` bit-field from the `NNA_CFG` register to zero (`NNA_NORMAL_RELU`).
- Leaky ReLU (negative side has 50%, 25%, 12.5%, 6.25%, 3.125%, 1.5625% or 0.78125% slope)
 - To use this activation, use the `NNA_RELU` bit-setting and set the `NNA_CFG_RELU_CFG` bit-field from the `NNA_CFG` register to a non-zero value(`NNA_LEAKY_RELU_*_PERCENT`).
- Tanh

$$\frac{2}{1 + e^{-2x}} - 1$$

- Sigmoid

$$\frac{1}{1 + e^{-x}}$$

Tanh and sigmoid functions are applied with bit 23 representing 1 (unity) in the 43-bit sum stage, which is obtained by a full-scale input and full-scale weight when multiplied in fractional mode. For 16-bit precision outputs the max tanh error is 3 LSB, max sigmoid error is 2 LSB, both of which represent an error of less than 0.01%. For 8-bit precision outputs, both functions have an error of less than 1 LSB.

The output data can be shifted as defined using the `NNA_CFG_OUTPUT_SHIFT` bit-field from the `NNA_CFG` register. To maintain a nominal output range of ± 1 (for 24 bits) when using linear, normal ReLU, or leaky ReLU functions the output should be shifted:

- 8-bit outputs are shifted right by 16
- 16-bit outputs are shifted right by 8
- 32-bit outputs are shifted left by 8

Sigmoid or tanh activation functions require no shifting as the output is already aligned to 24 bits.

The output data can also be shifted to change the nominal range for the supplied weights:

$$\frac{1}{1 + e^{-x}}$$

$$\frac{2}{1 + e^{-2x}} - 1$$

- 8-bit weights have a nominal weight range of ± 1 . Output data can also be shifted left to increase the nominal weight range, with each shift increasing the weight range by a multiple of 2 (up to a maximum of ± 256).
- 16-bit weights have a nominal weight range of ± 256 . Output data can also be shifted right to decrease the nominal weight range, with each shift decreasing the weight range by a multiple of 2 (down to a minimum of ± 1).

7.3 WEIGHT INTERLEAVING AND COMPRESSION

After training a neural network with at least one dense (fully-connected) layer, the 2D weight matrix (number-of-inputs columns by number-of-outputs rows) has to be converted to two's complement fixed-point format and then organized into a 1D array of interleaved weights in a special order. The Ezairo 8300 SDK sample code includes weight interleaving scripts in both Python and MATLAB®.

7.3.0.0.1 Interleaving with 4-Bit or 8-Bit Weights

The 4-bit and 8-bit weight interleaving, where the number of rows corresponds to the number of outputs, can be visualized as a number of (up to) 16x16 tiles containing the following entries of the non-interleaved weight array:

- Tile 1
 - Weights 1 to 16 for 1st output (along 1st row)
 - Weights 1 to 16 for 2nd output (along 2nd row)
 - ...
 - Weights 1 to 16 for 16th output (along 16th row)
- Tile 2
 - Weights 17 to 32 for 1st output (along 1st row)
 - Weights 17 to 32 for 2nd output (along 2nd row)
 - ...
 - Weights 17 to 32 for 16th output (along 16th row)
- Tile 3
 - ...
- ...
- Last tile for outputs 1 to 16 (1st to 16th rows)
 - Remaining 1 to 16 weights for 1st output (along 1st row)
 - Remaining 1 to 16 weights for 2nd output (along 2nd row)
 - ...
 - Remaining 1 to 16 weights for 16th output (along 16th row)
- Repeat for remaining outputs (rows)

Example 1

NOTE: These examples use labels indicating the row/column position, not possible 8-bit data, in the depiction for clarity.

Ezairo 8300 Hardware Reference

A 20x20 non-interleaved row-major weight matrix is shown in the "20 x 20 8-Bit Weight Matrix" figure (Figure 8), with a tiling appropriate for 8-bit weights. The first 256 weights of the final interleaved 1-dimensional array required by the NNA starts with the weights in tile 1 (orange box) with labels written out in row-major order i.e. 101, 102, ... and so on. The next sequence of 64 weights comes from tile 2 (always row-major), i.e. labels 117, 118, 119, 120, 217, 218, ..., and so on until all tiles are unpacked into a single 400 element 1-dimensional array.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120
2	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220
3	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320
4	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420
5	501	502	503	504	505	506	507	508	509	510	511	512	513	514	515	516	517	518	519	520
6	601	602	603	604	605	606	607	608	609	610	611	612	613	614	615	616	617	618	619	620
7	701	702	703	704	705	706	707	708	709	710	711	712	713	714	715	716	717	718	719	720
8	801	802	803	804	805	806	807	808	809	810	811	812	813	814	815	816	817	818	819	820
9	901	902	903	904	905	906	907	908	909	910	911	912	913	914	915	916	917	918	919	920
10	1001	1002	1003	1004	1005	1006	1007	1008	1009	1010	1011	1012	1013	1014	1015	1016	1017	1018	1019	1020
11	1101	1102	1103	1104	1105	1106	1107	1108	1109	1110	1111	1112	1113	1114	1115	1116	1117	1118	1119	1120
12	1201	1202	1203	1204	1205	1206	1207	1208	1209	1210	1211	1212	1213	1214	1215	1216	1217	1218	1219	1220
13	1301	1302	1303	1304	1305	1306	1307	1308	1309	1310	1311	1312	1313	1314	1315	1316	1317	1318	1319	1320
14	1401	1402	1403	1404	1405	1406	1407	1408	1409	1410	1411	1412	1413	1414	1415	1416	1417	1418	1419	1420
15	1501	1502	1503	1504	1505	1506	1507	1508	1509	1510	1511	1512	1513	1514	1515	1516	1517	1518	1519	1520
16	1601	1602	1603	1604	1605	1606	1607	1608	1609	1610	1611	1612	1613	1614	1615	1616	1617	1618	1619	1620
17	1701	1702	1703	1704	1705	1706	1707	1708	1709	1710	1711	1712	1713	1714	1715	1716	1717	1718	1719	1720
18	1801	1802	1803	1804	1805	1806	1807	1808	1809	1810	1811	1812	1813	1814	1815	1816	1817	1818	1819	1820
19	1901	1902	1903	1904	1905	1906	1907	1908	1909	1910	1911	1912	1913	1914	1915	1916	1917	1918	1919	1920
20	2001	2002	2003	2004	2005	2006	2007	2008	2009	2010	2011	2012	2013	2014	2015	2016	2017	2018	2019	2020

Figure 8. 20 x 20 8-Bit Weight Matrix

7.3.0.0.2 Interleaving with 16-Bit Weights

The 16-bit weight interleaving follows the same pattern along rows but only up to 8 columns at a time, resulting in 8 columns x 16 row tiles.

- Tile 1
 - Weights 1 to 8 for 1st output (along 1st row)
 - Weights 1 to 8 for 2nd output (along 2nd row)
 - ...
 - Weights 1 to 8 for 16th output (along 16th row)
- Tile 2
 - Weights 9 to 16 for 1st output (along 1st row)
 - Weights 9 to 16 for 2nd output (along 2nd row)
 - ...
 - Weights 9 to 16 for 16th output (along 16th row)
- Tile 3
 - ...
- ...
- Last tile for outputs 1 to 16 (1st to 16th rows)
 - Remaining 1 to 8 weights for 1st output (along 1st row)
 - Remaining 1 to 8 weights for 2nd output (along 2nd row)
 - ...
 - Remaining 1 to 8 weights for 16th output (along 16th row)
- Repeat for remaining outputs (rows)

Ezairo 8300 Hardware Reference

Example 2

The same 20x20 non-interleaved row-major weight matrix is shown in the "20 x 20 16-Bit Weight Matrix" figure (Figure 9) but with a tiling appropriate for 16-bit weights. The first 128 weights of the final interleaved 1-dimensional array starts with the weights in tile 1 (orange box) read out in row-major order i.e. labeled 101, 102, ... and so on. The next sequence of 128 weights are from tile 2 (always row-major) i.e. labeled 109, 110, ..., and so on until all tiles are unpacked in row-major order and in the tile pattern below.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
1	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120
2	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220
3	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320
4	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420
5	501	502	503	504	505	506	507	508	509	510	511	512	513	514	515	516	517	518	519	520
6	601	602	603	604	605	606	607	608	609	610	611	612	613	614	615	616	617	618	619	620
7	701	702	703	704	705	706	707	708	709	710	711	712	713	714	715	716	717	718	719	720
8	801	802	803	804	805	806	807	808	809	810	811	812	813	814	815	816	817	818	819	820
9	901	902	903	904	905	906	907	908	909	910	911	912	913	914	915	916	917	918	919	920
10	1001	1002	1003	1004	1005	1006	1007	1008	1009	1010	1011	1012	1013	1014	1015	1016	1017	1018	1019	1020
11	1101	1102	1103	1104	1105	1106	1107	1108	1109	1110	1111	1112	1113	1114	1115	1116	1117	1118	1119	1120
12	1201	1202	1203	1204	1205	1206	1207	1208	1209	1210	1211	1212	1213	1214	1215	1216	1217	1218	1219	1220
13	1301	1302	1303	1304	1305	1306	1307	1308	1309	1310	1311	1312	1313	1314	1315	1316	1317	1318	1319	1320
14	1401	1402	1403	1404	1405	1406	1407	1408	1409	1410	1411	1412	1413	1414	1415	1416	1417	1418	1419	1420
15	1501	1502	1503	1504	1505	1506	1507	1508	1509	1510	1511	1512	1513	1514	1515	1516	1517	1518	1519	1520
16	1601	1602	1603	1604	1605	1606	1607	1608	1609	1610	1611	1612	1613	1614	1615	1616	1617	1618	1619	1620
17	1701	1702	1703	1704	1705	1706	1707	1708	1709	1710	1711	1712	1713	1714	1715	1716	1717	1718	1719	1720
18	1801	1802	1803	1804	1805	1806	1807	1808	1809	1810	1811	1812	1813	1814	1815	1816	1817	1818	1819	1820
19	1901	1902	1903	1904	1905	1906	1907	1908	1909	1910	1911	1912	1913	1914	1915	1916	1917	1918	1919	1920
20	2001	2002	2003	2004	2005	2006	2007	2008	2009	2010	2011	2012	2013	2014	2015	2016	2017	2018	2019	2020

Figure 9. 20 x 20 16-Bit Weight Matrix

7.3.0.0.3 Weight Compression

The NNA supports the following weight compression methods:

1. Expansion of 4-bit lookup table values to 8-bit weights using a weight look-up table (LUT), useful for models demanding a non-uniform distribution of precision (e.g. where higher precision is needed for small values).
 - a. The default values of the lookup table registers are linear (4-bit values left aligned with zero padding). The alignment on the left is used to minimize power consumption, as the logic of the multipliers is toggled less compared to when alignment is on the right with sign extension in the MSB part. An output shift right of 4 is required to align the results on the right side. (This might not be necessary when custom look-up table values are used.)
 - b. The LUT allows for a programmable expansion method. For more details on how to construct an appropriate lookup table, see the following section.
 - c. For example, to use logarithmic weights, where higher precision for smaller values can be achieved without sacrificing range, you would load these registers as follows:

```
// Use 4-bit logarithmic values
NNA->WEIGHT_LUT_0_3 = 0x08040200;
NNA->WEIGHT_LUT_4_7 = 0x7F402010;
NNA->WEIGHT_LUT_8_11 = 0xF0E0C080;
NNA->WEIGHT_LUT_12_15 = 0xFFFFFCF8;
```

- d. You can restore the default linear weights as follows:

Ezairo 8300 Hardware Reference

```
// Use 4-bit linear values
NNA->WEIGHT_LUT_0_3 = 0x30201000;
NNA->WEIGHT_LUT_4_7 = 0x70605040;
NNA->WEIGHT_LUT_8_11 = 0xB0A09080;
NNA->WEIGHT_LUT_12_15 = 0xF0E0D0C0;
```

IMPORTANT: When using 4-bit weights, the number of inputs must be a multiple of 2. If this limitation is not respected, the NNA ignores the start command.

2. Expansion of a compressed encoding of weights, both from taking out zeros (4-bit, 8-bit or 16-bit) and from small value compression (16-bit only). If a larger, fully-connected (dense) model has been trained with significant dropout, then the set of weights potentially contains a large number of zero-valued weights representing deleted neurons. Rather than take up memory with these null values, weight compression encoding can be enabled and a header byte is used for every eight weight bytes. In this header byte each bit indicates whether the corresponding weight byte is pruned (bits 0 to 7 correspond to weight bytes 0 to 7). A bit value of 0 indicates that the weight byte is pruned and contains all zeros, while a value of 1 indicates that the weight byte is present. An exception is 16-bit weights, for which a high byte is considered pruned when all its bits are identical to the MSB of the low byte. This is because if the high byte is a pure sign extension of the low byte, it contains no information and can be pruned. The low byte will still be considered pruned if it only contains zeroes, regardless of what the high byte is. After the header byte, the non-pruned bytes follow in the byte stream in little endian manner (i.e. from lowest byte number to highest byte number). Weight compression can be used in combinations of all weight sizes:
 - a. 4-bit weight compression is effective when many adjacent weights are pruned.

IMPORTANT: For this configuration, the number of inputs must be a multiple of 16. If this limitation is not respected, the NNA ignores the start command.

Example of 4-bit weight compression:

- Uncompressed weights: 0xE, 0x0, 0x5, 0x1, 0xC, 0x0, 0x0, 0x4, 0xF, 0x0, 0x0, 0x0, 0x0, 0x0, 0x6, 0x3
- Uncompressed bytes: 0x0E, 0x15, 0x0C, 0x40, 0x0F, 0x00, 0x00, 0x36
- Compression header: 0b10011111 -> 0x9F
- Compressed bytes: 0x9F, 0x0E, 0x15, 0x0C, 0x40, 0x0F, 0x36

- b. 8-bit weight compression is effective as long as there are enough pruned weights.

IMPORTANT: For this configuration, the number of inputs must be a multiple of 8. If this limitation is not respected, the NNA ignores the start command.

Example of 8-bit weight compression:

- Uncompressed weights/bytes: 0x00, 0x12, 0x34, 0x00, 0x56, 0x00, 0x00, 0x78
- Compression header: 0b10010110 -> 0x96
- Compressed bytes: 0x96, 0x12, 0x34, 0x56, 0x78

Ezairo 8300 Hardware Reference

- c. 16-bit weight compression is effective when there are many pruned weights.

IMPORTANT: For this configuration, the number of inputs must be a multiple of 4. If this limitation is not respected, the NNA ignores the start command.

Example of 16-bit weight compression:

- Uncompressed weights: 0x0123, 0xFFE3, 0x0056, 0x0089
- Uncompressed bytes: 0x23, 0x01, 0xE3, 0xFF, 0x56, 0x00, 0x89, 0x00
- Compression header: 0b10110111 -> 0xD7
- Compressed bytes: 0xD7, 0x23, 0x01, 0xE3, 0x56, 0x89, 0x00

7.3.0.0.3.1 Implementing a Weight Lookup Table

The lookup table has 16 possible values, which are expanded internally to 8-bit according to the LUT. Thus, when implementing the lookup table, the indices are sent for the lookup table to map to the appropriate weights; the weights themselves are not sent. While using the `uint8_t` datatype, each weight byte is packed with two nibbles in little-endian order.

An example of this can be shown with the default LUT. A set of weights, {0x12, 0xA3, 0xBD, 0xFF, 0x41, 0x8E}, mapped closest to the values of the LUT, becomes {0x10, 0xA0, 0xC0, 0x00, 0x40, 0x90}. To obtain these weights, these indices are sent in: {0xA1, 0x0C, 0x94}.

Additionally, a new interleaving order might need to be considered. For example, if 8-bit weights are used, the corresponding interleaving tile is 16x16 (i.e. 16 input columns x 16 output rows). With the weight bytes each representing two weights, the interleaving tiles are 8 columns x 16 rows in bytes (but still 16x16 in nibbles). Given that interleaving occurs before compressing the weights into the weight bytes, this has no effect on the interleaving order.

A suggested implementation flow is described below:

1. Construct a sensible LUT and evaluate the impact on inference accuracy.
 - a. Perform this step on a reference model e.g. on MATLAB or Python, where it is easy to manipulate the weights.
 - b. It is useful to try the linear LUT first and see the impact.
 - c. Use the LUT that is best for the layer and the overall model accuracy.
2. Assuming the sample application has already been developed and the original weights have been implemented correctly with the NNA, start testing with the original, uncompressed weights. Run the app as usual under a test condition: a known input with an expected output.
 - a. Note the output values.
 - b. Start with a very basic input: for example, if the input vector is a row vector size N, then a simple input to work with is one that is all zeroes except for the first index.
 - c. Disable the bias and use linear activation if it is feasible, to perform the simplest calculation possible during verification. Disabling bias and using linear activation reduces the weight calculation to a multiplication between the inputs and the weights. This simplifies verification and debugging when comparing with results from steps 3 and 4.
3. Round the weights to their LUT 8-bit equivalent. While still using the 8-bit weight setting on NNA, run the app with the same input and settings as before.
 - a. For example, with the linear LUT: 0x1F => 0x20, 0xF1 => 0xF0 (see the "Linear Lookup Table: Weights In vs. Weights Out" figure (Figure 10)).

Ezairo 8300 Hardware Reference

- b. Or, using the logarithmic LUT as an example: 0x79 => 0x7F, 0xD1 => 0xE0 (see the "[Logarithmic Lookup Table: Weights In vs. Weights Out](#)" figure (Figure 11)).
 - c. Note the output values.
4. Compress the weights with the LUT. Mapping the weights can easily occur outside of the C application. One suggestion is to create a script that processes the weights against the LUT to generate a new header file for the weights. Keep in mind that each weight byte contains two index nibbles and these are packed in a little-endian order. Run the app once again.
 - a. Note the output values and compare them to the output from steps 2 and 3.

NOTE: Steps 3 and 4 give identical results because both cases are implementing the same 16 8-bit weight values, either by explicitly listing these 8-bit weights (step 3) or implicitly through 4-bit indices (step 4). Meanwhile, the values compared between step 2 and 3 are expected to differ (the weights may have different precision), but for many applications the loss of precision will not significantly affect the final inference.

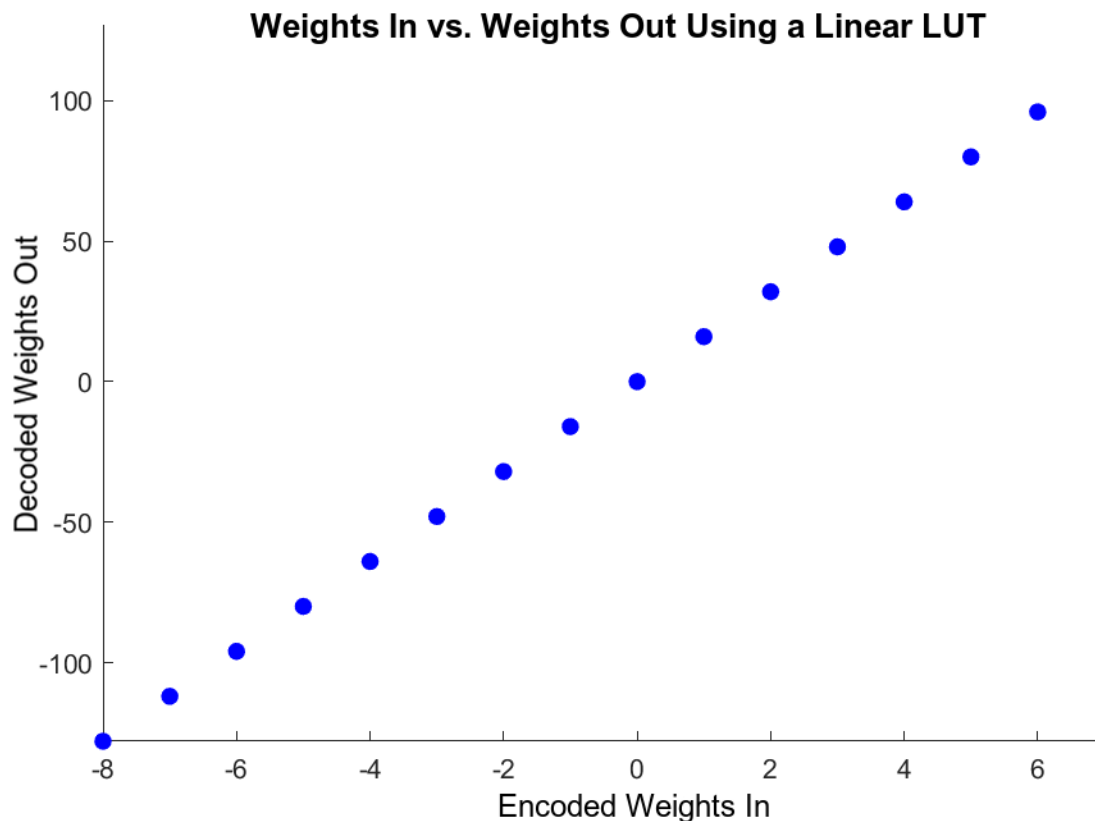


Figure 10. Linear Lookup Table: Weights In vs. Weights Out

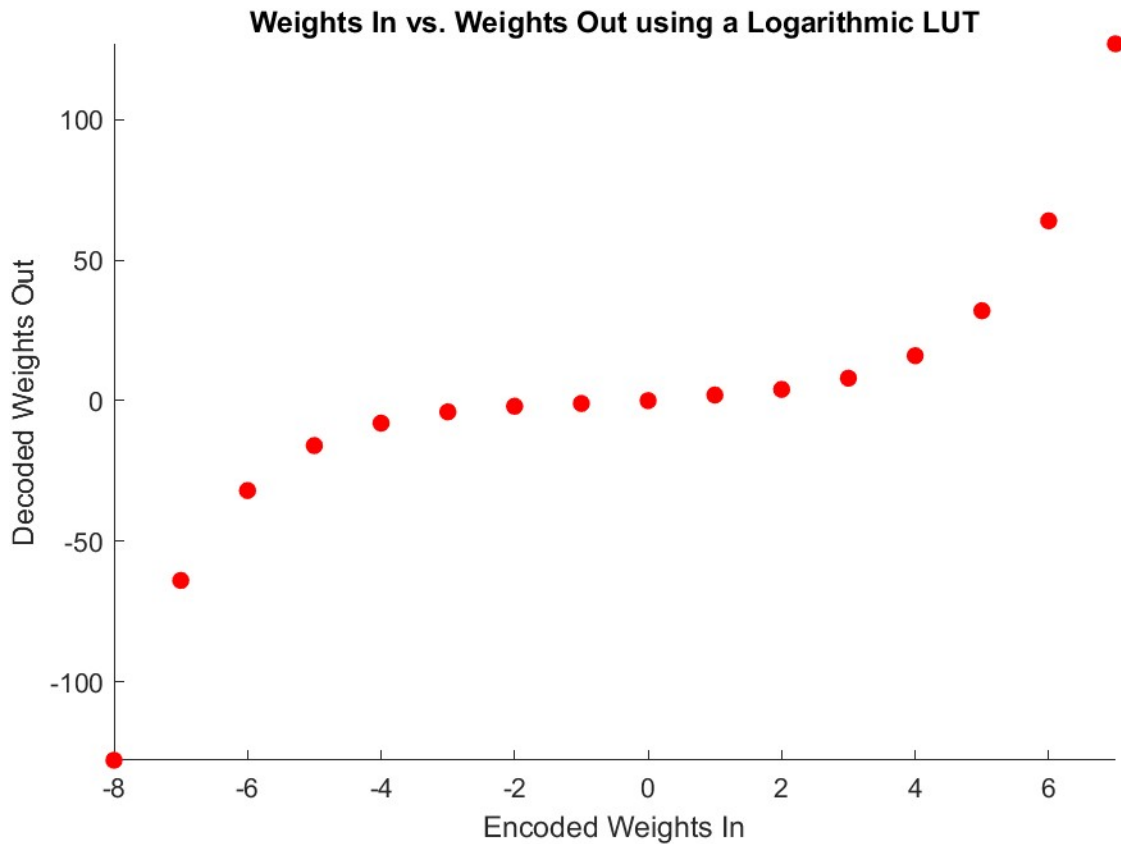


Figure 11. Logarithmic Lookup Table: Weights In vs. Weights Out

7.4 SUGGESTED DEVELOPMENT FLOW

Creating development flow with the NNA in mind can be a complex task. Consider the following suggestions:

1. Model the solution in floating-point, with these steps:
 - a. Use a deep learning framework (TensorFlow®, MATLAB Deep Learning Toolbox™, etc.) to design your solution and come up with a complete set of parameters required for, for instance, a forward inference of the model, or whatever deep learning task is considered runtime.
 - b. Make a considerable research effort to reduce the memory and computational footprint of the model.
 - i. Weight pruning can reduce the model's memory footprint, as it allows for the possibility of using weight compression methods (as described in [Section 7.3.0.0.3 “Weight Compression” on page 265](#)).
 - c. If at all possible, modify the model so that all (still floating-point) parameters have a $[-1, +1)$ range, i.e.: can be trivially converted to fractional fixed-point later.
 - d. Before any embedded programming is done, observe and analyze the data ranges at each layer of the model when computing a forward inference.
 - i. Collect statistics using the complete training set.
 - ii. Observe that while the inputs and weights/filters to layer N may be $[-1, +1)$ range, the intermediate outputs (between layer N and N+1) are not. It might or might not be necessary to reserve integer bits throughout the entire model to avoid complicated and computationally expensive re-normalization and/or shifting.

Ezairo 8300 Hardware Reference

- iii. Determine what amount of integer bit allocation is necessary. Some models respond well to clipping, some might not.
2. Implement the solution in non-embedded fixed-point C
 - C code is generally faster to write and debug on a computer than on the embedded target. Therefore, write a reference version of your implementation on a computer. It will not have NNA acceleration but is an excellent opportunity to test the fixed-point conversion, choice of radix points, and more. If using a simulator, cycle estimates and memory footprint can be obtained. Weight interleave testing can be optionally performed here.
3. Port to embedded - accelerate with NNA
 - Completing the previous two steps allows this step to be all about NNA integration, not application level debugging. Ideally, this phase focuses on integrating the data flow into, and out of, the application. Final cycle costs and memory footprints are determined here, as are changes ported back to the non-embedded fixed-point C, and even back to the deep learning framework model if appropriate.

7.5 NNA BEYOND FULLY-CONNECTED LAYERS

It is possible to leverage the NNA in convolutional layers. An example with eight 5x9 filters (convolving 45 pixels at a time) is as follows:

1. Interleave the ($8 \times 5 \times 9 = 360$) filter coefficients as an 8×45 weight matrix.
2. Use the eight filter biases as the NNA bias vector.
3. Flatten the 45 inputs to be the input to the NNA.
4. After calling NNA:
 - a. The 1st output node is the convolution result of the 1st filter.
 - b. The 2nd output node is the convolution result of the 2nd filter.
 - c. ...
 - d. The Nth output node is the convolution result of the Nth filter.

7.6 NNA CONTROL

The NNA is controlled by the `NNA_CTRL` register. NNA statuses can also be read from this register. This register provides the following functionality:

- Setting the `NNA_CTRL_RESET` bit to 1 resets the NNA. It is recommended that the NNA be reset before configuring and starting it, to ensure it is not still in a busy state (for example, when restarting the application).
- Setting the `NNA_CTRL_START` bit starts the NNA, allowing it to begin making necessary computations according to its configurations.
- The `NNA_CTRL_BUSY` bit indicates status, and can be read to determine whether the NNA is currently busy processing or it is idle.
- The `NNA_CTRL` register indicates errors that may occur while the NNA is running:
 - `NNA_CTRL_DSP_ACCESS_ERROR`: this flag is set when an illegal LPDSP32 register write access occurs.
 - `NNA_CTRL_PBUS_ACCESS_ERROR`: This flag is set when an illegal non-LPDSP32 register write access occurs.
 - `NNA_CTRL_ACCESS_ERROR` indicates if either error of those has occurred.
- If either access error occurs, the user can reset the error flag by setting the corresponding reset bit.
 - Set the `RESET_DSP_ACCESS_ERROR` bit if an LPDSP32 read access error has occurred.
 - Set the `RESET_PBUS_ACCESS_ERROR` bit if a peripheral bus read access error has occurred.

Ezairo 8300 Hardware Reference

- NNA_CTRL can also inform the user of saturation that may have occurred during processing. There are two saturation flags:
 - NNA_CTRL_SATURATION_FLAG indicates if the NNA has saturated at all.
 - NNA_CTRL_SATURATION2_FLAG indicates that the NNA has saturated with a value at least twice the output range. If this flag indicates an error, then the other saturation flag will also have a value of 1.
- If saturation occurs, the user can reset both flags by setting the RESET_SATURATION_FLAGS bit.

7.7 INTERFACING TO THE REST OF THE SYSTEM

In addition to being accessible to the CFX DSP and Arm Cortex-M3 processors, the Neural Network Accelerator is interfaced with the LPDSP32 in the following ways:

1. The memory mapped registers are mapped on the DMIO bus at addresses 0xC00100 to 0xC001FF. The 8 LSBs of the register mapping are identical to the mapping on the system bus of the Arm Cortex-M3 processor.
2. The NNA interrupt is mapped on the LPDSP32 (see section [Section 8.3 “Interrupt Handling” on page 281](#)).
3. NNA uses the logical memories that are mapped in the LPDSP32-DMA memory space. The same byte addressing is used as in the LPDSP32.
4. In cases of memory access conflict between the LPDSP32 and the NNA, the NNA is stalled.

7.8 NEURAL-NETWORK ACCELERATOR CONTROL AND CONFIGURATION REGISTERS

Register Name	Register Description	Address
NNA_CFG	NNA Configuration Register	P:0x17C0 0x40001F00 DMIO: 0x00C00100
NNA_CTRL	NNA Control Register	P:0x17C1 0x40001F04 DMIO: 0x00C00104
NNA_INPUT_CNT	NNA Input Count	P:0x17C2 0x40001F08 DMIO: 0x00C00108
NNA_OUTPUT_CNT	NNA Output Count	P:0x17C3 0x40001F0C DMIO: 0x00C0010C
NNA_CIRC_BASE	NNA Circular Buffer Base Address	P:0x17C4 0x40001F10 DMIO: 0x00C00110

onsemi Confidential
Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
NNA_CIRC_SIZE	NNA Circular Buffer Size (in input size elements)	P:0x17C5 0x40001F14 DMIO: 0x00C00114
NNA_INPUT_PTR	NNA Input Pointer	P:0x17C6 0x40001F18 DMIO: 0x00C00118
NNA_WEIGHT_PTR	NNA Weight Pointer	P:0x17C7 0x40001F1C DMIO: 0x00C0011C
NNA_BIAS_PTR	NNA Bias Pointer	P:0x17C8 0x40001F20 DMIO: 0x00C00120
NNA_OUTPUT_PTR	NNA Output Pointer	P:0x17C9 0x40001F24 DMIO: 0x00C00124
NNA_WEIGHT_LUT_0_3	NNA Weight Lookup Table Values 0 to 3	P:0x17CA 0x40001F28 DMIO: 0x00C00128
NNA_WEIGHT_LUT_4_7	NNA Weight Lookup Table Values 4 to 7	P:0x17CB 0x40001F2C DMIO: 0x00C0012C
NNA_WEIGHT_LUT_8_11	NNA Weight Lookup Table Values 8 to 11	P:0x17CC 0x40001F30 DMIO: 0x00C00130
NNA_WEIGHT_LUT_12_15	NNA Weight Lookup Table Values 12 to 15	P:0x17CD 0x40001F34 DMIO: 0x00C00134

Ezairo 8300 Hardware Reference

7.8.0.1 NNA_CFG

Bit Field	Read/Write	Field Name	Description
25:23	RW	RELU_CFG	ReLU configuration
22:21	RW	ACTIVATION_FUNCTION	Activation function
20	RW	OUTPUT_ROUNDING	Rounding
19:15	RW	OUTPUT_SHIFT	Output shift
14:13	RW	OUTPUT_BITS	Output word size
12:8	RW	BIAS_SHIFT	Bias shift
7:6	RW	BIAS_MODE	Bias mode
4	RW	WEIGHT_COMPRESSION	Weight compression
3:2	RW	WEIGHT_BITS	Weight word size
1	RW	INPUT_BITS	Input word size
0	RW	MULT_MODE	Fixed point or integer multiplier mode

Field Name	Value Symbol	Value Description	Hex Value
RELU_CFG	NNA_NORMAL_RELU	ReLU is normal (not leaky)	0x0*
	NNA_LEAKY_RELU_50_PERCENT	ReLU is leaky, negative side has 50% amplitude	0x1
	NNA_LEAKY_RELU_25_PERCENT	ReLU is leaky, negative side has 25% amplitude	0x2
	NNA_LEAKY_RELU_12P5_PERCENT	ReLU is leaky, negative side has 12.5% amplitude	0x3
	NNA_LEAKY_RELU_6P25_PERCENT	ReLU is leaky, negative side has 6.25% amplitude	0x4
	NNA_LEAKY_RELU_3P125_PERCENT	ReLU is leaky, negative side has 3.125% amplitude	0x5
	NNA_LEAKY_RELU_1P5625_PERCENT	ReLU is leaky, negative side has 1.5625% amplitude	0x6
	NNA_LEAKY_RELU_0P78125_PERCENT	ReLU is leaky, negative side has 0.78125% amplitude	0x7
ACTIVATION_FUNCTION	NNA_LINEAR	Linear activation function is used	0x0*
	NNA_RELU	ReLU activation function is used	0x1
	NNA_SIGMOID	Sigmoid activation function is used	0x2
	NNA_TANH	Tanh activation function is used	0x3
OUTPUT_ROUNDING	NNA_OUTPUT_NO_ROUNDING	No rounding is used in output	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		generation	
	NNA_OUTPUT_ROUNDING	Rounding is used in output generation	0x1
OUTPUT_SHIFT	NNA_OUTPUT_NO_SHIFT	Outputs are not shifted	0x0*
	NNA_OUTPUT_SHIFT_LEFT_1	Outputs are shifted left by 1 bit	0x1
	NNA_OUTPUT_SHIFT_LEFT_2	Outputs are shifted left by 2 bits	0x2
	NNA_OUTPUT_SHIFT_LEFT_3	Outputs are shifted left by 3 bits	0x3
	NNA_OUTPUT_SHIFT_LEFT_4	Outputs are shifted left by 4 bits	0x4
	NNA_OUTPUT_SHIFT_LEFT_5	Outputs are shifted left by 5 bits	0x5
	NNA_OUTPUT_SHIFT_LEFT_6	Outputs are shifted left by 6 bits	0x6
	NNA_OUTPUT_SHIFT_LEFT_7	Outputs are shifted left by 7 bits	0x7
	NNA_OUTPUT_SHIFT_LEFT_8	Outputs are shifted left by 8 bits	0x8
	NNA_OUTPUT_SHIFT_RIGHT_16	Outputs are shifted right by 16 bits	0x10
	NNA_OUTPUT_SHIFT_RIGHT_15	Outputs are shifted right by 15 bits	0x11
	NNA_OUTPUT_SHIFT_RIGHT_14	Outputs are shifted right by 14 bits	0x12
	NNA_OUTPUT_SHIFT_RIGHT_13	Outputs are shifted right by 13 bits	0x13
	NNA_OUTPUT_SHIFT_RIGHT_12	Outputs are shifted right by 12 bits	0x14
	NNA_OUTPUT_SHIFT_RIGHT_11	Outputs are shifted right by 11 bits	0x15
	NNA_OUTPUT_SHIFT_RIGHT_10	Outputs are shifted right by 10 bits	0x16
	NNA_OUTPUT_SHIFT_RIGHT_9	Outputs are shifted right by 9 bits	0x17
	NNA_OUTPUT_SHIFT_RIGHT_8	Outputs are shifted right by 8 bits	0x18
	NNA_OUTPUT_SHIFT_RIGHT_7	Outputs are shifted right by 7 bits	0x19
	NNA_OUTPUT_SHIFT_RIGHT_6	Outputs are shifted right by 6 bits	0x1A
	NNA_OUTPUT_SHIFT_RIGHT_5	Outputs are shifted right by 5 bits	0x1B
	NNA_OUTPUT_SHIFT_RIGHT_4	Outputs are shifted right by 4 bits	0x1C
	NNA_OUTPUT_SHIFT_RIGHT_3	Outputs are shifted right by 3 bits	0x1D
	NNA_OUTPUT_SHIFT_RIGHT_2	Outputs are shifted right by 2 bits	0x1E
	NNA_OUTPUT_SHIFT_RIGHT_1	Outputs are shifted right by 1 bits	0x1F
OUTPUT_BITS	NNA_OUTPUT_8_BITS	Outputs are 8-bit words	0x0*
	NNA_OUTPUT_16_BITS	Outputs are 16-bit words	0x1
	NNA_OUTPUT_32_BITS	Outputs are 32-bit words	0x2
BIAS_SHIFT	NNA_BIAS_NO_SHIFT	Bias is not shifted	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	NNA_BIAS_SHIFT_LEFT_1	Bias is shifted left by 1 bit	0x1
	NNA_BIAS_SHIFT_LEFT_2	Bias is shifted left by 2 bits	0x2
	NNA_BIAS_SHIFT_LEFT_3	Bias is shifted left by 3 bits	0x3
	NNA_BIAS_SHIFT_LEFT_4	Bias is shifted left by 4 bits	0x4
	NNA_BIAS_SHIFT_LEFT_5	Bias is shifted left by 5 bits	0x5
	NNA_BIAS_SHIFT_LEFT_6	Bias is shifted left by 6 bits	0x6
	NNA_BIAS_SHIFT_LEFT_7	Bias is shifted left by 7 bits	0x7
	NNA_BIAS_SHIFT_LEFT_8	Bias is shifted left by 8 bits	0x8
	NNA_BIAS_SHIFT_LEFT_9	Bias is shifted left by 9 bits	0x9
	NNA_BIAS_SHIFT_LEFT_10	Bias is shifted left by 10 bits	0xA
	NNA_BIAS_SHIFT_LEFT_11	Bias is shifted left by 11 bits	0xB
	NNA_BIAS_SHIFT_LEFT_12	Bias is shifted left by 12 bits	0xC
	NNA_BIAS_SHIFT_LEFT_13	Bias is shifted left by 13 bits	0xD
	NNA_BIAS_SHIFT_LEFT_14	Bias is shifted left by 14 bits	0xE
	NNA_BIAS_SHIFT_LEFT_15	Bias is shifted left by 15 bits	0xF
	NNA_BIAS_SHIFT_LEFT_16	Bias is shifted left by 16 bits	0x10
	NNA_BIAS_SHIFT_LEFT_17	Bias is shifted left by 17 bits	0x11
	NNA_BIAS_SHIFT_LEFT_18	Bias is shifted left by 18 bits	0x12
	NNA_BIAS_SHIFT_LEFT_19	Bias is shifted left by 19 bits	0x13
	NNA_BIAS_SHIFT_LEFT_20	Bias is shifted left by 20 bits	0x14
	NNA_BIAS_SHIFT_LEFT_21	Bias is shifted left by 21 bits	0x15
	NNA_BIAS_SHIFT_LEFT_22	Bias is shifted left by 22 bits	0x16
	NNA_BIAS_SHIFT_LEFT_23	Bias is shifted left by 23 bits	0x17
	NNA_BIAS_SHIFT_LEFT_24	Bias is shifted left by 24 bits	0x18
	NNA_BIAS_SHIFT_LEFT_25	Bias is shifted left by 25 bits	0x19
	NNA_BIAS_SHIFT_LEFT_26	Bias is shifted left by 26 bits	0x1A
	NNA_BIAS_SHIFT_LEFT_27	Bias is shifted left by 27 bits	0x1B
	NNA_BIAS_SHIFT_LEFT_28	Bias is shifted left by 28 bits	0x1C
	NNA_BIAS_SHIFT_LEFT_29	Bias is shifted left by 29 bits	0x1D
	NNA_BIAS_SHIFT_LEFT_30	Bias is shifted left by 30 bits	0x1E
	NNA_BIAS_SHIFT_LEFT_31	Bias is shifted left by 31 bits	0x1F

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
BIAS_MODE	NNA_BIAS_DISABLED	No bias is used	0x0*
	NNA_BIAS_8_BITS	Biases are 8-bit words	0x1
	NNA_BIAS_16_BITS	Biases are 16-bit words	0x2
	NNA_BIAS_32_BITS	Biases are 32-bit words	0x3
WEIGHT_COMPRESSION	NNA_WEIGHT_COMPRESSION_DISABLED	Weight compression disabled	0x0*
	NNA_WEIGHT_COMPRESSION_ENABLED	Weight compression enabled. Bytes with value 0 are encoded as a single bit '0'. All other bytes are encoded as 9 bits with a bit '1' added on the LSB side. An exception is 16-bit weights for which the high byte is encoded as a single bit '0' when all its bits are identical to the MSB of the low byte.	0x1
WEIGHT_BITS	NNA_WEIGHT_4_BITS	Weights are 4-bit words (using lookup table)	0x0*
	NNA_WEIGHT_8_BITS	Weights are 8-bit words (linear)	0x2
	NNA_WEIGHT_16_BITS	Weights are 16-bit words (linear)	0x3
INPUT_BITS	NNA_INPUT_8_BITS	Inputs are 8-bit words	0x0*
	NNA_INPUT_16_BITS	Inputs are 16-bit words	0x1
MULT_MODE	NNA_INTEGER_MULT	Multiplications are performed in integer mode	0x0*
	NNA_FRACTIONAL_MULT	Multiplications are performed in fractional mode	0x1

7.8.0.2 NNA_CTRL

Bit Field	Read/Write	Field Name	Description
13	R	ACCESS_ERROR	Peripheral bus or DSP access error
12	R	DSP_ACCESS_ERROR	DSP access error
11	R	PBUS_ACCESS_ERROR	Peripheral bus access error
10	R	SATURATION2_FLAG	NNA saturation 2 flag
9	R	SATURATION_FLAG	NNA saturation flag
8	R	BUSY	The NNA processing state
4	W	RESET_DSP_ACCESS_ERROR	Write a 1 to reset DSP access error
3	W	RESET_PBUS_ACCESS_ERROR	Write a 1 to reset peripheral bus access error

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
2	W	RESET_SATURATION_FLAGS	Write a 1 to reset saturation flags
1	W	RESET	Write a 1 to reset NNA
0	W	START	Write a 1 to start the NNA

Field Name	Value Symbol	Value Description	Hex Value
ACCESS_ERROR	NNA_NO_ACCESS_ERROR	No peripheral bus or DSP access error occurred	0x0*
	NNA_ACCESS_ERROR	Peripheral bus or DSP read access error occurred	0x1
DSP_ACCESS_ERROR	NNA_DSP_NO_ACCESS_ERROR	No DSP access error occurred	0x0*
	NNA_DSP_ACCESS_ERROR	DSP read access error occurred	0x1
PBUS_ACCESS_ERROR	NNA_PBUS_NO_ACCESS_ERROR	No peripheral bus access error occurred	0x0*
	NNA_PBUS_ACCESS_ERROR	Peripheral bus read access error occurred	0x1
SATURATION2_FLAG	NNA_SATURATION2_NOT_OCCURED	NNA has not saturated with a value at least twice the output range	0x0*
	NNA_SATURATION2_OCCURED	NNA has saturated with a value at least twice the output range	0x1
SATURATION_FLAG	NNA_SATURATION_NOT_OCCURED	NNA has not saturated	0x0*
	NNA_SATURATION_OCCURED	NNA has saturated	0x1
BUSY	NNA_IDLE	The NNA is idle	0x0*
	NNA_BUSY	The NNA is busy processing	0x1
RESET_DSP_ACCESS_ERROR	NNA_RESET_DSP_ACCESS_ERROR	Reset DSP access error	0x1
RESET_PBUS_ACCESS_ERROR	NNA_RESET_PBUS_ACCESS_ERROR	Reset peripheral bus access error	0x1
RESET_SATURATION_FLAGS	NNA_RESET_SATURATION_FLAGS	Reset saturation flags	0x1
RESET	NNA_RESET	Stop and reset the NNA	0x1
START	NNA_START	Start the NNA	0x1

7.8.0.3 NNA_INPUT_CNT

Bit Field	Read/Write	Field Name	Description
9:0	RW	INPUT_CNT	NNA input count (number of elements in input vector)

7.8.0.4 NNA_OUTPUT_CNT

Bit Field	Read/Write	Field Name	Description
9:0	RW	OUTPUT_CNT	NNA output count (number of output points to be calculated)

7.8.0.5 NNA_CIRC_BASE

Bit Field	Read/Write	Field Name	Description
18:0	RW	CIRC_BASE	NNA circular buffer base address (byte address in LPDSP32 DM-A)

7.8.0.6 NNA_CIRC_SIZE

Bit Field	Read/Write	Field Name	Description
9:0	RW	CIRC_SIZE	NNA circular buffer size (in input size elements)

7.8.0.7 NNA_INPUT_PTR

Bit Field	Read/Write	Field Name	Description
18:0	RW	INPUT_PTR	NNA input pointer (byte address in LPDSP32 DM-A)

7.8.0.8 NNA_WEIGHT_PTR

Bit Field	Read/Write	Field Name	Description
18:0	RW	WEIGHT_PTR	NNA weight pointer (byte address in LPDSP32 DM-A)

7.8.0.9 NNA_BIAS_PTR

Bit Field	Read/Write	Field Name	Description
18:0	RW	BIAS_PTR	NNA bias pointer (byte address in LPDSP32 DM-A)

7.8.0.10 NNA_OUTPUT_PTR

Bit Field	Read/Write	Field Name	Description
18:0	RW	OUTPUT_PTR	NNA output pointer (byte address in LPDSP32 DM-A)

7.8.0.11 NNA_WEIGHT_LUT_0_3

Bit Field	Read/Write	Field Name	Description
31:24	RW	LUT_ENTRY_3	Weight Lookup Table Entry for value 3
23:16	RW	LUT_ENTRY_2	Weight Lookup Table Entry for value 2
15:8	RW	LUT_ENTRY_1	Weight Lookup Table Entry for value 1
7:0	RW	LUT_ENTRY_0	Weight Lookup Table Entry for value 0

Ezairo 8300 Hardware Reference

7.8.0.12 NNA_WEIGHT_LUT_4_7

Bit Field	Read/Write	Field Name	Description
31:24	RW	LUT_ENTRY_7	Weight Lookup Table Entry for value 7
23:16	RW	LUT_ENTRY_6	Weight Lookup Table Entry for value 6
15:8	RW	LUT_ENTRY_5	Weight Lookup Table Entry for value 5
7:0	RW	LUT_ENTRY_4	Weight Lookup Table Entry for value 4

7.8.0.13 NNA_WEIGHT_LUT_8_11

Bit Field	Read/Write	Field Name	Description
31:24	RW	LUT_ENTRY_11	Weight Lookup Table Entry for value 11
23:16	RW	LUT_ENTRY_10	Weight Lookup Table Entry for value 10
15:8	RW	LUT_ENTRY_9	Weight Lookup Table Entry for value 9
7:0	RW	LUT_ENTRY_8	Weight Lookup Table Entry for value 8

7.8.0.14 NNA_WEIGHT_LUT_12_15

Bit Field	Read/Write	Field Name	Description
31:24	RW	LUT_ENTRY_15	Weight Lookup Table Entry for value 15
23:16	RW	LUT_ENTRY_14	Weight Lookup Table Entry for value 14
15:8	RW	LUT_ENTRY_13	Weight Lookup Table Entry for value 13
7:0	RW	LUT_ENTRY_12	Weight Lookup Table Entry for value 12

CHAPTER 8

LPDSP32 Processor

The LPDSP32 DSP is a low power, programmable, pipelined DSP that uses a dual-Harvard, dual-MAC architecture to efficiently process 32-bit signal data. This processor is included in the design to efficiently support digital signal processing tasks, such as audio codecs that might be required for wireless audio communication tasks, Deep Learning and other Artificial Intelligence (AI) functions, and other advanced developments requiring the additional processing power that this core provides.

IMPORTANT: To minimize power consumption when the LPDSP32 DSP is waiting for an interrupt, the core should be held in a wait state using the powerdown instruction. In C code, an application can use the `SYS_WAIT_FOR_INTERRUPT` macro (which wraps the `core_halt()` intrinsic function) to place the core into this wait state.

8.1 SYSTEM INTEGRATION

The LPDSP32 does not have a ROM. Instead, it relies on the CFX DSP or the Arm Cortex-M3 processor to initialize its memories and peripherals. Once initialized, the CFX DSP and/or the Arm Cortex-M3 processor control the LPDSP32 DSP's execution state.

The `SYSCTRL_DSP_CTRL` register controls the LPDSP32 execution. Bit-fields in this register include:

- The `SYSCTRL_DSP_CTRL_DSP_RESET` bit, which resets the LPDSP32 core, its interrupt handler, and the LPDSP32 loop cache when written with `DSP_RESET`.
- The `SYSCTRL_DSP_CTRL_DSP_PAUSE` and `SYSCTRL_DSP_CTRL_DSP_RESUME` bits, which respectively pause and resume the LPDSP32 processing.
- The `SYSCTRL_DSP_CTRL_DSP_RUNNING` bit, which indicates the current state (running or paused) of the LPDSP32. When the LPDSP32 core is paused, the clock provided to the core is stopped. This prevents the execution of any functions, and all responses to interrupts generated by the DMA, the CFX DSP, or the Arm Cortex-M3 processor.

When reset, the LPDSP32 begins execution from the reset vector location. This vector is placed at the base of the LPDSP32 vector table, as described in the "LPDSP32 Program Memory Mapping" table (Table 36).

8.2 MEMORY SUPPORT

The LPDSP32 is supported by data memory and program memory. The LPDSP32 has a 40-bit program memory bus, which is address translated to two 32-bit buses to access its program memory that is located in 32-bit RAM. Unused memory instances can be reassigned in the firmware, and in the linker configuration scripts used by the CFX DSP and the Arm Cortex-M3 processor. Similarly, if additional memory is needed for the LPDSP32 DSP's program or data, memory instances that are nominally assigned to the Arm Cortex-M3 processor can be reassigned to the LPDSP32 DSP. See [Section 11.6 "LPDSP32 DSP Memory Mapping" on page 367](#) for more information regarding the program and data memory.

When the LPDSP32 core is paused, all incomplete accesses it has made to any memory are put into a non-blocking wait state, and these accesses are completed once processing resumes on the LPDSP32 core. While the LPDSP32 core is paused, the CFX DSP or Arm Cortex-M3 core can access shared memory resources at any address without the LPDSP32 introducing additional delays.

Ezairo 8300 Hardware Reference

8.3 INTERRUPT HANDLING

To coordinate execution of data-processing functions on the LPDSP32 processor with data that is available to be processed, the LPDSP32 is supported by three sets of interrupts that can trigger execution on the LPDSP32:

1. Each DMA channel can issue one interrupt. To clear all pending DMA triggered interrupts to the LPDSP32, set the `SYSCTRL_DSP_CTRL_DSP_DMA_INT_RESET` bit from the `SYSCTRL_DSP_CTRL` register.
2. The CFX DSP or Arm Cortex-M3 processor can issue one of six application-defined commands by setting the appropriate bit in the `SYSCTRL_DSP_CMD` register. To clear all pending core triggered interrupts to the LPDSP32, set the `SYSCTRL_DSP_CTRL_DSP_CM3_INT_RESET` bit from the `SYSCTRL_DSP_CTRL` register.
3. The neural-network accelerator integrated with the LPDSP32 can issue one interrupt to the LPDSP32. This interrupt is also cleared when writing the `SYSCTRL_DSP_CTRL_DSP_CM3_INT_RESET` bit from the `SYSCTRL_DSP_CTRL` register.

The LPDSP32 vector table is provided in the "LPDSP32 Vectors" table (Table 16). This table contains both the reset vector and interrupt vectors for all of the interrupt handlers. The LPDSP32 does not support pre-emption of interrupt handler execution. If more than one interrupt is pending when execution completes on a previous interrupt handler, the handler for the lowest numbered pending interrupt is selected for execution.

Table 16. LPDSP32 Vectors

Interrupt Number	Description
-	Reset vector
0	DMA channel 0 interrupt
1	DMA channel 1 interrupt
2	DMA channel 2 interrupt
3	DMA channel 3 interrupt
4	DMA channel 4 interrupt
5	DMA channel 5 interrupt
6	DMA channel 6 interrupt
7	DMA channel 7 interrupt
8	Arm Cortex-M3 processor and CFX interrupt 0
9	Arm Cortex-M3 processor and CFX interrupt 1
10	Arm Cortex-M3 processor and CFX interrupt 2
11	Arm Cortex-M3 processor and CFX interrupt 3
12	Arm Cortex-M3 processor and CFX interrupt 4
13	Arm Cortex-M3 processor and CFX interrupt 5
14	NNA interrupt

8.4 OUTGOING INTERRUPTS

When any function completes, or at any other logical intermediate point, the LPDSP32 processor can send the Arm Cortex-M3 core and the CFX processor an interrupt through one of eight outgoing LPDSP32 interrupts, which are connected to the `DSP_*_IRQn` and `D_VECT_DSP_*` interrupts of each processor respectively. These interrupts are

Ezairo 8300 Hardware Reference

intended to notify Arm Cortex-M3 core or the CFX when the data processing on the LPDSP32 is complete, or to signal when the LPDSP32 is ready to receive requests from the Arm Cortex-M3 core or the CFX for additional processing. To trigger these interrupts on the Arm Cortex-M3 core or the CFX DSP, set the appropriate CM3_CFX_CMD_* bit in the LPDSP32's CM3_CFX_CMD register.

The interrupts can also be used as triggers on a DMA channel, causing it to copy processed data output to elsewhere in the system. To trigger any of these interrupts, set the corresponding bit (0 to 7) of the CM3_CFX_CMD register to 1.

For more information, see the *Ezairo 8300 Firmware Reference Manual*, the [Using the ASIP Programmer Toolchain for CFX and LPDSP32](#) section of the *Integrated Development Environment User's Guide for Ezairo 8300*, and the LPDSP32 documentation included with your Ezairo 8300 installation, located by default in *C:/Program Files (x86)/ON Semiconductor\Ezairo 8300 SDK\documentation\lpdsp32_3rd_party_docs*.

8.5 CONTROL AND CONFIGURATION REGISTERS

Register Name	Register Description	Address
SYSCTRL_DSP_CTRL	DSP Control	P:0x1001 0x40000004
SYSCTRL_DSP_DEBUG_CFG	DSP Debug Port Configuration	P:0x1002 0x40000008
IMSK	Interrupt Mask Register	N/A
IRQ_STAT	Interrupt Request Status Register	N/A
IRQ_CLR	Interrupt Clear Register	DMIO: 0xC00000
CM3_CFX_CMD	Arm Cortex-M3 Core and CFX Command Register	DMIO: 0xC00004

8.5.1 SYSCTRL_DSP_CTRL

Bit Field	Read/Write	Field Name	Description
8	R	DSP_RUNNING	DSP running status
4	W	DSP_CM3_CFX_NNA_INT_RESET	Write a 1 to reset pending CM3_CFX and NNA interrupts in the DSP interrupt controller
3	W	DSP_DMA_INT_RESET	Write a 1 to reset pending DMA interrupts in the DSP interrupt controller
2	W	DSP_RESET	Write a 1 to reset DSP
1	W	DSP_PAUSE	Write a 1 to pause DSP
0	W	DSP_RESUME	Write a 1 to run DSP

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DSP_RUNNING	DSP_STATE_PAUSE	DSP paused	0x0*
	DSP_STATE_RUN	DSP running	0x1
DSP_CM3_CFX_NNA_INT_RESET	DSP_CM3_CFX_NNA_INT_RESET	Reset CM3_CFX and NNA interrupts in the DSP interrupt controller	0x1
DSP_DMA_INT_RESET	DSP_DMA_INT_RESET	Reset DMA interrupts in the DSP interrupt controller	0x1
DSP_RESET	DSP_RESET	Reset DSP	0x1
DSP_PAUSE	DSP_PAUSE	Pause DSP	0x1
DSP_RESUME	DSP_RESUME	Resume DSP	0x1

8.5.2 IMSK

Bit Field	Read/Write	Field Name	Description
0	R/W	IMSK_DMA_0	DMA_0 unmasking bit
1	R/W	IMSK_DMA_1	DMA_1 unmasking bit
2	R/W	IMSK_DMA_2	DMA_2 unmasking bit
3	R/W	IMSK_DMA_3	DMA_3 unmasking bit
4	R/W	IMSK_DMA_4	DMA_4 unmasking bit
5	R/W	IMSK_DMA_5	DMA_5 unmasking bit
6	R/W	IMSK_DMA_6	DMA_6 unmasking bit
7	R/W	IMSK_DMA_7	DMA_7 unmasking bit
8	R/W	IMSK_CM3_CFX_0	CM3_CFX_0 unmasking bit
9	R/W	IMSK_CM3_CFX_1	CM3_CFX_1 unmasking bit
10	R/W	IMSK_CM3_CFX_2	CM3_CFX_2 unmasking bit
11	R/W	IMSK_CM3_CFX_3	CM3_CFX_3 unmasking bit
12	R/W	IMSK_CM3_CFX_4	CM3_CFX_4 unmasking bit
13	R/W	IMSK_CM3_CFX_5	CM3_CFX_5 unmasking bit
14	R/W	IMSK_NNA	NNA unmasking bit

Field Name	Value Symbol	Value Description	Hex Value
IMSK_DMA_0	IMSK_DMA_0_INT_ENABLE	DMA_0 interrupt unmasked	0x1
IMSK_DMA_1	IMSK_DMA_1_INT_ENABLE	DMA_1 interrupt unmasked	0x1
IMSK_DMA_2	IMSK_DMA_2_INT_ENABLE	DMA_2 interrupt unmasked	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
IMSK_DMA_3	IMSK_DMA_3_INT_ENABLE	DMA_3 interrupt unmasked	0x1
IMSK_DMA_4	IMSK_DMA_4_INT_ENABLE	DMA_4 interrupt unmasked	0x1
IMSK_DMA_5	IMSK_DMA_5_INT_ENABLE	DMA_5 interrupt unmasked	0x1
IMSK_DMA_6	IMSK_DMA_6_INT_ENABLE	DMA_6 interrupt unmasked	0x1
IMSK_DMA_7	IMSK_DMA_7_INT_ENABLE	DMA_7 interrupt unmasked	0x1
IMSK_CM3_CFX_0	IMSK_CM3_CFX_0_INT_ENABLE	CM3_CFX_0 interrupt unmasked	0x1
IMSK_CM3_CFX_1	IMSK_CM3_CFX_1_INT_ENABLE	CM3_CFX_1 interrupt unmasked	0x1
IMSK_CM3_CFX_2	IMSK_CM3_CFX_2_INT_ENABLE	CM3_CFX_2 interrupt unmasked	0x1
IMSK_CM3_CFX_3	IMSK_CM3_CFX_3_INT_ENABLE	CM3_CFX_3 interrupt unmasked	0x1
IMSK_CM3_CFX_4	IMSK_CM3_CFX_4_INT_ENABLE	CM3_CFX_4 interrupt unmasked	0x1
IMSK_CM3_CFX_5	IMSK_CM3_CFX_5_INT_ENABLE	CM3_CFX_5 interrupt unmasked	0x1
IMSK_NNA	IMSK_NNA_INT_ENABLE	NNA interrupt unmasked	0x1

8.5.3 IRQ_STAT

Bit Field	Read/Write	Field Name	Description
0	R	IRQ_STAT_DMA_0	IRQ_STAT_DMA_0 status
1	R	IRQ_STAT_DMA_1	IRQ_STAT_DMA_1 status
2	R	IRQ_STAT_DMA_2	IRQ_STAT_DMA_2 status
3	R	IRQ_STAT_DMA_3	IRQ_STAT_DMA_3 status
4	R	IRQ_STAT_DMA_4	IRQ_STAT_DMA_4 status
5	R	IRQ_STAT_DMA_5	IRQ_STAT_DMA_5 status
6	R	IRQ_STAT_DMA_6	IRQ_STAT_DMA_6 status
7	R	IRQ_STAT_DMA_7	IRQ_STAT_DMA_7 status
8	R	IRQ_STAT_CM3_CFX_0	IRQ_STAT_CM3_CFX_0 status
9	R	IRQ_STAT_CM3_CFX_1	IRQ_STAT_CM3_CFX_1 status
10	R	IRQ_STAT_CM3_CFX_2	IRQ_STAT_CM3_CFX_2 status
11	R	IRQ_STAT_CM3_CFX_3	IRQ_STAT_CM3_CFX_3 status
12	R	IRQ_STAT_CM3_CFX_4	IRQ_STAT_CM3_CFX_4 status
13	R	IRQ_STAT_CM3_CFX_5	IRQ_STAT_CM3_CFX_5 status
14	R	IRQ_STAT_NNA	IRQ_STAT_NNA status

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
IRQ_STAT_DMA_0	IRQ_STAT_DMA_0_INT_ACTIVE	IRQ_STAT_DMA_0 interrupt status	0x1
IRQ_STAT_DMA_1	IRQ_STAT_DMA_1_INT_ACTIVE	IRQ_STAT_DMA_1 interrupt status	0x1
IRQ_STAT_DMA_2	IRQ_STAT_DMA_2_INT_ACTIVE	IRQ_STAT_DMA_2 interrupt status	0x1
IRQ_STAT_DMA_3	IRQ_STAT_DMA_3_INT_ACTIVE	IRQ_STAT_DMA_3 interrupt status	0x1
IRQ_STAT_DMA_4	IRQ_STAT_DMA_4_INT_ACTIVE	IRQ_STAT_DMA_4 interrupt status	0x1
IRQ_STAT_DMA_5	IRQ_STAT_DMA_5_INT_ACTIVE	IRQ_STAT_DMA_5 interrupt status	0x1
IRQ_STAT_DMA_6	IRQ_STAT_DMA_6_INT_ACTIVE	IRQ_STAT_DMA_6 interrupt status	0x1
IRQ_STAT_DMA_7	IRQ_STAT_DMA_7_INT_ACTIVE	IRQ_STAT_DMA_7 interrupt status	0x1
IRQ_STAT_CM3_CFX_0	IRQ_STAT_CM3_CFX_0_INT_ACTIVE	IRQ_STAT_CM3_CFX_0 interrupt status	0x1
IRQ_STAT_CM3_CFX_1	IRQ_STAT_CM3_CFX_0_INT_ACTIVE	IRQ_STAT_CM3_CFX_1 interrupt status	0x1
IRQ_STAT_CM3_CFX_2	IRQ_STAT_CM3_CFX_0_INT_ACTIVE	IRQ_STAT_CM3_CFX_2 interrupt status	0x1
IRQ_STAT_CM3_CFX_3	IRQ_STAT_CM3_CFX_0_INT_ACTIVE	IRQ_STAT_CM3_CFX_3 interrupt status	0x1
IRQ_STAT_CM3_CFX_4	IRQ_STAT_CM3_CFX_0_INT_ACTIVE	IRQ_STAT_CM3_CFX_4 interrupt status	0x1
IRQ_STAT_CM3_CFX_5	IRQ_STAT_CM3_CFX_0_INT_ACTIVE	IRQ_STAT_CM3_CFX_5 interrupt status	0x1
IRQ_STAT_NNA	IRQ_STAT_NNA_INT_ACTIVE	IRQ_STAT_NNA interrupt status	0x1

8.5.4 IRQ_CLR

Bit Field	Read/Write	Field Name	Description
0	R/W	IRQ_CLR_DMA_0	IRQ_CLR_DMA_0 interrupt acknowledge
1	R/W	IRQ_CLR_DMA_1	IRQ_CLR_DMA_1 interrupt acknowledge
2	R/W	IRQ_CLR_DMA_2	IRQ_CLR_DMA_2 interrupt acknowledge
3	R/W	IRQ_CLR_DMA_3	IRQ_CLR_DMA_3 interrupt acknowledge
4	R/W	IRQ_CLR_DMA_4	IRQ_CLR_DMA_4 interrupt acknowledge
5	R/W	IRQ_CLR_DMA_5	IRQ_CLR_DMA_5 interrupt acknowledge
6	R/W	IRQ_CLR_DMA_6	IRQ_CLR_DMA_6 interrupt acknowledge
7	R/W	IRQ_CLR_DMA_7	IRQ_CLR_DMA_7 interrupt acknowledge
8	R/W	IRQ_CLR_CM3_CFX_0	IRQ_CLR_CM3_CFX_0 interrupt acknowledge

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
9	R/W	IRQ_CLR_CM3_CFX_1	IRQ_CLR_CM3_CFX_1 interrupt acknowledge
10	R/W	IRQ_CLR_CM3_CFX_2	IRQ_CLR_CM3_CFX_2 interrupt acknowledge
11	R/W	IRQ_CLR_CM3_CFX_3	IRQ_CLR_CM3_CFX_3 interrupt acknowledge
12	R/W	IRQ_CLR_CM3_CFX_4	IRQ_CLR_CM3_CFX_4 interrupt acknowledge
13	R/W	IRQ_CLR_CM3_CFX_5	IRQ_CLR_CM3_CFX_5 interrupt acknowledge
14	R/W	IRQ_CLR_NNA	IRQ_CLR_NNA interrupt acknowledge

Field Name	Value Symbol	Value Description	Hex Value
IRQ_CLR_DMA_0	IRQ_CLR_DMA_0_CLR	Acknowledge the IRQ_CLR_DMA_0 interrupt	0x1
IRQ_CLR_DMA_1	IRQ_CLR_DMA_1_CLR	Acknowledge the IRQ_CLR_DMA_1 interrupt	0x1
IRQ_CLR_DMA_2	IRQ_CLR_DMA_2_CLR	Acknowledge the IRQ_CLR_DMA_2 interrupt	0x1
IRQ_CLR_DMA_3	IRQ_CLR_DMA_3_CLR	Acknowledge the IRQ_CLR_DMA_3 interrupt	0x1
IRQ_CLR_DMA_4	IRQ_CLR_DMA_4_CLR	Acknowledge the IRQ_CLR_DMA_4 interrupt	0x1
IRQ_CLR_DMA_5	IRQ_CLR_DMA_5_CLR	Acknowledge the IRQ_CLR_DMA_5 interrupt	0x1
IRQ_CLR_DMA_6	IRQ_CLR_DMA_6_CLR	Acknowledge the IRQ_CLR_DMA_6 interrupt	0x1
IRQ_CLR_DMA_7	IRQ_CLR_DMA_7_CLR	Acknowledge the IRQ_CLR_DMA_7 interrupt	0x1
IRQ_CLR_CM3_CFX_0	IRQ_CLR_CM3_CFX_0_CLR	Acknowledge the IRQ_CLR_CM3_CFX_0 interrupt	0x1
IRQ_CLR_CM3_CFX_1	IRQ_CLR_CM3_CFX_1_CLR	Acknowledge the IRQ_CLR_CM3_CFX_1 interrupt	0x1
IRQ_CLR_CM3_CFX_2	IRQ_CLR_CM3_CFX_2_CLR	Acknowledge the IRQ_CLR_CM3_CFX_2 interrupt	0x1
IRQ_CLR_CM3_CFX_3	IRQ_CLR_CM3_CFX_3_CLR	Acknowledge the IRQ_CLR_CM3_CFX_3 interrupt	0x1
IRQ_CLR_CM3_CFX_4	IRQ_CLR_CM3_CFX_4_CLR	Acknowledge the IRQ_CLR_CM3_CFX_4 interrupt	0x1
IRQ_CLR_CM3_CFX_5	IRQ_CLR_CM3_CFX_5_CLR	Acknowledge the IRQ_CLR_CM3_CFX_5 interrupt	0x1
IRQ_CLR_NNA	IRQ_CLR_NNA_CLR	Acknowledge the IRQ_CLR_NNA interrupt	0x1

Ezairo 8300 Hardware Reference

8.5.5 CM3_CFX_CMD

Bit Field	Read/Write	Field Name	Description
7	W	CM3_CFX_CMD_7	Write a 1 to issue Arm Cortex-M3 core/CFX DSP command 7
6	W	CM3_CFX_CMD_6	Write a 1 to issue Arm Cortex-M3 core/CFX DSP command 6
5	W	CM3_CFX_CMD_5	Write a 1 to issue Arm Cortex-M3 core/CFX DSP command 5
4	W	CM3_CFX_CMD_4	Write a 1 to issue Arm Cortex-M3 core/CFX DSP command 4
3	W	CM3_CFX_CMD_3	Write a 1 to issue Arm Cortex-M3 core/CFX DSP command 3
2	W	CM3_CFX_CMD_2	Write a 1 to issue Arm Cortex-M3 core/CFX DSP command 2
1	W	CM3_CFX_CMD_1	Write a 1 to issue Arm Cortex-M3 core/CFX DSP command 1
0	W	CM3_CFX_CMD_0	Write a 1 to issue Arm Cortex-M3 core/CFX DSP command 0

Field Name	Value Symbol	Value Description	Hex Value
CM3_CFX_CMD_7	CM3_CFX_CMD_7	Issue Arm Cortex-M3 core/CFX DSP command 7	0x1
CM3_CFX_CMD_6	CM3_CFX_CMD_6	Issue Arm Cortex-M3 core/CFX DSP command 6	0x1
CM3_CFX_CMD_5	CM3_CFX_CMD_5	Issue Arm Cortex-M3 core/CFX DSP command 5	0x1
CM3_CFX_CMD_4	CM3_CFX_CMD_4	Issue Arm Cortex-M3 core/CFX DSP command 4	0x1
CM3_CFX_CMD_3	CM3_CFX_CMD_3	Issue Arm Cortex-M3 core/CFX DSP command 3	0x1
CM3_CFX_CMD_2	CM3_CFX_CMD_2	Issue Arm Cortex-M3 core/CFX DSP command 2	0x1
CM3_CFX_CMD_1	CM3_CFX_CMD_1	Issue Arm Cortex-M3 core/CFX DSP command 1	0x1
CM3_CFX_CMD_0	CM3_CFX_CMD_0	Issue Arm Cortex-M3 core/CFX DSP command 0	0x1

CHAPTER 9

Power Components

The power supply is a critical component of the Ezairo 8300 system, as system performance, particularly audio performance, is greatly affected by the supplied power.

The power supply is powered by a battery supply voltage (VBAT), which is used as the supply for a number of power sources, including:

- A regulated, low-noise voltage reference source for the analog components (VREG)
- Two configurable regulated supplies for digital components (VDDC, VDDM)
- An on-chip charge pump for the analog system components (VDDA)
- An on-chip charge pump for the digital interfaces and external components (VDDIF)

9.1 POWER SUPPLY

The components that make up the power supply can be divided into three types of supply voltages:

1. Power supply input voltages, described further in [Section 9.1.1 “Power Supply Inputs”](#)
2. Internal power supply voltages, described further in [Section 9.1.2 “Internal Power Supply Voltages”](#)
3. Power supply output voltages, described further in [Section 9.1.3 “Power Supply Outputs”](#)

The "[Top-Level Power Supply Block Diagram](#)" table (Figure 12) shows a block diagram of the main components of the power supply.

Ezairo 8300 Hardware Reference

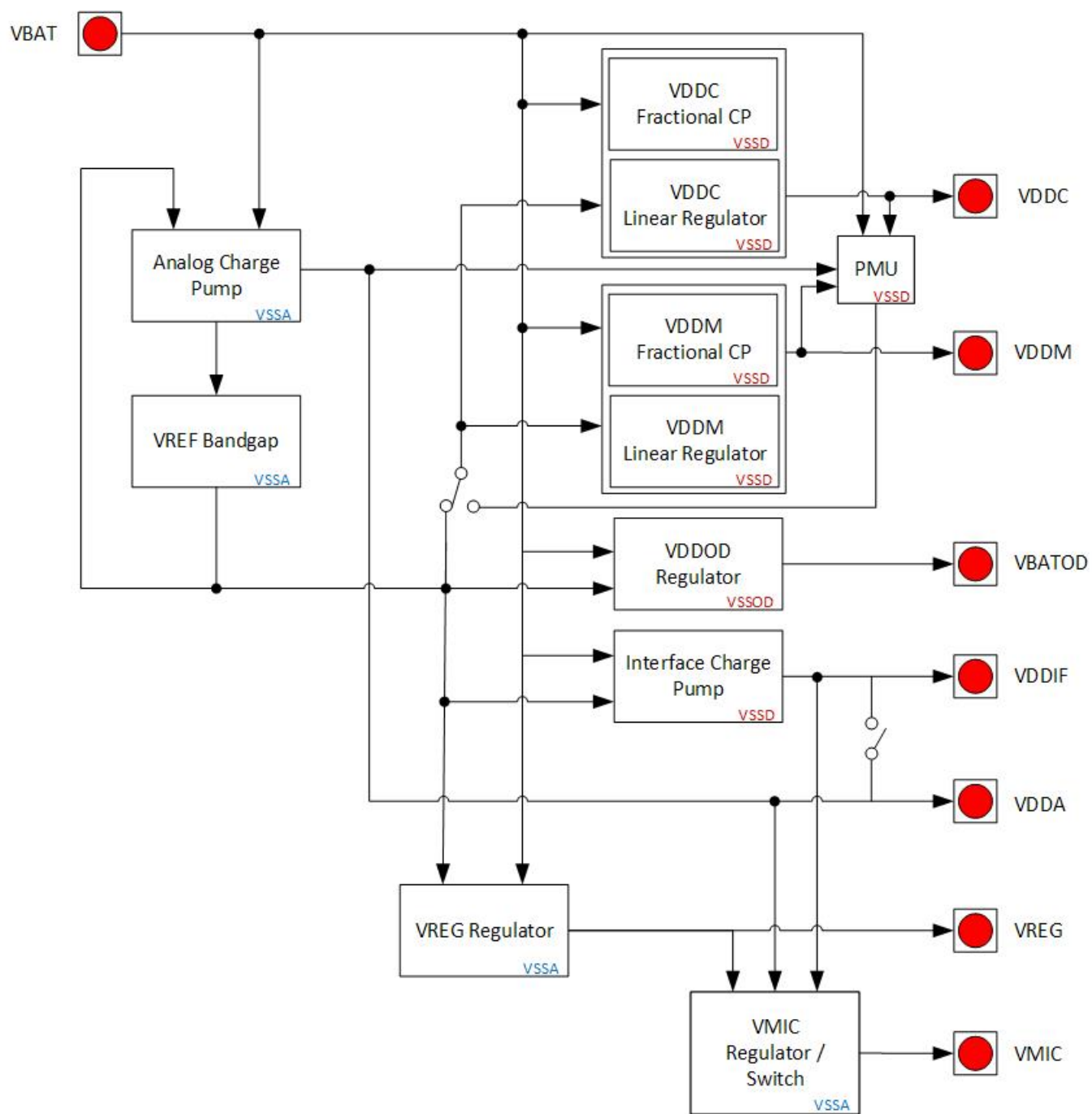


Figure 12. Top-Level Power Supply Block Diagram

Ezairo 8300 Hardware Reference

9.1.1 Power Supply Inputs

9.1.1.1 Battery Supply Voltage (VBAT)

The primary voltage supplied to an Ezairo 8300 chip is known as the battery supply voltage because this voltage is supplied by a battery in a typical application.

VBAT directly supplies each of the regulators and charge-pumps included in the Ezairo 8300 chip. This supply voltage is monitored by the power management unit and the battery monitor to ensure safe system operation, and to signal the current voltage level to user applications. See [Section 9.2 “Power Management Unit” on page 297](#), and [Section 18.5.4 “Power Supply Monitoring” on page 633](#) for more information about the power management unit and the battery monitor.

The minimum VBAT voltage required for basic operation is 0.9 V. However, the electrical performance of the VREF regulator, and the VDDA and VDDIF charge pumps, is reduced to the range 0.9 V to 1.0 V. The following effects are observed due to the reduced electrical performance:

- Degradation in the input dynamic range due to a reduction in the VREF regulator supply
- Likely reduction in CCO performance due to the effects on VDDA
- Restricted VDDIF load currents

9.1.1.2 Output Receiver Supply Voltage (VBATOD)

The direct-digital output drivers from the output stage are powered from the VBATOD. This supply voltage is typically connected to one of the following sources:

- Externally connected to VBAT to obtain the lowest output impedance
- Internally sourced from the VDDOD regulator to obtain the best PSRR, when large transient currents are drawn elsewhere in the Ezairo 8300-based system.

The VDDOD regulator can be enabled by setting the `ANALOG_VDDOD_CTRL_VDDOD_REG_EN` bit from the `ANALOG_VDDOD_CTRL` register. When disabled, VDDOD is tied internally to VBAT. When enabled, the output voltage from VDDOD is trimmed using the `ANALOG_VDDOD_CTRL_VDDOD_REG_TRIM` bit-field from the `ANALOG_VDDOD_CTRL`.

IMPORTANT: It is not recommended to rely on the internal short of the VDDOD regulator to VBAT, as this short will typically have an increased output impedance verses an external connection.

If VBATOD is supplied externally by a voltage from a different supply than that provided by VBAT, the supplied voltage must be at least 0.80 V and the VDDOD regulator should be enabled and set to the lowest trim setting (`VDDOD_REG_TRIM_0P80V`).

9.1.1.3 Digital Output Supply Voltages (VDDO1, VDDO2, VDDO3, and VDDO4)

The digital output supply voltages are attached to the I/O pads on an Ezairo 8300 chip. This includes:

- All DIO pads, as described in [Chapter 16.1 "Overview" on page 463](#)
- The SCL and SDA pads for the I²C debug ports and the general-purpose I²C interface that shares the same I²C bus (`D_I2C`). For more information, see [Section 18.4 “I2C Interfaces” on page 601](#).
- The NRESET input pad, described in [Section 9.3 “Resets” on page 299](#)

Ezairo 8300 Hardware Reference

The internal system digital logic is attached to the pads through internal level translators. They shift the voltage level from VDDC to the correct VDDO voltage for digital outputs, and translate input digital signals from the correct VDDO voltage to VDDC for digital inputs. The "Digital Supply Mapping" table (Table 17) provides a mapping of the digital pads to VDDO supply voltages.

Table 17. Digital Supply Mapping

Supply Voltage	Pads
VDDO1	DIO0 to DIO11
VDDO2	DIO12 to DIO23
VDDO3	DIO24 to DIO29
VDDO4	DIO30 to DIO35
	SCL and SDA, supporting the D_I2C interface
	NRESET

The VDDO inputs will generally be connected externally to VBAT or VDDIF based on the required voltages for digital communications using the associated I/O pads. All four VDDO voltages can be supplied from different sources or the same source, as required for communication with external system components.

IMPORTANT: To minimize power consumption, VDDO* power domains need to be powered at the lowest voltage possible (e.g., preference given to 1.25 V over 1.8 V wherever possible).

9.1.2 Internal Power Supply Voltages

9.1.2.1 Internal Band Gap Reference Voltage (VREF)

The Ezairo 8300 chip includes a band gap reference voltage that is stabilized over temperature and supply voltage variations. This reference voltage is used in the generation of all of the regulated voltages in the Ezairo 8300 chip, and can be used to provide a calibrated 0.90 V reference signal to all components using the reference voltage.

To trim the level of the band gap voltage, use the ANALOG_BG_CTRL_BG_LEVEL bit field from the ANALOG_BG_CTRL register. Temperature compensation circuitry for the band gap can be configured using the ANALOG_BG_CTRL_BG_TEMP_COEF_TRIM bit field from this register, and the optimal setting of 0xF is recommended for this bit field in all cases.

9.1.2.2 Internal Digital Supply Voltages (VDDC and VDDM)

The Ezairo 8300 chip includes a pair of internally regulated digital supply voltages:

- VDDC is the core digital voltage that is used for most of the digital components of the system.
- VDDM is the memory digital voltage that is used for memories of the Ezairo 8300 system. This voltage must always be greater than or equal to the VDDC voltage for proper operation.

Each of the digital supply voltages is generated using a fully-integrated fractional charge pump in parallel with a linear regulator. To configure these fractional charge pumps to be enabled or disabled, use the ANALOG_CP_VDDC_CTRL_ENABLE and ANALOG_CP_VDDM_CTRL_ENABLE bits from the ANALOG_CP_VDDC_CTRL and ANALOG_CP_VDDM_CTRL registers, respectively. The VDDC and VDDM fractional charge pumps can be configured to use different fractional ratios when converting the supply voltage by using the ANALOG_CP_VDDC_CTRL_CP_MODE and ANALOG_

Ezairo 8300 Hardware Reference

CP_VDDM_CTRL_CP_MODE bit fields from the ANALOG_CP_VDDC_CTRL and ANALOG_CP_VDDM_CTRL registers. For optimal performance, automatic charge pump control can be enabled to automatically enable, disable, and control the charge pump ratios based on the VDDC, VDDM and VBAT voltages as measured by the LSAD.

The following configuration steps are required for using VDDC and VDDM automatic charge pump control:

- Set LSAD_CFG_FREQ bit field of the LSAD_CFG register to LSAD_PRESCALE_160. Using a lower LSAD sampling frequency is possible as well but it will reduce the reaction time of the mechanism.
- Set D_CLK_CFG_CPCM_CP_VDDCM_CLK_SRC bit of the D_CLK_CFG_CPCM register to CPCMLK_SRC_SDMCLK.
- Set D_CLK_CFG_CPCM_CP_VDDCM_CLK_PRESCALE bit field of the D_CLK_CFG_CPCM register to CPCMLK_PRESCALE_2 (when SDMCLK is 3.84 MHz or 7.68 MHz) or CPCMLK_PRESCALE_4 (when SDMCLK is 15.36 MHz).
- Set D_CLK_CFG_CPCM_CP_VDDC_CLK_PHASE and D_CLK_CFG_CPCM_CP_VDDM_CLK_PHASE bit fields of the D_CLK_CFG_CPCM register to CPCCLK_NORMAL and CPMCLK_INVERTED, respectively.
- Set both ANALOG_CP_VDDC_CTRL_AUTO_CTRL and ANALOG_CP_VDDM_CTRL_AUTO_CTRL bit fields to VDDC_CTRL_AUTO and VDDM_CTRL_AUTO in the ANALOG_CP_VDDC_CTRL and ANALOG_CP_VDDM_CTRL registers, respectively, to enable automatic charge pump control.

When the digital charge pumps are configured for automatic charge pump control, the charge pumps controllers switch configurations as needed. By default, these charge pumps are configured to automatically switch between configurations that minimize the digital power consumption using the CP_VDDCM_CFG_LOW_POWER bit setting written to the ANALOG_CP_VDDCM_CFG register. As switching the charge pump configuration too often can sometimes introduce noise into the Ezairo 8300 system, the CP_VDDCM_CFG_QUIET bit setting can be written to the ANALOG_CP_VDDCM_CFG register to reduce the frequency at which switches occur.

The VDDC and VDDM fractional charge pumps use internal capacitances that are periodically refreshed to maintain the supply. The refresh frequencies for the VDDC and VDDM charge pumps can be divided from SYSCLK (see Section 10.2.1 “System Clock (SYSCLK)” on page 313) or SDMCLK (see Section 10.2.2 “Sigma-Delta Modulator Clock (SDMCLK)” on page 314) by configuring the D_CLK_CFG_CPCM_CP_VDDCM_CLK_SRC bit from the D_CLK_CFG_CPCM register. The clock division factor can be configured using the D_CLK_CFG_CPCM_CP_VDDCM_CLK_PRESCALE bit field from the D_CLK_CFG_CPCM register. The output voltage is synchronized on the rising edge of the charge pump clock by default. To configure the synchronization for the falling edge, set the D_CLK_CFG_CPCM_CP_VDDC_CLK_PHASE and D_CLK_CFG_CPCM_CP_VDDM_CLK_PHASE bits of the D_CLK_CFG_CPCM register. Selecting the falling edge can help distribute the current load on VBAT, and selecting the opposite clock edge can be helpful in mitigating crosstalk and noise issues.

The VDDC and VDDM linear regulator and charge pump output voltages can be calibrated by configuring the ANALOG_CP_VDDC_TRIM and ANALOG_CP_VDDM_TRIM registers, respectively. The linear regulator voltages need to be calibrated while the charge pumps are disabled, since the charge pump output voltages are delta voltages based on the linear regulator voltages. After the initial calibration, adaptive voltage scaling can be used to automatically adjust the trimming for VDDC.

NOTE: The charge pump clock and the charge pump mode should be configured appropriately before proceeding with the calibration. The recommended settings are as follows:

- Set the CLK_CFG_CPCM register to CPCMLK_SRC_SDMCLK | CPCCLK_NORMAL | CPMCLK_NORMAL | CPCMLK_PRESCALE_2 (when SDMCLK is 3.84 MHz or 7.68 MHz) or CPCMLK_SRC_SDMCLK | CPCCLK_NORMAL | CPMCLK_NORMAL | CPCMLK_PRESCALE_4 (when SDMCLK is 15.36 MHz).
- Set the ANALOG_CP_VDDC_CTRL_CP_MODE bit field in the ANALOG_CP_VDDC_CTRL register to VDDC_RATIO_4_5.

Ezairo 8300 Hardware Reference

- Similarly, if calibrating the VDDM charge pump as well, set the ANALOG_CP_VDDM_CTRL_CP_MODE bit field in the ANALOG_CP_VDDM_CTRL register to VDDM_RATIO_4_5.

IMPORTANT: As VDDC and VDDM are affected by the trim configuration of the band gap reference (VREF) and analog supply voltage (VDDA), VREF and VDDA must be calibrated before calibrating any trim settings for VDDC or VDDM.

IMPORTANT: To minimize power consumption, the VDDC and VDDM charge pump voltages should be trimmed to the lowest delta trimming value where the charge pump voltage for the supply exceeds the supply's LDO voltage by 7 mV or more.

The "Operating Frequency" table (Table 18) shows the relationship between VDDC and VDDM voltage trim settings for operation in both run mode and standby mode with memory retention. The placeholders <VDD* active> and <VDD* retention> represent target voltage trims that are specified in the Ezairo 8300 datasheet, which can be found at <https://www.onsemi.com/products/audio-video-assp/audiology-dsp-systems/ezairo-8300>.

Table 18. Operating Frequency

VDDC Voltage	VDDM Voltage	Maximum Operating Frequency
<VDDC retention>	<VDDM standby>, <VDDM retention>	100 kHz (standby mode, memories in retention - retention trim settings)
Device controlled between <VDDC limit> and <VDDC active>	<VDDM active>	Up to the datasheet specified maximum operating frequency (run mode, using adaptive voltage scaling as described below)
<VDDC active>	<VDDM active>	Up to the datasheet specified maximum operating frequency (run mode, without using adaptive voltage scaling)

NOTE: If VDDC exceeds VDDM by more than 50 mV, the leakage current for digital components increases.

NOTE: When calibrating the VDDC retention, VDDM standby, and VDDM retention trim settings, VDDC and VDDM must be configured to use the power management unit (PMU) reference. As this supply reference is less accurate than the bandgap, trim setting targets using this reference include extra margin to ensure that the device achieves the required minimum voltages. This trim setting can be forced by setting the ANALOG_CP_VDDCM_REF_VDDCM_REF_SEL bit to VDDCM_PMU in the ANALOG_CP_VDDCM_REF register. This setting does not need to be set when switching to standby or sleep mode as the reference supplies are automatically configured correctly by the PMU.

IMPORTANT: When switching to the standby power mode, applications must select between using VDDM standby and VDDM retention. If using VDDM standby, the memories are kept active when the cores are held in retention and wakeup time is less than five STANDBYCLK cycles. If using VDDM retention, the memories and cores are held in retention (the lowest power state) and wakeup time is between 25 and 50 STANDBYCLK cycles.

Ezairo 8300 Hardware Reference

These trim targets are the minimal VDDC and VDDM voltages that are needed to guarantee operation in the worst case conditions, assuming these voltages are trimmed with VBAT of 1.25 V at room temperature (~25°C). The following are recommendations for reduced supply voltage requirements if the operating temperature range is restricted to -5°C to 50°C:

- VDDC active and VDDM active settings can be decreased by 10 mV (-5 LSB in trimming value).
- VDDC retention trim setting (used in standby mode both with and without retention) can be decreased by 8 mV (-4 LSB in trimming value).
- VDDM standby trim setting can be decreased by 24 mV (-12 LSB in trimming value).
- VDDM retention trim setting can be decreased by 20 mV (-10 LSB in trimming value).

NOTE: VDDC limit cannot be decreased in the restricted operating range.

IMPORTANT: For more information about calibration and configuration of the internal digital supply voltages, refer to the Calibration Library chapter of the *Ezairo 8300 Firmware Reference*.

9.1.2.2.1 Adaptive Voltage Scaling for VDDC

The required minimum VDDC trim level is defined by the timing of the system's digital logic. Automatic control of the VDDC trim level using adaptive voltage scaling can be enabled to automatically adjust the VDDC trim in a user defined range, to take advantage of lower VDDC trim settings (and hence lower power consumption) when there are sufficient timing margins on the operation of the digital components. When adaptive voltage scaling is enabled, the system periodically measures the digital timing of critical paths in the digital logic and adjusts VDDC appropriately.

To configure the VDDC supply for adaptive voltage scaling configuration:

1. Set the ANALOG_CP_VDDC_AUTO_LIMITS_MIN_TRIM and ANALOG_CP_VDDC_AUTO_LIMITS_MAX_TRIM bit fields of the ANALOG_CP_VDDC_AUTO_LIMITS register to the <VDDC Limit> and <VDDC Active> calibrated trim values, respectively.
2. Set the ANALOG_CP_VDDC_AUTO_CTRL_MIN_MARGIN bit field of the ANALOG_CP_VDDC_AUTO_CTRL register to VDDC_MIN_MARGIN_CCO_MULT_4X if CCO multiplier 4x is used, and set it to VDDC_MIN_MARGIN_ALL_EXCEPT_CCO_MULT_4X otherwise. Refer to [Section 10.1.1 “Current-Controlled Oscillator” on page 311](#) for more information about CCO multiplier setting.
3. Enable VDDC voltage scaling by setting the ANALOG_CP_VDDC_AUTO_CTRL_ENABLE bit in the ANALOG_CP_VDDC_AUTO_CTRL register to VDDC_TRIM_AUTO.
4. If using the charge pump, enable it by setting the ANALOG_CP_VDDC_CTRL_ENABLE bit in the ANALOG_CP_VDDC_CTRL register to VDDC_ENABLE.

When changing the SYSCLK frequency to a lower frequency while using adaptive voltage scaling, the VDDC adaptive voltage scaling can be kept enabled. After changing the CCO multiplier from 4x to a smaller multiplier, set the ANALOG_CP_VDDC_AUTO_CTRL_MIN_MARGIN bit field to VDDC_MIN_MARGIN_ALL_EXCEPT_CCO_MULT_4X. When changing the SYSCLK frequency to a higher frequency, follow the steps below:

1. Disable the VDDC adaptive voltage scaling by setting the ANALOG_CP_VDDC_AUTO_CTRL_ENABLE bit in the ANALOG_CP_VDDC_AUTO_CTRL register to VDDC_TRIM_MANUAL.
2. Set ANALOG_CP_VDDC_CTRL_AUTO_CTRL and ANALOG_CP_VDDC_CTRL_ENABLE in the ANALOG_CP_VDDC_CTRL register to VDDC_CTRL_MANUAL and VDDC_DISABLE, respectively, if the VDDC charge pump is used.
3. Ensure the VDDC trim is configured for active mode (trimmed to <VDDC active> in the ANALOG_CP_VDDC_TRIM register).
4. Change the SYSCLK frequency.

Ezairo 8300 Hardware Reference

5. If the CCO multiplier was changed to 4x from a smaller multiplier, set the `ANALOG_CP_VDDC_AUTO_CTRL_MIN_MARGIN` bit field to `VDDC_MIN_MARGIN_CCO_MULT_4X`.
6. Enable the VDDC adaptive voltage scaling again by setting the `ANALOG_CP_VDDC_AUTO_CTRL_ENABLE` bit in the `ANALOG_CP_VDDC_AUTO_CTRL` register to `VDDC_TRIM_AUTO`.
7. If using the charge pump (with automatic charge pump control), wait until LSAD has completed a new conversion, sampled a new VDDC voltage (typically 125 μ s), and then re-enable the charge pump and automatic control by setting the `ANALOG_CP_VDDC_CTRL_AUTO_CTRL` and `ANALOG_CP_VDDC_CTRL` bits in the `ANALOG_CP_VDDC_CTRL` register to `VDDC_CTRL_AUTO` and `VDDC_ENABLE`, respectively.

9.1.2.3 Regulated Supply Voltage (VREG)

The Ezairo 8300 chip provides a regulated reference supply voltage at the VREG pad to allow for additional external filtering of the regulated voltages within the system. VREG is sourced by VBAT and provides a regulated voltage of 0.9 V.

9.1.2.4 Analog Supply Voltage (VDDA)

The analog supply voltage makes use of the internal charge pump to generate a configurable regulated supply voltage that is used for all of the analog audio components. In particular, this is used for the input stage of the system. This signal is typically configured as a 1.8 V supply voltage.

The VDDA charge pump uses an external capacitance that is periodically refreshed to maintain the supply voltage. The refresh frequency for the charge pump is divided from SLOWCLK (see [Section 10.2.4 “Slow Clock \(SLOWCLK\)” on page 315](#)) using the `CLK_CFG_CPA_CP_VDDA_CLK_PRESCALE` bit field from the `CLK_CFG_CPA` register.

The VDDA charge pump can be configured to operate in a high or low power mode by using the `ANALOG_CP_VDDA_CTRL_HIGH_POWER` bit from the `ANALOG_CP_VDDA_CTRL` register. When changing the power mode, the current trimming and the clock frequency of the charge pump need to be updated accordingly. The current trimming value can be configured using the `ANALOG_CP_VDDA_CTRL_ITRM` bit field of the `ANALOG_CP_VDDA_CTRL` register. The ["Recommended Trimming and Clock Frequency Values \(VDDA\)" table \(Table 19\)](#) lists the recommended trimming and clock frequency settings for the power modes. The nominal voltage supplied by VDDA can be calibrated using the `ANALOG_CP_VDDA_CTRL_VTRIM` bit field of the `ANALOG_CP_VDDA_CTRL` register.

Table 19. Recommended Trimming and Clock Frequency Values (VDDA)

Operating Mode	Frequency (kHz)	PRESCALE	ITRIM	Notes
Standby Mode	25	N/A	0x09	Bias from PMU. Only used during POR sequence. As VDDA is biased from the PMU directly in this mode, it requires a second calibrated trim setting relative to the PMU (<code>VTRIM_PMU</code>).
Low Power Mode	128	0x09	0x09	Bias from Bandgap reference In this mode, the calibrated trim setting needs to be incremented by 1 (i.e., use <code>VTRIM_BG+1</code>)
High Power Mode	320	0x03	0x12	Bias from Bandgap reference (<code>VTRIM_BG</code>).

IMPORTANT: To minimize power consumption, use the Low Power Mode wherever possible.

Ezairo 8300 Hardware Reference

NOTE: When calibrating the VDDA trim setting for standby mode, the VDDA must be configured to use the power management unit (PMU) reference. This trim setting can be forced by setting the ANALOG_CP_VDDA_CTRL_STANDBY bit to VDDA_STANDBY in the ANALOG_CP_VDDA_CTRL register. This setting does not need to be set when switching to standby mode as the reference supplies are automatically configured correctly by the PMU.

9.1.3 Power Supply Outputs

9.1.3.1 Regulated Interface Supply Voltage (VDDIF)

The regulated interface supply voltage makes use of the internal charge pump to generate a configurable regulated supply voltage that is used for communications with external components. It can also be used to power external devices that interface with the Ezairo 8300 chip. This signal is typically configured as a 1.8 V supply voltage, but can be configured to supply between 1.57 V and 1.98 V as needed.

The regulated interface supply voltage is available at the VDDIF output pad. This supply voltage meets the regulated voltage specifications from the Ezairo 8300 data sheet when the system is supplied with a VBAT supply voltage of at least 1.0 V.

The VDDIF charge pump can be enabled or disabled using the ANALOG_CP_VDDIF_CTRL_ENABLE bit from the ANALOG_CP_VDDIF_CTRL register. By default VDDIF is connected to VDDA when disabled. This can be changed by configuring the ANALOG_CP_VDDIF_CTRL_VDDA_CONNECT bit from the ANALOG_CP_VDDIF_CTRL register. The nominal voltage supplied through VDDIF can be trimmed using the ANALOG_CP_VDDIF_CTRL_VTRIM bit field of the ANALOG_CP_VDDIF_CTRL register. The current consumed by the VDDIF charge pump itself can be trimmed by using the ANALOG_CP_VDDIF_CTRL_ITRIM bit field from the ANALOG_CP_VDDIF_CTRL register.

The VDDIF charge pump uses an external capacitor that is periodically refreshed to maintain the supply. The refresh frequency for the charge pump is divided from SLOWCLK (see [Section 10.2.4 “Slow Clock \(SLOWCLK\)” on page 315](#)) using the CLK_CFG_CPIF_CP_VDDIF_CLK_PRESCALE bit field from the CLK_CFG_CPIF register.

Similar to the VDDA charge pump, the VDDIF charge pump can be used in multiple configurations. When changing the modes, the charge pump clock frequency and current trimming settings need to be updated accordingly. See the "[Recommended Trimming and Clock Frequency Values \(VDDIF\)](#)" table (Table 20) for a list of recommended settings for the modes of operation.

Table 20. Recommended Trimming and Clock Frequency Values (VDDIF)

Operating Mode	Frequency (kHz)	PRESCALE	ITRIM	Notes
Low Power Mode (< 1 mA load)	128	0x09	0x00	The calibrated trim setting needs to be decremented by 1 (i.e., use VTRIM-1)
High Power Mode (< 5 mA load)	160	0x07	0x08	The calibrated trim setting needs to be decremented by 1 (i.e., use VTRIM-1)
High Power Mode (< 8 mA load)	160	0x07	0x0C	The calibrated trim setting needs to be decremented by 2 (i.e., use VTRIM-2)
High Power Mode (< 10 mA load)	213	0x05	0x10	The calibrated trim setting needs to be decremented by 1 (i.e., use VTRIM-1)
High Power Mode (< 15 mA load)	320	0x03	0x10	VTRIM

IMPORTANT: To minimize power consumption:

- If VDDIF is not used, the VDDIF charge pump must be disabled and VDDIF disconnected from VDDA.
- If VDDIF is used, and VDDA can supply sufficient current for VDDIF use cases: disable the VDDIF charge pump, and connect VDDIF to VDDA.
- In all other configurations, the VDDIF charge pump should be enabled using the lowest maximum load configuration possible (VDDIF is automatically disconnected from VDDA in this case).

9.1.3.2 Microphone Supply Voltage (VMIC) and Microphone Ground Reference (GND_MIC)

The Ezairo 8300 chip provides VMIC as a configurable, noise-robust output supply voltage that can be used to supply external microphones. This supply voltage is paired with an associated noise-robust ground pad (GND_MIC) that is intended to be used to connect to the ground side of any external microphone powered from VMIC.

Ezairo 8300 can be configured using the `ANALOG_VMIC_CTRL` register to connect the VMIC pad to any of the following sources:

- VMIC regulator (output is HiZ if the regulator is disabled)
- VREG, see [Section 9.1.2.3 “Regulated Supply Voltage \(VREG\)”](#)
- VDDA, see [Section 9.1.2.4 “Analog Supply Voltage \(VDDA\)”](#)
- VDDIF, see [Section 9.1.3.1 “Regulated Interface Supply Voltage \(VDDIF\)”](#)

The `ANALOG_VMIC_CTRL_VMIC_MUX_CTRL` bit field from the `ANALOG_VMIC_CTRL` register can be configured to select between the different sources. The power supply for the VMIC regulator can be configured using the `ANALOG_VMIC_CTRL_VMIC_REG_PS` bit from the `ANALOG_VMIC_CTRL` register. The VMIC regulator powered by VBAT is a 2-stage regulator, as opposed to the single stage regulator when powered by VDDA. The VMIC regulator can be enabled by setting the `ANALOG_VMIC_CTRL_VMIC_REG_EN` bit from the `ANALOG_VMIC_CTRL` register.

NOTE: We recommend that you pre-charge VMIC to VREG before switching from HiZ mode to 2-stage regulation mode to avoid long startup times (typically 10 ms).

9.2 POWER MANAGEMENT UNIT

The power management unit (PMU) consists primarily of two components:

- A power supervisory circuit to ensure safe system operation under any battery condition
- A power-on reset (POR) component that ensures startup behavior and configuration

These components work together to ensure proper system behavior under all battery conditions. They also ensure that the system startup sequence behaves correctly, as voltage fluctuations related to system startup might otherwise result in an indeterminate system state. Neither component is user configurable.

Further information about each of these components is provided in the sections that follow.

NOTE: The PMU additionally contains secondary components that support system power management. When the Ezairo 8300 system is operating in Standby Mode, the PMU's low-power reference is provided for use by the Internal Digital Supply Voltages and Analog Supply Voltage (allowing the bandgap reference to be disabled in this mode).

Ezairo 8300 Hardware Reference

For more information, see [Section 9.4 “Standby Mode”](#) on page 300, [Section 9.1.2.2 “Internal Digital Supply Voltages \(VDDC and VDDM\)”](#) on page 291, and [Section 9.1.2.4 “Analog Supply Voltage \(VDDA\)”](#) on page 295.

9.2.1 Power Supervisory Unit

The power supervisory unit (PSU) monitors the battery supply voltage (VBAT) and the internal analog and digital supply voltages (VDDA, VDDC, VDDM), safely shutting down the system without user intervention when the supply voltages are below the thresholds required for valid system operation. This unit is used to ensure that the system operates correctly and is not damaged by the power supply fluctuations that are encountered when a battery is inserted or removed from a hearing aid.

When power is first supplied to the system, the power supervisory circuitry starts to monitor the battery supply voltage as it crosses the low VBAT threshold. If the voltage continues to rise past the high VBAT voltage threshold, and stays above the high VBAT threshold for an appropriate period of time, the system releases the reset signal and uses the power-on-reset module (see [Section 9.2.2 “Power-On-Reset \(POR\) Sequence”](#)) to ensure proper system startup behavior. This startup sequence is shown in the "Power Supervisory Behavior" figure (Figure 13).

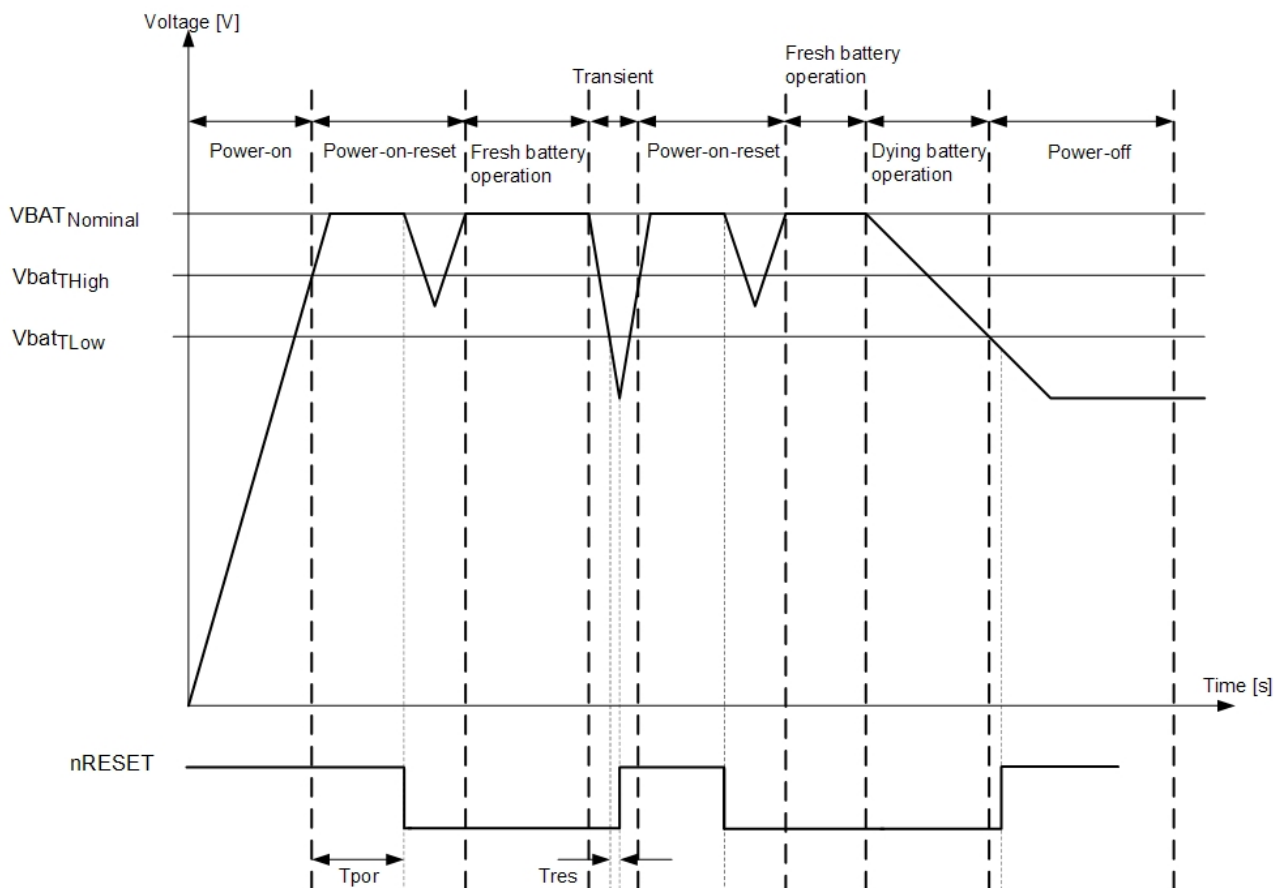


Figure 13. Power Supervisory Behavior

Ezairo 8300 Hardware Reference

If VBAT falls below the low voltage threshold due to a voltage transient or dying battery, the system is returned to a reset state. The system is held in a reset state until the voltage recovers, and VBAT is above the high voltage threshold for the POR delay. These possible transitions are also shown in the "Power Supervisory Behavior" figure (Figure 13). To manually detect the end of battery life, the battery monitor from the LSAD block can be used. See Section 18.5.4 "Power Supply Monitoring" on page 633 for further information.

The relevant threshold voltages and nominal POR delay are supplied in the "Power Supervisory Parameters" table (Table 21).

Table 21. Power Supervisory Parameters

Parameter	Level
Nominal POR Delay	9 ms
VBAT High Threshold	0.77 V $\pm$ 90 mV
VBAT Low Threshold	0.72 V $\pm$ 90 mV

NOTE: The voltage difference between the high and low VBAT voltage thresholds is used to prevent repeated system resets due to voltage oscillation around the low voltage threshold.

9.2.2 Power-On-Reset (POR) Sequence

The Ezairo 8300 device uses a power-on-reset (POR) sequence to ensure proper system behavior during startup, and to ensure proper system configuration after startup.

At the start of the POR sequence, audio output is disabled and all configuration and control registers are asynchronously reset to their default values. At the start of the POR sequence, the core registers for all five of the system's cores are cleared and the contents of all RAM is undefined.

The POR sequence consists of two components: voltage supply stabilization and boot ROM initialization. During the voltage supply stabilization component, the internal VDDC and VDDM regulators are enabled and allowed to stabilize.

The boot ROM initialization component is then run, initializing and enabling the VDDA and VDDIF charge pumps, amongst other components. Following a stabilization delay, the POR sequence is complete, and the device attempts to load data from a flash or EEPROM device using SPI on DIO0 to DIO3, or an eMMC flash using eMMC on DIO0 to DIO2. If no SPI flash or EEPROM is found, the eMMC interface is used in a one data line mode on DIO0 to DIO2 to look for an eMMC flash to boot from. For more information about:

- The boot ROM, see the Boot ROM section of the *Ezairo 8300 Firmware Reference*
- The SPI, see Section 18.7 "Serial Peripheral Interfaces (SPI)" on page 656
- The eMMC interface, see Section 18.3 "eMMC Interface" on page 565
- DIO configuration, see Chapter 16.1 "Overview" on page 463

9.3 RESETS

The Ezairo 8300 system contains a variety of reset sources that can be used to reset the whole Ezairo 8300 system, or a set of system components.

Exiting from any full system reset triggers the POR sequence, as described in Section 9.2.2 "Power-On-Reset (POR) Sequence". Full system resets can be triggered by:

Ezairo 8300 Hardware Reference

- The power supervisory
- The watchdog timer (see [Section 19.9 “Watchdog Timer” on page 768](#))
 - The CFX and Arm Cortex-M3 cores can force a watchdog timeout to trigger an immediate reset.
- The clock detector (see [Section 10.1.3.1 “Clock Detector” on page 312](#))
- The dedicated NRESET pad

NOTE: The NRESET pad is an asynchronous active-low pad that is pulled up using a weak pull-up resistor (250 kΩ), supported by a low-pass filter to avoid glitches of less than 50 ns resetting the device.

Partial resets supported by the Ezairo 8300 system include:

- A hard reset of a core, which resets the core registers and potentially the registers for its peripherals and interfaces. These resets can be triggered:
 - For the HEAR and its peripherals (including the FIFOs), by setting the `SYSTEM_CTRL_HEAR_RESET` bit from the `SYSTEM_CTRL` register
 - For the Arm Cortex-M3 processor, its interfaces, and its peripherals, by setting the `SCB_AIRCR_SYSRESETREQ` bit from the `SCB_AIRCR` register
 - For the Filter Engine, by setting the `FENG_CMDS_DECODER_RESET` and `FENG_CMDS_AUDIO_RESET` bits from the `FENG_CMDS` register
 - For the LPDSP32 processor, its interrupt handler, and the LPDSP32 loop cache, by setting the `SYSCTRL_DSP_CTRL_DSP_RESET` bit from the `SYSCTRL_DSP_CTRL` register
- A soft reset of a core, which resets only the core registers for a core. These resets can be triggered:
 - For the CFX and its peripherals, by clearing the CFX SR and executing a branch to address 0x00000.
 - For the Arm Cortex-M3 processor, by triggering a reset interrupt in the NVIC (NVIC interrupt vector 1)
 - For the Arm Cortex-M3 processor, by requesting a reset through the Arm Cortex-M3 processor's debug port, debug monitor, or using the `VECTRESET` bit from the `SCB_AIRCR` register, as described in [Section 4.6.1.6 “Interrupt Priority Registers” on page 195](#)

9.4 STANDBY MODE

The Ezairo 8300 device provides an ultra-low power mode where the cores and other digital components of the system are still active, but are being clocked from a `SYSCLK` that is using the standby clock, and the analog circuitry is disabled. Since the CFX and Arm Cortex-M3 are still active, an event on the GPIO interface or another system element can trigger a reactivation of the system from standby mode.

As the digital interfaces are still operative in standby mode, disconnect system elements such as the I/Os from the input and output stages when in standby mode to prevent unexpected behavior upon restoration of the normal clock. The `D_I2C` interface bus is disconnected by default when in standby mode, unless the `D_I2C_CFG_CONNECT_IN_STANDBY` bit from the `D_I2C_CFG` register is set (for more information about configuring the I²C debug ports and their bus, see [Chapter 17.1 “Debug” on page 523](#)). Other interfaces that are not explicitly disabled remain active, although they cannot operate at their typical communication frequencies since all interface clocks are scaled down proportionally with `SYSCLK`.

Use the following sequence of tasks to enter standby mode:

1. Configure the `ANALOG_CP_VDDIF_CTRL_VDDA_CONNECT` bit of the `ANALOG_CP_VDDIF_CTRL` register as desired. See [Section 9.1.3.1 “Regulated Interface Supply Voltage \(VDDIF\)” on page 296](#) for more information.
2. Disable the LSAD by clearing the `LSAD_CFG_FREQ` bit field from the `LSAD_CFG` register.

Ezairo 8300 Hardware Reference

3. Configure the VDDA charge pump in low power mode and apply the desired voltage trimming for standby mode. The VDDA clock divider has no impact in standby mode and can be kept as is. For information on how to configure the VDDA charge pump, see [Section 9.1.2.4 “Analog Supply Voltage \(VDDA\)” on page 295](#).
4. If adaptive voltage scaling is used, then disable it by writing `VDDC_TRIM_MANUAL` to `ANALOG_CP_VDDC_AUTO_CTRL` and restoring the high VDDC trimming value (typically stored in the `MAX_TRIM` bit field of `ANALOG_CP_VDDC_AUTO_LIMITS`).
5. Disable the VDDC and VDDM charge pumps by clearing the `ANALOG_CP_VDDC_CTRL_ENABLE` and `ANALOG_CP_VDDM_CTRL_ENABLE` bits from the `ANALOG_CP_VDDC_CTRL` and `ANALOG_CP_VDDM_CTRL` registers, respectively.
6. Apply the desired VDDM voltage trimming for standby mode now, if the current trimming is lower than desired. Refer to [Section 9.1.2.2 “Internal Digital Supply Voltages \(VDDC and VDDM\)” on page 291](#) for information on VDDM voltage trimming.
7. Prepare the disabling of the analog blocks by setting the `ANALOG_PWR_CTRL_ANALOG_DISABLE` bit in the `ANALOG_PWR_CTRL` control register.
8. Prepare to disable the internal oscillator (CCO) multiplier by clearing the `ANALOG_OSC_CTRL_1_ENABLE` bit field from the `ANALOG_OSC_CTRL_1` register.
9. Activate the standby mode by switching the source for `SYSCLK` to the standby clock. For more information about changing the source of `SYSCLK`, see [Section 10.2.1 “System Clock \(SYSCLK\)” on page 313](#). This step will power down the bandgap, VREG regulator, and VDDIF charge pump.
10. Change the VDDM and VDDC voltage trimmings to those for standby mode. Refer to [Section 9.1.2.2 “Internal Digital Supply Voltages \(VDDC and VDDM\)” on page 291](#) for information on trimming VDDM and VDDC.

In standby mode VREG is disabled and shorted to VBAT.

The current consumption in standby mode can be further lowered by putting the memories into retention mode. This can be achieved using the ROM functions in CFX or CM3. See the section on Program ROM in the *Ezairo 8300 Firmware Reference* for the ROM function tables. These functions can keep the memories in retention mode until an interrupt wakes the core up. While in retention mode, the secondary core needs to be in wait for interrupt state, and it cannot be woken up by the same interrupt as it takes time for the ROM routine to power up the memories again. The memory retention ROM functions take the VDDM voltage trimming for the retention mode as an input, and they restore the previous VDDM voltage trimming on exit.

To revert back to run mode from standby mode, perform the following steps:

1. Change the VDDM voltage trimming to that for run mode now, if the target trimming is higher than the current trimming.
2. Enable the analog blocks by clearing the `ANALOG_PWR_CTRL_ANALOG_DISABLE` bit in the `ANALOG_PWR_CTRL` control register.
3. Change the VDDC voltage trimming to that for run mode.
4. Configure the VDDA charge pump for the desired run mode, and update the current and voltage trimmings accordingly.
5. Switch the system clock source back to the internal oscillator (CCO).
6. Change the VDDM voltage trimming to that for run mode.
7. Enable the LSAD by restoring the previous value to the `LSAD_CFG_FREQ` bit field in the `LSAD_CFG` register.
8. Enable adaptive voltage scaling again (if it was used).

Ezairo 8300 Hardware Reference

9. If the VDDC and VDDM charge pumps were configured to be controlled automatically, they continue to do so now. In the rare case that these charge pumps are controlled by software, the software has to re-enable the charge pumps (it will likely also need LSAD conversion values as VBAT and VDDC might have changed during the standby period).

9.5 CONTROL AND CONFIGURATION REGISTERS

Register Name	Register Description	Address
ANALOG_BG_CTRL	Configuration registers of the bandgap	P:0x0991 0x40002644
ANALOG_PWR_CTRL	Analog power down control	P:0x0992 0x40002648
ANALOG_VMIC_CTRL	VMIC output control	P:0x0993 0x4000264C
ANALOG_VDDOD_CTRL	VDDOD regulator control	P:0x0994 0x40002650
ANALOG_CP_VDDA_CTRL	Analog charge pump configuration registers	P:0x09A0 0x40002680
ANALOG_CP_VDDIF_CTRL	Interface charge pump configuration registers	P:0x09A1 0x40002684
ANALOG_CP_VDDC_TRIM	Logic supply trimming register	P:0x09A2 0x40002688
ANALOG_CP_VDDC_CTRL	Logic supply control register	P:0x09A3 0x4000268C
ANALOG_CP_VDDC_AUTO_CTRL	Logic supply automatic trim level control	P:0x09AA 0x400026A8
ANALOG_CP_VDDC_AUTO_LIMITS	Logic supply automatic trim level control	P:0x09AB 0x400026AC
ANALOG_CP_VDDM_TRIM	Memory supply trimming register	P:0x09A4 0x40002690

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
ANALOG_CP_VDDM_CTRL	Memory supply control register	P:0x09A5 0x40002694
ANALOG_CP_VDDCM_REF	Logic and memory supply reference selection	P:0x09A6 0x40002698
ANALOG_CP_VDDCM_CFG	Digital charge pump configuration	P:0x09A9 0x400026A4

9.5.1 ANALOG_BG_CTRL

Bit Field	Read/Write	Field Name	Description
13	RW	BG_VTEMP_ENABLE	Temperature dependent voltage enable
12:8	RW	BG_TEMP_COEF_TRIM	Temperature coefficient trimming control
4:0	RW	BG_LEVEL	Reference voltage trimming control

Field Name	Value Symbol	Value Description	Hex Value
BG_VTEMP_ENABLE	BG_VTEMP_DISABLE	Temperature dependent voltage disabled	0x0*
	BG_VTEMP_ENABLE	Temperature dependent voltage enabled	0x1
BG_TEMP_COEF_TRIM	BG_TEMP_COEF_OPTIMAL	Optimal temperature coefficient	0xF
BG_LEVEL	BG_TRIM_NOMINAL	0.9 V (nominal value uncalibrated)	0x11

9.5.2 ANALOG_PWR_CTRL

Bit Field	Read/Write	Field Name	Description
0	RW	ANALOG_DISABLE	Analog power down control

Field Name	Value Symbol	Value Description	Hex Value
ANALOG_DISABLE	ANALOG_ENABLE	Analog blocks are enabled	0x0*
	ANALOG_DISABLE	All analog blocks except the PMU are disabled	0x1

Ezairo 8300 Hardware Reference

9.5.3 ANALOG_VMIC_CTRL

Bit Field	Read/Write	Field Name	Description
8	RW	VMIC_REG_PS	VMIC regulator power supply
7:3	RW	VMIC_REG_TRIM	VMIC regulator trimming
2	RW	VMIC_REG_EN	Enable of VMIC regulator
1:0	RW	VMIC_MUX_CTRL	Selection of the VMIC output signal

Field Name	Value Symbol	Value Description	Hex Value
VMIC_REG_PS	VMIC_REG_PS_VDDA	VMIC regulator is powered by VDDA	0x0*
	VMIC_REG_PS_VBAT	VMIC regulator is powered by VBAT	0x1
VMIC_REG_TRIM	VMIC_REG_TRIM_0P80V	0.8 V (uncalibrated)	0x0
	VMIC_REG_TRIM_0P825V	0.825 V (uncalibrated)	0x1
	VMIC_REG_TRIM_1P15V	1.15 V (uncalibrated)	0xE*
	VMIC_REG_TRIM_1P55V	1.55 V (uncalibrated)	0x1E
VMIC_REG_EN	VMIC_REG_TRIM_1P575V	1.575 V (uncalibrated)	0x1F
	VMIC_REG_DISABLE	VMIC regulator is disabled (hi-Z)	0x0*
	VMIC_REG_ENABLE	VMIC regulator is enabled	0x1
	VMIC_MUX_CTRL	VMIC output is from VMIC regulator; Output is hi-Z if regulator is disabled	0x0*
VMIC_MUX_CTRL	VMIC_SW_VMIC	VMIC output is from VMIC regulator; Output is hi-Z if regulator is disabled	0x0*
	VMIC_SW_VREG	VMIC output is VREG	0x1
	VMIC_SW_VDDA	VMIC output is VDDA	0x2
	VMIC_SW_VDDIF	VMIC output is VDDIF	0x3

9.5.4 ANALOG_VDDOD_CTRL

Bit Field	Read/Write	Field Name	Description
5:1	RW	VDDOD_REG_TRIM	VDDOD regulator trimming
0	RW	VDDOD_REG_EN	Enable of VDDOD regulator

Field Name	Value Symbol	Value Description	Hex Value
VDDOD_REG_TRIM	VDDOD_REG_TRIM_0P80V	0.8 V (uncalibrated)	0x0*
	VDDOD_REG_TRIM_0P825V	0.825 V (uncalibrated)	0x1
	VDDOD_REG_TRIM_1P15V	1.15 V (uncalibrated)	0xE

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	VDDOD_REG_TRIM_1P55V	1.55 V (uncalibrated)	0x1E
	VDDOD_REG_TRIM_1P575V	1.575 V (uncalibrated)	0x1F
VDDOD_REG_EN	VDDOD_REG_DISABLE	VDDOD regulator is disabled (tied to VBAT)	0x0
	VDDOD_REG_ENABLE	VDDOD regulator is enabled	0x1*

9.5.5 ANALOG_CP_VDDA_CTRL

Bit Field	Read/Write	Field Name	Description
12	RW	HIGH_POWER	Select high-power mode
11	RW	STANDBY	Select charge pump clock and reference sources
10:6	RW	ITRIM	Current trimming of the analog charge pump comparator
5:0	RW	VTRIM	CP output voltage trimming configuration at a step of 6.5 mV

Field Name	Value Symbol	Value Description	Hex Value
HIGH_POWER	VDDA_HIGH_POWER_DISABLE	Disable the charge pump high-power mode	0x0
	VDDA_HIGH_POWER_ENABLE	Enable the charge pump high-power mode	0x1*
ITRIM	VDDA_ITRIM_LP_0P5MA	Bias current for low-power 0.5 mA mode	0x9
	VDDA_ITRIM_HP_4MA	Bias current for high-power 4 mA mode	0x12*
VTRIM	VDDA_VTRIM_1P57V	Charge pump outputs 1.57 V (uncalibrated)	0x0
	VDDA_VTRIM_1P8V	Charge pump outputs 1.80 V (uncalibrated)	0x21*
	VDDA_VTRIM_1P98V	Charge pump outputs 1.98 V (uncalibrated)	0x3F

9.5.6 ANALOG_CP_VDDIF_CTRL

Bit Field	Read/Write	Field Name	Description
14	RW	VDDA_CONNECT	VDDIF to VDDA connection configuration
13	RW	HIGH_POWER	Select high-power mode
12	RW	ENABLE	Enables interface charge pump
10:6	RW	ITRIM	Current trimming of the interface charge pump comparator
5:0	RW	VTRIM	CP output voltage trimming configuration at a step of 6.5 mV

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
VDDA_CONNECT	VDDIF_VDDA_DISCONNECT	Never connect VDDIF to VDDA	0x0
	VDDIF_VDDA_AUTO_CONNECT	Automatically connect VDDIF to VDDA when VDDIF is disabled manually or when the entire analog part is disabled (standby mode)	0x1*
HIGH_POWER	VDDIF_HIGH_POWER_DISABLE	Disable the charge pump high-power mode	0x0
	VDDIF_HIGH_POWER_ENABLE	Enable the charge pump high-power mode	0x1*
ENABLE	VDDIF_DISABLE	Disable the charge pump	0x0
	VDDIF_ENABLE	Enable the charge pump	0x1*
ITRIM	VDDIF_ITRIM_LP_1MA	Bias current for low-power 1 mA mode	0x0
	VDDIF_ITRIM_HP_5MA	Bias current for high-power 5 mA mode	0x8
	VDDIF_ITRIM_HP_8MA	Bias current for high-power 8 mA mode	0xC
	VDDIF_ITRIM_HP_10MA	Bias current for high-power 10 mA mode	0x10*
	VDDIF_ITRIM_HP_15MA	Bias current for high-power 15 mA mode	0x10*
VTRIM	VDDIF_VTRIM_1P57V	Charge pump outputs 1.57 V (uncalibrated)	0x0
	VDDIF_VTRIM_1P8V	Charge pump outputs 1.80 V (uncalibrated)	0x23*
	VDDIF_VTRIM_1P98V	Charge pump outputs 1.98 V (uncalibrated)	0x3F

9.5.7 ANALOG_CP_VDDC_TRIM

Bit Field	Read/Write	Field Name	Description
13:8	RW	CP_DELTA_TRIM	Charge pump output voltage trimming in 1.7 mV steps (from -54 mV to 52 mV)
7:0	RW	LDO_TRIM	Linear regulator output voltage trimming in 1.7 mV steps

Field Name	Value Symbol	Value Description	Hex Value
CP_DELTA_TRIM	VDDC_CP_DELTA_0MV	CP = LDO + 0 mV (uncalibrated)	0x0
	VDDC_CP_DELTA_20MV	CP = LDO + 20 mV (uncalibrated)	0xC*
	VDDC_CP_DELTA_52MV	CP = LDO + 52 mV (uncalibrated)	0x1F

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	VDDC_CP_DELTA_MINUS_54mV	CP = LDO - 54 mV (uncalibrated)	0x20
LDO_TRIM	VDDC_LDO_0P45V	0.45 V (uncalibrated)	0x0
	VDDC_LDO_0P80V	0.80 V (uncalibrated)	0xD0*
	VDDC_LDO_0P88V	0.88 V (uncalibrated)	0xFF

9.5.8 ANALOG_CP_VDDC_CTRL

Bit Field	Read/Write	Field Name	Description
6:4	RW	CP_MODE	Charge pump mode
1	RW	AUTO_CTRL	Enable automatic charge pump ratio control (requires LSAD to be enabled)
0	RW	ENABLE	Charge pump enable control

Field Name	Value Symbol	Value Description	Hex Value
CP_MODE	VDDC_RATIO_1_3	1/3 ratio mode	0x0*
	VDDC_RATIO_1_2	1/2 ratio mode	0x1
	VDDC_RATIO_2_3	2/3 ratio mode	0x2
	VDDC_RATIO_3_4	3/4 ratio mode	0x3
	VDDC_RATIO_4_5	4/5 ratio mode	0x4
AUTO_CTRL	VDDC_CTRL_MANUAL	Manual control of CP ratio and enable	0x0*
	VDDC_CTRL_AUTO	Automatic control of CP ratio and enable	0x1
ENABLE	VDDC_DISABLE	Disable the charge pump	0x0*
	VDDC_ENABLE	Enable the charge pump	0x1

9.5.9 ANALOG_CP_VDDC_AUTO_CTRL

Bit Field	Read/Write	Field Name	Description
19:16	RW	MIN_MARGIN	Minimum timing margin
3	RW	ENABLE	VDDC automatic trim enable

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
MIN_MARGIN	VDDC_MIN_MARGIN_ALL_ EXCEPT_CCO_MULT_4X		0x8
	VDDC_MIN_MARGIN_CCO_MULT_ 4X		0x9
ENABLE	VDDC_TRIM_MANUAL	Manual control of VDDC trim level	0x0*
	VDDC_TRIM_AUTO	Automatic control of VDDC trim level	0x1

9.5.10 ANALOG_CP_VDDC_AUTO_LIMITS

Bit Field	Read/Write	Field Name	Description
15:8	RW	MAX_TRIM	Maximum VDDC trimming value (the automatic trim level control will not go above this trimming value)
7:0	RW	MIN_TRIM	Minimum VDDC trimming value (the automatic trim level control will not go below this trimming value)

9.5.11 ANALOG_CP_VDDM_TRIM

Bit Field	Read/Write	Field Name	Description
13:8	RW	CP_DELTA_TRIM	Charge pump output voltage trimming in 1.7 mV steps (from -54 mV to 52 mV)
7:0	RW	LDO_TRIM	Linear regulator output voltage trimming in 1.7 mV steps

Field Name	Value Symbol	Value Description	Hex Value
CP_DELTA_TRIM	VDDM_CP_DELTA_0MV	CP = LDO + 0 mV (uncalibrated)	0x0
	VDDM_CP_DELTA_20MV	CP = LDO + 20 mV (uncalibrated)	0xC*
	VDDM_CP_DELTA_52MV	CP = LDO + 52 mV (uncalibrated)	0x1F
	VDDM_CP_DELTA_MINUS_54MV	CP = LDO - 54 mV (uncalibrated)	0x20
LDO_TRIM	VDDM_LDO_0P45V	0.45 V (uncalibrated)	0x0
	VDDM_LDO_0P80V	0.80 V (uncalibrated)	0xD0*
	VDDM_LDO_0P88V	0.88 V (uncalibrated)	0xFF

Ezairo 8300 Hardware Reference

9.5.12 ANALOG_CP_VDDM_CTRL

Bit Field	Read/Write	Field Name	Description
6:4	RW	CP_MODE	Charge pump mode
1	RW	AUTO_CTRL	Enable automatic charge pump control (requires LSAD to be enabled)
0	RW	ENABLE	Charge pump enable control

Field Name	Value Symbol	Value Description	Hex Value
CP_MODE	VDDM_RATIO_1_3	1/3 ratio mode	0x0*
	VDDM_RATIO_1_2	1/2 ratio mode	0x1
	VDDM_RATIO_2_3	2/3 ratio mode	0x2
	VDDM_RATIO_3_4	3/4 ratio mode	0x3
	VDDM_RATIO_4_5	4/5 ratio mode	0x4
AUTO_CTRL	VDDM_CTRL_MANUAL	Manual control of CP ratio and enable	0x0*
	VDDM_CTRL_AUTO	Automatic control of CP ratio and enable	0x1
ENABLE	VDDM_DISABLE	Disable the charge pump	0x0*
	VDDM_ENABLE	Enable the charge pump	0x1

9.5.13 ANALOG_CP_VDDCM_REF

Bit Field	Read/Write	Field Name	Description
0	RW	VDDCM_REF_SEL	Select digital regulators reference selection

Field Name	Value Symbol	Value Description	Hex Value
VDDCM_REF_SEL	VDDCM_REF_PMU	Always use PMU reference	0x0
	VDDCM_REF_AUTO	Use bandgap reference except when the analog part is disabled (standby mode)	0x1*

9.5.14 ANALOG_CP_VDDCM_CFG

Bit Field	Read/Write	Field Name	Description
11:0	RW	AUTO_CTRL	Digital charge pump automatic charge pump ratio configuration

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
AUTO_CTRL	CP_VDDCM_CFG_LOW_POWER	Optimal charge pump configuration for lowest power	0xA9*
	CP_VDDCM_CFG_QUIET	Optimal charge pump configuration for minimal noise (avoid excessive enabling/disabling of charge pumps)	0x5A9

CHAPTER 10

Clock Components

All clocks and clock domains in the Ezairo 8300 system are derived from the system clock (SYSCLK). Because of its importance to the functioning of Ezairo 8300 and the execution of applications, this clock signal can be generated in multiple ways for maximum flexibility.

10.1 CLOCK GENERATION

The following are available as sources for the system clock:

- The internal CCO-based primary oscillator (discussed in [Section 10.1.1 “Current-Controlled Oscillator”](#))
- The internal standby oscillator (discussed in [Section 10.1.2 “Standby Oscillator”](#) on page 311)
- The external clock pad (discussed in [Section 10.1.3 “External Clock \(EXTCLK\)”](#) on page 312)

10.1.1 Current-Controlled Oscillator

The primary oscillator on Ezairo 8300 is a current-controlled oscillator (CCO). The oscillator is automatically enabled when it is selected as the system clock source, providing a clock with a nominal frequency of 7.68 MHz in its default configuration.

The base frequency of the CCO is set using the `ANALOG_OSC_CTRL_0_COARSE_FREQ_CTRL` and `ANALOG_OSC_CTRL_0_FINE_FREQ_CTRL` bit fields from the `ANALOG_OSC_CTRL_0` register. This base frequency is multiplied by 1, 2 or 4 using the `ANALOG_OSC_CTRL_1_MULT` bit field from the `ANALOG_OSC_CTRL_1` register.

NOTE: Although the coarse and fine trimmings are both monotonic in their respective ranges, multiple trim value combinations can exist for each frequency. Consequently, the `ANALOG_OSC_CTRL_0_COARSE_FREQ_CTRL` bit field of the `ANALOG_OSC_CTRL_0` register must be trimmed first, followed by the `ANALOG_OSC_CTRL_0_FINE_FREQ_CTRL` bit field, to accurately calibrate the oscillator.

CAUTION: Multiplying the base frequency of primary oscillator by three is not supported. Configuring the `ANALOG_OSC_CTRL_1_MULT` bit field from the `ANALOG_OSC_CTRL_1` register for a multiplication factor of 3 results in undefined system behavior.

IMPORTANT: To minimize power consumption, use of higher CCO multipliers (2x, 4x) are recommended over higher base frequency calibrations of the CCO.

10.1.2 Standby Oscillator

The Ezairo 8300 system includes a second oscillator to allow for low frequency operation in standby mode. When operating in this mode, the power consumption is greatly reduced; however, the system is still executing instructions and can be woken up to regular operation mode. When the standby clock is selected as the source for the system clock ([Section 10.2.1 “System Clock \(SYSCLK\)”](#) on page 313), the standby oscillator provides a 25 kHz clock signal.

Ezairo 8300 Hardware Reference

10.1.3 External Clock (EXTCLK)

The DIO35EXTCLK pad can be used as an external input clock (EXTCLK) to the system, which can be selected as the source for SYSCLK. To use EXTCLK as a clock input, configure DIO35 must be configured as an input. For more information about the DIO configuration, see [Chapter 16.1 "Overview" on page 463](#).

The clock signal provided to EXTCLK is passed through a prescaling integer clock divider before being used as a source for the SYSCLK. To set the divisor used for this clock division, configure the CLK_CFG_0_EXTCLK_PRESCALE bit field in the CLK_CFG_0 register.

NOTE: Reconfiguring from a divided setting to an undivided setting for the divisor is not allowed while EXTCLK is selected as the source for SYSCLK. When switching from a divided setting to an undivided one, an alternate source must be selected for SYSCLK.

10.1.3.1 Clock Detector

The external clock has a clock detector which can be used to detect when a clock signal appears or disappears on the EXTCLK pad. The clock detector is disabled by default. It can be enabled by setting the CLK_EXTCLK_DET_CFG_CLK_DET_ENABLE bit from the CLK_EXTCLK_DET_CFG register.

NOTE: Note: The clock detector is guaranteed to detect an EXTCLK input signal of at least 25 kHz

There are two ways to check whether the EXTCLK is active or not:

- By polling the CLK_EXTCLK_DET_CFG_CLK_DET_STATUS bit from the CLK_EXTCLK_DET_STATUS register
- Using the CLK_DET interrupt which is configured by writing to the CLK_EXTCLK_DET_CFG_CLK_DET_INT_SEL bit field from the CLK_EXTCLK_DET_CFG register

The clock detector always starts in an inactive state when enabled and it takes two standby clock cycles for the status to be updated. This means the interrupt can be generated shortly after enabling the clock detector when an EXTCLK signal is present. The interrupt can also be configured to trigger for when EXTCLK goes inactive or for any clock status change. This can be done by writing CLK_DET_INT_DEACTIVATED or CLK_DET_INT_ACTIVITY_CHANGE, respectively, to the CLK_EXTCLK_DET_CFG_CLK_DET_INT_SEL bit field in the CLK_EXTCLK_DET_CFG register.

10.2 CLOCK DISTRIBUTION

As noted in [Section 10.1 "Clock Generation" on page 311](#), the system clock (SYSCLK) is used as the source of all the other internal clocks and clock domains. The connections of the clock distribution network that provides SYSCLK in its original or divided forms can be seen in the ["System Clock Distribution" figure \(Figure 14\)](#).

Ezairo 8300 Hardware Reference

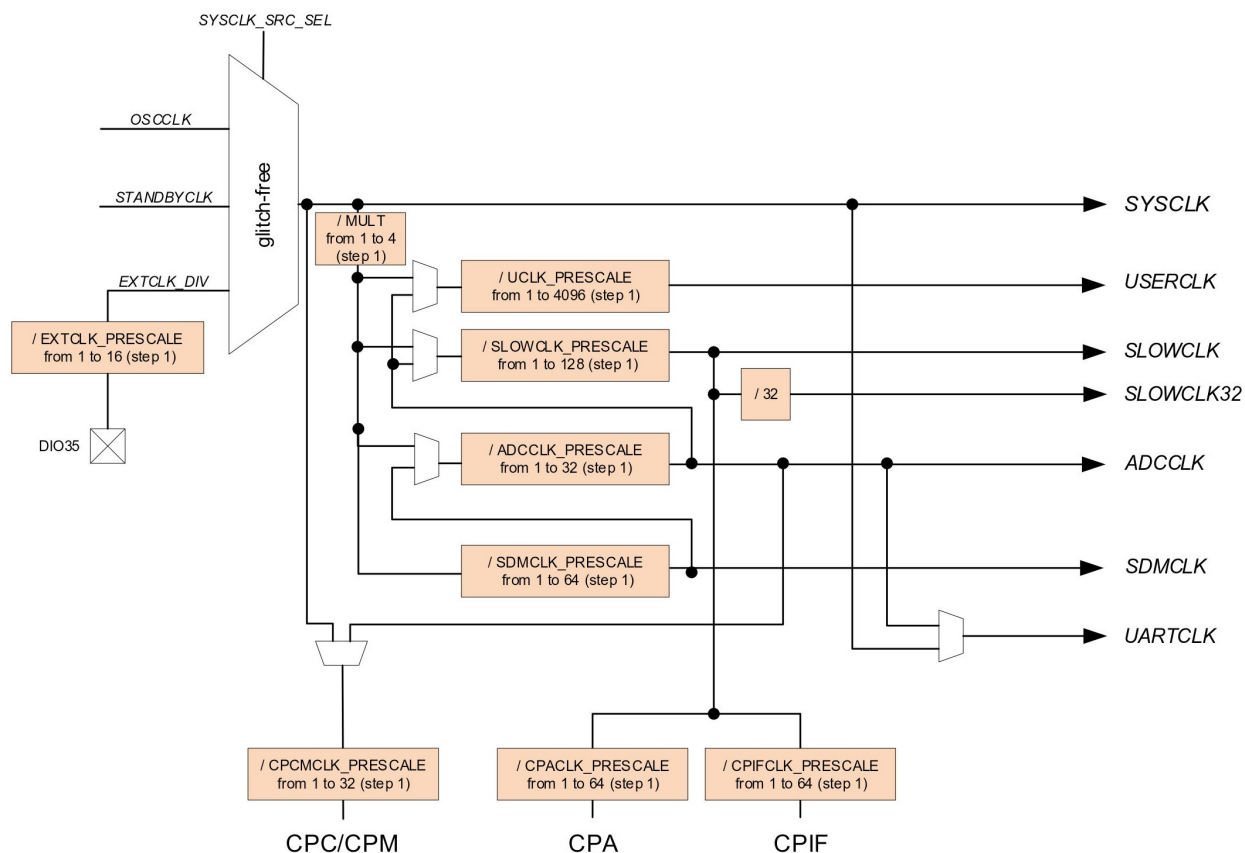


Figure 14. System Clock Distribution

NOTE: The divisor by MULT shown in the "System Clock Distribution" figure (Figure 14) shows the effect of bypassing the frequency multiplier of the primary oscillator when SYSCLK is sourced from the primary oscillator and multiplier-based clock throttling is used. For more information about this option, see the following sections.

Where a clock divider is used for only one component of the Ezairo 8300 device and has no side effects that change the behavior of other system components, further information about that clock divider can be found in the components description elsewhere in this group of topics. A description is provided in the following subsections for all the divided internal clocks and clock domains that can affect more than one component of the system.

10.2.1 System Clock (SYSCLK)

The system clock is the primary clock for the system and all other clocks in the Ezairo 8300 system are derived from this clock. SYSCLK is generated with a near 50% duty-cycle. All of the derivative clocks are also synchronously enabled.

Ezairo 8300 Hardware Reference

By default, this clock uses the clock signal generated by the current-controlled oscillator (CCO); however, you can also configure it to be sourced from the standby oscillator or the external clock. To select the standby oscillator or the external clock as the source for SYSCLK, configure the `CLK_CFG_0_SYSCLK_SEL` bit field from the `CLK_CFG_0` register. Configuring the external clock is further described in [Section 10.1.3 “External Clock \(EXTCLK\)” on page 312](#).

SYSCLK can be output through the digital I/O pads including the EXTCLK pad (DIO35). For more information about the DIO configuration, see [Chapter 16.1 “Overview” on page 463](#).

IMPORTANT: For all system clock frequencies, the VDDC digital supply voltage must be set appropriately to ensure that the system's digital logic and memories operate correctly. For more information about setting this supply voltage, see [Section 9.1.2.2 “Internal Digital Supply Voltages \(VDDC and VDDM\)” on page 291](#).

IMPORTANT: To minimize power consumption, set SYSCLK to the lowest frequency needed to support the required processing on all processors.

10.2.2 Sigma-Delta Modulator Clock (SDMCLK)

The sigma-delta modulator clock is used by the sigma-delta modulators of the audio output channels.

This clock derives from SYSCLK and uses a 6-bit integer divisor to scale SYSCLK to the correct sigma-delta modulator clock frequency. Set the value of this divisor by using the `CLK_CFG_1_SDMCLK_PRESCALE` bit field in the `CLK_CFG_1` register.

NOTE: For optimal audio and power performance, match SDMCLK to the unmultiplied CCO's frequency or an integer division of that oscillator. Recommended SDMCLK frequencies are 3.84 MHz (with `AUDIO_SDM_CTRL_SDM_CLK_RATIO` set), 7.68 MHz, and 15.36 MHz. For more information, see [Section 15.3 “Sigma-Delta Modulator” on page 457](#) and the Ezairo 8300 datasheet.

SDMCLK can be output through one or more digital I/O pads using the DIO components. For more information about the DIO configuration, see [Chapter 16.1 “Overview” on page 463](#).

NOTE: SDMCLK does not have a 50% duty cycle when it is divided from SYSCLK by an odd value. SDMCLK is equal to SYSCLK when both `ADDCLK_PRESCALE_1` and `SDMDLK_EBL` are used. Neither of these points has any effect on the correct behavior of the output driver.

If SYSCLK is sourced from the primary oscillator the `CLK_MULT_THROTTLING_CFG_SDMCLK_EBL` bit in the `CLK_MULT_THROTTLING_CFG` register can be set to derive the sigma-delta modulator clock directly from the primary oscillator instead of from SYSCLK. This configuration bypasses the frequency multiplier of the primary oscillator, ensuring that the sigma-delta modulator clock is unaffected by changes to this multiplier. For more information about multiplier-based clock throttling, see [Section 10.3.1 “Multiplier-Based Clock Throttling” on page 317](#).

The `CLK_CFG_1_SDMCLK_PRESCALE` bit field in the `CLK_CFG_1` register can be updated using the `CLK_SDMCLK_THROTTLING` register. This register is used as part of the clock throttling feature, which can be used to avoid artifacts in the audio stream. For information about frequency-based clock throttling, see [Section 10.3.2 “Frequency-Based Clock Throttling” on page 318](#).

IMPORTANT: To minimize power consumption, enable multiplier-based clock throttling for SDMCLK.

Ezairo 8300 Hardware Reference

10.2.3 ADC Clock (ADCCLK)

The ADCCLK is used by the analog and digital input related components of the system. This includes:

- All components of the input stages
- Interpolation filters of the output stages
- DMIC outputs

Either SYSCLK or SDMCLK can be selected as the source for ADCCLK, by configuring the CLK_CFG_1_ADCCLK_SRC bit from the CLK_CFG_1 register. ADCCLK is derived from this clock through a 5-bit integer division by the CLK_CFG_1_ADCCLK_PRESCALE bit field in the CLK_CFG_1 register.

NOTE: For optimal performance, the typical ADCCLK frequency is 3.84 MHz. For more information, see the Ezairo 8300 datasheet.

NOTE: ADCCLK cannot be Depending on how ADCCLK is derived from SYSCLK, ADCCLK may not have a 50% duty cycle clock. Although this does not affect the ADC sampling, it can be an issue if ADCCLK is used as an output clock.

ADCCLK can be output through the digital I/O pads. For more information about the DIO configuration, see [Chapter 16.1 "Overview" on page 463](#).

If SYSCLK is sourced from the primary oscillator and divided (CLK_CFG1_ADCCLK_PRESCALE is not set to ADCCLK_PRESCALE_1), the CLK_MULT_THROTTLING_CFG_ADCCLK_EBL bit in the D_CLK_MULT_THROTTLING_CFG register can be set to derive the ADCCLK directly from the primary oscillator instead of from SYSCLK (this configuration is only available when SYSCLK is selected as the source for ADCCLK using the CLK_CFG_1_ADCCLK_SRC bit from the CLK_CFG_1 register). This configuration bypasses the frequency multiplier of the primary oscillator, ensuring that the ADCCLK is unaffected by changes to this multiplier. For more information about multiplier-based clock throttling, see [Section 10.3.1 "Multiplier-Based Clock Throttling" on page 317](#).

IMPORTANT: ADCCLK must be divided at least by two from SYSCLK when multiplier-based clock throttling is used, regardless of the source clock selected for ADCCLK using the CLK_CFG_1_ADCCLK_SRC bit from the CLK_CFG_1 register.

The CLK_CFG_1_ADCCLK_PRESCALE bit field from the CLK_CFG_1 register can be updated using the CLK_ADCCLK_THROTTLING register. This register is used as part of the clock throttling feature, which can be used to avoid artifacts in the audio stream. For information about frequency-based clock throttling, see [Section 10.3.2 "Frequency-Based Clock Throttling" on page 318](#).

IMPORTANT: To minimize power consumption, select SDMCLK as the source for ADCCLK.

10.2.4 Slow Clock (SLOWCLK)

The slow clock is used by the power and timer related components of the system. This includes:

- The general-purpose, watchdog, and SYSTICK timers
- Various power components of the system: LSAD, analog charge pump (CPA), digital interfaces charge pump (CPIF), and digital power-on-reset (POR) delay

Ezairo 8300 Hardware Reference

This clock derives from SYSCLK or ADCCLK. Selecting ADCCLK as the source allows SLOWCLK to clock slower or not at all to save power. The CLK_CFG_1_SLOWCLK_SRC bit in the CLK_CFG_1 register can be set to SLOWCLK_SRC_ADCCLK to configure ADCCLK as the source for SLOWCLK. SLOWCLK uses a 7-bit integer divisor to scale down to the correct clock frequency. The value of this divisor can be set by using the CLK_CFG_1_SLOWCLK_PRESCALE bit field in the CLK_CFG_1 register.

IMPORTANT: The system has been optimized for a SLOWCLK frequency of 1.28 MHz. Use of SLOWCLK frequencies that are significantly different might result in unexpected or incorrect behavior, and are not supported.

NOTE: The SLOWCLK_DIV32 divider is disabled so that it runs at the SYSCLK rate when the STANDBYCLK is selected as the SYSCLK source.

SLOWCLK can be output through the digital I/O pads. For more information about the DIO configuration, see [Chapter 16.1 "Overview" on page 463](#).

If SYSCLK is sourced from the primary oscillator and divided (CLK_CFG1_SLOWCLK_PRESCALE is not set to SLOWCLK_PRESCALE_1), the CLK_MULT_THROTTLING_CFG_SLOWCLK_EBL bit in the D_CLK_MULT_THROTTLING_CFG register can be set to derive the slow clock directly from the primary oscillator instead of from SYSCLK. This configuration bypasses the frequency multiplier of the primary oscillator, ensuring that the slow clock is unaffected by changes to this multiplier. For more information about multiplier-based clock throttling, see [Section 10.3.1 "Multiplier-Based Clock Throttling" on page 317](#).

The CLK_CFG_1_SLOWCLK_PRESCALE bit field from the CLK_CFG_1 register can be updated using the CLK_SLOWCLK_THROTTLING register. This register is used as part of the clock throttling feature, which can be used to avoid artifacts in the audio stream. For information about frequency-based clock throttling, see [Section 10.3.2 "Frequency-Based Clock Throttling" on page 318](#).

IMPORTANT: To minimize power consumption, select ADCCLK as the source for SLOWCLK.

10.2.5 User Clock (UCLK)

The user clock is an output clock that you can use as a clock source for the PCM interfaces or for any external components. This clock is not used internally by the Ezairo 8300 system, so its usage is entirely dependent on the outside needs of the larger system containing Ezairo 8300.

Either SYSCLK or ADCCLK can be selected as the source for UCLK, by configuring the CLK_CFG_2_UCLK_SRC bit from the CLK_CFG_2 register. UCLK is derived from this clock through a 12-bit integer division by the CLK_CFG_2_UCLK_PRESCALE bit field in the CLK_CFG_2 register.

UCLK can be output through one or more digital I/O pads using the DIO components. For more information about the DIO configuration, see [Chapter 16.1 "Overview" on page 463](#).

If SYSCLK is sourced from the primary oscillator and UCLK is divided from SYSCLK (CLK_CFG_2_UCLK_PRESCALE is not set to UCLK_PRESCALE_1 and CLK_CFG_2_UCLK_SRC is set to UCLK_SRC_SYSCLK), the CLK_MULT_THROTTLING_CFG_UCLK_EBL bit in the CLK_MULT_THROTTLING_CFG register can be set to derive the user clock directly from the primary oscillator instead of from SYSCLK. This configuration bypasses the frequency multiplier of the primary oscillator, ensuring that UCLK is unaffected by changes to this multiplier. For more information about multiplier-based clock throttling, see [Section 10.3.1 "Multiplier-Based Clock Throttling" on page 317](#).

IMPORTANT: For frequency-based clock throttling, the UCLK frequency changes with its source clock frequency. To prevent changes to UCLK when using the frequency-based clock throttling, ADCCLK must be used as the source for UCLK.

IMPORTANT: UCLK does not have a 50% duty cycle if the UCLK prescaler is configured for an odd division from SYSCLK.

10.2.6 UART Clock (UARTCLK)

The UARTCLK is used by the UART interface as the reference clock. It directly impacts the possible baud rates and the baud rate error for the interface. Further information about UARTCLK's relationship with the UART interface can be read in [Section 18.8 “Universal Asynchronous Receiver-Transmitter \(UART\) Interfaces” on page 669](#).

Either SYSCLK or ADCCLK can be selected as the source for UARTCLK by configuring the CLK_CFG_1_UARTCLK_SRC bit from the CLK_CFG_1 register.

IMPORTANT: When clock throttling is used (either frequency- or multiplier-based), the UARTCLK frequency changes with its source clock frequency, except when its source clock is ADCCLK.

10.3 CLOCK THROTTLING

When the primary oscillator is selected as the source for SYSCLK, the frequency of SYSCLK can be throttled to accommodate changes to the processing needs of the algorithms used by the user application. Clock throttling is specifically designed to allow the dynamic frequency scaling of the system clock without audible artifacts.

When using clock throttling, the frequency of SYSCLK and of the clocks derived from SYSCLK (except for ADCCLK, SLOWCLK, and SDMCLK, UCLK if it is derived from ADCCLK, and UARTCLK if it is derived from ADCCLK) can be increased or decreased without affecting the audio path (supplied using ADDCLK, SLOWCLK, and SDMCLK). Additionally, multiplier-based clock throttling can be used to prevent the effects of this frequency change on UCLK when this clock is not sourced from ADCCLK.

IMPORTANT: Clock throttling is intended to support the changing needs of an algorithm by reducing or increasing the number of clock cycles between sample blocks. While it is theoretically possible to switch the clock at rates approaching the block sample rate, periodic switching of the clock at a rate close to the audio sampling frequency might result in audio artifacts.

10.3.1 Multiplier-Based Clock Throttling

Multiplier-based clock throttling uses the CCO multiplier to adjust the SYSCLK frequency. When multiplier-based clock throttling is enabled for a clock, that clock is divided from the base CCO clock frequency (i.e., those clocks are unaffected by the setting of the OSC_CTRL_1_MULT bit field from ANALOG_OSC_CTRL_1).

IMPORTANT: When using multiplier-based clock throttling, scaling of the SYSCLK frequency is limited to multiples of the base CCO frequency. Despite this limited frequency scaling range, multiplier-based clock throttling is recommended wherever possible as it is guaranteed to not create any artifacts in the audio path.

Ezairo 8300 Hardware Reference

To connect a clock to the base CCO output, multiplier-based clock throttling requires:

- The clock to be divided by at least a factor of two (this limitation applies to SLOWCLK, ADCCLK, and UCLK only).
- The bit corresponding to the specified clock to be set in the `CLK_MULT_THROTTLING_CFG` register, or the clock to use a clock source that is using multiplier-based clock throttling.

Multiplier-based clock throttling can be used for SLOWCLK, SDMCLK, ADCCLK, and UCLK as described below:

- If the frequency of ADCCLK is unchanged when scaling SYSCLK using multiplier-based clock throttling, this mechanism guarantees that no audio artifacts will occur in the input stage. To enable multiplier-based clock throttling of ADCCLK, set the `CLK_MULT_THROTTLING_CFG_ADCCLK_EBL` bit from the `CLK_MULT_THROTTLING_CFG` register, and configure ADCCLK to use any prescaler other than `ADCCLK_PRESCALE_1`.
- If the frequency of SDMCLK is unchanged when scaling SYSCLK using multiplier-based clock throttling, this mechanism guarantees that no audio artifacts will occur in the output stage. To enable multiplier-based clock throttling of SDMCLK, set the `CLK_MULT_THROTTLING_CFG_SDMCLK_EBL` bit from the `CLK_MULT_THROTTLING_CFG` register.
- If the frequency of SLOWCLK is unchanged when scaling SYSCLK using multiplier-based clock throttling, this mechanism guarantees that the accuracy of the timers and the LSAD will not be affected. To enable multiplier-based clock throttling of SLOWCLK, set the `CLK_MULT_THROTTLING_CFG_SLOWCLK_EBL` bit from the `CLK_MULT_THROTTLING_CFG` register, and configure SLOWCLK to use an prescaler other than `SLOWCLK_PRESCALE_1`.
- If the frequency of UCLK is unchanged when scaling SYSCLK using multiplier-based clock throttling, this mechanism guarantees that the UCLK output will be unaffected. To enable multiplier-based clock throttling of UCLK, set the `CLK_MULT_THROTTLING_CFG_UCLK_EBL` bit from the `CLK_MULT_THROTTLING_CFG` register, and configure UCLK to use any prescaler other than `UCLK_PRESCALE_1`.

NOTE: We recommend only using multiplier-based clock throttling on SDMCLK and derive ADCCLK, SLOWCLK and UCLK from it.

10.3.2 Frequency-Based Clock Throttling

Frequency-based clock throttling adjusts the base frequency of the CCO and updates the ADCCLK, SLOWCLK and SDMCLK prescalers in the same cycle. Because all four clocks are updated at the same time, the only artifacts produced are those caused by the settling time of the CCO.

IMPORTANT: The error introduced due to the settling time of the CCO is typically less than one SYSCLK clock cycle, which limits any artifacts that might occur due to a frequency change. For frequency-based throttling that double or halve the CCO frequency, artifacts observed should not be audible. If an audible artifact is produced for larger frequency-based clock throttling steps, consider using the advanced frequency-based clock throttling options described in [Section 10.3.2.1 “Advanced Frequency-Based Clock Throttling Options” on page 319](#) to limit or eliminate these artifacts.

When initiating a frequency-based clock throttling switch, a user application must:

1. Set the `ANALOG_OSC_CTRL_THROTTLING[0]` register with the updated frequency setting that will be copied to the `ANALOG_OSC_CTRL_0` register when applying clock throttling.

Ezairo 8300 Hardware Reference

2. Choose a new SDMCLK prescaler which ensures that the frequency of SDMCLK before and after the clock throttling event is unchanged. Set the CLK_SDMCLK_THROTTLING register with the updated value to use for the SDMCLK_PRESCALE bit field from the CLK_CFG_1 register after clock throttling.
3. Choose a new ADCCLK prescaler which ensures that the frequency of the ADCCLK before and after the clock throttling event is unchanged. Set the CLK_ADCCLK_THROTTLING register with the new value to use for the CLK_CFG_1_ADCCLK_PRESCALE bit field from the CLK_CFG_1 register after clock throttling.

NOTE: When an ADC is used, frequency-based throttling is only possible between configurations where the ADC clock division factor is 1 or an even multiple integer. For any other division factors, the ADC sampling might shift, causing a small audio artifact.

4. Choose a new SLOWCLK prescaler which ensures that the frequency of SLOWCLK before and after the clock throttling event is unchanged. Set the CLK_SLOWCLK_THROTTLING register with the new value to use for the CLK_CFG_1_SLOWCLK_PRESCALE bit field from the CLK_CFG_1 register after clock throttling.
5. Choose a new SDMCLK prescaler which ensures that the frequency of SDMCLK before and after the clock throttling event is unchanged. Set the CLK_SDMCLK_THROTTLING register with the updated value to use for the SDMCLK_PRESCALE bit field from the CLK_CFG_1 register after clock throttling.

NOTE: When an OD is used, frequency-based throttling is only possible between configurations where the ADC clock division factor is 1 or an even multiple integer. For any other division factors, the OD output might shift, causing a small audio artifact.

6. Set the clock throttling apply source by configuring the CLK_THROTTLING_CTRL_APPLY_SRC bit field from the CLK_THROTTLING_CTRL register. By default the new prescaler values are applied every SYSCLK cycle.
7. Write CLK_THROTTLE_APPLY to the CLK_THROTTLING_CTRL_APPLY bit field from the CLK_THROTTLING_CTRL register.

An application can then poll the CLK_THROTTLING_CTRL_STATUS bit field from the CLK_THROTTLING_CTRL register, which indicates if the frequency-based clock throttling has completed or is still pending.

The main error that can be observed when applying frequency-based clock throttling is the error in the updated CCO output frequency. To limit this effect, the initial frequency and the frequency switched to must both be calibrated configurations of the CCO, and both configurations must use the same multiplier value stored to the ANALOG_OSC_CTRL_1_MULT bit field from ANALOG_OSC_CTRL_1 register. Another possible source of error can be unsynchronized clock dividers/clocks. To avoid this, set the initial clock configuration simultaneously to ensure that all the clock dividers are initiated at the same time. The APPLY field mechanism of the CLK_THROTTLE_APPLY register can also be used to achieve this instead of a direct register write operation.

IMPORTANT: For optimal performance, limit frequency-based clock throttling to steps of lessno more than 400% of the lower frequency. For example, frequency-switching from 3.841.28 MHz to 7.685.12 MHz, or from 15.3610.24 MHz to 3.842.56 MHz, fits this recommendation. However, a clock frequency change from 3.841.28 MHz to 19.210.24 MHz needs to be transitioned through an intermediate frequency such as 7.68 MHzfirst.

10.3.2.1 Advanced Frequency-Based Clock Throttling Options

When switching the frequency of the system clocks using frequency-based clock throttling, the primary cause of audio artifacts relates to the error in the CCO output frequency. This output frequency generally settles in less than one clock period; however in some cases, the error in the first clock period can be enough to produce a detectable audio artifact due to an imbalance in the charge observed on the H-bridge of the digital drive outputs. If your system is using frequency-based clock throttling and audio artifacts are detected, two advanced clock throttling options are provided.

Frequency Correction

Ezairo 8300 Hardware Reference

To correct for frequency errors in the first cycles following the application of a clock throttling setting, the frequency configuration applied to the `ANALOG_OSC_CTRL_0` register can be updated up to three times in consecutive clock cycles of the configured clock (`SYSCLK`, `ADCCLK`, `SDMCLK` or `SLOWCLK`) to correct for the error in each clock cycle due to the switch in frequency. The clock is configured using the `APPLY_SRC` bit field of the `CLK_THROTTLING_CTRL` register.

In addition to the normal frequency-based clock throttling steps, the `ANALOG_OSC_CTRL_THROTTLING[1]` and `ANALOG_OSC_CTRL_THROTTLING[2]` registers are set with the intermediate values to be copied to the `ANALOG_OSC_CTRL_0` register.

When clock throttling is applied by writing `CLK_THROTTLE_APPLY_2` to the `CLK_THROTTLING_CTRL_APPLY` bit field from the `CLK_THROTTLING_CTRL` register, the value from `ANALOG_OSC_CTRL_THROTTLING[1]` is used in the first clock cycle, and the value in `ANALOG_OSC_CTRL_THROTTLING[0]` is used thereafter.

When clock throttling is applied by writing `CLK_THROTTLE_APPLY_3` to the `CLK_THROTTLING_CTRL_APPLY` bit field from the `CLK_THROTTLING_CTRL` register, the value from `ANALOG_OSC_CTRL_THROTTLING[2]` is used in the first clock cycle, the value from `ANALOG_OSC_CTRL_THROTTLING[1]` is used in the second clock cycle, and the value in `ANALOG_OSC_CTRL_THROTTLING[0]` is used thereafter.

NOTE: We have no recommendations for Ezairo 8300 on how to determine the intermediate CCO trimming values. Instead, we recommend using the Output State Control.

Output State Control

If the `AUDIO_SDM_CTRL_OD_DELAY`, `AUDIO_SDM_CTRL_TRI_STATE` or `AUDIO_SDM_CTRL_XSDM0` bits from the `AUDIO_SDM_CTRL` register are set, it is possible to have a zero state at the output to the sigma-delta modulator where the receiver pads of the H-bridge are outputting the same value. By setting the `CLK_THROTTLING_CFG_CLK_THROTTLING_ON_OD_ZERO` bit from the `CLK_THROTTLING_CFG` register, the application of the clock throttling values will wait until the H-bridge is in this state before switching the divisors and CCO frequency. Because the H-bridge is in a balanced state during the first clock cycle, any error observed at the CCO frequency does not charge either side of the H-bridge, which avoids the potential audio artifacts.

NOTE: When using this mode, it might take some time before a pending clock throttling is applied as the clocking block has to wait for the `ADCCLK`, `SLOWCLK`, `SDMCLK` and output drivers to be ready to update in the same cycle. This issue can be compounded by large amplitude outputs, or by having both output drivers enabled.

For more information about configuring the sigma-delta modulators, see [Section 15.3 “Sigma-Delta Modulator” on page 457](#).

onsemi Confidential

Ezairo 8300 Hardware Reference

10.4 CONTROL AND CONFIGURATION REGISTERS

Register Name	Register Description	Address
D_CLK_CFG_0 CLK_CFG_0	SYSCLK selection and external clock prescale registers	P:0x08C0 0x40002300
D_CLK_CFG_1 CLK_CFG_1	ADCCLK, SDMCLK and SLOWCLK prescale registers	P:0x08C1 0x40002304
D_CLK_CFG_2 CLK_CFG_2	UCLK prescale registers	P:0x08C2 0x40002308
D_CLK_CFG_CPA CLK_CFG_CPA	Prescale register for analog charge pump	P:0x08C4 0x40002310
D_CLK_CFG_CPIF CLK_CFG_CPIF	Prescale register for the digital interface charge pump	P:0x08C5 0x40002314
D_CLK_CFG_CPCM CLK_CFG_CPCM	Prescale registers for digital supply charge pumps	P:0x08C6 0x40002318
D_CLK_ADCCLK_THROTTLING CLK_ADCCLK_THROTTLING	ADC Clock throttling configuration register	P:0x08C7 0x4000231C
D_CLK_SDMCLK_THROTTLING CLK_SDMCLK_THROTTLING	SDM Clock throttling configuration register	P:0x08C8 0x40002320
D_CLK_SLOWCLK_THROTTLING CLK_SLOWCLK_THROTTLING	Slow Clock throttling configuration register	P:0x08C9 0x40002324
D_CLK_THROTTLING_CTRL CLK_THROTTLING_CTRL	CCO frequency based clock throttling control register	P:0x08CA 0x40002328
D_CLK_MULT_THROTTLING_CFG CLK_MULT_THROTTLING_CFG	CCO multiplier based clock throttling configuration register	P:0x08CB 0x4000232C
D_CLK_THROTTLING_CFG CLK_THROTTLING_CFG	CCO frequency based clock throttling configuration register	P:0x08CC 0x40002330
D_CLK_MULTI_SYNC_CFG CLK_MULTI_SYNC_CFG	Multi-chip synchronization configuration register	P:0x08CD 0x40002334
D_CLK_EXTCLK_DET_CFG	External clock detector configuration register	P:0x08CE

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
CLK_EXTCLK_DET_CFG		0x40002338
D_CLK_EXTCLK_DET_STATUS	External clock detector status register	P:0x08CF
CLK_EXTCLK_DET_STATUS		0x4000233C
ANALOG_OSC_CTRL_0	Internal oscillator frequency setting	P:0x0980 0x40002600
ANALOG_OSC_CTRL_1	Multiplier setting of the internal oscillator	P:0x0981 0x40002604
ANALOG_OSC_CTRL_THROTTLING	System clock throttling configuration register	P:0x0984- P:0x0986 0x40002610- 0x40002618

10.4.1 D_CLK_CFG_0, CLK_CFG_0

Bit Field	Read/Write	Field Name	Description
5:4	RW	SYSCLK_SEL	Controls the functionality of SYSCLK
3:0	RW	EXTCLK_PRESCALE	Prescale value for the input clock from pad EXTCLK (1 to 16 in steps of 1)

Field Name	Value Symbol	Value Description	Hex Value
SYSCLK_SEL	SYSCLK_SEL_OSCCLK	Select the default internal oscillator as the source for SYSCLK	0x0*
	SYSCLK_SEL_EXTCLK	Select the external clock input (DIO35) as the source for SYSCLK	0x1
	SYSCLK_SEL_STANDBY	Select the standby oscillator as the source for SYSCLK	0x2
EXTCLK_PRESCALE	EXTCLK_PRESCALE_1	Divide by 1 (Note: Switching to this setting while running on EXTCLK is not allowed as it can put the chip in an unknown state.)	0x0*
	EXTCLK_PRESCALE_2	Divide by 2	0x1
	EXTCLK_PRESCALE_3	Divide by 3	0x2
	EXTCLK_PRESCALE_4	Divide by 4	0x3
	EXTCLK_PRESCALE_5	Divide by 5	0x4

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	EXTCLK_PRESCALE_6	Divide by 6	0x5
	EXTCLK_PRESCALE_7	Divide by 7	0x6
	EXTCLK_PRESCALE_8	Divide by 8	0x7
	EXTCLK_PRESCALE_9	Divide by 9	0x8
	EXTCLK_PRESCALE_10	Divide by 10	0x9
	EXTCLK_PRESCALE_11	Divide by 11	0xA
	EXTCLK_PRESCALE_12	Divide by 12	0xB
	EXTCLK_PRESCALE_13	Divide by 13	0xC
	EXTCLK_PRESCALE_14	Divide by 14	0xD
	EXTCLK_PRESCALE_15	Divide by 15	0xE
	EXTCLK_PRESCALE_16	Divide by 16	0xF

10.4.2 D_CLK_CFG_1, CLK_CFG_1

Bit Field	Read/Write	Field Name	Description
24	RW	UARTCLK_SRC	Indicates the UART clock source (ADCCLK or SYSCCLK)
23	RW	SLOWCLK_SRC	Indicates the slow clock source (SYSCCLK or ADCCLK)
22:16	RW	SLOWCLK_PRESCALE	Prescale value for the slow clock (1 to 128 in steps of 1)
13:8	RW	SDMCLK_PRESCALE	Prescale value for the SDM clock (1 to 64 in steps of 1)
7	RW	ADCCLK_SRC	Indicates the ADC clock source (SYSCCLK or SDMCLK)
4:0	RW	ADCCLK_PRESCALE	Prescale value for the ADC clock (1 to 32 in steps of 1)

Field Name	Value Symbol	Value Description	Hex Value
UARTCLK_SRC	UARTCLK_SRC_ADCCLK	UARTCLK source is ADCCLK	0x0*
	UARTCLK_SRC_SYSCCLK	UARTCLK source is SYSCCLK	0x1
SLOWCLK_SRC	SLOWCLK_SRC_SYSCCLK	SLOWCLK source is SYSCCLK	0x0*
	SLOWCLK_SRC_ADCCLK	SLOWCLK source is ADCCLK	0x1
SLOWCLK_PRESCALE	SLOWCLK_PRESCALE_1	Divide by 1	0x0
	SLOWCLK_PRESCALE_2	Divide by 2	0x1
	SLOWCLK_PRESCALE_3	Divide by 3	0x2
	SLOWCLK_PRESCALE_4	Divide by 4	0x3
	SLOWCLK_PRESCALE_5	Divide by 5	0x4

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	SLOWCLK_PRESALE_6	Divide by 6	0x5*
	SLOWCLK_PRESALE_7	Divide by 7	0x6
	SLOWCLK_PRESALE_8	Divide by 8	0x7
	SLOWCLK_PRESALE_9	Divide by 9	0x8
	SLOWCLK_PRESALE_10	Divide by 10	0x9
	SLOWCLK_PRESALE_11	Divide by 11	0xA
	SLOWCLK_PRESALE_12	Divide by 12	0xB
	SLOWCLK_PRESALE_13	Divide by 13	0xC
	SLOWCLK_PRESALE_14	Divide by 14	0xD
	SLOWCLK_PRESALE_15	Divide by 15	0xE
	SLOWCLK_PRESALE_16	Divide by 16	0xF
	SLOWCLK_PRESALE_17	Divide by 17	0x10
	SLOWCLK_PRESALE_18	Divide by 18	0x11
	SLOWCLK_PRESALE_19	Divide by 19	0x12
	SLOWCLK_PRESALE_20	Divide by 20	0x13
	SLOWCLK_PRESALE_21	Divide by 21	0x14
	SLOWCLK_PRESALE_22	Divide by 22	0x15
	SLOWCLK_PRESALE_23	Divide by 23	0x16
	SLOWCLK_PRESALE_24	Divide by 24	0x17
	SLOWCLK_PRESALE_25	Divide by 25	0x18
	SLOWCLK_PRESALE_26	Divide by 26	0x19
	SLOWCLK_PRESALE_27	Divide by 27	0x1A
	SLOWCLK_PRESALE_28	Divide by 28	0x1B
	SLOWCLK_PRESALE_29	Divide by 29	0x1C
	SLOWCLK_PRESALE_30	Divide by 30	0x1D
	SLOWCLK_PRESALE_31	Divide by 31	0x1E
	SLOWCLK_PRESALE_32	Divide by 32	0x1F
	SLOWCLK_PRESALE_33	Divide by 33	0x20
	SLOWCLK_PRESALE_34	Divide by 34	0x21
	SLOWCLK_PRESALE_35	Divide by 35	0x22
	SLOWCLK_PRESALE_36	Divide by 36	0x23

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	SLOWCLK_PRESALE_37	Divide by 37	0x24
	SLOWCLK_PRESALE_38	Divide by 38	0x25
	SLOWCLK_PRESALE_39	Divide by 39	0x26
	SLOWCLK_PRESALE_40	Divide by 40	0x27
	SLOWCLK_PRESALE_41	Divide by 41	0x28
	SLOWCLK_PRESALE_42	Divide by 42	0x29
	SLOWCLK_PRESALE_43	Divide by 43	0x2A
	SLOWCLK_PRESALE_44	Divide by 44	0x2B
	SLOWCLK_PRESALE_45	Divide by 45	0x2C
	SLOWCLK_PRESALE_46	Divide by 46	0x2D
	SLOWCLK_PRESALE_47	Divide by 47	0x2E
	SLOWCLK_PRESALE_48	Divide by 48	0x2F
	SLOWCLK_PRESALE_49	Divide by 49	0x30
	SLOWCLK_PRESALE_50	Divide by 50	0x31
	SLOWCLK_PRESALE_51	Divide by 51	0x32
	SLOWCLK_PRESALE_52	Divide by 52	0x33
	SLOWCLK_PRESALE_53	Divide by 53	0x34
	SLOWCLK_PRESALE_54	Divide by 54	0x35
	SLOWCLK_PRESALE_55	Divide by 55	0x36
	SLOWCLK_PRESALE_56	Divide by 56	0x37
	SLOWCLK_PRESALE_57	Divide by 57	0x38
	SLOWCLK_PRESALE_58	Divide by 58	0x39
	SLOWCLK_PRESALE_59	Divide by 59	0x3A
	SLOWCLK_PRESALE_60	Divide by 60	0x3B
	SLOWCLK_PRESALE_61	Divide by 61	0x3C
	SLOWCLK_PRESALE_62	Divide by 62	0x3D
	SLOWCLK_PRESALE_63	Divide by 63	0x3E
	SLOWCLK_PRESALE_64	Divide by 64	0x3F
SDMCLK_PRESALE	SDMCLK_PRESALE_1	Divide by 1	0x0*
	SDMCLK_PRESALE_2	Divide by 2	0x1
	SDMCLK_PRESALE_3	Divide by 3	0x2

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	SDMCLK_PRESCALE_4	Divide by 4	0x3
	SDMCLK_PRESCALE_5	Divide by 5	0x4
	SDMCLK_PRESCALE_6	Divide by 6	0x5
	SDMCLK_PRESCALE_7	Divide by 7	0x6
	SDMCLK_PRESCALE_8	Divide by 8	0x7
	SDMCLK_PRESCALE_9	Divide by 9	0x8
	SDMCLK_PRESCALE_10	Divide by 10	0x9
	SDMCLK_PRESCALE_64	Divide by 64	0x3F
ADCCLK_SRC	ADCCLK_SRC_SYSCLK	ADCCLK source is SYSCLK	0x0*
	ADCCLK_SRC_SDMCLK	ADCCLK source is SDMCLK (not allowed when ADCCLK_PRESCALE_1, SDMCLK_PRESCALE_1 and CCO_MULT_THROTTLE_SDMCLK_ENABLE are used)	0x1
ADCCLK_PRESCALE	ADCCLK_PRESCALE_1	Divide by 1	0x0*
	ADCCLK_PRESCALE_2	Divide by 2	0x1
	ADCCLK_PRESCALE_3	Divide by 3	0x2
	ADCCLK_PRESCALE_4	Divide by 4	0x3
	ADCCLK_PRESCALE_5	Divide by 5	0x4
	ADCCLK_PRESCALE_6	Divide by 6	0x5
	ADCCLK_PRESCALE_7	Divide by 7	0x6
	ADCCLK_PRESCALE_8	Divide by 8	0x7
	ADCCLK_PRESCALE_9	Divide by 9	0x8
	ADCCLK_PRESCALE_10	Divide by 10	0x9
	ADCCLK_PRESCALE_11	Divide by 11	0xA
	ADCCLK_PRESCALE_12	Divide by 12	0xB
	ADCCLK_PRESCALE_13	Divide by 13	0xC
	ADCCLK_PRESCALE_14	Divide by 14	0xD
	ADCCLK_PRESCALE_15	Divide by 15	0xE
	ADCCLK_PRESCALE_16	Divide by 16	0xF
	ADCCLK_PRESCALE_17	Divide by 17	0x10
	ADCCLK_PRESCALE_18	Divide by 18	0x11

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	ADCCLK_PRESCALE_19	Divide by 19	0x12
	ADCCLK_PRESCALE_20	Divide by 20	0x13
	ADCCLK_PRESCALE_21	Divide by 21	0x14
	ADCCLK_PRESCALE_22	Divide by 22	0x15
	ADCCLK_PRESCALE_23	Divide by 23	0x16
	ADCCLK_PRESCALE_24	Divide by 24	0x17
	ADCCLK_PRESCALE_25	Divide by 25	0x18
	ADCCLK_PRESCALE_26	Divide by 26	0x19
	ADCCLK_PRESCALE_27	Divide by 27	0x1A
	ADCCLK_PRESCALE_28	Divide by 28	0x1B
	ADCCLK_PRESCALE_29	Divide by 29	0x1C
	ADCCLK_PRESCALE_30	Divide by 30	0x1D
	ADCCLK_PRESCALE_31	Divide by 31	0x1E
	ADCCLK_PRESCALE_32	Divide by 32	0x1F

10.4.3 D_CLK_CFG_2, CLK_CFG_2

Bit Field	Read/Write	Field Name	Description
15	RW	UCLK_SRC	Indicates the user clock source (SYSCLK or ADCCLK)
11:0	RW	UCLK_PRESCALE	Prescale value for the user clock (1 to 4096 in steps of 1)

Field Name	Value Symbol	Value Description	Hex Value
UCLK_SRC	UCLK_SRC_SYSCLK	UCLK source is SYSCLK	0x0*
	UCLK_SRC_ADCCLK	UCLK source is ADCCLK	0x1
UCLK_PRESCALE	UCLK_PRESCALE_1	Divide by 1	0x0*
	UCLK_PRESCALE_2	Divide by 2	0x1
	UCLK_PRESCALE_3	Divide by 3	0x2
	UCLK_PRESCALE_4	Divide by 4	0x3
	UCLK_PRESCALE_5	Divide by 5	0x4
	UCLK_PRESCALE_6	Divide by 6	0x5
	UCLK_PRESCALE_7	Divide by 7	0x6
	UCLK_PRESCALE_8	Divide by 8	0x7

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	UCLK_PRESCALE_9	Divide by 9	0x8
	UCLK_PRESCALE_10	Divide by 10	0x9
	UCLK_PRESCALE_16	Divide by 16	0xF
	UCLK_PRESCALE_32	Divide by 32	0x1F
	UCLK_PRESCALE_64	Divide by 64	0x3F
	UCLK_PRESCALE_128	Divide by 128	0x7F
	UCLK_PRESCALE_256	Divide by 256	0xFF
	UCLK_PRESCALE_512	Divide by 512	0x1FF
	UCLK_PRESCALE_1024	Divide by 1024	0x3FF
	UCLK_PRESCALE_2048	Divide by 2048	0x7FF
	UCLK_PRESCALE_4096	Divide by 4096	0xFFF

10.4.4 D_CLK_CFG_CPA, CLK_CFG_CPA

Bit Field	Read/Write	Field Name	Description
5:0	RW	CP_VDDA_CLK_PRESCALE	Prescale the clock for the VDDA charge pump (1 to 64 in steps of 1). For an optimal performance of VDDA charge pump use an even prescaler value.

Field Name	Value Symbol	Value Description	Hex Value
CP_VDDA_CLK_PRESCALE	CPACLK_PRESCALE_4	Divide by 4 for high-power mode (1.28 MHz -> 320 kHz)	0x3*
	CPACLK_PRESCALE_10	Divided by 10 for low-power mode (1.28 MHz -> 128 kHz)	0x9

10.4.5 D_CLK_CFG_CPIF, CLK_CFG_CPIF

Bit Field	Read/Write	Field Name	Description
5:0	RW	CP_VDDIF_CLK_PRESCALE	Prescale the clock for the VDDIF charge pump (1 to 64 in steps of 1). For an optimal performance of VDDIF charge pump use an even prescaler value.

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
CP_VDDIF_CLK_PRESCALE	CPIFCLK_PRESCALE_4	Divide by 4 for high-power (15 mA) mode (1.28 MHz -> 320 kHz)	0x3*
	CPIFCLK_PRESCALE_6	Divided by 6 for 10 mA mode (1.28 MHz -> 213 kHz)	0x5
	CPIFCLK_PRESCALE_8	Divided by 8 for 5 mA mode (1.28 MHz -> 160 kHz)	0x7
	CPIFCLK_PRESCALE_10	Divided by 10 for low-power (1 mA) mode (1.28 MHz -> 128 kHz)	0x9

10.4.6 D_CLK_CFG_CPCM, CLK_CFG_CPCM

Bit Field	Read/Write	Field Name	Description
7	RW	CP_VDDM_CLK_PHASE	VDDM charge pump clock phase
6	RW	CP_VDDC_CLK_PHASE	VDDC charge pump clock phase
5	RW	CP_VDDCM_CLK_SRC	Indicates the VDDC/VDDM charge pump clock source (SYSCLK or SDMCLK)
4:0	RW	CP_VDDCM_CLK_PRESCALE	Prescale the clock for the VDDC/VDDM charge pump (1 to 32 in steps of 1)

Field Name	Value Symbol	Value Description	Hex Value
CP_VDDM_CLK_PHASE	CPMCLK_NORMAL	CPMCLK is not inverted	0x0*
	CPMCLK_INVERTED	CPMCLK is inverted	0x1
CP_VDDC_CLK_PHASE	CPCCLK_NORMAL	CPCCLK is not inverted	0x0*
	CPCCLK_INVERTED	CPCCLK is inverted	0x1
CP_VDDCM_CLK_SRC	CPCMCLK_SRC_SYSCLK	CPCMCLK source is SYSCLK	0x0*
	CPCMCLK_SRC_SDMCLK	CPCMCLK source is SDMCLK (not allowed when CPCMCLK_PRESCALE_1, SDMCLK_PRESCALE_1 and CCO_MULT_THROTTLE_SDMCLK_ENABLE are used)	0x1
CP_VDDCM_CLK_PRESCALE	CPCMCLK_PRESCALE_1	Divide by 1	0x0*
	CPCMCLK_PRESCALE_2	Divide by 2	0x1
	CPCMCLK_PRESCALE_3	Divide by 3	0x2
	CPCMCLK_PRESCALE_4	Divide by 4	0x3
	CPCMCLK_PRESCALE_5	Divide by 5	0x4

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	CPCMCLK_PRESCALE_6	Divide by 6	0x5
	CPCMCLK_PRESCALE_7	Divide by 7	0x6
	CPCMCLK_PRESCALE_8	Divide by 8	0x7
	CPCMCLK_PRESCALE_9	Divide by 9	0x8
	CPCMCLK_PRESCALE_10	Divide by 10	0x9
	CPCMCLK_PRESCALE_11	Divide by 11	0xA
	CPCMCLK_PRESCALE_12	Divide by 12	0xB
	CPCMCLK_PRESCALE_13	Divide by 13	0xC
	CPCMCLK_PRESCALE_14	Divide by 14	0xD
	CPCMCLK_PRESCALE_15	Divide by 15	0xE
	CPCMCLK_PRESCALE_16	Divide by 16	0xF
	CPCMCLK_PRESCALE_17	Divide by 17	0x10
	CPCMCLK_PRESCALE_18	Divide by 18	0x11
	CPCMCLK_PRESCALE_19	Divide by 19	0x12
	CPCMCLK_PRESCALE_20	Divide by 20	0x13
	CPCMCLK_PRESCALE_21	Divide by 21	0x14
	CPCMCLK_PRESCALE_22	Divide by 22	0x15
	CPCMCLK_PRESCALE_23	Divide by 23	0x16
	CPCMCLK_PRESCALE_24	Divide by 24	0x17
	CPCMCLK_PRESCALE_25	Divide by 25	0x18
	CPCMCLK_PRESCALE_26	Divide by 26	0x19
	CPCMCLK_PRESCALE_27	Divide by 27	0x1A
	CPCMCLK_PRESCALE_28	Divide by 28	0x1B
	CPCMCLK_PRESCALE_29	Divide by 29	0x1C
	CPCMCLK_PRESCALE_30	Divide by 30	0x1D
	CPCMCLK_PRESCALE_31	Divide by 31	0x1E
	CPCMCLK_PRESCALE_32	Divide by 32	0x1F

10.4.7 D_CLK_ADCCLK_THROTTLING, CLK_ADCCLK_THROTTLING

Bit Field	Read/Write	Field Name	Description
4:0	RW	PRESALE	Future clock throttling value for ADCCLK_PRESCALE

Ezairo 8300 Hardware Reference

10.4.8 D_CLK_SDMCLK_THROTTLING, CLK_SDMCLK_THROTTLING

Bit Field	Read/Write	Field Name	Description
5:0	RW	PRESCALE	Future clock throttling value for SDMCLK_PRESCALE

10.4.9 D_CLK_SLOWCLK_THROTTLING, CLK_SLOWCLK_THROTTLING

Bit Field	Read/Write	Field Name	Description
6:0	RW	PRESCALE	Future clock throttling value for SLOWCLK_PRESCALE

10.4.10 D_CLK_THROTTLING_CTRL, CLK_THROTTLING_CTRL

Bit Field	Read/Write	Field Name	Description
3:2	RW	APPLY_SRC	Clock throttling apply source
1:0	R	STATUS	Clock throttling apply status
1:0	W	APPLY	Clock throttling apply command

Field Name	Value Symbol	Value Description	Hex Value
APPLY_SRC	CLK_THROTTLE_SYSCCLK	Apply new values every SYSCCLK cycle	0x0*
	CLK_THROTTLE_ADCCLK	Apply new values every ADCCLK cycle	0x1
	CLK_THROTTLE_SDMCLK	Apply new values every SDMCLK cycle	0x2
	CLK_THROTTLE_SLOWCLK	Apply new values every SLOWCLK cycle	0x3
STATUS	CLK_THROTTLE_DISABLED	Clock throttling inactive	0x0*
	CLK_THROTTLE_PENDING	New clock throttling values are pending	0x1
	CLK_THROTTLE_PENDING_2	Two new clock throttling values are pending	0x2
	CLK_THROTTLE_PENDING_3	Three new clock throttling values are pending	0x3
APPLY	CLK_THROTTLE_ABORT	Abort clock throttling if it is pending	0x0
	CLK_THROTTLE_APPLY	Apply new clock throttling values (as soon as the next counter value of the slow clock divider is zero)	0x1
	CLK_THROTTLE_APPLY_2	Apply two new clock throttling values (as soon as the next counter value of the slow clock divider is zero)	0x2
	CLK_THROTTLE_APPLY_3	Apply three new clock throttling values (as soon as the next counter value of the slow clock divider is zero)	0x3

Ezairo 8300 Hardware Reference

10.4.11 D_CLK_MULT_THROTTLING_CFG, CLK_MULT_THROTTLING_CFG

Bit Field	Read/Write	Field Name	Description
3	RW	ADCCLK_EBL	ADC clock CCO multiplier based clock throttling enable
2	RW	UCLK_EBL	User clock CCO multiplier based clock throttling enable
1	RW	SDMCLK_EBL	SDM clock CCO multiplier based clock throttling enable
0	RW	SLOWCLK_EBL	Slow clock CCO multiplier based clock throttling enable

Field Name	Value Symbol	Value Description	Hex Value
ADCCLK_EBL	CCO_MULT_THROTTLE_ADCCLK_DISABLE	ADC clock is not divided by the CCO multiplier value	0x0*
	CCO_MULT_THROTTLE_ADCCLK_ENABLE	ADC clock is divided by the CCO multiplier value (not allowed when ADCCLK_PRESCALE_1 is used)	0x1
UCLK_EBL	CCO_MULT_THROTTLE_UCLK_DISABLE	User clock is not divided by the CCO multiplier value	0x0*
	CCO_MULT_THROTTLE_UCLK_ENABLE	User clock is divided by the CCO multiplier value	0x1
SDMCLK_EBL	CCO_MULT_THROTTLE_SDMCLK_DISABLE	SDM clock is not divided by the CCO multiplier value	0x0*
	CCO_MULT_THROTTLE_SDMCLK_ENABLE	SDM clock is divided by the CCO multiplier value	0x1
SLOWCLK_EBL	CCO_MULT_THROTTLE_SLOWCLK_DISABLE	Slow clock is not divided by the CCO multiplier value	0x0*
	CCO_MULT_THROTTLE_SLOWCLK_ENABLE	Slow clock is divided by the CCO multiplier value	0x1

10.4.12 D_CLK_THROTTLING_CFG, CLK_THROTTLING_CFG

Bit Field	Read/Write	Field Name	Description
0	RW	CLK_THROTTLING_ON_OD_ZERO	Enable CCO frequency based clock throttling only on OD zero output

Field Name	Value Symbol	Value Description	Hex Value
CLK_THROTTLING_ON_OD_ZERO	CLK_THROTTLING_ON_OD_ZERO_DISABLE	Always allow CCO frequency based clock throttling	0x0*
	CLK_THROTTLING_ON_OD_ZERO_ENABLE	Only allow CCO frequency based clock throttling when OD_DELAY is disabled, or when both OD0 and OD1 are either disabled or output a zero	0x1

Ezairo 8300 Hardware Reference

10.4.13 D_CLK_MULTI_SYNC_CFG, CLK_MULTI_SYNC_CFG

Bit Field	Read/Write	Field Name	Description
0	RW	SLAVE_CFG	Multi-chip slave configuration

Field Name	Value Symbol	Value Description	Hex Value
SLAVE_CFG	MULT_SYNC_SLAVE_OFF	Multi-sync slave feature disabled on ADCCLK prescaler; master device runs on ADCCLK	0x0*
	MULT_SYNC_SLAVE_SYSCLK	Multi-sync slave enabled; master device runs on SYSCLK	0x1

10.4.14 D_CLK_EXTCLK_DET_CFG, CLK_EXTCLK_DET_CFG

Bit Field	Read/Write	Field Name	Description
6:5	RW	CLK_DET_INT_SEL	Clock detector interrupt configuration
0	RW	CLK_DET_ENABLE	Clock detector enable/disable

Field Name	Value Symbol	Value Description	Hex Value
CLK_DET_INT_SEL	CLK_DET_INT_DISABLE	Clock detector interrupt disabled	0x0*
	CLK_DET_INT_ACTIVATED	If the clock source becomes active an interrupt is created	0x1
	CLK_DET_INT_DEACTIVATED	If the clock source becomes inactive an interrupt is created	0x2
	CLK_DET_INT_ACTIVITY_CHANGE	Any clock source activity change creates an interrupt	0x3
CLK_DET_ENABLE	CLK_DET_DISABLE	Clock detector disabled	0x0*
	CLK_DET_ENABLE	Clock detector enabled	0x1

10.4.15 D_CLK_EXTCLK_DET_STATUS, CLK_EXTCLK_DET_STATUS

Bit Field	Read/Write	Field Name	Description
9	R	CLK_DET_INT_STATUS	Clock detector interrupt status
8	R	CLK_DET_STATUS	Clock detector status
0	W	CLK_DET_INT_STATUS_CLEAR	Clear clock detector interrupt status

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
CLK_DET_INT_STATUS	CLK_DET_INT_FALSE	Clock detector interrupt not triggered	0x0*
	CLK_DET_INT_TRUE	Clock detector interrupt triggered	0x1
CLK_DET_STATUS	CLK_NOT_ACTIVE	No clock detected	0x0*
	CLK_ACTIVE	Clock detected	0x1
CLK_DET_INT_STATUS_CLEAR	CLK_DET_INT_CLEAR	Clear clock detector interrupt status	0x1

10.4.16 ANALOG_OSC_CTRL_0

Bit Field	Read/Write	Field Name	Description
15:8	RW	COARSE_FREQ_CTRL	Coarse oscillator frequency programming control
7:0	RW	FINE_FREQ_CTRL	Fine oscillator frequency programming control

Field Name	Value Symbol	Value Description	Hex Value
COARSE_FREQ_CTRL	OSC_FREQ_MAX	Maximum frequency	0xFF
	OSC_FREQ_7M68	Nominal oscillator frequency of 7.68 MHz	0xF*
	OSC_FREQ_MIN	Minimum frequency	0x2
FINE_FREQ_CTRL	OSC_FREQ_FINE_MAX	Maximum fine trimming	0xFF
	OSC_FREQ_FINE_NOM	Nominal fine trimming	0x80*
	OSC_FREQ_FINE_MIN	Minimum fine trimming	0x0

10.4.17 ANALOG_OSC_CTRL_1

Bit Field	Read/Write	Field Name	Description
1:0	RW	MULT	Oscillator clock multiplier control

Field Name	Value Symbol	Value Description	Hex Value
MULT	OSC_MULTIPLY_BY_4	Multiply the oscillator frequency by 4	0x3
	OSC_MULTIPLY_BY_2	Multiply the oscillator frequency by 2	0x1
	OSC_MULTIPLY_BY_1	Multiply the oscillator frequency by 1	0x0*

Ezairo 8300 Hardware Reference

10.4.18 ANALOG_OSC_CTRL_THROTTLING*

Bit Field	Read/Write	Field Name	Description
15:8	RW	COARSE_FREQ_CTRL	Coarse oscillator frequency programming control. If 0x00 or 0x01 is written, the entire register keeps its previous value
7:0	RW	FINE_FREQ_CTRL	Fine oscillator frequency programming control

CHAPTER 11

Memory

The memory systems provided by the Ezairo 8300 system are constructed using a number of memories (memory instances), memory buses, memory controllers and memory arbiters. These memories and other memory-mapped elements are addressable from the CFX DSP and the Arm Cortex-M3 processor through a set of memory spaces (also known as address spaces).

11.1 INTRODUCTION

This group of topics describes:

- The memory instances, buses, controllers, and arbiters that form the memory architecture in [Section 11.2 “Memory Architecture” on page 336](#)
- The CFX Processor memory spaces in [Section 11.3 “CFX Processor Memory Mapping” on page 348](#)
 - Details on CFX program memory spaces in [Section 11.3.1 “Program and P Memory” on page 348](#)
 - Details on CFX data memory spaces in [Section 11.3.2 “Data Memory” on page 354](#)
- The Arm Cortex-M3 processor unified memory space in [Section 11.4 “Arm Cortex-M3 Processor Memory Mapping” on page 358](#)
- The HEAR Configurable Accelerator memory spaces in [Section 11.5 “HEAR Configurable Accelerator Memory Mapping” on page 365](#)
- The LPDSP32 DSP memory spaces in [Section 11.6 “LPDSP32 DSP Memory Mapping” on page 367](#)
- The Filter Engine memory space in [Section 11.7 “Filter Engine Memory” on page 370](#)
- Control and configuration registers for all of the memory components in [Section 11.8 “Control and Configuration Registers” on page 371](#)

11.2 MEMORY ARCHITECTURE

The Ezairo 8300 memory architecture consists of a number of single-port memories, interconnected with the cores, the NNA, the IOCs, and the DMA through a series of buses and data controllers. Memory is divided into memory instances attached to the:

- CFX DSP
- HEAR Configurable Accelerator
- Arm Cortex-M3 processor
- Filter Engine
- LPDSP32 DSP

All memory instances are accessible to the CFX DSP and the Arm Cortex-M3 processor. An overview of the layout of the memories and buses is illustrated in the [“System Memory Architecture” figure \(Figure 15\)](#).

NOTE: The only memory-mapped area that the CFX DSP cannot access is the Arm Cortex-M3 processor private peripheral configuration. The Arm Cortex-M3 processor has access to all memory mapped areas in the Ezairo 8300 system.

- All the memory instances connected to the other cores are also accessible by the CFX DSP through either the P memory bus or X memory bus. CFX DSP access to Arm Cortex-M3 processor memories is through a bus bridge; this adds two cycles of delay for a read access and one delay cycle in case of a write access that is immediately following another read or write access (i.e. a write access that is not directly following an access to Arm Cortex-M3 processor memories does add any delay cycles).

Ezairo 8300 Hardware Reference

- Arm Cortex-M3 processor access to CFX memories, FIFO memories, and memory-mapped registers on the CFX P Memory Bus is through separate bus bridges that add two cycles of delay for a read access and one delay cycle in case of a write access that is immediately following another read or write access (i.e. a write access that doesn't directly follow an access to the same bus bridge does not add any delay cycles).

For more information, see [Section 11.2.3 “Memory Controllers and Arbitration”](#) on page 344.

Ezairo 8300 Hardware Reference

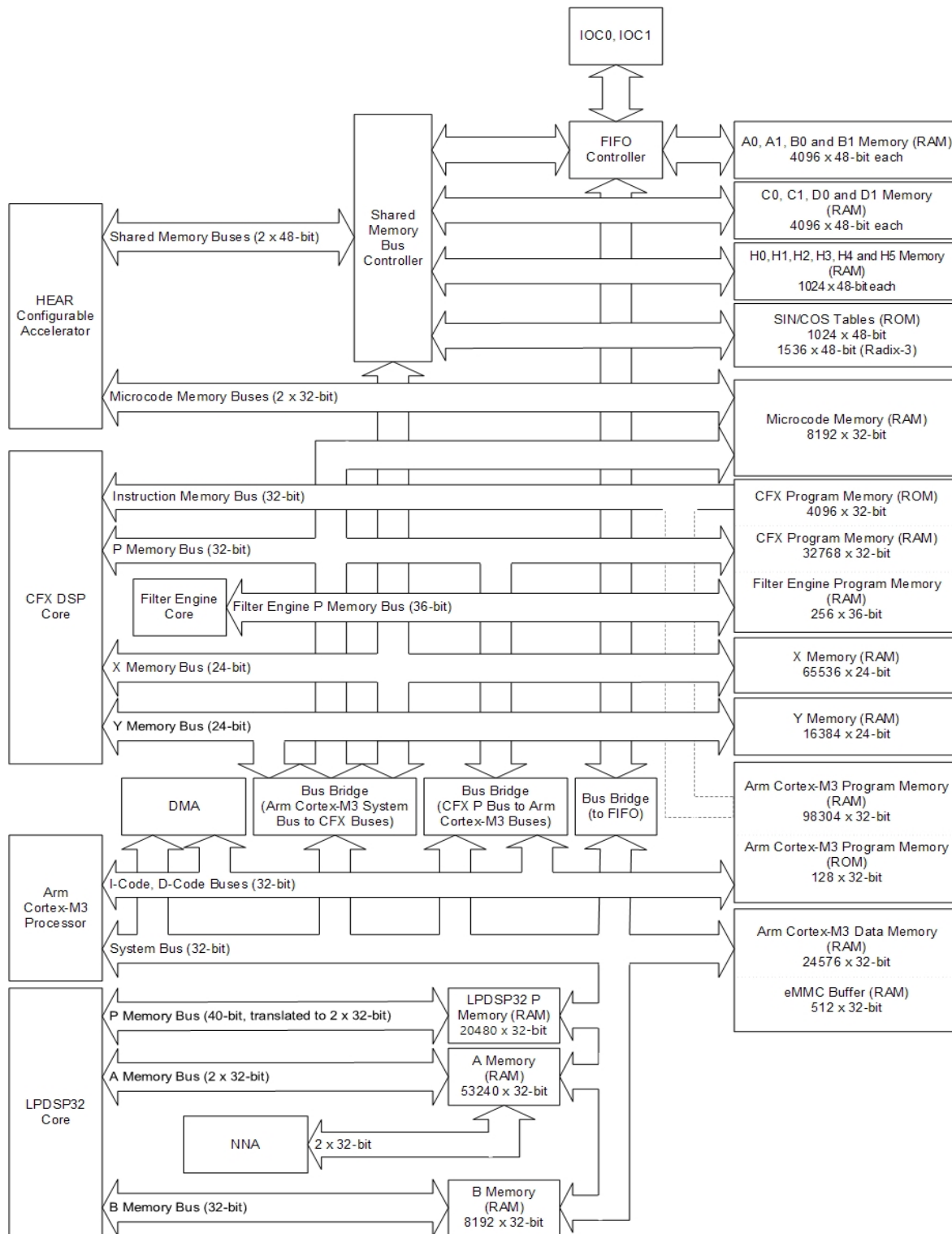


Figure 15. System Memory Architecture

Ezairo 8300 Hardware Reference

11.2.1 Memory Instances

The memory for the Ezairo 8300 device is divided into a number of different memory structures, which are sometimes further divided into several logical memory instances. This breakdown can be important to the efficiency of an application, as memory conflicts only occur when accesses are made to the same logical memory instance.

The "CFX Memory Instances" table (Table 22) lists the memory structures attached to the CFX, the size and width of each memory structure, and the bus or buses that the CFX DSP uses to access the memory.

Table 22. CFX Memory Instances

Memory Structure	Data Width	Memory Instances	Memory Length	CFX Access Using
Program memory (ROM)	32	-	4096	P, Instruction memory bus
Program memory (RAM)	32	PRAM0, PRAM1	2 x 16384 (4 x 16384 ¹)	P, Instruction memory bus
X memory (ROM, AES:S-Box)	24	-	1280	X
X memory (RAM)	24	XRAM0 to XRAM3	4 x 16384	XRAM0:X, XY memory bus XRAM1 to XRAM3: X memory bus
Y memory (RAM)	24	YRAM0	8192	Y, XY memory bus
		YRAM1, YRAM2	2 x 4096	Y, X, XY memory bus

The "Arm Cortex-M3 Core Memory Structures" table (Table 23) lists the memory structures attached to the Arm Cortex-M3 processor, the size and the width of each memory structure, and the bus that the CFX DSP uses to access the memory.

Table 23. Arm Cortex-M3 Core Memory Structures

Memory Structure	Data Width	Memory Instances	Memory Length	CFX Access Using
Program memory (ROM)	32	-	128	P memory bus
Program memory (RAM)	32	CM3_PRAM0 to CM3_PRAM4	5 x 16384 (3 x 16384 ²)	P memory bus
		CM3_PRAM5, CM3_PRAM6	2 x 8192	P memory bus
Data memory (RAM)	32	CM3_DRAM0, CM3_DRAM1	2 x 8192	P memory bus
		CM3_DRAM2, CM3_DRAM3	2 x 4096	

The "HEAR Memory Structures" table (Table 24) lists the memory structures attached to the HEAR, the size and width of each memory structure, and the bus that the CFX DSP uses to access the memory.

¹The CFX program memory can be extended by assigning the Arm Cortex-M3 processor's program memory sections 0 and 1 to the CFX. Configure these assignments using the `SYSCTRL_PRAM_CFG_PRAM*_ASSIGN` bits in the `SYSCTRL_PRAM_CFG` mapped register. When these memory instances are assigned to the Arm Cortex-M3 processor, the CFX DSP cannot execute from this memory; when it accesses it over the IOMEM bus, it will (automatically) use the Arm Cortex-M3 processor's shared memory interface.

²Two sections of the Arm Cortex-M3 processor's program memory can be assigned to the CFX DSP as described above.

Table 24. HEAR Memory Structures

Memory Structure	Data Width	Memory Instances	Memory Length	CFX Access Using
Microcode memory (RAM)	32	HEAR_MICROCODE_PMEM0, HEAR_MICROCODE_PMEM1	2 x 4096	P memory bus
H0 memory (RAM)	48	-	1024	X memory bus
H1 memory (RAM)	48	-	1024	X memory bus
H2 memory (RAM)	48	-	1024	X memory bus
H3 memory (RAM)	48	-	1024	X memory bus
H4 memory (RAM)	48	-	1024	X memory bus
H5 memory (RAM)	48	-	1024	X memory bus
A0 memory (RAM)	48	-	4096	X memory bus
A1 memory (RAM)	48	-	4096	X memory bus
B0 memory (RAM)	48	-	4096	X memory bus
B1 memory (RAM)	48	-	4096	X memory bus
C0 memory (RAM)	48	-	4096	X memory bus
C1 memory (RAM)	48	-	4096	X memory bus
D0 memory (RAM)	48	-	4096	X memory bus
D10 memory (RAM)	48	-	4096	X memory bus
SIN/COS Table (ROM)	48	-	1024	X memory bus
SIN/COS Radix-3 Table (ROM)	48	-	1536	X memory bus

The "LPDSP32 Core Memory Structures" table (Table 25) lists the memory structures attached to the LPDSP32 core, the size and the width of each memory structure, and the bus that the CFX DSP uses to access the memory.

Table 25. LPDSP32 Core Memory Structures

Memory Structure	Data Width	Memory Instances	Memory Length	CFX Access Using
Program memory (RAM)	32 ¹	DSP_PRAM0, DSP_PRAM1	2 x 2048	P memory bus
		DSP_PRAM2 to DSP_PRAM5	4 x 4096	
A memory (RAM)	32	DSP_ARAM0, DSP_ARAM1	2 x 8192	P memory bus
		DSP_ARAM2, DSP_ARAM3	2 x 16384	

¹The LPDSP32 uses 40-bit instructions accessed from 32-bit memory using a 2 x 20-bit bus. For more information about the mapping of bytes from the 32-bit memory instances to the 40-bit instructions used by the LPDSP32, refer to [Section 11.6.1 "Program Memory" on page 367](#).

Ezairo 8300 Hardware Reference

LPDSP32 Core Memory Structures (Continued)

Memory Structure	Data Width	Memory Instances	Memory Length	CFX Access Using
		DSP_ARAM4, DSP_ARAM5	2 x 2048	
B memory (RAM)	32	DSP_BRAM0, DSP_BRAM1	2 x 2048	P memory bus
		DSP_BRAM2	4096	

The "Other Memory Structures" table (Table 26) lists the other memory structures in the system, including those attached to the Filter Engine and the eMMC interface. This list includes the size and the width of each memory structure, and the bus that the CFX DSP uses to access the memory. These structures are described further in the following subsections.

Table 26. Other Memory Structures

Memory Structure	Data Width	Memory Length	CFX Access Using
Filter Engine Program memory (RAM)	36	256	P memory bus
Filter Engine State	24	320	
Filter Engine Tmp	28	64	
Filter Engine Coef	28	320	
eMMC Buffer (RAM)	32	512	P memory bus

11.2.1.1 Filter Engine Memory

The Filter Engine's functional block contains a 256 word (36 bits per word) program memory, and state variables and coefficient memories. The Filter Engine cannot write into the program and coefficients memories, but frequently reads from it, while it both reads and writes from the state variables memory. For more information about the Filter Engine's memory and registers, see [Chapter 6.1 "Processor Overview" on page 217](#).

11.2.1.2 eMMC Memory Instance

The eMMC interface is supported by a 512 word (32 bits per word) memory buffer, to simplify data transfers between the Ezairo 8300 device and high-density memories attached to the device using the eMMC interface. For more information about use of this buffer and the eMMC interface, see [Section 18.3 "eMMC Interface" on page 565](#).

The eMMC interface always has priority access to its memory instance. If a memory conflict occurs, and does not include the eMMC interface, the conflict is arbitrated as though the eMMC memory instance is one of the Arm Cortex-M3 processor's memory instances.

11.2.1.3 Memory Instance Power Control

Each memory instance in Ezairo 8300 can be put into 4 power states, controlled by the `SYSCTRL_MEM_POWER_CFG*` and `SYSCTRL_MEM_ACCESS_CFG*` registers:

Ezairo 8300 Hardware Reference

Disabled

If the memory instance's corresponding bit in `SYSCTRL_MEM_POWER_CFG*` is cleared, the memory is in Disabled mode. In this mode, memory is not powered and memory isolation is enabled. Memory instances do not retain their state in Disabled mode, and are inaccessible.

When a memory instance is in Disabled mode, its corresponding bit in the `SYSCTRL_MEM_ACCESS_CFG*` registers is cleared. To enable a memory instance, the instance must first be placed into Wakeup mode.

Retention

If the memory instance's corresponding bits in `SYSCTRL_MEM_POWER_CFG*` are set, the memory can be placed into Retention mode when using the ROM retention and sleep functions. In this mode, all memories that are powered are held in a low-power state, and memory isolation is enabled. Memory instances retain their state in Retention mode, but are inaccessible.

For more information on the ROM retention and sleep functions, see the *Ezairo 8300 Firmware Reference*.

Wakeup

If the memory instance's corresponding bit in `SYSCTRL_MEM_POWER_CFG*` is set, and the corresponding bit in the `SYSCTRL_MEM_ACCESS_CFG*` register is cleared, the memory is in Wakeup mode. In this mode, memory is held in a normal power state; however, memory isolation remains enabled. This mode is a transitional state between Retention or Disabled and Enabled modes, where memory instances retain their state, but are inaccessible.

NOTE: Transitions through the Wakeup mode last at least 500 ns. This delay is automatically enforced by inserting the necessary wait states during writes to `SYSCTRL_MEM_POWER_CFG*`. If multiple memories are set to the Wakeup mode at the same time, each memory instance is enabled in sequence at an automatically enforced interval of at least 500 ns to avoid drops on the VDDM supply voltage. The wait state delays are based on a single SLOWCLK period.

Enabled (default)

If the memory instance's corresponding bits in `SYSCTRL_MEM_POWER_CFG*` and `SYSCTRL_MEM_ACCESS_CFG*` are set, the memory is in Enabled mode. In this mode, memory is held in a normal power mode, with memory isolation disabled. Memories are fully functional in Enabled mode.

If a memory is not in Enabled mode, access to that memory produces a bus fault or `MEM_ERROR` interrupt:

- If the failed memory access occurs on the Arm Cortex-M3 processor, a bus fault is generated.
- If the failed memory access occurs on any other access, a `MEM_ERROR` has occurred. The `SYSCTRL_MEM_ERROR` register indicates if the error occurred on an access from the CFX (`CFX_MEM_ERROR`), LPDSP32 (`DSP_MEM_ERROR`), or DMA (`DMA_MEM_ERROR`).

IMPORTANT: To minimize power consumption, all unused memories should be placed in the disabled state.

Ezairo 8300 Hardware Reference

11.2.2 Memory Bus Structures

Four memory buses connect the various memory instances to the CFX DSP. The primary connections and uses for these buses, which can be seen in the "[System Memory Architecture](#)" figure (Figure 15), are as follows:

Instruction Bus

20-bit addressable bus used by the CFX DSP to fetch program instructions for execution. Additionally, this bus gives the CFX processor access to the interrupt vector table.

X Memory Bus

20-bit addressable bus used by the CFX DSP to access data in X memory, XY memory, and shared memory. This bus also provides the CFX with access to the memory-mapped control and configuration registers for the HEAR and for the FIFOs. This bus provides the least significant word of any accesses to the compound XY memory bus.

Y Memory Bus

20-bit addressable bus used by the CFX DSP to access data in Y memory and XY memory. This bus provides the most significant word of any accesses to the compound XY memory bus.

P Memory Bus

20-bit addressable bus used by the CFX DSP to access program and microcode memory as data. This bus also provides the CFX with access to the memory-mapped control and configuration registers used to configure analog and digital system components. This bus can also be referred to as the IO memory bus.

Buses connected to the Arm Cortex-M3 processor implement the standard Arm Cortex-M3 core memory map. All buses share the same 32-bit Arm Cortex-M3 processor memory space. These buses, which, with the exception of the Peripheral Bus and APB Bus, can also be seen in the "[System Memory Architecture](#)" figure (Figure 15), are as follows:

I-Code Bus

Allows the Arm Cortex-M3 processor to fetch instruction information from the Arm Cortex-M3 core program memory instance or PRAM0 memory instance

D-Code Bus

Allows the Arm Cortex-M3 processor to fetch data information from the Arm Cortex-M3 core program memory instance or PRAM0 memory instance

System Bus

Allows the Arm Cortex-M3 processor to fetch instructions and data information from the Arm Cortex-M3 core data memory instance. This bus also provides access to the Peripheral Bus, APB bus and bus bridges to the CFX X, Y, P and FIFO buses.

Peripheral Bus

Allows the Arm Cortex-M3 processor to access memory-mapped peripherals

APB Bus

Allows the Arm Cortex-M3 processor to access the I³C and eMMC interfaces

Private Peripheral Bus

Ezairo 8300 Hardware Reference

Allows the Arm Cortex-M3 processor to access standard memory-mapped peripherals. This includes the NVIC, SysTick timer, and the Arm Cortex-M3 processor debug port. This bus and the peripherals memory-mapped onto it are accessible by only the Arm Cortex-M3 processor.

An additional four buses attached to the HEAR Accelerator provide access to microcode memory and shared memory for the HEAR. These buses, which can also be seen in the "System Memory Architecture" figure (Figure 15), are set up in matched pairs to increase the data throughput of the HEAR:

Microcode Memory Buses

Two buses that allow the HEAR to fetch configuration information from the microcode memory instance

Shared Memory Buses

Two buses that allow the HEAR to access all of shared memory through the shared memory bus controller

The LPDSP32 DSP processor is supported by three buses:

LPDSP32 P Memory Bus

This bus provides access to the LPDSP32 program memory instances. This bus uses 24-bit addressing of 20-bit data words, with the combined 40-bit data organized in a big-endian manner.

A Memory Bus

This bus provides access to the A memories from the LPDSP32 data memory instances. This bus uses 24-bit addresses for byte addressed data, with four bytes organized into words in a little-endian manner.

B Memory Bus

This bus provides access to the B memories from the LPDSP32 data memory instances. This bus uses 24-bit addresses for byte addressed data, with four bytes organized into words in a little-endian manner.

11.2.3 Memory Controllers and Arbitration

As shown in the "System Memory Architecture" figure (Figure 15), the memory architecture contains several controllers, which operate between memory buses attached to the cores, and part or all of the shared memories. Memory access control, arbitration, and write buffering using these blocks is described in the following sub-sections.

External breakpoints on the CFX debug port can be used in memory conflict mode, to detect when any memory conflicts between any two of the CFX DSP, the HEAR Configurable Accelerator, and the Arm Cortex-M3 processor occur. For more information, see [Section 17.1.2.2.2 "Memory Conflict Modes" on page 531](#).

11.2.3.1 CFX Memory Arbitration

Memory accesses to CFX memories from the Arm Cortex-M3 processor and the debug port are performed using unobstrusive accesses. These accesses have the following fixed priority:

1. CFX DSP
2. Arm Cortex-M3 processor
3. Debug Port

Ezairo 8300 Hardware Reference

Unobtrusive access is granted whenever a higher priority core does not use the memory for one or more cycles, and there is no pending data from the relevant bus for a higher priority core due to a wait state.

Additionally, wait states for accesses to CFX memories can be introduced by:

- Instruction and P memory access conflicts on the CFX. See below for more information.
- Access conflicts with the HEAR configurable accelerator
 - Conflicts on microcode memory accesses are processed with instruction and P memory bus conflicts. See below for more information.
 - Conflicts on shared memory access are controlled by the Shared Memory Controller (see [Section 11.2.3.4 “Shared Memory Bus Controller” on page 347](#) for more information).
- An Arm Cortex-M3 or debug port unobtrusive accesses on a shared memory or memory-mapped register, which triggers a wait state

When the CFX P memory bus is used to read or write, a bus conflict occurs if the CFX instruction bus is currently using the memory block to be accessed. When a bus conflict occurs, the CFX DSP is held in a wait state, which incurs a delay in the completion of the instruction accessing P memory. This wait state holds the CFX pipeline to ensure that this delay will have no impact on the execution of instructions in delay slots.

Note that a bus conflict does *not* occur, and P memory access proceeds without introducing a delay, if any of the following conditions hold:

- The CFX instruction and data are located in different memory blocks (see [Section 11.2.1 “Memory Instances” on page 339](#) for a list of memory blocks).
- The CFX is currently in the first execution cycle of a multi-cycle instruction, including *Move-Move* instructions, so that it is not pre-fetching the next instruction.

Similar bus conflicts can also occur between the CFX P memory bus and the HEAR Accelerator core when both are accessing the microcode memory. This conflict is handled using the `SYSTEM_CTRL_PRIORITY_STATUS` bit field in the `D_SYSTEM_CTRL` register. This bit field is configured using the `SYSTEM_CTRL_PRIORITY_HEAR` and `SYSTEM_CTRL_PRIORITY_CFX` bits of the `D_SYSTEM_CTRL` register. The `SYSTEM_CTRL_PRIORITY_STATUS` bit field in the `D_SYSTEM_CTRL` register indicates which core has priority. The state of this bit can be changed with the `SYSTEM_CTRL_PRIORITY_HEAR` and `SYSTEM_CTRL_PRIORITY_CFX` bit fields in the `D_SYSTEM_CTRL` register. The processor with priority retains access to the conflicting memory structure until the conflict is removed or resolved, while the other core is held in a wait state.

11.2.3.2 CFX Memory Interface Write Buffers

The CFX DSP core contains a write buffer for each of the P memory, X memory, and Y memory buses. The buffers allow the CFX to perform any combination of memory-read and memory-write operations without stalling the instruction pipeline. A buffered memory-write operation generally does not affect program behavior and is transparent to the user application, but the CFX's write buffering is visible in some situations, which are explained further in the following subsections.

11.2.3.2.1 Buffering and Flushing of Write Operations

A memory-write operation is buffered by the CFX when a read operation from the same memory space as the write occurs in the clock cycle following the write. This is necessary, since a write is performed in the Execute stage of the instruction pipeline while a read is being performed in the Decode stage (see the *CFX DSP Architecture Manual for Ezairo 8300* for details about the Program Control Unit's execution pipeline). This means when a write is immediately followed by a read from the same memory, both operations need the bus at the same time. The read operation takes priority and the write operation is buffered.

Ezairo 8300 Hardware Reference

A buffered write is flushed (stored to memory) as soon as the bus has a cycle during which it is not busy, which occurs for any instruction that is not a read from the same memory space. This includes:

- Another write operation (during its Decode stage the memory bus is free)
- Memory operations to a different memory space
- Control-flow instructions (including interrupt handling)
- Register-based operations

In summary, when a write operation is followed by a consecutive sequence of read operations from the same memory, the write is only stored to memory after all of the read operations have occurred. If any of the read operations read from the same address to which the buffered write stores its value, the read operation returns the write operation's buffered value.

11.2.3.2.2 Visible Effects of Write Buffering

Write buffering can be visible to a user application for two reasons:

1. The delay in actually storing a value to memory
2. Reading a buffered value instead of memory. This is visible where one or more memory bits are not instantiated, or they use different read and write definitions.

Several control and configuration registers can behave unexpectedly for the second reason if you write to the register's P memory location, and then immediately read it back so that you read the buffered value. Examples of special register behaviors that are not implemented by a buffered value include:

- Some bits might not be defined (for example, `D_WATCHDOG_CFG`).
- Writing a one to a bit clears the bit that has been set, or otherwise changes the register values or initiates an action (for example, `D_SYSTEM_CTRL`).
- The bits in a register have different meanings when read than when written (for example, `D_INT_STATUS_0`).
- Reading a register changes its value or initiates an action (for example, `D_SPI_DATA` when the SPI interface is configured for auto mode).

Visible write buffering due to a delayed memory store is less common. It is usually combined with writing a control register that automatically modifies another memory location (typically another control register). Since the store is delayed, the side-effect modification has not yet occurred if the other memory location is read immediately after the write. For example:

- Blocks such as the CRC Generator, where a read of `D_CRC_VALUE` would not include the value written to any `D_CRC_ADD_x` registers in the previous cycle
- Reading back the interrupt enable or priority bits in `D_INT_ENABLE` or `D_INT_PRIORITY` immediately after writing them through `D_INT_EBL_x` or `D_INT_PRIO_x`

Although all of the preceding examples involve control registers in P memory, similar write buffering affects registers mapped into X or Y memory. Examples include the Math Accelerator registers, FIFO registers, HEAR function chain controller registers, and HEAR block floating point registers. The value written to a register such as `D_FIFO_xx_BLOCK_CNT` might not match the value returned by that register, and the value of other registers in the same block could be updated by the value written to the `D_FIFO_xx_BLOCK_CNT` register.

IMPORTANT: In all of the above examples, you can avoid the undesirable side effects of write buffering by using the CFX IOREG memory space (for C applications), or by inserting (after the write operation) any instruction that does not read the same memory space (for assembly applications). If you cannot move an existing instruction into place after the write, an NOP instruction can be used.

11.2.3.3 Arm Cortex-M3 Memory Arbitration

Memory accesses to memories in the Arm Cortex-M3 processor space, including the peripheral bus and APB bus supporting the I³C and eMMC interfaces, are controlled by an arbiter defined for each memory instance. The priority scheme for all of these arbiters is configurable using the `SYSCTRL_MEM_ARBITER_CFG_PRIORITY_ORDER` bit-field of the `SYSCTRL_MEM_ARBITER_CFG` register. This selects between the following fixed priority sequences:

- Arm Cortex-M3 processor > LPDSP32 > CFX > DMA
- Arm Cortex-M3 processor > CFX > LPDSP32 > DMA
- LPDSP32 > CFX > Arm Cortex-M3 processor > DMA
- CFX > LPDSP32 > Arm Cortex-M3 processor > DMA

High-priority accesses can be configured for memories using a maximum wait time mechanism, which is provided for each of the processors and the DMA. The delay before granting of a high-priority access is defined in the `SYSCTRL_MEM_ARBITER_CFG_DMA_MAX_WAIT`, `SYSCTRL_MEM_ARBITER_CFG_CFX_MAX_WAIT`, `SYSCTRL_MEM_ARBITER_CFG_DSP_MAX_WAIT` and `SYSCTRL_MEM_ARBITER_CFG_CM3_MAX_WAIT` fields of the `SYSCTRL_MEM_ARBITER_CFG` register. This configuration allows each source to have its access promoted to high priority after not being granted access for 3, 7, 15, or 31 cycles. If more than one source is promoted to high-priority at the same time, the fixed priority ordering listed above grants precedence amongst high-priority accesses.

NOTE: Individual DMA channels can be configured to have background priority, rather than real-time priority. DMA channels configured for background priority will never be granted high-priority access.

11.2.3.4 Shared Memory Bus Controller

The shared memory bus controller coordinates memory accesses and addressing into the underlying HEAR shared memories, separately directing data between the shared memory instances and:

- The X and Y memory buses that are attached to the CFX
- The shared memory buses attached to the HEAR

For more information about the shared memory itself, see [Section 11.5.2 “Shared Memory” on page 365](#).

If a memory conflict is detected, it is handled as described in [Section 11.2.3.1 “CFX Memory Arbitration” on page 344](#).

The shared memory controller pre-arbitrates some accesses to the A0, A1, B0, and B1 memories that are determined through access supervised by the FIFO controller. For more information, see [Section 11.2.3.5 “FIFO Controller”](#) below.

11.2.3.5 FIFO Controller

The FIFO controller is connected to the IOCs, the shared memory bus controller, and the Arm Cortex-M3 core through a bus bridge. This controller is used to provide FIFO all accesses/addressing into the A0, A1, B0 and B1 memories while coordinating and controlling memory access requests to these memories. To control which access has priority, and to set the sequence in which the FIFOs receive accesses, the FIFO controller uses a fixed access priority scheme. The priorities enforced by the FIFO controller are:

1. Direct read/write access by the IOCs. If multiple IOC triggers are pending at the same time, then they are handled in the following order:
 - a. IOC0 ADC/Filter Engine inputs
 - b. IOC1 ADC/Filter Engine inputs
 - c. IOC0 SDM outputs
 - d. IOC1 SDM outputs
 - e. PCM0-1 RX
 - f. PCM0-1 TX
2. Read/write access using an access register (D_FIFO_xx_ACCESS) for a FIFO through the shared memory bus controller
3. Write access to a block counter register (D_FIFO_xx_BLOCK_CNT) for a FIFO through the shared memory bus controller
4. Read/write access to any remaining FIFO register(s) through the shared memory bus controller
5. Read/write access to any area outside of a defined FIFO buffer through the shared memory bus controller (linear access)
6. Read/write access to an area within a FIFO buffer through the shared memory bus controller
7. Read/write access to an area outside of a FIFO buffer through the Arm Cortex-M3 core access to the FIFO controller.
8. Read/write access to an area within a FIFO buffer through the Arm Cortex-M3 core access to the FIFO controller.

For more information on the FIFO controller operation itself, see [Chapter 12.1 "Introduction to the FIFO Controller" on page 387](#).

11.2.3.6 Filter Engine Memory Accesses

Memory conflicts between the CFX DSP or Arm Cortex-M3 processor and the Filter Engine follow a strict priority scheme with no arbitration. In general, conflicting accesses between the CFX DSP/Arm Cortex-M3 processor and the Filter Engine result in a failure of the CFX or Arm Cortex-M3 processor's memory access. For more information about Filter Engine memory accesses conflicts, see [Section 6.5.3 "Memory and Register Access Conflicts" on page 227](#).

11.3 CFX PROCESSOR MEMORY MAPPING

For reference by an application, the start, end, and size of the memory mapped areas listed in the following sections are provided as `#define` statements in the `sk5_cfx_map.h` file.

11.3.1 Program and P Memory

The CFX program and P memory areas are related memory views that share an address space, and that are accessed through the instruction bus and P memory bus respectively. The "CFX Program and P Memory Map" figure ([Figure 16](#)) provides a visual representation of these memory spaces, which are described in detail in the following sections.

Ezairo 8300 Hardware Reference

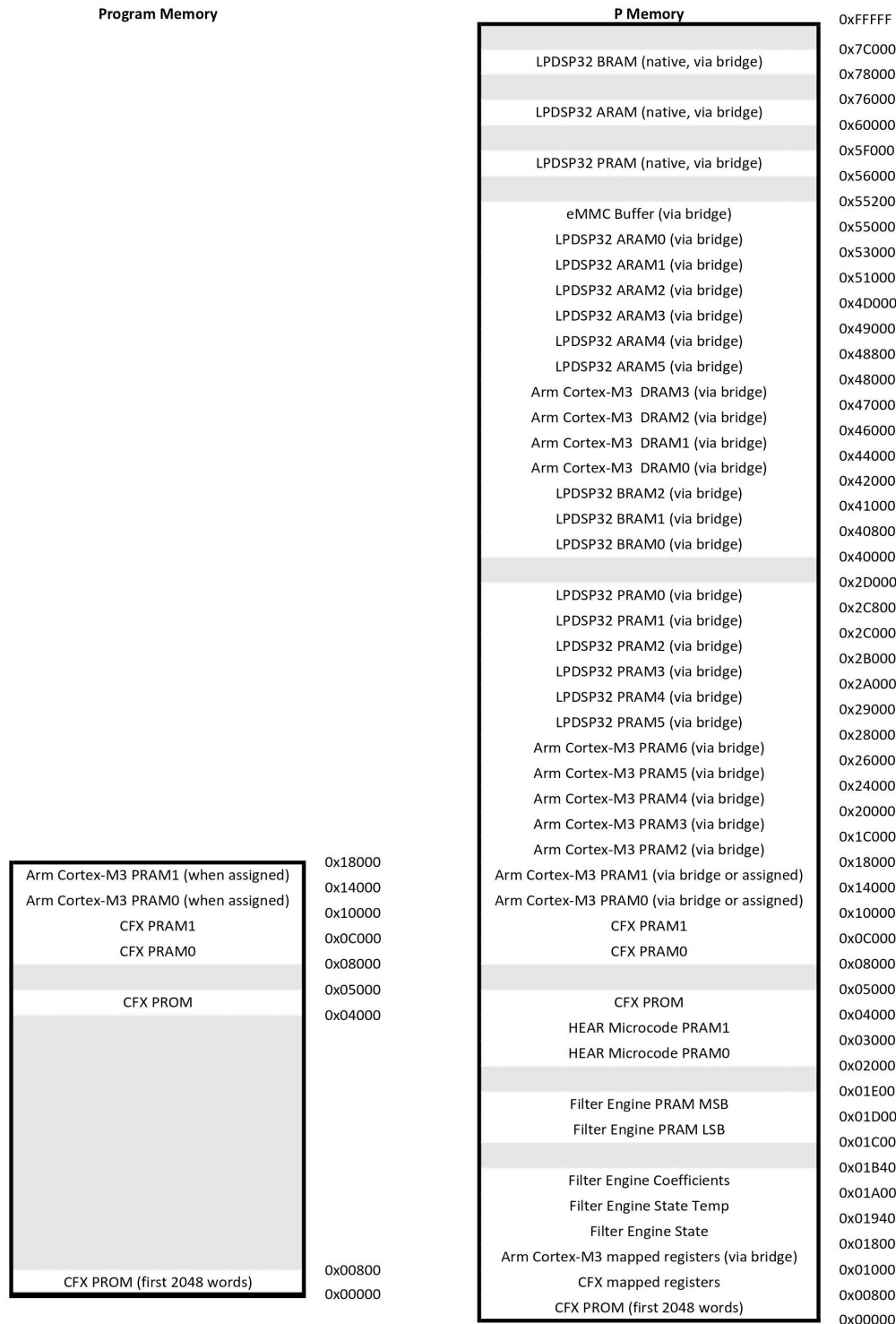


Figure 16. CFX Program and P Memory Map

Ezairo 8300 Hardware Reference

11.3.1.1 Program Memory Space

Instructions to execute on the CFX are fetched from the program memory through the dedicated instruction bus. Access on this bus is restricted to this use; therefore, all data access to the program memory areas occurs over the P memory bus.

The program memory space defines the RAM and ROM instances of CFX program memory that are accessible through the instruction bus. This is shown in the "Program Memory Mapping" table (Table 27).

Table 27. Program Memory Mapping

Start Address	End Address	Description	Words
0x00000	0x007FF	CFX PROM (partial mirror)	2048
0x04000	0x04FFF	CFX PROM	4096
0x08000	0x0BFFF	CFX PRAM0	16384
0x0C000	0x0FFFF	CFX PRAM1	16384
0x10000	0x13FFF	Arm Cortex-M3 PRAM0 (when assigned)	16384
0x14000	0x17FFF	Arm Cortex-M3 PRAM1 (when assigned)	16384

The program ROM is located from 0x4000 to 0x4FFF (first 2048 words mirrored from 0x0000 to 0x07FF) of the program memory space. This ROM contains code for booting the chip as well as system-level routines for performing common tasks. For detailed information about the contents of the program ROM, see the *Ezairo 8300 Firmware Reference*.

11.3.1.2 P Memory Space

The P memory space (P memory) allows the CFX to access a proper superset of the program memory space for use as data, as shown in Table 28, below. The P memory bus provides access to:

- Program memory instances for the CFX
- Arm Cortex-M3 program and data memory instances
- Microcode memory instances for the HEAR Configurable Accelerator
- LPDSP32 program and data memory instances
- Filter Engine memory instances and registers
- Memory-mapped control and configuration registers for the majority of analog and digital components (excludes elements on the Arm Cortex-M3 private peripheral bus)
- Access to interrupt vectors for use by the CFX interrupt controller and the CFX core

NOTE: The memories that are accessible to the LPDPS32 are mapped into the CFX P memory space's memory map in two locations. These mappings provide access to the same memories, ordered as they are accessible natively to the LPSPD32 in its memory maps (marked as "native" mappings in the "P Memory Mapping" table (Table 28)) and also as they are remapped for use by the CFX DSP for any memory instances not used by the LPDSP32.

onsemi Confidential

Ezairo 8300 Hardware Reference

Table 28. P Memory Mapping

Start Address	End Address	Description	Words
0x00000	0x007FF	CFX PROM (partial mirror)	2048
0x00800	0x00FFF	CFX mapped registers	2048
0x01000	0x017FF	Arm Cortex-M3 mapped registers (via bridge)	2048
0x01800	0x0193F	Filter Engine State Variables	320
0x01940	0x0197F	Filter Engine Temporary State Variables	64
0x01A00	0x01B3F	Filter Engine Coefficients	320
0x01C00	0x01CFF	Filter Engine PRAM LSB	256
0x01D00	0x01DFF	Filter Engine PRAM MSB	256
0x02000	0x02FFF	HEAR Microcode PRAM0	4096
0x03000	0x03FFF	HEAR Microcode PRAM1	4096
0x04000	0x04FFF	CFX PROM	4096
0x08000	0x0BFFF	CFX PRAM0	16384
0x0C000	0x0FFFF	CFX PRAM1	16384
0x10000	0x13FFF	Arm Cortex-M3 PRAM0 (via bridge or assigned)	16384
0x14000	0x17FFF	Arm Cortex-M3 PRAM1 (via bridge or assigned)	16384
0x18000	0x1BFFF	Arm Cortex-M3 PRAM2 (via bridge)	16384
0x1C000	0x1FFFF	Arm Cortex-M3 PRAM3 (via bridge)	16384
0x20000	0x23FFF	Arm Cortex-M3 PRAM4 (via bridge)	16384
0x24000	0x25FFF	Arm Cortex-M3 PRAM5 (via bridge)	8192
0x26000	0x27FFF	Arm Cortex-M3 PRAM6 (via bridge)	8192
0x28000	0x28FFF	LPDSP32 PRAM5 (via bridge)	4096
0x29000	0x29FFF	LPDSP32 PRAM4 (via bridge)	4096
0x2A000	0x2AFFF	LPDSP32 PRAM3 (via bridge)	4096
0x2B000	0x2BFFF	LPDSP32 PRAM2 (via bridge)	4096
0x2C000	0x2C7FF	LPDSP32 PRAM1 (via bridge)	2048
0x2C800	0x2CFFF	LPDSP32 PRAM0 (via bridge)	2048
0x40000	0x407FF	LPDSP32 BRAM0 (via bridge)	2048
0x40800	0x40FFF	LPDSP32 BRAM1 (via bridge)	2048
0x41000	0x41FFF	LPDSP32 BRAM2 (via bridge)	4096
0x42000	0x43FFF	Arm Cortex-M3 DRAM0 (via bridge)	8192
0x44000	0x45FFF	Arm Cortex-M3 DRAM1 (via bridge)	8192
0x46000	0x46FFF	Arm Cortex-M3 DRAM2 (via bridge)	4096

P Memory Mapping (Continued)

Start Address	End Address	Description	Words
0x47000	0x47FFF	Arm Cortex-M3 DRAM3 (via bridge)	4096
0x48000	0x487FF	LPDSP32 ARAM5 (via bridge)	2048
0x48800	0x48FFF	LPDSP32 ARAM4 (via bridge)	2048
0x49000	0x4CFFF	LPDSP32 ARAM3 (via bridge)	16384
0x4D000	0x50FFF	LPDSP32 ARAM2 (via bridge)	16384
0x51000	0x52FFF	LPDSP32 ARAM1 (via bridge)	8192
0x53000	0x54FFF	LPDSP32 ARAM0 (via bridge)	8192
0x55000	0x551FF	eMMC Buffer (via bridge)	512
0x56000	0x56FFF	LPDSP32 PRAM01 (native, via bridge)	4096
0x57000	0x58FFF	LPDSP32 PRAM23 (native, via bridge)	8192
0x59000	0x5AFFF	LPDSP32 PRAM45 (native, via bridge)	8192
0x5B000	0x5EFFF	Arm Cortex-M3 PRAM56 (LPDSP32 native, via bridge)	16384
0x60000	0x63FFF	LPDSP32 ARAM01 (native, via bridge)	16384
0x64000	0x6BFFF	LPDSP32 ARAM23 (native, via bridge)	32768
0x6C000	0x6CFFF	LPDSP32 ARAM45 (native, via bridge)	4096
0x6D000	0x6DFFF	LPDSP32 BRAM01 (native, via bridge)	4096
0x6E000	0x6FFFF	LPDSP32 PRAM45 (native, via bridge)	8192
0x70000	0x73FFF	Arm Cortex-M3 PRAM56 (LPDSP32 native, via bridge)	16384
0x74000	0x75FFF	Arm Cortex-M3 DRAM23 (LPDSP32 native, via bridge)	8192
0x78000	0x787FF	LPDSP32 BRAM0 (native, via bridge)	2048
0x78800	0x78FFF	LPDSP32 BRAM1 (native, via bridge)	2048
0x79000	0x79FFF	LPDSP32 BRAM2 (native, via bridge)	4096
0x7A000	0x7AFFF	Arm Cortex-M3 DRAM3 (LPDSP32 native, via bridge)	4096
0x7B000	0x7BFFF	Arm Cortex-M3 DRAM2 (LPDSP32 native, via bridge)	4096

11.3.1.3 Program Memory Cache

To increase the power-efficiency of the CFX DSP, a 32-word write-through instruction cache has been included on the instruction bus between the CFX DSP and the program memory. This cache is designed to reduce the power consumed by constantly accessing program memory to fetch subsequent instructions for the CFX DSP. There is no detectable difference in functional code execution when using the .

Ezairo 8300 Hardware Reference

The program memory cache automatically reconfigures whenever a backward branch of less than 256 words indicates that a loop has been detected. At this point, the cache aligns itself to ensure that it stores the largest possible number of instructions from the loop.

11.3.1.4 Interrupt Vector Mapping

The interrupt vector table, which is shared by the interrupt controller and the CFX DSP, is mapped into the program memory between addresses 0x8000 and 0x805D. The specific interrupt vectors and their offsets within this range are provided in [Section 3.2 “Interrupt Controller”](#) on page 61.

11.3.1.5 Configuration Register Mapping

The analog and digital control and configuration registers are mapped into the P memory space between addresses 0x00800 and 0x01800. These registers are organized by their area of control and mapped as blocks into the P memory space, as shown in the ["Analog and Digital Control and Configuration Register Block Mapping"](#) table (Table 29).

Table 29. Analog and Digital Control and Configuration Register Block Mapping

Controlled/Configured Area	Area Type	Block Start Address
System Control	CFX Digital	0x00800
Watchdog Timer Configuration	CFX Digital	0x00801
Boot Status	CFX Digital	0x00803
HEAR Function Chain Commands	CFX Digital	0x00804
PRAM Security	CFX Digital	0x00809
CFX/Arm Cortex-M3 Core Control and Configuration	CFX Digital	0x0080B
External Breakpoint Configuration	CFX Digital	0x00810
Hardware CRC	CFX Digital	0x00822
Chip Version Information	CFX Digital	0x00830
Debug Port Configuration	CFX Digital	0x00835
Interrupt Configuration	CFX Digital	0x00840
Clock Configuration	CFX Digital	0x008C0
IOC Configuration	CFX Digital	0x00090
Analog	CFX Analog	0x00980
Audio	CFX Audio	0x009C0
I2C Interface	CFX Digital	0x00A00
FENG Control and Configuration	CFX Digital	0x00B00
FENG Register File	CFX Digital	0x00B40
System Control	Arm Cortex-M3 Digital	0x01000
General-Purpose Timers [3:0]	Arm Cortex-M3 Digital	0x01040
DMA Controller Configuration and Control [7:0]	Arm Cortex-M3 Digital	0x01080

Ezairo 8300 Hardware Reference

Analog and Digital Control and Configuration Register Block Mapping (Continued)

Controlled/Configured Area	Area Type	Block Start Address
Audio Sink Clock Counters [1:0]	Arm Cortex-M3 Digital	0x010C0
ASRC Configuration and Control	Arm Cortex-M3 Digital	0x01140
CRC Generator Control	Arm Cortex-M3 Digital	0x01180
G.277 Configuration and Control	Arm Cortex-M3 Digital	0x011C0
Reed Solomon Configuration and Control	Arm Cortex-M3 Digital	0x01200
DIO Interface and Digital Pad Control	Arm Cortex-M3 Digital	0x01240
PCM Interface Configuration and Control [1:0]	Arm Cortex-M3 Digital	0x012C0
SPI Interface Configuration and Control [1:0]	Arm Cortex-M3 Digital	0x01340
UART Interface Configuration and Control	Arm Cortex-M3 Digital	0x013C0
I ² C Interface Configuration and Control [1:0]	Arm Cortex-M3 Digital	0x01400
Low Speed Analog-to-Digital Converter and Battery Monitoring	Arm Cortex-M3 Analog	0x01480
eMMC and I ³ C Interface Control	Arm Cortex-M3 Digital	0x014C0
I ³ C Interface Configuration and Control [1:0]	Arm Cortex-M3 Digital	0x01500
eMMC Interface Configuration Control	Arm Cortex-M3 Digital	0x01700
Neural Network Accelerator	Arm Cortex-M3 Digital	0x017C0
CFX Math Accelerators I/O and Control	CFX Digital (X)	X:0x00800
FIFO A0 Configuration and Control [7:0]	HEAR Digital	X:0x01000
FIFO A1 Configuration and Control [7:0]	HEAR Digital	X:0x01080
FIFO B0 Configuration and Control [7:0]	HEAR Digital	X:0x01100
FIFO B1 Configuration and Control [7:0]	HEAR Digital	X:0x01180
FIFO Interrupt Selection	HEAR Digital	X:0x01200
HEAR Interrupt Configuration and Control	HEAR Digital	X:0x01230
Block Floating Point Configuration and Control	HEAR Digital	X:0x01260
CFX Math Accelerators I/O	CFX Digital (Y)	Y:0x00800

11.3.2 Data Memory

The data memory areas are related memories that are used to store all types of signal and control data. The CFX has direct access to a number of data memories that a user application can access as needed. All data memory attached to the HEAR Accelerator core is also available to the CFX as shared memory, although all cores in the system access the memory through the shared memory bus controller. The "CFX X, Y Memory Maps" figure (Figure 17) provides a visual representation of these memory spaces, which are described in detail in the following sections.

Ezairo 8300 Hardware Reference

X Memory		Y Memory	
	0x80000		
BD1 memory	0x7C000		
BD0 memory	0x78000		
AC1 memory	0x74000		
AC0 memory	0x70000		
D1 memory	0x6E000		
D0 memory	0x6C000		
C1 memory	0x6A000		
C0 memory	0x68000		
B1 memory	0x66000		
B0 memory	0x64000		
A1 memory	0x62000		
A0 memory	0x60000		
CD1 memory	0x5C000		
CD0 memory	0x58000		
AB1 memory	0x54000		
AB0 memory	0x50000		
SIN/COS Data Table Radix-3 (ROM)	0x4D000		
SIN/COS Data Table (ROM)	0x4C000		
H12 memory	0x49000		
H03 memory	0x48000		
H13 memory	0x47000		
H02 memory	0x46000		
H5 memory	0x45800		
H4 memory	0x45000		
H3 memory	0x44800		
H2 memory	0x44000		
H1 memory	0x43800		
H0 memory	0x43000		
H45 memory	0x42000		
H23 memory	0x41000		
H01 memory	0x40000		
YRAM1 remapped	0x15000		
YRAM2 remapped	0x14000		
XRAM3	0x10000		
XRAM2	0x0C000		
XRAM1	0x08000		
			0x08000
		YRAM2	0x07000
		YRAM1	0x06000
		YRAM0	0x04000
			0x01000
		Math Accelerator registers	0x00800
			0x00000
XRAM0	0x04000		
HEAR registers	0x01000		
Math Accelerator registers	0x00800		
Synthesized ROM (AES S-box)	0x00000		

Figure 17. CFX X, Y Memory Maps

Ezairo 8300 Hardware Reference

This section provides detailed information about these memory components; all of the related components that are accessible through the X, Y, and shared memory buses; and memory components and related components that are located in the X, Y, and XY memory maps.

11.3.2.1 CFX Data Memories (RAM and ROM)

Data memory attached to the CFX DSP primarily consists of two banks of 24-bit memory, which can be accessed either individually as 24-bit memory, or in combination as a single bank of 48-bit memory. These two individual memories are referred to as X memory and Y memory respectively. X memory is located from 0x04000 to 0x13FFF in the X memory space and is accessed through the X memory bus. Y memory is located from 0x04000 to 0x07FFF in the Y memory space and is accessed through the Y memory bus. This is shown in the "XMEM Mapping" table (Table 30) and the "YMEM Mapping" table (Table 31).

Table 30. XMEM Mapping

Start Address	End Address	Description	Words
0x00000	0x004FF	Synthesized ROM (AES S-box)	1280
0x00800	0x00FFF	Mapped registers (Math Accelerators)	2048
0x01000	0x01FFF	HEAR mapped registers	4096
0x04000	0x07FFF	XRAM0	16384
0x08000	0x0BFFF	XRAM1	16384
0x0C000	0x0FFFF	XRAM2	16384
0x10000	0x13FFF	XRAM3	16384
0x14000	0x14FFF	YRAM2 remapped	4096
0x15000	0x15FFF	YRAM1 remapped	4096
0x40000	0x40FFF	H01 memory	4096
0x41000	0x41FFF	H23 memory	4096
0x42000	0x42FFF	H45 memory	4096
0x43000	0x437FF	H0 memory	2048
0x43800	0x43FFF	H1 memory	2048
0x44000	0x447FF	H2 memory	2048
0x44800	0x44FFF	H3 memory	2048
0x45000	0x457FF	H4 memory	2048
0x45800	0x45FFF	H5 memory	2048
0x46000	0x46FFF	H02 memory	4096
0x47000	0x47FFF	H13 memory	4096
0x48000	0x48FFF	H03 memory	4096
0x49000	0x49FFF	H12 memory	4096
0x4C000	0x4C7FF	SIN/COS Data Table (ROM)	2048
0x4D000	0x4DBFF	SIN/COS Data Table Radix-3 (ROM)	3072
0x50000	0x53FFF	AB0 memory	16384

XMEM Mapping (Continued)

Start Address	End Address	Description	Words
0x54000	0x57FFF	AB1 memory	16384
0x58000	0x5BFFF	CD0 memory	16384
0x5C000	0x5FFFF	CD1 memory	16384
0x60000	0x61FFF	A0 memory	8192
0x62000	0x63FFF	A1 memory	8192
0x64000	0x65FFF	B0 memory	8192
0x66000	0x67FFF	B1 memory	8192
0x68000	0x69FFF	C0 memory	8192
0x6A000	0x6BFFF	C1 memory	8192
0x6C000	0x6DFFF	D0 memory	8192
0x6E000	0x6FFFF	D1 memory	8192
0x70000	0x73FFF	AC0 memory	16384
0x74000	0x77FFF	AC1 memory	16384
0x78000	0x7BFFF	BD0 memory	16384
0x7C000	0x7FFFF	BD1 memory	16384

Table 31. YMEM Mapping

Start Address	End Address	Description	Words
0x00800	0x00FFF	Mapped registers (math accelerators)	2048
0x04000	0x05FFF	YRAM0	8192
0x06000	0x06FFF	YRAM1	4096
0x07000	0x07FFF	YRAM2	4096

XY memory is a combined memory space that is addressable from 0x04000 to 0x07FFF. Each addressable element is a double-word 48-bit value using the data at the corresponding X memory address as the low word, and at the corresponding Y memory address as the high word. XY memory is accessed through both the X and Y memory buses together as the XY memory bus.

NOTE: As XY memory uses physical memory containing both the X memory instance and the Y memory instance located between 0x04000 and 0x07FFF, we generally recommend that any space reserved for XY memory also be reserved in both X memory and Y memory.

Shared memories from the HEAR Configurable Accelerator are double-word structured 48-bit memories that allow 24-bit word addressing by the CFX through the X memory space. These memories are described in [Section 11.5.2 “Shared Memory” on page 365](#).

11.4 ARM CORTEX-M3 PROCESSOR MEMORY MAPPING

The Arm Cortex-M3 processor uses a combined 32-bit, byte addressable memory space for all of its memories. Four bytes form a 32-bit memory word in a little endian manner. The layout of the Arm Cortex-M3 memory map is fixed, and can be broken down into the following top-level sections:

- Code (0x00000000 to 0x1FFFFFFF)
- SRAM (0x20000000 to 0x3FFFFFFF)
- Peripherals (0x40000000 to 0x5FFFFFFF)
- Private Peripherals (0xE0000000 to 0xE00FFFFF)

Use of this memory mapping is shown in the "Arm Cortex-M3 Core System Memory Map" figure (Figure 18).

onsemi Confidential

Ezairo 8300 Hardware Reference

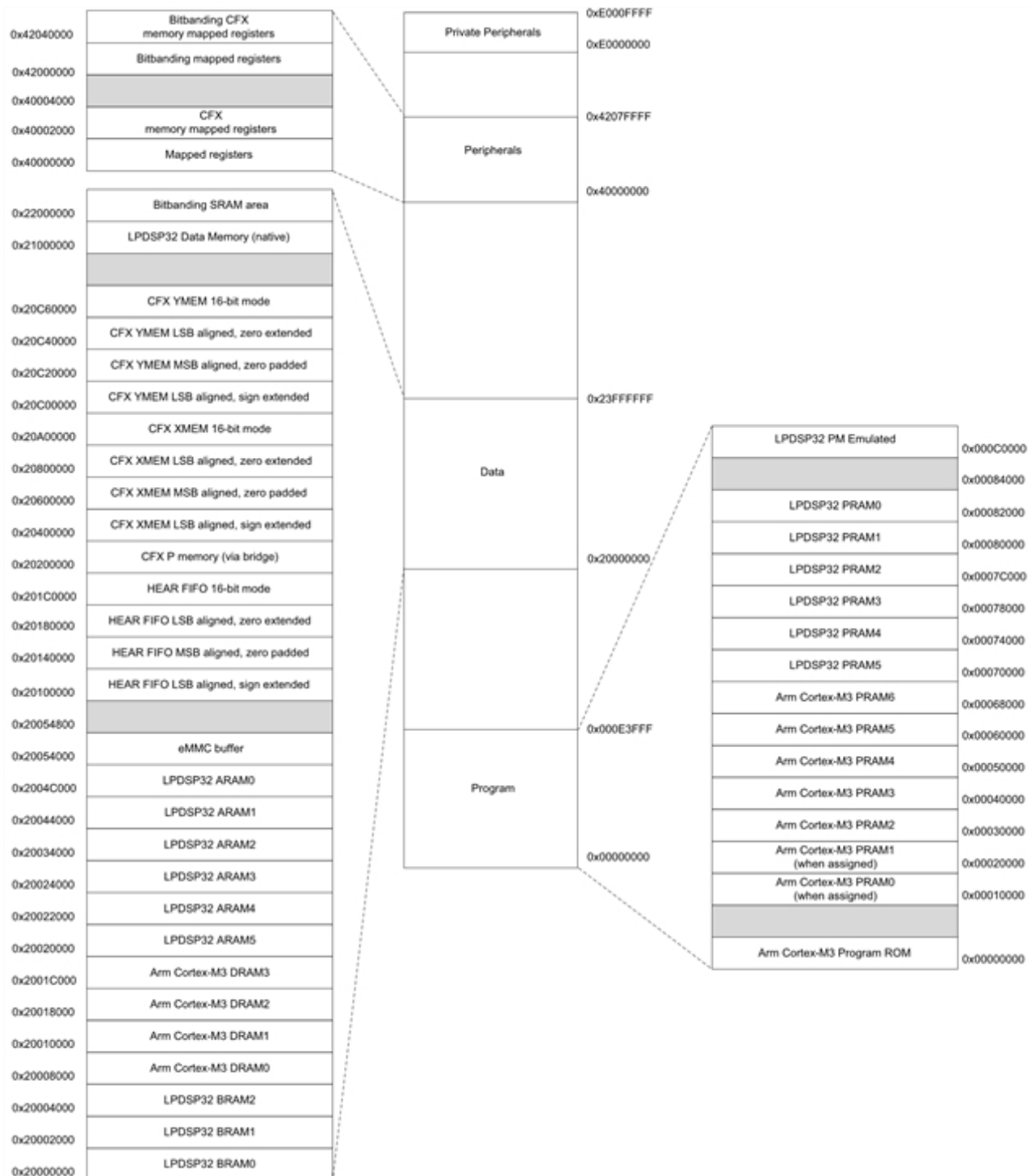


Figure 18. Arm Cortex-M3 Core System Memory Map

Ezairo 8300 Hardware Reference

11.4.1 Arm Cortex-M3 Memory Elements

The elements mapped into the Arm Cortex-M3 processor's memory space are listed in [Table 32](#).

NOTE: The memories that are accessible to the LPDPS32 are mapped into the Arm Cortex-M3 processor's memory map in two locations. These mappings provide access to the same memories, ordered as they are accessible natively to the LPDPS32 in its memory maps (marked as "native" mappings in the "[Arm Cortex-M3 Processor Memory Mapping](#)" table ([Table 32](#))) and also as they are remapped for use by the Arm Cortex-M3 processor for any memory instances not used by the LPDPS32.

NOTE: For information on the Arm Cortex-M3 core bootloader's use of memory, see Reserved Memory in the Firmware Overview section of the *Ezairo 8300 Firmware Reference*.

Table 32. Arm Cortex-M3 Processor Memory Mapping

Start Address	End Address	Description	Bytes
0x00000000	0x000001FF	Arm Cortex-M3 Program ROM	512
0x00010000	0x0001FFFF	Arm Cortex-M3 PRAM0 (when assigned)	65536
0x00020000	0x0002FFFF	Arm Cortex-M3 PRAM1 (when assigned)	65536
0x00030000	0x0003FFFF	Arm Cortex-M3 PRAM2	65536
0x00040000	0x0004FFFF	Arm Cortex-M3 PRAM3	65536
0x00050000	0x0005FFFF	Arm Cortex-M3 PRAM4	65536
0x00060000	0x00067FFF	Arm Cortex-M3 PRAM5	32768
0x00068000	0x0006FFFF	Arm Cortex-M3 PRAM6	32768
0x00070000	0x00073FFF	LPDPS32 PRAM5	16384
0x00074000	0x00077FFF	LPDPS32 PRAM4	16384
0x00078000	0x0007BFFF	LPDPS32 PRAM3	16384
0x0007C000	0x0007FFFF	LPDPS32 PRAM2	16384
0x00080000	0x00081FFF	LPDPS32 PRAM1	8192
0x00082000	0x00083FFF	LPDPS32 PRAM0	8192
0x000C0000	0x000C3FFF	LPDPS32 PRAM01	16384
0x000C4000	0x000CBFFF	LPDPS32 PRAM23	32768
0x000CC000	0x000D3FFF	LPDPS32 PRAM45	32768
0x000D4000	0x000E3FFF	Arm Cortex-M3 PRAM56	65536
0x20000000	0x20001FFF	LPDPS32 BRAM0	8192
0x20002000	0x20003FFF	LPDPS32 BRAM1	8192
0x20004000	0x20007FFF	LPDPS32 BRAM2	16384
0x20008000	0x2000FFFF	Arm Cortex-M3 DRAM0	32768
0x20010000	0x20017FFF	Arm Cortex-M3 DRAM1	32768

onsemi Confidential
Ezairo 8300 Hardware Reference

Arm Cortex-M3 Processor Memory Mapping (Continued)

Start Address	End Address	Description	Bytes
0x20018000	0x2001BFFF	Arm Cortex-M3 DRAM2	16384
0x2001C000	0x2001FFFF	Arm Cortex-M3 DRAM3	16384
0x20020000	0x20021FFF	LPDSP32 ARAM5	8192
0x20022000	0x20023FFF	LPDSP32 ARAM4	8192
0x20024000	0x20033FFF	LPDSP32 ARAM3	65536
0x20034000	0x20043FFF	LPDSP32 ARAM2	65536
0x20044000	0x2004BFFF	LPDSP32 ARAM1	32768
0x2004C000	0x20053FFF	LPDSP32 ARAM0	32768
0x20054000	0x200547FF	eMMC buffer	2048
0x20100000	0x2013FFFF	HEAR FIFO LSB aligned, sign extended	262144
0x20140000	0x2017FFFF	HEAR FIFO MSB aligned, zero padded	262144
0x20180000	0x201BFFFF	HEAR FIFO LSB aligned, zero extended	262144
0x201C0000	0x201FFFFF	HEAR FIFO 16-bit mode	262144
0x20200000	0x20201FFF	CFX PROM (via bridge)	8192
0x20206000	0x202064FF	Filter Engine State	1280
0x20206500	0x202065FF	Filter Engine State Temp	256
0x20206800	0x20206CFF	Filter Engine Coefficients	1280
0x20207000	0x202073FF	Filter Engine PRAM LSB	1024
0x20207400	0x202077FF	Filter Engine PRAM MSB	1024
0x20208000	0x2020BFFF	HEAR Microcode PRAM0	16384
0x2020C000	0x2020FFFF	HEAR Microcode PRAM1	16384
0x20220000	0x2022FFFF	CFX PRAM0	65536
0x20230000	0x2023FFFF	CFX PRAM1	65536
0x20240000	0x2024FFFF	Arm Cortex-M3 PRAM0 (when assigned to CFX)	65536
0x20250000	0x2025FFFF	Arm Cortex-M3 PRAM1 (when assigned to CFX)	65536
0x20400000	0x205FFFFFFF	CFX XMEM LSB aligned, sign extended	2097152
0x20600000	0x207FFFFFFF	CFX XMEM MSB aligned, zero padded	2097152
0x20800000	0x209FFFFFFF	CFX XMEM LSB aligned, zero extended	2097152
0x20A00000	0x20BFFFFFFF	CFX XMEM 16-bit mode	2097152
0x20C00000	0x20C0FFFF	CFX YMEM LSB aligned, sign extended	65536
0x20C20000	0x20C3FFFF	CFX YMEM MSB aligned, zero padded	131072
0x20C40000	0x20C5FFFF	CFX YMEM LSB aligned, zero extended	131072

Ezairo 8300 Hardware Reference

Arm Cortex-M3 Processor Memory Mapping (Continued)

Start Address	End Address	Description	Bytes
0x20C60000	0x20C7FFFF	CFX YMEM 16-bit mode	131072
0x21000000	0x2100FFFF	LPDSP32 ARAM01 (native)	65536
0x21010000	0x2102FFFF	LPDSP32 ARAM23 (native)	131072
0x21030000	0x21033FFF	LPDSP32 ARAM45 (native)	16384
0x21034000	0x21037FFF	LPDSP32 BRAM01 (native)	16384
0x21038000	0x2103FFFF	LPDSP32 PRAM45 (native)	32768
0x21040000	0x2104FFFF	Arm Cortex-M3 PRAM56 (LPDSP32 native)	65536
0x21050000	0x21057FFF	Arm Cortex-M3 DRAM23 (LPDSP32 native)	32768
0x21800000	0x21801FFF	LPDSP32 BRAM0 (native)	8192
0x21802000	0x21803FFF	LPDSP32 BRAM1 (native)	8192
0x21804000	0x21807FFF	LPDSP32 BRAM2 (native)	16384
0x21808000	0x2180BFFF	Arm Cortex-M3 DRAM3 (LPDSP32 native)	16384
0x2180C000	0x2180FFFF	Arm Cortex-M3 DRAM2 (LPDSP32 native)	16384
0x22000000	0x23FFFFFF	Bitbanding SRAM area	-
0x40000000	0x40001FFF	Arm Cortex-M3 memory mapped registers	8192
0x40002000	0x40003FFF	CFX memory mapped registers	8192
0x42000000	0x4203FFFF	Bitbanding for Arm Cortex-M3 memory mapped registers	-
0x42040000	0x4207FFFF	Bitbanding for CFX memory mapped registers	-
0xE0000000	0xE00FFFFF	Arm Cortex-M3 private peripheral registers	1048576

11.4.2 Program Memory Cache

To increase the power-efficiency of the Arm Cortex-M3 Processor, a 32-word write-through instruction cache has been included on the I-Code memory bus between the Arm Cortex-M3 Processor and its addressable program memory. This cache is designed to reduce the power consumed by constantly accessing program memory to fetch subsequent instructions for the Arm Cortex-M3 Processor. There is no detectable difference in functional code execution when using the cache, except for a slight change in timing due to the special case discussed in the note below. The Arm Cortex-M3 Processor's instruction cache is enabled by default, and can be reconfigured using the CM3_LOOP_CACHE_ENABLE bit from the SYSCTRL_CM3_LOOP_CACHE_CFG register.

NOTE: Access to an accessed program memory instance while the Arm Cortex-M3 Processor is fetching cached instructions from a loop ignores wait states from a memory conflict. As a result, when the program memory cache is being used for a loop that uses program memory as data, the core does not incur the delays due to conflicts that would otherwise occur.

The program memory cache automatically reconfigures whenever a backward branch of less than 256 words indicates that a loop has been detected. At this point, the cache aligns itself to ensure that it stores the largest possible number of instructions from the loop.

CAUTION: If the program memory area covered by the cache is changed due to an indirect access (for example, due to the memory being disabled or through CFX DSP or DMA writes), the cache should be disabled or reset to ensure the cached instruction words match the underlying memory.

11.4.3 Configuration Register Mapping

The analog and digital control and configuration registers are mapped into the peripherals area of the memory space. These registers are organized by their area of control and mapped as blocks into memory space as shown in the "Analog and Digital Control and Configuration Register Block Mapping" table (Table 33).

Table 33. Analog and Digital Control and Configuration Register Block Mapping

Controlled/Configured Area	Area Type	Block Start Address
Chip Identification	Arm Cortex-M3 Digital	0x1FFFFFFC
CFX Math Accelerators I/O and Control	CFX Digital (X)	0x20802000
FIFO A0 Configuration and Control [7:0]	HEAR Digital	0x20804000
FIFO A1 Configuration and Control [7:0]	HEAR Digital	0x20804200
FIFO B0 Configuration and Control [7:0]	HEAR Digital	0x20804400
FIFO B1 Configuration and Control [7:0]	HEAR Digital	0x20804600
FIFO Interrupt Selection	HEAR Digital	0x20804800
HEAR Interrupt Configuration and Control	HEAR Digital	0x208048C0
Block Floating Point Configuration and Control	HEAR Digital	0x20804980
CFX Math Accelerators I/O	CFX Digital (Y)	0x20C42000
System Control	Arm Cortex-M3 Digital	0x40000000
General-Purpose Timers [3:0]	Arm Cortex-M3 Digital	0x40000100
DMA Controller Configuration and Control [7:0]	Arm Cortex-M3 Digital	0x40000200
Audio Sink Clock Counters [1:0]	Arm Cortex-M3 Digital	0x40000300
ASRC Configuration and Control	Arm Cortex-M3 Digital	0x40000500
CRC Generator Control	Arm Cortex-M3 Digital	0x40000600
G.277 Configuration and Control	Arm Cortex-M3 Digital	0x40000700
Reed Solomon Configuration and Control	Arm Cortex-M3 Digital	0x40000800
DIO Interface and Digital Pad Control	Arm Cortex-M3 Digital	0x40000900
PCM Interface Configuration and Control [1:0]	Arm Cortex-M3 Digital	0x40000B00
SPI Interface Configuration and Control [1:0]	Arm Cortex-M3 Digital	0x40000D00

Ezairo 8300 Hardware Reference

Analog and Digital Control and Configuration Register Block Mapping
(Continued)

Controlled/Configured Area	Area Type	Block Start Address
UART Interface Configuration and Control	Arm Cortex-M3 Digital	0x40000F00
I ² C Interface Configuration and Control [1:0]	Arm Cortex-M3 Digital	0x40001000
Analog-to-Digital Converter and Battery Monitoring	Arm Cortex-M3 Analog	0x40001200
eMMC and I ³ C Interface Control	Arm Cortex-M3 Digital	0x40001300
I ³ C Interface Configuration and Control [1:0]	Arm Cortex-M3 Digital	0x40001400
eMMC Interface Configuration Control	Arm Cortex-M3 Digital	0x40001C00
Neural Network Accelerator	Arm Cortex-M3 Digital	0x40001F00
System Control	CFX Digital	0x40002000
Watchdog Timer Configuration	CFX Digital	0x40002004
Boot Status	CFX Digital	0x4000200C
HEAR Function Chain Commands	CFX Digital	0x40002010
PRAM Security	CFX Digital	0x40002024
CFX/Arm Cortex-M3 Core Control and Configuration	CFX Digital	0x4000202C
External Breakpoint Configuration	CFX Digital	0x40002040
Hardware CRC	CFX Digital	0x40002088
Chip Version Information	CFX Digital	0x400020C0
Debug Port Configuration	CFX Digital	0x400020D4
Interrupt Configuration	CFX Digital	0x40002100
Clock Configuration	CFX Digital	0x40002300
IOC Configuration	CFX Digital	0x40002400
Analog	CFX Analog	0x40002600
Audio	CFX Audio	0x40002700
I ² C Interface	CFX Digital	0x40002800
FENG Control and Configuration	CFX Digital	0x40002C00
FENG Register File	CFX Digital	0x40002D00
SYSTICK Timer	Arm Cortex-M3 Digital	0xE000E010
System Control and ID Register	Arm Cortex-M3 Digital	0xE000E000

Ezairo 8300 Hardware Reference

**Analog and Digital Control and Configuration Register Block Mapping
(Continued)**

Controlled/Configured Area	Area Type	Block Start Address
Nested Vector Interrupt Controller	Arm Cortex-M3 Digital	0xE000E100
System Control Block	Arm Cortex-M3 Digital	0xE000ED00
Debug Controller	Arm Cortex-M3 Digital	0xE000EDF0

11.5 HEAR CONFIGURABLE ACCELERATOR MEMORY MAPPING**11.5.1 Microcode Memory**

The microcode memory is loaded by the CFX or Arm Cortex-M3 processor with a microcode configuration for the HEAR. This memory is addressable in:

- CFX P memory space from 0x2000 to 0x3FFF
- Arm Cortex-M3 memory space from 0x20208000 to 0x2020FFFF

When each function chain is triggered, its microcode is loaded by the HEAR Configurable Accelerator using the microcode memory bus.

11.5.2 Shared Memory

All of the shared memories are double-word structured, 48-bit memories that allow 24-bit word addressing by both the CFX and HEAR. These memories and their locations in the shared memory map are outlined in the "[HEAR Shared Memory Instances](#)" table (Table 34). All accesses to these memories are controlled by the shared memory bus controller. The controller provides access for:

- The CFX DSP through the X memory spaces (offset by 0x40000 from the addresses listed below)
- The HEAR through the core's 48-bit shared memory buses

Table 34. HEAR Shared Memory Instances

Memory Name	Start Address	End Address
H0 memory	0x3000	0x37FF
H1 memory	0x3800	0x3FFF
H2 memory	0x4000	0x47FF
H3 memory	0x4800	0x4FFF
H4 memory	0x5000	0x57FF
H5 memory	0x5800	0x5FFF
SIN/COS Data Table (ROM)	0xC000	0xC7FF
SIN/COS Data Table Radix-3 (ROM)	0xD000	0xDBFF
A0 memory	0x20000	0x21FFF
A1 memory	0x22000	0x23FFF

Ezairo 8300 Hardware Reference

HEAR Shared Memory Instances (Continued)

Memory Name	Start Address	End Address
B0 memory	0x24000	0x25FFF
B1 memory	0x26000	0x27FFF
C0 memory	0x28000	0x29FFF
C1 memory	0x2A000	0x2BFFF
D0 memory	0x2C000	0x2DFFF
D1 memory	0x2E000	0x2FFFF

The shared memory instances are both accessible independently, as listed in the "HEAR Shared Memory Instances" table (Table 34), and interleaved with other shared memory instances. All interleaved combinations of shared memory instances are listed in the "HEAR Shared Memory Interleaving" table (Table 35). Both independent instances and interleaved memory instances are shown as appropriate in the X memory map (see the "CFX X, Y Memory Maps" figure (Figure 17)).

Table 35. HEAR Shared Memory Interleaving

Memory	Memory Space	First Memory	Second Memory	Start Address	End Address
H01 memory	X	H0 memory	H1 memory	0x0000	0x0FFF
H23 memory	X	H2 memory	H3 memory	0x1000	0x1FFF
H45 memory	X	H4 memory	H5 memory	0x2000	0x2FFF
H02 memory	X	H0 memory	H2 memory	0x6000	0x6FFF
H13 memory	X	H1 memory	H3 memory	0x7000	0x7FFF
H03 memory	X	H0 memory	H3 memory	0x8000	0x8FFF
H12 memory	X	H1 memory	H2 memory	0x9000	0x9FFF
AB0 memory	X	A0 memory	B0 memory	0x10000	0x13FFF
AB1 memory	X	A1 memory	B1 memory	0x14000	0x17FFF
CD0 memory	X	C0 memory	D0 memory	0x18000	0x1BFFF
CD1 memory	X	C1 memory	D1 memory	0x1C000	0x1FFFF
AC0 memory	Y	A0 memory	C0 memory	0x30000	0x33FFF
AC1 memory	Y	A1 memory	A0 memory	0x34000	0x37FFF
BD0 memory	Y	B0 memory	D0 memory	0x38000	0x3BFFF
BD1 memory	Y	B1 memory	D1 memory	0x3C000	0x3FFFF

The interleaving of memory is in 48-bit double-words, resulting in two addressable locations into each memory at each step. For example, the memory addressed between 0x10000 and 0x13FFF is an interleaved memory mapping that provides access to data in both A0 and B0 memory. Reading from 0x10000-0x10001 (the first double-word of the interleaved AB0 Memory) provides the first double-word of A0 memory. Reading from 0x10002-0x10003 (the second double-word of the interleaved AB0 Memory) provides the first double-word of B0 memory.

When using shared memory with both the CFX and HEAR cores, bus conflicts occur if both cores attempt to access the same memory instance in the same cycle. If a bus conflict occurs, access is granted to the CFX or HEAR based on the settings of the `D_SYSTEM_CTRL` register, as described in [Section 11.2.3.4 “Shared Memory Bus Controller” on page 347](#), while the other core is held in a wait state. When the CFX DSP is held in a wait state, the delay is not visible to instructions running on the DSP, as explained in [Section 11.2.3.1 “CFX Memory Arbitration” on page 344](#).

IMPORTANT: The `HEAR_COMEM`, `HEAR_C1MEM`, `HEAR_DOMEM`, and `HEAR_D1MEM` memory instances consume a bit more power than the other HEAR memory instances. To minimize power consumption, applications needs to prioritize the use of other memory instances.

11.6 LPDSP32 DSP MEMORY MAPPING

11.6.1 Program Memory

The LPDSP32's program memory space uses 24-bit addressing relative to 20-bit words. Two 20-bit words form a 40-bit memory word in a big endian manner. The DSP PRAM is subdivided into 6 instances (`DSP_PRAM0` - `DSP_PRAM5`) that can be independently accessed by various cores, using arbitration as defined in [Section 11.2.3.3 “Arm Cortex-M3 Memory Arbitration” on page 347](#). Program memory can be extended using a part of the Arm Cortex-M3 processor program memory.

The LPDSP32 program memory mapping is shown in the "LPDSP32 Program Memory Mapping" table (Table 36):

Table 36. LPDSP32 Program Memory Mapping

Start Address	End Address	Name	Size (KBs)
0x000000	0x001998	LPDSP32 PRAM0/PRAM1	16
0x00199A	0x004CCB	LPDSP32 PRAM2/PRAM3	32
0x004CCD	0x007FFF	LPDSP32 PRAM4/PRAM5	32
0x008000	0x00E665	Arm Cortex-M3 PRAM5/PRAM6	64

11.6.1.1 Program Memory Cache

To increase the power-efficiency of the LPDSP32, a 32-word write-through instruction cache has been included on the instruction bus between the LPDSP32 and its program memory. This cache is designed to reduce the power consumed by constantly accessing program memory to fetch subsequent instructions for the LPDSP32. There is no detectable difference in functional code execution when using the cache, except for a slight change in timing due to the special case discussed in the note below. The LPDSP32 instruction cache is enabled by default, and can be reconfigured using the `DSP_LOOP_CACHE_ENABLE` bit from the `SYSCTRL_DSP_LOOP_CACHE_CFG` register.

NOTE: Access to the LPDSP32 PRAM while the LPDSP32 is fetching cached instructions from a loop ignores wait states from a memory conflict. As a result, when the program memory cache is being used for a loop that uses program memory as data, the core does not incur the delays due to conflicts that would otherwise occur.

The program memory cache automatically reconfigures whenever a backward branch of less than 256 20-bit words or hardware loop start indicates that a loop has been detected. At this point, the cache aligns itself to ensure that it stores the largest possible number of instructions from the loop.

Ezairo 8300 Hardware Reference

CAUTION: If the program memory area covered by the cache is changed due to an indirect access (for example, due to the memory being disabled or through writes from the CFX DSP or Arm Cortex-M3 processor), the cache should be disabled or manually reset to ensure the cached instruction words match the underlying memory.

11.6.2 Data Memory

The LPDSP32's data memory space uses 24-bit addressing relative to 8-bit words. Four 8-bit words form a 32-bit memory word in a little endian manner.

The LPDSP32 has multiple usable data memory mappings (refer to the "[LPDSP32 Data Memory Mapping](#)" figure (Figure 19)), which are accessible through the LPDSP32 data memory A (DMA) and data memory B (DMB) buses. LPDSP32-DMA memory can be extended using a part of the LPDSP32-DMB memory, LPDSP-32 program memory, and Arm Cortex-M3 processor program and data memories. LPDSP32-DMB memory can be extended using a part of the Arm Cortex-M3 processor data memory. These optional mappings are listed in the "[LPDSP32 Data Memory Mapping](#)" table (Table 37).

IMPORTANT: The different data memory mappings do not require any hardware configuration. The mapping that is used must simply be respected by the software that is running on the LPDSP32 and Arm Cortex-M3 processor.

IMPORTANT: The software debug tools use a temporary memory location to access the internal registers of the core. Data memory address 0x000000 has been assigned for this use and four 32-bit words at this address must be excluded from applications for proper debug behavior, typically using the linker configuration in the *.bcf* file.

Table 37. LPDSP32 Data Memory Mapping

Start Address	End Address	Name	Size (KBs)
0x000000	0x00FFFF	LPDSP32 ARAM0/ARAM1	64
0x010000	0x02FFFF	LPDSP32 ARAM2/ARAM3	128
0x030000	0x033FFF	LPDSP32 ARAM4/ARAM5	16
0x034000	0x037FFF	LPDSP32 BRAM0/BRAM1	16
0x038000	0x03FFFF	LPDSP32 PRAM4/PRAM5	32
0x040000	0x04FFFF	Arm Cortex-M3 PRAM5/PRAM6	64
0x050000	0x057FFF	Arm Cortex-M3 DRAM2/DRAM3	32
0x800000	0x801FFF	LPDSP32 BRAM0	8
0x802000	0x803FFF	LPDSP32 BRAM1	8
0x804000	0x807FFF	LPDSP32 BRAM2	16
0x808000	0x80BFFF	Arm Cortex-M3 DRAM3	16
0x80C000	0x80FFFF	Arm Cortex-M3 DRAM2	16

Ezairo 8300 Hardware Reference

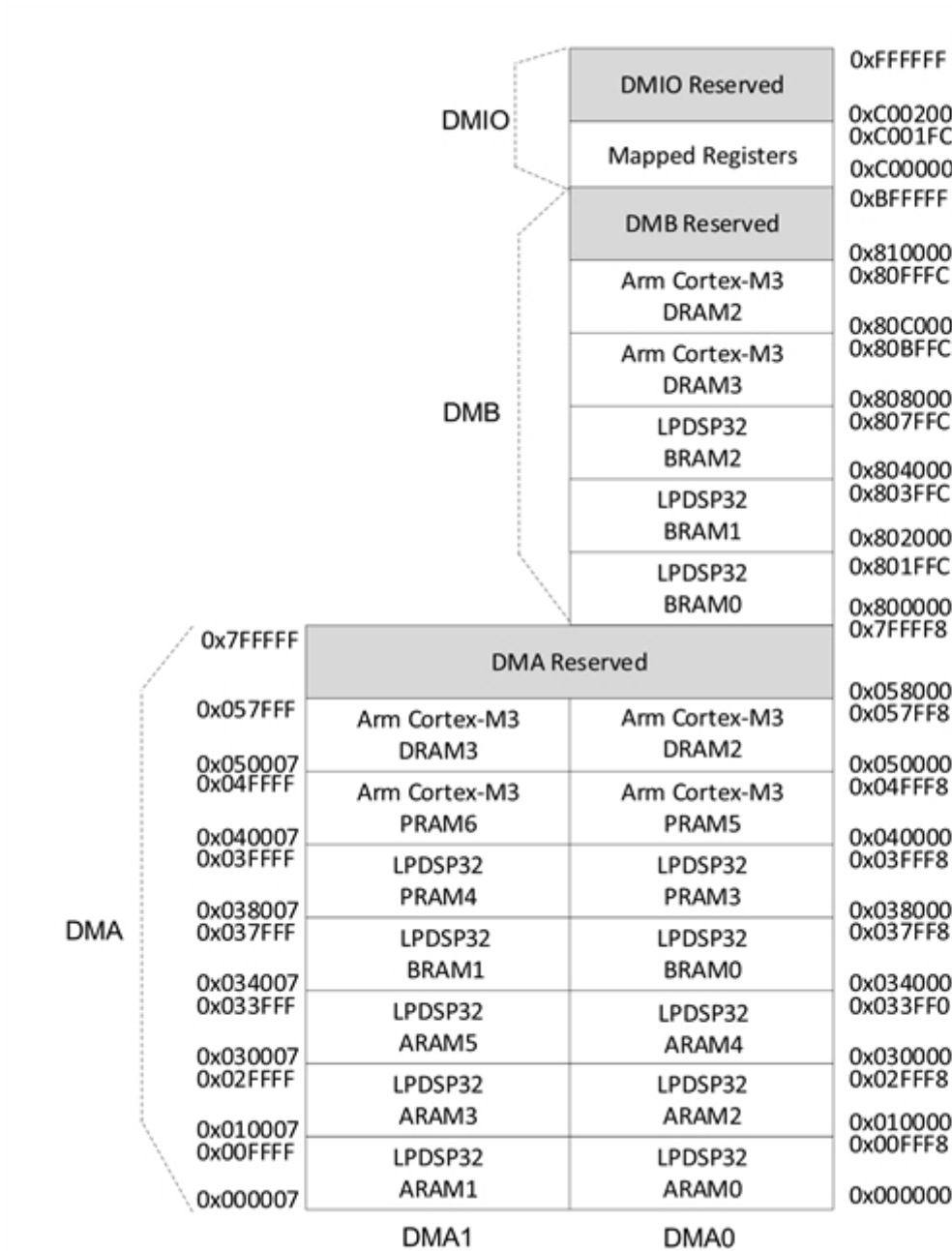


Figure 19. LPDSP32 Data Memory Mapping

IMPORTANT: The `DSP_ARAM2` and `DSP_ARAM3` memory instances consume a bit more power than the other memory instances accessible by the LPDSP32 and NNA. To minimize power consumption, applications need to prioritize the use of other memory instances.

Ezairo 8300 Hardware Reference

11.7 FILTER ENGINE MEMORY

The Filter Engine memory map is provided in the "Filter Engine Memory Map" table (Table 38). This memory map contains:

- Data registers:
 - Extended-precision
 - Single-precision
 - Input
 - Output
- Filter Coefficients
- State variables
- Filter Engine program memory

Filter Engine program memory words are 36-bits organized into 32 LSBs and 4 MSBs, with the LSBs and MSBs grouped separately.

Table 38. Filter Engine Memory Map

Start	End Address	Description	Size (words)	Data
0x0B40	0x0B5F	Extended-precision data registers	32	<i>Read:</i> Sign extension [31:28] data [27:0] <i>Write:</i> Sign extension [31:27], data [26:0]
0x0B60	0x0BA5	Single-precision data registers	70	Sign extension [31:24], data [23:0]
0x0BA6	0x0BB3	Input data registers	14	Sign extension [31:24], data [23:0]
0x0BB4	0x0BBF	Output data registers	12	Sign extension [31:24], data [23:0]
(0x1800+5* x)	(0x1804+5* x)	Core stage- x state variables x = 0,1,2,...63 Sx.y = Stage x State y States at stage x : Sx.1 , Sx.2 , Sx.3 , Sx.4 , Sx.5	320	Sign extension [31:24] data [23:0] Discard sign extension [31:24] of unsigned data [23:0] for Error Spectral Shaping (ESS) variable in Sx.5 . (For more about ESS, refer to Quantization Management information in the <i>Ezairo 8300 Filter Engine Reference</i> .)
0x1940	0x197F	Core stage- x temporary state variables x = 0,1,2,...63 Sx.y = Stage x State y States at stage x : Sx.5tmp	64	[27:24] ESS bits [23:0] state variables Discard sign extension [31:24] of unsigned data [23:0] for Error Spectral Shaping (ESS) variable. (For more about ESS, refer to Quantization Management information in the <i>Ezairo 8300 Filter Engine Reference</i> .)

Ezairo 8300 Hardware Reference

Filter Engine Memory Map (Continued)

Start	End Address	Description	Size (words)	Data
(0x1A00+5* <i>x</i>)	(0x1A04+5* <i>x</i>)	Core stage- <i>x</i> coefficients <i>x</i> = 0,1,2,...63 <i>C_{x.y}</i> = Stage <i>x</i> Coefficient <i>y</i> Coefficients at stage <i>x</i> : <i>C_{x.0}</i> , <i>C_{x.1}</i> , <i>C_{x.2}</i> , <i>C_{x.3}</i> , <i>C_{x.4}</i>	320	Sign extension [31:28], data [27:0]
0x1C00	0x1CFF	Program Memory (LSBs)	256	Data [31:0]
0x1D00	0x1DFF	Program Memory (MSBs)	256	No sign extension, data [3:0]

11.8 CONTROL AND CONFIGURATION REGISTERS

Register Name	Register Description	Address
SYSCTRL_MEM_POWER_CFG0	Memory Power Configuration 0	P:0x1008 0x40000020
SYSCTRL_MEM_POWER_CFG1	Memory Power Configuration 1	P:0x1009 0x40000024
SYSCTRL_MEM_ACCESS_CFG0	Memory Access Configuration 0	P:0x100A 0x40000028
SYSCTRL_MEM_ACCESS_CFG1	Memory Access Configuration 1	P:0x100B 0x4000002C
SYSCTRL_MEM_ARBITER_CFG	Memory Arbiter Configuration	P:0x100E 0x40000038
SYSCTRL_MEM_ERROR	Memory Access Error Flags	P:0x100F 0x4000003C
SYSCTRL_CM3_LOOP_CACHE_CFG	Arm Cortex-M3 Loop Cache Configuration	P:0x1005 0x40000014
SYSCTRL_DSP_LOOP_CACHE_CFG	DSP Loop Cache Configuration	P:0x1006 0x40000018
SYSCTRL_PRAM_CFG	PRAM Memory Assignment and Security Configuration	P:0x1007 0x4000001C

Ezairo 8300 Hardware Reference

11.8.1 SYSCTRL_MEM_POWER_CFG0

Bit Field	Read/Write	Field Name	Description
26	RW	DSP_BRAM2_POWER	DSP BRAM2 power configuration
25	RW	DSP_BRAM1_POWER	DSP BRAM1 power configuration
24	RW	DSP_BRAM0_POWER	DSP BRAM0 power configuration
23	RW	DSP_ARAM5_POWER	DSP ARAM5 power configuration
22	RW	DSP_ARAM4_POWER	DSP ARAM4 power configuration
21	RW	DSP_ARAM3_POWER	DSP ARAM3 power configuration
20	RW	DSP_ARAM2_POWER	DSP ARAM2 power configuration
19	RW	DSP_ARAM1_POWER	DSP ARAM1 power configuration
18	RW	DSP_ARAM0_POWER	DSP ARAM0 power configuration
17	RW	DSP_PRAM5_POWER	DSP PRAM5 power configuration
16	RW	DSP_PRAM4_POWER	DSP PRAM4 power configuration
15	RW	DSP_PRAM3_POWER	DSP PRAM3 power configuration
14	RW	DSP_PRAM2_POWER	DSP PRAM2 power configuration
13	RW	DSP_PRAM1_POWER	DSP PRAM1 power configuration
12	RW	DSP_PRAM0_POWER	DSP PRAM0 power configuration
11	RW	CM3_EMMC_RAM_POWER	eMMC RAM power configuration
10	RW	CM3_DRAM3_POWER	Arm Cortex-M3 DRAM3 power configuration
9	RW	CM3_DRAM2_POWER	Arm Cortex-M3 DRAM2 power configuration
8	RW	CM3_DRAM1_POWER	Arm Cortex-M3 DRAM1 power configuration
7	RW	CM3_DRAM0_POWER	Arm Cortex-M3 DRAM0 power configuration
6	RW	CM3_PRAM6_POWER	Arm Cortex-M3 PRAM6 power configuration
5	RW	CM3_PRAM5_POWER	Arm Cortex-M3 PRAM5 power configuration
4	RW	CM3_PRAM4_POWER	Arm Cortex-M3 PRAM4 power configuration
3	RW	CM3_PRAM3_POWER	Arm Cortex-M3 PRAM3 power configuration
2	RW	CM3_PRAM2_POWER	Arm Cortex-M3 PRAM2 power configuration
1	RW	CM3_PRAM1_POWER	Arm Cortex-M3 PRAM1 power configuration
0	RW	CM3_PRAM0_POWER	Arm Cortex-M3 PRAM0 power configuration

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DSP_BRAM2_POWER	DSP_BRAM2_POWER_DISABLE	DSP BRAM2 power disabled	0x0
	DSP_BRAM2_POWER_ENABLE	DSP BRAM2 power enabled	0x1*
DSP_BRAM1_POWER	DSP_BRAM1_POWER_DISABLE	DSP BRAM1 power disabled	0x0
	DSP_BRAM1_POWER_ENABLE	DSP BRAM1 power enabled	0x1*
DSP_BRAM0_POWER	DSP_BRAM0_POWER_DISABLE	DSP BRAM0 power disabled	0x0
	DSP_BRAM0_POWER_ENABLE	DSP BRAM0 power enabled	0x1*
DSP_ARAM5_POWER	DSP_ARAM5_POWER_DISABLE	DSP ARAM5 power disabled	0x0
	DSP_ARAM5_POWER_ENABLE	DSP ARAM5 power enabled	0x1*
DSP_ARAM4_POWER	DSP_ARAM4_POWER_DISABLE	DSP ARAM4 power disabled	0x0
	DSP_ARAM4_POWER_ENABLE	DSP ARAM4 power enabled	0x1*
DSP_ARAM3_POWER	DSP_ARAM3_POWER_DISABLE	DSP ARAM3 power disabled	0x0
	DSP_ARAM3_POWER_ENABLE	DSP ARAM3 power enabled	0x1*
DSP_ARAM2_POWER	DSP_ARAM2_POWER_DISABLE	DSP ARAM2 power disabled	0x0
	DSP_ARAM2_POWER_ENABLE	DSP ARAM2 power enabled	0x1*
DSP_ARAM1_POWER	DSP_ARAM1_POWER_DISABLE	DSP ARAM1 power disabled	0x0
	DSP_ARAM1_POWER_ENABLE	DSP ARAM1 power enabled	0x1*
DSP_ARAM0_POWER	DSP_ARAM0_POWER_DISABLE	DSP ARAM0 power disabled	0x0
	DSP_ARAM0_POWER_ENABLE	DSP ARAM0 power enabled	0x1*
DSP_PRAM5_POWER	DSP_PRAM5_POWER_DISABLE	DSP PRAM5 power disabled	0x0
	DSP_PRAM5_POWER_ENABLE	DSP PRAM5 power enabled	0x1*
DSP_PRAM4_POWER	DSP_PRAM4_POWER_DISABLE	DSP PRAM4 power disabled	0x0
	DSP_PRAM4_POWER_ENABLE	DSP PRAM4 power enabled	0x1*
DSP_PRAM3_POWER	DSP_PRAM3_POWER_DISABLE	DSP PRAM3 power disabled	0x0
	DSP_PRAM3_POWER_ENABLE	DSP PRAM3 power enabled	0x1*
DSP_PRAM2_POWER	DSP_PRAM2_POWER_DISABLE	DSP PRAM2 power disabled	0x0
	DSP_PRAM2_POWER_ENABLE	DSP PRAM2 power enabled	0x1*
DSP_PRAM1_POWER	DSP_PRAM1_POWER_DISABLE	DSP PRAM1 power disabled	0x0
	DSP_PRAM1_POWER_ENABLE	DSP PRAM1 power enabled	0x1*
DSP_PRAM0_POWER	DSP_PRAM0_POWER_DISABLE	DSP PRAM0 power disabled	0x0
	DSP_PRAM0_POWER_ENABLE	DSP PRAM0 power enabled	0x1*
CM3_EMMC_RAM_POWER	CM3_EMMC_RAM_POWER_DISABLE	Arm Cortex-M3 eMMC RAM power disabled	0x0

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	CM3_EMMC_RAM_POWER_ENABLE	Arm Cortex-M3 eMMC RAM power enabled	0x1*
CM3_DRAM3_POWER	CM3_DRAM3_POWER_DISABLE	Arm Cortex-M3 DRAM3 power disabled	0x0
	CM3_DRAM3_POWER_ENABLE	Arm Cortex-M3 DRAM3 power enabled	0x1*
CM3_DRAM2_POWER	CM3_DRAM2_POWER_DISABLE	Arm Cortex-M3 DRAM2 power disabled	0x0
	CM3_DRAM2_POWER_ENABLE	Arm Cortex-M3 DRAM2 power enabled	0x1*
CM3_DRAM1_POWER	CM3_DRAM1_POWER_DISABLE	Arm Cortex-M3 DRAM1 power disabled	0x0
	CM3_DRAM1_POWER_ENABLE	Arm Cortex-M3 DRAM1 power enabled	0x1*
CM3_DRAM0_POWER	CM3_DRAM0_POWER_DISABLE	Arm Cortex-M3 DRAM0 power disabled	0x0
	CM3_DRAM0_POWER_ENABLE	Arm Cortex-M3 DRAM0 power enabled	0x1*
CM3_PRAM6_POWER	CM3_PRAM6_POWER_DISABLE	Arm Cortex-M3 PRAM6 power disabled	0x0
	CM3_PRAM6_POWER_ENABLE	Arm Cortex-M3 PRAM6 power enabled	0x1*
CM3_PRAM5_POWER	CM3_PRAM5_POWER_DISABLE	Arm Cortex-M3 PRAM5 power disabled	0x0
	CM3_PRAM5_POWER_ENABLE	Arm Cortex-M3 PRAM5 power enabled	0x1*
CM3_PRAM4_POWER	CM3_PRAM4_POWER_DISABLE	Arm Cortex-M3 PRAM4 power disabled	0x0
	CM3_PRAM4_POWER_ENABLE	Arm Cortex-M3 PRAM4 power enabled	0x1*
CM3_PRAM3_POWER	CM3_PRAM3_POWER_DISABLE	Arm Cortex-M3 PRAM3 power disabled	0x0
	CM3_PRAM3_POWER_ENABLE	Arm Cortex-M3 PRAM3 power enabled	0x1*
CM3_PRAM2_POWER	CM3_PRAM2_POWER_DISABLE	Arm Cortex-M3 PRAM2 power disabled	0x0
	CM3_PRAM2_POWER_ENABLE	Arm Cortex-M3 PRAM2 power enabled	0x1*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
CM3_PRAM1_POWER	CM3_PRAM1_POWER_DISABLE	Arm Cortex-M3 PRAM1 power disabled	0x0
	CM3_PRAM1_POWER_ENABLE	Arm Cortex-M3 PRAM1 power enabled	0x1*
CM3_PRAM0_POWER	CM3_PRAM0_POWER_DISABLE	Arm Cortex-M3 PRAM0 power disabled	0x0
	CM3_PRAM0_POWER_ENABLE	Arm Cortex-M3 PRAM0 power enabled	0x1*

11.8.2 SYSCTRL_MEM_POWER_CFG1

Bit Field	Read/Write	Field Name	Description
19	RW	FENG_PRAM_POWER	FENG PRAM power configuration
18	RW	FENG_COEF_POWER	FENG COEF power configuration
17	RW	FENG_STATE_POWER	FENG STATE power configuration
16	RW	HEAR_ABCD_PART3_POWER	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 3 power configuration
15	RW	HEAR_ABCD_PART2_POWER	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 2 power configuration
14	RW	HEAR_ABCD_PART1_POWER	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 1 power configuration
13	RW	HEAR_ABCD_PART0_POWER	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 0 power configuration
12	RW	HEAR_HMEM_POWER	HEAR HMEM power configuration
11	RW	HEAR_PRAM1_POWER	HEAR PRAM1 power configuration
10	RW	HEAR_PRAM0_POWER	HEAR PRAM0 power configuration
9	RW	CFX_YRAM2_POWER	CFX YRAM2 power configuration
8	RW	CFX_YRAM1_POWER	CFX YRAM1 power configuration
7	RW	CFX_YRAM0_POWER	CFX YRAM0 power configuration
6	RW	CFX_XRAM3_POWER	CFX XRAM3 power configuration
5	RW	CFX_XRAM2_POWER	CFX XRAM2 power configuration
4	RW	CFX_XRAM1_POWER	CFX XRAM1 power configuration
3	RW	CFX_XRAM0_POWER	CFX XRAM0 power configuration
2	RW	CFX_PRAM1_POWER	CFX PRAM1 power configuration
1	RW	CFX_PRAM0_POWER	CFX PRAM0 power configuration
0	RW	CFX_PROM_POWER	CFX PROM power configuration

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
FENG_PRAM_POWER	FENG_PRAM_POWER_DISABLE	FENG PRAM power disabled	0x0
	FENG_PRAM_POWER_ENABLE	FENG PRAM power enabled	0x1*
FENG_COEF_POWER	FENG_COEF_POWER_DISABLE	FENG COEF power disabled	0x0
	FENG_COEF_POWER_ENABLE	FENG COEF power enabled	0x1*
FENG_STATE_POWER	FENG_STATE_POWER_DISABLE	FENG STATE power disabled	0x0
	FENG_STATE_POWER_ENABLE	FENG STATE power enabled	0x1*
HEAR_ABCD_PART3_POWER	HEAR_ABCD_PART3_POWER_DISABLE	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 3 power disabled	0x0
	HEAR_ABCD_PART3_POWER_ENABLE	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 3 power enabled	0x1*
HEAR_ABCD_PART2_POWER	HEAR_ABCD_PART2_POWER_DISABLE	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 2 power disabled	0x0
	HEAR_ABCD_PART2_POWER_ENABLE	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 2 power enabled	0x1*
HEAR_ABCD_PART1_POWER	HEAR_ABCD_PART1_POWER_DISABLE	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 1 power disabled	0x0
	HEAR_ABCD_PART1_POWER_ENABLE	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 1 power enabled	0x1*
HEAR_ABCD_PART0_POWER	HEAR_ABCD_PART0_POWER_DISABLE	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 0 power disabled	0x0
	HEAR_ABCD_PART0_POWER_ENABLE	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 0 power enabled	0x1*
HEAR_HMEM_POWER	HEAR_HMEM_POWER_DISABLE	HEAR HMEM power disabled	0x0
	HEAR_HMEM_POWER_ENABLE	HEAR HMEM power enabled	0x1*
HEAR_PRAM1_POWER	HEAR_PRAM1_POWER_DISABLE	HEAR PRAM1 power disabled	0x0
	HEAR_PRAM1_POWER_ENABLE	HEAR PRAM1 power enabled	0x1*
HEAR_PRAM0_POWER	HEAR_PRAM0_POWER_DISABLE	HEAR PRAM0 power disabled	0x0
	HEAR_PRAM0_POWER_ENABLE	HEAR PRAM0 power enabled	0x1*
CFX_YRAM2_POWER	CFX_YRAM2_POWER_DISABLE	CFX YRAM2 power disabled	0x0
	CFX_YRAM2_POWER_ENABLE	CFX YRAM2 power enabled	0x1*
CFX_YRAM1_POWER	CFX_YRAM1_POWER_DISABLE	CFX YRAM1 power disabled	0x0
	CFX_YRAM1_POWER_ENABLE	CFX YRAM1 power enabled	0x1*
CFX_YRAM0_POWER	CFX_YRAM0_POWER_DISABLE	CFX YRAM0 power disabled	0x0
	CFX_YRAM0_POWER_ENABLE	CFX YRAM0 power enabled	0x1*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
CFX_XRAM3_POWER	CFX_XRAM3_POWER_DISABLE	CFX XRAM3 power disabled	0x0
	CFX_XRAM3_POWER_ENABLE	CFX XRAM3 power enabled	0x1*
CFX_XRAM2_POWER	CFX_XRAM2_POWER_DISABLE	CFX XRAM2 power disabled	0x0
	CFX_XRAM2_POWER_ENABLE	CFX XRAM2 power enabled	0x1*
CFX_XRAM1_POWER	CFX_XRAM1_POWER_DISABLE	CFX XRAM1 power disabled	0x0
	CFX_XRAM1_POWER_ENABLE	CFX XRAM1 power enabled	0x1*
CFX_XRAM0_POWER	CFX_XRAM0_POWER_DISABLE	CFX XRAM0 power disabled	0x0
	CFX_XRAM0_POWER_ENABLE	CFX XRAM0 power enabled	0x1*
CFX_PRAM1_POWER	CFX_PRAM1_POWER_DISABLE	CFX PRAM1 power disabled	0x0
	CFX_PRAM1_POWER_ENABLE	CFX PRAM1 power enabled	0x1*
CFX_PRAM0_POWER	CFX_PRAM0_POWER_DISABLE	CFX PRAM0 power disabled	0x0
	CFX_PRAM0_POWER_ENABLE	CFX PRAM0 power enabled	0x1*
CFX_PROM_POWER	CFX_PROM_POWER_DISABLE	CFX PROM power disabled	0x0
	CFX_PROM_POWER_ENABLE	CFX PROM power enabled	0x1*

11.8.3 SYSCTRL_MEM_ACCESS_CFG0

Bit Field	Read/Write	Field Name	Description
26	RW	DSP_BRAM2_ACCESS	DSP BRAM2 access configuration
25	RW	DSP_BRAM1_ACCESS	DSP BRAM1 access configuration
24	RW	DSP_BRAM0_ACCESS	DSP BRAM0 access configuration
23	RW	DSP_ARAM5_ACCESS	DSP ARAM5 access configuration
22	RW	DSP_ARAM4_ACCESS	DSP ARAM4 access configuration
21	RW	DSP_ARAM3_ACCESS	DSP ARAM3 access configuration
20	RW	DSP_ARAM2_ACCESS	DSP ARAM2 access configuration
19	RW	DSP_ARAM1_ACCESS	DSP ARAM1 access configuration
18	RW	DSP_ARAM0_ACCESS	DSP ARAM0 access configuration
17	RW	DSP_PRAM5_ACCESS	DSP PRAM5 access configuration
16	RW	DSP_PRAM4_ACCESS	DSP PRAM4 access configuration
15	RW	DSP_PRAM3_ACCESS	DSP PRAM3 access configuration
14	RW	DSP_PRAM2_ACCESS	DSP PRAM2 access configuration
13	RW	DSP_PRAM1_ACCESS	DSP PRAM1 access configuration

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
12	RW	DSP_PRAM0_ACCESS	DSP PRAM0 access configuration
11	RW	CM3_EMMC_RAM_ACCESS	eMMC RAM access configuration
10	RW	CM3_DRAM3_ACCESS	Arm Cortex-M3 DRAM3 access configuration
9	RW	CM3_DRAM2_ACCESS	Arm Cortex-M3 DRAM2 access configuration
8	RW	CM3_DRAM1_ACCESS	Arm Cortex-M3 DRAM1 access configuration
7	RW	CM3_DRAM0_ACCESS	Arm Cortex-M3 DRAM0 access configuration
6	RW	CM3_PRAM6_ACCESS	Arm Cortex-M3 PRAM6 access configuration
5	RW	CM3_PRAM5_ACCESS	Arm Cortex-M3 PRAM5 access configuration
4	RW	CM3_PRAM4_ACCESS	Arm Cortex-M3 PRAM4 access configuration
3	RW	CM3_PRAM3_ACCESS	Arm Cortex-M3 PRAM3 access configuration
2	RW	CM3_PRAM2_ACCESS	Arm Cortex-M3 PRAM2 access configuration
1	RW	CM3_PRAM1_ACCESS	Arm Cortex-M3 PRAM1 access configuration
0	RW	CM3_PRAM0_ACCESS	Arm Cortex-M3 PRAM0 access configuration

Field Name	Value Symbol	Value Description	Hex Value
DSP_BRAM2_ACCESS	DSP_BRAM2_ACCESS_DISABLE	DSP BRAM2 access disabled	0x0
	DSP_BRAM2_ACCESS_ENABLE	DSP BRAM2 access enabled	0x1*
DSP_BRAM1_ACCESS	DSP_BRAM1_ACCESS_DISABLE	DSP BRAM1 access disabled	0x0
	DSP_BRAM1_ACCESS_ENABLE	DSP BRAM1 access enabled	0x1*
DSP_BRAM0_ACCESS	DSP_BRAM0_ACCESS_DISABLE	DSP BRAM0 access disabled	0x0
	DSP_BRAM0_ACCESS_ENABLE	DSP BRAM0 access enabled	0x1*
DSP_ARAM5_ACCESS	DSP_ARAM5_ACCESS_DISABLE	DSP ARAM5 access disabled	0x0
	DSP_ARAM5_ACCESS_ENABLE	DSP ARAM5 access enabled	0x1*
DSP_ARAM4_ACCESS	DSP_ARAM4_ACCESS_DISABLE	DSP ARAM4 access disabled	0x0
	DSP_ARAM4_ACCESS_ENABLE	DSP ARAM4 access enabled	0x1*
DSP_ARAM3_ACCESS	DSP_ARAM3_ACCESS_DISABLE	DSP ARAM3 access disabled	0x0
	DSP_ARAM3_ACCESS_ENABLE	DSP ARAM3 access enabled	0x1*
DSP_ARAM2_ACCESS	DSP_ARAM2_ACCESS_DISABLE	DSP ARAM2 access disabled	0x0
	DSP_ARAM2_ACCESS_ENABLE	DSP ARAM2 access enabled	0x1*
DSP_ARAM1_ACCESS	DSP_ARAM1_ACCESS_DISABLE	DSP ARAM1 access disabled	0x0
	DSP_ARAM1_ACCESS_ENABLE	DSP ARAM1 access enabled	0x1*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DSP_ARAM0_ACCESS	DSP_ARAM0_ACCESS_DISABLE	DSP ARAM0 access disabled	0x0
	DSP_ARAM0_ACCESS_ENABLE	DSP ARAM0 access enabled	0x1*
DSP_PRAM5_ACCESS	DSP_PRAM5_ACCESS_DISABLE	DSP PRAM5 access disabled	0x0
	DSP_PRAM5_ACCESS_ENABLE	DSP PRAM5 access enabled	0x1*
DSP_PRAM4_ACCESS	DSP_PRAM4_ACCESS_DISABLE	DSP PRAM4 access disabled	0x0
	DSP_PRAM4_ACCESS_ENABLE	DSP PRAM4 access enabled	0x1*
DSP_PRAM3_ACCESS	DSP_PRAM3_ACCESS_DISABLE	DSP PRAM3 access disabled	0x0
	DSP_PRAM3_ACCESS_ENABLE	DSP PRAM3 access enabled	0x1*
DSP_PRAM2_ACCESS	DSP_PRAM2_ACCESS_DISABLE	DSP PRAM2 access disabled	0x0
	DSP_PRAM2_ACCESS_ENABLE	DSP PRAM2 access enabled	0x1*
DSP_PRAM1_ACCESS	DSP_PRAM1_ACCESS_DISABLE	DSP PRAM1 access disabled	0x0
	DSP_PRAM1_ACCESS_ENABLE	DSP PRAM1 access enabled	0x1*
DSP_PRAM0_ACCESS	DSP_PRAM0_ACCESS_DISABLE	DSP PRAM0 access disabled	0x0
	DSP_PRAM0_ACCESS_ENABLE	DSP PRAM0 access enabled	0x1*
CM3_EMMC_RAM_ACCESS	CM3_EMMC_RAM_ACCESS_DISABLE	Arm Cortex-M3 eMMC RAM access disabled	0x0
	CM3_EMMC_RAM_ACCESS_ENABLE	Arm Cortex-M3 eMMC RAM access enabled	0x1*
CM3_DRAM3_ACCESS	CM3_DRAM3_ACCESS_DISABLE	Arm Cortex-M3 DRAM3 access disabled	0x0
	CM3_DRAM3_ACCESS_ENABLE	Arm Cortex-M3 DRAM3 access enabled	0x1*
CM3_DRAM2_ACCESS	CM3_DRAM2_ACCESS_DISABLE	Arm Cortex-M3 DRAM2 access disabled	0x0
	CM3_DRAM2_ACCESS_ENABLE	Arm Cortex-M3 DRAM2 access enabled	0x1*
CM3_DRAM1_ACCESS	CM3_DRAM1_ACCESS_DISABLE	Arm Cortex-M3 DRAM1 access disabled	0x0
	CM3_DRAM1_ACCESS_ENABLE	Arm Cortex-M3 DRAM1 access enabled	0x1*
CM3_DRAM0_ACCESS	CM3_DRAM0_ACCESS_DISABLE	Arm Cortex-M3 DRAM0 access disabled	0x0
	CM3_DRAM0_ACCESS_ENABLE	Arm Cortex-M3 DRAM0 access enabled	0x1*
CM3_PRAM6_ACCESS	CM3_PRAM6_ACCESS_DISABLE	Arm Cortex-M3 PRAM6 access	0x0

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		disabled	
	CM3_PRAM6_ACCESS_ENABLE	Arm Cortex-M3 PRAM6 access enabled	0x1*
CM3_PRAM5_ACCESS	CM3_PRAM5_ACCESS_DISABLE	Arm Cortex-M3 PRAM5 access disabled	0x0
	CM3_PRAM5_ACCESS_ENABLE	Arm Cortex-M3 PRAM5 access enabled	0x1*
CM3_PRAM4_ACCESS	CM3_PRAM4_ACCESS_DISABLE	Arm Cortex-M3 PRAM4 access disabled	0x0
	CM3_PRAM4_ACCESS_ENABLE	Arm Cortex-M3 PRAM4 access enabled	0x1*
CM3_PRAM3_ACCESS	CM3_PRAM3_ACCESS_DISABLE	Arm Cortex-M3 PRAM3 access disabled	0x0
	CM3_PRAM3_ACCESS_ENABLE	Arm Cortex-M3 PRAM3 access enabled	0x1*
CM3_PRAM2_ACCESS	CM3_PRAM2_ACCESS_DISABLE	Arm Cortex-M3 PRAM2 access disabled	0x0
	CM3_PRAM2_ACCESS_ENABLE	Arm Cortex-M3 PRAM2 access enabled	0x1*
CM3_PRAM1_ACCESS	CM3_PRAM1_ACCESS_DISABLE	Arm Cortex-M3 PRAM1 access disabled	0x0
	CM3_PRAM1_ACCESS_ENABLE	Arm Cortex-M3 PRAM1 access enabled	0x1*
CM3_PRAM0_ACCESS	CM3_PRAM0_ACCESS_DISABLE	Arm Cortex-M3 PRAM0 access disabled	0x0
	CM3_PRAM0_ACCESS_ENABLE	Arm Cortex-M3 PRAM0 access enabled	0x1*

11.8.4 SYSCTRL_MEM_ACCESS_CFG1

Bit Field	Read/Write	Field Name	Description
19	RW	FENG_PRAM_ACCESS	FENG PRAM access configuration
18	RW	FENG_COEF_ACCESS	FENG COEF access configuration
17	RW	FENG_STATE_ACCESS	FENG STATE access configuration
16	RW	HEAR_ABCD_PART3_ACCESS	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 3 access configuration
15	RW	HEAR_ABCD_PART2_ACCESS	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 2 access

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
			configuration
14	RW	HEAR_ABCD_PART1_ACCESS	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 1 access configuration
13	RW	HEAR_ABCD_PART0_ACCESS	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 0 access configuration
12	RW	HEAR_HMEM_ACCESS	HEAR HMEM access configuration
11	RW	HEAR_PRAM1_ACCESS	HEAR PRAM1 access configuration
10	RW	HEAR_PRAM0_ACCESS	HEAR PRAM0 access configuration
9	RW	CFX_YRAM2_ACCESS	CFX YRAM2 access configuration
8	RW	CFX_YRAM1_ACCESS	CFX YRAM1 access configuration
7	RW	CFX_YRAM0_ACCESS	CFX YRAM0 access configuration
6	RW	CFX_XRAM3_ACCESS	CFX XRAM3 access configuration
5	RW	CFX_XRAM2_ACCESS	CFX XRAM2 access configuration
4	RW	CFX_XRAM1_ACCESS	CFX XRAM1 access configuration
3	RW	CFX_XRAM0_ACCESS	CFX XRAM0 access configuration
2	RW	CFX_PRAM1_ACCESS	CFX PRAM1 access configuration
1	RW	CFX_PRAM0_ACCESS	CFX PRAM0 access configuration
0	RW	CFX_PROM_ACCESS	CFX PROM access configuration

Field Name	Value Symbol	Value Description	Hex Value
FENG_PRAM_ACCESS	FENG_PRAM_ACCESS_DISABLE	FENG PRAM access disabled	0x0
	FENG_PRAM_ACCESS_ENABLE	FENG PRAM access enabled	0x1*
FENG_COEF_ACCESS	FENG_COEF_ACCESS_DISABLE	FENG COEF access disabled	0x0
	FENG_COEF_ACCESS_ENABLE	FENG COEF access enabled	0x1*
FENG_STATE_ACCESS	FENG_STATE_ACCESS_DISABLE	FENG STATE access disabled	0x0
	FENG_STATE_ACCESS_ENABLE	FENG STATE access enabled	0x1*
HEAR_ABCD_PART3_ACCESS	HEAR_ABCD_PART3_ACCESS_DISABLE	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 3 access disabled	0x0
	HEAR_ABCD_PART3_ACCESS_ENABLE	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 3 access enabled	0x1*
HEAR_ABCD_PART2_ACCESS	HEAR_ABCD_PART2_ACCESS_DISABLE	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 2 access disabled	0x0
	HEAR_ABCD_PART2_ACCESS_	HEAR A0/A1/B0/B1/C0/C1/D0/D1	0x1*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	ENABLE	memories part 2 access enabled	
HEAR_ABCD_PART1_ACCESS	HEAR_ABCD_PART1_ACCESS_DISABLE	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 1 access disabled	0x0
	HEAR_ABCD_PART1_ACCESS_ENABLE	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 1 access enabled	0x1*
HEAR_ABCD_PART0_ACCESS	HEAR_ABCD_PART0_ACCESS_DISABLE	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 0 access disabled	0x0
	HEAR_ABCD_PART0_ACCESS_ENABLE	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 0 access enabled	0x1*
HEAR_HMEM_ACCESS	HEAR_HMEM_ACCESS_DISABLE	HEAR HMEM access disabled	0x0
	HEAR_HMEM_ACCESS_ENABLE	HEAR HMEM access enabled	0x1*
HEAR_PRAM1_ACCESS	HEAR_PRAM1_ACCESS_DISABLE	HEAR PRAM1 access disabled	0x0
	HEAR_PRAM1_ACCESS_ENABLE	HEAR PRAM1 access enabled	0x1*
HEAR_PRAM0_ACCESS	HEAR_PRAM0_ACCESS_DISABLE	HEAR PRAM0 access disabled	0x0
	HEAR_PRAM0_ACCESS_ENABLE	HEAR PRAM0 access enabled	0x1*
CFX_YRAM2_ACCESS	CFX_YRAM2_ACCESS_DISABLE	CFX YRAM2 access disabled	0x0
	CFX_YRAM2_ACCESS_ENABLE	CFX YRAM2 access enabled	0x1*
CFX_YRAM1_ACCESS	CFX_YRAM1_ACCESS_DISABLE	CFX YRAM1 access disabled	0x0
	CFX_YRAM1_ACCESS_ENABLE	CFX YRAM1 access enabled	0x1*
CFX_YRAM0_ACCESS	CFX_YRAM0_ACCESS_DISABLE	CFX YRAM0 access disabled	0x0
	CFX_YRAM0_ACCESS_ENABLE	CFX YRAM0 access enabled	0x1*
CFX_XRAM3_ACCESS	CFX_XRAM3_ACCESS_DISABLE	CFX XRAM3 access disabled	0x0
	CFX_XRAM3_ACCESS_ENABLE	CFX XRAM3 access enabled	0x1*
CFX_XRAM2_ACCESS	CFX_XRAM2_ACCESS_DISABLE	CFX XRAM2 access disabled	0x0
	CFX_XRAM2_ACCESS_ENABLE	CFX XRAM2 access enabled	0x1*
CFX_XRAM1_ACCESS	CFX_XRAM1_ACCESS_DISABLE	CFX XRAM1 access disabled	0x0
	CFX_XRAM1_ACCESS_ENABLE	CFX XRAM1 access enabled	0x1*
CFX_XRAM0_ACCESS	CFX_XRAM0_ACCESS_DISABLE	CFX XRAM0 access disabled	0x0
	CFX_XRAM0_ACCESS_ENABLE	CFX XRAM0 access enabled	0x1*
CFX_PRAM1_ACCESS	CFX_PRAM1_ACCESS_DISABLE	CFX PRAM1 access disabled	0x0
	CFX_PRAM1_ACCESS_ENABLE	CFX PRAM1 access enabled	0x1*
CFX_PRAM0_ACCESS	CFX_PRAM0_ACCESS_DISABLE	CFX PRAM0 access disabled	0x0

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	CFX_PRAM0_ACCESS_ENABLE	CFX PRAM0 access enabled	0x1*
CFX_PROM_ACCESS	CFX_PROM_ACCESS_DISABLE	CFX PROM access disabled	0x0
	CFX_PROM_ACCESS_ENABLE	CFX PROM access enabled	0x1*

11.8.5 SYSCTRL_MEM_ARBITER_CFG

Bit Field	Read/Write	Field Name	Description
9:8	RW	DSP_MAX_WAIT	DSP maximum wait cycles configuration
7:6	RW	CFX_MAX_WAIT	CFX maximum wait cycles configuration
5:4	RW	DMA_MAX_WAIT	DMA maximum wait cycles configuration
3:2	RW	CM3_MAX_WAIT	Arm Cortex-M3 maximum wait cycles configuration
1:0	RW	PRIORITY_ORDER	Arm Cortex-M3 and DSP memory priority order configuration

Field Name	Value Symbol	Value Description	Hex Value
DSP_MAX_WAIT	DSP_MAX_WAIT_3_CYCLES	DSP maximum wait time set to 3 cycles	0x0
	DSP_MAX_WAIT_7_CYCLES	DSP maximum wait time set to 7 cycles	0x1
	DSP_MAX_WAIT_15_CYCLES	DSP maximum wait time set to 15 cycles	0x2
	DSP_MAX_WAIT_31_CYCLES	DSP maximum wait time set to 31 cycles	0x3*
CFX_MAX_WAIT	CFX_MAX_WAIT_3_CYCLES	CFX maximum wait time set to 3 cycles	0x0
	CFX_MAX_WAIT_7_CYCLES	CFX maximum wait time set to 7 cycles	0x1
	CFX_MAX_WAIT_15_CYCLES	CFX maximum wait time set to 15 cycles	0x2
	CFX_MAX_WAIT_31_CYCLES	CFX maximum wait time set to 31 cycles	0x3*
DMA_MAX_WAIT	DMA_MAX_WAIT_3_CYCLES	DMA maximum wait time set to 3 cycles	0x0
	DMA_MAX_WAIT_7_CYCLES	DMA maximum wait time set to 7 cycles	0x1
	DMA_MAX_WAIT_15_CYCLES	DMA maximum wait time set to 15 cycles	0x2
	DMA_MAX_WAIT_31_CYCLES	DMA maximum wait time set to 31 cycles	0x3*
CM3_MAX_WAIT	CM3_MAX_WAIT_3_CYCLES	Arm Cortex-M3 maximum wait time set to 3 cycles	0x0
	CM3_MAX_WAIT_7_CYCLES	Arm Cortex-M3 maximum wait time set to 7 cycles	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		to 7 cycles	
	CM3_MAX_WAIT_15_CYCLES	Arm Cortex-M3 maximum wait time set to 15 cycles	0x2
	CM3_MAX_WAIT_31_CYCLES	Arm Cortex-M3 maximum wait time set to 31 cycles	0x3*
PRIORITY_ORDER	CM3_DSP_CFX_PRIORITY	Priority access to the memories is Arm Cortex-M3/DSP/CFX	0x0*
	CM3_CFX_DSP_PRIORITY	Priority access to the memories is Arm Cortex-M3/CFX/DSP	0x1
	DSP_CFX_CM3_PRIORITY	Priority access to the memories is DSP/CFX/Arm Cortex-M3	0x2
	CFX_DSP_CM3_PRIORITY	Priority access to the memories is CFX/DSP/Arm Cortex-M3	0x3

11.8.6 SYSCTRL_MEM_ERROR

Bit Field	Read/Write	Field Name	Description
3	R	DMA_MEM_ERROR	DMA memory error flag
2	R	CFX_MEM_ERROR	CFX memory error flag
1	R	DSP_MEM_ERROR	DSP memory error flag
0	W	MEM_ERROR_CLEAR	Write a 1 to clear the memory error flags

Field Name	Value Symbol	Value Description	Hex Value
DMA_MEM_ERROR	DMA_MEM_NO_ERROR_DETECTED	No DMA memory error detected	0x0*
	DMA_MEM_ERROR_DETECTED	DMA has accessed an isolated memory	0x1
CFX_MEM_ERROR	CFX_MEM_NO_ERROR_DETECTED	No CFX memory error detected	0x0*
	CFX_MEM_ERROR_DETECTED	CFX has accessed an isolated memory	0x1
DSP_MEM_ERROR	DSP_MEM_NO_ERROR_DETECTED	No DSP memory error detected	0x0*
	DSP_MEM_ERROR_DETECTED	DSP has accessed an isolated memory	0x1
MEM_ERROR_CLEAR	MEM_ERROR_CLEAR	Clear the memory error flags	0x1

11.8.7 SYSCTRL_CM3_LOOP_CACHE_CFG

Bit Field	Read/Write	Field Name	Description
0	RW	CM3_LOOP_CACHE_ENABLE	Arm Cortex-M3 loop cache enable

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
CM3_LOOP_CACHE_ENABLE	CM3_LOOP_CACHE_DISABLE	Arm Cortex-M3 loop cache disabled	0x0
	CM3_LOOP_CACHE_ENABLE	Arm Cortex-M3 loop cache enabled	0x1*

11.8.8 SYSCTRL_DSP_LOOP_CACHE_CFG

Bit Field	Read/Write	Field Name	Description
0	RW	DSP_LOOP_CACHE_ENABLE	DSP loop cache enable

Field Name	Value Symbol	Value Description	Hex Value
DSP_LOOP_CACHE_ENABLE	DSP_LOOP_CACHE_DISABLE	DSP loop cache disabled	0x0
	DSP_LOOP_CACHE_ENABLE	DSP loop cache enabled	0x1*

11.8.9 SYSCTRL_PRAM_CFG

Bit Field	Read/Write	Field Name	Description
14	W	PRAM6_LOCK	Lock PRAM6 access except for IBus
14	R	PRAM6_LOCK_STATUS	PRAM6 access lock status
13	W	PRAM5_LOCK	Lock PRAM5 access except for IBus
13	R	PRAM5_LOCK_STATUS	PRAM5 access lock status
12	W	PRAM4_LOCK	Lock PRAM4 access except for IBus
12	R	PRAM4_LOCK_STATUS	PRAM4 access lock status
11	W	PRAM3_LOCK	Lock PRAM3 access except for IBus
11	R	PRAM3_LOCK_STATUS	PRAM3 access lock status
10	W	PRAM2_LOCK	Lock PRAM2 access except for IBus
10	R	PRAM2_LOCK_STATUS	PRAM2 access lock status
9	W	PRAM1_LOCK	Lock PRAM1 access except for IBus
9	R	PRAM1_LOCK_STATUS	PRAM1 access lock status
8	W	PRAM0_LOCK	Lock PRAM0 access except for IBus
8	R	PRAM0_LOCK_STATUS	PRAM0 access lock status
2	W	PRAM_ASSIGN_LOCK	Lock PRAM0 and PRAM1 assignments
2	R	PRAM_ASSIGN_LOCK_STATUS	PRAM0 and PRAM1 assignments lock status
1	RW	PRAM1_ASSIGN	Assign PRAM1 to the Arm Cortex-M3 core or CFX
0	RW	PRAM0_ASSIGN	Assign PRAM0 to the Arm Cortex-M3 core or CFX

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
PRAM6_LOCK	PRAM6_LOCK	Lock PRAM6 access except for IBus	0x1
PRAM6_LOCK_STATUS	PRAM6_NOT_LOCKED	PRAM6 access is not locked	0x0*
	PRAM6_LOCKED	PRAM6 access is locked except for IBus	0x1
PRAM5_LOCK	PRAM5_LOCK	Lock PRAM5 access except for IBus	0x1
PRAM5_LOCK_STATUS	PRAM5_NOT_LOCKED	PRAM5 access is not locked	0x0*
	PRAM5_LOCKED	PRAM5 access is locked except for IBus	0x1
PRAM4_LOCK	PRAM4_LOCK	Lock PRAM4 access except for IBus	0x1
PRAM4_LOCK_STATUS	PRAM4_NOT_LOCKED	PRAM4 access is not locked	0x0*
	PRAM4_LOCKED	PRAM4 access is locked except for IBus	0x1
PRAM3_LOCK	PRAM3_LOCK	Lock PRAM3 access except for IBus	0x1
PRAM3_LOCK_STATUS	PRAM3_NOT_LOCKED	PRAM3 access is not locked	0x0*
	PRAM3_LOCKED	PRAM3 access is locked except for IBus	0x1
PRAM2_LOCK	PRAM2_LOCK	Lock PRAM2 access except for IBus	0x1
PRAM2_LOCK_STATUS	PRAM2_NOT_LOCKED	PRAM2 access is not locked	0x0*
	PRAM2_LOCKED	PRAM2 access is locked except for IBus	0x1
PRAM1_LOCK	PRAM1_LOCK	Lock PRAM1 access except for IBus	0x1
PRAM1_LOCK_STATUS	PRAM1_NOT_LOCKED	PRAM1 access is not locked	0x0*
	PRAM1_LOCKED	PRAM1 access is locked except for IBus	0x1
PRAM0_LOCK	PRAM0_LOCK	Lock PRAM0 access except for IBus	0x1
PRAM0_LOCK_STATUS	PRAM0_NOT_LOCKED	PRAM0 access is not locked	0x0*
	PRAM0_LOCKED	PRAM0 access is locked except for IBus	0x1
PRAM_ASSIGN_LOCK	PRAM_ASSIGN_LOCK	Lock PRAM0 and PRAM1 assignments	0x1
PRAM_ASSIGN_LOCK_STATUS	PRAM_ASSIGN_NOT_LOCKED	PRAM0 and PRAM1 assignments are not locked	0x0*
	PRAM_ASSIGN_LOCKED	PRAM0 and PRAM1 assignments are locked	0x1
PRAM1_ASSIGN	PRAM1_CM3	PRAM1 is assigned to the Arm Cortex-M3 core	0x0*
	PRAM1_CFX	PRAM1 is assigned to CFX	0x1
PRAM0_ASSIGN	PRAM0_CM3	PRAM0 is assigned to the Arm Cortex-M3 core	0x0*
	PRAM0_CFX	PRAM0 is assigned to CFX	0x1

CHAPTER 12

FIFO Controller

The Ezairo 8300 system's FIFO controller provides the ability to define up to 32 FIFO buffers. Up to 8 FIFOs can be defined in each of the A0, A1, B0 and B1 memories. To simplify discussion, the FIFOs are denoted as FIFO A0.*, FIFO A1.*, FIFO B0.*, and FIFO B1.* in documentation and defines. These FIFOs can be used as:

- Input FIFOs as used by the IOCs
- Output FIFOs as used by the IOCs
- Intermediate FIFOs for transferring data between the CFX, HEAR and Arm Cortex-M3 cores
- Software FIFOs for the CFX DSP or Arm Cortex-M3 processor through the use of access registers

12.1 INTRODUCTION TO THE FIFO CONTROLLER

Each individual FIFO is associated with a set of FIFO configuration registers that are mapped into X memory starting at X:0x01000 (0x20804000 from the Arm Cortex-M3 processor perspective, as defined in the "FIFO Base Addresses" table (Table 39)). Each FIFO can also be associated with one or more of the eight FIFO interrupts that are available for the CFX DSP, eight FIFO events that are available for the HEAR, and eight FIFO interrupts that are available for the Arm Cortex-M3 core.

IMPORTANT: Using the HEAR reset by setting the `SYSTEM_CTRL_HEAR_RESET` bit in the `D_SYSTEM_CTRL` register resets the FIFO controllers, clearing the current FIFO register settings.

12.1.1 FIFO Control and Configuration Register Conventions

Each memory has an identical set of eight FIFOs. As a result, all registers described in this chapter define x in `D_FIFO_x_*` to allow us to represent all of the FIFO registers as a group by replacing:

x

with the name of the memory space: A0, A1, B0 or B1

The base addresses for FIFOs of the CFX and Arm Cortex-M3 processors can be found in the "FIFO Base Addresses" table (Table 39).

Table 39. FIFO Base Addresses

FIFO	CFX Base Address	Arm Cortex-M3 Core Base Address
A0.0	X:0x1000	0x20804000
A0.1	X:0x1010	0x20804040
A0.2	X:0x1020	0x20804080
A0.3	X:0x1030	0x208040C0
A0.4	X:0x1040	0x20804100
A0.5	X:0x1050	0x20804140
A0.6	X:0x1060	0x20804180
A0.7	X:0x1070	0x208041C0
A1.0	X:0x1080	0x20804200
A1.1	X:0x1090	0x20804240

Ezairo 8300 Hardware Reference

FIFO Base Addresses (Continued)

FIFO	CFX Base Address	Arm Cortex-M3 Core Base Address
A1.2	X:0x10A0	0x20804280
A1.3	X:0x10B0	0x208042C0
A1.4	X:0x10C0	0x20804300
A1.5	X:0x10D0	0x20804340
A1.6	X:0x10E0	0x20804380
A1.7	X:0x10F0	0x208043C0
B0.0	X:0x1100	0x20804400
B0.1	X:0x1110	0x20804440
B0.2	X:0x1120	0x20804480
B0.3	X:0x1130	0x208044C0
B0.4	X:0x1140	0x20804500
B0.5	X:0x1150	0x20804540
B0.6	X:0x1160	0x20804580
B0.7	X:0x1170	0x208045C0
B1.0	X:0x1180	0x20804600
B1.1	X:0x1190	0x20804640
B1.2	X:0x11A0	0x20804680
B1.3	X:0x11B0	0x208046C0
B1.4	X:0x11C0	0x20804700
B1.5	X:0x11D0	0x20804740
B1.6	X:0x11E0	0x20804780
B1.7	X:0x11F0	0x208047C0

12.2 DEFINING MEMORY FOR USE AS A FIFO

A FIFO is a section of memory that is treated as a circular buffer, where components that use the memory use pointers to indicate the next location to be read or written. Each of the FIFO memories listed below in the ["FIFO Memories" table \(Table 40\)](#) can have eight FIFOs defined. All start and end addresses are defined relative to the address offsets listed in this table.

Table 40. FIFO Memories

Memory	Address Offset (CFX)	Address Offset (Arm Cortex-M3 processor, LSB sign extended)
A0	X:0x60000	0x20120000
A1	X:0x62000	0x20128000
B0	X:0x64000	0x20130000
B1	X:0x66000	0x20138000

Ezairo 8300 Hardware Reference

For each FIFO in memory, define a start and end point for the FIFO, and specify the size of the FIFO. The "Defining an Area of A0 Memory for Use as a FIFO" figure (Figure 20), below, shows how the FIFO start and end pointers, along with the FIFO size indicator, are used to define an area of memory for use as a FIFO.

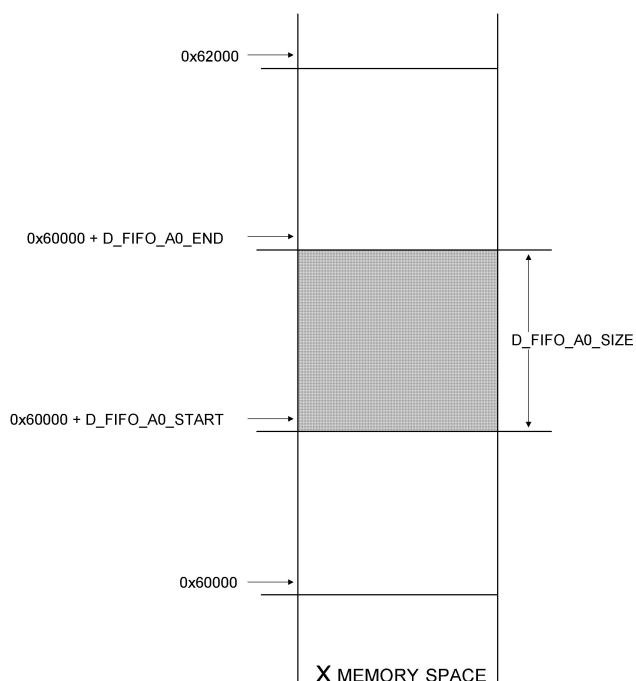


Figure 20. Defining an Area of A0 Memory for Use as a FIFO

The start address for a FIFO is defined by the associated `D_FIFO_x_START` register. This register defines the offset from the start of the memory to the FIFO. This start address is limited to steps of 32 words; therefore, the associated bit field of this register is aligned to truncate values that do not meet this requirement. The end address for a FIFO is similarly defined by an offset specified in the associated `D_FIFO_x_END` register, and this address is also limited to steps of 32 words by truncation. If a FIFO is enabled, the size of a FIFO must be defined by setting the `D_FIFO_x_SIZE` register to the value of $(D_FIFO_x_END - D_FIFO_x_START)$. FIFOs of up to 2048 words are supported. For all non-zero values, the `FIFO_x_SIZE_FIFO_ENABLE` bit from the `D_FIFO_x_SIZE` register is automatically set to indicate that the FIFO is active. The address that the `D_FIFO_x_END` register points to is not considered part of the area of memory allocated to the FIFO.

Ezairo 8300 Hardware Reference

NOTE: Setting the `D_FIFO_x_SIZE` register for a FIFO to zero disables a FIFO. All accesses from the IOCs or through the associated `D_FIFO_x_ACCESS` register are ignored.

CAUTION: Defining two or more FIFOs that overlap in memory results in undefined behavior. Further, defining an invalid FIFO configuration also results in undefined behavior. Some examples of invalid FIFO configurations include specifying a start address and end address that reserve less memory than the defined size, defining a FIFO that exceeds 2048 words, and defining an invalid area (as would be the case if the end address offset is lower than the start address offset).

All accesses to areas of the A0, A1, B0 or B1 memories that are included in defined FIFOs are processed by the FIFO controller as described throughout the remainder of this chapter. For areas of the A0, A1, B0 or B1 memories that are not included in the defined FIFO ranges, the FIFO controller provides no address manipulation. User applications access those areas of memory linearly.

12.2.1 Control and Configuration Registers

Register Name	Register Description	Address
<code>D_FIFO_x_START</code> <code>FIFO_x_START</code>	Start offset in x memory for FIFO x	X:(Base) (Base)
<code>D_FIFO_x_END</code> <code>FIFO_x_END</code>	End offset (+1) in x memory for FIFO x	X:(Base + 1) (Base + 1 * 4)
<code>D_FIFO_x_SIZE</code> <code>FIFO_x_SIZE</code>	Total size of x memory used in FIFO x	X:(Base + 2) (Base + 2 * 4)

12.2.1.1 D_FIFO_x_START, FIFO_x_START

Bit Field	Read/Write	Field Name	Description
12:5	RW	OFFSET	Start address for FIFO x

12.2.1.2 D_FIFO_x_END, FIFO_x_END

Bit Field	Read/Write	Field Name	Description
12:5	RW	OFFSET	End address (+1) for FIFO x

12.2.1.3 D_FIFO_x_SIZE, FIFO_x_SIZE

Bit Field	Read/Write	Field Name	Description
13	R	<code>FIFO_ENABLE</code>	FIFO enable for FIFO x
12:5	RW	<code>FIFO_SIZE</code>	Size (End-Start) for FIFO x

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
FIFO_ENABLE	FIFO_x_DISABLE	Disable the FIFO	0x0*
	FIFO_x_ENABLE	Enable the FIFO	0x1

12.3 FIFO BUFFER DATA AND CONFIGURATION

There are four primary uses for a FIFO in the Ezairo 8300 system:

1. To collect blocks of input data from the IOCs
2. To provide blocks of output data to the IOCs
3. In software, to supply an application with a hardware FIFO buffer
4. To transfer data between two cores, including:
 - Transferring data between the CFX DSP and the HEAR
 - Transferring data between the CFX DSP and the Arm Cortex-M3 processor
 - Transferring data between the HEAR and the Arm Cortex-M3 processor

To understand how a FIFO buffer is used for input from an IOC, it is useful to think of the FIFO buffer as divided into two areas: an area to store the input block of data that is currently being received, and an area containing previously received blocks of data. Received data is collected into a block until a complete block of samples has been received.

To understand how a FIFO buffer is used for output from an IOC, it is useful to think of the FIFO buffer as divided into two areas: an area from which the IOC is currently reading output data, and an area to store future output blocks of samples. This second area contains previously read blocks of output data until this data is overwritten by new (future) output samples.

Finally, a FIFO can be used in software as an input FIFO, an output FIFO, or both—except that data is stored or loaded through the appropriate access register (`D_FIFO_x_ACCESS`) rather than storing or loading data through the IOC. Alternatively, if a user application written for the CFX is not using the access register, it can read or write directly to the appropriate part of the FIFO and then force a block update by writing the block counter directly to advance the FIFO pointers.

CAUTION: The FIFOs are located in memory that has been organized into a 48-bit double word configuration; therefore, the access register is actually a 48-bit register that allows reading or writing of up to two adjacent words of a FIFO within one cycle. Since the CFX DSP, Arm Cortex-M3 processor, and related components do not have 48-bit access to this register, do not read or write the address immediately following an access register (i.e., the second half of the access register) because this is registered as a data access without reading or writing any useful data.

NOTE: All pointer updates, IOC accesses, and accesses through the access register use the FIFO buffer as a circular buffer. As a result, any update calculation that would result in a pointer or access outside of the FIFO itself is reduced modulo the size of the FIFO.

12.3.1 FIFO Input and Output Block

In each use case, the input/output block pointer and block size defines the area of input/output that the IOC or access register is currently accessing. The current block of input or output samples is commonly referred to as the *I/O block* or *IOBLOCK*, and is pointed to by the I/O block pointer. The elements related to the currently used I/O block are shown below in the "Current Input or Output Data Block for a FIFO" figure (Figure 21).

Ezairo 8300 Hardware Reference

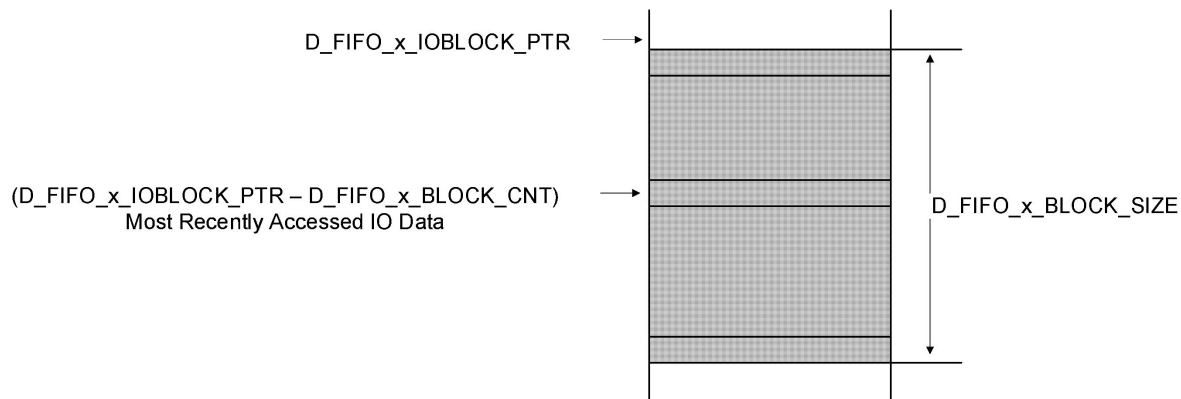


Figure 21. Current Input or Output Data Block for a FIFO

The block size for each I/O block is defined by the `D_FIFO_x_BLOCK_SIZE` register. The block size for a FIFO must not exceed the defined size of the FIFO. The block size is further limited to a range of 1 to 1024 words.

NOTE: A setting of 0 in the `D_FIFO_x_BLOCK_SIZE` register configures the FIFO for a block size of 1024. For all other block sizes, the register setting matches the block size configured.

The I/O block (shaded area of Figure 21) is an area of the block size that is filled starting at the top of the block. The I/O block pointer, which is accessible through the appropriate `D_FIFO_x_IOBLOCK_PTR` register, provides an offset within the FIFO to the first address above the top of this block, indicating that the first data item stored in the block is located one address below the pointer.

Finally, a block counter is maintained by the FIFO controller to indicate to the FIFO controller where to locate the next data access within the I/O block. This counter, which you can see through the appropriate `D_FIFO_x_BLOCK_CNT` register, indicates how many words have already been read or written in the current data block. This value is incremented following an IOC access to the FIFO data, or through the access register. The most recently accessed input or output data in the FIFO is located at $(D_FIFO_x_y_IOBLOCK_PTR - D_FIFO_x_y_BLOCK_CNT)$. The next location accessed to store data written to the FIFO, or provide data read from the FIFO, is $(D_FIFO_x_y_IOBLOCK_PTR - (D_FIFO_x_y_BLOCK_CNT + 1))$.

If at any point the block count is equal to or exceeds the block size, the FIFO configuration is updated to indicate that use of the current input/output data block is complete. This can occur when the block count is updated through one of the following:

- An IOC FIFO access
- An access register read/write
- By explicitly writing a value to the block count register from the CFX or the HEAR

When this happens, the following updates occur:

- The block count resets to 0.
- Both the I/O block pointer and base pointer are decremented by the block size. Both pointers must point to an even address, so if the block size is odd, the I/O block pointer and base pointer are decremented by (the block size + 1).

Ezairo 8300 Hardware Reference

- The interrupts associated with this FIFO are triggered. For more information about configuring CFX interrupts and using interrupts within an application, see [Section 4.6.1 “Nested Vectored Interrupt Controller \(NVIC\)” on page 135](#). For more information about configuring and using Arm Cortex-M3 processor interrupts within an application, see [Section 4.6.1 “Nested Vectored Interrupt Controller \(NVIC\)” on page 135](#).
- The HEAR receives a function chain controller event triggering the function chain linked to the FIFO. For more information about the function chain controller, see [Section 5.2 “Function Chain Controller” on page 207](#). For more information indicating which FIFO configurations to use with a HEAR microcode configuration, see the *HEAR Configurable Accelerator Reference Manual for Ezairo 8300*.

12.3.2 FIFO Processing Block

The data not currently being used for input or output from the FIFO is available for signal processing operations. A section of this area is normally reserved for use by these operations and is referred to as the processing block or processing area. The beginning of the processing block is defined by the base pointer, which you access through the appropriate `D_FIFO_x_BASE_PTR` register. The base pointer provides an offset into the FIFO that is updated along with the I/O block pointer. The base pointer is used to specify what actual hardware data is mapped to the base of the FIFO from a software application perspective. Normally the mapping is used so that the base pointer indicates where in the physical memory the most recent input blocks or output buffer area is located. The FIFO controller in turn uses this pointer to remap the data when addressed by software, so that when a software application accesses memory within a FIFO, the memory addressed at the `D_FIFO_x_START` register matches the physical memory pointed to by the base pointer.

The "Hardware Memory Mapping and Software View of FIFO Memory Remapping" figure (Figure 22) shows the area of one FIFO (shaded area) as it is bounded by the FIFO start and end points. The hardware memory mapping on the left shows where in memory the `D_FIFO_x_BASE_PTR` register actually points, and how memory would be aligned if the FIFO were disabled by setting its size to zero (see [Section 12.2 “Defining Memory for Use as a FIFO” on page 388](#) above). The software memory mapping on the right shows how memory appears to be aligned to software accessing the FIFO. This aligns the memory pointed to by the `D_FIFO_x_BASE_PTR` register to the base of the FIFO (located at the offset of the `D_FIFO_x_START` register).

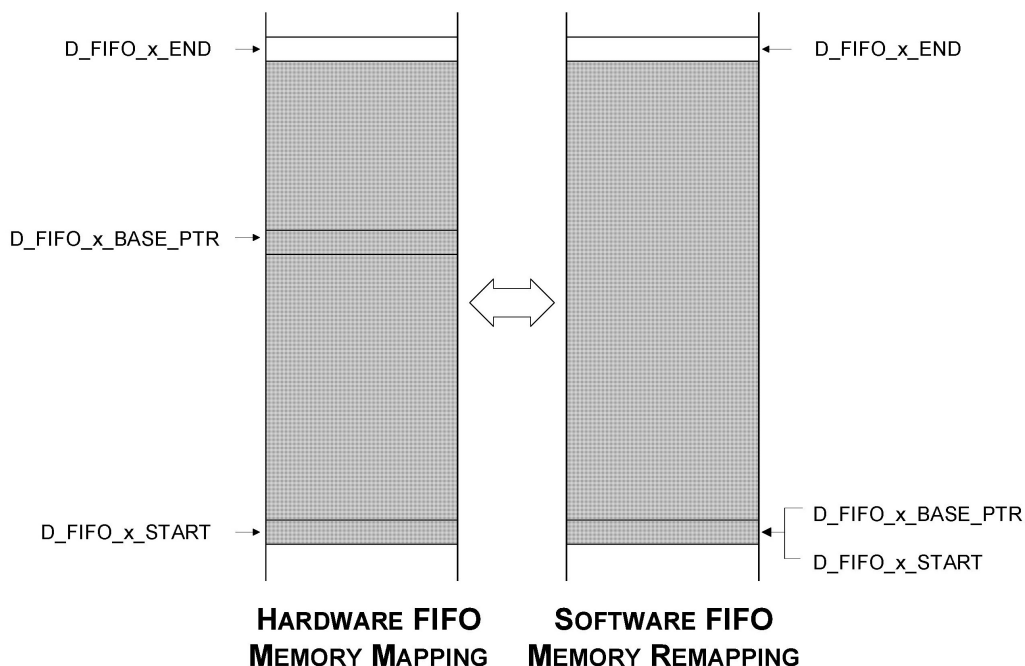


Figure 22. Hardware Memory Mapping and Software View of FIFO Memory Remapping

12.3.3 Automatic FIFO Mute

To prevent an IOC from repeatedly reading the same set of data from a FIFO, use the automatic mute feature. This feature can be enabled for a given FIFO by setting the `AUTO_MUTE_ENABLE` bit in the appropriate `D_FIFO_x_AUTO_MUTE` register.

When the automatic mute feature is enabled for a FIFO and either IOC has read all of the data in the defined FIFO memory since the last write to the FIFO, data read by the IOCs is forced to a value of `0x000000`. The IOC output data returns to normal operation whenever a write is detected or the auto mute feature is disabled.

12.3.4 Typical FIFO Configurations

12.3.4.1 Input FIFO Configuration

When using a FIFO for input, a typical initial configuration might be:

- Base pointer set to zero
- I/O block pointer set to zero

This results in the latest full block of input samples always being located at the base of memory, and provides the maximum amount of historical FIFO data to draw upon for algorithm calculations. The ["Input FIFO Hardware Memory Mapping and Software View of Memory Remapping" figure \(Figure 23\)](#) illustrates a typical input FIFO configuration.

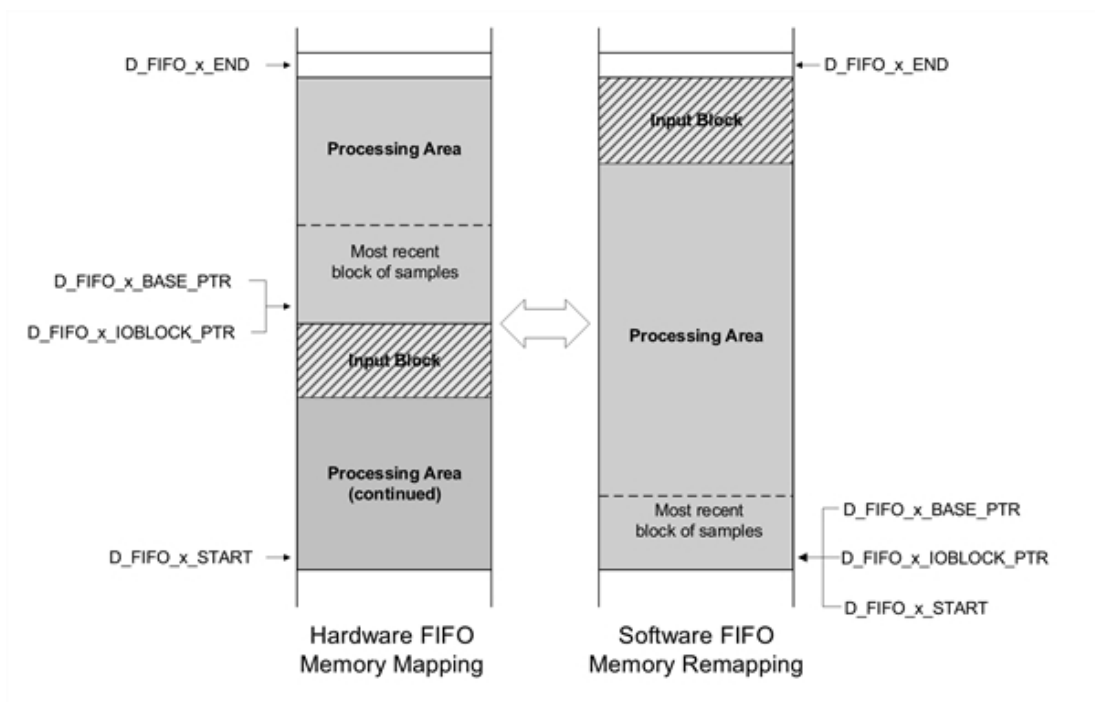


Figure 23. Input FIFO Hardware Memory Mapping and Software View of Memory Remapping

12.3.4.2 Output FIFO Configuration

When using a FIFO for output, a typical initial configuration might be:

- Base pointer set to zero
- I/O block pointer set to (processing area + block size)

This results in the top block of data from the processing area becoming the next output block each time the pointers are updated. It also ensures that the processing area is located at the start of the FIFO for convenient application access. The "Output FIFO Hardware Memory Mapping and Software View of Memory Remapping" figure (Figure 24) illustrates a typical output FIFO configuration.

Ezairo 8300 Hardware Reference

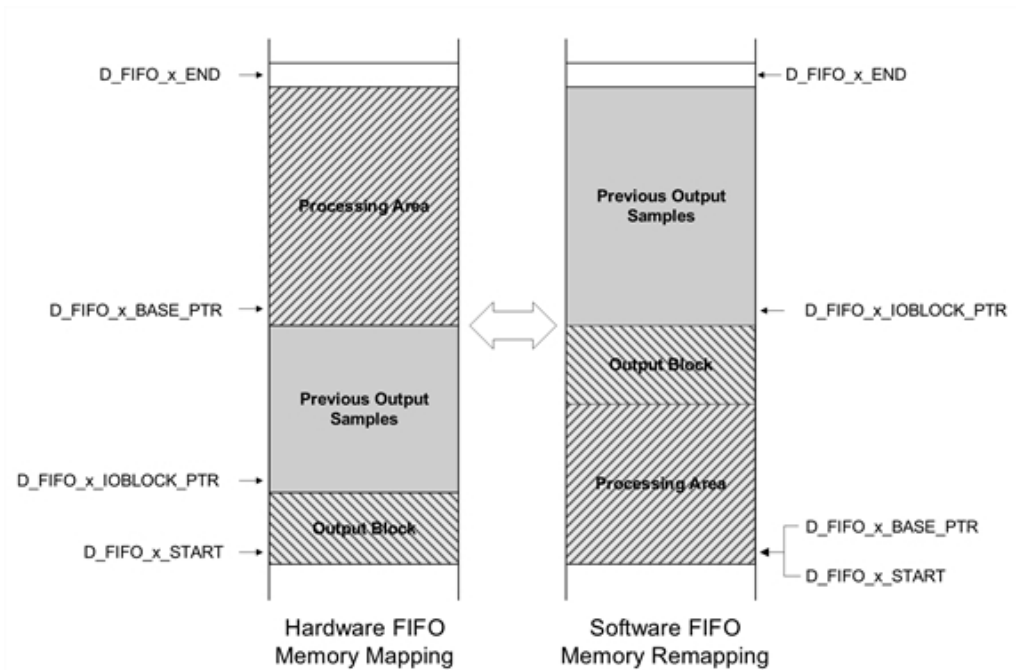


Figure 24. Output FIFO Hardware Memory Mapping and Software View of Memory Remapping

12.3.5 Control and Configuration Registers

These registers are the same for x and y, where x represents A0, A1, B0 or B1 memories, and y is 0 to 7. For FIFO base addresses, see the "FIFO Base Addresses" table (Table 39).

Register Name	Register Description	Address
FIFO_x_BASE_PTR	Pointer to the base of FIFO x	(Base + 4 * 4)
D_FIFO_x_BASE_PTR		X:(Base + 4)

Register Name	Register Description	Address
D_FIFO_x_BASE_PTR	Pointer to the base of FIFO x	X:(Base + 4)
FIFO_x_BASE_PTR		(Base + 4 * 4)
D_FIFO_x_IOBLOCK_PTR	Pointer to the latest I/O block in FIFO x	X:(Base + 5)
FIFO_x_IOBLOCK_PTR		(Base + 5 * 4)
D_FIFO_x_BLOCK_SIZE	Size of one block of samples for FIFO x	X:(Base + 8)
FIFO_x_BLOCK_SIZE		(Base + 8 * 4)
D_FIFO_x_BLOCK_CNT	Next read/write location in current sample block	X:(Base + C)

Register Name	Register Description	Address
FIFO_x_BLOCK_CNT		(Base + C * 4)
D_FIFO_x_ACCESS	Smart FIFO access for FIFO x	X:(Base + E)
FIFO_x_ACCESS		(Base + E * 4)
D_FIFO_x_AUTO_MUTE	Auto mute control of FIFO x	X:(Base +)
FIFO_x_AUTO_MUTE		(Base + * 4)

12.3.5.1 D_FIFO_x_BASE_PTR, FIFO_x_BASE_PTR

Bit Field	Read/Write	Field Name	Description
12:1	RW	FIFO_PTR	FIFO pointer for FIFO x

12.3.5.2 D_FIFO_x_IOBLOCK_PTR, FIFO_x_IOBLOCK_PTR

Bit Field	Read/Write	Field Name	Description
12:1	RW	IOC_PTR	IOC pointer for FIFO x

12.3.5.3 D_FIFO_x_BLOCK_SIZE, FIFO_x_BLOCK_SIZE

Bit Field	Read/Write	Field Name	Description
11:0	RW	BLOCK_SIZE	Block size for FIFO x

12.3.5.4 D_FIFO_x_BLOCK_CNT, FIFO_x_BLOCK_CNT

Bit Field	Read/Write	Field Name	Description
11:0	RW	BLOCK_CNT	Block counter for FIFO x

12.3.5.5 D_FIFO_x_ACCESS, FIFO_x_ACCESS

Bit Field	Read/Write	Field Name	Description
23:0	RW	FIFO_DATA	Smart FIFO access for FIFO x

12.3.5.6 D_FIFO_x_AUTO_MUTE, FIFO_x_AUTO_MUTE

Bit Field	Read/Write	Field Name	Description
0	RW	AUTO_MUTE_ENABLE	FIFO auto mute enable for FIFO x

Field Name	Value Symbol	Value Description	Hex Value
AUTO_MUTE_ENABLE	FIFO_x_AUTO_MUTE_DISABLE	Disable the auto mute feature	0x0*
	FIFO_x_AUTO_MUTE_ENABLE	Enable the auto mute feature	0x1

Ezairo 8300 Hardware Reference

12.4 FIFO INTERRUPT CONFIGURATION

The Ezairo 8300 system allows each of the 32 FIFOs to be defined as the source for one or more of the 24 FIFO interrupts and events that can be used to trigger processing on the CFX DSP, the HEAR accelerator, or the Arm Cortex-M3 processor.

To configure a FIFO as the source for an interrupt or event, set the `D_FIFO_INT_CFX_CFG_*`, `D_FIFO_INT_HEAR_CFG_*`, or `D_FIFO_INT_CM3_CFG_*` registers using the `FIFO_INT_x` bit settings to indicate which FIFO is used to trigger which interrupts and events. For example, to trigger Arm Cortex-M3 processor FIFO interrupt 2 on FIFO A1.3, write `FIFO_INT_A1_3` to `D_FIFO_INT_CM3_CFG_2`.

12.4.1 Control and Configuration Registers

Register Name	Register Description	Address
<code>D_FIFO_INT_CFX_CFG</code> <code>FIFO_INT_CFX_CFG</code>	CFX FIFO interrupt selection	X:0x1200-X:0x1207 0x20804800-0x2080481C
<code>D_FIFO_INT_HEAR_CFG</code> <code>FIFO_INT_HEAR_CFG</code>	HEAR FIFO event selection	X:0x1210-X:0x1217 0x20804840-0x2080485C
<code>D_FIFO_INT_CM3_CFG</code> <code>FIFO_INT_CM3_CFG</code>	Arm Cortex-M3 core FIFO interrupt selection	X:0x1220-X:0x1227 0x20804880-0x2080489C

12.4.1.1 `D_FIFO_INT_CFX_CFG*`, `FIFO_INT_CFX_CFG_*`

Bit Field	Read/Write	Field Name	Description
4:0	RW	<code>SELECT_INT</code>	Select the source for a CFX FIFO interrupt

Field Name	Value Symbol	Value Description	Hex Value
<code>SELECT_INT</code>	<code>FIFO_INT_A0_0</code>	Select FIFO controller A0 interrupt 0 (interrupt 0 default)	0x0*
	<code>FIFO_INT_A0_1</code>	Select FIFO controller A0 interrupt 1 (interrupt 1 default)	0x1
	<code>FIFO_INT_A0_2</code>	Select FIFO controller A0 interrupt 2	0x2
	<code>FIFO_INT_A0_3</code>	Select FIFO controller A0 interrupt 3	0x3
	<code>FIFO_INT_A0_4</code>	Select FIFO controller A0 interrupt 4	0x4
	<code>FIFO_INT_A0_5</code>	Select FIFO controller A0 interrupt 5	0x5
	<code>FIFO_INT_A0_6</code>	Select FIFO controller A0 interrupt 6	0x6
	<code>FIFO_INT_A0_7</code>	Select FIFO controller A0 interrupt 7	0x7
	<code>FIFO_INT_A1_0</code>	Select FIFO controller A1 interrupt 0 (interrupt 2 default)	0x8

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	FIFO_INT_A1_1	Select FIFO controller A1 interrupt 1 (interrupt 3 default)	0x9
	FIFO_INT_A1_2	Select FIFO controller A1 interrupt 2	0xA
	FIFO_INT_A1_3	Select FIFO controller A1 interrupt 3	0xB
	FIFO_INT_A1_4	Select FIFO controller A1 interrupt 4	0xC
	FIFO_INT_A1_5	Select FIFO controller A1 interrupt 5	0xD
	FIFO_INT_A1_6	Select FIFO controller A1 interrupt 6	0xE
	FIFO_INT_A1_7	Select FIFO controller A1 interrupt 7	0xF
	FIFO_INT_B0_0	Select FIFO controller B0 interrupt 0 (interrupt 4 default)	0x10
	FIFO_INT_B0_1	Select FIFO controller B0 interrupt 1 (interrupt 5 default)	0x11
	FIFO_INT_B0_2	Select FIFO controller B0 interrupt 2	0x12
	FIFO_INT_B0_3	Select FIFO controller B0 interrupt 3	0x13
	FIFO_INT_B0_4	Select FIFO controller B0 interrupt 4	0x14
	FIFO_INT_B0_5	Select FIFO controller B0 interrupt 5	0x15
	FIFO_INT_B0_6	Select FIFO controller B0 interrupt 6	0x16
	FIFO_INT_B0_7	Select FIFO controller B0 interrupt 7	0x17
	FIFO_INT_B1_0	Select FIFO controller B1 interrupt 0 (interrupt 6 default)	0x18
	FIFO_INT_B1_1	Select FIFO controller B1 interrupt 1 (interrupt 7 default)	0x19
	FIFO_INT_B1_2	Select FIFO controller B1 interrupt 2	0x1A
	FIFO_INT_B1_3	Select FIFO controller B1 interrupt 3	0x1B
	FIFO_INT_B1_4	Select FIFO controller B1 interrupt 4	0x1C
	FIFO_INT_B1_5	Select FIFO controller B1 interrupt 5	0x1D
	FIFO_INT_B1_6	Select FIFO controller B1 interrupt 6	0x1E
	FIFO_INT_B1_7	Select FIFO controller B1 interrupt 7	0x1F

12.4.1.2 D_FIFO_INT_HEAR_CFG*, FIFO_INT_HEAR_CFG_*

Bit Field	Read/Write	Field Name	Description
4:0	RW	SELECT_INT	Select the source for a HEAR FIFO event

Ezairo 8300 Hardware Reference

12.4.1.3 D_FIFO_INT_CM3_CFG*, FIFO_INT_CM3_CFG_*

Bit Field	Read/Write	Field Name	Description
4:0	RW	SELECT_INT	Select the source for an Arm Cortex-M3 core FIFO interrupt

12.5 ACCESS PRIORITY

All access to A0, A1, B0 and B1 memories, as well as to the FIFO configuration registers, goes through the FIFO controller. To control which access has priority and to set the sequence in which the FIFOs receive accesses, the FIFO controller uses a fixed access priority scheme. The fixed access priority that is enforced by the FIFO controller is:

1. Direct read/write access by IOC0
2. Direct read/write access by IOC1
3. Read/write access using an access register (D_FIFO_XX_ACCESS) for a FIFO through the shared memory bus controller
4. Write access to a block counter register (D_FIFO_XX_BLOCK_CNT) for a FIFO through the shared memory bus controller
5. Read/write access to any remaining FIFO register through the shared memory bus controller
6. Read/write access to any area outside of a defined FIFO buffer through the shared memory bus controller (linear access)
7. Read/write access to an area within a FIFO buffer through the shared memory bus controller

CHAPTER 13

Input/Output Audio Control

The input/output controllers (IOCs) and audio multiplexers control the audio signal data input and output from the system. The IOCs can be directed to acquire input data from a combination of:

- The four input A/D channels belonging to the input stage
- The four digital microphone inputs
- The eight IOC dedicated Filter Engine inputs
- Four shared PCM/I²S input channels from the two PCM/I²S interfaces

The IOCs are also used to direct output data, sending the data to a combination of:

- The two output data channels belonging to the output stage
- The two digital microphone outputs
- The four shared PCM/I²S output channels from the two PCM/I²S interfaces
- The four IOC dedicated Filter Engine inputs

13.1 AUDIO MULTIPLEXING

Input/output audio data can be routed along a number of different paths using the multiplexing options available in the Ezairo 8300 system. The audio multiplexing controls the paths for audio data passed within the Ezairo 8300 system, including:

- The input stage, described in [Chapter 14.1 "Overview" on page 438](#)
- The output stage, described in [Chapter 15.1 "Overview" on page 456](#)
- The Filter Engine, described in [Chapter 6.1 "Processor Overview" on page 217](#)
- The Digital Microphone (DMIC) in puts and outputs, described in [Section 18.2 "Digital Microphone \(DMIC\) Inputs and Outputs" on page 563](#)

The audio multiplexing ensures that input data received on all of the data sources is available for processing by the CFX DSP, the Arm Cortex-M3 processor, the HEAR accelerator, and the LPDSP32. Similarly, it ensures that data processed from these cores is provided as output data to all the appropriate data sinks.

There are two input/output controllers (IOCs) within the Ezairo 8300 system, referred to as IOC0 and IOC1. IOC0 is used to route data from the input stage and Filter Engine, and to the output stage, Filter Engine, and FIFOs. Trigger sources for both IOCs are configurable for all signals provided to or received from the Filter Engine.

The "Audio Filtering and Multiplexing, Simplified" figure ([Figure 25](#)) provides a simplified outline of audio filtering and multiplexing in the Ezairo 8300 system, while the "Audio Filtering and Multiplexing, Full Diagram" figure ([Figure 26](#)) shows the more complex details of the process, including configuration register names. In the "Audio Filtering and Multiplexing, Simplified" figure ([Figure 25](#)), the lighter lines indicate control signals, whereas the darker lines represent signal path options.

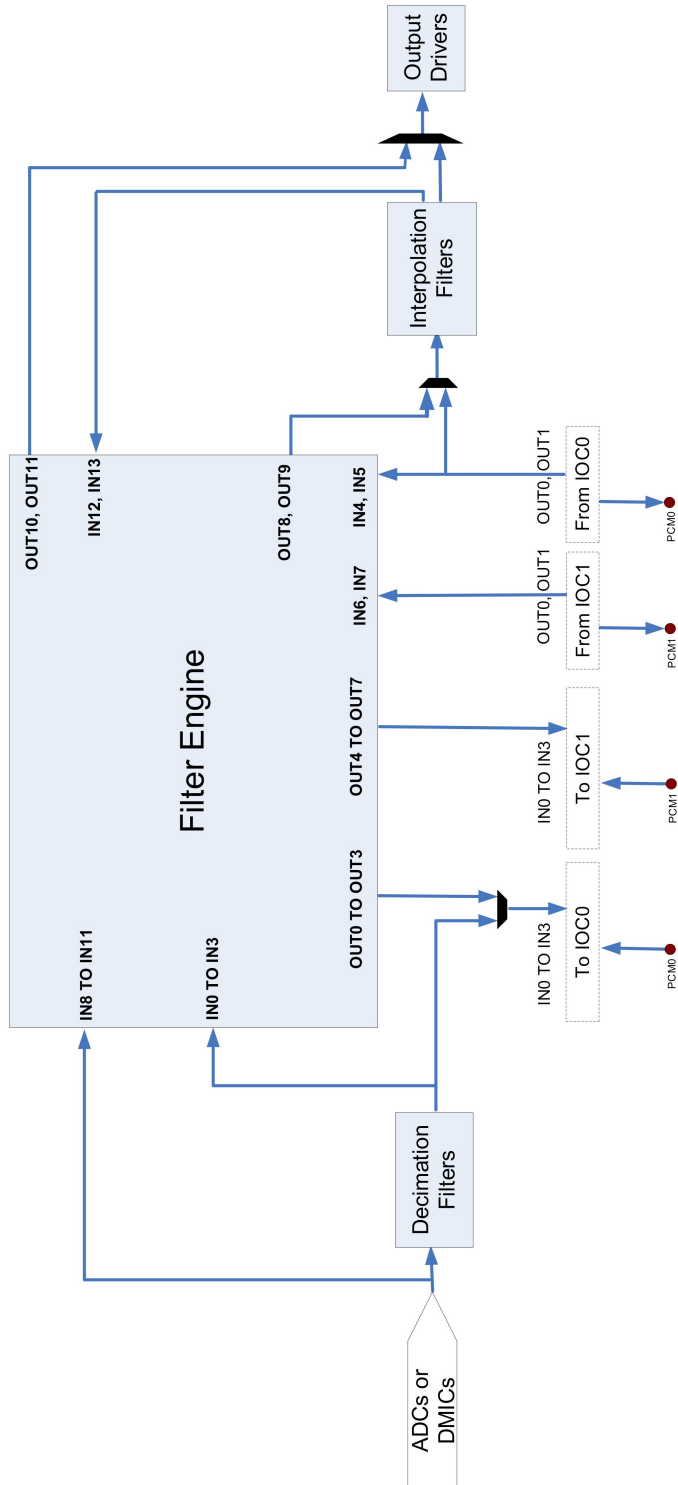


Figure 25. Audio Filtering and Multiplexing, Simplified

Ezairo 8300 Hardware Reference

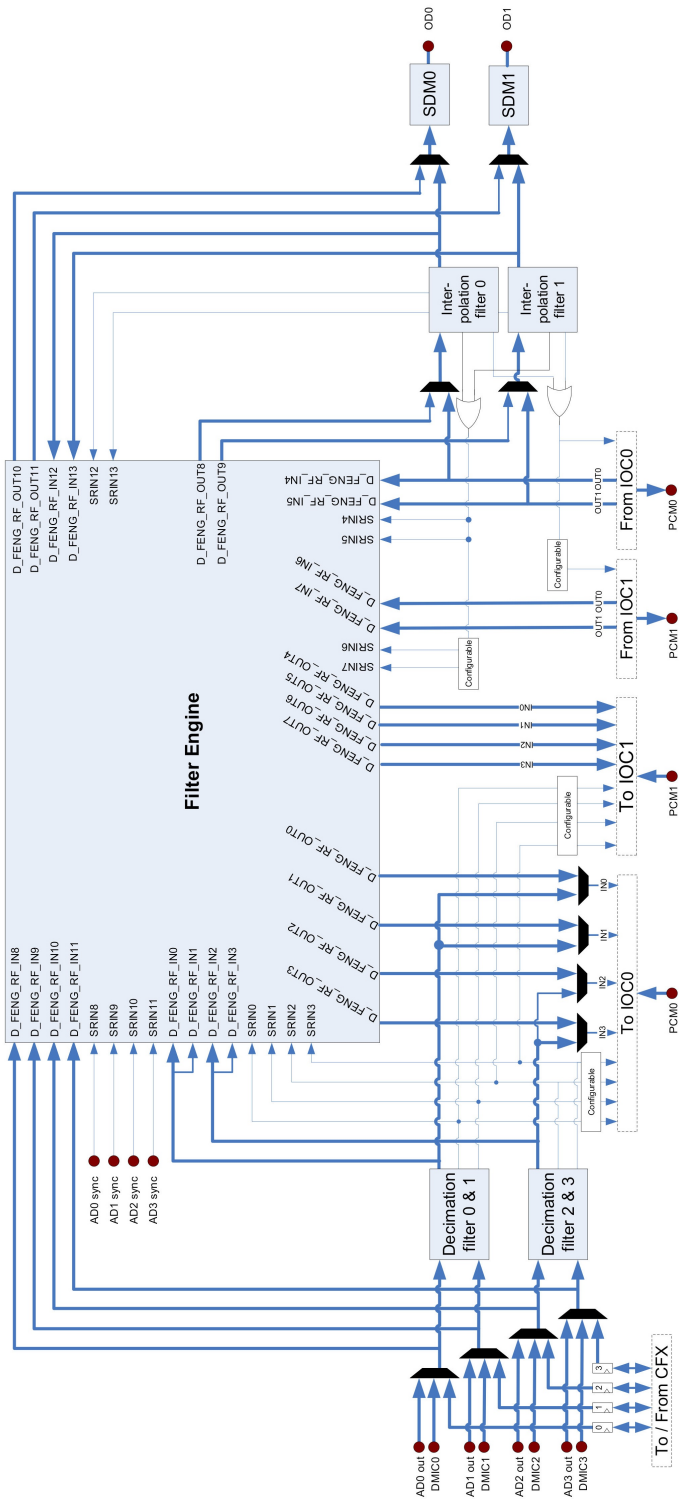


Figure 26. Audio Filtering and Multiplexing, Full Diagram

Ezairo 8300 Hardware Reference

13.1.1 Audio Multiplexing with Filter Engine Outputs

The Filter Engine is linked to the audio input and output stages to optionally pre-process the signals from the input stage and post-process the signals before they are provided to the output stage. The audio filtering and multiplexing block also provides a low group-delay processing path that can bypass or replace the input decimation filters and output interpolation filters. The "Audio Filtering and Multiplexing, Simplified" figure (Figure 25) shows how:

- The Filter Engine input and output signals are connected to the input and output stages.
- The audio signals that bypass the Filter Engine are multiplexed with Filter Engine pre-processed signals and post-processed signals.

The group-delay in the low group-delay processing path provided by the Filter Engine consists of delays introduced by:

- The ADC
- The Filter Engine processing itself, or shadow register delay
- The sigma-delta modulator
- The H-bridge
- Any filtering performed by the Filter Engine

The "Group-Delay of Low Delay Processing Paths" table (Table 41) below, shows the group-delay of the low delay processing paths:

Table 41. Group-Delay of Low Delay Processing Paths

Base SF [kHz]	Low-delay SF [kHz]	ADC delay [μ s]	OD delay [μ s]	Filter Engine shadow output register delay* [μ s]	Filter Engine filtering delay [μ s]
16	128	11.0	10.6	7.8	Application dependent
24	192	8.0	7.7	5.2	Application dependent
32	256	5.5	5.3	3.9	Application dependent
48	384	4.0	3.8	2.6	Application dependent

NOTE: The use of the Filter Engine shadow output registers is optional. When these are not used, then this delay is replaced with the actual Filter Engine processing time, which is dependent of the application.

To select the source that IOC0 uses for each input channel, set the AUDIO_MUX_CFG_IOC0_SRC* bit fields from the AUDIO_MUX_CFG register. Each source can be configured to use the output of the input stage's decimation filters, or Filter Engine outputs 0, 1, 2, 3 (for channels 0, 1, 2 and 3 respectively) from the Filter Engine. IOC1 provides input channels for outputs 4, 5, 6 and 7 from the Filter Engine.

To select the source for the interpolation filter inputs between IOC0 and Filter Engine outputs 8 or 9, set the AUDIO_MUX_CFG_INT_FILTER0 and AUDIO_MUX_CFG_INT_FILTER1 bit fields from the AUDIO_MUX_CFG register.

Ezairo 8300 Hardware Reference

To select the source for the sigma-delta modulators between the interpolation filter outputs and Filter Engine outputs 0 or 1, set the `AUDIO_MUX_CFG_SDM0` and `AUDIO_MUX_CFG_SDM1` bit fields from the `AUDIO_MUX_CFG` register.

For more information, see [Section 6.3 “Filter Engine in the Audio Path”](#) on page 219.

13.1.2 Audio Multiplexing of Audio Input Sources

Audio input data for each of the input channels can be configured to come from one of the following sources:

- The analog inputs and ADCs of the input stages as described in [Chapter 14.1 "Overview" on page 438](#)
- The digital microphone inputs, described in [Section 18.2 “Digital Microphone \(DMIC\) Inputs and Outputs” on page 563](#). Each input channel can be configured to be triggered from the rising or falling edge of the specified DMIC input.
- System controlled data registers bypass registers. The core provides this input data through the `D_IOC*_IN*_DATA` registers. These bypass registers can be used to form a feedback path, or to provide debug data at the inputs to the decimation filter and Filter Engine.

The data from the selected sources is used as the input to the input stage's decimation filters for each channel; the Filter Engine has access to the selected sources directly as inputs 8, 9, 10 and 11. To select the source for each channel, set the `AUDIO_MUX_CFG_INPUT_CH*_SRC` bit fields from the `AUDIO_MUX_CFG` register.

The `AUDIO_IOC_FENG_CFG` register controls what triggers IOC0 when the inputs to the IOC come from the Filter Engine. It also controls what triggers IOC1 and Filter Engine inputs 6 and 7.

NOTE: IOC1 requires use of the Filter Engine (if not using PCM interface 1), whereas IOC0 can be used without the Filter Engine.

13.1.3 Control and Configuration Registers

Register Name	Register Description	Address
<code>AUDIO_MUX_CFG</code>	Configuration register for the audio filtering and multiplexing block	P:0x09D4 0x40002750
<code>AUDIO_BYPASS_DATA</code>	Data registers for the input channel	P:0x09D8- P:0x09DB 0x40002760- 0x4000276C
<code>AUDIO_IOC_FENG_CFG</code>	IOC FENG trigger source configuration	P:0x09F0 0x400027C0

Ezairo 8300 Hardware Reference

13.1.3.1 AUDIO_MUX_CFG

Bit Field	Read/Write	Field Name	Description
15:14	RW	INPUT_CH3_SRC	Selects the source of the audio input channel 3
13:12	RW	INPUT_CH2_SRC	Selects the source of the audio input channel 2
11:10	RW	INPUT_CH1_SRC	Selects the source of the audio input channel 1
9:8	RW	INPUT_CH0_SRC	Selects the source of the audio input channel 0
7	RW	IOC0_SRC3	Configures the IOC0 audio input channel 3
6	RW	IOC0_SRC2	Configures the IOC0 audio input channel 2
5	RW	IOC0_SRC1	Configures the IOC0 audio input channel 1
4	RW	IOC0_SRC0	Configures the IOC0 audio input channel 0
3	RW	INT_FILTER1	Configures the interpolation filter input channel 1
2	RW	INT_FILTER0	Configures the interpolation filter input channel 0
1	RW	SDM1	Configures the Sigma-Delta input channel 1
0	RW	SDM0	Configures the Sigma-Delta input channel 0

Field Name	Value Symbol	Value Description	Hex Value
INPUT_CH3_SRC	ADC3_OUT	Data is provided by the ADC	0x0*
	BYPASS_DATA_REG3	Data is provided by the bypass register	0x1
	DMIC3_DATA_RE	Data is provided from DMIC (rising-edge)	0x2
	DMIC3_DATA_FE	Data is provided from DMIC (falling-edge)	0x3
INPUT_CH2_SRC	ADC2_OUT	Data is provided by the ADC	0x0*
	BYPASS_DATA_REG2	Data is provided by the bypass register	0x1
	DMIC2_DATA_RE	Data is provided from DMIC (rising-edge)	0x2
	DMIC2_DATA_FE	Data is provided from DMIC (falling-edge)	0x3
INPUT_CH1_SRC	ADC1_OUT	Data is provided by the ADC	0x0*
	BYPASS_DATA_REG1	Data is provided by the bypass register	0x1
	DMIC1_DATA_RE	Data is provided from DMIC (rising-edge)	0x2
	DMIC1_DATA_FE	Data is provided from DMIC (falling-edge)	0x3
INPUT_CH0_SRC	ADC0_OUT	Data is provided by the ADC	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	BYPASS_DATA_REG0	Data is provided by the bypass register	0x1
	DMIC0_DATA_RE	Data is provided from DMIC (rising-edge)	0x2
	DMIC0_DATA_FE	Data is provided from DMIC (falling-edge)	0x3
IOC0_SRC3	IOC0_SRC3_DEC_FILTER	Decimation filter 3 output	0x0*
	IOC0_SRC3_FENG	FENG output f_out3	0x1
IOC0_SRC2	IOC0_SRC2_DEC_FILTER	Decimation filter 2 output	0x0*
	IOC0_SRC2_FENG	FENG output f_out2	0x1
IOC0_SRC1	IOC0_SRC1_DEC_FILTER	Decimation filter 1 output	0x0*
	IOC0_SRC1_FENG	FENG output f_out1	0x1
IOC0_SRC0	IOC0_SRC0_DEC_FILTER	Decimation filter 0 output	0x0*
	IOC0_SRC0_FENG	FENG output f_out0	0x1
INT_FILTER1	INT_FILTER1_IOC0	IOC0 audio output 1	0x0*
	INT_FILTER1_FENG	FENG output f_out9	0x1
INT_FILTER0	INT_FILTER0_IOC0	IOC0 audio output 0	0x0*
	INT_FILTER0_FENG	FENG output f_out8	0x1
SDM1	SDM1_INT_FILTER	Interpolation filter 1 output	0x0*
	SDM1_FENG	FENG output f_out11	0x1
SDM0	SDM0_INT_FILTER	Interpolation filter 0 output	0x0*
	SDM0_FENG	FENG output f_out10	0x1

13.1.3.2 AUDIO_BYPASS_DATA*

Bit Field	Read/Write	Field Name	Description
23:0	RW	BYPASS_DATA	24-bit signed data for the input data bypass (sign extended to 32 bit)

13.1.3.3 AUDIO_IOC_FENG_CFG

Bit Field	Read/Write	Field Name	Description
26:24	RW	IOC1_FENG_IN6_IN7_TRIG	IOC1 FENG IN6 IN7 trigger source
23:21	RW	IOC1_FENG_OUT7_TRIG	IOC1 FENG OUT7 trigger source
20:18	RW	IOC1_FENG_OUT6_TRIG	IOC1 FENG OUT6 trigger source
17:15	RW	IOC1_FENG_OUT5_TRIG	IOC1 FENG OUT5 trigger source

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
14:12	RW	IOC1_FENG_OUT4_TRIG	IOC1 FENG OUT4 trigger source
11:9	RW	IOC0_FENG_OUT3_TRIG	IOC0 FENG OUT3 trigger source
8:6	RW	IOC0_FENG_OUT2_TRIG	IOC0 FENG OUT2 trigger source
5:3	RW	IOC0_FENG_OUT1_TRIG	IOC0 FENG OUT1 trigger source
2:0	RW	IOC0_FENG_OUT0_TRIG	IOC0 FENG OUT0 trigger source

Field Name	Value Symbol	Value Description	Hex Value
IOC1_FENG_IN6_IN7_TRIG	IOC1_FENG_IN6_IN7_DISABLED	IOC1 FENG IN6 IN7 trigger disabled	0x0*
	IOC1_FENG_IN6_IN7_INT_FILTER_LDP	IOC1 FENG IN6 IN7 trigger on interpolation filter 0 or 1 Low Delay Path	0x1
	IOC1_FENG_IN6_IN7_INT_FILTER	IOC1 FENG IN6 IN7 trigger on interpolation filter 0 or 1	0x2
	IOC1_FENG_IN6_IN7_ADC0_LDP	IOC1 FENG IN6 IN7 trigger on ADC0 Low Delay Path	0x4
	IOC1_FENG_IN6_IN7_ADC1_LDP	IOC1 FENG IN6 IN7 trigger on ADC1 Low Delay Path	0x5
	IOC1_FENG_IN6_IN7_ADC2_LDP	IOC1 FENG IN6 IN7 trigger on ADC2 Low Delay Path	0x6
	IOC1_FENG_IN6_IN7_ADC3_LDP	IOC1 FENG IN6 IN7 trigger on ADC3 Low Delay Path	0x7
IOC1_FENG_OUT7_TRIG	IOC1_FENG_OUT7_DISABLED	IOC1 FENG OUT7 trigger disabled	0x0*
	IOC1_FENG_OUT7_INT_FILTER_LDP	IOC1 FENG OUT7 triggers on interpolation filter 0 or 1 Low Delay Path	0x1
	IOC1_FENG_OUT7_ADC0_ADC2	IOC1 FENG OUT7 triggers on ADC0 or ADC2	0x2
	IOC1_FENG_OUT7_ADC1_ADC3	IOC1 FENG OUT7 triggers on ADC1 or ADC3	0x3
	IOC1_FENG_OUT7_ADC0_LDP	IOC1 FENG OUT7 triggers on ADC0 Low Delay Path	0x4
	IOC1_FENG_OUT7_ADC1_LDP	IOC1 FENG OUT7 triggers on ADC1 Low Delay Path	0x5
	IOC1_FENG_OUT7_ADC2_LDP	IOC1 FENG OUT7 triggers on ADC2 Low Delay Path	0x6
	IOC1_FENG_OUT7_ADC3_LDP	IOC1 FENG OUT7 triggers on ADC3 Low Delay Path	0x7

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
IOC1_FENG_OUT6_TRIG	IOC1_FENG_OUT6_DISABLED	IOC1 FENG OUT6 trigger disabled	0x0*
	IOC1_FENG_OUT6_INT_FILTER_LDP	IOC1 FENG OUT6 triggers on interpolation filter 0 or 1 Low Delay Path	0x1
	IOC1_FENG_OUT6_ADC0_ADC2	IOC1 FENG OUT6 triggers on ADC0 or ADC2	0x2
	IOC1_FENG_OUT6_ADC1_ADC3	IOC1 FENG OUT6 triggers on ADC1 or ADC3	0x3
	IOC1_FENG_OUT6_ADC0_LDP	IOC1 FENG OUT6 triggers on ADC0 Low Delay Path	0x4
	IOC1_FENG_OUT6_ADC1_LDP	IOC1 FENG OUT6 triggers on ADC1 Low Delay Path	0x5
	IOC1_FENG_OUT6_ADC2_LDP	IOC1 FENG OUT6 triggers on ADC2 Low Delay Path	0x6
	IOC1_FENG_OUT6_ADC3_LDP	IOC1 FENG OUT6 triggers on ADC3 Low Delay Path	0x7
IOC1_FENG_OUT5_TRIG	IOC1_FENG_OUT5_DISABLED	IOC1 FENG OUT5 trigger disabled	0x0*
	IOC1_FENG_OUT5_INT_FILTER_LDP	IOC1 FENG OUT5 triggers on interpolation filter 0 or 1 Low Delay Path	0x1
	IOC1_FENG_OUT5_ADC0_ADC2	IOC1 FENG OUT5 triggers on ADC0 or ADC2	0x2
	IOC1_FENG_OUT5_ADC1_ADC3	IOC1 FENG OUT5 triggers on ADC1 or ADC3	0x3
	IOC1_FENG_OUT5_ADC0_LDP	IOC1 FENG OUT5 triggers on ADC0 Low Delay Path	0x4
	IOC1_FENG_OUT5_ADC1_LDP	IOC1 FENG OUT5 triggers on ADC1 Low Delay Path	0x5
	IOC1_FENG_OUT5_ADC2_LDP	IOC1 FENG OUT5 triggers on ADC2 Low Delay Path	0x6
	IOC1_FENG_OUT5_ADC3_LDP	IOC1 FENG OUT5 triggers on ADC3 Low Delay Path	0x7
IOC1_FENG_OUT4_TRIG	IOC1_FENG_OUT4_DISABLED	IOC1 FENG OUT4 trigger disabled	0x0*
	IOC1_FENG_OUT4_INT_FILTER_LDP	IOC1 FENG OUT4 triggers on interpolation filter 0 or 1 Low Delay Path	0x1
	IOC1_FENG_OUT4_ADC0_ADC2	IOC1 FENG OUT4 triggers on ADC0 or ADC2	0x2
	IOC1_FENG_OUT4_ADC1_ADC3	IOC1 FENG OUT4 triggers on ADC1 or ADC3	0x3

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	IOC1_FENG_OUT4_ADC0_LDP	IOC1 FENG OUT4 triggers on ADC0 Low Delay Path	0x4
	IOC1_FENG_OUT4_ADC1_LDP	IOC1 FENG OUT4 triggers on ADC1 Low Delay Path	0x5
	IOC1_FENG_OUT4_ADC2_LDP	IOC1 FENG OUT4 triggers on ADC2 Low Delay Path	0x6
	IOC1_FENG_OUT4_ADC3_LDP	IOC1 FENG OUT4 triggers on ADC3 Low Delay Path	0x7
IOC0_FENG_OUT3_TRIG	IOC0_FENG_OUT3_DISABLED	IOC0 FENG OUT3 trigger disabled	0x0
	IOC0_FENG_OUT3_INT_FILTER_LDP	IOC0 FENG OUT3 triggers on interpolation filter 0 or 1 Low Delay Path	0x1
	IOC0_FENG_OUT3_ADC0_ADC2	IOC0 FENG OUT3 triggers on ADC0 or ADC2	0x2
	IOC0_FENG_OUT3_ADC1_ADC3	IOC0 FENG OUT3 triggers on ADC1 or ADC3	0x3*
	IOC0_FENG_OUT3_ADC0_LDP	IOC0 FENG OUT3 triggers on ADC0 Low Delay Path	0x4
	IOC0_FENG_OUT3_ADC1_LDP	IOC0 FENG OUT3 triggers on ADC1 Low Delay Path	0x5
	IOC0_FENG_OUT3_ADC2_LDP	IOC0 FENG OUT3 triggers on ADC2 Low Delay Path	0x6
	IOC0_FENG_OUT3_ADC3_LDP	IOC0 FENG OUT3 triggers on ADC3 Low Delay Path	0x7
IOC0_FENG_OUT2_TRIG	IOC0_FENG_OUT2_DISABLED	IOC0 FENG OUT2 trigger disabled	0x0
	IOC0_FENG_OUT2_INT_FILTER_LDP	IOC0 FENG OUT2 triggers on interpolation filter 0 or 1 Low Delay Path	0x1
	IOC0_FENG_OUT2_ADC0_ADC2	IOC0 FENG OUT2 triggers on ADC0 or ADC2	0x2*
	IOC0_FENG_OUT2_ADC1_ADC3	IOC0 FENG OUT2 triggers on ADC1 or ADC3	0x3
	IOC0_FENG_OUT2_ADC0_LDP	IOC0 FENG OUT2 triggers on ADC0 Low Delay Path	0x4
	IOC0_FENG_OUT2_ADC1_LDP	IOC0 FENG OUT2 triggers on ADC1 Low Delay Path	0x5
	IOC0_FENG_OUT2_ADC2_LDP	IOC0 FENG OUT2 triggers on ADC2 Low Delay Path	0x6
	IOC0_FENG_OUT2_ADC3_LDP	IOC0 FENG OUT2 triggers on ADC3	0x7

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		Low Delay Path	
IOC0_FENG_OUT1_TRIG	IOC0_FENG_OUT1_DISABLED	IOC0 FENG OUT1 trigger disabled	0x0
	IOC0_FENG_OUT1_INT_FILTER_LDP	IOC0 FENG OUT1 triggers on interpolation filter 0 or 1 Low Delay Path	0x1
	IOC0_FENG_OUT1_ADC0_ADC2	IOC0 FENG OUT1 triggers on ADC0 or ADC2	0x2
	IOC0_FENG_OUT1_ADC1_ADC3	IOC0 FENG OUT1 triggers on ADC1 or ADC3	0x3*
	IOC0_FENG_OUT1_ADC0_LDP	IOC0 FENG OUT1 triggers on ADC0 Low Delay Path	0x4
	IOC0_FENG_OUT1_ADC1_LDP	IOC0 FENG OUT1 triggers on ADC1 Low Delay Path	0x5
	IOC0_FENG_OUT1_ADC2_LDP	IOC0 FENG OUT1 triggers on ADC2 Low Delay Path	0x6
	IOC0_FENG_OUT1_ADC3_LDP	IOC0 FENG OUT1 triggers on ADC3 Low Delay Path	0x7
IOC0_FENG_OUT0_TRIG	IOC0_FENG_OUT0_DISABLED	IOC0 FENG OUT0 trigger disabled	0x0
	IOC0_FENG_OUT0_INT_FILTER_LDP	IOC0 FENG OUT0 triggers on interpolation filter 0 or 1 Low Delay Path	0x1
	IOC0_FENG_OUT0_ADC0_ADC2	IOC0 FENG OUT0 triggers on ADC0 or ADC2	0x2*
	IOC0_FENG_OUT0_ADC1_ADC3	IOC0 FENG OUT0 triggers on ADC1 or ADC3	0x3
	IOC0_FENG_OUT0_ADC0_LDP	IOC0 FENG OUT0 triggers on ADC0 Low Delay Path	0x4
	IOC0_FENG_OUT0_ADC1_LDP	IOC0 FENG OUT0 triggers on ADC1 Low Delay Path	0x5
	IOC0_FENG_OUT0_ADC2_LDP	IOC0 FENG OUT0 triggers on ADC2 Low Delay Path	0x6
	IOC0_FENG_OUT0_ADC3_LDP	IOC0 FENG OUT0 triggers on ADC3 Low Delay Path	0x7

13.2 INTRODUCTION TO THE INPUT/OUTPUT CONTROLLERS

The two instances of the input/output controller (IOC0 and IOC1) move signal data from the Ezairo 8300 system's input sources into system memory to be processed, and out of system memory to the system's output sinks. To do this, the IOCs are connected appropriately between each of the input sources and PCM interfaces (supported by a number of audio multiplexers as described in [Section 13.1 “Audio Multiplexing”](#)), and enjoy privileged access to the FIFOs through the FIFO controller. The IOCs use the FIFOs to:

Ezairo 8300 Hardware Reference

- Provide data for processing by the CFX DSP, the Arm Cortex-M3 processor, the HEAR Accelerator, and the LPDSP32 DSP.
- Provide source data to the system outputs and the Filter Engine
- Make data available to the LPDSP32 through the DMA

IOC0 interacts directly with each of the following components (as illustrated in the "Components Interfacing with an IOC" figure (Figure 27)):

- The input stage, where inputs come from the ADCs or the digital microphone inputs
- The output stage, where the outputs go to the DACs or the digital microphone outputs
- The PCM interface 0
- The FIFOs and the FIFO controller
- The Filter Engine

As a result, these components must be referenced when discussing the IOCs. For more details about any of these other system components, refer to their respective chapters in this document.

IOC1 has a similar interface, but there are no direct connections with the input stage or output stage. It only receives signals from, and sends signals to, the following components.

- The PCM interface 1
- The FIFOs and the FIFO controller
- The Filter Engine

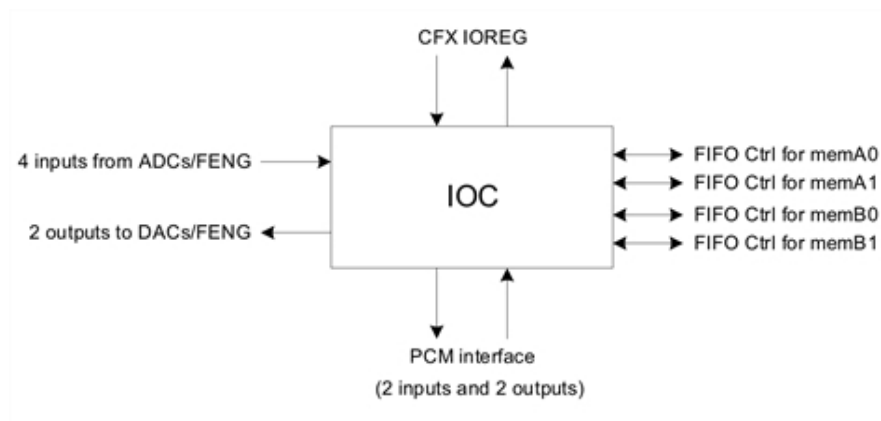


Figure 27. Components Interfacing with an IOC

NOTE: Writing to any of an IOC's configuration registers, even if no register settings are updated, resets the internal state machines of the IOC. This forces the IOC to resynchronize to the input sources from which it is reading data—and to the output sinks to which it is providing data. The IOC does not read input samples nor provide output samples until the next time it reads from IN0 (even if IN0 is disabled) and is resynchronized.

IMPORTANT: To synchronize all of the input and output signals, the IOCs operate at the sampling frequency determined by the ADC sample frequency configuration.

Ezairo 8300 Hardware Reference

13.3 IOC CONFIGURATION

Data being written or read by an IOC from a single FIFO can be accessed either one word at a time (single access mode), or two words at a time (double access mode). Using double access mode reduces the number of times an IOC accesses the FIFOs, which reduces the possibility of FIFO access conflicts with the CFX, HEAR and Arm Cortex-M3 cores.

If one input source or one output sink uses a FIFO, the `IOC_FIFO_ACCESS_CFG_FIFO_x_DBL_ACC` bit in the `D_IOC*_FIFO_ACCESS_CFG` register configures whether the IOC uses single or double access mode.

If more than one input source or output sink uses a FIFO, the `IOC_FIFO_ACCESS_CFG_FIFO_x_DBL_ACC` bit must be cleared. In this configuration, the IOC always uses single access mode for an odd number of sources or sinks, and the IOC always uses double access mode for an even number of sources or sinks.

NOTE: Writing to the `D_IOC*_INPUT_CFG`, `D_IOC*_INPUT_PCM_CFG` or the `D_IOC*_FIFO_ACCESS_CFG` register resets the input state machine of the IOCs.

13.4 IOC INPUT

On the input side, IOC0 is attached to the four input channels selected from the input stage and Filter Engine, denoted by IN0, IN1, IN2 and IN3 in constants and documentation. IOC1 is attached to four input channels from the Filter Engine and uses the same designations of IN0, IN1, IN2 and IN3 for its inputs.

NOTE: Data provided to the IOCs by the Filter Engine is treated as 24-bit data. As a result, if an input channel is configured to use the Filter Engine as its input source for a channel, this data is used directly by the IOCs.

For each of these input channels, there is an access register (`IOC_IN*_DATA`) for each IOC that can be used to supply the most recent data transferred for that input channel.

In addition to the input channels, each IOC can be attached to a maximum of two channels of input from its PCM interface. IOC0 can interface with PCM0, and IOC1 can interface with PCM1. The two receive channels for each PCM interface are denoted by RX0 and RX1 in constants and documentation, and the channels are nominally considered to be providing 24-bit input data. The PCM interface configuration can be used to align data that is shorter than 24 bits. Likewise, the two transmit channels for each of the two PCM interfaces are denoted by TX0 and TX1.

An IOC, when configured to do so, can direct each of the six input sources into individual FIFOs that have been configured. You can select which FIFO each channel is connected to, by writing the desired configuration to the `IOC_INPUT_CFG_XXX` fields in the `D_IOC*_INPUT_CFG` and `D_IOC*_INPUT_PCM_CFG` registers. Each of these bit fields allows you to select from one of the 32 FIFOs, or to not store the input at all.

IMPORTANT: Connecting IOC input sources to FIFOs that have not been configured results in undefined behavior.

Data from more than one input channel can be stored in the same FIFO. In a case where more than one channel of the input stage is directed into a FIFO, the data received is guaranteed to be written to the FIFO in a time sequence ordered ADC0, ADC2, ADC1 and ADC3. Similarly, data from more than one PCM input channel can be stored in the same FIFO. The data received from the PCM interface is guaranteed to be written to the FIFO in a sequence ordered PCM 0/1 RX0/1. For details about how input FIFO data is written to memory, see [Section 12.3.1 “FIFO Input and Output Block”](#) on page 391.

Ezairo 8300 Hardware Reference

IMPORTANT: The Ezairo 8300 system does not support inputs from both the input channels and the PCM interface being stored in the same FIFO. Because of this, users need to always store data from the input channels and the PCM interface to different FIFOs.

In cases of concurrent ADC/FENG and PCM accesses, the following order of priority applies:

- IN0, IN2, IN1, IN3
- OUT0, OUT1
- PCM0 RX
- PCM1 RX
- PCM0 and PCM1 TX

IMPORTANT: Precise data synchronization between the inputs/outputs and the PCM interface is not possible; but if necessary, approximate data synchronization can be attempted if these interfaces are using synchronous clock sources. Therefore, when clocking the PCM interface externally, the ADCs, SDMs and system need to be clocked externally by this same clock source. For all other configurations, processing of ADC and PCM input data needs to be triggered on separate FIFO interrupts or events.

13.4.1 Control and Configuration Registers

Register Name	Register Description	Address
D_IOC0_INPUT_CFG IOC0_INPUT_CFG	Configuration registers for IOC inputs	P:0x0900 0x40002400
D_IOC0_INPUT_PCM_CFG IOC0_INPUT_PCM_CFG	Configuration registers for IOC PCM inputs	P:0x0901 0x40002404
D_IOC0_FIFO_ACCESS_CFG IOC0_FIFO_ACCESS_CFG	Configuration registers for IOC FIFO access types	P:0x0902 0x40002408
D_IOC0_IN0_DATA IOC0_IN0_DATA	Audio sample from IN0	P:0x0910 0x40002440
D_IOC0_IN1_DATA IOC0_IN1_DATA	Audio sample from IN1	P:0x0911 0x40002444
D_IOC0_IN2_DATA IOC0_IN2_DATA	Audio sample from IN2	P:0x0912 0x40002448
D_IOC0_IN3_DATA IOC0_IN3_DATA	Audio sample from IN3	P:0x0913 0x4000244C
D_IOC1_INPUT_CFG IOC1_INPUT_CFG	Configuration registers for IOC inputs	P:0x0940 0x40002500

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
D_IOC1_INPUT_PCM_CFG IOC1_INPUT_PCM_CFG	Configuration registers for IOC PCM inputs	P:0x0941
		0x40002504
D_IOC1_FIFO_ACCESS_CFG IOC1_FIFO_ACCESS_CFG	Configuration registers for IOC FIFO access types	P:0x0942
		0x40002508
D_IOC1_IN0_DATA IOC1_IN0_DATA	Audio sample from IN0	P:0x0950
		0x40002540
D_IOC1_IN1_DATA IOC1_IN1_DATA	Audio sample from IN1	P:0x0951
		0x40002544
D_IOC1_IN2_DATA IOC1_IN2_DATA	Audio sample from IN2	P:0x0952
		0x40002548
D_IOC1_IN3_DATA IOC1_IN3_DATA	Audio sample from IN3	P:0x0953
		0x4000254C

13.4.1.1 D_IOC*_INPUT_CFG, IOC*_INPUT_CFG

Bit Field	Read/Write	Field Name	Description
29:24	RW	IN3_STORE	Select where to store IN3
21:16	RW	IN2_STORE	Select where to store IN2
13:8	RW	IN1_STORE	Select where to store IN1
5:0	RW	IN0_STORE	Select where to store IN0

Field Name	Value Symbol	Value Description	Hex Value
IN3_STORE	IOC_INPUT_CFG_IN3_FA0_0	Store IN3 in FIFO A0 0	0x0
	IOC_INPUT_CFG_IN3_FA0_1	Store IN3 in FIFO A0 1	0x1
	IOC_INPUT_CFG_IN3_FA0_2	Store IN3 in FIFO A0 2	0x2
	IOC_INPUT_CFG_IN3_FA0_3	Store IN3 in FIFO A0 3	0x3
	IOC_INPUT_CFG_IN3_FA0_4	Store IN3 in FIFO A0 4	0x4
	IOC_INPUT_CFG_IN3_FA0_5	Store IN3 in FIFO A0 5	0x5
	IOC_INPUT_CFG_IN3_FA0_6	Store IN3 in FIFO A0 6	0x6
	IOC_INPUT_CFG_IN3_FA0_7	Store IN3 in FIFO A0 7	0x7
	IOC_INPUT_CFG_IN3_FA1_0	Store IN3 in FIFO A1 0	0x8

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	IOC_INPUT_CFG_IN3_FA1_1	Store IN3 in FIFO A1 1	0x9
	IOC_INPUT_CFG_IN3_FA1_2	Store IN3 in FIFO A1 2	0xA
	IOC_INPUT_CFG_IN3_FA1_3	Store IN3 in FIFO A1 3	0xB
	IOC_INPUT_CFG_IN3_FA1_4	Store IN3 in FIFO A1 4	0xC
	IOC_INPUT_CFG_IN3_FA1_5	Store IN3 in FIFO A1 5	0xD
	IOC_INPUT_CFG_IN3_FA1_6	Store IN3 in FIFO A1 6	0xE
	IOC_INPUT_CFG_IN3_FA1_7	Store IN3 in FIFO A1 7	0xF
	IOC_INPUT_CFG_IN3_FB0_0	Store IN3 in FIFO B0 0	0x10
	IOC_INPUT_CFG_IN3_FB0_1	Store IN3 in FIFO B0 1	0x11
	IOC_INPUT_CFG_IN3_FB0_2	Store IN3 in FIFO B0 2	0x12
	IOC_INPUT_CFG_IN3_FB0_3	Store IN3 in FIFO B0 3	0x13
	IOC_INPUT_CFG_IN3_FB0_4	Store IN3 in FIFO B0 4	0x14
	IOC_INPUT_CFG_IN3_FB0_5	Store IN3 in FIFO B0 5	0x15
	IOC_INPUT_CFG_IN3_FB0_6	Store IN3 in FIFO B0 6	0x16
	IOC_INPUT_CFG_IN3_FB0_7	Store IN3 in FIFO B0 7	0x17
	IOC_INPUT_CFG_IN3_FB1_0	Store IN3 in FIFO B1 0	0x18
	IOC_INPUT_CFG_IN3_FB1_1	Store IN3 in FIFO B1 1	0x19
	IOC_INPUT_CFG_IN3_FB1_2	Store IN3 in FIFO B1 2	0x1A
	IOC_INPUT_CFG_IN3_FB1_3	Store IN3 in FIFO B1 3	0x1B
	IOC_INPUT_CFG_IN3_FB1_4	Store IN3 in FIFO B1 4	0x1C
	IOC_INPUT_CFG_IN3_FB1_5	Store IN3 in FIFO B1 5	0x1D
	IOC_INPUT_CFG_IN3_FB1_6	Store IN3 in FIFO B1 6	0x1E
	IOC_INPUT_CFG_IN3_FB1_7	Store IN3 in FIFO B1 7	0x1F
	IOC_INPUT_CFG_IN3_NONE	Not stored	0x3F*
IN2_STORE	IOC_INPUT_CFG_IN2_FA0_0	Store IN2 in FIFO A0 0	0x0
	IOC_INPUT_CFG_IN2_FA0_1	Store IN2 in FIFO A0 1	0x1
	IOC_INPUT_CFG_IN2_FA0_2	Store IN2 in FIFO A0 2	0x2
	IOC_INPUT_CFG_IN2_FA0_3	Store IN2 in FIFO A0 3	0x3
	IOC_INPUT_CFG_IN2_FA0_4	Store IN2 in FIFO A0 4	0x4
	IOC_INPUT_CFG_IN2_FA0_5	Store IN2 in FIFO A0 5	0x5
	IOC_INPUT_CFG_IN2_FA0_6	Store IN2 in FIFO A0 6	0x6

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	IOC_INPUT_CFG_IN2_FA0_7	Store IN2 in FIFO A0 7	0x7
	IOC_INPUT_CFG_IN2_FA1_0	Store IN2 in FIFO A1 0	0x8
	IOC_INPUT_CFG_IN2_FA1_1	Store IN2 in FIFO A1 1	0x9
	IOC_INPUT_CFG_IN2_FA1_2	Store IN2 in FIFO A1 2	0xA
	IOC_INPUT_CFG_IN2_FA1_3	Store IN2 in FIFO A1 3	0xB
	IOC_INPUT_CFG_IN2_FA1_4	Store IN2 in FIFO A1 4	0xC
	IOC_INPUT_CFG_IN2_FA1_5	Store IN2 in FIFO A1 5	0xD
	IOC_INPUT_CFG_IN2_FA1_6	Store IN2 in FIFO A1 6	0xE
	IOC_INPUT_CFG_IN2_FA1_7	Store IN2 in FIFO A1 7	0xF
	IOC_INPUT_CFG_IN2_FB0_0	Store IN2 in FIFO B0 0	0x10
	IOC_INPUT_CFG_IN2_FB0_1	Store IN2 in FIFO B0 1	0x11
	IOC_INPUT_CFG_IN2_FB0_2	Store IN2 in FIFO B0 2	0x12
	IOC_INPUT_CFG_IN2_FB0_3	Store IN2 in FIFO B0 3	0x13
	IOC_INPUT_CFG_IN2_FB0_4	Store IN2 in FIFO B0 4	0x14
	IOC_INPUT_CFG_IN2_FB0_5	Store IN2 in FIFO B0 5	0x15
	IOC_INPUT_CFG_IN2_FB0_6	Store IN2 in FIFO B0 6	0x16
	IOC_INPUT_CFG_IN2_FB0_7	Store IN2 in FIFO B0 7	0x17
	IOC_INPUT_CFG_IN2_FB1_0	Store IN2 in FIFO B1 0	0x18
	IOC_INPUT_CFG_IN2_FB1_1	Store IN2 in FIFO B1 1	0x19
	IOC_INPUT_CFG_IN2_FB1_2	Store IN2 in FIFO B1 2	0x1A
	IOC_INPUT_CFG_IN2_FB1_3	Store IN2 in FIFO B1 3	0x1B
	IOC_INPUT_CFG_IN2_FB1_4	Store IN2 in FIFO B1 4	0x1C
	IOC_INPUT_CFG_IN2_FB1_5	Store IN2 in FIFO B1 5	0x1D
	IOC_INPUT_CFG_IN2_FB1_6	Store IN2 in FIFO B1 6	0x1E
	IOC_INPUT_CFG_IN2_FB1_7	Store IN2 in FIFO B1 7	0x1F
	IOC_INPUT_CFG_IN2_NONE	Not stored	0x3F*
IN1_STORE	IOC_INPUT_CFG_IN1_FA0_0	Store IN1 in FIFO A0 0	0x0
	IOC_INPUT_CFG_IN1_FA0_1	Store IN1 in FIFO A0 1	0x1
	IOC_INPUT_CFG_IN1_FA0_2	Store IN1 in FIFO A0 2	0x2
	IOC_INPUT_CFG_IN1_FA0_3	Store IN1 in FIFO A0 3	0x3
	IOC_INPUT_CFG_IN1_FA0_4	Store IN1 in FIFO A0 4	0x4

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	IOC_INPUT_CFG_IN1_FA0_5	Store IN1 in FIFO A0 5	0x5
	IOC_INPUT_CFG_IN1_FA0_6	Store IN1 in FIFO A0 6	0x6
	IOC_INPUT_CFG_IN1_FA0_7	Store IN1 in FIFO A0 7	0x7
	IOC_INPUT_CFG_IN1_FA1_0	Store IN1 in FIFO A1 0	0x8
	IOC_INPUT_CFG_IN1_FA1_1	Store IN1 in FIFO A1 1	0x9
	IOC_INPUT_CFG_IN1_FA1_2	Store IN1 in FIFO A1 2	0xA
	IOC_INPUT_CFG_IN1_FA1_3	Store IN1 in FIFO A1 3	0xB
	IOC_INPUT_CFG_IN1_FA1_4	Store IN1 in FIFO A1 4	0xC
	IOC_INPUT_CFG_IN1_FA1_5	Store IN1 in FIFO A1 5	0xD
	IOC_INPUT_CFG_IN1_FA1_6	Store IN1 in FIFO A1 6	0xE
	IOC_INPUT_CFG_IN1_FA1_7	Store IN1 in FIFO A1 7	0xF
	IOC_INPUT_CFG_IN1_FB0_0	Store IN1 in FIFO B0 0	0x10
	IOC_INPUT_CFG_IN1_FB0_1	Store IN1 in FIFO B0 1	0x11
	IOC_INPUT_CFG_IN1_FB0_2	Store IN1 in FIFO B0 2	0x12
	IOC_INPUT_CFG_IN1_FB0_3	Store IN1 in FIFO B0 3	0x13
	IOC_INPUT_CFG_IN1_FB0_4	Store IN1 in FIFO B0 4	0x14
	IOC_INPUT_CFG_IN1_FB0_5	Store IN1 in FIFO B0 5	0x15
	IOC_INPUT_CFG_IN1_FB0_6	Store IN1 in FIFO B0 6	0x16
	IOC_INPUT_CFG_IN1_FB0_7	Store IN1 in FIFO B0 7	0x17
	IOC_INPUT_CFG_IN1_FB1_0	Store IN1 in FIFO B1 0	0x18
	IOC_INPUT_CFG_IN1_FB1_1	Store IN1 in FIFO B1 1	0x19
	IOC_INPUT_CFG_IN1_FB1_2	Store IN1 in FIFO B1 2	0x1A
	IOC_INPUT_CFG_IN1_FB1_3	Store IN1 in FIFO B1 3	0x1B
	IOC_INPUT_CFG_IN1_FB1_4	Store IN1 in FIFO B1 4	0x1C
	IOC_INPUT_CFG_IN1_FB1_5	Store IN1 in FIFO B1 5	0x1D
	IOC_INPUT_CFG_IN1_FB1_6	Store IN1 in FIFO B1 6	0x1E
	IOC_INPUT_CFG_IN1_FB1_7	Store IN1 in FIFO B1 7	0x1F
	IOC_INPUT_CFG_IN1_NONE	Not stored	0x3F*
IN0_STORE	IOC_INPUT_CFG_IN0_FA0_0	Store IN0 in FIFO A0 0	0x0
	IOC_INPUT_CFG_IN0_FA0_1	Store IN0 in FIFO A0 1	0x1
	IOC_INPUT_CFG_IN0_FA0_2	Store IN0 in FIFO A0 2	0x2

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	IOC_INPUT_CFG_IN0_FA0_3	Store IN0 in FIFO A0 3	0x3
	IOC_INPUT_CFG_IN0_FA0_4	Store IN0 in FIFO A0 4	0x4
	IOC_INPUT_CFG_IN0_FA0_5	Store IN0 in FIFO A0 5	0x5
	IOC_INPUT_CFG_IN0_FA0_6	Store IN0 in FIFO A0 6	0x6
	IOC_INPUT_CFG_IN0_FA0_7	Store IN0 in FIFO A0 7	0x7
	IOC_INPUT_CFG_IN0_FA1_0	Store IN0 in FIFO A1 0	0x8
	IOC_INPUT_CFG_IN0_FA1_1	Store IN0 in FIFO A1 1	0x9
	IOC_INPUT_CFG_IN0_FA1_2	Store IN0 in FIFO A1 2	0xA
	IOC_INPUT_CFG_IN0_FA1_3	Store IN0 in FIFO A1 3	0xB
	IOC_INPUT_CFG_IN0_FA1_4	Store IN0 in FIFO A1 4	0xC
	IOC_INPUT_CFG_IN0_FA1_5	Store IN0 in FIFO A1 5	0xD
	IOC_INPUT_CFG_IN0_FA1_6	Store IN0 in FIFO A1 6	0xE
	IOC_INPUT_CFG_IN0_FA1_7	Store IN0 in FIFO A1 7	0xF
	IOC_INPUT_CFG_IN0_FB0_0	Store IN0 in FIFO B0 0	0x10
	IOC_INPUT_CFG_IN0_FB0_1	Store IN0 in FIFO B0 1	0x11
	IOC_INPUT_CFG_IN0_FB0_2	Store IN0 in FIFO B0 2	0x12
	IOC_INPUT_CFG_IN0_FB0_3	Store IN0 in FIFO B0 3	0x13
	IOC_INPUT_CFG_IN0_FB0_4	Store IN0 in FIFO B0 4	0x14
	IOC_INPUT_CFG_IN0_FB0_5	Store IN0 in FIFO B0 5	0x15
	IOC_INPUT_CFG_IN0_FB0_6	Store IN0 in FIFO B0 6	0x16
	IOC_INPUT_CFG_IN0_FB0_7	Store IN0 in FIFO B0 7	0x17
	IOC_INPUT_CFG_IN0_FB1_0	Store IN0 in FIFO B1 0	0x18
	IOC_INPUT_CFG_IN0_FB1_1	Store IN0 in FIFO B1 1	0x19
	IOC_INPUT_CFG_IN0_FB1_2	Store IN0 in FIFO B1 2	0x1A
	IOC_INPUT_CFG_IN0_FB1_3	Store IN0 in FIFO B1 3	0x1B
	IOC_INPUT_CFG_IN0_FB1_4	Store IN0 in FIFO B1 4	0x1C
	IOC_INPUT_CFG_IN0_FB1_5	Store IN0 in FIFO B1 5	0x1D
	IOC_INPUT_CFG_IN0_FB1_6	Store IN0 in FIFO B1 6	0x1E
	IOC_INPUT_CFG_IN0_FB1_7	Store IN0 in FIFO B1 7	0x1F
	IOC_INPUT_CFG_IN0_NONE	Not stored	0x3F*

Ezairo 8300 Hardware Reference

13.4.1.2 D_IOC*_INPUT_PCM_CFG, IOC*_INPUT_PCM_CFG

Bit Field	Read/Write	Field Name	Description
13:8	RW	PCM_RX1_STORE	Select where to store PCM RX1
5:0	RW	PCM_RX0_STORE	Select where to store PCM RX0

Field Name	Value Symbol	Value Description	Hex Value
PCM_RX1_STORE	IOC_INPUT_CFG_PCM_RX1_FA0_0	Store PCM RX1 in FIFO A0 0	0x0
	IOC_INPUT_CFG_PCM_RX1_FA0_1	Store PCM RX1 in FIFO A0 1	0x1
	IOC_INPUT_CFG_PCM_RX1_FA0_2	Store PCM RX1 in FIFO A0 2	0x2
	IOC_INPUT_CFG_PCM_RX1_FA0_3	Store PCM RX1 in FIFO A0 3	0x3
	IOC_INPUT_CFG_PCM_RX1_FA0_4	Store PCM RX1 in FIFO A0 4	0x4
	IOC_INPUT_CFG_PCM_RX1_FA0_5	Store PCM RX1 in FIFO A0 5	0x5
	IOC_INPUT_CFG_PCM_RX1_FA0_6	Store PCM RX1 in FIFO A0 6	0x6
	IOC_INPUT_CFG_PCM_RX1_FA0_7	Store PCM RX1 in FIFO A0 7	0x7
	IOC_INPUT_CFG_PCM_RX1_FA1_0	Store PCM RX1 in FIFO A1 0	0x8
	IOC_INPUT_CFG_PCM_RX1_FA1_1	Store PCM RX1 in FIFO A1 1	0x9
	IOC_INPUT_CFG_PCM_RX1_FA1_2	Store PCM RX1 in FIFO A1 2	0xA
	IOC_INPUT_CFG_PCM_RX1_FA1_3	Store PCM RX1 in FIFO A1 3	0xB
	IOC_INPUT_CFG_PCM_RX1_FA1_4	Store PCM RX1 in FIFO A1 4	0xC
	IOC_INPUT_CFG_PCM_RX1_FA1_5	Store PCM RX1 in FIFO A1 5	0xD
	IOC_INPUT_CFG_PCM_RX1_FA1_6	Store PCM RX1 in FIFO A1 6	0xE
	IOC_INPUT_CFG_PCM_RX1_FA1_7	Store PCM RX1 in FIFO A1 7	0xF

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	IOC_INPUT_CFG_PCM_RX1_FB0_0	Store PCM RX1 in FIFO B0 0	0x10
	IOC_INPUT_CFG_PCM_RX1_FB0_1	Store PCM RX1 in FIFO B0 1	0x11
	IOC_INPUT_CFG_PCM_RX1_FB0_2	Store PCM RX1 in FIFO B0 2	0x12
	IOC_INPUT_CFG_PCM_RX1_FB0_3	Store PCM RX1 in FIFO B0 3	0x13
	IOC_INPUT_CFG_PCM_RX1_FB0_4	Store PCM RX1 in FIFO B0 4	0x14
	IOC_INPUT_CFG_PCM_RX1_FB0_5	Store PCM RX1 in FIFO B0 5	0x15
	IOC_INPUT_CFG_PCM_RX1_FB0_6	Store PCM RX1 in FIFO B0 6	0x16
	IOC_INPUT_CFG_PCM_RX1_FB0_7	Store PCM RX1 in FIFO B0 7	0x17
	IOC_INPUT_CFG_PCM_RX1_FB1_0	Store PCM RX1 in FIFO B1 0	0x18
	IOC_INPUT_CFG_PCM_RX1_FB1_1	Store PCM RX1 in FIFO B1 1	0x19
	IOC_INPUT_CFG_PCM_RX1_FB1_2	Store PCM RX1 in FIFO B1 2	0x1A
	IOC_INPUT_CFG_PCM_RX1_FB1_3	Store PCM RX1 in FIFO B1 3	0x1B
	IOC_INPUT_CFG_PCM_RX1_FB1_4	Store PCM RX1 in FIFO B1 4	0x1C
	IOC_INPUT_CFG_PCM_RX1_FB1_5	Store PCM RX1 in FIFO B1 5	0x1D
	IOC_INPUT_CFG_PCM_RX1_FB1_6	Store PCM RX1 in FIFO B1 6	0x1E
	IOC_INPUT_CFG_PCM_RX1_FB1_7	Store PCM RX1 in FIFO B1 7	0x1F
	IOC_INPUT_CFG_PCM_RX1_NONE	Not stored	0x3F*
PCM_RX0_STORE	IOC_INPUT_CFG_PCM_RX0_FA0_0	Store PCM RX0 in FIFO A0 0	0x0
	IOC_INPUT_CFG_PCM_RX0_FA0_1	Store PCM RX0 in FIFO A0 1	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	IOC_INPUT_CFG_PCM_RX0_FA0_2	Store PCM RX0 in FIFO A0 2	0x2
	IOC_INPUT_CFG_PCM_RX0_FA0_3	Store PCM RX0 in FIFO A0 3	0x3
	IOC_INPUT_CFG_PCM_RX0_FA0_4	Store PCM RX0 in FIFO A0 4	0x4
	IOC_INPUT_CFG_PCM_RX0_FA0_5	Store PCM RX0 in FIFO A0 5	0x5
	IOC_INPUT_CFG_PCM_RX0_FA0_6	Store PCM RX0 in FIFO A0 6	0x6
	IOC_INPUT_CFG_PCM_RX0_FA0_7	Store PCM RX0 in FIFO A0 7	0x7
	IOC_INPUT_CFG_PCM_RX0_FA1_0	Store PCM RX0 in FIFO A1 0	0x8
	IOC_INPUT_CFG_PCM_RX0_FA1_1	Store PCM RX0 in FIFO A1 1	0x9
	IOC_INPUT_CFG_PCM_RX0_FA1_2	Store PCM RX0 in FIFO A1 2	0xA
	IOC_INPUT_CFG_PCM_RX0_FA1_3	Store PCM RX0 in FIFO A1 3	0xB
	IOC_INPUT_CFG_PCM_RX0_FA1_4	Store PCM RX0 in FIFO A1 4	0xC
	IOC_INPUT_CFG_PCM_RX0_FA1_5	Store PCM RX0 in FIFO A1 5	0xD
	IOC_INPUT_CFG_PCM_RX0_FA1_6	Store PCM RX0 in FIFO A1 6	0xE
	IOC_INPUT_CFG_PCM_RX0_FA1_7	Store PCM RX0 in FIFO A1 7	0xF
	IOC_INPUT_CFG_PCM_RX0_FB0_0	Store PCM RX0 in FIFO B0 0	0x10
	IOC_INPUT_CFG_PCM_RX0_FB0_1	Store PCM RX0 in FIFO B0 1	0x11
	IOC_INPUT_CFG_PCM_RX0_FB0_2	Store PCM RX0 in FIFO B0 2	0x12
	IOC_INPUT_CFG_PCM_RX0_FB0_3	Store PCM RX0 in FIFO B0 3	0x13
	IOC_INPUT_CFG_PCM_RX0_FB0_4	Store PCM RX0 in FIFO B0 4	0x14

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	IOC_INPUT_CFG_PCM_RX0_FB0_5	Store PCM RX0 in FIFO B0 5	0x15
	IOC_INPUT_CFG_PCM_RX0_FB0_6	Store PCM RX0 in FIFO B0 6	0x16
	IOC_INPUT_CFG_PCM_RX0_FB0_7	Store PCM RX0 in FIFO B0 7	0x17
	IOC_INPUT_CFG_PCM_RX0_FB1_0	Store PCM RX0 in FIFO B1 0	0x18
	IOC_INPUT_CFG_PCM_RX0_FB1_1	Store PCM RX0 in FIFO B1 1	0x19
	IOC_INPUT_CFG_PCM_RX0_FB1_2	Store PCM RX0 in FIFO B1 2	0x1A
	IOC_INPUT_CFG_PCM_RX0_FB1_3	Store PCM RX0 in FIFO B1 3	0x1B
	IOC_INPUT_CFG_PCM_RX0_FB1_4	Store PCM RX0 in FIFO B1 4	0x1C
	IOC_INPUT_CFG_PCM_RX0_FB1_5	Store PCM RX0 in FIFO B1 5	0x1D
	IOC_INPUT_CFG_PCM_RX0_FB1_6	Store PCM RX0 in FIFO B1 6	0x1E
	IOC_INPUT_CFG_PCM_RX0_FB1_7	Store PCM RX0 in FIFO B1 7	0x1F
	IOC_INPUT_CFG_PCM_RX0_NONE	Not stored	0x3F*

13.4.1.3 D_IOC*_FIFO_ACCESS_CFG, IOC*_FIFO_ACCESS_CFG

Bit Field	Read/Write	Field Name	Description
31	RW	FIFO_B1_7_DBL_ACC	Defines FIFO B1 7 access type
30	RW	FIFO_B1_6_DBL_ACC	Defines FIFO B1 6 access type
29	RW	FIFO_B1_5_DBL_ACC	Defines FIFO B1 5 access type
28	RW	FIFO_B1_4_DBL_ACC	Defines FIFO B1 4 access type
27	RW	FIFO_B1_3_DBL_ACC	Defines FIFO B1 3 access type
26	RW	FIFO_B1_2_DBL_ACC	Defines FIFO B1 2 access type
25	RW	FIFO_B1_1_DBL_ACC	Defines FIFO B1 1 access type
24	RW	FIFO_B1_0_DBL_ACC	Defines FIFO B1 0 access type
23	RW	FIFO_B0_7_DBL_ACC	Defines FIFO B0 7 access type

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
22	RW	FIFO_B0_6_DBL_ACC	Defines FIFO B0 6 access type
21	RW	FIFO_B0_5_DBL_ACC	Defines FIFO B0 5 access type
20	RW	FIFO_B0_4_DBL_ACC	Defines FIFO B0 4 access type
19	RW	FIFO_B0_3_DBL_ACC	Defines FIFO B0 3 access type
18	RW	FIFO_B0_2_DBL_ACC	Defines FIFO B0 2 access type
17	RW	FIFO_B0_1_DBL_ACC	Defines FIFO B0 1 access type
16	RW	FIFO_B0_0_DBL_ACC	Defines FIFO B0 0 access type
15	RW	FIFO_A1_7_DBL_ACC	Defines FIFO A1 7 access type
14	RW	FIFO_A1_6_DBL_ACC	Defines FIFO A1 6 access type
13	RW	FIFO_A1_5_DBL_ACC	Defines FIFO A1 5 access type
12	RW	FIFO_A1_4_DBL_ACC	Defines FIFO A1 4 access type
11	RW	FIFO_A1_3_DBL_ACC	Defines FIFO A1 3 access type
10	RW	FIFO_A1_2_DBL_ACC	Defines FIFO A1 2 access type
9	RW	FIFO_A1_1_DBL_ACC	Defines FIFO A1 1 access type
8	RW	FIFO_A1_0_DBL_ACC	Defines FIFO A1 0 access type
7	RW	FIFO_A0_7_DBL_ACC	Defines FIFO A0 7 access type
6	RW	FIFO_A0_6_DBL_ACC	Defines FIFO A0 6 access type
5	RW	FIFO_A0_5_DBL_ACC	Defines FIFO A0 5 access type
4	RW	FIFO_A0_4_DBL_ACC	Defines FIFO A0 4 access type
3	RW	FIFO_A0_3_DBL_ACC	Defines FIFO A0 3 access type
2	RW	FIFO_A0_2_DBL_ACC	Defines FIFO A0 2 access type
1	RW	FIFO_A0_1_DBL_ACC	Defines FIFO A0 1 access type
0	RW	FIFO_A0_0_DBL_ACC	Defines FIFO A0 0 access type

Field Name	Value Symbol	Value Description	Hex Value
FIFO_B1_7_DBL_ACC	FIFO_B1_7_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B1_7_DBL_ACC_EN	Double access	0x1
FIFO_B1_6_DBL_ACC	FIFO_B1_6_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B1_6_DBL_ACC_EN	Double access	0x1
FIFO_B1_5_DBL_ACC	FIFO_B1_5_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B1_5_DBL_ACC_EN	Double access	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
FIFO_B1_4_DBL_ACC	FIFO_B1_4_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B1_4_DBL_ACC_EN	Double access	0x1
FIFO_B1_3_DBL_ACC	FIFO_B1_3_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B1_3_DBL_ACC_EN	Double access	0x1
FIFO_B1_2_DBL_ACC	FIFO_B1_2_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B1_2_DBL_ACC_EN	Double access	0x1
FIFO_B1_1_DBL_ACC	FIFO_B1_1_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B1_1_DBL_ACC_EN	Double access	0x1
FIFO_B1_0_DBL_ACC	FIFO_B1_0_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B1_0_DBL_ACC_EN	Double access	0x1
FIFO_B0_7_DBL_ACC	FIFO_B0_7_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B0_7_DBL_ACC_EN	Double access	0x1
FIFO_B0_6_DBL_ACC	FIFO_B0_6_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B0_6_DBL_ACC_EN	Double access	0x1
FIFO_B0_5_DBL_ACC	FIFO_B0_5_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B0_5_DBL_ACC_EN	Double access	0x1
FIFO_B0_4_DBL_ACC	FIFO_B0_4_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B0_4_DBL_ACC_EN	Double access	0x1
FIFO_B0_3_DBL_ACC	FIFO_B0_3_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B0_3_DBL_ACC_EN	Double access	0x1
FIFO_B0_2_DBL_ACC	FIFO_B0_2_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B0_2_DBL_ACC_EN	Double access	0x1
FIFO_B0_1_DBL_ACC	FIFO_B0_1_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B0_1_DBL_ACC_EN	Double access	0x1
FIFO_B0_0_DBL_ACC	FIFO_B0_0_DBL_ACC_DIS	Simple access	0x0*
	FIFO_B0_0_DBL_ACC_EN	Double access	0x1
FIFO_A1_7_DBL_ACC	FIFO_A1_7_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A1_7_DBL_ACC_EN	Double access	0x1
FIFO_A1_6_DBL_ACC	FIFO_A1_6_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A1_6_DBL_ACC_EN	Double access	0x1
FIFO_A1_5_DBL_ACC	FIFO_A1_5_DBL_ACC_DIS	Simple access	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	FIFO_A1_5_DBL_ACC_EN	Double access	0x1
FIFO_A1_4_DBL_ACC	FIFO_A1_4_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A1_4_DBL_ACC_EN	Double access	0x1
FIFO_A1_3_DBL_ACC	FIFO_A1_3_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A1_3_DBL_ACC_EN	Double access	0x1
FIFO_A1_2_DBL_ACC	FIFO_A1_2_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A1_2_DBL_ACC_EN	Double access	0x1
FIFO_A1_1_DBL_ACC	FIFO_A1_1_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A1_1_DBL_ACC_EN	Double access	0x1
FIFO_A1_0_DBL_ACC	FIFO_A1_0_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A1_0_DBL_ACC_EN	Double access	0x1
FIFO_A0_7_DBL_ACC	FIFO_A0_7_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A0_7_DBL_ACC_EN	Double access	0x1
FIFO_A0_6_DBL_ACC	FIFO_A0_6_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A0_6_DBL_ACC_EN	Double access	0x1
FIFO_A0_5_DBL_ACC	FIFO_A0_5_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A0_5_DBL_ACC_EN	Double access	0x1
FIFO_A0_4_DBL_ACC	FIFO_A0_4_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A0_4_DBL_ACC_EN	Double access	0x1
FIFO_A0_3_DBL_ACC	FIFO_A0_3_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A0_3_DBL_ACC_EN	Double access	0x1
FIFO_A0_2_DBL_ACC	FIFO_A0_2_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A0_2_DBL_ACC_EN	Double access	0x1
FIFO_A0_1_DBL_ACC	FIFO_A0_1_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A0_1_DBL_ACC_EN	Double access	0x1
FIFO_A0_0_DBL_ACC	FIFO_A0_0_DBL_ACC_DIS	Simple access	0x0*
	FIFO_A0_0_DBL_ACC_EN	Double access	0x1

13.4.1.4 D_IOC*_IN0_DATA, IOC*_IN0_DATA

Bit Field	Read/Write	Field Name	Description
31:0	R	IN_DATA	Latest audio sample from IN0 (m1p23 format, sign extended to 32bits)

Ezairo 8300 Hardware Reference

13.4.1.5 D_IOC*_IN1_DATA, IOC*_IN1_DATA

Bit Field	Read/Write	Field Name	Description
31:0	R	IN_DATA	Latest audio sample from IN1 (m1p23 format, sign extended to 32bits)

13.4.1.6 D_IOC*_IN2_DATA, IOC*_IN2_DATA

Bit Field	Read/Write	Field Name	Description
31:0	R	IN_DATA	Latest audio sample from IN2 (m1p23 format, sign extended to 32bits)

13.4.1.7 D_IOC*_IN3_DATA, IOC*_IN3_DATA

Bit Field	Read/Write	Field Name	Description
31:0	R	IN_DATA	Latest audio sample from IN3 (m1p23 format, sign extended to 32bits)

13.5 IOC OUTPUT

The output of IOC0 is attached to the two output channels from the output stage and the Filter Engine, denoted by OUT0 and OUT1 in constants and documentation. IOC1 is attached to two output channels to the Filter Engine (also denoted by OUT0 and OUT1). Data provided to the Filter Engine by the IOCs is processed as 24-bit data.

For each of these output channels, the IOCs provide an access register (D_OUT*_DATA) that can be used to supply data to be transferred to that output channel.

In addition to the output channels, each IOC can be attached to a maximum of two channels of output from its associated PCM interface. IOC0 can interface with PCM0, and IOC1 can interface with PCM1. The two output channels are denoted by TX0 and TX1 in constants and documentation, and these channels are nominally considered to be 24-bit output data. The PCM interface configuration can be used to align data that is less than 24 bits in size. Each of IOC0 and IOC1 has its own TX0 and TX1 signals.

Each IOC (if configured correctly) can direct data from the FIFOs to each of the four output sinks. Select the source FIFO for each output by configuring the IOC_OUTPUT_CFG_OUT0_LOAD, IOC_OUTPUT_CFG_OUT1_LOAD, IOC_OUTPUT_CFG_PCM_TX0_LOAD and IOC_OUTPUT_CFG_PCM_TX1_LOAD bit fields of the D_IOC*_OUTPUT_CFG register. Each of these bit fields allows you to select the source of your output data from one of the 32 FIFOs, any of the six input sources, or no source. Additionally, the OUT0 and OUT1 configurations can be used to select the D_IOC*_OUT*_DATA bypass data registers as their source.

IMPORTANT: Connecting IOC output sinks to FIFOs that have not been configured results in undefined behavior.

Data for more than one output channel can be read from the same FIFO. In such a case, the data written to the outputs is guaranteed to be read from the FIFO in the time sequence order OUT0, OUT1. Similarly, data for more than one PCM output channel can be loaded from the same FIFO. The data provided to the PCM interface is guaranteed to be in the order PCM 0/1 TX0/1. For details about how output FIFO data is read from memory, see [Section 12.3.1 “FIFO Input and Output Block” on page 391](#).

Ezairo 8300 Hardware Reference

IMPORTANT: The Ezairo 8300 system does not support storing outputs to both the output channels and the PCM interface in the same FIFO. Because of this, always use different FIFOs to provide data to the output channels and the PCM interface.

NOTE: Writing to the D_IOC*_OUTPUT_CFG or the D_IOC*_FIFO_ACCESS_CFG register resets the output state machine of the associated IOC.

13.5.1 Control and Configuration Registers

Register Name	Register Description	Address
D_IOC0_OUTPUT_CFG	Configuration registers for IOC to outputs	P:0x0904
IOC0_OUTPUT_CFG		0x40002410
D_IOC0_OUT0_DATA	Input sample for OUT0	P:0x0914
IOC0_OUT0_DATA		0x40002450
D_IOC0_OUT1_DATA	Input sample for OUT1	P:0x0915
IOC0_OUT1_DATA		0x40002454
D_IOC1_OUTPUT_CFG	Configuration registers for IOC to outputs	P:0x0944
IOC1_OUTPUT_CFG		0x40002510
D_IOC1_OUT0_DATA	Input sample for OUT0	P:0x0954
IOC1_OUT0_DATA		0x40002550
D_IOC1_OUT1_DATA	Input sample for OUT1	P:0x0955
IOC1_OUT1_DATA		0x40002554

13.5.1.1 D_IOC*_OUTPUT_CFG, IOC*_OUTPUT_CFG

Bit Field	Read/Write	Field Name	Description
29:24	RW	PCM_TX1_LOAD	Defines the origin of the audio sample for PCM TX1
21:16	RW	PCM_TX0_LOAD	Defines the origin of the audio sample for PCM TX0
13:8	RW	OUT1_LOAD	Defines the origin of the audio sample for OUT1
5:0	RW	OUT0_LOAD	Defines the origin of the audio sample for OUT0

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
PCM_TX1_LOAD	IOC_OUTPUT_CFG_PCM_TX1_ FA0_0	Load PCM TX1 from FIFO A0 0	0x0
	IOC_OUTPUT_CFG_PCM_TX1_ FA0_1	Load PCM TX1 from FIFO A0 1	0x1
	IOC_OUTPUT_CFG_PCM_TX1_ FA0_2	Load PCM TX1 from FIFO A0 2	0x2
	IOC_OUTPUT_CFG_PCM_TX1_ FA0_3	Load PCM TX1 from FIFO A0 3	0x3
	IOC_OUTPUT_CFG_PCM_TX1_ FA0_4	Load PCM TX1 from FIFO A0 4	0x4
	IOC_OUTPUT_CFG_PCM_TX1_ FA0_5	Load PCM TX1 from FIFO A0 5	0x5
	IOC_OUTPUT_CFG_PCM_TX1_ FA0_6	Load PCM TX1 from FIFO A0 6	0x6
	IOC_OUTPUT_CFG_PCM_TX1_ FA0_7	Load PCM TX1 from FIFO A0 7	0x7
	IOC_OUTPUT_CFG_PCM_TX1_ FA1_0	Load PCM TX1 from FIFO A1 0	0x8
	IOC_OUTPUT_CFG_PCM_TX1_ FA1_1	Load PCM TX1 from FIFO A1 1	0x9
	IOC_OUTPUT_CFG_PCM_TX1_ FA1_2	Load PCM TX1 from FIFO A1 2	0xA
	IOC_OUTPUT_CFG_PCM_TX1_ FA1_3	Load PCM TX1 from FIFO A1 3	0xB
	IOC_OUTPUT_CFG_PCM_TX1_ FA1_4	Load PCM TX1 from FIFO A1 4	0xC
	IOC_OUTPUT_CFG_PCM_TX1_ FA1_5	Load PCM TX1 from FIFO A1 5	0xD
	IOC_OUTPUT_CFG_PCM_TX1_ FA1_6	Load PCM TX1 from FIFO A1 6	0xE
	IOC_OUTPUT_CFG_PCM_TX1_ FA1_7	Load PCM TX1 from FIFO A1 7	0xF
	IOC_OUTPUT_CFG_PCM_TX1_ FB0_0	Load PCM TX1 from FIFO B0 0	0x10
	IOC_OUTPUT_CFG_PCM_TX1_ FB0_1	Load PCM TX1 from FIFO B0 1	0x11
	IOC_OUTPUT_CFG_PCM_TX1_ FB0_2	Load PCM TX1 from FIFO B0 2	0x12

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	IOC_OUTPUT_CFG_PCM_TX1_FB0_3	Load PCM TX1 from FIFO B0 3	0x13
	IOC_OUTPUT_CFG_PCM_TX1_FB0_4	Load PCM TX1 from FIFO B0 4	0x14
	IOC_OUTPUT_CFG_PCM_TX1_FB0_5	Load PCM TX1 from FIFO B0 5	0x15
	IOC_OUTPUT_CFG_PCM_TX1_FB0_6	Load PCM TX1 from FIFO B0 6	0x16
	IOC_OUTPUT_CFG_PCM_TX1_FB0_7	Load PCM TX1 from FIFO B0 7	0x17
	IOC_OUTPUT_CFG_PCM_TX1_FB1_0	Load PCM TX1 from FIFO B1 0	0x18
	IOC_OUTPUT_CFG_PCM_TX1_FB1_1	Load PCM TX1 from FIFO B1 1	0x19
	IOC_OUTPUT_CFG_PCM_TX1_FB1_2	Load PCM TX1 from FIFO B1 2	0x1A
	IOC_OUTPUT_CFG_PCM_TX1_FB1_3	Load PCM TX1 from FIFO B1 3	0x1B
	IOC_OUTPUT_CFG_PCM_TX1_FB1_4	Load PCM TX1 from FIFO B1 4	0x1C
	IOC_OUTPUT_CFG_PCM_TX1_FB1_5	Load PCM TX1 from FIFO B1 5	0x1D
	IOC_OUTPUT_CFG_PCM_TX1_FB1_6	Load PCM TX1 from FIFO B1 6	0x1E
	IOC_OUTPUT_CFG_PCM_TX1_FB1_7	Load PCM TX1 from FIFO B1 7	0x1F
	IOC_OUTPUT_CFG_PCM_TX1_PCM_RX0	Load PCM TX1 from PCM RX0	0x20
	IOC_OUTPUT_CFG_PCM_TX1_PCM_RX1	Load PCM TX1 from PCM RX1	0x21
	IOC_OUTPUT_CFG_PCM_TX1_IN0	Load PCM TX1 from IN0	0x22
	IOC_OUTPUT_CFG_PCM_TX1_IN1	Load PCM TX1 from IN1	0x23
	IOC_OUTPUT_CFG_PCM_TX1_IN2	Load PCM TX1 from IN2	0x24
	IOC_OUTPUT_CFG_PCM_TX1_IN3	Load PCM TX1 from IN3	0x25
	IOC_OUTPUT_CFG_PCM_TX1_NONE	Do not load (load 0 instead)	0x3F*
PCM_TX0_LOAD	IOC_OUTPUT_CFG_PCM_TX0_	Load PCM TX0 from FIFO A0 0	0x0

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	FA0_0		
	IOC_OUTPUT_CFG_PCM_TX0_ FA0_1	Load PCM TX0 from FIFO A0 1	0x1
	IOC_OUTPUT_CFG_PCM_TX0_ FA0_2	Load PCM TX0 from FIFO A0 2	0x2
	IOC_OUTPUT_CFG_PCM_TX0_ FA0_3	Load PCM TX0 from FIFO A0 3	0x3
	IOC_OUTPUT_CFG_PCM_TX0_ FA0_4	Load PCM TX0 from FIFO A0 4	0x4
	IOC_OUTPUT_CFG_PCM_TX0_ FA0_5	Load PCM TX0 from FIFO A0 5	0x5
	IOC_OUTPUT_CFG_PCM_TX0_ FA0_6	Load PCM TX0 from FIFO A0 6	0x6
	IOC_OUTPUT_CFG_PCM_TX0_ FA0_7	Load PCM TX0 from FIFO A0 7	0x7
	IOC_OUTPUT_CFG_PCM_TX0_ FA1_0	Load PCM TX0 from FIFO A1 0	0x8
	IOC_OUTPUT_CFG_PCM_TX0_ FA1_1	Load PCM TX0 from FIFO A1 1	0x9
	IOC_OUTPUT_CFG_PCM_TX0_ FA1_2	Load PCM TX0 from FIFO A1 2	0xA
	IOC_OUTPUT_CFG_PCM_TX0_ FA1_3	Load PCM TX0 from FIFO A1 3	0xB
	IOC_OUTPUT_CFG_PCM_TX0_ FA1_4	Load PCM TX0 from FIFO A1 4	0xC
	IOC_OUTPUT_CFG_PCM_TX0_ FA1_5	Load PCM TX0 from FIFO A1 5	0xD
	IOC_OUTPUT_CFG_PCM_TX0_ FA1_6	Load PCM TX0 from FIFO A1 6	0xE
	IOC_OUTPUT_CFG_PCM_TX0_ FA1_7	Load PCM TX0 from FIFO A1 7	0xF
	IOC_OUTPUT_CFG_PCM_TX0_ FB0_0	Load PCM TX0 from FIFO B0 0	0x10
	IOC_OUTPUT_CFG_PCM_TX0_ FB0_1	Load PCM TX0 from FIFO B0 1	0x11
	IOC_OUTPUT_CFG_PCM_TX0_ FB0_2	Load PCM TX0 from FIFO B0 2	0x12

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	IOC_OUTPUT_CFG_PCM_TX0_FB0_3	Load PCM TX0 from FIFO B0 3	0x13
	IOC_OUTPUT_CFG_PCM_TX0_FB0_4	Load PCM TX0 from FIFO B0 4	0x14
	IOC_OUTPUT_CFG_PCM_TX0_FB0_5	Load PCM TX0 from FIFO B0 5	0x15
	IOC_OUTPUT_CFG_PCM_TX0_FB0_6	Load PCM TX0 from FIFO B0 6	0x16
	IOC_OUTPUT_CFG_PCM_TX0_FB0_7	Load PCM TX0 from FIFO B0 7	0x17
	IOC_OUTPUT_CFG_PCM_TX0_FB1_0	Load PCM TX0 from FIFO B1 0	0x18
	IOC_OUTPUT_CFG_PCM_TX0_FB1_1	Load PCM TX0 from FIFO B1 1	0x19
	IOC_OUTPUT_CFG_PCM_TX0_FB1_2	Load PCM TX0 from FIFO B1 2	0x1A
	IOC_OUTPUT_CFG_PCM_TX0_FB1_3	Load PCM TX0 from FIFO B1 3	0x1B
	IOC_OUTPUT_CFG_PCM_TX0_FB1_4	Load PCM TX0 from FIFO B1 4	0x1C
	IOC_OUTPUT_CFG_PCM_TX0_FB1_5	Load PCM TX0 from FIFO B1 5	0x1D
	IOC_OUTPUT_CFG_PCM_TX0_FB1_6	Load PCM TX0 from FIFO B1 6	0x1E
	IOC_OUTPUT_CFG_PCM_TX0_FB1_7	Load PCM TX0 from FIFO B1 7	0x1F
	IOC_OUTPUT_CFG_PCM_TX0_PCM_RX0	Load PCM TX0 from PCM RX0	0x20
	IOC_OUTPUT_CFG_PCM_TX0_PCM_RX1	Load PCM TX0 from PCM RX1	0x21
	IOC_OUTPUT_CFG_PCM_TX0_IN0	Load PCM TX0 from IN0	0x22
	IOC_OUTPUT_CFG_PCM_TX0_IN1	Load PCM TX0 from IN1	0x23
	IOC_OUTPUT_CFG_PCM_TX0_IN2	Load PCM TX0 from IN2	0x24
	IOC_OUTPUT_CFG_PCM_TX0_IN3	Load PCM TX0 from IN3	0x25
	IOC_OUTPUT_CFG_PCM_TX0_NONE	Do not load (load 0 instead)	0x3F*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
OUT1_LOAD	IOC_OUTPUT_CFG_OUT1_FA0_0	Load OUT1 from FIFO A0 0	0x0
	IOC_OUTPUT_CFG_OUT1_FA0_1	Load OUT1 from FIFO A0 1	0x1
	IOC_OUTPUT_CFG_OUT1_FA0_2	Load OUT1 from FIFO A0 2	0x2
	IOC_OUTPUT_CFG_OUT1_FA0_3	Load OUT1 from FIFO A0 3	0x3
	IOC_OUTPUT_CFG_OUT1_FA0_4	Load OUT1 from FIFO A0 4	0x4
	IOC_OUTPUT_CFG_OUT1_FA0_5	Load OUT1 from FIFO A0 5	0x5
	IOC_OUTPUT_CFG_OUT1_FA0_6	Load OUT1 from FIFO A0 6	0x6
	IOC_OUTPUT_CFG_OUT1_FA0_7	Load OUT1 from FIFO A0 7	0x7
	IOC_OUTPUT_CFG_OUT1_FA1_0	Load OUT1 from FIFO A1 0	0x8
	IOC_OUTPUT_CFG_OUT1_FA1_1	Load OUT1 from FIFO A1 1	0x9
	IOC_OUTPUT_CFG_OUT1_FA1_2	Load OUT1 from FIFO A1 2	0xA
	IOC_OUTPUT_CFG_OUT1_FA1_3	Load OUT1 from FIFO A1 3	0xB
	IOC_OUTPUT_CFG_OUT1_FA1_4	Load OUT1 from FIFO A1 4	0xC
	IOC_OUTPUT_CFG_OUT1_FA1_5	Load OUT1 from FIFO A1 5	0xD
	IOC_OUTPUT_CFG_OUT1_FA1_6	Load OUT1 from FIFO A1 6	0xE
	IOC_OUTPUT_CFG_OUT1_FA1_7	Load OUT1 from FIFO A1 7	0xF
	IOC_OUTPUT_CFG_OUT1_FB0_0	Load OUT1 from FIFO B0 0	0x10
	IOC_OUTPUT_CFG_OUT1_FB0_1	Load OUT1 from FIFO B0 1	0x11
	IOC_OUTPUT_CFG_OUT1_FB0_2	Load OUT1 from FIFO B0 2	0x12
	IOC_OUTPUT_CFG_OUT1_FB0_3	Load OUT1 from FIFO B0 3	0x13
	IOC_OUTPUT_CFG_OUT1_FB0_4	Load OUT1 from FIFO B0 4	0x14
	IOC_OUTPUT_CFG_OUT1_FB0_5	Load OUT1 from FIFO B0 5	0x15
	IOC_OUTPUT_CFG_OUT1_FB0_6	Load OUT1 from FIFO B0 6	0x16
	IOC_OUTPUT_CFG_OUT1_FB0_7	Load OUT1 from FIFO B0 7	0x17
	IOC_OUTPUT_CFG_OUT1_FB1_0	Load OUT1 from FIFO B1 0	0x18
	IOC_OUTPUT_CFG_OUT1_FB1_1	Load OUT1 from FIFO B1 1	0x19
	IOC_OUTPUT_CFG_OUT1_FB1_2	Load OUT1 from FIFO B1 2	0x1A
	IOC_OUTPUT_CFG_OUT1_FB1_3	Load OUT1 from FIFO B1 3	0x1B
	IOC_OUTPUT_CFG_OUT1_FB1_4	Load OUT1 from FIFO B1 4	0x1C
	IOC_OUTPUT_CFG_OUT1_FB1_5	Load OUT1 from FIFO B1 5	0x1D
	IOC_OUTPUT_CFG_OUT1_FB1_6	Load OUT1 from FIFO B1 6	0x1E

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	IOC_OUTPUT_CFG_OUT1_FB1_7	Load OUT1 from FIFO B1 7	0x1F
	IOC_OUTPUT_CFG_OUT1_PCM_RX0	Load OUT1 from PCM RX0	0x20
	IOC_OUTPUT_CFG_OUT1_PCM_RX1	Load OUT1 from PCM RX1	0x21
	IOC_OUTPUT_CFG_OUT1_IN0	Load OUT1 from IN0	0x22
	IOC_OUTPUT_CFG_OUT1_IN1	Load OUT1 from IN1	0x23
	IOC_OUTPUT_CFG_OUT1_IN2	Load OUT1 from IN2	0x24
	IOC_OUTPUT_CFG_OUT1_IN3	Load OUT1 from IN3	0x25
	IOC_OUTPUT_CFG_OUT1_LOAD	Load OUT1 from the OUT1 load register	0x26
	IOC_OUTPUT_CFG_OUT1_NONE	Do not load (load 0 instead)	0x3F*
OUT0_LOAD	IOC_OUTPUT_CFG_OUT0_FA0_0	Load OUT0 from FIFO A0 0	0x0
	IOC_OUTPUT_CFG_OUT0_FA0_1	Load OUT0 from FIFO A0 1	0x1
	IOC_OUTPUT_CFG_OUT0_FA0_2	Load OUT0 from FIFO A0 2	0x2
	IOC_OUTPUT_CFG_OUT0_FA0_3	Load OUT0 from FIFO A0 3	0x3
	IOC_OUTPUT_CFG_OUT0_FA0_4	Load OUT0 from FIFO A0 4	0x4
	IOC_OUTPUT_CFG_OUT0_FA0_5	Load OUT0 from FIFO A0 5	0x5
	IOC_OUTPUT_CFG_OUT0_FA0_6	Load OUT0 from FIFO A0 6	0x6
	IOC_OUTPUT_CFG_OUT0_FA0_7	Load OUT0 from FIFO A0 7	0x7
	IOC_OUTPUT_CFG_OUT0_FA1_0	Load OUT0 from FIFO A1 0	0x8
	IOC_OUTPUT_CFG_OUT0_FA1_1	Load OUT0 from FIFO A1 1	0x9
	IOC_OUTPUT_CFG_OUT0_FA1_2	Load OUT0 from FIFO A1 2	0xA
	IOC_OUTPUT_CFG_OUT0_FA1_3	Load OUT0 from FIFO A1 3	0xB
	IOC_OUTPUT_CFG_OUT0_FA1_4	Load OUT0 from FIFO A1 4	0xC
	IOC_OUTPUT_CFG_OUT0_FA1_5	Load OUT0 from FIFO A1 5	0xD
	IOC_OUTPUT_CFG_OUT0_FA1_6	Load OUT0 from FIFO A1 6	0xE
	IOC_OUTPUT_CFG_OUT0_FA1_7	Load OUT0 from FIFO A1 7	0xF
	IOC_OUTPUT_CFG_OUT0_FB0_0	Load OUT0 from FIFO B0 0	0x10
	IOC_OUTPUT_CFG_OUT0_FB0_1	Load OUT0 from FIFO B0 1	0x11
	IOC_OUTPUT_CFG_OUT0_FB0_2	Load OUT0 from FIFO B0 2	0x12
	IOC_OUTPUT_CFG_OUT0_FB0_3	Load OUT0 from FIFO B0 3	0x13

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	IOC_OUTPUT_CFG_OUT0_FB0_4	Load OUT0 from FIFO B0 4	0x14
	IOC_OUTPUT_CFG_OUT0_FB0_5	Load OUT0 from FIFO B0 5	0x15
	IOC_OUTPUT_CFG_OUT0_FB0_6	Load OUT0 from FIFO B0 6	0x16
	IOC_OUTPUT_CFG_OUT0_FB0_7	Load OUT0 from FIFO B0 7	0x17
	IOC_OUTPUT_CFG_OUT0_FB1_0	Load OUT0 from FIFO B1 0	0x18
	IOC_OUTPUT_CFG_OUT0_FB1_1	Load OUT0 from FIFO B1 1	0x19
	IOC_OUTPUT_CFG_OUT0_FB1_2	Load OUT0 from FIFO B1 2	0x1A
	IOC_OUTPUT_CFG_OUT0_FB1_3	Load OUT0 from FIFO B1 3	0x1B
	IOC_OUTPUT_CFG_OUT0_FB1_4	Load OUT0 from FIFO B1 4	0x1C
	IOC_OUTPUT_CFG_OUT0_FB1_5	Load OUT0 from FIFO B1 5	0x1D
	IOC_OUTPUT_CFG_OUT0_FB1_6	Load OUT0 from FIFO B1 6	0x1E
	IOC_OUTPUT_CFG_OUT0_FB1_7	Load OUT0 from FIFO B1 7	0x1F
	IOC_OUTPUT_CFG_OUT0_PCM_RX0	Load OUT0 from PCM RX0	0x20
	IOC_OUTPUT_CFG_OUT0_PCM_RX1	Load OUT0 from PCM RX1	0x21
	IOC_OUTPUT_CFG_OUT0_IN0	Load OUT0 from IN0	0x22
	IOC_OUTPUT_CFG_OUT0_IN1	Load OUT0 from IN1	0x23
	IOC_OUTPUT_CFG_OUT0_IN2	Load OUT0 from IN2	0x24
	IOC_OUTPUT_CFG_OUT0_IN3	Load OUT0 from IN3	0x25
	IOC_OUTPUT_CFG_OUT0_LOAD	Load OUT0 from the OUT0 load register	0x26
	IOC_OUTPUT_CFG_OUT0_NONE	Do not load (load 0 instead)	0x3F*

13.5.1.2 D_IOC*_OUT0_DATA, IOC*_OUT0_DATA

Bit Field	Read/Write	Field Name	Description
23:0	W	OUT_DATA	Input sample for output channel 0 (m1p23 format)

13.5.1.3 D_IOC*_OUT1_DATA, IOC*_OUT1_DATA

Bit Field	Read/Write	Field Name	Description
23:0	W	OUT_DATA	Input sample for output channel 1 (m1p23 format)

Ezairo 8300 Hardware Reference

13.6 IOC DATA TRANSFER TIMING

The IOCs coordinate the sequential movement of data to and from the input and output components. In addition, the IOCs create a fixed ordering of data transfers to and from the FIFOs, resulting in the associated FIFO interrupts being generated in the same fixed order.

IMPORTANT: To provide a fixed ordering of data reception in the FIFOs (and consequently a fixed ordering of FIFO interrupts), the IOCs must not be providing data to, or requesting data from, any FIFO while that FIFO is being configured.

If the PCM interface shares a clock source with the other input and output components, interrupts generated by IOC data transfers between the PCM interface and the FIFOs have a guaranteed timing relative to the other FIFO interrupts (as described in [Section 13.4 “IOC Input” on page 413](#), and [Section 13.5 “IOC Output” on page 427](#)). If the clock source is not shared in this way, the relative timing of interrupts generated for PCM interface data to other FIFO interrupts cannot be guaranteed.

IMPORTANT: The sampling rate of the PCM interface must be identical to the sampling rates of the input and output stages if IOC accesses to the FIFOs are always performed in a stable order.

The timing of software FIFO interrupts depends on the application using the software FIFO and is not controlled by the IOCs.

The ordering and relative timing of IOC data transfers to and from the FIFOs (when the PCM interface is synchronized with the input and output stages) is found in the ["Relative IOC Data Transfer Timing Between FIFOs and System Components" table \(Table 42\)](#), below. The timing of the FIFO interrupts is based on this ordering and relative timing, and on the configuration of each FIFO's block size and pointers.

Table 42. Relative IOC Data Transfer Timing Between FIFOs and System Components

Ordering	Components	Relative Timing
1	IN0, IN2	Data is transferred from ADC0 and ADC2 (or corresponding Filter Engine outputs) at the beginning of each sample period.
2	OUT0, OUT1	Data is transferred after a delay of approximately 1/4 of the sampling period from the beginning of the sampling period.
3	IN1, IN3	Data is transferred after a delay of approximately 1/2 of the sampling period from the beginning of the sampling period.
4	PCM RX0, PCM RX1 and PCMTX0, PCM TX1	Data transfers due to PCM input data occur at the same time within a sample period as interrupts due to PCM output data requests at either of the following times, whichever happens later: • Immediately following the last enabled ADC or DAC • When data for the PCM interface arrives or is needed by the interface for transmission.

Ezairo 8300 Hardware Reference

This relative timing sequence is important for applications that process data synchronously, triggering their input and output processing on a single interrupt (typically a FIFO interrupt). In these situations, the application normally waits for the last interrupt to arrive before beginning to process the data. For example, consider the timing when a FIFO interrupt is generated for a block of data transferred from IN0. A block of data from IN1 that needs to be received and processed synchronously with this data would not be received completely under these circumstances, and a previous output result block for the output would not be transmitted completely.

To ensure that an IOC has completed transferring the current block of data for all of the relevant components, synchronize with the interrupt associated with the last scheduled data transfer. In this situation, a user application using IN0 for input and OUT0 for output uses the FIFO interrupt associated with OUT0 to synchronize processing. Similarly, if the same application instead uses IN1 and OUT0, the FIFO interrupt associated with IN1 is used. Writing to the registers `D_IOC*_INPUT_CFG`, `D_IOC*_PCM_CFG` or `D_IOC*_OUTPUT_CFG` will reset the associated IOC state machine to guarantee the FIFO order.

IMPORTANT: To ensure proper system behavior, if the PCM interface is not synchronized (i.e. does not share a clock source) with the input stage, use separate interrupts or events to trigger processing of input channel data and PCM data. Similarly, if the PCM interface does not share a clock source with the output stage, handle output channel data and PCM data separately.

CHAPTER 14

Input Stage

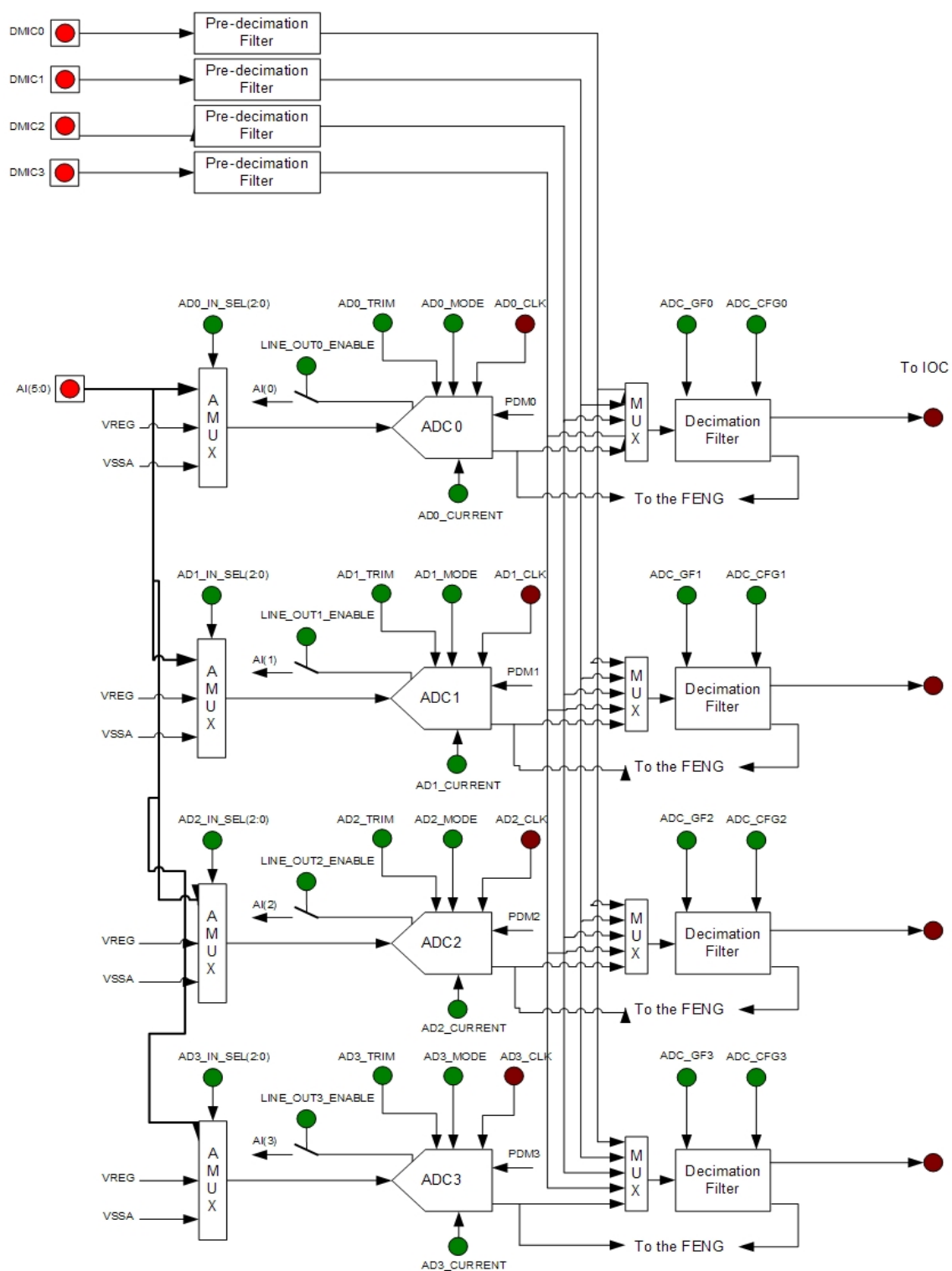
The audio input stage on is based on an ADC architecture that provides four input channels multiplexed from six analog inputs, with sufficient input dynamic range to not require an input preamplifier. Each of these input channels uses a number of components, including several filters and a 24-bit oversampling sigma-delta A/D converter, to provide audio input data to the IOCs for the rest of the system to use.

14.1 OVERVIEW

The input stage of an Ezairo 8300 provides four audio input channels that supply signal data to the rest of the Ezairo 8300 system. Each input channel includes:

- Input selection using an analog multiplexer to select between the input source from the available input sources and reference inputs
- An over-sampling analog-to-digital converter (ADC) which uses a programmable sampling frequency and provides a configurable sampling delay
- Selectable digital microphone inputs and bypass registers that can source data in place of the ADCs
- High-quality decimation filtering at all selectable sampling rates
- Selectable input muting

Each of these components are shown in the "Input Stage Block Diagram" figure (Figure 28), which shows the overall architecture of the input stage.



Ezairo 8300 Hardware Reference

The input data provided by the ADCs are multiplexed with the digital microphone inputs and bypass data registers, as described in [Section 13.1.2 “Audio Multiplexing of Audio Input Sources” on page 405](#). Signals from the input stage are then routed to the Filter Engine directly or through IOC0 to the rest of the system, as described in [Chapter 6.1 “Processor Overview” on page 217](#) and [Chapter 13.1 “Audio Multiplexing” on page 401](#) respectively.

14.2 INPUT SELECTION

The four input channels select their signal source from one of 8 sources using the `AUDIO_ADC_CTRL_AIN_SEL` bit field from the `AUDIO_ADC_CTRL` register. Sources can be selected from:

- Input pads (AI0 to AI5)
- The regulated power supply (VREG); for more information on this reference supply see [Section 9.1.2.3 “Regulated Supply Voltage \(VREG\)” on page 295](#)
- Analog ground (VSSA)

NOTE: Input pads AI4, AI5 are not currently routed to externally accessible pads.

The system also has the `AUDIO_ADC_CTRL_ADC_ENABLE` bits from the `AUDIO_ADC_CTRL_*` registers, which allow you to selectively enable or disable the receipt of data and components for each of the four input stage channels.

The ADC internal reference can be selected from VSSA, GNDMIC, VREG and VMIC. The selected reference must match the source reference to achieve low IRN:

- For a microphone that references VSSA, GNDMIC, or VMIC, the ADC reference must match the microphone reference.
- For a telecoil connected between VREG and the ADC input, select VREG as the reference.

The audio inputs are high-impedance inputs that do not require DC blocking decoupling capacitors. However, the inputs do require a bias voltage that adds a DC offset to the input signal. The DC bias is set by the external circuitry to provide a bias voltage within the minimum and maximum bias voltage limitations specified in the Ezairo 8300 datasheet.

NOTE: A typical microphone provides a bias voltage of 450 mV without any external components. A bias of 750 mV is recommended to achieve the maximum dynamic range for the system.

The audio input bias is controlled using the `AUDIO_ADC_CTRL_AIN_RBIAS_CONNECT` field in the `AUDIO_ADC_CTRL*` registers. This bit-field is used to select between the following options:

- `AUDIO_ADC_CTRL_AIN_RBIAS_CONNECT = 0x0`: ADC input high impedance (default)
- `AUDIO_ADC_CTRL_AIN_RBIAS_CONNECT = 0x1`: ADC input connected to VSSA through 1 M Ω
- `AUDIO_ADC_CTRL_AIN_RBIAS_CONNECT = 0x2`: ADC input connected to VREG through 1 M Ω
- `AUDIO_ADC_CTRL_AIN_RBIAS_CONNECT = 0x3`: ADC input connected to (VREG / 2) through 1 M Ω
- `AUDIO_ADC_CTRL_AIN_RBIAS_CONNECT = 0x4`: ADC input connected to VMIC through 1 M Ω
- `AUDIO_ADC_CTRL_AIN_RBIAS_CONNECT = 0x5`: ADC input connected to (VMIC / 2) through 1 M Ω

14.3 ANALOG-TO-DIGITAL CONVERTERS (ADCs)

The main components of the Ezairo 8300 analog input signal paths are the 4 multi-mode ADC converters.

Ezairo 8300 Hardware Reference

As shown in the Sigma Delta ADC modulator block diagram (the "ADC Modulator Architecture" figure (Figure 29)), the selected input signal is connected to the modulator V_{IN} input. The modulator is designed to provide a second order noise shaping function at low frequency and a first order low noise floor in the audio frequency region.

The ADC inputs are high-impedance inputs, as no net current flows through the input pad. The ADC input is combined with the output bit stream, using integrator that compares the input to the output from a feedback DAC that closely follows the input. The output bit-stream from the ADCs is provided to the decimation filters and Filter Engine.

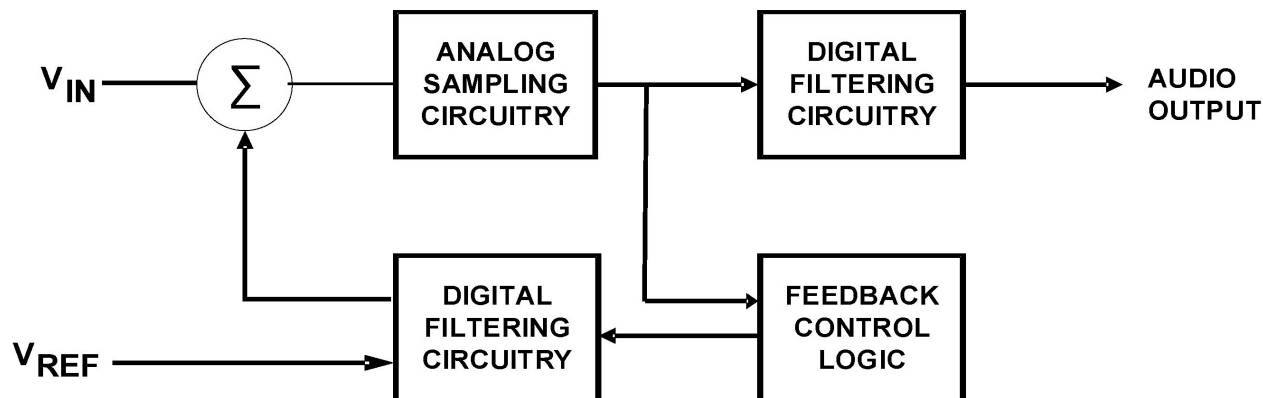


Figure 29. ADC Modulator Architecture

The 4 ADCs are configured using the `AUDIO_ADC_CTRL*` register. For more information on the ADC input connection, see [Section 14.2 “Input Selection” on page 440](#).

Each ADC is enabled individually, using the `AUDIO_ADC_CTRL_ADC_ENABLE` and `AUDIO_ADC_CTRL_COMP_ENABLE` fields in the `AUDIO_ADC_CTRL*` register.

The ADCs can operate in several modes. These modes are defined by the `AUDIO_ADC_CTRL_ADC_MODE` field in the `AUDIO_ADC_CTRL*` register. Modes of operation and their recommended settings are listed in the "ADC Modes of Operation" table (Table 43).

onsemi Confidential
Ezairo 8300 Hardware Reference

Table 43. ADC Modes of Operation

Mode	Mode Setting	Other Recommended Settings	Use Description
Normal	ADC_MODE	Modulator frequency (ADCCLK = 3.84 MHz) Nominal RIN (AUDIO_ADC_CFG_RIN_TRIM=ADC_RIN_4K57) ADC current normal mode (AUDIO_ADC_CFG_ADC_CURRENT = ADC_CURRENT_14P5UA) or low power mode (AUDIO_ADC_CFG_ADC_CURRENT=ADC_CURRENT_2P5UA) Comparator current normal mode (AUDIO_ADC_CFG_COMP_CURRENT = ADC_COMP_CURRENT_2UA) or low power mode (AUDIO_ADC_CFG_COMP_CURRENT=ADC_COMP_CURRENT_1UA) DC bias voltage range (around 0.6 V)	Good performance mode and low power, input range up to 1.2 V
Normal, Extended Range	ADC_MODE_MAX_1P6V	Same as Normal, above DC bias voltage range (around 0.8 V)	Normal mode with extended input range up to 1.6 V
Normal, Improved PSRR	ADC_MODE_HIPSRR	Same as Normal, above DC bias voltage (around 0.6 V)	Normal mode with VDDA regulator enabled for improved PSRR, input range up to 1.2 V
Normal, Improved PSRR, Extended Range	ADC_MODE_HIPSRR_MAX_1P6V	Same as Normal, above DC bias voltage (around 0.8 V)	Normal mode with VDDA regulator enabled for improved PSRR and extended input range up to 1.6 V

onsemi Confidential
Ezairo 8300 Hardware Reference

ADC Modes of Operation (Continued)

Mode	Mode Setting	Other Recommended Settings	Use Description
High Quality	ADC_MODE_HIQ	Modulator frequency (ADCCLK = 3.84 MHz) Increased RIN (AUDIO_ADC_CFG_RIN_TRIM = ADC_RIN_2K29) ADC current (AUDIO_ADC_CFG_ADC_CURRENT = ADC_CURRENT_25UA) Comparator current (AUDIO_ADC_CFG_COMP_CURRENT = ADC_COMP_CURRENT_2UA) DC bias voltage (around 0.6 V)	Higher performance mode, input range up to 1.2 V
High Quality, Extended Range	ADC_TEST_SW_HIQ_MAX_1P6V	Same as High Quality, above DC bias voltage (around 0.8 V)	Higher performance mode with extended input range up to 1.6 V
High Quality, Improved PSRR	ADC_MODE_HIQ_HIPSRR	Same as High Quality, above DC bias voltage (around 0.6 V)	Higher performance mode with VDDA regulator enabled for improved PSRR, input range up to 1.2 V
High Quality, Improved PSRR, Extended Range	ADC_MODE_HIQ_HIPSRR_MAX_1P6V	Same as High Quality, above DC bias voltage (around 0.8 V)	Higher performance mode with VDDA regulator enabled for improved PSRR and extended input range (up to 1.6 V)

Ezairo 8300 Hardware Reference

ADC Modes of Operation (Continued)

Mode	Mode Setting	Other Recommended Settings	Use Description
Signal Detection	ADC_MODE_SDU	Modulator frequency (ADCCLK = 1.92 MHz) Increased RIN (AUDIO_ADC_CFG_RIN_TRIM = ADC_RIN_8K) ADC current (AUDIO_ADC_CFG_ADC_CURRENT = ADC_CURRENT_2P5UA) Comparator current (AUDIO_ADC_CFG_COMP_CURRENT = ADC_COMP_CURRENT_1UA) DC bias voltage (around 0.45V)	Lowest power mode with reduced performance useful for signal detection, input range up to 0.8 V
DAC	ADC_MODE_DAC		In DAC mode, the feedback circuit is entirely controlled from the digital partside (gain and quantizer disabled), and the input OPAMP is configured to buffer the non-inverting input signal.
DAC, Improved PSRR	ADC_MODE_DAC_HIPSR		Same as DAC mode with VDDA regulator enabled for improved PSRR

The AUDIO_ADC_CFG_DAC_MIN_DRIVE bit field must always be set to ADC_FBDAC_MIN_DRIVE_1.

The AUDIO_ADC_CFG_DAC_MAX_DRIVE bit field must always be set to ADC_FBDAC_MAX_DRIVE_64.

The AUDIO_ADC_CFG_DEM_ENABLE bit field must always be set to ADC_FBDAC_DEM_DISABLE.

IMPORTANT: To minimize power consumption, use the ADC_MODE or ADC_MODE_MAX_1P6V mode with minimal ADC and comparator currents. To reduce the current consumption even further, use ADCCLK = 1.92 MHz and AUDIO_ADC_CFG_RIN_TRIM = ADC_RIN_8K.

To minimize noise due to differences in the feedback path within an ADC, offset trim parameters are provided for the ADCs. These ADC offsets can be calibrated using the calibration library. If not calibrated, the optimal values for these settings (which do not match the default values) are:

- AUDIO_ADC_OFFSET_TRIM0 = 0x028014
- AUDIO_ADC_OFFSET_TRIM1 = 0x0A0082
- AUDIO_ADC_OFFSET_TRIM2 = 0x0820A0

14.4 DECIMATION FILTERING

To obtain a final non-oversampled stream of input data, the Ezairo 8300 device uses a set of wave digital filters (WDF) as decimation filters that downsample and filter the oversampled signal to limit aliasing of signals from outside of the audio band. This filter shares components with the interpolation filtering block of the output stage

Ezairo 8300 Hardware Reference

and uses three low-pass wave digital filtering stages (5th, 7th, and 11th order). The source for each decimation filter selects from the following source options for each input channel using the `AUDIO_MUX_CFG_INPUT_CH*_SRC` bit fields from the `AUDIO_MUX_CFG` register:

- ADC
- Digital microphone
- Bypass data register

The decimation filters are enabled by setting the appropriate `ADC_DEC_CTRL_ADC_DECIMATOR_EBL` bits in the `AUDIO_ADC_DEC_CTRL` registers.

NOTE: All four ADC inputs are sampled at the same time relative to their fractional delay setting, but inputs from the input audio channels is time multiplexed in the decimation filters into two pairs (channels 0, 1; channels 2, 3) resulting in longer processing delays for channels 1, 3.

During the decimation filtering step, a configurable gain factor is added to the signal from each of the input channels. This digital gain factor is used to compensate for the analog and digital gain captured through the ADC and decimation filter, and is applied at the output of the decimation filter. The `ADC_DEC_CTRL_ADC_GF` bit field from the `AUDIO_ADC_DEC_CTRL` register controls this gain factor, and must be adapted as a function of the `ADC_SAMPLE_FREQ_CFG_SFCR` bit field setting in the `AUDIO_ADC_SAMPLE_FREQ_CFG` register.

It is recommended that the gain value for each filter be set in such a way that the signal level, after downsampling and filtering, is the same as it was before downsampling and filtering (i.e., unity gain). These gain factors are useful to fine tune the gain realized in the input stage, and are commonly used to compensate for changes in sampling frequency, or to match the gain between microphones in a multiple-microphone system. To obtain this unity gain for the input stage, `ADC_DEC_CTRL_ADC_GF` must be set according to the following equation:

$$\text{ADC_DEC_CTRL_ADC_GF} = \frac{2^{(11 + \log_2(\text{ADC_SAMPLE_FREQ_CFG_SFCR}))}}{(\text{ADC_SAMPLE_FREQ_CFG_SFCR} + 1)}$$

The decimation filter output can also be temporarily muted by setting the `ADC_DEC_CTRL_MUTE` bit from the appropriate `AUDIO_ADC_DEC_CTRL` register. Muting the decimation filter in this way is useful in an application where an input channel is temporarily disabled, but you want to maintain the data ordering of the data in the input FIFOs. To use this feature:

1. Disable the decimation filter and mute the decimation filter output using the `ADC_DEC_CTRL_MUTE` and `ADC_DEC_CTRL_DECIMATOR_EBL` bits of the appropriate `AUDIO_ADC_DEC_CTRL` register.
2. Disable the associated ADC.

To re-enable the input channel:

1. Enable the ADC.
2. Enable the decimation filter (continuing to mute the output).
3. Delay at least 200 μ s to allow the decimation filter to stabilize.
4. Unmute the decimation filter output.

IMPORTANT: The `AUDIO_ADC_FDEC_CTRL` register contains a set of internal configurations for the decimation filters. To obtain the optimal configuration for these settings, use the `ADC_FDEC_OPTIMAL` bit setting.

Ezairo 8300 Hardware Reference

14.4.1 DC Removal Filters

The decimation filtering component of the input stage includes a high-pass filter for each channel designed to remove any DC signal component from the input signal belonging to each channel. The DC removal filters are based on first-order digital filter structures with programmable cut-off frequencies. To configure these filters, use the `ADC_DEC_CTRL_DCRM_CUT_OFF` bit field of the `AUDIO_ADC_DEC_CTRL` registers to select between cut-off frequencies of 5 Hz, 10 Hz, and 20 Hz, or to bypass the DC removal filter.

The filter cut-off frequency scales linearly with the sampling frequency, and cut-off frequencies for the filters are defined in terms of a 16 kHz sampling frequency. For sampling frequencies other than 16 kHz, scale the cut-off frequency along with any increase or decrease in sampling frequency. For example, the cut-off frequencies for a sampling frequency of 32 kHz are 10 Hz, 20 Hz and 40 Hz respectively.

14.5 CONFIGURATION REGISTERS

Register Name	Register Description	Address
<code>AUDIO_ADC_CFG</code>	Input configuration registers	P:0x09C0-P:0x09C3 0x40002700-0x4000270C
<code>AUDIO_ADC_CTRL</code>	Input control registers	P:0x09C4-P:0x09C7 0x40002710-0x4000271C
<code>AUDIO_ADC_FDEC_CTRL</code>	ADC first decimation configuration registers	P:0x09C8-P:0x09CB 0x40002720-0x4000272C
<code>AUDIO_ADC_DEC_CTRL</code>	Decimation configuration registers	P:0x09CC-P:0x09CF 0x40002730-0x4000273C
<code>AUDIO_ADC_SAMPLE_FREQ_CFG</code>	Sampling frequency configuration	P:0x09D0 0x40002740
<code>AUDIO_DMIC_OUTPUT_CFG</code>	Configuration registers for the DMIC outputs	P:0x09DC 0x40002770
<code>AUDIO_OUTPUT_DRIVER_CFG</code>	Output driver configuration registers	P:0x09E0 0x40002780
<code>AUDIO_OUTPUT_GAIN</code>	Interpolation filter output gain settings	P:0x09E2 0x40002788
<code>AUDIO_OUTPUT_LIMITER</code>	Audio output limiter at SDM input	P:0x09E3 0x4000278C

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
AUDIO_ADC_OFFSET_TRIM0	ADC offset trimming values for drive 1 and 2	P:0x09F4-P:0x09F7 0x400027D0-0x400027DC
AUDIO_ADC_OFFSET_TRIM1	ADC offset trimming values for drive 8 and 16	P:0x09F8-P:0x09FB 0x400027E0-0x400027EC
AUDIO_ADC_OFFSET_TRIM2	ADC offset trimming values for drive 32 and 64	P:0x09FC-P:0x09FF 0x400027F0-0x400027FC

14.5.1 AUDIO_ADC_CFG*

Bit Field	Read/Write	Field Name	Description
23	RW	DEM_ENABLE	Enables dynamic element matching of the feedback DAC
22:20	RW	DAC_MAX_DRIVE	Feedback DAC maximum drive (powers of 2)
18:16	RW	DAC_MIN_DRIVE	Feedback DAC minimum drive (powers of 2)
11:8	RW	RIN_TRIM	Trimming of the integrator input resistor (number of parallel 32 kOhm resistors)
5:4	RW	COMP_CURRENT	Trimming of the ADC comparator current (1.0 uA + 0.5 uA steps on VDDA)
3:0	RW	ADC_CURRENT	Trimming of the ADC integrator current (2.5 uA + 1.5 uA steps on VDDA)

Field Name	Value Symbol	Value Description	Hex Value
DEM_ENABLE	ADC_FBDAC_DEM_DISABLE	DEM disabled	0x0
	ADC_FBDAC_DEM_ENABLE	DEM enabled	0x1*
DAC_MAX_DRIVE	ADC_FBDAC_MAX_DRIVE_0	Maximum feedback DAC drive of 0	0x0
	ADC_FBDAC_MAX_DRIVE_1	Maximum feedback DAC drive of 1	0x1
	ADC_FBDAC_MAX_DRIVE_64	Maximum feedback DAC drive of 64	0x7*
DAC_MIN_DRIVE	ADC_FBDAC_MIN_DRIVE_0	Minimum feedback DAC drive of 0	0x0
	ADC_FBDAC_MIN_DRIVE_1	Minimum feedback DAC drive of 1	0x1*
	ADC_FBDAC_MIN_DRIVE_64	Minimum feedback DAC drive of 64	0x7
RIN_TRIM	ADC_RIN_HIZ	RIN disconnected (DAC mode)	0x0
	ADC_RIN_8K	RIN set to 8 kOhm (SDU mode)	0x4
	ADC_RIN_4K57	RIN set to 4.57 kOhm (nominal value)	0x7*
	ADC_RIN_2K29	RIN set to 2.29 kOhm (high quality)	0xE

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		mode)	
COMP_CURRENT	ADC_COMP_CURRENT_1UA	Minimum current (low power mode)	0x0
	ADC_COMP_CURRENT_2UA	Nominal current	0x2*
ADC_CURRENT	ADC_CURRENT_2P5UA	Minimum current (low power mode)	0x0
	ADC_CURRENT_14P5UA	Nominal current	0x8*
	ADC_CURRENT_25UA	Maximum current (high quality mode)	0xF

14.5.2 AUDIO_ADC_CTRL*

Bit Field	Read/Write	Field Name	Description
30:28	RW	AIN_RBIAS_CONNECT	Analog input biasing connections (1 MOhm resistor)
27:16	RW	ADC_MODE	ADC operation mode
13:12	RW	ADC_REFERENCE_SEL	ADC reference selection
11:10	RW	DAC_SRC	DAC mode source selection
8	RW	DAC_ENABLE	Enables DAC mode by driving the feedback DAC
6:4	RW	AIN_SEL	Select the input applied to ADC
1	RW	COMP_ENABLE	Enables ADC comparator (ADC must be enabled)

Field Name	Value Symbol	Value Description	Hex Value
AIN_RBIAS_CONNECT	ADC_AIN_HIZ	ADC input high impedance (for use without external coupling capacitor)	0x0*
	ADC_AIN_RBIAS_VSSA	ADC input connected to VSSA through 1 MOhm	0x1
	ADC_AIN_RBIAS_VREG	ADC input connected to VREG through 1 MOhm (for use with external coupling capacitor)	0x2
	ADC_AIN_RBIAS_VREG_DIV2	ADC input connected to VREG/2 through 2x1 MOhm resistive divider (for use with external coupling capacitor)	0x3
	ADC_AIN_RBIAS_VMIC	ADC input connected to VMIC through 1 MOhm (for use with external coupling capacitor)	0x4
	ADC_AIN_RBIAS_VMIC_DIV2	ADC input connected to VMIC/2 through 2x1 MOhm resistive divider (for use with external coupling capacitor)	0x5
ADC_MODE	ADC_MODE	ADC mode configuration (input range	0x80

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		limited to 1.2 V)	
	ADC_MODE_MAX_1P6V	ADC mode configuration with input range extended to 1.6 V (at the expense of a small increase in harmonic distortion)	0xA0
	ADC_MODE_HIPSR	ADC mode configuration with VDDA regulator enabled for improved PSRR (input range limited to 1.2 V)	0x480
	ADC_MODE_HIPSR_MAX_1P6V	ADC mode configuration with VDDA regulator enabled for improved PSRR and input range extended to 1.6 V (at the expense of a small increase in harmonic distortion)	0x4A0
	ADC_MODE_HIQ	HiQ ADC mode configuration (input range limited to 1.2 V)	0x280
	ADC_MODE_HIQ_MAX_1P6V	HiQ ADC mode configuration with input range extended to 1.6 V (at the expense of a small increase in harmonic distortion)	0x2A0
	ADC_MODE_HIQ_HIPSR	HiQ ADC mode configuration with VDDA regulator enabled for improved PSRR (input range limited to 1.2 V)	0x680
	ADC_MODE_HIQ_HIPSR_MAX_1P6V	HiQ ADC mode configuration with VDDA regulator enabled for improved PSRR and input range extended to 1.6 V (at the expense of a small increase in harmonic distortion)	0x6A0
	ADC_MODE_SD	SDU ADC mode configuration (input range limited to 0.8 V)	0x800
	ADC_MODE_DAC	DAC mode configuration	0x12C
	ADC_MODE_DAC_HIPSR	DAC mode configuration with VDDA regulator enabled for improved PSRR	0x52C
ADC_REFERENCE_SEL	ADC_REF_VSSA	ADC reference connected to VSSA	0x0*
	ADC_REF_GNDMIC	ADC reference connected to GNDMIC	0x1
	ADC_REF_VREG	ADC reference connected to VREG	0x2
	ADC_REF_VMIC	ADC reference connected to VMIC	0x3
DAC_SRC	ADC_FBDAC_SRC_INT_FILT0	FBDAC driven from interpolator filter 0	0x0*
	ADC_FBDAC_SRC_INT_FILT1	FBDAC driven from interpolator filter 1	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	ADC_FBDAC_SRC_FENG_OUT10	FBDAC driven from FENG out10	0x2
	ADC_FBDAC_SRC_FENG_OUT11	FBDAC driven from FENG out11	0x3
DAC_ENABLE	ADC_FBDAC_INTERNAL	FBDAC driven from modulator	0x0*
	ADC_FBDAC_EXTERNAL	FBDAC driven from digital	0x1
AIN_SEL	ADC_SEL_AI0	AI0: Analog Input 0	0x0*
	ADC_SEL_AI1	AI1: Analog Input 1	0x1
	ADC_SEL_AI2	AI2: Analog Input 2	0x2
	ADC_SEL_AI3	AI3: Analog Input 3	0x3
	ADC_SEL_AI4	AI4: Analog Input 4	0x4
	ADC_SEL_AI5	AI5: Analog Input 5	0x5
	ADC_SEL_VREG	VREG: Input shorted to VREG (for offset calibration)	0x6
	ADC_SEL_VSSA	VSSA: Input shorted to VSSA (for gain calibration)	0x7

14.5.3 AUDIO_ADC_FDEC_CTRL*

Bit Field	Read/Write	Field Name	Description
31:0	RW	FDEC_CTRL	First decimation control

Field Name	Value Symbol	Value Description	Hex Value
FDEC_CTRL	ADC_FDEC_OPTIMAL	Optimal first decimation setting	0x51027F5

14.5.4 AUDIO_ADC_DEC_CTRL*

Bit Field	Read/Write	Field Name	Description
31	RW	MUTE	Mute output and enable data ready signal
30:28	RW	BAND_SELECT_ADC	Indicated which band is filtered in the decimation filter
26:24	RW	DELAY_INTEGER	Integer sample delay
21:16	RW	DELAY_FRACTIONAL	Fractional sample delay (valid values are from 0 to AUDIO_ADC_SAMPLE_FREQ_CFG:SFCR)
15	RW	DECIMATOR_EBL	Enables ADC decimation
14:12	RW	DCRM_CUT_OFF	Control the DC removal filter cut-off frequency of channel
11:0	RW	ADC_GF	Gain factor for ADC = $\text{round}[2^{(11+\text{floor}[\log_2(\text{sfcR})])}/(\text{sfcR}+1)]$

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
MUTE	ADC_UNMUTE	ADC output is not muted	0x0*
	ADC_MUTE	ADC output is muted and data ready enabled	0x1
BAND_SELECT_ADC	BAND_SELECT_ADC_0K_8K	Select the 0 to 8 kHz band (at Fs 16 kHz)	0x0*
	BAND_SELECT_ADC_8K_16K	Select the 8 to 16 kHz band (at Fs 16 kHz)	0x1
	BAND_SELECT_ADC_16K_24K	Select the 16 to 24 kHz band (at Fs 16 kHz)	0x2
	BAND_SELECT_ADC_24K_32K	Select the 24 to 32 kHz band (at Fs 16 kHz)	0x3
	BAND_SELECT_ADC_32K_40K	Select the 32 to 40 kHz band (at Fs 16 kHz)	0x4
	BAND_SELECT_ADC_40K_48K	Select the 40 to 48 kHz band (at Fs 16 kHz)	0x5
	BAND_SELECT_ADC_48K_56K	Select the 48 to 56 kHz band (at Fs 16 kHz)	0x6
	BAND_SELECT_ADC_56K_64K	Select the 56 to 64 kHz band (at Fs 16 kHz)	0x7
DELAY_INTEGER	ADC_INTEGER_DELAY_0	No integer sampling delay	0x0*
	ADC_INTEGER_DELAY_1	Delay the sampling by 1/8 of the sampling frequency	0x1
	ADC_INTEGER_DELAY_2	Delay the sampling by 2/8 of the sampling frequency	0x2
	ADC_INTEGER_DELAY_3	Delay the sampling by 3/8 of the sampling frequency	0x3
	ADC_INTEGER_DELAY_4	Delay the sampling by 4/8 of the sampling frequency	0x4
	ADC_INTEGER_DELAY_5	Delay the sampling by 5/8 of the sampling frequency	0x5
	ADC_INTEGER_DELAY_6	Delay the sampling by 6/8 of the sampling frequency	0x6
	ADC_INTEGER_DELAY_7	Delay the sampling by 7/8 of the sampling frequency	0x7
DECIMATOR_EBL	ADC_DEC_DISABLE	ADC decimation disabled	0x0*
	ADC_DEC_ENABLE	ADC decimation enabled	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DCRM_CUT_OFF	ADC_DC_REMOVE_CUTOFF_5HZ	Cut-off frequency at 5 Hz (at Fs 16 kHz)	0x0
	ADC_DC_REMOVE_CUTOFF_10HZ	Cut-off frequency at 10 Hz (at Fs 16 kHz)	0x1
	ADC_DC_REMOVE_CUTOFF_20HZ	Cut-off frequency at 20 Hz (at Fs 16 kHz)	0x2*
	ADC_DC_REMOVE_CUTOFF_40HZ	Cut-off frequency at 40 Hz (at Fs 16 kHz)	0x3
	ADC_DC_REMOVE_CUTOFF_80HZ	Cut-off frequency at 80 Hz (at Fs 16 kHz)	0x4
	ADC_DC_REMOVE_CUTOFF_160HZ	Cut-off frequency at 160 Hz (at Fs 16 kHz)	0x5
	ADC_DC_REMOVE_CUTOFF_BYPASS	No DC remove	0x6

14.5.5 AUDIO_ADC_SAMPLE_FREQ_CFG

Bit Field	Read/Write	Field Name	Description
5:0	RW	SFCR	Modulator to WDF input division factor ($SFCR = \text{round}(f_{\text{adcdk}}/8/f_{\text{sample}}) - 1$). Division factors lower than 8 ($SFCR < 7$) are not valid (not enough cycles for WDF filter)

Field Name	Value Symbol	Value Description	Hex Value
SFCR	ADC_MODDIV_BY8	Division by 8	0x7
	ADC_MODDIV_BY10	Division by 10	0x9
	ADC_MODDIV_BY15	Division by 15	0xE
	ADC_MODDIV_BY20	Division by 20	0x13
	ADC_MODDIV_BY24	Division by 24	0x17
	ADC_MODDIV_BY30	Division by 30	0x1D
	ADC_MODDIV_BY40	Division by 40	0x27
	ADC_MODDIV_BY60	Division by 60	0x3B*
	ADC_MODDIV_BY64	Division by 64	0x3F

Ezairo 8300 Hardware Reference

14.5.6 AUDIO_DMIC_OUTPUT_CFG

Bit Field	Read/Write	Field Name	Description
7	RW	DMIC1_HIGH_SRC	Selects the source of the DMIC1 high data channel
6	RW	DMIC1_LOW_SRC	Selects the source of the DMIC1 low data channel
5	RW	DMIC0_HIGH_SRC	Selects the source of the DMIC0 high data channel
4	RW	DMIC0_LOW_SRC	Selects the source of the DMIC0 low data channel
3	RW	DMIC1_HIGH_ENABLE	Enables/disables the DMIC1 high data output
2	RW	DMIC1_LOW_ENABLE	Enables/disables the DMIC1 low data output
1	RW	DMIC0_HIGH_ENABLE	Enables/disables the DMIC0 high data output
0	RW	DMIC0_LOW_ENABLE	Enables/disables the DMIC0 low data output

Field Name	Value Symbol	Value Description	Hex Value
DMIC1_HIGH_SRC	DMIC1_HIGH_SRC_CH0	Output channel 0	0x0*
	DMIC1_HIGH_SRC_CH1	Output channel 1	0x1
DMIC1_LOW_SRC	DMIC1_LOW_SRC_CH0	Output channel 0	0x0*
	DMIC1_LOW_SRC_CH1	Output channel 1	0x1
DMIC0_HIGH_SRC	DMIC0_HIGH_SRC_CH0	Output channel 0	0x0*
	DMIC0_HIGH_SRC_CH1	Output channel 1	0x1
DMIC0_LOW_SRC	DMIC0_LOW_SRC_CH0	Output channel 0	0x0*
	DMIC0_LOW_SRC_CH1	Output channel 1	0x1
DMIC1_HIGH_ENABLE	DMIC1_HIGH_DISABLE	DMIC1 high output disabled	0x0*
	DMIC1_HIGH_ENABLE	DMIC1 high output enabled	0x1
DMIC1_LOW_ENABLE	DMIC1_LOW_DISABLE	DMIC1 low output disabled	0x0*
	DMIC1_LOW_ENABLE	DMIC1 low output enabled	0x1
DMIC0_HIGH_ENABLE	DMIC0_HIGH_DISABLE	DMIC0 high output disabled	0x0*
	DMIC0_HIGH_ENABLE	DMIC0 high output enabled	0x1
DMIC0_LOW_ENABLE	DMIC0_LOW_DISABLE	DMIC0 low output disabled	0x0*
	DMIC0_LOW_ENABLE	DMIC0 low output enabled	0x1

Ezairo 8300 Hardware Reference

14.5.7 AUDIO_OUTPUT_DRIVER_CFG

Bit Field	Read/Write	Field Name	Description
11	RW	OD1_CLK_SRC	Clock source for channel 1 re-synchronization
8	RW	OD1_ENABLE	Enable RCVR1
3	RW	OD0_CLK_SRC	Clock source for channel 0 re-synchronization
0	RW	OD0_ENABLE	Enable RCVR0

Field Name	Value Symbol	Value Description	Hex Value
OD1_CLK_SRC	OD1_CLK_SRC_CCO	Output driver uses clock from CCO (recommended setting)	0x0*
	OD1_CLK_SRC_SDMCLK	Output driver uses SDMCLK (only required when external clock is used)	0x1
OD1_ENABLE	OD1_DISABLE	Output driver disabled	0x0*
	OD1_ENABLE	Output driver enabled	0x1
OD0_CLK_SRC	OD0_CLK_SRC_CCO	Output driver uses clock from CCO (recommended setting)	0x0*
	OD0_CLK_SRC_SDMCLK	Output driver uses SDMCLK (only required when external clock is used)	0x1
OD0_ENABLE	OD0_DISABLE	Output driver disabled	0x0*
	OD0_ENABLE	Output driver enabled	0x1

14.5.8 AUDIO_OUTPUT_GAIN

Bit Field	Read/Write	Field Name	Description
27:16	RW	OUTPUT1_GAIN	Interpolation filter output 1 gain setting
11:0	RW	OUTPUT0_GAIN	Interpolation filter output 0 gain setting

14.5.9 AUDIO_OUTPUT_LIMITER

Bit Field	Read/Write	Field Name	Description
17	W	OUTPUT_LIMITER_STATUS_CLEAR	Clear OUTPUT_LIMITER_STATUS bit
16	R	OUTPUT_LIMITER_STATUS	Sticky status bit indicating that the audio output signal has been limited
15:0	RW	OUTPUT_LIMITER	Limit of the audio output signal at SDM input

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
OUTPUT_LIMITER_STATUS_CLEAR	OUTPUT_LIMITER_STATUS_CLEAR	Clear OUTPUT_LIMITER_STATUS bit	0x1
OUTPUT_LIMITER_STATUS	OUTPUT_LIMITER_NOT_TRIGGERED	Audio output signal has not been limited	0x0*
	OUTPUT_LIMITER_TRIGGERED	Audio output signal has been limited	0x1
OUTPUT_LIMITER	OUTPUT_LIMITER_OFF	No output limiter	0x0*
	OUTPUT_LIMITER_MINUS_18_DB	Output is clipped at -18 dB	0x2000
	OUTPUT_LIMITER_MINUS_15_DB	Output is clipped at -15 dB	0x2D86
	OUTPUT_LIMITER_MINUS_12_DB	Output is clipped at -12 dB	0x4000
	OUTPUT_LIMITER_MINUS_9_DB	Output is clipped at -9 dB	0x5AD5
	OUTPUT_LIMITER_MINUS_6_DB	Output is clipped at -6 dB	0x8000
	OUTPUT_LIMITER_MINUS_3_DB	Output is clipped at -3 dB	0xB53C

14.5.10 AUDIO_ADC_OFFSET_TRIM0*

Bit Field	Read/Write	Field Name	Description
23:12	RW	OFFSET_TRIM_DRIVE_4	12-bit signed data for the offset trimming of drive 4
11:0	RW	OFFSET_TRIM_DRIVE_2	12-bit signed data for the offset trimming of drive 2

14.5.11 AUDIO_ADC_OFFSET_TRIM1*

Bit Field	Read/Write	Field Name	Description
23:12	RW	OFFSET_TRIM_DRIVE_16	12-bit signed data for the offset trimming of drive 16
11:0	RW	OFFSET_TRIM_DRIVE_8	12-bit signed data for the offset trimming of drive 8

14.5.12 AUDIO_ADC_OFFSET_TRIM2*

Bit Field	Read/Write	Field Name	Description
23:12	RW	OFFSET_TRIM_DRIVE_64	12-bit signed data for the offset trimming of drive 64
11:0	RW	OFFSET_TRIM_DRIVE_32	12-bit signed data for the offset trimming of drive 32

CHAPTER 15

Output Stage

The audio output stage on Ezairo 8300 provides two output channels. Each output channel uses a fourth-order, 24-bit oversampling sigma-delta D/A converter as the source for a direct digital output, which can be used to drive zero-bias hearing aid receivers.

15.1 OVERVIEW

The output stage of Ezairo 8300 provides two audio output channels that post-process signal data from the rest of the Ezairo 8300 system, and provide it to external receivers or speakers. The output channels include:

- High-quality interpolation filtering that automatically tracks the selected sample rate of the sigma-delta modulator
- An ultra low-power, high fidelity, over-sampled sigma-delta modulator with programmable sample rate
- A direct digital output driver for zero-bias hearing aid or headset receivers
- Selectable digital microphone outputs that can sink data in parallel with either output channel

Each of these components of the output channel is shown in the "Output Stage Block Diagram" figure (Figure 30), which shows the overall architecture of the output stage.

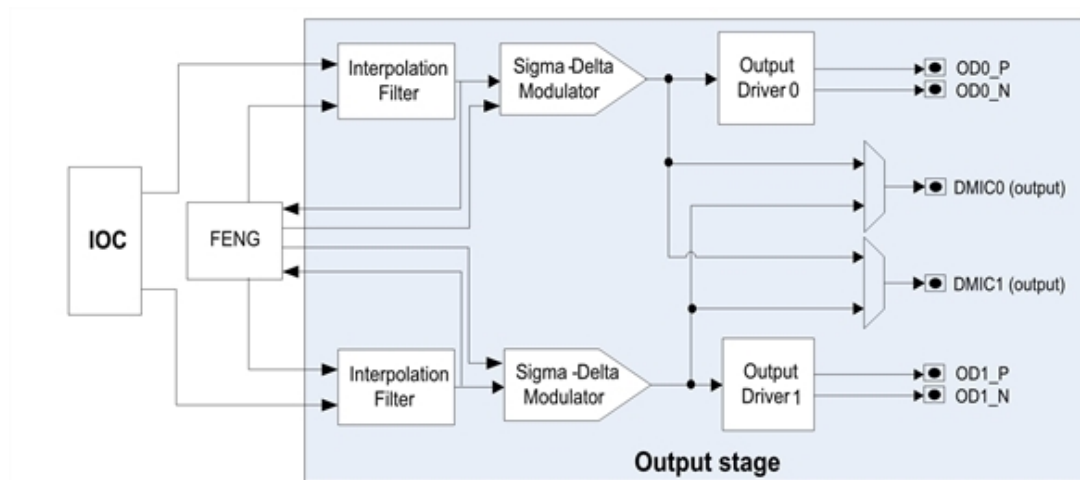


Figure 30. Output Stage Block Diagram

15.2 INTERPOLATION FILTERING

The output stage contains an interpolation filter for each output channel which upsamples and filters the signal that is being passed to the channel's sigma-delta modulator. This filter shares components with the decimation filtering block of the input stage and uses three low-pass wave digital filtering stages (11th, 7th, and 5th order). As a result of this link to the decimation filters, the interpolation filters automatically track the selected sample rate and upsampled frequency of the ADCs. For more information about configuring the sampling rate of the ADCs (and hence the decimation filters upsampled frequency), see [Section 14.3 “Analog-to-Digital Converters \(ADCs\)” on page 440](#).

Ezairo 8300 Hardware Reference

The interpolation filter component of each output channel provides the ability to apply a gain to the output signals. The amount of gain applied by the interpolation filters is configured using the `AUDIO_OUTPUT_GAIN_OUTPUT*_GAIN` bit fields in the `AUDIO_OUTPUT_GAIN` register. These bit fields use 12-bit signed values, with 2 integer bits and 10 fractional bits. This can be used to configure a gain in the range -2 to $(2 - 2^{-10})$. The default setting of 0x400 for this bit field corresponds to the application of no gain across the interpolation filter.

The output data directed to each interpolation filter is controlled by IOC0 and its audio multiplexers. For more information, see [Section 13.5 “IOC Output” on page 427](#).

15.3 SIGMA-DELTA MODULATOR

The sigma-delta modulator of each output channel produces a pulse density modulated (PDM) output signal from the oversampled signal data provided by either the interpolation filter or the Filter Engine. For information about selecting the data source used for the sigma-delta modulators, see [Section 13.1.1 “Audio Multiplexing with Filter Engine Outputs” on page 404](#).

The sigma-delta modulators are clocked using `SDMCLK`. For more information about this prescaled clock source, see [Section 10.2.2 “Sigma-Delta Modulator Clock \(SDMCLK\)” on page 314](#).

NOTE: Recommended `SDMCLK` frequencies between 3.84 MHz and 7.68 MHz if `AUDIO_SDM_CTRL_SDM_CLK_RATIO` is set, and between 7.68 MHz and 15.36 MHz otherwise. As the unmultiplied CCO frequency is typically 7.68 MHz or 15.36 MHz, typical `SDMCLK` frequencies are 3.84 MHz (with `AUDIO_SDM_CTRL_SDM_CLK_RATIO` set), 7.68 MHz, and 15.36 MHz.

IMPORTANT: To minimize power consumption and maximize audio quality, match `SDMCLK` to the unmultiplied CCO's frequency or an integer division of that oscillator's unmultiplied frequency. The optimal `SDMCLK` operating frequency for minimal power consumption depends on the load on the output stage. To minimize power consumption, `SDMCLK` should be set to 7.68 MHz for most receivers - although we recommend completing trials with the receiver used to confirm the optimal `SDMCLK` frequency needed to minimize power consumption for your selected receiver's output load. If your application prioritizes maximum audio quality, the `SDMCLK` frequency should be set to 15.36 MHz as audio quality improves for higher `SDMCLK` frequencies.

When the oversampled signal is received from the interpolation filters by each sigma-delta modulator, it is quantized to 1 bit by the modulator. This process shapes the noise of the output signal. As a result, the noise floor is no longer flat; instead, it is flat for a pass-band before the noise climbs at higher frequencies. The useful band is about 1/512 of the frequency of `SDMCLK` for XSDM mode and 1/128 of the frequency of `SDMCLK` for other modes.

The output signals from the sigma-delta modulator can create a periodic signal due to the quantization of the signal. To eliminate this possible issue, a dithering stage has been included at the output of the sigma-delta modulator to randomize the quantization error.

The sigma-delta modulator includes several advanced configuration options that can be used to adjust the behavior of the output modulator. Advanced configuration features of the sigma-delta modulators include:

Output Delay

The output from the sigma-delta modulator can be configured to provide an output to both the high and low sides of the digital output driver at the same time, or to delay one side by a cycle. Delaying one half of the output modulator signal relative to the other typically improves the noise

Ezairo 8300 Hardware Reference

performance of the system, and reduces the power consumption in the output stage. To configure the output delay, set or clear the `AUDIO_SDM_CTRL_OD_DELAY` bit from the `AUDIO_SDM_CTRL_*` register.

2-Level, 3-Level

The sigma-delta modulator output can be configured in 2-level (default) or 3-level using the `AUDIO_SDM_CTRL_TRI_STATE` bit in the `AUDIO_SDM_CTRL_*` registers. When using 3-level output configurations, the quantizer levels that corresponds to an output of zero (the third level) can be configured to select between $\pm 6 \cdot 2^{26}$ (default) and $\pm 5 \cdot 2^{26}$, using the `AUDIO_SDM_CTRL_QUANT_LEVEL` bit field in the `AUDIO_SDM_CTRL_*` register. For example, setting 0x6 outputs zero for values between $-6 \cdot 2^{26}$ and $+6 \cdot 2^{26}$.

XSDM Mode

This new extended sigma-delta modulation mode (XSDM) gives better performance for most receivers. The noise floor is lower than in traditional modes (e.g. by adding an output delay using `AUDIO_SDM_CTRL_OD_DELAY`) and the power consumption is heavily reduced. However, the maximum achievable $THD + N$ is degraded. Adding an offset to the audio signal (`AUDIO_SDM_OFFSET_CTRL_* = OFFSET_FOR_XSDM0_MODE`) register reduces harmonics in these new modes. The `AUDIO_SDM_CTRL_XSDM0` bit from the `AUDIO_SDM_CTRL` register is used to configure whether or not XSDM mode is used.

If the output stage is turned off during application, the offset has to be removed before turning off the output. Otherwise, there is an audible click in the output. There must also be no offset when enabling the output stage. The offset needs to be added or removed over a duration of roughly 500 ms (e.g., adding or subtracting `OFFSET_FOR_XSDM0_MODE/256` every 2 ms and doing this 256 times) when enabling or disabling the output stage.

In XSDM mode, the SDM has two stages working at different frequencies. The clock ratio between the stages can be set at 8 or at 4. The ratio of 4 needs to be selected for SDM clock frequencies below 7.68 MHz. The ratio between the main and second SDM clocks can be selected from 8 (default) to 4 by using the `AUDIO_SDM_CTRL_SDM_CLK_RATIO` bit in the `AUDIO_SDM_CTRL_*` register.

NOTE: The DCRM filters work on the lower clock frequency (when in XSDM mode). The cutoff frequency needs to be calculated based on the divided clock.

DC Removal

The sigma-delta modulator includes DC removal filters at the input (IDCRM) and output (DCRM) of the modulator. These filters are used to remove the effects of DC offsets due to quantization introduced in user applications and in the interpolation filter. To enable the DC removal filter, set the `AUDIO_SDM_DCRM_CTRL_IDCRM_ENABLE` and `AUDIO_SDM_DCRM_CTRL_DCRM_ENABLE` bits in the `AUDIO_SDM_DCRM_CTRL_*` register.

The cut-off frequency of the IDCRCM is roughly calculated by $f_{cut} = f_{SDMCLK} / 2^{(AUDIO_SDM_DCRM_CTRL_IDCRM_CUT_OFF/2 + 2.15)}$ where f_{SDMCLK} is the frequency of the main modulator (i.e. in XSDM mode the SDM clock frequency divided by 8). Set the desired value for this cut-off frequency in the `AUDIO_SDM_DCRM_CTRL_IDCRM_CUT_OFF` bit-field from the `AUDIO_SDM_DCRM_CTRL` register.

Ezairo 8300 Hardware Reference

The cut-off frequency of the DCRM is roughly calculated by $f_{cut} = f_{SDMCLK} * 2^{(AUDIO_SDM_DCRM_CTRL_DCRM_CUT_OFF/2 - 22.65)}$.

Output Offsets

The sigma-delta modulator can introduce a DC offset to the input signal after DC removal. This offset is divided into two parts: a small offset and a large offset.

The small offset can be used to correct for a known DC offset by adding an additional offset. This can be used with the DC removal filter, but is more commonly used in place of the DC removal filter.

The small offset is configured using the `AUDIO_SDM_OFFSET_CTRL_S_OFFSET` bit field from the `AUDIO_SDM_OFFSET_CTRL_*` register. A DC offset of LSBs is added to the input signal prior to processing through the sigma-delta modulator.

The large offset can be used to shift the frequency of undesired periodic signals generated in the sigma-delta modulator out of the audio band.

The large offset is configured using the `AUDIO_SDM_OFFSET_CTRL_OFFSET` bit field from the `AUDIO_SDM_OFFSET_CTRL_*` register. A DC offset of LSBs is added to the input signal prior to processing through the sigma-delta modulator.

Output Driver Toggling

The toggling optimization bit, `AUDIO_SDM_CTRL_TOGGLE_OPT` in the `AUDIO_SDM_CTRL_*` register, activates a switching optimization (to reduce toggling of output pins, and as a result, to minimize power consumption in the output driver). The toggling optimization cannot be used in XSDM mode.

15.3.1 Output Limiter

The Ezairo 8300 output stage includes an optional audio output limiter that can be used to saturate the input of the sigma-delta modulator at a register-defined value. This limiter can be configured using the `OUTPUT_LIMITER` bit-field from the `AUDIO_OUTPUT_LIMITER` register, allowing a limit to be placed between -3 and -18 dB in three dB steps. If the limiter has affected the output signal since the status was last cleared, the `OUTPUT_LIMITER_STATUS` bit will be set. To clear the `OUTPUT_LIMITER_STATUS` bit, write a 1 to the `OUTPUT_LIMITER_STATUS_CLEAR` bit.

15.4 OUTPUT DRIVER

The output H-bridge stage provides a direct digital output driver that is capable of driving a zero-bias hearing aid receiver coil or a passive headset receiver, without the need for an external amplifier.

The output stage is designed to drive typical zero bias receivers without audible artifacts. The block is symmetric for the ODP and ODN outputs.

The output driver has a test mode of operation that can be enabled using the `AUDIO_OUTPUT_TEST_TEST_ENABLE` bit from the `AUDIO_OUTPUT_TEST_*` register. Bit field `AUDIO_OUTPUT_TEST_SINGLE_ENDED` from the same register allows selection between differential output mode (default) and single ended positive or negative output active only. When test mode is enabled the state of positive and negative audio outputs can be switched from LOW (default) to HIGH using the `AUDIO_OUTPUT_TEST_*` register's `AUDIO_OUTPUT_TEST_POS_STATE` and `AUDIO_OUTPUT_TEST_NEG_STATE` bits, respectively.

onsemi Confidential

Ezairo 8300 Hardware Reference

15.5 CONFIGURATION REGISTERS

Register Name	Register Description	Address
AUDIO_SDM_CTRL	SDM configuration	P:0x09E4-P:0x09E5 0x40002790-0x40002794
AUDIO_SDM_OFFSET_CTRL	SDM offset configuration	P:0x09E6-P:0x09E7 0x40002798-0x4000279C
AUDIO_OUTPUT_TEST	Audio output test configuration at SDM output	P:0x09EA-P:0x09EB 0x400027A8-0x400027AC
AUDIO_SDM_DCRM_CTRL	DC remove filters configuration	P:0x09EC-P:0x09ED 0x400027B0-0x400027B4

15.5.1 AUDIO_SDM_CTRL*

Bit Field	Read/Write	Field Name	Description
15:12	RW	QUANT_LEVEL	Quantizer level for 3-level output
9	RW	TOGGLE_OPT	Toggle optimization mode
8	RW	TRI_STATE	Activation of three level output
7	RW	SDM_CLK_RATIO	Ratio between main and second SDM clock
6	RW	XSDM0	Activation of extended output modulation mode 0
0	RW	OD_DELAY	Delay on the RCVR

Field Name	Value Symbol	Value Description	Hex Value
QUANT_LEVEL	QUANT_LEVEL_5	Quantization level is 5 * 2 ²⁶	0x5
	QUANT_LEVEL_6	Quantization level is 6 * 2 ²⁶	0x6*
TOGGLE_OPT	TOGGLE_OPT_DISABLE	Normal mode	0x0*
	TOGGLE_OPT_ENABLE	Toggling optimization (use "00" instead of "11")	0x1
TRI_STATE	SD_2_LEVEL	Two level output	0x0*
	SD_3_LEVEL	Three level output	0x1
SDM_CLK_RATIO	CLK_RATIO_8	Clock ratio is 8	0x0*
	CLK_RATIO_4	Clock ratio is 4	0x1
XSDM0	XSDM0_DISABLE	Extended output modulation mode 0 disabled	0x0

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	XSDM0_ENABLE	Extended output modulation mode 0 enabled	0x1*
OD_DELAY	OD_DELAY_DISABLE	Two-state output mode	0x0*
	OD_DELAY_ENABLE	Three-state output mode	0x1

15.5.2 AUDIO_SDM_OFFSET_CTRL*

Bit Field	Read/Write	Field Name	Description
23:3	RW	OFFSET	Offset added at SDM input
2:0	RW	S_OFFSET	Small offset added at SDM input

Field Name	Value Symbol	Value Description	Hex Value
OFFSET	OFFSET_M524288	Offset of -524288 added	0x80000
	OFFSET_M1	Offset of -1 added	0xFFFFF
	OFFSET_0	No offset added	0x0*
	OFFSET_1	Offset of 1 added	0x1
	OFFSET_FOR_XSDM0_MODE	Ideal offset for XSDM0 mode	0x38000
	OFFSET_524287	Offset of 524287 added	0x7FFFF
S_OFFSET	S_OFFSET_NONE	No small offset added	0x0*
	S_OFFSET_1_8	Offset of 1/8 added	0x1
	S_OFFSET_2_8	Offset of 2/8 added	0x2
	S_OFFSET_7_8	Offset of 7/8 added	0x7

15.5.3 AUDIO_OUTPUT_TEST*

Bit Field	Read/Write	Field Name	Description
2	RW	TEST_ENABLE	Test mode enable
1	RW	POS_STATE	Test state for positive audio output
0	RW	NEG_STATE	Test state for negative audio output

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
TEST_ENABLE	AUDIO_OUTPUT_TEST_DISABLE	Test mode disabled	0x0*
	AUDIO_OUTPUT_TEST_ENABLE	Test mode enabled (on both POS and NEG outputs)	0x1
POS_STATE	AUDIO_OUTPUT_POS_0	Drive POS pin low in test mode	0x0*
	AUDIO_OUTPUT_POS_1	Drive POS pin high in test mode	0x1
NEG_STATE	AUDIO_OUTPUT_NEG_0	Drive NEG pin low in test mode	0x0*
	AUDIO_OUTPUT_NEG_1	Drive NEG pin high in test mode	0x1

15.5.4 AUDIO_SDM_DCRM_CTRL*

Bit Field	Read/Write	Field Name	Description
12:8	RW	DCRM_CUT_OFF	DCRM cut-off frequency ($f_{cut} = f_{SDMCLK} * 2^{(DCRM_CUT_OFF/2 - 22.65)}$)
7	RW	DCRM_ENABLE	DC remove enable
6:1	RW	IDCRM_CUT_OFF	Input DCRM cut-off frequency ($f_{cut} = f_{SDMCLK} / 2^{(IDCRM_CUT_OFF/2 + 2.15)}$)
0	RW	IDCRM_ENABLE	Input DCRM enable

Field Name	Value Symbol	Value Description	Hex Value
DCRM_CUT_OFF	DC_REMOVE_FREQ_0		0x0
	DC_REMOVE_FREQ_15	DC remove frequency = 54Hz @ 1.92MHz SDMCLK	0xF*
	DC_REMOVE_FREQ_31		0x1F
DCRM_ENABLE	DC_REMOVE_DISABLE	DC not removed from the output	0x0*
	DC_REMOVE_ENABLE	DC removed from the output	0x1
IDCRM_CUT_OFF	IDC_REMOVE_FREQ_24	DC remove frequency = 56Hz @ 0.96MHz SDMCLK	0x18
	IDC_REMOVE_FREQ_26	DC remove frequency = 54Hz @ 1.92MHz SDMCLK	0x1A*
	IDC_REMOVE_FREQ_28	DC remove frequency = 27Hz @ 1.92MHz SDMCLK	0x1C
	IDC_REMOVE_FREQ_127		0x7F
IDCRM_ENABLE	IDC_REMOVE_DISABLE	DC not removed from the input	0x0
	IDC_REMOVE_ENABLE	DC removed from the input	0x1*

CHAPTER 16

Input/Output Pad Configuration

The Ezairo 8300 system contains 36 datadigital input/output (DIO) pads and 8 analog pads that can be configured:

- To support the external interfaces, output clocks, and other I/Os
- As general-purpose I/Os (GPIO)
- Analog input/output function

16.1 OVERVIEW

DIOs support all digital inputs and output functions that are not supported directly by a dedicated I/O. For more information about the functional configuration of DIO pads, see [Section 16.2 “Functional Configuration”](#) below.

Dedicated digital I/Os are supplied for the following pads which are not part of the DIO pad set:

- CFX and Arm Cortex-M3 I2C debug port pads (SCL and SDA, shared with the D_I2C general-purpose I²C interface) which can be used to download firmware and debug application on the various cores on Ezairo 8300. For more information, see [Chapter 17.1 “Debug”](#) on page 523.

NOTE: Additional pads can be configured as a parallel interface to this I²C interface using `DIO_D_I2C_PAD_CFG` register.

- A reset pad (NRESET); for more information, see [Section 9.3 “Resets”](#) on page 299.

Additionally, certain DIOs have special fixed functionality that is only available on those particular I/Os. These configurations include:

- DIOs 8:11 and 20:23 can be used as LSAD inputs. This functionality is not enabled by default. For more information see [Section 18.5 “Low-Speed Analog-to-Digital \(LSAD\)”](#) on page 630.
- DIOs 30:33 which are assigned to the Arm Cortex-M3 processor SWJ-DP and enabled by default. DIOs 32:33 are used as TDI and TDO respectively, and are optional if using the port in serial-wire mode. For more information see [Section 4.6.3 “Debug Controller”](#) on page 199.
- DIO 35 can be used as the EXTCLK input. This functionality is not enabled by default. For more information see [Section 10.1.3 “External Clock \(EXTCLK\)”](#) on page 312

For more information, see [Section 16.2.1 “Special Functional Configurations”](#) on page 468.

The DIO pads support a variety of physical configuration parameters that can be required to properly interface with external components, including:

- Pull-up and pull-down resistors
- Low-pass input filtering
- Output drive strength
- Enabling or disabling DIO functionality

A complete description of the physical configuration of DIOs can be found in [Section 16.3 “Physical Configuration”](#) on page 469.

All of the DIO pads are powered on one of the four dedicated pads for VDDO power supply, to facilitate communication at different digital logic voltages. For more information about this power supply and its configuration, see [Section 9.1.1.3 “Digital Output Supply Voltages \(VDDO1, VDDO2, VDDO3, and VDDO4\)”](#) on page 290.

16.2 FUNCTIONAL CONFIGURATION

The DIO pads can be configured using the `DIO_CFG_IO_MODE` bit field from the `DIO_CFG_*` registers:

- For a variety of digital output modes
- For a general-purpose digital input mode, with the input function configured by the `DIO_SRC_<interface>` registers. For several interfaces (I2C, I3C, SPI, and PCM), these registers have a numeric postfix to indicate which of the interfaces is being configured. Supported interfaces and registers are:

PCM

`DIO_SRC_PCM`

SPI

`DIO_SRC_SPI, DIO_SRC_SPI_IO`

UART

`DIO_SRC_UART`

I²C

`DIO_SRC_I2C`

I³C

`DIO_SRC_I3C`

Audio Sync

`DIO_SRC_SYNC_SLAVE`

NMI

`DIO_SRC_NMI`

DMIC

`DIO_SRC_DMIC_CLK`

`DIO_SRC_DMIC_DATA`

eMMC

`DIO_SRC_EMMC_0, DIO_SRC_EMMC_1, DIO_SRC_EMMC_2`

CLKDIV

`DIO_SRC_CLKDIV`

LPDSP32 JTAG

`DIO_SRC_DSP_JTAG`

The "DIO Multiplexed Functionality" table (Table 44) contains a list of the functional modes for which a DIO can be configured. Table 45 on page 467 contains a list of the input sources which a given DIO can be assigned to supply.

In addition to standard digital functional configuration, certain DIOs can be configured for special modes. These special modes are described in Section 16.2.1 "Special Functional Configurations" on page 468.

Ezairo 8300 Hardware Reference

CAUTION: While a DIO can be configured to be both an output and an input, it is the user application's responsibility to ensure that the DIO is not driving an output to a pad that is also being driven externally, with an output high that is driving an output which is driving low. If a DIO pad signal has two drivers with two different voltages, the physical values and inputs that are read from this pad are considered undefined.

Table 44. DIO Multiplexed Functionality

Mode	Setting	Description
0x0	DIO_MODE_DISABLE	Disabled
0x1	DIO_MODE_INPUT	Input mode
0x2	DIO_MODE_GPIO_IN	GPIO input mode
0x3	DIO_MODE_GPIO_OUT	GPIO output mode
0x4	DIO_MODE_USRCLK	USRCLK (user clock) signal
0x5	DIO_MODE_SLOWCLK	SLOWCLK (slow clock) signal
0x7	DIO_MODE_ADCCLK	ADCCLK (ADC clock) signal
0x8	DIO_MODE_SYSCLK	SYSCLK (system clock) signal
0x9	DIO_MODE_UART_TX	UART_TX interface signal
0xA	DIO_MODE_PCM0_SERO	PCM0_SERO interface signal
0xB	DIO_MODE_PCM0_FRAME	PCM0_FRAME interface signal
0xC	DIO_MODE_PCM1_SERO	PCM1_SERO interface signal
0xD	DIO_MODE_PCM1_FRAME	PCM1_FRAME interface signal
0xE	DIO_MODE_DMIC0	DMIC 0 interface signal
0xF	DIO_MODE_DMIC1	DMIC 1 interface signal
0x10	DIO_MODE_SPI0_IO0	SPI0_IO0 interface signal (master SERO)
0x11	DIO_MODE_SPI0_IO1	SPI0_IO1 interface signal (slave SERO)
0x12	DIO_MODE_SPI0_IO2	SPI0_IO2 interface signal
0x13	DIO_MODE_SPI0_IO3	SPI0_IO3 interface signal
0x14	DIO_MODE_SPI0_CS	SPI0_CS interface signal
0x15	DIO_MODE_SPI0_CLK	SPI0_CLK interface signal
0x16	DIO_MODE_SPI1_IO0	SPI1_IO0 interface signal (master SERO)
0x17	DIO_MODE_SPI1_IO1	SPI1_IO1 interface signal (slave SERO)
0x18	DIO_MODE_SPI1_IO2	SPI1_IO2 interface signal
0x19	DIO_MODE_SPI1_IO3	SPI1_IO3 interface signal
0x1A	DIO_MODE_SPI1_CS	SPI1_CS interface signal

onsemi Confidential

Ezairo 8300 Hardware Reference

DIO Multiplexed Functionality (Continued)

Mode	Setting	Description
0x1B	DIO_MODE_SPI1_CLK	SPI1_CLK interface signal
0x1C	DIO_MODE_I2C0_SCL	I2C0 SCL interface signal (open collector)
0x1D	DIO_MODE_I2C0_SDA	I2C0 SDA interface signal (open collector)
0x1E	DIO_MODE_I2C1_SCL	I2C1 SCL interface signal (open collector)
0x1F	DIO_MODE_I2C1_SDA	I2C1 SDA interface signal (open collector)
0x20	DIO_MODE_I3C0_SCL	I3C0 SCL interface signal (open collector)
0x21	DIO_MODE_I3C0_SDA	I3C0 SDA interface signal (open collector)
0x22	DIO_MODE_I3C1_SCL	I3C1 SCL interface signal (open collector)
0x23	DIO_MODE_I3C1_SDA	I3C1 SDA interface signal (open collector)
0x24	DIO_MODE_EMMC_CLK	eMMC Clock
0x25	DIO_MODE_EMMC_CMD	eMMC Command/Response
0x26	DIO_MODE_EMMC_DAT0	eMMC Data[0]
0x27	DIO_MODE_EMMC_DAT1	eMMC Data[1]
0x28	DIO_MODE_EMMC_DAT2	eMMC Data[2]
0x29	DIO_MODE_EMMC_DAT3	eMMC Data[3]
0x2A	DIO_MODE_EMMC_DAT4	eMMC Data[4]
0x2B	DIO_MODE_EMMC_DAT5	eMMC Data[5]
0x2C	DIO_MODE_EMMC_DAT6	eMMC Data[6]
0x2D	DIO_MODE_EMMC_DAT7	eMMC Data[7]
0x2E	DIO_MODE_SYNC_MASTER	SYNC_MASTER interface signal
0x2F	DIO_MODE_DSP_TDO	DSP-TDO interface signal
0x30	DIO_MODE_OSCCLK	OSCCLK signal
0x31	DIO_MODE_STANDBYCLK	STANDBYCLK signal
0x32	DIO_MODE_EXTCLK_DIV	EXTCLK divider signal

onsemi Confidential

Ezairo 8300 Hardware Reference

Table 45. DIO Input Functionality

Configuration Register	Signal	Description
DIO_SRC_SYNC_SLAVE	SYNC	SYNC_SLAVE input selection
DIO_SRC_PCM	SERI	PCM_SERI input selection
	FRAME	PCM_FRAME input selection
	CLK	PCM_CLK input selection
DIO_SRC_SPI	CS	SPI_CS input selection
	CLK	SPI_CLK input selection
DIO_SRC_SPI_IO	IO3	SPI_IO3 input selection
	IO2	SPI_IO2 input selection
	IO1	SPI_IO1 input selection (master SERI)
	IO0	SPI_IO0 input selection (slave SERI)
DIO_SRC_I3C	SDA	SDA input selection
	SCL	SCL input selection
DIO_SRC_I2C	SDA	SDA input selection
	SCL	SCL input selection
DIO_SRC_CLKDIV	CLKDIV1	DIO Interrupt Clock Divider 1 Input Selection
	CLKDIV0	DIO Interrupt Clock Divider 0 Input Selection
DIO_SRC_NMI	NMI_POLARITY	NMI polarity
	NMI	NMI input selection
DIO_SRC_EMMC_0	EMMC_DATA3	eMMC DATA3 input selection
	EMMC_DATA2	eMMC DATA2 input selection
	EMMC_DATA1	eMMC DATA1 input selection
	EMMC_DATA0	eMMC DATA0 input selection
DIO_SRC_EMMC_1	EMMC_DATA7	eMMC DATA7 input selection
	EMMC_DATA6	eMMC DATA6 input selection
	EMMC_DATA5	eMMC DATA5 input selection
	EMMC_DATA4	eMMC DATA4 input selection
DIO_SRC_EMMC_2	EMMC_CMD	eMMC CMD input selection
DIO_SRC_DMIC_CLK	CLK	DMIC clock input selection

Ezairo 8300 Hardware Reference

DIO Input Functionality (Continued)

Configuration Register	Signal	Description
DIO_SRC_DMIC_DATA	DMIC3	DMIC data input selection
	DMIC2	DMIC data input selection
	DMIC1	DMIC data input selection
	DMIC0	DMIC data input selection
DIO_SRC_DSP_JTAG	TDI	DSP_TDI input selection
	TMS	DSP_TMS input selection
	TCK	DSP_TCK input selection
DIO_SRC_UART	RX	UART_RX input selection

16.2.1 Special Functional Configurations

Ezairo 8300 contains two special functional configurations that affect a subset of the DIOs. These special functional configurations are described in the ["Special DIO Functional Configurations" table \(Table 46\)](#).

Table 46. Special DIO Functional Configurations

Mode	Affected DIOs	Description
LSAD Input	8, 9, 10, 11, 20, 21, 22, and/or 23	Eight of the DIOs support a special configuration as Low Speed Analog to Digital converter (LSAD) inputs. These DIOs can be configured independently from one another using the DIO_CFG_IO_MODE bit field from their respective DIO_CFG_* registers. NOTE: For a DIO that is used in LSAD mode, the pad must be disconnected from both the digital input sensor and digital output driver by setting the DIO_MODE_DISABLE bit in the corresponding DIO_CFG_* register. This is done to prevent any digital signal from entering the LSAD input amplifier. For DIOs 8 to 11 and 20 to 23, the DIOs mentioned in the column pad are connected to the analog input multiplexer. For all other DIOs, the pad is completely disconnected. For more information about analog inputs, see Chapter 14.1 "Overview" on page 438.
Arm Cortex-M3 Processor SWJ-DP JTAG	DIO30 - DIO33	On power up, these pins are assigned as a four-wire JTAG interface to the Arm Cortex-M3 processor. The SWJ-DP debug port interface uses the following pads to implement a 4- or 5-wire JTAG interface: - JTAG TDI pin on DIO32, and TDO pin on DIO33 - JTAG SW-TCK pin connected to DIO30, and SW-TMS pin connected to DIO31 Arm Cortex-M3 processor JTAG TDI and TDO pins can be disconnected from the Arm Cortex-M3 processor JTAG by clearing the CM3_JTAG_DATA_EN bit in the DIO_CM3_JTAG_PAD_CFG register. All JTAG pins can be disabled altogether, freeing them up for IO operation, by clearing the CM3_JTAG_EN bit in the DIO_CM3_JTAG_PAD_CFG register.
EXTCLK	35	This DIO can be configured as an input clock source for the system, selectable as the source for SYSCLK. For more information about this clock, see Section 10.1.3 "External Clock (EXTCLK)" on page 312.

Ezairo 8300 Hardware Reference

16.3 PHYSICAL CONFIGURATION

The Ezairo 8300 system includes physical configuration parameters for each DIO. These parameters are set using configuration bits from the appropriate `DIO_CFG_*` register.

If the DIO is configured as an input pad, it has the following configuration options:

- The `DIO_CFG_PULL_CTRL` bit field is used to configure the pad to use a pull-up or pull-down resistor. Options include:
 - No pull resistor
 - A weak pull-up resistor (250 k Ω)
 - A pull-up resistor (60 k Ω)
 - A pull-down resistor (60 k Ω)
 - A strong (1 k Ω) pull-up resistor
- The `DIO_CFG_LPF` bit enables or disables a low-pass filter that can be used to clean up the DIO's received input signal. This filter suppresses all glitches of 50 ns or less.

IMPORTANT: For input signals of 2 MHz or less, we recommend enabling the low-pass filters provided for DIO inputs in order to reduce high-frequency noise on signals received at the DIO pads. For signals using a frequency that exceeds 2 MHz, the DIO low-pass filters need to be disabled to keep them from attenuating the input signals.

In addition to the configurable physical parameters, all DIO pads contain Schmitt trigger inputs.

If the DIO is configured as an output pad, it has the following configuration option:

- The `DIO_CFG_DRIVE` bit-field allows you to select the drive strength for the DIO output. This drive strength is configurable as a power-of-two multiple of the nominal drive strength (1, 2, 4 or 8).

16.4 DIO REGISTERS

Register Name	Register Description	Address
<code>DIO_CFG</code>	Digital IO Configuration	P:0x1240-P:0x1263 0x40000900-0x4000098C
<code>DIO_CM3_JTAG_PAD_CFG</code>	JTAG / SW Pad Configuration Register	P:0x127B 0x400009EC
<code>DIO_CLKDIV_CFG</code>	DIO Clock Divider Configuration Register	P:0x127C 0x400009F0
<code>DIO_SRC_SYNC_SLAVE</code>	Slave sync interface input selection	P:0x1280 0x40000A00
<code>DIO_SRC_PCM</code>	PCM Input Selection	P:0x1282-P:0x1283 0x40000A08-0x40000A0C

onsemi Confidential

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
DIO_SRC_SPI	DIO_SRC_SPI[1:0]	P:0x1284-P:0x1285 0x40000A10-0x40000A14
DIO_SRC_SPI_IO	SPI[1:0] Interface IO Selection	P:0x1286-P:0x1287 0x40000A18-0x40000A1C
DIO_SRC_I3C	I3C[1:0] Input Selection	P:0x1288-P:0x1289 0x40000A20-0x40000A24
DIO_SRC_I2C	I2C[1:0] Input Selection	P:0x128A-P:0x128B 0x40000A28-0x40000A2C
DIO_SRC_CLKDIV	DIO Interrupt Clock Divider Input Selection	P:0x128C 0x40000A30
DIO_SRC_NMI	NMI Input Selection	P:0x128D 0x40000A34
DIO_SRC_EMMC_0	eMMC Data[3:0] Input Selection	P:0x128E 0x40000A38
DIO_SRC_EMMC_1	eMMC Data[7:4] Input Selection	P:0x128F 0x40000A3C
DIO_SRC_EMMC_2	eMMC CMD Input Selection	P:0x1290 0x40000A40
DIO_SRC_DMIC_CLK	DMIC Clock Input Selection Register	P:0x1291 0x40000A44
DIO_SRC_DMIC_DATA	DMIC Data Input Selection Register	P:0x1292 0x40000A48
DIO_SRC_DSP_JTAG	DSP JTAG Configuration Register	P:0x1293 0x40000A4C
DIO_SRC_UART	UART Interface Input Selection	P:0x1294 0x40000A50

Ezairo 8300 Hardware Reference

16.4.1 DIO_CFG*

Bit Field	Read/Write	Field Name	Description
13:12	RW	DRIVE	Drive strength configuration
11	RW	LPF	Low Pass Filter enable
10:8	RW	PULL_CTRL	Pull selection
5:0	RW	IO_MODE	IO mode selection

Field Name	Value Symbol	Value Description	Hex Value
DRIVE	DIO_1X_DRIVE	1x drive strength	0x0
	DIO_2X_DRIVE	2x drive strength	0x1
	DIO_4X_DRIVE	4x drive strength	0x2
	DIO_8X_DRIVE	8x drive strength	0x3*
LPF	DIO_LPF_DISABLE	Disable low pass filter	0x0*
	DIO_LPF_ENABLE	Enable low pass filter	0x1
PULL_CTRL	DIO_NO_PULL	No pull selected	0x0
	DIO_250K_PULL_UP	Weak 250 kOhm pull-up selected	0x1*
	DIO_60K_PULL_UP	Medium 60 kOhm pull-up selected	0x2
	DIO_1K_PULL_UP	Strong 1 kOhm pull-up selected	0x3
	DIO_60K_PULL_DOWN	Medium 60 kOhm pull-down selected	0x4
IO_MODE	DIO_MODE_DISABLE	Disabled	0x0*
	DIO_MODE_INPUT	Input mode	0x1
	DIO_MODE_GPIO_IN	GPIO input mode	0x2
	DIO_MODE_GPIO_OUT	GPIO output mode	0x3
	DIO_MODE_USRCLK	Output USRCLK (user clock) signal	0x4
	DIO_MODE_SLOWCLK	Output SLOWCLK (slow clock) signal	0x5
	DIO_MODE_ADCCLK	Output ADCCLK (ADC clock) signal	0x7
	DIO_MODE_SYSCCLK	Output SYSCCLK (system clock) signal	0x8
	DIO_MODE_UART0_TX	Output UART0_TX interface signal	0x9
	DIO_MODE_PCM0_SERO	Output PCM0_SERO interface signal	0xA
	DIO_MODE_PCM0_FRAME	Output PCM0_FRAME interface signal	0xB
	DIO_MODE_PCM1_SERO	Output PCM1_SERO interface signal	0xC
	DIO_MODE_PCM1_FRAME	Output PCM1_FRAME interface signal	0xD

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DIO_MODE_DMIC0	Output DMIC 0 interface signal	0xE
	DIO_MODE_DMIC1	Output DMIC 1 interface signal	0xF
	DIO_MODE_SPI0_IO0	Output SPI0_IO0 interface signal (master SERO)	0x10
	DIO_MODE_SPI0_IO1	Output SPI0_IO1 interface signal (slave SERO)	0x11
	DIO_MODE_SPI0_IO2	Output SPI0_IO2 interface signal	0x12
	DIO_MODE_SPI0_IO3	Output SPI0_IO3 interface signal	0x13
	DIO_MODE_SPI0_CS	Output SPI0_CS interface signal	0x14
	DIO_MODE_SPI0_CLK	Output SPI0_CLK interface signal	0x15
	DIO_MODE_SPI1_IO0	Output SPI1_IO0 interface signal (master SERO)	0x16
	DIO_MODE_SPI1_IO1	Output SPI1_IO1 interface signal (slave SERO)	0x17
	DIO_MODE_SPI1_IO2	Output SPI1_IO2 interface signal	0x18
	DIO_MODE_SPI1_IO3	Output SPI1_IO3 interface signal	0x19
	DIO_MODE_SPI1_CS	Output SPI1_CS interface signal	0x1A
	DIO_MODE_SPI1_CLK	Output SPI1_CLK interface signal	0x1B
	DIO_MODE_I2C0_SCL	Output I2C0_SCL interface signal (open collector)	0x1C
	DIO_MODE_I2C0_SDA	Output I2C0_SDA interface signal (open collector)	0x1D
	DIO_MODE_I2C1_SCL	Output I2C1_SCL interface signal (open collector)	0x1E
	DIO_MODE_I2C1_SDA	Output I2C1_SDA interface signal (open collector)	0x1F
	DIO_MODE_I3C0_SCL	Output I3C0_SCL interface signal (open collector)	0x20
	DIO_MODE_I3C0_SDA_FIX_PU	Output I3C0_SDA interface signal (open collector, use fixed pull as defined in PULL_CTRL)	0x21
	DIO_MODE_I3C0_SDA_DYN_PU	Output I3C0_SDA interface signal (open collector, force 1 kOhm pull-up dynamically)	0x22
	DIO_MODE_I3C1_SCL	Output I3C1_SCL interface signal (open collector)	0x23

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DIO_MODE_I3C1_SDA_FIX_PU	Output I3C1 SDA interface signal (open collector, use fixed pull as defined in PULL_CTRL)	0x24
	DIO_MODE_I3C1_SDA_DYN_PU	Output I3C0 SDA interface signal (open collector, force 1 kOhm pull-up dynamically)	0x25
	DIO_MODE_EMMC_CLK	Output eMMC Clock	0x26
	DIO_MODE_EMMC_CMD	Output eMMC Command/Response	0x27
	DIO_MODE_EMMC_DAT0	Output eMMC Data[0]	0x28
	DIO_MODE_EMMC_DAT1	Output eMMC Data[1]	0x29
	DIO_MODE_EMMC_DAT2	Output eMMC Data[2]	0x2A
	DIO_MODE_EMMC_DAT3	Output eMMC Data[3]	0x2B
	DIO_MODE_EMMC_DAT4	Output eMMC Data[4]	0x2C
	DIO_MODE_EMMC_DAT5	Output eMMC Data[5]	0x2D
	DIO_MODE_EMMC_DAT6	Output eMMC Data[6]	0x2E
	DIO_MODE_EMMC_DAT7	Output eMMC Data[7]	0x2F
	DIO_MODE_SYNC_MASTER	Output SYNC_MASTER interface signal	0x30
	DIO_MODE_DSP_TDO	Output DSP-TDO interface signal	0x31
	DIO_MODE_OSCCLK	Output OSCCLK signal	0x32
	DIO_MODE_STANDBYCLK	Output STANDBYCLK signal	0x33
	DIO_MODE_EXTCLK_DIV	Output of EXTCLK divider signal	0x34

16.4.2 DIO_CM3_JTAG_PAD_CFG

Bit Field	Read/Write	Field Name	Description
1	RW	CM3_JTAG_DATA_EN	Arm Cortex-M3 JTAG TDI on DIO32 and TDO on DIO33
0	RW	CM3_JTAG_EN	Arm Cortex-M3 JTAG SW-TCK and SW-TMS on DIO[31:30]

Field Name	Value Symbol	Value Description	Hex Value
CM3_JTAG_DATA_EN	CM3_JTAG_DATA_DISABLED	Arm Cortex-M3 JTAG TDI and TDO not connected	0x0
	CM3_JTAG_DATA_ENABLED	Arm Cortex-M3 JTAG TDI connected to DIO32 and TDO connected to DIO33	0x1*
CM3_JTAG_EN	CM3_JTAG_DISABLED	Arm Cortex-M3 JTAG SW-TCK and	0x0

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		SW-TMS not connected	
	CM3_JTAG_ENABLED	Arm Cortex-M3 JTAG SW-TCK connected to DIO30 and SW-TMS connected to DIO31	0x1*

16.4.3 DIO_CLKDIV_CFG

Bit Field	Read/Write	Field Name	Description
24	W	DIO_CLKDIV1_RESET	DIO CLKDIV1 reset
21	RW	DIO_CLKDIV1_EDGE	DIO CLKDIV1 active input edge
20	RW	DIO_CLKDIV1_ENABLE	DIO CLKDIV1 enable
19:16	RW	DIO_CLKDIV1_PRESCALE	DIO CLKDIV1 prescale value
8	W	DIO_CLKDIV0_RESET	DIO CLKDIV0 reset
5	RW	DIO_CLKDIV0_EDGE	DIO CLKDIV0 active input edge
4	RW	DIO_CLKDIV0_ENABLE	DIO CLKDIV0 enable
3:0	RW	DIO_CLKDIV0_PRESCALE	DIO CLKDIV0 prescale value

Field Name	Value Symbol	Value Description	Hex Value
DIO_CLKDIV1_RESET	DIO_CLKDIV1_RESET	Reset the DIO CLKDIV1	0x1
DIO_CLKDIV1_EDGE	DIO_CLKDIV1_RISING_EDGE	DIO CLKDIV1 counter is incremented on the rising edge of the input signal	0x0*
	DIO_CLKDIV1_FALLING_EDGE	DIO CLKDIV1 counter is incremented on the falling edge of the input signal	0x1
DIO_CLKDIV1_ENABLE	DIO_CLKDIV1_DISABLE	DIO CLKDIV1 is disabled counter	0x0*
	DIO_CLKDIV1_ENABLE	DIO CLKDIV1 is enabled	0x1
DIO_CLKDIV1_PRESCALE	DIO_CLKDIV1_PRESCALE_2	Prescale the DIO source clock by 2	0x0*
	DIO_CLKDIV1_PRESCALE_4	Prescale the DIO source clock by 4	0x1
	DIO_CLKDIV1_PRESCALE_8	Prescale the DIO source clock by 8	0x2
	DIO_CLKDIV1_PRESCALE_16	Prescale the DIO source clock by 16	0x3
	DIO_CLKDIV1_PRESCALE_32	Prescale the DIO source clock by 32	0x4
	DIO_CLKDIV1_PRESCALE_64	Prescale the DIO source clock by 64	0x5
	DIO_CLKDIV1_PRESCALE_128	Prescale the DIO source clock by 128	0x6
	DIO_CLKDIV1_PRESCALE_256	Prescale the DIO source clock by 256	0x7

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DIO_CLKDIV1_PRESCALE_512	Prescale the DIO source clock by 512	0x8
	DIO_CLKDIV1_PRESCALE_1024	Prescale the DIO source clock by 1024	0x9
	DIO_CLKDIV1_PRESCALE_2048	Prescale the DIO source clock by 2048	0xA
	DIO_CLKDIV1_PRESCALE_4096	Prescale the DIO source clock by 4096	0xB
	DIO_CLKDIV1_PRESCALE_8192	Prescale the DIO source clock by 8192	0xC
	DIO_CLKDIV1_PRESCALE_16384	Prescale the DIO source clock by 16384	0xD
	DIO_CLKDIV1_PRESCALE_32768	Prescale the DIO source clock by 32768	0xE
	DIO_CLKDIV1_PRESCALE_65536	Prescale the DIO source clock by 65536	0xF
DIO_CLKDIV0_RESET	DIO_CLKDIV0_RESET	Reset the DIO CLKDIV0 counter	0x1
DIO_CLKDIV0_EDGE	DIO_CLKDIV0_RISING_EDGE	DIO CLKDIV0 counter is incremented on the rising edge of the input signal	0x0*
	DIO_CLKDIV0_FALLING_EDGE	DIO CLKDIV0 counter is incremented on the falling edge of the input signal	0x1
DIO_CLKDIV0_ENABLE	DIO_CLKDIV0_DISABLE	DIO CLKDIV0 is disabled	0x0*
	DIO_CLKDIV0_ENABLE	DIO CLKDIV0 is enabled	0x1
DIO_CLKDIV0_PRESCALE	DIO_CLKDIV0_PRESCALE_2	Prescale the DIO source clock by 2	0x0*
	DIO_CLKDIV0_PRESCALE_4	Prescale the DIO source clock by 4	0x1
	DIO_CLKDIV0_PRESCALE_8	Prescale the DIO source clock by 8	0x2
	DIO_CLKDIV0_PRESCALE_16	Prescale the DIO source clock by 16	0x3
	DIO_CLKDIV0_PRESCALE_32	Prescale the DIO source clock by 32	0x4
	DIO_CLKDIV0_PRESCALE_64	Prescale the DIO source clock by 64	0x5
	DIO_CLKDIV0_PRESCALE_128	Prescale the DIO source clock by 128	0x6
	DIO_CLKDIV0_PRESCALE_256	Prescale the DIO source clock by 256	0x7
	DIO_CLKDIV0_PRESCALE_512	Prescale the DIO source clock by 512	0x8
	DIO_CLKDIV0_PRESCALE_1024	Prescale the DIO source clock by 1024	0x9
	DIO_CLKDIV0_PRESCALE_2048	Prescale the DIO source clock by 2048	0xA
	DIO_CLKDIV0_PRESCALE_4096	Prescale the DIO source clock by 4096	0xB
	DIO_CLKDIV0_PRESCALE_8192	Prescale the DIO source clock by 8192	0xC
	DIO_CLKDIV0_PRESCALE_16384	Prescale the DIO source clock by 16384	0xD
	DIO_CLKDIV0_PRESCALE_32768	Prescale the DIO source clock by 32768	0xE
	DIO_CLKDIV0_PRESCALE_65536	Prescale the DIO source clock by 65536	0xF

Ezairo 8300 Hardware Reference

16.4.4 DIO_SRC_SYNC_SLAVE

Bit Field	Read/Write	Field Name	Description
5:0	RW	SYNC	SYNC_SLAVE input selection

Field Name	Value Symbol	Value Description	Hex Value
SYNC	SYNC_SRC_DIO_0	Select DIO[0] as source	0x0
	SYNC_SRC_DIO_1	Select DIO[1] as source	0x1
	SYNC_SRC_DIO_2	Select DIO[2] as source	0x2
	SYNC_SRC_DIO_3	Select DIO[3] as source	0x3
	SYNC_SRC_DIO_4	Select DIO[4] as source	0x4
	SYNC_SRC_DIO_5	Select DIO[5] as source	0x5
	SYNC_SRC_DIO_6	Select DIO[6] as source	0x6
	SYNC_SRC_DIO_7	Select DIO[7] as source	0x7
	SYNC_SRC_DIO_8	Select DIO[8] as source	0x8
	SYNC_SRC_DIO_9	Select DIO[9] as source	0x9
	SYNC_SRC_DIO_10	Select DIO[10] as source	0xA
	SYNC_SRC_DIO_11	Select DIO[11] as source	0xB
	SYNC_SRC_DIO_12	Select DIO[12] as source	0xC
	SYNC_SRC_DIO_13	Select DIO[13] as source	0xD
	SYNC_SRC_DIO_14	Select DIO[14] as source	0xE
	SYNC_SRC_DIO_15	Select DIO[15] as source	0xF
	SYNC_SRC_DIO_16	Select DIO[16] as source	0x10
	SYNC_SRC_DIO_17	Select DIO[17] as source	0x11
	SYNC_SRC_DIO_18	Select DIO[18] as source	0x12
	SYNC_SRC_DIO_19	Select DIO[19] as source	0x13
	SYNC_SRC_DIO_20	Select DIO[20] as source	0x14
	SYNC_SRC_DIO_21	Select DIO[21] as source	0x15
	SYNC_SRC_DIO_22	Select DIO[22] as source	0x16
	SYNC_SRC_DIO_23	Select DIO[23] as source	0x17
	SYNC_SRC_DIO_24	Select DIO[24] as source	0x18
	SYNC_SRC_DIO_25	Select DIO[25] as source	0x19
	SYNC_SRC_DIO_26	Select DIO[26] as source	0x1A

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	SYNC_SRC_DIO_27	Select DIO[27] as source	0x1B
	SYNC_SRC_DIO_28	Select DIO[28] as source	0x1C
	SYNC_SRC_DIO_29	Select DIO[29] as source	0x1D
	SYNC_SRC_DIO_30	Select DIO[30] as source	0x1E
	SYNC_SRC_DIO_31	Select DIO[31] as source	0x1F
	SYNC_SRC_DIO_32	Select DIO[32] as source	0x20
	SYNC_SRC_DIO_33	Select DIO[33] as source	0x21
	SYNC_SRC_DIO_34	Select DIO[34] as source	0x22
	SYNC_SRC_DIO_35	Select DIO[35] as source	0x23
	SYNC_SRC_CONST_LOW	Select constant low as source	0x24
	SYNC_SRC_CONST_HIGH	Select constant high as source	0x25*

16.4.5 DIO_SRC_PCM*

Bit Field	Read/Write	Field Name	Description
21:16	RW	SERI	PCM_SERI input selection
13:8	RW	FRAME	PCM_FRAME input selection
5:0	RW	CLK	PCM_CLK input selection

Field Name	Value Symbol	Value Description	Hex Value
SERI	PCM_SERI_SRC_DIO_0	Select DIO[0] as source	0x0
	PCM_SERI_SRC_DIO_1	Select DIO[1] as source	0x1
	PCM_SERI_SRC_DIO_2	Select DIO[2] as source	0x2
	PCM_SERI_SRC_DIO_3	Select DIO[3] as source	0x3
	PCM_SERI_SRC_DIO_4	Select DIO[4] as source	0x4
	PCM_SERI_SRC_DIO_5	Select DIO[5] as source	0x5
	PCM_SERI_SRC_DIO_6	Select DIO[6] as source	0x6
	PCM_SERI_SRC_DIO_7	Select DIO[7] as source	0x7
	PCM_SERI_SRC_DIO_8	Select DIO[8] as source	0x8
	PCM_SERI_SRC_DIO_9	Select DIO[9] as source	0x9
	PCM_SERI_SRC_DIO_10	Select DIO[10] as source	0xA
	PCM_SERI_SRC_DIO_11	Select DIO[11] as source	0xB

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	PCM_SERI_SRC_DIO_12	Select DIO[12] as source	0xC
	PCM_SERI_SRC_DIO_13	Select DIO[13] as source	0xD
	PCM_SERI_SRC_DIO_14	Select DIO[14] as source	0xE
	PCM_SERI_SRC_DIO_15	Select DIO[15] as source	0xF
	PCM_SERI_SRC_DIO_16	Select DIO[16] as source	0x10
	PCM_SERI_SRC_DIO_17	Select DIO[17] as source	0x11
	PCM_SERI_SRC_DIO_18	Select DIO[18] as source	0x12
	PCM_SERI_SRC_DIO_19	Select DIO[19] as source	0x13
	PCM_SERI_SRC_DIO_20	Select DIO[20] as source	0x14
	PCM_SERI_SRC_DIO_21	Select DIO[21] as source	0x15
	PCM_SERI_SRC_DIO_22	Select DIO[22] as source	0x16
	PCM_SERI_SRC_DIO_23	Select DIO[23] as source	0x17
	PCM_SERI_SRC_DIO_24	Select DIO[24] as source	0x18
	PCM_SERI_SRC_DIO_25	Select DIO[25] as source	0x19
	PCM_SERI_SRC_DIO_26	Select DIO[26] as source	0x1A
	PCM_SERI_SRC_DIO_27	Select DIO[27] as source	0x1B
	PCM_SERI_SRC_DIO_28	Select DIO[28] as source	0x1C
	PCM_SERI_SRC_DIO_29	Select DIO[29] as source	0x1D
	PCM_SERI_SRC_DIO_30	Select DIO[24] as source	0x1E
	PCM_SERI_SRC_DIO_31	Select DIO[25] as source	0x1F
	PCM_SERI_SRC_DIO_32	Select DIO[26] as source	0x20
	PCM_SERI_SRC_DIO_33	Select DIO[27] as source	0x21
	PCM_SERI_SRC_DIO_34	Select DIO[28] as source	0x22
	PCM_SERI_SRC_DIO_35	Select DIO[29] as source	0x23
	PCM_SERI_SRC_CONST_LOW	Select constant low as source	0x24
	PCM_SERI_SRC_CONST_HIGH	Select constant high as source	0x25*
FRAME	PCM_FRAME_SRC_DIO_0	Select DIO[0] as source	0x0
	PCM_FRAME_SRC_DIO_1	Select DIO[1] as source	0x1
	PCM_FRAME_SRC_DIO_2	Select DIO[2] as source	0x2
	PCM_FRAME_SRC_DIO_3	Select DIO[3] as source	0x3
	PCM_FRAME_SRC_DIO_4	Select DIO[4] as source	0x4

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	PCM_FRAME_SRC_DIO_5	Select DIO[5] as source	0x5
	PCM_FRAME_SRC_DIO_6	Select DIO[6] as source	0x6
	PCM_FRAME_SRC_DIO_7	Select DIO[7] as source	0x7
	PCM_FRAME_SRC_DIO_8	Select DIO[8] as source	0x8
	PCM_FRAME_SRC_DIO_9	Select DIO[9] as source	0x9
	PCM_FRAME_SRC_DIO_10	Select DIO[10] as source	0xA
	PCM_FRAME_SRC_DIO_11	Select DIO[11] as source	0xB
	PCM_FRAME_SRC_DIO_12	Select DIO[12] as source	0xC
	PCM_FRAME_SRC_DIO_13	Select DIO[13] as source	0xD
	PCM_FRAME_SRC_DIO_14	Select DIO[14] as source	0xE
	PCM_FRAME_SRC_DIO_15	Select DIO[15] as source	0xF
	PCM_FRAME_SRC_DIO_16	Select DIO[16] as source	0x10
	PCM_FRAME_SRC_DIO_17	Select DIO[17] as source	0x11
	PCM_FRAME_SRC_DIO_18	Select DIO[18] as source	0x12
	PCM_FRAME_SRC_DIO_19	Select DIO[19] as source	0x13
	PCM_FRAME_SRC_DIO_20	Select DIO[20] as source	0x14
	PCM_FRAME_SRC_DIO_21	Select DIO[21] as source	0x15
	PCM_FRAME_SRC_DIO_22	Select DIO[22] as source	0x16
	PCM_FRAME_SRC_DIO_23	Select DIO[23] as source	0x17
	PCM_FRAME_SRC_DIO_24	Select DIO[24] as source	0x18
	PCM_FRAME_SRC_DIO_25	Select DIO[25] as source	0x19
	PCM_FRAME_SRC_DIO_26	Select DIO[26] as source	0x1A
	PCM_FRAME_SRC_DIO_27	Select DIO[27] as source	0x1B
	PCM_FRAME_SRC_DIO_28	Select DIO[28] as source	0x1C
	PCM_FRAME_SRC_DIO_29	Select DIO[29] as source	0x1D
	PCM_FRAME_SRC_DIO_30	Select DIO[24] as source	0x1E
	PCM_FRAME_SRC_DIO_31	Select DIO[25] as source	0x1F
	PCM_FRAME_SRC_DIO_32	Select DIO[26] as source	0x20
	PCM_FRAME_SRC_DIO_33	Select DIO[27] as source	0x21
	PCM_FRAME_SRC_DIO_34	Select DIO[28] as source	0x22
	PCM_FRAME_SRC_DIO_35	Select DIO[29] as source	0x23

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	PCM_FRAME_SRC_CONST_LOW	Select constant low as source	0x24
	PCM_FRAME_SRC_CONST_HIGH	Select constant high as source	0x25*
CLK	PCM_CLK_SRC_DIO_0	Select DIO[0] as source	0x0
	PCM_CLK_SRC_DIO_1	Select DIO[1] as source	0x1
	PCM_CLK_SRC_DIO_2	Select DIO[2] as source	0x2
	PCM_CLK_SRC_DIO_3	Select DIO[3] as source	0x3
	PCM_CLK_SRC_DIO_4	Select DIO[4] as source	0x4
	PCM_CLK_SRC_DIO_5	Select DIO[5] as source	0x5
	PCM_CLK_SRC_DIO_6	Select DIO[6] as source	0x6
	PCM_CLK_SRC_DIO_7	Select DIO[7] as source	0x7
	PCM_CLK_SRC_DIO_8	Select DIO[8] as source	0x8
	PCM_CLK_SRC_DIO_9	Select DIO[9] as source	0x9
	PCM_CLK_SRC_DIO_10	Select DIO[10] as source	0xA
	PCM_CLK_SRC_DIO_11	Select DIO[11] as source	0xB
	PCM_CLK_SRC_DIO_12	Select DIO[12] as source	0xC
	PCM_CLK_SRC_DIO_13	Select DIO[13] as source	0xD
	PCM_CLK_SRC_DIO_14	Select DIO[14] as source	0xE
	PCM_CLK_SRC_DIO_15	Select DIO[15] as source	0xF
	PCM_CLK_SRC_DIO_16	Select DIO[16] as source	0x10
	PCM_CLK_SRC_DIO_17	Select DIO[17] as source	0x11
	PCM_CLK_SRC_DIO_18	Select DIO[18] as source	0x12
	PCM_CLK_SRC_DIO_19	Select DIO[19] as source	0x13
	PCM_CLK_SRC_DIO_20	Select DIO[20] as source	0x14
	PCM_CLK_SRC_DIO_21	Select DIO[21] as source	0x15
	PCM_CLK_SRC_DIO_22	Select DIO[22] as source	0x16
	PCM_CLK_SRC_DIO_23	Select DIO[23] as source	0x17
	PCM_CLK_SRC_DIO_24	Select DIO[24] as source	0x18
	PCM_CLK_SRC_DIO_25	Select DIO[25] as source	0x19
	PCM_CLK_SRC_DIO_26	Select DIO[26] as source	0x1A
	PCM_CLK_SRC_DIO_27	Select DIO[27] as source	0x1B
	PCM_CLK_SRC_DIO_28	Select DIO[28] as source	0x1C

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	PCM_CLK_SRC_DIO_29	Select DIO[29] as source	0x1D
	PCM_CLK_SRC_DIO_30	Select DIO[24] as source	0x1E
	PCM_CLK_SRC_DIO_31	Select DIO[25] as source	0x1F
	PCM_CLK_SRC_DIO_32	Select DIO[26] as source	0x20
	PCM_CLK_SRC_DIO_33	Select DIO[27] as source	0x21
	PCM_CLK_SRC_DIO_34	Select DIO[28] as source	0x22
	PCM_CLK_SRC_DIO_35	Select DIO[29] as source	0x23
	PCM_CLK_SRC_CONST_LOW	Select constant low as source	0x24
	PCM_CLK_SRC_CONST_HIGH	Select constant high as source	0x25*

16.4.6 DIO_SRC_SPI*

Bit Field	Read/Write	Field Name	Description
13:8	RW	CS	SPI_CS input selection
5:0	RW	CLK	SPI_CLK input selection

Field Name	Value Symbol	Value Description	Hex Value
CS	SPI_CS_SRC_DIO_0	Select DIO[0] as source	0x0
	SPI_CS_SRC_DIO_1	Select DIO[1] as source	0x1
	SPI_CS_SRC_DIO_2	Select DIO[2] as source	0x2
	SPI_CS_SRC_DIO_3	Select DIO[3] as source	0x3
	SPI_CS_SRC_DIO_4	Select DIO[4] as source	0x4
	SPI_CS_SRC_DIO_5	Select DIO[5] as source	0x5
	SPI_CS_SRC_DIO_6	Select DIO[6] as source	0x6
	SPI_CS_SRC_DIO_7	Select DIO[7] as source	0x7
	SPI_CS_SRC_DIO_8	Select DIO[8] as source	0x8
	SPI_CS_SRC_DIO_9	Select DIO[9] as source	0x9
	SPI_CS_SRC_DIO_10	Select DIO[10] as source	0xA
	SPI_CS_SRC_DIO_11	Select DIO[11] as source	0xB
	SPI_CS_SRC_DIO_12	Select DIO[12] as source	0xC
	SPI_CS_SRC_DIO_13	Select DIO[13] as source	0xD
	SPI_CS_SRC_DIO_14	Select DIO[14] as source	0xE

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	SPI_CS_SRC_DIO_15	Select DIO[15] as source	0xF
	SPI_CS_SRC_DIO_16	Select DIO[16] as source	0x10
	SPI_CS_SRC_DIO_17	Select DIO[17] as source	0x11
	SPI_CS_SRC_DIO_18	Select DIO[18] as source	0x12
	SPI_CS_SRC_DIO_19	Select DIO[19] as source	0x13
	SPI_CS_SRC_DIO_20	Select DIO[20] as source	0x14
	SPI_CS_SRC_DIO_21	Select DIO[21] as source	0x15
	SPI_CS_SRC_DIO_22	Select DIO[22] as source	0x16
	SPI_CS_SRC_DIO_23	Select DIO[23] as source	0x17
	SPI_CS_SRC_DIO_24	Select DIO[24] as source	0x18
	SPI_CS_SRC_DIO_25	Select DIO[25] as source	0x19
	SPI_CS_SRC_DIO_26	Select DIO[26] as source	0x1A
	SPI_CS_SRC_DIO_27	Select DIO[27] as source	0x1B
	SPI_CS_SRC_DIO_28	Select DIO[28] as source	0x1C
	SPI_CS_SRC_DIO_29	Select DIO[29] as source	0x1D
	SPI_CS_SRC_DIO_30	Select DIO[30] as source	0x1E
	SPI_CS_SRC_DIO_31	Select DIO[31] as source	0x1F
	SPI_CS_SRC_DIO_32	Select DIO[32] as source	0x20
	SPI_CS_SRC_DIO_33	Select DIO[33] as source	0x21
	SPI_CS_SRC_DIO_34	Select DIO[34] as source	0x22
	SPI_CS_SRC_DIO_35	Select DIO[35] as source	0x23
	SPI_CS_SRC_CONST_LOW	Select constant low as source	0x24
	SPI_CS_SRC_CONST_HIGH	Select constant high as source	0x25*
CLK	SPI_CLK_SRC_DIO_0	Select DIO[0] as source	0x0
	SPI_CLK_SRC_DIO_1	Select DIO[1] as source	0x1
	SPI_CLK_SRC_DIO_2	Select DIO[2] as source	0x2
	SPI_CLK_SRC_DIO_3	Select DIO[3] as source	0x3
	SPI_CLK_SRC_DIO_4	Select DIO[4] as source	0x4
	SPI_CLK_SRC_DIO_5	Select DIO[5] as source	0x5
	SPI_CLK_SRC_DIO_6	Select DIO[6] as source	0x6
	SPI_CLK_SRC_DIO_7	Select DIO[7] as source	0x7

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	SPI_CLK_SRC_DIO_8	Select DIO[8] as source	0x8
	SPI_CLK_SRC_DIO_9	Select DIO[9] as source	0x9
	SPI_CLK_SRC_DIO_10	Select DIO[10] as source	0xA
	SPI_CLK_SRC_DIO_11	Select DIO[11] as source	0xB
	SPI_CLK_SRC_DIO_12	Select DIO[12] as source	0xC
	SPI_CLK_SRC_DIO_13	Select DIO[13] as source	0xD
	SPI_CLK_SRC_DIO_14	Select DIO[14] as source	0xE
	SPI_CLK_SRC_DIO_15	Select DIO[15] as source	0xF
	SPI_CLK_SRC_DIO_16	Select DIO[16] as source	0x10
	SPI_CLK_SRC_DIO_17	Select DIO[17] as source	0x11
	SPI_CLK_SRC_DIO_18	Select DIO[18] as source	0x12
	SPI_CLK_SRC_DIO_19	Select DIO[19] as source	0x13
	SPI_CLK_SRC_DIO_20	Select DIO[20] as source	0x14
	SPI_CLK_SRC_DIO_21	Select DIO[21] as source	0x15
	SPI_CLK_SRC_DIO_22	Select DIO[22] as source	0x16
	SPI_CLK_SRC_DIO_23	Select DIO[23] as source	0x17
	SPI_CLK_SRC_DIO_24	Select DIO[24] as source	0x18
	SPI_CLK_SRC_DIO_25	Select DIO[25] as source	0x19
	SPI_CLK_SRC_DIO_26	Select DIO[26] as source	0x1A
	SPI_CLK_SRC_DIO_27	Select DIO[27] as source	0x1B
	SPI_CLK_SRC_DIO_28	Select DIO[28] as source	0x1C
	SPI_CLK_SRC_DIO_29	Select DIO[29] as source	0x1D
	SPI_CLK_SRC_DIO_30	Select DIO[30] as source	0x1E
	SPI_CLK_SRC_DIO_31	Select DIO[31] as source	0x1F
	SPI_CLK_SRC_DIO_32	Select DIO[32] as source	0x20
	SPI_CLK_SRC_DIO_33	Select DIO[33] as source	0x21
	SPI_CLK_SRC_DIO_34	Select DIO[34] as source	0x22
	SPI_CLK_SRC_DIO_35	Select DIO[35] as source	0x23
	SPI_CLK_SRC_CONST_LOW	Select constant low as source	0x24
	SPI_CLK_SRC_CONST_HIGH	Select constant high as source	0x25*

Ezairo 8300 Hardware Reference

16.4.6.1 DIO_SRC_SPI_IO*

Bit Field	Read/Write	Field Name	Description
29:24	RW	IO3	SPI_IO3 input selection
21:16	RW	IO2	SPI_IO2 input selection
13:8	RW	IO1	SPI_IO1 input selection (master SERI)
5:0	RW	IO0	SPI_IO0 input selection (slave SERI)

Field Name	Value Symbol	Value Description	Hex Value
IO3	SPI_IO3_SRC_DIO_0	Select DIO[0] as source	0x0
	SPI_IO3_SRC_DIO_1	Select DIO[1] as source	0x1
	SPI_IO3_SRC_DIO_2	Select DIO[2] as source	0x2
	SPI_IO3_SRC_DIO_3	Select DIO[3] as source	0x3
	SPI_IO3_SRC_DIO_4	Select DIO[4] as source	0x4
	SPI_IO3_SRC_DIO_5	Select DIO[5] as source	0x5
	SPI_IO3_SRC_DIO_6	Select DIO[6] as source	0x6
	SPI_IO3_SRC_DIO_7	Select DIO[7] as source	0x7
	SPI_IO3_SRC_DIO_8	Select DIO[8] as source	0x8
	SPI_IO3_SRC_DIO_9	Select DIO[9] as source	0x9
	SPI_IO3_SRC_DIO_10	Select DIO[10] as source	0xA
	SPI_IO3_SRC_DIO_11	Select DIO[11] as source	0xB
	SPI_IO3_SRC_DIO_12	Select DIO[12] as source	0xC
	SPI_IO3_SRC_DIO_13	Select DIO[13] as source	0xD
	SPI_IO3_SRC_DIO_14	Select DIO[14] as source	0xE
	SPI_IO3_SRC_DIO_15	Select DIO[15] as source	0xF
	SPI_IO3_SRC_DIO_16	Select DIO[16] as source	0x10
	SPI_IO3_SRC_DIO_17	Select DIO[17] as source	0x11
	SPI_IO3_SRC_DIO_18	Select DIO[18] as source	0x12
	SPI_IO3_SRC_DIO_19	Select DIO[19] as source	0x13
	SPI_IO3_SRC_DIO_20	Select DIO[20] as source	0x14
	SPI_IO3_SRC_DIO_21	Select DIO[21] as source	0x15
	SPI_IO3_SRC_DIO_22	Select DIO[22] as source	0x16
	SPI_IO3_SRC_DIO_23	Select DIO[23] as source	0x17
	SPI_IO3_SRC_DIO_24	Select DIO[24] as source	0x18

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	SPI_IO3_SRC_DIO_25	Select DIO[25] as source	0x19
	SPI_IO3_SRC_DIO_26	Select DIO[26] as source	0x1A
	SPI_IO3_SRC_DIO_27	Select DIO[27] as source	0x1B
	SPI_IO3_SRC_DIO_28	Select DIO[28] as source	0x1C
	SPI_IO3_SRC_DIO_29	Select DIO[29] as source	0x1D
	SPI_IO3_SRC_DIO_30	Select DIO[30] as source	0x1E
	SPI_IO3_SRC_DIO_31	Select DIO[31] as source	0x1F
	SPI_IO3_SRC_DIO_32	Select DIO[32] as source	0x20
	SPI_IO3_SRC_DIO_33	Select DIO[33] as source	0x21
	SPI_IO3_SRC_DIO_34	Select DIO[34] as source	0x22
	SPI_IO3_SRC_DIO_35	Select DIO[35] as source	0x23
	SPI_IO3_SRC_CONST_LOW	Select constant low as source	0x24
	SPI_IO3_SRC_CONST_HIGH	Select constant high as source	0x25*
IO2	SPI_IO2_SRC_DIO_0	Select DIO[0] as source	0x0
	SPI_IO2_SRC_DIO_1	Select DIO[1] as source	0x1
	SPI_IO2_SRC_DIO_2	Select DIO[2] as source	0x2
	SPI_IO2_SRC_DIO_3	Select DIO[3] as source	0x3
	SPI_IO2_SRC_DIO_4	Select DIO[4] as source	0x4
	SPI_IO2_SRC_DIO_5	Select DIO[5] as source	0x5
	SPI_IO2_SRC_DIO_6	Select DIO[6] as source	0x6
	SPI_IO2_SRC_DIO_7	Select DIO[7] as source	0x7
	SPI_IO2_SRC_DIO_8	Select DIO[8] as source	0x8
	SPI_IO2_SRC_DIO_9	Select DIO[9] as source	0x9
	SPI_IO2_SRC_DIO_10	Select DIO[10] as source	0xA
	SPI_IO2_SRC_DIO_11	Select DIO[11] as source	0xB
	SPI_IO2_SRC_DIO_12	Select DIO[12] as source	0xC
	SPI_IO2_SRC_DIO_13	Select DIO[13] as source	0xD
	SPI_IO2_SRC_DIO_14	Select DIO[14] as source	0xE
	SPI_IO2_SRC_DIO_15	Select DIO[15] as source	0xF
	SPI_IO2_SRC_DIO_16	Select DIO[16] as source	0x10
	SPI_IO2_SRC_DIO_17	Select DIO[17] as source	0x11

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	SPI_IO2_SRC_DIO_18	Select DIO[18] as source	0x12
	SPI_IO2_SRC_DIO_19	Select DIO[19] as source	0x13
	SPI_IO2_SRC_DIO_20	Select DIO[20] as source	0x14
	SPI_IO2_SRC_DIO_21	Select DIO[21] as source	0x15
	SPI_IO2_SRC_DIO_22	Select DIO[22] as source	0x16
	SPI_IO2_SRC_DIO_23	Select DIO[23] as source	0x17
	SPI_IO2_SRC_DIO_24	Select DIO[24] as source	0x18
	SPI_IO2_SRC_DIO_25	Select DIO[25] as source	0x19
	SPI_IO2_SRC_DIO_26	Select DIO[26] as source	0x1A
	SPI_IO2_SRC_DIO_27	Select DIO[27] as source	0x1B
	SPI_IO2_SRC_DIO_28	Select DIO[28] as source	0x1C
	SPI_IO2_SRC_DIO_29	Select DIO[29] as source	0x1D
	SPI_IO2_SRC_DIO_30	Select DIO[30] as source	0x1E
	SPI_IO2_SRC_DIO_31	Select DIO[31] as source	0x1F
	SPI_IO2_SRC_DIO_32	Select DIO[32] as source	0x20
	SPI_IO2_SRC_DIO_33	Select DIO[33] as source	0x21
	SPI_IO2_SRC_DIO_34	Select DIO[34] as source	0x22
	SPI_IO2_SRC_DIO_35	Select DIO[35] as source	0x23
	SPI_IO2_SRC_CONST_LOW	Select constant low as source	0x24
	SPI_IO2_SRC_CONST_HIGH	Select constant high as source	0x25*
IO1	SPI_IO1_SRC_DIO_0	Select DIO[0] as source	0x0
	SPI_IO1_SRC_DIO_1	Select DIO[1] as source	0x1
	SPI_IO1_SRC_DIO_2	Select DIO[2] as source	0x2
	SPI_IO1_SRC_DIO_3	Select DIO[3] as source	0x3
	SPI_IO1_SRC_DIO_4	Select DIO[4] as source	0x4
	SPI_IO1_SRC_DIO_5	Select DIO[5] as source	0x5
	SPI_IO1_SRC_DIO_6	Select DIO[6] as source	0x6
	SPI_IO1_SRC_DIO_7	Select DIO[7] as source	0x7
	SPI_IO1_SRC_DIO_8	Select DIO[8] as source	0x8
	SPI_IO1_SRC_DIO_9	Select DIO[9] as source	0x9
	SPI_IO1_SRC_DIO_10	Select DIO[10] as source	0xA

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	SPI_IO1_SRC_DIO_11	Select DIO[11] as source	0xB
	SPI_IO1_SRC_DIO_12	Select DIO[12] as source	0xC
	SPI_IO1_SRC_DIO_13	Select DIO[13] as source	0xD
	SPI_IO1_SRC_DIO_14	Select DIO[14] as source	0xE
	SPI_IO1_SRC_DIO_15	Select DIO[15] as source	0xF
	SPI_IO1_SRC_DIO_16	Select DIO[16] as source	0x10
	SPI_IO1_SRC_DIO_17	Select DIO[17] as source	0x11
	SPI_IO1_SRC_DIO_18	Select DIO[18] as source	0x12
	SPI_IO1_SRC_DIO_19	Select DIO[19] as source	0x13
	SPI_IO1_SRC_DIO_20	Select DIO[20] as source	0x14
	SPI_IO1_SRC_DIO_21	Select DIO[21] as source	0x15
	SPI_IO1_SRC_DIO_22	Select DIO[22] as source	0x16
	SPI_IO1_SRC_DIO_23	Select DIO[23] as source	0x17
	SPI_IO1_SRC_DIO_24	Select DIO[24] as source	0x18
	SPI_IO1_SRC_DIO_25	Select DIO[25] as source	0x19
	SPI_IO1_SRC_DIO_26	Select DIO[26] as source	0x1A
	SPI_IO1_SRC_DIO_27	Select DIO[27] as source	0x1B
	SPI_IO1_SRC_DIO_28	Select DIO[28] as source	0x1C
	SPI_IO1_SRC_DIO_29	Select DIO[29] as source	0x1D
	SPI_IO1_SRC_DIO_30	Select DIO[30] as source	0x1E
	SPI_IO1_SRC_DIO_31	Select DIO[31] as source	0x1F
	SPI_IO1_SRC_DIO_32	Select DIO[32] as source	0x20
	SPI_IO1_SRC_DIO_33	Select DIO[33] as source	0x21
	SPI_IO1_SRC_DIO_34	Select DIO[34] as source	0x22
	SPI_IO1_SRC_DIO_35	Select DIO[35] as source	0x23
	SPI_IO1_SRC_CONST_LOW	Select constant low as source	0x24
	SPI_IO1_SRC_CONST_HIGH	Select constant high as source	0x25*
IO0	SPI_IO0_SRC_DIO_0	Select DIO[0] as source	0x0
	SPI_IO0_SRC_DIO_1	Select DIO[1] as source	0x1
	SPI_IO0_SRC_DIO_2	Select DIO[2] as source	0x2
	SPI_IO0_SRC_DIO_3	Select DIO[3] as source	0x3

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	SPI_IO0_SRC_DIO_4	Select DIO[4] as source	0x4
	SPI_IO0_SRC_DIO_5	Select DIO[5] as source	0x5
	SPI_IO0_SRC_DIO_6	Select DIO[6] as source	0x6
	SPI_IO0_SRC_DIO_7	Select DIO[7] as source	0x7
	SPI_IO0_SRC_DIO_8	Select DIO[8] as source	0x8
	SPI_IO0_SRC_DIO_9	Select DIO[9] as source	0x9
	SPI_IO0_SRC_DIO_10	Select DIO[10] as source	0xA
	SPI_IO0_SRC_DIO_11	Select DIO[11] as source	0xB
	SPI_IO0_SRC_DIO_12	Select DIO[12] as source	0xC
	SPI_IO0_SRC_DIO_13	Select DIO[13] as source	0xD
	SPI_IO0_SRC_DIO_14	Select DIO[14] as source	0xE
	SPI_IO0_SRC_DIO_15	Select DIO[15] as source	0xF
	SPI_IO0_SRC_DIO_16	Select DIO[16] as source	0x10
	SPI_IO0_SRC_DIO_17	Select DIO[17] as source	0x11
	SPI_IO0_SRC_DIO_18	Select DIO[18] as source	0x12
	SPI_IO0_SRC_DIO_19	Select DIO[19] as source	0x13
	SPI_IO0_SRC_DIO_20	Select DIO[20] as source	0x14
	SPI_IO0_SRC_DIO_21	Select DIO[21] as source	0x15
	SPI_IO0_SRC_DIO_22	Select DIO[22] as source	0x16
	SPI_IO0_SRC_DIO_23	Select DIO[23] as source	0x17
	SPI_IO0_SRC_DIO_24	Select DIO[24] as source	0x18
	SPI_IO0_SRC_DIO_25	Select DIO[25] as source	0x19
	SPI_IO0_SRC_DIO_26	Select DIO[26] as source	0x1A
	SPI_IO0_SRC_DIO_27	Select DIO[27] as source	0x1B
	SPI_IO0_SRC_DIO_28	Select DIO[28] as source	0x1C
	SPI_IO0_SRC_DIO_29	Select DIO[29] as source	0x1D
	SPI_IO0_SRC_DIO_30	Select DIO[30] as source	0x1E
	SPI_IO0_SRC_DIO_31	Select DIO[31] as source	0x1F
	SPI_IO0_SRC_DIO_32	Select DIO[32] as source	0x20
	SPI_IO0_SRC_DIO_33	Select DIO[33] as source	0x21
	SPI_IO0_SRC_DIO_34	Select DIO[34] as source	0x22

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	SPI_IO0_SRC_DIO_35	Select DIO[35] as source	0x23
	SPI_IO0_SRC_CONST_LOW	Select constant low as source	0x24
	SPI_IO0_SRC_CONST_HIGH	Select constant high as source	0x25*

16.4.7 DIO_SRC_I3C*

Bit Field	Read/Write	Field Name	Description
13:8	RW	SDA	SDA input selection
5:0	RW	SCL	SCL input selection

Field Name	Value Symbol	Value Description	Hex Value
SDA	I3C_SDA_SRC_DIO_0	Select DIO[0] as source	0x0
	I3C_SDA_SRC_DIO_1	Select DIO[1] as source	0x1
	I3C_SDA_SRC_DIO_2	Select DIO[2] as source	0x2
	I3C_SDA_SRC_DIO_3	Select DIO[3] as source	0x3
	I3C_SDA_SRC_DIO_4	Select DIO[4] as source	0x4
	I3C_SDA_SRC_DIO_5	Select DIO[5] as source	0x5
	I3C_SDA_SRC_DIO_6	Select DIO[6] as source	0x6
	I3C_SDA_SRC_DIO_7	Select DIO[7] as source	0x7
	I3C_SDA_SRC_DIO_8	Select DIO[8] as source	0x8
	I3C_SDA_SRC_DIO_9	Select DIO[9] as source	0x9
	I3C_SDA_SRC_DIO_10	Select DIO[10] as source	0xA
	I3C_SDA_SRC_DIO_11	Select DIO[11] as source	0xB
	I3C_SDA_SRC_DIO_12	Select DIO[12] as source	0xC
	I3C_SDA_SRC_DIO_13	Select DIO[13] as source	0xD
	I3C_SDA_SRC_DIO_14	Select DIO[14] as source	0xE
	I3C_SDA_SRC_DIO_15	Select DIO[15] as source	0xF
	I3C_SDA_SRC_DIO_16	Select DIO[16] as source	0x10
	I3C_SDA_SRC_DIO_17	Select DIO[17] as source	0x11
	I3C_SDA_SRC_DIO_18	Select DIO[18] as source	0x12
	I3C_SDA_SRC_DIO_19	Select DIO[19] as source	0x13
	I3C_SDA_SRC_DIO_20	Select DIO[20] as source	0x14

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	I3C_SDA_SRC_DIO_21	Select DIO[21] as source	0x15
	I3C_SDA_SRC_DIO_22	Select DIO[22] as source	0x16
	I3C_SDA_SRC_DIO_23	Select DIO[23] as source	0x17
	I3C_SDA_SRC_DIO_24	Select DIO[24] as source	0x18
	I3C_SDA_SRC_DIO_25	Select DIO[25] as source	0x19
	I3C_SDA_SRC_DIO_26	Select DIO[26] as source	0x1A
	I3C_SDA_SRC_DIO_27	Select DIO[27] as source	0x1B
	I3C_SDA_SRC_DIO_28	Select DIO[28] as source	0x1C
	I3C_SDA_SRC_DIO_29	Select DIO[29] as source	0x1D
	I3C_SDA_SRC_DIO_30	Select DIO[30] as source	0x1E
	I3C_SDA_SRC_DIO_31	Select DIO[31] as source	0x1F
	I3C_SDA_SRC_DIO_32	Select DIO[32] as source	0x20
	I3C_SDA_SRC_DIO_33	Select DIO[33] as source	0x21
	I3C_SDA_SRC_DIO_34	Select DIO[34] as source	0x22
	I3C_SDA_SRC_DIO_35	Select DIO[35] as source	0x23
	I3C_SDA_SRC_CONST_LOW	Select constant low as source	0x24
	I3C_SDA_SRC_CONST_HIGH	Select constant high as source	0x25*
SCL	I3C_SCL_SRC_DIO_0	Select DIO[0] as source	0x0
	I3C_SCL_SRC_DIO_1	Select DIO[1] as source	0x1
	I3C_SCL_SRC_DIO_2	Select DIO[2] as source	0x2
	I3C_SCL_SRC_DIO_3	Select DIO[3] as source	0x3
	I3C_SCL_SRC_DIO_4	Select DIO[4] as source	0x4
	I3C_SCL_SRC_DIO_5	Select DIO[5] as source	0x5
	I3C_SCL_SRC_DIO_6	Select DIO[6] as source	0x6
	I3C_SCL_SRC_DIO_7	Select DIO[7] as source	0x7
	I3C_SCL_SRC_DIO_8	Select DIO[8] as source	0x8
	I3C_SCL_SRC_DIO_9	Select DIO[9] as source	0x9
	I3C_SCL_SRC_DIO_10	Select DIO[10] as source	0xA
	I3C_SCL_SRC_DIO_11	Select DIO[11] as source	0xB
	I3C_SCL_SRC_DIO_12	Select DIO[12] as source	0xC
	I3C_SCL_SRC_DIO_13	Select DIO[13] as source	0xD

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	I3C_SCL_SRC_DIO_14	Select DIO[14] as source	0xE
	I3C_SCL_SRC_DIO_15	Select DIO[15] as source	0xF
	I3C_SCL_SRC_DIO_16	Select DIO[16] as source	0x10
	I3C_SCL_SRC_DIO_17	Select DIO[17] as source	0x11
	I3C_SCL_SRC_DIO_18	Select DIO[18] as source	0x12
	I3C_SCL_SRC_DIO_19	Select DIO[19] as source	0x13
	I3C_SCL_SRC_DIO_20	Select DIO[20] as source	0x14
	I3C_SCL_SRC_DIO_21	Select DIO[21] as source	0x15
	I3C_SCL_SRC_DIO_22	Select DIO[22] as source	0x16
	I3C_SCL_SRC_DIO_23	Select DIO[23] as source	0x17
	I3C_SCL_SRC_DIO_24	Select DIO[24] as source	0x18
	I3C_SCL_SRC_DIO_25	Select DIO[25] as source	0x19
	I3C_SCL_SRC_DIO_26	Select DIO[26] as source	0x1A
	I3C_SCL_SRC_DIO_27	Select DIO[27] as source	0x1B
	I3C_SCL_SRC_DIO_28	Select DIO[28] as source	0x1C
	I3C_SCL_SRC_DIO_29	Select DIO[29] as source	0x1D
	I3C_SCL_SRC_DIO_30	Select DIO[30] as source	0x1E
	I3C_SCL_SRC_DIO_31	Select DIO[31] as source	0x1F
	I3C_SCL_SRC_DIO_32	Select DIO[32] as source	0x20
	I3C_SCL_SRC_DIO_33	Select DIO[33] as source	0x21
	I3C_SCL_SRC_DIO_34	Select DIO[34] as source	0x22
	I3C_SCL_SRC_DIO_35	Select DIO[35] as source	0x23
	I3C_SCL_SRC_CONST_LOW	Select constant low as source	0x24
	I3C_SCL_SRC_CONST_HIGH	Select constant high as source	0x25*

16.4.8 DIO_SRC_I2C*

Bit Field	Read/Write	Field Name	Description
13:8	RW	SDA	SDA input selection
5:0	RW	SCL	SCL input selection

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
SDA	I2C_SDA_SRC_DIO_0	Select DIO[0] as source	0x0
	I2C_SDA_SRC_DIO_1	Select DIO[1] as source	0x1
	I2C_SDA_SRC_DIO_2	Select DIO[2] as source	0x2
	I2C_SDA_SRC_DIO_3	Select DIO[3] as source	0x3
	I2C_SDA_SRC_DIO_4	Select DIO[4] as source	0x4
	I2C_SDA_SRC_DIO_5	Select DIO[5] as source	0x5
	I2C_SDA_SRC_DIO_6	Select DIO[6] as source	0x6
	I2C_SDA_SRC_DIO_7	Select DIO[7] as source	0x7
	I2C_SDA_SRC_DIO_8	Select DIO[8] as source	0x8
	I2C_SDA_SRC_DIO_9	Select DIO[9] as source	0x9
	I2C_SDA_SRC_DIO_10	Select DIO[10] as source	0xA
	I2C_SDA_SRC_DIO_11	Select DIO[11] as source	0xB
	I2C_SDA_SRC_DIO_12	Select DIO[12] as source	0xC
	I2C_SDA_SRC_DIO_13	Select DIO[13] as source	0xD
	I2C_SDA_SRC_DIO_14	Select DIO[14] as source	0xE
	I2C_SDA_SRC_DIO_15	Select DIO[15] as source	0xF
	I2C_SDA_SRC_DIO_16	Select DIO[16] as source	0x10
	I2C_SDA_SRC_DIO_17	Select DIO[17] as source	0x11
	I2C_SDA_SRC_DIO_18	Select DIO[18] as source	0x12
	I2C_SDA_SRC_DIO_19	Select DIO[19] as source	0x13
	I2C_SDA_SRC_DIO_20	Select DIO[20] as source	0x14
	I2C_SDA_SRC_DIO_21	Select DIO[21] as source	0x15
	I2C_SDA_SRC_DIO_22	Select DIO[22] as source	0x16
	I2C_SDA_SRC_DIO_23	Select DIO[23] as source	0x17
	I2C_SDA_SRC_DIO_24	Select DIO[24] as source	0x18
	I2C_SDA_SRC_DIO_25	Select DIO[25] as source	0x19
	I2C_SDA_SRC_DIO_26	Select DIO[26] as source	0x1A
	I2C_SDA_SRC_DIO_27	Select DIO[27] as source	0x1B
	I2C_SDA_SRC_DIO_28	Select DIO[28] as source	0x1C
	I2C_SDA_SRC_DIO_29	Select DIO[29] as source	0x1D
	I2C_SDA_SRC_DIO_30	Select DIO[30] as source	0x1E

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	I2C_SDA_SRC_DIO_31	Select DIO[31] as source	0x1F
	I2C_SDA_SRC_DIO_32	Select DIO[32] as source	0x20
	I2C_SDA_SRC_DIO_33	Select DIO[33] as source	0x21
	I2C_SDA_SRC_DIO_34	Select DIO[34] as source	0x22
	I2C_SDA_SRC_DIO_35	Select DIO[35] as source	0x23
	I2C_SDA_SRC_CONST_LOW	Select constant low as source	0x24
	I2C_SDA_SRC_CONST_HIGH	Select constant high as source	0x25*
SCL	I2C_SCL_SRC_DIO_0	Select DIO[0] as source	0x0
	I2C_SCL_SRC_DIO_1	Select DIO[1] as source	0x1
	I2C_SCL_SRC_DIO_2	Select DIO[2] as source	0x2
	I2C_SCL_SRC_DIO_3	Select DIO[3] as source	0x3
	I2C_SCL_SRC_DIO_4	Select DIO[4] as source	0x4
	I2C_SCL_SRC_DIO_5	Select DIO[5] as source	0x5
	I2C_SCL_SRC_DIO_6	Select DIO[6] as source	0x6
	I2C_SCL_SRC_DIO_7	Select DIO[7] as source	0x7
	I2C_SCL_SRC_DIO_8	Select DIO[8] as source	0x8
	I2C_SCL_SRC_DIO_9	Select DIO[9] as source	0x9
	I2C_SCL_SRC_DIO_10	Select DIO[10] as source	0xA
	I2C_SCL_SRC_DIO_11	Select DIO[11] as source	0xB
	I2C_SCL_SRC_DIO_12	Select DIO[12] as source	0xC
	I2C_SCL_SRC_DIO_13	Select DIO[13] as source	0xD
	I2C_SCL_SRC_DIO_14	Select DIO[14] as source	0xE
	I2C_SCL_SRC_DIO_15	Select DIO[15] as source	0xF
	I2C_SCL_SRC_DIO_16	Select DIO[16] as source	0x10
	I2C_SCL_SRC_DIO_17	Select DIO[17] as source	0x11
	I2C_SCL_SRC_DIO_18	Select DIO[18] as source	0x12
	I2C_SCL_SRC_DIO_19	Select DIO[19] as source	0x13
	I2C_SCL_SRC_DIO_20	Select DIO[20] as source	0x14
	I2C_SCL_SRC_DIO_21	Select DIO[21] as source	0x15
	I2C_SCL_SRC_DIO_22	Select DIO[22] as source	0x16
	I2C_SCL_SRC_DIO_23	Select DIO[23] as source	0x17

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	I2C_SCL_SRC_DIO_24	Select DIO[24] as source	0x18
	I2C_SCL_SRC_DIO_25	Select DIO[25] as source	0x19
	I2C_SCL_SRC_DIO_26	Select DIO[26] as source	0x1A
	I2C_SCL_SRC_DIO_27	Select DIO[27] as source	0x1B
	I2C_SCL_SRC_DIO_28	Select DIO[28] as source	0x1C
	I2C_SCL_SRC_DIO_29	Select DIO[29] as source	0x1D
	I2C_SCL_SRC_DIO_30	Select DIO[30] as source	0x1E
	I2C_SCL_SRC_DIO_31	Select DIO[31] as source	0x1F
	I2C_SCL_SRC_DIO_32	Select DIO[32] as source	0x20
	I2C_SCL_SRC_DIO_33	Select DIO[33] as source	0x21
	I2C_SCL_SRC_DIO_34	Select DIO[34] as source	0x22
	I2C_SCL_SRC_DIO_35	Select DIO[35] as source	0x23
	I2C_SCL_SRC_CONST_LOW	Select constant low as source	0x24
	I2C_SCL_SRC_CONST_HIGH	Select constant high as source	0x25*

16.4.9 DIO_SRC_CLKDIV

Bit Field	Read/Write	Field Name	Description
13:8	RW	CLKDIV1	DIO Interrupt Clock Divider 1 Input Selection
5:0	RW	CLKDIV0	DIO Interrupt Clock Divider 0 Input Selection

Field Name	Value Symbol	Value Description	Hex Value
CLKDIV1	DIO_CLKDIV1_SRC_DIO_0	Select DIO[0] as source	0x0
	DIO_CLKDIV1_SRC_DIO_1	Select DIO[1] as source	0x1
	DIO_CLKDIV1_SRC_DIO_2	Select DIO[2] as source	0x2
	DIO_CLKDIV1_SRC_DIO_3	Select DIO[3] as source	0x3
	DIO_CLKDIV1_SRC_DIO_4	Select DIO[4] as source	0x4
	DIO_CLKDIV1_SRC_DIO_5	Select DIO[5] as source	0x5
	DIO_CLKDIV1_SRC_DIO_6	Select DIO[6] as source	0x6
	DIO_CLKDIV1_SRC_DIO_7	Select DIO[7] as source	0x7
	DIO_CLKDIV1_SRC_DIO_8	Select DIO[8] as source	0x8
	DIO_CLKDIV1_SRC_DIO_9	Select DIO[9] as source	0x9

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DIO_CLKDIV1_SRC_DIO_10	Select DIO[10] as source	0xA
	DIO_CLKDIV1_SRC_DIO_11	Select DIO[11] as source	0xB
	DIO_CLKDIV1_SRC_DIO_12	Select DIO[12] as source	0xC
	DIO_CLKDIV1_SRC_DIO_13	Select DIO[13] as source	0xD
	DIO_CLKDIV1_SRC_DIO_14	Select DIO[14] as source	0xE
	DIO_CLKDIV1_SRC_DIO_15	Select DIO[15] as source	0xF
	DIO_CLKDIV1_SRC_DIO_16	Select DIO[16] as source	0x10
	DIO_CLKDIV1_SRC_DIO_17	Select DIO[17] as source	0x11
	DIO_CLKDIV1_SRC_DIO_18	Select DIO[18] as source	0x12
	DIO_CLKDIV1_SRC_DIO_19	Select DIO[19] as source	0x13
	DIO_CLKDIV1_SRC_DIO_20	Select DIO[20] as source	0x14
	DIO_CLKDIV1_SRC_DIO_21	Select DIO[21] as source	0x15
	DIO_CLKDIV1_SRC_DIO_22	Select DIO[22] as source	0x16
	DIO_CLKDIV1_SRC_DIO_23	Select DIO[23] as source	0x17
	DIO_CLKDIV1_SRC_DIO_24	Select DIO[24] as source	0x18
	DIO_CLKDIV1_SRC_DIO_25	Select DIO[25] as source	0x19
	DIO_CLKDIV1_SRC_DIO_26	Select DIO[26] as source	0x1A
	DIO_CLKDIV1_SRC_DIO_27	Select DIO[27] as source	0x1B
	DIO_CLKDIV1_SRC_DIO_28	Select DIO[28] as source	0x1C
	DIO_CLKDIV1_SRC_DIO_29	Select DIO[29] as source	0x1D
	DIO_CLKDIV1_SRC_DIO_30	Select DIO[30] as source	0x1E
	DIO_CLKDIV1_SRC_DIO_31	Select DIO[31] as source	0x1F
	DIO_CLKDIV1_SRC_DIO_32	Select DIO[32] as source	0x20
	DIO_CLKDIV1_SRC_DIO_33	Select DIO[33] as source	0x21
	DIO_CLKDIV1_SRC_DIO_34	Select DIO[34] as source	0x22
	DIO_CLKDIV1_SRC_DIO_35	Select DIO[35] as source	0x23
	DIO_CLKDIV1_SRC_CONST_LOW	Select constant low as source	0x24
	DIO_CLKDIV1_SRC_CONST_HIGH	Select constant high as source	0x25*
CLKDIV0	DIO_CLKDIV0_SRC_DIO_0	Select DIO[0] as source	0x0
	DIO_CLKDIV0_SRC_DIO_1	Select DIO[1] as source	0x1
	DIO_CLKDIV0_SRC_DIO_2	Select DIO[2] as source	0x2

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DIO_CLKDIV0_SRC_DIO_3	Select DIO[3] as source	0x3
	DIO_CLKDIV0_SRC_DIO_4	Select DIO[4] as source	0x4
	DIO_CLKDIV0_SRC_DIO_5	Select DIO[5] as source	0x5
	DIO_CLKDIV0_SRC_DIO_6	Select DIO[6] as source	0x6
	DIO_CLKDIV0_SRC_DIO_7	Select DIO[7] as source	0x7
	DIO_CLKDIV0_SRC_DIO_8	Select DIO[8] as source	0x8
	DIO_CLKDIV0_SRC_DIO_9	Select DIO[9] as source	0x9
	DIO_CLKDIV0_SRC_DIO_10	Select DIO[10] as source	0xA
	DIO_CLKDIV0_SRC_DIO_11	Select DIO[11] as source	0xB
	DIO_CLKDIV0_SRC_DIO_12	Select DIO[12] as source	0xC
	DIO_CLKDIV0_SRC_DIO_13	Select DIO[13] as source	0xD
	DIO_CLKDIV0_SRC_DIO_14	Select DIO[14] as source	0xE
	DIO_CLKDIV0_SRC_DIO_15	Select DIO[15] as source	0xF
	DIO_CLKDIV0_SRC_DIO_16	Select DIO[16] as source	0x10
	DIO_CLKDIV0_SRC_DIO_17	Select DIO[17] as source	0x11
	DIO_CLKDIV0_SRC_DIO_18	Select DIO[18] as source	0x12
	DIO_CLKDIV0_SRC_DIO_19	Select DIO[19] as source	0x13
	DIO_CLKDIV0_SRC_DIO_20	Select DIO[20] as source	0x14
	DIO_CLKDIV0_SRC_DIO_21	Select DIO[21] as source	0x15
	DIO_CLKDIV0_SRC_DIO_22	Select DIO[22] as source	0x16
	DIO_CLKDIV0_SRC_DIO_23	Select DIO[23] as source	0x17
	DIO_CLKDIV0_SRC_DIO_24	Select DIO[24] as source	0x18
	DIO_CLKDIV0_SRC_DIO_25	Select DIO[25] as source	0x19
	DIO_CLKDIV0_SRC_DIO_26	Select DIO[26] as source	0x1A
	DIO_CLKDIV0_SRC_DIO_27	Select DIO[27] as source	0x1B
	DIO_CLKDIV0_SRC_DIO_28	Select DIO[28] as source	0x1C
	DIO_CLKDIV0_SRC_DIO_29	Select DIO[29] as source	0x1D
	DIO_CLKDIV0_SRC_DIO_30	Select DIO[30] as source	0x1E
	DIO_CLKDIV0_SRC_DIO_31	Select DIO[31] as source	0x1F
	DIO_CLKDIV0_SRC_DIO_32	Select DIO[32] as source	0x20
	DIO_CLKDIV0_SRC_DIO_33	Select DIO[33] as source	0x21

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DIO_CLKDIV0_SRC_DIO_34	Select DIO[34] as source	0x22
	DIO_CLKDIV0_SRC_DIO_35	Select DIO[35] as source	0x23
	DIO_CLKDIV0_SRC_CONST_LOW	Select constant low as source	0x24
	DIO_CLKDIV0_SRC_CONST_HIGH	Select constant high as source	0x25*

16.4.9.1 DIO_SRC_NMI

Bit Field	Read/Write	Field Name	Description
7	RW	NMI_EDGE_LEVEL	NMI edge or level
6	RW	NMI_POLARITY	NMI polarity
5:0	RW	NMI	NMI input selection

Field Name	Value Symbol	Value Description	Hex Value
NMI_EDGE_LEVEL	NMI_EDGE_TRIGGERED	NMI is edge triggered	0x0*
	NMI_LEVEL_TRIGGERED	NMI is level triggered	0x1
NMI_POLARITY	NMI_ACTIVE_LOW	NMI active low	0x0
	NMI_ACTIVE_HIGH	NMI active high	0x1*
NMI	NMI_SRC_DIO_0	Select DIO[0] as source	0x0
	NMI_SRC_DIO_1	Select DIO[1] as source	0x1
	NMI_SRC_DIO_2	Select DIO[2] as source	0x2
	NMI_SRC_DIO_3	Select DIO[3] as source	0x3
	NMI_SRC_DIO_4	Select DIO[4] as source	0x4
	NMI_SRC_DIO_5	Select DIO[5] as source	0x5
	NMI_SRC_DIO_6	Select DIO[6] as source	0x6
	NMI_SRC_DIO_7	Select DIO[7] as source	0x7
	NMI_SRC_DIO_8	Select DIO[8] as source	0x8
	NMI_SRC_DIO_9	Select DIO[9] as source	0x9
	NMI_SRC_DIO_10	Select DIO[10] as source	0xA
	NMI_SRC_DIO_11	Select DIO[11] as source	0xB
	NMI_SRC_DIO_12	Select DIO[12] as source	0xC
	NMI_SRC_DIO_13	Select DIO[13] as source	0xD
	NMI_SRC_DIO_14	Select DIO[14] as source	0xE

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	NMI_SRC_DIO_15	Select DIO[15] as source	0xF
	NMI_SRC_DIO_16	Select DIO[16] as source	0x10
	NMI_SRC_DIO_17	Select DIO[17] as source	0x11
	NMI_SRC_DIO_18	Select DIO[18] as source	0x12
	NMI_SRC_DIO_19	Select DIO[19] as source	0x13
	NMI_SRC_DIO_20	Select DIO[20] as source	0x14
	NMI_SRC_DIO_21	Select DIO[21] as source	0x15
	NMI_SRC_DIO_22	Select DIO[22] as source	0x16
	NMI_SRC_DIO_23	Select DIO[23] as source	0x17
	NMI_SRC_DIO_24	Select DIO[24] as source	0x18
	NMI_SRC_DIO_25	Select DIO[25] as source	0x19
	NMI_SRC_DIO_26	Select DIO[26] as source	0x1A
	NMI_SRC_DIO_27	Select DIO[27] as source	0x1B
	NMI_SRC_DIO_28	Select DIO[28] as source	0x1C
	NMI_SRC_DIO_29	Select DIO[29] as source	0x1D
	NMI_SRC_DIO_30	Select DIO[30] as source	0x1E
	NMI_SRC_DIO_31	Select DIO[31] as source	0x1F
	NMI_SRC_DIO_32	Select DIO[32] as source	0x20
	NMI_SRC_DIO_33	Select DIO[33] as source	0x21
	NMI_SRC_DIO_34	Select DIO[34] as source	0x22
	NMI_SRC_DIO_35	Select DIO[35] as source	0x23
	NMI_SRC_CONST_LOW	Select constant low as source	0x24*
	NMI_SRC_CONST_HIGH	Select constant high as source	0x25

16.4.10 DIO_SRC_EMMC_0

Bit Field	Read/Write	Field Name	Description
29:24	RW	EMMC_DATA3	eMMC DATA3 input selection
21:16	RW	EMMC_DATA2	eMMC DATA2 input selection
13:8	RW	EMMC_DATA1	eMMC DATA1 input selection
5:0	RW	EMMC_DATA0	eMMC DATA0 input selection

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
EMMC_DATA3	EMMC_DATA3_SRC_DIO_0	Select DIO[0] as source	0x0
	EMMC_DATA3_SRC_DIO_1	Select DIO[1] as source	0x1
	EMMC_DATA3_SRC_DIO_2	Select DIO[2] as source	0x2
	EMMC_DATA3_SRC_DIO_3	Select DIO[3] as source	0x3
	EMMC_DATA3_SRC_DIO_4	Select DIO[4] as source	0x4
	EMMC_DATA3_SRC_DIO_5	Select DIO[5] as source	0x5
	EMMC_DATA3_SRC_DIO_6	Select DIO[6] as source	0x6
	EMMC_DATA3_SRC_DIO_7	Select DIO[7] as source	0x7
	EMMC_DATA3_SRC_DIO_8	Select DIO[8] as source	0x8
	EMMC_DATA3_SRC_DIO_9	Select DIO[9] as source	0x9
	EMMC_DATA3_SRC_DIO_10	Select DIO[10] as source	0xA
	EMMC_DATA3_SRC_DIO_11	Select DIO[11] as source	0xB
	EMMC_DATA3_SRC_DIO_12	Select DIO[12] as source	0xC
	EMMC_DATA3_SRC_DIO_13	Select DIO[13] as source	0xD
	EMMC_DATA3_SRC_DIO_14	Select DIO[14] as source	0xE
	EMMC_DATA3_SRC_DIO_15	Select DIO[15] as source	0xF
	EMMC_DATA3_SRC_DIO_16	Select DIO[16] as source	0x10
	EMMC_DATA3_SRC_DIO_17	Select DIO[17] as source	0x11
	EMMC_DATA3_SRC_DIO_18	Select DIO[18] as source	0x12
	EMMC_DATA3_SRC_DIO_19	Select DIO[19] as source	0x13
	EMMC_DATA3_SRC_DIO_20	Select DIO[20] as source	0x14
	EMMC_DATA3_SRC_DIO_21	Select DIO[21] as source	0x15
	EMMC_DATA3_SRC_DIO_22	Select DIO[22] as source	0x16
	EMMC_DATA3_SRC_DIO_23	Select DIO[23] as source	0x17
	EMMC_DATA3_SRC_DIO_24	Select DIO[24] as source	0x18
	EMMC_DATA3_SRC_DIO_25	Select DIO[25] as source	0x19
	EMMC_DATA3_SRC_DIO_26	Select DIO[26] as source	0x1A
	EMMC_DATA3_SRC_DIO_27	Select DIO[27] as source	0x1B
	EMMC_DATA3_SRC_DIO_28	Select DIO[28] as source	0x1C
	EMMC_DATA3_SRC_DIO_29	Select DIO[29] as source	0x1D
	EMMC_DATA3_SRC_DIO_30	Select DIO[30] as source	0x1E

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	EMMC_DATA3_SRC_DIO_31	Select DIO[31] as source	0x1F
	EMMC_DATA3_SRC_DIO_32	Select DIO[32] as source	0x20
	EMMC_DATA3_SRC_DIO_33	Select DIO[33] as source	0x21
	EMMC_DATA3_SRC_DIO_34	Select DIO[34] as source	0x22
	EMMC_DATA3_SRC_DIO_35	Select DIO[35] as source	0x23
	EMMC_DATA3_SRC_CONST_LOW	Select constant low as source	0x24
	EMMC_DATA3_SRC_CONST_HIGH	Select constant high as source	0x25*
EMMC_DATA2	EMMC_DATA2_SRC_DIO_0	Select DIO[0] as source	0x0
	EMMC_DATA2_SRC_DIO_1	Select DIO[1] as source	0x1
	EMMC_DATA2_SRC_DIO_2	Select DIO[2] as source	0x2
	EMMC_DATA2_SRC_DIO_3	Select DIO[3] as source	0x3
	EMMC_DATA2_SRC_DIO_4	Select DIO[4] as source	0x4
	EMMC_DATA2_SRC_DIO_5	Select DIO[5] as source	0x5
	EMMC_DATA2_SRC_DIO_6	Select DIO[6] as source	0x6
	EMMC_DATA2_SRC_DIO_7	Select DIO[7] as source	0x7
	EMMC_DATA2_SRC_DIO_8	Select DIO[8] as source	0x8
	EMMC_DATA2_SRC_DIO_9	Select DIO[9] as source	0x9
	EMMC_DATA2_SRC_DIO_10	Select DIO[10] as source	0xA
	EMMC_DATA2_SRC_DIO_11	Select DIO[11] as source	0xB
	EMMC_DATA2_SRC_DIO_12	Select DIO[12] as source	0xC
	EMMC_DATA2_SRC_DIO_13	Select DIO[13] as source	0xD
	EMMC_DATA2_SRC_DIO_14	Select DIO[14] as source	0xE
	EMMC_DATA2_SRC_DIO_15	Select DIO[15] as source	0xF
	EMMC_DATA2_SRC_DIO_16	Select DIO[16] as source	0x10
	EMMC_DATA2_SRC_DIO_17	Select DIO[17] as source	0x11
	EMMC_DATA2_SRC_DIO_18	Select DIO[18] as source	0x12
	EMMC_DATA2_SRC_DIO_19	Select DIO[19] as source	0x13
	EMMC_DATA2_SRC_DIO_20	Select DIO[20] as source	0x14
	EMMC_DATA2_SRC_DIO_21	Select DIO[21] as source	0x15
	EMMC_DATA2_SRC_DIO_22	Select DIO[22] as source	0x16
	EMMC_DATA2_SRC_DIO_23	Select DIO[23] as source	0x17

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	EMMC_DATA2_SRC_DIO_24	Select DIO[24] as source	0x18
	EMMC_DATA2_SRC_DIO_25	Select DIO[25] as source	0x19
	EMMC_DATA2_SRC_DIO_26	Select DIO[26] as source	0x1A
	EMMC_DATA2_SRC_DIO_27	Select DIO[27] as source	0x1B
	EMMC_DATA2_SRC_DIO_28	Select DIO[28] as source	0x1C
	EMMC_DATA2_SRC_DIO_29	Select DIO[29] as source	0x1D
	EMMC_DATA2_SRC_DIO_30	Select DIO[30] as source	0x1E
	EMMC_DATA2_SRC_DIO_31	Select DIO[31] as source	0x1F
	EMMC_DATA2_SRC_DIO_32	Select DIO[32] as source	0x20
	EMMC_DATA2_SRC_DIO_33	Select DIO[33] as source	0x21
	EMMC_DATA2_SRC_DIO_34	Select DIO[34] as source	0x22
	EMMC_DATA2_SRC_DIO_35	Select DIO[35] as source	0x23
	EMMC_DATA2_SRC_CONST_LOW	Select constant low as source	0x24
	EMMC_DATA2_SRC_CONST_HIGH	Select constant high as source	0x25*
EMMC_DATA1	EMMC_DATA1_SRC_DIO_0	Select DIO[0] as source	0x0
	EMMC_DATA1_SRC_DIO_1	Select DIO[1] as source	0x1
	EMMC_DATA1_SRC_DIO_2	Select DIO[2] as source	0x2
	EMMC_DATA1_SRC_DIO_3	Select DIO[3] as source	0x3
	EMMC_DATA1_SRC_DIO_4	Select DIO[4] as source	0x4
	EMMC_DATA1_SRC_DIO_5	Select DIO[5] as source	0x5
	EMMC_DATA1_SRC_DIO_6	Select DIO[6] as source	0x6
	EMMC_DATA1_SRC_DIO_7	Select DIO[7] as source	0x7
	EMMC_DATA1_SRC_DIO_8	Select DIO[8] as source	0x8
	EMMC_DATA1_SRC_DIO_9	Select DIO[9] as source	0x9
	EMMC_DATA1_SRC_DIO_10	Select DIO[10] as source	0xA
	EMMC_DATA1_SRC_DIO_11	Select DIO[11] as source	0xB
	EMMC_DATA1_SRC_DIO_12	Select DIO[12] as source	0xC
	EMMC_DATA1_SRC_DIO_13	Select DIO[13] as source	0xD
	EMMC_DATA1_SRC_DIO_14	Select DIO[14] as source	0xE
	EMMC_DATA1_SRC_DIO_15	Select DIO[15] as source	0xF
	EMMC_DATA1_SRC_DIO_16	Select DIO[16] as source	0x10

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	EMMC_DATA1_SRC_DIO_17	Select DIO[17] as source	0x11
	EMMC_DATA1_SRC_DIO_18	Select DIO[18] as source	0x12
	EMMC_DATA1_SRC_DIO_19	Select DIO[19] as source	0x13
	EMMC_DATA1_SRC_DIO_20	Select DIO[20] as source	0x14
	EMMC_DATA1_SRC_DIO_21	Select DIO[21] as source	0x15
	EMMC_DATA1_SRC_DIO_22	Select DIO[22] as source	0x16
	EMMC_DATA1_SRC_DIO_23	Select DIO[23] as source	0x17
	EMMC_DATA1_SRC_DIO_24	Select DIO[24] as source	0x18
	EMMC_DATA1_SRC_DIO_25	Select DIO[25] as source	0x19
	EMMC_DATA1_SRC_DIO_26	Select DIO[26] as source	0x1A
	EMMC_DATA1_SRC_DIO_27	Select DIO[27] as source	0x1B
	EMMC_DATA1_SRC_DIO_28	Select DIO[28] as source	0x1C
	EMMC_DATA1_SRC_DIO_29	Select DIO[29] as source	0x1D
	EMMC_DATA1_SRC_DIO_30	Select DIO[30] as source	0x1E
	EMMC_DATA1_SRC_DIO_31	Select DIO[31] as source	0x1F
	EMMC_DATA1_SRC_DIO_32	Select DIO[32] as source	0x20
	EMMC_DATA1_SRC_DIO_33	Select DIO[33] as source	0x21
	EMMC_DATA1_SRC_DIO_34	Select DIO[34] as source	0x22
	EMMC_DATA1_SRC_DIO_35	Select DIO[35] as source	0x23
	EMMC_DATA1_SRC_CONST_LOW	Select constant low as source	0x24
	EMMC_DATA1_SRC_CONST_HIGH	Select constant high as source	0x25*
EMMC_DATA0	EMMC_DATA0_SRC_DIO_0	Select DIO[0] as source	0x0
	EMMC_DATA0_SRC_DIO_1	Select DIO[1] as source	0x1
	EMMC_DATA0_SRC_DIO_2	Select DIO[2] as source	0x2
	EMMC_DATA0_SRC_DIO_3	Select DIO[3] as source	0x3
	EMMC_DATA0_SRC_DIO_4	Select DIO[4] as source	0x4
	EMMC_DATA0_SRC_DIO_5	Select DIO[5] as source	0x5
	EMMC_DATA0_SRC_DIO_6	Select DIO[6] as source	0x6
	EMMC_DATA0_SRC_DIO_7	Select DIO[7] as source	0x7
	EMMC_DATA0_SRC_DIO_8	Select DIO[8] as source	0x8
	EMMC_DATA0_SRC_DIO_9	Select DIO[9] as source	0x9

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	EMMC_DATA0_SRC_DIO_10	Select DIO[10] as source	0xA
	EMMC_DATA0_SRC_DIO_11	Select DIO[11] as source	0xB
	EMMC_DATA0_SRC_DIO_12	Select DIO[12] as source	0xC
	EMMC_DATA0_SRC_DIO_13	Select DIO[13] as source	0xD
	EMMC_DATA0_SRC_DIO_14	Select DIO[14] as source	0xE
	EMMC_DATA0_SRC_DIO_15	Select DIO[15] as source	0xF
	EMMC_DATA0_SRC_DIO_16	Select DIO[16] as source	0x10
	EMMC_DATA0_SRC_DIO_17	Select DIO[17] as source	0x11
	EMMC_DATA0_SRC_DIO_18	Select DIO[18] as source	0x12
	EMMC_DATA0_SRC_DIO_19	Select DIO[19] as source	0x13
	EMMC_DATA0_SRC_DIO_20	Select DIO[20] as source	0x14
	EMMC_DATA0_SRC_DIO_21	Select DIO[21] as source	0x15
	EMMC_DATA0_SRC_DIO_22	Select DIO[22] as source	0x16
	EMMC_DATA0_SRC_DIO_23	Select DIO[23] as source	0x17
	EMMC_DATA0_SRC_DIO_24	Select DIO[24] as source	0x18
	EMMC_DATA0_SRC_DIO_25	Select DIO[25] as source	0x19
	EMMC_DATA0_SRC_DIO_26	Select DIO[26] as source	0x1A
	EMMC_DATA0_SRC_DIO_27	Select DIO[27] as source	0x1B
	EMMC_DATA0_SRC_DIO_28	Select DIO[28] as source	0x1C
	EMMC_DATA0_SRC_DIO_29	Select DIO[29] as source	0x1D
	EMMC_DATA0_SRC_DIO_30	Select DIO[30] as source	0x1E
	EMMC_DATA0_SRC_DIO_31	Select DIO[31] as source	0x1F
	EMMC_DATA0_SRC_DIO_32	Select DIO[32] as source	0x20
	EMMC_DATA0_SRC_DIO_33	Select DIO[33] as source	0x21
	EMMC_DATA0_SRC_DIO_34	Select DIO[34] as source	0x22
	EMMC_DATA0_SRC_DIO_35	Select DIO[35] as source	0x23
	EMMC_DATA0_SRC_CONST_LOW	Select constant low as source	0x24
	EMMC_DATA0_SRC_CONST_HIGH	Select constant high as source	0x25*

Ezairo 8300 Hardware Reference

16.4.11 DIO_SRC_EMMC_1

Bit Field	Read/Write	Field Name	Description
29:24	RW	EMMC_DATA7	eMMC DATA7 input selection
21:16	RW	EMMC_DATA6	eMMC DATA6 input selection
13:8	RW	EMMC_DATA5	eMMC DATA5 input selection
5:0	RW	EMMC_DATA4	eMMC DATA4 input selection

Field Name	Value Symbol	Value Description	Hex Value
EMMC_DATA7	EMMC_DATA7_SRC_DIO_0	Select DIO[0] as source	0x0
	EMMC_DATA7_SRC_DIO_1	Select DIO[1] as source	0x1
	EMMC_DATA7_SRC_DIO_2	Select DIO[2] as source	0x2
	EMMC_DATA7_SRC_DIO_3	Select DIO[3] as source	0x3
	EMMC_DATA7_SRC_DIO_4	Select DIO[4] as source	0x4
	EMMC_DATA7_SRC_DIO_5	Select DIO[5] as source	0x5
	EMMC_DATA7_SRC_DIO_6	Select DIO[6] as source	0x6
	EMMC_DATA7_SRC_DIO_7	Select DIO[7] as source	0x7
	EMMC_DATA7_SRC_DIO_8	Select DIO[8] as source	0x8
	EMMC_DATA7_SRC_DIO_9	Select DIO[9] as source	0x9
	EMMC_DATA7_SRC_DIO_10	Select DIO[10] as source	0xA
	EMMC_DATA7_SRC_DIO_11	Select DIO[11] as source	0xB
	EMMC_DATA7_SRC_DIO_12	Select DIO[12] as source	0xC
	EMMC_DATA7_SRC_DIO_13	Select DIO[13] as source	0xD
	EMMC_DATA7_SRC_DIO_14	Select DIO[14] as source	0xE
	EMMC_DATA7_SRC_DIO_15	Select DIO[15] as source	0xF
	EMMC_DATA7_SRC_DIO_16	Select DIO[16] as source	0x10
	EMMC_DATA7_SRC_DIO_17	Select DIO[17] as source	0x11
	EMMC_DATA7_SRC_DIO_18	Select DIO[18] as source	0x12
	EMMC_DATA7_SRC_DIO_19	Select DIO[19] as source	0x13
	EMMC_DATA7_SRC_DIO_20	Select DIO[20] as source	0x14
	EMMC_DATA7_SRC_DIO_21	Select DIO[21] as source	0x15
	EMMC_DATA7_SRC_DIO_22	Select DIO[22] as source	0x16
	EMMC_DATA7_SRC_DIO_23	Select DIO[23] as source	0x17

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	EMMC_DATA7_SRC_DIO_24	Select DIO[24] as source	0x18
	EMMC_DATA7_SRC_DIO_25	Select DIO[25] as source	0x19
	EMMC_DATA7_SRC_DIO_26	Select DIO[26] as source	0x1A
	EMMC_DATA7_SRC_DIO_27	Select DIO[27] as source	0x1B
	EMMC_DATA7_SRC_DIO_28	Select DIO[28] as source	0x1C
	EMMC_DATA7_SRC_DIO_29	Select DIO[29] as source	0x1D
	EMMC_DATA7_SRC_DIO_30	Select DIO[30] as source	0x1E
	EMMC_DATA7_SRC_DIO_31	Select DIO[31] as source	0x1F
	EMMC_DATA7_SRC_DIO_32	Select DIO[32] as source	0x20
	EMMC_DATA7_SRC_DIO_33	Select DIO[33] as source	0x21
	EMMC_DATA7_SRC_DIO_34	Select DIO[34] as source	0x22
	EMMC_DATA7_SRC_DIO_35	Select DIO[35] as source	0x23
	EMMC_DATA7_SRC_CONST_LOW	Select constant low as source	0x24
	EMMC_DATA7_SRC_CONST_HIGH	Select constant high as source	0x25*
EMMC_DATA6	EMMC_DATA6_SRC_DIO_0	Select DIO[0] as source	0x0
	EMMC_DATA6_SRC_DIO_1	Select DIO[1] as source	0x1
	EMMC_DATA6_SRC_DIO_2	Select DIO[2] as source	0x2
	EMMC_DATA6_SRC_DIO_3	Select DIO[3] as source	0x3
	EMMC_DATA6_SRC_DIO_4	Select DIO[4] as source	0x4
	EMMC_DATA6_SRC_DIO_5	Select DIO[5] as source	0x5
	EMMC_DATA6_SRC_DIO_6	Select DIO[6] as source	0x6
	EMMC_DATA6_SRC_DIO_7	Select DIO[7] as source	0x7
	EMMC_DATA6_SRC_DIO_8	Select DIO[8] as source	0x8
	EMMC_DATA6_SRC_DIO_9	Select DIO[9] as source	0x9
	EMMC_DATA6_SRC_DIO_10	Select DIO[10] as source	0xA
	EMMC_DATA6_SRC_DIO_11	Select DIO[11] as source	0xB
	EMMC_DATA6_SRC_DIO_12	Select DIO[12] as source	0xC
	EMMC_DATA6_SRC_DIO_13	Select DIO[13] as source	0xD
	EMMC_DATA6_SRC_DIO_14	Select DIO[14] as source	0xE
	EMMC_DATA6_SRC_DIO_15	Select DIO[15] as source	0xF
	EMMC_DATA6_SRC_DIO_16	Select DIO[16] as source	0x10

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	EMMC_DATA6_SRC_DIO_17	Select DIO[17] as source	0x11
	EMMC_DATA6_SRC_DIO_18	Select DIO[18] as source	0x12
	EMMC_DATA6_SRC_DIO_19	Select DIO[19] as source	0x13
	EMMC_DATA6_SRC_DIO_20	Select DIO[20] as source	0x14
	EMMC_DATA6_SRC_DIO_21	Select DIO[21] as source	0x15
	EMMC_DATA6_SRC_DIO_22	Select DIO[22] as source	0x16
	EMMC_DATA6_SRC_DIO_23	Select DIO[23] as source	0x17
	EMMC_DATA6_SRC_DIO_24	Select DIO[24] as source	0x18
	EMMC_DATA6_SRC_DIO_25	Select DIO[25] as source	0x19
	EMMC_DATA6_SRC_DIO_26	Select DIO[26] as source	0x1A
	EMMC_DATA6_SRC_DIO_27	Select DIO[27] as source	0x1B
	EMMC_DATA6_SRC_DIO_28	Select DIO[28] as source	0x1C
	EMMC_DATA6_SRC_DIO_29	Select DIO[29] as source	0x1D
	EMMC_DATA6_SRC_DIO_30	Select DIO[30] as source	0x1E
	EMMC_DATA6_SRC_DIO_31	Select DIO[31] as source	0x1F
	EMMC_DATA6_SRC_DIO_32	Select DIO[32] as source	0x20
	EMMC_DATA6_SRC_DIO_33	Select DIO[33] as source	0x21
	EMMC_DATA6_SRC_DIO_34	Select DIO[34] as source	0x22
	EMMC_DATA6_SRC_DIO_35	Select DIO[35] as source	0x23
	EMMC_DATA6_SRC_CONST_LOW	Select constant low as source	0x24
	EMMC_DATA6_SRC_CONST_HIGH	Select constant high as source	0x25*
EMMC_DATA5	EMMC_DATA5_SRC_DIO_0	Select DIO[0] as source	0x0
	EMMC_DATA5_SRC_DIO_1	Select DIO[1] as source	0x1
	EMMC_DATA5_SRC_DIO_2	Select DIO[2] as source	0x2
	EMMC_DATA5_SRC_DIO_3	Select DIO[3] as source	0x3
	EMMC_DATA5_SRC_DIO_4	Select DIO[4] as source	0x4
	EMMC_DATA5_SRC_DIO_5	Select DIO[5] as source	0x5
	EMMC_DATA5_SRC_DIO_6	Select DIO[6] as source	0x6
	EMMC_DATA5_SRC_DIO_7	Select DIO[7] as source	0x7
	EMMC_DATA5_SRC_DIO_8	Select DIO[8] as source	0x8
	EMMC_DATA5_SRC_DIO_9	Select DIO[9] as source	0x9

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	EMMC_DATA5_SRC_DIO_10	Select DIO[10] as source	0xA
	EMMC_DATA5_SRC_DIO_11	Select DIO[11] as source	0xB
	EMMC_DATA5_SRC_DIO_12	Select DIO[12] as source	0xC
	EMMC_DATA5_SRC_DIO_13	Select DIO[13] as source	0xD
	EMMC_DATA5_SRC_DIO_14	Select DIO[14] as source	0xE
	EMMC_DATA5_SRC_DIO_15	Select DIO[15] as source	0xF
	EMMC_DATA5_SRC_DIO_16	Select DIO[16] as source	0x10
	EMMC_DATA5_SRC_DIO_17	Select DIO[17] as source	0x11
	EMMC_DATA5_SRC_DIO_18	Select DIO[18] as source	0x12
	EMMC_DATA5_SRC_DIO_19	Select DIO[19] as source	0x13
	EMMC_DATA5_SRC_DIO_20	Select DIO[20] as source	0x14
	EMMC_DATA5_SRC_DIO_21	Select DIO[21] as source	0x15
	EMMC_DATA5_SRC_DIO_22	Select DIO[22] as source	0x16
	EMMC_DATA5_SRC_DIO_23	Select DIO[23] as source	0x17
	EMMC_DATA5_SRC_DIO_24	Select DIO[24] as source	0x18
	EMMC_DATA5_SRC_DIO_25	Select DIO[25] as source	0x19
	EMMC_DATA5_SRC_DIO_26	Select DIO[26] as source	0x1A
	EMMC_DATA5_SRC_DIO_27	Select DIO[27] as source	0x1B
	EMMC_DATA5_SRC_DIO_28	Select DIO[28] as source	0x1C
	EMMC_DATA5_SRC_DIO_29	Select DIO[29] as source	0x1D
	EMMC_DATA5_SRC_DIO_30	Select DIO[30] as source	0x1E
	EMMC_DATA5_SRC_DIO_31	Select DIO[31] as source	0x1F
	EMMC_DATA5_SRC_DIO_32	Select DIO[32] as source	0x20
	EMMC_DATA5_SRC_DIO_33	Select DIO[33] as source	0x21
	EMMC_DATA5_SRC_DIO_34	Select DIO[34] as source	0x22
	EMMC_DATA5_SRC_DIO_35	Select DIO[35] as source	0x23
	EMMC_DATA5_SRC_CONST_LOW	Select constant low as source	0x24
	EMMC_DATA5_SRC_CONST_HIGH	Select constant high as source	0x25*
EMMC_DATA4	EMMC_DATA4_SRC_DIO_0	Select DIO[0] as source	0x0
	EMMC_DATA4_SRC_DIO_1	Select DIO[1] as source	0x1
	EMMC_DATA4_SRC_DIO_2	Select DIO[2] as source	0x2

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	EMMC_DATA4_SRC_DIO_3	Select DIO[3] as source	0x3
	EMMC_DATA4_SRC_DIO_4	Select DIO[4] as source	0x4
	EMMC_DATA4_SRC_DIO_5	Select DIO[5] as source	0x5
	EMMC_DATA4_SRC_DIO_6	Select DIO[6] as source	0x6
	EMMC_DATA4_SRC_DIO_7	Select DIO[7] as source	0x7
	EMMC_DATA4_SRC_DIO_8	Select DIO[8] as source	0x8
	EMMC_DATA4_SRC_DIO_9	Select DIO[9] as source	0x9
	EMMC_DATA4_SRC_DIO_10	Select DIO[10] as source	0xA
	EMMC_DATA4_SRC_DIO_11	Select DIO[11] as source	0xB
	EMMC_DATA4_SRC_DIO_12	Select DIO[12] as source	0xC
	EMMC_DATA4_SRC_DIO_13	Select DIO[13] as source	0xD
	EMMC_DATA4_SRC_DIO_14	Select DIO[14] as source	0xE
	EMMC_DATA4_SRC_DIO_15	Select DIO[15] as source	0xF
	EMMC_DATA4_SRC_DIO_16	Select DIO[16] as source	0x10
	EMMC_DATA4_SRC_DIO_17	Select DIO[17] as source	0x11
	EMMC_DATA4_SRC_DIO_18	Select DIO[18] as source	0x12
	EMMC_DATA4_SRC_DIO_19	Select DIO[19] as source	0x13
	EMMC_DATA4_SRC_DIO_20	Select DIO[20] as source	0x14
	EMMC_DATA4_SRC_DIO_21	Select DIO[21] as source	0x15
	EMMC_DATA4_SRC_DIO_22	Select DIO[22] as source	0x16
	EMMC_DATA4_SRC_DIO_23	Select DIO[23] as source	0x17
	EMMC_DATA4_SRC_DIO_24	Select DIO[24] as source	0x18
	EMMC_DATA4_SRC_DIO_25	Select DIO[25] as source	0x19
	EMMC_DATA4_SRC_DIO_26	Select DIO[26] as source	0x1A
	EMMC_DATA4_SRC_DIO_27	Select DIO[27] as source	0x1B
	EMMC_DATA4_SRC_DIO_28	Select DIO[28] as source	0x1C
	EMMC_DATA4_SRC_DIO_29	Select DIO[29] as source	0x1D
	EMMC_DATA4_SRC_DIO_30	Select DIO[30] as source	0x1E
	EMMC_DATA4_SRC_DIO_31	Select DIO[31] as source	0x1F
	EMMC_DATA4_SRC_DIO_32	Select DIO[32] as source	0x20
	EMMC_DATA4_SRC_DIO_33	Select DIO[33] as source	0x21

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	EMMC_DATA4_SRC_DIO_34	Select DIO[34] as source	0x22
	EMMC_DATA4_SRC_DIO_35	Select DIO[35] as source	0x23
	EMMC_DATA4_SRC_CONST_LOW	Select constant low as source	0x24
	EMMC_DATA4_SRC_CONST_HIGH	Select constant high as source	0x25*

16.4.12 DIO_SRC_EMMC_2

Bit Field	Read/Write	Field Name	Description
5:0	RW	EMMC_CMD	eMMC CMD input selection

Field Name	Value Symbol	Value Description	Hex Value
EMMC_CMD	EMMC_CMD_SRC_DIO_0	Select DIO[0] as source	0x0
	EMMC_CMD_SRC_DIO_1	Select DIO[1] as source	0x1
	EMMC_CMD_SRC_DIO_2	Select DIO[2] as source	0x2
	EMMC_CMD_SRC_DIO_3	Select DIO[3] as source	0x3
	EMMC_CMD_SRC_DIO_4	Select DIO[4] as source	0x4
	EMMC_CMD_SRC_DIO_5	Select DIO[5] as source	0x5
	EMMC_CMD_SRC_DIO_6	Select DIO[6] as source	0x6
	EMMC_CMD_SRC_DIO_7	Select DIO[7] as source	0x7
	EMMC_CMD_SRC_DIO_8	Select DIO[8] as source	0x8
	EMMC_CMD_SRC_DIO_9	Select DIO[9] as source	0x9
	EMMC_CMD_SRC_DIO_10	Select DIO[10] as source	0xA
	EMMC_CMD_SRC_DIO_11	Select DIO[11] as source	0xB
	EMMC_CMD_SRC_DIO_12	Select DIO[12] as source	0xC
	EMMC_CMD_SRC_DIO_13	Select DIO[13] as source	0xD
	EMMC_CMD_SRC_DIO_14	Select DIO[14] as source	0xE
	EMMC_CMD_SRC_DIO_15	Select DIO[15] as source	0xF
	EMMC_CMD_SRC_DIO_16	Select DIO[16] as source	0x10
	EMMC_CMD_SRC_DIO_17	Select DIO[17] as source	0x11
	EMMC_CMD_SRC_DIO_18	Select DIO[18] as source	0x12
	EMMC_CMD_SRC_DIO_19	Select DIO[19] as source	0x13
	EMMC_CMD_SRC_DIO_20	Select DIO[20] as source	0x14

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	EMMC_CMD_SRC_DIO_21	Select DIO[21] as source	0x15
	EMMC_CMD_SRC_DIO_22	Select DIO[22] as source	0x16
	EMMC_CMD_SRC_DIO_23	Select DIO[23] as source	0x17
	EMMC_CMD_SRC_DIO_24	Select DIO[24] as source	0x18
	EMMC_CMD_SRC_DIO_25	Select DIO[25] as source	0x19
	EMMC_CMD_SRC_DIO_26	Select DIO[26] as source	0x1A
	EMMC_CMD_SRC_DIO_27	Select DIO[27] as source	0x1B
	EMMC_CMD_SRC_DIO_28	Select DIO[28] as source	0x1C
	EMMC_CMD_SRC_DIO_29	Select DIO[29] as source	0x1D
	EMMC_CMD_SRC_DIO_30	Select DIO[30] as source	0x1E
	EMMC_CMD_SRC_DIO_31	Select DIO[31] as source	0x1F
	EMMC_CMD_SRC_DIO_32	Select DIO[32] as source	0x20
	EMMC_CMD_SRC_DIO_33	Select DIO[33] as source	0x21
	EMMC_CMD_SRC_DIO_34	Select DIO[34] as source	0x22
	EMMC_CMD_SRC_DIO_35	Select DIO[35] as source	0x23
	EMMC_CMD_SRC_CONST_LOW	Select constant low as source	0x24
	EMMC_CMD_SRC_CONST_HIGH	Select constant high as source	0x25*

16.4.13 DIO_SRC_DMIC_CLK

Bit Field	Read/Write	Field Name	Description
5:0	RW	CLK	DMIC clock input selection

Field Name	Value Symbol	Value Description	Hex Value
CLK	DMIC_CLK_SRC_DIO_0	Select DIO[0] as source	0x0
	DMIC_CLK_SRC_DIO_1	Select DIO[1] as source	0x1
	DMIC_CLK_SRC_DIO_2	Select DIO[2] as source	0x2
	DMIC_CLK_SRC_DIO_3	Select DIO[3] as source	0x3
	DMIC_CLK_SRC_DIO_4	Select DIO[4] as source	0x4
	DMIC_CLK_SRC_DIO_5	Select DIO[5] as source	0x5
	DMIC_CLK_SRC_DIO_6	Select DIO[6] as source	0x6
	DMIC_CLK_SRC_DIO_7	Select DIO[7] as source	0x7

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DMIC_CLK_SRC_DIO_8	Select DIO[8] as source	0x8
	DMIC_CLK_SRC_DIO_9	Select DIO[9] as source	0x9
	DMIC_CLK_SRC_DIO_10	Select DIO[10] as source	0xA
	DMIC_CLK_SRC_DIO_11	Select DIO[11] as source	0xB
	DMIC_CLK_SRC_DIO_12	Select DIO[12] as source	0xC
	DMIC_CLK_SRC_DIO_13	Select DIO[13] as source	0xD
	DMIC_CLK_SRC_DIO_14	Select DIO[14] as source	0xE
	DMIC_CLK_SRC_DIO_15	Select DIO[15] as source	0xF
	DMIC_CLK_SRC_DIO_16	Select DIO[16] as source	0x10
	DMIC_CLK_SRC_DIO_17	Select DIO[17] as source	0x11
	DMIC_CLK_SRC_DIO_18	Select DIO[18] as source	0x12
	DMIC_CLK_SRC_DIO_19	Select DIO[19] as source	0x13
	DMIC_CLK_SRC_DIO_20	Select DIO[20] as source	0x14
	DMIC_CLK_SRC_DIO_21	Select DIO[21] as source	0x15
	DMIC_CLK_SRC_DIO_22	Select DIO[22] as source	0x16
	DMIC_CLK_SRC_DIO_23	Select DIO[23] as source	0x17
	DMIC_CLK_SRC_DIO_24	Select DIO[24] as source	0x18
	DMIC_CLK_SRC_DIO_25	Select DIO[25] as source	0x19
	DMIC_CLK_SRC_DIO_26	Select DIO[26] as source	0x1A
	DMIC_CLK_SRC_DIO_27	Select DIO[27] as source	0x1B
	DMIC_CLK_SRC_DIO_28	Select DIO[28] as source	0x1C
	DMIC_CLK_SRC_DIO_29	Select DIO[29] as source	0x1D
	DMIC_CLK_SRC_DIO_30	Select DIO[30] as source	0x1E
	DMIC_CLK_SRC_DIO_31	Select DIO[31] as source	0x1F
	DMIC_CLK_SRC_DIO_32	Select DIO[32] as source	0x20
	DMIC_CLK_SRC_DIO_33	Select DIO[33] as source	0x21
	DMIC_CLK_SRC_DIO_34	Select DIO[34] as source	0x22
	DMIC_CLK_SRC_DIO_35	Select DIO[35] as source	0x23
	DMIC_CLK_SRC_CONST_LOW	Select constant low as source	0x24
	DMIC_CLK_SRC_CONST_HIGH	Select constant high as source	0x25*

Ezairo 8300 Hardware Reference

16.4.14 DIO_SRC_DMIC_DATA

Bit Field	Read/Write	Field Name	Description
29:24	RW	DMIC3	DMIC data input selection
21:16	RW	DMIC2	DMIC data input selection
13:8	RW	DMIC1	DMIC data input selection
5:0	RW	DMIC0	DMIC data input selection

Field Name	Value Symbol	Value Description	Hex Value
DMIC3	DMIC3_DATA_SRC_DIO_0	Select DIO[0] as source	0x0
	DMIC3_DATA_SRC_DIO_1	Select DIO[1] as source	0x1
	DMIC3_DATA_SRC_DIO_2	Select DIO[2] as source	0x2
	DMIC3_DATA_SRC_DIO_3	Select DIO[3] as source	0x3
	DMIC3_DATA_SRC_DIO_4	Select DIO[4] as source	0x4
	DMIC3_DATA_SRC_DIO_5	Select DIO[5] as source	0x5
	DMIC3_DATA_SRC_DIO_6	Select DIO[6] as source	0x6
	DMIC3_DATA_SRC_DIO_7	Select DIO[7] as source	0x7
	DMIC3_DATA_SRC_DIO_8	Select DIO[8] as source	0x8
	DMIC3_DATA_SRC_DIO_9	Select DIO[9] as source	0x9
	DMIC3_DATA_SRC_DIO_10	Select DIO[10] as source	0xA
	DMIC3_DATA_SRC_DIO_11	Select DIO[11] as source	0xB
	DMIC3_DATA_SRC_DIO_12	Select DIO[12] as source	0xC
	DMIC3_DATA_SRC_DIO_13	Select DIO[13] as source	0xD
	DMIC3_DATA_SRC_DIO_14	Select DIO[14] as source	0xE
	DMIC3_DATA_SRC_DIO_15	Select DIO[15] as source	0xF
	DMIC3_DATA_SRC_DIO_16	Select DIO[16] as source	0x10
	DMIC3_DATA_SRC_DIO_17	Select DIO[17] as source	0x11
	DMIC3_DATA_SRC_DIO_18	Select DIO[18] as source	0x12
	DMIC3_DATA_SRC_DIO_19	Select DIO[19] as source	0x13
	DMIC3_DATA_SRC_DIO_20	Select DIO[20] as source	0x14
	DMIC3_DATA_SRC_DIO_21	Select DIO[21] as source	0x15
	DMIC3_DATA_SRC_DIO_22	Select DIO[22] as source	0x16
	DMIC3_DATA_SRC_DIO_23	Select DIO[23] as source	0x17

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DMIC3_DATA_SRC_DIO_24	Select DIO[24] as source	0x18
	DMIC3_DATA_SRC_DIO_25	Select DIO[25] as source	0x19
	DMIC3_DATA_SRC_DIO_26	Select DIO[26] as source	0x1A
	DMIC3_DATA_SRC_DIO_27	Select DIO[27] as source	0x1B
	DMIC3_DATA_SRC_DIO_28	Select DIO[28] as source	0x1C
	DMIC3_DATA_SRC_DIO_29	Select DIO[29] as source	0x1D
	DMIC3_DATA_SRC_DIO_30	Select DIO[30] as source	0x1E
	DMIC3_DATA_SRC_DIO_31	Select DIO[31] as source	0x1F
	DMIC3_DATA_SRC_DIO_32	Select DIO[32] as source	0x20
	DMIC3_DATA_SRC_DIO_33	Select DIO[33] as source	0x21
	DMIC3_DATA_SRC_DIO_34	Select DIO[34] as source	0x22
	DMIC3_DATA_SRC_DIO_35	Select DIO[35] as source	0x23
	DMIC3_DATA_SRC_CONST_LOW	Select constant low as source	0x24
	DMIC3_DATA_SRC_CONST_HIGH	Select constant high as source	0x25*
DMIC2	DMIC2_DATA_SRC_DIO_0	Select DIO[0] as source	0x0
	DMIC2_DATA_SRC_DIO_1	Select DIO[1] as source	0x1
	DMIC2_DATA_SRC_DIO_2	Select DIO[2] as source	0x2
	DMIC2_DATA_SRC_DIO_3	Select DIO[3] as source	0x3
	DMIC2_DATA_SRC_DIO_4	Select DIO[4] as source	0x4
	DMIC2_DATA_SRC_DIO_5	Select DIO[5] as source	0x5
	DMIC2_DATA_SRC_DIO_6	Select DIO[6] as source	0x6
	DMIC2_DATA_SRC_DIO_7	Select DIO[7] as source	0x7
	DMIC2_DATA_SRC_DIO_8	Select DIO[8] as source	0x8
	DMIC2_DATA_SRC_DIO_9	Select DIO[9] as source	0x9
	DMIC2_DATA_SRC_DIO_10	Select DIO[10] as source	0xA
	DMIC2_DATA_SRC_DIO_11	Select DIO[11] as source	0xB
	DMIC2_DATA_SRC_DIO_12	Select DIO[12] as source	0xC
	DMIC2_DATA_SRC_DIO_13	Select DIO[13] as source	0xD
	DMIC2_DATA_SRC_DIO_14	Select DIO[14] as source	0xE
	DMIC2_DATA_SRC_DIO_15	Select DIO[15] as source	0xF
	DMIC2_DATA_SRC_DIO_16	Select DIO[16] as source	0x10

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DMIC2_DATA_SRC_DIO_17	Select DIO[17] as source	0x11
	DMIC2_DATA_SRC_DIO_18	Select DIO[18] as source	0x12
	DMIC2_DATA_SRC_DIO_19	Select DIO[19] as source	0x13
	DMIC2_DATA_SRC_DIO_20	Select DIO[20] as source	0x14
	DMIC2_DATA_SRC_DIO_21	Select DIO[21] as source	0x15
	DMIC2_DATA_SRC_DIO_22	Select DIO[22] as source	0x16
	DMIC2_DATA_SRC_DIO_23	Select DIO[23] as source	0x17
	DMIC2_DATA_SRC_DIO_24	Select DIO[24] as source	0x18
	DMIC2_DATA_SRC_DIO_25	Select DIO[25] as source	0x19
	DMIC2_DATA_SRC_DIO_26	Select DIO[26] as source	0x1A
	DMIC2_DATA_SRC_DIO_27	Select DIO[27] as source	0x1B
	DMIC2_DATA_SRC_DIO_28	Select DIO[28] as source	0x1C
	DMIC2_DATA_SRC_DIO_29	Select DIO[29] as source	0x1D
	DMIC2_DATA_SRC_DIO_30	Select DIO[30] as source	0x1E
	DMIC2_DATA_SRC_DIO_31	Select DIO[31] as source	0x1F
	DMIC2_DATA_SRC_DIO_32	Select DIO[32] as source	0x20
	DMIC2_DATA_SRC_DIO_33	Select DIO[33] as source	0x21
	DMIC2_DATA_SRC_DIO_34	Select DIO[34] as source	0x22
	DMIC2_DATA_SRC_DIO_35	Select DIO[35] as source	0x23
	DMIC2_DATA_SRC_CONST_LOW	Select constant low as source	0x24
	DMIC2_DATA_SRC_CONST_HIGH	Select constant high as source	0x25*
DMIC1	DMIC1_DATA_SRC_DIO_0	Select DIO[0] as source	0x0
	DMIC1_DATA_SRC_DIO_1	Select DIO[1] as source	0x1
	DMIC1_DATA_SRC_DIO_2	Select DIO[2] as source	0x2
	DMIC1_DATA_SRC_DIO_3	Select DIO[3] as source	0x3
	DMIC1_DATA_SRC_DIO_4	Select DIO[4] as source	0x4
	DMIC1_DATA_SRC_DIO_5	Select DIO[5] as source	0x5
	DMIC1_DATA_SRC_DIO_6	Select DIO[6] as source	0x6
	DMIC1_DATA_SRC_DIO_7	Select DIO[7] as source	0x7
	DMIC1_DATA_SRC_DIO_8	Select DIO[8] as source	0x8
	DMIC1_DATA_SRC_DIO_9	Select DIO[9] as source	0x9

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DMIC1_DATA_SRC_DIO_10	Select DIO[10] as source	0xA
	DMIC1_DATA_SRC_DIO_11	Select DIO[11] as source	0xB
	DMIC1_DATA_SRC_DIO_12	Select DIO[12] as source	0xC
	DMIC1_DATA_SRC_DIO_13	Select DIO[13] as source	0xD
	DMIC1_DATA_SRC_DIO_14	Select DIO[14] as source	0xE
	DMIC1_DATA_SRC_DIO_15	Select DIO[15] as source	0xF
	DMIC1_DATA_SRC_DIO_16	Select DIO[16] as source	0x10
	DMIC1_DATA_SRC_DIO_17	Select DIO[17] as source	0x11
	DMIC1_DATA_SRC_DIO_18	Select DIO[18] as source	0x12
	DMIC1_DATA_SRC_DIO_19	Select DIO[19] as source	0x13
	DMIC1_DATA_SRC_DIO_20	Select DIO[20] as source	0x14
	DMIC1_DATA_SRC_DIO_21	Select DIO[21] as source	0x15
	DMIC1_DATA_SRC_DIO_22	Select DIO[22] as source	0x16
	DMIC1_DATA_SRC_DIO_23	Select DIO[23] as source	0x17
	DMIC1_DATA_SRC_DIO_24	Select DIO[24] as source	0x18
	DMIC1_DATA_SRC_DIO_25	Select DIO[25] as source	0x19
	DMIC1_DATA_SRC_DIO_26	Select DIO[26] as source	0x1A
	DMIC1_DATA_SRC_DIO_27	Select DIO[27] as source	0x1B
	DMIC1_DATA_SRC_DIO_28	Select DIO[28] as source	0x1C
	DMIC1_DATA_SRC_DIO_29	Select DIO[29] as source	0x1D
	DMIC1_DATA_SRC_DIO_30	Select DIO[30] as source	0x1E
	DMIC1_DATA_SRC_DIO_31	Select DIO[31] as source	0x1F
	DMIC1_DATA_SRC_DIO_32	Select DIO[32] as source	0x20
	DMIC1_DATA_SRC_DIO_33	Select DIO[33] as source	0x21
	DMIC1_DATA_SRC_DIO_34	Select DIO[34] as source	0x22
	DMIC1_DATA_SRC_DIO_35	Select DIO[35] as source	0x23
	DMIC1_DATA_SRC_CONST_LOW	Select constant low as source	0x24
	DMIC1_DATA_SRC_CONST_HIGH	Select constant high as source	0x25*
DMIC0	DMIC0_DATA_SRC_DIO_0	Select DIO[0] as source	0x0
	DMIC0_DATA_SRC_DIO_1	Select DIO[1] as source	0x1
	DMIC0_DATA_SRC_DIO_2	Select DIO[2] as source	0x2

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DMIC0_DATA_SRC_DIO_3	Select DIO[3] as source	0x3
	DMIC0_DATA_SRC_DIO_4	Select DIO[4] as source	0x4
	DMIC0_DATA_SRC_DIO_5	Select DIO[5] as source	0x5
	DMIC0_DATA_SRC_DIO_6	Select DIO[6] as source	0x6
	DMIC0_DATA_SRC_DIO_7	Select DIO[7] as source	0x7
	DMIC0_DATA_SRC_DIO_8	Select DIO[8] as source	0x8
	DMIC0_DATA_SRC_DIO_9	Select DIO[9] as source	0x9
	DMIC0_DATA_SRC_DIO_10	Select DIO[10] as source	0xA
	DMIC0_DATA_SRC_DIO_11	Select DIO[11] as source	0xB
	DMIC0_DATA_SRC_DIO_12	Select DIO[12] as source	0xC
	DMIC0_DATA_SRC_DIO_13	Select DIO[13] as source	0xD
	DMIC0_DATA_SRC_DIO_14	Select DIO[14] as source	0xE
	DMIC0_DATA_SRC_DIO_15	Select DIO[15] as source	0xF
	DMIC0_DATA_SRC_DIO_16	Select DIO[16] as source	0x10
	DMIC0_DATA_SRC_DIO_17	Select DIO[17] as source	0x11
	DMIC0_DATA_SRC_DIO_18	Select DIO[18] as source	0x12
	DMIC0_DATA_SRC_DIO_19	Select DIO[19] as source	0x13
	DMIC0_DATA_SRC_DIO_20	Select DIO[20] as source	0x14
	DMIC0_DATA_SRC_DIO_21	Select DIO[21] as source	0x15
	DMIC0_DATA_SRC_DIO_22	Select DIO[22] as source	0x16
	DMIC0_DATA_SRC_DIO_23	Select DIO[23] as source	0x17
	DMIC0_DATA_SRC_DIO_24	Select DIO[24] as source	0x18
	DMIC0_DATA_SRC_DIO_25	Select DIO[25] as source	0x19
	DMIC0_DATA_SRC_DIO_26	Select DIO[26] as source	0x1A
	DMIC0_DATA_SRC_DIO_27	Select DIO[27] as source	0x1B
	DMIC0_DATA_SRC_DIO_28	Select DIO[28] as source	0x1C
	DMIC0_DATA_SRC_DIO_29	Select DIO[29] as source	0x1D
	DMIC0_DATA_SRC_DIO_30	Select DIO[30] as source	0x1E
	DMIC0_DATA_SRC_DIO_31	Select DIO[31] as source	0x1F
	DMIC0_DATA_SRC_DIO_32	Select DIO[32] as source	0x20
	DMIC0_DATA_SRC_DIO_33	Select DIO[33] as source	0x21

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DMIC0_DATA_SRC_DIO_34	Select DIO[34] as source	0x22
	DMIC0_DATA_SRC_DIO_35	Select DIO[35] as source	0x23
	DMIC0_DATA_SRC_CONST_LOW	Select constant low as source	0x24
	DMIC0_DATA_SRC_CONST_HIGH	Select constant high as source	0x25*

16.4.15 DIO_SRC_DSP_JTAG

Bit Field	Read/Write	Field Name	Description
21:16	RW	TDI	DSP_TDI input selection
13:8	RW	TMS	DSP_TMS input selection
5:0	RW	TCK	DSP_TCK input selection

Field Name	Value Symbol	Value Description	Hex Value
TDI	DSP_TDI_SRC_DIO_0	Select DIO[0] as source	0x0
	DSP_TDI_SRC_DIO_1	Select DIO[1] as source	0x1
	DSP_TDI_SRC_DIO_2	Select DIO[2] as source	0x2
	DSP_TDI_SRC_DIO_3	Select DIO[3] as source	0x3
	DSP_TDI_SRC_DIO_4	Select DIO[4] as source	0x4
	DSP_TDI_SRC_DIO_5	Select DIO[5] as source	0x5
	DSP_TDI_SRC_DIO_6	Select DIO[6] as source	0x6
	DSP_TDI_SRC_DIO_7	Select DIO[7] as source	0x7
	DSP_TDI_SRC_DIO_8	Select DIO[8] as source	0x8
	DSP_TDI_SRC_DIO_9	Select DIO[9] as source	0x9
	DSP_TDI_SRC_DIO_10	Select DIO[10] as source	0xA
	DSP_TDI_SRC_DIO_11	Select DIO[11] as source	0xB
	DSP_TDI_SRC_DIO_12	Select DIO[12] as source	0xC
	DSP_TDI_SRC_DIO_13	Select DIO[13] as source	0xD
	DSP_TDI_SRC_DIO_14	Select DIO[14] as source	0xE
	DSP_TDI_SRC_DIO_15	Select DIO[15] as source	0xF
	DSP_TDI_SRC_DIO_16	Select DIO[16] as source	0x10
	DSP_TDI_SRC_DIO_17	Select DIO[17] as source	0x11
	DSP_TDI_SRC_DIO_18	Select DIO[18] as source	0x12

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DSP_TDI_SRC_DIO_19	Select DIO[19] as source	0x13
	DSP_TDI_SRC_DIO_20	Select DIO[20] as source	0x14
	DSP_TDI_SRC_DIO_21	Select DIO[21] as source	0x15
	DSP_TDI_SRC_DIO_22	Select DIO[22] as source	0x16
	DSP_TDI_SRC_DIO_23	Select DIO[23] as source	0x17
	DSP_TDI_SRC_DIO_24	Select DIO[24] as source	0x18
	DSP_TDI_SRC_DIO_25	Select DIO[25] as source	0x19
	DSP_TDI_SRC_DIO_26	Select DIO[26] as source	0x1A
	DSP_TDI_SRC_DIO_27	Select DIO[27] as source	0x1B
	DSP_TDI_SRC_DIO_28	Select DIO[28] as source	0x1C
	DSP_TDI_SRC_DIO_29	Select DIO[29] as source	0x1D
	DSP_TDI_SRC_DIO_30	Select DIO[30] as source	0x1E
	DSP_TDI_SRC_DIO_31	Select DIO[31] as source	0x1F
	DSP_TDI_SRC_DIO_32	Select DIO[32] as source	0x20
	DSP_TDI_SRC_DIO_33	Select DIO[33] as source	0x21
	DSP_TDI_SRC_DIO_34	Select DIO[34] as source	0x22
	DSP_TDI_SRC_DIO_35	Select DIO[35] as source	0x23
	DSP_TDI_SRC_CONST_LOW	Select constant low as source	0x24
	DSP_TDI_SRC_CONST_HIGH	Select constant high as source	0x25*
TMS	DSP_TMS_SRC_DIO_0	Select DIO[0] as source	0x0
	DSP_TMS_SRC_DIO_1	Select DIO[1] as source	0x1
	DSP_TMS_SRC_DIO_2	Select DIO[2] as source	0x2
	DSP_TMS_SRC_DIO_3	Select DIO[3] as source	0x3
	DSP_TMS_SRC_DIO_4	Select DIO[4] as source	0x4
	DSP_TMS_SRC_DIO_5	Select DIO[5] as source	0x5
	DSP_TMS_SRC_DIO_6	Select DIO[6] as source	0x6
	DSP_TMS_SRC_DIO_7	Select DIO[7] as source	0x7
	DSP_TMS_SRC_DIO_8	Select DIO[8] as source	0x8
	DSP_TMS_SRC_DIO_9	Select DIO[9] as source	0x9
	DSP_TMS_SRC_DIO_10	Select DIO[10] as source	0xA
	DSP_TMS_SRC_DIO_11	Select DIO[11] as source	0xB

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DSP_TMS_SRC_DIO_12	Select DIO[12] as source	0xC
	DSP_TMS_SRC_DIO_13	Select DIO[13] as source	0xD
	DSP_TMS_SRC_DIO_14	Select DIO[14] as source	0xE
	DSP_TMS_SRC_DIO_15	Select DIO[15] as source	0xF
	DSP_TMS_SRC_DIO_16	Select DIO[16] as source	0x10
	DSP_TMS_SRC_DIO_17	Select DIO[17] as source	0x11
	DSP_TMS_SRC_DIO_18	Select DIO[18] as source	0x12
	DSP_TMS_SRC_DIO_19	Select DIO[19] as source	0x13
	DSP_TMS_SRC_DIO_20	Select DIO[20] as source	0x14
	DSP_TMS_SRC_DIO_21	Select DIO[21] as source	0x15
	DSP_TMS_SRC_DIO_22	Select DIO[22] as source	0x16
	DSP_TMS_SRC_DIO_23	Select DIO[23] as source	0x17
	DSP_TMS_SRC_DIO_24	Select DIO[24] as source	0x18
	DSP_TMS_SRC_DIO_25	Select DIO[25] as source	0x19
	DSP_TMS_SRC_DIO_26	Select DIO[26] as source	0x1A
	DSP_TMS_SRC_DIO_27	Select DIO[27] as source	0x1B
	DSP_TMS_SRC_DIO_28	Select DIO[28] as source	0x1C
	DSP_TMS_SRC_DIO_29	Select DIO[29] as source	0x1D
	DSP_TMS_SRC_DIO_30	Select DIO[30] as source	0x1E
	DSP_TMS_SRC_DIO_31	Select DIO[31] as source	0x1F
	DSP_TMS_SRC_DIO_32	Select DIO[32] as source	0x20
	DSP_TMS_SRC_DIO_33	Select DIO[33] as source	0x21
	DSP_TMS_SRC_DIO_34	Select DIO[34] as source	0x22
	DSP_TMS_SRC_DIO_35	Select DIO[35] as source	0x23
	DSP_TMS_SRC_CONST_LOW	Select constant low as source	0x24
	DSP_TMS_SRC_CONST_HIGH	Select constant high as source	0x25*
TCK	DSP_TCK_SRC_DIO_0	Select DIO[0] as source	0x0
	DSP_TCK_SRC_DIO_1	Select DIO[1] as source	0x1
	DSP_TCK_SRC_DIO_2	Select DIO[2] as source	0x2
	DSP_TCK_SRC_DIO_3	Select DIO[3] as source	0x3
	DSP_TCK_SRC_DIO_4	Select DIO[4] as source	0x4

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DSP_TCK_SRC_DIO_5	Select DIO[5] as source	0x5
	DSP_TCK_SRC_DIO_6	Select DIO[6] as source	0x6
	DSP_TCK_SRC_DIO_7	Select DIO[7] as source	0x7
	DSP_TCK_SRC_DIO_8	Select DIO[8] as source	0x8
	DSP_TCK_SRC_DIO_9	Select DIO[9] as source	0x9
	DSP_TCK_SRC_DIO_10	Select DIO[10] as source	0xA
	DSP_TCK_SRC_DIO_11	Select DIO[11] as source	0xB
	DSP_TCK_SRC_DIO_12	Select DIO[12] as source	0xC
	DSP_TCK_SRC_DIO_13	Select DIO[13] as source	0xD
	DSP_TCK_SRC_DIO_14	Select DIO[14] as source	0xE
	DSP_TCK_SRC_DIO_15	Select DIO[15] as source	0xF
	DSP_TCK_SRC_DIO_16	Select DIO[16] as source	0x10
	DSP_TCK_SRC_DIO_17	Select DIO[17] as source	0x11
	DSP_TCK_SRC_DIO_18	Select DIO[18] as source	0x12
	DSP_TCK_SRC_DIO_19	Select DIO[19] as source	0x13
	DSP_TCK_SRC_DIO_20	Select DIO[20] as source	0x14
	DSP_TCK_SRC_DIO_21	Select DIO[21] as source	0x15
	DSP_TCK_SRC_DIO_22	Select DIO[22] as source	0x16
	DSP_TCK_SRC_DIO_23	Select DIO[23] as source	0x17
	DSP_TCK_SRC_DIO_24	Select DIO[24] as source	0x18
	DSP_TCK_SRC_DIO_25	Select DIO[25] as source	0x19
	DSP_TCK_SRC_DIO_26	Select DIO[26] as source	0x1A
	DSP_TCK_SRC_DIO_27	Select DIO[27] as source	0x1B
	DSP_TCK_SRC_DIO_28	Select DIO[28] as source	0x1C
	DSP_TCK_SRC_DIO_29	Select DIO[29] as source	0x1D
	DSP_TCK_SRC_DIO_30	Select DIO[30] as source	0x1E
	DSP_TCK_SRC_DIO_31	Select DIO[31] as source	0x1F
	DSP_TCK_SRC_DIO_32	Select DIO[32] as source	0x20
	DSP_TCK_SRC_DIO_33	Select DIO[33] as source	0x21
	DSP_TCK_SRC_DIO_34	Select DIO[34] as source	0x22

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DSP_TCK_SRC_DIO_35	Select DIO[35] as source	0x23
	DSP_TCK_SRC_CONST_LOW	Select constant low as source	0x24
	DSP_TCK_SRC_CONST_HIGH	Select constant high as source	0x25*

16.4.16 DIO_SRC_UART

Bit Field	Read/Write	Field Name	Description
5:0	RW	RX	UART_RX input selection

Field Name	Value Symbol	Value Description	Hex Value
RX	UART_RX_SRC_DIO_0	Select DIO[0] as source	0x0
	UART_RX_SRC_DIO_1	Select DIO[1] as source	0x1
	UART_RX_SRC_DIO_2	Select DIO[2] as source	0x2
	UART_RX_SRC_DIO_3	Select DIO[3] as source	0x3
	UART_RX_SRC_DIO_4	Select DIO[4] as source	0x4
	UART_RX_SRC_DIO_5	Select DIO[5] as source	0x5
	UART_RX_SRC_DIO_6	Select DIO[6] as source	0x6
	UART_RX_SRC_DIO_7	Select DIO[7] as source	0x7
	UART_RX_SRC_DIO_8	Select DIO[8] as source	0x8
	UART_RX_SRC_DIO_9	Select DIO[9] as source	0x9
	UART_RX_SRC_DIO_10	Select DIO[10] as source	0xA
	UART_RX_SRC_DIO_11	Select DIO[11] as source	0xB
	UART_RX_SRC_DIO_12	Select DIO[12] as source	0xC
	UART_RX_SRC_DIO_13	Select DIO[13] as source	0xD
	UART_RX_SRC_DIO_14	Select DIO[14] as source	0xE
	UART_RX_SRC_DIO_15	Select DIO[15] as source	0xF
	UART_RX_SRC_DIO_16	Select DIO[16] as source	0x10
	UART_RX_SRC_DIO_17	Select DIO[17] as source	0x11
	UART_RX_SRC_DIO_18	Select DIO[18] as source	0x12
	UART_RX_SRC_DIO_19	Select DIO[19] as source	0x13
	UART_RX_SRC_DIO_20	Select DIO[20] as source	0x14
	UART_RX_SRC_DIO_21	Select DIO[21] as source	0x15

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	UART_RX_SRC_DIO_22	Select DIO[22] as source	0x16
	UART_RX_SRC_DIO_23	Select DIO[23] as source	0x17
	UART_RX_SRC_DIO_24	Select DIO[24] as source	0x18
	UART_RX_SRC_DIO_25	Select DIO[25] as source	0x19
	UART_RX_SRC_DIO_26	Select DIO[26] as source	0x1A
	UART_RX_SRC_DIO_27	Select DIO[27] as source	0x1B
	UART_RX_SRC_DIO_28	Select DIO[28] as source	0x1C
	UART_RX_SRC_DIO_29	Select DIO[29] as source	0x1D
	UART_RX_SRC_DIO_30	Select DIO[30] as source	0x1E
	UART_RX_SRC_DIO_31	Select DIO[31] as source	0x1F
	UART_RX_SRC_DIO_32	Select DIO[32] as source	0x20
	UART_RX_SRC_DIO_33	Select DIO[33] as source	0x21
	UART_RX_SRC_DIO_34	Select DIO[34] as source	0x22
	UART_RX_SRC_DIO_35	Select DIO[35] as source	0x23
	UART_RX_SRC_CONST_LOW	Select constant low as source	0x24
	UART_RX_SRC_CONST_HIGH	Select constant high as source	0x25*

CHAPTER 17

Debug Ports and Security

The Ezairo 8300 system supports two primary (I²C and SWJ-DP) debug ports, which can both be used for debugging the CFX DSP and the Arm Cortex-M3 processor. A secondary JTAG debug port, that can be configured using one of the primary debug ports, provides debug support for the LPDSP32 DSP. The system also includes components that can be used to secure intellectual property stored on the chip and attached NVM storage device.

17.1 DEBUG

The HEAR Configurable Accelerator and Filter Engine are not supported by debug ports. Integration of HEAR microcode is intended to be debugged through the CFX DSP or Arm Cortex-M3 processor and the primary debug interfaces. Integration and debug testing of the Filter Engine application is supported by a pair of command and control registers that provide basic debug functionality when combined with memory accesses over one of the primary debug ports.

17.1.1 Primary Debug Ports for the CFX DSP and the Arm Cortex-M3 Processor

17.1.1.1 I²C Debug Ports

The I²C debug ports for the Ezairo 8300 system provide both the CFX DSP and Arm Cortex-M3 processor with a full debugging capacity. The protocols used by these shared I²C debug ports is described in the *Communication Protocols Manual for Ezairo 8300*.

These shared I²C debug ports use dedicated I/O pads that form an I²C bus, which is also shared with the D_I2C I²C interface instance. For more information about I²C buses, and the D_I2C interface, see [Section 18.4 “I2C Interfaces” on page 601](#). Since three interfaces share the same bus, addressing these interfaces becomes an important concern. Be careful not to address the CFX debug I²C interface, ARM Cortex-M3 core debug I²C interface, and/or the general-purpose D_I2C I²C interface instance at the same address because if they share an address, the order of precedence for handling data using the shared address is:

1. The CFX debug I²C interface
2. The ARM Cortex-M3 core debug I²C interface
3. The general-purpose D_I2C I²C interface instance

The addressing for these interfaces is defined in the [“I2C Bus Addressing for the Debug Ports and CFX General-Purpose I2C Interface” table \(Table 47\)](#).

Ezairo 8300 Hardware Reference

Table 47. I²C Bus Addressing for the Debug Ports and CFX General-Purpose I²C Interface

Interface	Response Location	Default Address Setting
CFX Debug Port	Internally-programmable address stored to the D_I2C_CFX_DEBUG_ADDR1 register	0x60
	Externally-programmable address stored to the D_I2C_CFX_DEBUG_ADDR2 register. This address can be configured using the Write Address and Acquire Address debug port commands (for more information, see the <i>Communication Protocols Manual for Ezairo 8300</i>).	0x60
ARM Cortex-M3 I ² C Debug Port	Externally-programmable address stored to the D_I2C_CM3_DEBUG_ADDR register	0x40
D_I2C I ² C Instance	I ² C general call address	0x00 (fixed)
	Programmable address	None

CAUTION: If you are using the onsemi Communication Accelerator Adapter (CAA) to communicate with the Ezairo 8300 chip over either I²C interface, do not use the address 0x51 for I²C communications because it conflicts with the non-volatile memory storage of the CAA.

The Ezairo 8300 system includes four configuration parameters for the I²C bus itself:

- The DIO_CFX_D_PAD_CFG_PULL bit field in the DIO_D_I2C_PAD_CFG register configures the use of an internal pull-up resistor used for the I²C interface lines.
- The DIO_D_I2C_PAD_CFG_LPF bit in the DIO_D_I2C_PAD_CFG register enables a pair of low-pass filters for the bus lines that can be used to filter out glitches of up to 50 ns on the I²C bus signals.
- The DIO_D_I2C_PAD_CFG_DRIVE bit in the DIO_D_I2C_PAD_CFG register allows you to select the drive strength for the I²C interface lines.

CAUTION: Using the DIO_D_I2C_PAD_CFG_PULL bit field from the DIO_D_I2C_PAD_CFG register to disable the bus pull-up resistors is not recommended unless external pull-up resistors are used. We recommend that you consult onsemi before using configurations that do not use pull-up resistors because this will likely result in unintended behavior.

As multiple interfaces can communicate on this bus with one or more external devices, the D_I2C_STATUS_LINE_FREE bit in the D_I2C_STATUS register provides information on current bus traffic. This bit is only set when the internal I²C bus state machine is currently idle, and both bus lines are high.

For this same reason, a watchdog timer is provided for the I²C bus lines. This timer watches the bus lines to ensure that the Ezairo 8300 system does not hold them low for an extended period of time; otherwise, if noise or another I²C driver causes an error on the bus, the chip's I²C debug ports would be locked out. To configure the timeout period, set the D_I2C_WATCHDOG_CFG_TIMEOUT bit field in the D_I2C_WATCHDOG_CFG register. The timer is

Ezairo 8300 Hardware Reference

cleared, preventing a timeout, when the interface is not holding the bus lines and no start condition is pending. If a timeout does occur, the I²C interface is reset, the D_I2C_STATUS_BUS_OR_WATCHDOG_ERROR bit is set in the D_I2C_STATUS register, and the count in the D_I2C_ERROR_CNT register is incremented unless its value is 255.

The interface can detect a bus error which allows the use of multi-master bus configurations, and prevents issues from arising when an I²C bus master reads from multiple slaves at the same address. A bus error is detected whenever the value written to the I²C data line (SDA) does not match the physical value of this line. When a bus error is detected, the D_I2C_STATUS_BUS_OR_WATCHDOG_ERROR and D_I2C_STATUS_BUS_ERROR bits are set in the D_I2C_STATUS register.

Writing a 1 to the D_I2C_CTRL_ERROR_CNT_CLEAR bit in the D_I2C_CTRL register resets the D_I2C_ERROR_CNT register to 0. The I²C interface error bits are cleared using the appropriate bits in the D_I2C_STATUS register, as described in [Section 18.4 “I2C Interfaces” on page 601](#).

NOTE: The SCL and SDA pads used for the I²C bus are disconnected by default from the I²C interface when SYSCLK is sourced from the standby clock because this could cause the Ezairo 8300 debug ports to respond to I²C traffic on the bus even when they are not being addressed. To enable these pads when clocking the system using the standby clock, set the D_I2C_CFG_CONNECT_IN_STANDBY bit from the D_I2C_CFG register.

17.1.1.1.1 I²C Debug Port Control and Configuration Registers

Register Name	Register Description	Address
D_DEBUG_CFG	Debug port mode	P:0x0835
DEBUG_CFG		0x400020D4
D_DEBUG_DATA	Debug port data register	P:0x0836
DEBUG_DATA		0x400020D8
D_I2C_CFX_DEBUG_ADDR1	First CFX Debug Slave ID	P:0x0A07
		0x4000281C
D_I2C_CFX_DEBUG_ADDR2	Second CFX Debug Slave ID	P:0x0A08
		0x40002820
D_I2C_CM3_DEBUG_ADDR	Arm Cortex-M3 Debug Slave ID	P:0x0A09
		0x40002824
D_PRAM_SECURITY_CFG	PRAM security configuration	P:0x0809
PRAM_SECURITY_CFG		0x40002024
SYSCTRL_DBG_LOCK	Debug Port Access Configuration	P:0x1000
		0x40000000

Ezairo 8300 Hardware Reference

17.1.1.1.1.1 D_DEBUG_CFG, DEBUG_CFG

Bit Field	Read/Write	Field Name	Description
3	R	DBG_REG_WRITTEN	Data register write status
2	R	DBG_REG_READ	Data register read status
1	R	AUDIO_MODE	
0	RW	ACCESS_MODE	

Field Name	Value Symbol	Value Description	Hex Value
DBG_REG_WRITTEN	DEBUG_PORT_NOT_WRITTEN		0x0*
	DEBUG_PORT_WRITTEN		0x1
DBG_REG_READ	DEBUG_PORT_NOT_READ		0x0*
	DEBUG_PORT_READ		0x1
AUDIO_MODE	AUDIO_STREAM_DISABLED		0x0*
	AUDIO_STREAM_ENABLED		0x1
ACCESS_MODE	ACCESS_UNRESTRICTED		0x0
	ACCESS_RESTRICTED		0x1*

17.1.1.1.1.2 D_DEBUG_DATA, DEBUG_DATA

Bit Field	Read/Write	Field Name	Description
31:0	RW	DEBUG_DATA	

17.1.1.1.1.3 D_I2C_CFX_DEBUG_ADDR1

Bit Field	Read/Write	Field Name	Description
6:0	RW	DEBUG_ADDR	Defines the first CFX processor debug I2C slave address

Field Name	Value Symbol	Value Description	Hex Value
DEBUG_ADDR	DEFAULT_D_DBG_I2C_SLAVE_ADDR	Default CFX I2C debug slave address	0x60*

17.1.1.1.1.4 D_I2C_CFX_DEBUG_ADDR2

Bit Field	Read/Write	Field Name	Description
6:0	RW	DEBUG_ADDR	Defines the second CFX processor debug I2C slave address

Ezairo 8300 Hardware Reference

17.1.1.1.1.5 D_I2C_CM3_DEBUG_ADDR

Bit Field	Read/Write	Field Name	Description
6:0	RW	DEBUG_ADDR	Defines the Arm Cortex-M3 processor debug I2C slave address

Field Name	Value Symbol	Value Description	Hex Value
DEBUG_ADDR	DEFAULT_CM3_DBG_I2C_SLAVE_ADDR	Default Arm Cortex-M3 I2C debug slave address	0x40*

17.1.1.1.1.6 D_PRAM_SECURITY_CFG, PRAM_SECURITY_CFG

Bit Field	Read/Write	Field Name	Description
3	W	CM3_PRAM1_LOCK	Lock Arm Cortex-M3 PRAM1 access except for PMEM bus
3	R	CM3_PRAM1_LOCK_STATUS	Arm Cortex-M3 PRAM1 access lock status
2	W	CM3_PRAM0_LOCK	Lock Arm Cortex-M3 PRAM0 access except for PMEM bus
2	R	CM3_PRAM0_LOCK_STATUS	Arm Cortex-M3 PRAM0 access lock status
1	W	CFX_PRAM1_LOCK	Lock CFX PRAM1 access except for PMEM bus
1	R	CFX_PRAM1_LOCK_STATUS	CFX PRAM1 access lock status
0	W	CFX_PRAM0_LOCK	Lock CFX PRAM0 access except for PMEM bus
0	R	CFX_PRAM0_LOCK_STATUS	CFX PRAM0 access lock status

Field Name	Value Symbol	Value Description	Hex Value
CM3_PRAM1_LOCK	CM3_PRAM1_LOCK	Lock Arm Cortex-M3 PRAM1 access except for PMEM bus	0x1
CM3_PRAM1_LOCK_STATUS	CM3_PRAM1_NOT_LOCKED	Arm Cortex-M3 PRAM1 access is not locked	0x0*
	CM3_PRAM1_LOCKED	Arm Cortex-M3 PRAM1 access is locked except for PMEM bus	0x1
CM3_PRAM0_LOCK	CM3_PRAM0_LOCK	Lock Arm Cortex-M3 PRAM0 access except for PMEM bus	0x1
CM3_PRAM0_LOCK_STATUS	CM3_PRAM0_NOT_LOCKED	Arm Cortex-M3 PRAM0 access is not locked	0x0*
	CM3_PRAM0_LOCKED	Arm Cortex-M3 PRAM0 access is locked except for PMEM bus	0x1
CFX_PRAM1_LOCK	CFX_PRAM1_LOCK	Lock CFX PRAM1 access except for PMEM bus	0x1
CFX_PRAM1_LOCK_STATUS	CFX_PRAM1_NOT_LOCKED	CFX PRAM1 access is not locked	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	CFX_PRAM1_LOCKED	CFX PRAM1 access is locked except for PMEM bus	0x1
CFX_PRAM0_LOCK	CFX_PRAM0_LOCK	Lock CFX PRAM0 access except for PMEM bus	0x1
CFX_PRAM0_LOCK_STATUS	CFX_PRAM0_NOT_LOCKED	CFX PRAM0 access is not locked	0x0*
	CFX_PRAM0_LOCKED	CFX PRAM0 access is locked except for PMEM bus	0x1

17.1.1.1.1.7 SYSCTRL_DBG_LOCK

Bit Field	Read/Write	Field Name	Description
0	RW	DBG_LOCK	Debug port access lock

Field Name	Value Symbol	Value Description	Hex Value
DBG_LOCK	DBG_ACCESS_UNLOCK	Debug port access is unlocked	0x0
	DBG_ACCESS_LOCK	Debug port access is locked	0x1*

17.1.1.2 SWJ-DP Debug Port

The Ezairo 8300 system contains a standard Core Sight SWJ-DP debug port for the Arm Cortex-M3 processor. The SWJ-DP provides access to the standard Arm Cortex-M3 core debug controller that is on the private peripheral bus. This debug controller is used to provide access to all of the Arm Cortex-M3 processor registers, and to all of the Ezairo 8300 memories through the memory buses attached to the Arm Cortex-M3 processor. For more information about the Arm Cortex-M3 processor's debug controller, see [Section 4.6.3 “Debug Controller” on page 199](#).

By default, the SWJ-DP is accessed using the JTAG connection that uses DIOs 30 to 33, to form a 4-wire JTAG interface. This debug port can be reconfigured for serial-wire mode using only DIOs 30 and 31, or disabled entirely to allow these pads to be re-used for other use cases. These DIOs are reconfigured using the `DIO_CM3_JTAG_PAD_CFG` register, as described in [Section 16.2.1 “Special Functional Configurations” on page 468](#).

When using the SWJ-DP in serial wire mode, data is transferred using a single clock and single data signal. This interface mode is useful when the number of debug port connections needs to be limited, but it is only half-duplex and has a relatively high overhead to transfer data. When using the SWJ-DP in JTAG mode, data is transferred using a data control line, a pair of data lines, a clock, and an optional reset signal. This interface mode is useful when the number of debug port connections is not limited, and a higher throughput is required, as JTAG mode uses less data overhead than SW mode, full-duplex transfers.

To switch the SWJ-DP into JTAG mode from serial wire mode, complete the following initialization sequence:

1. Use SWCLK to send 0xE73C (transmitted LSB first) using SWDIO. This initialization pattern switches the debug port to JTAG mode.
2. Send at least 5 SWCLK cycles with SWDIO HIGH. This ensures that the JTAG interface enters its test-logic reset state.

To switch the SWJ-DP into serial wire mode from JTAG mode, complete the following initialization sequence:

Ezairo 8300 Hardware Reference

1. Send at least 50 clock cycles on SWCLK with the SWDIO pad held high. This ensures that the current interface is in its reset state.
2. Use SWCLK to send 0xE79E (transmitted LSB first) using SWDIO. This initialization pattern switches the debug port to serial wire mode.
3. Send at least 50 SWCLK cycles with SWDIO HIGH. This ensures that the serial wire interface enters its line reset state.

17.1.1.2.1 SWJ-DP as CFX Debug Port Bridge

The Arm Cortex-M3 SWJ-DP debug port can be used as a bridge to the CFX I²C debug port through the `SYSTRM_CFX_DEBUG` register. Through this register, the system can access all of the I²C protocols used by the CFX I²C debug port, as described in the *Communication Protocols Manual for Ezairo 8300*.

NOTE: Since the `SYSTRM_CFX_DEBUG` register simply accesses the CFX I²C debug port, all restrictions on commands (including restricted and unrestricted access modes and run state) apply to debug port accesses that use the `SYSTRM_CFX_DEBUG` register.

The CFX debug port bridge is enabled by setting the `SYSTRM_CFX_DEBUG_ENABLE` bit in the `SYSTRM_CFX_DEBUG` register. Once enabled, the bridge can emulate an I²C transfer using this register and the components described in [Table 48](#).

Table 48. CFX I²C Debug Port Bridge Register Bit Fields

Bit(s)	Bit Field	Notes
1	START	Issue a start condition on the CFX I ² C debug port
2	STOP	Issue a stop condition on the CFX I ² C debug port
3	DATA_READY	Indicate that the next byte is ready to be transferred to/from the CFX I ² C debug port. In read mode (see RNW below), this bit indicates that the <code>TX_DATA</code> field has been read and can now be reloaded by the CFX debug controller. In write mode, this bit indicates that the <code>RX_DATA</code> field has been written.
4	RNW	Read/Not write bit; set to 0 when writing to the debug port, set to 1 when reading from the debug port.
15:8	RX_DATA	Protocol command data written to the CFX DSP's I ² C debug port
23:16	TX_DATA	Protocol transmit data read from the CFX DSP's I ² C debug port

17.1.2 CFX Debug Port Controller

Access to the CFX debug port controller is locked by default, and can be locked or unlocked by a user application through the `D_DEBUG_CFG` register. If the `D_DEBUG_CFG_ACCESS_MODE` bit in the `D_DEBUG_CFG` register is set, the debug controller is in a locked state with access blocked to any instructions that require access to memory or registers. This bit can be cleared either by a user application, or through the use of the `JumpROM` commands as described in the *Communication Protocols Manual for Ezairo 8300*.

For more information about security modes and their use in the Ezairo 8300 system, see [Section 17.2 “System Security”](#) on page 537.

Ezairo 8300 Hardware Reference

The debug port uses the `D_DEBUG_DATA` register as a memory-mapped register for direct communication with the CFX DSP over the debug port while the system is running. When the debug port has written to this register, the `D_DEBUG_CFG_DBG_REG_WRITTEN` bit in the `D_DEBUG_CFG` register remains set until the register is read. Similarly, when the debug port has read this register, the `D_DEBUG_CFG_DBG_REG_READ` bit in the `D_DEBUG_CFG` register remains set until the register is written.

The debug port interface can be used with a PCM interface to provide digital audio streaming functionality. When operating in this mode, the I²C interface is connected to a PCM interface and the debug port is flipped into audio streaming mode. The debug port indicates this mode to the system by setting the `D_DEBUG_CFG_AUDIO_MODE` in the `D_DEBUG_CFG` register. More information about digital audio streaming is in [Section 17.1.2.3 “NOAHlink Digital Audio Streaming Configuration and Usage” on page 531](#), and in the *Communication Protocols Manual for Ezairo 8300*.

NOTE: For additional support to configure the debug port for use with audio streaming, contact onsemi Customer Support.

17.1.2.1 Breakpoints

The primary troubleshooting functionality of the debug port is provided by the different types of breakpoints that can be configured through the debug port protocol. These breakpoints include the following:

Software breakpoints

Included in code for the CFX processor as `breakpoint` instructions. See the *CFX DSP Architecture Manual for Ezairo 8300* for details about the `breakpoint` instruction, and the *Communication Protocols Manual for Ezairo 8300* for information about configuring the debug port to stop at software breakpoints.

Hardware breakpoints

Configured as pointers to P Memory that are triggered when the CFX DSP issues the instruction that is pointed to by the hardware breakpoint for execution. See the *Communication Protocols Manual for Ezairo 8300* for details about configuring hardware breakpoints, and configuring the debug port to stop at hardware breakpoints.

External breakpoints

Configured to trigger a breakpoint event based on accessed data at addresses corresponding to specified conditions and/or the value of the written data. See [Section 17.1.2.2 “External Breakpoint Configuration”](#) for details about configuring external breakpoints.

17.1.2.2 External Breakpoint Configuration

External breakpoints can be used to detect a variety of different memory access conditions. The different types of memory access conditions that can be tested for are grouped into comparison modes as follows:

- A double-address mode that monitors for data accesses relative to a pair of addresses; this mode is described in [Section 17.1.2.2.1 “Double-Address Mode” on page 531](#)
- Three memory conflict modes that detects if a memory conflict has occurred; these modes are described in [Section 17.1.2.2.2 “Memory Conflict Modes” on page 531](#)

The external breakpoint modes are enabled using the `D_EXT_BREAK_CFG` register.

IMPORTANT: When an external breakpoint condition is observed, the instructions currently loaded to the CFX DSP instruction pipeline complete before the core is stopped. This typically results in a perceived delay of 3 cycles between an external breakpoint event and the CFX DSP stopping.

17.1.2.2.1 Double-Address Mode

In external breakpoint double-address mode, an external breakpoint is triggered when a data access occurs and all of the following conditions are met:

- Access is on the specified memory bus and of the specified type
- The location where the data was accessed is at an address that meets the address conditions

The conditions are specified as follows:

- The bus of interest for the external breakpoint is selected by setting the `D_EXT_BREAK_CFG_BUS_SELECT` bit field from the `D_EXT_BREAK_CFG` register. If no bus is selected, the external breakpoint is disabled.
- Access conditions are used to restrict external breakpoint events based on the type of data access that occurred. A description of access conditions that can be used to trigger an external breakpoint are listed in [Table 3](#). Access conditions are selected using the `D_EXT_BREAK_CFG_ACCESS_COND` bit field from the `D_EXT_BREAK_CFG` register.

Table 49. Access Conditions for Double-Address Mode

Condition	Description
Disabled	External breakpoints are not generated
Read or Write Access	Address is considered when data is either read or written
Read Access Only	Address is only considered when data is read
Write Access Only	Address is only considered when data is written

17.1.2.2.2 Memory Conflict Modes

In external breakpoint memory conflict modes, an external breakpoint is triggered when an enabled memory conflict occurs. The `D_EXT_BREAK_CFG` register is used to enable the monitored memory conflicts detected in this mode. Possible conflicts that can be enabled for detection are:

- The CFX DSP core and HEAR Accelerator core
- The CFX DSP core and the Arm Cortex-M3 processor, DMA, or LPDSP32
- The Arm Cortex-M3 processor or DMA and any accesses to the FIFO-controlled portions of shared memory (excluding conflicts with the IOC)

17.1.2.3 NOAHlink Digital Audio Streaming Configuration and Usage

A PCM interface can be configured for compatibility using the digital audio streaming feature of a NOAHlink device to input data. This configuration, however, requires a number of signal configuration settings as shown in [Table 50](#). This configuration also requires several modifications to the hardware connections as described in [Table 51](#).

Ezairo 8300 Hardware Reference

Table 50. Required Configuration Settings for NOAHlink Digital Audio Streaming Configuration

Configuration Bit or Bit Field	Setting
PCM_CFG_WORD_SIZE	PCM_WORD_SIZE_16
PCM_CFG_FRAME_LENGTH	PCM_MULTIWORD_2
PCM_CFG_FRAME_WIDTH	PCM_FRAME_WIDTH_LONG
PCM_CFG_FRAME_ALIGN	PCM_FRAME_ALIGN_FIRST
PCM_CFG_BIT_ORDER	PCM_BIT_ORDER_LSB_FIRST
PCM_CFG_SUBFRAME	PCM_SUBFRAME_DISABLE

Table 51. Hardware Connection Configuration for NOAHlink Digital Audio Streaming

PCM Signal	Connected Signal	Connection	Comment
PCM_CLK	UCLK	Both signals routed to a shared DIO pad	Signal output or input at PCM clock is ignored
PCM_FRAME	D_I2C I ² C SDA	External; PCM_FRAME configured as an output	Disconnect the pull-ups on the I ² C SDA line to limit glitches in the connection to a NOAHlink audio streaming device
PCM_SERO	-	Unconnected	NOAHlink digital audio streaming is input only
PCM_SERI	D_I2C I ² C SCL	External; PCM_SERI configured as an input	Disconnect the pull-ups on I ² C SCL line to limit glitches in the connection to a NOAHlink device

To enable audio streaming when connected to a NOAHlink device, configure the debug port to operate in audio streaming mode.

NOTE: Support for configuring the debug port for use with audio streaming can be obtained by contacting onsemi Customer Support.

When in audio streaming mode with the required PCM interface configuration, the device streams audio data from a NOAHlink device. After configuring the PCM interface in this way and enabling audio streaming, the resulting signals will match the timing diagram shown in the "Signal Timing Diagram: NOAHlink Digital Audio Streaming PCM Configuration" figure (Figure 31).

IMPORTANT: If you configured the PCM interface for NOAHlink digital audio streaming by connecting the relevant PCM and I²C pads, analog audio streaming using the NOAHlink device and the I²C interface is not possible.

Ezairo 8300 Hardware Reference

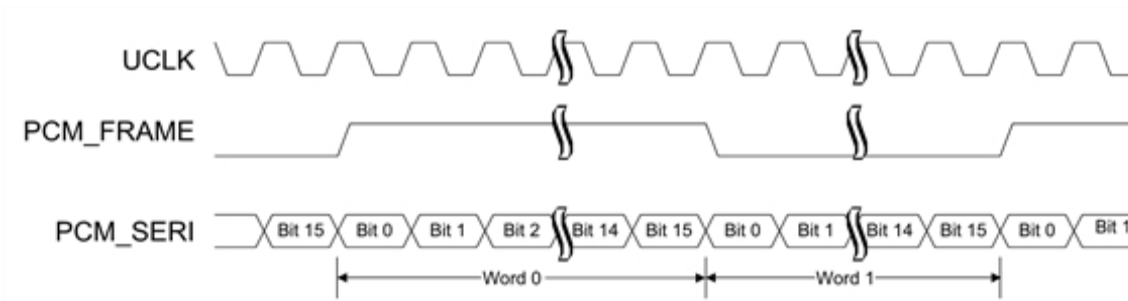


Figure 31. Signal Timing Diagram: NOAHlink Digital Audio Streaming PCM Configuration

17.1.2.4 CFX Debug Port Specific Control and Configuration Registers

Register Name	Register Description	Address
D_EXT_BREAK_CFG	External break point configuration	P:0x0810
EXT_BREAK_CFG		0x40002040
D_EXT_BREAK_ADDR_LOW	External break point low address	P:0x0811
EXT_BREAK_ADDR_LOW		0x40002044
D_EXT_BREAK_ADDR_HIGH	External break point high address	P:0x0812
EXT_BREAK_ADDR_HIGH		0x40002048

For control and configuration registers that apply to the D_I2C interfaces for the CFX and the Arm Cortex-M3 processor, as well as the D_I2C I²C interface instance, see [Section 17.1.1.1.1 “I2C Debug Port Control and Configuration Registers”](#) on page 525.

17.1.2.4.1 D_EXT_BREAK_CFG, EXT_BREAK_CFG

Bit Field	Read/Write	Field Name	Description
6	RW	CM3_FIFO_CONFLICT	Configured detection of FIFO memory conflicts, between the Arm Cortex-M3 core or DMA, and the HEAR or CFX
5	RW	CFX_CM3_CONFLICT	Configured detection of memory conflicts, between the CFX and the Arm Cortex-M3 core, LPDSP32 or DMA
4	RW	CFX_HEAR_CONFLICT	Configured detection of memory conflicts, between the CFX and HEAR
3:2	RW	ACCESS_COND	Select the access condition
1:0	RW	BUS_SELECT	Select the data bus to monitor for external breakpoints

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
CM3_FIFO_CONFLICT	EXTBREAK_CM3_FIFO_CONFLICT_DISABLE	Do not trigger on memory conflicts between the Arm Cortex-M3/DMA and HEAR/CFX when accessing FIFOs	0x0*
	EXTBREAK_CM3_FIFO_CONFLICT_ENABLE	Trigger on conflicts between the Arm Cortex-M3/DMA and HEAR/CFX when accessing FIFOs	0x1
CFX_CM3_CONFLICT	EXTBREAK_CFX_CM3_CONFLICT_DISABLE	Do not trigger on conflicts between the CFX and Arm Cortex-M3/LPDSP32/DMA	0x0*
	EXTBREAK_CFX_CM3_CONFLICT_ENABLE	Trigger on conflicts between the CFX and Arm Cortex-M3/LPDSP32/DMA	0x1
CFX_HEAR_CONFLICT	EXTBREAK_CFX_HEAR_CONFLICT_DISABLE	Do not trigger on conflicts between the CFX and HEAR	0x0*
	EXTBREAK_CFX_HEAR_CONFLICT_ENABLE	Trigger on conflicts between the CFX and HEAR	0x1
ACCESS_COND	EXTBREAK_ACCESS_FALSE	Double address mode: False (disabled)	0x0*
	EXTBREAK_ACCESS_RW	Double address mode: Read or write access	0x1
	EXTBREAK_ACCESS_READ	Double address mode: Read access	0x2
	EXTBREAK_ACCESS_WRITE	Double address mode: Write access	0x3
BUS_SELECT	EXTBREAK_BUS_DISABLED	No bus (disabled)	0x0*
	EXTBREAK_BUS_XMEM	XMEM	0x1
	EXTBREAK_BUS_YMEM	YMEM	0x2
	EXTBREAK_BUS_PMEM	PMEM	0x3

17.1.2.4.2 D_EXT_BREAK_ADDR_LOW, EXT_BREAK_ADDR_LOW

Bit Field	Read/Write	Field Name	Description
18:0	RW	EXT_BREAK_ADDR_LOW	External break low address

17.1.2.4.3 D_EXT_BREAK_ADDR_HIGH, EXT_BREAK_ADDR_HIGH

Bit Field	Read/Write	Field Name	Description
18:0	RW	EXT_BREAK_ADDR_HIGH	External break high address

Ezairo 8300 Hardware Reference

17.1.3 Arm Cortex-M3 Debug

Debugging the Arm Cortex-M3 processor in the Ezairo 8300 system using either of the primary debug ports accesses the standard Arm Cortex-M3 core debug controller on the Arm Cortex-M3 processor's private peripheral bus. This debug controller is used to provide access to all of the Arm Cortex-M3 processor registers, and to all of the Ezairo 8300 memories through the memory buses attached to the Arm Cortex-M3 processor. For more information about the Arm Cortex-M3 processor's debug controller, see [Section 4.6.3 “Debug Controller” on page 199](#).

Access to the debug controller is locked by default, and can be locked or unlocked by a user application through the `SYSCTRL_DBG_LOCK` register. If the `SYSCTRL_DBG_LOCK_DBG_LOCK` bit is set, the debug controller is in a locked state with access blocked to the debug access port bus bridge. In this state, only debug port commands that do not access memory (or memory-mapped elements) are functional.

NOTE: Access to Arm Cortex-M3 registers uses memory-mapped registers on the private-peripheral bus, and therefore are not accessible in locked mode.

For more information about security modes and their use in the Ezairo 8300 system, see [Section 17.2 “System Security” on page 537](#).

17.1.4 LPDSP32 DSP Debug Port

LPDSP32 is supported by a 4-wire JTAG debug port that is mapped onto the DIOs. This debug port is designed to work with the standard ASIP programmer debug tools provided for the LPDSP32 DSP. For information on assigning DIOs to be used as the LPDSP32 debug port, refer to [Chapter 16.1 “Overview” on page 463](#). For proper operation of this debug port, all four signals must be mapped to DIOs.

To enable the LPDSP32 debug port, set the `SYSCTRL_DSP_DEBUG_CFG_DSP_DEBUG_ENABLE` bit from the `SYSCTRL_DSP_DEBUG_CFG` register. The LPDSP32 must be enabled as well, using the `DSP_RESUME` bit in the `SYSCTRL_DSP_CTRL` register. Before enabling the LPDSP32, a program needs to be downloaded into the LPDSP32 program memory to ensure that the LPDSP32 is in a known state.

When the LPDSP32 is in power down state after executing a `POWERDOWN` instruction, it normally exits that state only when there is an unmasked interrupt pending. This is also the case in debug mode; however, when the `DSP_EXIT_POWERDOWN_WHEN_HALTED` bit is enabled, then the LPDSP32 also exits the power down mode when it is in the halted debug mode and the step or run debug command is executed.

IMPORTANT: To minimize power consumption, a production application should disable the LPDSP32's debug port.

17.1.5 Control and Configuration Registers

Register Name	Register Description	Address
<code>SYSCTRL_DSP_CTRL</code>	DSP Control	P:0x1001 0x40000004
<code>SYSCTRL_DSP_DEBUG_CFG</code>	DSP Debug Port Configuration	P:0x1002 0x40000008

Ezairo 8300 Hardware Reference

17.1.5.1 SYSCTRL_DSP_DEBUG_CFG

Bit Field	Read/Write	Field Name	Description
1	RW	DSP_EXIT_POWERDOWN_WHEN_HALTED	DSP exit powerdown mode configuration when halted
0	RW	DSP_DEBUG_ENABLE	DSP debug port enable

Field Name	Value Symbol	Value Description	Hex Value
DSP_EXIT_POWERDOWN_WHEN_HALTED	DSP_EXIT_POWERDOWN_WHEN_HALTED_DISABLED	DSP exit powerdown when halted disabled	0x0*
	DSP_EXIT_POWERDOWN_WHEN_HALTED_ENABLED	DSP exit powerdown when halted enabled	0x1
DSP_DEBUG_ENABLE	DSP_DEBUG_DISABLED	DSP debug port disabled	0x0*
	DSP_DEBUG_ENABLED	DSP debug port enabled	0x1

17.1.6 Filter Engine Debug Support Registers

The Filter Engine does not have a formal debug interface, but does provide control functionality through the `D_FENG_CMDS` and `D_FENG_CTRL` registers, which can be used to debug programs written for this core using the CFX DSP or Arm Cortex-M3 processor's debug ports. The debug support commands that can be issued using these registers are summarized in [Table 52](#).

Table 52. Filter Engine Debug Support

Debug Functionality	Bit	Notes
Pause	<code>D_FENG_CTRL_PAUSE</code> from <code>D_FENG_CTRL</code>	Pauses execution of the Filter Engine, prevents the sample ready signal from triggering execution.
Resume (Clean) ¹	<code>D_FENG_CTRL_RESUME_CLEAN</code> from <code>D_FENG_CTRL</code>	Clear all pending sample ready signals and the multi-rate counters, then enable the Filter Engine to execute on the next configured sample ready trigger.
Single Step	<code>D_FENG_CTRL_SINGLE_STEP</code> from <code>D_FENG_CTRL</code>	Execute one instruction on the Filter Engine
Manual Execution	<code>D_FENG_CMDS_EXEC</code> from <code>D_FENG_CMDS</code>	Manually trigger execution of the base processing and one phase of the current Filter Engine program NOTE: Requires the sample ready input to be disabled, by setting the <code>D_FENG_CFG_SRI_SEL</code> bit field of the <code>D_FENG_CFG</code> register to <code>FENG_SRI_DISABLE</code> .

1. The Filter Engine can also be started using the `SYNC_START` bit from the `D_FENG_CTRL` register; however, this configuration is designed for synchronization of input samples rather than for use in debugging the Filter Engine program, and therefore, is not described here.

1

Ezairo 8300 Hardware Reference

For more information about Filter Engine control support which can be used for debugging, see [Section 6.4.1 “Filter Engine Control” on page 219](#).

17.2 SYSTEM SECURITY

By default, the Ezairo 8300 system boots in a protective state that restricts the functionality of the debug ports. A description of the restrictions placed on debug port commands when restricted access is enabled can be found in the debug port reference materials provided by the *Communication Protocols Manual for Communication Protocols Manual for Ezairo 8300*. [Section 17.1.2 “CFX Debug Port Controller” on page 529](#) shows how to change the security mode of the I²C debug ports, while unrestricting the SWJ-DP port is explained in [Section 17.1.3 “Arm Cortex-M3 Debug” on page 535](#).

A complete explanation of the security model for protecting IP assets stored in an external NVM device and in the system RAM is provided in the security model section (Program ROM chapter) of the *Ezairo 8300 Firmware Reference*. The information provided here is an abbreviated explanation of the security model.

The security model is mostly implemented in the Ezairo 8300 ROM. It uses a two-step encryption process to guarantee that no unencrypted traffic goes through any external storage device, such as an SPI or eMMC flash device or an SPI EEPROM device, so as to prevent eavesdropping. A boot key stored in the ROM area is issued by onsemi to each customer, and the Program ROM can use this key to decrypt the bootloader application in the NVM image, along with a user-defined key stored in the User Key Area in the NVM device. The user-defined key can be used by the bootloader to decrypt user applications or other protected IP assets in the NVM image.

Once a protected IP has been decrypted to the system memory, it is protected by two measures:

1. A RAM wipe performed by the ROM during boot and before unlocking the device through the Jump ROM unlock command
2. Restricting the debug ports so that they cannot be used to access registers or memory, nor to access devices connected to device interfaces that might contain protected IP assets.

Complete information on key security components is provided in the appropriate sections of other manuals in the Ezairo 8300 documentation set:

- The RAM sections that are cleared by the RAM wipe are shown in the RAM clearing section (Program ROM chapter) of the *Ezairo 8300 Firmware Reference*.
- The non-volatile device file system security section and application encrypted sections section from the *Ezairo 8300 Firmware Reference* describe the NVM file system components.
- Program ROM support for AES and the decryption of encrypted file system components is documented in the file system decryption support section of the *Ezairo 8300 Firmware Reference*.
- The bootloader operation section of the *Ezairo 8300 Firmware Reference* describes the support in the bootloader for decryption, and the boot errors table lists the `BOOTSTRAP_RET_DECRYPT_KEYROM_ERROR` error code that the bootloader lists in case of a decryption error.
- The Program ROM function tables section in the *Ezairo 8300 Firmware Reference* provides links to the ROM-based implementations of the AES decryption functions.
- The protected resources section in the Algorithm Protection chapter of the *Communication Protocols Manual for Ezairo 8300* outlines how security is provided from the perspective of the debugger protocol.
- The Non-Volatile Memory Manager (NVMM) tool section in the Integrated Development Environment User's Guide for Ezairo 8300 describes the support for creating encrypted (or partially encrypted) non-volatile memory images.

CHAPTER 18

Interfaces

The majority of the interfaces in the Ezairo 8300 system are assigned to 36 data input/output (DIO) pads that are configured through the DIO block. Each of these pads can be configured as the source for a number of interfaces or as the outputs for a number of interface signals, system clocks and other system signals. The Ezairo 8300 system also contains a number of debug interfaces to assist with debugging applications running on the system's programmable cores.

18.1 GENERAL-PURPOSE I/O (GPIO) INTERFACE

Ezairo 8300 can configure any, or all, of the 36 DIO pads as software-controlled general-purpose DIO (GPIO) pads. The function of these GPIO pads is defined by the user application, which can use them for any general-purpose input or output. Each GPIO pad can be configured to generate interrupts to the CFX DSP and/or Arm Cortex-M3 processor.

NOTE: Since the total number of pads exceeds the 32 bits available in a physical addressable register, the bits are split across two registers. DIO[23:0] are mapped to the `DIO_INPUT_DATA_0`, `DIO_GPIO_OUTPUT_DATA_0`, `DIO_DIR_0`, `DIO_GPIO_DIR_0`, and `DIO_MODE_0` registers. DIO[35:24] are mapped to the `DIO_INPUT_DATA_1`, `DIO_GPIO_OUTPUT_DATA_1`, `DIO_DIR_1`, `DIO_GPIO_DIR_1`, and `DIO_MODE_1` registers.

All DIO pins can be configured in GPIO mode by setting the `IO_MODE` field in the `DIO_CFG` register to `DIO_MODE_GPIO_IN` or `DIO_MODE_GPIO_OUT`, which allows the application to configure a pad as GPIO and set the direction in one register assignment. The `DIO_MODE_*` registers indicate which DIO pads have been configured for GPIO functionality; bits in these registers are set (for example, `DIO_IS_GPIO`) if they are configured as GPIOs, and cleared otherwise (for example, `DIO_IS_NOT_GPIO`). The `DIO_DIR_*` registers can be used to read and set the input/output direction for any pads that are defined as GPIOs, changing the `IO_MODE` fields of the `DIO_CFG_*` registers. To set the direction as input, write `DIO*_INPUT` to the bit locations corresponding to the DIOs that need to be configured as inputs. To set the direction as output, write `DIO*_OUTPUT` to the bit locations corresponding to the DIOs that need to be configured as outputs. By default, the direction is set to input. Applications can use `DIO_MODE_*` registers to mask corresponding `DIO_INPUT_DATA_0` and `DIO_INPUT_DATA_1` read values.

The values observed at the digital input pads can be read from the `DIO_INPUT_DATA_*` registers. These values are read as 0 for all pads that are disabled, or that are configured as LSAD inputs (see [Section 18.5.1 “LSAD Input Configuration” on page 630](#)), because digital input is not enabled in this mode. For all other modes, the physical value of the pad is directly measured.

The output value for the digital output pads that are configured as GPIO outputs can be configured using three register sets:

- The two registers `DIO_GPIO_OUTPUT_DATA_0` and `DIO_GPIO_OUTPUT_DATA_1` are used to explicitly set the output values for all GPIO outputs.
- `DIO_GPIO_OUTPUT_DATA_*_SET` is used to atomically set GPIO outputs for either `DIO_GPIO_OUTPUT_DATA_0` or `DIO_GPIO_OUTPUT_DATA_1` to high (1).
- `DIO_GPIO_OUTPUT_DATA_*_CLR` is used to atomically clear GPIO outputs for either `DIO_GPIO_OUTPUT_DATA_0` or `DIO_GPIO_OUTPUT_DATA_1` to low (0).

NOTE: DIO[33:30] are mapped as Arm Cortex-M3 processor JTAG pads on power up. To re-use these pads as GPIO, the application writes to `DIO_CM3_JTAG_PAD_CFG` (see [Section 16.2.1 “Special Functional Configurations”](#) on page 468). When in JTAG mode, these pads are reported as outputs.

18.1.1 GPIO Interrupts

The GPIO interface provides a set of eight configurable interrupts which, when enabled, signal the occurrence of an event or a condition on a specified GPIO pad. See [Section 4.6.1 “Nested Vectored Interrupt Controller \(NVIC\)”](#) on page 135 for information regarding interrupt configuration and handling interrupts for the Arm Cortex-M3 processor. See [Section 3.2 “Interrupt Controller”](#) on page 61 for information regarding interrupt configuration and handling for the CFX DSP.

Each of the GPIO interrupts supports triggering an interrupt from the following input sources:

- Any of the 36 DIOs
- The CFX and Arm Cortex-M3 I2C debug port pads (SCL, SDA from the D_I2C interface)
- Either of the two DIO interrupt clock dividers (as described below)

Two clock dividers are provided, which allow an application to decrease the rate at which GPIO interrupts occur. The input source for each of these clock dividers is configured using the `DIO_SRC_CLKDIV_CLKDIV*` bit-fields from the `DIO_SRC_CLKDIV` register. Each clock divisor is configured using the `DIO_CLKDIV_CFG` register:

- To enable or disable the GPIO interrupt clock divisor, use the `DIO_CLKDIV_CFG_DIO_CLKDIV*_ENABLE` bit.
- To select between triggering on rising or falling edges of the input, use the `DIO_CLKDIV_CFG_DIO_CLKDIV*_EDGE` bit.
- To configure the prescale factor provided by the clock divisor, use the `DIO_CLKDIV_CFG_DIO_CLKDIV*_PRESACLE` bit-field.
- To reset the internal counters for the clock divisor, set the `DIO_CLKDIV_CFG_DIO_CLKDIV*_RESET` bit.

The GPIO interrupts also support triggering on one of five possible GPIO events. The source and event trigger for each interrupt can be configured using one of the `DIO_INT_CFG_*` registers. A list of the possible triggering events (including a description of each event) is listed in the ["GPIO Interrupt Events" table \(Table 53\)](#).

Table 53. GPIO Interrupt Events

Event	Event Description
Disabled	An interrupt is never generated.
High Level	An interrupt is generated when a logical 1 is detected.
Low Level	An interrupt is generated when a logical 0 is detected.
Rising Edge	An interrupt is generated when a transition from a logical 0 to a logical 1 is detected.
Falling Edge	An interrupt is generated when a transition from a logical 1 to a logical 0 is detected.
Transition	An interrupt is generated when any transition between a logical 0 and a logical 1 is detected.

Each GPIO interrupt can be configured to use a button debounce filter. When the debounce filter is used, the output of the filter functions as the source for the purpose of detecting GPIO interrupt events.

Ezairo 8300 Hardware Reference

NOTE: The debounce filter state for a GPIO interrupt is reset whenever the GPIO interrupt is disabled or the selected source for the interrupt changes.

When a transition is detected on the selected source for the GPIO interrupt, the debounce filter output immediately changes to the new value and an internal counter is loaded with the value from the `DIO_INT_DEBOUNCE_DEBOUNCE_COUNT` bit-field from the `DIO_INT_DEBOUNCE_DEBOUNCE` register. This internal counter is decremented using a division of `SLOWCLK` by either 32 or 1024, as configured using the `DIO_INT_DEBOUNCE_DEBOUNCE_CLK` bit from the `DIO_INT_DEBOUNCE_DEBOUNCE` register. The debounce filter output is held at the new value until the internal counter reaches zero, at which point the filter output once again matches the input to the filter.

Each of the eight GPIO interrupts is configured by a `DIO_INT_CFG` register:

- To select the DIO pad or other input source to use as a trigger for the interrupt, set the `DIO_INT_CFG_SRC` bit field.
- To select the event to use as a trigger for the interrupt, use the `DIO_INT_CFG_EVENT` bit field.
- To enable a button debounce filter, write `DIO_INT_DEBOUNCE_ENABLE` to the `DEBOUNCE_ENABLE` bit, and write `DIO_INT_DEBOUNCE_DISABLE` to disable this feature.

NOTE: To ensure proper initial behavior, GPIO interrupts will not trigger on the first three clock cycles after the `DIO_INT_CFG_SRC` or `DIO_INT_CFG_EVENT` bit field is changed.

The statuses of the eight DIO interrupts are latched in the `DIO_INT_STATUS` register, in the `DIO_INT*_STATUS` bits. The user application must manually clear the status of each interrupt, by writing to the corresponding `DIO_INT*_CLEAR` bit.

18.1.2 GPIO Registers

Register Name	Register Description	Address
DIO_INPUT_DATA_0	Digital IO [23:0] Input Data State	P:0x1264 0x40000990
DIO_INPUT_DATA_1	Digital IO [35:24] Input Data State	P:0x1265 0x40000994
DIO_GPIO_OUTPUT_DATA_0	GPIO [23:0] Output Data Register	P:0x1266 0x40000998
DIO_GPIO_OUTPUT_DATA_1	GPIO [35:24] Output Data Register	P:0x1267 0x4000099C
DIO_GPIO_OUTPUT_DATA_0_SET	GPIO [23:0] Output Data Set	P:0x1268 0x400009A0
DIO_GPIO_OUTPUT_DATA_1_SET	GPIO [35:24] Output Data Set	P:0x1269 0x400009A4

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
DIO_GPIO_OUTPUT_DATA_0_CLR	GPIO [23:0] Output Data Clear	P:0x126A 0x400009A8
DIO_GPIO_OUTPUT_DATA_1_CLR	GPIO [35:24] Output Data Clear	P:0x126B 0x400009AC
DIO_DIR_0	Digital IO [23:0] Direction State	P:0x126C 0x400009B0
DIO_DIR_1	Digital IO [35:24] Direction State	P:0x126D 0x400009B4
DIO_MODE_0	Digital IO [23:0] Mode State	P:0x126E 0x400009B8
DIO_MODE_1	Digital IO [35:24] Mode State	P:0x126F 0x400009BC
DIO_INT_CFG	DIO Interrupt Configuration	P:0x1270-P:0x1277 0x400009C0-0x400009DC
DIO_INT_STATUS	DIO interrupt status	P:0x1278 0x400009E0
DIO_INT_DEBOUNCE	DIO Interrupt Button Debounce Filter Time Configuration	P:0x1279 0x400009E4

18.1.2.1 DIO_INPUT_DATA_0

Bit Field	Read/Write	Field Name	Description
23:0	R	DIO	DIO[23:0] input data

Field Name	Value Symbol	Value Description	Hex Value
DIO	DIO0_LOW	DIO pad is low	0x0*
	DIO1_LOW	DIO pad is low	0x0*
	DIO2_LOW	DIO pad is low	0x0*
	DIO3_LOW	DIO pad is low	0x0*
	DIO4_LOW	DIO pad is low	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DIO5_LOW	DIO pad is low	0x0*
	DIO6_LOW	DIO pad is low	0x0*
	DIO7_LOW	DIO pad is low	0x0*
	DIO8_LOW	DIO pad is low	0x0*
	DIO9_LOW	DIO pad is low	0x0*
	DIO10_LOW	DIO pad is low	0x0*
	DIO11_LOW	DIO pad is low	0x0*
	DIO12_LOW	DIO pad is low	0x0*
	DIO13_LOW	DIO pad is low	0x0*
	DIO14_LOW	DIO pad is low	0x0*
	DIO15_LOW	DIO pad is low	0x0*
	DIO16_LOW	DIO pad is low	0x0*
	DIO17_LOW	DIO pad is low	0x0*
	DIO18_LOW	DIO pad is low	0x0*
	DIO19_LOW	DIO pad is low	0x0*
	DIO20_LOW	DIO pad is low	0x0*
	DIO21_LOW	DIO pad is low	0x0*
	DIO22_LOW	DIO pad is low	0x0*
	DIO23_LOW	DIO pad is low	0x0*
	DIO0_HIGH	DIO pad is high	0x1
	DIO1_HIGH	DIO pad is high	0x2
	DIO2_HIGH	DIO pad is high	0x4
	DIO3_HIGH	DIO pad is high	0x8
	DIO4_HIGH	DIO pad is high	0x10
	DIO5_HIGH	DIO pad is high	0x20
	DIO6_HIGH	DIO pad is high	0x40
	DIO7_HIGH	DIO pad is high	0x80
	DIO8_HIGH	DIO pad is high	0x100
	DIO9_HIGH	DIO pad is high	0x200
	DIO10_HIGH	DIO pad is high	0x400
	DIO11_HIGH	DIO pad is high	0x800

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DIO12_HIGH	DIO pad is high	0x1000
	DIO13_HIGH	DIO pad is high	0x2000
	DIO14_HIGH	DIO pad is high	0x4000
	DIO15_HIGH	DIO pad is high	0x8000
	DIO16_HIGH	DIO pad is high	0x10000
	DIO17_HIGH	DIO pad is high	0x20000
	DIO18_HIGH	DIO pad is high	0x40000
	DIO19_HIGH	DIO pad is high	0x80000
	DIO20_HIGH	DIO pad is high	0x100000
	DIO21_HIGH	DIO pad is high	0x200000
	DIO22_HIGH	DIO pad is high	0x400000
	DIO23_HIGH	DIO pad is high	0x800000

18.1.2.2 DIO_INPUT_DATA_1

Bit Field	Read/Write	Field Name	Description
11:0	R	DIO	DIO[35:24] input data

Field Name	Value Symbol	Value Description	Hex Value
DIO	DIO24_LOW	DIO pad is low	0x0*
	DIO25_LOW	DIO pad is low	0x0*
	DIO26_LOW	DIO pad is low	0x0*
	DIO27_LOW	DIO pad is low	0x0*
	DIO28_LOW	DIO pad is low	0x0*
	DIO29_LOW	DIO pad is low	0x0*
	DIO30_LOW	DIO pad is low	0x0*
	DIO31_LOW	DIO pad is low	0x0*
	DIO32_LOW	DIO pad is low	0x0*
	DIO33_LOW	DIO pad is low	0x0*
	DIO34_LOW	DIO pad is low	0x0*
	DIO35_LOW	DIO pad is low	0x0*
	DIO24_HIGH	DIO pad is high	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DIO25_HIGH	DIO pad is high	0x2
	DIO26_HIGH	DIO pad is high	0x4
	DIO27_HIGH	DIO pad is high	0x8
	DIO28_HIGH	DIO pad is high	0x10
	DIO29_HIGH	DIO pad is high	0x20
	DIO30_HIGH	DIO pad is high	0x40
	DIO31_HIGH	DIO pad is high	0x80
	DIO32_HIGH	DIO pad is high	0x100
	DIO33_HIGH	DIO pad is high	0x200
	DIO34_HIGH	DIO pad is high	0x400
	DIO35_HIGH	DIO pad is high	0x800

18.1.2.3 DIO_GPIO_OUTPUT_DATA_0

Bit Field	Read/Write	Field Name	Description
23:0	RW	GPIO	GPIO[23:0] output data

Field Name	Value Symbol	Value Description	Hex Value
GPIO	GPIO0_LOW	Set the GPIO output data low	0x0*
	GPIO1_LOW	Set the GPIO output data low	0x0*
	GPIO2_LOW	Set the GPIO output data low	0x0*
	GPIO3_LOW	Set the GPIO output data low	0x0*
	GPIO4_LOW	Set the GPIO output data low	0x0*
	GPIO5_LOW	Set the GPIO output data low	0x0*
	GPIO6_LOW	Set the GPIO output data low	0x0*
	GPIO7_LOW	Set the GPIO output data low	0x0*
	GPIO8_LOW	Set the GPIO output data low	0x0*
	GPIO9_LOW	Set the GPIO output data low	0x0*
	GPIO10_LOW	Set the GPIO output data low	0x0*
	GPIO11_LOW	Set the GPIO output data low	0x0*
	GPIO12_LOW	Set the GPIO output data low	0x0*
	GPIO13_LOW	Set the GPIO output data low	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	GPIO14_LOW	Set the GPIO output data low	0x0*
	GPIO15_LOW	Set the GPIO output data low	0x0*
	GPIO16_LOW	Set the GPIO output data low	0x0*
	GPIO17_LOW	Set the GPIO output data low	0x0*
	GPIO18_LOW	Set the GPIO output data low	0x0*
	GPIO19_LOW	Set the GPIO output data low	0x0*
	GPIO20_LOW	Set the GPIO output data low	0x0*
	GPIO21_LOW	Set the GPIO output data low	0x0*
	GPIO22_LOW	Set the GPIO output data low	0x0*
	GPIO23_LOW	Set the GPIO output data low	0x0*
	GPIO0_HIGH	Set the GPIO output data high	0x1
	GPIO1_HIGH	Set the GPIO output data high	0x2
	GPIO2_HIGH	Set the GPIO output data high	0x4
	GPIO3_HIGH	Set the GPIO output data high	0x8
	GPIO4_HIGH	Set the GPIO output data high	0x10
	GPIO5_HIGH	Set the GPIO output data high	0x20
	GPIO6_HIGH	Set the GPIO output data high	0x40
	GPIO7_HIGH	Set the GPIO output data high	0x80
	GPIO8_HIGH	Set the GPIO output data high	0x100
	GPIO9_HIGH	Set the GPIO output data high	0x200
	GPIO10_HIGH	Set the GPIO output data high	0x400
	GPIO11_HIGH	Set the GPIO output data high	0x800
	GPIO12_HIGH	Set the GPIO output data high	0x1000
	GPIO13_HIGH	Set the GPIO output data high	0x2000
	GPIO14_HIGH	Set the GPIO output data high	0x4000
	GPIO15_HIGH	Set the GPIO output data high	0x8000
	GPIO16_HIGH	Set the GPIO output data high	0x10000
	GPIO17_HIGH	Set the GPIO output data high	0x20000
	GPIO18_HIGH	Set the GPIO output data high	0x40000
	GPIO19_HIGH	Set the GPIO output data high	0x80000
	GPIO20_HIGH	Set the GPIO output data high	0x100000

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	GPIO21_HIGH	Set the GPIO output data high	0x200000
	GPIO22_HIGH	Set the GPIO output data high	0x400000
	GPIO23_HIGH	Set the GPIO output data high	0x800000

18.1.2.4 DIO_GPIO_OUTPUT_DATA_1

Bit Field	Read/Write	Field Name	Description
11:0	RW	GPIO	GPIO[35:24] output data

Field Name	Value Symbol	Value Description	Hex Value
GPIO	GPIO24_LOW	Set the GPIO output data low	0x0*
	GPIO25_LOW	Set the GPIO output data low	0x0*
	GPIO26_LOW	Set the GPIO output data low	0x0*
	GPIO27_LOW	Set the GPIO output data low	0x0*
	GPIO28_LOW	Set the GPIO output data low	0x0*
	GPIO29_LOW	Set the GPIO output data low	0x0*
	GPIO30_LOW	Set the GPIO output data low	0x0*
	GPIO31_LOW	Set the GPIO output data low	0x0*
	GPIO32_LOW	Set the GPIO output data low	0x0*
	GPIO33_LOW	Set the GPIO output data low	0x0*
	GPIO34_LOW	Set the GPIO output data low	0x0*
	GPIO35_LOW	Set the GPIO output data low	0x0*
	GPIO24_HIGH	Set the GPIO output data high	0x1
	GPIO25_HIGH	Set the GPIO output data high	0x2
	GPIO26_HIGH	Set the GPIO output data high	0x4
	GPIO27_HIGH	Set the GPIO output data high	0x8
	GPIO28_HIGH	Set the GPIO output data high	0x10
	GPIO29_HIGH	Set the GPIO output data high	0x20
	GPIO30_HIGH	Set the GPIO output data high	0x40
	GPIO31_HIGH	Set the GPIO output data high	0x80
	GPIO32_HIGH	Set the GPIO output data high	0x100

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	GPIO33_HIGH	Set the GPIO output data high	0x200
	GPIO34_HIGH	Set the GPIO output data high	0x400
	GPIO35_HIGH	Set the GPIO output data high	0x800

18.1.2.5 DIO_GPIO_OUTPUT_DATA_0_SET

Bit Field	Read/Write	Field Name	Description
23:0	W	GPIO	GPIO[23:0] output data set

Field Name	Value Symbol	Value Description	Hex Value
GPIO	GPIO0_SET	Set the GPIO output data high	0x1
	GPIO1_SET	Set the GPIO output data high	0x2
	GPIO2_SET	Set the GPIO output data high	0x4
	GPIO3_SET	Set the GPIO output data high	0x8
	GPIO4_SET	Set the GPIO output data high	0x10
	GPIO5_SET	Set the GPIO output data high	0x20
	GPIO6_SET	Set the GPIO output data high	0x40
	GPIO7_SET	Set the GPIO output data high	0x80
	GPIO8_SET	Set the GPIO output data high	0x100
	GPIO9_SET	Set the GPIO output data high	0x200
	GPIO10_SET	Set the GPIO output data high	0x400
	GPIO11_SET	Set the GPIO output data high	0x800
	GPIO12_SET	Set the GPIO output data high	0x1000
	GPIO13_SET	Set the GPIO output data high	0x2000
	GPIO14_SET	Set the GPIO output data high	0x4000
	GPIO15_SET	Set the GPIO output data high	0x8000
	GPIO16_SET	Set the GPIO output data high	0x10000
	GPIO17_SET	Set the GPIO output data high	0x20000
	GPIO18_SET	Set the GPIO output data high	0x40000
	GPIO19_SET	Set the GPIO output data high	0x80000
	GPIO20_SET	Set the GPIO output data high	0x100000

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	GPIO21_SET	Set the GPIO output data high	0x200000
	GPIO22_SET	Set the GPIO output data high	0x400000
	GPIO23_SET	Set the GPIO output data high	0x800000

18.1.2.6 DIO_GPIO_OUTPUT_DATA_1_SET

Bit Field	Read/Write	Field Name	Description
11:0	W	GPIO	GPIO[35:24] output data set

Field Name	Value Symbol	Value Description	Hex Value
GPIO	GPIO24_SET	Set the GPIO output data high	0x1
	GPIO25_SET	Set the GPIO output data high	0x2
	GPIO26_SET	Set the GPIO output data high	0x4
	GPIO27_SET	Set the GPIO output data high	0x8
	GPIO28_SET	Set the GPIO output data high	0x10
	GPIO29_SET	Set the GPIO output data high	0x20
	GPIO30_SET	Set the GPIO output data high	0x40
	GPIO31_SET	Set the GPIO output data high	0x80
	GPIO32_SET	Set the GPIO output data high	0x100
	GPIO33_SET	Set the GPIO output data high	0x200
	GPIO34_SET	Set the GPIO output data high	0x400
	GPIO35_SET	Set the GPIO output data high	0x800

18.1.2.7 DIO_GPIO_OUTPUT_DATA_0_CLR

Bit Field	Read/Write	Field Name	Description
23:0	W	GPIO	GPIO[23:0] output data clear

Field Name	Value Symbol	Value Description	Hex Value
GPIO	GPIO0_CLR	Set the GPIO output data low	0x1
	GPIO1_CLR	Set the GPIO output data low	0x2
	GPIO2_CLR	Set the GPIO output data low	0x4
	GPIO3_CLR	Set the GPIO output data low	0x8

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	GPIO4_CLR	Set the GPIO output data low	0x10
	GPIO5_CLR	Set the GPIO output data low	0x20
	GPIO6_CLR	Set the GPIO output data low	0x40
	GPIO7_CLR	Set the GPIO output data low	0x80
	GPIO8_CLR	Set the GPIO output data low	0x100
	GPIO9_CLR	Set the GPIO output data low	0x200
	GPIO10_CLR	Set the GPIO output data low	0x400
	GPIO11_CLR	Set the GPIO output data low	0x800
	GPIO12_CLR	Set the GPIO output data low	0x1000
	GPIO13_CLR	Set the GPIO output data low	0x2000
	GPIO14_CLR	Set the GPIO output data low	0x4000
	GPIO15_CLR	Set the GPIO output data low	0x8000
	GPIO16_CLR	Set the GPIO output data low	0x10000
	GPIO17_CLR	Set the GPIO output data low	0x20000
	GPIO18_CLR	Set the GPIO output data low	0x40000
	GPIO19_CLR	Set the GPIO output data low	0x80000
	GPIO20_CLR	Set the GPIO output data low	0x100000
	GPIO21_CLR	Set the GPIO output data low	0x200000
	GPIO22_CLR	Set the GPIO output data low	0x400000
	GPIO23_CLR	Set the GPIO output data low	0x800000

18.1.2.8 DIO_GPIO_OUTPUT_DATA_1_CLR

Bit Field	Read/Write	Field Name	Description
11:0	W	GPIO	GPIO[35:24] output data clear

Field Name	Value Symbol	Value Description	Hex Value
GPIO	GPIO24_CLR	Set the GPIO output data low	0x1
	GPIO25_CLR	Set the GPIO output data low	0x2
	GPIO26_CLR	Set the GPIO output data low	0x4
	GPIO27_CLR	Set the GPIO output data low	0x8
	GPIO28_CLR	Set the GPIO output data low	0x10

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	GPIO29_CLR	Set the GPIO output data low	0x20
	GPIO30_CLR	Set the GPIO output data low	0x40
	GPIO31_CLR	Set the GPIO output data low	0x80
	GPIO32_CLR	Set the GPIO output data low	0x100
	GPIO33_CLR	Set the GPIO output data low	0x200
	GPIO34_CLR	Set the GPIO output data low	0x400
	GPIO35_CLR	Set the GPIO output data low	0x800

18.1.2.9 DIO_DIR_0

Bit Field	Read/Write	Field Name	Description
23:0	R	DIO	Get DIO[23:0] direction
23:0	W	GPIO	GPIO[23:0] direction

Field Name	Value Symbol	Value Description	Hex Value
DIO	DIO0_INPUT	DIO[23:0] is an input	0x0*
	DIO1_INPUT	DIO[23:0] is an input	0x0*
	DIO2_INPUT	DIO[23:0] is an input	0x0*
	DIO3_INPUT	DIO[23:0] is an input	0x0*
	DIO4_INPUT	DIO[23:0] is an input	0x0*
	DIO5_INPUT	DIO[23:0] is an input	0x0*
	DIO6_INPUT	DIO[23:0] is an input	0x0*
	DIO7_INPUT	DIO[23:0] is an input	0x0*
	DIO8_INPUT	DIO[23:0] is an input	0x0*
	DIO9_INPUT	DIO[23:0] is an input	0x0*
	DIO10_INPUT	DIO[23:0] is an input	0x0*
	DIO11_INPUT	DIO[23:0] is an input	0x0*
	DIO12_INPUT	DIO[23:0] is an input	0x0*
	DIO13_INPUT	DIO[23:0] is an input	0x0*
	DIO14_INPUT	DIO[23:0] is an input	0x0*
	DIO15_INPUT	DIO[23:0] is an input	0x0*
	DIO16_INPUT	DIO[23:0] is an input	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DIO17_INPUT	DIO[23:0] is an input	0x0*
	DIO18_INPUT	DIO[23:0] is an input	0x0*
	DIO19_INPUT	DIO[23:0] is an input	0x0*
	DIO20_INPUT	DIO[23:0] is an input	0x0*
	DIO21_INPUT	DIO[23:0] is an input	0x0*
	DIO22_INPUT	DIO[23:0] is an input	0x0*
	DIO23_INPUT	DIO[23:0] is an input	0x0*
	DIO0_OUTPUT	DIO[23:0] is an output	0x1
	DIO1_OUTPUT	DIO[23:0] is an output	0x2
	DIO2_OUTPUT	DIO[23:0] is an output	0x4
	DIO3_OUTPUT	DIO[23:0] is an output	0x8
	DIO4_OUTPUT	DIO[23:0] is an output	0x10
	DIO5_OUTPUT	DIO[23:0] is an output	0x20
	DIO6_OUTPUT	DIO[23:0] is an output	0x40
	DIO7_OUTPUT	DIO[23:0] is an output	0x80
	DIO8_OUTPUT	DIO[23:0] is an output	0x100
	DIO9_OUTPUT	DIO[23:0] is an output	0x200
	DIO10_OUTPUT	DIO[23:0] is an output	0x400
	DIO11_OUTPUT	DIO[23:0] is an output	0x800
	DIO12_OUTPUT	DIO[23:0] is an output	0x1000
	DIO13_OUTPUT	DIO[23:0] is an output	0x2000
	DIO14_OUTPUT	DIO[23:0] is an output	0x4000
	DIO15_OUTPUT	DIO[23:0] is an output	0x8000
	DIO16_OUTPUT	DIO[23:0] is an output	0x10000
	DIO17_OUTPUT	DIO[23:0] is an output	0x20000
	DIO18_OUTPUT	DIO[23:0] is an output	0x40000
	DIO19_OUTPUT	DIO[23:0] is an output	0x80000
	DIO20_OUTPUT	DIO[23:0] is an output	0x100000
	DIO21_OUTPUT	DIO[23:0] is an output	0x200000
	DIO22_OUTPUT	DIO[23:0] is an output	0x400000
	DIO23_OUTPUT	DIO[23:0] is an output	0x800000

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
GPIO	GPIO0_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO1_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO2_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO3_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO4_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO5_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO6_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO7_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO8_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO9_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO10_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO11_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO12_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO13_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO14_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO15_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO16_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO17_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO18_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO19_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO20_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO21_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO22_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO23_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO0_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x1
	GPIO1_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x2
	GPIO2_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x4
	GPIO3_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x8
	GPIO4_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x10
	GPIO5_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x20
	GPIO6_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x40

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	GPIO7_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x80
	GPIO8_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x100
	GPIO9_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x200
	GPIO10_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x400
	GPIO11_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x800
	GPIO12_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x1000
	GPIO13_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x2000
	GPIO14_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x4000
	GPIO15_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x8000
	GPIO16_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x10000
	GPIO17_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x20000
	GPIO18_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x40000
	GPIO19_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x80000
	GPIO20_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x100000
	GPIO21_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x200000
	GPIO22_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x400000
	GPIO23_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x800000

18.1.2.10 DIO_DIR_1

Bit Field	Read/Write	Field Name	Description
11:0	R	DIO	Get DIO[35:24] direction
11:0	W	GPIO	GPIO[35:24] direction

Field Name	Value Symbol	Value Description	Hex Value
DIO	DIO24_INPUT	DIO[35:24] is an input	0x0
	DIO25_INPUT	DIO[35:24] is an input	0x0
	DIO26_INPUT	DIO[35:24] is an input	0x0
	DIO27_INPUT	DIO[35:24] is an input	0x0
	DIO28_INPUT	DIO[35:24] is an input	0x0
	DIO29_INPUT	DIO[35:24] is an input	0x0
	DIO30_INPUT	DIO[35:24] is an input	0x0

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DIO31_INPUT	DIO[35:24] is an input	0x0
	DIO32_INPUT	DIO[35:24] is an input	0x0
	DIO33_INPUT	DIO[35:24] is an input	0x0
	DIO34_INPUT	DIO[35:24] is an input	0x0
	DIO35_INPUT	DIO[35:24] is an input	0x0
	DIO24_OUTPUT	DIO[35:24] is an output	0x1
	DIO25_OUTPUT	DIO[35:24] is an output	0x2
	DIO26_OUTPUT	DIO[35:24] is an output	0x4
	DIO27_OUTPUT	DIO[35:24] is an output	0x8
	DIO28_OUTPUT	DIO[35:24] is an output	0x10
	DIO29_OUTPUT	DIO[35:24] is an output	0x20
	DIO30_OUTPUT	DIO[35:24] is an output	0x40
	DIO31_OUTPUT	DIO[35:24] is an output	0x80
	DIO32_OUTPUT	DIO[35:24] is an output	0x100
	DIO33_OUTPUT	DIO[35:24] is an output	0x200
	DIO34_OUTPUT	DIO[35:24] is an output	0x400
	DIO35_OUTPUT	DIO[35:24] is an output	0x800
	DIO32_DIO33_OUTPUT	DIO32 and DIO33 are outputs	0x300*
GPIO	GPIO24_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO25_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO26_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO27_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO28_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO29_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO30_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO31_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO32_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO33_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO34_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO35_INPUT	Set DIO to input if IO_MODE is GPIO	0x0*
	GPIO24_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	GPIO25_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x2
	GPIO26_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x4
	GPIO27_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x8
	GPIO28_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x10
	GPIO29_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x20
	GPIO30_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x40
	GPIO31_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x80
	GPIO32_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x100
	GPIO33_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x200
	GPIO34_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x400
	GPIO35_OUTPUT	Set DIO to output if IO_MODE is GPIO	0x800

18.1.2.11 DIO_MODE_0

Bit Field	Read/Write	Field Name	Description
23:0	R	DIO	DIO[23:0] mode

Field Name	Value Symbol	Value Description	Hex Value
DIO	DIO0_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO1_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO2_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO3_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO4_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO5_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO6_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO7_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO8_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DIO9_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO10_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO11_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO12_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO13_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO14_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO15_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO16_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO17_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO18_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO19_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO20_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO21_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO22_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO23_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO0_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x1
	DIO1_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x2
	DIO2_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x4
	DIO3_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x8

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DIO4_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x10
	DIO5_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x20
	DIO6_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x40
	DIO7_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x80
	DIO8_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x100
	DIO9_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x200
	DIO10_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x400
	DIO11_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x800
	DIO12_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x1000
	DIO13_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x2000
	DIO14_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x4000
	DIO15_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x8000
	DIO16_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x10000
	DIO17_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x20000
	DIO18_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x40000
	DIO19_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x80000
	DIO20_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x100000

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DIO21_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x200000
	DIO22_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x400000
	DIO23_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x800000

18.1.2.12 DIO_MODE_1

Bit Field	Read/Write	Field Name	Description
11:0	R	DIO	DIO[35:24] mode

Field Name	Value Symbol	Value Description	Hex Value
DIO	DIO24_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO25_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO26_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO27_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO28_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO29_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO30_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO31_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO32_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO33_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO34_IS_NOT_GPIO	This DIO is not configured as a Arm Cortex-M3 controlled GPIO	0x0*
	DIO24_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x1
	DIO25_IS_GPIO	This DIO is configured as a Arm	0x2

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		Cortex-M3 controlled GPIO	
	DIO26_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x4
	DIO27_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x8
	DIO28_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x10
	DIO29_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x20
	DIO30_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x40
	DIO31_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x80
	DIO32_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x100
	DIO33_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x200
	DIO34_IS_GPIO	This DIO is configured as a Arm Cortex-M3 controlled GPIO	0x400

18.1.2.13 DIO_INT_CFG*

Bit Field	Read/Write	Field Name	Description
11	RW	DEBOUNCE_ENABLE	Interrupt button debounce filter enable/disable
10:8	RW	EVENT	Interrupt event configuration
5:0	RW	SRC	Interrupt input selection

Field Name	Value Symbol	Value Description	Hex Value
DEBOUNCE_ENABLE	DIO_INT_DEBOUNCE_DISABLE	Button debounce filter disabled	0x0*
	DIO_INT_DEBOUNCE_ENABLE	Button debounce filter enabled	0x1
EVENT	DIO_INT_EVENT_NONE	Interrupt not triggered	0x0*
	DIO_INT_EVENT_HIGH_LEVEL	Interrupt triggered on high state	0x1
	DIO_INT_EVENT_LOW_LEVEL	Interrupt triggered on low state	0x2
	DIO_INT_EVENT_RISING_EDGE	Interrupt triggered on rising edge	0x3
	DIO_INT_EVENT_FALLING_EDGE	Interrupt triggered on falling edge	0x4

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DIO_INT_EVENT_TRANSITION	Interrupt triggered on any edge	0x5
SRC	DIO_INT_SRC_DIO_0	Select DIO[0] as source	0x0*
	DIO_INT_SRC_DIO_1	Select DIO[1] as source	0x1
	DIO_INT_SRC_DIO_2	Select DIO[2] as source	0x2
	DIO_INT_SRC_DIO_3	Select DIO[3] as source	0x3
	DIO_INT_SRC_DIO_4	Select DIO[4] as source	0x4
	DIO_INT_SRC_DIO_5	Select DIO[5] as source	0x5
	DIO_INT_SRC_DIO_6	Select DIO[6] as source	0x6
	DIO_INT_SRC_DIO_7	Select DIO[7] as source	0x7
	DIO_INT_SRC_DIO_8	Select DIO[8] as source	0x8
	DIO_INT_SRC_DIO_9	Select DIO[9] as source	0x9
	DIO_INT_SRC_DIO_10	Select DIO[10] as source	0xA
	DIO_INT_SRC_DIO_11	Select DIO[11] as source	0xB
	DIO_INT_SRC_DIO_12	Select DIO[12] as source	0xC
	DIO_INT_SRC_DIO_13	Select DIO[13] as source	0xD
	DIO_INT_SRC_DIO_14	Select DIO[14] as source	0xE
	DIO_INT_SRC_DIO_15	Select DIO[15] as source	0xF
	DIO_INT_SRC_DIO_16	Select DIO[16] as source	0x10
	DIO_INT_SRC_DIO_17	Select DIO[17] as source	0x11
	DIO_INT_SRC_DIO_18	Select DIO[18] as source	0x12
	DIO_INT_SRC_DIO_19	Select DIO[19] as source	0x13
	DIO_INT_SRC_DIO_20	Select DIO[20] as source	0x14
	DIO_INT_SRC_DIO_21	Select DIO[21] as source	0x15
	DIO_INT_SRC_DIO_22	Select DIO[22] as source	0x16
	DIO_INT_SRC_DIO_23	Select DIO[23] as source	0x17
	DIO_INT_SRC_DIO_24	Select DIO[24] as source	0x18
	DIO_INT_SRC_DIO_25	Select DIO[25] as source	0x19
	DIO_INT_SRC_DIO_26	Select DIO[26] as source	0x1A
	DIO_INT_SRC_DIO_27	Select DIO[27] as source	0x1B
	DIO_INT_SRC_DIO_28	Select DIO[28] as source	0x1C
	DIO_INT_SRC_DIO_29	Select DIO[29] as source	0x1D

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DIO_INT_SRC_DIO_30	Select DIO[30] as source	0x1E
	DIO_INT_SRC_DIO_31	Select DIO[31] as source	0x1F
	DIO_INT_SRC_DIO_32	Select DIO[32] as source	0x20
	DIO_INT_SRC_DIO_33	Select DIO[33] as source	0x21
	DIO_INT_SRC_DIO_34	Select DIO[34] as source	0x22
	DIO_INT_SRC_DIO_35	Select DIO[35] as source	0x23
	DIO_INT_SRC_SCL	Select SCL as source	0x24
	DIO_INT_SRC_SDA	Select SDA as source	0x25
	DIO_INT_SRC_CLKDIV0	Select CLKDIV0 as source	0x26
	DIO_INT_SRC_CLKDIV1	Select CLKDIV1 as source	0x27

18.1.2.14 DIO_INT_STATUS

Bit Field	Read/Write	Field Name	Description
15	R	DIO_INT7_STATUS	DIO interrupt 7 status
14	R	DIO_INT6_STATUS	DIO interrupt 6 status
13	R	DIO_INT5_STATUS	DIO interrupt 5 status
12	R	DIO_INT4_STATUS	DIO interrupt 4 status
11	R	DIO_INT3_STATUS	DIO interrupt 3 status
10	R	DIO_INT2_STATUS	DIO interrupt 2 status
9	R	DIO_INT1_STATUS	DIO interrupt 1 status
8	R	DIO_INT0_STATUS	DIO interrupt 0 status
7	W	DIO_INT7_CLEAR	DIO interrupt 7 clear
6	W	DIO_INT6_CLEAR	DIO interrupt 6 clear
5	W	DIO_INT5_CLEAR	DIO interrupt 5 clear
4	W	DIO_INT4_CLEAR	DIO interrupt 4 clear
3	W	DIO_INT3_CLEAR	DIO interrupt 3 clear
2	W	DIO_INT2_CLEAR	DIO interrupt 2 clear
1	W	DIO_INT1_CLEAR	DIO interrupt 1 clear
0	W	DIO_INT0_CLEAR	DIO interrupt 0 clear

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DIO_INT7_STATUS	DIO_INT7_FALSE	Interrupt 7 not triggered	0x0*
	DIO_INT7_TRUE	Interrupt 7 triggered	0x1
DIO_INT6_STATUS	DIO_INT6_FALSE	Interrupt 6 not triggered	0x0*
	DIO_INT6_TRUE	Interrupt 6 triggered	0x1
DIO_INT5_STATUS	DIO_INT5_FALSE	Interrupt 5 not triggered	0x0*
	DIO_INT5_TRUE	Interrupt 5 triggered	0x1
DIO_INT4_STATUS	DIO_INT4_FALSE	Interrupt 4 not triggered	0x0*
	DIO_INT4_TRUE	Interrupt 4 triggered	0x1
DIO_INT3_STATUS	DIO_INT3_FALSE	Interrupt 3 not triggered	0x0*
	DIO_INT3_TRUE	Interrupt 3 triggered	0x1
DIO_INT2_STATUS	DIO_INT2_FALSE	Interrupt 2 not triggered	0x0*
	DIO_INT2_TRUE	Interrupt 2 triggered	0x1
DIO_INT1_STATUS	DIO_INT1_FALSE	Interrupt 1 not triggered	0x0*
	DIO_INT1_TRUE	Interrupt 1 triggered	0x1
DIO_INT0_STATUS	DIO_INT0_FALSE	Interrupt 0 not triggered	0x0*
	DIO_INT0_TRUE	Interrupt 0 triggered	0x1
DIO_INT7_CLEAR	DIO_INT7_CLEAR	Interrupt 7 clear	0x1
DIO_INT6_CLEAR	DIO_INT6_CLEAR	Interrupt 6 clear	0x1
DIO_INT5_CLEAR	DIO_INT5_CLEAR	Interrupt 5 clear	0x1
DIO_INT4_CLEAR	DIO_INT4_CLEAR	Interrupt 4 clear	0x1
DIO_INT3_CLEAR	DIO_INT3_CLEAR	Interrupt 3 clear	0x1
DIO_INT2_CLEAR	DIO_INT2_CLEAR	Interrupt 2 clear	0x1
DIO_INT1_CLEAR	DIO_INT1_CLEAR	Interrupt 1 clear	0x1
DIO_INT0_CLEAR	DIO_INT0_CLEAR	Interrupt 0 clear	0x1

18.1.2.15 DIO_INT_DEBOUNCE

Bit Field	Read/Write	Field Name	Description
8	RW	DEBOUNCE_CLK	Interrupt button debounce filter clock
7:0	RW	DEBOUNCE_COUNT	Interrupt button debounce filter count

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DEBOUNCE_CLK	DIO_DEBOUNCE_SLOWCLK_DIV32	Button debounce filter runs on SLOWCLK divided by 32	0x0*
	DIO_DEBOUNCE_SLOWCLK_DIV1024	Button debounce filter runs on SLOWCLK divided by 1024	0x1

18.2 DIGITAL MICROPHONE (DMIC) INPUTS AND OUTPUTS

The DMIC block in Ezairo 8300 uses the input and output stage components to provide four inputs and two outputs. For the purpose of this text and for all of the inputs and outputs mentioned, the low data channel will represent the data sent when the DMIC clock is low, and the high data channel will represent the data sent when the DMIC clock is high.

18.2.1 Digital Microphone Inputs

Each of the input channels from the Ezairo 8300 system can optionally source their input data from a digital microphone instead of the channel's ADC. The source for each digital microphone and the sampling clock is selected from the digital I/Os and configured as described in [Chapter 16.1 "Overview" on page 463](#).

The DMIC inputs have pre-decimation filters that are implemented using a 6th order Sync filter, which supports variable decimation rates. The decimation factor for the DMIC input signals is the same as the ADCs, as defined in the `AUDIO_ADC_SAMPLE_FREQ_CFG` register. Similarly, the DMIC pre-decimation filters also support fractional delay in the same way that the ADCs do.

The clock used to sample the DMIC input data can be provided either by Ezairo 8300, or by an external clock source via the `EXTCLK` pad. In either case, the DMIC clock source must be configured to be the DIO or the `EXTCLK` pad on which the DMIC clock is available. If you are using `EXTCLK` as the source for the DMIC clock input, this clock signal must have the same nominal frequency as `ADCCLK`. This is done to ensure that the DMIC input data is synchronized with the Ezairo 8300 input stage.

The DMIC input signals from each of the DMIC inputs are configured to clock in the digital input data by setting the `AUDIO_MUX_CFG_INPUT_CH*_SRC` bit field from the `AUDIO_MUX_CFG` register to either:

- The `DMIC*_DATA_RE` bit setting for the rising edge of this clock
- The `DMIC*_DATA_FE` bit setting for the falling edge of this clock

If the DMIC input is a mono signal, the data can be captured on either the rising or the falling edge of the DMIC input clock. If the DMIC input is a stereo signal, assign both DMIC inputs to the same DIO pad. The low data channel is captured on the rising DMIC clock edge, while the high data channel is captured on the falling DMIC clock edge. The mapping of DMIC inputs to the input channels is assigned as appropriate to the user application. DMIC input timing is shown in the ["Relative DMIC input timing" figure \(Figure 32\)](#).

Ezairo 8300 Hardware Reference

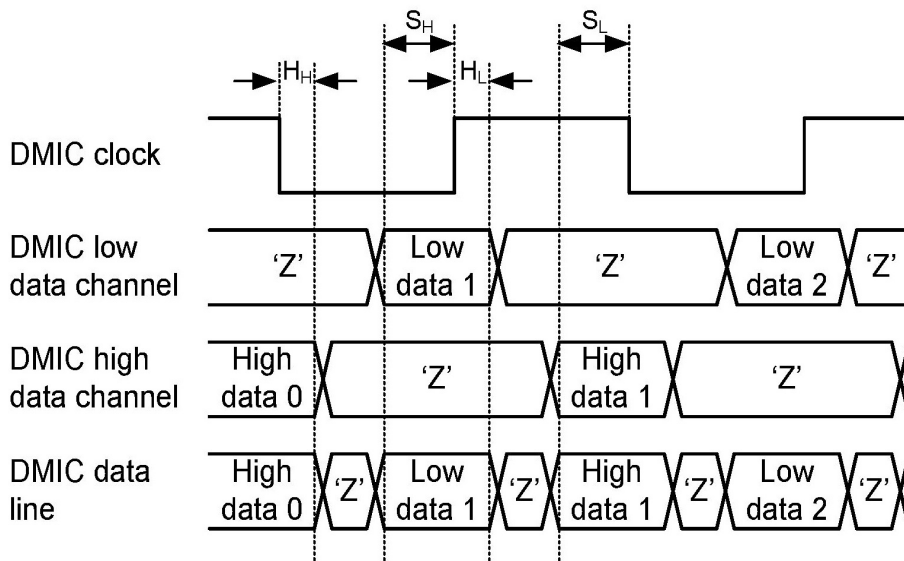


Figure 32. Relative DMIC input timing

18.2.2 Digital Microphone Outputs

The Ezairo 8300 system provides the ability to sink data from each of the output channels to a pair of digital microphone (DMIC) outputs. The pad used for each DMIC output is selected from the DIOs and configured as described in [Chapter 16.1 "Overview" on page 463](#).

The DMIC outputs are enabled using the `AUDIO_DMIC_OUTPUT_CFG_DMIC*_ENABLE` bits of the `AUDIO_DMIC_OUTPUT_CFG` register. These outputs take input data from the sigma-delta modulators of the output channels, and can be used either in parallel with or independently of the direct-digital outputs.

The DMIC output is synchronized using a clock provided to the Ezairo 8300 system through the `EXTCLK` pad. This clock input and the sigma-delta modulator have the following requirements for the DMIC outputs to function correctly:

- The DMIC clock must have the same nominal frequency as `SDMCLK`, or the frequency must be an integer multiplication of `SDMCLK`.
- The `AUDIO_SDM_CTRL` register of the sigma-delta modulator that generates the DMIC output signal must be configured with the following settings:
 - `AUDIO_SDM_CTRL_TRI_STATE` bit is set to `SD_2_LEVEL`
 - `AUDIO_SDM_XSDM0` bit is set to `XSDM0_DISABLE`
 - `AUDIO_SDM_OD_DELAY` bit is set to `OD_DELAY_DISABLE`

For each DMIC output, the DMIC data can be configured for the high and low portions of the `EXTCLK` clock cycle. While the clock is high, the `AUDIO_DMIC_OUTPUT_CFG_DMIC*_HIGH_SRC` bits from the `AUDIO_DMIC_OUTPUT_CFG` register choose between output channels 0 and 1 as the source for the DMIC output. While the clock is low, the `AUDIO_DMIC_OUTPUT_CFG_DMIC*_LOW_SRC` bits from the `AUDIO_DMIC_OUTPUT_CFG` register choose between output channels 0 and 1 as the source for the DMIC output.

Ezairo 8300 Hardware Reference

NOTE: You can drive a DMIC output line with two different Ezairo 8300 devices by configuring one device to drive the output when the DMIC clock is low, and configuring the other to drive the output when the DMIC clock is high. This can be used to transmit audio to a third device, or to transmit audio between two Ezairo 8300 devices.

NOTE: To treat a DMIC output as a mono output, we recommend configuring both the high and low sources to use the same output channel, because this is less likely to cause errors. To treat a DMIC output as a stereo output, the high and low sources can be derived independently from the output channels as needed by the remote system.

18.3 EMMC INTERFACE

The eMMC interface increases content capacity and bandwidth and adds faster switching, compared with other external synchronous interfaces available for use with Ezairo 8300. The "eMMC System-Level Block Diagram" figure (Figure 33), below, shows a block diagram of the eMMC interface system:

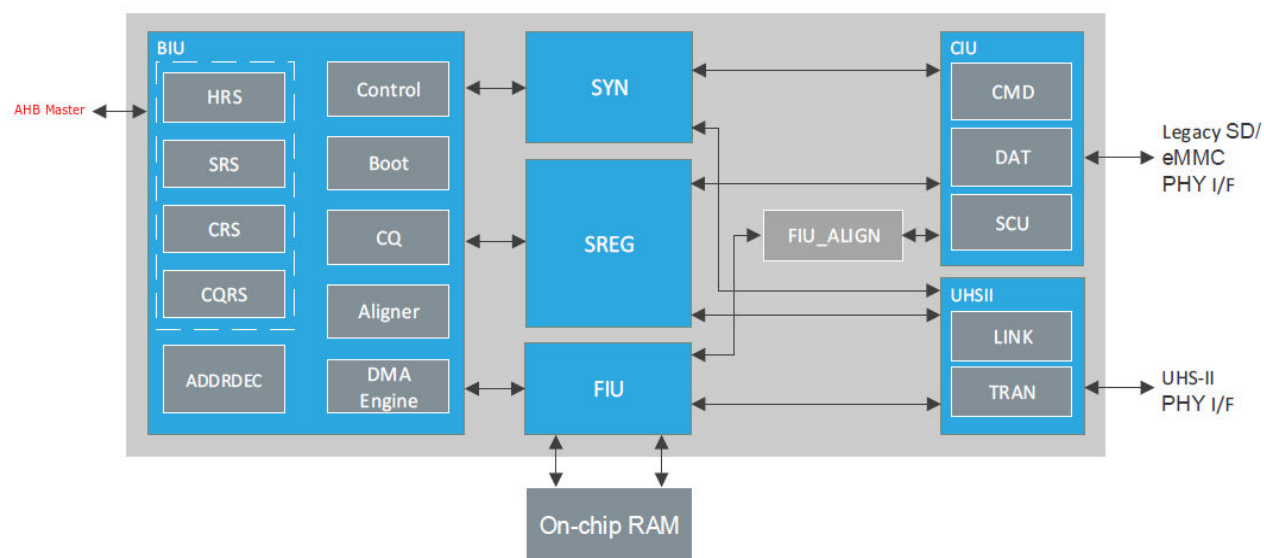


Figure 33. eMMC System-Level Block Diagram

IMPORTANT: This section provides an overview of this interface, with limited details as this interface is intended to be primarily used within the context of the NVM library, as described in the *Ezairo 8300 Firmware Reference*. For a more detailed description of the eMMC implementation itself, consult the *Cadence SD Host Controller IP User Guide*, included with your Ezairo 8300 documentation.

The main blocks shown in the "eMMC System-Level Block Diagram" figure (Figure 33) are described as follows:

- The AHB Master and bus interfaces are tied to the peripheral bus, and are accessible to the CFX and Arm Cortex-M3 cores within the Ezairo 8300 system. This includes the following control register sets:

Ezairo 8300 Hardware Reference

- HRS - Host register set includes implementation of additional registers that are not defined by the *SD Host Controller Standard Specification* (released by SD Association) and are specific to Ezairo 8300's implementation of the SD Host Controller.
- SRS - Slot register set and extended register set includes implementation of registers defined by the *SD Host Controller Standard Specification*.
- CRS - Common register set, implementation of registers in the common register area defined by *SD Host Controller Standard Specification*.
- CQRS - Command queuing register set, implementation of registers defined in the section *B.4. CQE Registers* of the *eMMC Standard 5.1*.
- CQ - The Command Queuing Engine module processes requested commands.
- Boot - The eMMC boot control module; this is not enabled in the Ezairo 8300.
- Control - The control module is a set of logic providing signals appropriate to the executed commands. Based on the information in HRS, CRS, and SRS, the control module stimulates other modules to interact with each other.

The "eMMC Interface Signals" table (Table 54), below, shows interface signals used by the eMMC.

Table 54. eMMC Interface Signals

Name	Type	Description
CLK	O	Clock
CMD	I/O/PP/OD	Command/Response
DAT0	I/O/PP	Data bit 0
DAT1	I/O/PP	Data bit 1
DAT2	I/O/PP	Data bit 2
DAT3	I/O/PP	Data bit 3
DAT4	I/O/PP	Data bit 4
DAT5	I/O/PP	Data bit 5
DAT6	I/O/PP	Data bit 6
DAT7	I/O/PP	Data bit 7
NOTES: Types: I = input; O = output; PP = push-pull; OD = open-drain. Number of bits is configurable to 1, 4, or 8.		

The eMMC includes the following features:

- Compatibility
 - SD Host Controller Standard Specification version 4.00
 - SD Physical Layer Specification version 4.00
 - SDIO Specification version 4.00
 - eMMC Specification 5.0
- Standard Register Set
 - Compatible with SD Host Controller Standard Specification version 4.00
 - Independent Interrupt and Wakeup outputs
- Data Buffering
 - Dedicated eMMC buffer for input and output data

Ezairo 8300 Hardware Reference

- eMMC interface
 - programmable bus width
 - hardware implementation of eMMC boot
- Low power features
 - Master SD card side clock can be switched off
 - Card clock can be switched off independently

The "Supported eMMC Host Functions" table (Table 55), below, shows the eMMC host functions and whether or not they are supported.

Table 55. Supported eMMC Host Functions

Function	Support Status
52 - 200 MHz transfer rate	Not supported
26-52 MHz transfer rate	Not supported
20-26 MHz transfer rate	Not supported
0-26 MHz transfer rate	Supported up to 15 MHz (SYSCLK/2)
2.7 V-3.6 V power supply	Supported with external supply to eMMC device but not for signaling
1.70 V-1.95 V power supply	Supported for signaling only
CCC 0 Basic	Supported
CCC 1 Sequential Read	Not supported
CCC 2 Block Read	Supported
CCC 3 Sequential Write	Not supported
CCC 4 Block Write	Supported
CCC 5 Erase	Supported
CCC 6 Write Protection Functions	Supported
CCC 7 Lock Device Commands	Supported
CCC 8 Application Specific Commands	Supported
CCC 9 Interrupt and Fast Read/Write	Supported
CCC 10 Security Protocols	Supported
CCC 11 Command Queuing	Supported
DSR	Supported
SPI Mode	Not supported

The following list presents details about eMMC control, interrupt, and DMA:

1. The `INTERFACE_EMMC_CTRL` register controls the eMMC IP's clock and reset. After power-on the IP is reset and not clocked, and only consumes leakage power.
2. The eMMC interface IP has an interrupt output that is connected to the system.
3. There are no dedicated DMA request lines for the eMMC interface. Instead, a simple DMA integrated with the eMMC provides fixed size DMA transfers to and from the eMMC interface, as configured with the `EMMC_SRS08` register.

onsemi Confidential
Ezairo 8300 Hardware Reference

18.3.1 eMMC Registers

Register Name	Register Description	Address
INTERFACE_EMMC_CTRL	EMMC Control Register	P:0x14C0 0x40001300
EMMC_HRS00	eMMC General Information Register	P:0x1700 0x40001C00
EMMC_HRS01	eMMC Debounce Setting Register	P:0x1701 0x40001C04
EMMC_HRS04	eMMC PHY Registers Interface	P:0x1704 0x40001C10
EMMC_HRS06	eMMC control registers	P:0x1706 0x40001C18
EMMC_HRS07	eMMC IO Delay Information Register	P:0x1707 0x40001C1C
EMMC_HRS30	eMMC Host Capability Register	P:0x171E 0x40001C78
EMMC_HRS31	eMMC Host Controller Version	P:0x171F 0x40001C7C
EMMC_HRS32	eMMC FSM Monitor Register	P:0x1720 0x40001C80
EMMC_HRS33	eMMC Tune Status 0 Register	P:0x1721 0x40001C84
EMMC_HRS34	eMMC Tune Status 1 Register	P:0x1722 0x40001C88
EMMC_HRS35	eMMC Tune Debug Register	P:0x1723 0x40001C8C
EMMC_HRS36	eMMC Boot Status Register	P:0x1724 0x40001C90
EMMC_HRS37	eMMC Read block gap coefficient interface mode select	P:0x1725

onsemi Confidential
Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
		0x40001C94
EMMC_HRS38	eMMC Read block gap coefficient	P:0x1726 0x40001C98
EMMC_CRS63	eMMC Host Controller Version/Slot Interrupt Status	P:0x173F 0x40001CFC
EMMC_SRS00	eMMC Auto CMD23 Argument	P:0x1780 0x40001E00
EMMC_SRS01	eMMC Block Size / Block Count	P:0x1781 0x40001E04
EMMC_SRS02	eMMC Argument 1	P:0x1782 0x40001E08
EMMC_SRS03	eMMC Command/Transfer Mode	P:0x1783 0x40001E0C
EMMC_SRS04	eMMC Response Register 1	P:0x1784 0x40001E10
EMMC_SRS05	eMMC Response Register 2	P:0x1785 0x40001E14
EMMC_SRS06	eMMC Response Register 3	P:0x1786 0x40001E18
EMMC_SRS07	eMMC Response Register 4	P:0x1787 0x40001E1C
EMMC_SRS08	eMMC Data Buffer	P:0x1788 0x40001E20
EMMC_SRS09	eMMC Present State Register	P:0x1789 0x40001E24
EMMC_SRS10	eMMC Host Control 1	P:0x178A 0x40001E28
EMMC_SRS11	eMMC Host Control 2	P:0x178B

onsemi Confidential

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
		0x40001E2C
EMMC_SRS12	eMMC Error/Normal Interrupt Status	P:0x178C 0x40001E30
EMMC_SRS13	eMMC Error/Normal Status Enable	P:0x178D 0x40001E34
EMMC_SRS14	eMMC Error/Normal Signal Enable	P:0x178E 0x40001E38
EMMC_SRS15	eMMC Host Control 2 / Auto CMD Error Status	P:0x178F 0x40001E3C
EMMC_SRS16	eMMC capabilities 1	P:0x1790 0x40001E40
EMMC_SRS17	eMMC capabilities 2	P:0x1791 0x40001E44
EMMC_SRS18	eMMC capabilities 3	P:0x1792 0x40001E48
EMMC_SRS19	eMMC capabilities 4	P:0x1793 0x40001E4C
EMMC_SRS20	eMMC Force Event	P:0x1794 0x40001E50
EMMC_SRS24	eMMC Preset Value Default Speed	P:0x1798 0x40001E60
EMMC_SRS25	eMMC Preset Value High Speed and SDR12	P:0x1799 0x40001E64
EMMC_SRS26	eMMC Preset Value SDR25 and SDR50	P:0x179A 0x40001E68
EMMC_SRS27	eMMC Preset Value for UHS-2	P:0x179B 0x40001E6C
EMMC_SRS29	eMMC Preset Value for UHS-II	P:0x179D 0x40001E74

Ezairo 8300 Hardware Reference

18.3.1.1 INTERFACE_EMMC_CTRL

Bit Field	Read/Write	Field Name	Description
8	R	CLK_ENABLE_STATUS	EMMC interface clock enable status
2	W	RESET	Reset the EMMC interface
1	W	CLK_DISABLE	Disable the EMMC interface clock
0	W	CLK_ENABLE	Enable the EMMC interface clock

Field Name	Value Symbol	Value Description	Hex Value
CLK_ENABLE_STATUS	EMMC_CLK_STATUS_DISABLED	EMMC interface clock is disabled	0x0*
	EMMC_CLK_STATUS_ENABLED	EMMC interface clock is enabled	0x1
RESET	EMMC_RESET	Reset the EMMC interface clock	0x1
CLK_DISABLE	EMMC_CLK_DISABLE	Disable the EMMC interface clock	0x1
CLK_ENABLE	EMMC_CLK_ENABLE	Enable the EMMC interface clock	0x1

18.3.1.2 EMMC_HRS00

Bit Field	Read/Write	Field Name	Description
31:24	R	HRS00_HWINIT1	HWINIT field
23:16	R	SAV	SAV - Slot Available
15:1	R	HRS00_HWINIT0	HWINIT field
0	RW	SWR	SWR - Software Reset

Field Name	Value Symbol	Value Description	Hex Value
SWR	EMMC_HWINIT0_SWR_DISABLED	The Software Reset bit is cleared; software reset is disabled	0x0*
	EMMC_HWINIT0_SWR_ENABLED	Software reset is enabled; this resets the entire core	0x1

18.3.1.3 EMMC_HRS01

Bit Field	Read/Write	Field Name	Description
31:24	R	HRS01_HWINIT0	HWINIT field
23:0	RW	DP	DP - Debounce Period: Defines the number of system (clk) clock cycles used by the debounce logic, which detects card insertion and removal events.

Ezairo 8300 Hardware Reference

18.3.1.4 EMMC_HRS04

Bit Field	Read/Write	Field Name	Description
26	R	UIS_ACK	UIS_ACK - PHY Transaction Acknowledge
25	RW	UIS_RD	UIS_RD - PHY Read Transaction Request
24	RW	UIS_WR	UIS_WR - PHY Write Transaction Request: This bit requests write transaction to the PHY register
23:16	R	UIS_RDATA	UIS_RDATA - PHY Read Data: This field stores data read from PHY register
15:8	RW	UIS_WDATA	UIS_WDATA - PHY Write Data: This field stores data to be written to the PHY register
5:0	RW	UIS_ADDR	UIS_ADDR - PHY Register Address: This field stores address of the PHY register which is to be read or written.

Field Name	Value Symbol	Value Description	Hex Value
UIS_ACK	EMMC_UIS_ACK_NO_TRANSACTION	No UIS Write or Read transaction has taken place	0x0*
	EMMC_UIS_ACK_TRANSACTION	UIS Write or Read transaction acknowledged	0x1
UIS_RD	EMMC_UIS_RD_NO_REQUEST	Read transaction requested from the PHY register	0x0*
	EMMC_UIS_RD_REQUEST	Read transaction is not requested from the PHY register	0x1
UIS_WR	EMMC_UIS_RW_NO_REQUEST	Write transaction requested from the PHY register	0x0*
	EMMC_UIS_RW_REQUEST	Write transaction is not requested from the PHY register	0x1

18.3.1.5 EMMC_HRS06

Bit Field	Read/Write	Field Name	Description
2:0	RW	EMM	EMM - eMMC Mode select: This field sets eMMC mode. The mode should reflect the eMMC device setting. If the SD card is in use, this field needs to be 0000b.

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
EMM	EMMC_EMM_SD_CARD	SD Card in use	0x0*
	EMMC_EMM_LEGACY	Legacy	0x1
	EMMC_EMM_SDR	SDR Mode	0x2
	EMMC_EMM_DDR	DDR Mode	0x3
	EMMC_EMM_HS200	HS200 Mode	0x4
	EMMC_EMM_HS400	HS400 Mode	0x5

18.3.1.6 EMMC_HRS07

Bit Field	Read/Write	Field Name	Description
20:16	RW	ODELAY_VAL	ODELAY_VAL - Output delay value for IO
4:0	RW	IDELAY_VAL	IDELAY_VAL - Input delay value for IO

18.3.1.7 EMMC_HRS30

Bit Field	Read/Write	Field Name	Description
1	R	HS400ESSUP	High Speed 400 Enhance Strobe Mode supported
0	R	CQSUP	Command Queuing supported This field informs whether Command Queuing is supported (1) or is not supported (0)

Field Name	Value Symbol	Value Description	Hex Value
HS400ESSUP	EMMC_HS400ESSUP_DISABLED	HS400 Enhanced Strobe Mode not supported	0x0*
	EMMC_HS400ESSUP_ENABLED	HS400 Enhanced Strobe Mode supported	0x1
CQSUP	EMMC_CQSUP_DISABLED	Command Queueing not supported	0x0*
	EMMC_CQSUP_ENABLED	Command Queueing supported	0x1

18.3.1.8 EMMC_HRS31

Bit Field	Read/Write	Field Name	Description
27:16	R	HOSTCTRLVER	Host Controller Version Release number of the Host Controller
7:0	R	HOSTFIXVER	Fix Version Number: Number of the fix related to the Host Controller Version

Ezairo 8300 Hardware Reference

18.3.1.9 EMMC_HRS32

Bit Field	Read/Write	Field Name	Description
31	RW	LOAD	LOAD - FSM monitor update request
30:16	W	ADDR	ADDR - FSM address: This field selects which FSM status will be read. All available status machines are listed in the Debug section of the User Guide.
15:0	R	DATA	DATA - FSM status: This register contains the Read FSM status. Before reading it, the user should select the FSM address (ADDR), set the LOAD bit, and wait until hardware clears it.

Field Name	Value Symbol	Value Description	Hex Value
LOAD	EMMC_LOAD_FSM_UPDATE_NOT_REQ	FSM monitor update not requested	0x0*
	EMMC_LOAD_FSM_UPDATE_REQ	FSM monitor update requested	0x1

18.3.1.10 EMMC_HRS33

Bit Field	Read/Write	Field Name	Description
31:0	R	STAT0	STAT0 - Tune status 0 After invoking the UHS-I tuning procedure, each bit of this register represents the status of one tuning step. This field corresponds to tuning steps 31-0. 0 - Step failed 1 - Step passed

18.3.1.11 EMMC_HRS34

Bit Field	Read/Write	Field Name	Description
7:0	R	STAT1	STAT1 - Tune status 1 This field is continuation of STAT0 field. Its value represents status of tuning steps 39-32.

18.3.1.12 EMMC_HRS35

Bit Field	Read/Write	Field Name	Description
31	RW	TFR	TFR - Tune Force Request
21:16	RW	TFV	TFV - Tune Force Value This register contains a value which can update the Tune value, and in consequence, the sample point. The new Tune value becomes valid when the TFR transits from 1 to 0.
5:0	R	TVAL	TVAL - Tune Current Value: This field represents the selected value of UHS-I delay

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
TFR	EMMC_NO_TFR	No tune value update requested from TFV	0x0*
	EMMC_TFR	Tune value update requested from TFV	0x1

18.3.1.13 EMMC_HRS36

Bit Field	Read/Write	Field Name	Description
5	R	BOOT_EDE	Boot Error - End Bit error
4	R	BOOT_EDC	Boot Error - Data CRC error
3	R	BOOT_EDT	Boot Error - Data Timeout error
2	R	BOOT_EAI	Boot Error - Invalid Acknowledge error
1	R	BOOT_EAT	Boot Error - Acknowledge Timeout error
0	R	BOOT_ACT	Boot Active - The BOOT is active and the operation cannot be interfered with by writing to any registers

Field Name	Value Symbol	Value Description	Hex Value
BOOT_EDE	EMMC_BOOT_EDE_NO_ERROR		0x0*
	EMMC_BOOT_EDE_ERROR		0x1
BOOT_EDC	EMMC_BOOT_EDC_NO_ERROR		0x0*
	EMMC_BOOT_EDC_ERROR		0x1
BOOT_EDT	EMMC_BOOT_EDT_NO_ERROR		0x0*
	EMMC_BOOT_EDT_ERROR		0x1
BOOT_EAI	EMMC_BOOT_EAI_NO_ERROR		0x0*
	EMMC_BOOT_EAI_ERROR		0x1
BOOT_EAT	EMMC_BOOT_EAT_NO_ERROR		0x0*
	EMMC_BOOT_EAT_ERROR		0x1
BOOT_ACT	EMMC_BOOT_NOT_ACT		0x0*
	EMMC_BOOT_ACT		0x1

18.3.1.14 EMMC_HRS37

Bit Field	Read/Write	Field Name	Description
5:0	RW	RGB_COEFF_IFM	RGB_COEFF_IFM - Reads block gap coefficient to select interface speed mode

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
RGB_COEFF_IFM	EMMC_RGB_COEFF_IFM_DS	Default Speed Mode (DS)	0x0*
	EMMC_RGB_COEFF_IFM_HS	High Speed Mode (HS)	0x1
	EMMC_RGB_COEFF_IFM_UHS_I_SDR12	UHS-I SDR12 Speed Mode	0x8
	EMMC_RGB_COEFF_IFM_UHS_I_SDR25	UHS-I SDR25 Speed Mode	0x9
	EMMC_RGB_COEFF_IFM_UHS_I_SDR50	UHS-I SDR50 Speed Mode	0xA
	EMMC_RGB_COEFF_IFM_UHS_I_DDR5	UHS-I DDR5 Speed Mode	0xC
	EMMC_RGB_COEFF_IFM_LEGACY	eMMC Legacy	0x20
	EMMC_RGB_COEFF_IFM_SDR	eMMC SDR Speed Mode	0x21
	EMMC_RGB_COEFF_IFM_DDR	eMMC DDR Speed Mode	0x22

18.3.1.15 EMMC_HRS38

Bit Field	Read/Write	Field Name	Description
3:0	RW	RGB_COEFF	Read block gap coefficient

18.3.1.16 EMMC_CRS63

Bit Field	Read/Write	Field Name	Description
31:24	R	CRS63_HWINIT1	HWINIT field
23:16	R	SVN	SVN - Specification Version Number: This field identifies the Host Controller Specification Version.
15:8	R	CRS63_HWINIT0	HWINIT field
7:0	R	ISES	ISES - Interrupt Signal for Each Slot

Field Name	Value Symbol	Value Description	Hex Value
SVN	EMMC_V1	Version 1.00	0x0
	EMMC_V2	Version 2.00	0x1
	EMMC_V3	Version 3.00	0x2
	EMMC_V4	Version 4.00	0x3*

onsemi Confidential
Ezairo 8300 Hardware Reference

18.3.1.17 EMMC_SRS00

Bit Field	Read/Write	Field Name	Description
31:0	RW	SAAR	Auto CMD23 Argument

18.3.1.18 EMMC_SRS01

Bit Field	Read/Write	Field Name	Description
31:16	RW	BCCT	Block Count For Current Transfer
11:0	RW	TBS	Transfer Block Size This field defines block sizes for block data transfers. During a data transfer, Read operations might return an invalid value, and Write operations are ignored.

18.3.1.19 EMMC_SRS02

Bit Field	Read/Write	Field Name	Description
31:0	RW	ARG1	Command Argument 1: This field contains a 32-bits argument for a command issued by SRS03.CIDX File Write.

18.3.1.20 EMMC_SRS03

Bit Field	Read/Write	Field Name	Description
29:24	RW	CIDX	Command Index: This field contains a command number (index) of the command to be sent. The index can be defined in range 00-63, which means all commands (CMD00-CMD63 and ACMD00-ACMD63) defined in the related specifications are supported.
23:22	RW	CT	Command Type: This field defines the specific type of a command.
21	RW	DPS	Data Present Select
20	RW	CICE	Command Index Check Enable: When set to 1, the host checks if the Command Index field in the response is equal to the SRS03.CIDX value. When set to 0, the check is not performed and the Command Index field of the response is ignored.
19	RW	CRCCE	Command CRC Check Enable
17:16	RW	RTS	Response Type Select: Defines the expected response length.
8	RW	RID	Response Interrupt Disable
7	RW	RECE	Response Error Check Enable
6	RW	RECT	Response Type R1/R5: Selects R1 or R5 response type for the response content checker.
5	RW	MSBS	Multi/Single Block data transfer select
4	RW	DTDS	Data Transfer Direction Select Selects direction of data transfer for commands with DPS=1
1	RW	BCE	Block Count Enable

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
CT	EMMC_CT_NORMAL	Normal command type - the default	0x0*
	EMMC_CT_SUSPEND	Not used	0x1
	EMMC_CT_RESUME	Not used	0x2
	EMMC_CT_ABORT	Abort command type - used to stop current Read or Write data transfer	0x3
DPS	EMMC_DPS_NON_TSF	Used for commands that do not transfer data	0x0*
	EMMC_DPS_TSF	Used for Read or Write data transfer commands using DAT line	0x1
CICE	EMMC_CICE_NOCHECK	Host does not check if the Command Index field in the response is equal to the SRS03.CIDX value; Command Index field is ignored	0x0*
	EMMC_CICE_CHECK	Host checks if the Command Index field in the response is equal to the SRS03.CIDX value	0x1
CRCCE	EMMC_CRCCE_NOCHECK	Host does not check if the CRC field of the response is valid; CRC field of the response is ignored	0x0*
	EMMC_CRCCE_CHECK	Host checks if the CRC field of the response is valid	0x1
RTS	EMMC_RTS_NORESP	No response	0x0*
	EMMC_RTS_136RESP	136-bit response	0x1
	EMMC_RTS_48RESP	48-bit response	0x2
	EMMC_RTS_48RESP_BUSY	48-bit response with BUSY	0x3
RID	EMMC_RID_CCINT_EN_OR_DIS	Command Complete Interrupt (SRS12.CC) enabled or disabled, depending on SRS13.CC_SE bit	0x0*
	EMMC_RID_CCINT_DISABLED	Command Complete Interrupt (SRS12.CC) disabled	0x1
RECE	EMMC_RECE_DISABLED	Response error check disabled	0x0*
	EMMC_RECE_ENABLED	Response error check enabled	0x1
RECT	EMMC_RECT_R1	Response type is R1 (SD Memory)	0x0*
	EMMC_RECT_R5	Response type is R5 (SDIO)	0x1
MSBS	EMMC_MSBS_SINGLE_BLOCK	Single-block data transfer selected	0x0*
	EMMC_MSBS_MULTI_BLOCK	Multi-block data transfer selected	0x1

onsemi Confidential
Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DTDS	EMMC_DTDS_WRITE	Data transfer direction = write	0x0*
	EMMC_DTDS_READ	Data transfer direction = read	0x1
BCE	EMMC_BCE_DISABLED	SRS01.BCCT Block Count is disabled	0x0*
	EMMC_BCE_ENABLED	SRS01.BCCT Block Count is enabled	0x1

18.3.1.21 EMMC_SRS04

Bit Field	Read/Write	Field Name	Description
31:0	R	RESP1	Response Register #1 (see short comment)

18.3.1.22 EMMC_SRS05

Bit Field	Read/Write	Field Name	Description
31:0	R	RESP2	Response Register #2 (see RESP1 short comment)

18.3.1.23 EMMC_SRS06

Bit Field	Read/Write	Field Name	Description
31:0	R	RESP3	Response Register #3 (see RESP1 short comment)

18.3.1.24 EMMC_SRS07

Bit Field	Read/Write	Field Name	Description
31:0	R	RESP4	Response Register #4 (see RESP1 short comment)

18.3.1.25 EMMC_SRS08

Bit Field	Read/Write	Field Name	Description
31:0	RW	BDP	Buffer Data Port The field is used to access the internal buffer (data block) in Non-DMA Transfer Mode.

18.3.1.26 EMMC_SRS09

Bit Field	Read/Write	Field Name	Description
24	R	CMDSL	CMD Line Signal Level (SD Mode only)
23:20	R	DATSL1	DAT[3:0] Line Signal Level (SD Mode only)
19	R	WPSL	Write Protect Switch Pin Level The value is equal to the actual signal level on Write Protect pin of the SD interface (sdwp_s#_n)
18	R	CDSL	Card Detect Pin Level This value is equal to the inverted signal level

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
			on the Card Detect pin of the SD interface (sdsl_s#_n)
17	R	CSS	Card State Stable Indicates if Card Detect Pin Level (CDSL) is stable
16	R	CI	Card Inserted Indicates if the card is inserted in the slot
11	R	BRE	Buffer Read Enable This field represents the data buffer (SRS08.BDP) state for Read transfer in Non-DMA Mode.
10	R	BWE	Buffer Write Enable This bit represents the data buffer (SRS08.BDP) state for Write transfer in Non-DMA Mode.
9	R	RTA	Read Transfer Active (SD Mode only) Indicates the status of the Read data transfer
8	R	WTA	Write Transfer Active (SD Mode only) Indicates the status of the Write data transfer.
7:4	R	DATSL2	DAT[7:4] Line Signal Level (SD Mode only)
2	R	DLA	DAT Line Active (SD Mode only) Indicates if the DAT lines of the SD interface are currently in use
1	R	CIDAT	Command Inhibit DAT (SD Mode only): Indicates if the host can issue a command which uses the DAT line
0	R	CICMD	Command Inhibit CMD: Indicates if the host can issue a command

Field Name	Value Symbol	Value Description	Hex Value
WPSL	EMMC_WPSL_WR_DISABLED	Write operation is disabled	0x0*
	EMMC_WPSL_WR_ENABLED	Write operation is enabled	0x1
CDSL	EMMC_CDSL_NO_CARD_IN	Card is not inserted	0x0*
	EMMC_CDSL_CARD_IN	Card is inserted	0x1
CSS	EMMC_CSS_CARD_NOT_STABLE	CDSL is not stable (during card insertion/removal or during the reset)	0x0*
	EMMC_CSS_CARD_STABLE	CDSL value is stable	0x1
CI	EMMC_CI_NO_CARD_IN	No card in slot	0x0*
	EMMC_CI_CARD_IN	Card is inserted in slot	0x1
BRE	EMMC_BRE_NO_DATA_RD	No valid data available to be read from the data buffer	0x0*
	EMMC_BRE_VALID_DATA_RD	Valid data can be read from the data buffer	0x1
BWE	EMMC_BWE_NO_DATA_WR	Data cannot be written to the data buffer	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	EMMC_BWE_VALID_DATA_WR	Data can be written to the data buffer	0x1
RTA	EMMC_RTA_NO_RDTSF	No data Read transfer in progress	0x0*
	EMMC_RTA_RDTSF	Data Read transfer in progress	0x1
WTA	EMMC_RTA_NO_WRTSF	No data Write transfer in progress	0x0*
	EMMC_RTA_WRTSF	Data Write transfer in progress	0x1
DLA	EMMC_DLA_RELEASED	DAT lines are released (not in use)	0x0*
	EMMC_DLA_ACTIVE	DAT lines are active (in use)	0x1
CIDAT	EMMC_CIDAT_DISABLED	Command using DAT line can be sent; Inhibit is disabled	0x0*
	EMMC_CIDAT_ENABLED	Command using DAT line cannot be sent; Inhibit is enabled	0x1
CICMD	EMMC_CICMD_DISABLED	Host can issue a command; Inhibit is disabled	0x0*
	EMMC_CICMD_ENABLED	Host cannot issue a command; Inhibit is enabled	0x1

18.3.1.27 EMMC_SRS10

Bit Field	Read/Write	Field Name	Description
26	RW	WORM	Wakeup Event Enable On SD Card Removal
25	RW	WOIS	Wake-Up Event Enable On Card Inserted
24	RW	WOIQ	Wakeup Event Enable On Card Interrupt
19	RW	IBG	Interrupt At Block Gap (SD Mode only)
18	RW	RWC	Read-Wait Control (SD Mode only)
17	RW	CREQ	Continue Request
16	RW	SBGR	Stop At Block Gap Request
15:13	RW	BVS2	SD Bus Voltage Select for VDD2
12	RW	BP2	SD Bus Power for VDD2
11:9	RW	BVS	SD Bus Voltage Select
8	RW	BP	SD Bus Power for VDD1
7	RW	CDSS	Card Detect Signal Selection The card detection mechanism is based on either s#_sdcn port or register value.
6	RW	CDTL	Card Detect Test Level Designates card insertion status when SRS10.CDSS=1. Bit provided for test purposes.

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
5	RW	EDTW	Extended Data Transfer Width This bit is to enable/disable 8-bit DAT bus width mode
2	RW	HSE	High Speed Enable (SD mode only)
1	RW	DTW	Data Transfer Width (SD mode only)
0	RW	LEDC	LED Control State of this bit directly drives LED port of the host in order to control the external LED diode

Field Name	Value Symbol	Value Description	Hex Value
WORM	EMMC_WORM_DISABLED	Wake-up event via Card Removal assertion on the SRS12.CR register disabled	0x0*
	EMMC_WORM_ENABLED	Wake-up event via Card Removal assertion on the SRS12.CR register enabled	0x1
WOIS	EMMC_WOIS_DISABLED	Wake-up event via Card Insertion assertion in the SRS12.CIN register disabled	0x0*
	EMMC_WOIS_ENABLED	Wake-up event via Card Insertion assertion in the SRS12.CIN register enabled	0x1
WOIQ	EMMC_WOIQ_DISABLED	Wake-up event via Card Interrupt assertion in the SRS12.CINT disabled	0x0*
	EMMC_WOIQ_ENABLED	Wake-up event via Card Interrupt assertion in the SRS12.CINT enabled	0x1
IBG	EMMC_IBG_DISABLED	Interrupt detection at the block gap for a multiple block transfer disabled	0x0*
	EMMC_IBG_ENABLED	Interrupt detection at the block gap for a multiple block transfer enabled	0x1
RWC	EMMC_RWC_DISABLED	Read Wait Control disabled	0x0*
	EMMC_RWC_ENABLED	Read Wait Control enabled	0x1
CREQ	EMMC_CREQ_DISABLED	Transfer previously stopped using the Stop At Block Gap is not restarted	0x0*
	EMMC_CREQ_ENABLED	Transfer previously stopped using the Stop At Block Gap is restarted	0x1
SBGR	EMMC_SBGR_DISABLED	Does not stop execution of Read and Write transactions at next possible block gap	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	EMMC_SBGR_ENABLED	Stops execution of Read and Write transactions at next possible block gap	0x1
BVS2	EMMC_BVS2_NOT_SUPPORTED	VDD2 is not supported	0x0*
	EMMC_BVS2_1P2V_RESERVED	Reserved for 1.2V	0x4
	EMMC_BVS2_1P8V	1.8V (1.8V VDD Support in Capability registers need to be set)	0x5
BP2	EMMC_BP2_VDD2_DISABLED	VDD2 power supply is disabled	0x0*
	EMMC_BP2_VDD2_ENABLED	VDD2 power supply is enabled	0x1
BVS	EMMC_BVS_RESERVED	Reserved	0x0*
	EMMC_BVS_1P8V	1.8V (typical) for embedded	0x5
	EMMC_BVS_3P0V	3.0V (typical)	0x6
	EMMC_BVS_3P3V	3.3V (typical)	0x7
BP	EMMC_BP_VDD1_DISABLED	VDD1 power supply is disabled	0x0*
	EMMC_BP_VDD1_ENABLED	VDD1 power supply is enabled	0x1
CDSS	EMMC_CDSS_NORMAL	Card detection mechanism is based on s#_sdcd_n pin (normal mode)	0x0*
	EMMC_CDSS_TESTING	Card detection mechanism is based on CDTL(SRS10.6) bit (testing mode)	0x1
CDTL	EMMC_CDTL_NO_CARD_INSERTED	Card is not inserted	0x0*
	EMMC_CDTL_CARD_INSERTED	Card is inserted	0x1
EDTW	EMMC_EDTW_DISABLED	8-bit DAT bus width mode is disabled; bus width is selected by SRS10.DTW	0x0*
	EMMC_EDTW_ENABLED	8-bit DAT bus width mode is enabled	0x1
HSE	EMMC_HSE_DISABLED	Operating mode is at Default Speed	0x0*
	EMMC_HSE_ENABLED	Operating mode is at High Speed	0x1
DTW	EMMC_DTW_1BIT	DAT bus width configured to 1-bit mode	0x0*
	EMMC_DTW_4BIT	DAT bus width configured to 4-bit mode	0x1
LEDC	EMMC_LEDC_OFF	External LED diode switched off	0x0*
	EMMC_LEDC_ON	External LED diode switched on	0x1

Ezairo 8300 Hardware Reference

18.3.1.28 EMMC_SRS11

Bit Field	Read/Write	Field Name	Description
26	RW	SRDAT	Software Reset For DAT Line (SD Mode only)
25	RW	SRCMD	Software Reset For CMD Line (SD Mode only)
24	RW	SRFA	Software Reset For All
19:16	RW	DTCV	Data Timeout Counter Value: the interval by which DAT line timeouts are detected
15:8	RW	SDCFSL	SDCLK Frequency Select (lower part): This register and SRS11.SDCFSH are used to calculate the frequency of the SDCLK clock.
7:6	RW	SDCFSH	SDCFSH - SDCLK Frequency Select (higher part): This register is an extension to SDCFSL.
2	RW	SDCE	SD Clock Enable
1	R	ICS	Internal Clock Stable
0	RW	ICE	Internal Clock Enable: This field is designated to control (enable/disable) the external clock generator (e.g. PLL).

Field Name	Value Symbol	Value Description	Hex Value
SRDAT	EMMC_SRDAT_DISABLED	Software reset for DAT line is disabled	0x0*
	EMMC_SRDAT_ENABLED	Software reset for DAT line is enabled; logic related to data path, including data buffers, is reset	0x1
SRCMD	EMMC_SRCMD_DISABLED	Software reset for CMD line is disabled	0x0*
	EMMC_SRCMD_ENABLED	Software reset for CMD line is enabled; logic related to command generation and response checking is reset	0x1
SRFA	EMMC_SRFA_DISABLED	Software reset for All is disabled	0x0*
	EMMC_SRFA_ENABLED	Software reset for All is disabled; entire slot is reset	0x1
DTCV	EMMC_DTCV_30	Data timeout counter value = $t_{sdmclk} \cdot 2^{(13+2)}$	0x0*
	EMMC_DTCV_32	Data timeout counter value = $t_{sdmclk} \cdot 2^{(14+2)}$	0x1
	EMMC_DTCV_34	Data timeout counter value = $t_{sdmclk} \cdot 2^{(15+2)}$	0x2
	EMMC_DTCV_36	Data timeout counter value = $t_{sdmclk} \cdot 2^{(16+2)}$	0x3

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	EMMC_DTCV_38	Data timeout counter value = $t_{sdmclk} \cdot 2 \cdot (17+2)$	0x4
	EMMC_DTCV_40	Data timeout counter value = $t_{sdmclk} \cdot 2 \cdot (18+2)$	0x5
	EMMC_DTCV_42	Data timeout counter value = $t_{sdmclk} \cdot 2 \cdot (19+2)$	0x6
	EMMC_DTCV_44	Data timeout counter value = $t_{sdmclk} \cdot 2 \cdot (20+2)$	0x7
	EMMC_DTCV_46	Data timeout counter value = $t_{sdmclk} \cdot 2 \cdot (21+2)$	0x8
	EMMC_DTCV_48	Data timeout counter value = $t_{sdmclk} \cdot 2 \cdot (22+2)$	0x9
	EMMC_DTCV_50	Data timeout counter value = $t_{sdmclk} \cdot 2 \cdot (23+2)$	0xA
	EMMC_DTCV_52	Data timeout counter value = $t_{sdmclk} \cdot 2 \cdot (24+2)$	0xB
	EMMC_DTCV_54	Data timeout counter value = $t_{sdmclk} \cdot 2 \cdot (25+2)$	0xC
	EMMC_DTCV_56	Data timeout counter value = $t_{sdmclk} \cdot 2 \cdot (26+2)$	0xD
	EMMC_DTCV_58	Data timeout counter value = $t_{sdmclk} \cdot 2 \cdot (27+2)$	0xE
SDCE	EMMC_SDCE_DISABLED	SDCLK clock is stopped	0x0*
	EMMC_SDCE_ENABLED	SDCLK clock is enabled	0x1
ICS	EMMC_ICS_NOT_STABLE	Clock on sdmclk pin of host is not yet stable (e.g.: external PLL that generates clock is not yet locked)	0x0*
	EMMC_ICS_STABLE	Clock on sdmclk pin of host is stable, after setting ICE to 1	0x1
ICE	EMMC_ICE_DISABLED	Internal clock generator is disabled	0x0*
	EMMC_ICE_ENABLED	Internal clock generator is enabled	0x1

Ezairo 8300 Hardware Reference

18.3.1.29 EMMC_SRS12

Bit Field	Read/Write	Field Name	Description
27	RW	ERSP	Response Error (SD Mode only) Generated on error detection in R1 or R5 response
24	RW	EAC	Auto CMD Error (SD mode only) Generated when an error occurs during Auto CMD12/Auto CMD23 command transmission
23	RW	ECL	Current Limit Error
22	RW	EDEB	Data End Bit Error (SD mode only)
21	RW	EDCRC	Data CRC Error (SD mode only)
20	RW	EDT	Data Timeout Error (SD mode only)
19	RW	ECI	Command Index Error (SD mode only)
18	RW	ECEB	Command End Bit Error (SD mode only)
17	RW	ECCRC	Command CRC Error (SD mode only)
16	RW	ECT	Command Timeout Error (SD mode only)
15	R	EINT	Error Interrupt The software can check for an error by reading this single bit first
8	R	CINT	Card Interrupt Indicates the card interrupt.
7	RW	CR	Card Removal Generated when the SRS09.CI bit changes from 1 to 0, indicating a card removal event.
6	RW	CIN	Card Insertion Generated when the SRS09.CI bit changes from 0 to 1, indicating card insertion
5	RW	BRR	Buffer Read Ready
4	RW	BWR	Buffer Write Ready
2	RW	BGE	Block Gap Event Generated when the read/write transaction is stopped at a block gap as the result of setting SRS10.SBGR to 1.
1	RW	TC	Transfer Complete
0	RW	CC	CC - Command Complete Generated when the end bit of the response is received, except the response for an Auto-CMD12 command; this type of command does not generate CC.

Field Name	Value Symbol	Value Description	Hex Value
ERSP	EMMC_ERSP_NO_ERROR	No error detected in R1 or R5 response	0x0*
	EMMC_ERSP_ERROR	Error detected in R1 or R5 response	0x1
EAC	EMMC_EAC_NO_ERROR	No error detected during Auto CMD12/ Auto CMD23 command transmission	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	EMMC_EAC_ERROR	Error detected during Auto CMD12/ Auto CMD23 command transmission	0x1
ECL	EMMC_ECL_NO_ERROR	No current limit error detected	0x0*
	EMMC_ECL_ERROR	Current limit error detected	0x1
EDEB	EMMC_EDEB_NO_ERROR	No Data End Bit error detected	0x0*
	EMMC_EDEB_ERROR	Data End Bit error detected	0x1
EDCRC	EMMC_EDCRC_NO_ERROR	No CRC error detected	0x0*
	EMMC_EDCRC_ERROR	CRC error detected when transferring Read data using the DAT line, or when Write CRC status value is not equal to 010	0x1
EDT	EMMC_EDT_NO_ERROR	No data timeout error detected	0x0*
	EMMC_EDT_ERROR	Data timeout error detected	0x1
ECI	EMMC_ECI_NO_ERROR	No index error detected in command response	0x0*
	EMMC_ECI_ERROR	Index error detected in command response	0x1
ECEB	EMMC_ECEB_NO_ERROR	No command end bit error detected; end bit of command response is not 0	0x0*
	EMMC_ECEB_ERROR	Command end bit error detected; end bit of command response is 0	0x1
ECCRC	EMMC_ECCRC_NO_ERROR	No command CRC error detected	0x0*
	EMMC_ECCRC_ERROR	Command CRC error detected	0x1
ECT	EMMC_ECT_NO_ERROR	No command timeout error detected	0x0*
	EMMC_ECT_ERROR	Command timeout error detected; no response returned within 64 SDCLK cycles from end bit of command	0x1
EINT	EMMC_EINT_NO_ERROR	No error interrupt detected; no bits in range SRS12[31:16] set	0x0*
	EMMC_EINT_ERROR	Error interrupt detected; one or more bits in range SRS12[31:16] set	0x1
CINT	EMCC_NO_CARD_INT	No card interrupt detected	0x0*
	EMCC_CARD_INT	Card interrupt detected	0x1
CR	EMMC_CR_CARD_STABLE	The card state is stable (still inserted or removed), or debouncing is in progress	0x0*
	EMMC_CR_CARD_REMOVED	The card has been removed from the	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		slot	
CIN	EMMC_CIN_CARD_STABLE	The card state is stable (still inserted or removed), or debouncing is in progress	0x0*
	EMMC_CIN_CARD_INSERTED	The card has been inserted in the slot	0x1
BRR	EMMC_BRR_NOT_READY	BRE value is 0; the data buffer cannot be read by the software	0x0*
	EMMC_BRR_READY	BRE value is 1; the data buffer can be read by the software	0x1
BWR	EMMC_BWR_NOT_READY	BWE value is 0; the data buffer cannot be written to by the software	0x0*
	EMMC_BWR_READY	BWE value is 1; the data buffer can be written to by the software	0x1
BGE	EMMC_BGE_NO_EVENT	No block gap event has occurred	0x0*
	EMMC_BGE_EVENT_OCCURRED	Block gap event has occurred	0x1
TC	EMMC_TC_TSF_NOT_COMPLETE	Transfer using the DAT line is not complete	0x0*
	EMMC_TC_TSF_COMPLETE	Transfer using the DAT line is complete	0x1
CC	EMMC_CC_CMD_NOT_COMPLETE	Command is not complete	0x0*
	EMMC_CC_CMD_COMPLETE	Command is complete	0x1

18.3.1.30 EMMC_SRS13

Bit Field	Read/Write	Field Name	Description
27	RW	ERSP_SE	Response Error Status Enable (SD mode only)
24	RW	EAC_SE	Auto CMD Error Status Enable (SD mode only)
23	RW	ECL_SE	Current Limit Error Status Enable (SD mode only)
22	RW	EDEB_SE	Data End Bit Error Status Enable (SD mode only)
21	RW	EDCRC_SE	Data CRC Error Status Enable (SD mode only)
20	RW	EDT_SE	Data Timeout Error Status Enable (SD mode only)
19	RW	ECI_SE	Command Index Error Status Enable (SD mode only)
18	RW	ECEB_SE	Command End Bit Error Status Enable (SD mode only)
17	RW	ECCRC_SE	Command CRC Error Status Enable (SD mode only)
16	RW	ECT_SE	Command Timeout Error Status Enable (SD mode only)
8	RW	CINT_SE	Card Interrupt Status Enable
7	RW	CR_SE	Card Removal Status Enable

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
6	RW	CIN_SE	Card Insertion Status Enable
5	RW	BRR_SE	Buffer Read Ready Status Enable
4	RW	BWR_SE	Buffer Write Ready Status Enable
2	RW	BGE_SE	Block Gap Event Status Enable
1	RW	TC_SE	Transfer Complete Status Enable
0	RW	CC_SE	Command Complete Status Enable

Field Name	Value Symbol	Value Description	Hex Value
ERSP_SE	EMMC_ERSP_SE_MASKED	Response error status masked	0x0*
	EMMC_ERSP_SE_ENABLED	Response error status enabled	0x1
EAC_SE	EMMC_EAC_SE_MASKED	Auto CMD error status masked	0x0*
	EMMC_EAC_SE_ENABLED	Auto CMD error status enabled	0x1
ECL_SE	EMMC_ECL_SE_MASKED	Current limit error status masked	0x0*
	EMMC_ECL_SE_ENABLED	Current limit error status enabled	0x1
EDEB_SE	EMMC_EDEB_SE_MASKED	Data end bit error status masked	0x0*
	EMMC_EDEB_SE_ENABLED	Data end bit error status enabled	0x1
EDCRC_SE	EMMC_EDCRC_SE_MASKED	Data CRC error status masked	0x0*
	EMMC_EDCRC_SE_ENABLED	Data CRC error status enabled	0x1
EDT_SE	EMMC_EDT_SE_MASKED	Data timeout error status masked	0x0*
	EMMC_EDT_SE_ENABLED	Data timeout error status enabled	0x1
ECI_SE	EMMC_ECI_SE_MASKED	Command index error status masked	0x0*
	EMMC_ECI_SE_ENABLED	Command index error status enabled	0x1
ECEB_SE	EMMC_ECEB_SE_MASKED	Command end bit error status masked	0x0*
	EMMC_ECEB_SE_ENABLED	Command end bit error status enabled	0x1
ECCRC_SE	EMMC_ECCRC_SE_MASKED	Command CRC error status masked	0x0*
	EMMC_ECCRC_SE_ENABLED	Command CRC error status enabled	0x1
ECT_SE	EMMC_ECT_SE_MASKED	Command timeout error status masked	0x0*
	EMMC_ECT_SE_ENABLED	Command timeout error status enabled	0x1
CINT_SE	EMMC_CINT_SE_MASKED	Card interrupt status masked	0x0*
	EMMC_CINT_SE_ENABLED	Card interrupt status enabled	0x1
CR_SE	EMMC_CR_SE_MASKED	Card removal status masked	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	EMMC_CR_SE_ENABLED	Card removal status enabled	0x1
CIN_SE	EMMC_CIN_SE_MASKED	Card insertion status masked	0x0*
	EMMC_CIN_SE_ENABLED	Card insertion status enabled	0x1
BRR_SE	EMMC_BRR_SE_MASKED	Buffer Read ready status masked	0x0*
	EMMC_BRR_SE_ENABLED	Buffer Read ready status enabled	0x1
BWR_SE	EMMC_BWR_SE_MASKED	Buffer Write ready status masked	0x0*
	EMMC_BWR_SE_ENABLED	Buffer Write ready status enabled	0x1
BGE_SE	EMMC_BGE_SE_MASKED	Block gap event status masked	0x0*
	EMMC_BGE_SE_ENABLED	Block gap event status enabled	0x1
TC_SE	EMMC_TC_SE_MASKED	Transfer complete status masked	0x0*
	EMMC_TC_SE_ENABLED	Transfer complete status enabled	0x1
CC_SE	EMMC_CC_SE_MASKED	Command complete status masked	0x0*
	EMMC_CC_SE_ENABLED	Command complete status enabled	0x1

18.3.1.31 EMMC_SRS14

Bit Field	Read/Write	Field Name	Description
27	RW	ERSP_IE	Response Error Interrupt Enable
24	RW	EAC_IE	Auto CMD Error Interrupt Enable (SD mode only)
23	RW	ECL_IE	Current Limit Error Interrupt Enable (SD mode only)
22	RW	EDEB_IE	Data End Bit Error Interrupt Enable (SD mode only)
21	RW	EDCRC_IE	Data CRC Error Interrupt Enable (SD mode only)
20	RW	EDT_IE	Data Timeout Error Interrupt Enable (SD mode only)
19	RW	ECI_IE	Command Index Error Interrupt Enable (SD mode only)
18	RW	ECEB_IE	Command End Bit Error Interrupt Enable (SD mode only)
17	RW	ECCRC_IE	Command CRC Error Interrupt Enable (SD mode only)
16	RW	ECT_IE	Command Timeout Error Interrupt Enable (SD mode only)
8	RW	CINT_IE	Card Interrupt - Interrupt Enable
7	RW	CR_IE	Card Removal Interrupt Enable
6	RW	CIN_IE	Card Insertion Interrupt Enable
5	RW	BRR_IE	Buffer Read Ready Interrupt Enable
4	RW	BWR_IE	Buffer Write Ready Interrupt Enable

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
2	RW	BGE_IE	Block Gap Event Interrupt Enable
1	RW	TC_IE	Transfer Complete Interrupt Enable
0	RW	CC_IE	Command Complete Interrupt Enable

Field Name	Value Symbol	Value Description	Hex Value
ERSP_IE	EMMC_ERSP_IE_MASKED	Response error interrupt masked	0x0*
	EMMC_ERSP_IE_ENABLED	Response error interrupt enabled	0x1
EAC_IE	EMMC_EAC_IE_MASKED	Auto CMD error interrupt masked	0x0*
	EMMC_EAC_IE_ENABLED	Auto CMD error interrupt enabled	0x1
ECL_IE	EMMC_ECL_IE_MASKED	Current limit error interrupt masked	0x0*
	EMMC_ECL_IE_ENABLED	Current limit error interrupt enabled	0x1
EDEB_IE	EMMC_EDEB_IE_MASKED	Data end bit error interrupt masked	0x0*
	EMMC_EDEB_IE_ENABLED	Data end bit error interrupt enabled	0x1
EDCRC_IE	EMMC_EDCRC_IE_MASKED	Data CRC error interrupt masked	0x0*
	EMMC_EDCRC_IE_ENABLED	Data CRC error interrupt enabled	0x1
EDT_IE	EMMC_EDT_IE_MASKED	Data timeout error interrupt masked	0x0*
	EMMC_EDT_IE_ENABLED	Data timeout error interrupt enabled	0x1
ECI_IE	EMMC_ECI_IE_MASKED	Command index error interrupt masked	0x0*
	EMMC_ECI_IE_ENABLED	Command index error interrupt enabled	0x1
ECEB_IE	EMMC_ECEB_IE_MASKED	Command end bit error interrupt masked	0x0*
	EMMC_ECEB_IE_ENABLED	Command end bit error interrupt enabled	0x1
ECCRC_IE	EMMC_ECCRC_IE_MASKED	Command CRC error interrupt masked	0x0*
	EMMC_ECCRC_IE_ENABLED	Command CRC error interrupt enabled	0x1
ECT_IE	EMMC_ECT_IE_MASKED	Command timeout error interrupt masked	0x0*
	EMMC_ECT_IE_ENABLED	Command timeout error interrupt enabled	0x1
CINT_IE	EMMC_CINT_IE_MASKED	Card interrupt interrupt masked	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	EMMC_CINT_IE_ENABLED	Card interrupt interrupt enabled	0x1
CR_IE	EMMC_CR_IE_MASKED	Card removal interrupt masked	0x0*
	EMMC_CR_IE_ENABLED	Card removal interrupt enabled	0x1
CIN_IE	EMMC_CIN_IE_MASKED	Card insertion interrupt masked	0x0*
	EMMC_CIN_IE_ENABLED	Card insertion interrupt enabled	0x1
BRR_IE	EMMC_BRR_IE_MASKED	Buffer Read ready interrupt masked	0x0*
	EMMC_BRR_IE_ENABLED	Buffer Read ready interrupt enabled	0x1
BWR_IE	EMMC_BWR_IE_MASKED	Buffer Write ready interrupt masked	0x0*
	EMMC_BWR_IE_ENABLED	Buffer Write ready interrupt enabled	0x1
BGE_IE	EMMC_BGE_IE_MASKED	Block gap event interrupt masked	0x0*
	EMMC_BGE_IE_ENABLED	Block gap event interrupt enabled	0x1
TC_IE	EMMC_TC_IE_MASKED	Transfer complete interrupt masked	0x0*
	EMMC_TC_IE_ENABLED	Transfer complete interrupt enabled	0x1
CC_IE	EMMC_CC_IE_MASKED	Command complete interrupt masked	0x0*
	EMMC_CC_IE_ENABLED	Command complete interrupt enabled	0x1

18.3.1.32 EMMC_SRS15

Bit Field	Read/Write	Field Name	Description
31	RW	PVE	Preset Value Enable
28	RW	HV4E	Host Version 4.00 Enable: Selects backward (SD Host 3.00 Version) compatibility mode or SD Host 4.00 Version mode.
23	RW	SCS	Sampling Clock Select (UHS-I only) The host updates this bit when the tuning procedure is finished
22	RW	EXTNG	Execute Tuning (UHS-I only) This register controls the tuning procedure.
21:20	RW	DSS	Driver Strength Select (UHS-I only) This bit controls the electric parameters of the I/O driver. Up to 4 configurations of I/O driver settings can be implemented.
19	RW	V18SE	1.8 V Signaling Enable (UHS-I only) This bit controls the I/O signaling voltage level. The SW driver will set this bit to 1 when in UHS-I Mode.
7	R	CNIACE	Command Not Issued By Auto CMD12 Error
5	R	ACRE	Auto CMD Response Error

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
4	R	ACIE	Auto CMD Index Error
3	R	ACEBE	Auto CMD End Bit Error
2	R	ACCE	Auto CMD CRC Error
1	R	ACTE	Auto CMD Timeout Error
0	R	ACNE	Auto CMD12 Not Executed: When set to 1, means the host cannot issue Auto CMD12 due to some error.

Field Name	Value Symbol	Value Description	Hex Value
PVE	EMMC_PVE_DISABLED	Preset value update disabled	0x0*
	EMMC_PVE_ENABLED	Preset value update enabled	0x1
HV4E	EMMC_HV4E_VERSION3	SD Host version 3.00 (backward compatibility) mode is set	0x0*
	EMMC_HV4E_VERSION4	SD Host version 4.00 mode is set	0x1
SCS	EMMC_SCS_FAILED	Tuning procedure has failed; clock tuning logic is disabled	0x0*
	EMMC_SCS_FINISHED	Tuning procedure has completed successfully	0x1
EXTNG	EMMC_EXTNG_NOT_IN_PROGRESS	Tuning procedure is not in progress	0x0*
	EMMC_EXTNG_IN_PROGRESS	Tuning procedure is in progress	0x1
DSS	EMMC_DSS_TYPE_B	I/O driver type B implemented	0x0*
	EMMC_DSS_TYPE_A	I/O driver type A implemented	0x1
	EMMC_DSS_TYPE_C	I/O driver type C implemented	0x2
	EMMC_DSS_TYPE_D	I/O driver type D implemented	0x3
V18SE	EMMC_V18SE_3POINT3	I/O uses 3.3 V signaling; also Default Speed or High Speed Mode (depending on selected SD interface mode)	0x0*
	EMMC_V18SE_1POINT8	I/O uses 1.8 V signaling; also UHS-I Mode (depending on selected SD interface mode)	0x1
CNIACE	EMMC_CNIACE_CMD23_ERROR	Auto CMD23 Error is detected (any of bits SRS15[4:1] is set).	0x0*
	EMMC_CNIACE_CMD12_ERROR	Command not executed by host due to previous Auto CMD12 error	0x1
ACRE	EMMC_ACRE_NO_ERROR	No error detected in response to Auto	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		Command	
	EMMC_ACRE_ERROR	Error has been detected in response to Auto Command	0x1
ACIE	EMMC_ACIE_NO_ERROR	No Command Index error has occurred in the Auto CMD response	0x0*
	EMMC_ACIE_ERROR	Command Index error has occurred in the Auto CMD response	0x1
ACEBE	EMMC_ACEBE_NO_ERROR	No error detected; end bit of Auto CMD response is not 0	0x0*
	EMMC_ACEBE_ERROR	Error detected; end bit of Auto CMD response is 0	0x1
ACCE	EMMC_ACCE_NO_ERROR	No CRC error detected in the Auto CMD response	0x0*
	EMMC_ACCE_ERROR	CRC error detected in the Auto CMD response, or conflict detected on CMD lines (see Additional Comments for specifics)	0x1
ACTE	EMMC_ACTE_NO_ERROR	No Auto CMD timeout error is detected	0x0*
	EMMC_ACTE_ERROR	Auto CMD timeout error has occurred: there has been no response within 64 SDCLK clock cycles from the end bit of the Auto CMD, or conflict detected on the CMD lines (see Additional Comments for specifics)	0x1
ACNE	EMMC_ACNE_NO_ERROR	No error detected in Auto CMD12 execution	0x0*
	EMMC_ACNE_ERROR	Host cannot issue an Auto CMD12 command due to some type of error	0x1

18.3.1.33 EMMC_SRS16

Bit Field	Read/Write	Field Name	Description
31:30	R	SLT	Type of Slot Provided
29	R	AIS	Asynchronous Interrupt Support
28	R	A64S	64-bit System Addressing Support
27	R	SRS16_HWINIT1	HWINIT field
26	R	VS18	Voltage Support 1.8 V

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
25	R	VS30	Voltage Support 3.0 V
24	R	VS33	Voltage Support 3.3 V
23	R	SRS	Suspend / Resume Support NOTE: The host controller does not support Suspend / Resume mechanism.
21	R	HSS	High Speed Support
18	R	EDS8	8-bit Embedded Device Support If this bit is 0, the SRS10.EDTW register is not implemented.
17:16	R	SRS16_MBL	Max Block Length This value indicates the maximum block size that can be transferred by the host
15:8	R	BCSDCLK	Base Clock Frequency for SD Clock Field defines the base clock frequency for the SD Clock in 1MHz units. If BCSDCLK = 0, the Host System has to obtain the clock information via another method (i.e. not defined by the specification).
7	R	TCU	Timeout Clock Unit Field defines the frequency unit for the SRS16.TCF
6	R	SRS16_HWINIT0	HWINIT field
5:0	R	TCF	Timeout Clock Frequency Defines the base clock frequency used to detect Data Timeout errors. If TCF = 0, the Host System has to obtain the clock information via another method (i.e. not defined by the specification).

Field Name	Value Symbol	Value Description	Hex Value
SLT	EMMC_SLT_REMOVABLE	Removable Card Slot	0x0
	EMMC_SLT_EMBEDDED	Embedded Slot for One Device	0x1*
	EMMC_SLT_SHARED	Shared Bus Slot	0x2
AIS	EMMC_AIS_NO_SUPPORT	Asynchronous interrupt is not supported	0x0*
	EMMC_AIS_SUPPORT	Asynchronous interrupt is supported	0x1
A64S	EMMC_A64S_NO_SUPPORT	64-bit system addressing is not supported	0x0
	EMMC_A64S_SUPPORT	64-bit system addressing is supported	0x1*
VS18	EMMC_VS18_NO_SUPPORT	1.8 V voltage is not supported	0x0
	EMMC_VS18_SUPPORT	1.8 V voltage is supported	0x1*
VS30	EMMC_VS30_NO_SUPPORT	3.0 V voltage is not supported	0x0*
	EMMC_VS30_SUPPORT	3.0 V voltage is supported	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
VS33	EMMC_VS33_NO_SUPPORT	3.3 V voltage is not supported	0x0*
	EMMC_VS33_SUPPORT	3.3 V voltage is supported	0x1
SRS	EMMC_VSRS_NO_SUPPORT	Suspend / resume is not supported	0x0*
	EMMC_VSRS_SUPPORT	Suspend / resume is supported	0x1
HSS	EMMC_HSS_NO_SUPPORT	High speed is not supported	0x0
	EMMC_HSS_SUPPORT	High speed is supported	0x1*
EDS8	EMMC_8EDS_NO_SUPPORT	8-bit embedded device is not supported	0x0*
	EMMC_8EDS_SUPPORT	8-bit embedded device is supported	0x1
SRS16_MBL	EMMC_MBL_512	512 bytes	0x0
	EMMC_MBL_1024	1024 bytes	0x1
	EMMC_MBL_2048	2048 bytes	0x2*
TCU	EMMC_TCU_KHZ	Timeout clock frequency unit is KHz	0x0
	EMMC_TCU_MHZ	Timeout clock frequency unit is MHz	0x1*

18.3.1.34 EMMC_SRS17

Bit Field	Read/Write	Field Name	Description
31:29	R	SRS17_HWINIT3	HWINIT field
28	R	VDD2S	VDD2 Supported Information about support or lack of support of optional VDD2
27:24	R	SRS17_HWINIT2	HWINIT field
23:16	R	CLKMPR	Clock Multiplier: This field is to be 0 (fixed), as the Clock Multiplier is not supported.
15:14	R	RTNGM	Re-Tuning Modes
13	R	UTSM50	Use Tuning for SDR50
12	R	SRS17_HWINIT1	HWINIT field
7	R	SRS17_HWINIT0	HWINIT field
6	R	DRVD	1.8 V Line Driver Type D Support
5	R	DRVC	1.8 V Line Driver Type C Support
4	R	DRVA	1.8 V Line Driver Type A Support
3	R	UHSII	UHS-II Support

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
2	R	DDR50	DDR50 Support
1	R	SDR104	SDR104 Support
0	R	SDR50	SDR50 - SDR50 Supported: 1 - SDR50 mode supported; 0 - SDR50 mode not supported

Field Name	Value Symbol	Value Description	Hex Value
VDD2S	EMMC_VDD2S_NO_SUPPORT		0x0*
	EMMC_VDD2S_SUPPORT		0x1
RTNGM	EMMC_RTNGM_MODE1	Retuning Mode 1 is used. (See Additional Comments for more information.)	0x0*
	EMMC_RTNGM_MODE2	Retuning Mode 2 is used. (See Additional Comments for more information.)	0x1
UTSM50	EMMC_UTSM50_NO_TUNING_NEEDED	Tuning operation is not necessary in SDR50 Mode	0x0*
	EMMC_UTSM50_TUNING_NEEDED	Tuning operation is necessary in SDR50 Mode	0x1
DRVD	EMMC_DRVD_NO_TYPE_D	Driver Type D is not supported	0x0
	EMMC_DRVD_TYPE_D	Driver Type D is supported	0x1*
DRVC	EMMC_DRVC_NO_TYPE_C	Driver Type C is not supported	0x0
	EMMC_DRVC_TYPE_C	Driver Type C is supported	0x1*
DRVA	EMMC_DRVA_NO_TYPE_A	Driver Type A is not supported	0x0
	EMMC_DRVA_TYPE_A	Driver Type A is supported	0x1*
UHSII	EMMC_UHSII_NO_SUPPORT	UHS-II is not supported	0x0*
	EMMC_UHSII_SUPPORT	UHS-II is supported	0x1
DDR50	EMMC_DDR50_NO_SUPPORT	DDR50 Mode is not supported	0x0*
	EMMC_DDR50_SUPPORT	DDR50 Mode is supported	0x1
SDR104	EMMC_SDR104_NO_SUPPORT	SDR104 Mode is not supported	0x0*
	EMMC_SDR104_SUPPORT	SDR104 Mode is supported	0x1
SDR50	EMMC_SDR50_NO_SUPPORT	SDR50 Mode is not supported	0x0*
	EMMC_SDR50_SUPPORT	SDR50 Mode is supported	0x1

onsemi Confidential

Ezairo 8300 Hardware Reference

18.3.1.35 EMMC_SRS18

Bit Field	Read/Write	Field Name	Description
31:24	R	SRS18_HWINIT0	HWINIT field
23:16	R	MC18	Maximum Current for 1.8 V
15:8	R	MC30	Maximum Current for 3.0 V
7:0	R	MC33	Maximum Current for 3.3 V

18.3.1.36 EMMC_SRS19

Bit Field	Read/Write	Field Name	Description
31:8	R	SRS19_HWINIT0	HWINIT field
7:0	R	MC18V2	Maximum Current for 1.8V VDD2

18.3.1.37 EMMC_SRS20

Bit Field	Read/Write	Field Name	Description
27	W	ERESP_FE	Force Response Error Event
26	W	ETUNE_FE	Force Tuning Error Event
24	W	EAC_FE	Force Auto CMD Error Event
23	W	ECL_FE	Force Current Limit Error Event
22	W	EDEB_FE	Force Data End Bit Error Event
21	W	EDCRC_FE	Force Data CRC Error Event
20	W	EDT_FE	Force Data Timeout Error Event
19	W	ECI_FE	Force Command Index Error Event
18	W	ECEB_FE	Force Command End Bit Error Event
17	W	ECCRC_FE	Force Command CRC Error Event
16	W	ECT_FE	Force Command Timeout Error Event
7	W	CNIACE_FE	Force Command Not Issued By Auto CMD12 Error Event
4	W	ACIE_FE	Force Auto CMD Index Error Event
3	W	ACEBE_FE	Force Auto CMD End Bit Error Event
2	W	ACCE_FE	Force Auto CMD CRC Error Event
1	W	ACTE_FE	Force Auto CMD Timeout Error Event
0	W	ACNE_FE	Force Auto CMD12 Not Executed Error Event

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
ERESP_FE	EMMC_ERESP_FE_ERROR	Force response error event has occurred	0x1
ETUNE_FE	EMMC_ETUNE_FE_ERROR	Force tuning error event has occurred	0x1
EAC_FE	EMMC_EAC_FE_ERROR	Force Auto CMD error event has occurred	0x1
ECL_FE	EMMC_ECL_FE_ERROR	Force current limit error event has occurred	0x1
EDEB_FE	EMMC_EDEB_FE_ERROR	Force data end bit error event has occurred	0x1
EDCRC_FE	EMMC_EDCRC_FE_ERROR	Force data CRC error event has occurred	0x1
EDT_FE	EMMC_EDT_FE_ERROR	Force data timeout error event has occurred	0x1
ECI_FE	EMMC_ECI_FE_ERROR	Force command index error event has occurred	0x1
ECEB_FE	EMMC_ECEB_FE_ERROR	Force command end bit error event has occurred	0x1
ECCRC_FE	EMMC_ECCRC_FE_ERROR	Force command CRC error event has occurred	0x1
ECT_FE	EMMC_ECT_FE_ERROR	Force command timeout error event has occurred	0x1
CNIACE_FE	EMMC_CNIACE_FE_ERROR	Force command not issued by Auto CMD12 error event has occurred	0x1
ACIE_FE	EMMC_ACIE_FE_ERROR	Force Auto CMD index error event has occurred	0x1
ACEBE_FE	EMMC_ACEBE_FE_ERROR	Force Auto CMD end bit error event has occurred	0x1
ACCE_FE	EMMC_ACCE_FE_ERROR	Force Auto CMD CRC error event has occurred	0x1
ACTE_FE	EMMC_ACTE_FE_ERROR	Force Auto CMD timeout error event has occurred	0x1
ACNE_FE	EMMC_ACNE_FE_ERROR	Force Auto CMD12 not executed error event has occurred	0x1

Ezairo 8300 Hardware Reference

18.3.1.38 EMMC_SRS24

Bit Field	Read/Write	Field Name	Description
31:30	R	SRS24_DSSPV_31_30	DSSPV## - Driver Strength Select - Preset Value: This field can be used by the software to update SRS15.DSS
29:26	R	SRS24_HWINIT1	HWINIT field
25:16	R	SRS24_SDCFSPV_25_16	SDCFSPV## - SDCLK Clock Frequency Select - Preset Value: This field can be used by the software to update SRS11.SDCFSH and SRS11.SDCFSL
15:0	R	SRS24_HWINIT0	HWINIT field

18.3.1.39 EMMC_SRS25

Bit Field	Read/Write	Field Name	Description
31:30	R	SRS25_DSSPV_31_30	DSSPV## - Driver Strength Select - Preset Value: This field can be used by the software to update SRS15.DSS
29:26	R	SRS25_HWINIT1	HWINIT field
25:16	R	SRS25_SDCFSPV_25_16	SDCFSPV## - SDCLK Clock Frequency Select - Preset Value: This field can be used by the software to update SRS11.SDCFSH and SRS11.SDCFSL
15:14	R	SRS25_DSSPV_15_14	DSSPV## - Driver Strength Select - Preset Value: This field can be used by the software to update SRS15.DSS.
13:10	R	SRS25_HWINIT0	HWINIT field
9:0	R	SRS25_SDCFSPV_09_00	SDCFSPV## - SDCLK Clock Frequency Select - Preset Value: This field can be used by the software to update SRS11.SDCFSH and SRS11.SDCFSL

18.3.1.40 EMMC_SRS26

Bit Field	Read/Write	Field Name	Description
31:30	R	SRS26_DSSPV_31_30	DSSPV## - Driver Strength Select - Preset Value: This field can be used by the software to update SRS15.DSS.
29:26	R	SRS26_HWINIT1	HWINIT field
25:16	R	SRS26_SDCFSPV_25_16	SDCFSPV## - SDCLK Clock Frequency Select - Preset Value: This field can be used by the software to update SRS11.SDCFSH and SRS11.SDCFSL.
15:14	R	SRS26_DSSPV_15_14	DSSPV## - Driver Strength Select - Preset Value: This field can be used by the software to update SRS15.DSS.

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
13:11	R	SRS26_HWINIT0	HWINIT field
10	R	SRS26_CGSPV_10	CGSPV## - Clock Generator Select - Preset Value: This field can be used by the software to update SRS11.CGS.
9:0	R	SRS26_SDCFSPV_09_00	SDCFSPV## - SDCLK Clock Frequency Select - Preset Value: This field can be used by the software to update SRS11.SDCFSL and SRS11.SDCFSL.

18.3.1.41 EMMC_SRS27

Bit Field	Read/Write	Field Name	Description
31:30	R	SRS27_DSSPV_31_30	DSSPV## - Driver Strength Select - Preset Value: This field can be used by the software to update SRS15.DSS.
29:26	R	SRS27_HWINIT1	HWINIT field
25:16	R	SRS27_SDCFSPV_25_16	SDCFSPV## - SDCLK Clock Frequency Select - Preset Value: This field can be used by the software to update SRS11.SDCFSL and SRS11.SDCFSL.
15:14	R	SRS27_DSSPV_15_14	DSSPV## - Driver Strength Select - Preset Value: This field can be used by the software to update SRS15.DSS.
13:10	R	SRS27_HWINIT0	HWINIT field
9:0	R	SRS27_SDCFSPV_09_00	SDCFSPV## - SDCLK Clock Frequency Select - Preset Value: This field can be used by the software to update SRS11.SDCFSL and SRS11.SDCFSL.

18.3.1.42 EMMC_SRS29

Bit Field	Read/Write	Field Name	Description
31:16	R	SRS29_HWINIT1	HWINIT field
15:14	R	SRS29_DSSPV_15_14	DSSPV## - Driver Strength Select - Preset Value: This field can be used by the software to update SRS15.DSS.
13:10	R	SRS29_HWINIT0	HWINIT field
9:0	R	SRS29_SDCFSPV_09_00	SDCFSPV## - SDCLK Clock Frequency Select - Preset Value: This field can be used by the software to update SRS11.SDCFSL and SRS11.SDCFSL.

18.4 I²C INTERFACES

The I²C interfaces are compatible with the *Inter-IC Bus Specification* from NXP Semiconductors. The I²C interfaces use a two-wire interface, which includes a bidirectional clock line (SCL) and a bidirectional data line (SDA). These interfaces are typically used for communication with external sensors and storage devices, and as control signals for wireless radios. The I²C interfaces support both master and slave mode operation. The I²C

Ezairo 8300 Hardware Reference

interfaces can be configured to connect to the DMA controller through a DMA channel, to support moving data without processor intervention. There are three interfaces on Ezairo 8300—I2C0, I2C1, and D_I2C—and the description that follows applies to all of them unless otherwise indicated.

NOTE: Each I²C interface has its own set of registers. In the text below, a reference to an I2C*_XXX register (ex., I2C*_CFG) refers to the I2C0, I2C1, and D_I2C versions of this register.

The I²C debug ports for the CFX DSP and Arm Cortex-M3 processor share fixed pads and an I²C bus with the D_I2C interface instance. For information about configuring the DIOs to create an I²C bus that is needed to support the I2C0 or I2C1 interface, see [Chapter 16.1 "Overview" on page 463](#).

IMPORTANT: I²C interfaces are defined using open-collector pads for the I²C bus. Because these pads do not drive a high signal for the bus signals—relying on the bus's pull-up resistors for proper operation—user applications working with this interface must use either the internal pull-up resistors implemented for the DIO pads, or a set of external pull-up resistors on the DIOs assigned to be the I²C bus lines.

The I²C interface uses the rising edges of a serial clock signal (SCL) to clock in data from a serial data signal (SDA) to communicate between devices. The I²C interface is designed to handle bus traffic operating at up to 1 MHz.

The example timing diagram shown in the ["I2C Signal Timing Diagram" figure \(Figure 34\)](#) provides some information about the important elements in an I²C transaction, which are described in further detail in the previously mentioned bus specification. These elements are:

Start Condition

The SDA transitions from the idle high state to the low state while the SCL remains high. This can also happen during a transmission as a repeated start condition, indicating that the transaction is starting again without an intermediate stop condition.

Address Bits

During the first byte transmitted, the first seven bits on the SDA (clocked in on a rising edge of SCL) provide the address of the device with which the master device wants to communicate. A device that is properly addressed needs to acknowledge this transaction if communications are to conform to the I²C standard.

Read/Write Direction Bit

During the first byte transmitted, the eighth bit on the SDA (clocked in on a rising edge of SCL) indicates the direction of the transaction. A zero for this bit indicates a write by the master; a one indicates a read by the master. If the master device has requested a read, the slave device controls the SDA line on subsequent bytes to transmit data.

Data Byte

All other bytes, excluding the addressing and read/write byte, that are transmitted on the SDA are considered data bytes for the transaction.

Acknowledge or Not Acknowledge Bit

Ezairo 8300 Hardware Reference

The acknowledge bit is used to indicate to the sender that the byte has been received. The device receiving data needs to acknowledge each data byte (ACK), including the address byte. During this time, the bus device that is sending data on the data bus stops driving the SDA and allows the line to be pulled high. To not acknowledge a byte (NACK), the receiving device does not respond. To ACK, the receiving device needs to pull the SDA low.

A receiving slave device needs to NACK if the slave device is not the device that has been addressed, or if the device cannot handle the byte received. A master device needs to NACK if the master is receiving and wants to end the transaction. If a NACK is encountered, the device transmitting data needs to generate a stop condition.

Stop Condition

The SDA transitions from a low state to high state while the SCL remains high. This ends an I²C transaction.

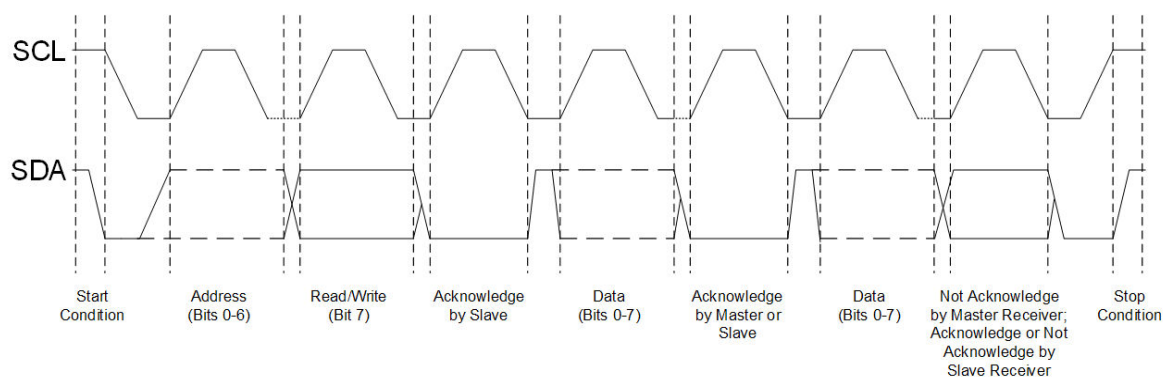


Figure 34. I²C Signal Timing Diagram

The I2C*_CTRL registers are the interface control registers. They contain write-only control bits, and read-only status bits to indicate the status of key control settings. Each of the write-only bits is used to trigger an event. Information about these bits is summarized in the "I2C Control Bits" table (Table 56), below.

Table 56. I²C Control Bits

Bit Field	Field Name	Field Description
9	I2C_CTRL_LAST_DATA_STATUS	Read-only bit indicating if the current byte is the last data byte for a transfer
8	I2C_CTRL_ENABLE_STATUS	Read-only bit indicating that I2C interface is enabled or disabled
7	D_I2C_ERROR_CNT_CLEAR	Clear the D_I2C_ERROR_CNT register. Available only on D_I2C interface.
6	I2C_CTRL_LAST_DATA	Indicate that the current data is the last byte
5	I2C_CTRL_STOP	Issue a stop condition on the I ² C interface
4	I2C_CTRL_NACK	Issue a negative acknowledgment of a sent/received byte.

Ezairo 8300 Hardware Reference

I2C Control Bits (Continued)

Bit Field	Field Name	Field Description
3	I2C_CTRL_ACK/I2C_CTRL_RESUME	Issue an acknowledgment of a send/received byte. Resume I2C internal state machine
2	I2C_CTRL_RESET	Reset the I2C interface
1	I2C_CTRL_DISABLE	Disable the I2C interface
0	I2C_CTRL_ENABLE	Enable the I2C interface

NOTE: The I2C_CTRL_ENABLE_STATUS, I2C_CTRL_DISABLE, and I2C_CTRL_ENABLE bits are not available on the D_I2C interface.

The I2C*_STATUS registers contain bits that are defined as read-only event bits, state bits, or write-only control bits used for updating the state of the I²C interface. The event bits are used to indicate that an event has occurred, and the state bits are used to indicate an interface operating state. Information about these status bits is shown in the "I2C Event and State Status Bits" table (Table 57), below.

Table 57. I²C Event and State Status Bits

Bit Field	Field Name	Bit Type	Field Description
26	I2C_STATUS_STOP_OR_REPEATED_START_DETECTED	Event	Indicate if a stop or repeated start has been detected
25	I2C_STATUS_REPEATED_START_DETECTED	Event	Indicate if an I2C repeated start has been detected during an active transaction in slave mode
24	I2C_STATUS_DMA_REQ	State	Indicate if the I2C interface is currently requesting DMA data.
23	D_I2C_STATUS_BUS_OR_WATCHDOG_ERROR	Event	Bus or watchdog error status bit
22	I2C_STATUS_BUS_ERROR	State	Indicate that a bus error or conflict occurred during transmission
21	I2C_STATUS_BUSY	Event	Indicate that the reception or transmission of the data is ongoing
20	I2C_STATUS_START_PENDING	Event	Master frame start pending status bit
19	I2C_STATUS_MASTER_MODE	Event	Master mode status bit.
18	I2C_STATUS_STOP_DETECTED	Event	Indicate if an I2C stop bit has been detected
17	I2C_STATUS_DATA_EVENT	Event	Indicate that I2C interface either needs data to transmit or has received data
16	I2C_STATUS_TX_REQ	State	Indicate that a TX data can be written
15	I2C_STATUS_RX_REQ	State	Indicate that an RX data byte is ready to be read

Ezairo 8300 Hardware Reference

I2C Event and State Status Bits (Continued)

Bit Field	Field Name	Bit Type	Field Description
14	I2C_STATUS_CLK_STRETCH	State	Indicate that the I2C clock (SCL) is being stretched
13	I2C_STATUS_LINE_FREE	State	Indicate that the I2C bus is free
12	I2C_STATUS_ADDR_DATA	State	Indicate if the information sent on the bus is an address or data byte
11	I2C_STATUS_READ_WRITE	State	Indicate if the current I2C transaction is a read or a write
10	I2C_STATUS_GEN_CALL	State	Indicate if the current I ² C transaction used the general call address or the interface's regular address
9	I2C_STATUS_ACK	State	Indicate if the data was acknowledged (ACK) or not-acknowledged (NACK)
8	I2C_STATUS_OVERRUN	State	Indicate that an overrun has occurred when receiving data
4	I2C_STATUS_TX_REQ_SET	Control	Write 1 to set TX_REQ status flag, flushing the current contents of the transmit buffers
3	I2C_STATUS_REPEATED_START_DETECTED_CLEAR	Control	Write 1 to clear REPEATED_START_DETECTED status flag
2	I2C_STATUS_STOP_DETECTED_CLEAR	Control	Write 1 to clear STOP_DETECTED status flag
1	I2C_STATUS_BUS_ERROR_CLEAR	Control	Write 1 to clear BUS_ERROR status flag
0	I2C_STATUS_OVERRUN_CLEAR	Control	Write 1 to clear OVERRUN status flag

NOTE: The interfaces support both master and slave operations, which can be simultaneously enabled. A user application can use the interface in both modes, provided the application ensures that the I²C bus is not currently in use. Check the I2C_STATUS_LINE_FREE bit in the I2C*_STATUS registers, to verify that the I²C bus is not currently in use, when attempting to initialize a transfer as a master device.

NOTE: D_I2C_STATUS_BUS_OR_WATCHDOG_ERROR is available only on the D_I2C interface.

IMPORTANT: When multiple masters co-exist on the bus, an I2C*_STATUS_BUS_ERROR is detected if the bit driven onto the serial data line is different from the data read from the serial data line. As soon as a BUS_ERROR is detected, the I²C interface stops driving the SCL and SDA lines, and an interrupt is generated if the BUS_ERROR_INT_ENABLE bit is set. The BUS_ERROR bit remains asserted until the SDA line is released by the other devices on the bus. A user application needs to wait for the bus to be free again and issue a stop condition to reset the slaves connected on the bus.

The I²C interfaces use the I2C*_TX_DATA and I2C*_RX_DATA registers to respectively transmit and receive data. The I²C interfaces use the I2C*_STATUS_TX_REQ and I2C*_STATUS_RX_REQ bits in the I2C*_STATUS registers to request data for transmission, and to indicate when a new byte is available to be read. Writing to and reading from

Ezairo 8300 Hardware Reference

I2C*_TX_DATA and I2C*_RX_DATA automatically clears the corresponding *TX_REQ bit or RX_REQ bit, respectively, in hardware. As use of the I2C*_RX_DATA registers can advance the I²C interface state machine; these registers are mirrored without impacting the I²C state machine with the I2C*_RX_DATA_MIRROR registers. Reading from these registers does not clear the I2C*_STATUS_RX_REQ bit in the I2C_STATUS register. A write only bit called I2C*_STATUS_REQ_SET in the I2C_STATUS register can be used to set the I2C*_STATUS_TX_REQ register and discard the current data in the buffer.

NOTE: The I2C*_STATUS_TX_REQ status flag is useful in slave mode, when a master ends a transaction and forces a stop condition (I²C). At this point the TX_REQ flag is zero, which indicates that there is still a byte pending in the TX buffer. To clear this state without resetting the interface, set I2C*_STATUS_TX_REQ_SET high to request new data again. This does not generate a new interrupt (TX); only the underlying buffer is flushed.

Each byte of an I²C transaction must be ACK or NACK. Use the I2C*_CFG_AUTO_ACK_ENABLE bit from the I2C*_CFG registers to select whether the application handles this acknowledgment manually, or the interface handles it automatically. In either case, if the application does not provide data in time or move the interface's internal state machine forward, the transaction is paused by stretching the clock signal until the interface can continue. If the clock is being stretched, the I2C*_STATUS_CLK_STRETCH bit from the I2C*_STATUS register is set as an indicator. Use of the above mentioned data registers and the general behavior of the I²C state machine is described in [Section 18.4.4.1 “Operation Using Manual Accomplishment” on page 609](#), for manual acknowledgment mode, and in [Section 18.4.4.2 “Operation Using Auto Accomplishment” on page 609](#), for automatic acknowledgment mode.

NOTE: If, at any point, when operating in either master or slave mode, a user application needs to reset the I²C bus lines (for example, after encountering a bus error condition), the application needs to write the I2C_CTRL_RESET bit from the I2C_CTRL register. This resets both the I²C bus and the internal state machines of the I²C interface, allowing the system to cleanly start again from a known state.

18.4.1 Slave Mode Specific Configuration

In slave mode, the device is receiving the serial clock signal from an external master device, which also controls addressing, direction, and the start/stop conditions for each I²C transfer. For communications to proceed on the I²C interface, the following relation between the system clock (which is used internally to clock the I²C interface) and the SCL must hold:

$$F_{\text{SYSCLK}} \geq 3.125 \times F_{\text{SCL}}$$

When in slave mode, the I²C interfaces can be held while waiting for data to be read, or for new data to be available for transmission. In this case, the SCL line is held low by the slave interface—stretching the clock. The I2C_CFG_SLAVE bit fields in the I2C*_CFG registers are used to configure the settling time required for the SDA line, by selecting the number of SYSCLK cycles between the time when data is put on the SDA line, and when the SCL line is released. It is important that the software be able to keep up with data transmitting from the master at the full rate (1 Mbps in Fast-mode Plus), to minimize the overhead due to clock stretching. [Section 18.4.3 “Setting the System Clock” on page 608](#)

To enable a device in slave mode, set the I2C_CFG_SLAVE bit from the appropriate I2C*_CFG register. Both master and slave operation can be enabled at the same time. The application can use the I2C_STATUS_MASTER_MODE bit field to identify which state machine is active when an action is being requested by the interface. The device responds to communications on both the I²C general call address (0x00; I²C write transactions only) and at a

Ezairo 8300 Hardware Reference

programmable slave address that can be selected using the `I2C_CFG_SLAVE_ADDRESS` bit fields in the `I2C*_CFG` registers. During a transaction, the `I2C_STATUS_GEN_CALL` bit in the appropriate `I2C*_STATUS` register can be queried to identify the addresses that were used to address the device.

A device receiving data in a slave mode configuration can use the `I2C_CTRL_LAST_DATA` bit from the appropriate `I2C*_CTRL` register to automatically NACK the last data byte, to indicate to a master device that it needs to send a stop condition.

NOTE: The `I2C_CTRL_LAST_DATA` bit is cleared when a stop condition is received.

18.4.2 Master Mode Specific Configuration

In master mode, the device provides the serial clock signal and controls all transfer information. To configure the clock signal, you need to select a prescaling division by 3 using the `I2C_CFG_MASTER_PRESCALE` bit field in the appropriate `I2C*_CFG` register. The interface clock frequency for a given configuration can be calculated using this formula:

$$f_{I2C \text{ MASTER}} = \frac{f_{SYSCLK}}{3 \cdot (I2C_CFG_MASTER_PRESCALE + 1)}$$

NOTE: The System clock needs to be set to at least 1.5 MHz for Fast-mode (400 kbps), and at least 6 MHz for Fast-mode Plus (1 Mbps). [Section 18.4.3 “Setting the System Clock” on page 608](#)

A user application must manually control all components of a master mode transaction. To transmit or receive using master mode, a device must:

1. Check that the I²C bus lines are not currently in use, by reading the `I2C_STATUS_LINE_FREE` bit in the appropriate `I2C*_STATUS` register. The device can only start a transaction if the lines are free.
2. Start a transaction, sending the start condition, address, and direction, by loading the appropriate address and direction to the `I2C*_ADDR_START` register.
3. Handle data and interrupts as appropriate for the transaction (see [Section 18.4.4 “I2C Interrupts” on page 608](#)).
4. Complete the transaction by sending a stop condition.
A stop condition can be generated using either the `I2C_CTRL_STOP` bit or the `I2C_CTRL_LAST_DATA` bit in the `I2C*_CTRL` registers.

When using the `I2C_CTRL_LAST_DATA` bit, a stop condition is automatically generated after the current data byte transfer is completed. If the interface is receiving data, this bit also automatically generates the required NACK of the last data byte that is received.

Multi-master mode can be supported in software by handling the `BUS_ERROR` status bit, which is set when a conflict occurs. The arbitration procedure and clock synchronization are active in master mode.

NOTE: If the I²C interface is used for transmitting, the last byte of data to be transmitted must be written to the `I2C_DATA` register before setting the `I2C_CTRL1_LAST_DATA` bit, as this bit triggers a stop condition at the end of the current transfer.

NOTE: The `I2C_CTRL1_LAST_DATA` bit is cleared when a stop condition is received on the I²C bus.

Ezairo 8300 Hardware Reference

When using the `I2C_CTRL_STOP` bit, a stop condition is issued immediately. This bit is normally set when the `I2C_CTRL_ACK` bit or `I2C_CTRL_NACK` bit in the `I2C_CTRL` register is set. In master mode, a stop condition is automatically generated if data or an address has not been acknowledged by the slave.

CAUTION: Using the stop event (STOP) immediately transmits a stop condition as soon as possible. This can result in a terminated transfer, and remote devices on the bus might detect I²C errors.

The following list shows the interrupt triggers which can be enabled, and the reasons why they would be triggered:

- `TX_INT_ENABLE` (triggered when a byte is transmitted)
- `RX_INT_ENABLE` (triggered when a byte is received)
- `BUS_ERROR_INT_ENABLE` (triggered when a bus error occurs)
- `OVERRUN_INT_ENABLE` (triggered when any data has been received that has overwritten the RX data buffer before the data in the buffer could be read)
- `STOP_INT_ENABLE` (triggered when a stop event occurs)
- `REPEATED_START_INT_ENABLE` (triggered when a repeated start event occurs)

18.4.3 Setting the System Clock

A maximum transmission rate of 400 kbps (Fast-mode) is fully supported by the I²C interface. In this mode, the system clock must be set to at least 1.5 MHz for Fast-mode (400 kbps), for a system with a sufficient pull-up resistance on SDA to provide a pull-up time of less than 150 ns.

A maximum transmission rate of 1 Mbps (Fast-mode plus) is supported with the exception of the pad drive strength required by the I²C standard (20 mA at 0.4 V). Multiple DIO pads can be used in parallel to increase the drive strength, but this might not be sufficient to meet the specifications, especially at low VDDO. In this mode, the system clock must be set to at least 6 MHz for Fast-mode Plus (1Mbps) for a system with a with a sufficient pull-up resistance unreasonably fast SDA to provide a pull-up time of less than 100 ns.

18.4.4 I²C Interrupts

The I²C interfaces use associated interrupts which, when enabled, signal the receipt of a correct address byte and the completion of each data byte in the transaction. Interrupts can also be enabled to identify bus-related conditions, such as bus errors during arbitration, stop conditions on the bus, and repeated start conditions (in slave mode only). In addition, the buffer overrun interrupt is available to identify firmware issues.

See [Section 4.6.1 “Nested Vectored Interrupt Controller \(NVIC\)” on page 135](#) for information regarding interrupt configuration and handling interrupts for the Arm Cortex-M3 processor. See [Section 3.2 “Interrupt Controller” on page 61](#) for information regarding interrupt configuration and handling for the CFX DSP, and [Section 4.6.1 “Nested Vectored Interrupt Controller \(NVIC\)” on page 135](#) for the equivalent information as applied to the Arm Cortex-M3 processor.

Several status indicators from the `I2C*_STATUS` registers can be used with the interrupt to identify the state of the I²C interface:

- If the `I2C_STATUS_ADDR_DATA` bit is set, the interrupt has been generated in response to a recognized sequence, including a start condition and address on the I²C interface. No interrupt is generated for I²C transactions that use an address which has not been recognized by the I²C interface.

Ezairo 8300 Hardware Reference

- If the `I2C_STATUS_STOP_DETECTED` bit is set, the interrupt has been generated in response to a stop interrupt being a condition for an active transaction on the I²C interface. To clear the status of this bit, write a 1 to the `I2C_STATUS_STOP_DETECTED_CLEAR` bit.
- If the `I2C_STATUS_DATA_EVENT` bit is set, the interrupt indicates that data has been received and can be read, or that data is needed in order to continue with the data transmission.
- If the `I2C_STATUS_BUS_ERROR` bit is set, then a bus error has occurred.
- If the `REPEATED_START_DETECTED` bit is set, a repeated start condition has been detected by the device. This interrupt occurs during an active transaction in slave mode.

18.4.4.1 Operation Using Manual Accomplishment

If an application is using manual acknowledgment (i.e., the `I2C*_CFG_AUTO_ACK_ENABLE` bit from the `I2C*_CFG` registers is disabled), the application needs to use the `I2C_CTRL_NACK` and `I2C_CTRL_ACK` bits from the `I2C*_CTRL` registers to acknowledge (ACK) or not-acknowledge (NACK) transaction bytes.

When handling a transaction, an interrupt is issued whenever data or an acknowledgment is required (in addition to interrupts triggered by stop conditions).

NOTE: An interrupt is not generated following a NACK for any data condition. However, an interrupt is generated (if operating in master mode or if stop interrupts are enabled) on the stop condition that is expected to follow the NACK.

A special interrupt is also generated for a master read, once the acknowledge bit from the address byte has been received. This scenario is a special case: it does not require data nor acknowledgment to trigger an interrupt, but instead is designed to allow the interface to check whether or not a slave device has acknowledged the transfer at that point. Provided the slave device has responded, the interface can issue a resume command by writing a 1 to the `I2C_CTRL_RESUME` bit command, which continues the transfer.

NOTE: Use of the `I2C_CTRL_NACK`, `I2C_CTRL_ACK` and `I2C_CTRL_STOP` bits from the `I2C*_CTRL` registers is only defined when an acknowledgment is needed during a data transfer. Interface behavior in response to these control bits being set at other times is undefined.

18.4.4.2 Operation Using Auto Accomplishment

When operating using auto acknowledgment (i.e., the `I2C_CFG_AUTO_ACK_ENABLE` bit from the appropriate `I2C*_CFG` register is enabled), transfers are controlled by the availability of bytes for transmission, or a place to put a byte when receiving data. This allows transfers to occur without clock stretching, if the firmware is able to handle each data byte fast enough. A DMA channel can be used in auto acknowledgment mode to reduce CPU utilization.

NOTE: DMA-based transfers are only supported in auto acknowledge mode.

When the interface handles a transaction, interrupts are issued whenever new data can be written or new data has been received. This allows a user application to maintain a transaction without introducing delay.

NOTE: An interrupt is not generated following a NACK for any data condition. However, an interrupt is generated (if operating in master mode or if stop interrupts are enabled) on the stop condition that is expected to follow the NACK.

Ezairo 8300 Hardware Reference

Additionally, interrupts are generated if data has been supplied but cannot be sent because the interface has received a NACK from the slave device, or is currently idle (for instance, after a stop condition). If data is required but is not currently available because the buffer is empty, the clock is stretched and an additional interrupt is generated. If new data has been received, but the previous data has not been read (resulting in the buffer not being available), the clock is stretched until the buffer becomes free.

IMPORTANT: When receiving data in master mode, the timing of interrupts for auto acknowledgment of data prevents a user application from cleanly terminating a transfer using the **LAST_DATA** event. As a result, if the interface is running in this mode, a dummy byte must be transferred to cleanly terminate the current transfer, with a stop condition following this byte.

18.4.4.3 I²C Main Modes of Operation

The main master and slave operation modes, with and without auto acknowledge enabled, are shown in the "I²C Master Modes of Operation" table (Table 58), below, and the "I²C Slave Modes of Operation" table (Table 59).

Table 58. I²C Master Modes of Operation

Mode	Auto ack off, Write frame	Auto ack on, Read frame	Auto ack on, Write frame	Auto ack on, Read frame
Start	Receive start, interrupt if repeated start and repeated start interrupt are enabled	Receive start, interrupt if repeated start and repeated start interrupt are enabled	Receive start, interrupt if repeated start and repeated start interrupt are enabled	Receive start, interrupt if repeated start and repeated start interrupt are enabled
Addr	Receive addr	Receive addr	Receive addr	Receive addr
Ack	Interrupt if addr is matched, stretch clock until ack/nack, send ack/nack	Interrupt if addr is matched, stretch clock until ack/nack, send ack/nack	Ack if addr is matched	Ack if addr is matched
Data0	Receive RX data	Interrupt if TX data not available, stretch until TX data available, send TX data	Receive RX data	Interrupt if TX data not available, stretch until TX data available, send TX data, interrupt
Ack	Interrupt, stretch clock until ack/nack, send ack/nack	Receive ack/nack from remote	Interrupt, stretch clock until RX data is read, ack	Receive ack/nack from remote
Data N	Receive RX data	Interrupt if TX data not available, stretch until TX data available, send TX data	Receive RX data	Interrupt if TX data not available, stretch until TX data available, interrupt, send TX data
Ack N	Interrupt, stretch clock until ack/nack, send ack/nack	Receive ack/nack from remote	Interrupt, stretch clock until RX data is read, ack	Receive ack/nack from remote
Stop	Receive stop, interrupt if stop interrupt is enabled	Receive stop, interrupt if stop interrupt is enabled	Receive stop, interrupt if stop interrupt is enabled	Receive stop, interrupt if stop interrupt is enabled

Ezairo 8300 Hardware Reference

Table 59. I²C Slave Modes of Operation

Mode	Auto ack off, Read frame	Auto ack off, Write frame	Auto ack on, Read frame	Auto ack on, Write frame
Start	Send start	Send start	Send start	Send start
Addr	Send addr	Send addr	Send addr	Interrupt, send addr
Ack	Receive ack/nack from remote	Receive ack/nack from remote	Receive ack/nack from remote	Receive ack/nack from remote
Data0	Interrupt, stretch clock until resume, receive RX data	Interrupt if TX data not available, stretch clock until TX data available, send TX data	Receive RX data	Interrupt if TX data not available, stretch clock until TX data available, interrupt, send TX data
Ack	Interrupt, stretch clock until ack/nack, send ack/nack	Receive ack/nack from remote	Interrupt, stretch clock until RX data is read, ack	Receive ack/nack from remote
Data N	Receive RX data	Interrupt if TX data not available, stretch clock until TX data available, send TX data	Receive RX data	Interrupt if TX data not available, stretch clock until TX data available, interrupt, send TX data
Ack N	Interrupt, stretch clock until ack/nack, send ack/nack	Receive ack/nack from remote	Interrupt, stretch clock until RX data is read, ack	Receive ack/nack from remote
Stop	Send stop, interrupt, interrupt if stop interrupt enabled	Send stop, interrupt, interrupt if stop interrupt enabled	Send stop, interrupt, interrupt if stop interrupt enabled	Send stop, interrupt, interrupt if stop interrupt enabled

18.4.5 I²C Registers

Register Name	Register Description	Address
D_I2C_CFG	I2C Configuration Register	P:0x0A00 0x40002800
D_I2C_CTRL	I2C Control Register	P:0x0A01 0x40002804
D_I2C_ADDR_START	I2C Master Address and Start Register	P:0x0A02 0x40002808
D_I2C_STATUS	I2C Status Register	P:0x0A03 0x4000280C

onsemi Confidential
Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
D_I2C_TX_DATA	I2C Transmit Data Register	P:0x0A04 0x40002810
D_I2C_RX_DATA	I2C Receive Data Register	P:0x0A05 0x40002814
D_I2C_RX_DATA_MIRROR	I2C Receive and Transmit Data (Mirror) Register	P:0x0A06 0x40002818
I2C0_CFG	I2C Configuration Register	P:0x1400 0x40001000
I2C0_CTRL	I2C Control Register	P:0x1401 0x40001004
I2C0_ADDR_START	I2C Master Address and Start Register	P:0x1402 0x40001008
I2C0_STATUS	I2C Status Register	P:0x1403 0x4000100C
I2C0_TX_DATA	I2C Transmit Data Register	P:0x1404 0x40001010
I2C0_RX_DATA	I2C Receive Data Register	P:0x1405 0x40001014
I2C0_RX_DATA_MIRROR	I2C Receive Data Mirror Register	P:0x1406 0x40001018
I2C1_CFG	I2C Configuration Register	P:0x1440 0x40001100
I2C1_CTRL	I2C Control Register	P:0x1441 0x40001104
I2C1_ADDR_START	I2C Master Address and Start Register	P:0x1442 0x40001108
I2C1_STATUS	I2C Status Register	P:0x1443 0x4000110C

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
I2C1_TX_DATA	I2C Transmit Data Register	P:0x1444 0x40001110
I2C1_RX_DATA	I2C Receive Data Register	P:0x1445 0x40001114
I2C1_RX_DATA_MIRROR	I2C Receive Data Mirror Register	P:0x1446 0x40001118

18.4.5.1 D_I2C_CFG

Bit Field	Read/Write	Field Name	Description
30	RW	REPEATED_START_INT_ENABLE	Configure whether repeated start interrupts will be generated by the I2C interface for active transactions in slave mode
29	RW	CONNECT_IN_STANDBY	Control if the I2C lines are connected when running on the standby clock
28	RW	TX_DMA_ENABLE	Enable/disable the TX DMA request
27	RW	RX_DMA_ENABLE	Enable/disable the RX DMA request
26	RW	TX_INT_ENABLE	Enable/disable the TX interrupt
25	RW	RX_INT_ENABLE	Enable/disable the RX interrupt
24	RW	BUS_ERROR_INT_ENABLE	Enable/disable the bus error interrupt
23	RW	OVERRUN_INT_ENABLE	Enable/disable the overrun interrupt
22	RW	STOP_INT_ENABLE	Configure whether stop interrupts are generated by the I2C interface
21	RW	AUTO_ACK_ENABLE	Select whether acknowledgment is automatically generated or not
20:16	RW	SLAVE_PRESCALE	Controls the number of SYSCLK wait cycles in case of clock stretching (in slave mode) between the moment the data is put on the SDA line and the SCL line is released.
15:8	RW	MASTER_PRESCALE	Prescaler used to divide SYSCLK to the correct SCL frequency when operating in master mode. SCL is prescaled by $(PRESCALE + 1) * 3$.
7:1	RW	SLAVE_ADDRESS	Set the I2C slave address for this device
0	RW	SLAVE	I2C slave mode enable

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
REPEATED_START_INT_ENABLE	D_I2C_REPEATED_START_INT_DISABLE	Repeated start interrupts are not generated	0x0*
	D_I2C_REPEATED_START_INT_ENABLE	A repeated start interrupt is generated when a repeated start condition occurs during an active transaction in slave mode	0x1
CONNECT_IN_STANDBY	D_I2C_DISCONNECT_IN_STANDBY	Disconnect the I2C lines when running on standby clock	0x0*
	D_I2C_CONNECT_IN_STANDBY	Keep the I2C lines connected when running on standby clock	0x1
TX_DMA_ENABLE	D_I2C_TX_DMA_DISABLE	No TX DMA request is generated	0x0*
	D_I2C_TX_DMA_ENABLE	A TX DMA request is generated when new data is requested by the I2C interface	0x1
RX_DMA_ENABLE	D_I2C_RX_DMA_DISABLE	No RX DMA request is generated	0x0*
	D_I2C_RX_DMA_ENABLE	An RX DMA request is generated when new data is received by the I2C interface	0x1
TX_INT_ENABLE	D_I2C_TX_INT_DISABLE	No TX interrupt is raised	0x0*
	D_I2C_TX_INT_ENABLE	A TX interrupt is raised when new data is requested by the I2C interface	0x1
RX_INT_ENABLE	D_I2C_RX_INT_DISABLE	No RX interrupt is raised	0x0*
	D_I2C_RX_INT_ENABLE	An RX interrupt is raised when new data is received by the I2C interface	0x1
BUS_ERROR_INT_ENABLE	D_I2C_BUS_ERROR_INT_DISABLE	No bus error interrupt is raised when an overrun is detected	0x0*
	D_I2C_BUS_ERROR_INT_ENABLE	A bus error interrupt is raised when an overrun occurs on the I2C interface	0x1
OVERRUN_INT_ENABLE	D_I2C_OVERRUN_INT_DISABLE	No overrun interrupt is raised when an overrun is detected	0x0*
	D_I2C_OVERRUN_INT_ENABLE	An overrun interrupt is raised when an overrun occurs on the I2C interface	0x1
STOP_INT_ENABLE	D_I2C_STOP_INT_DISABLE	Stop interrupts are not generated	0x0*
	D_I2C_STOP_INT_ENABLE	A stop interrupt are not generated when a stop condition occurs for an active transaction	0x1
AUTO_ACK_ENABLE	D_I2C_AUTO_ACK_DISABLE	Require manual acknowledgment of all I2C interface transfers	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	D_I2C_AUTO_ACK_ENABLE	Use automatic acknowledgment for I2C interface transfers	0x1
SLAVE_PRESCALE	D_I2C_SLAVE_PRESCALE_1	Slave Standard-mode: at least 250 ns +10% data set-up time with SYSCLK = 3 MHz; Slave Fast-mode: at least 100 ns +10% data set-up time with SYSCLK = 3, 4, 5, 8 MHz; Slave Fast-mode Plus: at least 50 ns +10% data set-up time with SYSCLK = 3, 4, 5, 8, 10, 12, 16 MHz	0x0*
	D_I2C_SLAVE_PRESCALE_2	Slave Standard-Mode: at least 250 ns +10% data set-up time with SYSCLK = 4, 5 MHz; Slave Fast-Mode: at least 100 ns +10% data set-up time with SYSCLK = 10, 12, 16 MHz; Slave Fast-mode Plus: at least 50 ns +10% data set-up time with SYSCLK = 20, 24 MHz	0x1
	D_I2C_SLAVE_PRESCALE_3	Slave Standard-Mode: at least 250 ns +10% data set-up time with SYSCLK = 8, 10 MHz; Slave Fast-Mode: at least 100 ns +10% data set-up time with SYSCLK = 20, 24 MHz; Slave Fast-mode Plus: at least 50 ns +10% data set-up time with SYSCLK = 48 MHz	0x2
	D_I2C_SLAVE_PRESCALE_4	Slave Standard-Mode: at least 250 ns +10% data set-up time with SYSCLK = 12 MHz	0x3
	D_I2C_SLAVE_PRESCALE_5	Slave Standard-Mode: at least 250 ns +10% data set-up time with SYSCLK = 16 MHz	0x4
	D_I2C_SLAVE_PRESCALE_6	Slave Standard-Mode: at least 250 ns +10% data set-up time with SYSCLK = 20 MHz; Slave Fast-Mode: at least 100 ns +10% data set-up time with SYSCLK = 48 MHz	0x5
	D_I2C_SLAVE_PRESCALE_7	Slave Standard-Mode: at least 250 ns +10% data set-up time with SYSCLK = 24 MHz	0x6
	D_I2C_SLAVE_PRESCALE_14	Slave Standard-Mode: at least 250 ns +10% data set-up time with SYSCLK = 48 MHz	0xD
	D_I2C_SLAVE_PRESCALE_32	Slave Standard-Mode: at least 250 ns	0x1F

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		+10% data set-up time with SYSCLK = 48 MHz	
MASTER_PRESCALE	D_I2C_MASTER_PRESCALE_3	Master mode: prescale SCL from SYSCLK by 3	0x0*
	D_I2C_MASTER_PRESCALE_6	Master mode: prescale SCL from SYSCLK by 6	0x1
	D_I2C_MASTER_PRESCALE_9	Master mode: prescale SCL from SYSCLK by 9	0x2
	D_I2C_MASTER_PRESCALE_12	Master mode: prescale SCL from SYSCLK by 12	0x3
	D_I2C_MASTER_PRESCALE_15	Master mode: prescale SCL from SYSCLK by 15	0x4
	D_I2C_MASTER_PRESCALE_18	Master mode: prescale SCL from SYSCLK by 18	0x5
	D_I2C_MASTER_PRESCALE_21	Master mode: prescale SCL from SYSCLK by 21	0x6
	D_I2C_MASTER_PRESCALE_24	Master mode: prescale SCL from SYSCLK by 24	0x7
	D_I2C_MASTER_PRESCALE_27	Master mode: prescale SCL from SYSCLK by 27	0x8
	D_I2C_MASTER_PRESCALE_30	Master mode: prescale SCL from SYSCLK by 30	0x9
	D_I2C_MASTER_PRESCALE_33	Master mode: prescale SCL from SYSCLK by 33	0xA
	D_I2C_MASTER_PRESCALE_36	Master mode: prescale SCL from SYSCLK by 36	0xB
	D_I2C_MASTER_PRESCALE_39	Master mode: prescale SCL from SYSCLK by 39	0xC
	D_I2C_MASTER_PRESCALE_42	Master mode: prescale SCL from SYSCLK by 42	0xD
	D_I2C_MASTER_PRESCALE_45	Master mode: prescale SCL from SYSCLK by 45	0xE
	D_I2C_MASTER_PRESCALE_48	Master mode: prescale SCL from SYSCLK by 48	0xF
	D_I2C_MASTER_PRESCALE_51	Master mode: prescale SCL from SYSCLK by 51	0x10
	D_I2C_MASTER_PRESCALE_54	Master mode: prescale SCL from SYSCLK by 54	0x11

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	D_I2C_MASTER_PRESCALE_57	Master mode: prescale SCL from SYSCLK by 57	0x12
	D_I2C_MASTER_PRESCALE_60	Master mode: prescale SCL from SYSCLK by 60	0x13
	D_I2C_MASTER_PRESCALE_120	Master mode: prescale SCL from SYSCLK by 120	0x27
	D_I2C_MASTER_PRESCALE_768	Master mode: prescale SCL from SYSCLK by 768	0xFF
SLAVE	D_I2C_SLAVE_DISABLE	Disable I2C slave mode	0x0*
	D_I2C_SLAVE_ENABLE	Enable I2C slave mode	0x1

18.4.5.2 D_I2C_CTRL

Bit Field	Read/Write	Field Name	Description
9	R	LAST_DATA_STATUS	I2C last data status
7	W	ERROR_CNT_CLEAR	Clear the D_I2C_ERROR_CNT register
6	W	LAST_DATA	Indicate that the current data is the last byte of a data transfer
5	W	STOP	Issue a stop condition on the I2C interface bus
4	W	NACK	Issue a not acknowledge on the I2C interface bus
3	W	ACK	Issue an acknowledge on the I2C interface bus
2	W	RESET	Reset the I2C interface

Field Name	Value Symbol	Value Description	Hex Value
LAST_DATA_STATUS	D_I2C_LAST_DATA_DISABLED	I2C last data is disabled	0x0*
	D_I2C_LAST_DATA_ENABLED	I2C last data is enabled	0x1
ERROR_CNT_CLEAR	D_I2C_ERROR_CNT_CLEAR	Clear the D_I2C_ERROR_CNT register	0x1
LAST_DATA	D_I2C_LAST_DATA	Indicate that the current data is the last byte of a data transfer	0x1
STOP	D_I2C_STOP	Issue a stop condition on the I2C interface bus	0x1
NACK	D_I2C_NACK	Issue a not acknowledge on the I2C interface bus	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
ACK	D_I2C_ACK	Issue an acknowledge on the I2C interface bus	0x1
	D_I2C_RESUME	Resume I2C interface after receiving address ACK/NACK in master read auto ack disable mode	0x1
RESET	D_I2C_RESET	Reset the I2C interface	0x1

18.4.5.3 D_I2C_ADDR_START

Bit Field	Read/Write	Field Name	Description
7:1	RW	ADDR	Address to send with the start condition over the I2C bus
0	RW	READ_WRITE	Select whether a read or a write transaction is started

Field Name	Value Symbol	Value Description	Hex Value
READ_WRITE	D_I2C_START_WRITE	Initiate a write transfer	0x0*
	D_I2C_START_READ	Initiate a read transfer	0x1

18.4.5.4 D_I2C_STATUS

Bit Field	Read/Write	Field Name	Description
26	R	STOP_OR_REPEATED_START_DETECTED	Indicate if STOP_DETECTED or REPEATED_START_DETECTED bit is set
25	R	REPEATED_START_DETECTED	Indicate if an I2C repeated start has been detected during an active transaction in slave mode
23	R	BUS_OR_WATCHDOG_ERROR	Bus or watchdog error status bit
22	R	BUS_ERROR	Bus error status bit
21	R	BUSY	Indicate that the reception or transmission of the data is ongoing
20	R	START_PENDING	Master frame start pending status bit
19	R	MASTER_MODE	Master mode status bit
18	R	STOP_DETECTED	Indicate if an I2C stop bit has been detected
17	R	DATA_EVENT	Indicate that I2C interface either needs data to transmit or has received data
16	R	TX_REQ	Indicate that a TX data can be written
15	R	RX_REQ	Indicate that a RX data can be read
14	R	CLK_STRETCH	Clock stretching flag

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
13	R	LINE_FREE	Line free flag
12	R	ADDR_DATA	Address / Data byte
11	R	READ_WRITE	Read / Write frame
10	R	GEN_CALL	General call flag
9	R	ACK	Acknowledge status
8	R	OVERRUN	Indicate that an overrun has occurred when receiving data
4	W	TX_REQ_SET	Set TX_REQ status flag
3	W	REPEATED_START_DETECTED_CLEAR	Clear REPEATED_START_DETECTED status flag
2	W	STOP_DETECTED_CLEAR	Clear STOP_DETECTED status flag
1	W	BUS_ERROR_CLEAR	Clear BUS_ERROR status flag
0	W	OVERRUN_CLEAR	Clear OVERRUN status flag

Field Name	Value Symbol	Value Description	Hex Value
STOP_OR_REPEATED_START_DETECTED	D_I2C_STOP_NOR_REPEATED_START_DETECTED	Neither STOP_DETECTED nor REPEATED_START_DETECTED bits are set	0x0*
	D_I2C_STOP_OR_REPEATED_START_DETECTED	STOP_DETECTED or REPEATED_START_DETECTED bit is set	0x1
REPEATED_START_DETECTED	D_I2C_NO_REPEATED_START_DETECTED	No repeated start condition has been detected on the I2C bus during an active transaction in slave mode	0x0*
	D_I2C_REPEATED_START_DETECTED	A repeated start condition has been detected on the I2C bus during an active transaction in slave mode	0x1
BUS_OR_WATCHDOG_ERROR	D_I2C_NO_ERROR	BUS_ERROR is 0 and I2C_ERROR_CNT equals 0	0x0*
	D_I2C_BUS_OR_WATCHDOG_ERROR	BUS_ERROR is 1 or I2C_ERROR_CNT is not equal to 0	0x1
BUS_ERROR	D_I2C_NO_BUS_ERROR	No I2C bus error has occurred	0x0*
	D_I2C_BUS_ERROR	An I2C bus error has occurred	0x1
BUSY	D_I2C_IDLE	I2C interface is idle	0x0*
	D_I2C_BUSY	I2C interface is busy	0x1
START_PENDING	D_I2C_START_NOT_PENDING	No pending master start frame	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	D_I2C_START_PENDING	A master frame start is pending (bit is set when I2C_ADDR_START is written)	0x1
MASTER_MODE	D_I2C_MASTER_INACTIVE	I2C interface is not operating in master mode	0x0*
	D_I2C_MASTER_ACTIVE	I2C interface is operating in master mode	0x1
STOP_DETECTED	D_I2C_NO_STOP_DETECTED	No stop condition has been detected on the I2C bus	0x0*
	D_I2C_STOP_DETECTED	A stop condition has been detected on the I2C bus	0x1
DATA_EVENT	D_I2C_NON_DATA_EVENT	No I2C data is needed or available	0x0*
	D_I2C_DATA_EVENT	I2C data is needed or is available	0x1
TX_REQ	D_I2C_TX_NO_REQ	I2C TX data has already been written	0x0
	D_I2C_TX_REQ	I2C TX data can be written	0x1*
RX_REQ	D_I2C_RX_NO_REQ	No new I2C RX data available	0x0*
	D_I2C_RX_REQ	New I2C RX data available	0x1
CLK_STRETCH	D_I2C_CLK_NOT_STRETCHED	I2C interface is currently not clock stretching	0x0*
	D_I2C_CLK_STRETCHED	I2C interface is currently clock stretching	0x1
LINE_FREE	D_I2C_BUS_BUSY	I2C transaction ongoing	0x0
	D_I2C_BUS_FREE	I2C bus is free	0x1*
ADDR_DATA	D_I2C_DATA_IS_DATA	The I2C data register holds data	0x0*
	D_I2C_DATA_IS_ADDR	The I2C data register holds an address	0x1
READ_WRITE	D_I2C_IS_WRITE	The current I2C transfer is a write	0x0*
	D_I2C_IS_READ	The current I2C transfer is a read	0x1
GEN_CALL	D_I2C_ADDR_OTHER	The address used for the current I2C transfer is not the general call address	0x0*
	D_I2C_ADDR_GEN_CALL	The address used for the current I2C transfer is the general call address	0x1
ACK	D_I2C_HAS_ACK	Indicate that the last I2C byte was acknowledged	0x0*
	D_I2C_HAS_NACK	Indicate that the last I2C byte was not acknowledged	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
OVERRUN	D_I2C_OVERRUN_FALSE	No I2C input overrun detected	0x0*
	D_I2C_OVERRUN_TRUE	I2C input overrun detected	0x1
TX_REQ_SET	D_I2C_TX_REQ_SET	Set the TX_REQ status flag	0x1
REPEATED_START_DETECTED_CLEAR	D_I2C_REPEATED_START_DETECTED_CLEAR	Clear the REPEATED_START_DETECTED status flag	0x1
STOP_DETECTED_CLEAR	D_I2C_STOP_DETECTED_CLEAR	Clear the STOP_DETECTED status flag	0x1
BUS_ERROR_CLEAR	D_I2C_BUS_ERROR_CLEAR	Clear the BUS_ERROR status flag	0x1
OVERRUN_CLEAR	D_I2C_OVERRUN_CLEAR	Clear the OVERRUN status flag	0x1

18.4.5.5 D_I2C_TX_DATA

Bit Field	Read/Write	Field Name	Description
7:0	RW	TX_DATA	Single byte buffer for data transmitted over the I2C interface

18.4.5.6 D_I2C_RX_DATA

Bit Field	Read/Write	Field Name	Description
7:0	R	RX_DATA	Single byte buffer for data received over the I2C interface

18.4.5.7 D_I2C_RX_DATA_MIRROR

Bit Field	Read/Write	Field Name	Description
7:0	R	RX_DATA	Mirror of the single byte buffer for data received over the I2C interface. Does not clear RX_REQ in D_I2C_STATUS

18.4.5.8 D_I2C_WATCHDOG_CFG

Bit Field	Read/Write	Field Name	Description
2:0	RW	TIMEOUT	Watchdog timeout configuration

Field Name	Value Symbol	Value Description	Hex Value
TIMEOUT	I2C_WATCHDOG_TIMEOUT_250M	Timeout is 0x0	0x0
	I2C_WATCHDOG_TIMEOUT_500M	Timeout is 0x1	0x1
	I2C_WATCHDOG_TIMEOUT_1000M	Timeout is 0x2	0x2*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	I2C_WATCHDOG_TIMEOUT_2000M	Timeout is 0x3	0x3
	I2C_WATCHDOG_TIMEOUT_4000M	Timeout is 0x4	0x4
	I2C_WATCHDOG_TIMEOUT_DISABLED	Disable the I2C watchdog timer	0x5

18.4.5.9 D_I2C_ERROR_CNT

Bit Field	Read/Write	Field Name	Description
7:0	R	ERROR_CNT	Counter the number of time the I2C-watchdog timeout

18.4.5.10 I2C*_CFG

Bit Field	Read/Write	Field Name	Description
30	RW	REPEATED_START_INT_ENABLE	Configure whether repeated start interrupts will be generated by the I2C interface for active transactions in slave mode
29	RW	CONNECT_IN_STANDBY	Control if the I2C lines are connected when running on the standby clock
28	RW	TX_DMA_ENABLE	Enable/disable the TX DMA request
27	RW	RX_DMA_ENABLE	Enable/disable the RX DMA request
26	RW	TX_INT_ENABLE	Enable/disable the TX interrupt
25	RW	RX_INT_ENABLE	Enable/disable the RX interrupt
24	RW	BUS_ERROR_INT_ENABLE	Enable/disable the bus error interrupt
23	RW	OVERRUN_INT_ENABLE	Enable/disable the overrun interrupt
22	RW	STOP_INT_ENABLE	Configure whether stop interrupts will be generated by the I2C interface
21	RW	AUTO_ACK_ENABLE	Select whether acknowledgment is automatically generated or not
20:16	RW	SLAVE_PRESCALE	Controls the number of SYSCLK wait cycles in case of clock stretching (in slave mode) between the moment the data is put on the SDA line and the SCL line is released.
15:8	RW	MASTER_PRESCALE	Prescaler used to divide SYSCLK to the correct SCL frequency when operating in master mode. SCL is prescaled by $(PRESCALE + 1) * 3$.
7:1	RW	SLAVE_ADDRESS	Set the I2C slave address for this device
0	RW	SLAVE	Select whether the I2C interface is enabled for slave mode or not

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
REPEATED_START_INT_ENABLE	I2C_REPEATED_START_INT_DISABLE	Repeated start interrupts are not generated	0x0*
	I2C_REPEATED_START_INT_ENABLE	A repeated start interrupt is generated when a repeated start condition occurs during an active transaction in slave mode	0x1
CONNECT_IN_STANDBY	I2C_DISCONNECT_IN_STANDBY	Disconnect the I2C lines when running on standby clock	0x0*
	I2C_CONNECT_IN_STANDBY	Keep the I2C lines connected when running on standby clock	0x1
TX_DMA_ENABLE	I2C_TX_DMA_DISABLE	No TX DMA request is generated	0x0*
	I2C_TX_DMA_ENABLE	A TX DMA request is generated when new data is requested by the I2C interface	0x1
RX_DMA_ENABLE	I2C_RX_DMA_DISABLE	No RX DMA request is generated	0x0*
	I2C_RX_DMA_ENABLE	An RX DMA request is generated when new data is received by the I2C interface	0x1
TX_INT_ENABLE	I2C_TX_INT_DISABLE	No TX interrupt is raised	0x0*
	I2C_TX_INT_ENABLE	A TX interrupt is raised when new data is requested by the I2C interface	0x1
RX_INT_ENABLE	I2C_RX_INT_DISABLE	No RX interrupt is raised	0x0*
	I2C_RX_INT_ENABLE	An RX interrupt is raised when new data is received by the I2C interface	0x1
BUS_ERROR_INT_ENABLE	I2C_BUS_ERROR_INT_DISABLE	No bus error interrupt is raised when an overrun is detected	0x0*
	I2C_BUS_ERROR_INT_ENABLE	A bus error interrupt is raised when an overrun occurs on the I2C interface	0x1
OVERRUN_INT_ENABLE	I2C_OVERRUN_INT_DISABLE	No overrun interrupt is raised when an overrun is detected	0x0*
	I2C_OVERRUN_INT_ENABLE	An overrun interrupt is raised when an overrun occurs on the I2C interface	0x1
STOP_INT_ENABLE	I2C_STOP_INT_DISABLE	Stop interrupts are not generated	0x0*
	I2C_STOP_INT_ENABLE	A stop interrupt is generated when a stop condition occurs for an active transaction	0x1
AUTO_ACK_ENABLE	I2C_AUTO_ACK_DISABLE	Require manual acknowledgment of all I2C interface transfers	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	I2C_AUTO_ACK_ENABLE	Use automatic acknowledgment for I2C interface transfers	0x1
SLAVE_PRESCALE	I2C_SLAVE_PRESCALE_1	Slave Standard-mode: at least 250 ns +10% data set-up time with SYSCLK = 3 MHz; Slave Fast-mode: at least 100 ns +10% data set-up time with SYSCLK = 3, 4, 5, 8 MHz; Slave Fast-mode Plus: at least 50 ns +10% data set-up time with SYSCLK = 3, 4, 5, 8, 10, 12, 16 MHz	0x0*
	I2C_SLAVE_PRESCALE_2	Slave Standard-Mode: at least 250 ns +10% data set-up time with SYSCLK = 4, 5 MHz; Slave Fast-Mode: at least 100 ns +10% data set-up time with SYSCLK = 10, 12, 16 MHz; Slave Fast-mode Plus: at least 50 ns +10% data set-up time with SYSCLK = 20, 24 MHz	0x1
	I2C_SLAVE_PRESCALE_3	Slave Standard-Mode: at least 250 ns +10% data set-up time with SYSCLK = 8, 10 MHz; Slave Fast-Mode: at least 100 ns +10% data set-up time with SYSCLK = 20, 24 MHz; Slave Fast-mode Plus: at least 50 ns +10% data set-up time with SYSCLK = 48 MHz	0x2
	I2C_SLAVE_PRESCALE_4	Slave Standard-Mode: at least 250 ns +10% data set-up time with SYSCLK = 12 MHz	0x3
	I2C_SLAVE_PRESCALE_5	Slave Standard-Mode: at least 250 ns +10% data set-up time with SYSCLK = 16 MHz	0x4
	I2C_SLAVE_PRESCALE_6	Slave Standard-Mode: at least 250 ns +10% data set-up time with SYSCLK = 20 MHz; Slave Fast-Mode: at least 100 ns +10% data set-up time with SYSCLK = 48 MHz	0x5
	I2C_SLAVE_PRESCALE_7	Slave Standard-Mode: at least 250 ns +10% data set-up time with SYSCLK = 24 MHz	0x6
	I2C_SLAVE_PRESCALE_14	Slave Standard-Mode: at least 250 ns +10% data set-up time with SYSCLK = 48 MHz	0xD
	I2C_SLAVE_PRESCALE_32	Maximum number of clock stretching	0x1F

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		cycles between SDA output and SCL release	
MASTER_PRESCALE	I2C_MASTER_PRESCALE_3	Master mode: prescale SCL from SYSCLK by 3	0x0*
	I2C_MASTER_PRESCALE_6	Master mode: prescale SCL from SYSCLK by 6	0x1
	I2C_MASTER_PRESCALE_9	Master mode: prescale SCL from SYSCLK by 9	0x2
	I2C_MASTER_PRESCALE_12	Master mode: prescale SCL from SYSCLK by 12	0x3
	I2C_MASTER_PRESCALE_15	Master mode: prescale SCL from SYSCLK by 15	0x4
	I2C_MASTER_PRESCALE_18	Master mode: prescale SCL from SYSCLK by 18	0x5
	I2C_MASTER_PRESCALE_21	Master mode: prescale SCL from SYSCLK by 21	0x6
	I2C_MASTER_PRESCALE_24	Master mode: prescale SCL from SYSCLK by 24	0x7
	I2C_MASTER_PRESCALE_27	Master mode: prescale SCL from SYSCLK by 27	0x8
	I2C_MASTER_PRESCALE_30	Master mode: prescale SCL from SYSCLK by 30	0x9
	I2C_MASTER_PRESCALE_33	Master mode: prescale SCL from SYSCLK by 33	0xA
	I2C_MASTER_PRESCALE_36	Master mode: prescale SCL from SYSCLK by 36	0xB
	I2C_MASTER_PRESCALE_39	Master mode: prescale SCL from SYSCLK by 39	0xC
	I2C_MASTER_PRESCALE_42	Master mode: prescale SCL from SYSCLK by 42	0xD
	I2C_MASTER_PRESCALE_45	Master mode: prescale SCL from SYSCLK by 45	0xE
	I2C_MASTER_PRESCALE_48	Master mode: prescale SCL from SYSCLK by 48	0xF
	I2C_MASTER_PRESCALE_51	Master mode: prescale SCL from SYSCLK by 51	0x10
	I2C_MASTER_PRESCALE_54	Master mode: prescale SCL from SYSCLK by 54	0x11

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	I2C_MASTER_PRESCALE_57	Master mode: prescale SCL from SYSCLK by 57	0x12
	I2C_MASTER_PRESCALE_60	Master mode: prescale SCL from SYSCLK by 60	0x13
	I2C_MASTER_PRESCALE_120	Master mode: prescale SCL from SYSCLK by 120	0x27
	I2C_MASTER_PRESCALE_768	Master mode: prescale SCL from SYSCLK by 768	0xFF
SLAVE	I2C_SLAVE_DISABLE	Disable I2C interface slave mode operation	0x0*
	I2C_SLAVE_ENABLE	Enable I2C interface slave mode operation	0x1

18.4.5.11 I2C*_CTRL

Bit Field	Read/Write	Field Name	Description
9	R	LAST_DATA_STATUS	I2C last data status
8	R	ENABLE_STATUS	I2C enable status
6	W	LAST_DATA	Indicate that the current data is the last byte of a data transfer
5	W	STOP	Issue a stop condition on the I2C interface bus
4	W	NACK	Issue a not acknowledge on the I2C interface bus
3	W	ACK	Issue an acknowledge on the I2C interface bus
2	W	RESET	Reset the I2C interface
1	W	DISABLE	Disable the I2C interface
0	W	ENABLE	Enable the I2C interface

Field Name	Value Symbol	Value Description	Hex Value
LAST_DATA_STATUS	I2C_LAST_DATA_DISABLED	I2C last data is disabled	0x0*
	I2C_LAST_DATA_ENABLED	I2C last data is enabled	0x1
ENABLE_STATUS	I2C_STATUS_DISABLED	I2C interface is disabled	0x0*
	I2C_STATUS_ENABLED	I2C interface is enabled	0x1
LAST_DATA	I2C_LAST_DATA	Indicate that the current data is the last byte of a data transfer	0x1
STOP	I2C_STOP	Issue a stop condition on the I2C interface bus	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
NACK	I2C_NACK	Issue a not acknowledge on the I2C interface bus	0x1
ACK	I2C_ACK	Issue an acknowledge on the I2C interface bus	0x1
	I2C_RESUME	Resume I2C interface after receiving address ACK/NACK in master read auto ack disable mode	0x1
RESET	I2C_RESET	Reset the I2C interface	0x1
DISABLE	I2C_DISABLE	Disable the I2C interface	0x1
ENABLE	I2C_ENABLE	Enable the I2C interface	0x1

18.4.5.12 I2C*_ADDR_START

Bit Field	Read/Write	Field Name	Description
7:1	RW	ADDRESS	I2C address to use for the transaction
0	RW	READ_WRITE	Select whether a read or a write transaction is started

Field Name	Value Symbol	Value Description	Hex Value
READ_WRITE	I2C_START_WRITE	Start an I2C write transaction	0x0*
	I2C_START_READ	Start an I2C read transaction	0x1

18.4.5.13 I2C*_STATUS

Bit Field	Read/Write	Field Name	Description
26	R	STOP_OR_REPEATED_START_DETECTED	Indicate if STOP_DETECTED or REPEATED_START_DETECTED bit is set
25	R	REPEATED_START_DETECTED	Indicate if an I2C repeated start has been detected during an active transaction in slave mode
22	R	BUS_ERROR	Bus error status bit
21	R	BUSY	Indicate that the reception or transmission of the data is ongoing
20	R	START_PENDING	Master frame start pending status bit
19	R	MASTER_MODE	Master mode status bit
18	R	STOP_DETECTED	Indicate if an I2C stop bit has been detected
17	R	DATA_EVENT	Indicate that I2C interface either needs data to transmit or has received data

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
16	R	TX_REQ	Indicate that a TX data can be written
15	R	RX_REQ	Indicate that a RX data can be read
14	R	CLK_STRETCH	Clock stretching flag
13	R	LINE_FREE	Line free flag
12	R	ADDR_DATA	Address / Data byte
11	R	READ_WRITE	Read / Write frame
10	R	GEN_CALL	General call flag
9	R	ACK	Acknowledge status
8	R	OVERRUN	Indicate that an overrun has occurred when receiving data
4	W	TX_REQ_SET	Set TX_REQ status flag
3	W	REPEATED_START_DETECTED_CLEAR	Clear REPEATED_START_DETECTED status flag
2	W	STOP_DETECTED_CLEAR	Clear STOP_DETECTED status flag
1	W	BUS_ERROR_CLEAR	Clear BUS_ERROR status flag
0	W	OVERRUN_CLEAR	Clear OVERRUN status flag

Field Name	Value Symbol	Value Description	Hex Value
STOP_OR_REPEATED_START_DETECTED	I2C_STOP_NOR_REPEATED_START_DETECTED	Neither STOP_DETECTED nor REPEATED_START_DETECTED bits are set	0x0*
	I2C_STOP_OR_REPEATED_START_DETECTED	STOP_DETECTED or REPEATED_START_DETECTED bit is set	0x1
REPEATED_START_DETECTED	I2C_NO_REPEATED_START_DETECTED	No repeated start condition has been detected on the I2C bus during an active transaction in slave mode	0x0*
	I2C_REPEATED_START_DETECTED	A repeated start condition has been detected on the I2C bus during an active transaction in slave mode	0x1
BUS_ERROR	I2C_NO_BUS_ERROR	No I2C bus error has occurred	0x0*
	I2C_BUS_ERROR	An I2C bus error has occurred	0x1
BUSY	I2C_IDLE	I2C interface is idle	0x0*
	I2C_BUSY	I2C interface is busy	0x1
START_PENDING	I2C_START_NOT_PENDING	No pending master start frame	0x0*
	I2C_START_PENDING	A master frame is pending to start (bit is	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		set when I2C_ADDR_START is written)	
MASTER_MODE	I2C_MASTER_INACTIVE	I2C interface is not operating in master mode	0x0*
	I2C_MASTER_ACTIVE	I2C interface is operating in master mode	0x1
STOP_DETECTED	I2C_NO_STOP_DETECTED	No stop condition has been detected on the I2C bus	0x0*
	I2C_STOP_DETECTED	A stop condition has been detected on the I2C bus	0x1
DATA_EVENT	I2C_NON_DATA_EVENT	No I2C data is needed or available	0x0*
	I2C_DATA_EVENT	I2C data is needed or is available	0x1
TX_REQ	I2C_TX_NO_REQ	I2C TX data has already been written	0x0
	I2C_TX_REQ	I2C TX data can be written	0x1*
RX_REQ	I2C_RX_NO_REQ	No new I2C RX data available	0x0*
	I2C_RX_REQ	New I2C RX data available	0x1
CLK_STRETCH	I2C_CLK_NOT_STRETCHED	I2C interface is currently not clock stretching	0x0*
	I2C_CLK_STRETCHED	I2C interface is currently clock stretching	0x1
LINE_FREE	I2C_BUS_BUSY	I2C transaction ongoing	0x0
	I2C_BUS_FREE	I2C bus is free	0x1*
ADDR_DATA	I2C_DATA_IS_DATA	The I2C data register holds data	0x0*
	I2C_DATA_IS_ADDR	The I2C data register holds an address	0x1
READ_WRITE	I2C_IS_WRITE	The current I2C transfer is a write	0x0*
	I2C_IS_READ	The current I2C transfer is a read	0x1
GEN_CALL	I2C_ADDR_OTHER	The address used for the current I2C transfer is not the general call address	0x0*
	I2C_ADDR_GEN_CALL	The address used for the current I2C transfer is the general call address	0x1
ACK	I2C_HAS_ACK	Indicate that the last I2C byte was acknowledged	0x0*
	I2C_HAS_NACK	Indicate that the last I2C byte was not acknowledged	0x1
OVERRUN	I2C_OVERRUN_FALSE	No I2C input overrun detected	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	I2C_OVERRUN_TRUE	I2C input overrun detected	0x1
TX_REQ_SET	I2C_TX_REQ_SET	Set the TX_REQ status flag	0x1
REPEATED_START_DETECTED_CLEAR	I2C_REPEATED_START_DETECTED_CLEAR	Clear the REPEATED_START_DETECTED status flag	0x1
STOP_DETECTED_CLEAR	I2C_STOP_DETECTED_CLEAR	Clear the STOP_DETECTED status flag	0x1
BUS_ERROR_CLEAR	I2C_BUS_ERROR_CLEAR	Clear the BUS_ERROR status flag	0x1
OVERRUN_CLEAR	I2C_OVERRUN_CLEAR	Clear the OVERRUN status flag	0x1

18.4.5.14 I2C*_TX_DATA

Bit Field	Read/Write	Field Name	Description
7:0	RW	TX_DATA	Single byte buffer for data transmitted over the I2C interface

18.4.5.15 I2C*_RX_DATA

Bit Field	Read/Write	Field Name	Description
7:0	R	RX_DATA	Single byte buffer for data received over the I2C interface

18.4.5.16 I2C*_RX_DATA_MIRROR

Bit Field	Read/Write	Field Name	Description
7:0	R	RX_DATA	Mirror of the single byte buffer for data received over the I2C interface. Does not clear RX_REQ in I2C_STATUS

18.5 LOW-SPEED ANALOG-TO-DIGITAL (LSAD)

The analog-to-digital converters (ADCs) provide an analog to digital conversion of up to eight signals, from a combination of four internal signals and four external signals. Each conversion is a single-ended measurement with 14 bits of resolution for the converter, at sampling rates up to 8 kHz.

IMPORTANT: For accurate ADC measurements across operating conditions, the reference VREG supply voltage level must be calibrated to 0.9 V. After verifying that VREG is calibrated correctly, the gain of the LSAD can be calibrated to ensure accurate LSAD measurements. For more information about VREG and LSAD calibration, see [Section 9.1.2.2 “Internal Digital Supply Voltages \(VDDC and VDDM\)”](#) on page 291, and the *Ezairo 8300 Firmware Reference Manual*.

18.5.1 LSAD Input Configuration

The purpose of the LSADs is to sample analog signals that are relevant to the user's application use cases—for example, the voltage associated with a potentiometer-based volume control, or a supply voltage for battery monitoring applications.

Ezairo 8300 Hardware Reference

The LSAD must be enabled when using the VDDC/VDDM charge pumps. See [Section 9.1.2.2 “Internal Digital Supply Voltages \(VDDC and VDDM\)” on page 291](#).

The fixed signals measured by the LSAD block are VREG (used for offset compensation), VBAT, VDDC, and VDDM. The configurable signals measured by the LSAD block are configured using the `LSAD_INPUT_SEL_CH_INPUT_SEL` bit-field from the `LSAD_INPUT_SEL_CH*` registers. The signal used for each measurement channel is selected from:

DIOs 8-11, 20-23

For more information about DIO configuration for LSADs, see [Chapter 16.1 "Overview" on page 463](#).

VSSA

Analog ground supply reference.

VMIC

The microphone biasing voltage. For more information about VMIC, see [Section 9.1.3.2 “Microphone Supply Voltage \(VMIC\) and Microphone Ground Reference \(GND_MIC\)” on page 297](#).

VDDOD

The output driver regulator voltage. For more information about VDDOD and VBATOD, see [Section 9.1.1.2 “Output Receiver Supply Voltage \(VBATOD\)” on page 290](#).

VDDA

The analog supply voltage. For more information about the VDDA supply, see [Section 9.1.2.4 “Analog Supply Voltage \(VDDA\)” on page 295](#).

VDDIF

The interface supply voltage. For more information about the VDDIF supply, see [Section 9.1.3.1 “Regulated Interface Supply Voltage \(VDDIF\)” on page 296](#).

VTEMP

The temperature reference from the bandgap supply voltage (VREF). For more information about the VREF supply, see [Section 9.1.2.1 “Internal Band Gap Reference Voltage \(VREF\)” on page 291](#).

NOTE: The sampling rate for a specific signal can be increased by assigning the same input to more than one configurable channel.

NOTE: When the LSAD block is stopped, it is recommended that you reset the source configuration register to its default state, to avoid leaving DIOs connected to the LSAD inputs.

18.5.2 LSAD Sampling Configuration

The eight channels measured by the LSAD are sampled at a nominal sampling rate of up to 1 kHz per channel. The LSAD block runs at a nominal frequency of 128 kHz, derived using a fixed prescaler of 10 from SLOWCLK. It takes 16 or more cycles to produce each LSAD sample.

LSAD channels are sampled sequentially, in the following order:

Ezairo 8300 Hardware Reference

1. Configurable channel 0 (CH0)
2. VREG
3. Configurable channel 1 (CH1)
4. VBAT
5. Configurable channel 2 (CH2)
6. VDDC
7. Configurable channel 3 (CH3)
8. VDDM

NOTE: When the LSAD is disabled, the state machine is reset automatically. When the LSAD is re-enabled, conversion starts again at the first slot (CH0).

Configuration of SLOWCLK is described in [Section 10.2.4 “Slow Clock \(SLOWCLK\)” on page 315](#), with SLOWCLK typically initialized to 1.28 MHz. The LSAD sample rate is configured by setting the `LSAD_CFG_FREQ` bit-field from the `LSAD_CFG` register. The selected sample rate affects the native voltage range of measurements, with higher sample rates providing a larger input range, and lower sample rates providing a higher resolution LSAD measurement. The resulting sample rates per channel as a function of SLOWCLK are shown in the "LSAD Sample Rate Configuration" table (Table 60), below.

Table 60. LSAD Sample Rate Configuration

Value	Setting	Fixed Divider	Configurable Division	Native Voltage Range (V)	Effective Division of SLOWCLK
0x0	LSAD_DISABLE	N/A	LSAD Disabled	N/A	LSAD Disabled
0x1	LSAD_PRESCALE_160	10	Divide by 16	-0.225 to 2.025	Divide by 1280
0x2	LSAD_PRESCALE_200	10	Divide by 20	-0.113 to 1.913	Divide by 1600
0x3	LSAD_PRESCALE_400	10	Divide by 40	-0.056 to 1.856	Divide by 3200
0x4	LSAD_PRESCALE_640	10	Divide by 64	-0.028 to 1.828	Divide by 5120
0x5	LSAD_PRESCALE_800	10	Divide by 80	-0.014 to 1.814	Divide by 6400
0x6	LSAD_PRESCALE_1600	10	Divide by 160	-0.007 to 1.807	Divide by 12800
0x7	LSAD_PRESCALE_3200	10	Divide by 320	-0.004 to 1.804	Divide by 25600

NOTE: For a typical SLOWCLK frequency of 1.28 MHz, the LSAD samples all eight channels at a configurable rate between 1 kHz and 50 Hz.

18.5.3 LSAD Output Data

The analog to digital conversion results from the LSAD are signed 15-bit values, relative to VREG in the range $-2 \times \text{VREG}$ to $2 \times \text{VREG}$. These output measurement results have an effective resolution of 14 bits.

The converted output from the most recent conversion of each fixed channel is provided in the following registers:

- VBAT is measured relative to VSSA and stored to the `LSAD_DATA_VBAT` register, in an unsigned 15-bit format.
- VDDC is measured relative to VSSA and stored to the `LSAD_DATA_VDDC` register, in an unsigned 13-bit format. If VDDC exceeds this range, it is saturated positively.

Ezairo 8300 Hardware Reference

- VDDM is measured relative to VSSA and stored to the `LSAD_DATA_VDDM` register, in an unsigned 13-bit format. If VDDM exceeds this range, it is saturated positively.

Each channel has four register sets that hold the most recent converted measurement from the channel. Each register in the set presents a different representation of the actual measured result. These register sets are:

`LSAD_DATA_AUDIO_CH*`

The output from the LSAD channel; data is output as a signed 15-bit value, which is relative to VREG in the range $-VREG$ to $3 \times VREG$. The output is scaled relative to `0xFFFFE000` ($VSSA$) and `0x00001FFF` ($2 \times VREG$).

`LSAD_DATA_AUDIO_SAT_CH*`

The LSAD channel data is output as a signed 14-bit value, which is relative to VREG saturated to the range $VSSA$ to $2 \times VREG$. The output is scaled relative to `0xFFFFE000` ($VSSA$) and `0x00001FFF` ($2 \times VREG$).

`LSAD_DATA_TRIM_CH*`

The LSAD channel data is output as a signed 15-bit value, which is relative to VREG saturated to the range $-VREG$ to $3 \times VREG$. The output is scaled relative to `0x00000000` ($VSSA$) and `0x00003FFF` ($2 \times VREG$).

`LSAD_DATA_TRIM_SAT_CH*`

The LSAD channel data is output as an unsigned 14-bit value, which is relative to VREG saturated to the range $VSSA$ to $2 \times VREG$. The output is scaled relative to `0x00000000` ($VSSA$) and `0x00003FFF` ($2 \times VREG$).

IMPORTANT: When enabling the LSAD, the first conversion result (configurable channel 0) is not valid and needs to be ignored.

The LSAD has an internal mechanism which uses VREG conversions and the loaded `LSAD_OFFSET_COMP` value to automatically compensate for offset errors ranging from -56 mV to 56 mV for all other LSAD measurements. This compensation allows for correcting the trimming inaccuracy of VREG, in order to have an input of 0.9 V correspond to a conversion value of zero in the `LSAD_DATA_AUDIO_*` output registers. The `LSAD_OFFSET_COMP` register needs to be loaded with a calibrated value before the LSAD is used in the application.

The output of the LSAD channels can be scaled using a gain compensation factor stored to the `LSAD_GAIN_COMP` register. The precision of the LSAD's conversion slope can be improved by calibrating the gain compensation factor stored in the `LSAD_GAIN_COMP` register. This value is multiplied by the measurement result for all LSAD channels except the offset compensation register, after applying offset compensation, and the output from this multiplication is stored to the various output registers.

18.5.4 Power Supply Monitoring

The LSAD block provides resources that can be used to monitor the power supplied to VBAT. From this voltage information, you can detect when the battery is getting close to its end of life.

The supply threshold level can be defined between 0 V and 1.8 V using 256 steps of approximately 7 mV. This is defined in the `LSAD_BATMON_CFG_SUPPLY_THRESHOLD` bit field in the `LSAD_BATMON_CFG` register.

Ezairo 8300 Hardware Reference

When the monitored supply is measured as being below the specified threshold, the value of the `LSAD_BATMON_COUNT_VAL` register is incremented. This counter register is reset when read. The value of this register is compared with the value stored to the `LSAD_BATMON_CFG_ALARM_COUNT_VALUE` bit-field from the `LSAD_BATMON_CFG` register, and if the counter value ever exceeds the alarm value, an alarm is generated.

18.5.5 LSAD and Power Supply Monitoring Interrupt

A single interrupt line is shared between the LSAD and the power supply monitoring circuitry. This interrupt can be independently configured to trigger when one or both of the following conditions are met:

- When a specified LSAD channel is sampled. To enable this trigger condition, use the `LSAD_INT_ENABLE` bit from the `LSAD_CFG` register. To specify the LSAD channel, use the `LSAD_CFG_INT_SEL` bit-field from this same register.
- When a power supply monitor alarm occurs.

The source of the interrupt can always be determined using fields within the `LSAD_BATMON_STATUS` register. The `LSAD_BATMON_STATUS_BATMON_ALARM_STAT` bit indicates an alarm condition within the battery monitoring circuit. The `LSAD_BATMON_STATUS_LSAD_READY_STAT` bit indicates that a new set of samples is available in the LSAD output registers.

NOTE: The flags in the `LSAD_BATMON_STATUS` register are sticky, and each must be separately cleared by setting the appropriate `LSAD_BATMON_STATUS_*_CLEAR` bit from the `LSAD_BATMON_STATUS` register.

See [Section 4.6.1 “Nested Vectored Interrupt Controller \(NVIC\)” on page 135](#) for information regarding interrupt configuration and handling interrupts for the Arm Cortex-M3 processor. See [Section 3.2 “Interrupt Controller” on page 61](#) for information regarding interrupt configuration and handling for the CFX DSP.

18.5.6 LSAD Gain Compensation Calibration

The LSAD gain compensation value can be calculated after bandgap trimming has been calibrated for a VREG of 0.9 V. This is required to ensure that the LSAD's reference (VREG) for conversions is accurate. Refer to the Calibration Library chapter of the Ezairo 8300 Firmware Reference for more information about VREG calibration.

After VREG has been calibrated, power VBAT with 1.7 V and save the LSAD code for VBAT. The following equation can then be used for calculating the LSAD gain compensation:

$$gain = \left[\left(\frac{[code(VBAT1.7V) - 8192]}{(7 \times 8192)} \times 9 \right) - 1 \right] \times \left(\frac{8192}{8} \right)$$

The result must be saturated to a 6-bit signed value and then copied to the `LSAD_GAIN_COMP` register before LSAD is used in the application.

onsemi Confidential
Ezairo 8300 Hardware Reference

18.5.7 LSAD Registers

Register Name	Register Description	Address
LSAD_DATA_AUDIO_CH	LSAD Conversion Result for Channel 0 to 3 in audio mode (signed)	P:0x1480- P:0x1483 0x40001200- 0x4000120C
LSAD_DATA_AUDIO_SAT_CH	LSAD Conversion Result for Channel 0 to 3 in saturated audio mode (signed)	P:0x1484- P:0x1487 0x40001210- 0x4000121C
LSAD_DATA_TRIM_CH	LSAD Conversion Result for Channel 0 to 3 in trimmer mode	P:0x1488- P:0x148B 0x40001220- 0x4000122C
LSAD_DATA_TRIM_SAT_CH	LSAD Conversion Result for Channel 0 to 3 in saturated trimmer mode	P:0x148C- P:0x148F 0x40001230- 0x4000123C
LSAD_INPUT_SEL_CH	LSAD Input Selection for Channel 0 to 3	P:0x1490- P:0x1493 0x40001240- 0x4000124C
LSAD_DATA_VBAT	LSAD Conversion Result for VBAT	P:0x1494 0x40001250
LSAD_DATA_VDDC	LSAD Conversion Result for VDDC	P:0x1495 0x40001254
LSAD_DATA_VDDM	LSAD Conversion Result for VDDM	P:0x1496 0x40001258
LSAD_CFG	LSAD Configuration Register	P:0x1498 0x40001260
LSAD_OFFSET_COMP	LSAD Offset Compensation	P:0x1499 0x40001264
LSAD_GAIN_COMP	LSAD Gain Compensation	P:0x149A

onsemi Confidential
Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
		0x40001268
LSAD_BATMON_CFG	Battery Monitoring Configuration Register	P:0x14A0 0x40001280
LSAD_BATMON_COUNT_VAL	Battery Monitoring Status Register	P:0x14A1 0x40001284
LSAD_BATMON_STATUS	LSAD / BATMON Status Register	P:0x14A2 0x40001288

18.5.7.1 LSAD_DATA_AUDIO_CH*

Bit Field	Read/Write	Field Name	Description
31:0	R	DATA	15-bit signed LSAD conversion result (sign extended to 32 bits)

18.5.7.2 LSAD_DATA_AUDIO_SAT_CH*

Bit Field	Read/Write	Field Name	Description
31:0	R	DATA	14-bit signed LSAD conversion result (sign extended to 32 bits)

18.5.7.3 LSAD_DATA_TRIM_CH*

Bit Field	Read/Write	Field Name	Description
31:0	R	DATA	16-bit signed LSAD conversion result (sign extended to 32 bits)

18.5.7.4 LSAD_DATA_TRIM_SAT_CH*

Bit Field	Read/Write	Field Name	Description
13:0	R	DATA	14-bit unsigned LSAD conversion result

18.5.7.5 LSAD_INPUT_SEL_CH*

Bit Field	Read/Write	Field Name	Description
3:0	RW	INPUT_SEL	Select channel source

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
INPUT_SEL	LSAD_INPUT_VSSA	Select VSSA as LSAD input	0x0*
	LSAD_INPUT_VMIC	Select VMIC as LSAD input	0x1
	LSAD_INPUT_VDDOD	Select VDDOD as LSAD input	0x2
	LSAD_INPUT_VDDA	Select VDDA as LSAD input	0x3
	LSAD_INPUT_VDDIF	Select VDDIF as LSAD input	0x4
	LSAD_INPUT_VTEMP	Select VTEMP as LSAD input	0x5
	LSAD_INPUT_DIO8	Select DIO8 pad as LSAD input	0x8
	LSAD_INPUT_DIO9	Select DIO9 pad as LSAD input	0x9
	LSAD_INPUT_DIO10	Select DIO10 pad as LSAD input	0xA
	LSAD_INPUT_DIO11	Select DIO11 pad as LSAD input	0xB
	LSAD_INPUT_DIO20	Select DIO20 pad as LSAD input	0xC
	LSAD_INPUT_DIO21	Select DIO21 pad as LSAD input	0xD
	LSAD_INPUT_DIO22	Select DIO22 pad as LSAD input	0xE
	LSAD_INPUT_DIO23	Select DIO23 pad as LSAD input	0xF

18.5.7.6 LSAD_DATA_VBAT

Bit Field	Read/Write	Field Name	Description
14:0	R	DATA	15-bit unsigned LSAD conversion result

18.5.7.7 LSAD_DATA_VDDC

Bit Field	Read/Write	Field Name	Description
12:0	R	DATA	13-bit unsigned LSAD conversion result

18.5.7.8 LSAD_DATA_VDDM

Bit Field	Read/Write	Field Name	Description
12:0	R	DATA	13-bit unsigned LSAD conversion result

18.5.7.9 LSAD_CFG

Bit Field	Read/Write	Field Name	Description
7:5	RW	INT_SEL	Select which conversion is triggering the LSAD interrupt
4	RW	INT_ENABLE	LSAD new sample ready interrupt mask
2:0	RW	FREQ	Defines the sampling frequency of the LSAD channels

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
INT_SEL	LSAD_INT_CH0	The LSAD interrupt is triggered when the LSAD CH0 data registers are updated	0x0*
	LSAD_INT_OFFSET	The LSAD interrupt is triggered when the LSAD offset compensation register is updated automatically	0x1
	LSAD_INT_CH1	The LSAD interrupt is triggered when the LSAD CH1 data registers are updated	0x2
	LSAD_INT_VBAT	The LSAD interrupt is triggered when the LSAD VBAT data register is updated	0x3
	LSAD_INT_CH2	The LSAD interrupt is triggered when the LSAD CH2 data registers are updated	0x4
	LSAD_INT_VDDC	The LSAD interrupt is triggered when the LSAD VDDC data register is updated	0x5
	LSAD_INT_CH3	The LSAD interrupt is triggered when the LSAD CH3 data registers are updated	0x6
	LSAD_INT_VDDM	The LSAD interrupt is triggered when the LSAD VDDM data register is updated	0x7
INT_ENABLE	LSAD_INT_DISABLE	No LSAD interrupt is raised	0x0*
	LSAD_INT_ENABLE	LSAD interrupt is raised when a new sample is ready for the selected channel	0x1
FREQ	LSAD_DISABLE	LSAD disabled	0x0*
	LSAD_PRESCALE_160	Sample rate is SLOWCLK/160 (low frequency mode)	0x1
	LSAD_PRESCALE_200	Sample rate is SLOWCLK/200 (low frequency mode)	0x2
	LSAD_PRESCALE_400	Sample rate is SLOWCLK/400 (low frequency mode)	0x3
	LSAD_PRESCALE_640	Sample rate is SLOWCLK/640 (low frequency mode)	0x4

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	LSAD_PRESCALE_800	Sample rate is SLOWCLK/800 (low frequency mode)	0x5
	LSAD_PRESCALE_1600	Sample rate is SLOWCLK/1600 (low frequency mode)	0x6
	LSAD_PRESCALE_3200	Sample rate is SLOWCLK/3200 (low frequency mode)	0x7

18.5.7.10 LSAD_OFFSET_COMP

Bit Field	Read/Write	Field Name	Description
31:0	R	OFFSET_COMP_RD	10-bit LSAD sign extended offset compensation (from -56 to +56 mV in steps of 0.11 mV, signed extended to 32 bits)
9:0	RW	OFFSET_COMP	10-bit LSAD signed offset compensation (from -56 to +56 mV in steps of 0.11 mV)

18.5.7.11 LSAD_GAIN_COMP

Bit Field	Read/Write	Field Name	Description
31:0	R	GAIN_COMP_RD	6-bit LSAD signed gain compensation (from -256/8192 to +248/8192 in steps of 8/8192, signed extended to 32 bits)
5:0	RW	GAIN_COMP	6-bit LSAD signed gain compensation (from -256/8192 to +248/8192 in steps of 8/8192)

18.5.7.12 LSAD_BATMON_CFG

Bit Field	Read/Write	Field Name	Description
15:8	RW	ALARM_COUNT_VALUE	An Alarm Status bit is set when SUPPLY_COUNT_VALUE=ALARM_COUNT_VALUE
7:0	RW	SUPPLY_THRESHOLD	Low voltage detection threshold (7 mV steps)

Field Name	Value Symbol	Value Description	Hex Value
ALARM_COUNT_VALUE	BATMON_ALARM_NONE	No Alarm is triggered	0x0*
	BATMON_ALARM_COUNT1	Alarm count value is 1	0x1
	BATMON_ALARM_COUNT255	Alarm count value is 255	0xFF
SUPPLY_THRESHOLD	BATMON_ALARM_0V95	Threshold is 0.95 V	0x87
	BATMON_ALARM_1V00	Threshold is 1.00 V	0x8E
	BATMON_ALARM_1V05	Threshold is 1.05 V	0x95*
	BATMON_ALARM_1V10	Threshold is 1.10 V	0x9C

Ezairo 8300 Hardware Reference

18.5.7.13 LSAD_BATMON_COUNT_VAL

Bit Field	Read/Write	Field Name	Description
7:0	R	SUPPLY_COUNT_VALUE	Number of times the battery voltage has fallen below the battery monitor voltage threshold. The counter is reset when read

18.5.7.14 LSAD_BATMON_STATUS

Bit Field	Read/Write	Field Name	Description
9	R	BATMON_ALARM_STAT	Battery monitoring alarm status bit
8	R	LSAD_READY_STAT	LSAD new sample ready status bit
1	W	BATMON_ALARM_CLEAR	Battery monitoring alarm status bit
0	W	LSAD_READY_CLEAR	LSAD new sample ready status bit

Field Name	Value Symbol	Value Description	Hex Value
BATMON_ALARM_STAT	BATMON_ALARM_FALSE	BATMON Alarm flag not set	0x0*
	BATMON_ALARM_TRUE	BATMON Alarm has been triggered	0x1
LSAD_READY_STAT	LSAD_READY_FALSE	No new LSAD samples available	0x0*
	LSAD_READY_TRUE	New LSAD samples are ready	0x1
BATMON_ALARM_CLEAR	BATMON_ALARM_CLEAR	Writing a 1 clears the BATMON Alarm status bit	0x1
LSAD_READY_CLEAR	LSAD_READY_CLEAR	Writing a 1 clears the LSAD Ready status bit	0x1

18.6 PULSE CODE MODULATION (PCM) INTERFACES

Ezairo 8300 has access to two pulse code modulation (PCM) interfaces, which can be used to stream control, configuration or signal data into and out of the microcontroller.

Each PCM interface connects to the processors through the ARM Cortex-M3 processor's peripheral bus. There are three possible ways PCM interfaces can handle transmission and reception buffers:

- By using the internally available data transmission and reception interrupts. The PCM transmission and reception interrupts can be enabled by setting the `PCM_CFG_RX_TX_INT_ENABLE` bit in the `PCM*_CFG` registers.
- By connecting to the DMA with two channels supporting transmit and receive operations. The data request lines to the DMA can be enabled by setting the `PCM_CFG_TX_DMA_ENABLE` and `PCM_CFG_RX_DMA_ENABLE` bits in the `PCM*_CFG` registers.
- By connecting to the IOCs with four FIFOs supporting transmit and receive operations. Each data channel for PCM transmit and receive buffers requires its own FIFO—as opposed to the case for using DMA, which allows one DMA channel to support both channel 0 and channel 1 for PCM. The data request lines to IOCs can be enabled by setting the `PCM_CFG_TX_IOC_ENABLE` and `PCM_CFG_RX_IOC_ENABLE` bits in the `PCM*_CFG` registers. PCM0 can only be connected to IOC0; similarly, PCM1 can only be connected to IOC1.

Ezairo 8300 Hardware Reference

The "High-Level PCM Interface Module Block Diagram" figure (Figure 35) shows a block diagram of the PC interface.

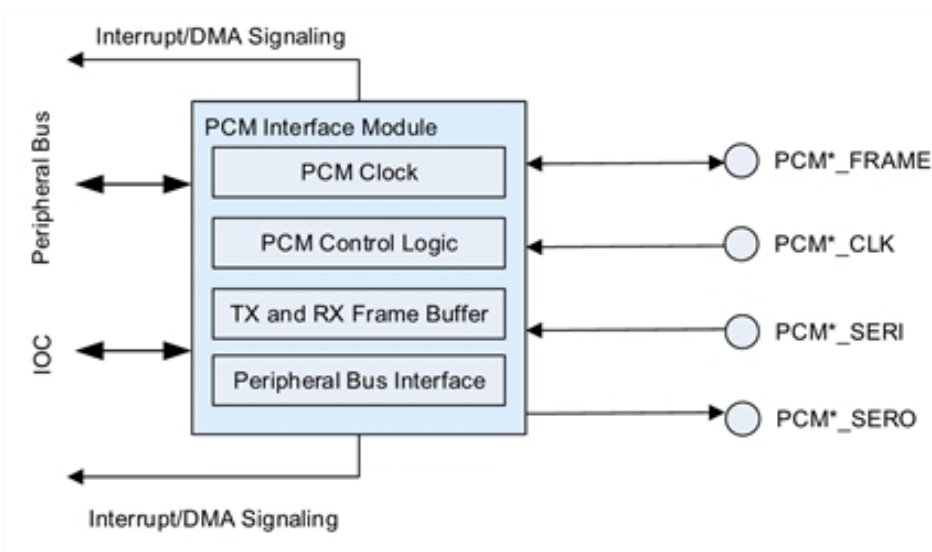


Figure 35. High-Level PCM Interface Module Block Diagram

Each PCM interface is multiplexed onto the DIO pads, which can be configured for the input and output signals that form the PCM interface with the necessary physical pad configuration. For more information about configuring the multiplexed DIO functionality, see [Section 16.2 “Functional Configuration” on page 464](#).

These interfaces make use of four external signals in communications. These signals are:

- An interface clock signal (PCM*_CLK)
- A bidirectional frame signal (PCM*_FRAME)
- A serial output data signal (PCM*_SERO)
- A serial input data signal (PCM*_SERI)

CAUTION: Disabling the bus pull-up and pull-down resistors is not recommended for the PCM clock unless this clock is sourced within the Ezairo 8300 system. Similarly, disabling the bus pull-up and pull-down resistors is not recommended for the PCM frame signal in slave mode. For all other configurations, attempting to use this interface without the pull resistors can result in unintended interface behavior that would result in the PCM interface transmitting and/or receiving undefined data.

Each PCM interface uses the PCM_CTRL_ENABLE bit in the PCM*_CTRL registers to enable and disable the PCM interface. When using the PCM interface in any mode, set this bit and ensure that the proper DIO multiplexing has been selected.

NOTE: While the PCM interface is disabled, the internal data registers are held at their current state, and the internal clocks are gated. Enabling the interface resumes the state machine of the interface, and disabling the interface pauses the interface's state machine. Note that the DMA requests and the IOC requests are not gated by the interface enable, so a DMA/IOC request can be generated with the PCM interface disabled.

Ezairo 8300 Hardware Reference

Each PCM interface can be configured as either a master device (controlling the frame signal, and potentially providing the interface PCM_CLK through an internally routed DIO function), or a slave device (receiving the frame signal and usually not providing the PCM_CLK). To select between these configurations, configure the PCM_CTRL_SLAVE bit as appropriate in the PCM*_CTRL registers. In master mode, configure the PCM frame signal as an output; in slave mode, configure it as an input. The PCM clock signal is always an input to the PCM interface; this clock signal can be sourced in the following ways:

- Internally, selecting a clock source as the DIO output mode for the pad that is also used for the PCM clock, to route an internal clock signal within the Ezairo 8300 system to the same pad
- Externally, using an externally generated clock signal

The PCM clock frequency can be the same as the system clock frequency. External clocks can also be used to drive the PCM interfaces; these clocks can be sourced via an external PCM device, or one of the output clocks from the Ezairo 8300 system can be used (e.g. USRCLK).

Each PCM interface is sensitive to only one edge of the PCM clock. All settings are clocked in and signal updates are captured on the rising or falling edge, as specified by the PCM_CFG_CLK_POLARITY bit from the PCM*_CFG registers. By default, the PCM interface samples its inputs on the falling edge of the PCM clock, outputting its signals on the rising clock edge. When configuring a PCM interface for I²S standard compatibility, it is recommended that you configure the interface to sample on the rising edges of this clock when using it in I²S mode. This is done by setting the PCM_CFG_CLK_POLARITY bit in the PCM*_CFG registers.

IMPORTANT: The PCM interface configuration and internal status registers are tightly synchronized to the input PCM clock. As a result, the PCM interfaces are only reset, and the internal configurations of the PCM interfaces are only updated, on their selected PCM clock edges.

18.6.1 PCM Signal Configuration

Each PCM interface uses a large number of configuration options to help define a set of signals that the system can interpret. For the purpose of clarifying the following explanations, the signals generated by the system in the default PCM configuration are shown in the "Signal Timing Diagram: Default PCM Configuration" figure (Figure 36), below.

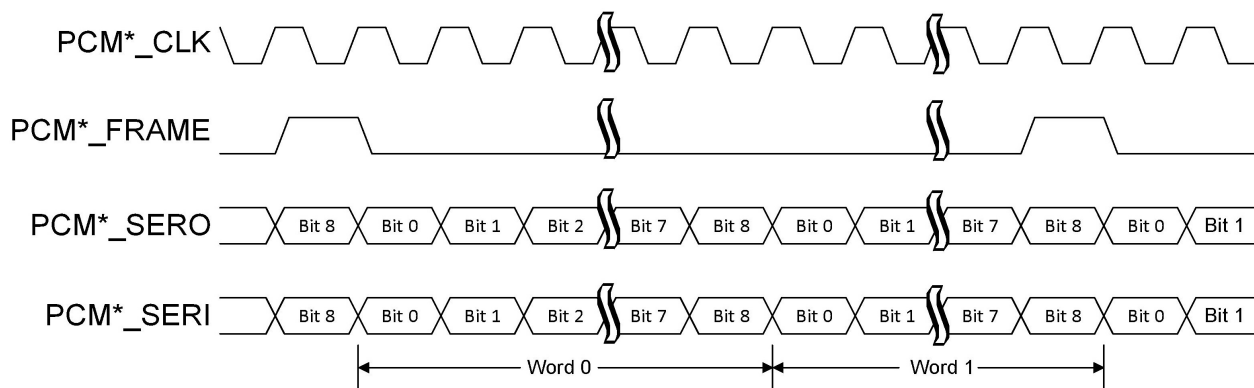


Figure 36. Signal Timing Diagram: Default PCM Configuration

Ezairo 8300 Hardware Reference

For more information about the required signal configuration and other interface configurations that are needed to allow either interface to be compatible with I²S, including an example timing diagram, see [Section 18.6.4 “I²S Configuration and Usage” on page 648](#).

IMPORTANT: To ensure defined behavior, the PCM interface needs to be in IDLE state when changing any configuration settings.

18.6.1.1 Frame Signal Configuration and Timing

Each PCM interface uses a signal, called a frame signal, to realign transmission over the PCM interface between two devices with every data frame transmitted.

The frame signal divides the communications on the PCM interface into data frames (or sub-frames, if the `PCM_CFG_SUBFRAME` bit of the `PCM*_CFG` registers is enabled). Each aspect of the PCM frame signal can be configured as follows (all bits are in the `PCM*_CFG` registers):

Interval

In the default configuration of the `PCM_CFG_FRAME_LENGTH` bit field (configured for two words per frame), and the `PCM_CFG_SUBFRAME` bit (configured to disable a frame signal being generated for each word within a frame), the frame signal is generated once for every two-word frame. Changing the `PCM_CFG_FRAME_LENGTH` bit field in PCM master mode allows the user to select the number of words per frame, incrementing in multiples of two words, and therefore the number of words per data frame. The `PCM_CFG_FRAME_LENGTH` bit field is not used for PCM slave mode; it needs to be left in the default configuration (configured for two words per frame), regardless of the actual frame length used, to produce the expected behavior. Alternately, if the `PCM_CFG_SUBFRAME` bit is configured to enable sub-frames, a frame signal is generated with every word of each data frame.

IMPORTANT: Configuring the `PCM_CFG_FRAME_LENGTH` bit field to something other than two words per frame, when setting the `PCM_CFG_SUBFRAME` bit, results in a configuration where the user receives a PCM frame signal for every sub-frame word. Despite the well-defined behavior of the frame signal generated for this configuration, the handling of data and interrupts by the PCM interfaces might not appear sensible and generally results in undefined or unexpected data.

Shape

The frame signal can be configured to take one of two shapes by setting the `PCM_CFG_FRAME_WIDTH` bit. By default, the frame signal is configured to produce a short width frame signal, which produces a single cycle pulse in the frame signal at the beginning of each frame. Alternatively, you can configure the signal to produce a long width frame signal, which uses a 50% duty cycle square wave and is spaced to align with the specified width between frame signal events.

Alignment

You can configure the `PCM*_FRAME` signal by using the `PCM_CFG_FRAME_ALIGN` bit, so that the first bit of sampled data is sampled at the same time as the frame signal, or the first bit of data is sampled one cycle after the frame signal. In this way, the frame signal is aligned with the first bit of the new data frame, or with the last bit of the previous data frame.

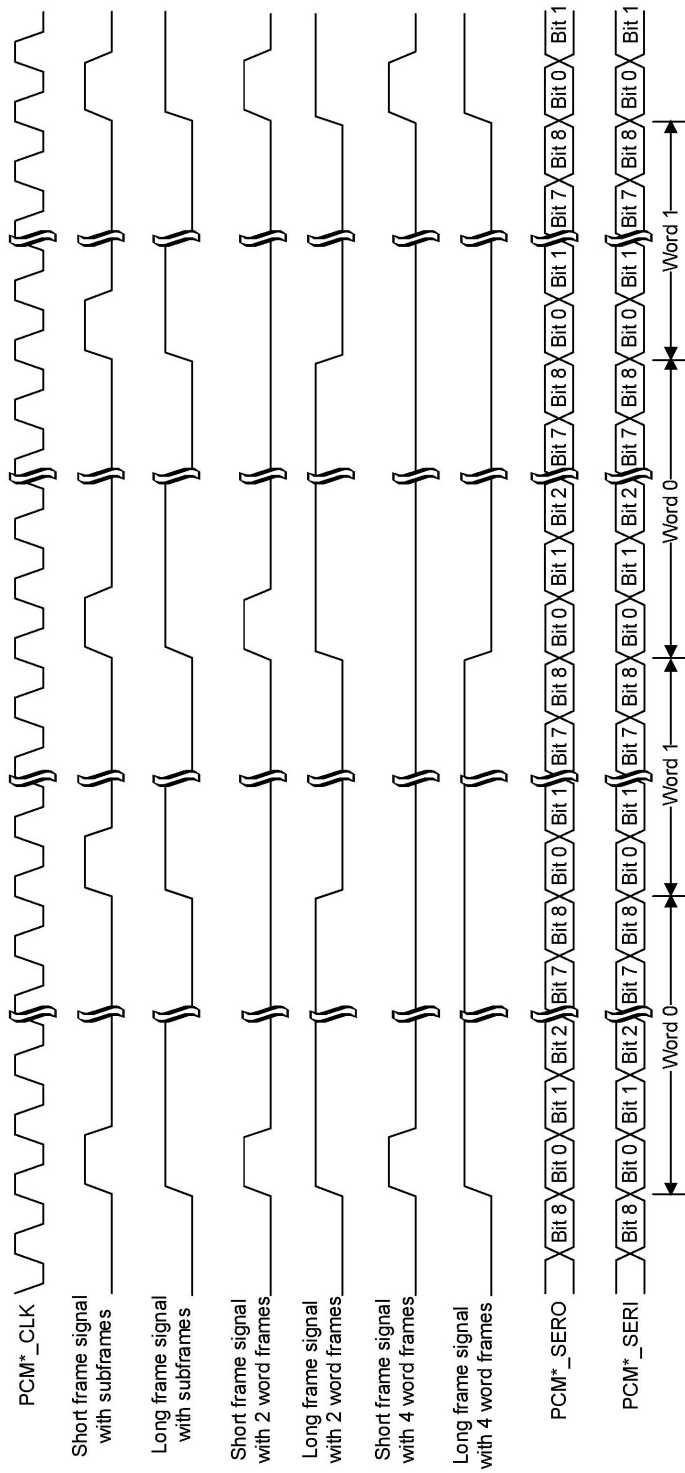
Ezairo 8300 Hardware Reference

NOTE: Regardless of the PCM*_FRAME signal alignment configuration, the PCM*_FRAME signal occurs at the beginning of each frame, and only one frame signal occurs for each frame received. The PCM*_FRAME signal, if aligned with the last bit, occurs one bit before the first data bit transmitted at the start of a transaction, and does not occur on the last bit of the last frame of a transaction. Similarly, if aligned with the first bit, the first frame signal is aligned with the first data bit transmitted at the start of a transaction, and no trailing frame signal occurs for a transaction.

IMPORTANT: To properly terminate a transmission, an expected frame signal pulse must be missed. When the frame signal is configured to be aligned with the first bit of a transmission, this requires one additional PCM*_CLK pulse following the transmission of the last bit of the last frame.

For clarification of the above PCM*_FRAME signal timing configuration explanations, several examples of different PCM*_FRAME signal traces are shown in the "Frame Signal Timing Examples" figure (Figure 37). In all cases the PCM_CFG_FRAME_ALIGN bit has been set to PCM_FRAME_ALIGN_FIRST (which differs from the default configuration shown in the "Signal Timing Diagram: Default PCM Configuration" figure (Figure 36)), to prevent any confusion as to the effects of the other frame signal configuration bits. The frame signal configurations shown are:

1. Configured to indicate the sub-frame words of a frame with a short pulse frame signal
2. Configured to indicate the sub-frame words of a frame with a long width frame signal
3. Configured to use a 2-word frame with a short pulse frame signal
4. Configured to use a 2-word frame with a long width frame signal
5. Configured to use a 4-word frame with a short pulse frame signal
6. Configured to use a 4-word frame with a long width frame signal



Ezairo 8300 Hardware Reference

The "PCM Frame Signal Configuration Options" table (Table 61) summarizes the options for PCM*_FRAME signal configuration.

Table 61. PCM Frame Signal Configuration Options

Configuration Option	Settings	Description
Sub-frames (Master mode only)	Sub-frames disabled (default), sub-frames enabled	When sub-frames are enabled, the PCM interface provides a frame signal for every word in the frame
Frame Size	2 (default), 4, 6, 8, 10, 12, 14 or 16 words by frames	Specifies the number of words that are expected in each frame (only applicable when sub-frames are disabled)
Frame signal width	Short (default), Long	A short PCM frame signal is held high for one PCM clock cycle. A long PCM frame signal is held high one half of the PCM frame NOTE: If sub-frame is enabled and the word size is odd, the frame signal is held high for a portion of the frame equal to half of the word size rounded down.
Alignment	Align to first data bit, align to last data bit (default)	The first (and possibly only) cycle in which the PCM frame signal is high; it is aligned with the specified data bit

18.6.1.2 Data Serial Input and Output Configuration

Each PCM interface allows data bits to be configured for transmission and reception in either an MSB first or an LSB first ordering. To select between these two configurations, set the PCM_CFG_BIT_ORDER bit in the PCM*_CFG registers.

The length of each word of a PCM transaction can be set to be between one and four bytes (8 and 32 bits) per word, by setting the PCM_CFG_WORD_SIZE bit field in the PCM*_CFG registers. When transmitting data that uses less than 32 bits per word, the PCM_CFG_TX_ALIGN bit from the PCM*_CFG registers configures whether the data is loaded from the most significant portion of the data register, or from the least significant portion of the data register (both configurations are equivalent when using 32-bit data words).

The "PCM Data Configuration Options" table (Table 62) summarizes the options available for data configuration.

Table 62. PCM Data Configuration Options

Configuration Option	Settings	Description
Data ordering	MSB first (default), LSB first	Selects whether the data is transmitted starting with the MSB or LSB
Word size	8 to 32-bit (default) by words	Specifies the number of bits in each PCM word
Transmit and receive bit-field	MSB bits-field (default), or the LSB bits-field	Selects the data alignment (MSB or LSB)

18.6.2 PCM/I²S Data Interface

Each PCM interface has two single word transmit registers called PCM_TX_DATA*, and two single word receive registers called PCM_RX_DATA*. The first word received after the rising edge of the frame is stored in the PCM_RX_DATA0 register, and the second word is stored in the PCM_RX_DATA1 register. Similarly, the PCM_TX_DATA0 register is transferred first, followed by the PCM_TX_DATA1 register.

Ezairo 8300 Hardware Reference

If the word size is greater than 2, both receive registers are overwritten with the next words, and the data in both transmit registers is sent. The order of capture and send is always the same.

18.6.3 PCM Interrupt Configuration

Each PCM interface uses two associated interrupts, which control transmission and reception of PCM data, and report any errors that have occurred while transmitting or receiving PCM data. These interrupts can be configured by setting the `PCM_CFG_RX_TX_INT_ENABLE`, `PCM_CFG_OVERRUN_INT_EN`, and `PCM_CFG_UNDERRUN_INT_EN` bit fields in the `PCM*_CFG` registers. See [Section 4.6.1 “Nested Vectored Interrupt Controller \(NVIC\)” on page 135](#) for information regarding interrupt configuration and handling interrupts for the Arm Cortex-M3 processor. See [Section 3.2 “Interrupt Controller” on page 61](#) for information regarding interrupt configuration and handling for the CFX DSP.

If a user application is transmitting data from a PCM interface, the `PCM_RX_TX` interrupt sends signals at the following times:

- When the `PCM_TX_DATA*` registers' value starts to be transmitted using the PCM interface
- When the `PCM_STATUS_TX_REQ` bit of the `PCM*_STATUS` register is set, signifying that the next data word(s) (of the specified size) to be transmitted can now be loaded into the `PCM_TX_DATA*` registers by the application
- When the `PCM_STATUS_RX_REQ` bit of the `PCM*_STATUS` register is set, signifying that a data word(s) of the specified size has been successfully received and written to the `PCM_RX_DATA*` registers

The PCM transmit and receive interrupts are generated for every two words of data transmitted in a valid PCM frame, regardless of the length of the PCM frame. The data register that acknowledges the transmission or reception can be configured by setting the `PCM_CFG_TX_ACK_SEL` and `PCM_CFG_RX_ACK_SEL` bit fields of the `PCM*_CFG` register.

The `PCM_ERROR` interrupt indicates one of the following conditions:

- An overrun has occurred when the received data from the `PCM_RX_DATA*` registers could not be read before being overwritten. The `PCM_STATUS_OVERRUN` bit in the `PCM*_STATUS` registers can be read for over-run status.
- An underrun occurred when new data could not be copied to the `PCM_TX_DATA*` registers before the next PCM transmission occurred. The `PCM_STATUS_UNDERRUN` bit in the `PCM*_STATUS` registers can be read for under-run status.

NOTE: The status bits for over-run and under-run errors in the `PCM*_STATUS` registers can be cleared by setting the associated clear bits in the same registers.

The "PCM Interrupt Options" table (Table 63) summarizes the options available for PCM interrupt configuration.

Ezairo 8300 Hardware Reference

Table 63. PCM Interrupt Options

Interrupt Type	Signal Name	Status Register Name	Description
Data received	pcm_rx_tx_irq	RX_REQ_STATUS	Interrupt when a new data word is ready to be read by processor
Data sent		TX_REQ_STATUS	Interrupt when a data word has been sent, transmit register is ready for a new value
Overrun detected	pcm_err_irq	OVERRUN_STATUS	A new data word has been received on the interface before the last received data has been read
Underrun detected		UNDERRUN_STATUS	A new data word has been sent on the interface before previous data has been written on the transmit register

18.6.4 I²S Configuration and Usage

Each PCM interface can be configured to be compatible with the I²S interface standard. The "Required Configuration Settings for I²S Configuration" table (Table 64), below, lists the required signal and data management settings for the PCM interface to enable I²S communication.

Table 64. Required Configuration Settings for I²S Configuration

Configuration Bit or Bit Field	Setting
PCM_CFG_FRAME_LENGTH	PCM_MULTIWORD_2
PCM_CFG_FRAME_WIDTH	PCM_FRAME_WIDTH_LONG
PCM_CFG_FRAME_ALIGN	PCM_FRAME_ALIGN_LAST
PCM_CFG_BIT_ORDER	PCM_BIT_ORDER_MSB_FIRST
PCM_CFG_SUBFRAME	PCM_SUBFRAME_DISABLE
PCM_CFG_CLK_POLARITY	PCM_SAMPLE_RISING_EDGE ¹

In master mode, the I²S configuration for the PCM interface has no effect.

In slave mode, the behavior of the PCM interfaces is slightly different from how it acts in any other configuration. When configured for slave mode and a long frame width, the PCM interfaces synchronize communications with both the rising edge and falling edge of the frame signal (instead of synchronizing with only the rising edge as it does in all other configurations). This means that if the word size used by the master is larger than the size configured in the corresponding PCM interface, the interface pads the remaining bits with zeroes for both words. If the size is less than the size configured, the interface only sends and receives the MSB bits of both words.

As shown in the I²S signal timing diagram (the "Signal Timing Diagram: I²S Configuration" figure (Figure 38)), after transmitting the LSB of the current data word, the device repeatedly transmits zeros until a rising or falling edge on the frame signal is detected. Similarly, if a rising or falling edge is detected on the frame signal before the

¹The PCM interfaces produce a data stream that is I²S standard compliant with either sampling edge. However, to comply completely with the standard, the interface must be configured to sample data on the rising edge of the clock.

Ezairo 8300 Hardware Reference

current data word has been completely transmitted, the current data word is truncated and the next data word is sent. In this way, the interface is compatible with I²S signals that transmit a number of bits different from that specified in the defined word size.

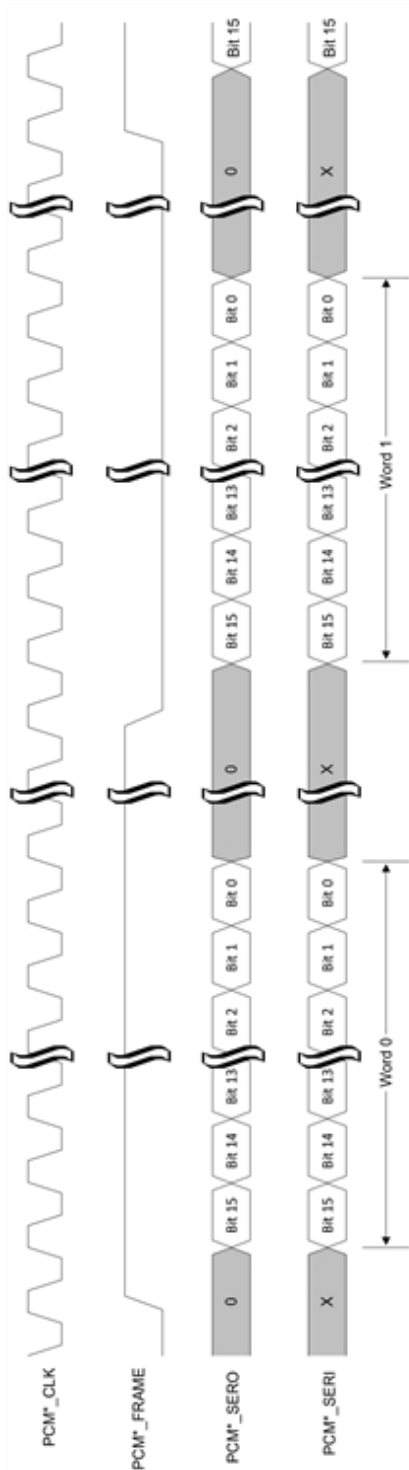


Figure 38. Signal Timing Diagram: I²S Configuration

18.6.5 PCM Registers

Register Name	Register Description	Address
PCM0_CFG	PCM Configuration Register	0x40000B00
PCM0_CTRL	PCM Control Register	0x40000B04
PCM0_STATUS	PCM Status Register	0x40000B08
PCM0_TX_DATA0	PCM Transmit Data 0 Register	0x40000B10
PCM0_TX_DATA1	PCM Transmit Data 1 Register	0x40000B14
PCM0_RX_DATA0	PCM Receive Data 0 Register	0x40000B18
PCM0_RX_DATA1	PCM Receive Data 1 Register	0x40000B1C
PCM1_CFG	PCM Configuration Register	0x40000C00
PCM1_CTRL	PCM Control Register	0x40000C04
PCM1_STATUS	PCM Status Register	0x40000C08
PCM1_TX_DATA0	PCM Transmit Data 0 Register	0x40000C10
PCM1_TX_DATA1	PCM Transmit Data 1 Register	0x40000C14
PCM1_RX_DATA0	PCM Receive Data 0 Register	0x40000C18
PCM1_RX_DATA1	PCM Receive Data 1 Register	0x40000C1C

Register Name	Register Description	Address
PCM0_CFG	PCM Configuration Register	P:0x12C0 0x40000B00
PCM0_CTRL	PCM Control Register	P:0x12C1 0x40000B04
PCM0_STATUS	PCM Status Register	P:0x12C2 0x40000B08
PCM0_TX_DATA0	PCM Transmit Data 0 Register	P:0x12C4 0x40000B10
PCM0_TX_DATA1	PCM Transmit Data 1 Register	P:0x12C5 0x40000B14
PCM0_RX_DATA0	PCM Receive Data 0 Register	P:0x12C6 0x40000B18

onsemi Confidential

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
PCM0_RX_DATA1	PCM Receive Data 1 Register	P:0x12C7 0x40000B1C
PCM1_CFG	PCM Configuration Register	P:0x1300 0x40000C00
PCM1_CTRL	PCM Control Register	P:0x1301 0x40000C04
PCM1_STATUS	PCM Status Register	P:0x1302 0x40000C08
PCM1_TX_DATA0	PCM Transmit Data 0 Register	P:0x1304 0x40000C10
PCM1_TX_DATA1	PCM Transmit Data 1 Register	P:0x1305 0x40000C14
PCM1_RX_DATA0	PCM Receive Data 0 Register	P:0x1306 0x40000C18
PCM1_RX_DATA1	PCM Receive Data 1 Register	P:0x1307 0x40000C1C

18.6.5.1 PCM*_CFG

Bit Field	Read/Write	Field Name	Description
25	RW	TX_ACK_SEL	Select which TX word acknowledges the TX request
24	RW	RX_ACK_SEL	Select which RX word acknowledges the RX request
23	RW	TX_DMA_ENABLE	Enable/disable the TX DMA request
22	RW	RX_DMA_ENABLE	Enable/disable the RX DMA request
21	RW	TX_IOC_ENABLE	Enable/disable the TX IOC request
20	RW	RX_IOC_ENABLE	Enable/disable the RX IOC request
19	RW	RX_TX_INT_ENABLE	Enable/disable the RX_TX interrupt
17	RW	OVERRUN_INT_EN	Enable/disable the overrun interrupt
16	RW	UNDERRUN_INT_EN	Enable/disable the underrun interrupt
15	RW	CLK_POLARITY	Select the PCM clock polarity

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
14	RW	TX_DATA_ALIGN	Select the TX data alignment
13	RW	RX_DATA_ALIGN	Select the RX data alignment
12:8	RW	WORD_SIZE	Select the number of bits per PCM word
7	RW	BIT_ORDER	Select whether the data is transmitted starting with the MSB or LSB
6	RW	FRAME_ALIGN	Align the PCM frame signal to the first/last bit
5	RW	FRAME_WIDTH	Use a long/short PCM frame signal
4:2	RW	FRAME_LENGTH	Select the number of words per PCM frame
1	RW	SUBFRAME	Enable the frame duration for each word
0	RW	SLAVE	Use the PCM interface as a master/slave

Field Name	Value Symbol	Value Description	Hex Value
TX_ACK_SEL	PCM_TX_ACK_DATA0	TX data0 acknowledges the TX request	0x0*
	PCM_TX_ACK_DATA1	TX data1 acknowledges the TX request	0x1
RX_ACK_SEL	PCM_RX_ACK_DATA0	RX data0 acknowledges the RX request	0x0*
	PCM_RX_ACK_DATA1	RX data1 acknowledges the RX request	0x1
TX_DMA_ENABLE	PCM_TX_DMA_DISABLE	No TX DMA request is generated	0x0*
	PCM_TX_DMA_ENABLE	A TX DMA request is generated when new data is requested by the PCM interface	0x1
RX_DMA_ENABLE	PCM_RX_DMA_DISABLE	No RX DMA request is generated	0x0*
	PCM_RX_DMA_ENABLE	An RX DMA request is generated when new data is received by the PCM interface	0x1
TX_IOC_ENABLE	PCM_TX_IOC_DISABLE	No TX IOC request is generated	0x0*
	PCM_TX_IOC_ENABLE	A TX IOC request is generated when new data is requested by the PCM interface	0x1
RX_IOC_ENABLE	PCM_RX_IOC_DISABLE	No RX IOC request is generated	0x0*
	PCM_RX_IOC_ENABLE	An RX IOC request is generated when new data is received by the PCM interface	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
RX_TX_INT_ENABLE	PCM_RX_TX_INT_DISABLE	No RX_TX interrupt is raised	0x0*
	PCM_RX_TX_INT_ENABLE	An RX_TX interrupt is raised when new data is received or requested by the PCM interface	0x1
OVERRUN_INT_EN	PCM_OVERRUN_INT_DISABLE	No ERROR interrupt is raised when an overrun is detected	0x0*
	PCM_OVERRUN_INT_ENABLE	An ERROR interrupt is raised when an overrun occurs on the PCM interface	0x1
UNDERRUN_INT_EN	PCM_UNDERRUN_INT_DISABLE	No ERROR interrupt is raised when an underrun is detected	0x0*
	PCM_UNDERRUN_INT_ENABLE	An ERROR interrupt is raised when an underrun occurs on the PCM interface	0x1
CLK_POLARITY	PCM_SAMPLE_FALLING_EDGE	PCM input data sampled on PCM_CLK falling edge	0x0*
	PCM_SAMPLE_RISING_EDGE	PCM input data sampled on PCM_CLK rising edge	0x1
TX_DATA_ALIGN	PCM_TX_DATA_ALIGN_MSB	TX data is aligned on the MSB of the 32-bit TX buffer	0x0*
	PCM_TX_DATA_ALIGN_LSB	TX data is aligned on the LSB of the 32-bit TX buffer	0x1
RX_DATA_ALIGN	PCM_RX_DATA_ALIGN_MSB	RX data is aligned on the MSB of the 32-bit RX buffer	0x0*
	PCM_RX_DATA_ALIGN_LSB	RX data is aligned on the LSB of the 32-bit RX buffer	0x1
WORD_SIZE	PCM_WORD_SIZE_8	Use 8-bit words	0x0
	PCM_WORD_SIZE_9	Use 9-bit words	0x1
	PCM_WORD_SIZE_10	Use 10-bit words	0x2
	PCM_WORD_SIZE_11	Use 11-bit words	0x3
	PCM_WORD_SIZE_12	Use 12-bit words	0x4
	PCM_WORD_SIZE_13	Use 13-bit words	0x5
	PCM_WORD_SIZE_14	Use 14-bit words	0x6
	PCM_WORD_SIZE_15	Use 15-bit words	0x7
	PCM_WORD_SIZE_16	Use 16-bit words	0x8*
	PCM_WORD_SIZE_17	Use 17-bit words	0x9
	PCM_WORD_SIZE_18	Use 18-bit words	0xA

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	PCM_WORD_SIZE_19	Use 19-bit words	0xB
	PCM_WORD_SIZE_20	Use 20-bit words	0xC
	PCM_WORD_SIZE_21	Use 21-bit words	0xD
	PCM_WORD_SIZE_22	Use 22-bit words	0xE
	PCM_WORD_SIZE_23	Use 23-bit words	0xF
	PCM_WORD_SIZE_24	Use 24-bit words	0x10
	PCM_WORD_SIZE_25	Use 25-bit words	0x11
	PCM_WORD_SIZE_26	Use 26-bit words	0x12
	PCM_WORD_SIZE_27	Use 27-bit words	0x13
	PCM_WORD_SIZE_28	Use 28-bit words	0x14
	PCM_WORD_SIZE_29	Use 29-bit words	0x15
	PCM_WORD_SIZE_30	Use 30-bit words	0x16
	PCM_WORD_SIZE_31	Use 31-bit words	0x17
	PCM_WORD_SIZE_32	Use 32-bit words	0x18
BIT_ORDER	PCM_BIT_ORDER_MSB_FIRST	PCM data is ordered from MSB to LSB.	0x0*
	PCM_BIT_ORDER_LSB_FIRST	PCM data is ordered from LSB to MSB.	0x1
FRAME_ALIGN	PCM_FRAME_ALIGN_LAST	Align the PCM frame signal to the last bit of the frame.	0x0*
	PCM_FRAME_ALIGN_FIRST	Align the PCM frame signal to the first bit of the frame.	0x1
FRAME_WIDTH	PCM_FRAME_WIDTH_SHORT	The frame is high for one PCM clock period.	0x0*
	PCM_FRAME_WIDTH_LONG	The frame is high for half of the frame length.	0x1
FRAME_LENGTH	PCM_MULTIWORD_2	A frame contains 2 words.	0x0*
	PCM_MULTIWORD_4	A frame contains 4 words.	0x1
	PCM_MULTIWORD_6	A frame contains 6 words.	0x2
	PCM_MULTIWORD_8	A frame contains 8 words.	0x3
	PCM_MULTIWORD_10	A frame contains 10 words.	0x4
	PCM_MULTIWORD_12	A frame contains 12 words.	0x5
	PCM_MULTIWORD_14	A frame contains 14 words.	0x6
	PCM_MULTIWORD_16	A frame contains 16 words.	0x7

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
SUBFRAME	PCM_SUBFRAME_DISABLE	Generate a frame signal every frame.	0x0*
	PCM_SUBFRAME_ENABLE	Generate a frame signal every word.	0x1
SLAVE	PCM_SELECT_MASTER	The PCM interface generates the PCM_FRAME.	0x0
	PCM_SELECT_SLAVE	The PCM interface received an external PCM_FRAME.	0x1*

18.6.5.2 PCM*_CTRL

Bit Field	Read/Write	Field Name	Description
8	R	ENABLE_STATUS	PCM enable status
2	W	RESET	Reset the PCM interface
1	W	DISABLE	Disable the PCM interface
0	W	ENABLE	Enable the PCM interface

Field Name	Value Symbol	Value Description	Hex Value
ENABLE_STATUS	PCM_STATUS_DISABLED	PCM interface is disabled	0x0*
	PCM_STATUS_ENABLED	PCM interface is enabled	0x1
RESET	PCM_RESET	Reset the PCM interface	0x1
DISABLE	PCM_DISABLE	Disable the PCM interface	0x1
ENABLE	PCM_ENABLE	Enable the PCM interface	0x1

18.6.5.3 PCM*_STATUS

Bit Field	Read/Write	Field Name	Description
12	R	BUSY	Indicate that the reception or transmission of the data is ongoing
11	R	TX_REQ	Indicate that TX data can be written
10	R	RX_REQ	Indicate that RX data can be read
9	R	OVERRUN	Indicate that an overrun occurred when receiving data
8	R	UNDERRUN	Indicate that an underrun occurred when transmitting data
1	W	OVERRUN_CLEAR	Clear the overrun status flag
0	W	UNDERRUN_CLEAR	Clear the underrun status flag

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
BUSY	PCM_IDLE	PCM idle	0x0*
	PCM_BUSY	PCM busy	0x1
TX_REQ	PCM_TX_NO_REQ	PCM TX data has already been written	0x0
	PCM_TX_REQ	PCM TX data can be written	0x1*
RX_REQ	PCM_RX_NO_REQ	No new PCM RX data available	0x0*
	PCM_RX_REQ	New PCM RX data available	0x1
OVERRUN	PCM_OVERRUN_FALSE	No PCM input overrun detected	0x0*
	PCM_OVERRUN_TRUE	PCM input overrun detected	0x1
UNDERRUN	PCM_UNDERRUN_FALSE	No PCM output underrun detected	0x0*
	PCM_UNDERRUN_TRUE	PCM output underrun detected	0x1
OVERRUN_CLEAR	PCM_OVERRUN_CLEAR	Clear the PCM overrun bit	0x1
UNDERRUN_CLEAR	PCM_UNDERRUN_CLEAR	Clear the PCM underrun bit	0x1

18.6.5.4 PCM*_TX_DATA0

Bit Field	Read/Write	Field Name	Description
31:0	RW	TX_DATA0	Data to transmit on channel 0

18.6.5.5 PCM*_TX_DATA1

Bit Field	Read/Write	Field Name	Description
31:0	RW	TX_DATA1	Data to transmit on channel 1

18.6.5.6 PCM*_RX_DATA0

Bit Field	Read/Write	Field Name	Description
31:0	R	RX_DATA0	Data received on channel 0

18.6.5.7 PCM*_RX_DATA1

Bit Field	Read/Write	Field Name	Description
31:0	R	RX_DATA1	Data received on channel 1

18.7 SERIAL PERIPHERAL INTERFACES (SPI)

The Ezairo 8300 system includes two Serial Peripheral Interfaces (SPIs). The SPI is a synchronous 4-wire interface that includes clock, chip select, and IO pads. The SPI supports single and dual I/O modes, as well as the

Ezairo 8300 Hardware Reference

ability to add two additional I/O pins in a quad I/O mode. These SPI interfaces allow the system to communicate with external components, including external analog front ends, external controllers, wireless transceivers, and non-volatile memories (NVMs).

NOTE: The SPIs support master/slave configuration as well as half/full duplex mode. Slave mode only supports the default SPI mode.

The SPI interfaces are multiplexed onto the DIO pads, which can be configured for the input and output signals that form each SPI with the necessary physical pad configuration. For more information about configuring the multiplexed DIO functionality, see [Section 16.2 “Functional Configuration” on page 464](#).

The SPI interfaces can be enabled or disabled using the `SPI_CTRL_ENABLE` and `SPI_CTRL_DISABLE` write only bits in the `SPI*_CTRL` registers. When the `SPI_CTRL_DISABLE` bit is set to 1, any ongoing transactions on the specified SPI interface are completed and the interface is disabled.

Data transfers using the SPI interfaces can be controlled directly by one of the host processors, or indirectly by using the DMA. To select the DMA channel, use the `SPI_CFG_TX_DMA_ENABLE` and `SPI_CFG_RX_DMA_ENABLE` bits in the `SPI*_CFG` registers. The RX/TX interrupts and RX/TX DMA requests can be generated concurrently.

18.7.1 SPI Data Transfers

The SPI interfaces can be configured to operate as an SPI master device or an SPI slave device by setting the `SPI_CFG_SLAVE` bit in the `SPI*_CFG` register to 0 or 1 respectively. When configured as SPI master, the `SPI_CFG_MODE` bit field selects the number of data pins used for communication. In slave mode, this field has no effect and the mode is always set to `SPI_MODE_SPI` (two separate RX and TX data pins). See the definition of `SPI*_IO[0:3]`, below, for an explanation of the data pins used for communication.

The differences in the SPI pad configurations between master and slave mode are as follows:

*SPI*_CLK*

In master mode, the `SPI*_CLK` signal is supplied by the Ezairo 8300. The `SPI*_CLK` signal is derived from `SYSCCLK` using a power of two prescaler (2^1 to 2^{10}), configurable via the `SPI_CFG_PRESCALE` bit field from the `SPI*_CFG` register, using the following equation:

$$f_{\text{SPI*}_\text{CLK}} = \frac{f_{\text{SYSCCLK}}}{2^{(\text{SPI*}_\text{CFG_PRESCALE}+1)}}$$

In slave mode, the `SPI*_CLK` signal is sourced from a remote SPI master device. For proper operation, the frequency of the `SPI*_CLK` inputs must abide by the following relation:

$$f_{\text{SYS CLK}} \geq 4 \times f_{\text{SPI*}_\text{CLK}}$$

Due to the re-timing of the input `SPI*_CLK`, the output on `SPI*_IO1` of the SPI slave pad, generated by writing to the `SPI*_TX_DATA` register, is delayed up to one `SYSCCLK` period when:

$$f_{\text{SYS CLK}} = 4 \times f_{\text{SPI*}_\text{CLK}}$$

Output can be delayed up to one half of a clock cycle when:

$$f_{SYS\ CLK} = 5 \times f_{SPI*_CLK}$$

No delay in output (i.e., the output delay only consists of logic and pad delays) occurs when:

$$f_{SYS\ CLK} \geq 6 \times f_{SPI*_CLK}$$

For both master and slave mode, the `SPI_CFG_CLK_POLARITY` bit in the `SPI*_CFG` registers is used to control the times when data changes and when data is sampled. An SPI using normal polarity updates output signals on the falling edge of `SPI*_CLK`, and samples input signals on the rising edge of `SPI*_CLK`. If the polarity is inverted, output signals change on the rising edge of `SPI*_CLK` and input signals are sampled on the falling edge. In normal mode, the idle state of the SPI clock is low; in the inverse mode, the SPI clock's idle state is high.

*SPI*_CS*

In master mode, the `SPI*_CS` pad is an output controlled by the `SPI_CTRL_CS*` bits from the `SPI*_CTRL` registers. The signal from this pad is generally routed to the chip select input of a slave device. Write to `SPI_CTRL_CS1` to set, or write to `SPI_CTRL_CS0` to clear.

In slave mode, the `SPI*_CS` pad is an input sourced from a remote SPI master device.

NOTE: For an SPI device, the chip select input is interpreted as an active-low signal. As such, if the signal on the `SPI*_CS` pad is high, the SPI slave ignores all communications using the interface. The minimum delay required between a falling edge on the `SPI*_CS` pad and the first edge on `SPI*_CLK` pad is:

$$\frac{1}{2} \cdot \text{SPI*_CLK Period} + 2 \cdot \text{SYSCLK Periods}$$

*SPI*_IO[0:3]*

These pads are used as data pins for the SPI. The `SPI_CFG_MODE` bit field in the `SPI*_CFG` register is used to configure the number of data pins used in SPI master mode. The SPI slave operates only in SPI normal mode; therefore, only `SPI*_IO0` and `SPI*_IO` pins are available. These pads are used to receive or transmit data when the following conditions are met:

- `SPI*_CLK` and `SPI*_CS` indicate that data needs to be transferred.
- The `SPI_CTRL_MODE*` bits from the `SPI*_CTRL` register are configured for an SPI read, write, or read-write (full-duplex) transfer.

The SPI timing for half and full duplex modes is shown in the "SPI Slave Mode Timing" figure (Figure 39).

Ezairo 8300 Hardware Reference

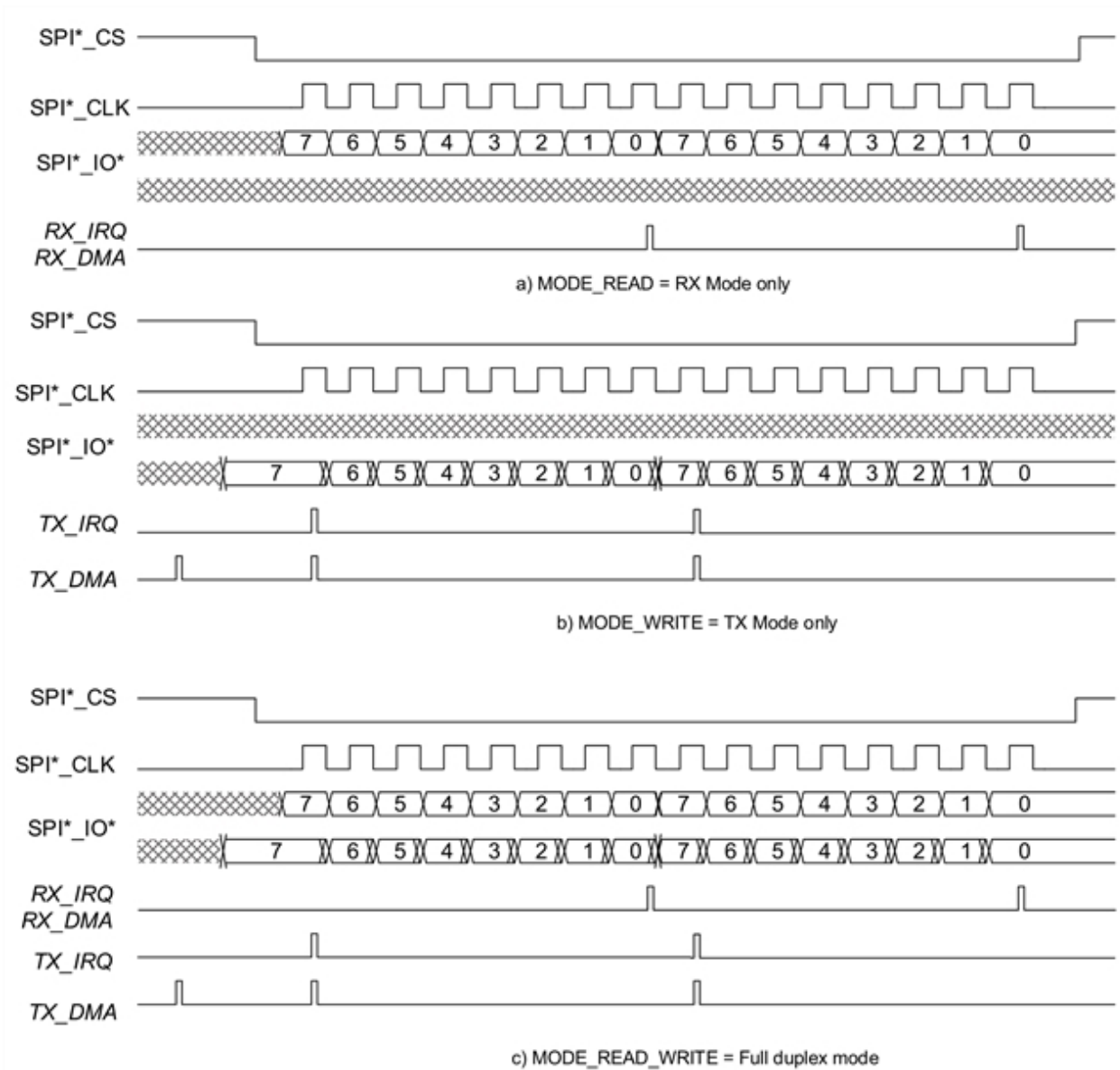


Figure 39. SPI Slave Mode Timing

The SPI operation is configured and controlled by the SPI^*_CFG and SPI^*_CTRL registers. Write or read access to these registers is not restricted when the SPI is active or idle. All updates in these registers are applied immediately, excluding the $SPI_CFG_CLK_POLARITY$ and $SPI_CFG_WORD_SIZE$ fields in the SPI^*_CFG register. Updating these registers with the same field values as their original content does not corrupt SPI communication, even if the SPI is active. As a result, you can update only the fields of interest without affecting the functionality associated with the remaining fields. It is recommended that you only update these registers when the SPI is idle.

Ezairo 8300 Hardware Reference

The SPIs support a configurable word size for each transfer of 1 to 32 bits of data per word, as specified in the `SPI_CFG_WORD_SIZE` bit field in the `SPI*_CFG` register. All data transactions using an SPI start with the MSB of the word received, and transfer $(\text{SPI_CFG_WORD_SIZE} + 1)$ bits of data per word.

- If the SPI is busy, the update to this register is not transferred to the active counter until the countdown reaches zero and the SPI is idle.
- In slave mode, the `WORD_SIZE` might not be updated during the transmission of the last two bits.
- In dual mode, the `WORD_SIZE` is required to be a multiple of 2. If the value written is not a multiple of 2, it is automatically set to the next multiple of 2.
- In quad mode, the `WORD_SIZE` is required to be a multiple of 4. If the value written is not a multiple of 4, it is automatically set to the next multiple of 4.

The `SPI*_CTRL` register `CMD` field is used to select either half duplex or full duplex mode as follows:

- Set `MODE_NOP` for no read or write operation
- Set `MODE_WRITE` for write-only data operation
- Set `MODE_READ` for read-only data operation
- Set `MODE_READ_WRITE` for read and write data operation (full duplex in normal SPI mode)

NOTE: In DSPI and QSPI modes, this setting is not valid. The application must set `MODE_WRITE` and `MODE_READ` during different phases of the transaction.

In SPI master mode, a read transaction is started when `SPI*_RX_DATA` is read, or when the `SPI_CTRL_START` bit is set to 1 in the `SPI*_CTRL` register. A write or a read/write operation is started when the TX buffer is full. If it is empty, the transfer starts when `SPI*_TX_DATA` is written. The SPI interface has parallel buffers for both transmitted and received data.

`SPI*_TX_DATA`

Shift register containing the data to transmit using the SPI.

`SPI*_RX_DATA`

Shift register containing the recent data word received from the SPI.

Two additional registers can be used to read the data received:

`SPI*_RX_DATA_NO_START`

Register for reading the received data without starting a new read transaction when in read mode. Read request flag `SPI_STATUS_RX_REQ` in the `SPI*_STATUS` register is cleared automatically on reading the `SPI*_RX_DATA_NO_START` register.

`SPI*_RX_DATA_MIRROR`

Register for reading the received data without starting a new read transaction when in read mode and without clearing the read request flag. Used for debugging, without modifying the behavior of the program.

The `SPI*_STATUS` register can be used by an application to identify the state of the SPI interface. The states are as follows:

Ezairo 8300 Hardware Reference

- The `SPI_STATUS_BUSY` bit in this register indicates that the reception or transmission of the data is ongoing. This bit is useful for determining when a transaction has completed on the bus. The application can use the `SPI_STATUS_TX_REQ` and `SPI_STATUS_RX_REQ` bits to determine when to read and write to or from the interface.
- When the `SPI_STATUS_TX_REQ` bit is set to 1, the SPI is ready to accept new data in the `SPI*_TX_DATA` register for transmission. The application must wait for this bit to be set before writing to the `SPI*_TX_DATA` register.
- The `SPI_STATUS_RX_REQ` bit indicates when new data is available to be read. The application must wait for this bit to be set before reading from the `SPI*_RX_DATA` register to ensure that the data is valid.
- The `SPI_STATUS_CS_RISE` bit indicates that a rising edge has been detected on the `SPI*_CS` pin. An SPI slave can use this interrupt to determine whether an SPI master is initiating a data transfer.
- The `SPI_STATUS_OVERRUN` and `SPI_STATUS_UNDERRUN` interrupts are used to determine whether a buffer overrun or buffer underrun condition has occurred during SPI communications.
- `SPI_STATUS_TX_REQ_SET` is provided in slave mode when a master ends a transaction by setting CS high (SPI). At this point the `SPI*_STATUS_TX_REQ` flag is zero, which indicates that there is still a byte pending in the TX buffer. To clear this state without resetting the interface, `TX_REQ_SET` can be set high to request new data. However, this does not generate a new interrupt (TX); only the underlying buffer is flushed empty.

NOTE: When the interface is disabled, any ongoing data transmission is allowed to complete before the interface shuts down.

NOTE: In slave mode, the first value to be transmitted in a frame (i.e., after the CS line goes low) needs to be written into the buffer prior to the CS going low. Therefore, after initializing the SPI interface in slave mode, `SPI*_TX_DATA` must be written to before an externally-connected master initiates a read transaction. Otherwise `SPI_STATUS_UNDERRUN` is set to indicate that an underrun has occurred.

IMPORTANT: When transmitting data as an SPI slave, the SPI interface must load the `SPI*_TX_DATA` register between `SPI*_CLK` edges that update the SPI output signals. If no delay occurs between the words of the SPI transfer, new data must be loaded in one `SPI*_CLK` cycle.

18.7.2 SPI Interrupts

Each of the SPI interfaces uses three interrupts that can indicate the following events:

- The start of a data transmission (TX start interrupt)
- The completion of a data transmission (TX end interrupt)
- The completion of data being received (RX interrupt)
- Buffer overrun/underrun/CS rise events

See [Section 4.6.1 “Nested Vectored Interrupt Controller \(NVIC\)” on page 135](#) for information regarding interrupt configuration and handling interrupts for the Arm Cortex-M3 processor. See [Section 3.2 “Interrupt Controller” on page 61](#) for information regarding interrupt configuration and handling for the CFX DSP.

If a user application is transmitting data using an SPI interface, the `SPI*_TX_START` interrupt indicates the following:

- That the interface has started transmitting the data value from the `SPI*_TX_DATA` register
- That the application can now load the next data word (of the specified size) to be transmitted

Ezairo 8300 Hardware Reference

The `SPI*_TX_END` interrupt can also be used when transmitting data. This interrupt indicates when the transmission is finished, which can be useful when transmitting the last data word at the end of a transmission. This indicates to the application when it is safe to reconfigure or disable the SPI interface, or to transition into a low power mode.

If a user application is receiving data from an SPI interface, the `SPI*_RX` interrupt indicates that a data word of the specified size has been successfully received and written to the `SPI*_RX_DATA` register.

`SPI*_COM` interrupts indicate buffer overrun, underrun, and a rising edge transition on the `SPI*_CS` pin. An overrun occurs when another data word has been loaded into the `SPI*_RX_DATA` register prior to the `SPI*_RX_DATA` register being read. An underrun occurs when data is taken from the `SPI*_TX_DATA` register for transmission before the `SPI*_TX_DATA` register has been updated with new data to be transmitted. A CS rise is detected when the interface is enabled in slave mode and the CS goes from low to high.

The status of the different interrupts can be monitored in the `SPI*_STATUS` register. The different status bits can be reset by writing 1 to the corresponding clear bit locations.

NOTE: In slave mode, if CS rises before a word transaction is completed, that transaction is aborted and the data is lost.

18.7.3 SPI DMA Control

The SPI can operate in DMA mode by enabling the bits `SPI_CFG_TX_DMA_ENABLE` and/or `SPI_CFG_RX_DMA_ENABLE` in the `SPI*_CFG` register. This allows the SPI to communicate with the DMA controller, which responds to requests by writing or reading to or from the data register.

NOTE: The DMA requests are not gated by the `SPI*_STATUS_ENABLE` bit of the SPI interface. Therefore, for TX_DMA transfers, the SPI causes the data to be transferred from the DMA into the `SPI*_TX_DATA` register even if SPI is disabled. However, the data itself is not shifted out on the SPI until the interface is enabled.

Additionally, while using the DMA to control data transfers over an SPI interface, the `SPI*_ERROR` interrupts indicate when an error has occurred.

- An overrun occurs when the DMA cannot read the received data from the `SPI*_RX_DATA` register and transfer this data to the DMA buffer before it is overwritten. To monitor for overrun events, set the `SPI_CFG_OVERRUN_INT_ENABLE` bit in the `SPI*_CFG` register. If an overrun occurs and this event monitor is enabled, the `SPI_STATUS_OVERRUN` bit in the appropriate `SPI*_STATUS` register is set.
- An underrun occurs when the DMA cannot write the `SPI*_TX_DATA` register before a second data transfer starts using this register's previous data. To monitor for underrun events, set the `SPI_CFG_UNDERRUN_INT_ENABLE` bit in the `SPI*_CFG` register. If an underrun occurs and this event monitor is enabled, the `SPI_STATUS_UNDERRUN` bit in the appropriate `SPI*_STATUS` register is set.

onsemi Confidential
Ezairo 8300 Hardware Reference

18.7.4 SPI Registers

Register Name	Register Description	Address
SPI0_CFG	SPI Configuration Register	P:0x1340 0x40000D00
SPI0_CTRL	SPI Control Register	P:0x1341 0x40000D04
SPI0_STATUS	SPI Status Register	P:0x1342 0x40000D08
SPI0_TX_DATA	SPI Transmit Data Register	P:0x1343 0x40000D0C
SPI0_RX_DATA	SPI Received Data Register	P:0x1344 0x40000D10
SPI0_RX_DATA_NO_START	SPI Received Data No Start Register	P:0x1345 0x40000D14
SPI0_RX_DATA_MIRROR	SPI Received Data Mirror Register	P:0x1346 0x40000D18
SPI1_CFG	SPI Configuration Register	P:0x1380 0x40000E00
SPI1_CTRL	SPI Control Register	P:0x1381 0x40000E04
SPI1_STATUS	SPI Status Register	P:0x1382 0x40000E08
SPI1_TX_DATA	SPI Transmit Data Register	P:0x1383 0x40000E0C
SPI1_RX_DATA	SPI Received Data Register	P:0x1384 0x40000E10
SPI1_RX_DATA_NO_START	SPI Received Data No Start Register	P:0x1385 0x40000E14
SPI1_RX_DATA_MIRROR	SPI Received Data Mirror Register	P:0x1386 0x40000E18

Ezairo 8300 Hardware Reference

18.7.4.1 SPI*_CFG

Bit Field	Read/Write	Field Name	Description
23	RW	TX_DMA_ENABLE	Enable/disable the TX DMA request
22	RW	RX_DMA_ENABLE	Enable/disable the RX DMA request
21	RW	TX_END_INT_ENABLE	Enable/disable the TX interrupt
20	RW	TX_START_INT_ENABLE	Enable/disable the TX interrupt
19	RW	RX_INT_ENABLE	Enable/disable the RX interrupt
18	RW	CS_RISE_INT_ENABLE	Enable/disable the CS rise interrupt (slave mode only)
17	RW	OVERRUN_INT_ENABLE	Enable/disable the overrun interrupt
16	RW	UNDERRUN_INT_ENABLE	Enable/disable the underrun interrupt
14:13	RW	MODE	Select the SPI master mode (ignored in slave mode)
12:8	RW	WORD_SIZE	Select the SPI word size (word size = SPI_WORD_SIZE + 1)
7:4	RW	PRESCALE	Prescale the SPI clock for master mode
1	RW	CLK_POLARITY	Select the SPI clock polarity
0	RW	SLAVE	Use the SPI interface as master or slave

Field Name	Value Symbol	Value Description	Hex Value
TX_DMA_ENABLE	SPI_TX_DMA_DISABLE	No TX DMA request is generated	0x0*
	SPI_TX_DMA_ENABLE	A TX DMA request is generated when new data is requested by the SPI interface	0x1
RX_DMA_ENABLE	SPI_RX_DMA_DISABLE	No RX DMA request is generated	0x0*
	SPI_RX_DMA_ENABLE	An RX DMA request is generated when new data is received by the SPI interface	0x1
TX_END_INT_ENABLE	SPI_TX_END_INT_DISABLE	No TX interrupt is raised	0x0*
	SPI_TX_END_INT_ENABLE	A TX interrupt is raised when a TX data transmission is finished	0x1
TX_START_INT_ENABLE	SPI_TX_START_INT_DISABLE	No TX interrupt is raised	0x0*
	SPI_TX_START_INT_ENABLE	A TX interrupt is raised when a new TX data send by the SPI interface is started	0x1
RX_INT_ENABLE	SPI_RX_INT_DISABLE	No RX interrupt is raised	0x0*
	SPI_RX_INT_ENABLE	An RX interrupt is raised when new data is received by the SPI interface	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
CS_RISE_INT_ENABLE	SPI_CS_RISE_INT_DISABLE	No interrupt is raised when the CS rises in slave mode	0x0*
	SPI_CS_RISE_INT_ENABLE	A common interrupt is raised when the CS rises in slave mode	0x1
OVERRUN_INT_ENABLE	SPI_OVERRUN_INT_DISABLE	No interrupt is raised when an overrun is detected	0x0*
	SPI_OVERRUN_INT_ENABLE	A common interrupt is raised when an overrun occurs on the SPI interface	0x1
UNDERRUN_INT_ENABLE	SPI_UNDERRUN_INT_DISABLE	No interrupt is raised when an underrun is detected	0x0*
	SPI_UNDERRUN_INT_ENABLE	A common interrupt is raised when an underrun occurs on the SPI interface	0x1
MODE	SPI_MODE_SPI	SPI normal mode (separate RX and TX data)	0x0*
	SPI_MODE_DSPI	SPI dual mode (two bidirectional data pins)	0x1
	SPI_MODE_QSPI	SPI quad mode (four bidirectional data pins)	0x2
WORD_SIZE	SPI_WORD_SIZE_1	SPI transfers use 1-bit words	0x0*
	SPI_WORD_SIZE_4	SPI transfers use 4-bit words	0x3
	SPI_WORD_SIZE_8	SPI transfers use 8-bit words	0x7
	SPI_WORD_SIZE_16	SPI transfers use 16-bit words	0xF
	SPI_WORD_SIZE_24	SPI transfers use 24-bit words	0x17
	SPI_WORD_SIZE_32	SPI transfers use 32-bit words	0x1F
PRESCALE	SPI_PRESCALE_2	Prescale the SPI interface clock by 2	0x0*
	SPI_PRESCALE_4	Prescale the SPI interface clock by 4	0x1
	SPI_PRESCALE_8	Prescale the SPI interface clock by 8	0x2
	SPI_PRESCALE_16	Prescale the SPI interface clock by 16	0x3
	SPI_PRESCALE_32	Prescale the SPI interface clock by 32	0x4
	SPI_PRESCALE_64	Prescale the SPI interface clock by 64	0x5
	SPI_PRESCALE_128	Prescale the SPI interface clock by 128	0x6
	SPI_PRESCALE_256	Prescale the SPI interface clock by 256	0x7
	SPI_PRESCALE_512	Prescale the SPI interface clock by 512	0x8
	SPI_PRESCALE_1024	Prescale the SPI interface clock by 1024	0x9

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
CLK_POLARITY	SPI_CLK_POLARITY_NORMAL	In both master and slave modes SERO changes on the falling edge of the SPI clock. The SERI is sampled in slave mode just after, and in master mode at, the rising edge of the SPI clock	0x0*
	SPI_CLK_POLARITY_INVERSE	In both master and slave modes SERO changes on the rising edge of the SPI clock. The SERI is sampled in slave mode just after, and in master mode at, the falling edge of the SPI clock	0x1
SLAVE	SPI_SELECT_MASTER	Use the SPI interface in master mode	0x0*
	SPI_SELECT_SLAVE	Use the SPI interface in slave mode	0x1

18.7.4.2 SPI*_CTRL

Bit Field	Read/Write	Field Name	Description
19	R	CS_STATUS	SPI CS status
18:17	R	MODE_STATUS	SPI mode status
16	R	ENABLE_STATUS	SPI enable status
9	W	CS_0	Lower the SPI chip-select line (master mode)
8	W	CS_1	Raise the SPI chip-select line (master mode)
7	W	MODE_NOP	Set mode to no operation
6	W	MODE_WRITE	Set mode to read operation
5	W	MODE_READ	Set mode to write operation
4	W	MODE_READ_WRITE	Set mode to read and write operation
3	W	START	Start a data transfer in master read mode
2	W	RESET	Reset the SPI interface
1	W	DISABLE	Disable the SPI interface
0	W	ENABLE	Enable the SPI interface

Field Name	Value Symbol	Value Description	Hex Value
CS_STATUS	SPI_STATUS_CS_0	SPI CS is low	0x0
	SPI_STATUS_CS_1	SPI CS is high	0x1*
MODE_STATUS	SPI_STATUS_MODE_NOP	Mode is no operation (not allowed in DSPI and QSPI modes)	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	SPI_STATUS_MODE_WRITE	Mode is write data	0x1
	SPI_STATUS_MODE_READ	Mode is read data	0x2
	SPI_STATUS_MODE_READ_WRITE	Mode is read and write data (not allowed in DSPI and QSPI modes)	0x3
ENABLE_STATUS	SPI_STATUS_DISABLED	SPI interface is disabled	0x0*
	SPI_STATUS_ENABLED	SPI interface is enabled	0x1
CS_0	SPI_CS_0	Set the SPI CS signal low	0x1
CS_1	SPI_CS_1	Set the SPI CS signal high	0x1
MODE_NOP	SPI_MODE_NOP	Set the mode to no operation	0x1
MODE_WRITE	SPI_MODE_WRITE	Set the mode to write operation	0x1
MODE_READ	SPI_MODE_READ	Set the mode to read operation	0x1
MODE_READ_WRITE	SPI_MODE_READ_WRITE	Set the mode to read and write operation (not allowed in DSPI and QSPI modes)	0x1
START	SPI_START_READ	Start a transfer on the SPI interface (master read mode only)	0x1
RESET	SPI_RESET	Reset the SPI interface	0x1
DISABLE	SPI_DISABLE	Disable the SPI interface	0x1
ENABLE	SPI_ENABLE	Enable the SPI interface	0x1

18.7.4.3 SPI*_STATUS

Bit Field	Read/Write	Field Name	Description
13	R	BUSY	Indicate that the reception or transmission of the data is ongoing
12	R	TX_REQ	Indicate that TX data can be written
11	R	RX_REQ	Indicate that RX data can be read
10	R	CS_RISE	Indicate that CS has risen in slave mode
9	R	OVERRUN	Indicate that an overrun has occurred when receiving data on the SPI interface
8	R	UNDERRUN	Indicate that an underrun has occurred when transmitting data on the SPI interface
4	W	TX_REQ_SET	Set TX_REQ status flag and clear internal TX buffer status
2	W	CS_RISE_CLEAR	Clear the CS rise status flag
1	W	OVERRUN_CLEAR	Clear the overrun status flag
0	W	UNDERRUN_CLEAR	Clear the underrun status flag

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
BUSY	SPI_IDLE	SPI idle	0x0*
	SPI_BUSY	SPI busy	0x1
TX_REQ	SPI_TX_NO_REQ	SPI TX data has already been written	0x0
	SPI_TX_REQ	SPI TX data can be written	0x1*
RX_REQ	SPI_RX_NO_REQ	No new SPI RX data available	0x0*
	SPI_RX_REQ	New SPI RX data available	0x1
CS_RISE	SPI_CS_RISE_FALSE	CS did not rise in slave mode	0x0*
	SPI_CS_RISE_TRUE	CS has risen in slave mode	0x1
OVERRUN	SPI_OVERRUN_FALSE	No SPI input overrun detected	0x0*
	SPI_OVERRUN_TRUE	SPI input overrun detected	0x1
UNDERRUN	SPI_UNDERRUN_FALSE	No SPI input underrun detected	0x0*
	SPI_UNDERRUN_TRUE	SPI input underrun detected	0x1
TX_REQ_SET	SPI_TX_REQ_SET	Set the TX_REQ status flag and clear internal TX buffer status	0x1
CS_RISE_CLEAR	SPI_CS_RISE_CLEAR	Clear the SPI CS rise status bit	0x1
OVERRUN_CLEAR	SPI_OVERRUN_CLEAR	Clear the SPI overrun status bit	0x1
UNDERRUN_CLEAR	SPI_UNDERRUN_CLEAR	Clear the SPI underrun status bit	0x1

18.7.4.4 SPI*_TX_DATA

Bit Field	Read/Write	Field Name	Description
31:0	RW	TX_DATA	Single word buffer for data to be transmitted. When in master write or read_write mode, the transaction is started automatically

18.7.4.5 SPI*_RX_DATA

Bit Field	Read/Write	Field Name	Description
31:0	R	RX_DATA	Single word buffer for received data. When in master read mode, a new transaction is started automatically

18.7.4.6 SPI*_RX_DATA_NO_START

Bit Field	Read/Write	Field Name	Description
31:0	R	RX_DATA	Single word buffer for received data. Does not start a new transaction in master read mode, but does clear the RX_REQ flag

Ezairo 8300 Hardware Reference

18.7.4.7 SPI*_RX_DATA_MIRROR

Bit Field	Read/Write	Field Name	Description
31:0	R	RX_DATA	Single word buffer for received data. Does not start a new transaction and does clear the RX_REQ flag

18.8 UNIVERSAL ASYNCHRONOUS RECEIVER-TRANSMITTER (UART) INTERFACES

The general-purpose Universal Asynchronous Receiver-Transmitter (UART) interface provides support for communicating with devices that use standard UART and RS-232 transmission protocols.

The UART interface is multiplexed onto the DIO pads, which can be configured for the UART interface's receiver and transmitter signals, with the necessary physical pad configuration (pull-up/pull-down resistor and low pass filtering configuration for the RX signal, and drive strength configuration for the TX signal). For more information about configuring the multiplexed DIO functionality, see [Chapter 16.1 "Overview" on page 463](#).

The UART interface can be enabled using the `UART_CTRL_ENABLE` bit or disabled using the `UART_CTRL_DISABLE` bit in the appropriate `UART_CTRL` register. When the `UART_CTRL_DISABLE` bit is set to 1, any ongoing transaction, TX or RX, is completed before disabling the interface, and the UART transmit line `UART_TX` is set high.

A reset can be performed on the UART interface by writing to the `UART_CTRL_RESET` bit in this register. This causes the UART interface to be disabled, and the register values to be reset to their power-on defaults.

The UART interface operates in half- or full-duplex mode using a standard data format of 1 start bit, 8 data bits, 1 stop bit, and no parity bits. All data bytes being sent or received are interpreted as starting with the LSB. The "UART Transaction Waveform" figure (Figure 40) shows the waveform for a UART transmit or receive transaction.

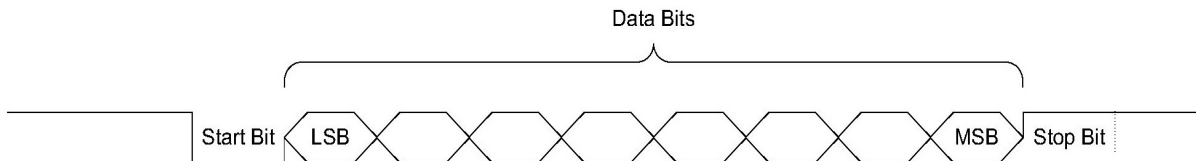


Figure 40. UART Transaction Waveform

Data transmission is started by writing to the `UART_TX_DATA` register. Data that has been received over the UART interface is stored in the `UART_RX_DATA` register. The `UART_TX_DATA` and `UART_RX_DATA` registers are only accessible after the interface has been enabled via the `UART_CTRL` register.

NOTE: When the UART interface is disabled using the `UART_CTRL_ENABLE` bit, data transmissions that are in progress complete before the interface shuts down.

The baud rates (specified in bits per second) for the UART interfaces are defined in terms of the `UARTCLK` frequency and several configuration parameters, as seen in the following equation for baud rate calculation, with `UARTCLK` frequency in Hz:

$$\text{baud_rate} = \frac{(\text{CNT_STEP} + 1) \times f_{\text{UARTCLK}}}{262144}$$

$$\text{CNT_STEP} = \text{round}\left(\frac{\text{baud_rate} \times 262144}{f_{\text{UARTCLK}}}\right) - 1$$

For a UARTCLK frequency of 3.84 MHz, the baud rate is determined as follows:

$$\text{baud_rate} = (\text{CNT_STEP} + 1) \times 14.6484375$$

$$\text{CNT_STEP} = \text{round}\left(\frac{\text{baud_rate}}{14.6484375}\right) - 1$$

For example, if a baud rate of 115200 bps is desired, the value of CNT_STEP must be set to 7863 for a UARTCLK frequency of 3.84 MHz, which results in an actual baud rate of 115195.3125 bps.

The UART interface runs on UARTCLK, which can be sourced from either ADCCLK or SYSCLK. This selection is available in the clock generation block (see [Section 1.0.1 “Setting the SCL Time Base” on page 1](#)). The ADCCLK, which is set to a frequency 3.84 MHz, is sufficient to cover all but very high baud rates. Therefore, on boot up, the UARTCLK source is set to ADCCLK. Definitions for standard baud rates are provided for the UART_CFG_CNT_STEP bit field in the UART_CFG register when sourcing from ADCCLK. Higher baud rates are possible when UARTCLK is sourced from SYSCLK, which runs at a higher frequency; however, sourcing from a higher frequency clock might not be desirable at a low frequency due to increased baud rate error.

IMPORTANT: For proper functionality, UART interfaces require both sides of a connection to have an absolute clock accuracy error of less than 2.5%. To allow for clock jitter, it is recommended that all UART communications use a clock with a maximum of 2% error versus the expected target frequency.

Data transfers using the UART interfaces can be controlled by one of the host processors using a memory-mapped register interface and/or the DMA block. The UART_CFG_TX_DMA_ENABLE and UART_CFG_RX_DMA_ENABLE bits in the UART_CFG register determine which request, whether read or write, needs to be completed using the DMA block. Both RX/TX interrupts and RX/TX DMA requests can be generated concurrently; if DMA requests are used, the data transfer requested is fulfilled by the DMA before the interrupt service routine is executed. User applications that do not use DMA or interrupts can poll the UART_STATUS_TX_REQ and UART_STATUS_RX_REQ bits in the UART_STATUS register to control data transfers. When UART_STATUS_TX_REQ is set to 1, the application can write to the UART_TX_DATA register to send data over the UART interface. When a new byte has been received over the UART interface and written to the UART_RX_DATA register, UART_STATUS_RX_REQ is set to 1. To avoid loss of data, the application must read the received byte using the processor or DMA before the next byte is received; otherwise an overrun occurs, and the UART_STATUS_OVERRUN flag is set to 1.

18.8.1 UART Interrupts

The UART interface uses three associated interrupts that separately control transmission and reception of UART data and handling of UART transmission errors. See [Section 4.6.1 “Nested Vectored Interrupt Controller \(NVIC\)” on page 135](#) for information regarding interrupt configuration and handling interrupts for the Arm Cortex-M3 processor. See [Section 3.2 “Interrupt Controller” on page 61](#) for information regarding interrupt configuration and handling for the CFX DSP.

Ezairo 8300 Hardware Reference

If a user application is transmitting data from the UART interface, the `UART_TX` interrupt indicates that data transmission can begin, and that the application can now load the first byte into the `UART_TX_DATA` register if this is the first interrupt, or load the next byte for subsequent interrupts. The next TX interrupt is generated as soon as the TX transmission has begun. To enable the `UART_TX` interrupt, set the `UART_CFG_TX_INT_ENABLE` bit in the `UART_CFG` register to 1.

If a user application is receiving data from the UART interface, the `UART_RX` interrupt indicates that a data byte has been successfully received and has been written to the `UART_RX_DATA` register. To enable the `UART_RX` interrupt, set the `UART_CFG_RX_INT_ENABLE` bit in the `UART_CFG` register to 1.

When using the DMA or the register interface to control data transfers over a UART interface, the `UART_ERROR` interrupt indicates that an overrun has occurred when data received from the UART interface has not been read by the DMA or processor before being overwritten. Detection of an overrun condition is enabled using the `UART_CTRL_OVERRUN_INT_ENABLE` bit in the `UART_CTRL` register. When this occurs, the `UART_STATUS_OVERRUN_STATUS` sticky bit from the `UART_STATUS` register is set. To clear it, write 1 to the `UART_STATUS_OVERRUN_CLEAR` bit.

NOTE: Once a value of 1 is written to the `UART_RX_OVERRUN_CLEAR` bit, it is automatically reset in the next clock cycle. Consequently, this bit is always read back as 0.

18.8.2 UART Interface Registers

Register Name	Register Description	Address
<code>UART0_CFG</code>	UART Configuration Register	P:0x13C0 0x40000F00
<code>UART0_CTRL</code>	UART Control Register	P:0x13C1 0x40000F04
<code>UART0_STATUS</code>	UART Status Register	P:0x13C2 0x40000F08
<code>UART0_TX_DATA</code>	UART Transmit Data Register	P:0x13C3 0x40000F0C
<code>UART0_RX_DATA</code>	UART Receive Data Register	P:0x13C4 0x40000F10

18.8.2.1 UART*_CFG

Bit Field	Read/Write	Field Name	Description
21	RW	<code>TX_DMA_ENABLE</code>	Enable/disable the TX DMA request
20	RW	<code>RX_DMA_ENABLE</code>	Enable/disable the RX DMA request
19	RW	<code>TX_END_INT_ENABLE</code>	Enable/disable the TX end interrupt
18	RW	<code>TX_START_INT_ENABLE</code>	Enable/disable the TX start interrupt

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
17	RW	RX_INT_ENABLE	Enable/disable the RX interrupt
16	RW	OVERRUN_INT_ENABLE	Enable/disable the overrun interrupt
15:0	RW	CNT_STEP	Counter step size that configures the baud rate

Field Name	Value Symbol	Value Description	Hex Value
TX_DMA_ENABLE	UART_TX_DMA_DISABLE	No TX DMA request is generated	0x0*
	UART_TX_DMA_ENABLE	A TX DMA request is generated when new data is requested by the UART interface	0x1
RX_DMA_ENABLE	UART_RX_DMA_DISABLE	No RX DMA request is generated	0x0*
	UART_RX_DMA_ENABLE	An RX DMA request is generated when new data is received by the UART interface	0x1
TX_END_INT_ENABLE	UART_TX_END_INT_DISABLE	No TX interrupt is raised	0x0*
	UART_TX_END_INT_ENABLE	A TX interrupt is raised when data transmission by the UART interface is finished	0x1
TX_START_INT_ENABLE	UART_TX_START_INT_DISABLE	No TX interrupt is raised	0x0*
	UART_TX_START_INT_ENABLE	A TX interrupt is raised when new data is requested by the UART interface	0x1
RX_INT_ENABLE	UART_RX_INT_DISABLE	No RX interrupt is raised	0x0*
	UART_RX_INT_ENABLE	An RX interrupt is raised when new data is received by the UART interface	0x1
OVERRUN_INT_ENABLE	UART_OVERRUN_INT_DISABLE	No ERROR interrupt is raised when an overrun is detected	0x0*
	UART_OVERRUN_INT_ENABLE	An ERROR interrupt is raised when an overrun occurs on the UART interface	0x1
CNT_STEP	UART_9600_BAUD	9600 baud at UARTCLK 3.84 MHz	0x28E
	UART_14400_BAUD	14400 baud at UARTCLK 3.84 MHz	0x3D6
	UART_19200_BAUD	19200 baud at UARTCLK 3.84 MHz	0x51E
	UART_38400_BAUD	38400 baud at UARTCLK 3.84 MHz	0xA3C
	UART_57600_BAUD	57600 baud at UARTCLK 3.84 MHz	0xF5B
	UART_115200_BAUD	115200 baud at UARTCLK 3.84 MHz	0x1EB7
	UART_120000_BAUD	120000 baud at UARTCLK 3.84 MHz	0x1FFF
	UART_240000_BAUD	240000 baud at UARTCLK 3.84 MHz	0x3FFF
	UART_480000_BAUD	480000 baud at UARTCLK 3.84 MHz	0x7FFF

Ezairo 8300 Hardware Reference

18.8.2.2 UART*_CTRL

Bit Field	Read/Write	Field Name	Description
8	R	ENABLE_STATUS	UART enable status
2	W	RESET	Reset the UART interface
1	W	DISABLE	Disable the UART interface
0	W	ENABLE	Enable the UART interface

Field Name	Value Symbol	Value Description	Hex Value
ENABLE_STATUS	UART_STATUS_DISABLED	UART interface is disabled	0x0*
	UART_STATUS_ENABLED	UART interface is enabled	0x1
RESET	UART_RESET	Reset the UART interface	0x1
DISABLE	UART_DISABLE	Disable the UART interface	0x1
ENABLE	UART_ENABLE	Enable the UART interface	0x1

18.8.2.3 UART*_STATUS

Bit Field	Read/Write	Field Name	Description
12	R	TX_BUSY	Indicate that a TX transaction is ongoing
11	R	RX_BUSY	Indicate that a RX transaction is ongoing
10	R	TX_REQ	Indicate that a TX data can be written
9	R	RX_REQ	Indicate that a RX data can be read
8	R	OVERRUN	Indicate that an overrun occurred when receiving data
0	W	OVERRUN_CLEAR	Clear the overrun status flag

Field Name	Value Symbol	Value Description	Hex Value
TX_BUSY	UART_TX_IDLE	No UART TX transaction is ongoing	0x0*
	UART_TX_BUSY	UART TX transaction is ongoing	0x1
RX_BUSY	UART_RX_IDLE	No UART RX transaction is ongoing	0x0*
	UART_RX_BUSY	UART RX transaction is ongoing	0x1
TX_REQ	UART_TX_NO_REQ	UART TX data has already been written	0x0
	UART_TX_REQ	UART TX data can be written	0x1*
RX_REQ	UART_RX_NO_REQ	No new UART RX data available	0x0*
	UART_RX_REQ	New UART RX data available	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
OVERRUN	UART_OVERRUN_FALSE	No UART RX overrun detected	0x0*
	UART_OVERRUN_TRUE	UART RX overrun detected	0x1
OVERRUN_CLEAR	UART_OVERRUN_CLEAR	Clear the RX overrun status bit	0x1

18.8.2.4 UART*_TX_DATA

Bit Field	Read/Write	Field Name	Description
7:0	RW	TX_DATA	Transmitted data

18.8.2.5 UART*_RX_DATA

Bit Field	Read/Write	Field Name	Description
7:0	R	RX_DATA	Received data

CHAPTER 19

Peripherals

The Ezairo 8300 system has a number of peripherals, accessible to the CFX DSP and the Arm Cortex-M3 core, to support various tasks that the system needs to perform. In addition, several system peripherals are accessible only to the Arm Cortex-M3 processor — the so-called “private peripherals”.

19.1 ASYNCHRONOUS SAMPLE RATE CONVERTER (ASRC)

The asynchronous sample rate converter (ASRC) block provides a means of synchronizing the audio sample rate, between the radio and a local source or sink, for the transferred audio data.

The ASRC is configured with the ASRC_CFG register, which sets up the following modes and interrupts:

- The selected ASRC frequency conversion mode. There are four frequency sets of conversion rates supported by different modes, controlled by the ASRC_CFG_MODE bit field. Use of these modes is dependent on the relationship between the source (F_{src}) and sink (F_{sink}) frequencies, as shown in Table 65.
- The ASRC bandwidth mode, configured with the ASRC_CFG_WDF_TYPE bit, which can select between:
 - Low delay mode: this mode provides the lowest group delay setting, but at a slightly lower bandwidth.
 - Wide band mode: this mode provides the highest bandwidth setting, but at a higher group delay.
- The ASRC supports five interrupts, configured using the ASRC_CFG_*_INT_ENABLE bits:
 - The ASRC_CFG_DATA_IN_INT_ENABLE bit can be set to trigger an ASRC_IN interrupt when more input data is required. This mode is used when providing audio samples using interrupts.
 - The ASRC_CFG_DATA_OUT_INT_ENABLE bit can be set to trigger an ASRC_OUT interrupt when more output data is available. This mode is used when reading audio samples using interrupts.
 - The ASRC_CFG_IN_ERR_INT_ENABLE bit can be set to trigger an ASRC_ERROR interrupt when data is written to the ASRC_IN register before processing of the previously written value is complete.
 - The ASRC_CFG_UPDATE_ERR_INT_ENABLE bit can be set to trigger an ASRC_ERROR interrupt when any of the ASRC_PHASE_CNT, ASRC_PHASE_INC, or ASRC_OUTPUT_CNT registers are written to while processing of a sample is ongoing.
 - The ASRC_CFG_PROC_DONE_INT_ENABLE bit can be set to trigger an interrupt when the ASRC is finished generating the last sample.

The ASRC_PHASE_INC register contains a signed 32-bit value in a signed m8p24 format, that is used to control the decimation factor of the ASRC. The ASRC_PHASE_INC value is dependent on ASRC_CFG_MODE, and on the difference between F_{src} and F_{sink} . The format adjusted formulas for setting ASRC_PHASE_INC are listed below in the “ASRC Settings” table (Table 65).

Table 65. ASRC Settings

ASRC_CFG_ASRC_MODE	F _{sink} range		ASRC_PHASE_INC	ASRC_STATE_MEM size	
	Minimum	Maximum		Low delay	Wide band
ASRC_INT_MODE	> F_{src}	N/A	$64 \left(\frac{F_{src} - F_{sink}}{F_{sink}} \right)$	12	18
ASRC_DEC_MODE1	$0.875 \cdot F_{src}$	$1.125 \cdot F_{src}$		18	30
ASRC_DEC_MODE2	$0.4 \cdot F_{src}$	F_{src}	64	17	23

Ezairo 8300 Hardware Reference

ASRC Settings (Continued)

ASRC_CFG_ASRC_MODE	F _{sink} range		ASRC_PHASE_INC	ASRC_STATE_MEM size	
	Minimum	Maximum		Low delay	Wide band
ASRC_DEC_MODE3	0.2*F _{src}	0.5*F _{src}	$16 \left(\frac{F_{src} - 2 \bullet F_{\sin k}}{F_{\sin k}} \right)$	20	26

The ASRC_PHASE_CNT register controls which filter coefficients are used by the ASRC block. The format for this register is unsigned m8p24, and the register is updated as follows for every output sample:

ASRC_CFG_MODE < 2

ASRC_PHASE_COUNT = *modulo*(ASRC_PHASE_CNT + 2*ASRC_PHASE_INC, WRAP)

where WRAP = 64

ASRC_CFG_MODE ≥ 2

ASRC_PHASE_COUNT = *modulo*(ASRC_PHASE_CNT + 4*ASRC_PHASE_INC, WRAP)

where WRAP = 32

For the cases where ASRC_PHASE_INC ≥ 0 (downsampling), wrapping for the ASRC_PHASE_CNT register is handled as follows:

If previous update caused a wrap

If ASRC_PHASE_CNT < WRAP: ASRC_PHASE_CNT = ASRC_PHASE_CNT

If ASRC_PHASE_CNT ≥ WRAP: ASRC_PHASE_CNT = ASRC_PHASE_CNT - WRAP

If previous update did not cause a wrap

If ASRC_PHASE_CNT + ASRC_PHASE_INC < WRAP: ASRC_PHASE_CNT = ASRC_PHASE_CNT + ASRC_PHASE_INC

If ASRC_PHASE_CNT + ASRC_PHASE_INC ≥ WRAP: ASRC_PHASE_CNT = ASRC_PHASE_CNT + ASRC_PHASE_INC - WRAP

The ASRC is controlled by the ASRC_CTRL register. This register provides the following functionality:

- The ASRC can be enabled by setting the ASRC_CTRL_ENABLE bit, and disabled by setting the ASRC_CTRL_DISABLE bit.
- The ASRC_CTRL_RESET bit can be used to reset the ASRC state memory (stored in the ASRC_STATE_MEM registers).

The status of the ASRC can be read using the ASRC_STATUS register. Status bits indicate the following:

- The ASRC active status can be read from the ASRC_STATUS_EN_STATUS bit.
- The ASRC_STATUS_BUSY bit indicates if the ASRC is currently processing data.
- The ASRC_STATUS_IN_REQ bit indicates if the ASRC_IN register is in use, or if it is ready for more data.

Ezairo 8300 Hardware Reference

- The `ASRC_STATUS_OUT_REQ` bit indicates if the `ASRC_OUT` register contains no new data, or if it has new data that can be read.
- The `ASRC_STATUS_*_ERR` bits indicate what errors have been captured by the ASRC block, and the `ASRC_STATUS_*_ERR_CLEAR` bits can be written to clear these error indication bits.

Data is provided to the ASRC through the `ASRC_IN` register, and the `ASRC_CNT_INPUT_CNT` bit field in the `ASRC_CNT` register indicates how many samples have been received. Data is read from the ASRC through the `ASRC_OUT` register, and the `ASRC_CNT_OUTPUT_CNT` bit field in the `ASRC_CNT` register indicates how many samples have been generated. It is recommended to clear the counters before processing a new block of samples, as the counters wrap at 1024.

Once the ASRC is enabled, the behavior is as follows:

1. When enabled from reset or with the input state as the last state, the ASRC input request status bit is set. This indicates to the Arm Cortex-M3 Processor or the DMA that the ASRC has new data to be read.
2. When enabled with the output state as the last state, the ASRC output request status bit is set, indicating that the ASRC is ready to receive new data.
3. The ASRC filter process starts when a 16-bit input sample is written into the `ASRC_IN` register and the `ASRC_STATUS_OUT_REQ` bit of the `ASRC_STATUS` register is cleared, indicating there is no pending output to be read. Since the `ASRC_IN` register does not support byte write access, single byte writes are assumed to be 16-bit writes.
4. Once the `ASRC_IN` register is updated, the `ASRC_STATUS_IN_REQ` bit of the `ASRC_STATUS` register is also cleared.
5. When the ASRC has finished processing the current input sample, one of three operations will occur:
 - If an output sample is not generated due to the sink sample rate being less than the source sample rate, the `ASRC_STATUS_IN_REQ` bit is set instead of the `ASRC_STATUS_OUT_REQ` bit, indicating to the Arm Cortex-M3 Processor or the DMA that the `ASRC_IN` register can be updated.
 - If a single output sample is generated, the `ASRC_OUT` register is updated and the `ASRC_STATUS_OUT_REQ` bit is set, indicating to the Arm Cortex-M3 Processor or the DMA that the `ASRC_OUT` register is available to read. When the `ASRC_OUT` register is read, the `ASRC_STATUS_OUT_REQ` bit is cleared and the `ASRC_STATUS_IN_REQ` bit is set, allowing a new sample to be written to the `ASRC_IN` register.
 - If multiple output samples are generated due to the sink sample rate being greater than the source sample rate, the `ASRC_OUT` register is updated and the `ASRC_STATUS_OUT_REQ` bit is set, indicating that the `ASRC_OUT` register is available to be read.
 - When the `ASRC_OUT` register is read, `ASRC_STATUS_OUT_REQ` bit is cleared, allowing the ASRC to generate another sample. On generation of a new sample, `ASRC_OUT` and `ASRC_OUT_MIRROR` registers are updated and the `ASRC_STATUS_OUT_REQ` bit is set again. This step is repeated until all the output samples have been generated.
 - Once all the output values have been read, the `ASRC_STATUS_OUT_REQ` bit is cleared and the `ASRC_STATUS_IN_REQ` bit is set, allowing a new sample to be written into the `ASRC_IN` register.
6. Steps 3-5 are repeated as necessary in order to process the current block of samples.

The ASRC can be disabled while the input data is still being processed. If the accelerator is not in an idle state, it continues with its processing until a new input sample is required or a new output sample is ready to be read. Any new data written to the `ASRC_IN` register is ignored. However, the rest of the ASRC registers and internal states are accessible in the disabled state. To determine when the ASRC is finished, the `ASRC_STATUS_BUSY` bit in the `ASRC_STATUS` register can be monitored, or the `ASRC_CFG_PROC_DONE_INT_ENABLE` bit in the `ASRC_CFG` register can be set to 1 to trigger an interrupt when the last sample is generated.

Ezairo 8300 Hardware Reference

The ASRC can process a fixed number of input or output samples by monitoring the ASRC_CNT_INPUT_CNT and ASRC_CNT_OUTPUT_CNT fields of the ASRC_CNT register, respectively. Once the desired count is achieved, the ASRC can be disabled by writing 1 to the ASRC_CTRL_DISABLE bit in the ASRC_CTRL register. To verify whether the ASRC is disabled, the ASRC_STATUS_BUSY bit in the ASRC_STATUS register can be monitored, or the ASRC_CFG_PROC_DONE_INT_ENABLE bit in the ASRC_CFG register can be set to enable the interrupt.

The time it takes the ASRC to process one input sample can vary based on the mode, the filter type, and the frequency ratio. Similarly, the THD+N performance also depends on the same factors and ranges between -90 dB to -96 dB. The "Average Processing Time (SYSCLK cycles)" table (Table 66) outlines the processing times for various conditions in terms of SYSCLK cycles.

Table 66. Average Processing Time (SYSCLK cycles)

Mode 0 ($F_{src} = 16$ kHz)			Mode 1 ($F_{src} = 16$ kHz)			Mode 2 ($F_{src} = 16$ kHz)			Mode 3 ($F_{src} = 32$ kHz)		
F_{sink} (kHz)	Wide Band	Low Delay	F_{sink} (kHz)	Wide Band	Low Delay	F_{sink} (kHz)	Wide Band	Low Delay	F_{sink} (kHz)	Wide Band	Low Delay
16.0	46.0	40.0	14.7	56.5	45.0	6.66	28.0	25.8	6.00	19.5	18.4
20.0	53.8	47.8	15.5	58.6	46.8	7.00	35.5	32.5	8.00	25.0	23.5
24.0	61.5	55.5	15.96	59.9	47.9	7.35	42.6	38.9	10.00	29.9	28.0
32.0	77.0	71.0	16.0	59.0	47.0	8.00	46.5	42.4	11.03	32.6	30.5
40.0	92.5	86.5	16.03	59.1	47.1	10.6	50.3	45.8	12.0	35.3	33.0
44.1	100.4	94.4	16.7	60.9	48.7	15.3	62.9	57.1	15.9	45.2	42.3
48.0	108.0	102.0	--	--	--	--	--	--	--	--	--

The ASRC is capable of processing one audio channel at a time. In cases where more than one channel needs to be processed, the ASRC block's internal states need to be saved to memory after a block has been processed, and restored before the next block is processed. The ordered steps for multi-channel processing are outlined below:

1. Disable the ASRC after processing one audio channel by setting the ASRC_CTRL_DISABLE bit in the ASRC_CTRL register to 1. Wait until the ASRC is idle by monitoring the ASRC_STATUS_BUSY bit in the ASRC_STATUS register.
2. Copy the ASRC_OUT_MIRROR and ASRC_PHASE_CNT registers and the used ASRC_STATE_MEM registers to memory, using either the Arm Cortex-M3 Processor or the DMA interface.
 - ASRC_PHASE_CNT: This register holds the current phase of the polyphase filter.
 - ASRC_PHASE_MIRROR: This register holds the current output sample.
 - ASRC_STATE_MEM: This 30-value register array holds the internal filter states of the polyphase filter. The number of states that need to be stored depends on the ASRC_CFG_MODE setting, as outlined in Table 65 on page 675.

NOTE: The ASRC_CFG register bits are also mapped to byte 3 of the ASRC_STATE_MEM[8] register, and therefore the ASRC_CFG register does not have to be saved and restored separately.

3. Using the Arm Cortex-M3 Processor or the DMA interface, update the saved ASRC registers back from memory for the channel that is to be processed next.
4. Enable the ASRC by setting the ASRC_CTRL_ENABLE bit in the ASRC_CTRL register to 1.

Ezairo 8300 Hardware Reference

IMPORTANT: The ASRC must be disabled when saving or restoring its internal states. The behavior of the ASRC is undefined if the ASRC is active while its state is being updated.

There are two types of errors that can occur while the ASRC accelerator is active:

1. Input error: this error occurs if the input register is updated while the accelerator is still processing the previous sample. The accelerator ignores this update and continues its operation. When the processing is finished, this false update does not trigger a new process, and the input register needs to be updated with fresh data once the ASRC_STATUS_IN_REQ bit of the ASRC_STATUS register is set.
2. Update error: this error occurs if the ASRC_CFG_MODE and ASRC_CFG_WDF_TYPE fields of the ASRC_CFG register are changed, and if any of the ASRC_PHASE_CNT, ASRC_PHASE_INC, ASRC_CNT, and ASRC_STATE_MEM registers are updated while the accelerator is busy. It is recommended to reset the ASRC if this error occurs.

For each of these errors, the appropriate bits in the ASRC_STATUS register are set, and the appropriate interrupts are also generated if they are enabled in the ASRC_CFG register.

19.1.1 ASRC Control and Configuration Registers

Register Name	Register Description	Address
ASRC_CFG	ASRC Configuration Register	P:0x1140 0x40000500
ASRC_CTRL	ASRC Control Register	P:0x1141 0x40000504
ASRC_STATUS	ASRC Status Register	P:0x1142 0x40000508
ASRC_CNT	ASRC Sample Counters Register	P:0x1143 0x4000050C
ASRC_IN	ASRC Input Data Register	P:0x1144 0x40000510
ASRC_OUT	ASRC Output Data Register	P:0x1145 0x40000514
ASRC_OUT_MIRROR	ASRC Output Data Mirror Register	P:0x1146 0x40000518

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
ASRC_PHASE_INC	ASRC Phase Counter Increment Register	P:0x1147 0x4000051C
ASRC_PHASE_CNT	ASRC Phase Counter Register	P:0x1148 0x40000520
ASRC_STATE_MEM	ASRC State Memory 0 to 29 (32 bit)	P:0x1149-P:0x1166 0x40000524-0x40000598

19.1.1.1 ASRC_CFG

Bit Field	Read/Write	Field Name	Description
7	RW	PROC_DONE_INT_ENABLE	The ASRC processing done interrupt mask
6	RW	UPDATE_ERR_INT_ENABLE	The ASRC state/configuration update error interrupt mask
5	RW	IN_ERR_INT_ENABLE	The ASRC input interface error interrupt mask
4	RW	DATA_OUT_INT_ENABLE	The ASRC_OUT register interrupt status
3	RW	DATA_IN_INT_ENABLE	The ASRC_IN register interrupt status
2	RW	WDF_TYPE	WDF Type Selection
1:0	RW	MODE	ASRC mode

Field Name	Value Symbol	Value Description	Hex Value
PROC_DONE_INT_ENABLE	ASRC_PROC_DONE_INT_DISABLE	This source can not set an interrupt	0x0
	ASRC_PROC_DONE_INT_ENABLE	This source can set the interrupt line	0x1*
UPDATE_ERR_INT_ENABLE	ASRC_UPDATE_ERR_INT_DISABLE	This source can not set an interrupt	0x0
	ASRC_UPDATE_ERR_INT_ENABLE	This source can set the interrupt line	0x1*
IN_ERR_INT_ENABLE	ASRC_IN_ERR_INT_DISABLE	This source can not set an interrupt	0x0*
	ASRC_IN_ERR_INT_ENABLE	This source can set the interrupt line	0x1
DATA_OUT_INT_ENABLE	ASRC_DATA_OUT_INT_DISABLE	This source can not set an interrupt	0x0*
	ASRC_DATA_OUT_INT_ENABLE	This source can set the interrupt line	0x1
DATA_IN_INT_ENABLE	ASRC_DATA_IN_INT_DISABLE	This source can not set an interrupt	0x0*
	ASRC_DATA_IN_INT_ENABLE	This source can set the interrupt line	0x1
WDF_TYPE	ASRC_LOW_DELAY	Low Delay filter	0x0*
	ASRC_WIDE_BAND	Wide band response filter	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
MODE	ASRC_INT_MODE	Interpolation mode	0x0*
	ASRC_DEC_MODE1	Decimation mode 1	0x1
	ASRC_DEC_MODE2	Decimation mode 2	0x2
	ASRC_DEC_MODE3	Decimation mode 3	0x3

19.1.1.2 ASRC_CTRL

Bit Field	Read/Write	Field Name	Description
8	R	ENABLE_STATUS	ASRC enable status
2	W	RESET	Write a 1 to reset ASRC
1	W	DISABLE	Disable the re-sampler block
0	W	ENABLE	Enable the re-sampler block

Field Name	Value Symbol	Value Description	Hex Value
ENABLE_STATUS	ASRC_DISABLED	The re-sampler is disabled	0x0*
	ASRC_ENABLED	The re-sampler is enabled	0x1
RESET	ASRC_RESET	Reset ASRC	0x1
DISABLE	ASRC_DISABLE	Disable the re-sampler	0x1*
ENABLE	ASRC_ENABLE	Enable the re-sampler block	0x1

19.1.1.3 ASRC_STATUS

Bit Field	Read/Write	Field Name	Description
12	R	BUSY	The ASRC processing state
11	R	OUT_REQ	The ASRC_OUT register status
10	R	IN_REQ	The ASRC_IN register status
9	R	UPDATE_ERR	The ASRC state/configuration update error interrupt status
8	R	IN_ERR	The ASRC input interface error interrupt status
1	W	UPDATE_ERR_CLEAR	Clear the ASRC update/configuration error interrupt status
0	W	IN_ERR_CLEAR	Clear the ASRC input interface error interrupt

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
BUSY	ASRC_IDLE	The ASRC is idle	0x0*
	ASRC_BUSY	The ASRC is busy processing a sample	0x1
OUT_REQ	ASRC_OUT_IDLE	The ASRC_OUT register is idle	0x0*
	ASRC_OUT_RDY	The ASRC_OUT register is ready to be read	0x1
IN_REQ	ASRC_IN_IDLE	The ASRC_IN register is idle	0x0*
	ASRC_IN_RDY	The ASRC_IN register is requesting an input	0x1
UPDATE_ERR	ASRC_UPDATE_ERR_INT_IDLE	This source has not set an interrupt	0x0*
	ASRC_UPDATE_ERR_INT_PENDING	The internal states or configuration updated while the accelerator is busy	0x1
IN_ERR	ASRC_IN_ERR_INT_IDLE	This source has not set an interrupt	0x0*
	ASRC_IN_ERR_INT_PENDING	The ASRC input interface register overwritten before the accelerator finished its processing	0x1
UPDATE_ERR_CLEAR	ASRC_UPDATE_ERR_CLEAR	Clear the ASRC update error interrupt	0x1
IN_ERR_CLEAR	ASRC_IN_ERR_CLEAR	Clear the ASRC IN error interrupt	0x1

19.1.1.4 ASRC_CNT

Bit Field	Read/Write	Field Name	Description
27:16	RW	OUTPUT_CNT	ASRC output sample counter
11:0	RW	INPUT_CNT	ASRC input sample counter

19.1.1.5 ASRC_IN

Bit Field	Read/Write	Field Name	Description
15:0	RW	ASRC_IN	Audio sample input

19.1.1.6 ASRC_OUT

Bit Field	Read/Write	Field Name	Description
15:0	R	ASRC_OUT	Audio sample output

19.1.1.7 ASRC_OUT_MIRROR

Bit Field	Read/Write	Field Name	Description
15:0	RW	ASRC_OUT	Audio sample mirror output

Ezairo 8300 Hardware Reference

19.1.1.8 ASRC_PHASE_INC

Bit Field	Read/Write	Field Name	Description
31:0	RW	STEP	ASRC_PHASE_INC

19.1.1.9 ASRC_PHASE_CNT

Bit Field	Read/Write	Field Name	Description
31:0	RW	PHASE_CNT	ASRC phase counter

19.1.1.10 ASRC_STATE_MEM*

Bit Field	Read/Write	Field Name	Description
31:0	RW	STATE_MEM	ASRC State Memory 0 to 29

19.2 AUDIO SINK CLOCK COUNTERS

Ezairo 8300 includes two asynchronous clock counters used to coordinate signals on a device operating as an audio sink. They can be used to measure the timing of the clock of a connected external device, and to measure the internal audio sampling rate. In addition, these asynchronous clock counters can be used for measuring the relative frequency of the internal standby clock oscillator and the CCO oscillator or an external clock source used as SYSCLK.

The following uses are supported:

- Asynchronous sample rate conversion on the audio samples being received from and/or sent to an external device

NOTE: If conversion is not performed against the internal sampling rate, then both audio sink clock counters are required: one for the audio sample source, and one for the external clock. The counts from both audio sink clock counters are used to calculate the required phase increment counter for asynchronous sample rate conversions. For more details on asynchronous sample rate conversion, see [Section 19.1 “Asynchronous Sample Rate Converter \(ASRC\)” on page 675](#).

- Generation of control information to synchronize the system's sampling frequency with the framing signal of an external audio source, such as an RF device that is receiving streaming audio
- Calibration of the internal CCO oscillator by inserting a reference clock on a DIO pad
- Measurement of the internal STANDBYCLK oscillator relative to the ADCCLK frequency through a DIO pad
- Measurement of the FIFO interrupt period relative to the ADCCLK frequency through a DIO pad

The clock being measured is sourced from a DIO or from STANDBYCLK, as specified by the DIO configuration. For more information on DIO configuration for use as the audio sink source, see [Section 16.2 “Functional Configuration” on page 464](#).

The timing diagram in the ["Audio Sink Timing" figure \(Figure 41\)](#) highlights the measurement registers in the Audio Sink module.

Ezairo 8300 Hardware Reference

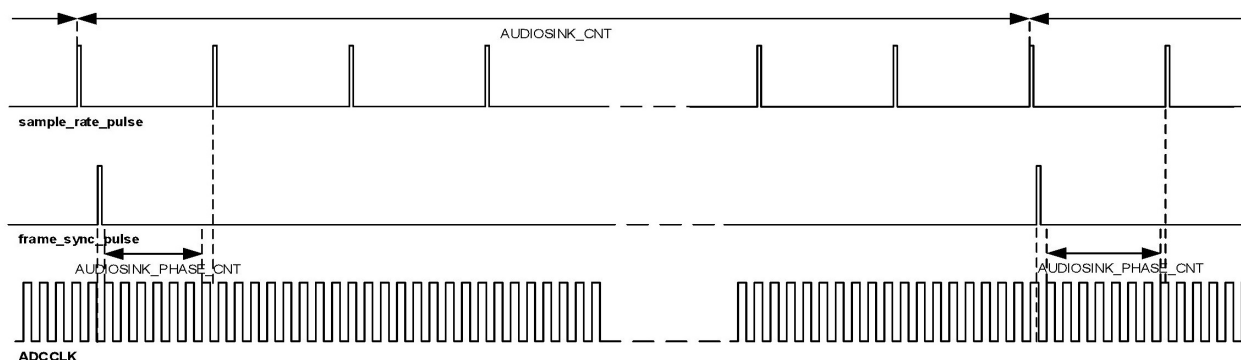


Figure 41. Audio Sink Timing

Bit fields from the `AUDIOSINK*_CFG` registers are used to configure the audio sink clock counters, including:

- Frame sync pulse selection, configured by writing to the `AUDIOSINK_CFG_FRAME_SYNC_SEL` bits. The frame sync source can be set to be one of the 8 available DIO interrupts. The frame sync pulses can also be manually generated by setting the `AUDIOSINK_CTRL_FRAME_SYNC_TRIGGER` bit in the `AUDIOSINK_CTRL` register.
- Sampling clock source selection, configured by writing to the `AUDIOSINK_CFG_AUDIOSINK_SEL` bits. The sampling clock can be sourced from any of the four ADCs, or from one of the eight available FIFO interrupts.
- Audio Sink enable status, configured by writing to the `AUDIOSINK_CFG_ENABLE` bit.

Instead of sampling from a clock source, the signal to be sampled can be derived from a FIFO interrupt or from the creation of decimated samples from the input stage. If the sampled signal source is a FIFO interrupt, the audio sink clock counter increments at the block rate of the selected FIFO. When deriving the clock from the creation of data in the input stage, it is important for the appropriate decimation filter to be enabled so that the ADC sample rate pulse is generated. If the signal source is a decimated ADC, the audio sink clock counter increments at the sampling rate. See [Chapter 12.1 "Introduction to the FIFO Controller" on page 387](#) and see [Chapter 14.1 "Overview" on page 438](#) for information on how to configure the registers for FIFO interrupts and ADC decimators, respectively.

The measurement registers are defined as follows:

- The `AUDIOSINK_CNT` registers hold the integer number of audio sink clock cycles being measured between consecutive frame synchronization pulses.
- The `AUDIOSINK_PHASE_CNT` register measures the time from the frame sync pulse to the first detected rising edge of the audio sink clock, in terms of `ADCCLK` cycles. This measurement is used to improve the resolution of the calculated number of audio sink clock cycles per frame. This counter is supported by:
 - The `AUDIOSINK_STATUS_PHASE_CNT_READY` bit, which indicates whether a new phase counter measurement is ready. This bit is reset automatically every time the `AUDIOSINK_PHASE_CNT` register is read.
 - The `AUDIOSINK_PHASE_CNT` register, which contains the number of `ADCCLK` cycles between when the counter starts and when a rising edge is detected on the audio sink clock. Note that:
 - Using the sample rate as the audio sink clock source results in the `AUDIOSINK_PHASE_CNT` register being a measurement between 0 and 512, as the ratio of `ADCCLK` to the sampling frequency has a maximum of 512.
 - When using a FIFO interrupt to control flow, it is possible that the phase measurement can be saturated, as this bit field is limited to 14-bits (0x3FFF) with no overflow.

Ezairo 8300 Hardware Reference

- The `AUDIOSINK_PHASE` interrupt, which is triggered when a rising edge occurs on the audio sink clock and the `AUDIOSINK_PHASE_CNT` is ready to be read

Using the values obtained from the `AUDIOSINK_CNT` and `AUDIOSINK_PHASE_CNT` registers plus the previous phase count value, you can calculate the number of audio sink clock cycles between the previous (i-1) and the current (i) synchronization pulses as follows:

$$\text{audio sink clock cycles} = \text{AUDIOSINK_CNT}[i] + \frac{\text{AUDIOSINK_PHASE_CNT}[i-1] - \text{AUDIOSINK_PHASE_CNT}[i]}{(\text{ADC_SAMPLE_FREQ_CFG} + 1) \cdot 8}$$

The recommended process for using the measurement registers is as follows:

- When the `AUDIOSINK_PHASE` interrupt occurs:
 - a. Save the `AUDIOSINK_PHASE_CNT` counter value from the previous frame.
 - b. Record the `AUDIOSINK_PHASE_CNT` counter value.
 - c. Record the value in the `AUDIOSINK_CNT` register.

19.2.1 Audio Sink Control and Configuration Registers

Register Name	Register Description	Address
<code>AUDIOSINK0_CFG</code>	Audio Sink Clock Counters Configuration Register	P:0x10C0 0x40000300
<code>AUDIOSINK0_CTRL</code>	Audio Sink Clock Counters Control Register	P:0x10C1 0x40000304
<code>AUDIOSINK0_STATUS</code>	Audio Sink Clock Counters Status Register	P:0x10C2 0x40000308
<code>AUDIOSINK0_CNT</code>	Audio Sink Clock Counter Register	P:0x10C3 0x4000030C
<code>AUDIOSINK0_PHASE_CNT</code>	Audio Sink Clock Phase Counter Register	P:0x10C4 0x40000310
<code>AUDIOSINK1_CFG</code>	Audio Sink Clock Counters Configuration Register	P:0x1100 0x40000400
<code>AUDIOSINK1_CTRL</code>	Audio Sink Clock Counters Control Register	P:0x1101 0x40000404

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
AUDIOSINK1_STATUS	Audio Sink Clock Counters Status Register	P:0x1102 0x40000408
AUDIOSINK1_CNT	Audio Sink Clock Counter Register	P:0x1103 0x4000040C
AUDIOSINK1_PHASE_CNT	Audio Sink Clock Phase Counter Register	P:0x1104 0x40000410

19.2.1.1 AUDIOSINK*_CFG

Bit Field	Read/Write	Field Name	Description
10:8	RW	FRAME_SYNC_SEL	Frame sync source selection
7:4	RW	AUDIOSINK_SEL	Audio sink clock source selection
0	RW	ENABLE	Enable the Audio Sink Clock Counters

19.2.1.2 AUDIOSINK*_CTRL

Bit Field	Read/Write	Field Name	Description
0	W	FRAME_SYNC_TRIGGER	Generate a frame sync pulse manually

Field Name	Value Symbol	Value Description	Hex Value
FRAME_SYNC_TRIGGER	FRAME_SYNC_TRIGGER	Generate a frame sync pulse manually	0x1

19.2.1.3 AUDIOSINK*_STATUS

Bit Field	Read/Write	Field Name	Description
0	R	PHASE_CNT_READY	Audio sink clock phase counter status

Field Name	Value Symbol	Value Description	Hex Value
PHASE_CNT_READY	PHASE_CNT_NOT_READY	No new audio sink clock phase counter value is available	0x0*
	PHASE_CNT_READY	A new audio sink clock phase counter value is available	0x1

Ezairo 8300 Hardware Reference

19.2.1.4 AUDIOSINK*_CNT

Bit Field	Read/Write	Field Name	Description
11:0	R	CNT	Sink clock counter value

19.2.1.5 AUDIOSINK*_PHASE_CNT

Bit Field	Read/Write	Field Name	Description
13:0	R	PHASE_CNT	Sink clock phase counter value

19.3 CYCLIC REDUNDANCY CHECK (CRC) GENERATORS

Creating a CRC is a common but moderately expensive software calculation; therefore, Ezairo 8300 contains two hardware CRC generator modules. Both modules are accessible from both the CFX DSP and the Arm Cortex-M3 processor, with unique register names for each generator that are consistent across both cores. Both CRC generator modules support the CRC-CCITT algorithm to create 16-bit CRCs. The CRC generator associated with the Arm Cortex-M3 processor also supports the CRC-32 algorithm (defined by the 802.3 Ethernet standard) to create 32-bit CRCs.

The calculated outputs from these generators can be employed by a user application to ensure data integrity of communications and non-volatile memory information. They ensure integrity by guaranteeing that all single-bit errors, two-bit errors, burst errors (i.e., multiple bit errors in a row), and any error containing an odd number of bits can be detected.

19.3.1 CRC-CCITT Generators

The CRC generator associated with the Arm Cortex-M3 processor can be configured to select the CRC-CCITT algorithm, by clearing the `CRC_CFG_CRC_TYPE` bit in the `CRC_CFG` register to the `CRC_CCITT` bit setting. The other CRC generator always uses the CRC-CCITT algorithm and has no mode configuration.

The parameters associated with the CRC-CCITT algorithm implementation are provided in the ["CRC-CCITT Algorithm Parameters" table \(Table 67\)](#).

Table 67. CRC-CCITT Algorithm Parameters

CRC Parameter	Parameter Value
Order	16
Polynomial	$x^{16} + x^{12} + x^5 + 1$
Polynomial (hex)	0x1021
Initial Value (hex)	0xFFFF
Final XOR Value (hex)	0x0000

To use either CRC-CCITT generator:

1. Write an initial value of 0xFFFF to the `CRC_VALUE/D_CRC_VALUE` register.

Ezairo 8300 Hardware Reference

- Write any data that needs to be included in the final CRC read from the generator to the CRC input registers (CRC_ADD_*/D_CRC_ADD_*, where * can be 1, 8, 16, 24 or 32 to add the specified number of bits to the calculation).

At any time, you can read the CRC_FINAL/D_CRC_FINAL register to obtain the CRC result for the data that has been added using the CRC input registers since the last initialization of the respective CRC_VALUE register. Additionally, under standard configuration, the CRC_VALUE register contains the same value as the CRC_FINAL register. If a non-standard variant of CRC-CCITT has been selected, the valid result resides in CRC_FINAL.

No data manipulation is required for the output CRC generated for the standard CRC-CCITT algorithm (i.e., no data byte reversal, reversal of the final result, or other finalization). To configure the generator to use non-standard variants of CRC-CCITT with finalization, see [Section 19.3.3 “Non-standard CRC Variants” on page 688](#).

19.3.2 CRC-32 Generator

The CRC generator associated with the Arm Cortex-M3 processor can be configured to select the CRC-32 algorithm, by setting the CRC_CFG_CRC_TYPE bit in the CRC_CFG register to the CRC_32 bit setting.

The parameters associated with the CRC-32 algorithm implementation are provided in the "CRC-32 Algorithm Parameters" table (Table 68), below. The output CRC generated for the standard CRC-32 algorithm requires both data byte reversal and reversal of the final result.

Table 68. CRC-32 Algorithm Parameters

CRC Parameter	Parameter Value
Order	32
Polynomial	$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$
Polynomial (hex)	0x4C11DB7
Initial Value (hex)	0xFFFFFFFF
Final XOR Value (hex)	0xFFFFFFFF

To use the CRC-32 generator located in the Arm Cortex-M3 processor:

- Write an initial value of 0xFFFFFFFF to the CRC_VALUE register.
- Write any data that needs to be included in the final CRC read from the generator to the CRC input registers (CRC_ADD_*, where * can be 1, 8, 16, 24 or 32 to add the specified number of bits to the calculation).

At any time, you can read the CRC_VALUE register to obtain the calculated CRC result for the data that has been added using the CRC input registers since the last initialization of the CRC_VALUE register, without any finalization, such as data-byte reversal or reversal of the final result. Similarly, the calculated CRC result, with finalization applied, can be found in the CRC_FINAL register.

19.3.3 Non-standard CRC Variants

The CRC generator associated with the Arm Cortex-M3 processor supports non-standard variants of the CRC-CCITT and CRC-32 standard implementation.

- To use a non-standard CRC ordering of data within each data byte, set the CRC_CFG_BIT_ORDER bit in the CRC_CFG register.

Ezairo 8300 Hardware Reference

- To use a non-standard CRC ordering of the final result, set the `CRC_CFG_FINAL_CRC_REVERSE` bit in the `CRC_CFG` register.
- To use a non-standard CRC XOR of the final result, set the `CRC_CFG_FINAL_CRC_XOR` bit in the `CRC_CFG` register. If configured for a non-standard XOR, this uses a final XOR value of 0xFFFF for CRC-CCITT and 0x00000000 for CRC-32.
- To change the endianness of bytes added to the CRC to a little endian direction, set the `CRC_CFG_BYTE_ORDER` bit in the `CRC_CFG` register. This affects the endianness of bytes added via the add 16, 24 and 32 bits registers.

19.3.4 CRC Registers

19.3.4.1 Arm Cortex-M3 Processor CRC Registers

Register Name	Register Description	Address
<code>CRC_CFG</code>	CRC Generator Control Register	P:0x1180 0x40000600
<code>CRC_VALUE</code>	CRC Generator Current Value Register	P:0x1181 0x40000604
<code>CRC_ADD_1</code>	CRC Generator - Add 1 Bit	P:0x1182 0x40000608
<code>CRC_ADD_8</code>	CRC Generator - Add 1 Byte	P:0x1183 0x4000060C
<code>CRC_ADD_16</code>	CRC Generator - Add 1 Half-word	P:0x1184 0x40000610
<code>CRC_ADD_24</code>	CRC Generator - Add 3 Bytes	P:0x1185 0x40000614
<code>CRC_ADD_32</code>	CRC Generator - Add 1 Word	P:0x1186 0x40000618
<code>CRC_FINAL</code>	CRC Generator Final Value	P:0x1187 0x4000061C

Ezairo 8300 Hardware Reference

19.3.4.1.1 CRC_CFG

Bit Field	Read/Write	Field Name	Description
4	RW	FINAL_CRC_XOR	Selects the final CRC XOR mode
3	RW	FINAL_CRC_REVERSE	Selects the final CRC reversal mode
2	RW	BIT_ORDER	Selects the bit order for bytes added to the CRC
1	RW	CRC_TYPE	Selects the CRC type
0	RW	BYTE_ORDER	Selects the endianness for bytes added to the CRC

Field Name	Value Symbol	Value Description	Hex Value
FINAL_CRC_XOR	CRC_FINAL_XOR_STANDARD	Final CRC XOR is done according to the standard (CRC-CCITT: no XOR; CRC-32: XOR with 0xFFFFFFFF)	0x0*
	CRC_FINAL_XOR_NON_STANDARD	Final CRC XOR is done in opposite of the standard	0x1
FINAL_CRC_REVERSE	CRC_FINAL_REVERSE_STANDARD	Final CRC reversal is done according to the standard (CRC-CCITT: normal; CRC-32 reversed)	0x0*
	CRC_FINAL_REVERSE_NON_STANDARD	Final CRC reversal is done in opposite of the standard	0x1
BIT_ORDER	CRC_BIT_ORDER_STANDARD	Bit order is as defined by the standard (CRC-CCITT: normal; CRC-32 reversed)	0x0*
	CRC_BIT_ORDER_NON_STANDARD	Bit order is opposite of the standard	0x1
CRC_TYPE	CRC_CCITT	CRC-CCITT algorithm selected	0x0*
	CRC_32	CRC-32 (IEEE 802.3) algorithm selected	0x1
BYTE_ORDER	CRC_BIG_ENDIAN	Bytes are added to the CRC in big-endian order	0x0*
	CRC_LITTLE_ENDIAN	Bytes are added to the CRC in little-endian order	0x1

19.3.4.1.2 CRC_VALUE

Bit Field	Read/Write	Field Name	Description
31:0	RW	CURRENT_CRC	CRC generator value: Write 0xFFFFFFFF (32) or 0xFFFF (CCITT) to initialize the CRC, read provides the current CRC value.

onsemi Confidential
Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
CURRENT_CRC	CRC_CCITT_INIT_VALUE	Initial value for the CRC CCITT calculation	0xFFFF*
	CRC_32_INIT_VALUE	Initial value for the CRC 32 calculation	0xFFFFFFFF

19.3.4.1.3 CRC_ADD_1

Bit Field	Read/Write	Field Name	Description
0	W	CRC_ADD_1	Add 1 bit to the CRC calculation

19.3.4.1.4 CRC_ADD_8

Bit Field	Read/Write	Field Name	Description
7:0	W	CRC_ADD_8	Add 1 byte (8 bits) to the CRC calculation

19.3.4.1.5 CRC_ADD_16

Bit Field	Read/Write	Field Name	Description
15:0	W	CRC_ADD_16	Add 1 half-word (16 bits) to the CRC calculation

19.3.4.1.6 CRC_ADD_24

Bit Field	Read/Write	Field Name	Description
23:0	W	CRC_ADD_24	Add 3 bytes (24 bits) to the CRC calculation

19.3.4.1.7 CRC_ADD_32

Bit Field	Read/Write	Field Name	Description
31:0	W	CRC_ADD_32	Add 1 word (32 bits) to the CRC calculation

19.3.4.1.8 CRC_FINAL

Bit Field	Read/Write	Field Name	Description
31:0	R	FINAL_CRC	CRC generator final value: After XOR for CCITT or byte reversal for CRC-32

19.3.4.2 CFX CRC Registers

Register Name	Register Description	Address
D_CRC_VALUE	CRC value (CRC16-CCITT)	P:0x0822 0x40002088
D_CRC_ADD_1	CRC 1-bit data input	P:0x0823 0x4000208C
D_CRC_ADD_8	CRC 8-bit data input	P:0x0824 0x40002090
D_CRC_ADD_16	CRC 16-bit data input	P:0x0825 0x40002094
D_CRC_ADD_24	CRC 24-bit data input	P:0x0826 0x40002098
D_CRC_ADD_32	CRC 32-bit data input	P:0x0827 0x4000209C

19.3.4.2.1 D_CRC_VALUE

Bit Field	Read/Write	Field Name	Description
15:0	RW	CRC_VALUE	Value of CRC-CCITT for supplied data

19.3.4.2.2 D_CRC_ADD_1

Bit Field	Read/Write	Field Name	Description
0	W	CRC_ADD_1	Add 1-bit data to the CRC

19.3.4.2.3 D_CRC_ADD_8

Bit Field	Read/Write	Field Name	Description
7:0	W	CRC_ADD_8	Add 8-bit data to the CRC

19.3.4.2.4 D_CRC_ADD_16

Bit Field	Read/Write	Field Name	Description
15:0	W	CRC_ADD_16	Add 16-bit data to the CRC

19.3.4.2.5 D_CRC_ADD_24

Bit Field	Read/Write	Field Name	Description
23:0	W	CRC_ADD_24	Add 24-bit data to the CRC

19.3.4.2.6 D_CRC_ADD_32

Bit Field	Read/Write	Field Name	Description
31:0	W	CRC_ADD_32	Add 32-bit data to the CRC

19.4 DIRECT MEMORY ACCESS (DMA) CONTROLLER**19.4.1 Block Overview**

The direct memory access controller (DMA) module allows background transfers between components on the peripheral bus (referred to in this section as peripherals) and memories without any processor intervention. This allows the processors to be used for other computational needs while enabling high speed sustained transfers to and from the peripherals/memories. The DMA has 8 independent configurable channels. Each channel can be configured for one of four modes:

- Data transferred from peripheral-to-memory (PM)
- Data transferred from memory-to-peripheral (MP)
- Data transferred between peripherals (PP)
- Data transferred between memory locations (MM)

The "DMA Overview" figure (Figure 42) gives an overview of the DMA function.

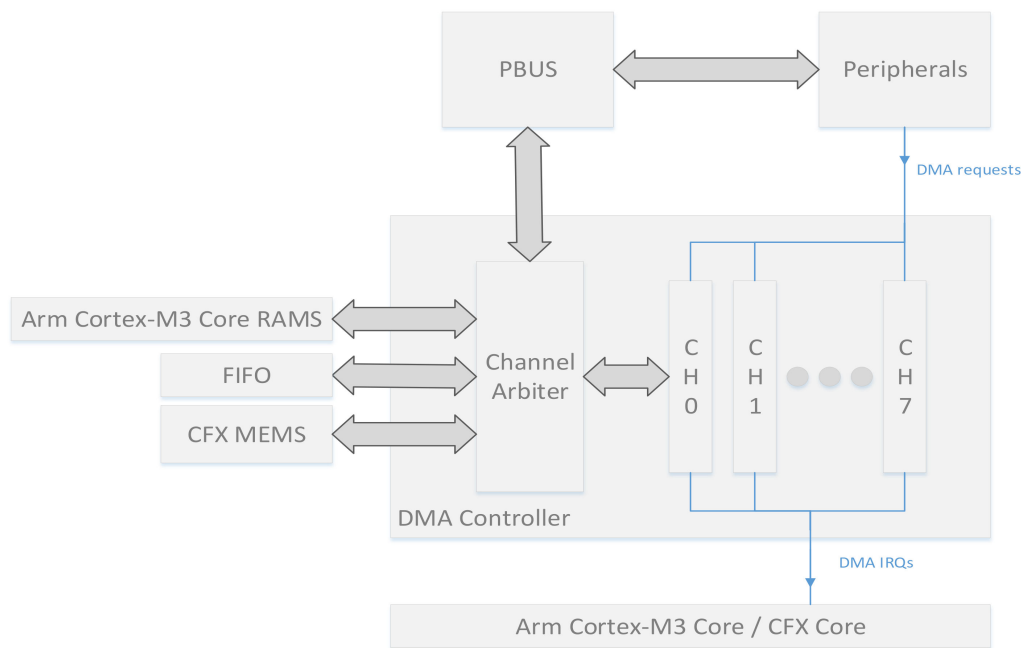


Figure 42. DMA Overview

When packing data, the DMA channel reads and packs input data into an internal buffer that is accessible through the `DMA_SRC_BUFFER` registers. When the DMA channel is unpacking data, it uses the same buffer as the source of packed data.

There is a separate interrupt line available for each of the CFX, Arm Cortex-M3, and LPDSP32 cores for each DMA channel. Each DMA channel can be configured to assert its interrupt line for several independent conditions, such as when a transfer, or a specified portion of it, is complete.

19.4.2 Functional Description

19.4.2.1 DMA to Peripherals Interface

The DMA supports several types of data transfers between the memories of the Arm Cortex-M3 processor and the CFX core, and the interfaces and peripherals connected to the peripheral bus, including:

- Memory-to-Memory transfers
- Memory-to-Peripheral transfers
- Peripheral-to-Memory transfers
- Peripheral-to-Peripheral transfers

Ezairo 8300 Hardware Reference

Each peripheral that supports DMA transfers is connected to the DMA controller through one or more DMA request signals. In most cases, a peripheral must be configured explicitly to operate in DMA mode to enable requests as required. The DMA supports 16 request lines from various peripherals included in Ezairo 8300, and connected independently to each channel.

The interfaces and peripherals mapped onto the peripheral bus that are valid data sources and destinations for a DMA data transfers, and their associated request lines, are listed in the ["DMA Request Lines and Their Associated Peripherals" table \(Table 69\)](#). For sources or destinations that do not need to wait for data (typically memories), the transfer uses an always on (DMA_SRC_ALWAYS_ON, DMA_DEST_ALWAYS_ON) configuration. To select the desired source and destination, set the DMA_CFG0_SRC_SELECT and DMA_CFG0_DEST_SELECT bit-fields from DMA_CFG0.

Table 69. DMA Request Lines and Their Associated Peripherals

DMA Request Line	Peripheral	Source	Destination
0	Always_on	-	-
1	SPI0	SPI0 RX	SPI0 TX
2	SPI1	SPI1 RX	SPI1 TX
3	PCM0	PCM0 RX	PCM0 TX
4	PCM1	PCM1 RX	PCM1 TX
5	UART	UART RX	UART TX
6	ASRC	ASRC OUT	ASRC IN
7	I2C0	I2C0 RX	I2C0 TX
8	I2C1	I2C1 RX	I2C1 TX
9	I3C0	I3C0 RX	I3C0 TX
10	I3C1	I3C1 RX	I3C1 TX
11	ACB	G722 ENC IN	G722 ENC OUT
12	ACB	G722 DEC IN	G722 DEC OUT
13	ECB	RS_ENC_IN	RS ENC OUT
14	ECB	RS DEC IN	RS DEC OUT
15	D_I2C	D_I2C RX	D_I2C TX

Each source interface or peripheral, when configured for DMA operation, asserts its DMA request signal when data can be read from the source. This signal is cleared automatically when a data value is read from the source using the peripheral bus.

Each destination interface or peripheral, when configured for DMA operation, asserts its DMA request signal when data can be written to the interface or peripheral. This signal is cleared automatically when a data value is written to that destination using the peripheral bus.

Ezairo 8300 Hardware Reference

CAUTION: When the DMA interface is used to control transfers that use an interface or peripheral, any access to that interface or peripheral's data registers using the peripheral bus clears the DMA request signals. If a DMA request signal is cleared due to a CFX DSP or an Arm Cortex-M3 processor access, the underlying DMA transfer becomes corrupted.

NOTE: To ease understanding, throughout the remainder of this section, all interfaces and peripherals used in DMA transfers are called *peripherals* due to their memory-mapping onto the peripheral bus.

19.4.2.2 Configuration and Status

The "DMA Module Specification" table (Table 70) describes DMA supported configurations and specifications:

Table 70. DMA Module Specification

Item	Specification
Number of DMA Channels	8
Priority Levels	4
Maximum Transfer Length	65536
Counter Interrupt Trigger Range	1 – 65535
Configurable Modes	Increment or Circular
Types of Transfers	Memory to Peripheral (MP) Peripheral to Memory (PM) Memory to Memory (MM) Peripheral to Peripheral (PP)
Interrupt Lines	8 (one per channel)
Interrupt Source per Channel	3: Transfer length counter, Complete, Word counter
Word Sizes	4-bit, 8-bit, 16-bit, 24-bit, and 32-bit

A DMA channel is configured independently through its associated memory mapped registers. Each DMA channel has one bit field: the `DMA*_CTRL_MODE` field of the `DMA_CTRL` register, which is used to enable one of the seven different modes in which the channel can be active (see the "DMA Channel Modes of Operation" table (Table 71), below). The DMA channel can be deactivated at any time by setting the `DMA_CTRL_MODE_ENABLE` bit to `DMA_DISABLE`. If the DMA channel is in the midst of an operation when it is disabled, the current read or write transaction is completed. All pending bus requests are subsequently aborted when the channel is disabled.

Ezairo 8300 Hardware Reference

Table 71. DMA Channel Modes of Operation

MODE_ENABLE Bit Value	Constant	Remarks
0	DMA_DISABLE	Disable the DMA channel (channel waits on current READ or WRITE access to finish before the active bit is cleared)
1	DMA_ENABLE	Enable the DMA channel, and when the transfer is completed, disable the DMA channel
2	DMA_ENABLE_WRAP	Enable the DMA channel, and when the transfer is completed, wrap addresses and disable the DMA channel
3	DMA_ENABLE_WRAP_RESTART	Enable the DMA channel, and when the transfer is completed, wrap addresses and restart the transfer
4	DMA_TRIGGER	When the source trigger occurs, enable the DMA channel; when the transfer is completed, disable the DMA channel
5	DMA_TRIGGER_WRAP	When the source trigger occurs, enable the DMA channel; when the transfer is completed, wrap addresses and disable the DMA channel
6	DMA_TRIGGER_WRAP_RESTART	When the source trigger occurs, enable the DMA channel; when the transfer is completed, wrap addresses and restart the DMA channel
7	DMA_TRIGGER_WRAP_TRIGGER_RESTART	When the source trigger occurs, enable the DMA channel; when the transfer is completed, wrap addresses, and upon next trigger event, restart the DMA channel

Other DMA channel configurations include:

Source and Destination Address Configuration

The DMA_SRC_ADDR registers define the base address of the source data for a DMA channel. The DMA_DEST_ADDR registers define the base address of the destination data for a DMA channel. The DMA supports byte addressing for source and destination addresses. Smaller word sizes are supported indirectly by selecting the source and destination word sizes, and through packing and unpacking. The next source address and next destination address are calculated by the DMA by using the current counter value.

These addresses need to be word aligned, a requirement for the 32-bit accesses this block performs.

Each DMA channel can be configured for two address modes: linear and circular. These address modes are selected while defining the mode of operation in the DMA control registers.

- In linear address mode, the DMA operation occurs and then the DMA channel is automatically disabled once the transfer is complete. The source and destination addresses are not reset to predefined addresses.
- In wrap restart mode (circular address mode), the DMA operation occurs and then the DMA channel configuration is reset to the initial configuration once the transfer is completed. This includes: current source address, current destination address, and length of transfer. The DMA channel remains enabled, continuing to run until explicitly stopped by the firmware when it sets the appropriate enabling bit to a disabled status.

Ezairo 8300 Hardware Reference

- In wrap linear mode, the addresses are wrapped at the end before the DMA channel is disabled.
- Each DMA channel has independent increment and decrement controls: between -8 to +7, with a step size of 1, for both source and destination addresses. The required step size can be configured using the `DMA_CFG0_SRC_ADDR_STEP` and `DMA_CFG0_DEST_ADDR_STEP` bit fields in the `DMA_CFG0` registers, for the source and destination addresses respectively. When the addressing step size is set to `DMA_DEST_ADDR_STATIC`, the address is neither incremented nor decremented after each word is transferred, i.e., it remains static. The address step size is applied to the address after each word is transferred. For example, in Memory-to-Peripheral operations, typically the source has the increment enabled, while the destination has the increment disabled.

NOTE: The source and destination address registers are not modified during a transfer; the increment setting only affects the calculation of the next address. When disabled, the next address is always equal to the base address.

Source and Destination Data

The source and destination data can be configured to be interpreted as packed into a variety of different word sizes and endian ordering. For more information, see [Section 19.4.2.6 “Packing and Unpacking” on page 700](#).

Transfer Length and Counter Interrupt Configuration

The DMA channels support variable transfer lengths up to 2^{16} packed words (32 bits per word). The length of a transfer can be set using the `DMA_CFG1_TRANSFER_LENGTH` bit field in the `DMA_CFG1` registers. When the DMA channel completes transferring this length of data to the destination, the `DMA_STATUS_COMPLETE_INT` flag is set, and an interrupt is generated if the `DMA_CFG0_COMPLETE_INT_ENABLE` bit is set in the `DMA_CFG0` registers.

The application can set a re-configurable trigger level for 1 to 2^{16} words in the `DMA_CFG1_INT_TRANSFER_LENGTH` bit field of the `DMA_CFG1` register for each DMA channel; set to 0 to disable. When the current transfer counter for a channel reaches the trigger level, an interrupt is generated if the counter interrupt is enabled, and all the data inside the source buffer register `DMA_SRC_BUFFER` is written to the destination.

Interrupt Configuration.

The interrupts used to coordinate with, and control, a DMA transfer can be defined using the `DMA_CFG0_*_INT_ENABLE` bits from the DMA channel's `DMA_CFG0` registers. For more information, see [Section 19.4.2.3 “DMA Interrupt Configuration” on page 699](#).

Transfer Priority.

The relative priority of this DMA channel's transfer. For more information, see [Section 19.4.2.9 “DMA Channel Arbiter” on page 707](#).

IMPORTANT: The DMA channel uses counters to calculate the next source address and next destination address. The base source address, base destination address, and starting length remain unchanged. Only the counter value is modified during a transfer. This allows the DMA channel configuration to be reused (either explicitly through firmware or in circular mode) without rewriting the configuration register for multiple transfers.

Ezairo 8300 Hardware Reference

The DMA channels each contain a status register (DMA_STATUS), indicating the completion status of the transfer (active, idle or complete), and the channel's interrupt status. The firmware can use this to quickly assess the status of a DMA channel.

IMPORTANT: When a DMA channel is enabled, the DMA_STATUS_ACTIVE bit stays low for one cycle. This can cause a problem when polling the DMA_STATUS_ACTIVE bit directly after enabling the channel, as the DMA status is incorrect for the first cycle (this only affects optimized C or assembly code executing on the Arm Cortex-M3 processor).

The DMA channel can be paused by disabling it. If the current transaction is a read, the interface fills up the source buffer and is ready for a write transfer. The channel configuration is not cleared unless DMA_CTRL_BUFFER_CLEAR and DMA_CTRL_CNTS_CLEAR are set.

19.4.2.3 DMA Interrupt Configuration

The DMA provides separate interrupts to the core for each channel independently. Each DMA channel can be configured using its DMA_CFG0 register to assert an interrupt for several independent conditions:

A transfer has reached the counter interrupt value

Set the DMA_CFG1_INT_TRANSFER_LENGTH bit field in the channel's DMA_CFG1 register, as defined by the DMA_CFG1_TRANSFER_LENGTH bit field in the channel's DMA_CFG1 register, to a non zero value to configure a trigger level, and set the DMA_CFG0_COUNTER_INT_ENABLE bit to enable it. If the counter is set to *n*, the transfer reaches the counter interrupt value when it writes the *n*th memory word to the DMA channel's destination. This counter can be cleared using the DMA_CTRL_BUFFER_CLEAR and DMA_CTRL_CNTS_CLEAR bits in the DMA_CTRL register. This interrupt can be used to allow the firmware to operate on partial blocks of data as they are available.

A transfer is complete

Set the DMA_CFG0_COMPLETE_INT_ENABLE bit to enable this interrupt. A transfer is considered complete when the last word is written to the DMA channel's destination. For wrap restart transfers (circular transfers), a complete interrupt is generated each time the circular transfer is completed.

The interrupts for DMA are configurable on a per-channel basis using the DMA_CFG0 registers. A user application can read sticky bits DMA_STATUS_CNT_INT and DMA_STATUS_COMPLETE_INT in the channel status registers (DMA_STATUS) to know which interrupts have been triggered. Consequently, the user application can clear sticky status bits of the interrupts by writing to the DMA_STATUS_COMPLETE_INT_CLEAR or DMA_STATUS_COMPLETE_INT_CLEAR bit fields.

NOTE: If a status bit for a particular event is not cleared, all subsequent interrupts for that same event are masked.

19.4.2.4 Circular Mode

By enabling DMA_CFG*_SRC_ADDR_LSB_TOGGLE, a DMA transfer can have its source address effective LSB toggled after each read, resulting in a circular buffer of 2 words. DMA_CFG0*_SRC_ADDR_STEP_SIZE must be set to zero in this case. For a source word size of 4 or 8 bits, address bit 0 is toggled; for a source word size of 16 bits, address bit 1 is toggled; and for a source word size of 32 bits, address bit 2 is toggled.

Ezairo 8300 Hardware Reference

By enabling `DMA_CFG*_DEST_ADDR_LSB_TOGGLE`, a DMA transfer can have its destination address's effective LSB toggled after each write, resulting in a circular buffer of 2 words. `DMA_CFG0*_DEST_ADDR_STEP_SIZE` must be set to zero in this case. For a destination word size of 4 or 8 bits, address bit 0 is toggled; for a destination word size of 16 bits, address bit 1 is toggled; and for a destination word size of 32 bits, address 2 is toggled.

19.4.2.5 Word Size

Each DMA channel can have different word sizes set for both the source and the destination. The word sizes for the source and destination data can be configured as 4-bit, 8-bit, 16-bit, 24-bit, or 32-bit, using the `DMA_CFG0*_SRC_DEST_WORD_SIZE` bit field from the `DMA_CFG0*` register. Smaller word sizes can be used to optimize memory utilization when retrieving or storing data. Both 4-bit and 8-bit words are byte addressable, 16-bit words are half word addressable, and 24-bit and 32-bit words are word addressable in memory.

The "Organization of a 32-Bit memory instance" figure (Figure 43) shows the organization of a 32-bit memory instance. From the DMA perspective, it can be comprised of the following:

- One 32-bit word (word 0), or
- One 24-bit word (word 0) (8 MSBs ignored when reading and set to 0 when writing), or
- Two 16-bit words (word 0, word 1), or
- Four 8-bit words (word 0, word 1, word 2, word 3), or
- Eight 4-bit words (word 0, word 1, word 2, word 3, word 4, word 5, word 6, word 7)

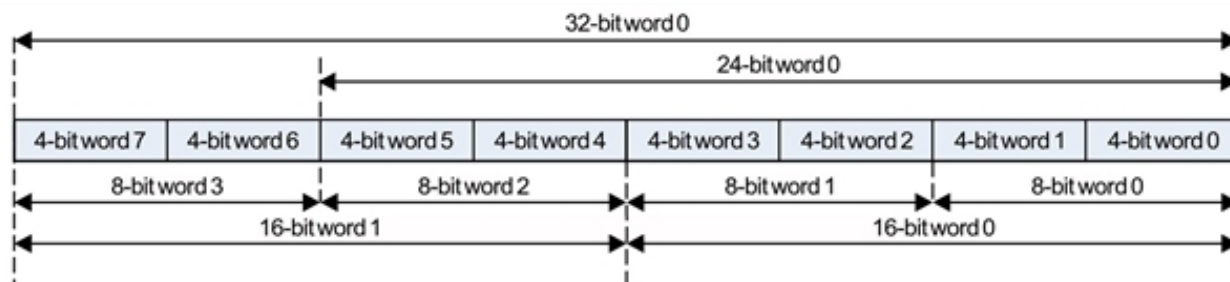


Figure 43. Organization of a 32-Bit memory instance

Read bus operations are always based on 32-bit words. However, the DMA can utilize only a portion of the data if the selected word sizes are smaller. Write bus operations can be 8-bit, 16-bit, or 32-bit words, as long as the memory system that is addressed supports this. The counter in the DMA channel always operates on the configured source word size. If the transfer type is memory-peripheral, the length sets the number of destination words transferred; otherwise it sets the number of source words transferred. The next address for reading or writing can be determined from the configured word size, the current counter, and the base address. This might result in an actual read or write, or use of the temporary register with the DMA.

NOTE: Source data using 24-bit words are only compatible with destination data comprising of 4-bit, 8-bit, or 24-bit words. Similarly, destination data using 24-bit words are only compatible with source data comprising of 4-bit, 8-bit, or 24-bit words.

19.4.2.6 Packing and Unpacking

The DMA performs packing and unpacking (shown in the "DMA packing and unpacking, Part 1" figure (Figure 44) and the "DMA packing and unpacking, Part 2" figure (Figure 45)) as required. Packing and unpacking are performed automatically, depending on the word size configuration, the byte address access, and the DMA operation

Ezairo 8300 Hardware Reference

mode. Packing and unpacking can be used for all types of DMA transfers, to minimize bus utilization and maximize memory use efficiency. For example, when writing four bytes to an 8-bit destination register, only one 32-bit read is required from the source memory instance. In this case, the memory instance word size is configured as 32-bit, and the peripheral word size is configured as 8-bit. When both source and destination are configured for the native word size (32-bit), no special processing is necessary (direct mode).

When either the source or destination word size is different from the native word size, the DMA performs packing and/or unpacking. The data is always right aligned and can be byte addressable. When the destination word size is less than 32 bits and the destination address is not byte addressed, the data is stored in the lower portion of the 32-bit word. The upper bits are always written as zero (padded mode). When the source word size is less than 32 bits, the data is read from the lower portion of the 32-bit word. The upper bits are ignored. In addition to packing/unpacking, the DMA supports both big and little endian transfers.

NOTE: Packing/unpacking with word sizes 24 to 16 bits, 16 to 24 bits, 32 to 24 bits, and 24 to 32 bits is not supported.

NOTE: Transfers that cannot be evenly packed into the destination word size (i.e., transfers that include an incomplete final word) are only supported for little endian transfers.

If byte ordering is enabled, the DMA can also order the bytes when the source word size and destination word size are the same. In the case of a word size of 8 bits for both source and destination, the nibble is ordered instead. No ordering occurs for word sizes of 4 bits. This is shown in the two examples that follow.

- Example 1 (see also the "DMA Transfer with Src Word Size < Dest Word Size (Example 1)" table (Table 4), below):
 - Transfer Type: Peripheral to Memory
 - Source Word Size: 8 bits (Peripheral, for instance SPI output)
 - Destination Word Size: 32 bits (Data Memory)
 - Transfer Length: 5 (11 symbols)
 - Byte Ordering: Little Endian

Table 72. DMA Transfer with Src Word Size < Dest Word Size (Example 1)

SOURCE	DESTINATION (Data Memory)
WordInput_8_0	
WordInput_8_1	
WordInput_8_2	
WordInput_8_3	WordOutput_32_0 = {W18_3, W18_2, W18_1, W18_0}
WordInput_8_4	WordOutput_32_1 = {0x00, 0x00, 0x00, W18_4}

- Example 2 (see also the "DMA Transfer with Src Word Size < Dest Word Size (Example 2)" table (Table 5)):
 - Transfer Type: Peripheral to Memory
 - Source Word Size: 8 bits (Peripheral, for instance SPI output)
 - Destination Word Size: 16 bits (Data Memory)
 - Transfer Length: 5 (11 symbols)
 - Byte Ordering: Big Endian

Table 73. DMA Transfer with Src Word Size < Dest Word Size (Example 2)

SOURCE	DESTINATION (Data Memory)
WordInput_8_0	
WordInput_8_1	WordOutput_32_0 = {0x00, 0x00, WI8_0, WI8_1}
WordInput_8_2	
WordInput_8_3	WordOutput_32_1 = {0x00, 0x00, WI8_2, WI8_3}
WordInput_8_4	WordOutput_32_2 = {0x00, 0x00, WI8_4, 0x00}

Ezairo 8300 Hardware Reference

Word size		Source Data	Destination Data	
SRC	DEST		Big Endian	Little Endian
32	32	H G F E D C B A _s	H G F E D C B A _d	H G F E D C B A _d
32	16	H G F E D C B A _s	X X X X H G F E _d X X X X D C B A _{d+p}	X X X X D C B A _d X X X X H G F E _{d+p}
32	8	H G F E D C B A _s	X X X X X X H G _d X X X X X X F E _{d+p} X X X X X X D C _{d+2p} X X X X X X B A _{d+3p}	X X X X X X B A _d X X X X X X D C _{d+p} X X X X X X F E _{d+2p} X X X X X X H G _{d+3p}
32	4	H G F E D C B A _s	X X X X X X X H _d X X X X X X X G _{d+p} X X X X X X X F _{d+2p} X X X X X X X E _{d+3p} X X X X X X X D _{d+4p} X X X X X X X C _{d+5p} X X X X X X X B _{d+6p} X X X X X X X A _{d+7p}	X X X X X X X A _d X X X X X X X B _{d+p} X X X X X X X C _{d+2p} X X X X X X X D _{d+3p} X X X X X X X E _{d+4p} X X X X X X X F _{d+5p} X X X X X X X G _{d+6p} X X X X X X X H _{d+7p}
24	24	X X F E D C B A _s	X X F E D C B A _d	X X F E D C B A _d
24	8	X X F E D C B A _s	X X X X X X F E _d X X X X X X D C _{d+p} X X X X X X B A _{d+2p}	X X X X X X B A _d X X X X X X D C _{d+p} X X X X X X F E _{d+2p}
24	4	X X F E D C B A _s	X X X X X X X F _d X X X X X X X E _{d+p} X X X X X X X D _{d+2p} X X X X X X X C _{d+3p} X X X X X X X B _{d+4p} X X X X X X X A _{d+5p}	X X X X X X X A _d X X X X X X X B _{d+p} X X X X X X X C _{d+2p} X X X X X X X D _{d+3p} X X X X X X X E _{d+4p} X X X X X X X F _{d+5p}
16	16	X X X X D C B A _s	X X X X D C B A _d	X X X X D C B A _d
16	8	X X X X D C B A _s	X X X X X X D C _d X X X X X X B A _{d+p}	X X X X X X B A _d X X X X X X D C _{d+p}
16	4	X X X X D C B A _s	X X X X X X X D _d X X X X X X X C _{d+p} X X X X X X X B _{d+2p} X X X X X X X A _{d+3p}	X X X X X X X A _d X X X X X X X B _{d+p} X X X X X X X C _{d+2p} X X X X X X X D _{d+3p}
8	32	X X X X X X X B A _s X X X X X X X D C _{s+p} X X X X X X X F E _{s+2p}		

Figure 44. DMA packing and unpacking, Part 1

Ezairo 8300 Hardware Reference

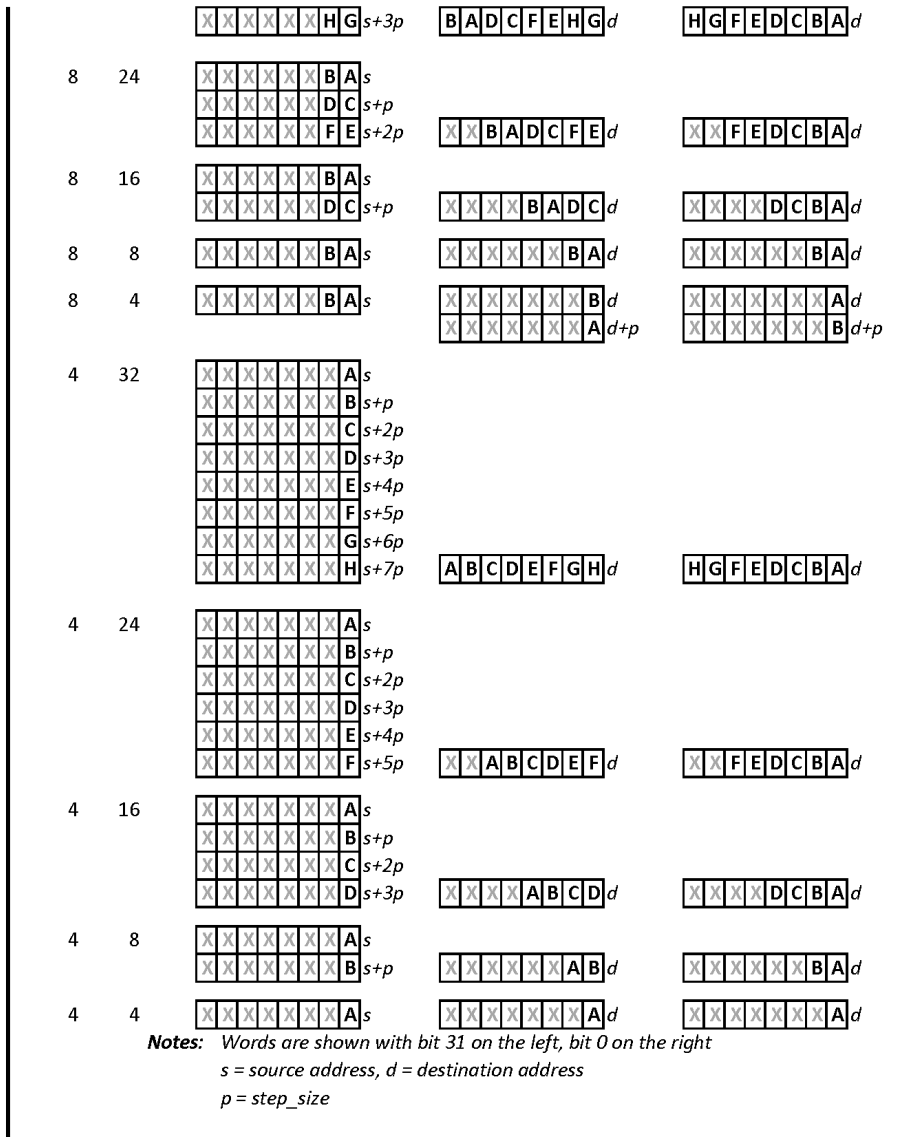


Figure 45. DMA packing and unpacking, Part 2

Ezairo 8300 Hardware Reference

19.4.2.7 Triggering DMA Channels

DMA channels can be triggered to start without firmware intervention, based on another DMA channel completing or a specified system interrupt occurring. This can be used to off-load interrupts and data transfer tasks from the Arm Cortex-M3 processor or the CFX core.

The start of the DMA transfer is defined by the triggering enable modes. For example, one DMA channel can send data to the CRC module, while a second DMA channel can pull data from the CRC module without the Arm Cortex-M3 core's intervention. In any triggering mode, the DMA channel waits until a configured triggering event occurs. This triggering event is defined by the `DMA_CTRL_TRIGGER_SOURCE` field in the `DMA_CTRL` registers.

Each DMA channel needs to be configured independently (i.e., different transfer length, word sizes, increment steps, source, destination, etc.).

IMPORTANT: A DMA channel must not be configured to be triggered by itself, or else the channel remains in pending state (`DMA_STATUS_DMA_STATUS_ACTIVE`).

NOTE: If two or more DMA channels are linked to the same trigger source, they run in parallel and start new DMA threads. Any conflict associated with cross-selecting DMA channels needs to be controlled by the firmware.

The destination counter value indicates the number of transfers sent to the destination address during each DMA transfer. Upon completion, the complete interrupt of the corresponding DMA channel is triggered if enabled.

At the end of the DMA transfer, the selected triggering mode can define how the channel behaves when the transfer length is reached:

- If `DMA_TRIGGER` has been selected, the channel is simply disabled automatically.
- If `DMA_TRIGGER_WRAP` has been selected, the addresses are wrapped to the configured base addresses and the channel is disabled.
- If `DMA_TRIGGER_WRAP_RESTART` has been selected, the addresses are wrapped to the configured base addresses, and restarts with another DMA transfer in a circular addressing mode with the same channel configuration. In this mode, the loop is broken when the DMA channel is disabled with the firmware's intervention.
- If `DMA_TRIGGER_WRAP_TRIGGER_RESTART` has been selected, the addresses are wrapped to the configured base addresses, the transfer counter is reset, the DMA channel becomes pending, waits to be triggered again, and restarts with another DMA transfer.

NOTE: Source and destination addresses can also be wrapped on every interrupt transfer length counter, as configured using the `DMA_CTRL_WRAP_DEST_ADDR_ON_INT_CNT` and `DMA_CTRL_WRAP_SOURCE_ADDR_ON_INT_CNT` bits from the `DMA_CTRL` registers.

19.4.2.8 DMA Operation

The DMA controller works as follows:

1. The DMA channel is configured and enabled by the firmware.
 - a. The operation sequence varies slightly if the transfer length depends on the destination or source word count. This can be selected using the bit location `DMA_CFG0_SRC_DEST_TRANS_LENGTH_SEL` in the `DMA_CFG0` register.

Ezairo 8300 Hardware Reference

- b. If bit `DMA_CFG0_SRC_DEST_TRANS_LENGTH_SEL` is set to `DEST_TRANS_LENGTH_SEL`, the transfer length depends on the destination transfer count. When `SRC_TRANS_LENGTH_SEL` is set, the transfer length depends on the source word count.
 2. The operation sequence is as follows when the transfer length depends on the destination word count:
 - a. The DMA channel reads from the source until its source buffer is filled with enough bits to write to its destination, in accordance with its word size.
 - i. If the DMA source request line is configured to be connected to a peripheral with DMA access, the channel reads from the source address when a request is issued by the peripheral.
 - ii. If DMA access is not available, or if the source is a memory location, then `DMA_SRC_ALWAYS_ON` can be selected, and the DMA channel reads immediately.
 - iii. The DMA channel reads from the source until its source buffer is filled with enough bits to write to its destination, in accordance with its word size.
 - b. As the source buffer is read by DMA, the `DMA_STATUS_SRC_BUFFER_FILL_LVL` in the `DMA_STATUS` register is updated to reflect the new source buffer status. The DMA channel writes to its destination, as long as the source buffer contains enough bits for the writing task, and the destination word counter is less than the configured transfer length.
 - i. If the DMA destination request line is configured to be connected to a peripheral with DMA access, the channel writes to the destination when this request is made by the peripheral.
 - ii. If DMA access is not available, or if the source is a memory location, then `DMA_DEST_ALWAYS_ON` is selected as the destination and the DMA channel writes immediately.
 - iii. When data is read by the DMA from the source or destination address, the data is copied into the `DMA_SRC_BUFFER` register. The data in this register is shifted to the right in accordance with the destination word size, and zeroes are shifted in. The `DMA_STATUS_SRC_BUFFER_FILL_LVL` field in `DMA0_STATUS` is updated to reflect the new source buffer status.
 - iv. The DMA continues to transfer data from source to destination, until the destination transfer length (configured through the `DMA_CFG1_TRANSFER_LENGTH` field in the `DMA_CFG1` registers) is reached. The DMA increments a destination word counter which can be read by the `DMA_CNTS_TRANSFER_WORD_CNT` field in the `DMA_CNTS` registers.
 - c. An interrupt can be generated for a transfer word count by writing to the `DMA_CFG1_INT_TRANSFER_LENGTH` field in the `DMA_CFG1` registers. This transfer word count is incremented with the destination word counter of the DMA. If a non zero value for the interrupt transfer word count is written in the register, and the interrupt transfer word count in the register value reaches the configured value, an interrupt triggers, the interrupt status bit is set, and the counter resets and begins to count again. The transfer word counter continues counting and generating interrupts in the same manner until the DMA operation is complete.
 - d. When the destination word count equals the transfer word count, the DMA channel triggers a completed interrupt (when it is enabled), its interrupt state bit is set, and the counter resets only in continuous modes.
 3. The operation is as follows when the transfer length depends on the source word count:
 - a. The source word counter is incremented until it reaches the transfer word count. The counter resets only in continuous modes.
 - b. When the destination word count equals the transfer word count, the DMA channel triggers a completed interrupt when it is enabled and its interrupt state bit is set. However, in this mode, if the DMA transfer is not completed, the channel restarts its operation.

Ezairo 8300 Hardware Reference

19.4.2.9 DMA Channel Arbiter

Only one transfer using the DMA channels can be actively serviced at a time. The DMA contains a channel arbiter, which is responsible for determining which DMA channel to activate. Each DMA channel is assigned a channel priority in the range of 0 - 3, using the `DMA_CFG0_CHANNEL_PRIORITY` bit-field from the `DMA_CFG0` registers. This channel priority is used by the arbiter to determine which DMA channel to service when multiple requests are pending.

Each DMA channel receives respective requests from peripherals connected to the DMA request lines. Once the DMA channel is ready to begin an operation, it indicates to the arbiter that it is ready by making a request. The DMA arbiter selects between multiple ready DMA channels based on configured channel priority.

The following situations describe scenarios when multiple requests might be pending:

- A DMA request is asserted by two or more peripherals (or a single peripheral serving two DMA channels) during the same clock cycle.
- One or more DMA requests come in during the processing of another DMA request.
- A DMA channel is enabled with multiple requests already pending.

When choosing which DMA channel to activate, the DMA arbiter applies the channel's priority settings as follows:

- When multiple channels are ready, the channel with the highest channel priority setting is activated.
- If more than one channel is ready to be activated and they have the same priority setting, the lowest numbered channel is activated.

When a single channel is ready, the arbiter grants access and the channel begins to service that request immediately.

IMPORTANT: A lower priority DMA channel might never be served if a higher priority DMA channel is generating requests too fast. This type of situation must be avoided by application design.

19.4.3 DMA Control and Configuration Registers

Register Name	Register Description	Address
DMA0_CFG0	DMA Channel Configuration Register	P:0x1080 0x40000200
DMA0_CFG1	DMA Channel Transfer Configuration Register	P:0x1081 0x40000204
DMA0_CTRL	DMA Channel Control	P:0x1082 0x40000208
DMA0_STATUS	DMA Channel Status	P:0x1083 0x4000020C
DMA0_SRC_ADDR	DMA Channel Source Address	P:0x1084

onsemi Confidential
Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
		0x40000210
DMA0_DEST_ADDR	DMA Channel Destination Address	P:0x1085 0x40000214
DMA0_CNTS	DMA Channel Counters	P:0x1086 0x40000218
DMA0_SRC_BUFFER	DMA Channel Source buffered data after packing	P:0x1087 0x4000021C
DMA1_CFG0	DMA Channel Configuration Register	P:0x1088 0x40000220
DMA1_CFG1	DMA Channel Transfer Configuration Register	P:0x1089 0x40000224
DMA1_CTRL	DMA Channel Control	P:0x108A 0x40000228
DMA1_STATUS	DMA Channel Status	P:0x108B 0x4000022C
DMA1_SRC_ADDR	DMA Channel Source Address	P:0x108C 0x40000230
DMA1_DEST_ADDR	DMA Channel Destination Address	P:0x108D 0x40000234
DMA1_CNTS	DMA Channel Counters	P:0x108E 0x40000238
DMA1_SRC_BUFFER	DMA Channel Source buffered data after packing	P:0x108F 0x4000023C
DMA2_CFG0	DMA Channel Configuration Register	P:0x1090 0x40000240
DMA2_CFG1	DMA Channel Transfer Configuration Register	P:0x1091 0x40000244
DMA2_CTRL	DMA Channel Control	P:0x1092

onsemi Confidential
Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
		0x40000248
DMA2_STATUS	DMA Channel Status	P:0x1093 0x4000024C
DMA2_SRC_ADDR	DMA Channel Source Address	P:0x1094 0x40000250
DMA2_DEST_ADDR	DMA Channel Destination Address	P:0x1095 0x40000254
DMA2_CNTS	DMA Channel Counters	P:0x1096 0x40000258
DMA2_SRC_BUFFER	DMA Channel Source buffered data after packing	P:0x1097 0x4000025C
DMA3_CFG0	DMA Channel Configuration Register	P:0x1098 0x40000260
DMA3_CFG1	DMA Channel Transfer Configuration Register	P:0x1099 0x40000264
DMA3_CTRL	DMA Channel Control	P:0x109A 0x40000268
DMA3_STATUS	DMA Channel Status	P:0x109B 0x4000026C
DMA3_SRC_ADDR	DMA Channel Source Address	P:0x109C 0x40000270
DMA3_DEST_ADDR	DMA Channel Destination Address	P:0x109D 0x40000274
DMA3_CNTS	DMA Channel Counters	P:0x109E 0x40000278
DMA3_SRC_BUFFER	DMA Channel Source buffered data after packing	P:0x109F 0x4000027C
DMA4_CFG0	DMA Channel Configuration Register	P:0x10A0

onsemi Confidential
Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
		0x40000280
DMA4_CFG1	DMA Channel Transfer Configuration Register	P:0x10A1 0x40000284
DMA4_CTRL	DMA Channel Control	P:0x10A2 0x40000288
DMA4_STATUS	DMA Channel Status	P:0x10A3 0x4000028C
DMA4_SRC_ADDR	DMA Channel Source Address	P:0x10A4 0x40000290
DMA4_DEST_ADDR	DMA Channel Destination Address	P:0x10A5 0x40000294
DMA4_CNTS	DMA Channel Counters	P:0x10A6 0x40000298
DMA4_SRC_BUFFER	DMA Channel Source buffered data after packing	P:0x10A7 0x4000029C
DMA5_CFG0	DMA Channel Configuration Register	P:0x10A8 0x400002A0
DMA5_CFG1	DMA Channel Transfer Configuration Register	P:0x10A9 0x400002A4
DMA5_CTRL	DMA Channel Control	P:0x10AA 0x400002A8
DMA5_STATUS	DMA Channel Status	P:0x10AB 0x400002AC
DMA5_SRC_ADDR	DMA Channel Source Address	P:0x10AC 0x400002B0
DMA5_DEST_ADDR	DMA Channel Destination Address	P:0x10AD 0x400002B4
DMA5_CNTS	DMA Channel Counters	P:0x10AE

onsemi Confidential
Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
		0x400002B8
DMA5_SRC_BUFFER	DMA Channel Source buffered data after packing	P:0x10AF 0x400002BC
DMA6_CFG0	DMA Channel Configuration Register	P:0x10B0 0x400002C0
DMA6_CFG1	DMA Channel Transfer Configuration Register	P:0x10B1 0x400002C4
DMA6_CTRL	DMA Channel Control	P:0x10B2 0x400002C8
DMA6_STATUS	DMA Channel Status	P:0x10B3 0x400002CC
DMA6_SRC_ADDR	DMA Channel Source Address	P:0x10B4 0x400002D0
DMA6_DEST_ADDR	DMA Channel Destination Address	P:0x10B5 0x400002D4
DMA6_CNTS	DMA Channel Counters	P:0x10B6 0x400002D8
DMA6_SRC_BUFFER	DMA Channel Source buffered data after packing	P:0x10B7 0x400002DC
DMA7_CFG0	DMA Channel Configuration Register	P:0x10B8 0x400002E0
DMA7_CFG1	DMA Channel Transfer Configuration Register	P:0x10B9 0x400002E4
DMA7_CTRL	DMA Channel Control	P:0x10BA 0x400002E8
DMA7_STATUS	DMA Channel Status	P:0x10BB 0x400002EC
DMA7_SRC_ADDR	DMA Channel Source Address	P:0x10BC

onsemi Confidential
Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
		0x400002F0
DMA7_DEST_ADDR	DMA Channel Destination Address	P:0x10BD 0x400002F4
DMA7_CNTS	DMA Channel Counters	P:0x10BE 0x400002F8
DMA7_SRC_BUFFER	DMA Channel Source buffered data after packing	P:0x10BF 0x400002FC

19.4.3.1 DMA*_CFG0

Bit Field	Read/Write	Field Name	Description
31	RW	COMPLETE_INT_ENABLE	Raise an interrupt when the DMA transfer completes
30	RW	CNT_INT_ENABLE	Raise an interrupt when the DMA transfer reaches the counter value
29	RW	DEST_ADDR_LSB_TOGGLE	Enable an address LSB toggling for the destination
28	RW	SRC_ADDR_LSB_TOGGLE	Enable an address LSB toggling for the source
27:24	RW	DEST_ADDR_STEP	Configure whether the destination address increments/decrements in terms of destination word size
23:20	RW	SRC_ADDR_STEP	Configure whether the source address increments/decrements in terms of source word size
19:14	RW	SRC_DEST_WORD_SIZE	Select the source and destination word sizes for the transfer
13:10	RW	DEST_SELECT	Select the request line for the destination
8:5	RW	SRC_SELECT	Select the request line for the source
3:2	RW	CHANNEL_PRIORITY	Select the priority level for this channel
1	RW	SRC_DEST_TRANS_LENGTH_SEL	Selects whether the transfer length counter depends on either the source word counts or the destination word count
0	RW	BYTE_ORDER	Select the byte ordering of the DMA channel

Field Name	Value Symbol	Value Description	Hex Value
COMPLETE_INT_ENABLE	DMA_COMPLETE_INT_DISABLE	Disable completion interrupts for DMA channel	0x0*
	DMA_COMPLETE_INT_ENABLE	Enable completion interrupts for DMA channel	0x1
CNT_INT_ENABLE	DMA_CNT_INT_DISABLE	Disable counter interrupts for DMA	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		channel	
	DMA_CNT_INT_ENABLE	Enable counter interrupts for DMA channel	0x1
DEST_ADDR_LSB_TOGGLE	DMA_DEST_ADDR_LSB_TOGGLE_DISABLE	No toggling on the destination address LSB	0x0*
	DMA_DEST_ADDR_LSB_TOGGLE_ENABLE	Toggle the destination address LSB	0x1
SRC_ADDR_LSB_TOGGLE	DMA_SRC_ADDR_LSB_TOGGLE_DISABLE	No toggling on the source address LSB	0x0*
	DMA_SRC_ADDR_LSB_TOGGLE_ENABLE	Toggle the source address LSB	0x1
DEST_ADDR_STEP	DMA_DEST_ADDR_STATIC	Do not increment the destination address used by DMA channel	0x0*
	DMA_DEST_ADDR_INCR_1	Set the step size of DMA channel destination address to 1	0x1
	DMA_DEST_ADDR_INCR_2	Set the step size of DMA channel destination address to 2	0x2
	DMA_DEST_ADDR_INCR_3	Set the step size of DMA channel destination address to 3	0x3
	DMA_DEST_ADDR_INCR_4	Set the step size of DMA channel destination address to 4	0x4
	DMA_DEST_ADDR_INCR_5	Set the step size of DMA channel destination address to 5	0x5
	DMA_DEST_ADDR_INCR_6	Set the step size of DMA channel destination address to 6	0x6
	DMA_DEST_ADDR_INCR_7	Set the step size of DMA channel destination address to 7	0x7
	DMA_DEST_ADDR_DECR_8	Set the step size of DMA channel destination address to minus 8	0x8
	DMA_DEST_ADDR_DECR_7	Set the step size of DMA channel destination address to minus 7	0x9
	DMA_DEST_ADDR_DECR_6	Set the step size of DMA channel destination address to minus 6	0xA
	DMA_DEST_ADDR_DECR_5	Set the step size of DMA channel destination address to minus 5	0xB
	DMA_DEST_ADDR_DECR_4	Set the step size of DMA channel destination address to minus 4	0xC

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DMA_DEST_ADDR_DECR_3	Set the step size of DMA channel destination address to minus 3	0xD
	DMA_DEST_ADDR_DECR_2	Set the step size of DMA channel destination address to minus 2	0xE
	DMA_DEST_ADDR_DECR_1	Set the step size of DMA channel destination address to minus 1	0xF
SRC_ADDR_STEP	DMA_SRC_ADDR_STATIC	Do not increment the source address used by DMA channel	0x0*
	DMA_SRC_ADDR_INCR_1	Set the step size of DMA channel source address to 1	0x1
	DMA_SRC_ADDR_INCR_2	Set the step size of DMA channel source address to 2	0x2
	DMA_SRC_ADDR_INCR_3	Set the step size of DMA channel source address to 3	0x3
	DMA_SRC_ADDR_INCR_4	Set the step size of DMA channel source address to 4	0x4
	DMA_SRC_ADDR_INCR_5	Set the step size of DMA channel source address to 5	0x5
	DMA_SRC_ADDR_INCR_6	Set the step size of DMA channel source address to 6	0x6
	DMA_SRC_ADDR_INCR_7	Set the step size of DMA channel source address to 7	0x7
	DMA_SRC_ADDR_DECR_8	Set the step size of DMA channel source address to minus 8	0x8
	DMA_SRC_ADDR_DECR_7	Set the step size of DMA channel source address to minus 7	0x9
	DMA_SRC_ADDR_DECR_6	Set the step size of DMA channel source address to minus 6	0xA
	DMA_SRC_ADDR_DECR_5	Set the step size of DMA channel source address to minus 5	0xB
	DMA_SRC_ADDR_DECR_4	Set the step size of DMA channel source address to minus 4	0xC
	DMA_SRC_ADDR_DECR_3	Set the step size of DMA channel source address to minus 3	0xD
	DMA_SRC_ADDR_DECR_2	Set the step size of DMA channel source address to minus 2	0xE
	DMA_SRC_ADDR_DECR_1	Set the step size of DMA channel source address to minus 1	0xF

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
SRC_DEST_WORD_SIZE	WORD_SIZE_32BITS_TO_32BITS	source data uses 32-bit word and destination data uses 32-bit word	0x0*
	WORD_SIZE_32BITS_TO_4BITS	source data uses 32-bit word and destination data uses 4-bit word	0x1
	WORD_SIZE_32BITS_TO_8BITS	source data uses 32-bit word and destination data uses 8-bit word	0x2
	WORD_SIZE_32BITS_TO_16BITS	source data uses 32-bit word and destination data uses 16-bit word	0x4
	WORD_SIZE_4BITS_TO_32BITS	source data uses 4-bit word and destination data uses 32-bit word	0x8
	WORD_SIZE_4BITS_TO_4BITS	source data uses 4-bit word and destination data uses 4-bit word	0x9
	WORD_SIZE_4BITS_TO_8BITS	source data uses 4-bit word and destination data uses 8-bit word	0xA
	WORD_SIZE_4BITS_TO_16BITS	source data uses 4-bit word and destination data uses 16-bit word	0xC
	WORD_SIZE_4BITS_TO_24BITS	source data uses 4-bit word and destination data uses 24-bit word	0xE
	WORD_SIZE_8BITS_TO_32BITS	source data uses 8-bit word and destination data uses 32-bit word	0x10
	WORD_SIZE_8BITS_TO_4BITS	source data uses 8-bit word and destination data uses 4-bit word	0x11
	WORD_SIZE_8BITS_TO_8BITS	source data uses 8-bit word and destination data uses 8-bit word	0x12
	WORD_SIZE_8BITS_TO_16BITS	source data uses 8-bit word and destination data uses 16-bit word	0x14
	WORD_SIZE_8BITS_TO_24BITS	source data uses 8-bit word and destination data uses 24-bit word	0x16
	WORD_SIZE_16BITS_TO_32BITS	source data uses 16-bit word and destination data uses 32-bit word	0x20
	WORD_SIZE_16BITS_TO_4BITS	source data uses 16-bit word and destination data uses 4-bit word	0x21
	WORD_SIZE_16BITS_TO_8BITS	source data uses 16-bit word and destination data uses 8-bit word	0x22
	WORD_SIZE_16BITS_TO_16BITS	source data uses 16-bit word and destination data uses 16-bit word	0x24
	WORD_SIZE_24BITS_TO_4BITS	source data uses 24-bit word and destination data uses 4-bit word	0x31

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	WORD_SIZE_24BITS_TO_8BITS	source data uses 24-bit word and destination data uses 8-bit word	0x32
	WORD_SIZE_24BITS_TO_24BITS	source data uses 24-bit word and destination data uses 24-bit word	0x36
DEST_SELECT	DMA_DEST_ALWAYS_ON	Data writes are always triggered	0x0*
	DMA_DEST_SPI0	Data writes are triggered by the SPI0 request line	0x1
	DMA_DEST_SPI1	Data writes are triggered by the SPI1 request line	0x2
	DMA_DEST_PCM0	Data writes are triggered by the PCM0 request line	0x3
	DMA_DEST_PCM1	Data writes are triggered by the PCM1 request line	0x4
	DMA_DEST_UART0	Data writes are triggered by the UART0 request line	0x5
	DMA_DEST_ASRC	Data writes are triggered by the ASRC request line	0x6
	DMA_DEST_I2C0	Data writes are triggered by the I2C0 request line	0x7
	DMA_DEST_I2C1	Data writes are triggered by the I2C1 request line	0x8
	DMA_DEST_I3C0	Data writes are triggered by the I3C0 request line	0x9
	DMA_DEST_I3C1	Data writes are triggered by the I3C1 request line	0xA
	DMA_DEST_G722_ENC	Data writes are triggered by the G.722 encoder request line	0xB
	DMA_DEST_G722_DEC	Data writes are triggered by the G.722 decoder request line	0xC
	DMA_DEST_RS_ENC	Data writes are triggered by the Reed-Solomon encoder request line	0xD
	DMA_DEST_RS_DEC	Data writes are triggered by the Reed-Solomon decoder request line	0xE
	DMA_DEST_D_I2C	Data writes are triggered by the D_I2C request line	0xF
SRC_SELECT	DMA_SRC_ALWAYS_ON	Data reads are always triggered	0x0*
	DMA_SRC_SPI0	Data reads are triggered by the SPI0	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		request line	
	DMA_SRC_SPI1	Data reads are triggered by the SPI1 request line	0x2
	DMA_SRC_PCM0	Data reads are triggered by the PCM0 request line	0x3
	DMA_SRC_PCM1	Data reads are triggered by the PCM1 request line	0x4
	DMA_SRC_UART0	Data reads are triggered by the UART0 request line	0x5
	DMA_SRC_ASRC	Data reads are triggered by the ASRC request line	0x6
	DMA_SRC_I2C0	Data reads are triggered by the I2C0 request line	0x7
	DMA_SRC_I2C1	Data reads are triggered by the I2C1 request line	0x8
	DMA_SRC_I3C0	Data reads are triggered by the I3C0 request line	0x9
	DMA_SRC_I3C1	Data reads are triggered by the I3C1 request line	0xA
	DMA_SRC_G722_ENC	Data reads are triggered by the G.722 encoder request line	0xB
	DMA_SRC_G722_DEC	Data reads are triggered by the G.722 decoder request line	0xC
	DMA_SRC_RS_ENC	Data reads are triggered by the Reed-Solomon encoder request line	0xD
	DMA_SRC_RS_DEC	Data reads are triggered by the Reed-Solomon decoder request line	0xE
	DMA_SRC_D_I2C	Data reads are triggered by the D_I2C request line	0xF
CHANNEL_PRIORITY	DMA_PRIORITY_0	Set the priority of DMA channel to 0 (Lowest)	0x0*
	DMA_PRIORITY_1	Set the priority of DMA channel to 1	0x1
	DMA_PRIORITY_2	Set the priority of DMA channel to 2	0x2
	DMA_PRIORITY_3	Set the priority of DMA channel to 3 (highest)	0x3
SRC_DEST_TRANS_LENGTH_SEL	DEST_TRANS_LENGTH_SEL	Transfer length counter depends on the destination word count	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	SRC_TRANS_LENGTH_SEL	Transfer length counter depends on the source word count	0x1
BYTE_ORDER	DMA_LITTLE_ENDIAN	DMA channel uses little endian mode	0x0*
	DMA_BIG_ENDIAN	DMA channel uses big endian mode	0x1

19.4.3.2 DMA*_CFG1

Bit Field	Read/Write	Field Name	Description
31:16	RW	INT_TRANSFER_LENGTH	Trigger a counter interrupt when the DMA transfer word count reaches this value
15:0	RW	TRANSFER_LENGTH	The length, in words, of each data transfer using DMA channel

19.4.3.3 DMA*_CTRL

Bit Field	Read/Write	Field Name	Description
13	RW	INT_CNT_TRIGGER_ENABLE	Enable waiting on a trigger when an interrupt counter event occurs
12:8	RW	TRIGGER_SOURCE	Selects which event triggers this DMA channel when triggering is enabled
6	RW	INT_CNT_ADDR_WRAP_ENABLE	Enable source or destination (depending on SRC_DEST_TRANS_LENGTH_SEL) address wrapping when an interrupt counter event occurs
5	RW	CLEAR_BUFFER_WHEN_WRAPPING	Clear buffer during address wrapping
4	W	BUFFER_CLEAR	Clear source buffer
3	W	CNTS_CLEAR	Clear counters
2:0	RW	MODE_ENABLE	Enable mode of operation of the DMA Channel

Field Name	Value Symbol	Value Description	Hex Value
INT_CNT_TRIGGER_ENABLE	DMA_INT_CNT_TRIGGER_DISABLE	The DMA channel does not pause when an interrupt counter event occurs	0x0*
	DMA_INT_CNT_TRIGGER_ENABLE	The DMA channel pauses when an interrupt counter event occurs and waits for a trigger source event to continue	0x1
TRIGGER_SOURCE	DMA_TRIGGER_CH0_COMPLETED	This DMA channel is triggered when DMA channel 0 is completed	0x0*
	DMA_TRIGGER_CH1_COMPLETED	This DMA channel is triggered when DMA channel 1 is completed	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DMA_TRIGGER_CH2_COMPLETED	This DMA channel is triggered when DMA channel 2 is completed	0x2
	DMA_TRIGGER_CH3_COMPLETED	This DMA channel is triggered when DMA channel 3 is completed	0x3
	DMA_TRIGGER_CH4_COMPLETED	This DMA channel is triggered when DMA channel 4 is completed	0x4
	DMA_TRIGGER_CH5_COMPLETED	This DMA channel is triggered when DMA channel 5 is completed	0x5
	DMA_TRIGGER_CH6_COMPLETED	This DMA channel is triggered when DMA channel 6 is completed	0x6
	DMA_TRIGGER_CH7_COMPLETED	This DMA channel is triggered when DMA channel 7 is completed	0x7
	DMA_TRIGGER_FIFO0_INT	This DMA channel is triggered when FIFO0 interrupt occurs	0x10
	DMA_TRIGGER_FIFO1_INT	This DMA channel is triggered when FIFO1 interrupt occurs	0x11
	DMA_TRIGGER_FIFO2_INT	This DMA channel is triggered when FIFO2 interrupt occurs	0x12
	DMA_TRIGGER_FIFO3_INT	This DMA channel is triggered when FIFO3 interrupt occurs	0x13
	DMA_TRIGGER_FIFO4_INT	This DMA channel is triggered when FIFO4 interrupt occurs	0x14
	DMA_TRIGGER_FIFO5_INT	This DMA channel is triggered when FIFO5 interrupt occurs	0x15
	DMA_TRIGGER_FIFO6_INT	This DMA channel is triggered when FIFO6 interrupt occurs	0x16
	DMA_TRIGGER_FIFO7_INT	This DMA channel is triggered when FIFO7 interrupt occurs	0x17
	DMA_TRIGGER_DSP0_INT	This DMA channel is triggered when DSP0 interrupt occurs	0x18
	DMA_TRIGGER_DSP1_INT	This DMA channel is triggered when DSP1 interrupt occurs	0x19
	DMA_TRIGGER_DSP2_INT	This DMA channel is triggered when DSP2 interrupt occurs	0x1A
	DMA_TRIGGER_DSP3_INT	This DMA channel is triggered when DSP3 interrupt occurs	0x1B
	DMA_TRIGGER_DSP4_INT	This DMA channel is triggered when DSP4 interrupt occurs	0x1C

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	DMA_TRIGGER_DSP5_INT	This DMA channel is triggered when DSP5 interrupt occurs	0x1D
	DMA_TRIGGER_DSP6_INT	This DMA channel is triggered when DSP6 interrupt occurs	0x1E
	DMA_TRIGGER_DSP7_INT	This DMA channel is triggered when DSP7 interrupt occurs	0x1F
INT_CNT_ADDR_WRAP_ENABLE	DMA_INT_CNT_ADDR_WRAP_DISABLE	Source address and destination addresses are wrapped normally	0x0*
	DMA_INT_CNT_ADDR_WRAP_ENABLE	When the interrupt counter is reached, source or destination address is wrapped depending on SRC_DEST_TRANS_LENGTH_SEL	0x1
CLEAR_BUFFER_WHEN_WRAPPING	DMA_KEEP_BUFFER_WHEN_WRAPPING	Buffer contents are not cleared during address wrapping	0x0*
	DMA_CLEAR_BUFFER_WHEN_WRAPPING	Buffer contents are cleared during address wrapping	0x1
BUFFER_CLEAR	DMA_CLEAR_BUFFER	Clears the DMA source buffered data	0x1
CNTS_CLEAR	DMA_CLEAR_CNTS	Clears the DMA counters	0x1
MODE_ENABLE	DMA_DISABLE	Disable DMA channel (channel waits on current READ or WRITE access to finish before the active bit is cleared)	0x0*
	DMA_ENABLE	Enable DMA channel and when completed disable	0x1
	DMA_ENABLE_WRAP	Enable DMA channel and when completed, wrap addresses and disable	0x2
	DMA_ENABLE_WRAP_RESTART	Enable DMA channel and when completed, wrap addresses and restart	0x3
	DMA_TRIGGER	When source trigger occurs enable DMA channel; When completed disable	0x4
	DMA_TRIGGER_WRAP	When source trigger occurs enable DMA channel; When completed wrap addresses and disable	0x5
	DMA_TRIGGER_WRAP_RESTART	When source trigger occurs enable DMA channel; When completed wrap addresses and restart	0x6
	DMA_TRIGGER_WRAP_TRIGGER_RESTART	When source trigger occurs enable DMA channel; When completed wrap addresses, and upon next trigger event restart	0x7

Ezairo 8300 Hardware Reference

19.4.3.4 DMA*_STATUS

Bit Field	Read/Write	Field Name	Description
10	R	ACTIVE	Active status of the channel
9	R	CNT_INT	Indicate if a counter interrupt has occurred on DMA channel
8	R	COMPLETE_INT	Indicate if a complete interrupt has occurred on DMA channel
6	W	CNT_INT_CLEAR	Clear the counter interrupt flag
5	W	COMPLETE_INT_CLEAR	Clear the complete interrupt flag
4	W	SRC_BUFFER_FILL_LVL_WR	Enable writing of SRC_BUFFER_FILL_LVL
3:0	RW	SRC_BUFFER_FILL_LVL	The number of nibbles currently in the source buffer (0: buffer is empty, 8: the 32-bit buffer is completely full)

Field Name	Value Symbol	Value Description	Hex Value
ACTIVE	DMA_NON_ACTIVE	DMA channel is disabled or waiting for a trigger event	0x0*
	DMA_ACTIVE	DMA channel is running	0x1
CNT_INT	DMA_CNT_INT_FALSE	Indicate that no counter interrupt has occurred	0x0*
	DMA_CNT_INT_TRUE	Indicate that a counter interrupt has occurred	0x1
COMPLETE_INT	DMA_COMPLETE_INT_FALSE	Indicate that a no complete interrupt has occurred	0x0*
	DMA_COMPLETE_INT_TRUE	Indicate that a complete interrupt has occurred	0x1
CNT_INT_CLEAR	DMA_CNT_INT_CLEAR	Clear the counter interrupt flag	0x1
COMPLETE_INT_CLEAR	DMA_COMPLETE_INT_CLEAR	Clear the complete interrupt flag	0x1
SRC_BUFFER_FILL_LVL_WR	DMA_SRC_BUFFER_FILL_LVL_WR	Enable writing of SRC_BUFFER_FILL_LVL	0x1

19.4.3.5 DMA*_SRC_ADDR

Bit Field	Read/Write	Field Name	Description
31:0	RW	SRC_ADDR	Address for the source of data transferred using DMA channel

19.4.3.6 DMA*_DEST_ADDR

Bit Field	Read/Write	Field Name	Description
31:0	RW	DEST_ADDR	Address for the destination of data transferred using DMA channel

Ezairo 8300 Hardware Reference

19.4.3.7 DMA*_CNTS

Bit Field	Read/Write	Field Name	Description
31:16	RW	INT_TRANSFER_WORD_CNT	Trigger a counter interrupt when the DMA transfer word count reaches this value
15:0	RW	TRANSFER_WORD_CNT	The number of words that have been transferred using DMA channel during the current transfer

19.4.3.8 DMA*_SRC_BUFFER

Bit Field	Read/Write	Field Name	Description
31:0	RW	SRC_BUFFER	Packed word which has been read from the source addresses.

19.5 G.722 HARDWARE ACCELERATORS

The G.722 accelerators provide an encoder and decoder for a subband audio compression compliant to the ITU-T G.722 standard. This standard provides a wide-band audio coding with a bandwidth of 50 Hz to 7 kHz compressed into a 64 kbps data stream. Further information about this CODEC is available from the International Telecommunications Union, described in the recommendation document available at <http://www.itu.int/rec/T-REC-G.722/en>.

The G.722 CODEC block is supported by the ACB_STATUS interrupt. This interrupt is used to indicate that errors have occurred when inputting data to any of the blocks. This interrupt can also be used to indicate when data can be provided to the G.722 decoder through the concealment inputs. To configure the interrupt sources that can trigger an ACB_STATUS interrupt, enable the desired events using the ACB_INT_ENABLE register. To read the status of these event sources, or to clear the cause of a pending ACB_STATUS interrupt, use the ACB_INT_STATUS register.

19.5.1 G.722 Decoder

The G.722 decoder accelerator consists of two main sections: the subband adaptive pulse code modulation (SB-ADPCM) decoder, and the quadrature mirror filter (QMF) filterbank. To enable simple communications between the CFX DSP or Arm Cortex-M3 processor and the accelerator, the G.722 decoder also includes input and output interfaces. A block diagram of the decoder (excluding the input and output interfaces) is shown in the "G.722 Codec Decoder Block Diagram" figure (Figure 46), which is taken from document *ITU-T G.722, 7 kHz audio-coding within 64 kbit/s*.

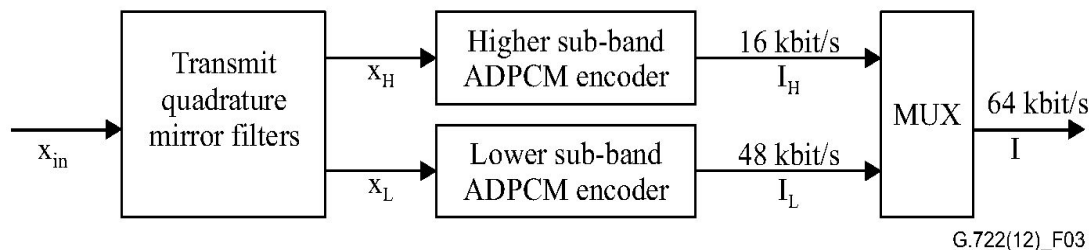


Figure 46. G.722 Codec Decoder Block Diagram

To enable the G.722 decoder, set the G722_CFG_DEC_ENABLE bit in the G722_CFG register.

19.5.1.1 G.722 SB-ADPCM Decoders

The subband decoder receives the 8-bit compressed data and splits it into lower and higher channels as:

$$I_H = I_{r1} I_{r2}$$

$$I_L = I_{r3} I_{r4} I_{r5} I_{r6} I_{r7} I_{r8}$$

In this notation, the I_{r1} stands for the MSB and I_{r8} stands for the LSB of the input data.

The 2-bit higher subband compressed data is channeled into the higher subband ADPCM decoder. A block diagram of this decoder is provided in the "G.722 Higher Subband Decoder Block Diagram" figure (Figure 47). The 6-bit lower subband compressed data is channeled into the lower subband ADPCM decoder. A block diagram of this decoder can be found in the "G.722 Lower Subband Decoder Block Diagram" figure (Figure 48). The figures are from document *ITU-T G.722, 7 kHz audio-coding within 64 kbit/s*.

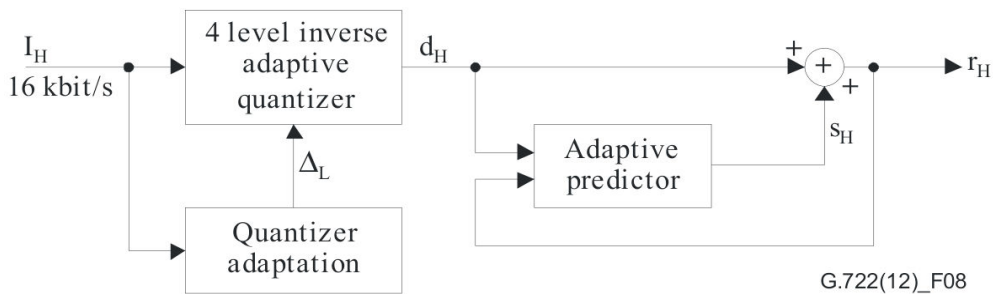


Figure 47. G.722 Higher Subband Decoder Block Diagram

Ezairo 8300 Hardware Reference

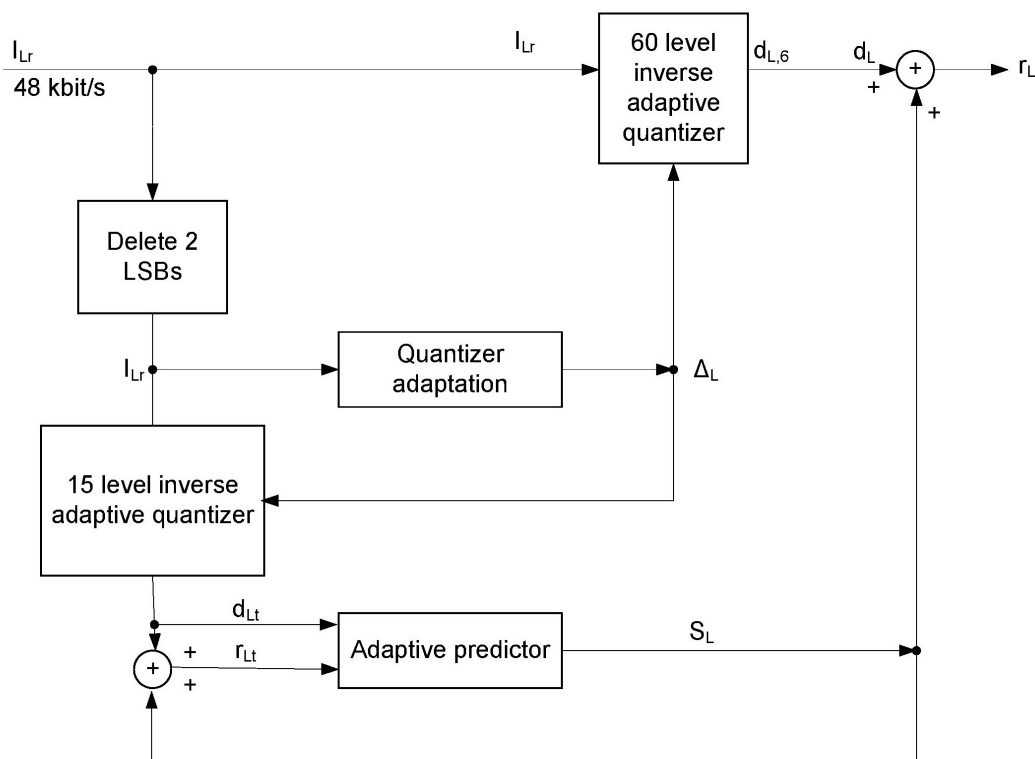


Figure 48. G.722 Lower Subband Decoder Block Diagram

This decoder output from each of the SB-ADPCM decoders is 16-bit data in mlp14 format with the sign bit repeated, sampled at half the decoder's output sample rate. These signals are output to the decoder QMF filterbank, which is used to combine the output signals.

NOTE: The details of the G.722 SB-ADPCM decoder's sub-blocks are not repeated here (refer to the G.722 standard). The decoder does not support auxiliary channel information that has been inserted into the low subband data, as the only mode supported is Mode 1, where all six bits supplied to the lower subband decoder are used in creating the output.

IMPORTANT: The G.722 implementation in Ezairo 8300 includes firmware support for Mode 2 and Mode 3 data. Neither the hardware accelerator nor the firmware provide an implementation that supports message insertion into the transmitted audio data.

The internal state of the SB-ADPCM decoder must be cleared prior to processing a new data stream, since this state, set by previous data streams, can introduce artifacts into the G.722 output. To clear the internal state of the G.722 SB-ADPCM decoder, set the `G722_CTRL_DEC_SBBND_CLR` bit in the `G722_CTRL` register. The G.722 SB-ADPCM decoder registers are cleared one cycle after this register bit is set.

Ezairo 8300 Hardware Reference

19.5.1.2 G.722 Decoder QMF Filterbank

The G.722 decoder accelerator uses a QMF filterbank that is equivalent to the filterbank used by the G.722 encoder, as required by G.722 specifications. For information about the filterbank itself, see [Section 19.5.2.1 “G.722 Encoder QMF Filterbank”](#) on page 727.

The filterbank receives two 16-bit subband samples in mlp14 format, with the sign bit repeated to extend the sample to 16 bits. G.722 specifications require that this data be provided at an 8 kHz sample rate, and interpolated by a factor of two, before being passed through the filterbank. The filterbank uses this input data to produce 16-bit output samples in the mlp14 format (with an extra sign bit) at the required 16 kHz sampling rate.

For a user application to implement a non-standard QMF filterbank, it can bypass the QMF filterbank by setting the `G722_CFG_DEC_FB_BYPASS` bit in the `G722_CFG` register. If the QMF filterbank is bypassed, the output audio data is provided prior to interpolation, with the high subband sample being provided in the upper half of the output interface register, and the low subband sample in the output interface register's lower half.

The internal state of the QMF filterbank must be cleared prior to processing a new data stream, since this state, set by previous data streams, can introduce artifacts into the G.722 output. To clear the internal state of the G.722 QMF filterbank, set the `G722_CTRL_DEC_FB_CLR` bit in the `G722_CTRL` register. The G.722 QMF filterbank registers are cleared one cycle after this register bit is set.

19.5.1.3 G.722 Decoder Concealment Support

The G.722 decoder accelerator provides support for an application to implement the packet loss concealment algorithms described in *Appendix IV of Recommendation ITU-T G.722*.

IMPORTANT: The G.722 decoder accelerator does not directly implement any packet loss concealment algorithm.

The concealment algorithm register accesses that are supported by the accelerator, as required by G.722 standards, include the following:

- The `G722_DEC_FBIN` register provides read and write access to the high and low intermediate data between the G.722 SB-ADPCM decoders and the decoder QMF filterbank.
- The `G722_CTRL_DEC_LSB_CNCL_CLR` bit in the `G722_CTRL` register can be used to clear the DLTi registers of the lower subband decoder.
- The `PLT1`, `PLT2`, `RLT1`, `DETL`, `NBL`, `SL`, and `SZL` registers of the low subband decoder can be overwritten using the `G722_DEC_PLT`, `G722_DEC_RLT`, `G722_DEC_SX`, and `G722_DEC_LSB_QUANT` registers.
- The `NBH` and `DETH` registers of the high subband decoder can be overwritten using the `G722_DEC_HSB_QUANT` register.

To enable concealment mode, set the `G722_CFG_DEC_CONCL_EBL` bit in the `G722_CFG` register. When operating in concealment mode, the accelerator produces a `G722_DEC_CONCEAL_RDY` interrupt when subband data is available to be overwritten according to the needs of the user application. The user application can read the current intermediate data from the `G722_DEC_FBIN` register, and can replace this data before the G.722 decoder completes any processing that uses the QMF filterbank. While held, the internal state registers from the SB-ADPCM decoders listed above can be updated by the user application.

19.5.1.4 G.722 Decoder Input and Output Interfaces

The input and output interfaces provided for the G.722 decoder accelerator coordinate the flow of samples through the accelerator. A block diagram highlighting these interfaces and how they relate to the G.722 SB-ADPCM decoders and QMF filterbank is provided in the "G.722 Decoder and Arm Cortex-M3 Processor Interface block diagram" figure (Figure 49), below.

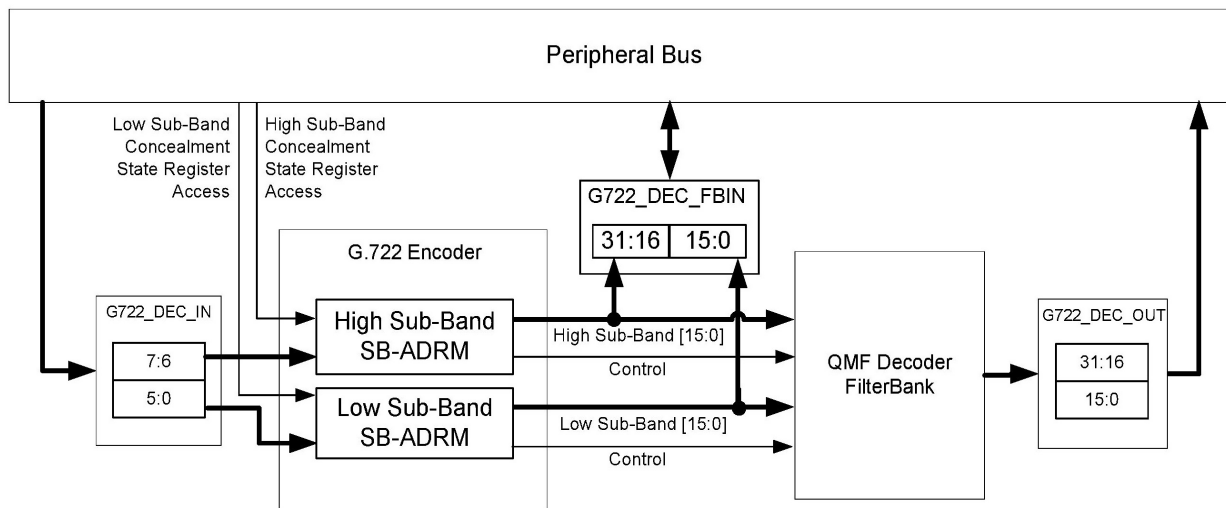


Figure 49. G.722 Decoder and Arm Cortex-M3 Processor Interface block diagram

The input interface to the G.722 decoder uses an 8-bit register, `G722_DEC_IN`, to supply the SB-ADPCM decoders with high and low subband input data. The SB-ADPCM decoders and QMF filterbank both remain idle until the input interface register has been written. A `G722_DEC_IN_RDY` interrupt is triggered when the G.722 decoder is enabled and idle. This interrupt indicates that the input interface is ready to accept more data.

The output interface uses a 32-bit register, `G722_DEC_OUT`, to store the decompressed data from the QMF filterbank (or a high subband and low subband output if the QMF filterbank is bypassed). When both output samples are available, the `G722_DEC_OUT_RDY` interrupt is triggered. After triggering this interrupt, the output interface waits until the output register is read before triggering another `G722_DEC_IN_RDY` interrupt indicating that the G.722 encoder is idle and ready for more data.

The G.722 standard requires output data to be formatted in an mlp14 format, where the sign bit is repeated to convert the 15-bit data to a 16-bit sample. The output data from the G.722 encoder can also be formatted in a standard 16-bit fractional (mlp15) format. To select between supporting 15-bit or 16-bit output data, configure the `G722_CFG_DEC_OUT_FORMAT` bit in the `G722_CFG` register.

If the input interface register is overwritten while the G.722 decoder accelerator is still processing the previous samples, a `G722_DEC_IN_ERR` interrupt is generated. In this situation, the accelerator continues its operation and disregards the new data written to the input register. To enable this interrupt, set the `G722_INT_ENABLE_DEC_IN_ERR` bit in the `G722_INT_ENABLE` register.

19.5.2 G.722 Encoder

The G.722 encoder accelerator consists of two main sections: the encoder quadrature mirror filter (QMF) FilterBank, and the subband adaptive pulse code modulation (SB-ADPCM) encoder. To enable simple communications with the CFX DSP or Arm Cortex-M3 processor, the accelerator also includes input and output interfaces. A block diagram of the encoder, excluding the input and output interfaces, is shown in the "[G.722 Encoder Block Diagram](#)" figure (Figure 50), taken from document ITU-T G.722, *7 kHz audio-coding within 64 kbit/s*.

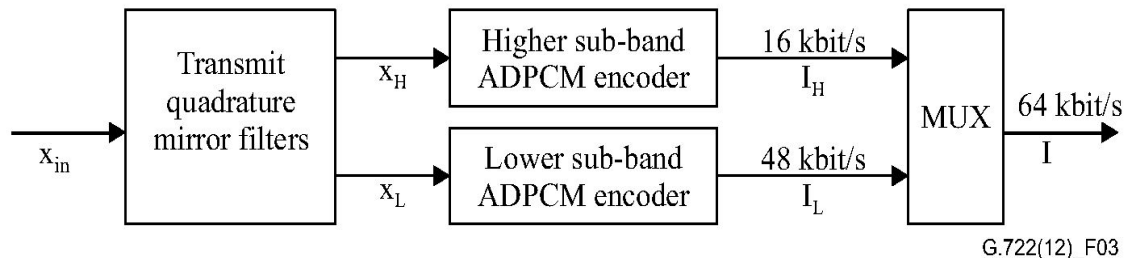


Figure 50. G.722 Encoder Block Diagram

To enable the G.722 encoder, set the `G722_CFG_ENC_ENABLE` bit in the `G722_CFG` register.

19.5.2.1 G.722 Encoder QMF Filterbank

The G.722 encoder implements the QMF filterbank dictated by G.722 standards. This filterbank is a symmetric, perfect reconstruction FIR filterbank with 24 coefficients (12 of which are unique coefficients). The standard dictates that the filterbank processes data at a 16 kHz sample rate. The magnitude response of the QMF filterbank is shown in the "[G.722 QMF Filterbank Magnitude Response](#)" figure (Figure 51), below.

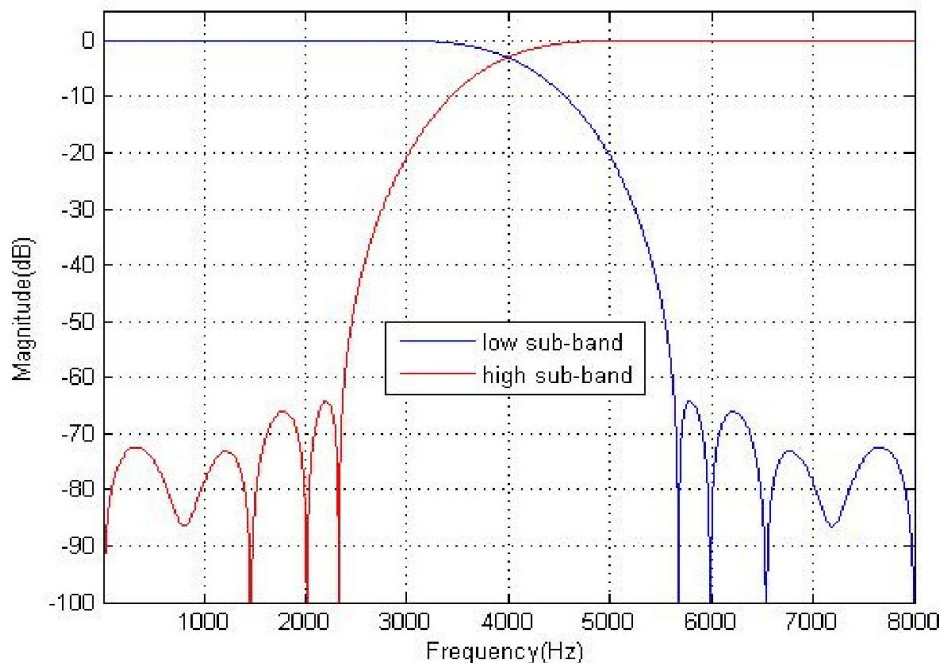


Figure 51. G.722 QMF Filterbank Magnitude Response

NOTE: Because of the 23 taps in the filter, the encoder filterbank has a constant group delay of 11.5 input samples (0.72 ms for 16 kHz sampling).

The output sampling rate for the QMF filterbank is decimated by a factor of 2 for each subband. That is, the G.722 QMF filterbank requires two input samples to create one subband output sample in the high subband, and one subband output sample in the low subband. The input and output data from the QMF filterbank uses a data word length of 16 bits.

If a user application wants to implement a non-standard QMF filterbank, the QMF filterbank can be bypassed by setting the `G722_CFG_ENC_FB_BYPASS` bit in the `G722_CFG` register. If the QMF filterbank is bypassed, the input audio must be pre-filtered before being provided to the G.722 encoder accelerator. The decimated pre-filtered input data is provided to the input interface with:

- The decimated high subband sample provided to the upper half of the input interface
- The decimated low subband sample provided to the lower half of the input interface

The internal state of the QMF filterbank must be cleared prior to processing a new data stream, since this state, set by previous data streams, can introduce artifacts into the G.722 output. To clear the internal state of the G.722 QMF filterbank, set the `G722_CTRL_ENC_FB_CLR` bit in the `G722_CTRL` register. The G.722 QMF filterbank registers are cleared one cycle after this register bit is set.

19.5.2.2 G.722 SB-ADPCM Encoders

The G.722 subband adaptive pulse-code modulation (SB-ADPCM) encoder follows the QMF filterbank. A block diagram showing the lower SB-ADPCM encoder is provided in the "Lower SB-ADPCM Encoder" figure (Figure 52), below, and a block diagram of the higher SB-ADPCM encoder can be found in the "Higher SB-ADPCM Encoder" figure (Figure 53). These diagrams are taken from document *ITU-T G.722, 7 kHz audio-coding within 64 kbit/s*.

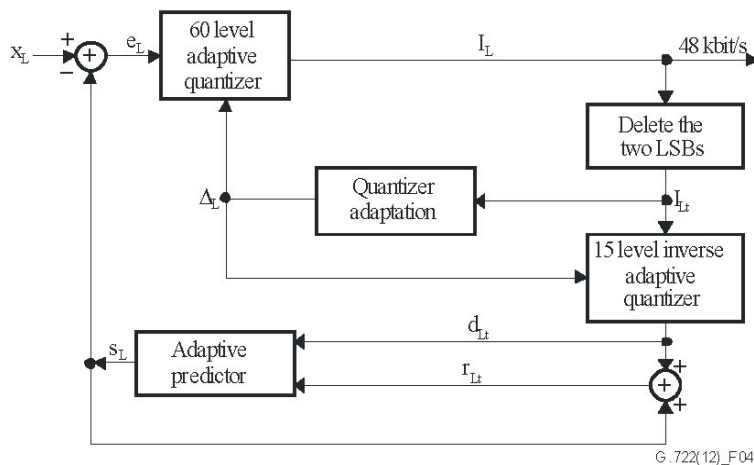


Figure 52. Lower SB-ADPCM Encoder

Ezairo 8300 Hardware Reference

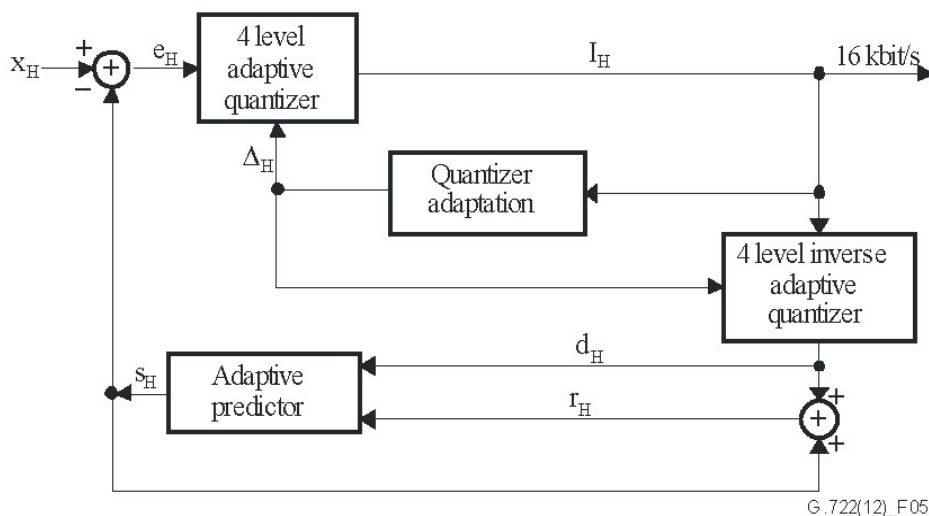


Figure 53. Higher SB-ADPCM Encoder

The lower SB-ADPCM encoder generates a 6-bit output from its 16-bit input, while the higher SB-ADPCM encoder generates a 2-bit output from its 16-bit input. These bits are combined at the output from the G.722 encoder accelerator in following format:

$$I_{H1}I_{H2}I_{L1}I_{L2}I_{L3}I_{L4}I_{L5}I_{L6}$$

In this octet format, the I_{H1} and I_{L1} are MSBs of the higher and lower subband encoder outputs respectively.

NOTE: The details of the sub-blocks of the G.722 SB-ADPCM encoder are not repeated here (refer to G.722 standards). The encoder does not allow any auxiliary channel insertion at the output, as the only mode supported is Mode 1, where all 6 bits coming out of the lower band encoder are used.

IMPORTANT: The G.722 implementation in Ezairo 8300 includes firmware support for Mode 2 and Mode 3 data. Neither the hardware accelerator nor the firmware provides an implementation that supports message insertion into the transmitted audio data.

The internal state of the SB-ADPCM encoder must be cleared prior to processing a new data stream, since this state, set by previous data streams, can introduce artifacts into the G.722 output. To clear the internal state of the G.722 SB-ADPCM encoder, set the `G722_CTRL_ENC_SBBND_CLR` bit in the `G722_CTRL` register. The G.722 SB-ADPCM encoder registers are cleared one cycle after this register bit is set.

19.5.2.3 G.722 Encoder Input and Output Interfaces

The input and output interfaces provided for the G.722 encoder accelerator coordinate the flow of samples through the accelerator. A block diagram highlighting these interfaces and how they relate to the G.722 QMF filterbank and SB-ADPCM encoders is provided in the "G.722 Encoder with Input and Output Interfaces" figure (Figure 54).

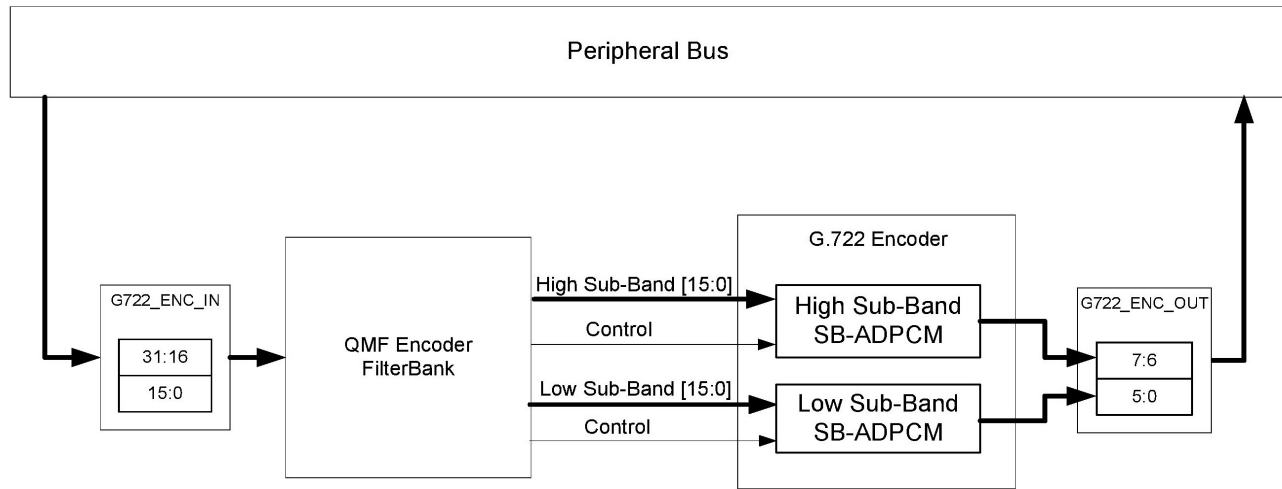


Figure 54. G.722 Encoder with Input and Output Interfaces

The G.722 standard requires input data to be formatted in an mlp14 format, where the sign bit is repeated to convert the 15-bit data to a 16-bit sample. If input data to the G.722 encoder is formatted in a standard frac16 (mlp15) format, the G.722 encoder can be configured for 16-bit input mode, which quantizes the LSB of the input data to convert the 16-bit input data to the required 15-bit format. To select between supporting 15-bit or 16-bit input data, configure the `G722_CFG_ENC_IN_FORMAT` bit in the `G722_CFG` register.

The input interface uses a 32-bit register, `G722_ENC_IN`, to supply the QMF filterbank with two 16-bit audio samples (or a high subband and low subband input if the QMF filterbank is bypassed). The QMF filterbank and SB-ADPCM encoder remain idle until both of the 16-bit sample fields in the input interface register have been written to. When data is provided to the G.722 encoder, the input interface transfers the low sample first, then the high sample from the G.722 input register to the QMF filterbank. A `G722_ENC_IN_RDY` interrupt is triggered when the G.722 encoder is enabled and idle. This interrupt indicates that the input interface is ready to accept more data.

The output interface uses an 8-bit register, `G722_ENC_OUT`, to store the compressed data from the two SB-ADPCM encoders. When this output is updated, a `G722_ENC_OUT_RDY` interrupt is triggered. After triggering this interrupt, the output interface waits until the output register is read before triggering another `G722_ENC_IN_RDY` interrupt indicating that the G.722 encoder is idle and ready for more data.

If the input interface registers are overwritten while the G.722 encoder accelerator is still processing the previous samples, a `G722_ENC_IN_ERR` interrupt is generated. In this situation, the accelerator continues its operation and disregards the new data written to the input registers. To enable this interrupt, set the `G722_INT_ENABLE_ENC_IN_ERR` bit in the `G722_INT_ENABLE` register.

19.5.3 G.722 Decoder and Encoder Configuration Registers

Register Name	Register Description	Address
G722_STATUS	G722 Interrupt Status Register	P:0x11C0 0x40000700
G722_INT_ENABLE	G722 Interrupt Mask Register	P:0x11C1 0x40000704
G722_CFG	G722 Configuration Registers	P:0x11C2 0x40000708
G722_CTRL	G722 Control Registers	P:0x11C3 0x4000070C
G722_ENC_IN	G722 Encoder Input Data Register	P:0x11C4 0x40000710
G722_ENC_OUT	G722 Encoder Output Data Register	P:0x11C5 0x40000714
G722_DEC_IN	G722 Decoder Input Data Register	P:0x11C6 0x40000718
G722_DEC_OUT	G722 Decoder Output Data Register	P:0x11C7 0x4000071C
G722_DEC_FBIN	G722 Decoder Filterbank Input Registers	P:0x11C8 0x40000720
G722_DEC_PLT	G722 Decoder Lower Subband Decoder PLT Registers	P:0x11C9 0x40000724
G722_DEC_RLT	G722 Decoder Lower Subband Decoder RLT Register	P:0x11CA 0x40000728
G722_DEC_SX	G722 Decoder Lower Subband Decoder SL and SLZ Pipeline Registers	P:0x11CB 0x4000072C
G722_DEC_LSB_QUANT	G722 Decoder Lower Subband Quantizer Registers	P:0x11CC 0x40000730
G722_DEC_HSB_QUANT	G722 Decoder Higher Subband Quantizer Registers	P:0x11CD 0x40000734

Ezairo 8300 Hardware Reference

19.5.3.1 G722_STATUS

Bit Field	Read/Write	Field Name	Description
10	R	DEC_CONCEAL_RDY	The G722 decoder concealment flag interrupt status
9	R	DEC_IN_ERR	The G722 decoder input interface error interrupt status
8	R	ENC_IN_ERR	The G722 encoder input interface error interrupt status
2	W	DEC_CONCEAL_RDY_CLEAR	The G722 decoder concealment flag interrupt clear
1	W	DEC_IN_ERR_CLEAR	The G722 decoder input interface error interrupt clear
0	W	ENC_IN_ERR_CLEAR	The G722 encoder input interface error interrupt clear

Field Name	Value Symbol	Value Description	Hex Value
DEC_CONCEAL_RDY	G722_DEC_INT_CONCEAL_RDY_INT_IDLE	This source has not set an interrupt	0x0*
	G722_DEC_INT_CONCEAL_RDY_INT_PENDING	The G722 decoder finished the decoding and waiting for the sub-band signal update from Arm Cortex-M3 to continue with filterbank	0x1
DEC_IN_ERR	G722_DEC_INT_IN_ERR_INT_IDLE	This source has not set an interrupt	0x0*
	G722_DEC_INT_IN_ERR_INT_PENDING	The G722 decoder input interface register overwritten before the accelerator finished its processing	0x1
ENC_IN_ERR	G722_ENC_INT_IN_ERR_INT_IDLE	This source has not set an interrupt	0x0*
	G722_ENC_INT_IN_ERR_INT_PENDING	The G722 encoder input interface register overwritten before the accelerator finished its processing	0x1
DEC_CONCEAL_RDY_CLEAR	G722_DEC_CONCEAL_RDY_INT_CLEAR	Clear the G722 decoder concealment interrupt	0x1
DEC_IN_ERR_CLEAR	G722_DEC_IN_ERR_INT_CLEAR	Clear the G722 decoder input interface error interrupt	0x1
ENC_IN_ERR_CLEAR	G722_ENC_IN_ERR_INT_CLEAR	Clear the G722 encoder input interface error interrupt	0x1

19.5.3.2 G722_INT_ENABLE

Bit Field	Read/Write	Field Name	Description
2	RW	DEC_CONCEAL_RDY	The G722 decoder concealment flag interrupt mask
1	RW	DEC_IN_ERR	The G722 decoder input interface error interrupt mask
0	RW	ENC_IN_ERR	The G722 encoder input interface error interrupt mask

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DEC_CONCEAL_RDY	G722_DEC_CONCEAL_RDY_INT_DISABLE	The G722 decoder concealment flag can not set an interrupt	0x0*
	G722_DEC_CONCEAL_RDY_INT_ENABLE	The G722 decoder concealment flag can set the G722_COM interrupt line	0x1
DEC_IN_ERR	G722_DEC_IN_ERR_INT_DISABLE	The G722 decoder input interface error can not set an interrupt	0x0*
	G722_DEC_IN_ERR_INT_ENABLE	The G722 decoder input interface error can set the G722_COM interrupt line	0x1
ENC_IN_ERR	G722_ENC_IN_ERR_INT_DISABLE	The G722 encoder input interface error can not set an interrupt	0x0*
	G722_ENC_IN_ERR_INT_ENABLE	The G722 encoder input interface error can set the G722_COM interrupt line	0x1

19.5.3.3 G722_CFG

Bit Field	Read/Write	Field Name	Description
11	RW	DEC_OUT_FORMAT	G722 Decoder Output Format
10	RW	DEC_FB_BYPASS	G722 Decoder filterbank bypass control
9	RW	DEC_CONCL_EBL	G722 Decoder concealment mode control
8	RW	DEC_ENABLE	Enable/disable the G722 Decoder
2	RW	ENC_IN_FORMAT	G722 Encoder Input Data Format
1	RW	ENC_FB_BYPASS	G722 Encoder filterbank bypass control
0	RW	ENC_ENABLE	Enable/disable the G722 Encoder

Field Name	Value Symbol	Value Description	Hex Value
DEC_OUT_FORMAT	G722_DEC_OUT_16BIT	The m1p14 output data is aligned to m1p15 format (zero padded)	0x0*
	G722_DEC_OUT_15BIT	The m1p14 output data is aligned to m2p14 format (sign extended)	0x1
DEC_FB_BYPASS	G722_DEC_FB_ENABLE	The G722 decoder filterbank is active.	0x0*
	G722_DEC_FB_BYPASS	The G722 decoder filterbank is bypassed; Its input is passed to its output as is.	0x1
DEC_CONCL_EBL	G722_CONCL_DISABLE	The G722 decoder is in normal operation mode.	0x0*
	G722_CONCL_ENABLE	The G722 decoder is in concealment	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
		mode. It waits for Arm Cortex-M3 to update the filterbank inputs after post-processing the sub-band decoder outputs.	
DEC_ENABLE	G722_DEC_DISABLE	Disable the G.722 decoder	0x0*
	G722_DEC_ENABLE	Enable the G.722 decoder	0x1
ENC_IN_FORMAT	G722_ENC_IN_16BIT	The input data is in m1p15 format. It is internally rounded to m2p14 format (standard)	0x0*
	G722_ENC_IN_15BIT	The input data is in m2p14 format. The MSB (extra sign bit) is dropped	0x1
ENC_FB_BYPASS	G722_ENC_FB_ENABLE	The G722 encoder filterbank is active.	0x0*
	G722_ENC_FB_BYPASS	The G722 encoder filterbank is bypassed; Its input is passed to its output as is.	0x1
ENC_ENABLE	G722_ENC_DISABLE	Disable the G.722 encoder	0x0*
	G722_ENC_ENABLE	Enable the G.722 encoder	0x1

19.5.3.4 G722_CTRL

Bit Field	Read/Write	Field Name	Description
10	W	DEC_SBBND_CLR	G722 subband decoders section clear
9	W	DEC_FB_CLR	G722 decoder filterbank section clear
8	W	DEC_LSB_CNCL_CLR	G722 decoder lower subband concealment clear
1	W	ENC_SBBND_CLR	G722 subband encoders section clear
0	W	ENC_FB_CLR	G722 encoder filterbank section clear

Field Name	Value Symbol	Value Description	Hex Value
DEC_SBBND_CLR	G722_DEC_SBBND_CLEAR	All registers will be set to their reset value immediately	0x1
DEC_FB_CLR	G722_DEC_FB_CLEAR	All registers will be set to their reset value immediately	0x1
DEC_LSB_CNCL_CLR	G722_DEC_LSB_CNCL_CLEAR	Decoder lower subband register registers are cleared to their reset values	0x1
ENC_SBBND_CLR	G722_ENC_SBBND_CLEAR	All registers will be set to their reset value immediately	0x1
ENC_FB_CLR	G722_ENC_FB_CLEAR	All registers will be set to their reset value immediately	0x1

19.5.3.5 G722_ENC_IN

Bit Field	Read/Write	Field Name	Description
31:16	RW	ENC_IN1	Audio sample 1
15:0	RW	ENC_IN0	Audio sample 0

19.5.3.6 G722_ENC_OUT

Bit Field	Read/Write	Field Name	Description
7:0	R	ENC_OUT	Compressed G722 encoder output

19.5.3.7 G722_DEC_IN

Bit Field	Read/Write	Field Name	Description
7:0	RW	DEC_IN	G722 decoder input to decompress

19.5.3.8 G722_DEC_OUT

Bit Field	Read/Write	Field Name	Description
31:16	R	DEC_OUT1	Audio sample 1
15:0	R	DEC_OUT0	Audio sample 0

19.5.3.9 G722_DEC_FBIN

Bit Field	Read/Write	Field Name	Description
31:16	RW	DEC_RH	Higher subband decoder output to the QMF
15:0	RW	DEC_RL	Lower subband decoder output to the QMF

19.5.3.10 G722_DEC_PLT

Bit Field	Read/Write	Field Name	Description
31	RW	DEC_LSB_PLT2	Decoder lower subband PLT2 sign register
15	RW	DEC_LSB_PLT1	Decoder lower subband PLT1 sign register

19.5.3.11 G722_DEC_RLT

Bit Field	Read/Write	Field Name	Description
15:0	RW	DEC_LSB_RLT1	Decoder lower subband RLT1 register

Ezairo 8300 Hardware Reference

19.5.3.12 G722_DEC_SX

Bit Field	Read/Write	Field Name	Description
31:16	RW	DEC_LSB_SZL	Decoder lower subband SZL pipeline register
15:0	RW	DEC_LSB_SL	Decoder lower subband SL pipeline register

19.5.3.13 G722_DEC_LSB_QUANT

Bit Field	Read/Write	Field Name	Description
30:18	RW	DEC_LSB_DETL	Decoder lower subband DETL register
14:0	RW	DEC_LSB_NBL	Decoder lower subband NBL register

19.5.3.14 G722_DEC_HSB_QUANT

Bit Field	Read/Write	Field Name	Description
30:18	RW	DEC_HSB_DETH	Decoder higher subband DETH register
14:0	RW	DEC_HSB_NBH	Decoder higher subband NBH register

19.6 MATH ACCELERATORS

Ezairo 8300 provides accelerators for the following mathematical operations:

- Log_2
- Exp_2
- 2nd, 3rd, 4th Root
- Division

Calculations are triggered by writing to the math accelerators' input registers. After completing execution, each accelerator stores the calculated results to its output registers. The accelerators are deterministic in their execution cycles, and bus wait states are used to stall accesses when an accelerator is not ready to be accessed. There is no interrupt generated at completion or if errors occur.

The math accelerators are located on the CFX X memory and Y memory buses, and are therefore subject to write buffering from that core. Care must be taken to ensure that the write to the math accelerators' input registers has occurred when determining when execution completes. For more information, see [Section 11.2.3.2 “CFX Memory Interface Write Buffers”](#) on page 345.

NOTE: Log_2 , Exp_2 , and Root accelerators share the same hardware. For this reason, bus wait states are used to stall accesses when an accelerator is not ready to be accessed. There is no interrupt generated at completion or if errors occur, so take care if these accelerators are accessed in close succession.

19.6.1 Log 2

The Log_2 accelerator takes a positive fractional input in mlp23 or mlp47 format, and calculates an output in m8p24 format, in two cycles. If a negative input is provided, the `D_Y_MATH_LOG_EXP_STATUS_LOG2_IN_NEG_ERR` flag bit is set in the `D_Y_MATH_LOG_EXP_STATUS` register. For negative inputs, the output depends on which register

Ezairo 8300 Hardware Reference

triggers the calculation. More information is provided in the paragraphs that follow. An input of zero returns -64 (0xC00000), because negative infinity has no fixed-point representation.

Inputs are written to the `D_Y_MATH_LOG2_IN_MSB` register in mlp23 format, and optionally the `D_X_MATH_LOG2_IN_LSB` register in p24 format for extended precision. If mlp23 input format is used, it is recommended to clear the lower 24 bits, to avoid a bias in the result if the bits happen to be non-zero. The input registers are write-only. Both input registers can be written in a single cycle using an XY memory access. With this input data combination, the accelerator calculation is triggered when the `D_Y_MATH_LOG2_IN_MSB` register is written.

Input data can also be written at the `D_Y_MATH_ABS_LOG2_IN_MSB` and `D_X_MATH_ABS_LOG2_IN_LSB` registers. The upper portion of the input word is taken from the `D_Y_MATH_ABS_LOG2_IN_MSB` register in mlp23 format, and the lower portion of the input word is taken from the `D_X_MATH_ABS_LOG2_IN_LSB` register in p24 format. These are write-only registers, and the read back from them returns all zeros. With this input data combination, the execution of the accelerator is triggered by the write to the `D_Y_MATH_ABS_LOG2_IN_MSB` register. Also note that both input registers can be written in a single cycle using an XY memory access.

If the calculation is triggered through the `D_Y_MATH_LOG2_IN_MSB` register and the input data is negative, the `D_Y_MATH_LOG_EXP_STATUS_LOG2_IN_NEG_ERR` flag bit is set, and the input to the calculation is forced to be 0. Therefore, the calculation always returns -64 in this case, which is the smallest integer value.

If the calculation is triggered through `D_Y_MATH_ABS_LOG2_IN_MSB` and the written data is negative, the `D_Y_MATH_LOG_EXP_STATUS_LOG2_IN_NEG_ERR` flag bit is set and the absolute value of the input data is fed into the log2 calculation. In the case of an input close to positive or negative fullscale, the log2 returns exactly 0.

Outputs are available in the `D_Y_MATH_LOG2_OUT_MSB` register in m8p16 format, and in the `D_X_MATH_LOG2_OUT_LSB` register in p8 format, MSB-aligned, with the remaining bits set to 0. The overall error of this function is less than half of one bit.

If the accelerator is utilized in both the main program and the ISRs, the content of the `D_Y_MATH_LOG_EXP_STATUS`, `D_Y_MATH_LOG2_OUT_MSB` and `D_X_MATH_LOG2_OUT_LSB` registers needs to be pushed to the stack for back-up at the start of the ISR. At ISR completion, the state of the accelerator needs to be restored by popping the content of these registers. If the accelerator is utilized in both the main program and the ISRs, and the input is not written in a single cycle, the `X_MATH_LOG_EXP_ROOT_LSB` content needs to be pushed onto the stack and popped after the ISR.

19.6.2 Exp 2

The Exp2 accelerator takes a negative input in m8p16 or m8p24 format, and calculates an output in mlp47 format, in two cycles. For positive inputs, the output depends on which register triggers the calculation. More information is provided in the paragraphs that follow.

Inputs are written to the `D_Y_MATH_EXP2_IN_MSB` register in m8p16 format and, optionally, the `D_X_MATH_EXP2_IN_LSB` register in p8 format, MSB-aligned, with the remaining bits ignored for extended precision. If mlp23 format is used, it is recommended to clear the lower 24 bits, to avoid a bias in the result if the bits happen to be non-zero. A read back from these registers always returns 0. You can write both input registers in a single cycle using an XY memory access. With this input data combination, the accelerator calculation is triggered when the `D_Y_MATH_EXP2_IN_MSB` register is written.

Input data can also be written over X and Y memories in parallel at the `D_Y_MATH_ABSNEG_EXP2_IN_MSB` and `D_X_MATH_ABSNEG_EXP2_IN_LSB` registers. The upper portion of the input word is taken from the `D_Y_MATH_ABSNEG_EXP2_IN_MSB` register in m8p16 format, and the lower portion of the input word is taken from the `D_X_`

Ezairo 8300 Hardware Reference

MATH_ABSNEG_EXP2_IN_LSB register MSBs in p8 format. These are write only registers, and the read back from them returns all zeros. With this input data combination, the execution of the accelerator is triggered by the write to the D_Y_MATH_ABSNEG_EXP2_IN_MSB register.

An input of 0 results the largest positive number at the output (0x7FFFFFFFFFFFFF). If the input data is exactly 0, the error flag is not set.

If the calculation is triggered through the D_Y_MATH_EXP2_IN_MSB register and the input data is positive, the D_Y_MATH_LOG_EXP_STATUS_EXP2_IN_POS_ERR flag bit is set and the calculation input is forced to be 0. Therefore, the calculation returns the largest positive number.

If the calculation is triggered through the D_Y_MATH_ABSNEG_EXP2_IN_MSB register and the input data is positive, the D_Y_MATH_LOG_EXP_STATUS_EXP2_IN_POS_ERR flag bit is set and the negated absolute value of the input data is fed into the exp2 calculation.

Outputs are available in the D_Y_MATH_EXP2_OUT_MSB register in mlp23 format, and in the D_X_MATH_EXP2_OUT_LSB register in p24 format. The overall error of this function is less than half of one bit.

If the accelerator is utilized in both the main program and the ISRs, the content of the D_Y_MATH_LOG_EXP_STATUS, D_Y_MATH_EXP2_OUT_MSB, and D_X_MATH_EXP2_OUT_LSB registers needs to be pushed to the stack for back-up at the start of the ISR. Before ISR completion, the state of the accelerator needs to be restored by popping the content of these registers. If not using single cycle access, also save and restore X_MATH_LOG_EXP_ROOT_LSB.

19.6.3 Roots

The Root accelerator takes a positive fractional input in mlp23 or mlp47 format, and calculates an output in mlp47 format. The square root and fourth root each take four cycles to complete. The cubic root takes five cycles to complete. For root2 and root4 operations, if a negative input is provided, the D_Y_MATH_LOG_EXP_STATUS_LOG2_IN_NEG_ERR flag bit is set in the D_Y_MATH_LOG_EXP_STATUS register. In the case of a negative data entry, the output response depends on which register has triggered the calculation.

The result of the root calculation is always positive or zero, and fractional in mlp47 format for root2 and root4 operations. The sign of the root3 output depends on the sign of the input data. For root3 negative inputs, the result is a negative output, and the D_Y_MATH_LOG_EXP_STATUS_LOG2_IN_NEG_ERR flag is not set.

Inputs need to be written to the appropriate registers, depending on which root calculation you are performing:

- Square root inputs are written to the D_Y_MATH_ROOT2_IN_MSB register in mlp23 format and, optionally, the D_X_MATH_ROOT2_IN_LSB register in p24 format for extended precision.
- Cubic root inputs are written to the D_Y_MATH_ROOT3_IN_MSB register in mlp23 format and, optionally, the D_X_MATH_ROOT3_IN_LSB register in p24 format for extended precision.
- Fourth root inputs are written to the D_Y_MATH_ROOT4_IN_MSB register in mlp23 format and, optionally, the D_X_MATH_ROOT4_IN_LSB register in p24 format for extended precision.

You can write to both the input registers in a single cycle. The accelerator calculation is triggered when the D_Y_MATH_ROOT2_IN_MSB, D_Y_MATH_ROOT3_IN_MSB, or D_Y_MATH_ROOT4_IN_MSB register is written.

When the input to the root2, root3 or root4 calculation is 0, it returns exactly 0 as the result. When the root3 input is close to fullscale -1, the conversion results in exactly -1. When the input is close to 1, the output is the largest positive fullscale number in fractional format.

Ezairo 8300 Hardware Reference

If the root2 calculation is triggered through the `D_Y_MATH_ROOT2_IN_MSB` register or the root4 calculation is triggered through the `D_Y_MATH_ROOT4_IN_MSB` and the input data is negative, the `D_Y_MATH_LOG_EXP_STATUS_LOG2_IN_NEG_ERR` flag bit is set and the input to the calculation is forced to be 0. Therefore, the calculation always returns 0.

If the root2 calculation is triggered through the `D_Y_MATH_ABS_ROOT2_IN_MSB` register or the root4 calculation is triggered through the `D_Y_MATH_ABS_ROOT4_IN_MSB` and the input data is negative, the `D_Y_MATH_LOG_EXP_STATUS_LOG2_IN_NEG_ERR` flag bit is set and the input to the calculation is the absolute value of the written data.

Outputs are available in the `D_Y_MATH_ROOT_OUT_MSB` register in mlp24 format, and in the `D_MATH_ROOT_OUT_LSB` register in p24 format.

If the accelerator is utilized in both the main program and the ISRs, the content of the `D_Y_MATH_LOG_EXP_STATUS`, `D_Y_MATH_ROOT_OUT_MSB`, and `D_X_MATH_ROOT_OUT_LSB` registers needs to be pushed to the stack for back-up at the start of the ISR. Before ISR completion, the state of the accelerator needs to be restored by popping the content of these registers. If not using single cycle access, the `X_MATH_LOG_EXP_ROOT_LSB` register needs to be pushed onto the stack at the start of the ISR, and popped before the end of the ISR.

19.6.4 Division

The division accelerator can be configured to work with signed or unsigned numbers, selected using the `DIV_SM` bit in the `D_Y_MATH_DIV_STATUS` register.

- In signed mode, the accelerator calculates the quotient of a dividend in mlp23 and a divisor in mlp23 format.
- In unsigned mode, the accelerator calculates the quotient of a dividend in p24 and a divisor in p24 format.

In both signed and unsigned modes, the result is returned in m24p24 format and the calculation takes six cycles to complete. For example, division of signed 0x800000 by signed 0xFFFFF results in signed 0x7FFFFFFFFF.

Division by zero sets the `D_Y_MATH_DIV_STATUS_DIV_ZERO` flag bit in the `D_Y_MATH_DIV_STATUS` register. The result of the division by zero in signed mode depends on the sign bit of the dividend. If the dividend is positive, the quotient is 0x7FFFFFFFFF. If the dividend is negative, the quotient is 0x800000000000. In unsigned mode, the result of a division by zero is 0xFFFFFFFF. The `D_Y_MATH_DIV_STATUS_DIV_ZERO` bit is writable. This is necessary so that the status of the `D_Y_MATH_DIV_STATUS` register can be restored after an interrupt is serviced.

The dividend can be written to the `D_Y_MATH_DIVIDEND` register or the `D_Y_MATH_DIVIDEND_TRG` register. The divisor can be written to the `D_X_MATH_DIVISOR` register or the `D_X_MATH_DIVISOR_TRG` register. You can write both input registers in a single cycle using an XY memory access. The accelerator calculation is triggered when either the `D_Y_MATH_DIVIDEND_TRG` register or the `D_X_MATH_DIVISOR_TRG` register is written.

Toggling the state of the `Y_MATH_DIV_STATUS_DIV_SM` bit in the `D_Y_MATH_DIV_STATUS` register also triggers processing, but processing triggered in this way does not update the `Y_MATH_DIV_STATUS_DIV_ZERO` bit in the `D_Y_MATH_DIV_STATUS` register. This behavior is required in order to allow proper backing up and restoring of the entire state of the accelerator in an ISR.

Outputs are available in the `D_Y_MATH_DIV_OUT_MSB` register in m24 format, and in the `D_X_DIV_OUT_LSB` register in p24 format.

Ezairo 8300 Hardware Reference

Unlike the other math accelerators, division restores its input registers, since the dividend and divisor can be updated at different times. The process of restoring these registers can trigger more than one division calculation, as described below:

1. The reloading of the `D_Y_MATH_DIVIDEND` and `D_X_MATH_DIVISOR_TRG` registers, or the `D_Y_MATH_DIVIDEND_TRG` and `D_X_MATH_DIVISOR` registers, triggers a new division calculation. The status register update that is popped is held back with the wait signal, until the division calculation has been completed.
2. The division result updates the output registers to their state prior to the ISR. The status register is then overwritten with the popped content.
3. If the `D_Y_MATH_DIV_STATUS_DIV_SM` bit has a different value from the popped content, this triggers another calculation, but does not overwrite the `Y_MATH_DIV_STATUS_DIV_ZERO` bit that has been popped.

As seen in step 3 above, in a worst case scenario it takes 16 clock cycles for the accelerator to restore its state after an ISR. This does not occur in the following cases:

- An application always uses exclusively signed or unsigned data when using the division accelerator.
- All writes to the input registers are performed in a single cycle using an XY memory access.

If the accelerator is utilized in both the main program and the ISRs, the content of the `D_Y_MATH_DIV_STATUS`, `D_Y_MATH_DIVIDEND`, and `D_X_MATH_DIVISOR_TRG` registers needs to be pushed to the stack for back-up at the start of the ISR, beginning with the `D_Y_MATH_DIV_STATUS` register. At ISR completion, the state of the accelerator needs to be restored by popping the content of these registers, starting with the `D_Y_MATH_DIVIDEND` register and the `D_X_MATH_DIVISOR` register. The `D_Y_MATH_DIV_STATUS` register needs to be popped as well, after the dividend and divisor.

IMPORTANT: The dividend input register does not prevent writes before the previous divider output has been calculated. To ensure the divider output is correct, the dividend input register to the division math accelerator must be stable throughout the calculation.

19.6.5 Control and Configuration Registers

Register Name	Register Description	Address
<code>X_MATH_DIVISOR_TRG</code>	The DIV Accelerator Divisor Input in m1p23 or p24 format	0x20802004
<code>D_X_MATH_DIVISOR_TRG</code>		X:0x0801
<code>X_MATH_LOG_EXP_ROOT_LSB</code>	The LOG2/EXP2/ROOT Accelerator Input LSBs in p24 format (automatically updated when any LSB register is written without the MSB part; used when MSB register is written without the LSB part)	0x20802008
<code>D_X_MATH_LOG_EXP_ROOT_LSB</code>		X:0x0802
<code>X_MATH_LOG2_OUT_LSB</code>	The LOG2 Accelerator Output LSBs in p8 format	0x2080200C
<code>D_X_MATH_LOG2_OUT_LSB</code>		X:0x0803
<code>X_MATH_EXP2_OUT_LSB</code>	The EXP2 Accelerator Output LSBs in p24 format	0x20802010
<code>D_X_MATH_EXP2_OUT_LSB</code>		X:0x0804
<code>X_MATH_ROOT_OUT_LSB</code>	The ROOT Accelerator Output LSBs in p24 format	0x20802014
<code>D_X_MATH_ROOT_OUT_LSB</code>		X:0x0805

onsemi Confidential
Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
X_MATH_LOG2_IN_LSB	The LOG2 Accelerator Input LSBs in p24 format (negative input is treated as zero)	0x20802018
D_X_MATH_LOG2_IN_LSB		X:0x0806
X_MATH_ABS_LOG2_IN_LSB	The LOG2 Accelerator Input LSBs in p24 format (negative input is negated)	0x2080201C
D_X_MATH_ABS_LOG2_IN_LSB		X:0x0807
X_MATH_EXP2_IN_LSB	The EXP2 Accelerator Input LSBs in p8 format (positive input is treated as zero)	0x20802020
D_X_MATH_EXP2_IN_LSB		X:0x0808
X_MATH_ABSNEG_EXP2_IN_LSB	The EXP2 Accelerator Input LSBs in p8 format (positive input is negated)	0x20802024
D_X_MATH_ABSNEG_EXP2_IN_LSB		X:0x0809
X_MATH_ROOT2_IN_LSB	The Square ROOT Accelerator Input LSBs in p24 format (negative input is treated as zero)	0x20802028
D_X_MATH_ROOT2_IN_LSB		X:0x080A
X_MATH_ABS_ROOT2_IN_LSB	The Square ROOT Accelerator Input LSBs in p24 format (negative input is negated)	0x2080202C
D_X_MATH_ABS_ROOT2_IN_LSB		X:0x080B
X_MATH_ROOT3_IN_LSB	The Cubic ROOT Accelerator Input LSBs in p24 format	0x20802030
D_X_MATH_ROOT3_IN_LSB		X:0x080C
X_MATH_ROOT4_IN_LSB	The Fourth ROOT Accelerator Input LSBs in p24 format (negative input is treated as zero)	0x20802038
D_X_MATH_ROOT4_IN_LSB		X:0x080E
X_MATH_ABS_ROOT4_IN_LSB	The Fourth ROOT Accelerator Input LSBs in p24 format (negative input is negated)	0x2080203C
D_X_MATH_ABS_ROOT4_IN_LSB		X:0x080F
X_MATH_DIVISOR	The DIV Accelerator Divisor Input in m1p23 or p24 format	0x20802040
D_X_MATH_DIVISOR		X:0x0810
X_MATH_DIV_OUT_LSB	The DIV Accelerator Output LSBs in p24 format	0x20802044
D_X_MATH_DIV_OUT_LSB		X:0x0811
Y_MATH_DIV_STATUS	The Division Math Accelerator Status	0x20C42000
D_Y_MATH_DIV_STATUS		Y:0x0800
Y_MATH_DIVIDEND	The DIV Accelerator Dividend Input in m1p23 or p24 format	0x20C42004
D_Y_MATH_DIVIDEND		Y:0x0801
Y_MATH_LOG_EXP_STATUS	The Log2 and Exp2 Math Accelerator Status	0x20C42008
D_Y_MATH_LOG_EXP_STATUS		Y:0x0802
Y_MATH_LOG2_OUT_MSB	The LOG2 Accelerator Output MSBs in m8p16 format	0x20C4200C
D_Y_MATH_LOG2_OUT_MSB		Y:0x0803

onsemi Confidential
Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
Y_MATH_EXP2_OUT_MSB	The EXP2 Accelerator Output MSBs in m1p23 format	0x20C42010
D_Y_MATH_EXP2_OUT_MSB		Y:0x0804
Y_MATH_ROOT_OUT_MSB	The ROOT Accelerator Output MSBs in m1p23 format	0x20C42014
D_Y_MATH_ROOT_OUT_MSB		Y:0x0805
Y_MATH_LOG2_IN_MSB	The LOG2 Accelerator Input MSBs in m1p23 format (negative input is treated as zero)	0x20C42018
D_Y_MATH_LOG2_IN_MSB		Y:0x0806
Y_MATH_ABS_LOG2_IN_MSB	The LOG2 Accelerator Input MSBs in m1p23 format (negative input is negated)	0x20C4201C
D_Y_MATH_ABS_LOG2_IN_MSB		Y:0x0807
Y_MATH_EXP2_IN_MSB	The EXP2 Accelerator Input MSBs in m8p16 format (positive input is treated as zero)	0x20C42020
D_Y_MATH_EXP2_IN_MSB		Y:0x0808
Y_MATH_ABSNEG_EXP2_IN_MSB	The EXP2 Accelerator Input MSBs in m8p16 format (positive input is negated)	0x20C42024
D_Y_MATH_ABSNEG_EXP2_IN_MSB		Y:0x0809
Y_MATH_ROOT2_IN_MSB	The Square ROOT Accelerator Input MSBs in m1p23 format (negative input is treated as zero)	0x20C42028
D_Y_MATH_ROOT2_IN_MSB		Y:0x080A
Y_MATH_ABS_ROOT2_IN_MSB	The Square ROOT Accelerator Input MSBs in m1p23 format (negative input is negated)	0x20C4202C
D_Y_MATH_ABS_ROOT2_IN_MSB		Y:0x080B
Y_MATH_ROOT3_IN_MSB	The Cubic ROOT Accelerator Input MSBs in m1p23 format	0x20C42030
D_Y_MATH_ROOT3_IN_MSB		Y:0x080C
Y_MATH_ROOT4_IN_MSB	The Fourth ROOT Accelerator Input MSBs in m1p23 format (negative input is treated as zero)	0x20C42038
D_Y_MATH_ROOT4_IN_MSB		Y:0x080E
Y_MATH_ABS_ROOT4_IN_MSB	The Fourth ROOT Accelerator Input MSBs in m1p23 format (negative input is negated)	0x0000080F
D_Y_MATH_ABS_ROOT4_IN_MSB		Y:0x080F
Y_MATH_DIVIDEND_TRG	The DIV Accelerator Dividend Input in m1p23 or p24 format	0x00000810
D_Y_MATH_DIVIDEND_TRG		Y:0x0810
Y_MATH_DIV_OUT_MSB	The DIV Accelerator Output MSBs in m1p23 format	0x00000811
D_Y_MATH_DIV_OUT_MSB		Y:0x0811

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
D_X_MATH_DIVISOR_TRG X_MATH_DIVISOR_TRG	The Division Accelerator Divisor Input in m1p23 or p24 format	X:0x0801 0x20802004
D_X_MATH_LOG_EXP_ROOT_LSB X_MATH_LOG_EXP_ROOT_LSB	The Log2/Exp2/Root Accelerator Input LSBs in p24 format (automatically updated when any LSB register is written without the MSB part; used when MSB register is written without the LSB part)	X:0x0802 0x20802008
D_X_MATH_LOG2_OUT_LSB X_MATH_LOG2_OUT_LSB	The Log2 Accelerator Output LSBs in p8 format	X:0x0803 0x2080200C
D_X_MATH_EXP2_OUT_LSB X_MATH_EXP2_OUT_LSB	The Exp2 Accelerator Output LSBs in p24 format	X:0x0804 0x20802010
D_X_MATH_ROOT_OUT_LSB X_MATH_ROOT_OUT_LSB	The Root Accelerator Output LSBs in p24 format	X:0x0805 0x20802014
D_X_MATH_LOG2_IN_LSB X_MATH_LOG2_IN_LSB	The Log2 Accelerator Input LSBs in p24 format (negative input is treated as zero)	X:0x0806 0x20802018
D_X_MATH_ABS_LOG2_IN_LSB X_MATH_ABS_LOG2_IN_LSB	The Log2 Accelerator Input LSBs in p24 format (negative input is negated)	X:0x0807 0x2080201C
D_X_MATH_EXP2_IN_LSB X_MATH_EXP2_IN_LSB	The Exp2 Accelerator Input LSBs in p8 format (positive input is treated as zero)	X:0x0808 0x20802020
D_X_MATH_ABSNEG_EXP2_IN_LSB X_MATH_ABSNEG_EXP2_IN_LSB	The Exp2 Accelerator Input LSBs in p8 format (positive input is negated)	X:0x0809 0x20802024
D_X_MATH_ROOT2_IN_LSB X_MATH_ROOT2_IN_LSB	The Square Root Accelerator Input LSBs in p24 format (negative input is treated as zero)	X:0x080A 0x20802028
D_X_MATH_ABS_ROOT2_IN_LSB X_MATH_ABS_ROOT2_IN_LSB	The Square Root Accelerator Input LSBs in p24 format (negative input is negated)	X:0x080B 0x2080202C
D_X_MATH_ROOT3_IN_LSB X_MATH_ROOT3_IN_LSB	The Cubic Root Accelerator Input LSBs in p24 format	X:0x080C 0x20802030
D_X_MATH_ROOT4_IN_LSB X_MATH_ROOT4_IN_LSB	The Fourth Root Accelerator Input LSBs in p24 format (negative input is treated as zero)	X:0x080E 0x20802038
D_X_MATH_ABS_ROOT4_IN_LSB X_MATH_ABS_ROOT4_IN_LSB	The Fourth Root Accelerator Input LSBs in p24 format (negative input is negated)	X:0x080F 0x2080203C

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
D_X_MATH_DIVISOR X_MATH_DIVISOR	The Division Accelerator Divisor Input in m1p23 or p24 format	X:0x0810 0x20802040
D_X_MATH_DIV_OUT_LSB X_MATH_DIV_OUT_LSB	The Division Accelerator Output LSBs in p24 format	X:0x0811 0x20802044
D_Y_MATH_DIV_STATUS Y_MATH_DIV_STATUS	The Division Accelerator Status	Y:0x0800 0x20C42000
D_Y_MATH_DIVIDEND Y_MATH_DIVIDEND	The Division Accelerator Dividend Input in m1p23 or p24 format	Y:0x0801 0x20C42004
D_Y_MATH_LOG_EXP_STATUS Y_MATH_LOG_EXP_STATUS	The Log2 and Exp2 Accelerator Status	Y:0x0802 0x20C42008
D_Y_MATH_LOG2_OUT_MSB Y_MATH_LOG2_OUT_MSB	The Log2 Accelerator Output MSBs in m8p16 format	Y:0x0803 0x20C4200C
D_Y_MATH_EXP2_OUT_MSB Y_MATH_EXP2_OUT_MSB	The Exp2 Accelerator Output MSBs in m1p23 format	Y:0x0804 0x20C42010
D_Y_MATH_ROOT_OUT_MSB Y_MATH_ROOT_OUT_MSB	The Root Accelerator Output MSBs in m1p23 format	Y:0x0805 0x20C42014
D_Y_MATH_LOG2_IN_MSB Y_MATH_LOG2_IN_MSB	The Log2 Accelerator Input MSBs in m1p23 format (negative input is treated as zero)	Y:0x0806 0x20C42018
D_Y_MATH_ABS_LOG2_IN_MSB Y_MATH_ABS_LOG2_IN_MSB	The Log2 Accelerator Input MSBs in m1p23 format (negative input is negated)	Y:0x0807 0x20C4201C
D_Y_MATH_EXP2_IN_MSB Y_MATH_EXP2_IN_MSB	The Exp2 Accelerator Input MSBs in m8p16 format (positive input is treated as zero)	Y:0x0808 0x20C42020
D_Y_MATH_ABSNEG_EXP2_IN_MSB Y_MATH_ABSNEG_EXP2_IN_MSB	The Exp2 Accelerator Input MSBs in m8p16 format (positive input is negated)	Y:0x0809 0x20C42024
D_Y_MATH_ROOT2_IN_MSB Y_MATH_ROOT2_IN_MSB	The Square Root Accelerator Input MSBs in m1p23 format (negative input is treated as zero)	Y:0x080A 0x20C42028
D_Y_MATH_ABS_ROOT2_IN_MSB Y_MATH_ABS_ROOT2_IN_MSB	The Square Root Accelerator Input MSBs in m1p23 format (negative input is negated)	Y:0x080B 0x20C4202C

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
D_Y_MATH_ROOT3_IN_MSB Y_MATH_ROOT3_IN_MSB	The Cubic Root Accelerator Input MSBs in m1p23 format	Y:0x080C 0x20C42030
D_Y_MATH_ROOT4_IN_MSB Y_MATH_ROOT4_IN_MSB	The Fourth Root Accelerator Input MSBs in m1p23 format (negative input is treated as zero)	Y:0x080E 0x20C42038
D_Y_MATH_ABS_ROOT4_IN_MSB Y_MATH_ABS_ROOT4_IN_MSB	The Fourth Root Accelerator Input MSBs in m1p23 format (negative input is negated)	Y:0x080F 0x0000080F
D_Y_MATH_DIVIDEND_TRG Y_MATH_DIVIDEND_TRG	The Division Accelerator Dividend Input in m1p23 or p24 format	Y:0x0810 0x00000810
D_Y_MATH_DIV_OUT_MSB Y_MATH_DIV_OUT_MSB	The Division Accelerator Output MSBs in m1p23 format	Y:0x0811 0x00000811

19.6.5.1 D_X_MATH_DIVISOR_TRG, X_MATH_DIVISOR_TRG

Bit Field	Read/Write	Field Name	Description
23:0	RW	DIVISOR_TRG	The Division accelerator Divisor Input. It triggers division calculation

19.6.5.2 D_X_MATH_LOG_EXP_ROOT_LSB, X_MATH_LOG_EXP_ROOT_LSB

Bit Field	Read/Write	Field Name	Description
23:0	RW	LOG_EXP_ROOT_IN_LSB	The Log/Exp/Root Accelerator Input LSBs

19.6.5.3 D_X_MATH_LOG2_OUT_LSB, X_MATH_LOG2_OUT_LSB

Bit Field	Read/Write	Field Name	Description
23:16	RW	LOG2_OUT_LSB	The Log2 Accelerator Output LSBs. The MSBs are in Y memory

19.6.5.4 D_X_MATH_EXP2_OUT_LSB, X_MATH_EXP2_OUT_LSB

Bit Field	Read/Write	Field Name	Description
23:0	RW	EXP2_OUT_LSB	The Exp2 Accelerator Output LSBs. The MSBs are in Y memory

19.6.5.5 D_X_MATH_ROOT_OUT_LSB, X_MATH_ROOT_OUT_LSB

Bit Field	Read/Write	Field Name	Description
23:0	RW	ROOT_OUT_LSB	The Root Accelerator Output LSBs. The MSBs are in Y memory

Ezairo 8300 Hardware Reference

19.6.5.6 D_X_MATH_LOG2_IN_LSB, X_MATH_LOG2_IN_LSB

Bit Field	Read/Write	Field Name	Description
23:0	W	LOG2_IN_LSB	The Log2 Accelerator Input LSBs. The MSBs are in Y memory

19.6.5.7 D_X_MATH_ABS_LOG2_IN_LSB, X_MATH_ABS_LOG2_IN_LSB

Bit Field	Read/Write	Field Name	Description
23:0	W	ABS_LOG2_IN_LSB	The Log2 Accelerator Input LSBs. The MSBs are in Y memory

19.6.5.8 D_X_MATH_EXP2_IN_LSB, X_MATH_EXP2_IN_LSB

Bit Field	Read/Write	Field Name	Description
23:16	W	EXP2_IN_LSB	The Exp2 Accelerator Input LSBs. The MSBs are in Y memory

19.6.5.9 D_X_MATH_ABSNEG_EXP2_IN_LSB, X_MATH_ABSNEG_EXP2_IN_LSB

Bit Field	Read/Write	Field Name	Description
23:16	W	ABSNEG_EXP2_IN_LSB	The Exp2 Accelerator Input LSBs. The MSBs are in Y memory

19.6.5.10 D_X_MATH_ROOT2_IN_LSB, X_MATH_ROOT2_IN_LSB

Bit Field	Read/Write	Field Name	Description
23:0	W	ROOT2_IN_LSB	The Root2 Accelerator Input LSBs. The MSBs are in Y memory

19.6.5.11 D_X_MATH_ABS_ROOT2_IN_LSB, X_MATH_ABS_ROOT2_IN_LSB

Bit Field	Read/Write	Field Name	Description
23:0	W	ABS_ROOT2_IN_LSB	The Root2 Accelerator Input LSBs. The MSBs are in Y memory

19.6.5.12 D_X_MATH_ROOT3_IN_LSB, X_MATH_ROOT3_IN_LSB

Bit Field	Read/Write	Field Name	Description
23:0	W	ROOT3_IN_LSB	The Root3 Accelerator Input LSBs. The MSBs are in Y memory

19.6.5.13 D_X_MATH_ROOT4_IN_LSB, X_MATH_ROOT4_IN_LSB

Bit Field	Read/Write	Field Name	Description
23:0	W	ROOT4_IN_LSB	The Root4 Accelerator Input LSBs. The MSBs are in Y memory

19.6.5.14 D_X_MATH_ABS_ROOT4_IN_LSB, X_MATH_ABS_ROOT4_IN_LSB

Bit Field	Read/Write	Field Name	Description
23:0	W	ABS_ROOT4_IN_LSB	The Root4 Accelerator Input LSBs. The MSBs are in Y memory

Ezairo 8300 Hardware Reference

19.6.5.15 D_X_MATH_DIVISOR, X_MATH_DIVISOR

Bit Field	Read/Write	Field Name	Description
23:0	W	DIVISOR_NOTRG	The Division accelerator Divisor Input. It does not trigger division calculation

19.6.5.16 D_X_MATH_DIV_OUT_LSB, X_MATH_DIV_OUT_LSB

Bit Field	Read/Write	Field Name	Description
23:0	R	DIV_OUT_LSB	The Division Accelerator Output LSBs. The MSBs are in Y memory

19.6.5.17 D_Y_MATH_DIV_STATUS, Y_MATH_DIV_STATUS

Bit Field	Read/Write	Field Name	Description
1	RW	DIV_ZERO	The Division Accelerator division by zero status
0	RW	DIV_SM	The Division Accelerator sign mode

Field Name	Value Symbol	Value Description	Hex Value
DIV_ZERO	DIV_BY_ZERO	There has been a division executed with a zero divisor	0x1
	DIV_BY_NONZERO	There have been divisions executed only with non-zero divisors	0x0*
DIV_SM	DIV_UNSIGNED	The accelerator inputs are unsigned numbers	0x1
	DIV_SIGNED	The accelerator inputs are signed numbers	0x0*

19.6.5.18 D_Y_MATH_DIVIDEND, Y_MATH_DIVIDEND

Bit Field	Read/Write	Field Name	Description
23:0	RW	DIVIDEND_NOTRG	The Division Accelerator Dividend Input. It does not trigger calculation

19.6.5.19 D_Y_MATH_LOG_EXP_STATUS, Y_MATH_LOG_EXP_STATUS

Bit Field	Read/Write	Field Name	Description
1	RW	EXP2_IN_POS_ERR	The Exp2 Accelerator input positive data entry status
0	RW	LOG2_IN_NEG_ERR	The Log2 Accelerator input negative data entry status

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
EXP2_IN_POS_ERR	EXP2_IN_POS	The accelerator noted positive data entry at its input	0x1
	EXP2_IN_NEG	The accelerator has not seen any positive data entry at its input	0x0*
LOG2_IN_NEG_ERR	LOG2_IN_NEG	The accelerator noted negative data entry at its input	0x1
	LOG2_IN_POS	The accelerator has not seen any negative data entry at its input	0x0*

19.6.5.20 D_Y_MATH_LOG2_OUT_MSB, Y_MATH_LOG2_OUT_MSB

Bit Field	Read/Write	Field Name	Description
23:0	RW	LOG2_OUT_MSB	The Log2 Accelerator Output MSBs. The LSBs are in X memory

19.6.5.21 D_Y_MATH_EXP2_OUT_MSB, Y_MATH_EXP2_OUT_MSB

Bit Field	Read/Write	Field Name	Description
23:0	RW	EXP2_OUT_MSB	The Exp2 Accelerator Output MSBs. The LSBs are in X memory

19.6.5.22 D_Y_MATH_ROOT_OUT_MSB, Y_MATH_ROOT_OUT_MSB

Bit Field	Read/Write	Field Name	Description
23:0	RW	ROOT_OUT_MSB	The Root Accelerator Output MSBs. The LSBs are in X memory

19.6.5.23 D_Y_MATH_LOG2_IN_MSB, Y_MATH_LOG2_IN_MSB

Bit Field	Read/Write	Field Name	Description
23:0	W	LOG2_IN_MSB	The Log2 Accelerator Input MSBs. The LSBs are in X memory

19.6.5.24 D_Y_MATH_ABS_LOG2_IN_MSB, Y_MATH_ABS_LOG2_IN_MSB

Bit Field	Read/Write	Field Name	Description
23:0	W	ABS_LOG2_IN_MSB	The Log2 Accelerator Input MSBs. The LSBs are in X memory

19.6.5.25 D_Y_MATH_EXP2_IN_MSB, Y_MATH_EXP2_IN_MSB

Bit Field	Read/Write	Field Name	Description
23:0	W	EXP2_IN_MSB	The Exp2 Accelerator Input MSBs. The LSBs are in X memory

Ezairo 8300 Hardware Reference

19.6.5.26 D_Y_MATH_ABSNEG_EXP2_IN_MSB, Y_MATH_ABSNEG_EXP2_IN_MSB

Bit Field	Read/Write	Field Name	Description
23:0	W	ABSNEG_EXP2_IN_MSB	The Exp2 Accelerator Input MSBs. The LSBs are in X memory

19.6.5.27 D_Y_MATH_ROOT2_IN_MSB, Y_MATH_ROOT2_IN_MSB

Bit Field	Read/Write	Field Name	Description
23:0	W	ROOT2_IN_MSB	The Root2 Accelerator Input MSBs. The LSBs are in X memory

19.6.5.28 D_Y_MATH_ABS_ROOT2_IN_MSB, Y_MATH_ABS_ROOT2_IN_MSB

Bit Field	Read/Write	Field Name	Description
23:0	W	ABS_ROOT2_IN_MSB	The Root2 Accelerator Input MSBs. The LSBs are in X memory

19.6.5.29 D_Y_MATH_ROOT3_IN_MSB, Y_MATH_ROOT3_IN_MSB

Bit Field	Read/Write	Field Name	Description
23:0	W	ROOT3_IN_MSB	The Root3 Accelerator Input MSBs. The LSBs are in X memory

19.6.5.30 D_Y_MATH_ROOT4_IN_MSB, Y_MATH_ROOT4_IN_MSB

Bit Field	Read/Write	Field Name	Description
23:0	W	ROOT4_IN_MSB	The Root4 Accelerator Input MSBs. The LSBs are in X memory

19.6.5.31 D_Y_MATH_ABS_ROOT4_IN_MSB, Y_MATH_ABS_ROOT4_IN_MSB

Bit Field	Read/Write	Field Name	Description
23:0	W	ABS_ROOT4_IN_MSB	The Root4 Accelerator Input MSBs. The LSBs are in X memory

19.6.5.32 D_Y_MATH_DIVIDEND_TRG, Y_MATH_DIVIDEND_TRG

Bit Field	Read/Write	Field Name	Description
23:0	W	DIVIDEND_TRG	The Division accelerator Divdend Input. It triggers calculation

19.6.5.33 D_Y_MATH_DIV_OUT_MSB, Y_MATH_DIV_OUT_MSB

Bit Field	Read/Write	Field Name	Description
23:0	R	DIV_OUT_MSB	The Root Accelerator Output MSBs. The LSBs are in X memory

19.7 REED-SOLOMON HARDWARE ACCELERATORS

19.7.1 Reed-Solomon Encoder

The Reed-Solomon encoder implements a [15,11] Reed-Solomon code, which creates encoded data blocks consisting of 15 four-bit symbols from 11 data symbols and 4 code symbols. Each data block contains 60 bits, of which 44 bits are data and 16 bits are code.

The encoder defines the code symbols using the input data passed through a set of four generator polynomials. Each is a separate Galois field polynomial, which calculates one code symbol when the 11 data symbols are shifted through the polynomial calculator. As shown in the timing diagram (the "Timing Diagram of Reed-Solomon Core Encoder" figure (Figure 55)), the Reed-Solomon encoder core receives one input symbol per clock cycle, and provides one output symbol on the following clock cycle. The effective delay through the encoder core itself is one system clock cycle. A new block of raw data is accepted only after the previous block has been encoded and read by the CFX DSP or Arm Cortex-M3 processor.

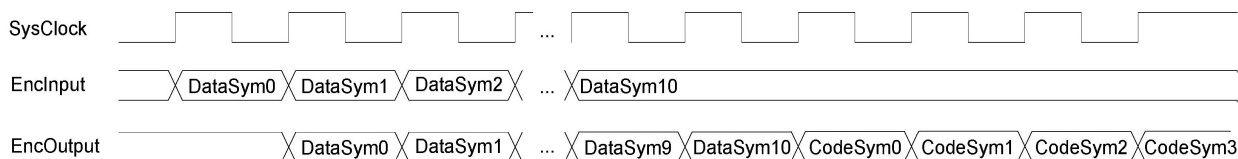


Figure 55. Timing Diagram of Reed-Solomon Core Encoder

The Reed-Solomon encoder block has a programmable counter, which can be set or read from the `RS_BLK_CNT_ENC_BLK_CNT` bit field in the `RS_BLK_CNT` register. It sets the number of Reed-Solomon blocks encoded in a block set, and is decremented as blocks are processed. When this counter reaches zero, the Reed-Solomon encoder is disabled. The countdown considers a block complete only when all of the 15 encoded data symbols are sent to the output interface. When the counter reaches zero, an `RS_COM` interrupt is generated indicating that an `ENC_BLK_END` event has occurred. If operating in DMA mode, the Reed-Solomon encoder input interface does not generate any new requests until the block counter is updated with a non-zero value.

IMPORTANT: To enable the Reed-Solomon encoder to continuously encode a stream of data blocks, set the block count to 0xFF. With this configuration, the Reed-Solomon block counter is not decremented when blocks are encoded.

Input and output interfaces are included with the Reed-Solomon encoder to support smooth data transfers between the CFX DSP or the Arm Cortex-M3 processor and the encoder.

The input interface to the encoder uses the `RS_ENC_IN` register. The input interface uses a circular pointer, advancing four bits at a time, to split the input register into symbols. The input interface block consists of the least significant 1, 2, 4 or 8 four-bit symbols (4, 8, 16 or 32 bits) from the input interface register. To select how many symbols are used as inputs to the Reed-Solomon encoder for each write to the `RS_ENC_IN` register, configure the `RS_CFG_ENC_IN_PTR_MODE` bit field in the `RS_CFG` register. The register content is transferred to the encoder one symbol at a time, using the interface pointer. This pointer can be configured to operate in increment mode (`RS_ENC_IN_INC`) or decrement mode (`RS_ENC_IN_DEC`) using the `RS_CFG_ENC_IN_PTR_DIR` bit in the `RS_CFG` register.

- In increment mode, transfers are from the symbol in the least significant position to the symbol in the most significant position.

Ezairo 8300 Hardware Reference

- In decrement mode, transfers are from the symbol in the most significant position to the symbol in the least significant position.

The "Input/Output Pointer Modes" figure (Figure 56) shows the order in which the pointer selects symbols to be used by the encoder from the input interface for each input pointer mode.

The current pointer value can be read from the RS_PTR_ENC_IN_PTR bit field in the RS_PTR register at any time, but cannot be overwritten.

	32-bit, Inc	16-bit, Inc	8-bit, Inc	4-bit	32-bit, Dec	16-bit, Dec	8-bit, Dec
Bit 28	Symbol 7				Symbol 0		
	Symbol 6				Symbol 1		
	Symbol 5				Symbol 2		
	Symbol 4				Symbol 3		
	Symbol 3	Symbol 3			Symbol 4	Symbol 0	
	Symbol 2	Symbol 2			Symbol 5	Symbol 1	
	Symbol 1	Symbol 1			Symbol 6	Symbol 2	Symbol 0
Bit 0	Symbol 0	Symbol 0	Symbol 0	Symbol 0	Symbol 7	Symbol 3	Symbol 1

Figure 56. Input/Output Pointer Modes

When the input interface pointer is at the zero address and no update on the RS_ENC_IN register is detected, an RS_ENC_IN_RDY interrupt indicates that the register is empty and ready to be refilled. The input interface does not transfer any new symbols to the RS encoder core until the interface register is refilled. If the input interface register is overwritten before all encoded symbols are read out, an RS_COM interrupt is generated, indicating that an ENC_IN_ERR event has occurred.

The output interface from the encoder uses the RS_ENC_OUT register. The output interface also uses a circular pointer, advancing four bits at a time, to split the output register into symbols in a separately configurable way from the input. Similar to the input interface pointer, the output interface pointer:

- Can be configured for 4-, 8-, 16-, or 32-bit (1, 2, 4, or 8 symbol) mode using the RS_CFG_ENC_OUT_PTR_MODE bit field in the RS_CFG register
- Can be configured for increment or decrement mode using the RS_CFG_ENC_OUT_PTR_DIR bit in the RS_CFG register
- Can be read (but not written) using the RS_PTR_ENC_OUT_PTR bit field in the RS_PTR register
- The "Input/Output Pointer Modes" figure (Figure 56) shows the order in which the pointer places symbols for generation by the encoder in the output interface for all pointer modes

The output register RS_ENC_OUT is considered to be full when the next output pointer value is zero. This generates an RS_ENC_OUT_RDY interrupt, indicating that the output register is ready to be read. If the output register is read before the pointer wraps back to zero, the error interrupt is not set, to allow for partial output interface reads. A partial read can be necessary to complete the transfer of the last symbols of a set of encoded data blocks that do not form a multiple of the word size.

CAUTION: Partial reads must only be used when the block counter is zero and leftover symbols are in the output register. If the output register is read when the encoder is still processing data, a symbol can be missed due to a race condition.

NOTE: The RS encoder does not have any internal data buffer. Therefore, to read any new symbol, it has to make sure that the output interface is ready to capture a new symbol. The RS encoder core provides an acknowledgment to the input interface, indicating that the new symbol is being read. This acknowledgment increments the input pointer. The RS encoder also provides a similar signal to the output interface so that the new symbol is captured and the output pointer is incremented.

To reset the Reed-Solomon encoder's internal state machines, set the `RS_CTRL_ENC_CLR` bit field in the `RS_CTRL` register. The encoder can be reset even when disabled.

19.7.1.1 Using the Reed-Solomon Encoder with Interrupts

To use the Reed-Solomon encoder with interrupts, enable the `RS_ENC_IN_RDY` and `RS_ENC_OUT_RDY` interrupts.

Once the encoder input and output pointers are configured, encode one set of blocks with the following steps:

1. Reset the Reed-Solomon encoder.
2. Enable the encoder by setting the block count to the desired number of blocks.

When interrupts are raised, perform the appropriate action:

- When an `RS_ENC_IN_RDY` interrupt is raised, write one word to the `RS_ENC_IN` register. The number of symbols written need to match the input pointer mode because additional symbols are ignored.
- When an `RS_ENC_OUT_RDY` interrupt is raised, read the `RS_STATUS` register to check the `RS_STATUS_ENC_BLK_END` status bit, which indicates whether the values available in the `RS_ENC_OUT` register are the last symbols of the block.
 - If the `RS_STATUS_ENC_BLK_END` status bit is set, clear it, then read the last set of symbols from the `RS_ENC_OUT` register. To verify how many symbols are valid in the data read from the `RS_ENC_OUT` register, read the `RS_PTR_ENC_OUT_PTR` bit field in the `RS_PTR` register, which indicates where the next symbol is expected to be written.
 - If the `RS_STATUS_ENC_BLK_END` status bit is not set, read one word from the `RS_ENC_OUT` register. This register contains the number of symbols specified by the output pointer mode.

For information on configuring the NVIC, see [Section 4.6.1 “Nested Vectored Interrupt Controller \(NVIC\)” on page 135](#).

19.7.1.2 Using the Reed-Solomon Encoder with the DMA Controller

To use the Reed-Solomon encoder with the DMA controller for input or output, disable the appropriate `RS_ENC_IN_RDY` or `RS_ENC_OUT_RDY` interrupts. Instead of using those interrupts, enable the `RS_COM` interrupt, and use the `RS_INT_ENABLE` register to enable the `ENC_IN_ERR` and `ENC_BLK_END` interrupt triggers for the `RS_COM` interrupt.

To encode one set of blocks, configure the transfer input pointer, output pointer, internal state, and block count as required for interrupts (described in [Section 19.7.1.1 “Using the Reed-Solomon Encoder with Interrupts”](#)). Then configure a DMA channel to handle the input or output as desired, ensuring that:

Ezairo 8300 Hardware Reference

- If a DMA channel is used to control Reed-Solomon encoder inputs:
 - The RS_ENC_IN register address is set as the destination address.
 - The Reed-Solomon encoder is selected as the destination for the DMA channel.
- If a DMA channel is used to control Reed-Solomon encoder outputs:
 - The RS_ENC_OUT register address is set as the source address.
 - The Reed-Solomon encoder is selected as the source for the DMA channel.
- The word size for the DMA transfer matches the input and/or output pointer configuration.
- The transfer length is configured to match the number of words that must be transferred to complete the block set.

Once the DMA channel is configured, wait for the RS_COM interrupt to indicate the ENC_BLK_END event, which signifies that the DMA transfer has been completed and that the remaining encoded symbols are available in the RS_ENC_OUT register. Clear the RS_STATUS_ENC_BLK_END status bit in the RS_STATUS register, then read the last set of symbols from the RS_ENC_OUT register.

To verify how many symbols are valid in the data read from the RS_ENC_OUT register, read the RS_PTR_ENC_PTR bit field in the RS_PTR register, which indicates where the next symbol is expected to be written.

If an RS_COM interrupt indicates that an ENC_IN_ERR event has occurred, the Arm Cortex-M3 processor or CFX DSP must manually handle this event.

For more information about the DMA, see [Section 19.4 “Direct Memory Access \(DMA\) Controller” on page 693](#).

19.7.2 Reed-Solomon Decoder

The Reed-Solomon decoder implements a [15,11] Reed-Solomon code, which creates decoded data blocks consisting of 15 four-bit symbols from 11 data symbols and 4 code symbols. Each data block contains 60 bits, of which 44 bits are data and 16 bits are code. The decoder block performs forward error correction, using data that has been previously encoded (including by a block such as the Reed-Solomon encoder described in [Section 19.7.1 “Reed-Solomon Encoder” on page 750](#)).

This Reed-Solomon decoder can detect and correct up to two error-containing symbols per block of encoded data. The decoding algorithm operation output is undefined in the presence of more than two symbol errors. This means that the decoder can flag severely corrupted data as correctable, even though this algorithm can only correct up to two symbol errors.

The Reed-Solomon decoder block has a programmable counter that can be set or read from the RS_BLK_CNT_DEC_BLK_CNT bit field in the RS_BLK_CNT register. This counter sets the number of Reed-Solomon blocks decoded in a block set, and is decremented. When this counter reaches zero, the Reed-Solomon decoder is disabled. The countdown only considers a block complete when all of the 11 data symbols are sent to the output interface. When the counter reaches zero, an RS_COM interrupt is generated, indicating that a DEC_BLK_END event has occurred. If operating in DMA mode, the Reed-Solomon decoder input interface does not generate any new requests until the block counter is updated with a non-zero value.

IMPORTANT: To enable the Reed-Solomon decoder to continuously decode a stream of data blocks, set the block count to 0xFF. If configured in this way, the Reed-Solomon block counter is not decremented when blocks are decoded.

Ezairo 8300 Hardware Reference

NOTE: If the Reed-Solomon decoder's counter is cleared while processing is ongoing, the decoder completes the current block's processing and transfers the output data to the output interface before being deactivated.

The Reed-Solomon decoder contains an internal buffer that holds one encoded Reed-Solomon block. The decoder starts the error detection and correction operation automatically once the block of data to be decoded has been loaded into this decoder. The Reed-Solomon decoder has a variable latency, as listed in the "[Latencies in Reed-Solomon Decoder](#)" table (Table 74), below. This latency is measured from the time that the last symbol to be decoded is copied to the buffer, to the time when the error correction is finished. Therefore, the latency information does not take into account the transfer of the symbols between the decoder core and the input and output interfaces.

Table 74. Latencies in Reed-Solomon Decoder

Number of Symbols with Errors	Latency (system clock cycle)
0	7
1	64
2	79
Uncorrectable	65

Input and output interfaces are included with the Reed-Solomon decoder to support smooth data transfers between the Arm Cortex-M3 processor or the CFX DSP and the decoder.

The input interface from the decoder uses the RS_DEC_IN register. The input interface also uses a circular pointer, advancing four bits at a time, to split the input register into symbols. Similar to the Reed-Solomon encoder input interface pointer, the decoder's input interface pointer:

- Can be configured for 4-, 8-, 16-, or 32-bit (1, 2, 4, or 8 symbol) mode using the RS_CFG_DEC_IN_PTR_MODE bit field of the RS_CFG register
- Can be configured for increment or decrement mode using the RS_CFG_DEC_IN_PTR_DIR bit in the RS_CFG register
- Can be read (but not written) using the RS_PTR_DEC_IN_PTR bit field in the RS_PTR register
- [Figure 56 on page 751](#) shows the order in which the pointer selects symbols to be used by the decoder from the input interface for each input pointer mode.

Similarly, the output interface from the encoder uses the RS_DEC_OUT register. The output interface also uses a circular pointer, advancing four bits at a time, to split the output register into symbols in a way separately configurable from that of the input. Similar to the input interface pointer, the output interface pointer:

- Can be configured for 4-, 8-, 16-, or 32-bit (1, 2, 4, or 8 symbol) mode using the RS_CFG_DEC_OUT_PTR_MODE bit field of the RS_CFG register
- Can be configured for increment or decrement mode using the RS_CFG_DEC_OUT_PTR_DIR bit in the RS_CFG register
- Can be read (but not written) using the RS_PTR_DEC_OUT_PTR bit field in the RS_PTR register
- The "[Input/Output Pointer Modes](#)" figure ([Figure 56](#)) shows the order in which the pointer places symbols for generation by the decoder in the output interface for all pointer modes.

Ezairo 8300 Hardware Reference

The input and output interface blocks for the Reed-Solomon decoder cannot operate simultaneously. The input interface is active when starting up, and the output interface is not active until the internal buffer is filled. When the buffer is full, the input interface idles and waits until the error correction is completed. When the error correction is finished, the output interface is active until it has emptied the buffer.

When the input interface pointer is at the zero address and more data is needed to fill the internal buffer, an `RS_DEC_IN_RDY` interrupt indicates that the register is empty and ready to be refilled. The input interface does not transfer any new symbols to the RS encoder core until the interface register is refilled. If the input interface register is overwritten before all encoded symbols are copied to the internal buffer, an `RS_COM` interrupt is generated, indicating that a `DEC_IN_ERR` event has occurred.

The output register `RS_DEC_OUT` is considered to be full when the next output pointer value is zero. This generates an `RS_ENC_OUT_RDY` interrupt, indicating that the output register is ready to be read. If the output register is read before the pointer wraps back to zero, the error interrupt is not set, to allow for partial output interface reads. A partial read can be necessary to complete the transfer of the last symbols of a set of encoded data blocks that do not form a multiple of the word size.

CAUTION: Partial reads must only be used when the block counter is zero and leftover symbols are in the output register. If the output register is read when the encoder is still processing data, a symbol can be missed due to a race condition.

The `RS_STATUS` register contains two bit fields that are used to report the number of errors that have been corrected by the Reed-Solomon decoder. These bit fields are:

- `RS_STATUS_DEC_BLK_ERR`, which indicates the number of symbols that have been corrected in the most recently completed block
- `RS_STATUS_DEC_ACC_ERR`, which indicates the number of symbols that have been corrected since the most recent decoder reset, or the last set of the block counter.

If the Reed-Solomon decoder detects that a symbol error cannot be corrected, the decoder generates an `RS_COM` interrupt, indicating that a `DEC_MISS_ERR` event has occurred.

To reset the Reed-Solomon decoder's internal state machines, set the `RS_CTRL_DEC_CLR` bit field in the `RS_CTRL` register. The decoder can be reset even when disabled.

19.7.2.1 Using the Reed-Solomon Decoder with Interrupts

To use the Reed-Solomon decoder with interrupts, use the NVIC to enable the `RS_DEC_IN_RDY` and `RS_DEC_OUT_RDY` interrupts.

Once the decoder input and output pointers are configured, decode one set of blocks with the following steps:

1. Reset the Reed-Solomon decoder.
2. Enable the decoder by setting the block count to the desired number of blocks.

When interrupts are raised, perform the appropriate action:

- When an `RS_DEC_IN_RDY` interrupt is raised, write one word to the `RS_DEC_IN` register. The number of symbols written needs to match the input pointer mode (additional symbols are ignored).
- When an `RS_DEC_OUT_RDY` interrupt is raised, read the `RS_STATUS` register to check the `RS_STATUS_DEC_BLK_END` status bit, which indicates whether the values available in the `RS_DEC_OUT` register are the last symbols of the block.

Ezairo 8300 Hardware Reference

- If the `DEC_BLK_END` status bit is set, clear it, then read the last set of symbols from the `RS_DEC_OUT` register.
To verify how many symbols are valid in the data read from the `RS_DEC_OUT` register, read the `RS_PTR_DEC_OUT_PTR` bit field in the `RS_PTR` register, which indicates where the next symbol is expected to be written.
- If the `RS_STATUS_DEC_BLK_END` status bit is not set, read one word from the `RS_DEC_OUT` register.
This register contains the number of symbols specified by the output pointer mode.

For every `RS_DEC_OUT_RDY` interrupt handled, the user application can read the `RS_STATUS` register. This register supports providing an application with information about how many errors were detected in the last encoded data block, and how many errors have been detected since the beginning of the decoding process.

For information on configuring the NVIC, see [Section 4.6.1 “Nested Vectored Interrupt Controller \(NVIC\)” on page 135](#).

19.7.2.2 Using the Reed-Solomon Decoder with the DMA Controller

To use the Reed-Solomon decoder with the DMA controller for input or output, disable the appropriate `RS_DEC_IN_RDY` or `RS_DEC_OUT_RDY` interrupts. Instead of using those interrupts, enable the `RS_COM` interrupt, and use the `RS_INT_ENABLE` register to enable the `DEC_IN_ERR`, `DEC_BLK_END`, and `DEC_MISS_ERR` interrupt triggers for the `RS_COM` interrupt.

To decode one set of blocks, configure the transfer input pointer, output pointer, internal state, and block count, as required for interrupts (described in [Section 19.7.2.1 “Using the Reed-Solomon Decoder with Interrupts” on page 755](#)). Then configure a DMA channel to handle the input or output as desired, ensuring that:

- If a DMA channel is used to control Reed-Solomon decoder inputs:
 - The `RS_DEC_IN` register address is set as the destination address.
 - The Reed-Solomon decoder is selected as the destination for the DMA channel.
- If a DMA channel is used to control Reed-Solomon decoder outputs:
 - The `RS_DEC_OUT` register address is set as the source address.
 - The Reed-Solomon decoder is selected as the source for the DMA channel.
- The word size for the DMA transfer matches the input and/or output pointer configuration.
- The transfer length is configured to match the number of words that must be transferred to complete the block set.

Once the DMA channel is configured, wait for the `RS_COM` interrupt:

- The `DEC_BLK_END` event indicates that the DMA transfer is complete and that the remaining decoded symbols are available in the `RS_DEC_OUT` register.
To handle this event, clear the `RS_STATUS_DEC_BLK_END` status bit, then read the last set of symbols from the `RS_DEC_OUT` register. To verify how many symbols are valid in the data read from the `RS_DEC_OUT` register, read the `RS_PTR_DEC_OUT_PTR` bit field in the `RS_PTR` register, which indicates where the next symbol is expected to be written.
- The `DEC_IN_ERR` and `DEC_MISS_ERR` events indicate the occurrence of an error that cannot be directly handled by the DMA or the Reed-Solomon decoder. The CFX DSP or Arm Cortex-M3 processor must handle these errors manually.

onsemi Confidential
Ezairo 8300 Hardware Reference

For more information about the DMA, see [Section 19.4 “Direct Memory Access \(DMA\) Controller”](#) on page 693.

19.7.3 Reed-Solomon Configuration and Control Registers

Register Name	Register Description	Address
RS_STATUS	Reed Solomon Interrupt Status Register	P:0x1200 0x40000800
RS_INT_ENABLE	Reed Solomon Interrupt Mask Register	P:0x1201 0x40000804
RS_CFG	Reed Solomon Configuration Register	P:0x1202 0x40000808
RS_BLK_CNT	Reed Solomon Block Counter Values	P:0x1203 0x4000080C
RS_CTRL	Reed Solomon Control Register	P:0x1204 0x40000810
RS_ENC_IN	Reed Solomon Encoder Input Data	P:0x1205 0x40000814
RS_ENC_OUT	Reed Solomon Encoder Output Data	P:0x1206 0x40000818
RS_DEC_IN	Reed Solomon Decoder Input Data	P:0x1207 0x4000081C
RS_DEC_OUT	Reed Solomon Decoder Output Data	P:0x1208 0x40000820
RS_PTR	Reed Solomon Pointer Values	P:0x1209 0x40000824

19.7.3.1 RS_STATUS

Bit Field	Read/Write	Field Name	Description
25:24	R	DEC_BLK_ERR	Reed Solomon decoder error count in the last block
21:16	R	DEC_ACC_ERR	Reed Solomon decoder accumulated error count
12	R	DEC_MISS_ERR	Reed Solomon decoder missed error interrupt status
11	R	DEC_BLK_END	Reed Solomon decoder block end interrupt status

Ezairo 8300 Hardware Reference

Bit Field	Read/Write	Field Name	Description
10	R	DEC_IN_ERR	Reed Solomon decoder input interface error interrupt status
9	R	ENC_BLK_END	Reed Solomon encoder block end interrupt status
8	R	ENC_IN_ERR	Reed Solomon encoder input interface error interrupt status
4	W	DEC_MISS_ERR_CLEAR	Reed Solomon decoder missed error interrupt clear
3	W	DEC_BLK_END_CLEAR	Reed Solomon decoder block end interrupt clear
2	W	DEC_IN_ERR_CLEAR	Reed Solomon decoder output interface error interrupt clear
1	W	ENC_BLK_END_CLEAR	Reed Solomon encoder block end interrupt clear
0	W	ENC_IN_ERR_CLEAR	Reed Solomon encoder input interface error interrupt clear

Field Name	Value Symbol	Value Description	Hex Value
DEC_MISS_ERR	RS_DEC_MISS_ERR_INT_IDLE	This source has not set an interrupt	0x0*
	RS_DEC_MISS_ERR_INT_PENDING	Reed Solomon decoder could not fix an error in the block	0x1
DEC_BLK_END	RS_DEC_BLK_END_INT_IDLE	This source has not set an interrupt	0x0*
	RS_DEC_BLK_END_INT_PENDING	Reed Solomon decoder block counter counted down to zero	0x1
DEC_IN_ERR	RS_DEC_IN_ERR_INT_IDLE	This source has not set an interrupt	0x0*
	RS_DEC_IN_ERR_INT_PENDING	Reed Solomon decoder input register was overwritten before the input registers were read.	0x1
ENC_BLK_END	RS_ENC_BLK_END_INT_IDLE	This source has not set an interrupt	0x0*
	RS_ENC_BLK_END_INT_PENDING	Reed Solomon encoder block counter counted down to zero	0x1
ENC_IN_ERR	RS_ENC_IN_ERR_INT_IDLE	This source has not set an interrupt	0x0*
	RS_ENC_IN_ERR_INT_PENDING	Reed Solomon encoder input register was overwritten before the output registers were read.	0x1
DEC_MISS_ERR_CLEAR	RS_DEC_MISS_ERR_INT_CLEAR	Clear the Reed Solomon decoder missed error interrupt	0x1
DEC_BLK_END_CLEAR	RS_DEC_BLK_END_INT_CLEAR	Clear the Reed Solomon decoder block end interrupt	0x1

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DEC_IN_ERR_CLEAR	RS_DEC_IN_ERR_INT_CLEAR	Clear the Reed Solomon decoder output interface error	0x1
ENC_BLK_END_CLEAR	RS_ENC_BLK_END_INT_CLEAR	Clear the Reed Solomon encoder block end interrupt	0x1
ENC_IN_ERR_CLEAR	RS_ENC_IN_ERR_INT_CLEAR	Clear the Reed Solomon encoder input interface error interrupt	0x1

19.7.3.2 RS_INT_ENABLE

Bit Field	Read/Write	Field Name	Description
4	RW	DEC_MISS_ERR	The Reed Solomon decoder missed error interrupt mask
3	RW	DEC_BLK_END	The Reed Solomon decoder block end interrupt mask
2	RW	DEC_IN_ERR	The Reed Solomon decoder input interface error interrupt mask
1	RW	ENC_BLK_END	The Reed Solomon encoder block end interrupt mask
0	RW	ENC_IN_ERR	The Reed Solomon Encoder input interface error interrupt mask

Field Name	Value Symbol	Value Description	Hex Value
DEC_MISS_ERR	RS_DEC_MISS_ERR_INT_DISABLE	The Reed Solomon decoder missed error can not set an interrupt	0x0*
	RS_DEC_MISS_ERR_INT_ENABLE	The Reed Solomon decoder missed error can set the RS_COM interrupt line	0x1
DEC_BLK_END	RS_DEC_BLK_END_INT_DISABLE	The Reed Solomon decoder block end can not set an interrupt	0x0*
	RS_DEC_BLK_END_INT_ENABLE	The Reed Solomon decoder block end can set the RS_COM interrupt line	0x1
DEC_IN_ERR	RS_DEC_IN_ERR_INT_DISABLE	The Reed Solomon decoder input interface error can not set an interrupt	0x0*
	RS_DEC_IN_ERR_INT_ENABLE	The Reed Solomon decoder input interface error can set the RS_COM interrupt line	0x1
ENC_BLK_END	RS_ENC_BLK_END_INT_DISABLE	The Reed Solomon encoder block end can not set an interrupt	0x0*

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
	RS_ENC_BLK_END_INT_ENABLE	The Reed Solomon encoder block end can set the RS_COM interrupt line	0x1
ENC_IN_ERR	RS_ENC_IN_ERR_INT_DISABLE	The Reed Solomon Encoder input interface error can not set an interrupt	0x0*
	RS_ENC_IN_ERR_INT_ENABLE	The Reed Solomon Encoder input interface error can set the RS_COM interrupt line	0x1

19.7.3.3 RS_CFG

Bit Field	Read/Write	Field Name	Description
13	RW	DEC_IN_PTR_DIR	Reed Solomon Decoder Input Register Pointer Update Direction
12	RW	DEC_OUT_PTR_DIR	Reed Solomon Decoder Output Register Pointer Update Direction
11:10	RW	DEC_IN_PTR_MODE	Reed Solomon Decoder Input Register Pointer Mode
9:8	RW	DEC_OUT_PTR_MODE	Reed Solomon Decoder Output Register Pointer Mode
5	RW	ENC_IN_PTR_DIR	Reed Solomon Encoder Input Register Pointer Update Direction
4	RW	ENC_OUT_PTR_DIR	Reed Solomon Encoder Output Register Pointer Update Direction
3:2	RW	ENC_IN_PTR_MODE	Reed Solomon Encoder Input Register Pointer Mode
1:0	RW	ENC_OUT_PTR_MODE	Reed Solomon Encoder Output Register Pointer Mode

Field Name	Value Symbol	Value Description	Hex Value
DEC_IN_PTR_DIR	RS_DEC_IN_INC	Decoder input pointer increments by 1 RS symbol	0x0*
	RS_DEC_IN_DEC	Decoder input pointer decrements by 1 RS symbol	0x1
DEC_OUT_PTR_DIR	RS_DEC_OUT_INC	Decoder Output pointer increments by 1 RS symbol	0x0*
	RS_DEC_OUT_DEC	Decoder Output pointer decrements by 1 RS symbol	0x1
DEC_IN_PTR_MODE	RS_DEC_IN_32BIT	Input data uses 32-bits per word	0x0*
	RS_DEC_IN_16BIT	Input data uses 16-bits per word (LSB aligned)	0x1
	RS_DEC_IN_8BIT	Input data uses 8-bits per word (LSB aligned)	0x2
	RS_DEC_IN_4BIT	Input data uses 4-bits per word (LSB aligned)	0x3

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
DEC_OUT_PTR_MODE	RS_DEC_OUT_32BIT	Output data uses 32-bits per word	0x0*
	RS_DEC_OUT_16BIT	Output data uses 16-bits per word (LSB aligned)	0x1
	RS_DEC_OUT_8BIT	Output data uses 8-bits per word (LSB aligned)	0x2
	RS_DEC_OUT_4BIT	Output data uses 4-bits per word (LSB aligned)	0x3
ENC_IN_PTR_DIR	RS_ENC_IN_INC	Encoder input pointer increments by 1 RS symbol	0x0*
	RS_ENC_IN_DEC	Encoder input pointer decrements by 1 RS symbol	0x1
ENC_OUT_PTR_DIR	RS_ENC_OUT_INC	Encoder output pointer increments by 1 RS symbol	0x0*
	RS_ENC_OUT_DEC	Encoder output pointer decrements by 1 RS symbol	0x1
ENC_IN_PTR_MODE	RS_ENC_IN_32BIT	Input data uses 32-bits per word	0x0*
	RS_ENC_IN_16BIT	Input data uses 16-bits per word (LSB aligned)	0x1
	RS_ENC_IN_8BIT	Input data uses 8-bits per word (LSB aligned)	0x2
	RS_ENC_IN_4BIT	Input data uses 4-bits per word (LSB aligned)	0x3
ENC_OUT_PTR_MODE	RS_ENC_OUT_32BIT	Output data uses 32-bits per word	0x0*
	RS_ENC_OUT_16BIT	Output data uses 16-bits per word (LSB aligned)	0x1
	RS_ENC_OUT_8BIT	Output data uses 8-bits per word (LSB aligned)	0x2
	RS_ENC_OUT_4BIT	Output data uses 4-bits per word (LSB aligned)	0x3

19.7.3.4 RS_BLK_CNT

Bit Field	Read/Write	Field Name	Description
15:8	RW	DEC_BLK_CNT	Reed Solomon Decoder block size to count down from (255 is continuous run)
7:0	RW	ENC_BLK_CNT	Reed Solomon Encoder block size to count down from (255 is continuous run)

19.7.3.5 RS_CTRL

Bit Field	Read/Write	Field Name	Description
1	W	DEC_CLR	Reed Solomon Decoder Clear
0	W	ENC_CLR	Reed Solomon Encoder Clear

Field Name	Value Symbol	Value Description	Hex Value
DEC_CLR	RS_DEC_CLEAR	Reset the RS decoder state	0x1
ENC_CLR	RS_ENC_CLEAR	Reset the RS encoder state	0x1

19.7.3.6 RS_ENC_IN

Bit Field	Read/Write	Field Name	Description
31:0	RW	ENC_IN	Reed Solomon encoder input register

19.7.3.7 RS_ENC_OUT

Bit Field	Read/Write	Field Name	Description
31:0	RW	ENC_OUT	Reed Solomon encoder output register

19.7.3.8 RS_DEC_IN

Bit Field	Read/Write	Field Name	Description
31:0	RW	DEC_IN	Reed Solomon decoder input register

19.7.3.9 RS_DEC_OUT

Bit Field	Read/Write	Field Name	Description
31:0	RW	DEC_OUT	Reed Solomon decoder output register

19.7.3.10 RS_PTR

Bit Field	Read/Write	Field Name	Description
13:11	RW	DEC_IN_PTR	Reed Solomon decoder input pointer value
10:8	RW	DEC_OUT_PTR	Reed Solomon decoder output pointer value
5:3	RW	ENC_IN_PTR	Reed Solomon encoder input pointer value
2:0	RW	ENC_OUT_PTR	Reed Solomon encoder output pointer value

19.8 TIMERS

The Ezairo 8300 system provides five timers, including:

- The SysTick timer from the Arm Cortex-M3 processor, which is described in [Section 4.6.2 “SysTick” on page 196](#)
- Four general-purpose timers

Each general-purpose timer provides:

- A 24-bit counter
- A 3-bit prescale factor that increases by a factor of 2 at each step, scaling between a prescaler of 1 and 128.
- Three operating modes: single-shot/multiple-shot, free-run, and DIO interrupt capture
- A dedicated interrupt that can be used to signal timer expiration
- Dedicated configuration and status registers

The general-purpose timers are clocked from `SLOWCLK`, divided by 32. The available power of 2 prescaling divisions results in the timers achieving a wider range of timing; however, the granularity at which the timer can be configured to trigger increases in parallel. Use the `TIMER_CFG0_PRESCALE` bit field to set the power of two scaling factor.

Throughout this section, the asterisk * indicates that any of the 4 timer channels, numbered 0-3, can be substituted.

After prescaling the timer clock, each timer can be configured to trigger after 1 to 2^{24} cycles of the prescaled clock, by setting the `TIMER_CFG0_TIMEOUT_VALUE` bit field in the appropriate `TIMER*_CFG0` register. The resulting timer delay is equal to:

$$\text{DELAY} = \frac{2^{(\text{TIMER_CFG0_PRESCALE})} \times (\text{TIMER_CFG0_TIMEOUT_VALUE} + 1)}{f_{\text{SLOWCLK_DIV32}}}$$

19.8.1 Starting or Stopping Timers

The state of a timer can be read from the `TIMER_CTRL_BUSY` bit of its `TIMER_CTRL` register.

If the timer is not running, it can be started by setting the `TIMER_CTRL_START` bit of its `TIMER_CTRL` register. If the timer is running, setting this same bit restarts the timer by reloading the timer value.

Each timer can be stopped at any time, by setting the `TIMER_CTRL_STOP` bit of its `TIMER_CTRL` register.

In a case where the `START` and `STOP` bits are set at the same time, the `STOP` bit takes precedence over the `START` bit and the timer is not started.

19.8.2 Mode Selection

The timers support selection between free-run and single-/multi-shot mode, using the `TIMER_CFG1_MODE` bit of the `TIMER_CFG1` register.

- In Free-Run Mode, the timer loads the initial time-out value and counts down to 0. When it reaches 0, it issues an interrupt, reloads the time-out value, and restarts the countdown timer. The process is repeated indefinitely until the timer is explicitly stopped by writing a 1 to the `TIMER_CTRL_STOP` bit in the `TIMER_CTRL` register.

Ezairo 8300 Hardware Reference

- In Multi-Shot Mode, the timer loads the initial time-out value and counts down to 0. When it reaches 0, it issues an interrupt, and checks the `TIMER_CFG1_MULTI_COUNT` bit field in the `TIMER_CFG1` register to determine if the countdown timer must be restarted. This process repeats $(\text{TIMER_CFG1_MULTI_COUNT} + 1)$ times before the timer is disabled, unless explicitly stopped by the `TIMER_STOP` bit. Single-shot mode is a special case of Multi-Shot Mode, where the timer is configured to trigger an interrupt only one time.

19.8.3 GPIO Interrupt Capture Mode

The timers can be used to indicate when a GPIO interrupt occurs. If the timer is configured to monitor GPIO interrupts, and a GPIO interrupt is triggered, the timer stores the current timer value to the `TIMER_VAL_DIO` register to indicate when the interrupt has occurred. To enable this mode, set the `TIMER_CFG1_DIO_INT_ENABLE` bit in the `TIMER_CFG1` register. To select the GPIO interrupt source used as the trigger for the timer, use the `TIMER_CFG1_DIO_INT_SRC` bit-field in the `TIMER_CFG1` register.

DIO interrupt mode can be configured to operate in single or continuous mode, using the `TIMER_CFG1_DIO_INT_MODE` bit in the `TIMER_CFG1` register.

- When operating in single mode, the timer captures the timing of one DIO interrupt. The DIO interrupt capture mechanism is automatically disabled once the DIO interrupt is generated. Setting the `TIMER_CFG1_DIO_INT_ENABLE` bit high again schedules another single shot capture.
- When operating in continuous mode, the timer constantly runs and updates the `TIMER_VAL_DIO` register when a DIO interrupt is triggered. The `TIMER_CFG1_DIO_INT_ENABLE` bit holds its value in this mode and must be cleared to disable this mode.

When the DIO interrupt based capture is enabled, but the timer is off, the state of the `TIMER_VAL_DIO` register is preserved.

For more information on configuring GPIO interrupts, see [Section 18.1.1 “GPIO Interrupts” on page 539](#).

19.8.4 Timer Registers

Register Name	Register Description	Address
<code>TIMER0_CFG0</code>	Timer Configuration 0 Register	P:0x1040 0x40000100
<code>TIMER0_CFG1</code>	Timer Configuration 1 Register	P:0x1041 0x40000104
<code>TIMER0_CTRL</code>	General-Purpose timer Control / Status Register	P:0x1042 0x40000108
<code>TIMER0_VAL</code>	Timer Current Value Register	P:0x1043 0x4000010C
<code>TIMER0_VAL_CAPTURE</code>	Timer DIO Interrupt Captured Value Register	P:0x1044 0x40000110

onsemi Confidential
Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
TIMER1_CFG0	Timer Configuration 0 Register	P:0x1048 0x40000120
TIMER1_CFG1	Timer Configuration 1 Register	P:0x1049 0x40000124
TIMER1_CTRL	General-Purpose timer Control / Status Register	P:0x104A 0x40000128
TIMER1_VAL	Timer Current Value Register	P:0x104B 0x4000012C
TIMER1_VAL_CAPTURE	Timer DIO Interrupt Captured Value Register	P:0x104C 0x40000130
TIMER2_CFG0	Timer Configuration 0 Register	P:0x1050 0x40000140
TIMER2_CFG1	Timer Configuration 1 Register	P:0x1051 0x40000144
TIMER2_CTRL	General-Purpose timer Control / Status Register	P:0x1052 0x40000148
TIMER2_VAL	Timer Current Value Register	P:0x1053 0x4000014C
TIMER2_VAL_CAPTURE	Timer DIO Interrupt Captured Value Register	P:0x1054 0x40000150
TIMER3_CFG0	Timer Configuration 0 Register	P:0x1058 0x40000160
TIMER3_CFG1	Timer Configuration 1 Register	P:0x1059 0x40000164

Ezairo 8300 Hardware Reference

Register Name	Register Description	Address
TIMER3_CTRL	General-Purpose timer Control / Status Register	P:0x105A 0x40000168
TIMER3_VAL	Timer Current Value Register	P:0x105B 0x4000016C
TIMER3_VAL_CAPTURE	Timer DIO Interrupt Captured Value Register	P:0x105C 0x40000170

19.8.4.1 TIMER*_CFG0

Bit Field	Read/Write	Field Name	Description
26:24	RW	PRESCALE	Prescale value of the timer
23:0	RW	TIMEOUT_VALUE	Number of Timer clock cycles to time-out

Field Name	Value Symbol	Value Description	Hex Value
PRESCALE	TIMER_PRESCALE_1	Divide the input clock frequency by 1	0x0*
	TIMER_PRESCALE_2	Divide the input clock frequency by 2	0x1
	TIMER_PRESCALE_4	Divide the input clock frequency by 4	0x2
	TIMER_PRESCALE_8	Divide the input clock frequency by 8	0x3
	TIMER_PRESCALE_16	Divide the input clock frequency by 16	0x4
	TIMER_PRESCALE_32	Divide the input clock frequency by 32	0x5
	TIMER_PRESCALE_64	Divide the input clock frequency by 64	0x6
	TIMER_PRESCALE_128	Divide the input clock frequency by 128	0x7

19.8.4.2 TIMER*_CFG1

Bit Field	Read/Write	Field Name	Description
10:8	RW	MULTI_COUNT	Multi-count value
6:4	RW	DIO_INT_SRC	DIO interrupt source selection
2	RW	DIO_INT_MODE	DIO interrupt capture mode
1	RW	DIO_INT_ENABLE	DIO interrupt capture enable
0	RW	MODE	Timer mode

Ezairo 8300 Hardware Reference

Field Name	Value Symbol	Value Description	Hex Value
MULTI_COUNT	TIMER_MULTI_COUNT_1	Stop on 1st Time-out occurrence and issue an interrupt	0x0*
	TIMER_MULTI_COUNT_2	Stop on 2nd Time-out occurrence and issue an interrupt	0x1
	TIMER_MULTI_COUNT_3	Stop on 3rd Time-out occurrence and issue an interrupt	0x2
	TIMER_MULTI_COUNT_4	Stop on 4th Time-out occurrence and issue an interrupt	0x3
	TIMER_MULTI_COUNT_5	Stop on 5th Time-out occurrence and issue an interrupt	0x4
	TIMER_MULTI_COUNT_6	Stop on 6th Time-out occurrence and issue an interrupt	0x5
	TIMER_MULTI_COUNT_7	Stop on 7th Time-out occurrence and issue an interrupt	0x6
	TIMER_MULTI_COUNT_8	Stop on 8th Time-out occurrence and issue an interrupt	0x7
DIO_INT_SRC	TIMER_SRC_DIO_INT0	Select DIO interrupt defined in DIO_INT_CFG0	0x0*
	TIMER_SRC_DIO_INT1	Select DIO interrupt defined in DIO_INT_CFG1	0x1
	TIMER_SRC_DIO_INT2	Select DIO interrupt defined in DIO_INT_CFG2	0x2
	TIMER_SRC_DIO_INT3	Select DIO interrupt defined in DIO_INT_CFG3	0x3
	TIMER_SRC_DIO_INT4	Select DIO interrupt defined in DIO_INT_CFG4	0x4
	TIMER_SRC_DIO_INT5	Select DIO interrupt defined in DIO_INT_CFG5	0x5
	TIMER_SRC_DIO_INT6	Select DIO interrupt defined in DIO_INT_CFG5	0x6
	TIMER_SRC_DIO_INT7	Select DIO interrupt defined in DIO_INT_CFG7	0x7
DIO_INT_MODE	TIMER_DIO_INT_SINGLE	DIO interrupt single capture mode	0x0*
	TIMER_DIO_INT_CONTINUOUS	DIO interrupt continuous capture mode	0x1
DIO_INT_ENABLE	TIMER_DIO_INT_DISABLE	Disable DIO interrupt capture	0x0*
	TIMER_DIO_INT_ENABLE	Enable DIO interrupt capture	0x1
MODE	TIMER_SHOT_MODE	One shot / multi shot mode	0x0*
	TIMER_FREE_RUN	Free-run mode	0x1

Ezairo 8300 Hardware Reference

19.8.4.3 TIMER*_CTRL

Bit Field	Read/Write	Field Name	Description
8	R	BUSY	Indicate if the timer is active or not
1	W	START	Start or restart the timer
0	W	STOP	Stop the timer

Field Name	Value Symbol	Value Description	Hex Value
BUSY	TIMER_INACTIVE	The timer is inactive	0x0*
	TIMER_ACTIVE	The timer is active	0x1
START	TIMER_START	Writing a 1 starts or restarts the timer	0x1
STOP	TIMER_STOP	Writing a 1 stops the timer	0x1

19.8.4.4 TIMER*_VAL

Bit Field	Read/Write	Field Name	Description
26:24	R	MULTI_COUNT_VAL	Current multi counter value
23:0	R	TIMER_VALUE	Current timer value

19.8.4.5 TIMER*_VAL_CAPTURE

Bit Field	Read/Write	Field Name	Description
26:24	RW	MULTI_COUNT_VAL_DIO	Current multi counter value captured by the DIO interrupt defined in DIO_INT_SRC
23:0	RW	TIMER_VALUE_DIO	Timer value captured by the DIO interrupt defined in DIO_INT_SRC

19.9 WATCHDOG TIMER

The watchdog timer is a simple safety system that, in most cases, resets a system that has malfunctioned. It is a countdown timer that must be periodically acknowledged by writing `SYSTEM_WATCHDOG_REFRESH` to the `D_SYSTEM_CTRL` register before the countdown reaches zero. Setting this bit does not affect the other bits in this register, as these bits only change when they are explicitly set. The system assumes that the application's failure to acknowledge this countdown timer before it reaches zero for the second time indicates that the system is malfunctioning and must be reset. The countdown timer value for the watchdog timer is not visible to the CFX DSP or the Arm Cortex-M3 processor.

IMPORTANT: The watchdog timer cannot be disabled by either the CFX DSP or the Arm Cortex-M3 processor directly. The watchdog timer is automatically disabled when a connection is established with either of the debug ports. For more information on the debug ports, see [Chapter 17.1 "Debug" on page 523](#).

Ezairo 8300 Hardware Reference

The watchdog timer runs on the SLOWCLK_DIV32 divided by 32. This clock is used to decrement the value in a 13-bit counter. The delay before this timer times out can be configured using the WATCHDOG_CFG_TIMEOUT field of the D_WATCHDOG_CFG register.

Since this field is defined in terms of SLOWCLK, the effective timeout delays are different for different SLOWCLK frequencies (although the configuration defines for this timer are specified in terms of the typical 1.28 MHz SLOWCLK configuration). The "Watchdog Timeout Intervals for Different SLOWCLK Settings" table (Table 75) shows the delays associated with the SLOWCLK setting of 1.28 MHz. For other SLOWCLK frequencies, the watchdog timeout intervals are scaled appropriately.

Table 75. Watchdog Timeout Intervals for Different SLOWCLK Settings

Watchdog Scaler Setting	Approximate Timeout: SLOWCLK=1.28 MHz (ms)
WATCHDOG_TIMEOUT_1M6	1.6
WATCHDOG_TIMEOUT_3M2	3.2
WATCHDOG_TIMEOUT_6M4	6.4
WATCHDOG_TIMEOUT_12M8	12.8
WATCHDOG_TIMEOUT_25M6	25.6
WATCHDOG_TIMEOUT_51M2	51.2
WATCHDOG_TIMEOUT_102M4	102.4
WATCHDOG_TIMEOUT_204M8	204.8
WATCHDOG_TIMEOUT_409M6	409.6
WATCHDOG_TIMEOUT_819M2	819.2
WATCHDOG_TIMEOUT_1638M4	1638.4
WATCHDOG_TIMEOUT_3276M8	3276.8

For information on SLOWCLK frequency configuration, see [Section 10.2.4 “Slow Clock \(SLOWCLK\)”](#) on [page 315](#).

The watchdog timer block also contains a bypass to the watchdog reset itself, so that an application can force an immediate watchdog reset. This reset is triggered by writing WATCHDOG_FORCE_RESET to the D_WATCHDOG_RESET register.

19.9.1 Watchdog Interrupt

The watchdog has an associated interrupt that triggers whenever the watchdog times out. If the watchdog timer has not been refreshed and a second watchdog timeout occurs, the entire system resets. See [Section 3.2 “Interrupt Controller”](#) on [page 61](#) and [Section 4.6.1 “Nested Vectored Interrupt Controller \(NVIC\)”](#) on [page 135](#) for information regarding watchdog interrupt configuration and handling.

Ezairo 8300 Hardware Reference

19.9.2 Control and Configuration Registers

Register Name	Register Description	Address
D_WATCHDOG_CFG	Watchdog timeout configuration	P:0x0801
WATCHDOG_CFG		0x40002004
D_WATCHDOG_RESET	Force watchdog reset to occur immediately	P:0x0802
WATCHDOG_RESET		0x40002008

19.9.2.1 D_WATCHDOG_CFG, WATCHDOG_CFG

Bit Field	Read/Write	Field Name	Description
3:0	RW	TIMEOUT	Watchdog timeout period. Values 0xC to 0xF result in the same timeout period as the value 0xB

Field Name	Value Symbol	Value Description	Hex Value
TIMEOUT	WATCHDOG_TIMEOUT_1M6	1.6ms	0x0
	WATCHDOG_TIMEOUT_3M2	3.2ms	0x1
	WATCHDOG_TIMEOUT_6M4	6.4ms	0x2
	WATCHDOG_TIMEOUT_12M8	12.8ms	0x3
	WATCHDOG_TIMEOUT_25M6	25.6ms	0x4
	WATCHDOG_TIMEOUT_51M2	51.2ms	0x5
	WATCHDOG_TIMEOUT_102M4	102.4ms	0x6
	WATCHDOG_TIMEOUT_204M8	204.8ms	0x7
	WATCHDOG_TIMEOUT_409M6	409.6ms	0x8
	WATCHDOG_TIMEOUT_819M2	819.2ms	0x9
	WATCHDOG_TIMEOUT_1638M4	1.64s	0xA
	WATCHDOG_TIMEOUT_3276M8	3.28s	0xB*

19.9.2.2 WATCHDOG_RESET, D_WATCHDOG_RESET

Bit Field	Read/Write	Field Name	Description
31:0	W	FORCE_RESET	Force watchdog reset to occur immediately when the correct key is written

Field Name	Value Symbol	Value Description	Hex Value
FORCE_RESET	WATCHDOG_FORCE_RESET	Force watchdog reset to occur immediately	0x87FDF035

APPENDIX A

Control and Configuration Registers

This topic lists all the registers that are available. Refer to the appropriate section for information about the control and configuration registers for a block. The sections are:

- Section A.1 “System Control” on page 773
- Section A.2 “Watchdog Timer Configuration” on page 774
- Section A.3 “Boot Status” on page 775
- Section A.4 “HEAR Function Chain Commands” on page 776
- Section A.5 “PRAM Security” on page 777
- Section A.6 “CFX/Arm Cortex-M3 Control and Configuration” on page 778
- Section A.7 “External Breakpoint Configuration” on page 779
- Section A.8 “Hardware CRC” on page 780
- Section A.9 “Chip Version Information” on page 781
- Section A.10 “Debug Port Configuration” on page 782
- Section A.11 “Interrupt Configuration” on page 783
- Section A.12 “Clock Configuration” on page 816
- Section A.13 “IOC Configuration” on page 819
- Section A.14 “Analog” on page 827
- Section A.15 “Audio” on page 831
- Section A.16 “I2C Interface” on page 838
- Section A.17 “FENG Control and Configuration” on page 842
- Section A.18 “FENG Register File” on page 862
- Section A.19 “Chip Identification” on page 863
- Section A.20 “System Control” on page 864
- Section A.21 “General-Purpose Timers” on page 875
- Section A.22 “DMA Controller Configuration and Control” on page 879
- Section A.23 “Audio Sink Clock Counters” on page 901
- Section A.24 “ASRC Configuration and Control” on page 903
- Section A.25 “CRC Generator Control” on page 905
- Section A.26 “G.722 Configuration and Control” on page 907
- Section A.27 “Reed Solomon Configuration and Control” on page 910
- Section A.28 “DIO Interface and Digital Pad control” on page 913
- Section A.29 “PCM Interface Configuration and Control” on page 920
- Section A.30 “SPI Interface Configuration and Control” on page 925
- Section A.31 “UART Interface Configuration and Control” on page 931
- Section A.32 “I2C Interface Configuration and Control” on page 933
- Section A.33 “Analog-to-Digital Converter and Battery Monitoring” on page 940
- Section A.34 “eMMC and I3C Interface Control” on page 943
- Section A.35 “I3C Interface Configuration and Control” on page 945
- Section A.36 “eMMC Interface Configuration Control” on page 985
- Section A.37 “Neural Network Accelerator” on page 1007
- Section A.38 “SYSTICK Timer” on page 1011
- Section A.39 “System Control and ID register not in the SCB” on page 1012
- Section A.40 “Nested Vector Interrupt Controller” on page 1013
- Section A.41 “System Control Block” on page 1040
- Section A.42 “Debug Controller” on page 1045
- Section A.43 “CFX Math Accelerators I/O and Control” on page 1047
- Section A.44 “FIFO A0 Configuration and Control” on page 1049
- Section A.45 “FIFO A1 Configuration and Control” on page 1057
- Section A.46 “FIFO B0 Configuration and Control” on page 1065

Ezairo 8300 Hardware Reference

- Section A.47 “FIFO B1 Configuration and Control” on page 1073
- Section A.48 “FIFO Interrupt Selection” on page 1081
- Section A.49 “HEAR Interrupt Configuration and Control” on page 1082
- Section A.50 “Block Floating Point Configuration and Control” on page 1088
- Section A.51 “CFX Math Accelerators I/O” on page 1089

A.1 SYSTEM CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0x0800	D_SYSTEM_CTRL	–	(9) PRIORITY_STATUS	0x0	CFX/HEAR priority status
0x40002000	SYSTEM_CTRL				
		–	(8) HEAR_RUNNING	0x0	HEAR running status
		(7) PRIORITY_HEAR	–	N/A	Write a 1 to give HEAR priority
		(6) PRIORITY_CFX	–	N/A	Write a 1 to give CFX priority
		(5) HEAR_PAUSE	–	N/A	Write a 1 to pause HEAR
		(4) HEAR_RESUME	–	N/A	Write a 1 to run HEAR
		(3) HEAR_RESET	–	N/A	Write a 1 to reset HEAR
		(0) WATCHDOG_REFRESH	–	N/A	Write a 1 to reset the watchdog

A.2 WATCHDOG TIMER CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0x0801 0x40002004	D_WATCHDOG_CFG WATCHDOG_CFG	(3:0) TIMEOUT	(3:0) TIMEOUT	0xB	Watchdog timeout period. Values 0xC to 0xF result in the same timeout period as the value 0xB
P:0x0802 0x40002008	D_WATCHDOG_RESET WATCHDOG_RESET	(31:0) FORCE_RESET	–	N/A	Force watchdog reset to occur immediately when the correct key is written

A.3 BOOT STATUS

Address	Register Name	Register Write	Register Read	Default	Description
P:0x0803	D_BOOT_STATUS	–	(0) STATUS	0x0	CFX boot status
0x4000200C	BOOT_STATUS				
		(0) SET	–	N/A	Set CFX boot status

A.4 HEAR FUNCTION CHAIN COMMANDS

Address	Register Name	Register Write	Register Read	Default	Description
P:0x0804 0x40002010	D_HEAR_CMDS_0_TO_7 HEAR_CMDS_0_TO_7	(7) HEAR_CMD_7	–	N/A	Write a 1 to issue HEAR start command 7
		(6) HEAR_CMD_6	–	N/A	Write a 1 to issue HEAR start command 6
		(5) HEAR_CMD_5	–	N/A	Write a 1 to issue HEAR start command 5
		(4) HEAR_CMD_4	–	N/A	Write a 1 to issue HEAR start command 4
		(3) HEAR_CMD_3	–	N/A	Write a 1 to issue HEAR start command 3
		(2) HEAR_CMD_2	–	N/A	Write a 1 to issue HEAR start command 2
		(1) HEAR_CMD_1	–	N/A	Write a 1 to issue HEAR start command 1
		(0) HEAR_CMD_0	–	N/A	Write a 1 to issue HEAR start command 0
P:0x0805 0x40002014	D_HEAR_CMDS_8_TO_15 HEAR_CMDS_8_TO_15	(7) HEAR_CMD_15	–	N/A	Write a 1 to issue HEAR start command 15
		(6) HEAR_CMD_14	–	N/A	Write a 1 to issue HEAR start command 14
		(5) HEAR_CMD_13	–	N/A	Write a 1 to issue HEAR start command 13
		(4) HEAR_CMD_12	–	N/A	Write a 1 to issue HEAR start command 12
		(3) HEAR_CMD_11	–	N/A	Write a 1 to issue HEAR start command 11
		(2) HEAR_CMD_10	–	N/A	Write a 1 to issue HEAR start command 10
		(1) HEAR_CMD_9	–	N/A	Write a 1 to issue HEAR start command 9
		(0) HEAR_CMD_8	–	N/A	Write a 1 to issue HEAR start command 8

A.5 PRAM SECURITY

Address	Register Name	Register Write	Register Read	Default	Description
P:0x0809 0x40002024	D_PRAM_SECURITY_CFG PRAM_SECURITY_CFG	(3) CM3_PRAM1_LOCK	–	N/A	Lock Arm Cortex-M3 PRAM1 access except for PMEM bus
		–	(3) CM3_PRAM1_LOCK_STATUS	0x0	Arm Cortex-M3 PRAM1 access lock status
		(2) CM3_PRAM0_LOCK	–	N/A	Lock Arm Cortex-M3 PRAM0 access except for PMEM bus
		–	(2) CM3_PRAM0_LOCK_STATUS	0x0	Arm Cortex-M3 PRAM0 access lock status
		(1) CFX_PRAM1_LOCK	–	N/A	Lock CFX PRAM1 access except for PMEM bus
		–	(1) CFX_PRAM1_LOCK_STATUS	0x0	CFX PRAM1 access lock status
		(0) CFX_PRAM0_LOCK	–	N/A	Lock CFX PRAM0 access except for PMEM bus
		–	(0) CFX_PRAM0_LOCK_STATUS	0x0	CFX PRAM0 access lock status

A.6 CFX/ARM CORTEX-M3 CONTROL AND CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P: 0x080B	D_CM3_CMDS	(7) CM3_CMD_7	–	N/A	Write a 1 to issue Arm Cortex-M3 processor start command 7
0x4000202C	CM3_CMDS				
		(6) CM3_CMD_6	–	N/A	Write a 1 to issue Arm Cortex-M3 processor start command 6
		(5) CM3_CMD_5	–	N/A	Write a 1 to issue Arm Cortex-M3 processor start command 5
		(4) CM3_CMD_4	–	N/A	Write a 1 to issue Arm Cortex-M3 processor start command 4
		(3) CM3_CMD_3	–	N/A	Write a 1 to issue Arm Cortex-M3 processor start command 3
		(2) CM3_CMD_2	–	N/A	Write a 1 to issue Arm Cortex-M3 processor start command 2
		(1) CM3_CMD_1	–	N/A	Write a 1 to issue Arm Cortex-M3 processor start command 1
		(0) CM3_CMD_0	–	N/A	Write a 1 to issue Arm Cortex-M3 processor start command 0

A.7 EXTERNAL BREAKPOINT CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0x0810 0x40002040	D_EXT_BREAK_CFG EXT_BREAK_CFG	(6) CM3_FIFO_CONFLICT	(6) CM3_FIFO_CONFLICT	0x0	Configured detection of FIFO memory conflicts, between the Arm Cortex-M3 core or DMA, and the HEAR or CFX
		(5) CFX_CM3_CONFLICT	(5) CFX_CM3_CONFLICT	0x0	Configured detection of memory conflicts, between the CFX and the Arm Cortex-M3 core, LPDSP32 or DMA
		(4) CFX_HEAR_CONFLICT	(4) CFX_HEAR_CONFLICT	0x0	Configured detection of memory conflicts, between the CFX and HEAR
		(3:2) ACCESS_COND	(3:2) ACCESS_COND	0x0	Select the access condition
		(1:0) BUS_SELECT	(1:0) BUS_SELECT	0x0	Select the data bus to monitor for external breakpoints
P:0x0811 0x40002044	D_EXT_BREAK_ADDR_LOW EXT_BREAK_ADDR_LOW	(18:0) EXT_BREAK_ADDR_LOW	(18:0) EXT_BREAK_ADDR_LOW	0x0	External break low address
P:0x0812 0x40002048	D_EXT_BREAK_ADDR_HIGH EXT_BREAK_ADDR_HIGH	(18:0) EXT_BREAK_ADDR_HIGH	(18:0) EXT_BREAK_ADDR_HIGH	0x0	External break high address

A.8 HARDWARE CRC

Address	Register Name	Register Write	Register Read	Default	Description
P:0x0822 0x40002088	D_CRC_VALUE	(15:0) CRC_VALUE	(15:0) CRC_VALUE	0xFFFF	Value of CRC-CCITT for supplied data
P:0x0823 0x4000208C	D_CRC_ADD_1	(0) CRC_ADD_1	–	N/A	Add 1-bit data to the CRC
P:0x0824 0x40002090	D_CRC_ADD_8	(7:0) CRC_ADD_8	–	N/A	Add 8-bit data to the CRC
P:0x0825 0x40002094	D_CRC_ADD_16	(15:0) CRC_ADD_16	–	N/A	Add 16-bit data to the CRC
P:0x0826 0x40002098	D_CRC_ADD_24	(23:0) CRC_ADD_24	–	N/A	Add 24-bit data to the CRC
P:0x0827 0x4000209C	D_CRC_ADD_32	(31:0) CRC_ADD_32	–	N/A	Add 32-bit data to the CRC

A.9 CHIP VERSION INFORMATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0x0830 0x400020C0	D_CHIP_ID_FAMILY CHIP_ID_FAMILY	–	(7:0) CHIP_FAMILY	0xA	Chip family number
P:0x0831 0x400020C4	D_CHIP_ID_VERSION CHIP_ID_VERSION	–	(7:0) CHIP_VERSION	0x1	Chip version number
P:0x0832 0x400020C8	D_CHIP_ID_REVISION CHIP_ID_REVISION	–	(15:0) CHIP_REVISION	0x101	Chip revision number
P:0x0833 0x400020CC	D_CHIP_ID_CFX_DEBUG_VERSION CHIP_ID_CFX_DEBUG_VERSION	–	(15:0) CFX_DEBUG_VERSION	0x300	CFX debug port version number
P:0x0834 0x400020D0	D_CHIP_ID_CM3_DEBUG_VERSION CHIP_ID_CM3_DEBUG_VERSION	–	(15:0) CM3_DEBUG_VERSION	0x300	Arm Cortex-M3 debug version number

A.10 DEBUG PORT CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0x0835 0x400020D4	D_DEBUG_CFG DEBUG_CFG	–	(3) DBG_REG_WRITTEN	0x0	Data register write status
		–	(2) DBG_REG_READ	0x0	Data register read status
		–	(1) AUDIO_MODE	0x0	
		(0) ACCESS_MODE	(0) ACCESS_MODE	0x1	
P:0x0836 0x400020D8	D_DEBUG_DATA DEBUG_DATA	(31:0) DEBUG_DATA	(31:0) DEBUG_DATA	0x0	

A.11 INTERRUPT CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0x0840 0x40002100	D_INT_MASTER_ENABLE INT_MASTER_ENABLE	(0) MASTER	(0) MASTER	0x1	MASTER enable
P:0x0841 0x40002104	D_INT_ENABLE_0 INT_ENABLE_0	(31) SPI_0_TX	(31) SPI_0_TX	0x0	SPI_0_TX enable
		(30) SPI_0_RX	(30) SPI_0_RX	0x0	SPI_0_RX enable
		(29) SPI_0_COM	(29) SPI_0_COM	0x0	SPI_0_COM enable
		(28) D_I2C	(28) D_I2C	0x0	D_I2C enable
		(27) I2C_1	(27) I2C_1	0x0	I2C_1 enable
		(26) I2C_0	(26) I2C_0	0x0	I2C_0 enable
		(25) I3C_1	(25) I3C_1	0x0	I3C_1 enable
		(24) I3C_0	(24) I3C_0	0x0	I3C_0 enable
		(23) DIO_7	(23) DIO_7	0x0	DIO_7 enable
		(22) DIO_6	(22) DIO_6	0x0	DIO_6 enable
		(21) DIO_5	(21) DIO_5	0x0	DIO_5 enable
		(20) DIO_4	(20) DIO_4	0x0	DIO_4 enable
		(19) DIO_3	(19) DIO_3	0x0	DIO_3 enable
		(18) DIO_2	(18) DIO_2	0x0	DIO_2 enable
		(17) DIO_1	(17) DIO_1	0x0	DIO_1 enable
		(16) DIO_0	(16) DIO_0	0x0	DIO_0 enable
		(15) DMA_7	(15) DMA_7	0x0	DMA_7 enable
		(14) DMA_6	(14) DMA_6	0x0	DMA_6 enable

Address	Register Name	Register Write	Register Read	Default	Description
		(13) DMA_5	(13) DMA_5	0x0	DMA_5 enable
		(12) DMA_4	(12) DMA_4	0x0	DMA_4 enable
		(11) DMA_3	(11) DMA_3	0x0	DMA_3 enable
		(10) DMA_2	(10) DMA_2	0x0	DMA_2 enable
		(9) DMA_1	(9) DMA_1	0x0	DMA_1 enable
		(8) DMA_0	(8) DMA_0	0x0	DMA_0 enable
		(7) TIMER_3	(7) TIMER_3	0x0	TIMER_3 enable
		(6) TIMER_2	(6) TIMER_2	0x0	TIMER_2 enable
		(5) TIMER_1	(5) TIMER_1	0x0	TIMER_1 enable
		(4) TIMER_0	(4) TIMER_0	0x0	TIMER_0 enable
		(3) AUDIOSINK_1	(3) AUDIOSINK_1	0x0	AUDIOSINK_1 enable
		(2) AUDIOSINK_0	(2) AUDIOSINK_0	0x0	AUDIOSINK_0 enable
		(1) LSAD_BATMON	(1) LSAD_BATMON	0x0	LSAD_BATMON enable
		(0) WATCHDOG	(0) WATCHDOG	0x0	WATCHDOG enable
P:0x0842 0x40002108	D_INT_ENABLE_1 INT_ENABLE_1	(31) FIFO_7	(31) FIFO_7	0x0	FIFO_7 enable
		(30) FIFO_6	(30) FIFO_6	0x0	FIFO_6 enable
		(29) FIFO_5	(29) FIFO_5	0x0	FIFO_5 enable
		(28) FIFO_4	(28) FIFO_4	0x0	FIFO_4 enable
		(27) FIFO_3	(27) FIFO_3	0x0	FIFO_3 enable
		(26) FIFO_2	(26) FIFO_2	0x0	FIFO_2 enable
		(25) FIFO_1	(25) FIFO_1	0x0	FIFO_1 enable
		(24) FIFO_0	(24) FIFO_0	0x0	FIFO_0 enable

Address	Register Name	Register Write	Register Read	Default	Description
		(23) RS_DEC_OUT_RDY	(23) RS_DEC_OUT_RDY	0x0	RS_DEC_OUT_RDY enable
		(22) RS_DEC_IN_RDY	(22) RS_DEC_IN_RDY	0x0	RS_DEC_IN_RDY enable
		(21) RS_ENC_OUT_RDY	(21) RS_ENC_OUT_RDY	0x0	RS_ENC_OUT_RDY enable
		(20) RS_ENC_IN_RDY	(20) RS_ENC_IN_RDY	0x0	RS_ENC_IN_RDY enable
		(19) G722_DEC_OUT_RDY	(19) G722_DEC_OUT_RDY	0x0	G722_DEC_OUT_RDY enable
		(18) G722_DEC_IN_RDY	(18) G722_DEC_IN_RDY	0x0	G722_DEC_IN_RDY enable
		(17) G722_ENC_OUT_RDY	(17) G722_ENC_OUT_RDY	0x0	G722_ENC_OUT_RDY enable
		(16) G722_ENC_IN_RDY	(16) G722_ENC_IN_RDY	0x0	G722_ENC_IN_RDY enable
		(15) ASRC_IDLE	(15) ASRC_IDLE	0x0	ASRC_IDLE enable
		(14) ASRC_OUT	(14) ASRC_OUT	0x0	ASRC_OUT enable
		(13) ASRC_IN	(13) ASRC_IN	0x0	ASRC_IN enable
		(12) ASRC_ERROR	(12) ASRC_ERROR	0x0	ASRC_ERROR enable
		(11) PCM_1_RX_TX	(11) PCM_1_RX_TX	0x0	PCM_1_RX_TX enable
		(10) PCM_1_ERROR	(10) PCM_1_ERROR	0x0	PCM_1_ERROR enable
		(9) PCM_0_RX_TX	(9) PCM_0_RX_TX	0x0	PCM_0_RX_TX enable
		(8) PCM_0_ERROR	(8) PCM_0_ERROR	0x0	PCM_0_ERROR enable
		(7) FENG	(7) FENG	0x0	FENG enable
		(6) UART_0_TX	(6) UART_0_TX	0x0	UART_0_TX enable
		(5) UART_0_RX	(5) UART_0_RX	0x0	UART_0_RX enable
		(4) UART_0_ERROR	(4) UART_0_ERROR	0x0	UART_0_ERROR enable

Address	Register Name	Register Write	Register Read	Default	Description
		(3) EMMC	(3) EMMC	0x0	EMMC enable
		(2) SPI_1_TX	(2) SPI_1_TX	0x0	SPI_1_TX enable
		(1) SPI_1_RX	(1) SPI_1_RX	0x0	SPI_1_RX enable
		(0) SPI_1_COM	(0) SPI_1_COM	0x0	SPI_1_COM enable
P:0x0843 0x4000210C	D_INT_ENABLE_2 INT_ENABLE_2	(29) RS_COM	(29) RS_COM	0x0	RS_COM enable
		(28) G722_COM	(28) G722_COM	0x0	G722_COM enable
		(27) MEM_ERROR	(27) MEM_ERROR	0x0	MEM_ERROR enable
		(26) LSAD	(26) LSAD	0x0	LSAD enable
		(25) CLK_DET	(25) CLK_DET	0x0	CLK_DET enable
		(24) NNA	(24) NNA	0x0	NNA enable
		(23) CM3_7	(23) CM3_7	0x0	CM3_7 enable
		(22) CM3_6	(22) CM3_6	0x0	CM3_6 enable
		(21) CM3_5	(21) CM3_5	0x0	CM3_5 enable
		(20) CM3_4	(20) CM3_4	0x0	CM3_4 enable
		(19) CM3_3	(19) CM3_3	0x0	CM3_3 enable
		(18) CM3_2	(18) CM3_2	0x0	CM3_2 enable
		(17) CM3_1	(17) CM3_1	0x0	CM3_1 enable
		(16) CM3_0	(16) CM3_0	0x0	CM3_0 enable
		(15) DSP_7	(15) DSP_7	0x0	DSP_7 enable
		(14) DSP_6	(14) DSP_6	0x0	DSP_6 enable
		(13) DSP_5	(13) DSP_5	0x0	DSP_5 enable
		(12) DSP_4	(12) DSP_4	0x0	DSP_4 enable

Address	Register Name	Register Write	Register Read	Default	Description
		(11) DSP_3	(11) DSP_3	0x0	DSP_3 enable
		(10) DSP_2	(10) DSP_2	0x0	DSP_2 enable
		(9) DSP_1	(9) DSP_1	0x0	DSP_1 enable
		(8) DSP_0	(8) DSP_0	0x0	DSP_0 enable
		(7) HEAR_7	(7) HEAR_7	0x0	HEAR_7 enable
		(6) HEAR_6	(6) HEAR_6	0x0	HEAR_6 enable
		(5) HEAR_5	(5) HEAR_5	0x0	HEAR_5 enable
		(4) HEAR_4	(4) HEAR_4	0x0	HEAR_4 enable
		(3) HEAR_3	(3) HEAR_3	0x0	HEAR_3 enable
		(2) HEAR_2	(2) HEAR_2	0x0	HEAR_2 enable
		(1) HEAR_1	(1) HEAR_1	0x0	HEAR_1 enable
		(0) HEAR_0	(0) HEAR_0	0x0	HEAR_0 enable
P:0x0844 0x40002110	D_INT_STATUS_0 INT_STATUS_0	(31) SPI_0_TX	(31) SPI_0_TX	0x0	SPI_0_TX status/acknowledge
		(30) SPI_0_RX	(30) SPI_0_RX	0x0	SPI_0_RX status/acknowledge
		(29) SPI_0_COM	(29) SPI_0_COM	0x0	SPI_0_COM status/acknowledge
		(28) D_I2C	(28) D_I2C	0x0	D_I2C status/acknowledge
		(27) I2C_1	(27) I2C_1	0x0	I2C_1 status/acknowledge
		(26) I2C_0	(26) I2C_0	0x0	I2C_0 status/acknowledge
		(25) I3C_1	(25) I3C_1	0x0	I3C_1 status/acknowledge
		(24) I3C_0	(24) I3C_0	0x0	I3C_0 status/acknowledge
		(23) DIO_7	(23) DIO_7	0x0	DIO_7 status/acknowledge
		(22) DIO_6	(22) DIO_6	0x0	DIO_6 status/acknowledge

Address	Register Name	Register Write	Register Read	Default	Description
		(21) DIO_5	(21) DIO_5	0x0	DIO_5 status/acknowledge
		(20) DIO_4	(20) DIO_4	0x0	DIO_4 status/acknowledge
		(19) DIO_3	(19) DIO_3	0x0	DIO_3 status/acknowledge
		(18) DIO_2	(18) DIO_2	0x0	DIO_2 status/acknowledge
		(17) DIO_1	(17) DIO_1	0x0	DIO_1 status/acknowledge
		(16) DIO_0	(16) DIO_0	0x0	DIO_0 status/acknowledge
		(15) DMA_7	(15) DMA_7	0x0	DMA_7 status/acknowledge
		(14) DMA_6	(14) DMA_6	0x0	DMA_6 status/acknowledge
		(13) DMA_5	(13) DMA_5	0x0	DMA_5 status/acknowledge
		(12) DMA_4	(12) DMA_4	0x0	DMA_4 status/acknowledge
		(11) DMA_3	(11) DMA_3	0x0	DMA_3 status/acknowledge
		(10) DMA_2	(10) DMA_2	0x0	DMA_2 status/acknowledge
		(9) DMA_1	(9) DMA_1	0x0	DMA_1 status/acknowledge
		(8) DMA_0	(8) DMA_0	0x0	DMA_0 status/acknowledge
		(7) TIMER_3	(7) TIMER_3	0x0	TIMER_3 status/acknowledge
		(6) TIMER_2	(6) TIMER_2	0x0	TIMER_2 status/acknowledge
		(5) TIMER_1	(5) TIMER_1	0x0	TIMER_1 status/acknowledge
		(4) TIMER_0	(4) TIMER_0	0x0	TIMER_0 status/acknowledge
		(3) AUDIOSINK_1	(3) AUDIOSINK_1	0x0	AUDIOSINK_1 status/acknowledge
		(2) AUDIOSINK_0	(2) AUDIOSINK_0	0x0	AUDIOSINK_0 status/acknowledge
		(1) LSAD_BATMON	(1) LSAD_BATMON	0x0	LSAD_BATMON status/acknowledge
		(0) WATCHDOG	(0) WATCHDOG	0x0	WATCHDOG status/acknowledge
P:0x0845	D_INT_STATUS_1	(31) FIFO_7	(31) FIFO_7	0x0	FIFO_7 status/acknowledge

Address	Register Name	Register Write	Register Read	Default	Description
0x40002114	INT_STATUS_1				
		(30) FIFO_6	(30) FIFO_6	0x0	FIFO_6 status/acknowledge
		(29) FIFO_5	(29) FIFO_5	0x0	FIFO_5 status/acknowledge
		(28) FIFO_4	(28) FIFO_4	0x0	FIFO_4 status/acknowledge
		(27) FIFO_3	(27) FIFO_3	0x0	FIFO_3 status/acknowledge
		(26) FIFO_2	(26) FIFO_2	0x0	FIFO_2 status/acknowledge
		(25) FIFO_1	(25) FIFO_1	0x0	FIFO_1 status/acknowledge
		(24) FIFO_0	(24) FIFO_0	0x0	FIFO_0 status/acknowledge
		(23) RS_DEC_OUT_RDY	(23) RS_DEC_OUT_RDY	0x0	RS_DEC_OUT_RDY status/acknowledge
		(22) RS_DEC_IN_RDY	(22) RS_DEC_IN_RDY	0x0	RS_DEC_IN_RDY status/acknowledge
		(21) RS_ENC_OUT_RDY	(21) RS_ENC_OUT_RDY	0x0	RS_ENC_OUT_RDY status/acknowledge
		(20) RS_ENC_IN_RDY	(20) RS_ENC_IN_RDY	0x0	RS_ENC_IN_RDY status/acknowledge
		(19) G722_DEC_OUT_RDY	(19) G722_DEC_OUT_RDY	0x0	G722_DEC_OUT_RDY status/acknowledge
		(18) G722_DEC_IN_RDY	(18) G722_DEC_IN_RDY	0x0	G722_DEC_IN_RDY status/acknowledge
		(17) G722_ENC_OUT_RDY	(17) G722_ENC_OUT_RDY	0x0	G722_ENC_OUT_RDY status/acknowledge
		(16) G722_ENC_IN_RDY	(16) G722_ENC_IN_RDY	0x0	G722_ENC_IN_RDY status/acknowledge
		(15) ASRC_IDLE	(15) ASRC_IDLE	0x0	ASRC_IDLE status/acknowledge
		(14) ASRC_OUT	(14) ASRC_OUT	0x0	ASRC_OUT status/acknowledge
		(13) ASRC_IN	(13) ASRC_IN	0x0	ASRC_IN status/acknowledge
		(12) ASRC_ERROR	(12) ASRC_ERROR	0x0	ASRC_ERROR status/acknowledge

Address	Register Name	Register Write	Register Read	Default	Description
		(11) PCM_1_RX_TX	(11) PCM_1_RX_TX	0x0	PCM_1_RX_TX status/acknowledge
		(10) PCM_1_ERROR	(10) PCM_1_ERROR	0x0	PCM_1_ERROR status/acknowledge
		(9) PCM_0_RX_TX	(9) PCM_0_RX_TX	0x0	PCM_0_RX_TX status/acknowledge
		(8) PCM_0_ERROR	(8) PCM_0_ERROR	0x0	PCM_0_ERROR status/acknowledge
		(7) FENG	(7) FENG	0x0	FENG status/acknowledge
		(6) UART_0_TX	(6) UART_0_TX	0x0	UART_0_TX status/acknowledge
		(5) UART_0_RX	(5) UART_0_RX	0x0	UART_0_RX status/acknowledge
		(4) UART_0_ERROR	(4) UART_0_ERROR	0x0	UART_0_ERROR status/acknowledge
		(3) EMMC	(3) EMMC	0x0	EMMC status/acknowledge
		(2) SPI_1_TX	(2) SPI_1_TX	0x0	SPI_1_TX status/acknowledge
		(1) SPI_1_RX	(1) SPI_1_RX	0x0	SPI_1_RX status/acknowledge
		(0) SPI_1_COM	(0) SPI_1_COM	0x0	SPI_1_COM status/acknowledge
P:0x0846 0x40002118	D_INT_STATUS_2 INT_STATUS_2	(29) RS_COM	(29) RS_COM	0x0	RS_COM status/acknowledge
		(28) G722_COM	(28) G722_COM	0x0	G722_COM status/acknowledge
		(27) MEM_ERROR	(27) MEM_ERROR	0x0	MEM_ERROR status/acknowledge
		(26) LSAD	(26) LSAD	0x0	LSAD status/acknowledge
		(25) CLK_DET	(25) CLK_DET	0x0	CLK_DET status/acknowledge
		(24) NNA	(24) NNA	0x0	NNA status/acknowledge
		(23) CM3_7	(23) CM3_7	0x0	CM3_7 status/acknowledge
		(22) CM3_6	(22) CM3_6	0x0	CM3_6 status/acknowledge
		(21) CM3_5	(21) CM3_5	0x0	CM3_5 status/acknowledge
		(20) CM3_4	(20) CM3_4	0x0	CM3_4 status/acknowledge

Address	Register Name	Register Write	Register Read	Default	Description
		(19) CM3_3	(19) CM3_3	0x0	CM3_3 status/acknowledge
		(18) CM3_2	(18) CM3_2	0x0	CM3_2 status/acknowledge
		(17) CM3_1	(17) CM3_1	0x0	CM3_1 status/acknowledge
		(16) CM3_0	(16) CM3_0	0x0	CM3_0 status/acknowledge
		(15) DSP_7	(15) DSP_7	0x0	DSP_7 status/acknowledge
		(14) DSP_6	(14) DSP_6	0x0	DSP_6 status/acknowledge
		(13) DSP_5	(13) DSP_5	0x0	DSP_5 status/acknowledge
		(12) DSP_4	(12) DSP_4	0x0	DSP_4 status/acknowledge
		(11) DSP_3	(11) DSP_3	0x0	DSP_3 status/acknowledge
		(10) DSP_2	(10) DSP_2	0x0	DSP_2 status/acknowledge
		(9) DSP_1	(9) DSP_1	0x0	DSP_1 status/acknowledge
		(8) DSP_0	(8) DSP_0	0x0	DSP_0 status/acknowledge
		(7) HEAR_7	(7) HEAR_7	0x0	HEAR_7 status/acknowledge
		(6) HEAR_6	(6) HEAR_6	0x0	HEAR_6 status/acknowledge
		(5) HEAR_5	(5) HEAR_5	0x0	HEAR_5 status/acknowledge
		(4) HEAR_4	(4) HEAR_4	0x0	HEAR_4 status/acknowledge
		(3) HEAR_3	(3) HEAR_3	0x0	HEAR_3 status/acknowledge
		(2) HEAR_2	(2) HEAR_2	0x0	HEAR_2 status/acknowledge
		(1) HEAR_1	(1) HEAR_1	0x0	HEAR_1 status/acknowledge
		(0) HEAR_0	(0) HEAR_0	0x0	HEAR_0 status/acknowledge
P:0x0847 0x4000211C	D_INT_PRIORITY_0 INT_PRIORITY_0	(31) SPI_0_TX	(31) SPI_0_TX	0x0	SPI_0_TX interrupt priority
		(30) SPI_0_RX	(30) SPI_0_RX	0x0	SPI_0_RX interrupt priority

Address	Register Name	Register Write	Register Read	Default	Description
		(29) SPI_0_COM	(29) SPI_0_COM	0x0	SPI_0_COM interrupt priority
		(28) D_I2C	(28) D_I2C	0x0	D_I2C interrupt priority
		(27) I2C_1	(27) I2C_1	0x0	I2C_1 interrupt priority
		(26) I2C_0	(26) I2C_0	0x0	I2C_0 interrupt priority
		(25) I3C_1	(25) I3C_1	0x0	I3C_1 interrupt priority
		(24) I3C_0	(24) I3C_0	0x0	I3C_0 interrupt priority
		(23) DIO_7	(23) DIO_7	0x0	DIO_7 interrupt priority
		(22) DIO_6	(22) DIO_6	0x0	DIO_6 interrupt priority
		(21) DIO_5	(21) DIO_5	0x0	DIO_5 interrupt priority
		(20) DIO_4	(20) DIO_4	0x0	DIO_4 interrupt priority
		(19) DIO_3	(19) DIO_3	0x0	DIO_3 interrupt priority
		(18) DIO_2	(18) DIO_2	0x0	DIO_2 interrupt priority
		(17) DIO_1	(17) DIO_1	0x0	DIO_1 interrupt priority
		(16) DIO_0	(16) DIO_0	0x0	DIO_0 interrupt priority
		(15) DMA_7	(15) DMA_7	0x0	DMA_7 interrupt priority
		(14) DMA_6	(14) DMA_6	0x0	DMA_6 interrupt priority
		(13) DMA_5	(13) DMA_5	0x0	DMA_5 interrupt priority
		(12) DMA_4	(12) DMA_4	0x0	DMA_4 interrupt priority
		(11) DMA_3	(11) DMA_3	0x0	DMA_3 interrupt priority
		(10) DMA_2	(10) DMA_2	0x0	DMA_2 interrupt priority
		(9) DMA_1	(9) DMA_1	0x0	DMA_1 interrupt priority
		(8) DMA_0	(8) DMA_0	0x0	DMA_0 interrupt priority
		(7) TIMER_3	(7) TIMER_3	0x0	TIMER_3 interrupt priority

Address	Register Name	Register Write	Register Read	Default	Description
		(6) TIMER_2	(6) TIMER_2	0x0	TIMER_2 interrupt priority
		(5) TIMER_1	(5) TIMER_1	0x0	TIMER_1 interrupt priority
		(4) TIMER_0	(4) TIMER_0	0x0	TIMER_0 interrupt priority
		(3) AUDIOSINK_1	(3) AUDIOSINK_1	0x0	AUDIOSINK_1 interrupt priority
		(2) AUDIOSINK_0	(2) AUDIOSINK_0	0x0	AUDIOSINK_0 interrupt priority
		(1) LSAD_BATMON	(1) LSAD_BATMON	0x0	LSAD_BATMON interrupt priority
		(0) WATCHDOG	(0) WATCHDOG	0x0	WATCHDOG interrupt priority
P:0x0848	D_INT_PRIORITY_1	(31) FIFO_7	(31) FIFO_7	0x0	FIFO_7 interrupt priority
0x40002120	INT_PRIORITY_1				
		(30) FIFO_6	(30) FIFO_6	0x0	FIFO_6 interrupt priority
		(29) FIFO_5	(29) FIFO_5	0x0	FIFO_5 interrupt priority
		(28) FIFO_4	(28) FIFO_4	0x0	FIFO_4 interrupt priority
		(27) FIFO_3	(27) FIFO_3	0x0	FIFO_3 interrupt priority
		(26) FIFO_2	(26) FIFO_2	0x0	FIFO_2 interrupt priority
		(25) FIFO_1	(25) FIFO_1	0x0	FIFO_1 interrupt priority
		(24) FIFO_0	(24) FIFO_0	0x0	FIFO_0 interrupt priority
		(23) RS_DEC_OUT_RDY	(23) RS_DEC_OUT_RDY	0x0	RS_DEC_OUT_RDY interrupt priority
		(22) RS_DEC_IN_RDY	(22) RS_DEC_IN_RDY	0x0	RS_DEC_IN_RDY interrupt priority
		(21) RS_ENC_OUT_RDY	(21) RS_ENC_OUT_RDY	0x0	RS_ENC_OUT_RDY interrupt priority
		(20) RS_ENC_IN_RDY	(20) RS_ENC_IN_RDY	0x0	RS_ENC_IN_RDY interrupt priority
		(19) G722_DEC_OUT_RDY	(19) G722_DEC_OUT_RDY	0x0	G722_DEC_OUT_RDY interrupt priority
		(18) G722_DEC_IN_RDY	(18) G722_DEC_IN_RDY	0x0	G722_DEC_IN_RDY interrupt priority

Address	Register Name	Register Write	Register Read	Default	Description
		(17) G722_ENC_OUT_RDY	(17) G722_ENC_OUT_RDY	0x0	G722_ENC_OUT_RDY interrupt priority
		(16) G722_ENC_IN_RDY	(16) G722_ENC_IN_RDY	0x0	G722_ENC_IN_RDY interrupt priority
		(15) ASRC_IDLE	(15) ASRC_IDLE	0x0	ASRC_IDLE interrupt priority
		(14) ASRC_OUT	(14) ASRC_OUT	0x0	ASRC_OUT interrupt priority
		(13) ASRC_IN	(13) ASRC_IN	0x0	ASRC_IN interrupt priority
		(12) ASRC_ERROR	(12) ASRC_ERROR	0x0	ASRC_ERROR interrupt priority
		(11) PCM_1_RX_TX	(11) PCM_1_RX_TX	0x0	PCM_1_RX_TX interrupt priority
		(10) PCM_1_ERROR	(10) PCM_1_ERROR	0x0	PCM_1_ERROR interrupt priority
		(9) PCM_0_RX_TX	(9) PCM_0_RX_TX	0x0	PCM_0_RX_TX interrupt priority
		(8) PCM_0_ERROR	(8) PCM_0_ERROR	0x0	PCM_0_ERROR interrupt priority
		(7) FENG	(7) FENG	0x0	FENG interrupt priority
		(6) UART_0_TX	(6) UART_0_TX	0x0	UART_0_TX interrupt priority
		(5) UART_0_RX	(5) UART_0_RX	0x0	UART_0_RX interrupt priority
		(4) UART_0_ERROR	(4) UART_0_ERROR	0x0	UART_0_ERROR interrupt priority
		(3) EMMC	(3) EMMC	0x0	EMMC interrupt priority
		(2) SPI_1_TX	(2) SPI_1_TX	0x0	SPI_1_TX interrupt priority
		(1) SPI_1_RX	(1) SPI_1_RX	0x0	SPI_1_RX interrupt priority
		(0) SPI_1_COM	(0) SPI_1_COM	0x0	SPI_1_COM interrupt priority
P:0x0849 0x40002124	D_INT_PRIORITY_2 INT_PRIORITY_2	(29) RS_COM	(29) RS_COM	0x0	RS_COM interrupt priority
		(28) G722_COM	(28) G722_COM	0x0	G722_COM interrupt priority

Address	Register Name	Register Write	Register Read	Default	Description
		(27) MEM_ERROR	(27) MEM_ERROR	0x0	MEM_ERROR interrupt priority
		(26) LSAD	(26) LSAD	0x0	LSAD interrupt priority
		(25) CLK_DET	(25) CLK_DET	0x0	CLK_DET interrupt priority
		(24) NNA	(24) NNA	0x0	NNA interrupt priority
		(23) CM3_7	(23) CM3_7	0x0	CM3_7 interrupt priority
		(22) CM3_6	(22) CM3_6	0x0	CM3_6 interrupt priority
		(21) CM3_5	(21) CM3_5	0x0	CM3_5 interrupt priority
		(20) CM3_4	(20) CM3_4	0x0	CM3_4 interrupt priority
		(19) CM3_3	(19) CM3_3	0x0	CM3_3 interrupt priority
		(18) CM3_2	(18) CM3_2	0x0	CM3_2 interrupt priority
		(17) CM3_1	(17) CM3_1	0x0	CM3_1 interrupt priority
		(16) CM3_0	(16) CM3_0	0x0	CM3_0 interrupt priority
		(15) DSP_7	(15) DSP_7	0x0	DSP_7 interrupt priority
		(14) DSP_6	(14) DSP_6	0x0	DSP_6 interrupt priority
		(13) DSP_5	(13) DSP_5	0x0	DSP_5 interrupt priority
		(12) DSP_4	(12) DSP_4	0x0	DSP_4 interrupt priority
		(11) DSP_3	(11) DSP_3	0x0	DSP_3 interrupt priority
		(10) DSP_2	(10) DSP_2	0x0	DSP_2 interrupt priority
		(9) DSP_1	(9) DSP_1	0x0	DSP_1 interrupt priority
		(8) DSP_0	(8) DSP_0	0x0	DSP_0 interrupt priority
		(7) HEAR_7	(7) HEAR_7	0x0	HEAR_7 interrupt priority
		(6) HEAR_6	(6) HEAR_6	0x0	HEAR_6 interrupt priority
		(5) HEAR_5	(5) HEAR_5	0x0	HEAR_5 interrupt priority

Address	Register Name	Register Write	Register Read	Default	Description
		(4) HEAR_4	(4) HEAR_4	0x0	HEAR_4 interrupt priority
		(3) HEAR_3	(3) HEAR_3	0x0	HEAR_3 interrupt priority
		(2) HEAR_2	(2) HEAR_2	0x0	HEAR_2 interrupt priority
		(1) HEAR_1	(1) HEAR_1	0x0	HEAR_1 interrupt priority
		(0) HEAR_0	(0) HEAR_0	0x0	HEAR_0 interrupt priority
P:0x0850	D_INT_EBL_0	(7) TIMER_3	–	N/A	TIMER_3 interrupt enable
0x40002140	INT_EBL_0				
		(6) TIMER_2	–	N/A	TIMER_2 interrupt enable
		(5) TIMER_1	–	N/A	TIMER_1 interrupt enable
		(4) TIMER_0	–	N/A	TIMER_0 interrupt enable
		(3) AUDIOSINK_1	–	N/A	AUDIOSINK_1 interrupt enable
		(2) AUDIOSINK_0	–	N/A	AUDIOSINK_0 interrupt enable
		(1) LSAD_BATMON	–	N/A	LSAD_BATMON interrupt enable
		(0) WATCHDOG	–	N/A	WATCHDOG interrupt enable
P:0x0851	D_INT_EBL_1	(7) DMA_7	–	N/A	DMA_7 interrupt enable
0x40002144	INT_EBL_1				
		(6) DMA_6	–	N/A	DMA_6 interrupt enable
		(5) DMA_5	–	N/A	DMA_5 interrupt enable
		(4) DMA_4	–	N/A	DMA_4 interrupt enable
		(3) DMA_3	–	N/A	DMA_3 interrupt enable
		(2) DMA_2	–	N/A	DMA_2 interrupt enable
		(1) DMA_1	–	N/A	DMA_1 interrupt enable

Address	Register Name	Register Write	Register Read	Default	Description
		(0) DMA_0	–	N/A	DMA_0 interrupt enable
P:0x0852	D_INT_EBL_2	(7) DIO_7	–	N/A	DIO_7 interrupt enable
0x40002148	INT_EBL_2				
		(6) DIO_6	–	N/A	DIO_6 interrupt enable
		(5) DIO_5	–	N/A	DIO_5 interrupt enable
		(4) DIO_4	–	N/A	DIO_4 interrupt enable
		(3) DIO_3	–	N/A	DIO_3 interrupt enable
		(2) DIO_2	–	N/A	DIO_2 interrupt enable
		(1) DIO_1	–	N/A	DIO_1 interrupt enable
		(0) DIO_0	–	N/A	DIO_0 interrupt enable
P:0x0853	D_INT_EBL_3	(7) SPI_0_TX	–	N/A	SPI_0_TX interrupt enable
0x4000214C	INT_EBL_3				
		(6) SPI_0_RX	–	N/A	SPI_0_RX interrupt enable
		(5) SPI_0_COM	–	N/A	SPI_0_COM interrupt enable
		(4) D_I2C	–	N/A	D_I2C interrupt enable
		(3) I2C_1	–	N/A	I2C_1 interrupt enable
		(2) I2C_0	–	N/A	I2C_0 interrupt enable
		(1) I3C_1	–	N/A	I3C_1 interrupt enable
		(0) I3C_0	–	N/A	I3C_0 interrupt enable
P:0x0854	D_INT_EBL_4	(7) FENG	–	N/A	FENG interrupt enable
0x40002150	INT_EBL_4				
		(6) UART_0_TX	–	N/A	UART_0_TX interrupt enable

Address	Register Name	Register Write	Register Read	Default	Description
		(5) UART_0_RX	–	N/A	UART_0_RX interrupt enable
		(4) UART_0_ERROR	–	N/A	UART_0_ERROR interrupt enable
		(3) EMMC	–	N/A	EMMC interrupt enable
		(2) SPI_1_TX	–	N/A	SPI_1_TX interrupt enable
		(1) SPI_1_RX	–	N/A	SPI_1_RX interrupt enable
		(0) SPI_1_COM	–	N/A	SPI_1_COM interrupt enable
P:0x0855 0x40002154	D_INT_EBL_5 INT_EBL_5	(7) ASRC_IDLE	–	N/A	ASRC_IDLE interrupt enable
		(6) ASRC_OUT	–	N/A	ASRC_OUT interrupt enable
		(5) ASRC_IN	–	N/A	ASRC_IN interrupt enable
		(4) ASRC_ERROR	–	N/A	ASRC_ERROR interrupt enable
		(3) PCM_1_RX_TX	–	N/A	PCM_1_RX_TX interrupt enable
		(2) PCM_1_ERROR	–	N/A	PCM_1_ERROR interrupt enable
		(1) PCM_0_RX_TX	–	N/A	PCM_0_RX_TX interrupt enable
		(0) PCM_0_ERROR	–	N/A	PCM_0_ERROR interrupt enable
P:0x0856 0x40002158	D_INT_EBL_6 INT_EBL_6	(7) RS_DEC_OUT_RDY	–	N/A	RS_DEC_OUT_RDY interrupt enable
		(6) RS_DEC_IN_RDY	–	N/A	RS_DEC_IN_RDY interrupt enable
		(5) RS_ENC_OUT_RDY	–	N/A	RS_ENC_OUT_RDY interrupt enable
		(4) RS_ENC_IN_RDY	–	N/A	RS_ENC_IN_RDY interrupt enable
		(3) G722_DEC_OUT_RDY	–	N/A	G722_DEC_OUT_RDY interrupt enable
		(2) G722_DEC_IN_RDY	–	N/A	G722_DEC_IN_RDY interrupt enable

Address	Register Name	Register Write	Register Read	Default	Description
		(1) G722_ENC_OUT_RDY	–	N/A	G722_ENC_OUT_RDY interrupt enable
		(0) G722_ENC_IN_RDY	–	N/A	G722_ENC_IN_RDY interrupt enable
P:0x0857 0x4000215C	D_INT_EBL_7 INT_EBL_7	(7) FIFO_7	–	N/A	FIFO_7 interrupt enable
		(6) FIFO_6	–	N/A	FIFO_6 interrupt enable
		(5) FIFO_5	–	N/A	FIFO_5 interrupt enable
		(4) FIFO_4	–	N/A	FIFO_4 interrupt enable
		(3) FIFO_3	–	N/A	FIFO_3 interrupt enable
		(2) FIFO_2	–	N/A	FIFO_2 interrupt enable
		(1) FIFO_1	–	N/A	FIFO_1 interrupt enable
		(0) FIFO_0	–	N/A	FIFO_0 interrupt enable
P:0x0858 0x40002160	D_INT_EBL_8 INT_EBL_8	(7) HEAR_7	–	N/A	HEAR_7 interrupt enable
		(6) HEAR_6	–	N/A	HEAR_6 interrupt enable
		(5) HEAR_5	–	N/A	HEAR_5 interrupt enable
		(4) HEAR_4	–	N/A	HEAR_4 interrupt enable
		(3) HEAR_3	–	N/A	HEAR_3 interrupt enable
		(2) HEAR_2	–	N/A	HEAR_2 interrupt enable
		(1) HEAR_1	–	N/A	HEAR_1 interrupt enable
		(0) HEAR_0	–	N/A	HEAR_0 interrupt enable
P:0x0859 0x40002164	D_INT_EBL_9 INT_EBL_9	(7) DSP_7	–	N/A	DSP_7 interrupt enable

Address	Register Name	Register Write	Register Read	Default	Description
		(6) DSP_6	–	N/A	DSP_6 interrupt enable
		(5) DSP_5	–	N/A	DSP_5 interrupt enable
		(4) DSP_4	–	N/A	DSP_4 interrupt enable
		(3) DSP_3	–	N/A	DSP_3 interrupt enable
		(2) DSP_2	–	N/A	DSP_2 interrupt enable
		(1) DSP_1	–	N/A	DSP_1 interrupt enable
		(0) DSP_0	–	N/A	DSP_0 interrupt enable
P:0x085A 0x40002168	D_INT_EBL_10 INT_EBL_10	(7) CM3_7	–	N/A	CM3_7 interrupt enable
		(6) CM3_6	–	N/A	CM3_6 interrupt enable
		(5) CM3_5	–	N/A	CM3_5 interrupt enable
		(4) CM3_4	–	N/A	CM3_4 interrupt enable
		(3) CM3_3	–	N/A	CM3_3 interrupt enable
		(2) CM3_2	–	N/A	CM3_2 interrupt enable
		(1) CM3_1	–	N/A	CM3_1 interrupt enable
		(0) CM3_0	–	N/A	CM3_0 interrupt enable
P:0x085B 0x4000216C	D_INT_EBL_11 INT_EBL_11	(5) RS_COM	–	N/A	RS_COM interrupt enable
		(4) G722_COM	–	N/A	G722_COM interrupt enable
		(3) MEM_ERROR	–	N/A	MEM_ERROR interrupt enable
		(2) LSAD	–	N/A	LSAD interrupt enable
		(1) CLK_DET	–	N/A	CLK_DET interrupt enable

Address	Register Name	Register Write	Register Read	Default	Description
		(0) NNA	–	N/A	NNA interrupt enable
P:0x085C	D_INT_DIS_0	(7) TIMER_3	–	N/A	TIMER_3 interrupt disable
0x40002170	INT_DIS_0				
		(6) TIMER_2	–	N/A	TIMER_2 interrupt disable
		(5) TIMER_1	–	N/A	TIMER_1 interrupt disable
		(4) TIMER_0	–	N/A	TIMER_0 interrupt disable
		(3) AUDIOSINK_1	–	N/A	AUDIOSINK_1 interrupt disable
		(2) AUDIOSINK_0	–	N/A	AUDIOSINK_0 interrupt disable
		(1) LSAD_BATMON	–	N/A	LSAD_BATMON interrupt disable
		(0) WATCHDOG	–	N/A	WATCHDOG interrupt disable
P:0x085D	D_INT_DIS_1	(7) DMA_7	–	N/A	DMA_7 interrupt disable
0x40002174	INT_DIS_1				
		(6) DMA_6	–	N/A	DMA_6 interrupt disable
		(5) DMA_5	–	N/A	DMA_5 interrupt disable
		(4) DMA_4	–	N/A	DMA_4 interrupt disable
		(3) DMA_3	–	N/A	DMA_3 interrupt disable
		(2) DMA_2	–	N/A	DMA_2 interrupt disable
		(1) DMA_1	–	N/A	DMA_1 interrupt disable
		(0) DMA_0	–	N/A	DMA_0 interrupt disable
P:0x085E	D_INT_DIS_2	(7) DIO_7	–	N/A	DIO_7 interrupt disable
0x40002178	INT_DIS_2				
		(6) DIO_6	–	N/A	DIO_6 interrupt disable

Address	Register Name	Register Write	Register Read	Default	Description
		(5) DIO_5	–	N/A	DIO_5 interrupt disable
		(4) DIO_4	–	N/A	DIO_4 interrupt disable
		(3) DIO_3	–	N/A	DIO_3 interrupt disable
		(2) DIO_2	–	N/A	DIO_2 interrupt disable
		(1) DIO_1	–	N/A	DIO_1 interrupt disable
		(0) DIO_0	–	N/A	DIO_0 interrupt disable
P:0x085F 0x4000217C	D_INT_DIS_3 INT_DIS_3	(7) SPI_0_TX	–	N/A	SPI_0_TX interrupt disable
		(6) SPI_0_RX	–	N/A	SPI_0_RX interrupt disable
		(5) SPI_0_COM	–	N/A	SPI_0_COM interrupt disable
		(4) D_I2C	–	N/A	D_I2C interrupt disable
		(3) I2C_1	–	N/A	I2C_1 interrupt disable
		(2) I2C_0	–	N/A	I2C_0 interrupt disable
		(1) I3C_1	–	N/A	I3C_1 interrupt disable
		(0) I3C_0	–	N/A	I3C_0 interrupt disable
P:0x0860 0x40002180	D_INT_DIS_4 INT_DIS_4	(7) FENG	–	N/A	FENG interrupt disable
		(6) UART_0_TX	–	N/A	UART_0_TX interrupt disable
		(5) UART_0_RX	–	N/A	UART_0_RX interrupt disable
		(4) UART_0_ERROR	–	N/A	UART_0_ERROR interrupt disable
		(3) EMMC	–	N/A	EMMC interrupt disable
		(2) SPI_1_TX	–	N/A	SPI_1_TX interrupt disable

Address	Register Name	Register Write	Register Read	Default	Description
		(1) SPI_1_RX	–	N/A	SPI_1_RX interrupt disable
		(0) SPI_1_COM	–	N/A	SPI_1_COM interrupt disable
P:0x0861 0x40002184	D_INT_DIS_5 INT_DIS_5	(7) ASRC_IDLE	–	N/A	ASRC_IDLE interrupt disable
		(6) ASRC_OUT	–	N/A	ASRC_OUT interrupt disable
		(5) ASRC_IN	–	N/A	ASRC_IN interrupt disable
		(4) ASRC_ERROR	–	N/A	ASRC_ERROR interrupt disable
		(3) PCM_1_RX_TX	–	N/A	PCM_1_RX_TX interrupt disable
		(2) PCM_1_ERROR	–	N/A	PCM_1_ERROR interrupt disable
		(1) PCM_0_RX_TX	–	N/A	PCM_0_RX_TX interrupt disable
		(0) PCM_0_ERROR	–	N/A	PCM_0_ERROR interrupt disable
P:0x0862 0x40002188	D_INT_DIS_6 INT_DIS_6	(7) RS_DEC_OUT_RDY	–	N/A	RS_DEC_OUT_RDY interrupt disable
		(6) RS_DEC_IN_RDY	–	N/A	RS_DEC_IN_RDY interrupt disable
		(5) RS_ENC_OUT_RDY	–	N/A	RS_ENC_OUT_RDY interrupt disable
		(4) RS_ENC_IN_RDY	–	N/A	RS_ENC_IN_RDY interrupt disable
		(3) G722_DEC_OUT_RDY	–	N/A	G722_DEC_OUT_RDY interrupt disable
		(2) G722_DEC_IN_RDY	–	N/A	G722_DEC_IN_RDY interrupt disable
		(1) G722_ENC_OUT_RDY	–	N/A	G722_ENC_OUT_RDY interrupt disable
		(0) G722_ENC_IN_RDY	–	N/A	G722_ENC_IN_RDY interrupt disable
P:0x0863 0x4000218C	D_INT_DIS_7 INT_DIS_7	(7) FIFO_7	–	N/A	FIFO_7 interrupt disable

Address	Register Name	Register Write	Register Read	Default	Description
		(6) FIFO_6	–	N/A	FIFO_6 interrupt disable
		(5) FIFO_5	–	N/A	FIFO_5 interrupt disable
		(4) FIFO_4	–	N/A	FIFO_4 interrupt disable
		(3) FIFO_3	–	N/A	FIFO_3 interrupt disable
		(2) FIFO_2	–	N/A	FIFO_2 interrupt disable
		(1) FIFO_1	–	N/A	FIFO_1 interrupt disable
		(0) FIFO_0	–	N/A	FIFO_0 interrupt disable
P:0x0864	D_INT_DIS_8	(7) HEAR_7	–	N/A	HEAR_7 interrupt disable
0x40002190	INT_DIS_8				
		(6) HEAR_6	–	N/A	HEAR_6 interrupt disable
		(5) HEAR_5	–	N/A	HEAR_5 interrupt disable
		(4) HEAR_4	–	N/A	HEAR_4 interrupt disable
		(3) HEAR_3	–	N/A	HEAR_3 interrupt disable
		(2) HEAR_2	–	N/A	HEAR_2 interrupt disable
		(1) HEAR_1	–	N/A	HEAR_1 interrupt disable
		(0) HEAR_0	–	N/A	HEAR_0 interrupt disable
P:0x0865	D_INT_DIS_9	(7) DSP_7	–	N/A	DSP_7 interrupt disable
0x40002194	INT_DIS_9				
		(6) DSP_6	–	N/A	DSP_6 interrupt disable
		(5) DSP_5	–	N/A	DSP_5 interrupt disable
		(4) DSP_4	–	N/A	DSP_4 interrupt disable
		(3) DSP_3	–	N/A	DSP_3 interrupt disable

Address	Register Name	Register Write	Register Read	Default	Description
		(2) DSP_2	–	N/A	DSP_2 interrupt disable
		(1) DSP_1	–	N/A	DSP_1 interrupt disable
		(0) DSP_0	–	N/A	DSP_0 interrupt disable
P:0x0866 0x40002198	D_INT_DIS_10 INT_DIS_10	(7) CM3_7	–	N/A	CM3_7 interrupt disable
		(6) CM3_6	–	N/A	CM3_6 interrupt disable
		(5) CM3_5	–	N/A	CM3_5 interrupt disable
		(4) CM3_4	–	N/A	CM3_4 interrupt disable
		(3) CM3_3	–	N/A	CM3_3 interrupt disable
		(2) CM3_2	–	N/A	CM3_2 interrupt disable
		(1) CM3_1	–	N/A	CM3_1 interrupt disable
		(0) CM3_0	–	N/A	CM3_0 interrupt disable
P:0x0867 0x4000219C	D_INT_DIS_11 INT_DIS_11	(5) RS_COM	–	N/A	RS_COM interrupt disable
		(4) G722_COM	–	N/A	G722_COM interrupt disable
		(3) MEM_ERROR	–	N/A	MEM_ERROR interrupt disable
		(2) LSAD	–	N/A	LSAD interrupt disable
		(1) CLK_DET	–	N/A	CLK_DET interrupt disable
		(0) NNA	–	N/A	NNA interrupt disable
P:0x0868 0x400021A0	D_INT_ACK_0 INT_ACK_0	(7) TIMER_3	–	N/A	TIMER_3 interrupt acknowledge
		(6) TIMER_2	–	N/A	TIMER_2 interrupt acknowledge

Address	Register Name	Register Write	Register Read	Default	Description
		(5) TIMER_1	–	N/A	TIMER_1 interrupt acknowledge
		(4) TIMER_0	–	N/A	TIMER_0 interrupt acknowledge
		(3) AUDIOSINK_1	–	N/A	AUDIOSINK_1 interrupt acknowledge
		(2) AUDIOSINK_0	–	N/A	AUDIOSINK_0 interrupt acknowledge
		(1) LSAD_BATMON	–	N/A	LSAD_BATMON interrupt acknowledge
		(0) WATCHDOG	–	N/A	WATCHDOG interrupt acknowledge
P:0x0869 0x400021A4	D_INT_ACK_1 INT_ACK_1	(7) DMA_7	–	N/A	DMA_7 interrupt acknowledge
		(6) DMA_6	–	N/A	DMA_6 interrupt acknowledge
		(5) DMA_5	–	N/A	DMA_5 interrupt acknowledge
		(4) DMA_4	–	N/A	DMA_4 interrupt acknowledge
		(3) DMA_3	–	N/A	DMA_3 interrupt acknowledge
		(2) DMA_2	–	N/A	DMA_2 interrupt acknowledge
		(1) DMA_1	–	N/A	DMA_1 interrupt acknowledge
		(0) DMA_0	–	N/A	DMA_0 interrupt acknowledge
P:0x086A 0x400021A8	D_INT_ACK_2 INT_ACK_2	(7) DIO_7	–	N/A	DIO_7 interrupt acknowledge
		(6) DIO_6	–	N/A	DIO_6 interrupt acknowledge
		(5) DIO_5	–	N/A	DIO_5 interrupt acknowledge
		(4) DIO_4	–	N/A	DIO_4 interrupt acknowledge
		(3) DIO_3	–	N/A	DIO_3 interrupt acknowledge
		(2) DIO_2	–	N/A	DIO_2 interrupt acknowledge

Address	Register Name	Register Write	Register Read	Default	Description
		(1) DIO_1	–	N/A	DIO_1 interrupt acknowledge
		(0) DIO_0	–	N/A	DIO_0 interrupt acknowledge
P:0x086B 0x400021AC	D_INT_ACK_3 INT_ACK_3	(7) SPI_0_TX	–	N/A	SPI_0_TX interrupt acknowledge
		(6) SPI_0_RX	–	N/A	SPI_0_RX interrupt acknowledge
		(5) SPI_0_COM	–	N/A	SPI_0_COM interrupt acknowledge
		(4) D_I2C	–	N/A	D_I2C interrupt acknowledge
		(3) I2C_1	–	N/A	I2C_1 interrupt acknowledge
		(2) I2C_0	–	N/A	I2C_0 interrupt acknowledge
		(1) I3C_1	–	N/A	I3C_1 interrupt acknowledge
		(0) I3C_0	–	N/A	I3C_0 interrupt acknowledge
P:0x086C 0x400021B0	D_INT_ACK_4 INT_ACK_4	(7) FENG	–	N/A	FENG interrupt acknowledge
		(6) UART_0_TX	–	N/A	UART_0_TX interrupt acknowledge
		(5) UART_0_RX	–	N/A	UART_0_RX interrupt acknowledge
		(4) UART_0_ERROR	–	N/A	UART_0_ERROR interrupt acknowledge
		(3) EMMC	–	N/A	EMMC interrupt acknowledge
		(2) SPI_1_TX	–	N/A	SPI_1_TX interrupt acknowledge
		(1) SPI_1_RX	–	N/A	SPI_1_RX interrupt acknowledge
		(0) SPI_1_COM	–	N/A	SPI_1_COM interrupt acknowledge
P:0x086D 0x400021B4	D_INT_ACK_5 INT_ACK_5	(7) ASRC_IDLE	–	N/A	ASRC_IDLE interrupt acknowledge

Address	Register Name	Register Write	Register Read	Default	Description
		(6) ASRC_OUT	–	N/A	ASRC_OUT interrupt acknowledge
		(5) ASRC_IN	–	N/A	ASRC_IN interrupt acknowledge
		(4) ASRC_ERROR	–	N/A	ASRC_ERROR interrupt acknowledge
		(3) PCM_1_RX_TX	–	N/A	PCM_1_RX_TX interrupt acknowledge
		(2) PCM_1_ERROR	–	N/A	PCM_1_ERROR interrupt acknowledge
		(1) PCM_0_RX_TX	–	N/A	PCM_0_RX_TX interrupt acknowledge
		(0) PCM_0_ERROR	–	N/A	PCM_0_ERROR interrupt acknowledge
P:0x086E 0x400021B8	D_INT_ACK_6 INT_ACK_6	(7) RS_DEC_OUT_RDY	–	N/A	RS_DEC_OUT_RDY interrupt acknowledge
		(6) RS_DEC_IN_RDY	–	N/A	RS_DEC_IN_RDY interrupt acknowledge
		(5) RS_ENC_OUT_RDY	–	N/A	RS_ENC_OUT_RDY interrupt acknowledge
		(4) RS_ENC_IN_RDY	–	N/A	RS_ENC_IN_RDY interrupt acknowledge
		(3) G722_DEC_OUT_RDY	–	N/A	G722_DEC_OUT_RDY interrupt acknowledge
		(2) G722_DEC_IN_RDY	–	N/A	G722_DEC_IN_RDY interrupt acknowledge
		(1) G722_ENC_OUT_RDY	–	N/A	G722_ENC_OUT_RDY interrupt acknowledge
		(0) G722_ENC_IN_RDY	–	N/A	G722_ENC_IN_RDY interrupt acknowledge
P:0x086F 0x400021BC	D_INT_ACK_7 INT_ACK_7	(7) FIFO_7	–	N/A	FIFO_7 interrupt acknowledge
		(6) FIFO_6	–	N/A	FIFO_6 interrupt acknowledge

Address	Register Name	Register Write	Register Read	Default	Description
		(5) FIFO_5	–	N/A	FIFO_5 interrupt acknowledge
		(4) FIFO_4	–	N/A	FIFO_4 interrupt acknowledge
		(3) FIFO_3	–	N/A	FIFO_3 interrupt acknowledge
		(2) FIFO_2	–	N/A	FIFO_2 interrupt acknowledge
		(1) FIFO_1	–	N/A	FIFO_1 interrupt acknowledge
		(0) FIFO_0	–	N/A	FIFO_0 interrupt acknowledge
P:0x0870 0x400021C0	D_INT_ACK_8 INT_ACK_8	(7) HEAR_7	–	N/A	HEAR_7 interrupt acknowledge
		(6) HEAR_6	–	N/A	HEAR_6 interrupt acknowledge
		(5) HEAR_5	–	N/A	HEAR_5 interrupt acknowledge
		(4) HEAR_4	–	N/A	HEAR_4 interrupt acknowledge
		(3) HEAR_3	–	N/A	HEAR_3 interrupt acknowledge
		(2) HEAR_2	–	N/A	HEAR_2 interrupt acknowledge
		(1) HEAR_1	–	N/A	HEAR_1 interrupt acknowledge
		(0) HEAR_0	–	N/A	HEAR_0 interrupt acknowledge
P:0x0871 0x400021C4	D_INT_ACK_9 INT_ACK_9	(7) DSP_7	–	N/A	DSP_7 interrupt acknowledge
		(6) DSP_6	–	N/A	DSP_6 interrupt acknowledge
		(5) DSP_5	–	N/A	DSP_5 interrupt acknowledge
		(4) DSP_4	–	N/A	DSP_4 interrupt acknowledge
		(3) DSP_3	–	N/A	DSP_3 interrupt acknowledge
		(2) DSP_2	–	N/A	DSP_2 interrupt acknowledge

Address	Register Name	Register Write	Register Read	Default	Description
		(1) DSP_1	–	N/A	DSP_1 interrupt acknowledge
		(0) DSP_0	–	N/A	DSP_0 interrupt acknowledge
P:0x0872 0x400021C8	D_INT_ACK_10 INT_ACK_10	(7) CM3_7	–	N/A	CM3_7 interrupt acknowledge
		(6) CM3_6	–	N/A	CM3_6 interrupt acknowledge
		(5) CM3_5	–	N/A	CM3_5 interrupt acknowledge
		(4) CM3_4	–	N/A	CM3_4 interrupt acknowledge
		(3) CM3_3	–	N/A	CM3_3 interrupt acknowledge
		(2) CM3_2	–	N/A	CM3_2 interrupt acknowledge
		(1) CM3_1	–	N/A	CM3_1 interrupt acknowledge
		(0) CM3_0	–	N/A	CM3_0 interrupt acknowledge
P:0x0873 0x400021CC	D_INT_ACK_11 INT_ACK_11	(5) RS_COM	–	N/A	RS_COM interrupt acknowledge
		(4) G722_COM	–	N/A	G722_COM interrupt acknowledge
		(3) MEM_ERROR	–	N/A	MEM_ERROR interrupt acknowledge
		(2) LSAD	–	N/A	LSAD interrupt acknowledge
		(1) CLK_DET	–	N/A	CLK_DET interrupt acknowledge
		(0) NNA	–	N/A	NNA interrupt acknowledge
P:0x0874 0x400021D0	D_INT_PRIO_0 INT_PRIO_0	(7) TIMER_3	(7) TIMER_3	0x0	TIMER_3 interrupt priority
		(6) TIMER_2	(6) TIMER_2	0x0	TIMER_2 interrupt priority
		(5) TIMER_1	(5) TIMER_1	0x0	TIMER_1 interrupt priority

Address	Register Name	Register Write	Register Read	Default	Description
		(4) TIMER_0	(4) TIMER_0	0x0	TIMER_0 interrupt priority
		(3) AUDIOSINK_1	(3) AUDIOSINK_1	0x0	AUDIOSINK_1 interrupt priority
		(2) AUDIOSINK_0	(2) AUDIOSINK_0	0x0	AUDIOSINK_0 interrupt priority
		(1) LSAD_BATMON	(1) LSAD_BATMON	0x0	LSAD_BATMON interrupt priority
		(0) WATCHDOG	(0) WATCHDOG	0x0	WATCHDOG interrupt priority
P:0x0875 0x400021D4	D_INT_PRIO_1 INT_PRIO_1	(7) DMA_7	(7) DMA_7	0x0	DMA_7 interrupt priority
		(6) DMA_6	(6) DMA_6	0x0	DMA_6 interrupt priority
		(5) DMA_5	(5) DMA_5	0x0	DMA_5 interrupt priority
		(4) DMA_4	(4) DMA_4	0x0	DMA_4 interrupt priority
		(3) DMA_3	(3) DMA_3	0x0	DMA_3 interrupt priority
		(2) DMA_2	(2) DMA_2	0x0	DMA_2 interrupt priority
		(1) DMA_1	(1) DMA_1	0x0	DMA_1 interrupt priority
		(0) DMA_0	(0) DMA_0	0x0	DMA_0 interrupt priority
P:0x0876 0x400021D8	D_INT_PRIO_2 INT_PRIO_2	(7) DIO_7	(7) DIO_7	0x0	DIO_7 interrupt priority
		(6) DIO_6	(6) DIO_6	0x0	DIO_6 interrupt priority
		(5) DIO_5	(5) DIO_5	0x0	DIO_5 interrupt priority
		(4) DIO_4	(4) DIO_4	0x0	DIO_4 interrupt priority
		(3) DIO_3	(3) DIO_3	0x0	DIO_3 interrupt priority
		(2) DIO_2	(2) DIO_2	0x0	DIO_2 interrupt priority
		(1) DIO_1	(1) DIO_1	0x0	DIO_1 interrupt priority

Address	Register Name	Register Write	Register Read	Default	Description
		(0) DIO_0	(0) DIO_0	0x0	DIO_0 interrupt priority
P:0x0877	D_INT_PRIO_3	(7) SPI_0_TX	(7) SPI_0_TX	0x0	SPI_0_TX interrupt priority
0x400021DC	INT_PRIO_3				
		(6) SPI_0_RX	(6) SPI_0_RX	0x0	SPI_0_RX interrupt priority
		(5) SPI_0_COM	(5) SPI_0_COM	0x0	SPI_0_COM interrupt priority
		(4) D_I2C	(4) D_I2C	0x0	D_I2C interrupt priority
		(3) I2C_1	(3) I2C_1	0x0	I2C_1 interrupt priority
		(2) I2C_0	(2) I2C_0	0x0	I2C_0 interrupt priority
		(1) I3C_1	(1) I3C_1	0x0	I3C_1 interrupt priority
		(0) I3C_0	(0) I3C_0	0x0	I3C_0 interrupt priority
P:0x0878	D_INT_PRIO_4	(7) FENG	(7) FENG	0x0	FENG interrupt priority
0x400021E0	INT_PRIO_4				
		(6) UART_0_TX	(6) UART_0_TX	0x0	UART_0_TX interrupt priority
		(5) UART_0_RX	(5) UART_0_RX	0x0	UART_0_RX interrupt priority
		(4) UART_0_ERROR	(4) UART_0_ERROR	0x0	UART_0_ERROR interrupt priority
		(3) EMMC	(3) EMMC	0x0	EMMC interrupt priority
		(2) SPI_1_TX	(2) SPI_1_TX	0x0	SPI_1_TX interrupt priority
		(1) SPI_1_RX	(1) SPI_1_RX	0x0	SPI_1_RX interrupt priority
		(0) SPI_1_COM	(0) SPI_1_COM	0x0	SPI_1_COM interrupt priority
P:0x0879	D_INT_PRIO_5	(7) ASRC_IDLE	(7) ASRC_IDLE	0x0	ASRC_IDLE interrupt priority
0x400021E4	INT_PRIO_5				
		(6) ASRC_OUT	(6) ASRC_OUT	0x0	ASRC_OUT interrupt priority

Address	Register Name	Register Write	Register Read	Default	Description
		(5) ASRC_IN	(5) ASRC_IN	0x0	ASRC_IN interrupt priority
		(4) ASRC_ERROR	(4) ASRC_ERROR	0x0	ASRC_ERROR interrupt priority
		(3) PCM_1_RX_TX	(3) PCM_1_RX_TX	0x0	PCM_1_RX_TX interrupt priority
		(2) PCM_1_ERROR	(2) PCM_1_ERROR	0x0	PCM_1_ERROR interrupt priority
		(1) PCM_0_RX_TX	(1) PCM_0_RX_TX	0x0	PCM_0_RX_TX interrupt priority
		(0) PCM_0_ERROR	(0) PCM_0_ERROR	0x0	PCM_0_ERROR interrupt priority
P:0x087A 0x400021E8	D_INT_PRIO_6 INT_PRIO_6	(7) RS_DEC_OUT_RDY	(7) RS_DEC_OUT_RDY	0x0	RS_DEC_OUT_RDY interrupt priority
		(6) RS_DEC_IN_RDY	(6) RS_DEC_IN_RDY	0x0	RS_DEC_IN_RDY interrupt priority
		(5) RS_ENC_OUT_RDY	(5) RS_ENC_OUT_RDY	0x0	RS_ENC_OUT_RDY interrupt priority
		(4) RS_ENC_IN_RDY	(4) RS_ENC_IN_RDY	0x0	RS_ENC_IN_RDY interrupt priority
		(3) G722_DEC_OUT_RDY	(3) G722_DEC_OUT_RDY	0x0	G722_DEC_OUT_RDY interrupt priority
		(2) G722_DEC_IN_RDY	(2) G722_DEC_IN_RDY	0x0	G722_DEC_IN_RDY interrupt priority
		(1) G722_ENC_OUT_RDY	(1) G722_ENC_OUT_RDY	0x0	G722_ENC_OUT_RDY interrupt priority
		(0) G722_ENC_IN_RDY	(0) G722_ENC_IN_RDY	0x0	G722_ENC_IN_RDY interrupt priority
P:0x087B 0x400021EC	D_INT_PRIO_7 INT_PRIO_7	(7) FIFO_7	(7) FIFO_7	0x0	FIFO_7 interrupt priority
		(6) FIFO_6	(6) FIFO_6	0x0	FIFO_6 interrupt priority
		(5) FIFO_5	(5) FIFO_5	0x0	FIFO_5 interrupt priority
		(4) FIFO_4	(4) FIFO_4	0x0	FIFO_4 interrupt priority
		(3) FIFO_3	(3) FIFO_3	0x0	FIFO_3 interrupt priority

Address	Register Name	Register Write	Register Read	Default	Description
		(2) FIFO_2	(2) FIFO_2	0x0	FIFO_2 interrupt priority
		(1) FIFO_1	(1) FIFO_1	0x0	FIFO_1 interrupt priority
		(0) FIFO_0	(0) FIFO_0	0x0	FIFO_0 interrupt priority
P:0x087C 0x400021F0	D_INT_PRIO_8 INT_PRIO_8	(7) HEAR_7	(7) HEAR_7	0x0	HEAR_7 interrupt priority
		(6) HEAR_6	(6) HEAR_6	0x0	HEAR_6 interrupt priority
		(5) HEAR_5	(5) HEAR_5	0x0	HEAR_5 interrupt priority
		(4) HEAR_4	(4) HEAR_4	0x0	HEAR_4 interrupt priority
		(3) HEAR_3	(3) HEAR_3	0x0	HEAR_3 interrupt priority
		(2) HEAR_2	(2) HEAR_2	0x0	HEAR_2 interrupt priority
		(1) HEAR_1	(1) HEAR_1	0x0	HEAR_1 interrupt priority
		(0) HEAR_0	(0) HEAR_0	0x0	HEAR_0 interrupt priority
P:0x087D 0x400021F4	D_INT_PRIO_9 INT_PRIO_9	(7) DSP_7	(7) DSP_7	0x0	DSP_7 interrupt priority
		(6) DSP_6	(6) DSP_6	0x0	DSP_6 interrupt priority
		(5) DSP_5	(5) DSP_5	0x0	DSP_5 interrupt priority
		(4) DSP_4	(4) DSP_4	0x0	DSP_4 interrupt priority
		(3) DSP_3	(3) DSP_3	0x0	DSP_3 interrupt priority
		(2) DSP_2	(2) DSP_2	0x0	DSP_2 interrupt priority
		(1) DSP_1	(1) DSP_1	0x0	DSP_1 interrupt priority
		(0) DSP_0	(0) DSP_0	0x0	DSP_0 interrupt priority
P:0x087E 0x400021F8	D_INT_PRIO_10 INT_PRIO_10	(7) CM3_7	(7) CM3_7	0x0	CM3_7 interrupt priority

Address	Register Name	Register Write	Register Read	Default	Description
		(6) CM3_6	(6) CM3_6	0x0	CM3_6 interrupt priority
		(5) CM3_5	(5) CM3_5	0x0	CM3_5 interrupt priority
		(4) CM3_4	(4) CM3_4	0x0	CM3_4 interrupt priority
		(3) CM3_3	(3) CM3_3	0x0	CM3_3 interrupt priority
		(2) CM3_2	(2) CM3_2	0x0	CM3_2 interrupt priority
		(1) CM3_1	(1) CM3_1	0x0	CM3_1 interrupt priority
		(0) CM3_0	(0) CM3_0	0x0	CM3_0 interrupt priority
P:0x087F	D_INT_PRIO_11	(5) RS_COM	(5) RS_COM	0x0	RS_COM interrupt priority
0x400021FC	INT_PRIO_11				
		(4) G722_COM	(4) G722_COM	0x0	G722_COM interrupt priority
		(3) MEM_ERROR	(3) MEM_ERROR	0x0	MEM_ERROR interrupt priority
		(2) LSAD	(2) LSAD	0x0	LSAD interrupt priority
		(1) CLK_DET	(1) CLK_DET	0x0	CLK_DET interrupt priority
		(0) NNA	(0) NNA	0x0	NNA interrupt priority

A.12 CLOCK CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0x08C0 0x40002300	D_CLK_CFG_0 CLK_CFG_0	(5:4) SYSCLK_SEL	(5:4) SYSCLK_SEL	0x0	Controls the functionality of SYSCLK
		(3:0) EXTCLK_PRESCALE	(3:0) EXTCLK_PRESCALE	0x0	Prescale value for the input clock from pad EXTCLK (1 to 16 in steps of 1)
P:0x08C1 0x40002304	D_CLK_CFG_1 CLK_CFG_1	(24) UARTCLK_SRC	(24) UARTCLK_SRC	0x0	Indicates the UART clock source (ADCCLK or SYSCLK)
		(23) SLOWCLK_SRC	(23) SLOWCLK_SRC	0x0	Indicates the slow clock source (SYSCLK or ADCCLK)
		(22:16) SLOWCLK_PRESCALE	(22:16) SLOWCLK_PRESCALE	0x5	Prescale value for the slow clock (1 to 128 in steps of 1)
		(13:8) SDMCLK_PRESCALE	(13:8) SDMCLK_PRESCALE	0x0	Prescale value for the SDM clock (1 to 64 in steps of 1)
		(7) ADCCLK_SRC	(7) ADCCLK_SRC	0x0	Indicates the ADC clock source (SYSCLK or SDMCLK)
		(4:0) ADCCLK_PRESCALE	(4:0) ADCCLK_PRESCALE	0x0	Prescale value for the ADC clock (1 to 32 in steps of 1)
P:0x08C2 0x40002308	D_CLK_CFG_2 CLK_CFG_2	(15) UCLK_SRC	(15) UCLK_SRC	0x0	Indicates the user clock source (SYSCLK or ADCCLK)
		(11:0) UCLK_PRESCALE	(11:0) UCLK_PRESCALE	0x0	Prescale value for the user clock (1 to 4096 in steps of 1)
P:0x08C4 0x40002310	D_CLK_CFG_CPA CLK_CFG_CPA	(5:0) CP_VDDA_CLK_PRESCALE	(5:0) CP_VDDA_CLK_PRESCALE	0x3	Prescale the clock for the VDDA charge pump (1 to 64 in steps of 1). For an optimal performance of VDDA charge pump use an even prescaler value.

Address	Register Name	Register Write	Register Read	Default	Description
P:0x08C5 0x40002314	D_CLK_CFG_CPIF CLK_CFG_CPIF	(5:0) CP_VDDIF_CLK_ PRESCALE	(5:0) CP_VDDIF_CLK_ PRESCALE	0x3	Prescale the clock for the VDDIF charge pump (1 to 64 in steps of 1). For an optimal performance of VDDIF charge pump use an even prescaler value.
P:0x08C6 0x40002318	D_CLK_CFG_CPCM CLK_CFG_CPCM	(7) CP_VDDM_CLK_PHASE	(7) CP_VDDM_CLK_ PHASE	0x0	VDDM charge pump clock phase
		(6) CP_VDDC_CLK_PHASE	(6) CP_VDDC_CLK_ PHASE	0x0	VDDC charge pump clock phase
		(5) CP_VDDCM_CLK_SRC	(5) CP_VDDCM_CLK_SRC	0x0	Indicates the VDDC/VDDM charge pump clock source (SYSClk or SDMCLK)
		(4:0) CP_VDDCM_CLK_ PRESCALE	(4:0) CP_VDDCM_CLK_ PRESCALE	0x0	Prescale the clock for the VDDC/VDDM charge pump (1 to 32 in steps of 1)
P:0x08C7 0x4000231C	D_CLK_ADCCLK_THROTTLING CLK_ADCCLK_THROTTLING	(4:0) PRESCALE	(4:0) PRESCALE	0x0	Future clock throttling value for ADCCLK_PRESCALE
P:0x08C8 0x40002320	D_CLK_SDMCLK_THROTTLING CLK_SDMCLK_THROTTLING	(5:0) PRESCALE	(5:0) PRESCALE	0x0	Future clock throttling value for SDMCLK_PRESCALE
P:0x08C9 0x40002324	D_CLK_SLOWCLK_THROTTLING CLK_SLOWCLK_THROTTLING	(6:0) PRESCALE	(6:0) PRESCALE	0x0	Future clock throttling value for SLOWCLK_PRESCALE
P:0x08CA 0x40002328	D_CLK_THROTTLING_CTRL CLK_THROTTLING_CTRL	(3:2) APPLY_SRC	(3:2) APPLY_SRC	0x0	Clock throttling apply source
		–	(1:0) STATUS	0x0	Clock throttling apply status
		(1:0) APPLY	–	N/A	Clock throttling apply command

Address	Register Name	Register Write	Register Read	Default	Description
P:0x08CB 0x4000232C	D_CLK_MULT_THROTTLING_CFG CLK_MULT_THROTTLING_CFG	(3) ADCCLK_EBL	(3) ADCCLK_EBL	0x0	ADC clock CCO multiplier based clock throttling enable
		(2) UCLK_EBL	(2) UCLK_EBL	0x0	User clock CCO multiplier based clock throttling enable
		(1) SDMCLK_EBL	(1) SDMCLK_EBL	0x0	SDM clock CCO multiplier based clock throttling enable
		(0) SLOWCLK_EBL	(0) SLOWCLK_EBL	0x0	Slow clock CCO multiplier based clock throttling enable
P:0x08CC 0x40002330	D_CLK_THROTTLING_CFG CLK_THROTTLING_CFG	(0) CLK_THROTTLING_ON_OD_ZERO	(0) CLK_THROTTLING_ON_OD_ZERO	0x0	Enable CCO frequency based clock throttling only on OD zero output
P:0x08CD 0x40002334	D_CLK_MULTI_SYNC_CFG CLK_MULTI_SYNC_CFG	(0) SLAVE_CFG	(0) SLAVE_CFG	0x0	Multi-chip slave configuration
P:0x08CE 0x40002338	D_CLK_EXTCLK_DET_CFG CLK_EXTCLK_DET_CFG	(6:5) CLK_DET_INT_SEL	(6:5) CLK_DET_INT_SEL	0x0	Clock detector interrupt configuration
		(0) CLK_DET_ENABLE	(0) CLK_DET_ENABLE	0x0	Clock detector enable/disable
P:0x08CF 0x4000233C	D_CLK_EXTCLK_DET_STATUS CLK_EXTCLK_DET_STATUS	-	(9) CLK_DET_INT_STATUS	0x0	Clock detector interrupt status
		-	(8) CLK_DET_STATUS	0x0	Clock detector status
		(0) CLK_DET_INT_STATUS_CLEAR	-	N/A	Clear clock detector interrupt status

A.13 IOC CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0x0900 0x40002400	D_IOC0_INPUT_CFG IOC0_INPUT_CFG	(29:24) IN3_STORE	(29:24) IN3_STORE	0x3F	Select where to store IN3
		(21:16) IN2_STORE	(21:16) IN2_STORE	0x3F	Select where to store IN2
		(13:8) IN1_STORE	(13:8) IN1_STORE	0x3F	Select where to store IN1
		(5:0) IN0_STORE	(5:0) IN0_STORE	0x3F	Select where to store IN0
P:0x0901 0x40002404	D_IOC0_INPUT_PCM_CFG IOC0_INPUT_PCM_CFG	(13:8) PCM_RX1_STORE	(13:8) PCM_RX1_STORE	0x3F	Select where to store PCM RX1
		(5:0) PCM_RX0_STORE	(5:0) PCM_RX0_STORE	0x3F	Select where to store PCM RX0
P:0x0902 0x40002408	D_IOC0_FIFO_ACCESS_CFG IOC0_FIFO_ACCESS_CFG	(31) FIFO_B1_7_DBL_ACC	(31) FIFO_B1_7_DBL_ACC	0x0	Defines FIFO B1 7 access type
		(30) FIFO_B1_6_DBL_ACC	(30) FIFO_B1_6_DBL_ACC	0x0	Defines FIFO B1 6 access type
		(29) FIFO_B1_5_DBL_ACC	(29) FIFO_B1_5_DBL_ACC	0x0	Defines FIFO B1 5 access type
		(28) FIFO_B1_4_DBL_ACC	(28) FIFO_B1_4_DBL_ACC	0x0	Defines FIFO B1 4 access type
		(27) FIFO_B1_3_DBL_ACC	(27) FIFO_B1_3_DBL_ACC	0x0	Defines FIFO B1 3 access type
		(26) FIFO_B1_2_DBL_ACC	(26) FIFO_B1_2_DBL_ACC	0x0	Defines FIFO B1 2 access type
		(25) FIFO_B1_1_DBL_ACC	(25) FIFO_B1_1_DBL_ACC	0x0	Defines FIFO B1 1 access type
		(24) FIFO_B1_0_DBL_ACC	(24) FIFO_B1_0_DBL_ACC	0x0	Defines FIFO B1 0 access type

Address	Register Name	Register Write	Register Read	Default	Description
		(23) FIFO_B0_7_DBL_ACC	(23) FIFO_B0_7_DBL_ACC	0x0	Defines FIFO B0 7 access type
		(22) FIFO_B0_6_DBL_ACC	(22) FIFO_B0_6_DBL_ACC	0x0	Defines FIFO B0 6 access type
		(21) FIFO_B0_5_DBL_ACC	(21) FIFO_B0_5_DBL_ACC	0x0	Defines FIFO B0 5 access type
		(20) FIFO_B0_4_DBL_ACC	(20) FIFO_B0_4_DBL_ACC	0x0	Defines FIFO B0 4 access type
		(19) FIFO_B0_3_DBL_ACC	(19) FIFO_B0_3_DBL_ACC	0x0	Defines FIFO B0 3 access type
		(18) FIFO_B0_2_DBL_ACC	(18) FIFO_B0_2_DBL_ACC	0x0	Defines FIFO B0 2 access type
		(17) FIFO_B0_1_DBL_ACC	(17) FIFO_B0_1_DBL_ACC	0x0	Defines FIFO B0 1 access type
		(16) FIFO_B0_0_DBL_ACC	(16) FIFO_B0_0_DBL_ACC	0x0	Defines FIFO B0 0 access type
		(15) FIFO_A1_7_DBL_ACC	(15) FIFO_A1_7_DBL_ACC	0x0	Defines FIFO A1 7 access type
		(14) FIFO_A1_6_DBL_ACC	(14) FIFO_A1_6_DBL_ACC	0x0	Defines FIFO A1 6 access type
		(13) FIFO_A1_5_DBL_ACC	(13) FIFO_A1_5_DBL_ACC	0x0	Defines FIFO A1 5 access type
		(12) FIFO_A1_4_DBL_ACC	(12) FIFO_A1_4_DBL_ACC	0x0	Defines FIFO A1 4 access type
		(11) FIFO_A1_3_DBL_ACC	(11) FIFO_A1_3_DBL_ACC	0x0	Defines FIFO A1 3 access type
		(10) FIFO_A1_2_DBL_ACC	(10) FIFO_A1_2_DBL_ACC	0x0	Defines FIFO A1 2 access type

Address	Register Name	Register Write	Register Read	Default	Description
		(9) FIFO_A1_1_DBL_ACC	(9) FIFO_A1_1_DBL_ACC	0x0	Defines FIFO A1 1 access type
		(8) FIFO_A1_0_DBL_ACC	(8) FIFO_A1_0_DBL_ACC	0x0	Defines FIFO A1 0 access type
		(7) FIFO_A0_7_DBL_ACC	(7) FIFO_A0_7_DBL_ACC	0x0	Defines FIFO A0 7 access type
		(6) FIFO_A0_6_DBL_ACC	(6) FIFO_A0_6_DBL_ACC	0x0	Defines FIFO A0 6 access type
		(5) FIFO_A0_5_DBL_ACC	(5) FIFO_A0_5_DBL_ACC	0x0	Defines FIFO A0 5 access type
		(4) FIFO_A0_4_DBL_ACC	(4) FIFO_A0_4_DBL_ACC	0x0	Defines FIFO A0 4 access type
		(3) FIFO_A0_3_DBL_ACC	(3) FIFO_A0_3_DBL_ACC	0x0	Defines FIFO A0 3 access type
		(2) FIFO_A0_2_DBL_ACC	(2) FIFO_A0_2_DBL_ACC	0x0	Defines FIFO A0 2 access type
		(1) FIFO_A0_1_DBL_ACC	(1) FIFO_A0_1_DBL_ACC	0x0	Defines FIFO A0 1 access type
		(0) FIFO_A0_0_DBL_ACC	(0) FIFO_A0_0_DBL_ACC	0x0	Defines FIFO A0 0 access type
P:0x0904 0x40002410	D_IOC0_OUTPUT_CFG IOC0_OUTPUT_CFG	(29:24) PCM_TX1_LOAD	(29:24) PCM_TX1_LOAD	0x3F	Defines the origin of the audio sample for PCM TX1
		(21:16) PCM_TX0_LOAD	(21:16) PCM_TX0_LOAD	0x3F	Defines the origin of the audio sample for PCM TX0
		(13:8) OUT1_LOAD	(13:8) OUT1_LOAD	0x3F	Defines the origin of the audio sample for OUT1
		(5:0) OUT0_LOAD	(5:0) OUT0_LOAD	0x3F	Defines the origin of the audio sample for

Address	Register Name	Register Write	Register Read	Default	Description
					OUT0
P:0x0910 0x40002440	D_IOC0_IN0_DATA IOC0_IN0_DATA	–	(31:0) IN_DATA	0x0	Latest audio sample from IN0 (m1p23 format, sign extended to 32bits)
P:0x0911 0x40002444	D_IOC0_IN1_DATA IOC0_IN1_DATA	–	(31:0) IN_DATA	0x0	Latest audio sample from IN1 (m1p23 format, sign extended to 32bits)
P:0x0912 0x40002448	D_IOC0_IN2_DATA IOC0_IN2_DATA	–	(31:0) IN_DATA	0x0	Latest audio sample from IN2 (m1p23 format, sign extended to 32bits)
P:0x0913 0x4000244C	D_IOC0_IN3_DATA IOC0_IN3_DATA	–	(31:0) IN_DATA	0x0	Latest audio sample from IN3 (m1p23 format, sign extended to 32bits)
P:0x0914 0x40002450	D_IOC0_OUT0_DATA IOC0_OUT0_DATA	(23:0) OUT_DATA	–	N/A	Input sample for output channel 0 (m1p23 format)
P:0x0915 0x40002454	D_IOC0_OUT1_DATA IOC0_OUT1_DATA	(23:0) OUT_DATA	–	N/A	Input sample for output channel 1 (m1p23 format)
P:0x0940 0x40002500	D_IOC1_INPUT_CFG IOC1_INPUT_CFG	(29:24) IN3_STORE	(29:24) IN3_STORE	0x3F	Select where to store IN3
		(21:16) IN2_STORE	(21:16) IN2_STORE	0x3F	Select where to store IN2
		(13:8) IN1_STORE	(13:8) IN1_STORE	0x3F	Select where to store IN1
		(5:0) IN0_STORE	(5:0) IN0_STORE	0x3F	Select where to store IN0
P:0x0941 0x40002504	D_IOC1_INPUT_PCM_CFG IOC1_INPUT_PCM_CFG	(13:8) PCM_RX1_STORE	(13:8) PCM_RX1_STORE	0x3F	Select where to store PCM RX1
		(5:0) PCM_RX0_STORE	(5:0) PCM_RX0_STORE	0x3F	Select where to store PCM RX0
P:0x0942	D_IOC1_FIFO_ACCESS_CFG	(31) FIFO_B1_7_DBL_	(31) FIFO_B1_7_DBL_	0x0	Defines FIFO B1 7 access type

Address	Register Name	Register Write	Register Read	Default	Description
0x40002508	IOC1_FIFO_ACCESS_CFG	ACC	ACC		
		(30) FIFO_B1_6_DBL_ ACC	(30) FIFO_B1_6_DBL_ ACC	0x0	Defines FIFO B1 6 access type
		(29) FIFO_B1_5_DBL_ ACC	(29) FIFO_B1_5_DBL_ ACC	0x0	Defines FIFO B1 5 access type
		(28) FIFO_B1_4_DBL_ ACC	(28) FIFO_B1_4_DBL_ ACC	0x0	Defines FIFO B1 4 access type
		(27) FIFO_B1_3_DBL_ ACC	(27) FIFO_B1_3_DBL_ ACC	0x0	Defines FIFO B1 3 access type
		(26) FIFO_B1_2_DBL_ ACC	(26) FIFO_B1_2_DBL_ ACC	0x0	Defines FIFO B1 2 access type
		(25) FIFO_B1_1_DBL_ ACC	(25) FIFO_B1_1_DBL_ ACC	0x0	Defines FIFO B1 1 access type
		(24) FIFO_B1_0_DBL_ ACC	(24) FIFO_B1_0_DBL_ ACC	0x0	Defines FIFO B1 0 access type
		(23) FIFO_B0_7_DBL_ ACC	(23) FIFO_B0_7_DBL_ ACC	0x0	Defines FIFO B0 7 access type
		(22) FIFO_B0_6_DBL_ ACC	(22) FIFO_B0_6_DBL_ ACC	0x0	Defines FIFO B0 6 access type
		(21) FIFO_B0_5_DBL_ ACC	(21) FIFO_B0_5_DBL_ ACC	0x0	Defines FIFO B0 5 access type
		(20) FIFO_B0_4_DBL_ ACC	(20) FIFO_B0_4_DBL_ ACC	0x0	Defines FIFO B0 4 access type
		(19) FIFO_B0_3_DBL_ ACC	(19) FIFO_B0_3_DBL_ ACC	0x0	Defines FIFO B0 3 access type
		(18) FIFO_B0_2_DBL_ ACC	(18) FIFO_B0_2_DBL_ ACC	0x0	Defines FIFO B0 2 access type
		(17) FIFO_B0_1_DBL_ ACC	(17) FIFO_B0_1_DBL_ ACC	0x0	Defines FIFO B0 1 access type

Address	Register Name	Register Write	Register Read	Default	Description
		ACC	ACC		
		(16) FIFO_B0_0_DBL_ACC	(16) FIFO_B0_0_DBL_ACC	0x0	Defines FIFO B0 0 access type
		(15) FIFO_A1_7_DBL_ACC	(15) FIFO_A1_7_DBL_ACC	0x0	Defines FIFO A1 7 access type
		(14) FIFO_A1_6_DBL_ACC	(14) FIFO_A1_6_DBL_ACC	0x0	Defines FIFO A1 6 access type
		(13) FIFO_A1_5_DBL_ACC	(13) FIFO_A1_5_DBL_ACC	0x0	Defines FIFO A1 5 access type
		(12) FIFO_A1_4_DBL_ACC	(12) FIFO_A1_4_DBL_ACC	0x0	Defines FIFO A1 4 access type
		(11) FIFO_A1_3_DBL_ACC	(11) FIFO_A1_3_DBL_ACC	0x0	Defines FIFO A1 3 access type
		(10) FIFO_A1_2_DBL_ACC	(10) FIFO_A1_2_DBL_ACC	0x0	Defines FIFO A1 2 access type
		(9) FIFO_A1_1_DBL_ACC	(9) FIFO_A1_1_DBL_ACC	0x0	Defines FIFO A1 1 access type
		(8) FIFO_A1_0_DBL_ACC	(8) FIFO_A1_0_DBL_ACC	0x0	Defines FIFO A1 0 access type
		(7) FIFO_A0_7_DBL_ACC	(7) FIFO_A0_7_DBL_ACC	0x0	Defines FIFO A0 7 access type
		(6) FIFO_A0_6_DBL_ACC	(6) FIFO_A0_6_DBL_ACC	0x0	Defines FIFO A0 6 access type
		(5) FIFO_A0_5_DBL_ACC	(5) FIFO_A0_5_DBL_ACC	0x0	Defines FIFO A0 5 access type
		(4) FIFO_A0_4_DBL_ACC	(4) FIFO_A0_4_DBL_ACC	0x0	Defines FIFO A0 4 access type
		(3) FIFO_A0_3_DBL_ACC	(3) FIFO_A0_3_DBL_ACC	0x0	Defines FIFO A0 3 access type

Address	Register Name	Register Write	Register Read	Default	Description
			ACC		
		(2) FIFO_A0_2_DBL_ACC	(2) FIFO_A0_2_DBL_ACC	0x0	Defines FIFO A0 2 access type
		(1) FIFO_A0_1_DBL_ACC	(1) FIFO_A0_1_DBL_ACC	0x0	Defines FIFO A0 1 access type
		(0) FIFO_A0_0_DBL_ACC	(0) FIFO_A0_0_DBL_ACC	0x0	Defines FIFO A0 0 access type
P:0x0944 0x40002510	D_IOC1_OUTPUT_CFG IOC1_OUTPUT_CFG	(29:24) PCM_TX1_LOAD	(29:24) PCM_TX1_LOAD	0x3F	Defines the origin of the audio sample for PCM TX1
		(21:16) PCM_TX0_LOAD	(21:16) PCM_TX0_LOAD	0x3F	Defines the origin of the audio sample for PCM TX0
		(13:8) OUT1_LOAD	(13:8) OUT1_LOAD	0x3F	Defines the origin of the audio sample for OUT1
		(5:0) OUT0_LOAD	(5:0) OUT0_LOAD	0x3F	Defines the origin of the audio sample for OUT0
P:0x0950 0x40002540	D_IOC1_IN0_DATA IOC1_IN0_DATA	–	(31:0) IN_DATA	0x0	Latest audio sample from IN0 (m1p23 format, sign extended to 32bits)
P:0x0951 0x40002544	D_IOC1_IN1_DATA IOC1_IN1_DATA	–	(31:0) IN_DATA	0x0	Latest audio sample from IN1 (m1p23 format, sign extended to 32bits)
P:0x0952 0x40002548	D_IOC1_IN2_DATA IOC1_IN2_DATA	–	(31:0) IN_DATA	0x0	Latest audio sample from IN2 (m1p23 format, sign extended to 32bits)

Address	Register Name	Register Write	Register Read	Default	Description
P:0x0953 0x4000254C	D_IOC1_IN3_DATA IOC1_IN3_DATA	–	(31:0) IN_DATA	0x0	Latest audio sample from IN3 (m1p23 format, sign extended to 32bits)
P:0x0954 0x40002550	D_IOC1_OUT0_DATA IOC1_OUT0_DATA	(23:0) OUT_DATA	–	N/A	Input sample for output channel 0 (m1p23 format)
P:0x0955 0x40002554	D_IOC1_OUT1_DATA IOC1_OUT1_DATA	(23:0) OUT_DATA	–	N/A	Input sample for output channel 1 (m1p23 format)

A.14 ANALOG

Address	Register Name	Register Write	Register Read	Default	Description
P:0x0980 0x40002600	ANALOG_OSC_CTRL_0	(15:8) COARSE_FREQ_CTRL	(15:8) COARSE_FREQ_CTRL	0xF	Coarse oscillator frequency programming control
		(7:0) FINE_FREQ_CTRL	(7:0) FINE_FREQ_CTRL	0x80	Fine oscillator frequency programming control
P:0x0981 0x40002604	ANALOG_OSC_CTRL_1	(1:0) MULT	(1:0) MULT	0x0	Oscillator clock multiplier control
P:0x0984-P:0x0986 0x40002610 - 0x40002618	ANALOG_OSC_CTRL_THROTTLING_*	(15:8) COARSE_FREQ_CTRL	(15:8) COARSE_FREQ_CTRL	0x10	Coarse oscillator frequency programming control. If 0x00 or 0x01 is written, the entire register keeps its previous value
		(7:0) FINE_FREQ_CTRL	(7:0) FINE_FREQ_CTRL	0x80	Fine oscillator frequency programming control
P:0x0991 0x40002644	ANALOG_BG_CTRL	(13) BG_VTEMP_ENABLE	(13) BG_VTEMP_ENABLE	0x0	Temperature dependent voltage enable
		(12:8) BG_TEMP_COEF_TRIM	(12:8) BG_TEMP_COEF_TRIM	0x10	Temperature coefficient trimming control
		(4:0) BG_LEVEL	(4:0) BG_LEVEL	0xF	Reference voltage trimming control
P:0x0992 0x40002648	ANALOG_PWR_CTRL	(0) ANALOG_DISABLE	(0) ANALOG_DISABLE	0x0	Analog power down control
P:0x0993 0x4000264C	ANALOG_VMIC_CTRL	(8) VMIC_REG_PS	(8) VMIC_REG_PS	0x0	VMIC regulator power supply
		(7:3) VMIC_REG_TRIM	(7:3) VMIC_REG_TRIM	0xE	VMIC regulator trimming

Address	Register Name	Register Write	Register Read	Default	Description
		(2) VMIC_REG_EN	(2) VMIC_REG_EN	0x0	Enable of VMIC regulator
		(1:0) VMIC_MUX_CTRL	(1:0) VMIC_MUX_CTRL	0x0	Selection of the VMIC output signal
P:0x0994 0x40002650	ANALOG_VDDOD_CTRL	(5:1) VDDOD_REG_TRIM	(5:1) VDDOD_REG_TRIM	0xE	VDDOD regulator trimming
		(0) VDDOD_REG_EN	(0) VDDOD_REG_EN	0x0	Enable of VDDOD regulator
P:0x09A0 0x40002680	ANALOG_CP_VDDA_CTRL	(12) HIGH_POWER	(12) HIGH_POWER	0x1	Select high-power mode
		(11) STANDBY	(11) STANDBY	0x0	Select charge pump clock and reference sources
		(10:6) ITRIM	(10:6) ITRIM	0x12	Current trimming of the analog charge pump comparator
		(5:0) VTRIM	(5:0) VTRIM	0x21	CP output voltage trimming configuration at a step of 6.5 mV
P:0x09A1 0x40002684	ANALOG_CP_VDDIF_CTRL	(14) VDDA_CONNECT	(14) VDDA_CONNECT	0x1	VDDIF to VDDA connection configuration
		(13) HIGH_POWER	(13) HIGH_POWER	0x1	Select high-power mode
		(12) ENABLE	(12) ENABLE	0x1	Enables interface charge pump
		(10:6) ITRIM	(10:6) ITRIM	0x10	Current trimming of the interface charge pump comparator
		(5:0) VTRIM	(5:0) VTRIM	0x23	CP output voltage trimming configuration at a step of 6.5 mV
P:0x09A2 0x40002688	ANALOG_CP_VDDC_TRIM	(13:8) CP_DELTA_TRIM	(13:8) CP_DELTA_TRIM	0xC	Charge pump output voltage trimming in 1.7 mV steps (from -54 mV to 52 mV)
		(7:0) LDO_TRIM	(7:0) LDO_TRIM	0xD0	Linear regulator output voltage trimming

Address	Register Name	Register Write	Register Read	Default	Description
					in 1.7 mV steps
P:0x09A3 0x4000268C	ANALOG_CP_VDDC_CTRL	(6:4) CP_MODE	(6:4) CP_MODE	0x0	Charge pump mode
		(1) AUTO_CTRL	(1) AUTO_CTRL	0x0	Enable automatic charge pump ratio control (requires LSAD to be enabled)
		(0) ENABLE	(0) ENABLE	0x0	Charge pump enable control
P:0x09A4 0x40002690	ANALOG_CP_VDDM_TRIM	(13:8) CP_DELTA_TRIM	(13:8) CP_DELTA_TRIM	0xC	Charge pump output voltage trimming in 1.7 mV steps (from -54 mV to 52 mV)
		(7:0) LDO_TRIM	(7:0) LDO_TRIM	0xD0	Linear regulator output voltage trimming in 1.7 mV steps
P:0x09A5 0x40002694	ANALOG_CP_VDDM_CTRL	(6:4) CP_MODE	(6:4) CP_MODE	0x0	Charge pump mode
		(1) AUTO_CTRL	(1) AUTO_CTRL	0x0	Enable automatic charge pump control (requires LSAD to be enabled)
		(0) ENABLE	(0) ENABLE	0x0	Charge pump enable control
P:0x09A6 0x40002698	ANALOG_CP_VDDCM_REF	(0) VDDCM_REF_SEL	(0) VDDCM_REF_SEL	0x0	Select digital regulators reference selection
P:0x09A9 0x400026A4	ANALOG_CP_VDDCM_CFG	(11:0) AUTO_CTRL	(11:0) AUTO_CTRL	0xA9	Digital charge pump automatic charge pump ratio configuration
P:0x09AA 0x400026A8	ANALOG_CP_VDDC_AUTO_CTRL	(19:16) MIN_MARGIN	(19:16) MIN_MARGIN	0x0	Minimum timing margin

Address	Register Name	Register Write	Register Read	Default	Description
		(3) ENABLE	(3) ENABLE	0x0	VDDC automatic trim enable
P:0x09AB 0x400026AC	ANALOG_CP_VDDC_AUTO_LIMITS	(15:8) MAX_TRIM	(15:8) MAX_TRIM	0x0	Maximum VDDC trimming value (the automatic trim level control will not go above this trimming value)
		(7:0) MIN_TRIM	(7:0) MIN_TRIM	0x0	Minimum VDDC trimming value (the automatic trim level control will not go below this trimming value)

A.15 AUDIO

Address	Register Name	Register Write	Register Read	Default	Description
P:0x09C0-P:0x09C3 0x40002700 - 0x4000270C	AUDIO_ADC_CFG_*	(23) DEM_ENABLE	(23) DEM_ENABLE	0x1	Enables dynamic element matching of the feedback DAC
		(22:20) DAC_MAX_DRIVE	(22:20) DAC_MAX_DRIVE	0x7	Feedback DAC maximum drive (powers of 2)
		(18:16) DAC_MIN_DRIVE	(18:16) DAC_MIN_DRIVE	0x1	Feedback DAC minimum drive (powers of 2)
		(11:8) RIN_TRIM	(11:8) RIN_TRIM	0x7	Trimming of the integrator input resistor (number of parallel 32 kOhm resistors)
		(5:4) COMP_CURRENT	(5:4) COMP_CURRENT	0x2	Trimming of the ADC comparator current (1.0 uA + 0.5 uA steps on VDDA)
		(3:0) ADC_CURRENT	(3:0) ADC_CURRENT	0x8	Trimming of the ADC integrator current (2.5 uA + 1.5 uA steps on VDDA)
P:0x09C4-P:0x09C7 0x40002710 - 0x4000271C	AUDIO_ADC_CTRL_*	(30:28) AIN_RBIA_CONNECT	(30:28) AIN_RBIA_CONNECT	0x0	Analog input biasing connections (1 MOhm resistor)
		(27:16) ADC_MODE	(27:16) ADC_MODE	0x0	ADC operation mode
		(13:12) ADC_REFERENCE_SEL	(13:12) ADC_REFERENCE_SEL	0x0	ADC reference selection
		(11:10) DAC_SRC	(11:10) DAC_SRC	0x0	DAC mode source selection
		(8) DAC_ENABLE	(8) DAC_ENABLE	0x0	Enables DAC mode by driving the feedback DAC
		(6:4) AIN_SEL	(6:4) AIN_SEL	0x0	Select the input applied to ADC

Address	Register Name	Register Write	Register Read	Default	Description
		(1) COMP_ENABLE	(1) COMP_ENABLE	0x1	Enables ADC comparator (ADC must be enabled)
		(0) ADC_ENABLE	(0) ADC_ENABLE	0x0	Enables ADC
P:0x09C8-P:0x09CB 0x40002720 - 0x4000272C	AUDIO_ADC_FDEC_CTRL_*	(31:0) FDEC_CTRL	(31:0) FDEC_CTRL	0x0	First decimation control
P:0x09CC-P:0x09CF 0x40002730 - 0x4000273C	AUDIO_ADC_DEC_CTRL_*	(31) MUTE	(31) MUTE	0x0	Mute output and enable data ready signal
		(30:28) BAND_SELECT_ADC	(30:28) BAND_SELECT_ADC	0x0	Indicated which band is filtered in the decimation filter
		(26:24) DELAY_INTEGER	(26:24) DELAY_INTEGER	0x0	Integer sample delay
		(21:16) DELAY_FRACTIONAL	(21:16) DELAY_FRACTIONAL	0x0	Fractional sample delay (valid values are from 0 to AUDIO_ADC_SAMPLE_FREQ_CFG:SFCR)
		(15) DECIMATOR_EBL	(15) DECIMATOR_EBL	0x0	Enables ADC decimation
		(14:12) DCRM_CUT_OFF	(14:12) DCRM_CUT_OFF	0x2	Control the DC removal filter cut-off frequency of channel
		(11:0) ADC_GF	(11:0) ADC_GF	0x400	Gain factor for ADC = $\text{round}[2^{(11+\text{floor}[\log_2(\text{sfc})])}/(\text{sfc}+1)]$
P:0x09D0 0x40002740	AUDIO_ADC_SAMPLE_FREQ_CFG	(5:0) SFCR	(5:0) SFCR	0x3B	Modulator to WDF input division factor (SFCR = $\text{round}(f_{\text{adclk}}/8/f_{\text{sample}})-1$). Division factors lower than 8 (SFCR < 7) are not valid (not enough cycles for WDF filter)
P:0x09D4	AUDIO_MUX_CFG	(15:14) INPUT_CH3_SRC	(15:14) INPUT_CH3_SRC	0x0	Selects the source of the audio input channel 3

Address	Register Name	Register Write	Register Read	Default	Description
0x40002750					
		(13:12) INPUT_CH2_SRC	(13:12) INPUT_CH2_SRC	0x0	Selects the source of the audio input channel 2
		(11:10) INPUT_CH1_SRC	(11:10) INPUT_CH1_SRC	0x0	Selects the source of the audio input channel 1
		(9:8) INPUT_CH0_SRC	(9:8) INPUT_CH0_SRC	0x0	Selects the source of the audio input channel 0
		(7) IOC0_SRC3	(7) IOC0_SRC3	0x0	Configures the IOC0 audio input channel 3
		(6) IOC0_SRC2	(6) IOC0_SRC2	0x0	Configures the IOC0 audio input channel 2
		(5) IOC0_SRC1	(5) IOC0_SRC1	0x0	Configures the IOC0 audio input channel 1
		(4) IOC0_SRC0	(4) IOC0_SRC0	0x0	Configures the IOC0 audio input channel 0
		(3) INT_FILTER1	(3) INT_FILTER1	0x0	Configures the interpolation filter input channel 1
		(2) INT_FILTER0	(2) INT_FILTER0	0x0	Configures the interpolation filter input channel 0
		(1) SDM1	(1) SDM1	0x0	Configures the Sigma-Delta input channel 1
		(0) SDM0	(0) SDM0	0x0	Configures the Sigma-Delta input channel 0
P:0x09D8-P:0x09DB 0x40002760 - 0x4000276C	AUDIO_BYPASS_DATA_*	(23:0) BYPASS_DATA	(23:0) BYPASS_DATA	0x0	24-bit signed data for the input data bypass (sign extended to 32 bit)
P:0x09DC	AUDIO_DMIC_OUTPUT_CFG	(7) DMIC1_HIGH_SRC	(7) DMIC1_HIGH_SRC	0x0	Selects the source of the DMIC1 high

Address	Register Name	Register Write	Register Read	Default	Description
0x40002770					data channel
		(6) DMIC1_LOW_SRC	(6) DMIC1_LOW_SRC	0x0	Selects the source of the DMIC1 low data channel
		(5) DMIC0_HIGH_SRC	(5) DMIC0_HIGH_SRC	0x0	Selects the source of the DMIC0 high data channel
		(4) DMIC0_LOW_SRC	(4) DMIC0_LOW_SRC	0x0	Selects the source of the DMIC0 low data channel
		(3) DMIC1_HIGH_ENABLE	(3) DMIC1_HIGH_ENABLE	0x0	Enables/disables the DMIC1 high data output
		(2) DMIC1_LOW_ENABLE	(2) DMIC1_LOW_ENABLE	0x0	Enables/disables the DMIC1 low data output
		(1) DMIC0_HIGH_ENABLE	(1) DMIC0_HIGH_ENABLE	0x0	Enables/disables the DMIC0 high data output
		(0) DMIC0_LOW_ENABLE	(0) DMIC0_LOW_ENABLE	0x0	Enables/disables the DMIC0 low data output
P:0x09E0 0x40002780	AUDIO_OUTPUT_DRIVER_CFG	(11) OD1_CLK_SRC	(11) OD1_CLK_SRC	0x0	Clock source for channel 1 re-synchronization
		(8) OD1_ENABLE	(8) OD1_ENABLE	0x0	Enable RCVR1
		(3) OD0_CLK_SRC	(3) OD0_CLK_SRC	0x0	Clock source for channel 0 re-synchronization
		(0) OD0_ENABLE	(0) OD0_ENABLE	0x0	Enable RCVR0
P:0x09E2 0x40002788	AUDIO_OUTPUT_GAIN	(27:16) OUTPUT1_GAIN	(27:16) OUTPUT1_GAIN	0x400	Interpolation filter output 1 gain setting
		(11:0) OUTPUT0_GAIN	(11:0) OUTPUT0_GAIN	0x400	Interpolation filter output 0 gain setting
P:0x09E3	AUDIO_OUTPUT_LIMITER	(17) OUTPUT_LIMITER_STATUS_CLEAR	-	N/A	Clear OUTPUT_LIMITER_STATUS bit

Address	Register Name	Register Write	Register Read	Default	Description
0x4000278C					
		–	(16) OUTPUT_LIMITER_STATUS	0x0	Sticky status bit indicating that the audio output signal has been limited
		(15:0) OUTPUT_LIMITER	(15:0) OUTPUT_LIMITER	0x0	Limit of the audio output signal at SDM input
P:0x09E4–P:0x09E5 0x40002790 – 0x40002794	AUDIO_SDM_CTRL_*	(15:12) QUANT_LEVEL	(15:12) QUANT_LEVEL	0x6	Quantizer level for 3-level output
		(9) TOGGLE_OPT	(9) TOGGLE_OPT	0x0	Toggle optimization mode
		(8) TRI_STATE	(8) TRI_STATE	0x0	Activation of three level output
		(7) SDM_CLK_RATIO	(7) SDM_CLK_RATIO	0x0	Ratio between main and second SDM clock
		(6) XSDM0	(6) XSDM0	0x1	Activation of extended output modulation mode 0
		(0) OD_DELAY	(0) OD_DELAY	0x0	Delay on the RCVR
P:0x09E6–P:0x09E7 0x40002798 – 0x4000279C	AUDIO_SDM_OFFSET_CTRL_*	(23:3) OFFSET	(23:3) OFFSET	0x0	Offset added at SDM input
		(2:0) S_OFFSET	(2:0) S_OFFSET	0x0	Small offset added at SDM input
P:0x09EA–P:0x09EB 0x400027A8 – 0x400027AC	AUDIO_OUTPUT_TEST_*	(2) TEST_ENABLE	(2) TEST_ENABLE	0x0	Test mode enable
		(1) POS_STATE	(1) POS_STATE	0x0	Test state for positive audio output
		(0) NEG_STATE	(0) NEG_STATE	0x0	Test state for negative audio output
P:0x09EC–P:0x09ED	AUDIO_SDM_DCRM_CTRL_*	(12:8) DCRM_CUT_OFF	(12:8) DCRM_CUT_OFF	0xF	DCRM cut-off frequency (fcut =

Address	Register Name	Register Write	Register Read	Default	Description
0x400027B0 - 0x400027B4					$f_{SDMCLK} * 2^{(DCRM_CUT_OFF/2 - 22.65)}$
		(7) DCRM_ENABLE	(7) DCRM_ENABLE	0x0	DC remove enable
		(6:1) IDCRM_CUT_OFF	(6:1) IDCRM_CUT_OFF	0x1A	Input DCRM cut-off frequency ($f_{cut} = f_{SDMCLK} / 2^{(IDCRM_CUT_OFF/2 + 2.15)}$)
		(0) IDCRM_ENABLE	(0) IDCRM_ENABLE	0x1	Input DCRM enable
P:0x09F0 0x400027C0	AUDIO_IOC_FENG_CFG	(26:24) IOC1_FENG_IN6_IN7_TRIG	(26:24) IOC1_FENG_IN6_IN7_TRIG	0x0	IOC1 FENG IN6 IN7 trigger source
		(23:21) IOC1_FENG_OUT7_TRIG	(23:21) IOC1_FENG_OUT7_TRIG	0x0	IOC1 FENG OUT7 trigger source
		(20:18) IOC1_FENG_OUT6_TRIG	(20:18) IOC1_FENG_OUT6_TRIG	0x0	IOC1 FENG OUT6 trigger source
		(17:15) IOC1_FENG_OUT5_TRIG	(17:15) IOC1_FENG_OUT5_TRIG	0x0	IOC1 FENG OUT5 trigger source
		(14:12) IOC1_FENG_OUT4_TRIG	(14:12) IOC1_FENG_OUT4_TRIG	0x0	IOC1 FENG OUT4 trigger source
		(11:9) IOC0_FENG_OUT3_TRIG	(11:9) IOC0_FENG_OUT3_TRIG	0x3	IOC0 FENG OUT3 trigger source
		(8:6) IOC0_FENG_OUT2_TRIG	(8:6) IOC0_FENG_OUT2_TRIG	0x2	IOC0 FENG OUT2 trigger source
		(5:3) IOC0_FENG_OUT1_TRIG	(5:3) IOC0_FENG_OUT1_TRIG	0x3	IOC0 FENG OUT1 trigger source
		(2:0) IOC0_FENG_OUT0_TRIG	(2:0) IOC0_FENG_OUT0_TRIG	0x2	IOC0 FENG OUT0 trigger source
P:0x09F4-P:0x09F7 0x400027D0 -	AUDIO_ADC_OFFSET_TRIM0_*	(23:12) OFFSET_TRIM_DRIVE_4	(23:12) OFFSET_TRIM_DRIVE_4	0x0	12-bit signed data for the offset trimming of drive 4

Address	Register Name	Register Write	Register Read	Default	Description
0x400027DC					
		(11:0) OFFSET_TRIM_DRIVE_2	(11:0) OFFSET_TRIM_DRIVE_2	0x0	12-bit signed data for the offset trimming of drive 2
P:0x09F8-P:0x09FB 0x400027E0 - 0x400027EC	AUDIO_ADC_OFFSET_TRIM1_*	(23:12) OFFSET_TRIM_DRIVE_16	(23:12) OFFSET_TRIM_DRIVE_16	0x0	12-bit signed data for the offset trimming of drive 16
		(11:0) OFFSET_TRIM_DRIVE_8	(11:0) OFFSET_TRIM_DRIVE_8	0x0	12-bit signed data for the offset trimming of drive 8
P:0x09FC-P:0x09FF 0x400027F0 - 0x400027FC	AUDIO_ADC_OFFSET_TRIM2_*	(23:12) OFFSET_TRIM_DRIVE_64	(23:12) OFFSET_TRIM_DRIVE_64	0x0	12-bit signed data for the offset trimming of drive 64
		(11:0) OFFSET_TRIM_DRIVE_32	(11:0) OFFSET_TRIM_DRIVE_32	0x0	12-bit signed data for the offset trimming of drive 32

A.16 I2C INTERFACE

Address	Register Name	Register Write	Register Read	Default	Description
P: 0x0A00 0x40002800	D_I2C_CFG	(30) REPEATED_START_INT_ENABLE	(30) REPEATED_START_INT_ENABLE	0x0	Configure whether repeated start interrupts will be generated by the I2C interface for active transactions in slave mode
		(29) CONNECT_IN_STANDBY	(29) CONNECT_IN_STANDBY	0x0	Control if the I2C lines are connected when running on the standby clock
		(28) TX_DMA_ENABLE	(28) TX_DMA_ENABLE	0x0	Enable/disable the TX DMA request
		(27) RX_DMA_ENABLE	(27) RX_DMA_ENABLE	0x0	Enable/disable the RX DMA request
		(26) TX_INT_ENABLE	(26) TX_INT_ENABLE	0x0	Enable/disable the TX interrupt
		(25) RX_INT_ENABLE	(25) RX_INT_ENABLE	0x0	Enable/disable the RX interrupt
		(24) BUS_ERROR_INT_ENABLE	(24) BUS_ERROR_INT_ENABLE	0x0	Enable/disable the bus error interrupt
		(23) OVERRUN_INT_ENABLE	(23) OVERRUN_INT_ENABLE	0x0	Enable/disable the overrun interrupt
		(22) STOP_INT_ENABLE	(22) STOP_INT_ENABLE	0x0	Configure whether stop interrupts are generated by the I2C interface
		(21) AUTO_ACK_ENABLE	(21) AUTO_ACK_ENABLE	0x0	Select whether acknowledgment is automatically generated or not
		(20:16) SLAVE_PRESCALE	(20:16) SLAVE_PRESCALE	0x0	Controls the number of SYSCLK wait cycles in case of clock stretching (in slave mode) between the moment the data is put on the SDA line and the SCL line is released.
		(15:8) MASTER_PRESCALE	(15:8) MASTER_PRESCALE	0x0	Prescaler used to divide SYSCLK to the correct SCL frequency when operating in master mode. SCL is prescaled by (PRESCALE + 1) * 3.

Address	Register Name	Register Write	Register Read	Default	Description
		(7:1) SLAVE_ADDRESS	(7:1) SLAVE_ADDRESS	0x10	Set the I2C slave address for this device
		(0) SLAVE	(0) SLAVE	0x0	I2C slave mode enable
P:0x0A01 0x40002804	D_I2C_CTRL	–	(9) LAST_DATA_STATUS	0x0	I2C last data status
		(7) ERROR_CNT_CLEAR	–	N/A	Clear the D_I2C_ERROR_CNT register
		(6) LAST_DATA	–	N/A	Indicate that the current data is the last byte of a data transfer
		(5) STOP	–	N/A	Issue a stop condition on the I2C interface bus
		(4) NACK	–	N/A	Issue a not acknowledge on the I2C interface bus
		(3) ACK	–	N/A	Issue an acknowledge on the I2C interface bus
		(2) RESET	–	N/A	Reset the I2C interface
P:0x0A02 0x40002808	D_I2C_ADDR_START	(7:1) ADDR	(7:1) ADDR	0x0	Address to send with the start condition over the I2C bus
		(0) READ_WRITE	(0) READ_WRITE	0x0	Select whether a read or a write transaction is started
P:0x0A03 0x4000280C	D_I2C_STATUS	–	(26) STOP_OR_REPEATED_START_DETECTED	0x0	Indicate if STOP_DETECTED or REPEATED_START_DETECTED bit is set
		–	(25) REPEATED_START_DETECTED	0x0	Indicate if an I2C repeated start has been detected during an active transaction in slave mode
		–	(23) BUS_OR_	0x0	Bus or watchdog error status bit

Address	Register Name	Register Write	Register Read	Default	Description
			WATCHDOG_ERROR		
		–	(22) BUS_ERROR	0x0	Bus error status bit
		–	(21) BUSY	0x0	Indicate that the reception or transmission of the data is ongoing
		–	(20) START_PENDING	0x0	Master frame start pending status bit
		–	(19) MASTER_MODE	0x0	Master mode status bit
		–	(18) STOP_DETECTED	0x0	Indicate if an I2C stop bit has been detected
		–	(17) DATA_EVENT	0x0	Indicate that I2C interface either needs data to transmit or has received data
		–	(16) TX_REQ	0x1	Indicate that a TX data can be written
		–	(15) RX_REQ	0x0	Indicate that a RX data can be read
		–	(14) CLK_STRETCH	0x0	Clock stretching flag
		–	(13) LINE_FREE	0x1	Line free flag
		–	(12) ADDR_DATA	0x0	Address / Data byte
		–	(11) READ_WRITE	0x0	Read / Write frame
		–	(10) GEN_CALL	0x0	General call flag
		–	(9) ACK	0x0	Acknowledge status
		–	(8) OVERRUN	0x0	Indicate that an overrun has occurred when receiving data
		(4) TX_REQ_SET	–	N/A	Set TX_REQ status flag
		(3) REPEATED_START_DETECTED_CLEAR	–	N/A	Clear REPEATED_START_DETECTED status flag
		(2) STOP_DETECTED_CLEAR	–	N/A	Clear STOP_DETECTED status flag

Address	Register Name	Register Write	Register Read	Default	Description
		(1) BUS_ERROR_CLEAR	–	N/A	Clear BUS_ERROR status flag
		(0) OVERRUN_CLEAR	–	N/A	Clear OVERRUN status flag
P:0x0A04 0x40002810	D_I2C_TX_DATA	(7:0) TX_DATA	(7:0) TX_DATA	0x0	Single byte buffer for data transmitted over the I2C interface
P:0x0A05 0x40002814	D_I2C_RX_DATA	–	(7:0) RX_DATA	0x0	Single byte buffer for data received over the I2C interface
P:0x0A06 0x40002818	D_I2C_RX_DATA_MIRROR	–	(7:0) RX_DATA	0x0	Mirror of the single byte buffer for data received over the I2C interface. Does not clear RX_REQ in D_I2C_STATUS
P:0x0A07 0x4000281C	D_I2C_CFX_DEBUG_ADDR1	(6:0) DEBUG_ADDR	(6:0) DEBUG_ADDR	0x60	Defines the first CFX processor debug I2C slave address
P:0x0A08 0x40002820	D_I2C_CFX_DEBUG_ADDR2	(6:0) DEBUG_ADDR	(6:0) DEBUG_ADDR	0x60	Defines the second CFX processor debug I2C slave address
P:0x0A09 0x40002824	D_I2C_CM3_DEBUG_ADDR	(6:0) DEBUG_ADDR	(6:0) DEBUG_ADDR	0x40	Defines the Arm Cortex-M3 processor debug I2C slave address
P:0x0A0A 0x40002828	D_I2C_WATCHDOG_CFG	(2:0) TIMEOUT	(2:0) TIMEOUT	0x2	Watchdog timeout configuration
P:0x0A0B 0x4000282C	D_I2C_ERROR_CNT	–	(7:0) ERROR_CNT	0x0	Counter the number of time the I2C-watchdog timeout

A.17 FENG CONTROL AND CONFIGURATION

Address	Register Name	Register Write	Register Read	Default	Description
P:0x0B00 0x40002C00	D_FENG_INT_STATUS FENG_INT_STATUS	–	(9) SRIN	0x0	New FENG sample input and signal ready arrival interrupt status. Check the INT_STATUS_IO register for the individual input status
		–	(8) OUT	0x0	New data capture at FENG output interrupt status. Check the INT_STATUS_IO register for the individual output status
		–	(7) OVERLOAD	0x0	Processor overload interrupt status
		–	(6) ALU_SAT	0x0	ALU saturation interrupt status
		–	(5) CORE_SAT	0x0	Core saturation interrupt status
		–	(4) MEM_ERR	0x0	Illegal memory access interrupt status
		–	(3) REG_ERR	0x0	Register write access collision interrupt status
		–	(2) CMP	0x0	Compare instruction interrupt status
		–	(1) PGE	0x0	End of PGM execution interrupt status
		–	(0) ISHC	0x0	Input shadow registers capture in RF interrupt status
P:0x0B01 0x40002C04	D_FENG_INT_STATUS_IO FENG_INT_STATUS_IO	–	(27) OUT11	0x0	New data capture at FENG output 11 interrupt status
		–	(26) OUT10	0x0	New data capture at FENG output

Address	Register Name	Register Write	Register Read	Default	Description
					10 interrupt status
		–	(25) OUT9	0x0	New data capture at FENG output 9 interrupt status
		–	(24) OUT8	0x0	New data capture at FENG output 8 interrupt status
		–	(23) OUT7	0x0	New data capture at FENG output 7 interrupt status
		–	(22) OUT6	0x0	New data capture at FENG output 6 interrupt status
		–	(21) OUT5	0x0	New data capture at FENG output 5 interrupt status
		–	(20) OUT4	0x0	New data capture at FENG output 4 interrupt status
		–	(19) OUT3	0x0	New data capture at FENG output 3 interrupt status
		–	(18) OUT2	0x0	New data capture at FENG output 2 interrupt status
		–	(17) OUT1	0x0	New data capture at FENG output 1 interrupt status
		–	(16) OUT0	0x0	New data capture at FENG output 0 interrupt status
		–	(13) SRIN13	0x0	New FENG input 13 and signal ready arrival interrupt status
		–	(12) SRIN12	0x0	New FENG input 12 and signal ready arrival interrupt status
		–	(11) SRIN11	0x0	New FENG input 11 and signal ready arrival interrupt status
		–	(10) SRIN10	0x0	New FENG input 10 and signal

Address	Register Name	Register Write	Register Read	Default	Description
					ready arrival interrupt status
		–	(9) SRIN9	0x0	New FENG input 9 and signal ready arrival interrupt status
		–	(8) SRIN8	0x0	New FENG input 8 and signal ready arrival interrupt status
		–	(7) SRIN7	0x0	New FENG input 7 and signal ready arrival interrupt status
		–	(6) SRIN6	0x0	New FENG input 6 and signal ready arrival interrupt status
		–	(5) SRIN5	0x0	New FENG input 5 and signal ready arrival interrupt status
		–	(4) SRIN4	0x0	New FENG input 4 and signal ready arrival interrupt status
		–	(3) SRIN3	0x0	New FENG input 3 and signal ready arrival interrupt status
		–	(2) SRIN2	0x0	New FENG input 2 and signal ready arrival interrupt status
		–	(1) SRIN1	0x0	New FENG input 1 and signal ready arrival interrupt status
		–	(0) SRIN0	0x0	New FENG input 0 and signal ready arrival interrupt status
P:0x0B02 0x40002C08	D_FENG_INT_ACK FENG_INT_ACK	(9) SRIN_ACK	–	N/A	FENG sample input and signal ready arrival interrupt acknowledge. Acknowledging with this bit clears the SRIN bit under the FENG_INT_STATUS as well as all the SRIN fields under the FENG_INT_STATUS_IO

Address	Register Name	Register Write	Register Read	Default	Description
		(8) OUT_ACK	–	N/A	New data capture at FENG output interrupt acknowledge. Acknowledging with this bit clears the OUT bit under the FENG_INT_STATUS as well as all the OUT fields under the FENG_INT_STATUS_IO
		(7) OVERLOAD_ACK	–	N/A	Processor overload interrupt acknowledge
		(6) ALU_SAT_ACK	–	N/A	ALU saturation interrupt acknowledge
		(5) CORE_SAT_ACK	–	N/A	Core saturation interrupt acknowledge
		(4) MEM_ERR_ACK	–	N/A	Illegal memory access interrupt acknowledge
		(3) REG_ERR_ACK	–	N/A	Register write access collision interrupt acknowledge
		(2) CMP_ACK	–	N/A	Compare instruction interrupt acknowledge
		(1) PGE_ACK	–	N/A	End of PGM execution interrupt acknowledge
		(0) ISHC_ACK	–	N/A	Input shadow registers capture in RF interrupt acknowledge
P:0x0B03 0x40002C0C	D_FENG_INT_ACK_IO FENG_INT_ACK_IO	(27) OUT11_ACK	–	N/A	Data capture at FENG output11 interrupt acknowledge
		(26) OUT10_ACK	–	N/A	Data capture at FENG output10 interrupt acknowledge
		(25) OUT9_ACK	–	N/A	Data capture at FENG output9

Address	Register Name	Register Write	Register Read	Default	Description
					interrupt acknowledge
		(24) OUT8_ACK	–	N/A	Data capture at FENG output8 interrupt acknowledge
		(23) OUT7_ACK	–	N/A	Data capture at FENG output7 interrupt acknowledge
		(22) OUT6_ACK	–	N/A	Data capture at FENG output6 interrupt acknowledge
		(21) OUT5_ACK	–	N/A	Data capture at FENG output5 interrupt acknowledge
		(20) OUT4_ACK	–	N/A	Data capture at FENG output4 interrupt acknowledge
		(19) OUT3_ACK	–	N/A	Data capture at FENG output3 interrupt acknowledge
		(18) OUT2_ACK	–	N/A	Data capture at FENG output2 interrupt acknowledge
		(17) OUT1_ACK	–	N/A	Data capture at FENG output1 interrupt acknowledge
		(16) OUT0_ACK	–	N/A	Data capture at FENG output0 interrupt acknowledge
		(13) SRIN13_ACK	–	N/A	FENG input13 and signal ready arrival interrupt acknowledge
		(12) SRIN12_ACK	–	N/A	FENG input12 and signal ready arrival interrupt acknowledge
		(11) SRIN11_ACK	–	N/A	FENG input11 and signal ready arrival interrupt acknowledge
		(10) SRIN10_ACK	–	N/A	FENG input10 and signal ready arrival interrupt acknowledge
		(9) SRIN9_ACK	–	N/A	FENG input9 and signal ready

Address	Register Name	Register Write	Register Read	Default	Description
					arrival interrupt acknowledge
		(8) SRIN8_ACK	–	N/A	FENG input8 and signal ready arrival interrupt acknowledge
		(7) SRIN7_ACK	–	N/A	FENG input7 and signal ready arrival interrupt acknowledge
		(6) SRIN6_ACK	–	N/A	FENG input6 and signal ready arrival interrupt acknowledge
		(5) SRIN5_ACK	–	N/A	FENG input5 and signal ready arrival interrupt acknowledge
		(4) SRIN4_ACK	–	N/A	FENG input4 and signal ready arrival interrupt acknowledge
		(3) SRIN3_ACK	–	N/A	FENG input3 and signal ready arrival interrupt acknowledge
		(2) SRIN2_ACK	–	N/A	FENG input2 and signal ready arrival interrupt acknowledge
		(1) SRIN1_ACK	–	N/A	FENG input1 and signal ready arrival interrupt acknowledge
		(0) SRIN0_ACK	–	N/A	FENG input0 and signal ready arrival interrupt acknowledge
P:0x0B04 0x40002C10	D_FENG_INT_ENABLE FENG_INT_ENABLE	(9) SRIN	(9) SRIN	0x0	New FENG sample input and signal ready arrival interrupt enable. This enable propagates the SRIN interrupts enabled in the INT_ENABLE_IO register
		(8) OUT	(8) OUT	0x0	New data capture at FENG output interrupt enable. This enable propagates the OUT interrupts enabled in the INT_ENABLE_IO register

Address	Register Name	Register Write	Register Read	Default	Description
		(7) OVERLOAD	(7) OVERLOAD	0x0	Processor overload interrupt enable
		(6) ALU_SAT	(6) ALU_SAT	0x0	ALU saturation interrupt enable
		(5) CORE_SAT	(5) CORE_SAT	0x0	Core saturation interrupt enable
		(4) MEM_ERR	(4) MEM_ERR	0x0	Illegal memory access interrupt enable
		(3) REG_ERR	(3) REG_ERR	0x0	Register write access collision interrupt enable
		(2) CMP	(2) CMP	0x0	Compare instruction interrupt enable
		(1) PGE	(1) PGE	0x0	End of PGM execution interrupt enable
		(0) ISHC	(0) ISHC	0x0	Input shadow registers capture in RF interrupt enable
P:0x0B05 0x40002C14	D_FENG_INT_ENABLE_IO FENG_INT_ENABLE_IO	(27) OUT11	(27) OUT11	0x0	New data capture at FENG output 11 interrupt enable
		(26) OUT10	(26) OUT10	0x0	New data capture at FENG output 10 interrupt enable
		(25) OUT9	(25) OUT9	0x0	New data capture at FENG output 9 interrupt enable
		(24) OUT8	(24) OUT8	0x0	New data capture at FENG output 8 interrupt enable
		(23) OUT7	(23) OUT7	0x0	New data capture at FENG output 7 interrupt enable
		(22) OUT6	(22) OUT6	0x0	New data capture at FENG output 6 interrupt enable
		(21) OUT5	(21) OUT5	0x0	New data capture at FENG output

Address	Register Name	Register Write	Register Read	Default	Description
					5 interrupt enable
		(20) OUT4	(20) OUT4	0x0	New data capture at FENG output 4 interrupt enable
		(19) OUT3	(19) OUT3	0x0	New data capture at FENG output 3 interrupt enable
		(18) OUT2	(18) OUT2	0x0	New data capture at FENG output 2 interrupt enable
		(17) OUT1	(17) OUT1	0x0	New data capture at FENG output 1 interrupt enable
		(16) OUT0	(16) OUT0	0x0	New data capture at FENG output 0 interrupt enable
		(13) SRIN13	(13) SRIN13	0x0	New FENG input 13 and signal ready arrival interrupt enable
		(12) SRIN12	(12) SRIN12	0x0	New FENG input 12 and signal ready arrival interrupt enable
		(11) SRIN11	(11) SRIN11	0x0	New FENG input 11 and signal ready arrival interrupt enable
		(10) SRIN10	(10) SRIN10	0x0	New FENG input 10 and signal ready arrival interrupt enable
		(9) SRIN9	(9) SRIN9	0x0	New FENG input 9 and signal ready arrival interrupt enable
		(8) SRIN8	(8) SRIN8	0x0	New FENG input 8 and signal ready arrival interrupt enable
		(7) SRIN7	(7) SRIN7	0x0	New FENG input 7 and signal ready arrival interrupt enable
		(6) SRIN6	(6) SRIN6	0x0	New FENG input 6 and signal ready arrival interrupt enable
		(5) SRIN5	(5) SRIN5	0x0	New FENG input 5 and signal

Address	Register Name	Register Write	Register Read	Default	Description
					ready arrival interrupt enable
		(4) SRIN4	(4) SRIN4	0x0	New FENG input 4 and signal ready arrival interrupt enable
		(3) SRIN3	(3) SRIN3	0x0	New FENG input 3 and signal ready arrival interrupt enable
		(2) SRIN2	(2) SRIN2	0x0	New FENG input 2 and signal ready arrival interrupt enable
		(1) SRIN1	(1) SRIN1	0x0	New FENG input 1 and signal ready arrival interrupt enable
		(0) SRIN0	(0) SRIN0	0x0	New FENG input 0 and signal ready arrival interrupt enable
P:0x0B06 0x40002C18	D_FENG_CFG FENG_CFG	(9:4) COEFF_OFFSET	(9:4) COEFF_OFFSET	0x0	The coefficient decoder offset
		(3:0) SRI_SEL	(3:0) SRI_SEL	0xF	Sample ready input starting the FENG processing
P:0x0B07 0x40002C1C	D_FENG_CFG_IO FENG_CFG_IO	(27) OSR11_EBL	(27) OSR11_EBL	0x1	Shadow register capture mode for output 11
		(26) OSR10_EBL	(26) OSR10_EBL	0x1	Shadow register capture mode for output 10
		(25) OSR9_EBL	(25) OSR9_EBL	0x1	Shadow register capture mode for output 9
		(24) OSR8_EBL	(24) OSR8_EBL	0x1	Shadow register capture mode for output 8
		(23) OSR7_EBL	(23) OSR7_EBL	0x1	Shadow register capture mode for output 7
		(22) OSR6_EBL	(22) OSR6_EBL	0x1	Shadow register capture mode for output 6

Address	Register Name	Register Write	Register Read	Default	Description
		(21) OSR5_EBL	(21) OSR5_EBL	0x1	Shadow register capture mode for output 5
		(20) OSR4_EBL	(20) OSR4_EBL	0x1	Shadow register capture mode for output 4
		(19) OSR3_EBL	(19) OSR3_EBL	0x1	Shadow register capture mode for output 3
		(18) OSR2_EBL	(18) OSR2_EBL	0x1	Shadow register capture mode for output 2
		(17) OSR1_EBL	(17) OSR1_EBL	0x1	Shadow register capture mode for output 1
		(16) OSR0_EBL	(16) OSR0_EBL	0x1	Shadow register capture mode for output 0
		(13) FIN13_EBL	(13) FIN13_EBL	0x0	Enable/Disable sample capture on input 13
		(12) FIN12_EBL	(12) FIN12_EBL	0x0	Enable/Disable sample capture on input 12
		(11) FIN11_EBL	(11) FIN11_EBL	0x0	Enable/Disable sample capture on input 11
		(10) FIN10_EBL	(10) FIN10_EBL	0x0	Enable/Disable sample capture on input 10
		(9) FIN9_EBL	(9) FIN9_EBL	0x0	Enable/Disable sample capture on input 9
		(8) FIN8_EBL	(8) FIN8_EBL	0x0	Enable/Disable sample capture on input 8
		(7) FIN7_EBL	(7) FIN7_EBL	0x0	Enable/Disable sample capture on input 7
		(6) FIN6_EBL	(6) FIN6_EBL	0x0	Enable/Disable sample capture on input 6

Address	Register Name	Register Write	Register Read	Default	Description
		(5) FIN5_EBL	(5) FIN5_EBL	0x0	Enable/Disable sample capture on input 5
		(4) FIN4_EBL	(4) FIN4_EBL	0x0	Enable/Disable sample capture on input 4
		(3) FIN3_EBL	(3) FIN3_EBL	0x0	Enable/Disable sample capture on input 3
		(2) FIN2_EBL	(2) FIN2_EBL	0x0	Enable/Disable sample capture on input 2
		(1) FIN1_EBL	(1) FIN1_EBL	0x0	Enable/Disable sample capture on input 1
		(0) FIN0_EBL	(0) FIN0_EBL	0x0	Enable/Disable sample capture on input 0
P:0x0B08 0x40002C20	D_FENG_MRCNT FENG_MRCNT	(19) MRCNT_SYNC_EN	(19) MRCNT_SYNC_EN	0x0	The continuous synchronization enable of the MRCNT Phase0
		(18:16) MRCNT_SYNC_SOURCE	(18:16) MRCNT_SYNC_SOURCE	0x0	Sample ready input to synchronize the Multirate Counters
		(15:12) MRCNT1	(15:12) MRCNT1	0x0	Multirate Counter 1 Value
		(11:8) MRCNT0	(11:8) MRCNT0	0x0	Multirate Counter 0 Value
		(7:4) MRCNT1_DR	(7:4) MRCNT1_DR	0x0	Multirate Counter 1 Divide Ratio (0 to15)
		(3:0) MRCNT0_DR	(3:0) MRCNT0_DR	0x0	Multirate Counter 0 Divide Ratio (0 to15)
P:0x0B09 0x40002C24	D_FENG_PGM_SECTOR_EN FENG_PGM_SECTOR_EN	(31:16) CNT1	(31:16) CNT1	0x0	FENG_MRCNT_MRCNT1 counter phase controlled sectors enable
		(15:0) CNT0	(15:0) CNT0	0x0	FENG_MRCNT_MRCNT0 counter phase controlled sectors

Address	Register Name	Register Write	Register Read	Default	Description
					enable
P:0x0B0A 0x40002C28	D_FENG_PHM_PH0_PH1 FENG_PHM_PH0_PH1	(31:24) PHM_CNT1PH1	(31:24) PHM_CNT1PH1	0x0	FENG_MRCNT_MRCNT1 phase 1 controlled program memory sector start
		(23:16) PHM_CNT0PH1	(23:16) PHM_CNT0PH1	0x0	FENG_MRCNT_MRCNT0 phase 1 controlled program memory sector start
		(15:8) PHM_CNT1PH0	(15:8) PHM_CNT1PH0	0x0	FENG_MRCNT_MRCNT1 phase 0 controlled program memory sector start
		(7:0) PHM_CNT0PH0	(7:0) PHM_CNT0PH0	0x0	FENG_MRCNT_MRCNT0 phase 0 controlled program memory sector start
P:0x0B0B 0x40002C2C	D_FENG_PHM_PH2_PH3 FENG_PHM_PH2_PH3	(31:24) PHM_CNT1PH3	(31:24) PHM_CNT1PH3	0x0	FENG_MRCNT_MRCNT1 phase 3 controlled program memory sector start
		(23:16) PHM_CNT0PH3	(23:16) PHM_CNT0PH3	0x0	FENG_MRCNT_MRCNT0 phase 3 controlled program memory sector start
		(15:8) PHM_CNT1PH2	(15:8) PHM_CNT1PH2	0x0	FENG_MRCNT_MRCNT1 phase 2 controlled program memory sector start
		(7:0) PHM_CNT0PH2	(7:0) PHM_CNT0PH2	0x0	FENG_MRCNT_MRCNT0 phase 2 controlled program memory sector start
P:0x0B0C 0x40002C30	D_FENG_PHM_PH4_PH5 FENG_PHM_PH4_PH5	(31:24) PHM_CNT1PH5	(31:24) PHM_CNT1PH5	0x0	FENG_MRCNT_MRCNT1 phase 5 controlled program memory sector start
		(23:16) PHM_CNT0PH5	(23:16) PHM_CNT0PH5	0x0	FENG_MRCNT_MRCNT0 phase 5 controlled program memory

Address	Register Name	Register Write	Register Read	Default	Description
					sector start
		(15:8) PHM_CNT1PH4	(15:8) PHM_CNT1PH4	0x0	FENG_MRCNT_MRCNT1 phase 4 controlled program memory sector start
		(7:0) PHM_CNT0PH4	(7:0) PHM_CNT0PH4	0x0	FENG_MRCNT_MRCNT0 phase 4 controlled program memory sector start
P:0x0B0D 0x40002C34	D_FENG_PHM_PH6_PH7 FENG_PHM_PH6_PH7	(31:24) PHM_CNT1PH7	(31:24) PHM_CNT1PH7	0x0	FENG_MRCNT_MRCNT1 phase 7 controlled program memory sector start
		(23:16) PHM_CNT0PH7	(23:16) PHM_CNT0PH7	0x0	FENG_MRCNT_MRCNT0 phase 7 controlled program memory sector start
		(15:8) PHM_CNT1PH6	(15:8) PHM_CNT1PH6	0x0	FENG_MRCNT_MRCNT1 phase 6 controlled program memory sector start
		(7:0) PHM_CNT0PH6	(7:0) PHM_CNT0PH6	0x0	FENG_MRCNT_MRCNT0 phase 6 controlled program memory sector start
P:0x0B0E 0x40002C38	D_FENG_PHM_PH8_PH9 FENG_PHM_PH8_PH9	(31:24) PHM_CNT1PH9	(31:24) PHM_CNT1PH9	0x0	FENG_MRCNT_MRCNT1 phase 9 controlled program memory sector start
		(23:16) PHM_CNT0PH9	(23:16) PHM_CNT0PH9	0x0	FENG_MRCNT_MRCNT0 phase 9 controlled program memory sector start
		(15:8) PHM_CNT1PH8	(15:8) PHM_CNT1PH8	0x0	FENG_MRCNT_MRCNT1 phase 8 controlled program memory sector start
		(7:0) PHM_CNT0PH8	(7:0) PHM_CNT0PH8	0x0	FENG_MRCNT_MRCNT0 phase 8 controlled program memory

Address	Register Name	Register Write	Register Read	Default	Description
					sector start
P:0x0B0F 0x40002C3C	D_FENG_PHM_PH10_PH11 FENG_PHM_PH10_PH11	(31:24) PHM_CNT1PH11	(31:24) PHM_CNT1PH11	0x0	FENG_MRCNT_MRCNT1 phase 11 controlled program memory sector start
		(23:16) PHM_CNT0PH11	(23:16) PHM_CNT0PH11	0x0	FENG_MRCNT_MRCNT0 phase 11 controlled program memory sector start
		(15:8) PHM_CNT1PH10	(15:8) PHM_CNT1PH10	0x0	FENG_MRCNT_MRCNT1 phase 10 controlled program memory sector start
		(7:0) PHM_CNT0PH10	(7:0) PHM_CNT0PH10	0x0	FENG_MRCNT_MRCNT0 phase 10 controlled program memory sector start
P:0x0B10 0x40002C40	D_FENG_PHM_PH12_PH13 FENG_PHM_PH12_PH13	(31:24) PHM_CNT1PH13	(31:24) PHM_CNT1PH13	0x0	FENG_MRCNT_MRCNT1 phase 13 controlled program memory sector start
		(23:16) PHM_CNT0PH13	(23:16) PHM_CNT0PH13	0x0	FENG_MRCNT_MRCNT0 phase 13 controlled program memory sector start
		(15:8) PHM_CNT1PH12	(15:8) PHM_CNT1PH12	0x0	FENG_MRCNT_MRCNT1 phase 12 controlled program memory sector start
		(7:0) PHM_CNT0PH12	(7:0) PHM_CNT0PH12	0x0	FENG_MRCNT_MRCNT0 phase 12 controlled program memory sector start
P:0x0B11 0x40002C44	D_FENG_PHM_PH14_PH15 FENG_PHM_PH14_PH15	(31:24) PHM_CNT1PH15	(31:24) PHM_CNT1PH15	0x0	FENG_MRCNT_MRCNT1 phase 15 controlled program memory sector start
		(23:16) PHM_CNT0PH15	(23:16) PHM_CNT0PH15	0x0	FENG_MRCNT_MRCNT0 phase 15 controlled program memory

Address	Register Name	Register Write	Register Read	Default	Description
					sector start
		(15:8) PHM_CNT1PH14	(15:8) PHM_CNT1PH14	0x0	FENG_MRCNT_MRCNT1 phase 14 controlled program memory sector start
		(7:0) PHM_CNT0PH14	(7:0) PHM_CNT0PH14	0x0	FENG_MRCNT_MRCNT0 phase 14 controlled program memory sector start
P:0x0B12 0x40002C48	D_FENG_PHM_PH15_END FENG_PHM_PH15_END	(7:0) PH15_END	(7:0) PH15_END	0x0	FENG_MRCNT_MRCNT1 phase 15 program memory sector end
P:0x0B13 0x40002C4C	D_FENG_COEFF_OFFSET_EN_SHADOW0 FENG_COEFF_OFFSET_EN_SHADOW0	(31:0) COEFF_OFFSET_EN_SHADOW0	(31:0) COEFF_OFFSET_EN_SHADOW0	0x0	The coefficients 0 to 31 offset enable shadow register
P:0x0B14 0x40002C50	D_FENG_COEFF_OFFSET_EN_SHADOW1 FENG_COEFF_OFFSET_EN_SHADOW1	(31:0) COEFF_OFFSET_EN_SHADOW1	(31:0) COEFF_OFFSET_EN_SHADOW1	0x0	The coefficients 32 to 63 offset enable shadow register
P:0x0B15 0x40002C54	D_FENG_SH_SHADOW FENG_SH_SHADOW	(23:21) SHREG7_SHADOW	(23:21) SHREG7_SHADOW	0x0	SH shadow value for address 7 mapping
		(20:18) SHREG6_SHADOW	(20:18) SHREG6_SHADOW	0x0	SH shadow value for address 6 mapping
		(17:15) SHREG5_SHADOW	(17:15) SHREG5_SHADOW	0x0	SH shadow value for address 5 mapping
		(14:12) SHREG4_SHADOW	(14:12) SHREG4_SHADOW	0x0	SH shadow value for address 4 mapping
		(11:9) SHREG3_SHADOW	(11:9) SHREG3_SHADOW	0x0	SH shadow value for address 3

Address	Register Name	Register Write	Register Read	Default	Description
					mapping
		(8:6) SHREG2_SHADOW	(8:6) SHREG2_SHADOW	0x0	SH shadow value for address 2 mapping
		(5:3) SHREG1_SHADOW	(5:3) SHREG1_SHADOW	0x0	SH shadow value for address 1 mapping
		(2:0) SHREG0_SHADOW	(2:0) SHREG0_SHADOW	0x0	SH shadow value for address 0 mapping
P:0x0B16 0x40002C58	D_FENG_TC_SHADOW0 FENG_TC_SHADOW0	(31:28) TCREG7_SHADOW	(31:28) TCREG7_SHADOW	0x0	ATC or DTC shadow value for address 7 mapping
		(27:24) TCREG6_SHADOW	(27:24) TCREG6_SHADOW	0x0	ATC or DTC shadow value for address 6 mapping
		(23:20) TCREG5_SHADOW	(23:20) TCREG5_SHADOW	0x0	ATC or DTC shadow value for address 5 mapping
		(19:16) TCREG4_SHADOW	(19:16) TCREG4_SHADOW	0x0	ATC or DTC shadow value for address 4 mapping
		(15:12) TCREG3_SHADOW	(15:12) TCREG3_SHADOW	0x0	ATC or DTC shadow value for address 3 mapping
		(11:8) TCREG2_SHADOW	(11:8) TCREG2_SHADOW	0x0	ATC or DTC shadow value for address 2 mapping
		(7:4) TCREG1_SHADOW	(7:4) TCREG1_SHADOW	0x0	ATC or DTC shadow value for address 1 mapping
		(3:0) TCREG0_SHADOW	(3:0) TCREG0_SHADOW	0x0	ATC or DTC shadow value for address 0 mapping
P:0x0B17 0x40002C5C	D_FENG_TC_SHADOW1 FENG_TC_SHADOW1	(31:28) TCREG15_SHADOW	(31:28) TCREG15_SHADOW	0x0	ATC or DTC shadow value for address 15 mapping
		(27:24) TCREG14_SHADOW	(27:24) TCREG14_SHADOW	0x0	ATC or DTC shadow value for address 14 mapping

Address	Register Name	Register Write	Register Read	Default	Description
		(23:20) TCREG13_SHADOW	(23:20) TCREG13_SHADOW	0x0	ATC or DTC shadow value for address 13 mapping
		(19:16) TCREG12_SHADOW	(19:16) TCREG12_SHADOW	0x0	ATC or DTC shadow value for address 12 mapping
		(15:12) TCREG11_SHADOW	(15:12) TCREG11_SHADOW	0x0	ATC or DTC shadow value for address 11 mapping
		(11:8) TCREG10_SHADOW	(11:8) TCREG10_SHADOW	0x0	ATC or DTC shadow value for address 10 mapping
		(7:4) TCREG9_SHADOW	(7:4) TCREG9_SHADOW	0x0	ATC or DTC shadow value for address 9 mapping
		(3:0) TCREG8_SHADOW	(3:0) TCREG8_SHADOW	0x0	ATC or DTC shadow value for address 8 mapping
P:0x0B18 0x40002C60	D_FENG_STATE_W_EN FENG_STATE_W_EN	(31:0) STATE_W_EN	(31:0) STATE_W_EN	0xFFFFFFFF	The conditional processing state registers write enable
P:0x0B19 0x40002C64	D_FENG_STATE_SHADOW FENG_STATE_SHADOW	(31:0) STATE_SHADOW	(31:0) STATE_SHADOW	0xFFFFFFFF	The conditional processing state shadow register
P:0x0B1A 0x40002C68	D_FENG_STATE_ACTIVE FENG_STATE_ACTIVE	(31:0) STATE_ACTIVE	(31:0) STATE_ACTIVE	0xFFFFFFFF	The conditional processing state active register
P:0x0B1B 0x40002C6C	D_FENG_COEFF_OFFSET_EN_ACTIVE0 FENG_COEFF_OFFSET_EN_ACTIVE0	(31:0) COEFF_OFFSET_EN_ACTIVE0	(31:0) COEFF_OFFSET_EN_ACTIVE0	0x0	The coefficients 0 to 31 offset enable active register
P:0x0B1C 0x40002C70	D_FENG_COEFF_OFFSET_EN_ACTIVE1 FENG_COEFF_OFFSET_EN_ACTIVE1	(31:0) COEFF_OFFSET_EN_ACTIVE1	(31:0) COEFF_OFFSET_EN_ACTIVE1	0x0	The coefficients 32 to 63 offset enable active register

Address	Register Name	Register Write	Register Read	Default	Description
P:0x0B1D 0x40002C74	D_FENG_SH_ACTIVE FENG_SH_ACTIVE	(23:21) SHREG7_ACTIVE	(23:21) SHREG7_ACTIVE	0x0	SH active value for address 7
		(20:18) SHREG6_ACTIVE	(20:18) SHREG6_ACTIVE	0x0	SH active value for address 6
		(17:15) SHREG5_ACTIVE	(17:15) SHREG5_ACTIVE	0x0	SH active value for address 5
		(14:12) SHREG4_ACTIVE	(14:12) SHREG4_ACTIVE	0x0	SH active value for address 4
		(11:9) SHREG3_ACTIVE	(11:9) SHREG3_ACTIVE	0x0	SH active value for address 3
		(8:6) SHREG2_ACTIVE	(8:6) SHREG2_ACTIVE	0x0	SH active value for address 2
		(5:3) SHREG1_ACTIVE	(5:3) SHREG1_ACTIVE	0x0	SH active value for address 1
		(2:0) SHREG0_ACTIVE	(2:0) SHREG0_ACTIVE	0x0	SH active value for address 0
P:0x0B1E 0x40002C78	D_FENG_TC_ACTIVE0 FENG_TC_ACTIVE0	(31:28) TCREG7_ACTIVE	(31:28) TCREG7_ACTIVE	0x0	ATC or DTC active value for address 7 mapping
		(27:24) TCREG6_ACTIVE	(27:24) TCREG6_ACTIVE	0x0	ATC or DTC active value for address 6 mapping
		(23:20) TCREG5_ACTIVE	(23:20) TCREG5_ACTIVE	0x0	ATC or DTC active value for address 5 mapping
		(19:16) TCREG4_ACTIVE	(19:16) TCREG4_ACTIVE	0x0	ATC or DTC active value for address 4 mapping
		(15:12) TCREG3_ACTIVE	(15:12) TCREG3_ACTIVE	0x0	ATC or DTC active value for address 3 mapping
		(11:8) TCREG2_ACTIVE	(11:8) TCREG2_ACTIVE	0x0	ATC or DTC active value for address 2 mapping
		(7:4) TCREG1_ACTIVE	(7:4) TCREG1_ACTIVE	0x0	ATC or DTC active value for address 1 mapping

Address	Register Name	Register Write	Register Read	Default	Description
		(3:0) TCREG0_ACTIVE	(3:0) TCREG0_ACTIVE	0x0	ATC or DTC active value for address 0 mapping
P:0x0B1F 0x40002C7C	D_FENG_TC_ACTIVE1 FENG_TC_ACTIVE1	(31:28) TCREG15_ACTIVE	(31:28) TCREG15_ACTIVE	0x0	ATC or DTC active value for address 15 mapping
		(27:24) TCREG14_ACTIVE	(27:24) TCREG14_ACTIVE	0x0	ATC or DTC active value for address 14 mapping
		(23:20) TCREG13_ACTIVE	(23:20) TCREG13_ACTIVE	0x0	ATC or DTC active value for address 13 mapping
		(19:16) TCREG12_ACTIVE	(19:16) TCREG12_ACTIVE	0x0	ATC or DTC active value for address 12 mapping
		(15:12) TCREG11_ACTIVE	(15:12) TCREG11_ACTIVE	0x0	ATC or DTC active value for address 11 mapping
		(11:8) TCREG10_ACTIVE	(11:8) TCREG10_ACTIVE	0x0	ATC or DTC active value for address 10 mapping
		(7:4) TCREG9_ACTIVE	(7:4) TCREG9_ACTIVE	0x0	ATC or DTC active value for address 9 mapping
		(3:0) TCREG8_ACTIVE	(3:0) TCREG8_ACTIVE	0x0	ATC or DTC active value for address 8 mapping
P:0x0B20 0x40002C80	D_FENG_CMDS FENG_CMDS	(3) SHADOW_TRANSFER	–	N/A	FENG Shadow to Active Transfer
		(2) EXEC	–	N/A	FENG Program Execution Start
		(1) DECODER_RESET	–	N/A	FENG Decoder Reset
		(0) AUDIO_RESET	–	N/A	FENG Audio Reset
P:0x0B21 0x40002C84	D_FENG_CTRL FENG_CTRL	(4) SYNC_START	–	N/A	If set, the FENG_ENABLE bit is set only after the SRIN arrival defined by the MRCNT_SYNC_SOURCE.

Address	Register Name	Register Write	Register Read	Default	Description
		(3) RESUME_CLEAN	–	N/A	If set, the DECODER_RESET is issued, all the pending sample ready flags are cleared and multirate counters are reset. Thereafter the ENABLE bit is set
		(2) PAUSE	(2) PAUSE	0x0	If set, at the next execution start FENG is paused
		(1) SINGLE_STEP	–	N/A	If set, the CTRL_ENABLE is set for one clock cycle
		(0) ENABLE	(0) ENABLE	0x0	If cleared, all the clocks are shut down (CFX can still access FENG)
P:0x0B22 0x40002C88	D_FENG_WR_KEY FENG_WR_KEY	(31:0) FENG_WR_KEY	–	N/A	The FENG_WR_PROTECT register write access grant key
P:0x0B23 0x40002C8C	D_FENG_WR_PROTECT FENG_WR_PROTECT	–	(5) WR_KEY_STATUS	0x0	If set, the FENG_WR_KEY has been entered properly and there has not been an access to FENG_WR_PROTECT thereafter
		(4) CFG_WR_PROTECT	(4) CFG_WR_PROTECT	0x0	If set, the configuration registers content cannot be updated
		(3) RF_WR_PROTECT	(3) RF_WR_PROTECT	0x0	If set, the register file content cannot be updated
		(2) COEF_WR_PROTECT	(2) COEF_WR_PROTECT	0x0	If set, the coefficient memory cannot be updated
		(1) SV_WR_PROTECT	(1) SV_WR_PROTECT	0x0	If set, the state variable memory cannot be updated
		(0) PGM_WR_PROTECT	(0) PGM_WR_PROTECT	0x0	If set, the program memory cannot be updated
P:0x0B24 0x40002C90	D_FENG_PGM_ADDR FENG_PGM_ADDR	–	(7:0) DECODER	0x0	Program memory decoder address

A.18 FENG REGISTER FILE

Address	Register Name	Register Write	Register Read	Default	Description
P:0x0B40-P:0x0B5F 0x40002D00 - 0x40002D7C	D_FENG_RF_EXTENDED FENG_RF_EXTENDED_*	(26:0) RF_EXTENDED	(26:0) RF_EXTENDED	0x0	Data read in m5p23 format, written in m4p23 format (sign extended to 32bits)
P:0x0B60-P:0x0BA5 0x40002D80 - 0x40002E94	D_FENG_RF_SINGLE FENG_RF_SINGLE_*	(23:0) RF_SINGLE	(23:0) RF_SINGLE	0x0	Data in m1p23 format (sign extended to 32bits)
P:0x0BA6-P:0x0BB3 0x40002E98 - 0x40002ECC	D_FENG_RF_IN FENG_RF_IN_*	(23:0) RF_IN	(23:0) RF_IN	0x0	Data in m1p23 format (sign extended to 32bits)
P:0x0BB4-P:0x0BBF 0x40002ED0 - 0x40002EFC	D_FENG_RF_OUT FENG_RF_OUT_*	(23:0) RF_OUT	(23:0) RF_OUT	0x0	Data in m1p23 format (sign extended to 32bits)

A.19 CHIP IDENTIFICATION

Address	Register Name	Register Write	Register Read	Default	Description
0x1FFFFFFC	AHBREGS_CHIP_ID_NUM	–	(31:24) CHIP_FAMILY	0xA	Chip Family number
		–	(23:16) CHIP_VERSION	0x1	Chip Version number
		–	(15:8) CHIP_MAJOR_REVISION	0x1	Chip Major Revision number
		–	(7:0) CHIP_MINOR_REVISION	0x1	Chip Minor Revision number

A.20 SYSTEM CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1000 0x40000000	SYSCTRL_DBG_LOCK	(0) DBG_LOCK	(0) DBG_LOCK	0x1	Debug port access lock
P:0x1001 0x40000004	SYSCTRL_DSP_CTRL	–	(8) DSP_RUNNING	0x0	DSP running status
		(4) DSP_CM3_CFX_NNA_INT_RESET	–	N/A	Write a 1 to reset pending CM3_CFX and NNA interrupts in the DSP interrupt controller
		(3) DSP_DMA_INT_RESET	–	N/A	Write a 1 to reset pending DMA interrupts in the DSP interrupt controller
		(2) DSP_RESET	–	N/A	Write a 1 to reset DSP
		(1) DSP_PAUSE	–	N/A	Write a 1 to pause DSP
		(0) DSP_RESUME	–	N/A	Write a 1 to run DSP
P:0x1002 0x40000008	SYSCTRL_DSP_DEBUG_CFG	(1) DSP_EXIT_POWERDOWN_WHEN_HALTED	(1) DSP_EXIT_POWERDOWN_WHEN_HALTED	0x0	DSP exit powerdown mode configuration when halted
		(0) DSP_DEBUG_ENABLE	(0) DSP_DEBUG_ENABLE	0x0	DSP debug port enable
P:0x1003 0x4000000C	SYSCTRL_DSP_CMD	(5) DSP_CMD_5	–	N/A	Write a 1 to issue DSP command 5
		(4) DSP_CMD_4	–	N/A	Write a 1 to issue DSP command 4
		(3) DSP_CMD_3	–	N/A	Write a 1 to issue DSP command 3
		(2) DSP_CMD_2	–	N/A	Write a 1 to issue DSP command 2
		(1) DSP_CMD_1	–	N/A	Write a 1 to issue DSP command 1

Address	Register Name	Register Write	Register Read	Default	Description
		(0) DSP_CMD_0	–	N/A	Write a 1 to issue DSP command 0
P:0x1004 0x40000010	SYSCTRL_CFX_CMD	(7) CFX_CMD_7	–	N/A	Write a 1 to issue CFX command 7
		(6) CFX_CMD_6	–	N/A	Write a 1 to issue CFX command 6
		(5) CFX_CMD_5	–	N/A	Write a 1 to issue CFX command 5
		(4) CFX_CMD_4	–	N/A	Write a 1 to issue CFX command 4
		(3) CFX_CMD_3	–	N/A	Write a 1 to issue CFX command 3
		(2) CFX_CMD_2	–	N/A	Write a 1 to issue CFX command 2
		(1) CFX_CMD_1	–	N/A	Write a 1 to issue CFX command 1
		(0) CFX_CMD_0	–	N/A	Write a 1 to issue CFX command 0
P:0x1005 0x40000014	SYSCTRL_CM3_LOOP_CACHE_CFG	(0) CM3_LOOP_CACHE_ENABLE	(0) CM3_LOOP_CACHE_ENABLE	0x1	Arm Cortex-M3 loop cache enable
P:0x1006 0x40000018	SYSCTRL_DSP_LOOP_CACHE_CFG	(0) DSP_LOOP_CACHE_ENABLE	(0) DSP_LOOP_CACHE_ENABLE	0x1	DSP loop cache enable
P:0x1007 0x4000001C	SYSCTRL_PRAM_CFG	(14) PRAM6_LOCK	–	N/A	Lock PRAM6 access except for IBus
		–	(14) PRAM6_LOCK_STATUS	0x0	PRAM6 access lock status
		(13) PRAM5_LOCK	–	N/A	Lock PRAM5 access except for IBus
		–	(13) PRAM5_LOCK_STATUS	0x0	PRAM5 access lock status
		(12) PRAM4_LOCK	–	N/A	Lock PRAM4 access except for IBus
		–	(12) PRAM4_LOCK_STATUS	0x0	PRAM4 access lock status

Address	Register Name	Register Write	Register Read	Default	Description
			STATUS		
		(11) PRAM3_LOCK	–	N/A	Lock PRAM3 access except for IBus
		–	(11) PRAM3_LOCK_STATUS	0x0	PRAM3 access lock status
		(10) PRAM2_LOCK	–	N/A	Lock PRAM2 access except for IBus
		–	(10) PRAM2_LOCK_STATUS	0x0	PRAM2 access lock status
		(9) PRAM1_LOCK	–	N/A	Lock PRAM1 access except for IBus
		–	(9) PRAM1_LOCK_STATUS	0x0	PRAM1 access lock status
		(8) PRAM0_LOCK	–	N/A	Lock PRAM0 access except for IBus
		–	(8) PRAM0_LOCK_STATUS	0x0	PRAM0 access lock status
		(2) PRAM_ASSIGN_LOCK	–	N/A	Lock PRAM0 and PRAM1 assignments
		–	(2) PRAM_ASSIGN_LOCK_STATUS	0x0	PRAM0 and PRAM1 assignments lock status
		(1) PRAM1_ASSIGN	(1) PRAM1_ASSIGN	0x0	Assign PRAM1 to the Arm Cortex-M3 core or CFX
		(0) PRAM0_ASSIGN	(0) PRAM0_ASSIGN	0x0	Assign PRAM0 to the Arm Cortex-M3 core or CFX
P:0x1008 0x40000020	SYSCTRL_MEM_POWER_CFG0	(26) DSP_BRAM2_POWER	(26) DSP_BRAM2_POWER	0x1	DSP BRAM2 power configuration
		(25) DSP_BRAM1_POWER	(25) DSP_BRAM1_POWER	0x1	DSP BRAM1 power configuration
		(24) DSP_BRAM0_POWER	(24) DSP_BRAM0_POWER	0x1	DSP BRAM0 power configuration

Address	Register Name	Register Write	Register Read	Default	Description
		(23) DSP_ARAM5_POWER	(23) DSP_ARAM5_POWER	0x1	DSP ARAM5 power configuration
		(22) DSP_ARAM4_POWER	(22) DSP_ARAM4_POWER	0x1	DSP ARAM4 power configuration
		(21) DSP_ARAM3_POWER	(21) DSP_ARAM3_POWER	0x1	DSP ARAM3 power configuration
		(20) DSP_ARAM2_POWER	(20) DSP_ARAM2_POWER	0x1	DSP ARAM2 power configuration
		(19) DSP_ARAM1_POWER	(19) DSP_ARAM1_POWER	0x1	DSP ARAM1 power configuration
		(18) DSP_ARAM0_POWER	(18) DSP_ARAM0_POWER	0x1	DSP ARAM0 power configuration
		(17) DSP_PRAM5_POWER	(17) DSP_PRAM5_POWER	0x1	DSP PRAM5 power configuration
		(16) DSP_PRAM4_POWER	(16) DSP_PRAM4_POWER	0x1	DSP PRAM4 power configuration
		(15) DSP_PRAM3_POWER	(15) DSP_PRAM3_POWER	0x1	DSP PRAM3 power configuration
		(14) DSP_PRAM2_POWER	(14) DSP_PRAM2_POWER	0x1	DSP PRAM2 power configuration
		(13) DSP_PRAM1_POWER	(13) DSP_PRAM1_POWER	0x1	DSP PRAM1 power configuration
		(12) DSP_PRAM0_POWER	(12) DSP_PRAM0_POWER	0x1	DSP PRAM0 power configuration
		(11) CM3_EMMC_RAM_POWER	(11) CM3_EMMC_RAM_POWER	0x1	eMMC RAM power configuration
		(10) CM3_DRAM3_POWER	(10) CM3_DRAM3_POWER	0x1	Arm Cortex-M3 DRAM3 power configuration

Address	Register Name	Register Write	Register Read	Default	Description
		(9) CM3_DRAM2_POWER	(9) CM3_DRAM2_POWER	0x1	Arm Cortex-M3 DRAM2 power configuration
		(8) CM3_DRAM1_POWER	(8) CM3_DRAM1_POWER	0x1	Arm Cortex-M3 DRAM1 power configuration
		(7) CM3_DRAM0_POWER	(7) CM3_DRAM0_POWER	0x1	Arm Cortex-M3 DRAM0 power configuration
		(6) CM3_PRAM6_POWER	(6) CM3_PRAM6_POWER	0x1	Arm Cortex-M3 PRAM6 power configuration
		(5) CM3_PRAM5_POWER	(5) CM3_PRAM5_POWER	0x1	Arm Cortex-M3 PRAM5 power configuration
		(4) CM3_PRAM4_POWER	(4) CM3_PRAM4_POWER	0x1	Arm Cortex-M3 PRAM4 power configuration
		(3) CM3_PRAM3_POWER	(3) CM3_PRAM3_POWER	0x1	Arm Cortex-M3 PRAM3 power configuration
		(2) CM3_PRAM2_POWER	(2) CM3_PRAM2_POWER	0x1	Arm Cortex-M3 PRAM2 power configuration
		(1) CM3_PRAM1_POWER	(1) CM3_PRAM1_POWER	0x1	Arm Cortex-M3 PRAM1 power configuration
		(0) CM3_PRAM0_POWER	(0) CM3_PRAM0_POWER	0x1	Arm Cortex-M3 PRAM0 power configuration
P:0x1009 0x40000024	SYSCTRL_MEM_POWER_CFG1	(19) FENG_PRAM_POWER	(19) FENG_PRAM_POWER	0x1	FENG PRAM power configuration
		(18) FENG_COEF_POWER	(18) FENG_COEF_POWER	0x1	FENG COEF power configuration
		(17) FENG_STATE_POWER	(17) FENG_STATE_POWER	0x1	FENG STATE power configuration
		(16) HEAR_ABCD_PART3_	(16) HEAR_ABCD_	0x1	HEAR A0/A1/B0/B1/C0/C1/D0/D1

Address	Register Name	Register Write	Register Read	Default	Description
		POWER	PART3_POWER		memories part 3 power configuration
		(15) HEAR_ABCD_PART2_POWER	(15) HEAR_ABCD_PART2_POWER	0x1	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 2 power configuration
		(14) HEAR_ABCD_PART1_POWER	(14) HEAR_ABCD_PART1_POWER	0x1	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 1 power configuration
		(13) HEAR_ABCD_PART0_POWER	(13) HEAR_ABCD_PART0_POWER	0x1	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 0 power configuration
		(12) HEAR_HMEM_POWER	(12) HEAR_HMEM_POWER	0x1	HEAR HMEM power configuration
		(11) HEAR_PRAM1_POWER	(11) HEAR_PRAM1_POWER	0x1	HEAR PRAM1 power configuration
		(10) HEAR_PRAM0_POWER	(10) HEAR_PRAM0_POWER	0x1	HEAR PRAM0 power configuration
		(9) CFX_YRAM2_POWER	(9) CFX_YRAM2_POWER	0x1	CFX YRAM2 power configuration
		(8) CFX_YRAM1_POWER	(8) CFX_YRAM1_POWER	0x1	CFX YRAM1 power configuration
		(7) CFX_YRAM0_POWER	(7) CFX_YRAM0_POWER	0x1	CFX YRAM0 power configuration
		(6) CFX_XRAM3_POWER	(6) CFX_XRAM3_POWER	0x1	CFX XRAM3 power configuration
		(5) CFX_XRAM2_POWER	(5) CFX_XRAM2_POWER	0x1	CFX XRAM2 power configuration
		(4) CFX_XRAM1_POWER	(4) CFX_XRAM1_POWER	0x1	CFX XRAM1 power configuration
		(3) CFX_XRAM0_POWER	(3) CFX_XRAM0_POWER	0x1	CFX XRAM0 power configuration
		(2) CFX_PRAM1_POWER	(2) CFX_PRAM1_POWER	0x1	CFX PRAM1 power configuration
		(1) CFX_PRAM0_POWER	(1) CFX_PRAM0_POWER	0x1	CFX PRAM0 power configuration
		(0) CFX_PROM_POWER	(0) CFX_PROM_POWER	0x1	CFX PROM power configuration
P:0x100A 0x40000028	SYSCTRL_MEM_ACCESS_CFG0	(26) DSP_BRAM2_ACCESS	(26) DSP_BRAM2_ACCESS	0x1	DSP BRAM2 access configuration

Address	Register Name	Register Write	Register Read	Default	Description
		(25) DSP_BRAM1_ACCESS	(25) DSP_BRAM1_ACCESS	0x1	DSP BRAM1 access configuration
		(24) DSP_BRAM0_ACCESS	(24) DSP_BRAM0_ACCESS	0x1	DSP BRAM0 access configuration
		(23) DSP_ARAM5_ACCESS	(23) DSP_ARAM5_ACCESS	0x1	DSP ARAM5 access configuration
		(22) DSP_ARAM4_ACCESS	(22) DSP_ARAM4_ACCESS	0x1	DSP ARAM4 access configuration
		(21) DSP_ARAM3_ACCESS	(21) DSP_ARAM3_ACCESS	0x1	DSP ARAM3 access configuration
		(20) DSP_ARAM2_ACCESS	(20) DSP_ARAM2_ACCESS	0x1	DSP ARAM2 access configuration
		(19) DSP_ARAM1_ACCESS	(19) DSP_ARAM1_ACCESS	0x1	DSP ARAM1 access configuration
		(18) DSP_ARAM0_ACCESS	(18) DSP_ARAM0_ACCESS	0x1	DSP ARAM0 access configuration
		(17) DSP_PRAM5_ACCESS	(17) DSP_PRAM5_ACCESS	0x1	DSP PRAM5 access configuration
		(16) DSP_PRAM4_ACCESS	(16) DSP_PRAM4_ACCESS	0x1	DSP PRAM4 access configuration
		(15) DSP_PRAM3_ACCESS	(15) DSP_PRAM3_ACCESS	0x1	DSP PRAM3 access configuration
		(14) DSP_PRAM2_ACCESS	(14) DSP_PRAM2_ACCESS	0x1	DSP PRAM2 access configuration
		(13) DSP_PRAM1_ACCESS	(13) DSP_PRAM1_ACCESS	0x1	DSP PRAM1 access configuration
		(12) DSP_PRAM0_ACCESS	(12) DSP_PRAM0_ACCESS	0x1	DSP PRAM0 access configuration

Address	Register Name	Register Write	Register Read	Default	Description
		(11) CM3_EMMC_RAM_ACCESS	(11) CM3_EMMC_RAM_ACCESS	0x1	eMMC RAM access configuration
		(10) CM3_DRAM3_ACCESS	(10) CM3_DRAM3_ACCESS	0x1	Arm Cortex-M3 DRAM3 access configuration
		(9) CM3_DRAM2_ACCESS	(9) CM3_DRAM2_ACCESS	0x1	Arm Cortex-M3 DRAM2 access configuration
		(8) CM3_DRAM1_ACCESS	(8) CM3_DRAM1_ACCESS	0x1	Arm Cortex-M3 DRAM1 access configuration
		(7) CM3_DRAM0_ACCESS	(7) CM3_DRAM0_ACCESS	0x1	Arm Cortex-M3 DRAM0 access configuration
		(6) CM3_PRAM6_ACCESS	(6) CM3_PRAM6_ACCESS	0x1	Arm Cortex-M3 PRAM6 access configuration
		(5) CM3_PRAM5_ACCESS	(5) CM3_PRAM5_ACCESS	0x1	Arm Cortex-M3 PRAM5 access configuration
		(4) CM3_PRAM4_ACCESS	(4) CM3_PRAM4_ACCESS	0x1	Arm Cortex-M3 PRAM4 access configuration
		(3) CM3_PRAM3_ACCESS	(3) CM3_PRAM3_ACCESS	0x1	Arm Cortex-M3 PRAM3 access configuration
		(2) CM3_PRAM2_ACCESS	(2) CM3_PRAM2_ACCESS	0x1	Arm Cortex-M3 PRAM2 access configuration
		(1) CM3_PRAM1_ACCESS	(1) CM3_PRAM1_ACCESS	0x1	Arm Cortex-M3 PRAM1 access configuration
		(0) CM3_PRAM0_ACCESS	(0) CM3_PRAM0_ACCESS	0x1	Arm Cortex-M3 PRAM0 access configuration
P:0x100B 0x4000002C	SYSCTRL_MEM_ACCESS_CFG1	(19) FENG_PRAM_ACCESS	(19) FENG_PRAM_ACCESS	0x1	FENG PRAM access configuration
		(18) FENG_COEF_ACCESS	(18) FENG_COEF_	0x1	FENG COEF access configuration

Address	Register Name	Register Write	Register Read	Default	Description
			ACCESS		
		(17) FENG_STATE_ACCESS	(17) FENG_STATE_ACCESS	0x1	FENG STATE access configuration
		(16) HEAR_ABCD_PART3_ACCESS	(16) HEAR_ABCD_PART3_ACCESS	0x1	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 3 access configuration
		(15) HEAR_ABCD_PART2_ACCESS	(15) HEAR_ABCD_PART2_ACCESS	0x1	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 2 access configuration
		(14) HEAR_ABCD_PART1_ACCESS	(14) HEAR_ABCD_PART1_ACCESS	0x1	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 1 access configuration
		(13) HEAR_ABCD_PART0_ACCESS	(13) HEAR_ABCD_PART0_ACCESS	0x1	HEAR A0/A1/B0/B1/C0/C1/D0/D1 memories part 0 access configuration
		(12) HEAR_HMEM_ACCESS	(12) HEAR_HMEM_ACCESS	0x1	HEAR HMEM access configuration
		(11) HEAR_PRAM1_ACCESS	(11) HEAR_PRAM1_ACCESS	0x1	HEAR PRAM1 access configuration
		(10) HEAR_PRAM0_ACCESS	(10) HEAR_PRAM0_ACCESS	0x1	HEAR PRAM0 access configuration
		(9) CFX_YRAM2_ACCESS	(9) CFX_YRAM2_ACCESS	0x1	CFX YRAM2 access configuration
		(8) CFX_YRAM1_ACCESS	(8) CFX_YRAM1_ACCESS	0x1	CFX YRAM1 access configuration
		(7) CFX_YRAM0_ACCESS	(7) CFX_YRAM0_ACCESS	0x1	CFX YRAM0 access configuration
		(6) CFX_XRAM3_ACCESS	(6) CFX_XRAM3_ACCESS	0x1	CFX XRAM3 access configuration
		(5) CFX_XRAM2_ACCESS	(5) CFX_XRAM2_ACCESS	0x1	CFX XRAM2 access configuration
		(4) CFX_XRAM1_ACCESS	(4) CFX_XRAM1_ACCESS	0x1	CFX XRAM1 access configuration

Address	Register Name	Register Write	Register Read	Default	Description
			ACCESS		
		(3) CFX_XRAM0_ACCESS	(3) CFX_XRAM0_ACCESS	0x1	CFX XRAM0 access configuration
		(2) CFX_PRAM1_ACCESS	(2) CFX_PRAM1_ACCESS	0x1	CFX PRAM1 access configuration
		(1) CFX_PRAM0_ACCESS	(1) CFX_PRAM0_ACCESS	0x1	CFX PRAM0 access configuration
		(0) CFX_PROM_ACCESS	(0) CFX_PROM_ACCESS	0x1	CFX PROM access configuration
P:0x100E 0x40000038	SYSCTRL_MEM_ARBITER_CFG	(9:8) DSP_MAX_WAIT	(9:8) DSP_MAX_WAIT	0x3	DSP maximum wait cycles configuration
		(7:6) CFX_MAX_WAIT	(7:6) CFX_MAX_WAIT	0x3	CFX maximum wait cycles configuration
		(5:4) DMA_MAX_WAIT	(5:4) DMA_MAX_WAIT	0x3	DMA maximum wait cycles configuration
		(3:2) CM3_MAX_WAIT	(3:2) CM3_MAX_WAIT	0x3	Arm Cortex-M3 maximum wait cycles configuration
		(1:0) PRIORITY_ORDER	(1:0) PRIORITY_ORDER	0x0	Arm Cortex-M3 and DSP memory priority order configuration
P:0x100F 0x4000003C	SYSCTRL_MEM_ERROR	–	(3) DMA_MEM_ERROR	0x0	DMA memory error flag
		–	(2) CFX_MEM_ERROR	0x0	CFX memory error flag
		–	(1) DSP_MEM_ERROR	0x0	DSP memory error flag
		(0) MEM_ERROR_CLEAR	–	N/A	Write a 1 to clear the memory error flags
P:0x1010 0x40000040	SYSCTRL_CNT_CTRL	–	(8) CNT_STATUS	0x0	Activity counters status bit
		(2) CNT_CLEAR	–	N/A	Clear activity counters

Address	Register Name	Register Write	Register Read	Default	Description
		(1) CNT_STOP	–	N/A	Stop activity counters
		(0) CNT_START	–	N/A	Start activity counters
P:0x1011 0x40000044	SYSCTRL_SYSCLK_CNT	(31:0) SYSCLK_CNT	(31:0) SYSCLK_CNT	0x0	System clock counter value
P:0x1012 0x40000048	SYSCTRL_CM3_CNT	(31:0) CM3_CNT	(31:0) CM3_CNT	0x0	Arm Cortex-M3 activity counter value
P:0x1013 0x4000004C	SYSCTRL_DSP_CNT	(31:0) DSP_CNT	(31:0) DSP_CNT	0x0	DSP activity counter value
P:0x1014 0x40000050	SYSCTRL_CFX_CNT	(31:0) CFX_CNT	(31:0) CFX_CNT	0x0	CFX activity counter value
P:0x1015 0x40000054	SYSCTRL_HEAR_CNT	(31:0) HEAR_CNT	(31:0) HEAR_CNT	0x0	HEAR activity counter value
P:0x1016 0x40000058	SYSCTRL_FENG_CNT	(31:0) FENG_CNT	(31:0) FENG_CNT	0x0	FENG activity counter value
P:0x1019 0x40000064	SYSCTRL_BOOT_STATUS	–	(0) STATUS	0x0	Arm Cortex-M3 boot status

A.21 GENERAL-PURPOSE TIMERS

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1040 0x40000100	TIMER0_CFG0	(26:24) PRESCALE	(26:24) PRESCALE	0x0	Prescale value of the timer
		(23:0) TIMEOUT_VALUE	(23:0) TIMEOUT_VALUE	0x0	Number of Timer clock cycles to time-out
P:0x1041 0x40000104	TIMER0_CFG1	(10:8) MULTI_COUNT	(10:8) MULTI_COUNT	0x0	Multi-count value
		(6:4) DIO_INT_SRC	(6:4) DIO_INT_SRC	0x0	DIO interrupt source selection
		(2) DIO_INT_MODE	(2) DIO_INT_MODE	0x0	DIO interrupt capture mode
		(1) DIO_INT_ENABLE	(1) DIO_INT_ENABLE	0x0	DIO interrupt capture enable
		(0) MODE	(0) MODE	0x0	Timer mode
P:0x1042 0x40000108	TIMER0_CTRL	–	(8) BUSY	0x0	Indicate if the timer is active or not
		(1) START	–	N/A	Start or restart the timer
		(0) STOP	–	N/A	Stop the timer
P:0x1043 0x4000010C	TIMER0_VAL	–	(26:24) MULTI_COUNT_VAL	0x0	Current multi counter value
		–	(23:0) TIMER_VALUE	0x0	Current timer value
P:0x1044 0x40000110	TIMER0_VAL_CAPTURE	(26:24) MULTI_COUNT_VAL_DIO	(26:24) MULTI_COUNT_VAL_DIO	0x0	Current multi counter value captured by the DIO interrupt defined in DIO_INT_SRC
		(23:0) TIMER_VALUE_DIO	(23:0) TIMER_VALUE_DIO	0x0	Timer value captured by the DIO interrupt defined in DIO_INT_SRC

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1048 0x40000120	TIMER1_CFG0	(26:24) PRESCALE	(26:24) PRESCALE	0x0	Prescale value of the timer
		(23:0) TIMEOUT_VALUE	(23:0) TIMEOUT_VALUE	0x0	Number of Timer clock cycles to time-out
P:0x1049 0x40000124	TIMER1_CFG1	(10:8) MULTI_COUNT	(10:8) MULTI_COUNT	0x0	Multi-count value
		(6:4) DIO_INT_SRC	(6:4) DIO_INT_SRC	0x0	DIO interrupt source selection
		(2) DIO_INT_MODE	(2) DIO_INT_MODE	0x0	DIO interrupt capture mode
		(1) DIO_INT_ENABLE	(1) DIO_INT_ENABLE	0x0	DIO interrupt capture enable
		(0) MODE	(0) MODE	0x0	Timer mode
P:0x104A 0x40000128	TIMER1_CTRL	-	(8) BUSY	0x0	Indicate if the timer is active or not
		(1) START	-	N/A	Start or restart the timer
		(0) STOP	-	N/A	Stop the timer
P:0x104B 0x4000012C	TIMER1_VAL	-	(26:24) MULTI_COUNT_VAL	0x0	Current multi counter value
		-	(23:0) TIMER_VALUE	0x0	Current timer value
P:0x104C 0x40000130	TIMER1_VAL_CAPTURE	(26:24) MULTI_COUNT_VAL_DIO	(26:24) MULTI_COUNT_VAL_DIO	0x0	Current multi counter value captured by the DIO interrupt defined in DIO_INT_SRC
		(23:0) TIMER_VALUE_DIO	(23:0) TIMER_VALUE_DIO	0x0	Timer value captured by the DIO interrupt defined in DIO_INT_SRC
P:0x1050 0x40000140	TIMER2_CFG0	(26:24) PRESCALE	(26:24) PRESCALE	0x0	Prescale value of the timer

Address	Register Name	Register Write	Register Read	Default	Description
		(23:0) TIMEOUT_VALUE	(23:0) TIMEOUT_VALUE	0x0	Number of Timer clock cycles to time-out
P:0x1051 0x40000144	TIMER2_CFG1	(10:8) MULTI_COUNT	(10:8) MULTI_COUNT	0x0	Multi-count value
		(6:4) DIO_INT_SRC	(6:4) DIO_INT_SRC	0x0	DIO interrupt source selection
		(2) DIO_INT_MODE	(2) DIO_INT_MODE	0x0	DIO interrupt capture mode
		(1) DIO_INT_ENABLE	(1) DIO_INT_ENABLE	0x0	DIO interrupt capture enable
		(0) MODE	(0) MODE	0x0	Timer mode
P:0x1052 0x40000148	TIMER2_CTRL	-	(8) BUSY	0x0	Indicate if the timer is active or not
		(1) START	-	N/A	Start or restart the timer
		(0) STOP	-	N/A	Stop the timer
P:0x1053 0x4000014C	TIMER2_VAL	-	(26:24) MULTI_COUNT_VAL	0x0	Current multi counter value
		-	(23:0) TIMER_VALUE	0x0	Current timer value
P:0x1054 0x40000150	TIMER2_VAL_CAPTURE	(26:24) MULTI_COUNT_VAL_DIO	(26:24) MULTI_COUNT_VAL_DIO	0x0	Current multi counter value captured by the DIO interrupt defined in DIO_INT_SRC
		(23:0) TIMER_VALUE_DIO	(23:0) TIMER_VALUE_DIO	0x0	Timer value captured by the DIO interrupt defined in DIO_INT_SRC
P:0x1058 0x40000160	TIMER3_CFG0	(26:24) PRESCALE	(26:24) PRESCALE	0x0	Prescale value of the timer
		(23:0) TIMEOUT_VALUE	(23:0) TIMEOUT_VALUE	0x0	Number of Timer clock cycles to time-out

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1059 0x40000164	TIMER3_CFG1	(10:8) MULTI_COUNT	(10:8) MULTI_COUNT	0x0	Multi-count value
		(6:4) DIO_INT_SRC	(6:4) DIO_INT_SRC	0x0	DIO interrupt source selection
		(2) DIO_INT_MODE	(2) DIO_INT_MODE	0x0	DIO interrupt capture mode
		(1) DIO_INT_ENABLE	(1) DIO_INT_ENABLE	0x0	DIO interrupt capture enable
		(0) MODE	(0) MODE	0x0	Timer mode
P:0x105A 0x40000168	TIMER3_CTRL	-	(8) BUSY	0x0	Indicate if the timer is active or not
		(1) START	-	N/A	Start or restart the timer
		(0) STOP	-	N/A	Stop the timer
P:0x105B 0x4000016C	TIMER3_VAL	-	(26:24) MULTI_COUNT_VAL	0x0	Current multi counter value
		-	(23:0) TIMER_VALUE	0x0	Current timer value
P:0x105C 0x40000170	TIMER3_VAL_CAPTURE	(26:24) MULTI_COUNT_VAL_DIO	(26:24) MULTI_COUNT_VAL_DIO	0x0	Current multi counter value captured by the DIO interrupt defined in DIO_INT_SRC
		(23:0) TIMER_VALUE_DIO	(23:0) TIMER_VALUE_DIO	0x0	Timer value captured by the DIO interrupt defined in DIO_INT_SRC

A.22 DMA CONTROLLER CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1080 0x40000200	DMA0_CFG0	(31) COMPLETE_INT_ENABLE	(31) COMPLETE_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer completes
		(30) CNT_INT_ENABLE	(30) CNT_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer reaches the counter value
		(29) DEST_ADDR_LSB_TOGGLE	(29) DEST_ADDR_LSB_TOGGLE	0x0	Enable an address LSB toggling for the destination
		(28) SRC_ADDR_LSB_TOGGLE	(28) SRC_ADDR_LSB_TOGGLE	0x0	Enable an address LSB toggling for the source
		(27:24) DEST_ADDR_STEP	(27:24) DEST_ADDR_STEP	0x0	Configure whether the destination address increments/decrements in terms of destination word size
		(23:20) SRC_ADDR_STEP	(23:20) SRC_ADDR_STEP	0x0	Configure whether the source address increments/decrements in terms of source word size
		(19:14) SRC_DEST_WORD_SIZE	(19:14) SRC_DEST_WORD_SIZE	0x0	Select the source and destination word sizes for the transfer
		(13:10) DEST_SELECT	(13:10) DEST_SELECT	0x0	Select the request line for the destination
		(8:5) SRC_SELECT	(8:5) SRC_SELECT	0x0	Select the request line for the source
		(3:2) CHANNEL_PRIORITY	(3:2) CHANNEL_PRIORITY	0x0	Select the priority level for this channel
		(1) SRC_DEST_TRANS_LENGTH_SEL	(1) SRC_DEST_TRANS_LENGTH_SEL	0x0	Selects whether the transfer length counter depends on either the source word counts or the destination word count
		(0) BYTE_ORDER	(0) BYTE_ORDER	0x0	Select the byte ordering of the DMA

Address	Register Name	Register Write	Register Read	Default	Description
					channel
P:0x1081 0x40000204	DMA0_CFG1	(31:16) INT_TRANSFER_LENGTH	(31:16) INT_TRANSFER_LENGTH	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value
		(15:0) TRANSFER_LENGTH	(15:0) TRANSFER_LENGTH	0x0	The length, in words, of each data transfer using DMA channel
P:0x1082 0x40000208	DMA0_CTRL	(13) INT_CNT_TRIGGER_ENABLE	(13) INT_CNT_TRIGGER_ENABLE	0x0	Enable waiting on a trigger when an interrupt counter event occurs
		(12:8) TRIGGER_SOURCE	(12:8) TRIGGER_SOURCE	0x0	Selects which event triggers this DMA channel when triggering is enabled
		(6) INT_CNT_ADDR_WRAP_ENABLE	(6) INT_CNT_ADDR_WRAP_ENABLE	0x0	Enable source or destination (depending on SRC_DEST_TRANS_LENGTH_SEL) address wrapping when an interrupt counter event occurs
		(5) CLEAR_BUFFER_WHEN_WRAPPING	(5) CLEAR_BUFFER_WHEN_WRAPPING	0x0	Clear buffer during address wrapping
		(4) BUFFER_CLEAR	–	N/A	Clear source buffer
		(3) CNTS_CLEAR	–	N/A	Clear counters
		(2:0) MODE_ENABLE	(2:0) MODE_ENABLE	0x0	Enable mode of operation of the DMA Channel
P:0x1083 0x4000020C	DMA0_STATUS	–	(10) ACTIVE	0x0	Active status of the channel
		–	(9) CNT_INT	0x0	Indicate if a counter interrupt has occurred on DMA channel
		–	(8) COMPLETE_INT	0x0	Indicate if a complete interrupt has occurred on DMA channel

Address	Register Name	Register Write	Register Read	Default	Description
		(6) CNT_INT_CLEAR	–	N/A	Clear the counter interrupt flag
		(5) COMPLETE_INT_CLEAR	–	N/A	Clear the complete interrupt flag
		(4) SRC_BUFFER_FILL_LVL_WR	–	N/A	Enable writing of SRC_BUFFER_FILL_LVL
		(3:0) SRC_BUFFER_FILL_LVL	(3:0) SRC_BUFFER_FILL_LVL	0x0	The number of nibbles currently in the source buffer (0: buffer is empty, 8: the 32-bit buffer is completely full)
P:0x1084 0x40000210	DMA0_SRC_ADDR	(31:0) SRC_ADDR	(31:0) SRC_ADDR	0x0	Address for the source of data transferred using DMA channel
P:0x1085 0x40000214	DMA0_DEST_ADDR	(31:0) DEST_ADDR	(31:0) DEST_ADDR	0x0	Address for the destination of data transferred using DMA channel
P:0x1086 0x40000218	DMA0_CNTS	(31:16) INT_TRANSFER_WORD_CNT	(31:16) INT_TRANSFER_WORD_CNT	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value
		(15:0) TRANSFER_WORD_CNT	(15:0) TRANSFER_WORD_CNT	0x0	The number of words that have been transferred using DMA channel during the current transfer
P:0x1087 0x4000021C	DMA0_SRC_BUFFER	(31:0) SRC_BUFFER	(31:0) SRC_BUFFER	0x0	Packed word which has been read from the source addresses.
P:0x1088 0x40000220	DMA1_CFG0	(31) COMPLETE_INT_ENABLE	(31) COMPLETE_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer completes
		(30) CNT_INT_ENABLE	(30) CNT_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer reaches the counter value
		(29) DEST_ADDR_LSB_TOGGLE	(29) DEST_ADDR_LSB_TOGGLE	0x0	Enable an address LSB toggling for the destination

Address	Register Name	Register Write	Register Read	Default	Description
		(28) SRC_ADDR_LSB_TOGGLE	(28) SRC_ADDR_LSB_TOGGLE	0x0	Enable an address LSB toggling for the source
		(27:24) DEST_ADDR_STEP	(27:24) DEST_ADDR_STEP	0x0	Configure whether the destination address increments/decrements in terms of destination word size
		(23:20) SRC_ADDR_STEP	(23:20) SRC_ADDR_STEP	0x0	Configure whether the source address increments/decrements in terms of source word size
		(19:14) SRC_DEST_WORD_SIZE	(19:14) SRC_DEST_WORD_SIZE	0x0	Select the source and destination word sizes for the transfer
		(13:10) DEST_SELECT	(13:10) DEST_SELECT	0x0	Select the request line for the destination
		(8:5) SRC_SELECT	(8:5) SRC_SELECT	0x0	Select the request line for the source
		(3:2) CHANNEL_PRIORITY	(3:2) CHANNEL_PRIORITY	0x0	Select the priority level for this channel
		(1) SRC_DEST_TRANS_LENGTH_SEL	(1) SRC_DEST_TRANS_LENGTH_SEL	0x0	Selects whether the transfer length counter depends on either the source word counts or the destination word count
		(0) BYTE_ORDER	(0) BYTE_ORDER	0x0	Select the byte ordering of the DMA channel
P:0x1089 0x40000224	DMA1_CFG1	(31:16) INT_TRANSFER_LENGTH	(31:16) INT_TRANSFER_LENGTH	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value
		(15:0) TRANSFER_LENGTH	(15:0) TRANSFER_LENGTH	0x0	The length, in words, of each data transfer using DMA channel
P:0x108A 0x40000228	DMA1_CTRL	(13) INT_CNT_TRIGGER_ENABLE	(13) INT_CNT_TRIGGER_ENABLE	0x0	Enable waiting on a trigger when an interrupt counter event occurs

Address	Register Name	Register Write	Register Read	Default	Description
		(12:8) TRIGGER_SOURCE	(12:8) TRIGGER_SOURCE	0x0	Selects which event triggers this DMA channel when triggering is enabled
		(6) INT_CNT_ADDR_WRAP_ENABLE	(6) INT_CNT_ADDR_WRAP_ENABLE	0x0	Enable source or destination (depending on SRC_DEST_TRANS_LENGTH_SEL) address wrapping when an interrupt counter event occurs
		(5) CLEAR_BUFFER_WHEN_WRAPPING	(5) CLEAR_BUFFER_WHEN_WRAPPING	0x0	Clear buffer during address wrapping
		(4) BUFFER_CLEAR	–	N/A	Clear source buffer
		(3) CNTS_CLEAR	–	N/A	Clear counters
		(2:0) MODE_ENABLE	(2:0) MODE_ENABLE	0x0	Enable mode of operation of the DMA Channel
P:0x108B 0x4000022C	DMA1_STATUS	–	(10) ACTIVE	0x0	Active status of the channel
		–	(9) CNT_INT	0x0	Indicate if a counter interrupt has occurred on DMA channel
		–	(8) COMPLETE_INT	0x0	Indicate if a complete interrupt has occurred on DMA channel
		(6) CNT_INT_CLEAR	–	N/A	Clear the counter interrupt flag
		(5) COMPLETE_INT_CLEAR	–	N/A	Clear the complete interrupt flag
		(4) SRC_BUFFER_FILL_LVL_WR	–	N/A	Enable writing of SRC_BUFFER_FILL_LVL
		(3:0) SRC_BUFFER_FILL_LVL	(3:0) SRC_BUFFER_FILL_LVL	0x0	The number of nibbles currently in the source buffer (0: buffer is empty, 8: the 32-bit buffer is completely full)
P:0x108C	DMA1_SRC_ADDR	(31:0) SRC_ADDR	(31:0) SRC_ADDR	0x0	Address for the source of data

Address	Register Name	Register Write	Register Read	Default	Description
0x40000230					transferred using DMA channel
P:0x108D 0x40000234	DMA1_DEST_ADDR	(31:0) DEST_ADDR	(31:0) DEST_ADDR	0x0	Address for the destination of data transferred using DMA channel
P:0x108E 0x40000238	DMA1_CNTS	(31:16) INT_TRANSFER_WORD_CNT	(31:16) INT_TRANSFER_WORD_CNT	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value
		(15:0) TRANSFER_WORD_CNT	(15:0) TRANSFER_WORD_CNT	0x0	The number of words that have been transferred using DMA channel during the current transfer
P:0x108F 0x4000023C	DMA1_SRC_BUFFER	(31:0) SRC_BUFFER	(31:0) SRC_BUFFER	0x0	Packed word which has been read from the source addresses.
P:0x1090 0x40000240	DMA2_CFG0	(31) COMPLETE_INT_ENABLE	(31) COMPLETE_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer completes
		(30) CNT_INT_ENABLE	(30) CNT_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer reaches the counter value
		(29) DEST_ADDR_LSB_TOGGLE	(29) DEST_ADDR_LSB_TOGGLE	0x0	Enable an address LSB toggling for the destination
		(28) SRC_ADDR_LSB_TOGGLE	(28) SRC_ADDR_LSB_TOGGLE	0x0	Enable an address LSB toggling for the source
		(27:24) DEST_ADDR_STEP	(27:24) DEST_ADDR_STEP	0x0	Configure whether the destination address increments/decrements in terms of destination word size
		(23:20) SRC_ADDR_STEP	(23:20) SRC_ADDR_STEP	0x0	Configure whether the source address increments/decrements in terms of source word size
		(19:14) SRC_DEST_WORD_SIZE	(19:14) SRC_DEST_WORD_SIZE	0x0	Select the source and destination word sizes for the transfer

Address	Register Name	Register Write	Register Read	Default	Description
		(13:10) DEST_SELECT	(13:10) DEST_SELECT	0x0	Select the request line for the destination
		(8:5) SRC_SELECT	(8:5) SRC_SELECT	0x0	Select the request line for the source
		(3:2) CHANNEL_PRIORITY	(3:2) CHANNEL_PRIORITY	0x0	Select the priority level for this channel
		(1) SRC_DEST_TRANS_LENGTH_SEL	(1) SRC_DEST_TRANS_LENGTH_SEL	0x0	Selects whether the transfer length counter depends on either the source word counts or the destination word count
		(0) BYTE_ORDER	(0) BYTE_ORDER	0x0	Select the byte ordering of the DMA channel
P:0x1091 0x40000244	DMA2_CFG1	(31:16) INT_TRANSFER_LENGTH	(31:16) INT_TRANSFER_LENGTH	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value
		(15:0) TRANSFER_LENGTH	(15:0) TRANSFER_LENGTH	0x0	The length, in words, of each data transfer using DMA channel
P:0x1092 0x40000248	DMA2_CTRL	(13) INT_CNT_TRIGGER_ENABLE	(13) INT_CNT_TRIGGER_ENABLE	0x0	Enable waiting on a trigger when an interrupt counter event occurs
		(12:8) TRIGGER_SOURCE	(12:8) TRIGGER_SOURCE	0x0	Selects which event triggers this DMA channel when triggering is enabled
		(6) INT_CNT_ADDR_WRAP_ENABLE	(6) INT_CNT_ADDR_WRAP_ENABLE	0x0	Enable source or destination (depending on SRC_DEST_TRANS_LENGTH_SEL) address wrapping when an interrupt counter event occurs
		(5) CLEAR_BUFFER_WHEN_WRAPPING	(5) CLEAR_BUFFER_WHEN_WRAPPING	0x0	Clear buffer during address wrapping
		(4) BUFFER_CLEAR	–	N/A	Clear source buffer

Address	Register Name	Register Write	Register Read	Default	Description
		(3) CNTS_CLEAR	–	N/A	Clear counters
		(2:0) MODE_ENABLE	(2:0) MODE_ENABLE	0x0	Enable mode of operation of the DMA Channel
P:0x1093 0x4000024C	DMA2_STATUS	–	(10) ACTIVE	0x0	Active status of the channel
		–	(9) CNT_INT	0x0	Indicate if a counter interrupt has occurred on DMA channel
		–	(8) COMPLETE_INT	0x0	Indicate if a complete interrupt has occurred on DMA channel
		(6) CNT_INT_CLEAR	–	N/A	Clear the counter interrupt flag
		(5) COMPLETE_INT_CLEAR	–	N/A	Clear the complete interrupt flag
		(4) SRC_BUFFER_FILL_LVL_WR	–	N/A	Enable writing of SRC_BUFFER_FILL_LVL
		(3:0) SRC_BUFFER_FILL_LVL	(3:0) SRC_BUFFER_FILL_LVL	0x0	The number of nibbles currently in the source buffer (0: buffer is empty, 8: the 32-bit buffer is completely full)
P:0x1094 0x40000250	DMA2_SRC_ADDR	(31:0) SRC_ADDR	(31:0) SRC_ADDR	0x0	Address for the source of data transferred using DMA channel
P:0x1095 0x40000254	DMA2_DEST_ADDR	(31:0) DEST_ADDR	(31:0) DEST_ADDR	0x0	Address for the destination of data transferred using DMA channel
P:0x1096 0x40000258	DMA2_CNTS	(31:16) INT_TRANSFER_WORD_CNT	(31:16) INT_TRANSFER_WORD_CNT	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value
		(15:0) TRANSFER_WORD_CNT	(15:0) TRANSFER_WORD_CNT	0x0	The number of words that have been transferred using DMA channel during

Address	Register Name	Register Write	Register Read	Default	Description
					the current transfer
P:0x1097 0x4000025C	DMA2_SRC_BUFFER	(31:0) SRC_BUFFER	(31:0) SRC_BUFFER	0x0	Packed word which has been read from the source addresses.
P:0x1098 0x40000260	DMA3_CFG0	(31) COMPLETE_INT_ENABLE	(31) COMPLETE_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer completes
		(30) CNT_INT_ENABLE	(30) CNT_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer reaches the counter value
		(29) DEST_ADDR_LSB_TOGGLE	(29) DEST_ADDR_LSB_TOGGLE	0x0	Enable an address LSB toggling for the destination
		(28) SRC_ADDR_LSB_TOGGLE	(28) SRC_ADDR_LSB_TOGGLE	0x0	Enable an address LSB toggling for the source
		(27:24) DEST_ADDR_STEP	(27:24) DEST_ADDR_STEP	0x0	Configure whether the destination address increments/decrements in terms of destination word size
		(23:20) SRC_ADDR_STEP	(23:20) SRC_ADDR_STEP	0x0	Configure whether the source address increments/decrements in terms of source word size
		(19:14) SRC_DEST_WORD_SIZE	(19:14) SRC_DEST_WORD_SIZE	0x0	Select the source and destination word sizes for the transfer
		(13:10) DEST_SELECT	(13:10) DEST_SELECT	0x0	Select the request line for the destination
		(8:5) SRC_SELECT	(8:5) SRC_SELECT	0x0	Select the request line for the source
		(3:2) CHANNEL_PRIORITY	(3:2) CHANNEL_PRIORITY	0x0	Select the priority level for this channel
		(1) SRC_DEST_TRANS_LENGTH_SEL	(1) SRC_DEST_TRANS_LENGTH_SEL	0x0	Selects whether the transfer length counter depends on either the source word counts or the destination word

Address	Register Name	Register Write	Register Read	Default	Description
					count
		(0) BYTE_ORDER	(0) BYTE_ORDER	0x0	Select the byte ordering of the DMA channel
P:0x1099 0x40000264	DMA3_CFG1	(31:16) INT_TRANSFER_LENGTH	(31:16) INT_TRANSFER_LENGTH	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value
		(15:0) TRANSFER_LENGTH	(15:0) TRANSFER_LENGTH	0x0	The length, in words, of each data transfer using DMA channel
P:0x109A 0x40000268	DMA3_CTRL	(13) INT_CNT_TRIGGER_ENABLE	(13) INT_CNT_TRIGGER_ENABLE	0x0	Enable waiting on a trigger when an interrupt counter event occurs
		(12:8) TRIGGER_SOURCE	(12:8) TRIGGER_SOURCE	0x0	Selects which event triggers this DMA channel when triggering is enabled
		(6) INT_CNT_ADDR_WRAP_ENABLE	(6) INT_CNT_ADDR_WRAP_ENABLE	0x0	Enable source or destination (depending on SRC_DEST_TRANS_LENGTH_SEL) address wrapping when an interrupt counter event occurs
		(5) CLEAR_BUFFER_WHEN_WRAPPING	(5) CLEAR_BUFFER_WHEN_WRAPPING	0x0	Clear buffer during address wrapping
		(4) BUFFER_CLEAR	–	N/A	Clear source buffer
		(3) CNTS_CLEAR	–	N/A	Clear counters
		(2:0) MODE_ENABLE	(2:0) MODE_ENABLE	0x0	Enable mode of operation of the DMA Channel
P:0x109B 0x4000026C	DMA3_STATUS	–	(10) ACTIVE	0x0	Active status of the channel
		–	(9) CNT_INT	0x0	Indicate if a counter interrupt has occurred on DMA channel
		–	(8) COMPLETE_INT	0x0	Indicate if a complete interrupt has

Address	Register Name	Register Write	Register Read	Default	Description
					occurred on DMA channel
		(6) CNT_INT_CLEAR	–	N/A	Clear the counter interrupt flag
		(5) COMPLETE_INT_CLEAR	–	N/A	Clear the complete interrupt flag
		(4) SRC_BUFFER_FILL_LVL_WR	–	N/A	Enable writing of SRC_BUFFER_FILL_LVL
		(3:0) SRC_BUFFER_FILL_LVL	(3:0) SRC_BUFFER_FILL_LVL	0x0	The number of nibbles currently in the source buffer (0: buffer is empty, 8: the 32-bit buffer is completely full)
P:0x109C 0x40000270	DMA3_SRC_ADDR	(31:0) SRC_ADDR	(31:0) SRC_ADDR	0x0	Address for the source of data transferred using DMA channel
P:0x109D 0x40000274	DMA3_DEST_ADDR	(31:0) DEST_ADDR	(31:0) DEST_ADDR	0x0	Address for the destination of data transferred using DMA channel
P:0x109E 0x40000278	DMA3_CNTS	(31:16) INT_TRANSFER_WORD_CNT	(31:16) INT_TRANSFER_WORD_CNT	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value
		(15:0) TRANSFER_WORD_CNT	(15:0) TRANSFER_WORD_CNT	0x0	The number of words that have been transferred using DMA channel during the current transfer
P:0x109F 0x4000027C	DMA3_SRC_BUFFER	(31:0) SRC_BUFFER	(31:0) SRC_BUFFER	0x0	Packed word which has been read from the source addresses.
P:0x10A0 0x40000280	DMA4_CFG0	(31) COMPLETE_INT_ENABLE	(31) COMPLETE_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer completes
		(30) CNT_INT_ENABLE	(30) CNT_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer reaches the counter value

Address	Register Name	Register Write	Register Read	Default	Description
		(29) DEST_ADDR_LSB_TOGGLE	(29) DEST_ADDR_LSB_TOGGLE	0x0	Enable an address LSB toggling for the destination
		(28) SRC_ADDR_LSB_TOGGLE	(28) SRC_ADDR_LSB_TOGGLE	0x0	Enable an address LSB toggling for the source
		(27:24) DEST_ADDR_STEP	(27:24) DEST_ADDR_STEP	0x0	Configure whether the destination address increments/decrements in terms of destination word size
		(23:20) SRC_ADDR_STEP	(23:20) SRC_ADDR_STEP	0x0	Configure whether the source address increments/decrements in terms of source word size
		(19:14) SRC_DEST_WORD_SIZE	(19:14) SRC_DEST_WORD_SIZE	0x0	Select the source and destination word sizes for the transfer
		(13:10) DEST_SELECT	(13:10) DEST_SELECT	0x0	Select the request line for the destination
		(8:5) SRC_SELECT	(8:5) SRC_SELECT	0x0	Select the request line for the source
		(3:2) CHANNEL_PRIORITY	(3:2) CHANNEL_PRIORITY	0x0	Select the priority level for this channel
		(1) SRC_DEST_TRANS_LENGTH_SEL	(1) SRC_DEST_TRANS_LENGTH_SEL	0x0	Selects whether the transfer length counter depends on either the source word counts or the destination word count
		(0) BYTE_ORDER	(0) BYTE_ORDER	0x0	Select the byte ordering of the DMA channel
P:0x10A1 0x40000284	DMA4_CFG1	(31:16) INT_TRANSFER_LENGTH	(31:16) INT_TRANSFER_LENGTH	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value
		(15:0) TRANSFER_LENGTH	(15:0) TRANSFER_LENGTH	0x0	The length, in words, of each data transfer using DMA channel
P:0x10A2	DMA4_CTRL	(13) INT_CNT_TRIGGER_	(13) INT_CNT_	0x0	Enable waiting on a trigger when an

Address	Register Name	Register Write	Register Read	Default	Description
0x40000288		ENABLE	TRIGGER_ENABLE		interrupt counter event occurs
		(12:8) TRIGGER_SOURCE	(12:8) TRIGGER_SOURCE	0x0	Selects which event triggers this DMA channel when triggering is enabled
		(6) INT_CNT_ADDR_WRAP_ENABLE	(6) INT_CNT_ADDR_WRAP_ENABLE	0x0	Enable source or destination (depending on SRC_DEST_TRANS_LENGTH_SEL) address wrapping when an interrupt counter event occurs
		(5) CLEAR_BUFFER_WHEN_WRAPPING	(5) CLEAR_BUFFER_WHEN_WRAPPING	0x0	Clear buffer during address wrapping
		(4) BUFFER_CLEAR	–	N/A	Clear source buffer
		(3) CNTS_CLEAR	–	N/A	Clear counters
		(2:0) MODE_ENABLE	(2:0) MODE_ENABLE	0x0	Enable mode of operation of the DMA Channel
P:0x10A3 0x4000028C	DMA4_STATUS	–	(10) ACTIVE	0x0	Active status of the channel
		–	(9) CNT_INT	0x0	Indicate if a counter interrupt has occurred on DMA channel
		–	(8) COMPLETE_INT	0x0	Indicate if a complete interrupt has occurred on DMA channel
		(6) CNT_INT_CLEAR	–	N/A	Clear the counter interrupt flag
		(5) COMPLETE_INT_CLEAR	–	N/A	Clear the complete interrupt flag
		(4) SRC_BUFFER_FILL_LVL_WR	–	N/A	Enable writing of SRC_BUFFER_FILL_LVL
		(3:0) SRC_BUFFER_FILL_LVL	(3:0) SRC_BUFFER_FILL_LVL	0x0	The number of nibbles currently in the source buffer (0: buffer is empty, 8: the 32-bit buffer is completely full)

Address	Register Name	Register Write	Register Read	Default	Description
P:0x10A4 0x40000290	DMA4_SRC_ADDR	(31:0) SRC_ADDR	(31:0) SRC_ADDR	0x0	Address for the source of data transferred using DMA channel
P:0x10A5 0x40000294	DMA4_DEST_ADDR	(31:0) DEST_ADDR	(31:0) DEST_ADDR	0x0	Address for the destination of data transferred using DMA channel
P:0x10A6 0x40000298	DMA4_CNTS	(31:16) INT_TRANSFER_WORD_CNT	(31:16) INT_TRANSFER_WORD_CNT	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value
		(15:0) TRANSFER_WORD_CNT	(15:0) TRANSFER_WORD_CNT	0x0	The number of words that have been transferred using DMA channel during the current transfer
P:0x10A7 0x4000029C	DMA4_SRC_BUFFER	(31:0) SRC_BUFFER	(31:0) SRC_BUFFER	0x0	Packed word which has been read from the source addresses.
P:0x10A8 0x400002A0	DMA5_CFG0	(31) COMPLETE_INT_ENABLE	(31) COMPLETE_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer completes
		(30) CNT_INT_ENABLE	(30) CNT_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer reaches the counter value
		(29) DEST_ADDR_LSB_TOGGLE	(29) DEST_ADDR_LSB_TOGGLE	0x0	Enable an address LSB toggling for the destination
		(28) SRC_ADDR_LSB_TOGGLE	(28) SRC_ADDR_LSB_TOGGLE	0x0	Enable an address LSB toggling for the source
		(27:24) DEST_ADDR_STEP	(27:24) DEST_ADDR_STEP	0x0	Configure whether the destination address increments/decrements in terms of destination word size
		(23:20) SRC_ADDR_STEP	(23:20) SRC_ADDR_STEP	0x0	Configure whether the source address increments/decrements in terms of source word size

Address	Register Name	Register Write	Register Read	Default	Description
		(19:14) SRC_DEST_WORD_SIZE	(19:14) SRC_DEST_WORD_SIZE	0x0	Select the source and destination word sizes for the transfer
		(13:10) DEST_SELECT	(13:10) DEST_SELECT	0x0	Select the request line for the destination
		(8:5) SRC_SELECT	(8:5) SRC_SELECT	0x0	Select the request line for the source
		(3:2) CHANNEL_PRIORITY	(3:2) CHANNEL_PRIORITY	0x0	Select the priority level for this channel
		(1) SRC_DEST_TRANS_LENGTH_SEL	(1) SRC_DEST_TRANS_LENGTH_SEL	0x0	Selects whether the transfer length counter depends on either the source word counts or the destination word count
		(0) BYTE_ORDER	(0) BYTE_ORDER	0x0	Select the byte ordering of the DMA channel
P:0x10A9 0x400002A4	DMA5_CFG1	(31:16) INT_TRANSFER_LENGTH	(31:16) INT_TRANSFER_LENGTH	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value
		(15:0) TRANSFER_LENGTH	(15:0) TRANSFER_LENGTH	0x0	The length, in words, of each data transfer using DMA channel
P:0x10AA 0x400002A8	DMA5_CTRL	(13) INT_CNT_TRIGGER_ENABLE	(13) INT_CNT_TRIGGER_ENABLE	0x0	Enable waiting on a trigger when an interrupt counter event occurs
		(12:8) TRIGGER_SOURCE	(12:8) TRIGGER_SOURCE	0x0	Selects which event triggers this DMA channel when triggering is enabled
		(6) INT_CNT_ADDR_WRAP_ENABLE	(6) INT_CNT_ADDR_WRAP_ENABLE	0x0	Enable source or destination (depending on SRC_DEST_TRANS_LENGTH_SEL) address wrapping when an interrupt counter event occurs
		(5) CLEAR_BUFFER_WHEN_WRAPPING	(5) CLEAR_BUFFER_WHEN_WRAPPING	0x0	Clear buffer during address wrapping

Address	Register Name	Register Write	Register Read	Default	Description
		(4) BUFFER_CLEAR	–	N/A	Clear source buffer
		(3) CNTS_CLEAR	–	N/A	Clear counters
		(2:0) MODE_ENABLE	(2:0) MODE_ENABLE	0x0	Enable mode of operation of the DMA Channel
P:0x10AB 0x400002AC	DMA5_STATUS	–	(10) ACTIVE	0x0	Active status of the channel
		–	(9) CNT_INT	0x0	Indicate if a counter interrupt has occurred on DMA channel
		–	(8) COMPLETE_INT	0x0	Indicate if a complete interrupt has occurred on DMA channel
		(6) CNT_INT_CLEAR	–	N/A	Clear the counter interrupt flag
		(5) COMPLETE_INT_CLEAR	–	N/A	Clear the complete interrupt flag
		(4) SRC_BUFFER_FILL_LVL_WR	–	N/A	Enable writing of SRC_BUFFER_FILL_LVL
		(3:0) SRC_BUFFER_FILL_LVL	(3:0) SRC_BUFFER_FILL_LVL	0x0	The number of nibbles currently in the source buffer (0: buffer is empty, 8: the 32-bit buffer is completely full)
P:0x10AC 0x400002B0	DMA5_SRC_ADDR	(31:0) SRC_ADDR	(31:0) SRC_ADDR	0x0	Address for the source of data transferred using DMA channel
P:0x10AD 0x400002B4	DMA5_DEST_ADDR	(31:0) DEST_ADDR	(31:0) DEST_ADDR	0x0	Address for the destination of data transferred using DMA channel
P:0x10AE 0x400002B8	DMA5_CNTS	(31:16) INT_TRANSFER_WORD_CNT	(31:16) INT_TRANSFER_WORD_CNT	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value
		(15:0) TRANSFER_WORD_	(15:0) TRANSFER_	0x0	The number of words that have been

Address	Register Name	Register Write	Register Read	Default	Description
		CNT	WORD_CNT		transferred using DMA channel during the current transfer
P:0x10AF 0x400002BC	DMA5_SRC_BUFFER	(31:0) SRC_BUFFER	(31:0) SRC_BUFFER	0x0	Packed word which has been read from the source addresses.
P:0x10B0 0x400002C0	DMA6_CFG0	(31) COMPLETE_INT_ENABLE	(31) COMPLETE_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer completes
		(30) CNT_INT_ENABLE	(30) CNT_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer reaches the counter value
		(29) DEST_ADDR_LSB_TOGGLE	(29) DEST_ADDR_LSB_TOGGLE	0x0	Enable an address LSB toggling for the destination
		(28) SRC_ADDR_LSB_TOGGLE	(28) SRC_ADDR_LSB_TOGGLE	0x0	Enable an address LSB toggling for the source
		(27:24) DEST_ADDR_STEP	(27:24) DEST_ADDR_STEP	0x0	Configure whether the destination address increments/decrements in terms of destination word size
		(23:20) SRC_ADDR_STEP	(23:20) SRC_ADDR_STEP	0x0	Configure whether the source address increments/decrements in terms of source word size
		(19:14) SRC_DEST_WORD_SIZE	(19:14) SRC_DEST_WORD_SIZE	0x0	Select the source and destination word sizes for the transfer
		(13:10) DEST_SELECT	(13:10) DEST_SELECT	0x0	Select the request line for the destination
		(8:5) SRC_SELECT	(8:5) SRC_SELECT	0x0	Select the request line for the source
		(3:2) CHANNEL_PRIORITY	(3:2) CHANNEL_PRIORITY	0x0	Select the priority level for this channel
		(1) SRC_DEST_TRANS_LENGTH_SEL	(1) SRC_DEST_TRANS_LENGTH_SEL	0x0	Selects whether the transfer length counter depends on either the source

Address	Register Name	Register Write	Register Read	Default	Description
					word counts or the destination word count
		(0) BYTE_ORDER	(0) BYTE_ORDER	0x0	Select the byte ordering of the DMA channel
P:0x10B1 0x400002C4	DMA6_CFG1	(31:16) INT_TRANSFER_LENGTH	(31:16) INT_TRANSFER_LENGTH	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value
		(15:0) TRANSFER_LENGTH	(15:0) TRANSFER_LENGTH	0x0	The length, in words, of each data transfer using DMA channel
P:0x10B2 0x400002C8	DMA6_CTRL	(13) INT_CNT_TRIGGER_ENABLE	(13) INT_CNT_TRIGGER_ENABLE	0x0	Enable waiting on a trigger when an interrupt counter event occurs
		(12:8) TRIGGER_SOURCE	(12:8) TRIGGER_SOURCE	0x0	Selects which event triggers this DMA channel when triggering is enabled
		(6) INT_CNT_ADDR_WRAP_ENABLE	(6) INT_CNT_ADDR_WRAP_ENABLE	0x0	Enable source or destination (depending on SRC_DEST_TRANS_LENGTH_SEL) address wrapping when an interrupt counter event occurs
		(5) CLEAR_BUFFER_WHEN_WRAPPING	(5) CLEAR_BUFFER_WHEN_WRAPPING	0x0	Clear buffer during address wrapping
		(4) BUFFER_CLEAR	–	N/A	Clear source buffer
		(3) CNTS_CLEAR	–	N/A	Clear counters
		(2:0) MODE_ENABLE	(2:0) MODE_ENABLE	0x0	Enable mode of operation of the DMA Channel
P:0x10B3 0x400002CC	DMA6_STATUS	–	(10) ACTIVE	0x0	Active status of the channel
		–	(9) CNT_INT	0x0	Indicate if a counter interrupt has occurred on DMA channel

Address	Register Name	Register Write	Register Read	Default	Description
		–	(8) COMPLETE_INT	0x0	Indicate if a complete interrupt has occurred on DMA channel
		(6) CNT_INT_CLEAR	–	N/A	Clear the counter interrupt flag
		(5) COMPLETE_INT_CLEAR	–	N/A	Clear the complete interrupt flag
		(4) SRC_BUFFER_FILL_LVL_WR	–	N/A	Enable writing of SRC_BUFFER_FILL_LVL
		(3:0) SRC_BUFFER_FILL_LVL	(3:0) SRC_BUFFER_FILL_LVL	0x0	The number of nibbles currently in the source buffer (0: buffer is empty, 8: the 32-bit buffer is completely full)
P:0x10B4 0x400002D0	DMA6_SRC_ADDR	(31:0) SRC_ADDR	(31:0) SRC_ADDR	0x0	Address for the source of data transferred using DMA channel
P:0x10B5 0x400002D4	DMA6_DEST_ADDR	(31:0) DEST_ADDR	(31:0) DEST_ADDR	0x0	Address for the destination of data transferred using DMA channel
P:0x10B6 0x400002D8	DMA6_CNTS	(31:16) INT_TRANSFER_WORD_CNT	(31:16) INT_TRANSFER_WORD_CNT	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value
		(15:0) TRANSFER_WORD_CNT	(15:0) TRANSFER_WORD_CNT	0x0	The number of words that have been transferred using DMA channel during the current transfer
P:0x10B7 0x400002DC	DMA6_SRC_BUFFER	(31:0) SRC_BUFFER	(31:0) SRC_BUFFER	0x0	Packed word which has been read from the source addresses.
P:0x10B8 0x400002E0	DMA7_CFG0	(31) COMPLETE_INT_ENABLE	(31) COMPLETE_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer completes
		(30) CNT_INT_ENABLE	(30) CNT_INT_ENABLE	0x0	Raise an interrupt when the DMA transfer reaches the counter value

Address	Register Name	Register Write	Register Read	Default	Description
		(29) DEST_ADDR_LSB_TOGGLE	(29) DEST_ADDR_LSB_TOGGLE	0x0	Enable an address LSB toggling for the destination
		(28) SRC_ADDR_LSB_TOGGLE	(28) SRC_ADDR_LSB_TOGGLE	0x0	Enable an address LSB toggling for the source
		(27:24) DEST_ADDR_STEP	(27:24) DEST_ADDR_STEP	0x0	Configure whether the destination address increments/decrements in terms of destination word size
		(23:20) SRC_ADDR_STEP	(23:20) SRC_ADDR_STEP	0x0	Configure whether the source address increments/decrements in terms of source word size
		(19:14) SRC_DEST_WORD_SIZE	(19:14) SRC_DEST_WORD_SIZE	0x0	Select the source and destination word sizes for the transfer
		(13:10) DEST_SELECT	(13:10) DEST_SELECT	0x0	Select the request line for the destination
		(8:5) SRC_SELECT	(8:5) SRC_SELECT	0x0	Select the request line for the source
		(3:2) CHANNEL_PRIORITY	(3:2) CHANNEL_PRIORITY	0x0	Select the priority level for this channel
		(1) SRC_DEST_TRANS_LENGTH_SEL	(1) SRC_DEST_TRANS_LENGTH_SEL	0x0	Selects whether the transfer length counter depends on either the source word counts or the destination word count
		(0) BYTE_ORDER	(0) BYTE_ORDER	0x0	Select the byte ordering of the DMA channel
P:0x10B9 0x400002E4	DMA7_CFG1	(31:16) INT_TRANSFER_LENGTH	(31:16) INT_TRANSFER_LENGTH	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value
		(15:0) TRANSFER_LENGTH	(15:0) TRANSFER_LENGTH	0x0	The length, in words, of each data transfer using DMA channel
P:0x10BA	DMA7_CTRL	(13) INT_CNT_TRIGGER_	(13) INT_CNT_	0x0	Enable waiting on a trigger when an

Address	Register Name	Register Write	Register Read	Default	Description
0x400002E8		ENABLE	TRIGGER_ENABLE		interrupt counter event occurs
		(12:8) TRIGGER_SOURCE	(12:8) TRIGGER_SOURCE	0x0	Selects which event triggers this DMA channel when triggering is enabled
		(6) INT_CNT_ADDR_WRAP_ENABLE	(6) INT_CNT_ADDR_WRAP_ENABLE	0x0	Enable source or destination (depending on SRC_DEST_TRANS_LENGTH_SEL) address wrapping when an interrupt counter event occurs
		(5) CLEAR_BUFFER_WHEN_WRAPPING	(5) CLEAR_BUFFER_WHEN_WRAPPING	0x0	Clear buffer during address wrapping
		(4) BUFFER_CLEAR	–	N/A	Clear source buffer
		(3) CNTS_CLEAR	–	N/A	Clear counters
		(2:0) MODE_ENABLE	(2:0) MODE_ENABLE	0x0	Enable mode of operation of the DMA Channel
P:0x10BB 0x400002EC	DMA7_STATUS	–	(10) ACTIVE	0x0	Active status of the channel
		–	(9) CNT_INT	0x0	Indicate if a counter interrupt has occurred on DMA channel
		–	(8) COMPLETE_INT	0x0	Indicate if a complete interrupt has occurred on DMA channel
		(6) CNT_INT_CLEAR	–	N/A	Clear the counter interrupt flag
		(5) COMPLETE_INT_CLEAR	–	N/A	Clear the complete interrupt flag
		(4) SRC_BUFFER_FILL_LVL_WR	–	N/A	Enable writing of SRC_BUFFER_FILL_LVL
		(3:0) SRC_BUFFER_FILL_LVL	(3:0) SRC_BUFFER_FILL_LVL	0x0	The number of nibbles currently in the source buffer (0: buffer is empty, 8: the 32-bit buffer is completely full)

Address	Register Name	Register Write	Register Read	Default	Description
P:0x10BC 0x400002F0	DMA7_SRC_ADDR	(31:0) SRC_ADDR	(31:0) SRC_ADDR	0x0	Address for the source of data transferred using DMA channel
P:0x10BD 0x400002F4	DMA7_DEST_ADDR	(31:0) DEST_ADDR	(31:0) DEST_ADDR	0x0	Address for the destination of data transferred using DMA channel
P:0x10BE 0x400002F8	DMA7_CNTS	(31:16) INT_TRANSFER_WORD_CNT	(31:16) INT_TRANSFER_WORD_CNT	0x0	Trigger a counter interrupt when the DMA transfer word count reaches this value
		(15:0) TRANSFER_WORD_CNT	(15:0) TRANSFER_WORD_CNT	0x0	The number of words that have been transferred using DMA channel during the current transfer
P:0x10BF 0x400002FC	DMA7_SRC_BUFFER	(31:0) SRC_BUFFER	(31:0) SRC_BUFFER	0x0	Packed word which has been read from the source addresses.

A.23 AUDIO SINK CLOCK COUNTERS

Address	Register Name	Register Write	Register Read	Default	Description
P:0x10C0 0x40000300	AUDIOSINK0_CFG	(10:8) FRAME_SYNC_SEL	(10:8) FRAME_SYNC_SEL	0x0	Frame sync source selection
		(7:4) AUDIOSINK_SEL	(7:4) AUDIOSINK_SEL	0x0	Audio sink clock source selection
		(0) ENABLE	(0) ENABLE	0x0	Enable the Audio Sink Clock Counters
P:0x10C1 0x40000304	AUDIOSINK0_CTRL	(0) FRAME_SYNC_TRIGGER	-	N/A	Generate a frame sync pulse manually
P:0x10C2 0x40000308	AUDIOSINK0_STATUS	-	(0) PHASE_CNT_READY	0x0	Audio sink clock phase counter status
P:0x10C3 0x4000030C	AUDIOSINK0_CNT	-	(11:0) CNT	0x0	Sink clock counter value
P:0x10C4 0x40000310	AUDIOSINK0_PHASE_CNT	-	(13:0) PHASE_CNT	0x0	Sink clock phase counter value
P:0x1100 0x40000400	AUDIOSINK1_CFG	(10:8) FRAME_SYNC_SEL	(10:8) FRAME_SYNC_SEL	0x0	Frame sync source selection
		(7:4) AUDIOSINK_SEL	(7:4) AUDIOSINK_SEL	0x0	Audio sink clock source selection
		(0) ENABLE	(0) ENABLE	0x0	Enable the Audio Sink Clock Counters
P:0x1101 0x40000404	AUDIOSINK1_CTRL	(0) FRAME_SYNC_TRIGGER	-	N/A	Generate a frame sync pulse manually

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1102 0x40000408	AUDIOSINK1_STATUS	–	(0) PHASE_CNT_READY	0x0	Audio sink clock phase counter status
P:0x1103 0x4000040C	AUDIOSINK1_CNT	–	(11:0) CNT	0x0	Sink clock counter value
P:0x1104 0x40000410	AUDIOSINK1_PHASE_CNT	–	(13:0) PHASE_CNT	0x0	Sink clock phase counter value

A.24 ASRC CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1140 0x40000500	ASRC_CFG	(7) PROC_DONE_INT_ENABLE	(7) PROC_DONE_INT_ENABLE	0x1	The ASRC processing done interrupt mask
		(6) UPDATE_ERR_INT_ENABLE	(6) UPDATE_ERR_INT_ENABLE	0x1	The ASRC state/configuration update error interrupt mask
		(5) IN_ERR_INT_ENABLE	(5) IN_ERR_INT_ENABLE	0x0	The ASRC input interface error interrupt mask
		(4) DATA_OUT_INT_ENABLE	(4) DATA_OUT_INT_ENABLE	0x0	The ASRC_OUT register interrupt status
		(3) DATA_IN_INT_ENABLE	(3) DATA_IN_INT_ENABLE	0x0	The ASRC_IN register interrupt status
		(2) WDF_TYPE	(2) WDF_TYPE	0x0	WDF Type Selection
		(1:0) MODE	(1:0) MODE	0x0	ASRC mode
P:0x1141 0x40000504	ASRC_CTRL	-	(8) ENABLE_STATUS	0x0	ASRC enable status
		(2) RESET	-	N/A	Write a 1 to reset ASRC
		(1) DISABLE	-	N/A	Disable the re-sampler block
		(0) ENABLE	-	N/A	Enable the re-sampler block
P:0x1142 0x40000508	ASRC_STATUS	-	(12) BUSY	0x0	The ASRC processing state
		-	(11) OUT_REQ	0x0	The ASRC_OUT register status
		-	(10) IN_REQ	0x0	The ASRC_IN register status
		-	(9) UPDATE_ERR	0x0	The ASRC state/configuration update error interrupt status

Address	Register Name	Register Write	Register Read	Default	Description
		–	(8) IN_ERR	0x0	The ASRC input interface error interrupt status
		(1) UPDATE_ERR_CLEAR	–	N/A	Clear the ASRC update/configuration error interrupt status
		(0) IN_ERR_CLEAR	–	N/A	Clear the ASRC input interface error interrupt
P:0x1143 0x4000050C	ASRC_CNT	(27:16) OUTPUT_CNT	(27:16) OUTPUT_CNT	0x0	ASRC output sample counter
		(11:0) INPUT_CNT	(11:0) INPUT_CNT	0x0	ASRC input sample counter
P:0x1144 0x40000510	ASRC_IN	(15:0) ASRC_IN	(15:0) ASRC_IN	0x0	Audio sample input
P:0x1145 0x40000514	ASRC_OUT	–	(15:0) ASRC_OUT	0x0	Audio sample output
P:0x1146 0x40000518	ASRC_OUT_MIRROR	(15:0) ASRC_OUT	(15:0) ASRC_OUT	0x0	Audio sample mirror output
P:0x1147 0x4000051C	ASRC_PHASE_INC	(31:0) STEP	(31:0) STEP	0x0	ASRC_PHASE_INC
P:0x1148 0x40000520	ASRC_PHASE_CNT	(31:0) PHASE_CNT	(31:0) PHASE_CNT	0x0	ASRC phase counter
P:0x1149–P:0x1166 0x40000524 – 0x40000598	ASRC_STATE_MEM_*	(31:0) STATE_MEM	(31:0) STATE_MEM	0x0	ASRC State Memory 0 to 29

A.25 CRC GENERATOR CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1180 0x40000600	CRC_CFG	(4) FINAL_CRC_XOR	(4) FINAL_CRC_XOR	0x0	Selects the final CRC XOR mode
		(3) FINAL_CRC_REVERSE	(3) FINAL_CRC_REVERSE	0x0	Selects the final CRC reversal mode
		(2) BIT_ORDER	(2) BIT_ORDER	0x0	Selects the bit order for bytes added to the CRC
		(1) CRC_TYPE	(1) CRC_TYPE	0x0	Selects the CRC type
		(0) BYTE_ORDER	(0) BYTE_ORDER	0x0	Selects the endianness for bytes added to the CRC
P:0x1181 0x40000604	CRC_VALUE	(31:0) CURRENT_CRC	(31:0) CURRENT_CRC	0xFFFF	CRC generator value: Write 0xFFFFFFFF (32) or 0xFFFF (CCITT) to initialize the CRC, read provides the current CRC value.
P:0x1182 0x40000608	CRC_ADD_1	(0) CRC_ADD_1	–	N/A	Add 1 bit to the CRC calculation
P:0x1183 0x4000060C	CRC_ADD_8	(7:0) CRC_ADD_8	–	N/A	Add 1 byte (8 bits) to the CRC calculation
P:0x1184 0x40000610	CRC_ADD_16	(15:0) CRC_ADD_16	–	N/A	Add 1 half-word (16 bits) to the CRC calculation

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1185 0x40000614	CRC_ADD_24	(23:0) CRC_ADD_24	–	N/A	Add 3 bytes (24 bits) to the CRC calculation
P:0x1186 0x40000618	CRC_ADD_32	(31:0) CRC_ADD_32	–	N/A	Add 1 word (32 bits) to the CRC calculation
P:0x1187 0x4000061C	CRC_FINAL	–	(31:0) FINAL_CRC	0x0	CRC generator final value: After XOR for CCITT or byte reversal for CRC-32

A.26 G.722 CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0x11C0 0x40000700	G722_STATUS	–	(10) DEC_CONCEAL_RDY	0x0	The G722 decoder concealment flag interrupt status
		–	(9) DEC_IN_ERR	0x0	The G722 decoder input interface error interrupt status
		–	(8) ENC_IN_ERR	0x0	The G722 encoder input interface error interrupt status
		(2) DEC_CONCEAL_RDY_CLEAR	–	N/A	The G722 decoder concealment flag interrupt clear
		(1) DEC_IN_ERR_CLEAR	–	N/A	The G722 decoder input interface error interrupt clear
		(0) ENC_IN_ERR_CLEAR	–	N/A	The G722 encoder input interface error interrupt clear
P:0x11C1 0x40000704	G722_INT_ENABLE	(2) DEC_CONCEAL_RDY	(2) DEC_CONCEAL_RDY	0x0	The G722 decoder concealment flag interrupt mask
		(1) DEC_IN_ERR	(1) DEC_IN_ERR	0x0	The G722 decoder input interface error interrupt mask
		(0) ENC_IN_ERR	(0) ENC_IN_ERR	0x0	The G722 encoder input interface error interrupt mask
P:0x11C2 0x40000708	G722_CFG	(11) DEC_OUT_FORMAT	(11) DEC_OUT_FORMAT	0x0	G722 Decoder Output Format
		(10) DEC_FB_BYPASS	(10) DEC_FB_BYPASS	0x0	G722 Decoder filterbank bypass control
		(9) DEC_CONCL_EBL	(9) DEC_CONCL_EBL	0x0	G722 Decoder concealment mode control
		(8) DEC_ENABLE	(8) DEC_ENABLE	0x0	Enable/disable the G722 Decoder
		(2) ENC_IN_FORMAT	(2) ENC_IN_FORMAT	0x0	G722 Encoder Input Data Format

Address	Register Name	Register Write	Register Read	Default	Description
		(1) ENC_FB_BYPASS	(1) ENC_FB_BYPASS	0x0	G722 Encoder filterbank bypass control
		(0) ENC_ENABLE	(0) ENC_ENABLE	0x0	Enable/disable the G722 Encoder
P:0x11C3 0x4000070C	G722_CTRL	(10) DEC_SBBND_CLR	–	N/A	G722 subband decoders section clear
		(9) DEC_FB_CLR	–	N/A	G722 decoder filterbank section clear
		(8) DEC_LSB_CNCL_CLR	–	N/A	G722 decoder lower subband concealment clear
		(1) ENC_SBBND_CLR	–	N/A	G722 subband encoders section clear
		(0) ENC_FB_CLR	–	N/A	G722 encoder filterbank section clear
P:0x11C4 0x40000710	G722_ENC_IN	(31:16) ENC_IN1	(31:16) ENC_IN1	0x0	Audio sample 1
		(15:0) ENC_IN0	(15:0) ENC_IN0	0x0	Audio sample 0
P:0x11C5 0x40000714	G722_ENC_OUT	–	(7:0) ENC_OUT	0x0	Compressed G722 encoder output
P:0x11C6 0x40000718	G722_DEC_IN	(7:0) DEC_IN	(7:0) DEC_IN	0x0	G722 decoder input to decompress
P:0x11C7 0x4000071C	G722_DEC_OUT	–	(31:16) DEC_OUT1	0x0	Audio sample 1
		–	(15:0) DEC_OUT0	0x0	Audio sample 0
P:0x11C8 0x40000720	G722_DEC_FBIN	(31:16) DEC_RH	(31:16) DEC_RH	0x0	Higher subband decoder output to the QMF
		(15:0) DEC_RL	(15:0) DEC_RL	0x0	Lower subband decoder output to the QMF

Address	Register Name	Register Write	Register Read	Default	Description
P:0x11C9 0x40000724	G722_DEC_PLT	(31) DEC_LSB_PLT2	(31) DEC_LSB_PLT2	0x0	Decoder lower subband PLT2 sign register
		(15) DEC_LSB_PLT1	(15) DEC_LSB_PLT1	0x0	Decoder lower subband PLT1 sign register
P:0x11CA 0x40000728	G722_DEC_RLT	(15:0) DEC_LSB_RLT1	(15:0) DEC_LSB_RLT1	0x0	Decoder lower subband RLT1 register
P:0x11CB 0x4000072C	G722_DEC_SX	(31:16) DEC_LSB_SZL	(31:16) DEC_LSB_SZL	0x0	Decoder lower subband SZL pipeline register
		(15:0) DEC_LSB_SL	(15:0) DEC_LSB_SL	0x0	Decoder lower subband SL pipeline register
P:0x11CC 0x40000730	G722_DEC_LSB_QUANT	(30:18) DEC_LSB_DETL	(30:18) DEC_LSB_DETL	0x8	Decoder lower subband DETL register
		(14:0) DEC_LSB_NBL	(14:0) DEC_LSB_NBL	0x0	Decoder lower subband NBL register
P:0x11CD 0x40000734	G722_DEC_HSB_QUANT	(30:18) DEC_HSB_DETH	(30:18) DEC_HSB_DETH	0x2	Decoder higher subband DETH register
		(14:0) DEC_HSB_NBH	(14:0) DEC_HSB_NBH	0x0	Decoder higher subband NBH register

A.27 REED SOLOMON CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1200 0x40000800	RS_STATUS	-	(25:24) DEC_BLK_ERR	0x0	Reed Solomon decoder error count in the last block
		-	(21:16) DEC_ACC_ERR	0x0	Reed Solomon decoder accumulated error count
		-	(12) DEC_MISS_ERR	0x0	Reed Solomon decoder missed error interrupt status
		-	(11) DEC_BLK_END	0x0	Reed Solomon decoder block end interrupt status
		-	(10) DEC_IN_ERR	0x0	Reed Solomon decoder input interface error interrupt status
		-	(9) ENC_BLK_END	0x0	Reed Solomon encoder block end interrupt status
		-	(8) ENC_IN_ERR	0x0	Reed Solomon encoder input interface error interrupt status
		(4) DEC_MISS_ERR_CLEAR	-	N/A	Reed Solomon decoder missed error interrupt clear
		(3) DEC_BLK_END_CLEAR	-	N/A	Reed Solomon decoder block end interrupt clear
		(2) DEC_IN_ERR_CLEAR	-	N/A	Reed Solomon decoder output interface error interrupt clear
		(1) ENC_BLK_END_CLEAR	-	N/A	Reed Solomon encoder block end interrupt clear
		(0) ENC_IN_ERR_CLEAR	-	N/A	Reed Solomon encoder input interface error interrupt clear
P:0x1201	RS_INT_ENABLE	(4) DEC_MISS_ERR	(4) DEC_MISS_ERR	0x0	The Reed Solomon decoder missed error interrupt mask

Address	Register Name	Register Write	Register Read	Default	Description
0x40000804					
		(3) DEC_BLK_END	(3) DEC_BLK_END	0x0	The Reed Solomon decoder block end interrupt mask
		(2) DEC_IN_ERR	(2) DEC_IN_ERR	0x0	The Reed Solomon decoder input interface error interrupt mask
		(1) ENC_BLK_END	(1) ENC_BLK_END	0x0	The Reed Solomon encoder block end interrupt mask
		(0) ENC_IN_ERR	(0) ENC_IN_ERR	0x0	The Reed Solomon Encoder input interface error interrupt mask
P:0x1202 0x40000808	RS_CFG	(13) DEC_IN_PTR_DIR	(13) DEC_IN_PTR_DIR	0x0	Reed Solomon Decoder Input Register Pointer Update Direction
		(12) DEC_OUT_PTR_DIR	(12) DEC_OUT_PTR_DIR	0x0	Reed Solomon Decoder Output Register Pointer Update Direction
		(11:10) DEC_IN_PTR_MODE	(11:10) DEC_IN_PTR_MODE	0x0	Reed Solomon Decoder Input Register Pointer Mode
		(9:8) DEC_OUT_PTR_MODE	(9:8) DEC_OUT_PTR_MODE	0x0	Reed Solomon Decoder Output Register Pointer Mode
		(5) ENC_IN_PTR_DIR	(5) ENC_IN_PTR_DIR	0x0	Reed Solomon Encoder Input Register Pointer Update Direction
		(4) ENC_OUT_PTR_DIR	(4) ENC_OUT_PTR_DIR	0x0	Reed Solomon Encoder Output Register Pointer Update Direction
		(3:2) ENC_IN_PTR_MODE	(3:2) ENC_IN_PTR_MODE	0x0	Reed Solomon Encoder Input Register Pointer Mode
		(1:0) ENC_OUT_PTR_MODE	(1:0) ENC_OUT_PTR_MODE	0x0	Reed Solomon Encoder Output Register Pointer Mode
P:0x1203 0x4000080C	RS_BLK_CNT	(15:8) DEC_BLK_CNT	(15:8) DEC_BLK_CNT	0x0	Reed Solomon Decoder block size to count down from (255 is continuous run)

Address	Register Name	Register Write	Register Read	Default	Description
		(7:0) ENC_BLK_CNT	(7:0) ENC_BLK_CNT	0x0	Reed Solomon Encoder block size to count down from (255 is continuous run)
P:0x1204 0x40000810	RS_CTRL	(1) DEC_CLR	–	N/A	Reed Solomon Decoder Clear
		(0) ENC_CLR	–	N/A	Reed Solomon Encoder Clear
P:0x1205 0x40000814	RS_ENC_IN	(31:0) ENC_IN	(31:0) ENC_IN	0x0	Reed Solomon encoder input register
P:0x1206 0x40000818	RS_ENC_OUT	(31:0) ENC_OUT	(31:0) ENC_OUT	0x0	Reed Solomon encoder output register
P:0x1207 0x4000081C	RS_DEC_IN	(31:0) DEC_IN	(31:0) DEC_IN	0x0	Reed Solomon decoder input register
P:0x1208 0x40000820	RS_DEC_OUT	(31:0) DEC_OUT	(31:0) DEC_OUT	0x0	Reed Solomon decoder output register
P:0x1209 0x40000824	RS_PTR	(13:11) DEC_IN_PTR	(13:11) DEC_IN_PTR	0x0	Reed Solomon decoder input pointer value
		(10:8) DEC_OUT_PTR	(10:8) DEC_OUT_PTR	0x0	Reed Solomon decoder output pointer value
		(5:3) ENC_IN_PTR	(5:3) ENC_IN_PTR	0x0	Reed Solomon encoder input pointer value
		(2:0) ENC_OUT_PTR	(2:0) ENC_OUT_PTR	0x0	Reed Solomon encoder output pointer value

A.28 DIO INTERFACE AND DIGITAL PAD CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1240-P:0x1263 0x40000900 - 0x4000098C	DIO_CFG_*	(13:12) DRIVE	(13:12) DRIVE	0x3	Drive strength configuration
		(11) LPF	(11) LPF	0x0	Low Pass Filter enable
		(10:8) PULL_CTRL	(10:8) PULL_CTRL	0x1	Pull selection
		(5:0) IO_MODE	(5:0) IO_MODE	0x0	IO mode selection
P:0x1264 0x40000990	DIO_INPUT_DATA_0	-	(23:0) DIO	0x0	DIO[23:0] input data
P:0x1265 0x40000994	DIO_INPUT_DATA_1	-	(11:0) DIO	0x0	DIO[35:24] input data
P:0x1266 0x40000998	DIO_GPIO_OUTPUT_DATA_0	(23:0) GPIO	(23:0) GPIO	0x0	GPIO[23:0] output data
P:0x1267 0x4000099C	DIO_GPIO_OUTPUT_DATA_1	(11:0) GPIO	(11:0) GPIO	0x0	GPIO[35:24] output data
P:0x1268 0x400009A0	DIO_GPIO_OUTPUT_DATA_0_ SET	(23:0) GPIO	-	N/A	GPIO[23:0] output data set
P:0x1269 0x400009A4	DIO_GPIO_OUTPUT_DATA_1_ SET	(11:0) GPIO	-	N/A	GPIO[35:24] output data set
P:0x126A 0x400009A8	DIO_GPIO_OUTPUT_DATA_0_ CLR	(23:0) GPIO	-	N/A	GPIO[23:0] output data clear

Address	Register Name	Register Write	Register Read	Default	Description
P:0x126B 0x400009AC	DIO_GPIO_OUTPUT_DATA_1_CLR	(11:0) GPIO	–	N/A	GPIO[35:24] output data clear
P:0x126C 0x400009B0	DIO_DIR_0	–	(23:0) DIO	0x0	Get DIO[23:0] direction
		(23:0) GPIO	–	N/A	GPIO[23:0] direction
P:0x126D 0x400009B4	DIO_DIR_1	–	(11:0) DIO	0x300	Get DIO[35:24] direction
		(11:0) GPIO	–	N/A	GPIO[35:24] direction
P:0x126E 0x400009B8	DIO_MODE_0	–	(23:0) DIO	0x0	DIO[23:0] mode
P:0x126F 0x400009BC	DIO_MODE_1	–	(11:0) DIO	0x0	DIO[35:24] mode
P:0x1270–P:0x1277 0x400009C0 – 0x400009DC	DIO_INT_CFG_*	(11) DEBOUNCE_ENABLE	(11) DEBOUNCE_ENABLE	0x0	Interrupt button debounce filter enable/disable
		(10:8) EVENT	(10:8) EVENT	0x0	Interrupt event configuration
		(5:0) SRC	(5:0) SRC	0x0	Interrupt input selection
P:0x1278 0x400009E0	DIO_INT_STATUS	–	(15) DIO_INT7_STATUS	0x0	DIO interrupt 7 status
		–	(14) DIO_INT6_STATUS	0x0	DIO interrupt 6 status
		–	(13) DIO_INT5_STATUS	0x0	DIO interrupt 5 status

Address	Register Name	Register Write	Register Read	Default	Description
		–	(12) DIO_INT4_STATUS	0x0	DIO interrupt 4 status
		–	(11) DIO_INT3_STATUS	0x0	DIO interrupt 3 status
		–	(10) DIO_INT2_STATUS	0x0	DIO interrupt 2 status
		–	(9) DIO_INT1_STATUS	0x0	DIO interrupt 1 status
		–	(8) DIO_INT0_STATUS	0x0	DIO interrupt 0 status
		(7) DIO_INT7_CLEAR	–	N/A	DIO interrupt 7 clear
		(6) DIO_INT6_CLEAR	–	N/A	DIO interrupt 6 clear
		(5) DIO_INT5_CLEAR	–	N/A	DIO interrupt 5 clear
		(4) DIO_INT4_CLEAR	–	N/A	DIO interrupt 4 clear
		(3) DIO_INT3_CLEAR	–	N/A	DIO interrupt 3 clear
		(2) DIO_INT2_CLEAR	–	N/A	DIO interrupt 2 clear
		(1) DIO_INT1_CLEAR	–	N/A	DIO interrupt 1 clear
		(0) DIO_INT0_CLEAR	–	N/A	DIO interrupt 0 clear
P:0x1279 0x400009E4	DIO_INT_DEBOUNCE	(8) DEBOUNCE_CLK	(8) DEBOUNCE_CLK	0x0	Interrupt button debounce filter clock
		(7:0) DEBOUNCE_COUNT	(7:0) DEBOUNCE_COUNT	0x0	Interrupt button debounce filter count
P:0x127A 0x400009E8	DIO_D_I2C_PAD_CFG	(8) LPF	(8) LPF	0x1	D_I2C Low-Pass-Filter enable / disable
		(3:2) DRIVE	(3:2) DRIVE	0x3	D_I2C drive strength
		(1:0) PULL	(1:0) PULL	0x3	D_I2C pull-up enable / disable

Address	Register Name	Register Write	Register Read	Default	Description
P:0x127B 0x400009EC	DIO_CM3_JTAG_PAD_CFG	(1) CM3_JTAG_DATA_EN	(1) CM3_JTAG_DATA_EN	0x1	Arm Cortex-M3 JTAG TDI on DIO32 and TDO on DIO33
		(0) CM3_JTAG_EN	(0) CM3_JTAG_EN	0x1	Arm Cortex-M3 JTAG SW-TCK and SW-TMS on DIO[31:30]
P:0x127C 0x400009F0	DIO_CLKDIV_CFG	(24) DIO_CLKDIV1_RESET	-	N/A	DIO_CLKDIV1 reset
		(21) DIO_CLKDIV1_EDGE	(21) DIO_CLKDIV1_EDGE	0x0	DIO_CLKDIV1 active input edge
		(20) DIO_CLKDIV1_ENABLE	(20) DIO_CLKDIV1_ENABLE	0x0	DIO_CLKDIV1 enable
		(19:16) DIO_CLKDIV1_PRESCALE	(19:16) DIO_CLKDIV1_PRESCALE	0x0	DIO_CLKDIV1 prescale value
		(8) DIO_CLKDIV0_RESET	-	N/A	DIO_CLKDIV0 reset
		(5) DIO_CLKDIV0_EDGE	(5) DIO_CLKDIV0_EDGE	0x0	DIO_CLKDIV0 active input edge
		(4) DIO_CLKDIV0_ENABLE	(4) DIO_CLKDIV0_ENABLE	0x0	DIO_CLKDIV0 enable
		(3:0) DIO_CLKDIV0_PRESCALE	(3:0) DIO_CLKDIV0_PRESCALE	0x0	DIO_CLKDIV0 prescale value
P:0x1280 0x40000A00	DIO_SRC_SYNC_SLAVE	(5:0) SYNC	(5:0) SYNC	0x25	SYNC_SLAVE input selection
P:0x1282-P:0x1283 0x40000A08 - 0x40000A0C	DIO_SRC_PCM_*	(21:16) SERI	(21:16) SERI	0x25	PCM_SERI input selection
		(13:8) FRAME	(13:8) FRAME	0x25	PCM_FRAME input selection

Address	Register Name	Register Write	Register Read	Default	Description
		(5:0) CLK	(5:0) CLK	0x25	PCM_CLK input selection
P:0x1284-P:0x1285 0x40000A10 - 0x40000A14	DIO_SRC_SPI_*	(13:8) CS	(13:8) CS	0x25	SPI_CS input selection
		(5:0) CLK	(5:0) CLK	0x25	SPI_CLK input selection
P:0x1286-P:0x1287 0x40000A18 - 0x40000A1C	DIO_SRC_SPI_IO_*	(29:24) IO3	(29:24) IO3	0x25	SPI_IO3 input selection
		(21:16) IO2	(21:16) IO2	0x25	SPI_IO2 input selection
		(13:8) IO1	(13:8) IO1	0x25	SPI_IO1 input selection (master SERI)
		(5:0) IO0	(5:0) IO0	0x25	SPI_IO0 input selection (slave SERI)
P:0x1288-P:0x1289 0x40000A20 - 0x40000A24	DIO_SRC_I3C_*	(13:8) SDA	(13:8) SDA	0x25	SDA input selection
		(5:0) SCL	(5:0) SCL	0x25	SCL input selection
P:0x128A-P:0x128B 0x40000A28 - 0x40000A2C	DIO_SRC_I2C_*	(13:8) SDA	(13:8) SDA	0x25	SDA input selection
		(5:0) SCL	(5:0) SCL	0x25	SCL input selection
P:0x128C 0x40000A30	DIO_SRC_CLKDIV	(13:8) CLKDIV1	(13:8) CLKDIV1	0x25	DIO Interrupt Clock Divider 1 Input Selection
		(5:0) CLKDIV0	(5:0) CLKDIV0	0x25	DIO Interrupt Clock Divider 0 Input Selection
P:0x128D	DIO_SRC_NMI	(7) NMI_EDGE_LEVEL	(7) NMI_EDGE_LEVEL	0x0	NMI edge or level

Address	Register Name	Register Write	Register Read	Default	Description
0x40000A34					
		(6) NMI_POLARITY	(6) NMI_POLARITY	0x1	NMI polarity
		(5:0) NMI	(5:0) NMI	0x24	NMI input selection
P:0x128E 0x40000A38	DIO_SRC_EMMC_0	(29:24) EMMC_DATA3	(29:24) EMMC_DATA3	0x25	eMMC DATA3 input selection
		(21:16) EMMC_DATA2	(21:16) EMMC_DATA2	0x25	eMMC DATA2 input selection
		(13:8) EMMC_DATA1	(13:8) EMMC_DATA1	0x25	eMMC DATA1 input selection
		(5:0) EMMC_DATA0	(5:0) EMMC_DATA0	0x25	eMMC DATA0 input selection
P:0x128F 0x40000A3C	DIO_SRC_EMMC_1	(29:24) EMMC_DATA7	(29:24) EMMC_DATA7	0x25	eMMC DATA7 input selection
		(21:16) EMMC_DATA6	(21:16) EMMC_DATA6	0x25	eMMC DATA6 input selection
		(13:8) EMMC_DATA5	(13:8) EMMC_DATA5	0x25	eMMC DATA5 input selection
		(5:0) EMMC_DATA4	(5:0) EMMC_DATA4	0x25	eMMC DATA4 input selection
P:0x1290 0x40000A40	DIO_SRC_EMMC_2	(5:0) EMMC_CMD	(5:0) EMMC_CMD	0x25	eMMC CMD input selection
P:0x1291 0x40000A44	DIO_SRC_DMIC_CLK	(5:0) CLK	(5:0) CLK	0x25	DMIC clock input selection
P:0x1292 0x40000A48	DIO_SRC_DMIC_DATA	(29:24) DMIC3	(29:24) DMIC3	0x25	DMIC data input selection
		(21:16) DMIC2	(21:16) DMIC2	0x25	DMIC data input selection
		(13:8) DMIC1	(13:8) DMIC1	0x25	DMIC data input selection
		(5:0) DMIC0	(5:0) DMIC0	0x25	DMIC data input selection

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1293 0x40000A4C	DIO_SRC_DSP_JTAG	(21:16) TDI	(21:16) TDI	0x25	DSP_TDI input selection
		(13:8) TMS	(13:8) TMS	0x25	DSP_TMS input selection
		(5:0) TCK	(5:0) TCK	0x25	DSP_TCK input selection
P:0x1294 0x40000A50	DIO_SRC_UART	(5:0) RX	(5:0) RX	0x25	UART_RX input selection

A.29 PCM INTERFACE CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0x12C0 0x40000B00	PCM0_CFG	(25) TX_ACK_SEL	(25) TX_ACK_SEL	0x0	Select which TX word acknowledges the TX request
		(24) RX_ACK_SEL	(24) RX_ACK_SEL	0x0	Select which RX word acknowledges the RX request
		(23) TX_DMA_ENABLE	(23) TX_DMA_ENABLE	0x0	Enable/disable the TX DMA request
		(22) RX_DMA_ENABLE	(22) RX_DMA_ENABLE	0x0	Enable/disable the RX DMA request
		(21) TX_IOC_ENABLE	(21) TX_IOC_ENABLE	0x0	Enable/disable the TX IOC request
		(20) RX_IOC_ENABLE	(20) RX_IOC_ENABLE	0x0	Enable/disable the RX IOC request
		(19) RX_TX_INT_ENABLE	(19) RX_TX_INT_ENABLE	0x0	Enable/disable the RX_TX interrupt
		(17) OVERRUN_INT_EN	(17) OVERRUN_INT_EN	0x0	Enable/disable the overrun interrupt
		(16) UNDERRUN_INT_EN	(16) UNDERRUN_INT_EN	0x0	Enable/disable the underrun interrupt
		(15) CLK_POLARITY	(15) CLK_POLARITY	0x0	Select the PCM clock polarity
		(14) TX_DATA_ALIGN	(14) TX_DATA_ALIGN	0x0	Select the TX data alignment
		(13) RX_DATA_ALIGN	(13) RX_DATA_ALIGN	0x0	Select the RX data alignment
		(12:8) WORD_SIZE	(12:8) WORD_SIZE	0x8	Select the number of bits per PCM word
		(7) BIT_ORDER	(7) BIT_ORDER	0x0	Select whether the data is transmitted starting with the MSB or LSB
		(6) FRAME_ALIGN	(6) FRAME_ALIGN	0x0	Align the PCM frame signal to the first/last bit
		(5) FRAME_WIDTH	(5) FRAME_WIDTH	0x0	Use a long/short PCM frame signal
		(4:2) FRAME_LENGTH	(4:2) FRAME_LENGTH	0x0	Select the number of words per PCM frame

Address	Register Name	Register Write	Register Read	Default	Description
		(1) SUBFRAME	(1) SUBFRAME	0x0	Enable the frame duration for each word
		(0) SLAVE	(0) SLAVE	0x1	Use the PCM interface as a master/slave
P:0x12C1 0x40000B04	PCM0_CTRL	–	(8) ENABLE_STATUS	0x0	PCM enable status
		(2) RESET	–	N/A	Reset the PCM interface
		(1) DISABLE	–	N/A	Disable the PCM interface
		(0) ENABLE	–	N/A	Enable the PCM interface
P:0x12C2 0x40000B08	PCM0_STATUS	–	(12) BUSY	0x0	Indicate that the reception or transmission of the data is ongoing
		–	(11) TX_REQ	0x1	Indicate that TX data can be written
		–	(10) RX_REQ	0x0	Indicate that RX data can be read
		–	(9) OVERRUN	0x0	Indicate that an overrun occurred when receiving data
		–	(8) UNDERRUN	0x0	Indicate that an underrun occurred when transmitting data
		(1) OVERRUN_CLEAR	–	N/A	Clear the overrun status flag
		(0) UNDERRUN_CLEAR	–	N/A	Clear the underrun status flag
P:0x12C4 0x40000B10	PCM0_TX_DATA0	(31:0) TX_DATA0	(31:0) TX_DATA0	0x0	Data to transmit on channel 0
P:0x12C5 0x40000B14	PCM0_TX_DATA1	(31:0) TX_DATA1	(31:0) TX_DATA1	0x0	Data to transmit on channel 1
P:0x12C6 0x40000B18	PCM0_RX_DATA0	–	(31:0) RX_DATA0	0x0	Data received on channel 0

Address	Register Name	Register Write	Register Read	Default	Description
P:0x12C7 0x40000B1C	PCM0_RX_DATA1	–	(31:0) RX_DATA1	0x0	Data received on channel 1
P:0x1300 0x40000C00	PCM1_CFG	(25) TX_ACK_SEL	(25) TX_ACK_SEL	0x0	Select which TX word acknowledges the TX request
		(24) RX_ACK_SEL	(24) RX_ACK_SEL	0x0	Select which RX word acknowledges the RX request
		(23) TX_DMA_ENABLE	(23) TX_DMA_ENABLE	0x0	Enable/disable the TX DMA request
		(22) RX_DMA_ENABLE	(22) RX_DMA_ENABLE	0x0	Enable/disable the RX DMA request
		(21) TX_IOC_ENABLE	(21) TX_IOC_ENABLE	0x0	Enable/disable the TX IOC request
		(20) RX_IOC_ENABLE	(20) RX_IOC_ENABLE	0x0	Enable/disable the RX IOC request
		(19) RX_TX_INT_ENABLE	(19) RX_TX_INT_ENABLE	0x0	Enable/disable the RX_TX interrupt
		(17) OVERRUN_INT_EN	(17) OVERRUN_INT_EN	0x0	Enable/disable the overrun interrupt
		(16) UNDERRUN_INT_EN	(16) UNDERRUN_INT_EN	0x0	Enable/disable the underrun interrupt
		(15) CLK_POLARITY	(15) CLK_POLARITY	0x0	Select the PCM clock polarity
		(14) TX_DATA_ALIGN	(14) TX_DATA_ALIGN	0x0	Select the TX data alignment
		(13) RX_DATA_ALIGN	(13) RX_DATA_ALIGN	0x0	Select the RX data alignment
		(12:8) WORD_SIZE	(12:8) WORD_SIZE	0x8	Select the number of bits per PCM word
		(7) BIT_ORDER	(7) BIT_ORDER	0x0	Select whether the data is transmitted starting with the MSB or LSB
		(6) FRAME_ALIGN	(6) FRAME_ALIGN	0x0	Align the PCM frame signal to the first/last bit
		(5) FRAME_WIDTH	(5) FRAME_WIDTH	0x0	Use a long/short PCM frame signal
		(4:2) FRAME_LENGTH	(4:2) FRAME_LENGTH	0x0	Select the number of words per PCM

Address	Register Name	Register Write	Register Read	Default	Description
					frame
		(1) SUBFRAME	(1) SUBFRAME	0x0	Enable the frame duration for each word
		(0) SLAVE	(0) SLAVE	0x1	Use the PCM interface as a master/slave
P:0x1301 0x40000C04	PCM1_CTRL	–	(8) ENABLE_STATUS	0x0	PCM enable status
		(2) RESET	–	N/A	Reset the PCM interface
		(1) DISABLE	–	N/A	Disable the PCM interface
		(0) ENABLE	–	N/A	Enable the PCM interface
P:0x1302 0x40000C08	PCM1_STATUS	–	(12) BUSY	0x0	Indicate that the reception or transmission of the data is ongoing
		–	(11) TX_REQ	0x1	Indicate that TX data can be written
		–	(10) RX_REQ	0x0	Indicate that RX data can be read
		–	(9) OVERRUN	0x0	Indicate that an overrun occurred when receiving data
		–	(8) UNDERRUN	0x0	Indicate that an underrun occurred when transmitting data
		(1) OVERRUN_CLEAR	–	N/A	Clear the overrun status flag
		(0) UNDERRUN_CLEAR	–	N/A	Clear the underrun status flag
P:0x1304 0x40000C10	PCM1_TX_DATA0	(31:0) TX_DATA0	(31:0) TX_DATA0	0x0	Data to transmit on channel 0

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1305 0x40000C14	PCM1_TX_DATA1	(31:0) TX_DATA1	(31:0) TX_DATA1	0x0	Data to transmit on channel 1
P:0x1306 0x40000C18	PCM1_RX_DATA0	–	(31:0) RX_DATA0	0x0	Data received on channel 0
P:0x1307 0x40000C1C	PCM1_RX_DATA1	–	(31:0) RX_DATA1	0x0	Data received on channel 1

A.30 SPI INTERFACE CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1340 0x40000D00	SPI0_CFG	(23) TX_DMA_ENABLE	(23) TX_DMA_ENABLE	0x0	Enable/disable the TX DMA request
		(22) RX_DMA_ENABLE	(22) RX_DMA_ENABLE	0x0	Enable/disable the RX DMA request
		(21) TX_END_INT_ENABLE	(21) TX_END_INT_ENABLE	0x0	Enable/disable the TX interrupt
		(20) TX_START_INT_ENABLE	(20) TX_START_INT_ENABLE	0x0	Enable/disable the TX interrupt
		(19) RX_INT_ENABLE	(19) RX_INT_ENABLE	0x0	Enable/disable the RX interrupt
		(18) CS_RISE_INT_ENABLE	(18) CS_RISE_INT_ENABLE	0x0	Enable/disable the CS rise interrupt (slave mode only)
		(17) OVERRUN_INT_ENABLE	(17) OVERRUN_INT_ENABLE	0x0	Enable/disable the overrun interrupt
		(16) UNDERRUN_INT_ENABLE	(16) UNDERRUN_INT_ENABLE	0x0	Enable/disable the underrun interrupt
		(14:13) MODE	(14:13) MODE	0x0	Select the SPI master mode (ignored in slave mode)
		(12:8) WORD_SIZE	(12:8) WORD_SIZE	0x0	Select the SPI word size (word size = SPI_WORD_SIZE + 1)
		(7:4) PRESCALE	(7:4) PRESCALE	0x0	Prescale the SPI clock for master mode
		(1) CLK_POLARITY	(1) CLK_POLARITY	0x0	Select the SPI clock polarity
		(0) SLAVE	(0) SLAVE	0x0	Use the SPI interface as master or slave
P:0x1341 0x40000D04	SPI0_CTRL	-	(19) CS_STATUS	0x1	SPI CS status

Address	Register Name	Register Write	Register Read	Default	Description
		–	(18:17) MODE_STATUS	0x0	SPI mode status
		–	(16) ENABLE_STATUS	0x0	SPI enable status
		(9) CS_0	–	N/A	Lower the SPI chip-select line (master mode)
		(8) CS_1	–	N/A	Raise the SPI chip-select line (master mode)
		(7) MODE_NOP	–	N/A	Set mode to no operation
		(6) MODE_WRITE	–	N/A	Set mode to read operation
		(5) MODE_READ	–	N/A	Set mode to write operation
		(4) MODE_READ_WRITE	–	N/A	Set mode to read and write operation
		(3) START	–	N/A	Start a data transfer in master read mode
		(2) RESET	–	N/A	Reset the SPI interface
		(1) DISABLE	–	N/A	Disable the SPI interface
		(0) ENABLE	–	N/A	Enable the SPI interface
P:0x1342 0x40000D08	SPI0_STATUS	–	(13) BUSY	0x0	Indicate that the reception or transmission of the data is ongoing
		–	(12) TX_REQ	0x1	Indicate that TX data can be written
		–	(11) RX_REQ	0x0	Indicate that RX data can be read
		–	(10) CS_RISE	0x0	Indicate that CS has risen in slave mode
		–	(9) OVERRUN	0x0	Indicate that an overrun has occurred when receiving data on the SPI interface
		–	(8) UNDERRUN	0x0	Indicate that an underrun has occurred

Address	Register Name	Register Write	Register Read	Default	Description
					when transmitting data on the SPI interface
		(4) TX_REQ_SET	–	N/A	Set TX_REQ status flag and clear internal TX buffer status
		(2) CS_RISE_CLEAR	–	N/A	Clear the CS rise status flag
		(1) OVERRUN_CLEAR	–	N/A	Clear the overrun status flag
		(0) UNDERRUN_CLEAR	–	N/A	Clear the underrun status flag
P:0x1343 0x40000D0C	SPI0_TX_DATA	(31:0) TX_DATA	(31:0) TX_DATA	0x0	Single word buffer for data to be transmitted. When in master write or read_write mode, the transaction is started automatically
P:0x1344 0x40000D10	SPI0_RX_DATA	–	(31:0) RX_DATA	0x0	Single word buffer for received data. When in master read mode, a new transaction is started automatically
P:0x1345 0x40000D14	SPI0_RX_DATA_NO_START	–	(31:0) RX_DATA	0x0	Single word buffer for received data. Does not start a new transaction in master read mode, but does clear the RX_REQ flag
P:0x1346 0x40000D18	SPI0_RX_DATA_MIRROR	–	(31:0) RX_DATA	0x0	Single word buffer for received data. Does not start a new transaction and does clear the RX_REQ flag
P:0x1380 0x40000E00	SPI1_CFG	(23) TX_DMA_ENABLE	(23) TX_DMA_ENABLE	0x0	Enable/disable the TX DMA request
		(22) RX_DMA_ENABLE	(22) RX_DMA_ENABLE	0x0	Enable/disable the RX DMA request
		(21) TX_END_INT_ENABLE	(21) TX_END_INT_ENABLE	0x0	Enable/disable the TX interrupt
		(20) TX_START_INT_ENABLE	(20) TX_START_INT_ENABLE	0x0	Enable/disable the TX interrupt

Address	Register Name	Register Write	Register Read	Default	Description
		(19) RX_INT_ENABLE	(19) RX_INT_ENABLE	0x0	Enable/disable the RX interrupt
		(18) CS_RISE_INT_ENABLE	(18) CS_RISE_INT_ENABLE	0x0	Enable/disable the CS rise interrupt (slave mode only)
		(17) OVERRUN_INT_ENABLE	(17) OVERRUN_INT_ENABLE	0x0	Enable/disable the overrun interrupt
		(16) UNDERRUN_INT_ENABLE	(16) UNDERRUN_INT_ENABLE	0x0	Enable/disable the underrun interrupt
		(14:13) MODE	(14:13) MODE	0x0	Select the SPI master mode (ignored in slave mode)
		(12:8) WORD_SIZE	(12:8) WORD_SIZE	0x0	Select the SPI word size (word size = SPI_WORD_SIZE + 1)
		(7:4) PRESCALE	(7:4) PRESCALE	0x0	Prescale the SPI clock for master mode
		(1) CLK_POLARITY	(1) CLK_POLARITY	0x0	Select the SPI clock polarity
		(0) SLAVE	(0) SLAVE	0x0	Use the SPI interface as master or slave
P:0x1381 0x40000E04	SPI1_CTRL	-	(19) CS_STATUS	0x1	SPI CS status
		-	(18:17) MODE_STATUS	0x0	SPI mode status
		-	(16) ENABLE_STATUS	0x0	SPI enable status
		(9) CS_0	-	N/A	Lower the SPI chip-select line (master mode)
		(8) CS_1	-	N/A	Raise the SPI chip-select line (master mode)
		(7) MODE_NOP	-	N/A	Set mode to no operation
		(6) MODE_WRITE	-	N/A	Set mode to read operation

Address	Register Name	Register Write	Register Read	Default	Description
		(5) MODE_READ	–	N/A	Set mode to write operation
		(4) MODE_READ_WRITE	–	N/A	Set mode to read and write operation
		(3) START	–	N/A	Start a data transfer in master read mode
		(2) RESET	–	N/A	Reset the SPI interface
		(1) DISABLE	–	N/A	Disable the SPI interface
		(0) ENABLE	–	N/A	Enable the SPI interface
P:0x1382 0x40000E08	SPI1_STATUS	–	(13) BUSY	0x0	Indicate that the reception or transmission of the data is ongoing
		–	(12) TX_REQ	0x1	Indicate that TX data can be written
		–	(11) RX_REQ	0x0	Indicate that RX data can be read
		–	(10) CS_RISE	0x0	Indicate that CS has risen in slave mode
		–	(9) OVERRUN	0x0	Indicate that an overrun has occurred when receiving data on the SPI interface
		–	(8) UNDERRUN	0x0	Indicate that an underrun has occurred when transmitting data on the SPI interface
		(4) TX_REQ_SET	–	N/A	Set TX_REQ status flag and clear internal TX buffer status
		(2) CS_RISE_CLEAR	–	N/A	Clear the CS rise status flag
		(1) OVERRUN_CLEAR	–	N/A	Clear the overrun status flag
		(0) UNDERRUN_CLEAR	–	N/A	Clear the underrun status flag
P:0x1383	SPI1_TX_DATA	(31:0) TX_DATA	(31:0) TX_DATA	0x0	Single word buffer for data to be transmitted. When in master write or

Address	Register Name	Register Write	Register Read	Default	Description
0x40000E0C					read_write mode, the transaction is started automatically
P:0x1384 0x40000E10	SPI1_RX_DATA	–	(31:0) RX_DATA	0x0	Single word buffer for received data. When in master read mode, a new transaction is started automatically
P:0x1385 0x40000E14	SPI1_RX_DATA_NO_START	–	(31:0) RX_DATA	0x0	Single word buffer for received data. Does not start a new transaction in master read mode, but does clear the RX_REQ flag
P:0x1386 0x40000E18	SPI1_RX_DATA_MIRROR	–	(31:0) RX_DATA	0x0	Single word buffer for received data. Does not start a new transaction and does clear the RX_REQ flag

A.31 UART INTERFACE CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0x13C0 0x40000F00	UART0_CFG	(21) TX_DMA_ENABLE	(21) TX_DMA_ENABLE	0x0	Enable/disable the TX DMA request
		(20) RX_DMA_ENABLE	(20) RX_DMA_ENABLE	0x0	Enable/disable the RX DMA request
		(19) TX_END_INT_ENABLE	(19) TX_END_INT_ENABLE	0x0	Enable/disable the TX end interrupt
		(18) TX_START_INT_ENABLE	(18) TX_START_INT_ENABLE	0x0	Enable/disable the TX start interrupt
		(17) RX_INT_ENABLE	(17) RX_INT_ENABLE	0x0	Enable/disable the RX interrupt
		(16) OVERRUN_INT_ENABLE	(16) OVERRUN_INT_ENABLE	0x0	Enable/disable the overrun interrupt
		(15:0) CNT_STEP	(15:0) CNT_STEP	0x0	Counter step size that configures the baud rate
P:0x13C1 0x40000F04	UART0_CTRL	–	(8) ENABLE_STATUS	0x0	UART enable status
		(2) RESET	–	N/A	Reset the UART interface
		(1) DISABLE	–	N/A	Disable the UART interface
		(0) ENABLE	–	N/A	Enable the UART interface
P:0x13C2 0x40000F08	UART0_STATUS	–	(12) TX_BUSY	0x0	Indicate that a TX transaction is ongoing
		–	(11) RX_BUSY	0x0	Indicate that a RX transaction is ongoing
		–	(10) TX_REQ	0x1	Indicate that a TX data can be written
		–	(9) RX_REQ	0x0	Indicate that a RX data can be read

Address	Register Name	Register Write	Register Read	Default	Description
		–	(8) OVERRUN	0x0	Indicate that an overrun occurred when receiving data
		(0) OVERRUN_CLEAR	–	N/A	Clear the overrun status flag
P:0x13C3 0x40000F0C	UART0_TX_DATA	(7:0) TX_DATA	(7:0) TX_DATA	0x0	Transmitted data
P:0x13C4 0x40000F10	UART0_RX_DATA	–	(7:0) RX_DATA	0x0	Received data

A.32 I2C INTERFACE CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1400 0x40001000	I2C0_CFG	(30) REPEATED_START_INT_ENABLE	(30) REPEATED_START_INT_ENABLE	0x0	Configure whether repeated start interrupts will be generated by the I2C interface for active transactions in slave mode
		(29) CONNECT_IN_STANDBY	(29) CONNECT_IN_STANDBY	0x0	Control if the I2C lines are connected when running on the standby clock
		(28) TX_DMA_ENABLE	(28) TX_DMA_ENABLE	0x0	Enable/disable the TX DMA request
		(27) RX_DMA_ENABLE	(27) RX_DMA_ENABLE	0x0	Enable/disable the RX DMA request
		(26) TX_INT_ENABLE	(26) TX_INT_ENABLE	0x0	Enable/disable the TX interrupt
		(25) RX_INT_ENABLE	(25) RX_INT_ENABLE	0x0	Enable/disable the RX interrupt
		(24) BUS_ERROR_INT_ENABLE	(24) BUS_ERROR_INT_ENABLE	0x0	Enable/disable the bus error interrupt
		(23) OVERRUN_INT_ENABLE	(23) OVERRUN_INT_ENABLE	0x0	Enable/disable the overrun interrupt
		(22) STOP_INT_ENABLE	(22) STOP_INT_ENABLE	0x0	Configure whether stop interrupts will be generated by the I2C interface
		(21) AUTO_ACK_ENABLE	(21) AUTO_ACK_ENABLE	0x0	Select whether acknowledgment is automatically generated or not
		(20:16) SLAVE_PRESCALE	(20:16) SLAVE_PRESCALE	0x0	Controls the number of SYSCLK wait cycles in case of clock stretching (in slave mode) between the moment the data is put on the SDA line and the SCL line is released.
		(15:8) MASTER_PRESCALE	(15:8) MASTER_PRESCALE	0x0	Prescaler used to divide SYSCLK to the correct SCL frequency when operating in master mode. SCL is prescaled by (PRESCALE + 1) * 3.

Address	Register Name	Register Write	Register Read	Default	Description
		(7:1) SLAVE_ADDRESS	(7:1) SLAVE_ADDRESS	0x10	Set the I2C slave address for this device
		(0) SLAVE	(0) SLAVE	0x0	Select whether the I2C interface is enabled for slave mode or not
P:0x1401 0x40001004	I2C0_CTRL	–	(9) LAST_DATA_STATUS	0x0	I2C last data status
		–	(8) ENABLE_STATUS	0x0	I2C enable status
		(6) LAST_DATA	–	N/A	Indicate that the current data is the last byte of a data transfer
		(5) STOP	–	N/A	Issue a stop condition on the I2C interface bus
		(4) NACK	–	N/A	Issue a not acknowledge on the I2C interface bus
		(3) ACK	–	N/A	Issue an acknowledge on the I2C interface bus
		(2) RESET	–	N/A	Reset the I2C interface
		(1) DISABLE	–	N/A	Disable the I2C interface
		(0) ENABLE	–	N/A	Enable the I2C interface
P:0x1402 0x40001008	I2C0_ADDR_START	(7:1) ADDRESS	(7:1) ADDRESS	0x0	I2C address to use for the transaction
		(0) READ_WRITE	(0) READ_WRITE	0x0	Select whether a read or a write transaction is started
P:0x1403 0x4000100C	I2C0_STATUS	–	(26) STOP_OR_REPEATED_START_DETECTED	0x0	Indicate if STOP_DETECTED or REPEATED_START_DETECTED bit is set
		–	(25) REPEATED_START_	0x0	Indicate if an I2C repeated start has

Address	Register Name	Register Write	Register Read	Default	Description
			DETECTED		been detected during an active transaction in slave mode
		–	(22) BUS_ERROR	0x0	Bus error status bit
		–	(21) BUSY	0x0	Indicate that the reception or transmission of the data is ongoing
		–	(20) START_PENDING	0x0	Master frame start pending status bit
		–	(19) MASTER_MODE	0x0	Master mode status bit
		–	(18) STOP_DETECTED	0x0	Indicate if an I2C stop bit has been detected
		–	(17) DATA_EVENT	0x0	Indicate that I2C interface either needs data to transmit or has received data
		–	(16) TX_REQ	0x1	Indicate that a TX data can be written
		–	(15) RX_REQ	0x0	Indicate that a RX data can be read
		–	(14) CLK_STRETCH	0x0	Clock stretching flag
		–	(13) LINE_FREE	0x1	Line free flag
		–	(12) ADDR_DATA	0x0	Address / Data byte
		–	(11) READ_WRITE	0x0	Read / Write frame
		–	(10) GEN_CALL	0x0	General call flag
		–	(9) ACK	0x0	Acknowledge status
		–	(8) OVERRUN	0x0	Indicate that an overrun has occurred when receiving data
		(4) TX_REQ_SET	–	N/A	Set TX_REQ status flag
		(3) REPEATED_START_DETECTED_CLEAR	–	N/A	Clear REPEATED_START_DETECTED status flag
		(2) STOP_DETECTED_CLEAR	–	N/A	Clear STOP_DETECTED status flag

Address	Register Name	Register Write	Register Read	Default	Description
		(1) BUS_ERROR_CLEAR	–	N/A	Clear BUS_ERROR status flag
		(0) OVERRUN_CLEAR	–	N/A	Clear OVERRUN status flag
P:0x1404 0x40001010	I2C0_TX_DATA	(7:0) TX_DATA	(7:0) TX_DATA	0x0	Single byte buffer for data transmitted over the I2C interface
P:0x1405 0x40001014	I2C0_RX_DATA	–	(7:0) RX_DATA	0x0	Single byte buffer for data received over the I2C interface
P:0x1406 0x40001018	I2C0_RX_DATA_MIRROR	–	(7:0) RX_DATA	0x0	Mirror of the single byte buffer for data received over the I2C interface. Does not clear RX_REQ in I2C_STATUS
P:0x1440 0x40001100	I2C1_CFG	(30) REPEATED_START_INT_ENABLE	(30) REPEATED_START_INT_ENABLE	0x0	Configure whether repeated start interrupts will be generated by the I2C interface for active transactions in slave mode
		(29) CONNECT_IN_STANDBY	(29) CONNECT_IN_STANDBY	0x0	Control if the I2C lines are connected when running on the standby clock
		(28) TX_DMA_ENABLE	(28) TX_DMA_ENABLE	0x0	Enable/disable the TX DMA request
		(27) RX_DMA_ENABLE	(27) RX_DMA_ENABLE	0x0	Enable/disable the RX DMA request
		(26) TX_INT_ENABLE	(26) TX_INT_ENABLE	0x0	Enable/disable the TX interrupt
		(25) RX_INT_ENABLE	(25) RX_INT_ENABLE	0x0	Enable/disable the RX interrupt
		(24) BUS_ERROR_INT_ENABLE	(24) BUS_ERROR_INT_ENABLE	0x0	Enable/disable the bus error interrupt
		(23) OVERRUN_INT_ENABLE	(23) OVERRUN_INT_ENABLE	0x0	Enable/disable the overrun interrupt
		(22) STOP_INT_ENABLE	(22) STOP_INT_ENABLE	0x0	Configure whether stop interrupts will be generated by the I2C interface
		(21) AUTO_ACK_ENABLE	(21) AUTO_ACK_ENABLE	0x0	Select whether acknowledgment is

Address	Register Name	Register Write	Register Read	Default	Description
					automatically generated or not
		(20:16) SLAVE_PRESCALE	(20:16) SLAVE_PRESCALE	0x0	Controls the number of SYSCLK wait cycles in case of clock stretching (in slave mode) between the moment the data is put on the SDA line and the SCL line is released.
		(15:8) MASTER_PRESCALE	(15:8) MASTER_PRESCALE	0x0	Prescaler used to divide SYSCLK to the correct SCL frequency when operating in master mode. SCL is prescaled by $(PRESCALE + 1) * 3$.
		(7:1) SLAVE_ADDRESS	(7:1) SLAVE_ADDRESS	0x10	Set the I2C slave address for this device
		(0) SLAVE	(0) SLAVE	0x0	Select whether the I2C interface is enabled for slave mode or not
P:0x1441 0x40001104	I2C1_CTRL	–	(9) LAST_DATA_STATUS	0x0	I2C last data status
		–	(8) ENABLE_STATUS	0x0	I2C enable status
		(6) LAST_DATA	–	N/A	Indicate that the current data is the last byte of a data transfer
		(5) STOP	–	N/A	Issue a stop condition on the I2C interface bus
		(4) NACK	–	N/A	Issue a not acknowledge on the I2C interface bus
		(3) ACK	–	N/A	Issue an acknowledge on the I2C interface bus
		(2) RESET	–	N/A	Reset the I2C interface
		(1) DISABLE	–	N/A	Disable the I2C interface

Address	Register Name	Register Write	Register Read	Default	Description
		(0) ENABLE	–	N/A	Enable the I2C interface
P:0x1442 0x40001108	I2C1_ADDR_START	(7:1) ADDRESS	(7:1) ADDRESS	0x0	I2C address to use for the transaction
		(0) READ_WRITE	(0) READ_WRITE	0x0	Select whether a read or a write transaction is started
P:0x1443 0x4000110C	I2C1_STATUS	–	(26) STOP_OR_REPEATED_START_DETECTED	0x0	Indicate if STOP_DETECTED or REPEATED_START_DETECTED bit is set
		–	(25) REPEATED_START_DETECTED	0x0	Indicate if an I2C repeated start has been detected during an active transaction in slave mode
		–	(22) BUS_ERROR	0x0	Bus error status bit
		–	(21) BUSY	0x0	Indicate that the reception or transmission of the data is ongoing
		–	(20) START_PENDING	0x0	Master frame start pending status bit
		–	(19) MASTER_MODE	0x0	Master mode status bit
		–	(18) STOP_DETECTED	0x0	Indicate if an I2C stop bit has been detected
		–	(17) DATA_EVENT	0x0	Indicate that I2C interface either needs data to transmit or has received data
		–	(16) TX_REQ	0x1	Indicate that a TX data can be written
		–	(15) RX_REQ	0x0	Indicate that a RX data can be read
		–	(14) CLK_STRETCH	0x0	Clock stretching flag
		–	(13) LINE_FREE	0x1	Line free flag
		–	(12) ADDR_DATA	0x0	Address / Data byte

Address	Register Name	Register Write	Register Read	Default	Description
		–	(11) READ_WRITE	0x0	Read / Write frame
		–	(10) GEN_CALL	0x0	General call flag
		–	(9) ACK	0x0	Acknowledge status
		–	(8) OVERRUN	0x0	Indicate that an overrun has occurred when receiving data
		(4) TX_REQ_SET	–	N/A	Set TX_REQ status flag
		(3) REPEATED_START_DETECTED_CLEAR	–	N/A	Clear REPEATED_START_DETECTED status flag
		(2) STOP_DETECTED_CLEAR	–	N/A	Clear STOP_DETECTED status flag
		(1) BUS_ERROR_CLEAR	–	N/A	Clear BUS_ERROR status flag
		(0) OVERRUN_CLEAR	–	N/A	Clear OVERRUN status flag
P:0x1444 0x40001110	I2C1_TX_DATA	(7:0) TX_DATA	(7:0) TX_DATA	0x0	Single byte buffer for data transmitted over the I2C interface
P:0x1445 0x40001114	I2C1_RX_DATA	–	(7:0) RX_DATA	0x0	Single byte buffer for data received over the I2C interface
P:0x1446 0x40001118	I2C1_RX_DATA_MIRROR	–	(7:0) RX_DATA	0x0	Mirror of the single byte buffer for data received over the I2C interface. Does not clear RX_REQ in I2C_STATUS

A.33 ANALOG-TO-DIGITAL CONVERTER AND BATTERY MONITORING

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1480-P:0x1483 0x40001200 - 0x4000120C	LSAD_DATA_AUDIO_CH_*	–	(31:0) DATA	0x-2000	15-bit signed LSAD conversion result (sign extended to 32 bits)
P:0x1484-P:0x1487 0x40001210 - 0x4000121C	LSAD_DATA_AUDIO_SAT_CH_*	–	(31:0) DATA	0x-2000	14-bit signed LSAD conversion result (sign extended to 32 bits)
P:0x1488-P:0x148B 0x40001220 - 0x4000122C	LSAD_DATA_TRIM_CH_*	–	(31:0) DATA	0x0	16-bit signed LSAD conversion result (sign extended to 32 bits)
P:0x148C-P:0x148F 0x40001230 - 0x4000123C	LSAD_DATA_TRIM_SAT_CH_*	–	(13:0) DATA	0x0	14-bit unsigned LSAD conversion result
P:0x1490-P:0x1493 0x40001240 - 0x4000124C	LSAD_INPUT_SEL_CH_*	(3:0) INPUT_SEL	(3:0) INPUT_SEL	0x0	Select channel source
P:0x1494 0x40001250	LSAD_DATA_VBAT	–	(14:0) DATA	0x0	15-bit unsigned LSAD conversion result
P:0x1495 0x40001254	LSAD_DATA_VDDC	–	(12:0) DATA	0x0	13-bit unsigned LSAD conversion result
P:0x1496 0x40001258	LSAD_DATA_VDDM	–	(12:0) DATA	0x0	13-bit unsigned LSAD conversion result
P:0x1498	LSAD_CFG	(7:5) INT_SEL	(7:5) INT_SEL	0x0	Select which conversion is triggering the

Address	Register Name	Register Write	Register Read	Default	Description
0x40001260					LSAD interrupt
		(4) INT_ENABLE	(4) INT_ENABLE	0x0	LSAD new sample ready interrupt mask
		(2:0) FREQ	(2:0) FREQ	0x0	Defines the sampling frequency of the LSAD channels
P:0x1499 0x40001264	LSAD_OFFSET_COMP	–	(31:0) OFFSET_COMP_RD	0x0	10-bit LSAD sign extended offset compensation (from -56 to +56 mV in steps of 0.11 mV, signed extended to 32 bits)
		(9:0) OFFSET_COMP	(9:0) OFFSET_COMP	0x0	10-bit LSAD signed offset compensation (from -56 to +56 mV in steps of 0.11 mV)
P:0x149A 0x40001268	LSAD_GAIN_COMP	–	(31:0) GAIN_COMP_RD	0x0	6-bit LSAD signed gain compensation (from -256/8192 to +248/8192 in steps of 8/8192, signed extended to 32 bits)
		(5:0) GAIN_COMP	(5:0) GAIN_COMP	0x0	6-bit LSAD signed gain compensation (from -256/8192 to +248/8192 in steps of 8/8192)
P:0x14A0 0x40001280	LSAD_BATMON_CFG	(15:8) ALARM_COUNT_VALUE	(15:8) ALARM_COUNT_VALUE	0x0	An Alarm Status bit is set when SUPPLY_COUNT_VALUE=ALARM_COUNT_VALUE
		(7:0) SUPPLY_THRESHOLD	(7:0) SUPPLY_THRESHOLD	0x95	Low voltage detection threshold (7 mV steps)
P:0x14A1 0x40001284	LSAD_BATMON_COUNT_VAL	–	(7:0) SUPPLY_COUNT_VALUE	0x0	Number of times the battery voltage has fallen below the battery monitor voltage threshold. The counter is reset when read
P:0x14A2 0x40001288	LSAD_BATMON_STATUS	–	(9) BATMON_ALARM_STAT	0x0	Battery monitoring alarm status bit

Address	Register Name	Register Write	Register Read	Default	Description
		–	(8) LSAD_READY_STAT	0x0	LSAD new sample ready status bit
		(1) BATMON_ALARM_CLEAR	–	N/A	Battery monitoring alarm status bit
		(0) LSAD_READY_CLEAR	–	N/A	LSAD new sample ready status bit

A.34 EMMC AND I3C INTERFACE CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0x14C0 0x40001300	INTERFACE_EMMC_CTRL	-	(8) CLK_ENABLE_STATUS	0x0	EMMC interface clock enable status
		(2) RESET	-	N/A	Reset the EMMC interface
		(1) CLK_DISABLE	-	N/A	Disable the EMMC interface clock
		(0) CLK_ENABLE	-	N/A	Enable the EMMC interface clock
P:0x14C2-P:0x14C3 0x40001308 - 0x4000130C	INTERFACE_I3C_CTRL_*	-	(11) IDLE	0x1	I3C interface idle status
		-	(10) CLK_DIO_WAKEUP_STATUS	0x0	I3C interface clock DIO wakeup status
		-	(9) CLK_STOP_REQ_STATUS	0x0	I3C interface clock stop request status
		-	(8) CLK_ENABLE_STATUS	0x0	I3C interface clock enable status
		(6) CLK_DIO_WAKEUP_DISABLE	-	N/A	Disable DIO wakeup of the I3C interface clock
		(5) CLK_DIO_WAKEUP_ENABLE	-	N/A	Enable DIO wakeup of the I3C interface clock
		(4) CLK_STOP_REQ_DISABLE	-	N/A	Disable request to stop the I3C interface clock
		(3) CLK_STOP_REQ_ENABLE	-	N/A	Enable request to stop the I3C interface clock
		(2) RESET	-	N/A	Reset the I3C interface clock
		(1) CLK_DISABLE	-	N/A	Disable the I3C interface clock

Address	Register Name	Register Write	Register Read	Default	Description
		(0) CLK_ENABLE	–	N/A	Enable the I3C interface clock
P:0x14C4–P:0x14C5 0x40001310 – 0x40001314	INTERFACE_I3C_CFG_*	(14:12) DIO_INT_ WAKEUP_SEL	(14:12) DIO_INT_ WAKEUP_SEL	0x0	DIO interrupt selection for automatic wakeup (clock enable)
		(11) DEV_ROLE	(11) DEV_ROLE	0x0	Selects between Main and Secondary Master roles
		(10:4) STAT_ADDR	(10:4) STAT_ADDR	0x20	I3C-Slave static address (Used as device address until Dynamic Address is assigned)
		(3:0) PID_INSTANCE_ID	(3:0) PID_INSTANCE_ ID	0xC	Provisional Instance ID identifying individual I3C device (differentiates several I3C-Slave devices)
P:0x14C7 0x4000131C	INTERFACE_I3C_TIMERS	(25:8) BUS_IDLE_TIMER	(25:8) BUS_IDLE_ TIMER	0x1E07	Correlates the SYSCLK frequency with the timing requirement of Bus Idle Condition (SYSCLK freq * 1.0 us)
		(7:0) BUS_AVAIL_TIMER	(7:0) BUS_AVAIL_ TIMER	0x7	Correlates the SYSCLK frequency with the timing requirement of Bus Available Condition (SYSCLK freq * 1.0 us)
P:0x14C8–P:0x14C9 0x40001320 – 0x40001324	INTERFACE_I3C_DMA_CFG_*	(3) TX_DMA_SRC_SEL	(3) TX_DMA_SRC_SEL	0x0	Select DMA TX request source
		(2) RX_DMA_SRC_SEL	(2) RX_DMA_SRC_SEL	0x0	Select DMA RX request source
		(1) TX_DMA_ENABLE	(1) TX_DMA_ENABLE	0x0	Enable/disable the TX DMA request
		(0) RX_DMA_ENABLE	(0) RX_DMA_ENABLE	0x0	Enable/disable the RX DMA request

A.35 I3C INTERFACE CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1500 0x40001400	I3C0_DEV_ID	–	(15:0) DEV_ID	0x5034	IP identifier
P:0x1501 0x40001404	I3C0_CONF_STATUS0	–	(31:29) CMDR_MEM_DEPTH	0x0	Command response queue depth
		–	(28:24) ASF	0x0	ASF checks available
		–	(23:16) GPO_NUM	0x1	GPO available
		–	(15:8) GPI_NUM	0x1	GPI available
		–	(7:6) IBIR_MEM_DEPTH	0x1	IBI response queue depth
		–	(5) DDR	0x1	DDR support
		–	(4) DEV_ROLE	0x0	Device role
		–	(3:0) DEVS_NUM	0x3	Slave retaining registers
P:0x1502 0x40001408	I3C0_CONF_STATUS1	–	(31:28) IBI_HW_RES	0x2	IBI resources
		–	(27:26) CMD_MEM_DEPTH	0x0	Command memory depth
		–	(25:21) SLV_DDR_RX_MEM_DEPTH	0x1F	Slave DDR Rx memory depth
		–	(20:16) SLV_DDR_TX_MEM_DEPTH	0x1F	Slave DDR Tx memory depth
		–	(12:10) IBI_MEM_DEPTH	0x1	IBI memory depth

Address	Register Name	Register Write	Register Read	Default	Description
		–	(9:5) RX_MEM_DEPTH	0x0	Rx memory depth
		–	(4:0) TX_MEM_DEPTH	0x0	Tx memory depth
P:0x1504 0x40001410	I3C0_CTRL	(31) DEV_EN	(31) DEV_EN	0x0	Enable/Disable I3C master
		(30) HALT_EN	(30) HALT_EN	0x0	Enable/Disable halt on abort behavior
		(29) MCS	–	N/A	Manual command start (MCS_EN=1)
		(28) MCS_EN	(28) MCS_EN	0x0	Enable/Disable manual command send
		(26) I3C_11_SUPP	(26) I3C_11_SUPP	0x0	Timing parameter tCASr_min
		(25:24) THD_DEL	(25:24) THD_DEL	0x0	Data hold tHD_DAT delay
		(8) HJ_DISEC	(8) HJ_DISEC	0x0	Hot-Join ACK'ed response(HJ_ACK=1)
		(7) MST_ACK	(7) MST_ACK	0x1	Mastership Accept (GETACCMST) response (slave mode = 1)
		(6) HJ_ACK	(6) HJ_ACK	0x1	Hot-Join acknowledgment
		(5) HJ_INIT	–	N/A	Initiate Hot-Join
		(4) MST_INIT	–	N/A	Initiate Mastership request
		(3) AHDR_OPT	(3) AHDR_OPT	0x0	Enable/Disable address header optimization
		(1:0) BUS_MODE	(1:0) BUS_MODE	0x0	Bus Mode
P:0x1505 0x40001414	I3C0_PRESC1_CTRL0	(31:16) I2C	(31:16) I2C	0x7C	Prescaler value for I2C SCL clock generation
		(9:0) I3C	(9:0) I3C	0x4	Prescaler value for I3C Push-Pull SDR Mode SCL clock generation
P:0x1506 0x40001418	I3C0_PRESC1_CTRL1	(15:8) PP_LOW	(15:8) PP_LOW	0x0	Counter for low period of SCL clock for Push-Pull in I3C

Address	Register Name	Register Write	Register Read	Default	Description
		(7:0) OD_LOW	(7:0) OD_LOW	0x9	Counter for low period of SCL clock for Open Drain in I3C
P:0x1508 0x40001420	I3C0_MST_IER	(18) HALTED	–	N/A	HALTED interrupt enable
		(17) MR_DONE	–	N/A	MR_DONE interrupt enable
		(16) IMM_COMP	–	N/A	IMM_COMP interrupt enable
		(15) TX_THR	–	N/A	TX_THR interrupt enable
		(14) TX_OVF	–	N/A	TX_OVF interrupt enable
		(12) IBID_THR	–	N/A	IBID_THR interrupt enable
		(11) IBID_UNF	–	N/A	IBID_UNF interrupt enable
		(10) IBIR_THR	–	N/A	IBIR_THR interrupt enable
		(9) IBIR_UNF	–	N/A	IBIR_UNF interrupt enable
		(8) IBIR_OVF	–	N/A	IBIR_OVF interrupt enable
		(7) RX_THR	–	N/A	RX_THR interrupt enable
		(6) RX_UNF	–	N/A	RX_UNF interrupt enable
		(5) CMDD_EMP	–	N/A	CMDD_EMP interrupt enable
		(4) CMDD_THR	–	N/A	CMDD_THR interrupt enable
		(3) CMDD_OVF	–	N/A	CMDD_OVF interrupt enable
		(2) CMDR_THR	–	N/A	CMDR_THR interrupt enable
		(1) CMDR_UNF	–	N/A	CMDR_UNF interrupt enable
		(0) CMDR_OVF	–	N/A	CMDR_OVF interrupt enable
P:0x1509 0x40001424	I3C0_MST_IDR	(18) HALTED	–	N/A	HALTED interrupt disable

Address	Register Name	Register Write	Register Read	Default	Description
		(17) MR_DONE	–	N/A	MR_DONE interrupt disable
		(16) IMM_COMP	–	N/A	IMM_COMP interrupt disable
		(15) TX_THR	–	N/A	TX_THR interrupt disable
		(14) TX_OVF	–	N/A	TX_OVF interrupt disable
		(12) IBID_THR	–	N/A	IBID_THR interrupt disable
		(11) IBID_UNF	–	N/A	IBID_UNF interrupt disable
		(10) IBIR_THR	–	N/A	IBIR_THR interrupt disable
		(9) IBIR_UNF	–	N/A	IBIR_UNF interrupt disable
		(8) IBIR_OVF	–	N/A	IBIR_OVF interrupt disable
		(7) RX_THR	–	N/A	RX_THR interrupt disable
		(6) RX_UNF	–	N/A	RX_UNF interrupt disable
		(5) CMDD_EMP	–	N/A	CMDD_EMP interrupt disable
		(4) CMDD_THR	–	N/A	CMDD_THR interrupt disable
		(3) CMDD_OVF	–	N/A	CMDD_OVF interrupt disable
		(2) CMDR_THR	–	N/A	CMDR_THR interrupt disable
		(1) CMDR_UNF	–	N/A	CMDR_UNF interrupt disable
		(0) CMDR_OVF	–	N/A	CMDR_OVF interrupt disable
P:0x150A 0x40001428	I3C0_MST_IMR	–	(18) HALTED	0x0	HALTED interrupt mask
		–	(17) MS_DONE	0x0	MR_DONE interrupt mask
		–	(16) IMM_COMP	0x0	IMM_COMP interrupt mask
		–	(15) TX_THR	0x0	TX_THR interrupt mask
		–	(14) TX_OVF	0x0	TX_OVF interrupt mask

Address	Register Name	Register Write	Register Read	Default	Description
		–	(12) IBID_THR	0x0	IBID_THR interrupt mask
		–	(11) IBID_UNF	0x0	IBIR_UNF interrupt mask
		–	(10) IBIR_THR	0x0	IBIR_OVF interrupt mask
		–	(9) IBIR_UNF	0x0	IBIR_UNF interrupt mask
		–	(8) IBIR_OVF	0x0	IBIR_OVF interrupt mask
		–	(7) RX_THR	0x0	RX_THR interrupt mask
		–	(6) RX_UNF	0x0	RX_UNF interrupt mask
		–	(5) CMDD_EMP	0x0	CMDD_EMP interrupt mask
		–	(4) CMDD_THR	0x0	CMDD_THR interrupt mask
		–	(3) CMDD_OVF	0x0	CMDD_OVF interrupt mask
		–	(2) CMDR_THR	0x0	CMDR_THR interrupt mask
		–	(1) CMDR_UNF	0x0	CMDR_UNF interrupt mask
		–	(0) CMDR_OVF	0x0	CMDR_OVF interrupt mask
P:0x150B 0x4000142C	I3C0_MST_ICR	(18) HALTED	–	N/A	HALTED interrupt clear
		(17) MR_DONE	–	N/A	MR_DONE interrupt clear
		(16) IMM_COMP	–	N/A	IMM_COMP interrupt clear
		(15) TX_THR	–	N/A	TX_THR interrupt clear
		(14) TX_OVF	–	N/A	TX_OVF interrupt clear
		(12) IBID_THR	–	N/A	IBID_THR interrupt clear
		(11) IBID_UNF	–	N/A	IBID_UNF interrupt clear
		(10) IBIR_THR	–	N/A	IBIR_THR interrupt clear
		(9) IBIR_UNF	–	N/A	IBIR_UNF interrupt clear

Address	Register Name	Register Write	Register Read	Default	Description
		(8) IBIR_OVF	–	N/A	IBIR_OVF interrupt clear
		(7) RX_THR	–	N/A	RX_THR interrupt clear
		(6) RX_UNF	–	N/A	RX_UNF interrupt clear
		(5) CMDD_EMP	–	N/A	CMDD_EMP interrupt clear
		(4) CMDD_THR	–	N/A	CMDD_THR interrupt clear
		(3) CMDD_OVF	–	N/A	CMDD_OVF interrupt clear
		(2) CMDR_THR	–	N/A	CMDR_THR interrupt clear
		(1) CMDR_UNF	–	N/A	CMDR_UNF interrupt clear
		(0) CMDR_OVF	–	N/A	CMDR_OVF interrupt clear
P:0x150C 0x40001430	I3C0_MST_ISR	–	(18) HALTED	0x0	HALTED interrupt status
		–	(17) MR_DONE	0x0	MR_DONE interrupt status
		–	(16) IMM_COMP	0x0	IMM_COMP interrupt status
		–	(15) TX_THR	0x0	TX_THR interrupt status
		–	(14) TX_OVF	0x0	TX_OVF interrupt status
		–	(12) IBID_THR	0x0	IBID_THR interrupt status
		–	(11) IBID_UNF	0x0	IBIR_UNF interrupt status
		–	(10) IBIR_THR	0x0	IBIR_OVF interrupt status
		–	(9) IBIR_UNF	0x0	IBIR_UNF interrupt status
		–	(8) IBIR_OVF	0x0	IBIR_OVF interrupt status
		–	(7) RX_THR	0x0	RX_THR interrupt status
		–	(6) RX_UNF	0x0	RX_UNF interrupt status
		–	(5) CMDD_EMP	0x0	CMDD_EMP interrupt status

Address	Register Name	Register Write	Register Read	Default	Description
		–	(4) CMDD_THR	0x0	CMDD_THR interrupt status
		–	(3) CMDD_OVF	0x0	CMDD_OVF interrupt status
		–	(2) CMDR_THR	0x0	CMDR_THR interrupt status
		–	(1) CMDR_UNF	0x0	CMDR_UNF interrupt status
		–	(0) CMDR_OVF	0x0	CMDR_OVF interrupt status
P:0x150D 0x40001434	I3C0_MST_STATUS0	–	(18) IDLE	0x1	Core is idle
		(17) HALTED	(17) HALTED	0x0	Core halted (W1C to un halt core)
		–	(16) OP_MODE	0x1	Indicates current mode of the controller
		–	(13) TX_FULL	0x0	TX Full
		–	(12) IBID_FULL	0x0	IBID Full
		–	(11) IBIR_FULL	0x0	IBIR Full
		–	(10) RX_FULL	0x0	RX Full
		–	(9) CMDD_FULL	0x0	CMDD Full
		–	(8) CMDR_FULL	0x0	CMDR Full
		–	(5) TX_EMP	0x1	TX Empty
		–	(4) IBID_EMP	0x1	IBID Empty
		–	(3) IBIR_EMP	0x1	IBIR Empty
		–	(2) RX_EMP	0x1	RX Empty
		–	(1) CMDD_EMP	0x1	CMDD Empty
		–	(0) CMDR_EMP	0x1	CMDR Empty
P:0x150E 0x40001438	I3C0_CMDR	–	(27:24) ERROR	0x0	Error code during command execution

Address	Register Name	Register Write	Register Read	Default	Description
		–	(19:8) XFER_BYTES	0x0	Number of bytes transferred
		–	(7:0) CMD_ID	0x0	Command Identifier field
P:0x150F 0x4000143C	I3C0_IBIR	–	(12) RESP	0x0	Master response
		–	(11:8) SLV_ID	0x0	Slave ID
		–	(7) ERROR	0x0	Overflow Error
		–	(6:2) XFER_BYTES	0x0	Number of received data bytes
		–	(1:0) IBI_TYPE	0x0	IBI type
P:0x1510 0x40001440	I3C0_SLV_IER	(21) DEFSLVS	–	N/A	DEFSLVS interrupt enable
		(20) TM	–	N/A	TM interrupt enable
		(19) ERROR	–	N/A	ERROR interrupt enable
		(18) EVENT_UP	–	N/A	EVENT_UP interrupt enable
		(17) HJ_DONE	–	N/A	HJ_DONE interrupt enable
		(16) MH_DONE	–	N/A	MH_DONE interrupt enable
		(15) DA_UPDATE	–	N/A	DA_UPDATE interrupt enable
		(14) SDR_FAIL	–	N/A	SDR_FAIL interrupt enable
		(13) DDR_FAIL	–	N/A	DDR_FAIL interrupt enable
		(12) M_RD_ABORT	–	N/A	M_RD_ABORT interrupt enable
		(11) DDR_RX_THR	–	N/A	DDR_RX_THR interrupt enable
		(10) DDR_TX_THR	–	N/A	DDR_TX_THR interrupt enable
		(9) SDR_RX_THR	–	N/A	SDR_RX_THR interrupt enable

Address	Register Name	Register Write	Register Read	Default	Description
		(8) SDR_TX_THR	–	N/A	SDR_TX_THR interrupt enable
		(7) DDR_RX_UNF	–	N/A	DDR_RX_UNF interrupt enable
		(6) DDR_TX_OVF	–	N/A	DDR_TX_OVF interrupt enable
		(5) SDR_RX_UNF	–	N/A	SDR_RX_UNF interrupt enable
		(4) SDR_TX_OVF	–	N/A	SDR_TX_OVF interrupt enable
		(3) DDR_RD_COMP	–	N/A	DDR_RD_COMP interrupt enable
		(2) DDR_WR_COMP	–	N/A	DDR_WR_COMP interrupt enable
		(1) SDR_RD_COMP	–	N/A	SDR_RD_COMP interrupt enable
		(0) SDR_WR_COMP	–	N/A	SDR_WR_COMP interrupt enable
P:0x1511 0x40001444	I3C0_SLV_IDR	(21) DEFSLVS	–	N/A	DEFSLVS interrupt disable
		(20) TM	–	N/A	TM interrupt disable
		(19) ERROR	–	N/A	ERROR interrupt disable
		(18) EVENT_UP	–	N/A	EVENT_UP interrupt disable
		(17) HJ_DONE	–	N/A	HJ_DONE interrupt disable
		(16) MH_DONE	–	N/A	MH_DONE interrupt disable
		(15) DA_UPDATE	–	N/A	DA_UPDATE interrupt disable
		(14) SDR_FAIL	–	N/A	SDR_FAIL interrupt disable
		(13) DDR_FAIL	–	N/A	DDR_FAIL interrupt disable
		(12) M_RD_ABORT	–	N/A	M_RD_ABORT interrupt disable
		(11) DDR_RX_THR	–	N/A	DDR_RX_THR interrupt disable
		(10) DDR_TX_THR	–	N/A	DDR_TX_THR interrupt disable
		(9) SDR_RX_THR	–	N/A	SDR_RX_THR interrupt disable

Address	Register Name	Register Write	Register Read	Default	Description
		(8) SDR_TX_THR	–	N/A	SDR_TX_THR interrupt disable
		(7) DDR_RX_UNF	–	N/A	DDR_RX_UNF interrupt disable
		(6) DDR_TX_OVF	–	N/A	DDR_TX_OVF interrupt disable
		(5) SDR_RX_UNF	–	N/A	SDR_RX_UNF interrupt disable
		(4) SDR_TX_OVF	–	N/A	SDR_TX_OVF interrupt disable
		(3) DDR_RD_COMP	–	N/A	DDR_RD_COMP interrupt disable
		(2) DDR_WR_COMP	–	N/A	DDR_WR_COMP interrupt disable
		(1) SDR_RD_COMP	–	N/A	SDR_RD_COMP interrupt disable
		(0) SDR_WR_COMP	–	N/A	SDR_WR_COMP interrupt disable
P:0x1512 0x40001448	I3C0_SLV_IMR	–	(21) DEFSLVS	0x0	DEFSLVS Interrupt mask
		–	(20) TM	0x0	TM Interrupt mask
		–	(19) ERROR	0x0	ERROR Interrupt mask
		–	(18) EVENT_UP	0x0	EVENT_UP Interrupt mask
		–	(17) HJ_DONE	0x0	HJ_DONE Interrupt mask
		–	(16) MH_DONE	0x0	MH_DONE Interrupt mask
		–	(15) DA_UPDATE	0x0	DA_UPDATE Interrupt mask
		–	(14) SDR_FAIL	0x0	SDR_FAIL Interrupt mask
		–	(13) DDR_FAIL	0x0	DDR_FAIL Interrupt mask
		–	(12) M_RD_ABORT	0x0	M_RD_ABORT Interrupt mask
		–	(11) DDR_RX_THR	0x0	DDR_RX_THR Interrupt mask
		–	(10) DDR_TX_THR	0x0	DDR_TX_THR Interrupt mask
		–	(9) SDR_RX_THR	0x0	SDR_RX_THR Interrupt mask

Address	Register Name	Register Write	Register Read	Default	Description
		–	(8) SDR_TX_THR	0x0	SDR_TX_THR Interrupt mask
		–	(7) DDR_RX_UNF	0x0	DDR_RX_UNF Interrupt mask
		–	(6) DDR_TX_OVF	0x0	DDR_TX_OVF Interrupt mask
		–	(5) SDR_RX_UNF	0x0	SDR_RX_UNF Interrupt mask
		–	(4) SDR_TX_OVF	0x0	SDR_TX_OVF Interrupt mask
		–	(3) DDR_RD_COMP	0x0	DDR_RD_COMP Interrupt mask
		–	(2) DDR_WR_COMP	0x0	DDR_WR_COMP Interrupt mask
		–	(1) SDR_RD_COMP	0x0	SDR_RD_COMP Interrupt mask
		–	(0) SDR_WR_COMP	0x0	SDR_WR_COMP Interrupt mask
P:0x1513 0x4000144C	I3C0_SLV_ICR	(21) DEFSLVS	–	N/A	DEFSLVS interrupt clear
		(20) TM	–	N/A	TM interrupt clear
		(19) ERROR	–	N/A	ERROR interrupt clear
		(18) EVENT_UP	–	N/A	EVENT_UP interrupt clear
		(17) HJ_DONE	–	N/A	HJ_DONE interrupt clear
		(16) MH_DONE	–	N/A	MH_DONE interrupt clear
		(15) DA_UPDATE	–	N/A	DA_UPDATE interrupt clear
		(14) SDR_FAIL	–	N/A	SDR_FAIL interrupt clear
		(13) DDR_FAIL	–	N/A	DDR_FAIL interrupt clear
		(12) M_RD_ABORT	–	N/A	M_RD_ABORT interrupt clear
		(11) DDR_RX_THR	–	N/A	DDR_RX_THR interrupt clear
		(10) DDR_TX_THR	–	N/A	DDR_TX_THR interrupt clear
		(9) SDR_RX_THR	–	N/A	SDR_RX_THR interrupt clear

Address	Register Name	Register Write	Register Read	Default	Description
		(8) SDR_TX_THR	–	N/A	SDR_TX_THR interrupt clear
		(7) DDR_RX_UNF	–	N/A	DDR_RX_UNF interrupt clear
		(6) DDR_TX_OVF	–	N/A	DDR_TX_OVF interrupt clear
		(5) SDR_RX_UNF	–	N/A	SDR_RX_UNF interrupt clear
		(4) SDR_TX_OVF	–	N/A	SDR_TX_OVF interrupt clear
		(3) DDR_RD_COMP	–	N/A	DDR_RD_COMP interrupt clear
		(2) DDR_WR_COMP	–	N/A	DDR_WR_COMP interrupt clear
		(1) SDR_RD_COMP	–	N/A	SDR_RD_COMP interrupt clear
		(0) SDR_WR_COMP	–	N/A	SDR_WR_COMP interrupt clear
P:0x1514 0x40001450	I3C0_SLV_ISR	–	(21) DEFSLVS	0x0	DEFSLVS Interrupt status
		–	(20) TM	0x0	TM Interrupt status
		–	(19) ERROR	0x0	ERROR Interrupt status
		–	(18) EVENT_UP	0x0	EVENT_UP Interrupt status
		–	(17) HJ_DONE	0x0	HJ_DONE Interrupt status
		–	(16) MH_DONE	0x0	MH_DONE Interrupt status
		–	(15) DA_UPDATE	0x0	DA_UPDATE Interrupt status
		–	(14) SDR_FAIL	0x0	SDR_FAIL Interrupt status
		–	(13) DDR_FAIL	0x0	DDR_FAIL Interrupt status
		–	(12) M_RD_ABORT	0x0	M_RD_ABORT Interrupt status
		–	(11) DDR_RX_THR	0x0	DDR_RX_THR Interrupt status
		–	(10) DDR_TX_THR	0x0	DDR_TX_THR Interrupt status
		–	(9) SDR_RX_THR	0x0	SDR_RX_THR Interrupt status

Address	Register Name	Register Write	Register Read	Default	Description
		–	(8) SDR_TX_THR	0x0	SLV_SDR_TX_THR Interrupt status
		–	(7) DDR_RX_UNF	0x0	DDR_RX_UNF Interrupt status
		–	(6) DDR_TX_OVF	0x0	DDR_TX_OVF Interrupt status
		–	(5) SDR_RX_UNF	0x0	SDR_RX_UNF Interrupt status
		–	(4) SDR_TX_OVF	0x0	SDR_TX_OVF Interrupt status
		–	(3) DDR_RD_COMP	0x0	DDR_RD_COMP Interrupt status
		–	(2) DDR_WR_COMP	0x0	DDR_WR_COMP Interrupt status
		–	(1) SDR_RD_COMP	0x0	SDR_RD_COMP Interrupt status
		–	(0) SDR_WR_COMP	0x0	SDR_WR_COMP Interrupt status
P:0x1515 0x40001454	I3C0_SLV_STATUS0	–	(23:16) REG_ADDR	0x0	Private Read/Write Address
		–	(15:0) XFERRED_BYTES	0x0	Number of transferred bytes in SDR transactions
P:0x1516 0x40001458	I3C0_SLV_STATUS1	–	(21:20) ENTAS	0x0	Activity state
		–	(19) VEN_TM	0x0	Vendor test mode detect status
		–	(18) HJ_DIS	0x1	Hot-Join disable status
		–	(17) MR_DIS	0x1	Mastership disable status
		–	(16) PROT_ERROR	0x0	Protocol error condition
		–	(15:9) DA	0x11	Slave Dynamic Address
		–	(8) HAS_DA	0x1	Slave Dynamic Address assigned
		–	(7) DDRRX_FULL	0x0	SLV_DDR_RX_FIFO full
		–	(6) DDRTX_FULL	0x0	SLV_DDR_TX_FIFO full

Address	Register Name	Register Write	Register Read	Default	Description
		–	(5) DDRX_EMPTY	0x1	SLV_DDR_RX_FIFO empty
		–	(4) DDRTX_EMPTY	0x1	SLV_DDR_TX_FIFO empty
		–	(3) SDRRX_FULL	0x0	This bit is set whenever SDR_RX_FIFO full
		–	(2) SDRTX_FULL	0x0	This bit is set whenever SDR_TX_FIFO full
		–	(1) SDRRX_EMPTY	0x1	This bit is set whenever SDR_RX_FIFO empty
		–	(0) SDRTX_EMPTY	0x1	This bit is set whenever SDR_TX_FIFO empty
P:0x1518 0x40001460	I3C0_CMD0_FIFO	(31) IS_DDR	–	N/A	Denotes whether it is a DDR command
		(30) IS_CCC	–	N/A	Denotes whether it is a CCC command
		(29) BCH	–	N/A	Enable/Disable broadcast header (enabled by default for CCC transfer)
		(28:27) XMIT_MODE	–	N/A	Private read/write transfer mode (non CCC transfer)
		(26) SBCA	–	N/A	Sixteen bits register sub-addressing (private transfer)
		(25) RSBC	–	N/A	Repeated start between commands
		(24) IS10B	–	N/A	Enable/Disable 10 bit extended addr mode (legacy I2C)
		(23:12) PL_LEN	–	N/A	Payload length
		(10:8) DEV_ADDR_MSB	–	N/A	MSB 10-bit extended address (IS10B = 1)
		(7:1) DEV_ADDR	–	N/A	Slave static/dynamic address

Address	Register Name	Register Write	Register Read	Default	Description
		(0) RNW	–	N/A	Transfer direction
P:0x1519 0x40001464	I3C0_CMD1_FIFO	(31:24) CMD_ID	–	N/A	Command ID
		(15:8) CSRADDR1	–	N/A	CSR ADDR1 - second byte of the CSR address in 16-bit addressing mode
		(7:0) CCC_CSRADDR0	–	N/A	CCC/CSR ADDR0 - CCC or CSR Address
P:0x151A 0x40001468	I3C0_TX_FIFO	(31:0) DATA	–	N/A	Tx Data FIFO
P:0x151C 0x40001470	I3C0_IMD_CMD0	(14:12) PL_LEN	–	N/A	Payload Length
		(7:1) DEV_ADDR	–	N/A	Slave static/dynamic address
		(0) RNW	–	N/A	Transfer direction
P:0x151D 0x40001474	I3C0_IMD_CMD1	(31:24) CMD_ID	–	N/A	Command ID
		(7:0) CCC	–	N/A	CCC code
P:0x151E 0x40001478	I3C0_IMD_DATA	(31:0) DATA	(31:0) DATA	0x0	Payload/Data for a particular immediate command
P:0x1520 0x40001480	I3C0_RX_FIFO	–	(31:0) DATA	0x0	Rx Data FIFO
P:0x1521 0x40001484	I3C0_IBI_DATA_FIFO	–	(31:0) DATA	0x0	IBI Data receive FIFO
P:0x1522	I3C0_SLV_DDR_TX_FIFO	(19:0) DATA	–	N/A	Slave Tx DDR send FIFO

Address	Register Name	Register Write	Register Read	Default	Description
0x40001488					
P:0x1523 0x4000148C	I3C0_SLV_DDR_RX_FIFO	–	(19:0) DATA	0x0	Slave Tx DDR receive FIFO
P:0x1524 0x40001490	I3C0_CMD_IBI_THR_CTRL	(29:24) IBIR_THR	(29:24) IBIR_THR	0x1	IBI response queue threshold
		(20:16) CMDR_THR	(20:16) CMDR_THR	0x1	Command response queue threshold
		(13:8) IBID_THR	(13:8) IBID_THR	0x1	IBI data FIFO threshold
		(4:0) CMDD_THR	(4:0) CMDD_THR	0x1	Command descriptor memory threshold
P:0x1525 0x40001494	I3C0_TX_RX_THR_CTRL	(31:16) RX_THR	(31:16) RX_THR	0x1	Rx data memory threshold
		(15:0) TX_THR	(15:0) TX_THR	0x1	Tx data memory threshold
P:0x1526 0x40001498	I3C0_SLV_DDR_TX_RX_THR_CTRL	(31:16) SLV_DDR_RX_THR	(31:16) SLV_DDR_RX_THR	0x1	Slave Mode DDR Rx data memory threshold
		(15:0) SLV_DDR_TX_THR	(15:0) SLV_DDR_TX_THR	0x1	Slave Mode DDR Tx data memory threshold
P:0x1527 0x4000149C	I3C0_FLUSH_CTRL	(24) IBI_RESP	–	N/A	IBI response queue flush
		(23) CMD_RESP	–	N/A	Command response queue flush
		(22) SLV_DDR_RX	–	N/A	Slave DDR Rx FIFO flush
		(21) SLV_DDR_TX	–	N/A	Slave DDR Tx FIFO flush
		(20) IMM_CMD	–	N/A	Immediate command and data register flush
		(19) IBI	–	N/A	IBI received data FIFO flush

Address	Register Name	Register Write	Register Read	Default	Description
		(18) RX	–	N/A	Rx FIFO flush
		(17) TX	–	N/A	Tx FIFO flush
		(16) CMD	–	N/A	Command FIFO flush
P:0x152C 0x400014B0	I3C0_TTO_PRESC1_CTRL0	(25:16) DIV_B	(25:16) DIV_B	0x0	Divider B
		(10:0) DIV_A	(10:0) DIV_A	0x0	Divider A
P:0x152D 0x400014B4	I3C0_TTO_PRESC1_CTRL1	(25:16) DIV_B	(25:16) DIV_B	0x0	Divider B
		(7:0) DIV_A	(7:0) DIV_A	0x0	Divider A
P:0x152E 0x400014B8	I3C0_DEVS_CTRL	(19) DEV3_CLEAR	–	N/A	Clear DevID3 retaining registers set. Self-cleared bit.
		(18) DEV2_CLEAR	–	N/A	Clear DevID2 retaining registers set. Self-cleared bit.
		(17) DEV1_CLEAR	–	N/A	Clear DevID1 retaining registers set. Self-cleared bit.
		(4) DEV4_CLEAR	(4) DEV4_CLEAR	0x0	DevID4 is active - has either valid DA or SA
		(3) DEV3_ACTIVE	(3) DEV3_ACTIVE	0x0	DevID3 is active - has either valid DA or SA
		(2) DEV2_ACTIVE	(2) DEV2_ACTIVE	0x0	DevID2 is active - has either valid DA or SA
		(1) DEV1_ACTIVE	(1) DEV1_ACTIVE	0x0	DevID1 is active - has either valid DA or SA
		(0) DEV0_ACTIVE	(0) DEV0_ACTIVE	0x1	DevID0 is active - has either valid DA or SA

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1530 0x400014C0	I3C0_DEV_ID0_RR0	(15:13) LVR_SA_MSB	(15:13) LVR_SA_MSB	0x0	MSB bits of Legacy I2C device with 10-bit addressing
		(11) LVR_EXT_ADDR	(11) LVR_EXT_ADDR	0x0	Address mode
		-	(9) IS_I3C	0x1	I3C mode
		(7:0) DEV_ADDR	(7:0) DEV_ADDR	0x10	Slave Dynamic (Static/Legacy) Address
P:0x1531 0x400014C4	I3C0_DEV_ID0_RR1	(31:0) PID_MSB	(31:0) PID_MSB	0x25D0010	Provisional ID MSB 32-bits (48 to 16)
P:0x1532 0x400014C8	I3C0_DEV_ID0_RR2	(31:16) PID_LSB	(31:16) PID_LSB	0xC000	Provisional ID LSB 16-bits (15 to 0)
		(15:8) BCR	(15:8) BCR	0x62	BCR register
		(7:0) DCR_LVR	(7:0) DCR_LVR	0x5A	DCR/LVR register
P:0x1534 0x400014D0	I3C0_DEV_ID1_RR0	(15:13) LVR_SA_MSB	(15:13) LVR_SA_MSB	0x0	MSB bits of Legacy I2C Device with 10-bit addressing
		(11) LVR_EXT_ADDR	(11) LVR_EXT_ADDR	0x0	Address mode
		(9) IS_I3C	(9) IS_I3C	0x1	I3C mode
		(7:0) DEV_ADDR	(7:0) DEV_ADDR	0x13	Slave Dynamic (Static/Legacy) Address
P:0x1535 0x400014D4	I3C0_DEV_ID1_RR1	(31:0) PID_MSB	(31:0) PID_MSB	0x0	Provisional ID MSB 32-bits (48 to 16)
P:0x1536 0x400014D8	I3C0_DEV_ID1_RR2	(31:16) PID_LSB	(31:16) PID_LSB	0x0	Provisional ID MSB 32-bits (48 to 16)
		(15:8) BCR	(15:8) BCR	0x0	BCR register
		(7:0) DCR_LVR	(7:0) DCR_LVR	0x0	DCR/LVR register

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1538 0x400014E0	I3C0_DEV_ID2_RR0	(15:13) LVR_SA_MSB	(15:13) LVR_SA_MSB	0x0	MSB bits of Legacy I2C Device with 10-bit addressing
		(11) LVR_EXT_ADDR	(11) LVR_EXT_ADDR	0x0	Address mode
		(9) IS_I3C	(9) IS_I3C	0x1	I3C mode
		(7:0) DEV_ADDR	(7:0) DEV_ADDR	0x15	Slave Dynamic (Static/Legacy) Address
P:0x1539 0x400014E4	I3C0_DEV_ID2_RR1	(31:0) PID_MSB	(31:0) PID_MSB	0x0	Provisional ID MSB 32-bits (48 to 16)
P:0x153A 0x400014E8	I3C0_DEV_ID2_RR2	(31:16) PID_LSB	(31:16) PID_LSB	0x0	Provisional ID LSB 16-bits (15 to 0)
		(15:8) BCR	(15:8) BCR	0x0	BCR register
		(7:0) DCR_LVR	(7:0) DCR_LVR	0x0	DCR/LVR register
P:0x153C 0x400014F0	I3C0_DEV_ID3_RR0	(15:13) LVR_SA_MSB	(15:13) LVR_SA_MSB	0x0	MSB bits of Legacy I2C Device with 10-bit addressing
		(11) LVR_EXT_ADDR	(11) LVR_EXT_ADDR	0x0	Address mode
		(9) IS_I3C	(9) IS_I3C	0x1	I3C mode
		(7:0) DEV_ADDR	(7:0) DEV_ADDR	0x16	Slave Dynamic (Static/Legacy) Address
P:0x153D 0x400014F4	I3C0_DEV_ID3_RR1	(31:0) PID_MSB	(31:0) PID_MSB	0x0	Provisional ID MSB 32-bits (48 to 16)
P:0x153E 0x400014F8	I3C0_DEV_ID3_RR2	(31:16) PID_LSB	(31:16) PID_LSB	0x0	Provisional ID MSB 32-bits (48 to 16)
		(15:8) BCR	(15:8) BCR	0x0	BCR register
		(7:0) DCR_LVR	(7:0) DCR_LVR	0x0	DCR/LVR register

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1560 0x40001580	I3C0_SIR_MAP0	(31:30) DEVID1_ROLE	(31:30) DEVID1_ROLE	0x0	Device ID1 BCR role
		(29) DEVID1_SLOW	(29) DEVID1_SLOW	0x0	Device ID1 Maximum data speed limit
		(28:24) DEVID1_PL	(28:24) DEVID1_PL	0x0	Device ID1 Payload length
		(23:17) DEVID1_DA	(23:17) DEVID1_DA	0x7F	Device ID1 DA
		(16) DEVID1_RESP	(16) DEVID1_RESP	0x0	Device ID1 ACK/NACK response
		(15:14) DEVID0_ROLE	(15:14) DEVID0_ROLE	0x0	Device ID0 BCR role
		(13) DEVID0_SLOW	(13) DEVID0_SLOW	0x0	Device ID0 Maximum data speed limit
		(12:8) DEVID0_PL	(12:8) DEVID0_PL	0x0	Device ID0 Payload length
		(7:1) DEVID0_DA	(7:1) DEVID0_DA	0x7F	Device ID0 DA
		(0) DEVID0_RESP	(0) DEVID0_RESP	0x0	Device ID0 ACK/NACK response
P:0x1561 0x40001584	I3C0_SIR_MAP1	(15:14) DEVID2_ROLE	(15:14) DEVID2_ROLE	0x0	Device ID2 BCR role
		(13) DEVID2_SLOW	(13) DEVID2_SLOW	0x0	Device ID2 Maximum data speed limit
		(12:8) DEVID2_PL	(12:8) DEVID2_PL	0x0	Device ID2 Payload length
		(7:1) DEVID2_DA	(7:1) DEVID2_DA	0x7F	Device ID2 DA
		(0) DEVID2_RESP	(0) DEVID2_RESP	0x0	Device ID2 ACK/NACK response
P:0x1568 0x400015A0	I3C0_GPIR_WORD0	–	(7:0) GPIO	0x0	User Defined GPI Register 0
P:0x1588 0x40001620	I3C0_GPOR_WORD0	–	(7:0) GPO0	0x0	User Defined GPO Register 0
P:0x1600	I3C1_DEV_ID	–	(15:0) DEV_ID	0x5034	IP identifier

Address	Register Name	Register Write	Register Read	Default	Description
0x40001800					
P:0x1601 0x40001804	I3C1_CONF_STATUS0	–	(31:29) CMDR_MEM_DEPTH	0x0	Command response queue depth
		–	(28:24) ASF	0x0	ASF checks available
		–	(23:16) GPO_NUM	0x1	GPO available
		–	(15:8) GPI_NUM	0x1	GPI available
		–	(7:6) IBIR_MEM_DEPTH	0x1	IBI response queue depth
		–	(5) DDR	0x1	DDR support
		–	(4) DEV_ROLE	0x0	Device role
		–	(3:0) DEVS_NUM	0x3	Slave retaining registers
P:0x1602 0x40001808	I3C1_CONF_STATUS1	–	(31:28) IBI_HW_RES	0x2	IBI resources
		–	(27:26) CMD_MEM_DEPTH	0x0	Command memory depth
		–	(25:21) SLV_DDR_RX_MEM_DEPTH	0x1F	Slave DDR Rx memory depth
		–	(20:16) SLV_DDR_TX_MEM_DEPTH	0x1F	Slave DDR Tx memory depth
		–	(12:10) IBI_MEM_DEPTH	0x1	IBI memory depth
		–	(9:5) RX_MEM_DEPTH	0x0	Rx memory depth
		–	(4:0) TX_MEM_DEPTH	0x0	Tx memory depth
P:0x1604	I3C1_CTRL	(31) DEV_EN	(31) DEV_EN	0x0	Enable/Disable I3C master

Address	Register Name	Register Write	Register Read	Default	Description
0x40001810					
		(30) HALT_EN	(30) HALT_EN	0x0	Enable/Disable halt on abort behavior
		(29) MCS	–	N/A	Manual command start (MCS_EN=1)
		(28) MCS_EN	(28) MCS_EN	0x0	Enable/Disable manual command send
		(26) I3C_11_SUPP	(26) I3C_11_SUPP	0x0	Timing parameter tCASr_min
		(25:24) THD_DEL	(25:24) THD_DEL	0x0	Data hold tHD_DAT delay
		(8) HJ_DISEC	(8) HJ_DISEC	0x0	Hot-Join ACK'ed response(HJ_ACK=1)
		(7) MST_ACK	(7) MST_ACK	0x1	Mastership Accept (GETACCMST) response (slave mode = 1)
		(6) HJ_ACK	(6) HJ_ACK	0x1	Hot-Join acknowledgment
		(5) HJ_INIT	–	N/A	Initiate Hot-Join
		(4) MST_INIT	–	N/A	Initiate Mastership request
		(3) AHDR_OPT	(3) AHDR_OPT	0x0	Enable/Disable address header optimization
		(1:0) BUS_MODE	(1:0) BUS_MODE	0x0	Bus Mode
P:0x1605 0x40001814	I3C1_PRESC1_CTRL0	(31:16) I2C	(31:16) I2C	0x7C	Prescaler value for I2C SCL clock generation
		(9:0) I3C	(9:0) I3C	0x4	Prescaler value for I3C Push-Pull SDR Mode SCL clock generation
P:0x1606 0x40001818	I3C1_PRESC1_CTRL1	(15:8) PP_LOW	(15:8) PP_LOW	0x0	Counter for low period of SCL clock for Push-Pull in I3C
		(7:0) OD_LOW	(7:0) OD_LOW	0x9	Counter for low period of SCL clock for Open Drain in I3C
P:0x1608	I3C1_MST_IER	(18) HALTED	–	N/A	HALTED interrupt enable

Address	Register Name	Register Write	Register Read	Default	Description
0x40001820					
		(17) MR_DONE	–	N/A	MR_DONE interrupt enable
		(16) IMM_COMP	–	N/A	IMM_COMP interrupt enable
		(15) TX_THR	–	N/A	TX_THR interrupt enable
		(14) TX_OVF	–	N/A	TX_OVF interrupt enable
		(12) IBID_THR	–	N/A	IBID_THR interrupt enable
		(11) IBID_UNF	–	N/A	IBID_UNF interrupt enable
		(10) IBIR_THR	–	N/A	IBIR_THR interrupt enable
		(9) IBIR_UNF	–	N/A	IBIR_UNF interrupt enable
		(8) IBIR_OVF	–	N/A	IBIR_OVF interrupt enable
		(7) RX_THR	–	N/A	RX_THR interrupt enable
		(6) RX_UNF	–	N/A	RX_UNF interrupt enable
		(5) CMDD_EMP	–	N/A	CMDD_EMP interrupt enable
		(4) CMDD_THR	–	N/A	CMDD_THR interrupt enable
		(3) CMDD_OVF	–	N/A	CMDD_OVF interrupt enable
		(2) CMDR_THR	–	N/A	CMDR_THR interrupt enable
		(1) CMDR_UNF	–	N/A	CMDR_UNF interrupt enable
		(0) CMDR_OVF	–	N/A	CMDR_OVF interrupt enable
P:0x1609	I3C1_MST_IDR	(18) HALTED	–	N/A	HALTED interrupt disable
0x40001824					
		(17) MR_DONE	–	N/A	MR_DONE interrupt disable
		(16) IMM_COMP	–	N/A	IMM_COMP interrupt disable
		(15) TX_THR	–	N/A	TX_THR interrupt disable

Address	Register Name	Register Write	Register Read	Default	Description
		(14) TX_OVF	–	N/A	TX_OVF interrupt disable
		(12) IBID_THR	–	N/A	IBID_THR interrupt disable
		(11) IBID_UNF	–	N/A	IBID_UNF interrupt disable
		(10) IBIR_THR	–	N/A	IBIR_THR interrupt disable
		(9) IBIR_UNF	–	N/A	IBIR_UNF interrupt disable
		(8) IBIR_OVF	–	N/A	IBIR_OVF interrupt disable
		(7) RX_THR	–	N/A	RX_THR interrupt disable
		(6) RX_UNF	–	N/A	RX_UNF interrupt disable
		(5) CMDD_EMP	–	N/A	CMDD_EMP interrupt disable
		(4) CMDD_THR	–	N/A	CMDD_THR interrupt disable
		(3) CMDD_OVF	–	N/A	CMDD_OVF interrupt disable
		(2) CMDR_THR	–	N/A	CMDR_THR interrupt disable
		(1) CMDR_UNF	–	N/A	CMDR_UNF interrupt disable
		(0) CMDR_OVF	–	N/A	CMDR_OVF interrupt disable
P:0x160A 0x40001828	I3C1_MST_IMR	–	(18) HALTED	0x0	HALTED interrupt mask
		–	(17) MS_DONE	0x0	MR_DONE interrupt mask
		–	(16) IMM_COMP	0x0	IMM_COMP interrupt mask
		–	(15) TX_THR	0x0	TX_THR interrupt mask
		–	(14) TX_OVF	0x0	TX_OVF interrupt mask
		–	(12) IBID_THR	0x0	IBID_THR interrupt mask
		–	(11) IBID_UNF	0x0	IBIR_UNF interrupt mask
		–	(10) IBIR_THR	0x0	IBIR_OVF interrupt mask

Address	Register Name	Register Write	Register Read	Default	Description
		–	(9) IBIR_UNF	0x0	IBIR_UNF interrupt mask
		–	(8) IBIR_OVF	0x0	IBIR_OVF interrupt mask
		–	(7) RX_THR	0x0	RX_THR interrupt mask
		–	(6) RX_UNF	0x0	RX_UNF interrupt mask
		–	(5) CMDD_EMP	0x0	CMDD_EMP interrupt mask
		–	(4) CMDD_THR	0x0	CMDD_THR interrupt mask
		–	(3) CMDD_OVF	0x0	CMDD_OVF interrupt mask
		–	(2) CMDR_THR	0x0	CMDR_THR interrupt mask
		–	(1) CMDR_UNF	0x0	CMDR_UNF interrupt mask
		–	(0) CMDR_OVF	0x0	CMDR_OVF interrupt mask
P:0x160B 0x4000182C	I3C1_MST_ICR	(18) HALTED	–	N/A	HALTED interrupt clear
		(17) MR_DONE	–	N/A	MR_DONE interrupt clear
		(16) IMM_COMP	–	N/A	IMM_COMP interrupt clear
		(15) TX_THR	–	N/A	TX_THR interrupt clear
		(14) TX_OVF	–	N/A	TX_OVF interrupt clear
		(12) IBID_THR	–	N/A	IBID_THR interrupt clear
		(11) IBID_UNF	–	N/A	IBID_UNF interrupt clear
		(10) IBIR_THR	–	N/A	IBIR_THR interrupt clear
		(9) IBIR_UNF	–	N/A	IBIR_UNF interrupt clear
		(8) IBIR_OVF	–	N/A	IBIR_OVF interrupt clear
		(7) RX_THR	–	N/A	RX_THR interrupt clear
		(6) RX_UNF	–	N/A	RX_UNF interrupt clear

Address	Register Name	Register Write	Register Read	Default	Description
		(5) CMDD_EMP	–	N/A	CMDD_EMP interrupt clear
		(4) CMDD_THR	–	N/A	CMDD_THR interrupt clear
		(3) CMDD_OVF	–	N/A	CMDD_OVF interrupt clear
		(2) CMDR_THR	–	N/A	CMDR_THR interrupt clear
		(1) CMDR_UNF	–	N/A	CMDR_UNF interrupt clear
		(0) CMDR_OVF	–	N/A	CMDR_OVF interrupt clear
P:0x160C 0x40001830	I3C1_MST_ISR	–	(18) HALTED	0x0	HALTED interrupt status
		–	(17) MR_DONE	0x0	MR_DONE interrupt status
		–	(16) IMM_COMP	0x0	IMM_COMP interrupt status
		–	(15) TX_THR	0x0	TX_THR interrupt status
		–	(14) TX_OVF	0x0	TX_OVF interrupt status
		–	(12) IBID_THR	0x0	IBID_THR interrupt status
		–	(11) IBID_UNF	0x0	IBIR_UNF interrupt status
		–	(10) IBIR_THR	0x0	IBIR_OVF interrupt status
		–	(9) IBIR_UNF	0x0	IBIR_UNF interrupt status
		–	(8) IBIR_OVF	0x0	IBIR_OVF interrupt status
		–	(7) RX_THR	0x0	RX_THR interrupt status
		–	(6) RX_UNF	0x0	RX_UNF interrupt status
		–	(5) CMDD_EMP	0x0	CMDD_EMP interrupt status
		–	(4) CMDD_THR	0x0	CMDD_THR interrupt status
		–	(3) CMDD_OVF	0x0	CMDD_OVF interrupt status
		–	(2) CMDR_THR	0x0	CMDR_THR interrupt status

Address	Register Name	Register Write	Register Read	Default	Description
		–	(1) CMDR_UNF	0x0	CMDR_UNF interrupt status
		–	(0) CMDR_OVF	0x0	CMDR_OVF interrupt status
P:0x160D 0x40001834	I3C1_MST_STATUS0	–	(18) IDLE	0x1	Core is idle
		(17) HALTED	(17) HALTED	0x0	Core halted (W1C to un halt core)
		–	(16) OP_MODE	0x1	Indicates current mode of the controller
		–	(13) TX_FULL	0x0	TX Full
		–	(12) IBID_FULL	0x0	IBID Full
		–	(11) IBIR_FULL	0x0	IBIR Full
		–	(10) RX_FULL	0x0	RX Full
		–	(9) CMDD_FULL	0x0	CMDD Full
		–	(8) CMDR_FULL	0x0	CMDR Full
		–	(5) TX_EMP	0x1	TX Empty
		–	(4) IBID_EMP	0x1	IBID Empty
		–	(3) IBIR_EMP	0x1	IBIR Empty
		–	(2) RX_EMP	0x1	RX Empty
		–	(1) CMDD_EMP	0x1	CMDD Empty
		–	(0) CMDR_EMP	0x1	CMDR Empty
P:0x160E 0x40001838	I3C1_CMDR	–	(27:24) ERROR	0x0	Error code during command execution
		–	(19:8) XFER_BYTES	0x0	Number of bytes transferred
		–	(7:0) CMD_ID	0x0	Command Identifier field
P:0x160F	I3C1_IBIR	–	(12) RESP	0x0	Master response

Address	Register Name	Register Write	Register Read	Default	Description
0x4000183C					
		–	(11:8) SLV_ID	0x0	Slave ID
		–	(7) ERROR	0x0	Overflow Error
		–	(6:2) XFER_BYTES	0x0	Number of received data bytes
		–	(1:0) IBI_TYPE	0x0	IBI type
P:0x1610	I3C1_SLV_IER	(21) DEFSLVS	–	N/A	DEFSLVS interrupt enable
0x40001840					
		(20) TM	–	N/A	TM interrupt enable
		(19) ERROR	–	N/A	ERROR interrupt enable
		(18) EVENT_UP	–	N/A	EVENT_UP interrupt enable
		(17) HJ_DONE	–	N/A	HJ_DONE interrupt enable
		(16) MH_DONE	–	N/A	MH_DONE interrupt enable
		(15) DA_UPDATE	–	N/A	DA_UPDATE interrupt enable
		(14) SDR_FAIL	–	N/A	SDR_FAIL interrupt enable
		(13) DDR_FAIL	–	N/A	DDR_FAIL interrupt enable
		(12) M_RD_ABORT	–	N/A	M_RD_ABORT interrupt enable
		(11) DDR_RX_THR	–	N/A	DDR_RX_THR interrupt enable
		(10) DDR_TX_THR	–	N/A	DDR_TX_THR interrupt enable
		(9) SDR_RX_THR	–	N/A	SDR_RX_THR interrupt enable
		(8) SDR_TX_THR	–	N/A	SDR_TX_THR interrupt enable
		(7) DDR_RX_UNF	–	N/A	DDR_RX_UNF interrupt enable
		(6) DDR_TX_OVF	–	N/A	DDR_TX_OVF interrupt enable
		(5) SDR_RX_UNF	–	N/A	SDR_RX_UNF interrupt enable

Address	Register Name	Register Write	Register Read	Default	Description
		(4) SDR_TX_OVF	–	N/A	SDR_TX_OVF interrupt enable
		(3) DDR_RD_COMP	–	N/A	DDR_RD_COMP interrupt enable
		(2) DDR_WR_COMP	–	N/A	DDR_WR_COMP interrupt enable
		(1) SDR_RD_COMP	–	N/A	SDR_RD_COMP interrupt enable
		(0) SDR_WR_COMP	–	N/A	SDR_WR_COMP interrupt enable
P:0x1611 0x40001844	I3C1_SLV_IDR	(21) DEFSLVS	–	N/A	DEFSLVS interrupt disable
		(20) TM	–	N/A	TM interrupt disable
		(19) ERROR	–	N/A	ERROR interrupt disable
		(18) EVENT_UP	–	N/A	EVENT_UP interrupt disable
		(17) HJ_DONE	–	N/A	HJ_DONE interrupt disable
		(16) MH_DONE	–	N/A	MH_DONE interrupt disable
		(15) DA_UPDATE	–	N/A	DA_UPDATE interrupt disable
		(14) SDR_FAIL	–	N/A	SDR_FAIL interrupt disable
		(13) DDR_FAIL	–	N/A	DDR_FAIL interrupt disable
		(12) M_RD_ABORT	–	N/A	M_RD_ABORT interrupt disable
		(11) DDR_RX_THR	–	N/A	DDR_RX_THR interrupt disable
		(10) DDR_TX_THR	–	N/A	DDR_TX_THR interrupt disable
		(9) SDR_RX_THR	–	N/A	SDR_RX_THR interrupt disable
		(8) SDR_TX_THR	–	N/A	SDR_TX_THR interrupt disable
		(7) DDR_RX_UNF	–	N/A	DDR_RX_UNF interrupt disable
		(6) DDR_TX_OVF	–	N/A	DDR_TX_OVF interrupt disable
		(5) SDR_RX_UNF	–	N/A	SDR_RX_UNF interrupt disable

Address	Register Name	Register Write	Register Read	Default	Description
		(4) SDR_TX_OVF	–	N/A	SDR_TX_OVF interrupt disable
		(3) DDR_RD_COMP	–	N/A	DDR_RD_COMP interrupt disable
		(2) DDR_WR_COMP	–	N/A	DDR_WR_COMP interrupt disable
		(1) SDR_RD_COMP	–	N/A	SDR_RD_COMP interrupt disable
		(0) SDR_WR_COMP	–	N/A	SDR_WR_COMP interrupt disable
P:0x1612 0x40001848	I3C1_SLV_IMR	–	(21) DEFSLVS	0x0	DEFSLVS Interrupt mask
		–	(20) TM	0x0	TM Interrupt mask
		–	(19) ERROR	0x0	ERROR Interrupt mask
		–	(18) EVENT_UP	0x0	EVENT_UP Interrupt mask
		–	(17) HJ_DONE	0x0	HJ_DONE Interrupt mask
		–	(16) MH_DONE	0x0	MH_DONE Interrupt mask
		–	(15) DA_UPDATE	0x0	DA_UPDATE Interrupt mask
		–	(14) SDR_FAIL	0x0	SDR_FAIL Interrupt mask
		–	(13) DDR_FAIL	0x0	DDR_FAIL Interrupt mask
		–	(12) M_RD_ABORT	0x0	M_RD_ABORT Interrupt mask
		–	(11) DDR_RX_THR	0x0	DDR_RX_THR Interrupt mask
		–	(10) DDR_TX_THR	0x0	DDR_TX_THR Interrupt mask
		–	(9) SDR_RX_THR	0x0	SDR_RX_THR Interrupt mask
		–	(8) SDR_TX_THR	0x0	SDR_TX_THR Interrupt mask
		–	(7) DDR_RX_UNF	0x0	DDR_RX_UNF Interrupt mask
		–	(6) DDR_TX_OVF	0x0	DDR_TX_OVF Interrupt mask
		–	(5) SDR_RX_UNF	0x0	SDR_RX_UNF Interrupt mask

Address	Register Name	Register Write	Register Read	Default	Description
		–	(4) SDR_TX_OVF	0x0	SDR_TX_OVF Interrupt mask
		–	(3) DDR_RD_COMP	0x0	DDR_RD_COMP Interrupt mask
		–	(2) DDR_WR_COMP	0x0	DDR_WR_COMP Interrupt mask
		–	(1) SDR_RD_COMP	0x0	SDR_RD_COMP Interrupt mask
		–	(0) SDR_WR_COMP	0x0	SDR_WR_COMP Interrupt mask
P:0x1613 0x4000184C	I3C1_SLV_ICR	(21) DEFSLVS	–	N/A	DEFSLVS interrupt clear
		(20) TM	–	N/A	TM interrupt clear
		(19) ERROR	–	N/A	ERROR interrupt clear
		(18) EVENT_UP	–	N/A	EVENT_UP interrupt clear
		(17) HJ_DONE	–	N/A	HJ_DONE interrupt clear
		(16) MH_DONE	–	N/A	MH_DONE interrupt clear
		(15) DA_UPDATE	–	N/A	DA_UPDATE interrupt clear
		(14) SDR_FAIL	–	N/A	SDR_FAIL interrupt clear
		(13) DDR_FAIL	–	N/A	DDR_FAIL interrupt clear
		(12) M_RD_ABORT	–	N/A	M_RD_ABORT interrupt clear
		(11) DDR_RX_THR	–	N/A	DDR_RX_THR interrupt clear
		(10) DDR_TX_THR	–	N/A	DDR_TX_THR interrupt clear
		(9) SDR_RX_THR	–	N/A	SDR_RX_THR interrupt clear
		(8) SDR_TX_THR	–	N/A	SDR_TX_THR interrupt clear
		(7) DDR_RX_UNF	–	N/A	DDR_RX_UNF interrupt clear
		(6) DDR_TX_OVF	–	N/A	DDR_TX_OVF interrupt clear
		(5) SDR_RX_UNF	–	N/A	SDR_RX_UNF interrupt clear

Address	Register Name	Register Write	Register Read	Default	Description
		(4) SDR_TX_OVF	–	N/A	SDR_TX_OVF interrupt clear
		(3) DDR_RD_COMP	–	N/A	DDR_RD_COMP interrupt clear
		(2) DDR_WR_COMP	–	N/A	DDR_WR_COMP interrupt clear
		(1) SDR_RD_COMP	–	N/A	SDR_RD_COMP interrupt clear
		(0) SDR_WR_COMP	–	N/A	SDR_WR_COMP interrupt clear
P:0x1614 0x40001850	I3C1_SLV_ISR	–	(21) DEFSLVS	0x0	DEFSLVS Interrupt status
		–	(20) TM	0x0	TM Interrupt status
		–	(19) ERROR	0x0	ERROR Interrupt status
		–	(18) EVENT_UP	0x0	EVENT_UP Interrupt status
		–	(17) HJ_DONE	0x0	HJ_DONE Interrupt status
		–	(16) MH_DONE	0x0	MH_DONE Interrupt status
		–	(15) DA_UPDATE	0x0	DA_UPDATE Interrupt status
		–	(14) SDR_FAIL	0x0	SDR_FAIL Interrupt status
		–	(13) DDR_FAIL	0x0	DDR_FAIL Interrupt status
		–	(12) M_RD_ABORT	0x0	M_RD_ABORT Interrupt status
		–	(11) DDR_RX_THR	0x0	DDR_RX_THR Interrupt status
		–	(10) DDR_TX_THR	0x0	DDR_TX_THR Interrupt status
		–	(9) SDR_RX_THR	0x0	SDR_RX_THR Interrupt status
		–	(8) SDR_TX_THR	0x0	SLV_SDR_TX_THR Interrupt status
		–	(7) DDR_RX_UNF	0x0	DDR_RX_UNF Interrupt status
		–	(6) DDR_TX_OVF	0x0	DDR_TX_OVF Interrupt status
		–	(5) SDR_RX_UNF	0x0	SDR_RX_UNF Interrupt status

Address	Register Name	Register Write	Register Read	Default	Description
		–	(4) SDR_TX_OVF	0x0	SDR_TX_OVF Interrupt status
		–	(3) DDR_RD_COMP	0x0	DDR_RD_COMP Interrupt status
		–	(2) DDR_WR_COMP	0x0	DDR_WR_COMP Interrupt status
		–	(1) SDR_RD_COMP	0x0	SDR_RD_COMP Interrupt status
		–	(0) SDR_WR_COMP	0x0	SDR_WR_COMP Interrupt status
P:0x1615 0x40001854	I3C1_SLV_STATUS0	–	(23:16) REG_ADDR	0x0	Private Read/Write Address
		–	(15:0) XFERRED_BYTES	0x0	Number of transferred bytes in SDR transactions
P:0x1616 0x40001858	I3C1_SLV_STATUS1	–	(21:20) ENTAS	0x0	Activity state
		–	(19) VEN_TM	0x0	Vendor test mode detect status
		–	(18) HJ_DIS	0x1	Hot-Join disable status
		–	(17) MR_DIS	0x1	Mastership disable status
		–	(16) PROT_ERROR	0x0	Protocol error condition
		–	(15:9) DA	0x11	Slave Dynamic Address
		–	(8) HAS_DA	0x1	Slave Dynamic Address assigned
		–	(7) DDRRX_FULL	0x0	SLV_DDR_RX_FIFO full
		–	(6) DDRTX_FULL	0x0	SLV_DDR_TX_FIFO full
		–	(5) DDRRX_EMPTY	0x1	SLV_DDR_RX_FIFO empty
		–	(4) DDRTX_EMPTY	0x1	SLV_DDR_TX_FIFO empty
		–	(3) SDRRX_FULL	0x0	This bit is set whenever SDR_RX_FIFO full
		–	(2) SDRTX_FULL	0x0	This bit is set whenever SDR_TX_FIFO

Address	Register Name	Register Write	Register Read	Default	Description
					full
		–	(1) SDRRX_EMPTY	0x1	This bit is set whenever SDR_RX_FIFO empty
		–	(0) SDRTX_EMPTY	0x1	This bit is set whenever SDR_TX_FIFO empty
P:0x1618 0x40001860	I3C1_CMD0_FIFO	(31) IS_DDR	–	N/A	Denotes whether it is a DDR command
		(30) IS_CCC	–	N/A	Denotes whether it is a CCC command
		(29) BCH	–	N/A	Enable/Disable broadcast header (enabled by default for CCC transfer)
		(28:27) XMIT_MODE	–	N/A	Private read/write transfer mode (non CCC transfer)
		(26) SBCA	–	N/A	Sixteen bits register sub-addressing (private transfer)
		(25) RSBC	–	N/A	Repeated start between commands
		(24) IS10B	–	N/A	Enable/Disable 10 bit extended addr mode (legacy I2C)
		(23:12) PL_LEN	–	N/A	Payload length
		(10:8) DEV_ADDR_MSB	–	N/A	MSB 10-bit extended address (IS10B = 1)
		(7:1) DEV_ADDR	–	N/A	Slave static/dynamic address
		(0) RNW	–	N/A	Transfer direction
P:0x1619 0x40001864	I3C1_CMD1_FIFO	(31:24) CMD_ID	–	N/A	Command ID
		(15:8) CSRADDR1	–	N/A	CSR ADDR1 - second byte of the CSR address in 16-bit addressing mode

Address	Register Name	Register Write	Register Read	Default	Description
		(7:0) CCC_CSRADDR0	–	N/A	CCC/CSR ADDR0 - CCC or CSR Address
P:0x161A 0x40001868	I3C1_TX_FIFO	(31:0) DATA	–	N/A	Tx Data FIFO
P:0x161C 0x40001870	I3C1_IMD_CMD0	(14:12) PL_LEN	–	N/A	Payload Length
		(7:1) DEV_ADDR	–	N/A	Slave static/dynamic address
		(0) RNW	–	N/A	Transfer direction
P:0x161D 0x40001874	I3C1_IMD_CMD1	(31:24) CMD_ID	–	N/A	Command ID
		(7:0) CCC	–	N/A	CCC code
P:0x161E 0x40001878	I3C1_IMD_DATA	(31:0) DATA	(31:0) DATA	0x0	Payload/Data for a particular immediate command
P:0x1620 0x40001880	I3C1_RX_FIFO	–	(31:0) DATA	0x0	Rx Data FIFO
P:0x1621 0x40001884	I3C1_IBI_DATA_FIFO	–	(31:0) DATA	0x0	IBI Data receive FIFO
P:0x1622 0x40001888	I3C1_SLV_DDR_TX_FIFO	(19:0) DATA	–	N/A	Slave Tx DDR send FIFO
P:0x1623 0x4000188C	I3C1_SLV_DDR_RX_FIFO	–	(19:0) DATA	0x0	Slave Tx DDR receive FIFO
P:0x1624	I3C1_CMD_IBI_THR_CTRL	(29:24) IBIR_THR	(29:24) IBIR_THR	0x1	IBI response queue threshold

Address	Register Name	Register Write	Register Read	Default	Description
0x40001890					
		(20:16) CMDR_THR	(20:16) CMDR_THR	0x1	Command response queue threshold
		(13:8) IBID_THR	(13:8) IBID_THR	0x1	IBI data FIFO threshold
		(4:0) CMDD_THR	(4:0) CMDD_THR	0x1	Command descriptor memory threshold
P:0x1625 0x40001894	I3C1_TX_RX_THR_CTRL	(31:16) RX_THR	(31:16) RX_THR	0x1	Rx data memory threshold
		(15:0) TX_THR	(15:0) TX_THR	0x1	Tx data memory threshold
P:0x1626 0x40001898	I3C1_SLV_DDR_TX_RX_THR_CTRL	(31:16) SLV_DDR_RX_THR	(31:16) SLV_DDR_RX_THR	0x1	Slave Mode DDR Rx data memory threshold
		(15:0) SLV_DDR_TX_THR	(15:0) SLV_DDR_TX_THR	0x1	Slave Mode DDR Tx data memory threshold
P:0x1627 0x4000189C	I3C1_FLUSH_CTRL	(24) IBI_RESP	–	N/A	IBI response queue flush
		(23) CMD_RESP	–	N/A	Command response queue flush
		(22) SLV_DDR_RX	–	N/A	Slave DDR Rx FIFO flush
		(21) SLV_DDR_TX	–	N/A	Slave DDR Tx FIFO flush
		(20) IMM_CMD	–	N/A	Immediate command and data register flush
		(19) IBI	–	N/A	IBI received data FIFO flush
		(18) RX	–	N/A	Rx FIFO flush
		(17) TX	–	N/A	Tx FIFO flush
		(16) CMD	–	N/A	Command FIFO flush
P:0x162C	I3C1_TTO_PRESC1_CTRL0	(25:16) DIV_B	(25:16) DIV_B	0x0	Divider B

Address	Register Name	Register Write	Register Read	Default	Description
0x400018B0					
		(10:0) DIV_A	(10:0) DIV_A	0x0	Divider A
P:0x162D 0x400018B4	I3C1_TTO_PRESCL_CTRL1	(25:16) DIV_B	(25:16) DIV_B	0x0	Divider B
		(7:0) DIV_A	(7:0) DIV_A	0x0	Divider A
P:0x162E 0x400018B8	I3C1_DEVS_CTRL	(19) DEV3_CLEAR	–	N/A	Clear DevID3 retaining registers set. Self-cleared bit.
		(18) DEV2_CLEAR	–	N/A	Clear DevID2 retaining registers set. Self-cleared bit.
		(17) DEV1_CLEAR	–	N/A	Clear DevID1 retaining registers set. Self-cleared bit.
		(4) DEV4_CLEAR	(4) DEV4_CLEAR	0x0	DevID4 is active - has either valid DA or SA
		(3) DEV3_ACTIVE	(3) DEV3_ACTIVE	0x0	DevID3 is active - has either valid DA or SA
		(2) DEV2_ACTIVE	(2) DEV2_ACTIVE	0x0	DevID2 is active - has either valid DA or SA
		(1) DEV1_ACTIVE	(1) DEV1_ACTIVE	0x0	DevID1 is active - has either valid DA or SA
		(0) DEV0_ACTIVE	(0) DEV0_ACTIVE	0x1	DevID0 is active - has either valid DA or SA
P:0x1630 0x400018C0	I3C1_DEV_ID0_RR0	(15:13) LVR_SA_MSB	(15:13) LVR_SA_MSB	0x0	MSB bits of Legacy I2C device with 10-bit addressing
		(11) LVR_EXT_ADDR	(11) LVR_EXT_ADDR	0x0	Address mode
		–	(9) IS_I3C	0x1	I3C mode

Address	Register Name	Register Write	Register Read	Default	Description
		(7:0) DEV_ADDR	(7:0) DEV_ADDR	0x10	Slave Dynamic (Static/Legacy) Address
P:0x1631 0x400018C4	I3C1_DEV_ID0_RR1	(31:0) PID_MSB	(31:0) PID_MSB	0x25D0010	Provisional ID MSB 32-bits (48 to 16)
P:0x1632 0x400018C8	I3C1_DEV_ID0_RR2	(31:16) PID_LSB	(31:16) PID_LSB	0xC000	Provisional ID LSB 16-bits (15 to 0)
		(15:8) BCR	(15:8) BCR	0x62	BCR register
		(7:0) DCR_LVR	(7:0) DCR_LVR	0x5A	DCR/LVR register
P:0x1634 0x400018D0	I3C1_DEV_ID1_RR0	(15:13) LVR_SA_MSB	(15:13) LVR_SA_MSB	0x0	MSB bits of Legacy I2C Device with 10-bit addressing
		(11) LVR_EXT_ADDR	(11) LVR_EXT_ADDR	0x0	Address mode
		(9) IS_I3C	(9) IS_I3C	0x1	I3C mode
		(7:0) DEV_ADDR	(7:0) DEV_ADDR	0x13	Slave Dynamic (Static/Legacy) Address
P:0x1635 0x400018D4	I3C1_DEV_ID1_RR1	(31:0) PID_MSB	(31:0) PID_MSB	0x0	Provisional ID MSB 32-bits (48 to 16)
P:0x1636 0x400018D8	I3C1_DEV_ID1_RR2	(31:16) PID_LSB	(31:16) PID_LSB	0x0	Provisional ID MSB 32-bits (48 to 16)
		(15:8) BCR	(15:8) BCR	0x0	BCR register
		(7:0) DCR_LVR	(7:0) DCR_LVR	0x0	DCR/LVR register
P:0x1638 0x400018E0	I3C1_DEV_ID2_RR0	(15:13) LVR_SA_MSB	(15:13) LVR_SA_MSB	0x0	MSB bits of Legacy I2C Device with 10-bit addressing
		(11) LVR_EXT_ADDR	(11) LVR_EXT_ADDR	0x0	Address mode
		(9) IS_I3C	(9) IS_I3C	0x1	I3C mode

Address	Register Name	Register Write	Register Read	Default	Description
		(7:0) DEV_ADDR	(7:0) DEV_ADDR	0x15	Slave Dynamic (Static/Legacy) Address
P:0x1639 0x400018E4	I3C1_DEV_ID2_RR1	(31:0) PID_MSB	(31:0) PID_MSB	0x0	Provisional ID MSB 32-bits (48 to 16)
P:0x163A 0x400018E8	I3C1_DEV_ID2_RR2	(31:16) PID_LSB	(31:16) PID_LSB	0x0	Provisional ID LSB 16-bits (15 to 0)
		(15:8) BCR	(15:8) BCR	0x0	BCR register
		(7:0) DCR_LVR	(7:0) DCR_LVR	0x0	DCR/LVR register
P:0x163C 0x400018F0	I3C1_DEV_ID3_RR0	(15:13) LVR_SA_MSB	(15:13) LVR_SA_MSB	0x0	MSB bits of Legacy I2C Device with 10-bit addressing
		(11) LVR_EXT_ADDR	(11) LVR_EXT_ADDR	0x0	Address mode
		(9) IS_I3C	(9) IS_I3C	0x1	I3C mode
		(7:0) DEV_ADDR	(7:0) DEV_ADDR	0x16	Slave Dynamic (Static/Legacy) Address
P:0x163D 0x400018F4	I3C1_DEV_ID3_RR1	(31:0) PID_MSB	(31:0) PID_MSB	0x0	Provisional ID MSB 32-bits (48 to 16)
P:0x163E 0x400018F8	I3C1_DEV_ID3_RR2	(31:16) PID_LSB	(31:16) PID_LSB	0x0	Provisional ID MSB 32-bits (48 to 16)
		(15:8) BCR	(15:8) BCR	0x0	BCR register
		(7:0) DCR_LVR	(7:0) DCR_LVR	0x0	DCR/LVR register
P:0x1660 0x40001980	I3C1_SIR_MAP0	(31:30) DEVID1_ROLE	(31:30) DEVID1_ROLE	0x0	Device ID1 BCR role
		(29) DEVID1_SLOW	(29) DEVID1_SLOW	0x0	Device ID1 Maximum data speed limit
		(28:24) DEVID1_PL	(28:24) DEVID1_PL	0x0	Device ID1 Payload length

Address	Register Name	Register Write	Register Read	Default	Description
		(23:17) DEVID1_DA	(23:17) DEVID1_DA	0x7F	Device ID1 DA
		(16) DEVID1_RESP	(16) DEVID1_RESP	0x0	Device ID1 ACK/NACK response
		(15:14) DEVID0_ROLE	(15:14) DEVID0_ROLE	0x0	Device ID0 BCR role
		(13) DEVID0_SLOW	(13) DEVID0_SLOW	0x0	Device ID0 Maximum data speed limit
		(12:8) DEVID0_PL	(12:8) DEVID0_PL	0x0	Device ID0 Payload length
		(7:1) DEVID0_DA	(7:1) DEVID0_DA	0x7F	Device ID0 DA
		(0) DEVID0_RESP	(0) DEVID0_RESP	0x0	Device ID0 ACK/NACK response
P:0x1661 0x40001984	I3C1_SIR_MAP1	(15:14) DEVID2_ROLE	(15:14) DEVID2_ROLE	0x0	Device ID2 BCR role
		(13) DEVID2_SLOW	(13) DEVID2_SLOW	0x0	Device ID2 Maximum data speed limit
		(12:8) DEVID2_PL	(12:8) DEVID2_PL	0x0	Device ID2 Payload length
		(7:1) DEVID2_DA	(7:1) DEVID2_DA	0x7F	Device ID2 DA
		(0) DEVID2_RESP	(0) DEVID2_RESP	0x0	Device ID2 ACK/NACK response
P:0x1668 0x400019A0	I3C1_GPIR_WORD0	–	(7:0) GPIO	0x0	User Defined GPI Register 0
P:0x1688 0x40001A20	I3C1_GPOR_WORD0	–	(7:0) GPO0	0x0	User Defined GPO Register 0

A.36 EMMC INTERFACE CONFIGURATION CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1700 0x40001C00	EMMC_HRS00	–	(31:24) HRS00_ HWINIT1	0x0	HWINIT field
		–	(23:16) SAV	0x1	SAV - Slot Available
		–	(15:1) HRS00_HWINIT0	0x0	HWINIT field
		(0) SWR	(0) SWR	0x0	SWR - Software Reset
P:0x1701 0x40001C04	EMMC_HRS01	–	(31:24) HRS01_ HWINIT0	0x0	HWINIT field
		(23:0) DP	(23:0) DP	0x32	DP - Debounce Period: Defines the number of system (clk) clock cycles used by the debounce logic, which detects card insertion and removal events.
P:0x1704 0x40001C10	EMMC_HRS04	–	(26) UIS_ACK	0x0	UIS_ACK - PHY Transaction Acknowledge
		(25) UIS_RD	(25) UIS_RD	0x0	UIS_RD - PHY Read Transaction Request
		(24) UIS_WR	(24) UIS_WR	0x0	UIS_WR - PHY Write Transaction Request: This bit requests write transaction to the PHY register
		–	(23:16) UIS_RDATA	0x0	UIS_RDATA - PHY Read Data: This field stores data read from PHY register
		(15:8) UIS_WDATA	(15:8) UIS_WDATA	0x0	UIS_WDATA - PHY Write Data: This field stores data to be written to the PHY register

Address	Register Name	Register Write	Register Read	Default	Description
		(5:0) UIS_ADDR	(5:0) UIS_ADDR	0x0	UIS_ADDR - PHY Register Address: This field stores address of the PHY register which is to be read or written.
P:0x1706 0x40001C18	EMMC_HRS06	(2:0) EMM	(2:0) EMM	0x0	EMM - eMMC Mode select: This field sets eMMC mode. The mode should reflect the eMMC device setting. If the SD card is in use, this field needs to be 0000b.
P:0x1707 0x40001C1C	EMMC_HRS07	(20:16) ODELAY_VAL	(20:16) ODELAY_VAL	0x0	ODELAY_VAL - Output delay value for IO
		(4:0) IDELAY_VAL	(4:0) IDELAY_VAL	0x0	IDELAY_VAL - Input delay value for IO
P:0x171E 0x40001C78	EMMC_HRS30	-	(1) HS400ESSUP	0x0	High Speed 400 Enhance Strobe Mode supported
		-	(0) CQSUP	0x0	Command Queuing supported This field informs whether Command Queuing is supported (1) or is not supported (0)
P:0x171F 0x40001C7C	EMMC_HRS31	-	(27:16) HOSTCTRLVER	0x411	Host Controller Version Release number of the Host Controller
		-	(7:0) HOSTFIXVER	0x0	Fix Version Number: Number of the fix related to the Host Controller Version
P:0x1720 0x40001C80	EMMC_HRS32	(31) LOAD	(31) LOAD	0x0	LOAD - FSM monitor update request
		(30:16) ADDR	-	N/A	ADDR - FSM address: This field

Address	Register Name	Register Write	Register Read	Default	Description
					selects which FSM status will be read. All available status machines are listed in the Debug section of the User Guide.
		–	(15:0) DATA	0x0	DATA - FSM status: This register contains the Read FSM status. Before reading it, the user should select the FSM address (ADDR), set the LOAD bit, and wait until hardware clears it.
P:0x1721 0x40001C84	EMMC_HRS33	–	(31:0) STAT0	0x0	STAT0 - Tune status 0 After invoking the UHS-I tuning procedure, each bit of this register represents the status of one tuning step. This field corresponds to tuning steps 31-0. 0 - Step failed 1 - Step passed
P:0x1722 0x40001C88	EMMC_HRS34	–	(7:0) STAT1	0x0	STAT1 - Tune status 1 This field is continuation of STAT0 field. Its value represents status of tuning steps 39-32.
P:0x1723 0x40001C8C	EMMC_HRS35	(31) TFR	(31) TFR	0x0	TFR - Tune Force Request
		(21:16) TFV	(21:16) TFV	0x0	TFV - Tune Force Value This register contains a value which can update the Tune value, and in consequence, the sample point. The new Tune value becomes valid when the TFR transits from 1 to 0.
		–	(5:0) TVAL	0x0	TVAL - Tune Current Value: This field represents the selected value of

Address	Register Name	Register Write	Register Read	Default	Description
					UHS-I delay
P:0x1724 0x40001C90	EMMC_HRS36	–	(5) BOOT_EDE	0x0	Boot Error - End Bit error
		–	(4) BOOT_EDC	0x0	Boot Error - Data CRC error
		–	(3) BOOT_EDT	0x0	Boot Error - Data Timeout error
		–	(2) BOOT_EAI	0x0	Boot Error - Invalid Acknowledge error
		–	(1) BOOT_EAT	0x0	Boot Error - Acknowledge Timeout error
		–	(0) BOOT_ACT	0x0	Boot Active - The BOOT is active and the operation cannot be interfered with by writing to any registers
P:0x1725 0x40001C94	EMMC_HRS37	(5:0) RGB_COEFF_IFM	(5:0) RGB_COEFF_IFM	0x0	RGB_COEFF_IFM - Reads block gap coefficient to select interface speed mode
P:0x1726 0x40001C98	EMMC_HRS38	(3:0) RGB_COEFF	(3:0) RGB_COEFF	0x0	Read block gap coefficient
P:0x173F 0x40001CFC	EMMC_CRS63	–	(31:24) CRS63_HWINIT1	0x0	HWINIT field
		–	(23:16) SVN	0x3	SVN - Specification Version Number: This field identifies the Host Controller Specification Version.
		–	(15:8) CRS63_HWINIT0	0x0	HWINIT field
		–	(7:0) ISES	0x0	ISES - Interrupt Signal for Each Slot

Address	Register Name	Register Write	Register Read	Default	Description
P:0x1780 0x40001E00	EMMC_SRS00	(31:0) SAAR	(31:0) SAAR	0x0	Auto CMD23 Argument
P:0x1781 0x40001E04	EMMC_SRS01	(31:16) BCCT	(31:16) BCCT	0x0	Block Count For Current Transfer
		(11:0) TBS	(11:0) TBS	0x0	Transfer Block Size This field defines block sizes for block data transfers. During a data transfer, Read operations might return an invalid value, and Write operations are ignored.
P:0x1782 0x40001E08	EMMC_SRS02	(31:0) ARG1	(31:0) ARG1	0x0	Command Argument 1: This field contains a 32-bits argument for a command issued by SRS03.CIDX File Write.
P:0x1783 0x40001E0C	EMMC_SRS03	(29:24) CIDX	(29:24) CIDX	0x0	Command Index: This field contains a command number (index) of the command to be sent. The index can be defined in range 00-63, which means all commands (CMD00-CMD63 and ACMD00-ACMD63) defined in the related specifications are supported.
		(23:22) CT	(23:22) CT	0x0	Command Type: This field defines the specific type of a command.
		(21) DPS	(21) DPS	0x0	Data Present Select
		(20) CICE	(20) CICE	0x0	Command Index Check Enable: When set to 1, the host checks if the Command Index field in the response is equal to the

Address	Register Name	Register Write	Register Read	Default	Description
					SRS03.CIDX value. When set to 0, the check is not performed and the Command Index field of the response is ignored.
		(19) CRCCE	(19) CRCCE	0x0	Command CRC Check Enable
		(17:16) RTS	(17:16) RTS	0x0	Response Type Select: Defines the expected response length.
		(8) RID	(8) RID	0x0	Response Interrupt Disable
		(7) RECE	(7) RECE	0x0	Response Error Check Enable
		(6) RECT	(6) RECT	0x0	Response Type R1/R5: Selects R1 or R5 response type for the response content checker.
		(5) MSBS	(5) MSBS	0x0	Multi/Single Block data transfer select
		(4) DTDS	(4) DTDS	0x0	Data Transfer Direction Select Selects direction of data transfer for commands with DPS=1
		(1) BCE	(1) BCE	0x0	Block Count Enable
P:0x1784 0x40001E10	EMMC_SRS04	–	(31:0) RESP1	0x0	Response Register #1 (see short comment)
P:0x1785 0x40001E14	EMMC_SRS05	–	(31:0) RESP2	0x0	Response Register #2 (see RESP1 short comment)
P:0x1786 0x40001E18	EMMC_SRS06	–	(31:0) RESP3	0x0	Response Register #3 (see RESP1 short comment)
P:0x1787	EMMC_SRS07	–	(31:0) RESP4	0x0	Response Register #4 (see RESP1 short comment)

Address	Register Name	Register Write	Register Read	Default	Description
0x40001E1C					
P:0x1788 0x40001E20	EMMC_SRS08	(31:0) BDP	(31:0) BDP	0x0	Buffer Data Port The field is used to access the internal buffer (data block) in Non-DMA Transfer Mode.
P:0x1789 0x40001E24	EMMC_SRS09	–	(24) CMDSL	0x0	CMD Line Signal Level (SD Mode only)
		–	(23:20) DATSL1	0x0	DAT[3:0] Line Signal Level (SD Mode only)
		–	(19) WPSL	0x0	Write Protect Switch Pin Level The value is equal to the actual signal level on Write Protect pin of the SD interface (sdwp_s#_n)
		–	(18) CDSL	0x0	Card Detect Pin Level This value is equal to the inverted signal level on the Card Detect pin of the SD interface (sdsl_s#_n)
		–	(17) CSS	0x0	Card State Stable Indicates if Card Detect Pin Level (CDSL) is stable
		–	(16) CI	0x0	Card Inserted Indicates if the card is inserted in the slot
		–	(11) BRE	0x0	Buffer Read Enable This field represents the data buffer (SRS08.BDP) state for Read transfer in Non-DMA Mode.
		–	(10) BWE	0x0	Buffer Write Enable This bit represents the data buffer (SRS08.BDP) state for Write transfer in Non-DMA Mode.
		–	(9) RTA	0x0	Read Transfer Active (SD Mode

Address	Register Name	Register Write	Register Read	Default	Description
					only) Indicates the status of the Read data transfer
		–	(8) WTA	0x0	Write Transfer Active (SD Mode only) Indicates the status of the Write data transfer.
		–	(7:4) DATSL2	0x0	DAT[7:4] Line Signal Level (SD Mode only)
		–	(2) DLA	0x0	DAT Line Active (SD Mode only) Indicates if the DAT lines of the SD interface are currently in use
		–	(1) CIDAT	0x0	Command Inhibit DAT (SD Mode only): Indicates if the host can issue a command which uses the DAT line
		–	(0) CICMD	0x0	Command Inhibit CMD: Indicates if the host can issue a command
P:0x178A 0x40001E28	EMMC_SRS10	(26) WORM	(26) WORM	0x0	Wakeup Event Enable On SD Card Removal
		(25) WOIS	(25) WOIS	0x0	Wake-Up Event Enable On Card Inserted
		(24) WOIQ	(24) WOIQ	0x0	Wakeup Event Enable On Card Interrupt
		(19) IBG	(19) IBG	0x0	Interrupt At Block Gap (SD Mode only)
		(18) RWC	(18) RWC	0x0	Read-Wait Control (SD Mode only)
		(17) CREQ	(17) CREQ	0x0	Continue Request
		(16) SBGR	(16) SBGR	0x0	Stop At Block Gap Request
		(15:13) BVS2	(15:13) BVS2	0x0	SD Bus Voltage Select for VDD2

Address	Register Name	Register Write	Register Read	Default	Description
		(12) BP2	(12) BP2	0x0	SD Bus Power for VDD2
		(11:9) BVS	(11:9) BVS	0x0	SD Bus Voltage Select
		(8) BP	(8) BP	0x0	SD Bus Power for VDD1
		(7) CDSS	(7) CDSS	0x0	Card Detect Signal Selection The card detection mechanism is based on either s#_sdcd_n port or register value.
		(6) CDTL	(6) CDTL	0x0	Card Detect Test Level Designates card insertion status when SRS10.CDSS=1. Bit provided for test purposes.
		(5) EDTW	(5) EDTW	0x0	Extended Data Transfer Width This bit is to enable/disable 8-bit DAT bus width mode
		(2) HSE	(2) HSE	0x0	High Speed Enable (SD mode only)
		(1) DTW	(1) DTW	0x0	Data Transfer Width (SD mode only)
		(0) LEDC	(0) LEDC	0x0	LED Control State of this bit directly drives LED port of the host in order to control the external LED diode
P:0x178B 0x40001E2C	EMMC_SRS11	(26) SRDAT	(26) SRDAT	0x0	Software Reset For DAT Line (SD Mode only)
		(25) SRCMD	(25) SRCMD	0x0	Software Reset For CMD Line (SD Mode only)
		(24) SRFA	(24) SRFA	0x0	Software Reset For All
		(19:16) DTCV	(19:16) DTCV	0x0	Data Timeout Counter Value: the interval by which DAT line timeouts are detected

Address	Register Name	Register Write	Register Read	Default	Description
		(15:8) SDCFSL	(15:8) SDCFSL	0x0	SDCLK Frequency Select (lower part): This register and SRS11.SDCFSH are used to calculate the frequency of the SDCLK clock.
		(7:6) SDCFSH	(7:6) SDCFSH	0x0	SDCFSH - SDCLK Frequency Select (higher part): This register is an extension to SDCFSL.
		(2) SDCE	(2) SDCE	0x0	SD Clock Enable
		–	(1) ICS	0x0	Internal Clock Stable
		(0) ICE	(0) ICE	0x0	Internal Clock Enable: This field is designated to control (enable/disable) the external clock generator (e.g. PLL).
P:0x178C 0x40001E30	EMMC_SRS12	(27) ERSP	(27) ERSP	0x0	Response Error (SD Mode only) Generated on error detection in R1 or R5 response
		(24) EAC	(24) EAC	0x0	Auto CMD Error (SD mode only) Generated when an error occurs during Auto CMD12/Auto CMD23 command transmission
		(23) ECL	(23) ECL	0x0	Current Limit Error
		(22) EDEB	(22) EDEB	0x0	Data End Bit Error (SD mode only)
		(21) EDCRC	(21) EDCRC	0x0	Data CRC Error (SD mode only)
		(20) EDT	(20) EDT	0x0	Data Timeout Error (SD mode only)
		(19) ECI	(19) ECI	0x0	Command Index Error (SD mode only)
		(18) ECEB	(18) ECEB	0x0	Command End Bit Error (SD mode only)

Address	Register Name	Register Write	Register Read	Default	Description
		(17) ECCRC	(17) ECCRC	0x0	Command CRC Error (SD mode only)
		(16) ECT	(16) ECT	0x0	Command Timeout Error (SD mode only)
		–	(15) EINT	0x0	Error Interrupt The software can check for an error by reading this single bit first
		–	(8) CINT	0x0	Card Interrupt Indicates the card interrupt.
		(7) CR	(7) CR	0x0	Card Removal Generated when the SRS09.CI bit changes from 1 to 0, indicating a card removal event.
		(6) CIN	(6) CIN	0x0	Card Insertion Generated when the SRS09.CI bit changes from 0 to 1, indicating card insertion
		(5) BRR	(5) BRR	0x0	Buffer Read Ready
		(4) BWR	(4) BWR	0x0	Buffer Write Ready
		(2) BGE	(2) BGE	0x0	Block Gap Event Generated when the read/write transaction is stopped at a block gap as the result of setting SRS10.SBGR to 1.
		(1) TC	(1) TC	0x0	Transfer Complete
		(0) CC	(0) CC	0x0	CC - Command Complete Generated when the end bit of the response is received, except the response for an Auto-CMD12 command; this type of command does not generate CC.
P:0x178D	EMMC_SRS13	(27) ERSP_SE	(27) ERSP_SE	0x0	Response Error Status Enable (SD

Address	Register Name	Register Write	Register Read	Default	Description
0x40001E34					mode only)
		(24) EAC_SE	(24) EAC_SE	0x0	Auto CMD Error Status Enable (SD mode only)
		(23) ECL_SE	(23) ECL_SE	0x0	Current Limit Error Status Enable (SD mode only)
		(22) EDEB_SE	(22) EDEB_SE	0x0	Data End Bit Error Status Enable (SD mode only)
		(21) EDCRC_SE	(21) EDCRC_SE	0x0	Data CRC Error Status Enable (SD mode only)
		(20) EDT_SE	(20) EDT_SE	0x0	Data Timeout Error Status Enable (SD mode only)
		(19) ECI_SE	(19) ECI_SE	0x0	Command Index Error Status Enable (SD mode only)
		(18) ECEB_SE	(18) ECEB_SE	0x0	Command End Bit Error Status Enable (SD mode only)
		(17) ECCRC_SE	(17) ECCRC_SE	0x0	Command CRC Error Status Enable (SD mode only)
		(16) ECT_SE	(16) ECT_SE	0x0	Command Timeout Error Status Enable (SD mode only)
		(8) CINT_SE	(8) CINT_SE	0x0	Card Interrupt Status Enable
		(7) CR_SE	(7) CR_SE	0x0	Card Removal Status Enable
		(6) CIN_SE	(6) CIN_SE	0x0	Card Insertion Status Enable
		(5) BRR_SE	(5) BRR_SE	0x0	Buffer Read Ready Status Enable
		(4) BWR_SE	(4) BWR_SE	0x0	Buffer Write Ready Status Enable
		(2) BGE_SE	(2) BGE_SE	0x0	Block Gap Event Status Enable
		(1) TC_SE	(1) TC_SE	0x0	Transfer Complete Status Enable

Address	Register Name	Register Write	Register Read	Default	Description
		(0) CC_SE	(0) CC_SE	0x0	Command Complete Status Enable
P:0x178E 0x40001E38	EMMC_SRS14	(27) ERSP_IE	(27) ERSP_IE	0x0	Response Error Interrupt Enable
		(24) EAC_IE	(24) EAC_IE	0x0	Auto CMD Error Interrupt Enable (SD mode only)
		(23) ECL_IE	(23) ECL_IE	0x0	Current Limit Error Interrupt Enable (SD mode only)
		(22) EDEB_IE	(22) EDEB_IE	0x0	Data End Bit Error Interrupt Enable (SD mode only)
		(21) EDCRC_IE	(21) EDCRC_IE	0x0	Data CRC Error Interrupt Enable (SD mode only)
		(20) EDT_IE	(20) EDT_IE	0x0	Data Timeout Error Interrupt Enable (SD mode only)
		(19) ECI_IE	(19) ECI_IE	0x0	Command Index Error Interrupt Enable (SD mode only)
		(18) ECEB_IE	(18) ECEB_IE	0x0	Command End Bit Error Interrupt Enable (SD mode only)
		(17) ECCRC_IE	(17) ECCRC_IE	0x0	Command CRC Error Interrupt Enable (SD mode only)
		(16) ECT_IE	(16) ECT_IE	0x0	Command Timeout Error Interrupt Enable (SD mode only)
		(8) CINT_IE	(8) CINT_IE	0x0	Card Interrupt - Interrupt Enable
		(7) CR_IE	(7) CR_IE	0x0	Card Removal Interrupt Enable
		(6) CIN_IE	(6) CIN_IE	0x0	Card Insertion Interrupt Enable
		(5) BRR_IE	(5) BRR_IE	0x0	Buffer Read Ready Interrupt Enable
		(4) BWR_IE	(4) BWR_IE	0x0	Buffer Write Ready Interrupt Enable

Address	Register Name	Register Write	Register Read	Default	Description
		(2) BGE_IE	(2) BGE_IE	0x0	Block Gap Event Interrupt Enable
		(1) TC_IE	(1) TC_IE	0x0	Transfer Complete Interrupt Enable
		(0) CC_IE	(0) CC_IE	0x0	Command Complete Interrupt Enable
P:0x178F 0x40001E3C	EMMC_SRS15	(31) PVE	(31) PVE	0x0	Preset Value Enable
		(28) HV4E	(28) HV4E	0x0	Host Version 4.00 Enable: Selects backward (SD Host 3.00 Version) compatibility mode or SD Host 4.00 Version mode.
		(23) SCS	(23) SCS	0x0	Sampling Clock Select (UHS-I only) The host updates this bit when the tuning procedure is finished
		(22) EXTNG	(22) EXTNG	0x0	Execute Tuning (UHS-I only) This register controls the tuning procedure.
		(21:20) DSS	(21:20) DSS	0x0	Driver Strength Select (UHS-I only) This bit controls the electric parameters of the I/O driver. Up to 4 configurations of I/O driver settings can be implemented.
		(19) V18SE	(19) V18SE	0x0	1.8 V Signaling Enable (UHS-I only) This bit controls the I/O signaling voltage level. The SW driver will set this bit to 1 when in UHS-I Mode.
		-	(7) CNIACE	0x0	Command Not Issued By Auto CMD12 Error
		-	(5) ACRE	0x0	Auto CMD Response Error

Address	Register Name	Register Write	Register Read	Default	Description
		–	(4) ACIE	0x0	Auto CMD Index Error
		–	(3) ACEBE	0x0	Auto CMD End Bit Error
		–	(2) ACCE	0x0	Auto CMD CRC Error
		–	(1) ACTE	0x0	Auto CMD Timeout Error
		–	(0) ACNE	0x0	Auto CMD12 Not Executed: When set to 1, means the host cannot issue Auto CMD12 due to some error.
P:0x1790 0x40001E40	EMMC_SRS16	–	(31:30) SLT	0x1	Type of Slot Provided
		–	(29) AIS	0x0	Asynchronous Interrupt Support
		–	(28) A64S	0x1	64-bit System Addressing Support
		–	(27) SRS16_HWINIT1	0x0	HWINIT field
		–	(26) VS18	0x1	Voltage Support 1.8 V
		–	(25) VS30	0x0	Voltage Support 3.0 V
		–	(24) VS33	0x0	Voltage Support 3.3 V
		–	(23) SRS	0x0	Suspend / Resume Support NOTE: The host controller does not support Suspend / Resume mechanism.
		–	(21) HSS	0x1	High Speed Support
		–	(18) EDS8	0x0	8-bit Embedded Device Support If this bit is 0, the SRS10.EDTW register is not implemented.
		–	(17:16) SRS16_MBL	0x2	Max Block Length This value indicates the maximum block size that can be transferred by the host

Address	Register Name	Register Write	Register Read	Default	Description
		–	(15:8) BCSDCLK	0x0	Base Clock Frequency for SD Clock Field defines the base clock frequency for the SD Clock in 1MHz units. If BCSDCLK = 0, the Host System has to obtain the clock information via another method (i.e. not defined by the specification).
		–	(7) TCU	0x1	Timeout Clock Unit Field defines the frequency unit for the SRS16.TCF
		–	(6) SRS16_HWINIT0	0x0	HWINIT field
		–	(5:0) TCF	0x0	Timeout Clock Frequency Defines the base clock frequency used to detect Data Timeout errors. If TCF = 0, the Host System has to obtain the clock information via another method (i.e. not defined by the specification).
P:0x1791 0x40001E44	EMMC_SRS17	–	(31:29) SRS17_ HWINIT3	0x0	HWINIT field
		–	(28) VDD2S	0x0	VDD2 Supported Information about support or lack of support of optional VDD2
		–	(27:24) SRS17_ HWINIT2	0x0	HWINIT field
		–	(23:16) CLKMPR	0x0	Clock Multiplier: This field is to be 0 (fixed), as the Clock Multiplier is not supported.
		–	(15:14) RTNGM	0x0	Re-Tuning Modes
		–	(13) UTSM50	0x0	Use Tuning for SDR50

Address	Register Name	Register Write	Register Read	Default	Description
		–	(12) SRS17_HWINIT1	0x0	HWINIT field
		–	(7) SRS17_HWINIT0	0x0	HWINIT field
		–	(6) DRVD	0x1	1.8 V Line Driver Type D Support
		–	(5) DRVC	0x1	1.8 V Line Driver Type C Support
		–	(4) DRVA	0x1	1.8 V Line Driver Type A Support
		–	(3) UHSII	0x0	UHS-II Support
		–	(2) DDR50	0x0	DDR50 Support
		–	(1) SDR104	0x0	SDR104 Support
		–	(0) SDR50	0x0	SDR50 - SDR50 Supported: 1 - SDR50 mode supported; 0 - SDR50 mode not supported
P:0x1792 0x40001E48	EMMC_SRS18	–	(31:24) SRS18_HWINIT0	0x0	HWINIT field
		–	(23:16) MC18	0x20	Maximum Current for 1.8 V
		–	(15:8) MC30	0x20	Maximum Current for 3.0 V
		–	(7:0) MC33	0x20	Maximum Current for 3.3 V
P:0x1793 0x40001E4C	EMMC_SRS19	–	(31:8) SRS19_HWINIT0	0x0	HWINIT field
		–	(7:0) MC18V2	0x20	Maximum Current for 1.8V VDD2
P:0x1794 0x40001E50	EMMC_SRS20	(27) ERESP_FE	–	N/A	Force Response Error Event
		(26) ETUNE_FE	–	N/A	Force Tuning Error Event
		(24) EAC_FE	–	N/A	Force Auto CMD Error Event

Address	Register Name	Register Write	Register Read	Default	Description
		(23) ECL_FE	–	N/A	Force Current Limit Error Event
		(22) EDEB_FE	–	N/A	Force Data End Bit Error Event
		(21) EDCRC_FE	–	N/A	Force Data CRC Error Event
		(20) EDT_FE	–	N/A	Force Data Timeout Error Event
		(19) ECI_FE	–	N/A	Force Command Index Error Event
		(18) ECEB_FE	–	N/A	Force Command End Bit Error Event
		(17) ECCRC_FE	–	N/A	Force Command CRC Error Event
		(16) ECT_FE	–	N/A	Force Command Timeout Error Event
		(7) CNIACE_FE	–	N/A	Force Command Not Issued By Auto CMD12 Error Event
		(4) ACIE_FE	–	N/A	Force Auto CMD Index Error Event
		(3) ACEBE_FE	–	N/A	Force Auto CMD End Bit Error Event
		(2) ACCE_FE	–	N/A	Force Auto CMD CRC Error Event
		(1) ACTE_FE	–	N/A	Force Auto CMD Timeout Error Event
		(0) ACNE_FE	–	N/A	Force Auto CMD12 Not Executed Error Event
P:0x1798 0x40001E60	EMMC_SRS24	–	(31:30) SRS24_DSSPV_ 31_30	0x0	DSSPV## - Driver Strength Select - Preset Value: This field can be used by the software to update SRS15.DSS
		–	(29:26) SRS24_ HWINIT1	0x0	HWINIT field

Address	Register Name	Register Write	Register Read	Default	Description
		–	(25:16) SRS24_ SDCFSPV_25_16	0x4	SDCFSPV## - SDCLK Clock Frequency Select - Preset Value: This field can be used by the software to update SRS11.SDCFSH and SRS11.SDCFSL
		–	(15:0) SRS24_HWINIT0	0xFA	HWINIT field
P:0x1799 0x40001E64	EMMC_SRS25	–	(31:30) SRS25_DSSPV_ 31_30	0x0	DSSPV## - Driver Strength Select - Preset Value: This field can be used by the software to update SRS15.DSS
		–	(29:26) SRS25_ HWINIT1	0x0	HWINIT field
		–	(25:16) SRS25_ SDCFSPV_25_16	0x4	SDCFSPV## - SDCLK Clock Frequency Select - Preset Value: This field can be used by the software to update SRS11.SDCFSH and SRS11.SDCFSL
		–	(15:14) SRS25_DSSPV_ 15_14	0x0	DSSPV## - Driver Strength Select - Preset Value: This field can be used by the software to update SRS15.DSS.
		–	(13:10) SRS25_ HWINIT0	0x0	HWINIT field
		–	(9:0) SRS25_SDCFSPV_ 09_00	0x2	SDCFSPV## - SDCLK Clock Frequency Select - Preset Value: This field can be used by the software to update SRS11.SDCFSH and SRS11.SDCFSL

Address	Register Name	Register Write	Register Read	Default	Description
P:0x179A 0x40001E68	EMMC_SRS26	–	(31:30) SRS26_DSSPV_ 31_30	0x0	DSSPV## - Driver Strength Select - Preset Value: This field can be used by the software to update SRS15.DSS.
		–	(29:26) SRS26_ HWINIT1	0x0	HWINIT field
		–	(25:16) SRS26_ SDCFSPV_25_16	0x1	SDCFSPV## - SDCLK Clock Frequency Select - Preset Value: This field can be used by the software to update SRS11.SDCFSH and SRS11.SDCFSL.
		–	(15:14) SRS26_DSSPV_ 15_14	0x0	DSSPV## - Driver Strength Select - Preset Value: This field can be used by the software to update SRS15.DSS.
		–	(13:11) SRS26_ HWINIT0	0x0	HWINIT field
		–	(10) SRS26_CGSPV_10	0x0	CGSPV## - Clock Generator Select - Preset Value: This field can be used by the software to update SRS11.CGS.
		–	(9:0) SRS26_SDCFSPV_ 09_00	0x2	SDCFSPV## - SDCLK Clock Frequency Select - Preset Value: This field can be used by the software to update SRS11.SDCFSH and SRS11.SDCFSL.
P:0x179B 0x40001E6C	EMMC_SRS27	–	(31:30) SRS27_DSSPV_ 31_30	0x0	DSSPV## - Driver Strength Select - Preset Value: This field can be used by the software to update

Address	Register Name	Register Write	Register Read	Default	Description
					SRS15.DSS.
		–	(29:26) SRS27_ HWINIT1	0x0	HWINIT field
		–	(25:16) SRS27_ SDCFSPV_25_16	0x2	SDCFSPV## - SDCLK Clock Frequency Select - Preset Value: This field can be used by the software to update SRS11.SDCFSH and SRS11.SDCFSL.
		–	(15:14) SRS27_DSSPV_ 15_14	0x0	DSSPV## - Driver Strength Select - Preset Value: This field can be used by the software to update SRS15.DSS.
		–	(13:10) SRS27_ HWINIT0	0x0	HWINIT field
		–	(9:0) SRS27_SDCFSPPV_ 09_00	0x0	SDCFSPV## - SDCLK Clock Frequency Select - Preset Value: This field can be used by the software to update SRS11.SDCFSH and SRS11.SDCFSL.
P:0x179D 0x40001E74	EMMC_SRS29	–	(31:16) SRS29_ HWINIT1	0x0	HWINIT field

Address	Register Name	Register Write	Register Read	Default	Description
		–	(15:14) SRS29_DSSPV_ 15_14	0x0	DSSPV## - Driver Strength Select - Preset Value: This field can be used by the software to update SRS15.DSS.
		–	(13:10) SRS29_ HWINIT0	0x0	HWINIT field
		–	(9:0) SRS29_SDCFSPV_ 09_00	0x0	SDCFSPV## - SDCLK Clock Frequency Select - Preset Value: This field can be used by the software to update SRS11.SDCFSH and SRS11.SDCFSL.

A.37 NEURAL NETWORK ACCELERATOR

Address	Register Name	Register Write	Register Read	Default	Description
P:0x17C0 0x40001F00	NNA_CFG	(25:23) RELU_CFG	(25:23) RELU_CFG	0x0	ReLu configuration
		(22:21) ACTIVATION_ FUNCTION	(22:21) ACTIVATION_ FUNCTION	0x0	Activation function
		(20) OUTPUT_ROUNDING	(20) OUTPUT_ ROUNDING	0x0	Rounding
		(19:15) OUTPUT_SHIFT	(19:15) OUTPUT_ SHIFT	0x0	Output shift
		(14:13) OUTPUT_BITS	(14:13) OUTPUT_BITS	0x0	Output word size
		(12:8) BIAS_SHIFT	(12:8) BIAS_SHIFT	0x0	Bias shift
		(7:6) BIAS_MODE	(7:6) BIAS_MODE	0x0	Bias mode
		(4) WEIGHT_ COMPRESSION	(4) WEIGHT_ COMPRESSION	0x0	Weight compression
		(3:2) WEIGHT_BITS	(3:2) WEIGHT_BITS	0x0	Weight word size
		(1) INPUT_BITS	(1) INPUT_BITS	0x0	Input word size
		(0) MULT_MODE	(0) MULT_MODE	0x0	Fixed point or integer multiplier mode
P:0x17C1 0x40001F04	NNA_CTRL	–	(13) ACCESS_ERROR	0x0	Peripheral bus or DSP access error
		–	(12) DSP_ACCESS_ ERROR	0x0	DSP access error
		–	(11) PBUS_ACCESS_ ERROR	0x0	Peripheral bus access error
		–	(10) SATURATION2_ FLAG	0x0	NNA saturation 2 flag

Address	Register Name	Register Write	Register Read	Default	Description
		–	(9) SATURATION_FLAG	0x0	NNA saturation flag
		–	(8) BUSY	0x0	The NNA processing state
		(4) RESET_DSP_ACCESS_ERROR	–	N/A	Write a 1 to reset DSP access error
		(3) RESET_PBUS_ACCESS_ERROR	–	N/A	Write a 1 to reset peripheral bus access error
		(2) RESET_SATURATION_FLAGS	–	N/A	Write a 1 to reset saturation flags
		(1) RESET	–	N/A	Write a 1 to reset NNA
		(0) START	–	N/A	Write a 1 to start the NNA
P:0x17C2 0x40001F08	NNA_INPUT_CNT	(9:0) INPUT_CNT	(9:0) INPUT_CNT	0x0	NNA input count (number of elements in input vector)
P:0x17C3 0x40001F0C	NNA_OUTPUT_CNT	(9:0) OUTPUT_CNT	(9:0) OUTPUT_CNT	0x0	NNA output count (number of output points to be calculated)
P:0x17C4 0x40001F10	NNA_CIRC_BASE	(18:0) CIRC_BASE	(18:0) CIRC_BASE	0x0	NNA circular buffer base address (byte address in LPDSP32 DM-A)
P:0x17C5 0x40001F14	NNA_CIRC_SIZE	(9:0) CIRC_SIZE	(9:0) CIRC_SIZE	0x0	NNA circular buffer size (in input size elements)
P:0x17C6 0x40001F18	NNA_INPUT_PTR	(18:0) INPUT_PTR	(18:0) INPUT_PTR	0x0	NNA input pointer (byte address in LPDSP32 DM-A)
P:0x17C7 0x40001F1C	NNA_WEIGHT_PTR	(18:0) WEIGHT_PTR	(18:0) WEIGHT_PTR	0x0	NNA weight pointer (byte address in LPDSP32 DM-A)
P:0x17C8	NNA_BIAS_PTR	(18:0) BIAS_PTR	(18:0) BIAS_PTR	0x0	NNA bias pointer (byte address in

Address	Register Name	Register Write	Register Read	Default	Description
0x40001F20					LPDSP32 DM-A)
P:0x17C9 0x40001F24	NNA_OUTPUT_PTR	(18:0) OUTPUT_PTR	(18:0) OUTPUT_PTR	0x0	NNA output pointer (byte address in LPDSP32 DM-A)
P:0x17CA 0x40001F28	NNA_WEIGHT_LUT_0_3	(31:24) LUT_ENTRY_3	(31:24) LUT_ENTRY_3	0x30	Weight Lookup Table Entry for value 3
		(23:16) LUT_ENTRY_2	(23:16) LUT_ENTRY_2	0x20	Weight Lookup Table Entry for value 2
		(15:8) LUT_ENTRY_1	(15:8) LUT_ENTRY_1	0x10	Weight Lookup Table Entry for value 1
		(7:0) LUT_ENTRY_0	(7:0) LUT_ENTRY_0	0x0	Weight Lookup Table Entry for value 0
P:0x17CB 0x40001F2C	NNA_WEIGHT_LUT_4_7	(31:24) LUT_ENTRY_7	(31:24) LUT_ENTRY_7	0x70	Weight Lookup Table Entry for value 7
		(23:16) LUT_ENTRY_6	(23:16) LUT_ENTRY_6	0x60	Weight Lookup Table Entry for value 6
		(15:8) LUT_ENTRY_5	(15:8) LUT_ENTRY_5	0x50	Weight Lookup Table Entry for value 5
		(7:0) LUT_ENTRY_4	(7:0) LUT_ENTRY_4	0x40	Weight Lookup Table Entry for value 4
P:0x17CC 0x40001F30	NNA_WEIGHT_LUT_8_11	(31:24) LUT_ENTRY_11	(31:24) LUT_ENTRY_11	0xB0	Weight Lookup Table Entry for value 11
		(23:16) LUT_ENTRY_10	(23:16) LUT_ENTRY_10	0xA0	Weight Lookup Table Entry for value 10
		(15:8) LUT_ENTRY_9	(15:8) LUT_ENTRY_9	0x90	Weight Lookup Table Entry for value 9
		(7:0) LUT_ENTRY_8	(7:0) LUT_ENTRY_8	0x80	Weight Lookup Table Entry for value 8
P:0x17CD 0x40001F34	NNA_WEIGHT_LUT_12_15	(31:24) LUT_ENTRY_15	(31:24) LUT_ENTRY_15	0xF0	Weight Lookup Table Entry for value 15

Address	Register Name	Register Write	Register Read	Default	Description
		(23:16) LUT_ENTRY_14	(23:16) LUT_ENTRY_14	0xE0	Weight Lookup Table Entry for value 14
		(15:8) LUT_ENTRY_13	(15:8) LUT_ENTRY_13	0xD0	Weight Lookup Table Entry for value 13
		(7:0) LUT_ENTRY_12	(7:0) LUT_ENTRY_12	0xC0	Weight Lookup Table Entry for value 12

A.38 SYSTICK TIMER

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E010	SysTick_CTRL	–	(16) COUNTFLAG	0x0	Reads as 1 if SYSTICK counter has reached 0 since the last time the timer has reached 0. Clears to 0 on read
		(2) CLKSOURCE	(2) CLKSOURCE	0x0	SYSTICK timer clock source
		(1) TICKINT	(1) TICKINT	0x0	SYSTICK timer interrupt enable
		(0) ENABLE	(0) ENABLE	0x0	SYSTICK timer enable
0xE000E014	SysTick_LOAD	(23:0) RELOAD	(23:0) RELOAD	0x0	Counter reload value for the SYSTICK timer when it reaches 0
0xE000E018	SysTick_VAL	(23:0) CURRENT	(23:0) CURRENT	0x0	Current value of the SYSTICK counter value. Write to clear counter
0xE000E01C	SysTick_CALIB	–	(31) NOREF	0x0	Indicates if a reference clock is available
		–	(30) SKEW	0x0	Indicates if calibration value is exactly 10 ms or not
		–	(23:0) TENMS	0x190	SYSTICK counter calibration value for 10 ms. A value of 0 means the calibration value is not available

A.39 SYSTEM CONTROL AND ID REGISTER NOT IN THE SCB

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E004	SCnSCB_ICTR	–	(4:0) INTLINESNUM	0x2	Number of interrupt inputs in steps of 32

A.40 NESTED VECTOR INTERRUPT CONTROLLER

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E100	NVIC_ISER0	(31) SPI_0_TX	(31) SPI_0_TX	0x0	SPI_0_TX interrupt set enable
		(30) SPI_0_RX	(30) SPI_0_RX	0x0	SPI_0_RX interrupt set enable
		(29) SPI_0_COM	(29) SPI_0_COM	0x0	SPI_0_COM interrupt set enable
		(28) D_I2C	(28) D_I2C	0x0	D_I2C interrupt set enable
		(27) I2C_1	(27) I2C_1	0x0	I2C_1 interrupt set enable
		(26) I2C_0	(26) I2C_0	0x0	I2C_0 interrupt set enable
		(25) I3C_1	(25) I3C_1	0x0	I3C_1 interrupt set enable
		(24) I3C_0	(24) I3C_0	0x0	I3C_0 interrupt set enable
		(23) DIO_7	(23) DIO_7	0x0	DIO_7 interrupt set enable
		(22) DIO_6	(22) DIO_6	0x0	DIO_6 interrupt set enable
		(21) DIO_5	(21) DIO_5	0x0	DIO_5 interrupt set enable
		(20) DIO_4	(20) DIO_4	0x0	DIO_4 interrupt set enable
		(19) DIO_3	(19) DIO_3	0x0	DIO_3 interrupt set enable
		(18) DIO_2	(18) DIO_2	0x0	DIO_2 interrupt set enable
		(17) DIO_1	(17) DIO_1	0x0	DIO_1 interrupt set enable
		(16) DIO_0	(16) DIO_0	0x0	DIO_0 interrupt set enable
		(15) DMA_7	(15) DMA_7	0x0	DMA_7 interrupt set enable
		(14) DMA_6	(14) DMA_6	0x0	DMA_6 interrupt set enable
		(13) DMA_5	(13) DMA_5	0x0	DMA_5 interrupt set enable
		(12) DMA_4	(12) DMA_4	0x0	DMA_4 interrupt set enable
		(11) DMA_3	(11) DMA_3	0x0	DMA_3 interrupt set enable

Address	Register Name	Register Write	Register Read	Default	Description
		(10) DMA_2	(10) DMA_2	0x0	DMA_2 interrupt set enable
		(9) DMA_1	(9) DMA_1	0x0	DMA_1 interrupt set enable
		(8) DMA_0	(8) DMA_0	0x0	DMA_0 interrupt set enable
		(7) TIMER_3	(7) TIMER_3	0x0	TIMER_3 interrupt set enable
		(6) TIMER_2	(6) TIMER_2	0x0	TIMER_2 interrupt set enable
		(5) TIMER_1	(5) TIMER_1	0x0	TIMER_1 interrupt set enable
		(4) TIMER_0	(4) TIMER_0	0x0	TIMER_0 interrupt set enable
		(3) AUDIOSINK_1	(3) AUDIOSINK_1	0x0	AUDIOSINK_1 interrupt set enable
		(2) AUDIOSINK_0	(2) AUDIOSINK_0	0x0	AUDIOSINK_0 interrupt set enable
		(1) LSAD_BATMON	(1) LSAD_BATMON	0x0	LSAD_BATMON interrupt set enable
		(0) WATCHDOG	(0) WATCHDOG	0x0	WATCHDOG interrupt set enable
0xE000E104	NVIC_IUSER1	(31) FIFO_7	(31) FIFO_7	0x0	FIFO_7 interrupt set enable
		(30) FIFO_6	(30) FIFO_6	0x0	FIFO_6 interrupt set enable
		(29) FIFO_5	(29) FIFO_5	0x0	FIFO_5 interrupt set enable
		(28) FIFO_4	(28) FIFO_4	0x0	FIFO_4 interrupt set enable
		(27) FIFO_3	(27) FIFO_3	0x0	FIFO_3 interrupt set enable
		(26) FIFO_2	(26) FIFO_2	0x0	FIFO_2 interrupt set enable
		(25) FIFO_1	(25) FIFO_1	0x0	FIFO_1 interrupt set enable
		(24) FIFO_0	(24) FIFO_0	0x0	FIFO_0 interrupt set enable
		(23) RS_DEC_OUT_RDY	(23) RS_DEC_OUT_RDY	0x0	RS_DEC_OUT_RDY interrupt set enable
		(22) RS_DEC_IN_RDY	(22) RS_DEC_IN_RDY	0x0	RS_DEC_IN_RDY interrupt set enable
		(21) RS_ENC_OUT_RDY	(21) RS_ENC_OUT_RDY	0x0	RS_ENC_OUT_RDY interrupt set enable
		(20) RS_ENC_IN_RDY	(20) RS_ENC_IN_RDY	0x0	RS_ENC_IN_RDY interrupt set enable

Address	Register Name	Register Write	Register Read	Default	Description
		(19) G722_DEC_OUT_RDY	(19) G722_DEC_OUT_RDY	0x0	G722_DEC_OUT_RDY interrupt set enable
		(18) G722_DEC_IN_RDY	(18) G722_DEC_IN_RDY	0x0	G722_DEC_IN_RDY interrupt set enable
		(17) G722_ENC_OUT_RDY	(17) G722_ENC_OUT_RDY	0x0	G722_ENC_OUT_RDY interrupt set enable
		(16) G722_ENC_IN_RDY	(16) G722_ENC_IN_RDY	0x0	G722_ENC_IN_RDY interrupt set enable
		(15) ASRC_IDLE	(15) ASRC_IDLE	0x0	ASRC_IDLE interrupt set enable
		(14) ASRC_OUT	(14) ASRC_OUT	0x0	ASRC_OUT interrupt set enable
		(13) ASRC_IN	(13) ASRC_IN	0x0	ASRC_IN interrupt set enable
		(12) ASRC_ERROR	(12) ASRC_ERROR	0x0	ASRC_ERROR interrupt set enable
		(11) PCM_1_RX_TX	(11) PCM_1_RX_TX	0x0	PCM_1_RX_TX interrupt set enable
		(10) PCM_1_ERROR	(10) PCM_1_ERROR	0x0	PCM_1_ERROR interrupt set enable
		(9) PCM_0_RX_TX	(9) PCM_0_RX_TX	0x0	PCM_0_RX_TX interrupt set enable
		(8) PCM_0_ERROR	(8) PCM_0_ERROR	0x0	PCM_0_ERROR interrupt set enable
		(7) FENG	(7) FENG	0x0	FENG interrupt set enable
		(6) UART_0_TX	(6) UART_0_TX	0x0	UART_0_TX interrupt set enable
		(5) UART_0_RX	(5) UART_0_RX	0x0	UART_0_RX interrupt set enable
		(4) UART_0_ERROR	(4) UART_0_ERROR	0x0	UART_0_ERROR interrupt set enable
		(3) EMMC	(3) EMMC	0x0	EMMC interrupt set enable
		(2) SPI_1_TX	(2) SPI_1_TX	0x0	SPI_1_TX interrupt set enable
		(1) SPI_1_RX	(1) SPI_1_RX	0x0	SPI_1_RX interrupt set enable
		(0) SPI_1_COM	(0) SPI_1_COM	0x0	SPI_1_COM interrupt set enable

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E108	NVIC_ISER2	(29) RS_COM	(29) RS_COM	0x0	RS_COM interrupt set enable
		(28) G722_COM	(28) G722_COM	0x0	G722_COM interrupt set enable
		(27) MEM_ERROR	(27) MEM_ERROR	0x0	MEM_ERROR interrupt set enable
		(26) LSAD	(26) LSAD	0x0	LSAD interrupt set enable
		(25) CLK_DET	(25) CLK_DET	0x0	CLK_DET interrupt set enable
		(24) NNA	(24) NNA	0x0	NNA interrupt set enable
		(23) CFX_7	(23) CFX_7	0x0	CFX_7 interrupt set enable
		(22) CFX_6	(22) CFX_6	0x0	CFX_6 interrupt set enable
		(21) CFX_5	(21) CFX_5	0x0	CFX_5 interrupt set enable
		(20) CFX_4	(20) CFX_4	0x0	CFX_4 interrupt set enable
		(19) CFX_3	(19) CFX_3	0x0	CFX_3 interrupt set enable
		(18) CFX_2	(18) CFX_2	0x0	CFX_2 interrupt set enable
		(17) CFX_1	(17) CFX_1	0x0	CFX_1 interrupt set enable
		(16) CFX_0	(16) CFX_0	0x0	CFX_0 interrupt set enable
		(15) DSP_7	(15) DSP_7	0x0	DSP_7 interrupt set enable
		(14) DSP_6	(14) DSP_6	0x0	DSP_6 interrupt set enable
		(13) DSP_5	(13) DSP_5	0x0	DSP_5 interrupt set enable
		(12) DSP_4	(12) DSP_4	0x0	DSP_4 interrupt set enable
		(11) DSP_3	(11) DSP_3	0x0	DSP_3 interrupt set enable
		(10) DSP_2	(10) DSP_2	0x0	DSP_2 interrupt set enable
		(9) DSP_1	(9) DSP_1	0x0	DSP_1 interrupt set enable
		(8) DSP_0	(8) DSP_0	0x0	DSP_0 interrupt set enable
		(7) HEAR_7	(7) HEAR_7	0x0	HEAR_7 interrupt set enable

Address	Register Name	Register Write	Register Read	Default	Description
		(6) HEAR_6	(6) HEAR_6	0x0	HEAR_6 interrupt set enable
		(5) HEAR_5	(5) HEAR_5	0x0	HEAR_5 interrupt set enable
		(4) HEAR_4	(4) HEAR_4	0x0	HEAR_4 interrupt set enable
		(3) HEAR_3	(3) HEAR_3	0x0	HEAR_3 interrupt set enable
		(2) HEAR_2	(2) HEAR_2	0x0	HEAR_2 interrupt set enable
		(1) HEAR_1	(1) HEAR_1	0x0	HEAR_1 interrupt set enable
		(0) HEAR_0	(0) HEAR_0	0x0	HEAR_0 interrupt set enable
0xE000E180	NVIC_ICER0	(31) SPI_0_TX	(31) SPI_0_TX	0x0	SPI_0_TX interrupt clear enable
		(30) SPI_0_RX	(30) SPI_0_RX	0x0	SPI_0_RX interrupt clear enable
		(29) SPI_0_COM	(29) SPI_0_COM	0x0	SPI_0_COM interrupt clear enable
		(28) D_I2C	(28) D_I2C	0x0	D_I2C interrupt clear enable
		(27) I2C_1	(27) I2C_1	0x0	I2C_1 interrupt clear enable
		(26) I2C_0	(26) I2C_0	0x0	I2C_0 interrupt clear enable
		(25) I3C_1	(25) I3C_1	0x0	I3C_1 interrupt clear enable
		(24) I3C_0	(24) I3C_0	0x0	I3C_0 interrupt clear enable
		(23) DIO_7	(23) DIO_7	0x0	DIO_7 interrupt clear enable
		(22) DIO_6	(22) DIO_6	0x0	DIO_6 interrupt clear enable
		(21) DIO_5	(21) DIO_5	0x0	DIO_5 interrupt clear enable
		(20) DIO_4	(20) DIO_4	0x0	DIO_4 interrupt clear enable
		(19) DIO_3	(19) DIO_3	0x0	DIO_3 interrupt clear enable
		(18) DIO_2	(18) DIO_2	0x0	DIO_2 interrupt clear enable
		(17) DIO_1	(17) DIO_1	0x0	DIO_1 interrupt clear enable
		(16) DIO_0	(16) DIO_0	0x0	DIO_0 interrupt clear enable

Address	Register Name	Register Write	Register Read	Default	Description
		(15) DMA_7	(15) DMA_7	0x0	DMA_7 interrupt clear enable
		(14) DMA_6	(14) DMA_6	0x0	DMA_6 interrupt clear enable
		(13) DMA_5	(13) DMA_5	0x0	DMA_5 interrupt clear enable
		(12) DMA_4	(12) DMA_4	0x0	DMA_4 interrupt clear enable
		(11) DMA_3	(11) DMA_3	0x0	DMA_3 interrupt clear enable
		(10) DMA_2	(10) DMA_2	0x0	DMA_2 interrupt clear enable
		(9) DMA_1	(9) DMA_1	0x0	DMA_1 interrupt clear enable
		(8) DMA_0	(8) DMA_0	0x0	DMA_0 interrupt clear enable
		(7) TIMER_3	(7) TIMER_3	0x0	TIMER_3 interrupt clear enable
		(6) TIMER_2	(6) TIMER_2	0x0	TIMER_2 interrupt clear enable
		(5) TIMER_1	(5) TIMER_1	0x0	TIMER_1 interrupt clear enable
		(4) TIMER_0	(4) TIMER_0	0x0	TIMER_0 interrupt clear enable
		(3) AUDIOSINK_1	(3) AUDIOSINK_1	0x0	AUDIOSINK_1 interrupt clear enable
		(2) AUDIOSINK_0	(2) AUDIOSINK_0	0x0	AUDIOSINK_0 interrupt clear enable
		(1) LSAD_BATMON	(1) LSAD_BATMON	0x0	LSAD_BATMON interrupt clear enable
		(0) WATCHDOG	(0) WATCHDOG	0x0	WATCHDOG interrupt clear enable
0xE000E184	NVIC_ICER1	(31) FIFO_7	(31) FIFO_7	0x0	FIFO_7 interrupt clear enable
		(30) FIFO_6	(30) FIFO_6	0x0	FIFO_6 interrupt clear enable
		(29) FIFO_5	(29) FIFO_5	0x0	FIFO_5 interrupt clear enable
		(28) FIFO_4	(28) FIFO_4	0x0	FIFO_4 interrupt clear enable
		(27) FIFO_3	(27) FIFO_3	0x0	FIFO_3 interrupt clear enable
		(26) FIFO_2	(26) FIFO_2	0x0	FIFO_2 interrupt clear enable
		(25) FIFO_1	(25) FIFO_1	0x0	FIFO_1 interrupt clear enable

Address	Register Name	Register Write	Register Read	Default	Description
		(24) FIFO_0	(24) FIFO_0	0x0	FIFO_0 interrupt clear enable
		(23) RS_DEC_OUT_RDY	(23) RS_DEC_OUT_RDY	0x0	RS_DEC_OUT_RDY interrupt clear enable
		(22) RS_DEC_IN_RDY	(22) RS_DEC_IN_RDY	0x0	RS_DEC_IN_RDY interrupt clear enable
		(21) RS_ENC_OUT_RDY	(21) RS_ENC_OUT_RDY	0x0	RS_ENC_OUT_RDY interrupt clear enable
		(20) RS_ENC_IN_RDY	(20) RS_ENC_IN_RDY	0x0	RS_ENC_IN_RDY interrupt clear enable
		(19) G722_DEC_OUT_RDY	(19) G722_DEC_OUT_RDY	0x0	G722_DEC_OUT_RDY interrupt clear enable
		(18) G722_DEC_IN_RDY	(18) G722_DEC_IN_RDY	0x0	G722_DEC_IN_RDY interrupt clear enable
		(17) G722_ENC_OUT_RDY	(17) G722_ENC_OUT_RDY	0x0	G722_ENC_OUT_RDY interrupt clear enable
		(16) G722_ENC_IN_RDY	(16) G722_ENC_IN_RDY	0x0	G722_ENC_IN_RDY interrupt clear enable
		(15) ASRC_IDLE	(15) ASRC_IDLE	0x0	ASRC_IDLE interrupt clear enable
		(14) ASRC_OUT	(14) ASRC_OUT	0x0	ASRC_OUT interrupt clear enable
		(13) ASRC_IN	(13) ASRC_IN	0x0	ASRC_IN interrupt clear enable
		(12) ASRC_ERROR	(12) ASRC_ERROR	0x0	ASRC_ERROR interrupt clear enable
		(11) PCM_1_RX_TX	(11) PCM_1_RX_TX	0x0	PCM_1_RX_TX interrupt clear enable
		(10) PCM_1_ERROR	(10) PCM_1_ERROR	0x0	PCM_1_ERROR interrupt clear enable
		(9) PCM_0_RX_TX	(9) PCM_0_RX_TX	0x0	PCM_0_RX_TX interrupt clear enable
		(8) PCM_0_ERROR	(8) PCM_0_ERROR	0x0	PCM_0_ERROR interrupt clear enable
		(7) FENG	(7) FENG	0x0	FENG interrupt clear enable
		(6) UART_0_TX	(6) UART_0_TX	0x0	UART_0_TX interrupt clear enable

Address	Register Name	Register Write	Register Read	Default	Description
		(5) UART_0_RX	(5) UART_0_RX	0x0	UART_0_RX interrupt clear enable
		(4) UART_0_ERROR	(4) UART_0_ERROR	0x0	UART_0_ERROR interrupt clear enable
		(3) EMMC	(3) EMMC	0x0	EMMC interrupt clear enable
		(2) SPI_1_TX	(2) SPI_1_TX	0x0	SPI_1_TX interrupt clear enable
		(1) SPI_1_RX	(1) SPI_1_RX	0x0	SPI_1_RX interrupt clear enable
		(0) SPI_1_COM	(0) SPI_1_COM	0x0	SPI_1_COM interrupt clear enable
0xE000E188	NVIC_ICER2	(29) RS_COM	(29) RS_COM	0x0	RS_COM interrupt clear enable
		(28) G722_COM	(28) G722_COM	0x0	G722_COM interrupt clear enable
		(27) MEM_ERROR	(27) MEM_ERROR	0x0	MEM_ERROR interrupt clear enable
		(26) LSAD	(26) LSAD	0x0	LSAD interrupt clear enable
		(25) CLK_DET	(25) CLK_DET	0x0	CLK_DET interrupt clear enable
		(24) NNA	(24) NNA	0x0	NNA interrupt clear enable
		(23) CFX_7	(23) CFX_7	0x0	CFX_7 interrupt clear enable
		(22) CFX_6	(22) CFX_6	0x0	CFX_6 interrupt clear enable
		(21) CFX_5	(21) CFX_5	0x0	CFX_5 interrupt clear enable
		(20) CFX_4	(20) CFX_4	0x0	CFX_4 interrupt clear enable
		(19) CFX_3	(19) CFX_3	0x0	CFX_3 interrupt clear enable
		(18) CFX_2	(18) CFX_2	0x0	CFX_2 interrupt clear enable
		(17) CFX_1	(17) CFX_1	0x0	CFX_1 interrupt clear enable
		(16) CFX_0	(16) CFX_0	0x0	CFX_0 interrupt clear enable
		(15) DSP_7	(15) DSP_7	0x0	DSP_7 interrupt clear enable
		(14) DSP_6	(14) DSP_6	0x0	DSP_6 interrupt clear enable
		(13) DSP_5	(13) DSP_5	0x0	DSP_5 interrupt clear enable

Address	Register Name	Register Write	Register Read	Default	Description
		(12) DSP_4	(12) DSP_4	0x0	DSP_4 interrupt clear enable
		(11) DSP_3	(11) DSP_3	0x0	DSP_3 interrupt clear enable
		(10) DSP_2	(10) DSP_2	0x0	DSP_2 interrupt clear enable
		(9) DSP_1	(9) DSP_1	0x0	DSP_1 interrupt clear enable
		(8) DSP_0	(8) DSP_0	0x0	DSP_0 interrupt clear enable
		(7) HEAR_7	(7) HEAR_7	0x0	HEAR_7 interrupt clear enable
		(6) HEAR_6	(6) HEAR_6	0x0	HEAR_6 interrupt clear enable
		(5) HEAR_5	(5) HEAR_5	0x0	HEAR_5 interrupt clear enable
		(4) HEAR_4	(4) HEAR_4	0x0	HEAR_4 interrupt clear enable
		(3) HEAR_3	(3) HEAR_3	0x0	HEAR_3 interrupt clear enable
		(2) HEAR_2	(2) HEAR_2	0x0	HEAR_2 interrupt clear enable
		(1) HEAR_1	(1) HEAR_1	0x0	HEAR_1 interrupt clear enable
		(0) HEAR_0	(0) HEAR_0	0x0	HEAR_0 interrupt clear enable
0xE000E200	NVIC_ISPR0	(31) SPI_0_TX	(31) SPI_0_TX	0x0	SPI_0_TX interrupt set pending
		(30) SPI_0_RX	(30) SPI_0_RX	0x0	SPI_0_RX interrupt set pending
		(29) SPI_0_COM	(29) SPI_0_COM	0x0	SPI_0_COM interrupt set pending
		(28) D_I2C	(28) D_I2C	0x0	D_I2C interrupt set pending
		(27) I2C_1	(27) I2C_1	0x0	I2C_1 interrupt set pending
		(26) I2C_0	(26) I2C_0	0x0	I2C_0 interrupt set pending
		(25) I3C_1	(25) I3C_1	0x0	I3C_1 interrupt set pending
		(24) I3C_0	(24) I3C_0	0x0	I3C_0 interrupt set pending
		(23) DIO_7	(23) DIO_7	0x0	DIO_7 interrupt set pending
		(22) DIO_6	(22) DIO_6	0x0	DIO_6 interrupt set pending

Address	Register Name	Register Write	Register Read	Default	Description
		(21) DIO_5	(21) DIO_5	0x0	DIO_5 interrupt set pending
		(20) DIO_4	(20) DIO_4	0x0	DIO_4 interrupt set pending
		(19) DIO_3	(19) DIO_3	0x0	DIO_3 interrupt set pending
		(18) DIO_2	(18) DIO_2	0x0	DIO_2 interrupt set pending
		(17) DIO_1	(17) DIO_1	0x0	DIO_1 interrupt set pending
		(16) DIO_0	(16) DIO_0	0x0	DIO_0 interrupt set pending
		(15) DMA_7	(15) DMA_7	0x0	DMA_7 interrupt set pending
		(14) DMA_6	(14) DMA_6	0x0	DMA_6 interrupt set pending
		(13) DMA_5	(13) DMA_5	0x0	DMA_5 interrupt set pending
		(12) DMA_4	(12) DMA_4	0x0	DMA_4 interrupt set pending
		(11) DMA_3	(11) DMA_3	0x0	DMA_3 interrupt set pending
		(10) DMA_2	(10) DMA_2	0x0	DMA_2 interrupt set pending
		(9) DMA_1	(9) DMA_1	0x0	DMA_1 interrupt set pending
		(8) DMA_0	(8) DMA_0	0x0	DMA_0 interrupt set pending
		(7) TIMER_3	(7) TIMER_3	0x0	TIMER_3 interrupt set pending
		(6) TIMER_2	(6) TIMER_2	0x0	TIMER_2 interrupt set pending
		(5) TIMER_1	(5) TIMER_1	0x0	TIMER_1 interrupt set pending
		(4) TIMER_0	(4) TIMER_0	0x0	TIMER_0 interrupt set pending
		(3) AUDIOSINK_1	(3) AUDIOSINK_1	0x0	AUDIOSINK_1 interrupt set pending
		(2) AUDIOSINK_0	(2) AUDIOSINK_0	0x0	AUDIOSINK_0 interrupt set pending
		(1) LSAD_BATMON	(1) LSAD_BATMON	0x0	LSAD_BATMON interrupt set pending
		(0) WATCHDOG	(0) WATCHDOG	0x0	WATCHDOG interrupt set pending
0xE000E204	NVIC_ISPR1	(31) FIFO_7	(31) FIFO_7	0x0	FIFO_7 interrupt set pending

Address	Register Name	Register Write	Register Read	Default	Description
		(30) FIFO_6	(30) FIFO_6	0x0	FIFO_6 interrupt set pending
		(29) FIFO_5	(29) FIFO_5	0x0	FIFO_5 interrupt set pending
		(28) FIFO_4	(28) FIFO_4	0x0	FIFO_4 interrupt set pending
		(27) FIFO_3	(27) FIFO_3	0x0	FIFO_3 interrupt set pending
		(26) FIFO_2	(26) FIFO_2	0x0	FIFO_2 interrupt set pending
		(25) FIFO_1	(25) FIFO_1	0x0	FIFO_1 interrupt set pending
		(24) FIFO_0	(24) FIFO_0	0x0	FIFO_0 interrupt set pending
		(23) RS_DEC_OUT_RDY	(23) RS_DEC_OUT_RDY	0x0	RS_DEC_OUT_RDY interrupt set pending
		(22) RS_DEC_IN_RDY	(22) RS_DEC_IN_RDY	0x0	RS_DEC_IN_RDY interrupt set pending
		(21) RS_ENC_OUT_RDY	(21) RS_ENC_OUT_RDY	0x0	RS_ENC_OUT_RDY interrupt set pending
		(20) RS_ENC_IN_RDY	(20) RS_ENC_IN_RDY	0x0	RS_ENC_IN_RDY interrupt set pending
		(19) G722_DEC_OUT_RDY	(19) G722_DEC_OUT_RDY	0x0	G722_DEC_OUT_RDY interrupt set pending
		(18) G722_DEC_IN_RDY	(18) G722_DEC_IN_RDY	0x0	G722_DEC_IN_RDY interrupt set pending
		(17) G722_ENC_OUT_RDY	(17) G722_ENC_OUT_RDY	0x0	G722_ENC_OUT_RDY interrupt set pending
		(16) G722_ENC_IN_RDY	(16) G722_ENC_IN_RDY	0x0	G722_ENC_IN_RDY interrupt set pending
		(15) ASRC_IDLE	(15) ASRC_IDLE	0x0	ASRC_IDLE interrupt set pending
		(14) ASRC_OUT	(14) ASRC_OUT	0x0	ASRC_OUT interrupt set pending
		(13) ASRC_IN	(13) ASRC_IN	0x0	ASRC_IN interrupt set pending
		(12) ASRC_ERROR	(12) ASRC_ERROR	0x0	ASRC_ERROR interrupt set pending

Address	Register Name	Register Write	Register Read	Default	Description
		(11) PCM_1_RX_TX	(11) PCM_1_RX_TX	0x0	PCM_1_RX_TX interrupt set pending
		(10) PCM_1_ERROR	(10) PCM_1_ERROR	0x0	PCM_1_ERROR interrupt set pending
		(9) PCM_0_RX_TX	(9) PCM_0_RX_TX	0x0	PCM_0_RX_TX interrupt set pending
		(8) PCM_0_ERROR	(8) PCM_0_ERROR	0x0	PCM_0_ERROR interrupt set pending
		(7) FENG	(7) FENG	0x0	FENG interrupt set pending
		(6) UART_0_TX	(6) UART_0_TX	0x0	UART_0_TX interrupt set pending
		(5) UART_0_RX	(5) UART_0_RX	0x0	UART_0_RX interrupt set pending
		(4) UART_0_ERROR	(4) UART_0_ERROR	0x0	UART_0_ERROR interrupt set pending
		(3) EMMC	(3) EMMC	0x0	EMMC interrupt set pending
		(2) SPI_1_TX	(2) SPI_1_TX	0x0	SPI_1_TX interrupt set pending
		(1) SPI_1_RX	(1) SPI_1_RX	0x0	SPI_1_RX interrupt set pending
		(0) SPI_1_COM	(0) SPI_1_COM	0x0	SPI_1_COM interrupt set pending
0xE000E208	NVIC_ISPR2	(29) RS_COM	(29) RS_COM	0x0	RS_COM interrupt set pending
		(28) G722_COM	(28) G722_COM	0x0	G722_COM interrupt set pending
		(27) MEM_ERROR	(27) MEM_ERROR	0x0	MEM_ERROR interrupt set pending
		(26) LSAD	(26) LSAD	0x0	LSAD interrupt set pending
		(25) CLK_DET	(25) CLK_DET	0x0	CLK_DET interrupt set pending
		(24) NNA	(24) NNA	0x0	NNA interrupt set pending
		(23) CFX_7	(23) CFX_7	0x0	CFX_7 interrupt set pending
		(22) CFX_6	(22) CFX_6	0x0	CFX_6 interrupt set pending
		(21) CFX_5	(21) CFX_5	0x0	CFX_5 interrupt set pending
		(20) CFX_4	(20) CFX_4	0x0	CFX_4 interrupt set pending
		(19) CFX_3	(19) CFX_3	0x0	CFX_3 interrupt set pending

Address	Register Name	Register Write	Register Read	Default	Description
		(18) CFX_2	(18) CFX_2	0x0	CFX_2 interrupt set pending
		(17) CFX_1	(17) CFX_1	0x0	CFX_1 interrupt set pending
		(16) CFX_0	(16) CFX_0	0x0	CFX_0 interrupt set pending
		(15) DSP_7	(15) DSP_7	0x0	DSP_7 interrupt set pending
		(14) DSP_6	(14) DSP_6	0x0	DSP_6 interrupt set pending
		(13) DSP_5	(13) DSP_5	0x0	DSP_5 interrupt set pending
		(12) DSP_4	(12) DSP_4	0x0	DSP_4 interrupt set pending
		(11) DSP_3	(11) DSP_3	0x0	DSP_3 interrupt set pending
		(10) DSP_2	(10) DSP_2	0x0	DSP_2 interrupt set pending
		(9) DSP_1	(9) DSP_1	0x0	DSP_1 interrupt set pending
		(8) DSP_0	(8) DSP_0	0x0	DSP_0 interrupt set pending
		(7) HEAR_7	(7) HEAR_7	0x0	HEAR_7 interrupt set pending
		(6) HEAR_6	(6) HEAR_6	0x0	HEAR_6 interrupt set pending
		(5) HEAR_5	(5) HEAR_5	0x0	HEAR_5 interrupt set pending
		(4) HEAR_4	(4) HEAR_4	0x0	HEAR_4 interrupt set pending
		(3) HEAR_3	(3) HEAR_3	0x0	HEAR_3 interrupt set pending
		(2) HEAR_2	(2) HEAR_2	0x0	HEAR_2 interrupt set pending
		(1) HEAR_1	(1) HEAR_1	0x0	HEAR_1 interrupt set pending
		(0) HEAR_0	(0) HEAR_0	0x0	HEAR_0 interrupt set pending
0xE000E280	NVIC_ICPR0	(31) SPI_0_TX	(31) SPI_0_TX	0x0	SPI_0_TX interrupt clear pending
		(30) SPI_0_RX	(30) SPI_0_RX	0x0	SPI_0_RX interrupt clear pending
		(29) SPI_0_COM	(29) SPI_0_COM	0x0	SPI_0_COM interrupt clear pending
		(28) D_I2C	(28) D_I2C	0x0	D_I2C interrupt clear pending

Address	Register Name	Register Write	Register Read	Default	Description
		(27) I2C_1	(27) I2C_1	0x0	I2C_1 interrupt clear pending
		(26) I2C_0	(26) I2C_0	0x0	I2C_0 interrupt clear pending
		(25) I3C_1	(25) I3C_1	0x0	I3C_1 interrupt clear pending
		(24) I3C_0	(24) I3C_0	0x0	I3C_0 interrupt clear pending
		(23) DIO_7	(23) DIO_7	0x0	DIO_7 interrupt clear pending
		(22) DIO_6	(22) DIO_6	0x0	DIO_6 interrupt clear pending
		(21) DIO_5	(21) DIO_5	0x0	DIO_5 interrupt clear pending
		(20) DIO_4	(20) DIO_4	0x0	DIO_4 interrupt clear pending
		(19) DIO_3	(19) DIO_3	0x0	DIO_3 interrupt clear pending
		(18) DIO_2	(18) DIO_2	0x0	DIO_2 interrupt clear pending
		(17) DIO_1	(17) DIO_1	0x0	DIO_1 interrupt clear pending
		(16) DIO_0	(16) DIO_0	0x0	DIO_0 interrupt clear pending
		(15) DMA_7	(15) DMA_7	0x0	DMA_7 interrupt clear pending
		(14) DMA_6	(14) DMA_6	0x0	DMA_6 interrupt clear pending
		(13) DMA_5	(13) DMA_5	0x0	DMA_5 interrupt clear pending
		(12) DMA_4	(12) DMA_4	0x0	DMA_4 interrupt clear pending
		(11) DMA_3	(11) DMA_3	0x0	DMA_3 interrupt clear pending
		(10) DMA_2	(10) DMA_2	0x0	DMA_2 interrupt clear pending
		(9) DMA_1	(9) DMA_1	0x0	DMA_1 interrupt clear pending
		(8) DMA_0	(8) DMA_0	0x0	DMA_0 interrupt clear pending
		(7) TIMER_3	(7) TIMER_3	0x0	TIMER_3 interrupt clear pending
		(6) TIMER_2	(6) TIMER_2	0x0	TIMER_2 interrupt clear pending
		(5) TIMER_1	(5) TIMER_1	0x0	TIMER_1 interrupt clear pending

Address	Register Name	Register Write	Register Read	Default	Description
		(4) TIMER_0	(4) TIMER_0	0x0	TIMER_0 interrupt clear pending
		(3) AUDIOSINK_1	(3) AUDIOSINK_1	0x0	AUDIOSINK_1 interrupt clear pending
		(2) AUDIOSINK_0	(2) AUDIOSINK_0	0x0	AUDIOSINK_0 interrupt clear pending
		(1) LSAD_BATMON	(1) LSAD_BATMON	0x0	LSAD_BATMON interrupt clear pending
		(0) WATCHDOG	(0) WATCHDOG	0x0	WATCHDOG interrupt clear pending
0xE000E284	NVIC_ICPR1	(31) FIFO_7	(31) FIFO_7	0x0	FIFO_7 interrupt clear pending
		(30) FIFO_6	(30) FIFO_6	0x0	FIFO_6 interrupt clear pending
		(29) FIFO_5	(29) FIFO_5	0x0	FIFO_5 interrupt clear pending
		(28) FIFO_4	(28) FIFO_4	0x0	FIFO_4 interrupt clear pending
		(27) FIFO_3	(27) FIFO_3	0x0	FIFO_3 interrupt clear pending
		(26) FIFO_2	(26) FIFO_2	0x0	FIFO_2 interrupt clear pending
		(25) FIFO_1	(25) FIFO_1	0x0	FIFO_1 interrupt clear pending
		(24) FIFO_0	(24) FIFO_0	0x0	FIFO_0 interrupt clear pending
		(23) RS_DEC_OUT_RDY	(23) RS_DEC_OUT_RDY	0x0	RS_DEC_OUT_RDY interrupt clear pending
		(22) RS_DEC_IN_RDY	(22) RS_DEC_IN_RDY	0x0	RS_DEC_IN_RDY interrupt clear pending
		(21) RS_ENC_OUT_RDY	(21) RS_ENC_OUT_RDY	0x0	RS_ENC_OUT_RDY interrupt clear pending
		(20) RS_ENC_IN_RDY	(20) RS_ENC_IN_RDY	0x0	RS_ENC_IN_RDY interrupt clear pending
		(19) G722_DEC_OUT_RDY	(19) G722_DEC_OUT_RDY	0x0	G722_DEC_OUT_RDY interrupt clear pending
		(18) G722_DEC_IN_RDY	(18) G722_DEC_IN_RDY	0x0	G722_DEC_IN_RDY interrupt clear pending

Address	Register Name	Register Write	Register Read	Default	Description
		(17) G722_ENC_OUT_RDY	(17) G722_ENC_OUT_RDY	0x0	G722_ENC_OUT_RDY interrupt clear pending
		(16) G722_ENC_IN_RDY	(16) G722_ENC_IN_RDY	0x0	G722_ENC_IN_RDY interrupt clear pending
		(15) ASRC_IDLE	(15) ASRC_IDLE	0x0	ASRC_IDLE interrupt clear pending
		(14) ASRC_OUT	(14) ASRC_OUT	0x0	ASRC_OUT interrupt clear pending
		(13) ASRC_IN	(13) ASRC_IN	0x0	ASRC_IN interrupt clear pending
		(12) ASRC_ERROR	(12) ASRC_ERROR	0x0	ASRC_ERROR interrupt clear pending
		(11) PCM_1_RX_TX	(11) PCM_1_RX_TX	0x0	PCM_1_RX_TX interrupt clear pending
		(10) PCM_1_ERROR	(10) PCM_1_ERROR	0x0	PCM_1_ERROR interrupt clear pending
		(9) PCM_0_RX_TX	(9) PCM_0_RX_TX	0x0	PCM_0_RX_TX interrupt clear pending
		(8) PCM_0_ERROR	(8) PCM_0_ERROR	0x0	PCM_0_ERROR interrupt clear pending
		(7) FENG	(7) FENG	0x0	FENG interrupt clear pending
		(6) UART_0_TX	(6) UART_0_TX	0x0	UART_0_TX interrupt clear pending
		(5) UART_0_RX	(5) UART_0_RX	0x0	UART_0_RX interrupt clear pending
		(4) UART_0_ERROR	(4) UART_0_ERROR	0x0	UART_0_ERROR interrupt clear pending
		(3) EMMC	(3) EMMC	0x0	EMMC interrupt clear pending
		(2) SPI_1_TX	(2) SPI_1_TX	0x0	SPI_1_TX interrupt clear pending
		(1) SPI_1_RX	(1) SPI_1_RX	0x0	SPI_1_RX interrupt clear pending
		(0) SPI_1_COM	(0) SPI_1_COM	0x0	SPI_1_COM interrupt clear pending
0xE000E288	NVIC_ICPR2	(29) RS_COM	(29) RS_COM	0x0	RS_COM interrupt clear pending
		(28) G722_COM	(28) G722_COM	0x0	G722_COM interrupt clear pending
		(27) MEM_ERROR	(27) MEM_ERROR	0x0	MEM_ERROR interrupt clear pending
		(26) LSAD	(26) LSAD	0x0	LSAD interrupt clear pending

Address	Register Name	Register Write	Register Read	Default	Description
		(25) CLK_DET	(25) CLK_DET	0x0	CLK_DET interrupt clear pending
		(24) NNA	(24) NNA	0x0	NNA interrupt clear pending
		(23) CFX_7	(23) CFX_7	0x0	CFX_7 interrupt clear pending
		(22) CFX_6	(22) CFX_6	0x0	CFX_6 interrupt clear pending
		(21) CFX_5	(21) CFX_5	0x0	CFX_5 interrupt clear pending
		(20) CFX_4	(20) CFX_4	0x0	CFX_4 interrupt clear pending
		(19) CFX_3	(19) CFX_3	0x0	CFX_3 interrupt clear pending
		(18) CFX_2	(18) CFX_2	0x0	CFX_2 interrupt clear pending
		(17) CFX_1	(17) CFX_1	0x0	CFX_1 interrupt clear pending
		(16) CFX_0	(16) CFX_0	0x0	CFX_0 interrupt clear pending
		(15) DSP_7	(15) DSP_7	0x0	DSP_7 interrupt clear pending
		(14) DSP_6	(14) DSP_6	0x0	DSP_6 interrupt clear pending
		(13) DSP_5	(13) DSP_5	0x0	DSP_5 interrupt clear pending
		(12) DSP_4	(12) DSP_4	0x0	DSP_4 interrupt clear pending
		(11) DSP_3	(11) DSP_3	0x0	DSP_3 interrupt clear pending
		(10) DSP_2	(10) DSP_2	0x0	DSP_2 interrupt clear pending
		(9) DSP_1	(9) DSP_1	0x0	DSP_1 interrupt clear pending
		(8) DSP_0	(8) DSP_0	0x0	DSP_0 interrupt clear pending
		(7) HEAR_7	(7) HEAR_7	0x0	HEAR_7 interrupt clear pending
		(6) HEAR_6	(6) HEAR_6	0x0	HEAR_6 interrupt clear pending
		(5) HEAR_5	(5) HEAR_5	0x0	HEAR_5 interrupt clear pending
		(4) HEAR_4	(4) HEAR_4	0x0	HEAR_4 interrupt clear pending
		(3) HEAR_3	(3) HEAR_3	0x0	HEAR_3 interrupt clear pending

Address	Register Name	Register Write	Register Read	Default	Description
		(2) HEAR_2	(2) HEAR_2	0x0	HEAR_2 interrupt clear pending
		(1) HEAR_1	(1) HEAR_1	0x0	HEAR_1 interrupt clear pending
		(0) HEAR_0	(0) HEAR_0	0x0	HEAR_0 interrupt clear pending
0xE000E300	NVIC_IABR0	–	(31) SPI_0_TX	0x0	Set the SPI_0_TX interrupt as active
		–	(30) SPI_0_RX	0x0	Set the SPI_0_RX interrupt as active
		–	(29) SPI_0_COM	0x0	Set the SPI_0_COM interrupt as active
		–	(28) D_I2C	0x0	Set the D_I2C interrupt as active
		–	(27) I2C_1	0x0	Set the I2C_1 interrupt as active
		–	(26) I2C_0	0x0	Set the I2C_0 interrupt as active
		–	(25) I3C_1	0x0	Set the I3C_1 interrupt as active
		–	(24) I3C_0	0x0	Set the I3C_0 interrupt as active
		–	(23) DIO_7	0x0	Set the DIO_7 interrupt as active
		–	(22) DIO_6	0x0	Set the DIO_6 interrupt as active
		–	(21) DIO_5	0x0	Set the DIO_5 interrupt as active
		–	(20) DIO_4	0x0	Set the DIO_4 interrupt as active
		–	(19) DIO_3	0x0	Set the DIO_3 interrupt as active
		–	(18) DIO_2	0x0	Set the DIO_2 interrupt as active
		–	(17) DIO_1	0x0	Set the DIO_1 interrupt as active
		–	(16) DIO_0	0x0	Set the DIO_0 interrupt as active
		–	(15) DMA_7	0x0	Set the DMA_7 interrupt as active
		–	(14) DMA_6	0x0	Set the DMA_6 interrupt as active
		–	(13) DMA_5	0x0	Set the DMA_5 interrupt as active
		–	(12) DMA_4	0x0	Set the DMA_4 interrupt as active

Address	Register Name	Register Write	Register Read	Default	Description
		–	(11) DMA_3	0x0	Set the DMA_3 interrupt as active
		–	(10) DMA_2	0x0	Set the DMA_2 interrupt as active
		–	(9) DMA_1	0x0	Set the DMA_1 interrupt as active
		–	(8) DMA_0	0x0	Set the DMA_0 interrupt as active
		–	(7) TIMER_3	0x0	Set the TIMER_3 interrupt as active
		–	(6) TIMER_2	0x0	Set the TIMER_2 interrupt as active
		–	(5) TIMER_1	0x0	Set the TIMER_1 interrupt as active
		–	(4) TIMER_0	0x0	Set the TIMER_0 interrupt as active
		–	(3) AUDIOSINK_1	0x0	Set the AUDIOSINK_1 interrupt as active
		–	(2) AUDIOSINK_0	0x0	Set the AUDIOSINK_0 interrupt as active
		–	(1) LSAD_BATMON	0x0	Set the LSAD_BATMON interrupt as active
		–	(0) WATCHDOG	0x0	Set the WATCHDOG interrupt as active
0xE000E304	NVIC_IABR1	–	(31) FIFO_7	0x0	Set the FIFO_7 interrupt as active
		–	(30) FIFO_6	0x0	Set the FIFO_6 interrupt as active
		–	(29) FIFO_5	0x0	Set the FIFO_5 interrupt as active
		–	(28) FIFO_4	0x0	Set the FIFO_4 interrupt as active
		–	(27) FIFO_3	0x0	Set the FIFO_3 interrupt as active
		–	(26) FIFO_2	0x0	Set the FIFO_2 interrupt as active
		–	(25) FIFO_1	0x0	Set the FIFO_1 interrupt as active
		–	(24) FIFO_0	0x0	Set the FIFO_0 interrupt as active
		–	(23) RS_DEC_OUT_RDY	0x0	Set the RS_DEC_OUT_RDY interrupt as active
		–	(22) RS_DEC_IN_RDY	0x0	Set the RS_DEC_IN_RDY interrupt as active

Address	Register Name	Register Write	Register Read	Default	Description
					active
		–	(21) RS_ENC_OUT_RDY	0x0	Set the RS_ENC_OUT_RDY interrupt as active
		–	(20) RS_ENC_IN_RDY	0x0	Set the RS_ENC_IN_RDY interrupt as active
		–	(19) G722_DEC_OUT_RDY	0x0	Set the G722_DEC_OUT_RDY interrupt as active
		–	(18) G722_DEC_IN_RDY	0x0	Set the G722_DEC_IN_RDY interrupt as active
		–	(17) G722_ENC_OUT_RDY	0x0	Set the G722_ENC_OUT_RDY interrupt as active
		–	(16) G722_ENC_IN_RDY	0x0	Set the G722_ENC_IN_RDY interrupt as active
		–	(15) ASRC_IDLE	0x0	Set the ASRC_IDLE interrupt as active
		–	(14) ASRC_OUT	0x0	Set the ASRC_OUT interrupt as active
		–	(13) ASRC_IN	0x0	Set the ASRC_IN interrupt as active
		–	(12) ASRC_ERROR	0x0	Set the ASRC_ERROR interrupt as active
		–	(11) PCM_1_RX_TX	0x0	Set the PCM_1_RX_TX interrupt as active
		–	(10) PCM_1_ERROR	0x0	Set the PCM_1_ERROR interrupt as active
		–	(9) PCM_0_RX_TX	0x0	Set the PCM_0_RX_TX interrupt as active
		–	(8) PCM_0_ERROR	0x0	Set the PCM_0_ERROR interrupt as active
		–	(7) FENG	0x0	Set the FENG interrupt as active

Address	Register Name	Register Write	Register Read	Default	Description
		–	(6) UART_0_TX	0x0	Set the UART_0_TX interrupt as active
		–	(5) UART_0_RX	0x0	Set the UART_0_RX interrupt as active
		–	(4) UART_0_ERROR	0x0	Set the UART_0_ERROR interrupt as active
		–	(3) EMMC	0x0	Set the EMMC interrupt as active
		–	(2) SPI_1_TX	0x0	Set the SPI_1_TX interrupt as active
		–	(1) SPI_1_RX	0x0	Set the SPI_1_RX interrupt as active
		–	(0) SPI_1_COM	0x0	Set the SPI_1_COM interrupt as active
0xE000E308	NVIC_IABR2	–	(29) RS_COM	0x0	Set the RS_COM interrupt as active
		–	(28) G722_COM	0x0	Set the G722_COM interrupt as active
		–	(27) MEM_ERROR	0x0	Set the MEM_ERROR interrupt as active
		–	(26) LSAD	0x0	Set the LSAD interrupt as active
		–	(25) CLK_DET	0x0	Set the CLK_DET interrupt as active
		–	(24) NNA	0x0	Set the NNA interrupt as active
		–	(23) CFX_7	0x0	Set the CFX_7 interrupt as active
		–	(22) CFX_6	0x0	Set the CFX_6 interrupt as active
		–	(21) CFX_5	0x0	Set the CFX_5 interrupt as active
		–	(20) CFX_4	0x0	Set the CFX_4 interrupt as active
		–	(19) CFX_3	0x0	Set the CFX_3 interrupt as active
		–	(18) CFX_2	0x0	Set the CFX_2 interrupt as active
		–	(17) CFX_1	0x0	Set the CFX_1 interrupt as active
		–	(16) CFX_0	0x0	Set the CFX_0 interrupt as active
		–	(15) DSP_7	0x0	Set the DSP_7 interrupt as active

Address	Register Name	Register Write	Register Read	Default	Description
		–	(14) DSP_6	0x0	Set the DSP_6 interrupt as active
		–	(13) DSP_5	0x0	Set the DSP_5 interrupt as active
		–	(12) DSP_4	0x0	Set the DSP_4 interrupt as active
		–	(11) DSP_3	0x0	Set the DSP_3 interrupt as active
		–	(10) DSP_2	0x0	Set the DSP_2 interrupt as active
		–	(9) DSP_1	0x0	Set the DSP_1 interrupt as active
		–	(8) DSP_0	0x0	Set the DSP_0 interrupt as active
		–	(7) HEAR_7	0x0	Set the HEAR_7 interrupt as active
		–	(6) HEAR_6	0x0	Set the HEAR_6 interrupt as active
		–	(5) HEAR_5	0x0	Set the HEAR_5 interrupt as active
		–	(4) HEAR_4	0x0	Set the HEAR_4 interrupt as active
		–	(3) HEAR_3	0x0	Set the HEAR_3 interrupt as active
		–	(2) HEAR_2	0x0	Set the HEAR_2 interrupt as active
		–	(1) HEAR_1	0x0	Set the HEAR_1 interrupt as active
		–	(0) HEAR_0	0x0	Set the HEAR_0 interrupt as active
0xE000E400	NVIC_IP0	(31:24) AUDIOSINK_1	(31:24) AUDIOSINK_1	0x0	Configure the AUDIOSINK_1 interrupt priority
		(23:16) AUDIOSINK_0	(23:16) AUDIOSINK_0	0x0	Configure the AUDIOSINK_0 interrupt priority
		(15:8) LSAD_BATMON	(15:8) LSAD_BATMON	0x0	Configure the LSAD_BATMON interrupt priority
		(7:0) WATCHDOG	(7:0) WATCHDOG	0x0	Configure the WATCHDOG interrupt priority
0xE000E404	NVIC_IP1	(31:24) TIMER_3	(31:24) TIMER_3	0x0	Configure the TIMER_3 interrupt priority

Address	Register Name	Register Write	Register Read	Default	Description
		(23:16) TIMER_2	(23:16) TIMER_2	0x0	Configure the TIMER_2 interrupt priority
		(15:8) TIMER_1	(15:8) TIMER_1	0x0	Configure the TIMER_1 interrupt priority
		(7:0) TIMER_0	(7:0) TIMER_0	0x0	Configure the TIMER_0 interrupt priority
0xE000E408	NVIC_IP2	(31:24) DMA_3	(31:24) DMA_3	0x0	Configure the DMA_3 interrupt priority
		(23:16) DMA_2	(23:16) DMA_2	0x0	Configure the DMA_2 interrupt priority
		(15:8) DMA_1	(15:8) DMA_1	0x0	Configure the DMA_1 interrupt priority
		(7:0) DMA_0	(7:0) DMA_0	0x0	Configure the DMA_0 interrupt priority
0xE000E40C	NVIC_IP3	(31:24) DMA_7	(31:24) DMA_7	0x0	Configure the DMA_7 interrupt priority
		(23:16) DMA_6	(23:16) DMA_6	0x0	Configure the DMA_6 interrupt priority
		(15:8) DMA_5	(15:8) DMA_5	0x0	Configure the DMA_5 interrupt priority
		(7:0) DMA_4	(7:0) DMA_4	0x0	Configure the DMA_4 interrupt priority
0xE000E410	NVIC_IP4	(31:24) DIO_3	(31:24) DIO_3	0x0	Configure the DIO_3 interrupt priority
		(23:16) DIO_2	(23:16) DIO_2	0x0	Configure the DIO_2 interrupt priority
		(15:8) DIO_1	(15:8) DIO_1	0x0	Configure the DIO_1 interrupt priority
		(7:0) DIO_0	(7:0) DIO_0	0x0	Configure the DIO_0 interrupt priority
0xE000E414	NVIC_IP5	(31:24) DIO_7	(31:24) DIO_7	0x0	Configure the DIO_7 interrupt priority
		(23:16) DIO_6	(23:16) DIO_6	0x0	Configure the DIO_6 interrupt priority
		(15:8) DIO_5	(15:8) DIO_5	0x0	Configure the DIO_5 interrupt priority
		(7:0) DIO_4	(7:0) DIO_4	0x0	Configure the DIO_4 interrupt priority
0xE000E418	NVIC_IP6	(31:24) I2C_1	(31:24) I2C_1	0x0	Configure the I2C_1 interrupt priority
		(23:16) I2C_0	(23:16) I2C_0	0x0	Configure the I2C_0 interrupt priority
		(15:8) I3C_1	(15:8) I3C_1	0x0	Configure the I3C_1 interrupt priority
		(7:0) I3C_0	(7:0) I3C_0	0x0	Configure the I3C_0 interrupt priority

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E41C	NVIC_IP7	(31:24) SPI_0_TX	(31:24) SPI_0_TX	0x0	Configure the SPI_0_TX interrupt priority
		(23:16) SPI_0_RX	(23:16) SPI_0_RX	0x0	Configure the SPI_0_RX interrupt priority
		(15:8) SPI_0_COM	(15:8) SPI_0_COM	0x0	Configure the SPI_0_COM interrupt priority
		(7:0) D_I2C	(7:0) D_I2C	0x0	Configure the D_I2C interrupt priority
0xE000E420	NVIC_IP8	(31:24) EMMC	(31:24) EMMC	0x0	Configure the EMMC interrupt priority
		(23:16) SPI_1_TX	(23:16) SPI_1_TX	0x0	Configure the SPI_1_TX interrupt priority
		(15:8) SPI_1_RX	(15:8) SPI_1_RX	0x0	Configure the SPI_1_RX interrupt priority
		(7:0) SPI_1_COM	(7:0) SPI_1_COM	0x0	Configure the SPI_1_COM interrupt priority
0xE000E424	NVIC_IP9	(31:24) FENG	(31:24) FENG	0x0	Configure the FENG interrupt priority
		(23:16) UART_0_TX	(23:16) UART_0_TX	0x0	Configure the UART_0_TX interrupt priority
		(15:8) UART_0_RX	(15:8) UART_0_RX	0x0	Configure the UART_0_RX interrupt priority
		(7:0) UART_0_ERROR	(7:0) UART_0_ERROR	0x0	Configure the UART_0_ERROR interrupt priority
0xE000E428	NVIC_IP10	(31:24) PCM_1_RX_TX	(31:24) PCM_1_RX_TX	0x0	Configure the PCM_1_RX_TX interrupt priority
		(23:16) PCM_1_ERROR	(23:16) PCM_1_ERROR	0x0	Configure the PCM_1_ERROR interrupt priority
		(15:8) PCM_0_RX_TX	(15:8) PCM_0_RX_TX	0x0	Configure the PCM_0_RX_TX interrupt priority
		(7:0) PCM_0_ERROR	(7:0) PCM_0_ERROR	0x0	Configure the PCM_0_ERROR interrupt priority
0xE000E42C	NVIC_IP11	(31:24) ASRC_IDLE	(31:24) ASRC_IDLE	0x0	Configure the ASRC_IDLE interrupt priority

Address	Register Name	Register Write	Register Read	Default	Description
		(23:16) ASRC_OUT	(23:16) ASRC_OUT	0x0	Configure the ASRC_OUT interrupt priority
		(15:8) ASRC_IN	(15:8) ASRC_IN	0x0	Configure the ASRC_IN interrupt priority
		(7:0) ASRC_ERROR	(7:0) ASRC_ERROR	0x0	Configure the ASRC_ERROR interrupt priority
0xE000E430	NVIC_IP12	(31:24) G722_DEC_OUT_RDY	(31:24) G722_DEC_OUT_RDY	0x0	Configure the G722_DEC_OUT_RDY interrupt priority
		(23:16) G722_DEC_IN_RDY	(23:16) G722_DEC_IN_RDY	0x0	Configure the G722_DEC_IN_RDY interrupt priority
		(15:8) G722_ENC_OUT_RDY	(15:8) G722_ENC_OUT_RDY	0x0	Configure the G722_ENC_OUT_RDY interrupt priority
		(7:0) G722_ENC_IN_RDY	(7:0) G722_ENC_IN_RDY	0x0	Configure the G722_ENC_IN_RDY interrupt priority
0xE000E434	NVIC_IP13	(31:24) RS_DEC_OUT_RDY	(31:24) RS_DEC_OUT_RDY	0x0	Configure the RS_DEC_OUT_RDY interrupt priority
		(23:16) RS_DEC_IN_RDY	(23:16) RS_DEC_IN_RDY	0x0	Configure the RS_DEC_IN_RDY interrupt priority
		(15:8) RS_ENC_OUT_RDY	(15:8) RS_ENC_OUT_RDY	0x0	Configure the RS_ENC_OUT_RDY interrupt priority
		(7:0) RS_ENC_IN_RDY	(7:0) RS_ENC_IN_RDY	0x0	Configure the RS_ENC_IN_RDY interrupt priority
0xE000E438	NVIC_IP14	(31:24) FIFO_3	(31:24) FIFO_3	0x0	Configure the FIFO_3 interrupt priority
		(23:16) FIFO_2	(23:16) FIFO_2	0x0	Configure the FIFO_2 interrupt priority
		(15:8) FIFO_1	(15:8) FIFO_1	0x0	Configure the FIFO_1 interrupt priority
		(7:0) FIFO_0	(7:0) FIFO_0	0x0	Configure the FIFO_0 interrupt priority
0xE000E43C	NVIC_IP15	(31:24) FIFO_7	(31:24) FIFO_7	0x0	Configure the FIFO_7 interrupt priority

Address	Register Name	Register Write	Register Read	Default	Description
		(23:16) FIFO_6	(23:16) FIFO_6	0x0	Configure the FIFO_6 interrupt priority
		(15:8) FIFO_5	(15:8) FIFO_5	0x0	Configure the FIFO_5 interrupt priority
		(7:0) FIFO_4	(7:0) FIFO_4	0x0	Configure the FIFO_4 interrupt priority
0xE000E440	NVIC_IP16	(31:24) HEAR_3	(31:24) HEAR_3	0x0	Configure the HEAR_3 interrupt priority
		(23:16) HEAR_2	(23:16) HEAR_2	0x0	Configure the HEAR_2 interrupt priority
		(15:8) HEAR_1	(15:8) HEAR_1	0x0	Configure the HEAR_1 interrupt priority
		(7:0) HEAR_0	(7:0) HEAR_0	0x0	Configure the HEAR_0 interrupt priority
0xE000E444	NVIC_IP17	(31:24) HEAR_7	(31:24) HEAR_7	0x0	Configure the HEAR_7 interrupt priority
		(23:16) HEAR_6	(23:16) HEAR_6	0x0	Configure the HEAR_6 interrupt priority
		(15:8) HEAR_5	(15:8) HEAR_5	0x0	Configure the HEAR_5 interrupt priority
		(7:0) HEAR_4	(7:0) HEAR_4	0x0	Configure the HEAR_4 interrupt priority
0xE000E444	NVIC_IP18	(31:24) DSP_3	(31:24) DSP_3	0x0	Configure the DSP_3 interrupt priority
		(23:16) DSP_2	(23:16) DSP_2	0x0	Configure the DSP_2 interrupt priority
		(15:8) DSP_1	(15:8) DSP_1	0x0	Configure the DSP_1 interrupt priority
		(7:0) DSP_0	(7:0) DSP_0	0x0	Configure the DSP_0 interrupt priority
0xE000E444	NVIC_IP19	(31:24) DSP_7	(31:24) DSP_7	0x0	Configure the DSP_7 interrupt priority
		(23:16) DSP_6	(23:16) DSP_6	0x0	Configure the DSP_6 interrupt priority
		(15:8) DSP_5	(15:8) DSP_5	0x0	Configure the DSP_5 interrupt priority
		(7:0) DSP_4	(7:0) DSP_4	0x0	Configure the DSP_4 interrupt priority
0xE000E444	NVIC_IP20	(31:24) CFX_3	(31:24) CFX_3	0x0	Configure the CFX_3 interrupt priority
		(23:16) CFX_2	(23:16) CFX_2	0x0	Configure the CFX_2 interrupt priority
		(15:8) CFX_1	(15:8) CFX_1	0x0	Configure the CFX_1 interrupt priority
		(7:0) CFX_0	(7:0) CFX_0	0x0	Configure the CFX_0 interrupt priority

Address	Register Name	Register Write	Register Read	Default	Description
0xE000E444	NVIC_IP21	(31:24) CFX_7	(31:24) CFX_7	0x0	Configure the CFX_7 interrupt priority
		(23:16) CFX_6	(23:16) CFX_6	0x0	Configure the CFX_6 interrupt priority
		(15:8) CFX_5	(15:8) CFX_5	0x0	Configure the CFX_5 interrupt priority
		(7:0) CFX_4	(7:0) CFX_4	0x0	Configure the CFX_4 interrupt priority
0xE000E444	NVIC_IP22	(31:24) MEM_ERROR	(31:24) MEM_ERROR	0x0	Configure the MEM_ERROR interrupt priority
		(23:16) LSAD	(23:16) LSAD	0x0	Configure the LSAD interrupt priority
		(15:8) CLK_DET	(15:8) CLK_DET	0x0	Configure the CLK_DET interrupt priority
		(7:0) NNA	(7:0) NNA	0x0	Configure the NNA interrupt priority
0xE000E444	NVIC_IP23	(15:8) RS_COM	(15:8) RS_COM	0x0	Configure the RS_COM interrupt priority
		(7:0) G722_COM	(7:0) G722_COM	0x0	Configure the G722_COM interrupt priority
0xE000EF00	NVIC_STIR	(7:0) INTID	–	N/A	The interrupt number to trigger

A.41 SYSTEM CONTROL BLOCK

Address	Register Name	Register Write	Register Read	Default	Description
0xE000ED00	SCB_CPUID	–	(31:24) IMPLEMENTER	0x41	Implementer code
		–	(23:20) VARIANT	0x2	Implementation variant
		–	(19:16) ARCHITECTURE	0xF	Architecture number
		–	(15:4) PARTNO	0xC23	Part number
		–	(3:0) REVISION	0x1	Revision code
0xE000ED04	SCB_ICSR	(31) NMIPENDSET	(31) NMIPENDSET	0x0	Indicates the NMI exception is pending
		(28) PENDSVSET	(28) PENDSVSET	0x0	Write 1 to set pending status for PendSV exception. Read indicates PendSV pending status
		(27) PENDSVCLR	–	N/A	Write 1 to clear PendSV exception pending status
		(26) PENDSTSET	(26) PENDSTSET	0x0	Write 1 to set pending status for SYSTICK exception. Read value indicates SYSTICK pending status
		(25) PENDSTCLR	–	N/A	Write 1 to clear SYSTICK exception pending status
		–	(23) ISRPREEMPT	0x0	Indicates that a pending interrupt is going to be active in the next step
		–	(22) ISRPENDING	0x0	Indicates that an external interrupt is pending
		–	(21:12) VECTPENDING	0x0	Pending external interrupt number
		–	(11) RETTOBASE	0x0	Set to 1 when the an exception handler is being run
		–	(9:0) VECTACTIVE	0x0	Number of current running interrupt

Address	Register Name	Register Write	Register Read	Default	Description
					service routine
0xE000ED08	SCB_VTOR	(31:7) TBLOFF	(31:7) TBLOFF	0x0	Table offset value in code or RAM region. Must be multiple of 128
0xE000ED0C	SCB_AICR	(31:16) VECTKEY	(31:16) VECTKEY	0xFA05	Access key for writing this register. Must be set to 0x05FA to write to the other register fields
		–	(15) ENDIANESS	0x0	Indicates endianness for data.
		(10:8) PRIGROUP	(10:8) PRIGROUP	0x7	Priority group setting. Controls how many bits in the priority register are used for pre-empt priority vs. sub-priority
		(2) SYSRESETREQ	–	N/A	Requests a chip-level system reset
		(1) VECTCLRACTIVE	–	N/A	Clears all active exception state information
		(0) VECTRESET	–	N/A	Reset the Arm Cortex-M3
0xE000ED10	SCB_SCR	(4) SEVONPEND	(4) SEVONPEND	0x0	Set to 1 to cause the WFE to wake up if a new interrupt is pending
		(2) SLEEPDEEP	(2) SLEEPDEEP	0x0	Enable SLEEPDEEP output signal when entering sleep mode
		(1) SLEEPONEXIT	(1) SLEEPONEXIT	0x0	Enable sleep on exit feature when returning from handler to thread mode
0xE000ED14	SCB_CCR	(9) STKALIGN	(9) STKALIGN	0x0	Force stack to be double-word aligned
		(8) BFHFNMIGN	(8) BFHFNMIGN	0x0	Ignore data bus faults in hard fault and NMI exceptions
		(4) DIV_0_TRP	(4) DIV_0_TRP	0x0	Trap on divide by zero
		(3) UNALIGN_TRP	(3) UNALIGN_TRP	0x0	Trap on unaligned data access
		(1) USERSETMPEND	(1) USERSETMPEND	0x0	Allow user code to write to software trigger interrupt register

Address	Register Name	Register Write	Register Read	Default	Description
		(0) NONBASETHRDENA	(0) NONBASETHRDENA	0x0	Non base thread mode enable
0xE000ED18	SCB_SHP0	(23:16) NVIC_USAGE_FAULT_PRIORITY	(23:16) NVIC_USAGE_FAULT_PRIORITY	0x0	Configure the usage fault priority
		(15:8) NVIC_BUS_FAULT_PRIORITY	(15:8) NVIC_BUS_FAULT_PRIORITY	0x0	Configure the bus fault priority
		(7:0) NVIC_MEM_FAULT_PRIORITY	(7:0) NVIC_MEM_FAULT_PRIORITY	0x0	Configure the memory fault priority
0xE000ED1C	SCB_SHP1	(31:24) NVIC_SVC_PRIORITY	(31:24) NVIC_SVC_PRIORITY	0x0	Configure the SVC priority
0xE000ED20	SCB_SHP2	(31:24) NVIC_SYSTICK_PRIORITY	(31:24) NVIC_SYSTICK_PRIORITY	0x0	Configure the SysTick interrupt priority
		(23:16) NVIC_PENDSV_PRIORITY	(23:16) NVIC_PENDSV_PRIORITY	0x0	Configure the PendSV priority
		(7:0) NVIC_MONITOR_PRIORITY	(7:0) NVIC_MONITOR_PRIORITY	0x0	Configure the monitor priority
0xE000ED24	SCB_SHCSR	(18) USGFAULTENA	(18) USGFAULTENA	0x0	Usage fault handler enable
		(17) BUSFAULTENA	(17) BUSFAULTENA	0x0	Bus fault handler enable
		(16) MEMFAULTENA	(16) MEMFAULTENA	0x0	Memory management fault handler enable
		(15) SVCALLPENDED	(15) SVCALLPENDED	0x0	SVCAll is pending or was started and replaced by a higher priority exception
		(14) BUSFAULTPENDED	(14) BUSFAULTPENDED	0x0	Bus fault is pending or was started and replaced by a higher priority exception
		(13) MEMFAULTPENDED	(13) MEMFAULTPENDED	0x0	Memory management fault is pending or was started and replaced by a higher priority exception
		(12) USGFAULTPENDED	(12) USGFAULTPENDED	0x0	Usage fault is pending or was started and replaced by a higher priority

Address	Register Name	Register Write	Register Read	Default	Description
					exception
		(11) SYSTICKACT	(11) SYSTICKACT	0x0	SYSTICK exception handler is active
		(10) PENDSVACT	(10) PENDSVACT	0x0	PendSV exception handler is active
		(8) MONITORACT	(8) MONITORACT	0x0	Debug monitor exception handler is active
		(7) SVCALLACT	(7) SVCALLACT	0x0	SVCAll exception handler is active
		(3) USGFAULTACT	(3) USGFAULTACT	0x0	Usage fault exception handler is active
		(1) BUSFAULTACT	(1) BUSFAULTACT	0x0	Bus fault exception handler is active
		(0) MEMFAULTACT	(0) MEMFAULTACT	0x0	Memory management fault exception handler is active
0xE000ED28	SCB_CFSR	(25) DIVBYZERO	(25) DIVBYZERO	0x0	Indicates divide by zero has occurred
		(24) UNALIGNED	(24) UNALIGNED	0x0	Indicates unaligned access has occurred
		(19) NOCP	(19) NOCP	0x0	Attempt to execute a coprocessor instruction
		(18) INVPC	(18) INVPC	0x0	Attempt to execute exception with bad value in EXC_RETURN number
		(17) INVSTATE	(17) INVSTATE	0x0	Attempt to switch to invalid (e.g. ARM) state
		(16) UNDEFINSTR	(16) UNDEFINSTR	0x0	Attempt to execute an undefined instruction
		–	(15) BFARVALID	0x0	Indicates if bus fault address register is valid
		(12) STKERR	(12) STKERR	0x0	Indicates stacking error
		(11) UNSTKERR	(11) UNSTKERR	0x0	Indicates unstacking error
		(10) IMPRECISERR	(10) IMPRECISERR	0x0	Indicates imprecise data access violation

Address	Register Name	Register Write	Register Read	Default	Description
		(9) PRECISERR	(9) PRECISERR	0x0	Indicates precise data access violation
		(8) IBUSERR	(8) IBUSERR	0x0	Indicates instruction access violation
		–	(7) MMARVALID	0x0	Indicates if memory management fault address register is valid
		(4) MSTKERR	(4) MSTKERR	0x0	Indicates stacking error
		(3) MUNSTKERR	(3) MUNSTKERR	0x0	Indicates unstacking error
		(1) DACCVIOL	(1) DACCVIOL	0x0	Indicates data access violation
		(0) IACCVIOL	(0) IACCVIOL	0x0	Indicates instruction access violation
0xE000ED2C	SCB_HFSR	(31) DEBUGEVT	(31) DEBUGEVT	0x0	Indicates hard fault is triggered by debug event
		(30) FORCED	(30) FORCED	0x0	Indicates hard fault is taken because of a lower priority (e.g., bus, memory management or usage) fault
		(1) VECTBL	(1) VECTBL	0x0	Indicates hard fault is taken due to failed vector fetch
0xE000ED30	SCB_DFSR	(4) EXTERNAL	(4) EXTERNAL	0x0	Indicates external debug request signal asserted
		(3) VCATCH	(3) VCATCH	0x0	Indicates vector fetch occurred
		(2) DWTTRAP	(2) DWTTRAP	0x0	Indicates DWT match occurred
		(1) BKPT	(1) BKPT	0x0	Indicates BKPT instruction executed
		(0) HALTED	(0) HALTED	0x0	Indicates halt requested by NVIC
0xE000ED34	SCB_MMFAR	–	(31:0) NVIC_MMAR	0x0	
0xE000ED38	SCB_BFAR	–	(31:0) NVIC_BFAR	0x0	

A.42 DEBUG CONTROLLER

Address	Register Name	Register Write	Register Read	Default	Description
0xE000EDF0	DEBUG_DHCSR	(31:16) DBGKEY	–	N/A	Debug key must be written to this field in order to write the rest of the register
		–	(25) S_RESET_ST	0x0	Core has been reset or is being reset. Bit is cleared on read
		–	(24) S_RETIRE_ST	0x0	Instruction is completed since last read. Bit is cleared on read
		–	(19) S_LOCKUP	0x0	Indicates if core is in lockup state
		–	(18) S_SLEEP	0x0	Indicates if core is in sleep mode
		–	(17) S_HALT	0x0	Indicates if core is halted
		–	(16) S_REGRDY	0x0	Indicates register read/write operation is completed
		(5) C_SNAPSTALL	(5) C_SNAPSTALL	0x0	Set to break a stalled memory access
		(3) C_MASKINTS	(3) C_MASKINTS	0x0	Mask interrupts while stepping
		(2) C_STEP	(2) C_STEP	0x0	Single step the processor
		(1) C_HALT	(1) C_HALT	0x0	Halt the processor
		(0) C_DEBUGEN	(0) C_DEBUGEN	0x1	Enable halt mode debugging
0xE000EDF4	DEBUG_DCRSR	(16) REGWnR	–	N/A	Indicates direction of register transfer
		(4:0) REGSEL	–	N/A	Indicates register to be accessed
0xE000EDF8	DEBUG_DCRDR	(31:0) DEBUG_REGDATA	(31:0) DEBUG_REGDATA	0x0	Register read/write data for debugging
0xE000EDFC	DEBUG_DEMCR	(24) TRCENA	(24) TRCENA	0x0	Trace system enable
		(19) MON_REQ	(19) MON_REQ	0x0	Indicates that the debug monitor is caused by a manual pending request rather than a hardware event

Address	Register Name	Register Write	Register Read	Default	Description
		(18) MON_STEP	(18) MON_STEP	0x0	Single step the processor
		(17) MON_PEND	(17) MON_PEND	0x0	Pend the monitor exception request
		(16) MON_EN	(16) MON_EN	0x0	Enable the debug monitor exception
		(10) VC_HARDERR	(10) VC_HARDERR	0x0	Debug trap on hard faults
		(9) VC_INTERR	(9) VC_INTERR	0x0	Debug trap on interrupt service errors
		(8) VC_BUSERR	(8) VC_BUSERR	0x0	Debug trap on bus faults
		(7) VC_STATERR	(7) VC_STATERR	0x0	Debug trap on usage fault state errors
		(6) VC_CHKERR	(6) VC_CHKERR	0x0	Debug trap on fault-enabled checking errors (e.g. unaligned access, divide by zero, etc.)
		(5) VC_NOCPERR	(5) VC_NOCPERR	0x0	Debug trap on usage fault no coprocessor errors
		(4) VC_MMERR	(4) VC_MMERR	0x0	Debug trap on memory management fault
		(0) VC_CORERESSET	(0) VC_CORERESSET	0x0	Debug trap on core reset

A.43 CFX MATH ACCELERATORS I/O AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
X:0x0801 0x20802004	D_X_MATH_DIVISOR_TRG X_MATH_DIVISOR_TRG	(23:0) DIVISOR_TRG	(23:0) DIVISOR_TRG	0x0	The Division accelerator Divisor Input. It triggers division calculation
X:0x0802 0x20802008	D_X_MATH_LOG_EXP_ROOT_LSB X_MATH_LOG_EXP_ROOT_LSB	(23:0) LOG_EXP_ROOT_IN_LSB	(23:0) LOG_EXP_ROOT_IN_LSB	0x0	The Log/Exp/Root Accelerator Input LSBs
X:0x0803 0x2080200C	D_X_MATH_LOG2_OUT_LSB X_MATH_LOG2_OUT_LSB	(23:16) LOG2_OUT_LSB	(23:16) LOG2_OUT_LSB	0x0	The Log2 Accelerator Output LSBs. The MSBs are in Y memory
X:0x0804 0x20802010	D_X_MATH_EXP2_OUT_LSB X_MATH_EXP2_OUT_LSB	(23:0) EXP2_OUT_LSB	(23:0) EXP2_OUT_LSB	0x0	The Exp2 Accelerator Output LSBs. The MSBs are in Y memory
X:0x0805 0x20802014	D_X_MATH_ROOT_OUT_LSB X_MATH_ROOT_OUT_LSB	(23:0) ROOT_OUT_LSB	(23:0) ROOT_OUT_LSB	0x0	The Root Accelerator Output LSBs. The MSBs are in Y memory
X:0x0806 0x20802018	D_X_MATH_LOG2_IN_LSB X_MATH_LOG2_IN_LSB	(23:0) LOG2_IN_LSB	–	N/A	The Log2 Accelerator Input LSBs. The MSBs are in Y memory
X:0x0807 0x2080201C	D_X_MATH_ABS_LOG2_IN_LSB X_MATH_ABS_LOG2_IN_LSB	(23:0) ABS_LOG2_IN_LSB	–	N/A	The Log2 Accelerator Input LSBs. The MSBs are in Y memory
X:0x0808 0x20802020	D_X_MATH_EXP2_IN_LSB X_MATH_EXP2_IN_LSB	(23:16) EXP2_IN_LSB	–	N/A	The Exp2 Accelerator Input LSBs. The MSBs are in Y memory
X:0x0809 0x20802024	D_X_MATH_ABSNEG_EXP2_IN_LSB X_MATH_ABSNEG_EXP2_IN_LSB	(23:16) ABSNEG_EXP2_IN_LSB	–	N/A	The Exp2 Accelerator Input LSBs. The MSBs are in Y memory

Address	Register Name	Register Write	Register Read	Default	Description
	LSB				
X:0x080A 0x20802028	D_X_MATH_ROOT2_IN_LSB X_MATH_ROOT2_IN_LSB	(23:0) ROOT2_IN_LSB	–	N/A	The Root2 Accelerator Input LSBs. The MSBs are in Y memory
X:0x080B 0x2080202C	D_X_MATH_ABS_ROOT2_IN_LSB X_MATH_ABS_ROOT2_IN_LSB	(23:0) ABS_ROOT2_IN_LSB	–	N/A	The Root2 Accelerator Input LSBs. The MSBs are in Y memory
X:0x080C 0x20802030	D_X_MATH_ROOT3_IN_LSB X_MATH_ROOT3_IN_LSB	(23:0) ROOT3_IN_LSB	–	N/A	The Root3 Accelerator Input LSBs. The MSBs are in Y memory
X:0x080E 0x20802038	D_X_MATH_ROOT4_IN_LSB X_MATH_ROOT4_IN_LSB	(23:0) ROOT4_IN_LSB	–	N/A	The Root4 Accelerator Input LSBs. The MSBs are in Y memory
X:0x080F 0x2080203C	D_X_MATH_ABS_ROOT4_IN_LSB X_MATH_ABS_ROOT4_IN_LSB	(23:0) ABS_ROOT4_IN_LSB	–	N/A	The Root4 Accelerator Input LSBs. The MSBs are in Y memory
X:0x0810 0x20802040	D_X_MATH_DIVISOR X_MATH_DIVISOR	(23:0) DIVISOR_NOTRG	–	N/A	The Division accelerator Divisor Input. It does not trigger division calculation
X:0x0811 0x20802044	D_X_MATH_DIV_OUT_LSB X_MATH_DIV_OUT_LSB	–	(23:0) DIV_OUT_LSB	0x0	The Division Accelerator Output LSBs. The MSBs are in Y memory

A.44 FIFO A0 CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
X:0x1000 0x20804000	D_FIFO_A0_0_START FIFO_A0_0_START	(12:5) OFFSET	(12:5) OFFSET	0x0	Start address for FIFO A0
X:0x1001 0x20804004	D_FIFO_A0_0_END FIFO_A0_0_END	(12:5) OFFSET	(12:5) OFFSET	0x0	End address (+1) for FIFO A0
X:0x1002 0x20804008	D_FIFO_A0_0_SIZE FIFO_A0_0_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO A0
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A0
X:0x1003 0x2080400C	D_FIFO_A0_0_AUTO_MUTE FIFO_A0_0_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A0
X:0x1004 0x20804010	D_FIFO_A0_0_BASE_PTR FIFO_A0_0_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO A0
X:0x1005 0x20804014	D_FIFO_A0_0_IOBLOCK_PTR FIFO_A0_0_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO A0
X:0x1008 0x20804020	D_FIFO_A0_0_BLOCK_SIZE FIFO_A0_0_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO A0
X:0x100C 0x20804030	D_FIFO_A0_0_BLOCK_CNT FIFO_A0_0_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO A0
X:0x100E 0x20804038	D_FIFO_A0_0_ACCESS FIFO_A0_0_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A0

Address	Register Name	Register Write	Register Read	Default	Description
X:0x1010 0x20804040	D_FIFO_A0_1_START FIFO_A0_1_START	(12:5) OFFSET	(12:5) OFFSET	0x0	Start address for FIFO A0
X:0x1011 0x20804044	D_FIFO_A0_1_END FIFO_A0_1_END	(12:5) OFFSET	(12:5) OFFSET	0x0	End address (+1) for FIFO A0
X:0x1012 0x20804048	D_FIFO_A0_1_SIZE FIFO_A0_1_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO A0
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A0
X:0x1013 0x2080404C	D_FIFO_A0_1_AUTO_MUTE FIFO_A0_1_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A0
X:0x1014 0x20804050	D_FIFO_A0_1_BASE_PTR FIFO_A0_1_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO A0
X:0x1015 0x20804054	D_FIFO_A0_1_IOBLOCK_PTR FIFO_A0_1_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO A0
X:0x1018 0x20804060	D_FIFO_A0_1_BLOCK_SIZE FIFO_A0_1_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO A0
X:0x101C 0x20804070	D_FIFO_A0_1_BLOCK_CNT FIFO_A0_1_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO A0
X:0x101E 0x20804078	D_FIFO_A0_1_ACCESS FIFO_A0_1_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A0
X:0x1020 0x20804080	D_FIFO_A0_2_START FIFO_A0_2_START	(12:5) OFFSET	(12:5) OFFSET	0x0	Start address for FIFO A0

Address	Register Name	Register Write	Register Read	Default	Description
X:0x1021 0x20804084	D_FIFO_A0_2_END FIFO_A0_2_END	(12:5) OFFSET	(12:5) OFFSET	0x0	End address (+1) for FIFO A0
X:0x1022 0x20804088	D_FIFO_A0_2_SIZE FIFO_A0_2_SIZE	-	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO A0
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A0
X:0x1023 0x2080408C	D_FIFO_A0_2_AUTO_MUTE FIFO_A0_2_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A0
X:0x1024 0x20804090	D_FIFO_A0_2_BASE_PTR FIFO_A0_2_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO A0
X:0x1025 0x20804094	D_FIFO_A0_2_IOBLOCK_PTR FIFO_A0_2_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO A0
X:0x1028 0x208040A0	D_FIFO_A0_2_BLOCK_SIZE FIFO_A0_2_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO A0
X:0x102C 0x208040B0	D_FIFO_A0_2_BLOCK_CNT FIFO_A0_2_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO A0
X:0x102E 0x208040B8	D_FIFO_A0_2_ACCESS FIFO_A0_2_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A0
X:0x1030 0x208040C0	D_FIFO_A0_3_START FIFO_A0_3_START	(12:5) OFFSET	(12:5) OFFSET	0x0	Start address for FIFO A0
X:0x1031 0x208040C4	D_FIFO_A0_3_END FIFO_A0_3_END	(12:5) OFFSET	(12:5) OFFSET	0x0	End address (+1) for FIFO A0

Address	Register Name	Register Write	Register Read	Default	Description
X:0x1032 0x208040C8	D_FIFO_A0_3_SIZE FIFO_A0_3_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO A0
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A0
X:0x1033 0x208040CC	D_FIFO_A0_3_AUTO_MUTE FIFO_A0_3_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A0
X:0x1034 0x208040D0	D_FIFO_A0_3_BASE_PTR FIFO_A0_3_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO A0
X:0x1035 0x208040D4	D_FIFO_A0_3_IOBLOCK_PTR FIFO_A0_3_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO A0
X:0x1038 0x208040E0	D_FIFO_A0_3_BLOCK_SIZE FIFO_A0_3_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO A0
X:0x103C 0x208040F0	D_FIFO_A0_3_BLOCK_CNT FIFO_A0_3_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO A0
X:0x103E 0x208040F8	D_FIFO_A0_3_ACCESS FIFO_A0_3_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A0
X:0x1040 0x20804100	D_FIFO_A0_4_START FIFO_A0_4_START	(12:5) OFFSET	(12:5) OFFSET	0x0	Start address for FIFO A0
X:0x1041 0x20804104	D_FIFO_A0_4_END FIFO_A0_4_END	(12:5) OFFSET	(12:5) OFFSET	0x0	End address (+1) for FIFO A0
X:0x1042 0x20804108	D_FIFO_A0_4_SIZE FIFO_A0_4_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO A0

Address	Register Name	Register Write	Register Read	Default	Description
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A0
X:0x1043 0x2080410C	D_FIFO_A0_4_AUTO_MUTE FIFO_A0_4_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A0
X:0x1044 0x20804110	D_FIFO_A0_4_BASE_PTR FIFO_A0_4_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO A0
X:0x1045 0x20804114	D_FIFO_A0_4_IOBLOCK_PTR FIFO_A0_4_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO A0
X:0x1048 0x20804120	D_FIFO_A0_4_BLOCK_SIZE FIFO_A0_4_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO A0
X:0x104C 0x20804130	D_FIFO_A0_4_BLOCK_CNT FIFO_A0_4_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO A0
X:0x104E 0x20804138	D_FIFO_A0_4_ACCESS FIFO_A0_4_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A0
X:0x1050 0x20804140	D_FIFO_A0_5_START FIFO_A0_5_START	(12:5) OFFSET	(12:5) OFFSET	0x0	Start address for FIFO A0
X:0x1051 0x20804144	D_FIFO_A0_5_END FIFO_A0_5_END	(12:5) OFFSET	(12:5) OFFSET	0x0	End address(+1) for FIFO A0
X:0x1052 0x20804148	D_FIFO_A0_5_SIZE FIFO_A0_5_SIZE	—	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO A0
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A0
X:0x1053	D_FIFO_A0_5_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_	0x0	FIFO auto mute enable for FIFO A0

Address	Register Name	Register Write	Register Read	Default	Description
0x2080414C	FIFO_A0_5_AUTO_MUTE		ENABLE		
X:0x1054 0x20804150	D_FIFO_A0_5_BASE_PTR FIFO_A0_5_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO A0
X:0x1055 0x20804154	D_FIFO_A0_5_IOBLOCK_PTR FIFO_A0_5_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO A0
X:0x1058 0x20804160	D_FIFO_A0_5_BLOCK_SIZE FIFO_A0_5_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO A0
X:0x105C 0x20804170	D_FIFO_A0_5_BLOCK_CNT FIFO_A0_5_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO A0
X:0x105E 0x20804178	D_FIFO_A0_5_ACCESS FIFO_A0_5_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A0
X:0x1060 0x20804180	D_FIFO_A0_6_START FIFO_A0_6_START	(12:5) OFFSET	(12:5) OFFSET	0x0	Start address for FIFO A0
X:0x1061 0x20804184	D_FIFO_A0_6_END FIFO_A0_6_END	(12:5) OFFSET	(12:5) OFFSET	0x0	End address (+1) for FIFO A0
X:0x1062 0x20804188	D_FIFO_A0_6_SIZE FIFO_A0_6_SIZE	-	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO A0
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A0
X:0x1063 0x2080418C	D_FIFO_A0_6_AUTO_MUTE FIFO_A0_6_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A0
X:0x1064	D_FIFO_A0_6_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO A0

Address	Register Name	Register Write	Register Read	Default	Description
0x20804190	FIFO_A0_6_BASE_PTR				
X:0x1065 0x20804194	D_FIFO_A0_6_IOBLOCK_PTR FIFO_A0_6_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO A0
X:0x1068 0x208041A0	D_FIFO_A0_6_BLOCK_SIZE FIFO_A0_6_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO A0
X:0x106C 0x208041B0	D_FIFO_A0_6_BLOCK_CNT FIFO_A0_6_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO A0
X:0x106E 0x208041B8	D_FIFO_A0_6_ACCESS FIFO_A0_6_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A0
X:0x1070 0x208041C0	D_FIFO_A0_7_START FIFO_A0_7_START	(12:5) OFFSET	(12:5) OFFSET	0x0	Start address for FIFO A0
X:0x1071 0x208041C4	D_FIFO_A0_7_END FIFO_A0_7_END	(12:5) OFFSET	(12:5) OFFSET	0x0	End address (+1) for FIFO A0
X:0x1072 0x208041C8	D_FIFO_A0_7_SIZE FIFO_A0_7_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO A0
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A0
X:0x1073 0x208041CC	D_FIFO_A0_7_AUTO_MUTE FIFO_A0_7_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A0
X:0x1074 0x208041D0	D_FIFO_A0_7_BASE_PTR FIFO_A0_7_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO A0
X:0x1075	D_FIFO_A0_7_IOBLOCK_	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO A0

Address	Register Name	Register Write	Register Read	Default	Description
0x208041D4	PTR FIFO_A0_7_IOBLOCK_PTR				
X:0x1078 0x208041E0	D_FIFO_A0_7_BLOCK_SIZE FIFO_A0_7_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO A0
X:0x107C 0x208041F0	D_FIFO_A0_7_BLOCK_CNT FIFO_A0_7_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO A0
X:0x107E 0x208041F8	D_FIFO_A0_7_ACCESS FIFO_A0_7_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A0

A.45 FIFO A1 CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
X:0x1080 0x20804200	D_FIFO_A1_0_START FIFO_A1_0_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO A1
X:0x1081 0x20804204	D_FIFO_A1_0_END FIFO_A1_0_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO A1
X:0x1082 0x20804208	D_FIFO_A1_0_SIZE FIFO_A1_0_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO A1
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A1
X:0x1083 0x2080420C	D_FIFO_A1_0_AUTO_MUTE FIFO_A1_0_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A1
X:0x1084 0x20804210	D_FIFO_A1_0_BASE_PTR FIFO_A1_0_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO A1
X:0x1085 0x20804214	D_FIFO_A1_0_IOBLOCK_PTR FIFO_A1_0_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO A1
X:0x1088 0x20804220	D_FIFO_A1_0_BLOCK_SIZE FIFO_A1_0_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO A1
X:0x108C 0x20804230	D_FIFO_A1_0_BLOCK_CNT FIFO_A1_0_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO A1
X:0x108E 0x20804238	D_FIFO_A1_0_ACCESS FIFO_A1_0_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A1

Address	Register Name	Register Write	Register Read	Default	Description
X:0x1090 0x20804240	D_FIFO_A1_1_START FIFO_A1_1_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO A1
X:0x1091 0x20804244	D_FIFO_A1_1_END FIFO_A1_1_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO A1
X:0x1092 0x20804248	D_FIFO_A1_1_SIZE FIFO_A1_1_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO A1
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A1
X:0x1093 0x2080424C	D_FIFO_A1_1_AUTO_MUTE FIFO_A1_1_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A1
X:0x1094 0x20804250	D_FIFO_A1_1_BASE_PTR FIFO_A1_1_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO A1
X:0x1095 0x20804254	D_FIFO_A1_1_IOBLOCK_PTR FIFO_A1_1_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO A1
X:0x1098 0x20804260	D_FIFO_A1_1_BLOCK_SIZE FIFO_A1_1_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO A1
X:0x109C 0x20804270	D_FIFO_A1_1_BLOCK_CNT FIFO_A1_1_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO A1
X:0x109E 0x20804278	D_FIFO_A1_1_ACCESS FIFO_A1_1_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A1
X:0x10A0 0x20804280	D_FIFO_A1_2_START FIFO_A1_2_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO A1

Address	Register Name	Register Write	Register Read	Default	Description
X:0x10A1 0x20804284	D_FIFO_A1_2_END FIFO_A1_2_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO A1
X:0x10A2 0x20804288	D_FIFO_A1_2_SIZE FIFO_A1_2_SIZE	-	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO A1
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A1
X:0x10A3 0x2080428C	D_FIFO_A1_2_AUTO_MUTE FIFO_A1_2_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A1
X:0x10A4 0x20804290	D_FIFO_A1_2_BASE_PTR FIFO_A1_2_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO A1
X:0x10A5 0x20804294	D_FIFO_A1_2_IOBLOCK_PTR FIFO_A1_2_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO A1
X:0x10A8 0x208042A0	D_FIFO_A1_2_BLOCK_SIZE FIFO_A1_2_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO A1
X:0x10AC 0x208042B0	D_FIFO_A1_2_BLOCK_CNT FIFO_A1_2_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO A1
X:0x10AE 0x208042B8	D_FIFO_A1_2_ACCESS FIFO_A1_2_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A1
X:0x10B0 0x208042C0	D_FIFO_A1_3_START FIFO_A1_3_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO A1
X:0x10B1 0x208042C4	D_FIFO_A1_3_END FIFO_A1_3_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO A1

Address	Register Name	Register Write	Register Read	Default	Description
X:0x10B2 0x208042C8	D_FIFO_A1_3_SIZE FIFO_A1_3_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO A1
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A1
X:0x10B3 0x208042CC	D_FIFO_A1_3_AUTO_MUTE FIFO_A1_3_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A1
X:0x10B4 0x208042D0	D_FIFO_A1_3_BASE_PTR FIFO_A1_3_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO A1
X:0x10B5 0x208042D4	D_FIFO_A1_3_IOBLOCK_PTR FIFO_A1_3_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO A1
X:0x10B8 0x208042E0	D_FIFO_A1_3_BLOCK_SIZE FIFO_A1_3_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO A1
X:0x10BC 0x208042F0	D_FIFO_A1_3_BLOCK_CNT FIFO_A1_3_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO A1
X:0x10BE 0x208042F8	D_FIFO_A1_3_ACCESS FIFO_A1_3_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A1
X:0x10C0 0x20804300	D_FIFO_A1_4_START FIFO_A1_4_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO A1
X:0x10C1 0x20804304	D_FIFO_A1_4_END FIFO_A1_4_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO A1
X:0x10C2 0x20804308	D_FIFO_A1_4_SIZE FIFO_A1_4_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO A1

Address	Register Name	Register Write	Register Read	Default	Description
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A1
X:0x10C3 0x2080430C	D_FIFO_A1_4_AUTO_MUTE FIFO_A1_4_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A1
X:0x10C4 0x20804310	D_FIFO_A1_4_BASE_PTR FIFO_A1_4_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO A1
X:0x10C5 0x20804314	D_FIFO_A1_4_IOBLOCK_PTR FIFO_A1_4_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO A1
X:0x10C8 0x20804320	D_FIFO_A1_4_BLOCK_SIZE FIFO_A1_4_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO A1
X:0x10CC 0x20804330	D_FIFO_A1_4_BLOCK_CNT FIFO_A1_4_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO A1
X:0x10CE 0x20804338	D_FIFO_A1_4_ACCESS FIFO_A1_4_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A1
X:0x10D0 0x20804340	D_FIFO_A1_5_START FIFO_A1_5_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO A1
X:0x10D1 0x20804344	D_FIFO_A1_5_END FIFO_A1_5_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address(+1) for FIFO A1
X:0x10D2 0x20804348	D_FIFO_A1_5_SIZE FIFO_A1_5_SIZE	—	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO A1
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A1
X:0x10D3	D_FIFO_A1_5_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_	0x0	FIFO auto mute enable for FIFO A1

Address	Register Name	Register Write	Register Read	Default	Description
0x2080434C	FIFO_A1_5_AUTO_MUTE		ENABLE		
X:0x10D4 0x20804350	D_FIFO_A1_5_BASE_PTR FIFO_A1_5_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO A1
X:0x10D5 0x20804354	D_FIFO_A1_5_IOBLOCK_PTR FIFO_A1_5_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO A1
X:0x10D8 0x20804360	D_FIFO_A1_5_BLOCK_SIZE FIFO_A1_5_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO A1
X:0x10DC 0x20804370	D_FIFO_A1_5_BLOCK_CNT FIFO_A1_5_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO A1
X:0x10DE 0x20804378	D_FIFO_A1_5_ACCESS FIFO_A1_5_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A1
X:0x10E0 0x20804380	D_FIFO_A1_6_START FIFO_A1_6_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO A1
X:0x10E1 0x20804384	D_FIFO_A1_6_END FIFO_A1_6_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO A1
X:0x10E2 0x20804388	D_FIFO_A1_6_SIZE FIFO_A1_6_SIZE	-	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO A1
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A1
X:0x10E3 0x2080438C	D_FIFO_A1_6_AUTO_MUTE FIFO_A1_6_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A1
X:0x10E4	D_FIFO_A1_6_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO A1

Address	Register Name	Register Write	Register Read	Default	Description
0x20804390	FIFO_A1_6_BASE_PTR				
X:0x10E5 0x20804394	D_FIFO_A1_6_IOBLOCK_PTR FIFO_A1_6_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO A1
X:0x10E8 0x208043A0	D_FIFO_A1_6_BLOCK_SIZE FIFO_A1_6_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO A1
X:0x10EC 0x208043B0	D_FIFO_A1_6_BLOCK_CNT FIFO_A1_6_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO A1
X:0x10EE 0x208043B8	D_FIFO_A1_6_ACCESS FIFO_A1_6_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A1
X:0x10F0 0x208043C0	D_FIFO_A1_7_START FIFO_A1_7_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO A1
X:0x10F1 0x208043C4	D_FIFO_A1_7_END FIFO_A1_7_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO A1
X:0x10F2 0x208043C8	D_FIFO_A1_7_SIZE FIFO_A1_7_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO A1
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO A1
X:0x10F3 0x208043CC	D_FIFO_A1_7_AUTO_MUTE FIFO_A1_7_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO A1
X:0x10F4 0x208043D0	D_FIFO_A1_7_BASE_PTR FIFO_A1_7_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO A1
X:0x10F5	D_FIFO_A1_7_IOBLOCK_	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO A1

Address	Register Name	Register Write	Register Read	Default	Description
0x208043D4	PTR FIFO_A1_7_IOBLOCK_PTR				
X:0x10F8 0x208043E0	D_FIFO_A1_7_BLOCK_SIZE FIFO_A1_7_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO A1
X:0x10FC 0x208043F0	D_FIFO_A1_7_BLOCK_CNT FIFO_A1_7_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO A1
X:0x10FE 0x208043F8	D_FIFO_A1_7_ACCESS FIFO_A1_7_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO A1

A.46 FIFO B0 CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
X:0x1100 0x20804400	D_FIFO_B0_0_START FIFO_B0_0_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO B0
X:0x1101 0x20804404	D_FIFO_B0_0_END FIFO_B0_0_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO B0
X:0x1102 0x20804408	D_FIFO_B0_0_SIZE FIFO_B0_0_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO B0
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B0
X:0x1103 0x2080440C	D_FIFO_B0_0_AUTO_MUTE FIFO_B0_0_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B0
X:0x1104 0x20804410	D_FIFO_B0_0_BASE_PTR FIFO_B0_0_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO B0
X:0x1105 0x20804414	D_FIFO_B0_0_IOBLOCK_PTR FIFO_B0_0_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO B0
X:0x1108 0x20804420	D_FIFO_B0_0_BLOCK_SIZE FIFO_B0_0_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO B0
X:0x110C 0x20804430	D_FIFO_B0_0_BLOCK_CNT FIFO_B0_0_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO B0
X:0x110E 0x20804438	D_FIFO_B0_0_ACCESS FIFO_B0_0_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B0

Address	Register Name	Register Write	Register Read	Default	Description
X:0x1110 0x20804440	D_FIFO_B0_1_START FIFO_B0_1_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO B0
X:0x1111 0x20804444	D_FIFO_B0_1_END FIFO_B0_1_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO B0
X:0x1112 0x20804448	D_FIFO_B0_1_SIZE FIFO_B0_1_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO B0
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B0
X:0x1113 0x2080444C	D_FIFO_B0_1_AUTO_MUTE FIFO_B0_1_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B0
X:0x1114 0x20804450	D_FIFO_B0_1_BASE_PTR FIFO_B0_1_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO B0
X:0x1115 0x20804454	D_FIFO_B0_1_IOBLOCK_PTR FIFO_B0_1_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO B0
X:0x1118 0x20804460	D_FIFO_B0_1_BLOCK_SIZE FIFO_B0_1_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO B0
X:0x111C 0x20804470	D_FIFO_B0_1_BLOCK_CNT FIFO_B0_1_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO B0
X:0x111E 0x20804478	D_FIFO_B0_1_ACCESS FIFO_B0_1_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B0
X:0x1120 0x20804480	D_FIFO_B0_2_START FIFO_B0_2_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO B0

Address	Register Name	Register Write	Register Read	Default	Description
X:0x1121 0x20804484	D_FIFO_B0_2_END FIFO_B0_2_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO B0
X:0x1122 0x20804488	D_FIFO_B0_2_SIZE FIFO_B0_2_SIZE	-	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO B0
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B0
X:0x1123 0x2080448C	D_FIFO_B0_2_AUTO_MUTE FIFO_B0_2_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B0
X:0x1124 0x20804490	D_FIFO_B0_2_BASE_PTR FIFO_B0_2_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO B0
X:0x1125 0x20804494	D_FIFO_B0_2_IOBLOCK_PTR FIFO_B0_2_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO B0
X:0x1128 0x208044A0	D_FIFO_B0_2_BLOCK_SIZE FIFO_B0_2_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO B0
X:0x112C 0x208044B0	D_FIFO_B0_2_BLOCK_CNT FIFO_B0_2_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO B0
X:0x112E 0x208044B8	D_FIFO_B0_2_ACCESS FIFO_B0_2_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B0
X:0x1130 0x208044C0	D_FIFO_B0_3_START FIFO_B0_3_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO B0
X:0x1131 0x208044C4	D_FIFO_B0_3_END FIFO_B0_3_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO B0

Address	Register Name	Register Write	Register Read	Default	Description
X:0x1132 0x208044C8	D_FIFO_B0_3_SIZE FIFO_B0_3_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO B0
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B0
X:0x1133 0x208044CC	D_FIFO_B0_3_AUTO_MUTE FIFO_B0_3_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B0
X:0x1134 0x208044D0	D_FIFO_B0_3_BASE_PTR FIFO_B0_3_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO B0
X:0x1135 0x208044D4	D_FIFO_B0_3_IOBLOCK_PTR FIFO_B0_3_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO B0
X:0x1138 0x208044E0	D_FIFO_B0_3_BLOCK_SIZE FIFO_B0_3_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO B0
X:0x113C 0x208044F0	D_FIFO_B0_3_BLOCK_CNT FIFO_B0_3_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO B0
X:0x113E 0x208044F8	D_FIFO_B0_3_ACCESS FIFO_B0_3_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B0
X:0x1140 0x20804500	D_FIFO_B0_4_START FIFO_B0_4_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO B0
X:0x1141 0x20804504	D_FIFO_B0_4_END FIFO_B0_4_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO B0
X:0x1142 0x20804508	D_FIFO_B0_4_SIZE FIFO_B0_4_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO B0

Address	Register Name	Register Write	Register Read	Default	Description
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B0
X:0x1143 0x2080450C	D_FIFO_B0_4_AUTO_MUTE FIFO_B0_4_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B0
X:0x1144 0x20804510	D_FIFO_B0_4_BASE_PTR FIFO_B0_4_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO B0
X:0x1145 0x20804514	D_FIFO_B0_4_IOBLOCK_PTR FIFO_B0_4_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO B0
X:0x1148 0x20804520	D_FIFO_B0_4_BLOCK_SIZE FIFO_B0_4_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO B0
X:0x114C 0x20804530	D_FIFO_B0_4_BLOCK_CNT FIFO_B0_4_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO B0
X:0x114E 0x20804538	D_FIFO_B0_4_ACCESS FIFO_B0_4_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B0
X:0x1150 0x20804540	D_FIFO_B0_5_START FIFO_B0_5_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO B0
X:0x1151 0x20804544	D_FIFO_B0_5_END FIFO_B0_5_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address(+1) for FIFO B0
X:0x1152 0x20804548	D_FIFO_B0_5_SIZE FIFO_B0_5_SIZE	—	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO B0
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B0
X:0x1153	D_FIFO_B0_5_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_	0x0	FIFO auto mute enable for FIFO B0

Address	Register Name	Register Write	Register Read	Default	Description
0x2080454C	FIFO_B0_5_AUTO_MUTE		ENABLE		
X:0x1154 0x20804550	D_FIFO_B0_5_BASE_PTR FIFO_B0_5_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO B0
X:0x1155 0x20804554	D_FIFO_B0_5_IOBLOCK_PTR FIFO_B0_5_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO B0
X:0x1158 0x20804560	D_FIFO_B0_5_BLOCK_SIZE FIFO_B0_5_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO B0
X:0x115C 0x20804570	D_FIFO_B0_5_BLOCK_CNT FIFO_B0_5_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO B0
X:0x115E 0x20804578	D_FIFO_B0_5_ACCESS FIFO_B0_5_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B0
X:0x1160 0x20804580	D_FIFO_B0_6_START FIFO_B0_6_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO B0
X:0x1161 0x20804584	D_FIFO_B0_6_END FIFO_B0_6_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO B0
X:0x1162 0x20804588	D_FIFO_B0_6_SIZE FIFO_B0_6_SIZE	-	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO B0
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B0
X:0x1163 0x2080458C	D_FIFO_B0_6_AUTO_MUTE FIFO_B0_6_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B0
X:0x1164	D_FIFO_B0_6_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO B0

Address	Register Name	Register Write	Register Read	Default	Description
0x20804590	FIFO_B0_6_BASE_PTR				
X:0x1165 0x20804594	D_FIFO_B0_6_IOBLOCK_PTR FIFO_B0_6_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO B0
X:0x1168 0x208045A0	D_FIFO_B0_6_BLOCK_SIZE FIFO_B0_6_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO B0
X:0x116C 0x208045B0	D_FIFO_B0_6_BLOCK_CNT FIFO_B0_6_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO B0
X:0x116E 0x208045B8	D_FIFO_B0_6_ACCESS FIFO_B0_6_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B0
X:0x1170 0x208045C0	D_FIFO_B0_7_START FIFO_B0_7_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO B0
X:0x1171 0x208045C4	D_FIFO_B0_7_END FIFO_B0_7_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO B0
X:0x1172 0x208045C8	D_FIFO_B0_7_SIZE FIFO_B0_7_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO B0
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B0
X:0x1173 0x208045CC	D_FIFO_B0_7_AUTO_MUTE FIFO_B0_7_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B0
X:0x1174 0x208045D0	D_FIFO_B0_7_BASE_PTR FIFO_B0_7_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO B0
X:0x1175	D_FIFO_B0_7_IOBLOCK_	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO B0

Address	Register Name	Register Write	Register Read	Default	Description
0x208045D4	PTR FIFO_B0_7_IOBLOCK_PTR				
X:0x1178 0x208045E0	D_FIFO_B0_7_BLOCK_SIZE FIFO_B0_7_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO B0
X:0x117C 0x208045F0	D_FIFO_B0_7_BLOCK_CNT FIFO_B0_7_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO B0
X:0x117E 0x208045F8	D_FIFO_B0_7_ACCESS FIFO_B0_7_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B0

A.47 FIFO B1 CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
X:0x1180 0x20804600	D_FIFO_B1_0_START FIFO_B1_0_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO B1
X:0x1181 0x20804604	D_FIFO_B1_0_END FIFO_B1_0_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO B1
X:0x1182 0x20804608	D_FIFO_B1_0_SIZE FIFO_B1_0_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO B1
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B1
X:0x1183 0x2080460C	D_FIFO_B1_0_AUTO_MUTE FIFO_B1_0_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B1
X:0x1184 0x20804610	D_FIFO_B1_0_BASE_PTR FIFO_B1_0_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO B1
X:0x1185 0x20804614	D_FIFO_B1_0_IOBLOCK_PTR FIFO_B1_0_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO B1
X:0x1188 0x20804620	D_FIFO_B1_0_BLOCK_SIZE FIFO_B1_0_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO B1
X:0x118C 0x20804630	D_FIFO_B1_0_BLOCK_CNT FIFO_B1_0_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO B1
X:0x118E 0x20804638	D_FIFO_B1_0_ACCESS FIFO_B1_0_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B1

Address	Register Name	Register Write	Register Read	Default	Description
X:0x1190 0x20804640	D_FIFO_B1_1_START FIFO_B1_1_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO B1
X:0x1191 0x20804644	D_FIFO_B1_1_END FIFO_B1_1_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO B1
X:0x1192 0x20804648	D_FIFO_B1_1_SIZE FIFO_B1_1_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO B1
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B1
X:0x1193 0x2080464C	D_FIFO_B1_1_AUTO_MUTE FIFO_B1_1_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B1
X:0x1194 0x20804650	D_FIFO_B1_1_BASE_PTR FIFO_B1_1_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO B1
X:0x1195 0x20804654	D_FIFO_B1_1_IOBLOCK_PTR FIFO_B1_1_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO B1
X:0x1198 0x20804660	D_FIFO_B1_1_BLOCK_SIZE FIFO_B1_1_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO B1
X:0x119C 0x20804670	D_FIFO_B1_1_BLOCK_CNT FIFO_B1_1_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO B1
X:0x119E 0x20804678	D_FIFO_B1_1_ACCESS FIFO_B1_1_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B1
X:0x11A0 0x20804680	D_FIFO_B1_2_START FIFO_B1_2_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO B1

Address	Register Name	Register Write	Register Read	Default	Description
X:0x11A1 0x20804684	D_FIFO_B1_2_END FIFO_B1_2_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO B1
X:0x11A2 0x20804688	D_FIFO_B1_2_SIZE FIFO_B1_2_SIZE	-	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO B1
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B1
X:0x11A3 0x2080468C	D_FIFO_B1_2_AUTO_MUTE FIFO_B1_2_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B1
X:0x11A4 0x20804690	D_FIFO_B1_2_BASE_PTR FIFO_B1_2_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO B1
X:0x11A5 0x20804694	D_FIFO_B1_2_IOBLOCK_PTR FIFO_B1_2_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO B1
X:0x11A8 0x208046A0	D_FIFO_B1_2_BLOCK_SIZE FIFO_B1_2_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO B1
X:0x11AC 0x208046B0	D_FIFO_B1_2_BLOCK_CNT FIFO_B1_2_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO B1
X:0x11AE 0x208046B8	D_FIFO_B1_2_ACCESS FIFO_B1_2_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B1
X:0x11B0 0x208046C0	D_FIFO_B1_3_START FIFO_B1_3_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO B1
X:0x11B1 0x208046C4	D_FIFO_B1_3_END FIFO_B1_3_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO B1

Address	Register Name	Register Write	Register Read	Default	Description
X:0x11B2 0x208046C8	D_FIFO_B1_3_SIZE FIFO_B1_3_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO B1
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B1
X:0x11B3 0x208046CC	D_FIFO_B1_3_AUTO_MUTE FIFO_B1_3_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B1
X:0x11B4 0x208046D0	D_FIFO_B1_3_BASE_PTR FIFO_B1_3_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO B1
X:0x11B5 0x208046D4	D_FIFO_B1_3_IOBLOCK_PTR FIFO_B1_3_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO B1
X:0x11B8 0x208046E0	D_FIFO_B1_3_BLOCK_SIZE FIFO_B1_3_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO B1
X:0x11BC 0x208046F0	D_FIFO_B1_3_BLOCK_CNT FIFO_B1_3_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO B1
X:0x11BE 0x208046F8	D_FIFO_B1_3_ACCESS FIFO_B1_3_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B1
X:0x11C0 0x20804700	D_FIFO_B1_4_START FIFO_B1_4_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO B1
X:0x11C1 0x20804704	D_FIFO_B1_4_END FIFO_B1_4_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO B1
X:0x11C2 0x20804708	D_FIFO_B1_4_SIZE FIFO_B1_4_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO B1

Address	Register Name	Register Write	Register Read	Default	Description
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B1
X:0x11C3 0x2080470C	D_FIFO_B1_4_AUTO_MUTE FIFO_B1_4_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B1
X:0x11C4 0x20804710	D_FIFO_B1_4_BASE_PTR FIFO_B1_4_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO B1
X:0x11C5 0x20804714	D_FIFO_B1_4_IOBLOCK_PTR FIFO_B1_4_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO B1
X:0x11C8 0x20804720	D_FIFO_B1_4_BLOCK_SIZE FIFO_B1_4_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO B1
X:0x11CC 0x20804730	D_FIFO_B1_4_BLOCK_CNT FIFO_B1_4_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO B1
X:0x11CE 0x20804738	D_FIFO_B1_4_ACCESS FIFO_B1_4_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B1
X:0x11D0 0x20804740	D_FIFO_B1_5_START FIFO_B1_5_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO B1
X:0x11D1 0x20804744	D_FIFO_B1_5_END FIFO_B1_5_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address(+1) for FIFO B1
X:0x11D2 0x20804748	D_FIFO_B1_5_SIZE FIFO_B1_5_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO B1
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B1
X:0x11D3	D_FIFO_B1_5_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_	0x0	FIFO auto mute enable for FIFO B1

Address	Register Name	Register Write	Register Read	Default	Description
0x2080474C	FIFO_B1_5_AUTO_MUTE		ENABLE		
X:0x11D4 0x20804750	D_FIFO_B1_5_BASE_PTR FIFO_B1_5_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO B1
X:0x11D5 0x20804754	D_FIFO_B1_5_IOBLOCK_PTR FIFO_B1_5_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO B1
X:0x11D8 0x20804760	D_FIFO_B1_5_BLOCK_SIZE FIFO_B1_5_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO B1
X:0x11DC 0x20804770	D_FIFO_B1_5_BLOCK_CNT FIFO_B1_5_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO B1
X:0x11DE 0x20804778	D_FIFO_B1_5_ACCESS FIFO_B1_5_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B1
X:0x11E0 0x20804780	D_FIFO_B1_6_START FIFO_B1_6_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO B1
X:0x11E1 0x20804784	D_FIFO_B1_6_END FIFO_B1_6_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO B1
X:0x11E2 0x20804788	D_FIFO_B1_6_SIZE FIFO_B1_6_SIZE	-	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO B1
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B1
X:0x11E3 0x2080478C	D_FIFO_B1_6_AUTO_MUTE FIFO_B1_6_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B1
X:0x11E4	D_FIFO_B1_6_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO B1

Address	Register Name	Register Write	Register Read	Default	Description
0x20804790	FIFO_B1_6_BASE_PTR				
X:0x11E5 0x20804794	D_FIFO_B1_6_IOBLOCK_PTR FIFO_B1_6_IOBLOCK_PTR	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO B1
X:0x11E8 0x208047A0	D_FIFO_B1_6_BLOCK_SIZE FIFO_B1_6_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO B1
X:0x11EC 0x208047B0	D_FIFO_B1_6_BLOCK_CNT FIFO_B1_6_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO B1
X:0x11EE 0x208047B8	D_FIFO_B1_6_ACCESS FIFO_B1_6_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B1
X:0x11F0 0x208047C0	D_FIFO_B1_7_START FIFO_B1_7_START	(12:5) START_ADDR	(12:5) START_ADDR	0x0	Start address for FIFO B1
X:0x11F1 0x208047C4	D_FIFO_B1_7_END FIFO_B1_7_END	(12:5) END_ADDR	(12:5) END_ADDR	0x0	End address (+1) for FIFO B1
X:0x11F2 0x208047C8	D_FIFO_B1_7_SIZE FIFO_B1_7_SIZE	–	(13) FIFO_ENABLE	0x0	FIFO enable for FIFO B1
		(12:5) FIFO_SIZE	(12:5) FIFO_SIZE	0x0	Size (End-Start) for FIFO B1
X:0x11F3 0x208047CC	D_FIFO_B1_7_AUTO_MUTE FIFO_B1_7_AUTO_MUTE	(0) AUTO_MUTE_ENABLE	(0) AUTO_MUTE_ENABLE	0x0	FIFO auto mute enable for FIFO B1
X:0x11F4 0x208047D0	D_FIFO_B1_7_BASE_PTR FIFO_B1_7_BASE_PTR	(12:1) FIFO_PTR	(12:1) FIFO_PTR	0x0	FIFO pointer for FIFO B1
X:0x11F5	D_FIFO_B1_7_IOBLOCK_	(12:1) IOC_PTR	(12:1) IOC_PTR	0x0	IOC pointer for FIFO B1

Address	Register Name	Register Write	Register Read	Default	Description
0x208047D4	PTR FIFO_B1_7_IOBLOCK_PTR				
X:0x11F8 0x208047E0	D_FIFO_B1_7_BLOCK_SIZE FIFO_B1_7_BLOCK_SIZE	(11:0) BLOCK_SIZE	(11:0) BLOCK_SIZE	0x0	Block size for FIFO B1
X:0x11FC 0x208047F0	D_FIFO_B1_7_BLOCK_CNT FIFO_B1_7_BLOCK_CNT	(11:0) BLOCK_CNT	(11:0) BLOCK_CNT	0x0	Block counter for FIFO B1
X:0x11FE 0x208047F8	D_FIFO_B1_7_ACCESS FIFO_B1_7_ACCESS	(23:0) FIFO_DATA	(23:0) FIFO_DATA	0x0	Smart FIFO access for FIFO B1

A.48 FIFO INTERRUPT SELECTION

Address	Register Name	Register Write	Register Read	Default	Description
X:0x1200-X:0x1207 0x20804800 - 0x2080481C	D_FIFO_INT_CFX_CFG FIFO_INT_CFX_CFG_*	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for a CFX FIFO interrupt
X:0x1210-X:0x1217 0x20804840 - 0x2080485C	D_FIFO_INT_HEAR_CFG FIFO_INT_HEAR_CFG_*	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for a HEAR FIFO event
X:0x1220-X:0x1227 0x20804880 - 0x2080489C	D_FIFO_INT_CM3_CFG FIFO_INT_CM3_CFG_*	(4:0) SELECT_INT	(4:0) SELECT_INT	0x0	Select the source for an Arm Cortex-M3 core FIFO interrupt

A.49 HEAR INTERRUPT CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
X:0x1230	D_HEAR_FC_ENABLE	(23) FIFO_7	(23) FIFO_7	0x0	Enable/disable HEAR FIFO function chain 7
0x208048C0	HEAR_FC_ENABLE				
		(22) FIFO_6	(22) FIFO_6	0x0	Enable/disable HEAR FIFO function chain 6
		(21) FIFO_5	(21) FIFO_5	0x0	Enable/disable HEAR FIFO function chain 5
		(20) FIFO_4	(20) FIFO_4	0x0	Enable/disable HEAR FIFO function chain 4
		(19) FIFO_3	(19) FIFO_3	0x0	Enable/disable HEAR FIFO function chain 3
		(18) FIFO_2	(18) FIFO_2	0x0	Enable/disable HEAR FIFO function chain 2
		(17) FIFO_1	(17) FIFO_1	0x0	Enable/disable HEAR FIFO function chain 1
		(16) FIFO_0	(16) FIFO_0	0x0	Enable/disable HEAR FIFO function chain 0
		(15) CMD15	(15) CMD15	0x0	Enable/disable CFX function chain 15
		(14) CMD14	(14) CMD14	0x0	Enable/disable CFX function chain 14
		(13) CMD13	(13) CMD13	0x0	Enable/disable CFX function chain 13
		(12) CMD12	(12) CMD12	0x0	Enable/disable CFX function chain 12
		(11) CMD11	(11) CMD11	0x0	Enable/disable CFX function chain 11
		(10) CMD10	(10) CMD10	0x0	Enable/disable CFX function chain 10
		(9) CMD9	(9) CMD9	0x0	Enable/disable CFX function chain 9
		(8) CMD8	(8) CMD8	0x0	Enable/disable CFX function chain 8

Address	Register Name	Register Write	Register Read	Default	Description
		(7) CMD7	(7) CMD7	0x0	Enable/disable CFX function chain 7
		(6) CMD6	(6) CMD6	0x0	Enable/disable CFX function chain 6
		(5) CMD5	(5) CMD5	0x0	Enable/disable CFX function chain 5
		(4) CMD4	(4) CMD4	0x0	Enable/disable CFX function chain 4
		(3) CMD3	(3) CMD3	0x0	Enable/disable CFX function chain 3
		(2) CMD2	(2) CMD2	0x0	Enable/disable CFX function chain 2
		(1) CMD1	(1) CMD1	0x0	Enable/disable CFX function chain 1
		(0) CMD0	(0) CMD0	0x0	Enable/disable CFX function chain 0
X:0x1231 0x208048C4	D_HEAR_FC_PRIORITY HEAR_FC_PRIORITY	(23) FIFO_7	(23) FIFO_7	0x0	Prioritize the HEAR FIFO function chain 7
		(22) FIFO_6	(22) FIFO_6	0x0	Prioritize the HEAR FIFO function chain 6
		(21) FIFO_5	(21) FIFO_5	0x0	Prioritize the HEAR FIFO function chain 5
		(20) FIFO_4	(20) FIFO_4	0x0	Prioritize the HEAR FIFO function chain 4
		(19) FIFO_3	(19) FIFO_3	0x0	Prioritize the HEAR FIFO function chain 3
		(18) FIFO_2	(18) FIFO_2	0x0	Prioritize the HEAR FIFO function chain 2
		(17) FIFO_1	(17) FIFO_1	0x0	Prioritize the HEAR FIFO function chain 1
		(16) FIFO_0	(16) FIFO_0	0x0	Prioritize the HEAR FIFO function chain 0
		(15) CFX_15	(15) CFX_15	0x0	Prioritize the CFX function chain 15
		(14) CFX_14	(14) CFX_14	0x0	Prioritize the CFX function chain 14
		(13) CMD13	(13) CMD13	0x0	Prioritize the CFX function chain 13
		(12) CMD12	(12) CMD12	0x0	Prioritize the CFX function chain 12
		(11) CMD11	(11) CMD11	0x0	Prioritize the CFX function chain 11
		(10) CMD10	(10) CMD10	0x0	Prioritize the CFX function chain 10

Address	Register Name	Register Write	Register Read	Default	Description
		(9) CMD9	(9) CMD9	0x0	Prioritize the CFX function chain 9
		(8) CMD8	(8) CMD8	0x0	Prioritize the CFX function chain 8
		(7) CMD7	(7) CMD7	0x0	Prioritize the CFX function chain 7
		(6) CMD6	(6) CMD6	0x0	Prioritize the CFX function chain 6
		(5) CMD5	(5) CMD5	0x0	Prioritize the CFX function chain 5
		(4) CMD4	(4) CMD4	0x0	Prioritize the CFX function chain 4
		(3) CMD3	(3) CMD3	0x0	Prioritize the CFX function chain 3
		(2) CMD2	(2) CMD2	0x0	Prioritize the CFX function chain 2
		(1) CMD1	(1) CMD1	0x0	Prioritize the CFX function chain 1
		(0) CMD0	(0) CMD0	0x0	Prioritize the CFX function chain 0
X:0x1237 0x208048DC	D_HEAR_FC_STATUS HEAR_FC_STATUS	–	(23) FIFO_HEAR_7_ PENDING	0x0	Read the HEAR FIFO pending function chain 7 status
		(23) FIFO_HEAR_7_ CLEAR	–	N/A	Clear the HEAR FIFO pending function chain 7 status
		–	(22) FIFO_HEAR_6_ PENDING	0x0	Read the HEAR FIFO pending function chain 6 status
		(22) FIFO_HEAR_6_ CLEAR	–	N/A	Clear the HEAR FIFO pending function chain 6 status
		–	(21) FIFO_HEAR_5_ PENDING	0x0	Read the HEAR FIFO pending function chain 5 status
		(21) FIFO_HEAR_5_ CLEAR	–	N/A	Clear the HEAR FIFO pending function chain 5 status
		–	(20) FIFO_HEAR_4_ PENDING	0x0	Read the HEAR FIFO pending function chain 4 status
		(20) FIFO_HEAR_4_ CLEAR	–	N/A	Clear the HEAR FIFO pending function

Address	Register Name	Register Write	Register Read	Default	Description
		CLEAR			chain 4 status
		–	(19) FIFO_HEAR_3_PENDING	0x0	Read the HEAR FIFO pending function chain 3 status
		(19) FIFO_HEAR_3_CLEAR	–	N/A	Clear the HEAR FIFO pending function chain 3 status
		–	(18) FIFO_HEAR_2_PENDING	0x0	Read the HEAR FIFO pending function chain 2 status
		(18) FIFO_HEAR_2_CLEAR	–	N/A	Clear the HEAR FIFO pending function chain 2 status
		–	(17) FIFO_HEAR_1_PENDING	0x0	Read the HEAR FIFO pending function chain 1 status
		(17) FIFO_HEAR_1_CLEAR	–	N/A	Clear the HEAR FIFO pending function chain 1 status
		–	(16) FIFO_HEAR_0_PENDING	0x0	Read the HEAR FIFO pending function chain 0 status
		(16) FIFO_HEAR_0_CLEAR	–	N/A	Clear the HEAR FIFO pending function chain 0 status
		–	(15) CFX_15_PENDING	0x0	Read the CFX pending function chain 15 status
		(15) CFX_15_CLEAR	–	N/A	Clear the CFX pending function chain 15 status
		–	(14) CFX_14_PENDING	0x0	Read the CFX pending function chain 14 status
		(14) CFX_14_CLEAR	–	N/A	Clear the CFX pending function chain 14 status
		–	(13) CFX_13_PENDING	0x0	Read the CFX pending function chain 13 status
		(13) CFX_13_CLEAR	–	N/A	Clear the CFX pending function chain 13

Address	Register Name	Register Write	Register Read	Default	Description
					status
		–	(12) CFX_12_PENDING	0x0	Read the CFX pending function chain 12 status
		(12) CFX_12_CLEAR	–	N/A	Clear the CFX pending function chain 12 status
		–	(11) CFX_11_PENDING	0x0	Read the CFX pending function chain 11 status
		(11) CFX_11_CLEAR	–	N/A	Clear the CFX pending function chain 11 status
		–	(10) CFX_10_PENDING	0x0	Read the CFX pending function chain 10 status
		(10) CFX_10_CLEAR	–	N/A	Clear the CFX pending function chain 10 status
		–	(9) CFX_9_PENDING	0x0	Read the CFX pending function chain 9 status
		(9) CFX_9_CLEAR	–	N/A	Clear the CFX pending function chain 9 status
		–	(8) CFX_8_PENDING	0x0	Read the CFX pending function chain 8 status
		(8) CFX_8_CLEAR	–	N/A	Clear the CFX pending function chain 8 status
		–	(7) CFX_7_PENDING	0x0	Read the CFX pending function chain 7 status
		(7) CFX_7_CLEAR	–	N/A	Clear the CFX pending function chain 7 status
		–	(6) CFX_6_PENDING	0x0	Read the CFX pending function chain 6 status
		(6) CFX_6_CLEAR	–	N/A	Clear the CFX pending function chain 6

Address	Register Name	Register Write	Register Read	Default	Description
					status
		–	(5) CFX_5_PENDING	0x0	Read the CFX pending function chain 5 status
		(5) CFX_5_CLEAR	–	N/A	Clear the CFX pending function chain 5 status
		–	(4) CFX_4_PENDING	0x0	Read the CFX pending function chain 4 status
		(4) CFX_4_CLEAR	–	N/A	Clear the CFX pending function chain 4 status
		–	(3) CFX_3_PENDING	0x0	Read the CFX pending function chain 3 status
		(3) CFX_3_CLEAR	–	N/A	Clear the CFX pending function chain 3 status
		–	(2) CFX_2_PENDING	0x0	Read the CFX pending function chain 2 status
		(2) CFX_2_CLEAR	–	N/A	Clear the CFX pending function chain 2 status
		–	(1) CFX_1_PENDING	0x0	Read the CFX pending function chain 1 status
		(1) CFX_1_CLEAR	–	N/A	Clear the CFX pending function chain 1 status
		–	(0) CFX_0_PENDING	0x0	Read the CFX pending function chain 0 status
		(0) CFX_0_CLEAR	–	N/A	Clear the CFX pending function chain 0 status

A.50 BLOCK FLOATING POINT CONFIGURATION AND CONTROL

Address	Register Name	Register Write	Register Read	Default	Description
X:0x1260	D_HEAR_BFP_CFG	(21:16) BLOCK_EXP	(21:16) BLOCK_EXP	0x0	Stored block exponent for the data
0x20804980	HEAR_BFP_CFG				
		(15) FMD_MODE	(15) FMD_MODE	0x0	Enable/disable FMD mode
		(12:8) BLOCK_MAX_MAG	(12:8) BLOCK_MAX_MAG	0x0	Maximum magnitude for a block of data
		(5:4) SCALE_FACTOR	(5:4) SCALE_FACTOR	0x0	Select a scaling factor for BFP
		(1:0) FORMAT	(1:0) FORMAT	0x0	Select a format based on the number of bits
X:0x1261	D_HEAR_BFP_BLOCK_EXP	(5:0) MIRROR	(5:0) MIRROR	0x0	Mirror of the block exponent for the data
0x20804984	HEAR_BFP_BLOCK_EXP				

A.51 CFX MATH ACCELERATORS I/O

Address	Register Name	Register Write	Register Read	Default	Description
Y:0x0800 0x20C42000	D_Y_MATH_DIV_STATUS Y_MATH_DIV_STATUS	(1) DIV_ZERO	(1) DIV_ZERO	0x0	The Division Accelerator division by zero status
		(0) DIV_SM	(0) DIV_SM	0x0	The Division Accelerator sign mode
Y:0x0801 0x20C42004	D_Y_MATH_DIVIDEND Y_MATH_DIVIDEND	(23:0) DIVIDEND_NOTRG	(23:0) DIVIDEND_NOTRG	0x0	The Division Accelerator Dividend Input. It does not trigger calculation
Y:0x0802 0x20C42008	D_Y_MATH_LOG_EXP_STATUS Y_MATH_LOG_EXP_STATUS	(1) EXP2_IN_POS_ERR	(1) EXP2_IN_POS_ERR	0x0	The Exp2 Accelerator input positive data entry status
		(0) LOG2_IN_NEG_ERR	(0) LOG2_IN_NEG_ERR	0x0	The Log2 Accelerator input negative data entry status
Y:0x0803 0x20C4200C	D_Y_MATH_LOG2_OUT_MSB Y_MATH_LOG2_OUT_MSB	(23:0) LOG2_OUT_MSB	(23:0) LOG2_OUT_MSB	0x0	The Log2 Accelerator Output MSBs. The LSBs are in X memory
Y:0x0804 0x20C42010	D_Y_MATH_EXP2_OUT_MSB Y_MATH_EXP2_OUT_MSB	(23:0) EXP2_OUT_MSB	(23:0) EXP2_OUT_MSB	0x0	The Exp2 Accelerator Output MSBs. The LSBs are in X memory
Y:0x0805 0x20C42014	D_Y_MATH_ROOT_OUT_MSB Y_MATH_ROOT_OUT_MSB	(23:0) ROOT_OUT_MSB	(23:0) ROOT_OUT_MSB	0x0	The Root Accelerator Output MSBs. The LSBs are in X memory
Y:0x0806 0x20C42018	D_Y_MATH_LOG2_IN_MSB Y_MATH_LOG2_IN_MSB	(23:0) LOG2_IN_MSB	–	N/A	The Log2 Accelerator Input MSBs. The LSBs are in X memory
Y:0x0807 0x20C4201C	D_Y_MATH_ABS_LOG2_IN_MSB Y_MATH_ABS_LOG2_IN_MSB	(23:0) ABS_LOG2_IN_MSB	–	N/A	The Log2 Accelerator Input MSBs. The LSBs are in X memory
Y:0x0808	D_Y_MATH_EXP2_IN_MSB	(23:0) EXP2_IN_MSB	–	N/A	The Exp2 Accelerator Input MSBs. The

Address	Register Name	Register Write	Register Read	Default	Description
0x20C42020	Y_MATH_EXP2_IN_MSB				LSBs are in X memory
Y:0x0809 0x20C42024	D_Y_MATH_ABSNEG_EXP2_IN_MSB Y_MATH_ABSNEG_EXP2_IN_MSB	(23:0) ABSNEG_EXP2_IN_MSB	–	N/A	The Exp2 Accelerator Input MSBs. The LSBs are in X memory
Y:0x080A 0x20C42028	D_Y_MATH_ROOT2_IN_MSB Y_MATH_ROOT2_IN_MSB	(23:0) ROOT2_IN_MSB	–	N/A	The Root2 Accelerator Input MSBs. The LSBs are in X memory
Y:0x080B 0x20C4202C	D_Y_MATH_ABS_ROOT2_IN_MSB Y_MATH_ABS_ROOT2_IN_MSB	(23:0) ABS_ROOT2_IN_MSB	–	N/A	The Root2 Accelerator Input MSBs. The LSBs are in X memory
Y:0x080C 0x20C42030	D_Y_MATH_ROOT3_IN_MSB Y_MATH_ROOT3_IN_MSB	(23:0) ROOT3_IN_MSB	–	N/A	The Root3 Accelerator Input MSBs. The LSBs are in X memory
Y:0x080E 0x20C42038	D_Y_MATH_ROOT4_IN_MSB Y_MATH_ROOT4_IN_MSB	(23:0) ROOT4_IN_MSB	–	N/A	The Root4 Accelerator Input MSBs. The LSBs are in X memory
Y:0x080F 0x0000080F	D_Y_MATH_ABS_ROOT4_IN_MSB Y_MATH_ABS_ROOT4_IN_MSB	(23:0) ABS_ROOT4_IN_MSB	–	N/A	The Root4 Accelerator Input MSBs. The LSBs are in X memory
Y:0x0810 0x00000810	D_Y_MATH_DIVIDEND_TRG Y_MATH_DIVIDEND_TRG	(23:0) DIVIDEND_TRG	–	N/A	The Division accelerator Divdend Input. It triggers calculation
Y:0x0811 0x00000811	D_Y_MATH_DIV_OUT_MSB Y_MATH_DIV_OUT_MSB	–	(23:0) DIV_OUT_MSB	0x0	The Root Accelerator Output MSBs. The LSBs are in X memory

APPENDIX B

Sampling Frequency Reference

Table 76 is a lookup table that provides a summary of the possible sampling frequencies for a set of common ADCCLK frequencies. All sampling frequency values are provided in Hz.

Table 76. Summary of Possible Sampling Frequencies

Register Setting For SFCR from AUDIO_ADC_SAMPLE_FREQ_CFG	Desired ADCCLK Frequency:	
	3.84 MHz	5.12 MHz
0x07	60000.00	80000.00
0x08	53333.33	71111.11
0x09	48000.00	64000.00
0x0A	43636.36	58181.82
0x0B	40000.00	53333.33
0x0C	36923.08	49230.77
0x0D	34285.71	45714.29
0x0E	32000.00	42666.67
0x0F	30000.00	40000.00
0x10	28235.29	37647.06
0x11	26666.67	35555.56
0x12	25263.16	33684.21
0x13	24000.00	32000.00
0x14	22857.14	30476.19
0x15	21818.18	29090.91
0x16	20869.57	27826.09
0x17	20000.00	26666.67
0x18	19200.00	25600.00
0x19	18461.54	24615.38
0x1A	17777.78	23703.70
0x1B	17142.86	22857.14
0x1C	16551.72	22068.97
0x1D	16000.00	21333.33
0x1E	15483.87	20645.16
0x1F	15000.00	20000.00
0x20	14545.45	19393.94
0x21	14117.65	18823.53
0x22	13714.29	18285.71

Ezairo 8300 Hardware Reference

Summary of Possible Sampling Frequencies (Continued)

Register Setting For SFCR from AUDIO_ADC_SAMPLE_FREQ_CFG	Desired ADCCLK Frequency:	
	3.84 MHz	5.12 MHz
0x23	13333.33	17777.78
0x24	12972.97	17297.30
0x25	12631.58	16842.11
0x26	12307.69	16410.26
0x27	12000.00	16000.00
0x28	11707.32	15609.76
0x29	11428.57	15238.10
0x2A	11162.79	14883.72
0x2B	10909.09	14545.45
0x2C	10666.67	14222.22
0x2D	10434.78	13913.04
0x2E	10212.77	13617.02
0x2F	10000.00	13333.33
0x30	9795.92	13061.22
0x31	9600.00	12800.00
0x32	9411.76	12549.02
0x33	9230.77	12307.69
0x34	9056.60	12075.47
0x35	8888.89	11851.85
0x36	8727.27	11636.36
0x37	8571.43	11428.57
0x38	8421.05	11228.07
0x39	8275.86	11034.48
0x3A	8135.59	10847.46
0x3B	8000.00	10666.67
0x3C	7868.85	10491.80
0x3D	7741.94	10322.58
0x3E	7619.05	10158.73
0x3F	7500.00	10000.00

APPENDIX C

Glossary

The following abbreviations and terms are used in this group of topics:

<i>ADC</i>	analog-to-digital converter
<i>AFE</i>	analog front-end
<i>Arm Cortex-M3 processor</i>	processing core that can be used to configure the overall system after initial boot, and is typically used to coordinate the system interaction with external components
<i>ASCC</i>	audio sink clock counter
<i>ASRC</i>	asynchronous sample rate converter
<i>CCC</i>	common command codes (I ³ C)
<i>CCO</i>	current-controlled oscillator
<i>CFX Digital Signal Processor</i>	processing core that is used to configure the system at initial boot; this core can also be used to configure the overall system, and coordinate the flow of signal data progressing through the system
<i>CID</i>	chip identifier
<i>CMSIS</i>	Cortex Microcontroller Software Interface Standard
<i>CRC</i>	cyclic redundancy check
<i>DA</i>	dynamic address (I ³ C)
<i>DAA</i>	dynamic address assignment (I ³ C)
<i>DAC</i>	digital-to-analog converter
<i>DIO</i>	

Ezairo 8300 Hardware Reference

	digital input/output
<i>DMA</i>	direct memory access
<i>DSP</i>	digital signal processor
<i>ECC</i>	error correcting code
<i>EEPROM</i>	electrically-erasable, programmable, read-only memory, a type of non-volatile memory
<i>eMMC</i>	embedded multi-media controller
<i>FIFO</i>	A section of memory that is treated as a circular buffer where components that use this memory use pointers to handle data in a first-in, first-out manner. A hardware FIFO is a FIFO instance where its registers are controlled by the IOCs. A software FIFO is a FIFO instance where its registers are controlled by an application on the CFX, an application on the Arm Cortex-M3 processor, or a microcode configuration on the HEAR.
<i>Filter Engine</i>	a specialized processing core that is optimized for basic audio filtering of both decimated and undecimated data; commonly used in the creation of a low-delay path
<i>GPIO</i>	general-purpose input/output
<i>HEAR Configurable Accelerator</i>	a microcode configurable signal processing engine used to execute a variety of signal processing functions
<i>HIZ</i>	high impedance
<i>HJ</i>	hot join (I ³ C)
<i>I²C</i>	inter-IC communication protocol
<i>I²S</i>	inter-IC sound protocol
<i>I³C</i>	improved inter-IC communication protocol (MIPI Alliance)

<i>INL</i>	integral non-linearity
<i>IOC</i>	input/output controller
<i>JTAG</i>	joint test action group (developer of IEEE standard 1149.1-1990)
<i>LDP</i>	low delay path
<i>LPDSP32</i>	low-power digital signal processor with a 32-bit data path; a processing core that is used to accelerate codec, neural network, and similar functions
<i>LSAD</i>	low-speed analog-to-digital
<i>LSB</i>	least significant bit
<i>LUT</i>	look up table
<i>MCU</i>	microcontroller unit
<i>memory block instance</i>	an instance of RAM or ROM
<i>memory bus</i>	a communication system that transfers data between memory or memory-mapped components and the cores that operate on that data
<i>memory space</i>	a mapping of memory and memory-mapped items to a memory address; synonym for address space
<i>MSB</i>	most significant bit
<i>MUX</i>	multiplexer, selector of one signal from many
<i>NNA</i>	Neural Network Accelerator
<i>NVIC</i>	nested vectored interrupt controller

<i>NVM</i>	non-volatile memory
<i>PCM</i>	pulse code modulation
<i>PMU</i>	power management unit
<i>POR</i>	power-on-reset
<i>PSU</i>	power supervisory unit
<i>PWM</i>	pulse width modulation
<i>QMF</i>	quadrature mirror filter
<i>RAM</i>	random-access memory
<i>ROM</i>	read-only memory
<i>SCL</i>	serial clock (part of an I ² C or I ³ C bus)
<i>SDA</i>	serial data (part of an I ² C or I ³ C bus)
<i>SDM</i>	sigma-delta modulator
<i>SFCR</i>	sampling frequency configuration register
<i>SIR map</i>	slave initiated register map (I ³ C)
<i>SPI</i>	Serial Peripheral Interface
<i>SWD</i>	serial wire debug, two-wire interface used for communication with Arm cores
<i>SWJ-DP</i>	serial wire and JTAG debug port

onsemi Confidential
Ezairo 8300 Hardware Reference

<i>UART</i>	universal asynchronous receiver-transmitter
<i>VBAT</i>	supply voltage to the system (typically supplied from a battery)
<i>VDDA</i>	supply voltage for the non-RF analog blocks and flash memory
<i>VDDC</i>	supply voltage for the digital logic
<i>VDDIF</i>	supply voltage supporting external components and their interfaces
<i>VDDM</i>	supply voltage for the memories
<i>VDDO*</i>	the four input supplies for the digital I/O pads, including the debug ports (I ² C, SWJ-DP)
<i>VMIC</i>	supply voltage for microphone units
<i>VREF</i>	bandgap reference voltage
<i>VREG</i>	regulated reference supply voltage
<i>VSSA</i>	analog ground supply reference
<i>WDF</i>	wave digital filter

onsemi Confidential

Ezairo 8300 Hardware Reference

Sections of this manual relating to the Arm Cortex-M3 processor have been republished from the Cortex-M3 Technical Reference (version r2p1) with permission.



Arm, the Arm logo and Cortex are trademarks or registered trademarks of Arm Ltd. Ezairo is either a trademark or registered trademark of SCILLC. All other brand names and product names appearing in this document are trademarks of their respective holders.

onsemi and the onsemi logo are trademarks of Semiconductor Components Industries, LLC dba onsemi or its subsidiaries in the United States and/or other countries. onsemi owns the rights to a number of patents, trademarks, copyrights, trade secrets, and other intellectual property. A listing of onsemi's product/patent coverage may be accessed at www.onsemi.com/site/pdf/Patent-Marking.pdf. onsemi reserves the right to make changes without further notice to any products herein. onsemi makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does onsemi assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. Buyer is responsible for its products and applications using onsemi products, including compliance with all laws, regulations and safety requirements or standards, regardless of any support or applications information provided by onsemi. "Typical" parameters which may be provided in onsemi data sheets and/or specifications can and do vary in different applications and actual performance may vary over time. All operating parameters, including "Typicals" must be validated for each customer application by customer's technical experts. onsemi does not convey any license under its patent rights nor the rights of others. onsemi products are not designed, intended, or authorized for use as a critical component in life support systems or any FDA Class 3 medical devices or medical devices with a same or similar classification in a foreign jurisdiction or any devices intended for implantation in the human body. Should Buyer purchase or use onsemi products for any such unintended or unauthorized application, Buyer shall indemnify and hold onsemi and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use, even if such claim alleges that onsemi was negligent regarding the design or manufacture of the part. onsemi is an Equal Opportunity/Affirmative Action Employer. This literature is subject to all applicable copyright laws and is not for resale in any manner.

PUBLICATION ORDERING INFORMATION

LITERATURE FULFILLMENT:

Literature Distribution Center for onsemi

19521 E. 32nd Pkwy, Aurora, Colorado 80011 USA

Phone: 303-675-2175 or 800-344-3860 Toll Free USA/Canada

Fax: 303-675-2176 or 800-344-3867 Toll Free USA/Canada

Email: orderlit@onsemi.com

N. American Technical Support:

800-282-9855 Toll Free USA/Canada

Europe, Middle East and Africa Technical

Support: Phone: 421 33 790 2910

onsemi Website: www.onsemi.com

Order Literature: <http://www.onsemi.com/orderlit>

For additional information, please contact your local Sales Representative

M-20852-019